Custom Compiler™ Environment User Guide

Version X-2025.06-SP1, September 2025

SYNOPSYS®

Copyright and Proprietary Information Notice

© 2025 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at https://www.synopsys.com/company/legal/trademarks-brands.html.
All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Feedback

Contents

New in This Release 23
Related Products, Publications, and Trademarks 23
Conventions 24
Customer Support 25
Statement on Inclusivity and Diversity 26

1. **Environment Overview** 27
Design Phases and Products to Use 27
Software User Interface 30
Home Tab 31
File Menu 33
Tools Menu 34
License Menu 35
Options Menu 36
Window Menu 36
Help Menu 38
Console Tab 38
Menu Bar 42
Standard Toolbar 49
Tabs 50
Command Input Text Box 53
Tabs for Tools and Designs 55
Shortcut Keys 57
Common Graphical User Interface Operations 57
Pinning and Unpinning Tabs 57
Pinning Tabs 58
Unpinning Tabs 58
Undocking and Docking Assistants 58
Undocking Assistants 59
Docking Assistants 59
Freezing Table Columns 59
Environment Management 60
Session Settings 61
Technology Data and Process Constraints 61

Display Resources 62
Instance Object Parameters 62
Design Import and Export Requirements 62
Job Process Monitoring 63
Errors and Event Logs 64
Custom Compiler Tcl 64
Executing Individual Tcl Commands 64
Running Tcl Scripts 65
Using the Tcl Help Utility 65
Python in the Custom Compiler Environment 70
About Using Python in the Custom Compiler Environment 70
How to Run Python Expressions and Scripts 71
Customization 72

2. Getting Started 73
Starting Sessions 73
About the custom_compiler Command 73
Starting the Custom Compiler Tool 74
custom_compiler Command Syntax and Arguments 74
License Queuing 76
Ending Sessions 77
Defining Startup Settings 78
Locating Common Files and Startup Directories 79
Setting Environment Variables 80
Viewing Installed Directories and Files 82
Defining Session Settings 83
Setting General Options 84
Managing Option Settings from the Home Page 97
Saving Option Settings as Preferences 97
Loading Option Settings Saved as Preferences 98
Saving Dialog Box Settings as Preferences 99
Specifying Design Preferences 100
Synchronizing Design Windows, Viewports, Net Selections, and Commands Across Opened Designs 101
Customizing Keyboard Shortcuts 104
About the Keyboard Shortcuts in the Custom Compiler Environment 104
Selecting the Shortcut Key Binding Set 106

Viewing Keyboard Shortcuts 107
Creating and Editing Keyboard Shortcuts 110
Creating Keyboard Shortcuts Using Tcl Commands 116
Importing and Exporting Keyboard Shortcut Information 118
Restricting Design Access to Read-Only 120

3. Supplying Mapping Files for GDSII Stream and OASIS Format Translations . 121
Layer Mapping Files 121
Layer Mapping File Format and Contents 122
OA, GDSII, and OASIS Mappings 124
Mask Color and Color Lock Attributes 124
Material and Mask Number Attributes 126
Cut Size Attribute 126
Mapping OpenAccess Objects in the Layer Mapping File 126
Examples 127
Object Mapping Files 128
Cell Mapping Files 128
Font Mapping Files (GDSII Stream Only) 129
Label Mapping Files (GDSII Stream Only) 129
Property Mapping Files (GDSII Stream Only) 130
Property Mapping Files (OASIS Only) 131
Reference Mapping Files (GDSII Stream Only) 132
Text Mapping Files 133
OASIS Text Mapping Files 133
GDSII Stream Text Mapping Files 133

4. Import 136
Importing Design Data in GDSII Stream Format 136
Importing Design Data in OASIS Format 150
Importing Design Data in Library Exchange Format 163
LEF Import Example 167
Importing Design Data in Design Exchange Format 167
Import DEF Example 171
Importing Design Netlist Data as Text Views 171
Generating Device Mapping File 184

Device Mapping File Format 185
devType 187
cellType 188
models 188
regModels 189
portProp 189
ports 190
portDefaultNets 190
propMatching 191
lcv 191
propMapping 191
stringParameters 192
appProps 192
Device Mapping File Examples 192
Importing Design Netlist Data for Schematic Generation 195
Importing Design Data in EDIF 200 Format 204
Using the Import EDIF200 Dialog Box 205

5. Export 210
Exporting Design Data in GDSII Stream Format 210
Exporting Design Data in OASIS Format 230
Exporting Design Data in Library Exchange Format (LEF) 246
Exporting Design Data in Design Exchange Format (DEF) 250
Exporting Design Data in Liberty Library File Format 252
Exporting Design Data in Netlist Format 254
Selecting Multiple Cellviews 265

6. General Designing 268
Creating Designs 268
Opening Designs 270
From the Console Tab 271
From an Editor Window 271
From the Library Manager 272
Opening Two Designs 274
Opening a Bookmarked Design 275
Responding to Warnings while Opening Designs 276
Reopening Recently Opened Designs 277

Navigating the Design Hierarchy 277
Navigating One Instance at a Time 277
Navigating Using Alternate Methods 280
Viewing the Design 281
Zooming 281
Magnifying 282
Peeking 282
Fitting 283
Panning 283
Redrawing Designs 284
Switching to Previous or Next Viewports 284
Interrupting Rendering and Layout Edits with Ctrl+C 285
Using Filter Boxes 285
Filtering Lists 286
Displaying Selections in Filter Boxes 286
Changing Edit Modes 287
Reverting to Last-Saved Edits 287
Saving Your Designs 288
Saving Designs 288
Saving Designs With Hierarchy 288
Saving Designs With New Names 289
Saving Designs With Schematic and Config Views 289
Saving Hierarchical Design Layouts 289
Checking and Saving Individual Design Schematics 290
Checking and Saving Designs Represented as Text 291
Checking and Saving All Designs in Libraries 291
Saving Designs With Backup 292
Automatically Saving Designs 293
Retrieving Your Designs 294
Displaying Design Summary Information 295
Generating a Design Summary Report in the Layout Editor 295
Generating a Design Summary Report in the Schematic Editor 299
Generating a Schematic Design Summary Report 299
About Cell Mapping File 302
Specifying Result Type for Schematic Summary 306
Consolidating Schematic Summary Results 308
Printing and Plotting Designs 309

Submitting a Print Request 309
Checking the Print Queue 316
Canceling a Print Request 316
Exporting Design Data 316
Exporting Design Images 317
Closing Designs 318
Configuring Custom Cells 319
Grouping Custom Cells by Tags 319

7. Assistants for Designing 322
Deciding Which Assistant to Use 323
Arranging Assistant Tabs 325
Synchronizing Assistants 326
About Assistant Synchronization 326
Annotation Toolbar 328
Clipboard Assistant 328
Displaying the Clipboard Assistant 330
Using the Clipboard Assistant 330
Selecting the Clipboard Library 331
Opening and Closing the Clipboard Library 331
Copying Selected Clipboards 333
Deleting Selected Clipboards 334
Reordering Selected Clipboards 334
Console Assistant 334
Displaying the Console Assistant 336
Viewing New Messages Without Displaying the Console Assistant 337
Displaying Console Message Tooltips Automatically 337
Viewing the Session Log Information 339
Viewing the Command History 340
Viewing Only Error and Warning Messages 342
Typing Tcl Commands and Canvas Coordinates 343
Typing Python Commands 345
Constraint Editor Assistant 350
Constraint Types and Sources 350
About the Constraint Editor 352
Displaying the Constraint Editor 354

Viewing Constraints 354
Using Common View Features 355
Customizing Views and Setting View Options 357
Viewing and Correcting Invalid Constraint Settings 358
Importing and Exporting Design-Specific Constraints 359
Importing Design-Specific Constraints 359
Exporting Design-Specific Constraints 361
Constraint Notes 363
Design Navigator Assistant 365
About the Design Navigator 366
Displaying the Design Navigator Assistant 367
Switching Between the Layout and Schematic Design Modes 367
Locating Components 368
Renaming Components 372
Manipulating Components 372
Placing Design Areas in Shadow 377
Setting Flight Line Display Options 377
Showing and Hiding Flight Lines From the Canvas 379
Working With Nets 380
Before You Start 381
Adding Nets to Net Groups 385
Creating Net Groups 386
Viewing Grouped Nets 388
Setting Signal Types 389
Viewing Information About Nets 390
Viewing Open Nets 391
Viewing Shorted Nets 393
Reporting Information about Nets 395
Working With Figure Groups 396
Displaying the List of Figure Groups 397
Filtering the List of Figure Groups 398
Selecting and Unselecting Figure Groups 400
Operating on Figure Groups 401
Showing Synchronous Clone Groups 404
Adding Figure Groups to Synchronous Groups 405
Breaking Synchronization of Cloned Groups 407
Synchronizing Cloned Groups to Peers 408
Device Palette Assistant 408
Displaying the Device Palette Assistant 409
Adding Favorite Devices to the Device Palette 409
Editing Favorite Devices in the Device Palette 410

Placing Devices from the Palette 411
Adjusting Device Palette Contents 412
Error Viewer Assistant 413
Opening the Error Viewer Assistant 414
Using the Error Viewer Assistant in the Schematic Editor 415
Managing the Marker Tree 416
Displaying Category Levels in the Marker Tree 417
Automatically Expanding Marker Levels in Error Viewer 418
Configuring the Marker Tree Contents 419
Wildcard Matching in the Display Filter 421
Viewing Marker Categories 422
Viewing Error Markers 422
Viewing Error Markers Hierarchically 423
Manipulating Error Markers 425
Sorting Error Markers 438
Deleting Error Markers 439
Customizing Marker Information to Display 439
Showing Tooltips of Markers 440
Customizing the Error Viewer Layout Style 441
Viewing Markers in Shadow 441
Managing Highlight Colors 441
Highlighting Markers When Selecting 445
Highlighting With an X Symbol 446
Setting Zoom Mode after Highlighting 447
Setting Highlight Actions 448
Specifying the Maximum Number of Markers to Display 450
Clearing All Highlights When Closing Error Viewer 451
Selecting First Item of Marker Table 452
Setting Filter Type for Categories 453
Loading Calibre Result 454
Loading RedHawk Result 456
Loading Generic Format File 457
Exporting to a Text File 459
Exporting Waived List 460
Hierarchy Navigator Assistant 462
Displaying the Hierarchy Navigator Assistant 462
Setting Navigation by Instance 462
Setting Navigation by Master 464
Exporting Design Hierarchy 465

Resolving Context Issues 466
Searching Instances 467
Showing Quick Access Menu 467
Object Filter Assistant 468
Opening the Object Filter Assistant 468
Filtering Schematic Objects 468
Filtering Symbol Objects 469
Object/Layer Panel Assistant 470
Opening the Object/Layer Panel Assistant 472
Manipulating the Object/Layer Panel 472
Locating LPPs, Layers, and Objects 473
Displaying Color Annotations 475
Displaying Presets 477
Working With LPPs 479
Using LPPs Tab Menu 480
Creating User-Defined LPP Sets 481
Editing User-defined LPP Sets 482
Removing User-defined LPP Sets 483
Setting the Active Layer Purpose Pair 484
Making LPP Sets Visible and Selectable 486
Showing Abbreviations and Stream Layer Numbers 488
Changing the Drawing Priority of a Layer Purpose Pair 489
Sorting Layer Purpose Pairs 489
Controlling the Visibility and Selectability of Boundary Objects 489
System LPPs 490
Working With Layers 491
Working With Objects 493
Sharing OLP Settings 495
Controlling Area Selections of Figure Groups 497
Working With Display Resources 498
Setting OLP Options 498
Probe Assistant 504
Opening the Probe Assistant 504
Cycling Through Probed Objects 505
Deleting Probes 506
Viewing Probes in Shadow 506
Viewing Probed Object Details 507
Property Editor Assistant 509
About the Property Editor 509

Setting Editor Annotation Options 511
Displaying the Property Editor 513
Opening the Properties Dialog Box 514
Viewing Design Object Properties 515
Editing and Deleting Design Properties 518
Creating New Design Properties 521
Enabling Zoom Operations 523
Adjusting Parameters in the Selected Set 523
Resetting to Default Parameter Values 524
Instance Terminal Region 524
Property Editor Toolbar 525
Selection Table Assistant 526
Opening the Selection Table Assistant 526
Showing Columns 527
Viewing Parameters 530
Viewing Figure Group Members 532
Showing Objects in Shadow 533
Fitting Viewport to Selected Objects 534
Track Pattern Assistant 535
About the Track Pattern Assistant 535
Displaying the Track Pattern Assistant 536
Viewing Track Patterns 537
Creating and Editing Track Patterns 540
Copying Track Patterns 541
Renaming and Deleting Track Patterns 542
Creating and Editing Track Pattern Definitions 542
Assigning New Definitions 546
Track Region Assistant 546
About Track Regions 547
About the Track Region Assistant 548
Grouping Track Grids and Grid Groups 549
Before You Start 556
Defining Track Presets 556
Displaying the Track Region Assistant 558
Setting Track Region Assistant Options 558
Creating Track Region PCells 562
Creating Track Regions 569
Creating a Global Track Region 570

Creating Tracks 572
Creating Track Group Definitions 573
Creating Track Region Definitions 574
Deleting Track Regions 576
Editing Tracks 577
Editing Track Group Definitions 577
Editing Track Region Definitions 578
Editing Track Region Properties 580
Filtering the List of Track Regions 581
Displaying and Hiding Track Regions on the Canvas 582
Making Track Regions, Track Groups, and Individual Tracks Active 584
Transaction History Assistant 585
Displaying the Transaction History Assistant 586
Using the Transaction History List 587

8. Tools for Designing 588
Deciding Which Tool to Use 588
Bookmark Manager Tool 589
Adding a Bookmark 590
Displaying the Bookmark Manager Tool 591
Managing Bookmarks 591
Loading Bookmarks 591
Saving Bookmarks 592
Deleting Bookmarks 592
Renaming Bookmarks 592
Constraint Manager Tool 593
Constraint Types and Sources 594
About the Constraint Manager 595
Displaying the Constraint Manager 596
Adding and Deleting Process Constraint Parameters 597
Adding Parameters 597
Deleting Parameters 597
Grouping a Set of Process Constraints 598
Specifying User-Defined Constraint Groups 599
Enabling Alternate Foundry Constraints 600
Workflow for Enabling Alternate Foundry Constraints 601
Before You Start 601
Defining Alternate Foundry Constraint Groups 602

Applying the Alternate Foundry Rules in the Design 602
Associating Track Region Definitions With Alternate Foundry Constraint Groups 603
Constraint Manager Reference 604
Constraint Manager Menus 605
Constraint Manager Toolbar 605
Display Resource Editor Tool 606
About the Display Resource Data 606
Displaying the Display Resource Editor 607
Managing Display Definitions 609
Creating Display Definitions 610
Editing Display Attributes 610
Renaming Display Definitions 611
Deleting Display Definitions 611
Creating and Editing Display Packets and Packet Aliases 611
Creating Packets 612
Creating Packet Aliases 613
Using Active Packet Aliases 613
Creating Colors, Stipples, and Line Styles 615
Creating Fill and Outline Colors 615
Creating Stipple Patterns 615
Creating Line Styles 616
Managing Display Resources 617
Saving Display Resource Data 618
Loading Display Resource Files 618
Reloading Last-Saved Display Resource Files 618
Reloading Default Display Resource Files 619
Merging Display Resource Files 620
Commonly Used Display Packets 622
Hierarchy Editor Tool 625
Displaying the Hierarchy Editor 625
Creating Hierarchy Configurations 626
Creating Configuration Views From an Existing Design 627
Creating Configuration Views as New CellViews 630
Editing Hierarchy Configurations 631
Undoing and Redoing Configuration View Modifications 631
Explaining Found Hierarchy Views 631
Importing Attributes From Existing Configuration Views 632
Specifying a Template for a Hierarchy Configuration View 632
Using Hierarchy View Name Groups 633
Setting Hierarchy Editor Options 634
Searching Configuration Views 635

Filtering Configuration Views on Libraries or Cells 635
Controlling Viewed Information 636
Synchronizing Design and Hierarchy Data 636
Enabling Automatic Cross Selection Between Hierarchy Editor and Design Editor 637
Showing Hierarchy View Search and Stop Lists 638
Showing Hierarchy Tree or Cell Views 638
Showing and Hiding Vector Bits 639
Selecting the View or Library 640
Viewing Unbound Cell or Instance Information 641
Selecting and Clearing Scoped Views 643
Hierarchy Editor Menus 643
Hierarchy Editor Toolbar 647
Library Manager Tool 649
About the Library Manager 650
Displaying the Library Manager 652
Showing and Hiding Libraries and Cells 653
Before You Start 654
Showing and Hiding Libraries 654
Showing and Hiding Cells 657
Viewing Library Attributes 659
Opening a Terminal From the Directory of a Library 661
Displaying or Hiding Cellview Version Information 662
Displaying Cellview Version Information 662
Hiding Cellview Version Information 663
Displaying or Hiding the Library and Cell File Browser 664
Displaying the Library and Cell File Browser 664
Hiding the Library and Cell File Browser 665
Copying Library, Cell, and Cellview Names 665
From the Libraries, Cells, or Views Sections 665
From the File Browser 666
Copying Library, Cell, and Cellview Paths 667
From the Libraries, Cells, or Views Sections 668
From the File Browser 668
Creating New Libraries 670
Editing Library Definitions 672
Using the Library Definitions Editor 672
Using a Text Editor 674
Setting the Path to Display 675
Creating Library Categories 675
Controlling Library Availability 678

Specifying Library Availability 678
Combining Libraries 680
Hiding Libraries 680
Loading Libraries Automatically 681
Compressing and Uncompressing Libraries 681
Defining Compression Level 682
Preparing Libraries for Compressing 682
Modifying Component Description Format (CDF) Overlays 684
Managing Individual Libraries 685
Copying Libraries 685
Moving Libraries 688
Deleting Libraries 690
Updating Library Name References 691
Updating Library References 691
Managing Cells 694
Creating New Cells 695
Creating New and Reusing Cell Hierarchies 696
Copying a Hierarchy of Cells 698
Moving Cells 703
Deleting Cells 705
Enabling Rotation and Scaling 705
Managing Cellviews 706
Creating Cellviews 706
Copying Cellviews 707
Moving Cellviews 711
Deleting Cellviews 713
Managing Duplicate Name Conflicts 715
Managing Properties 715
Opening the Context-Sensitive Menu to Manage Properties 716
Opening Library Properties 716
Copying Cell Properties 719
Opening a File in the File Browser 720
Using Cell Categories 721
About Cell Categories and Subcategories 721
Viewing Cell Categories and Subcategories 724
Creating New Cell Categories 724
Managing Cell Categories 725
Post-processing MCells 726
Importing MCells 726
Exporting MCells 727
Refreshing MCells 728
Library Manager Menus 729

File Menu . . . 730
Edit Menu . . . 731
View Menu . . . 732
Window Menu . . . 733
Help Menu . . . 734
Library Manager Toolbar . . . 734
Parameter Definition Editor Tool . . . 736
Displaying the Parameter Definition Editor Tool . . . 736
Viewing Parameters . . . 737
Adding Parameters . . . 738
Adjusting Parameters . . . 738
Editing Components Parameters . . . 738
Viewing CDF Overlay Parameters . . . 740
Editing CDF Overlay Parameters . . . 742
Parameter Definition Editor Menus . . . 743
Parameter Definition Editor Toolbar . . . 746
Search and Replace Tool . . . 747
Displaying the Search and Replace Tool . . . 748
Setting Search and Replace Options . . . 749
Setting the Design Root . . . 751
Setting Search Criteria for Attributes and Properties . . . 752
Searching for Design Components . . . 753
Browsing Component Search Results . . . 758
Refining Component Search Results . . . 759
Checking Whether a Property Exists on an Instance . . . 760
Finding Instances With the Default Value of a Given Property . . . 760
Finding Instances With a Specific Property Value . . . 761
Finding Instances With a Property Not Equal to a Specific Value . . . 761
Finding Instances With a Property Value in a Given Range . . . 762
Finding Paths or Path Segments in Layouts . . . 762
Finding Text on Layer Shapes in Layouts . . . 764
Finding Text Objects in Layouts . . . 765
Finding Vectored Instances . . . 766
Finding Via Vector Values . . . 767
Finding Vias in Layouts . . . 768
Getting Total Number of Instances Used in a Design . . . 769
Replacing Component Aspects Using Search Results . . . 770
Replacing Scientific Notation With Equivalent Value . . . 771
Replacing Property Values . . . 772

Replacing Values Within Search Results 772
Replacing Values With Value Multiplied by a Constant 772
Technology Manager Tool 773
Displaying the Technology Manager Tool 774
Considering Technology Management Options 774
Changing the Technology Referenced by a Design Library 775
Importing Technology Data 775
Exporting Technology Data 777
Adding a Reference 778
Deleting a Reference 778
Saving a Modified Technology Database 779
Reverting Changes to a Modified Technology Database 779
Text Viewer Tool 779
Displaying the Text Viewer 780
Viewing Job Process Text 780
Printing Files From the Text Viewer 781
Design Technology Co-optimization Tool 782
About the Design Technology Co-optimization 783
Workflow for Design Technology Co-optimization 784
Preparing for Design Technology Co-optimization 786
Setting Up the DTCO Parameters 788
Displaying and Filtering the DTCO Parameters 788
Adding and Editing DTCO Parameters 790
Exporting and Importing DTCO Parameters 792
Loading DTCO Parameters 794
Saving DTCO Parameters 794
Running Transistor Modeling 795
Running Interconnect Characterization 798
Running Cell Parasitic Extraction 800
Running Digital Implementation 803
Creating a Split 807
Comparing Split Results 810
Monitoring Computations in Job Monitor 812

9. Design Reviews 813
About the Design Review System 813
Review Assistant 814
Review Manager 817

Review Data Structure 820
Review Database 820
Review Process 821
Workflows for Design Reviews 822
Workflow for Setting Up Design Reviews 822
Workflow for Conducting Reviews 822
Setting Up Design Review Policies 824
About Design Review Policies 824
Default Design Review Policy 824
Creating Design Review Policies 825
Modify Existing Design Review Policies 825
Setting Up Automatic Review Notifications 825
Creating, Editing, and Managing Design Reviews 826
Displaying the Review Assistant 827
Starting Design Review Sessions 828
Creating New Reviews 829
Copying Reviews 830
Cloning Reviews 832
Editing Reviews 832
Adding Annotations in a Review 833
Editing, Moving, and Deleting Annotations 837
Adding Images to a Review 839
Displaying Annotations and Images 841
Hiding Annotations 842
Deleting Issues and Reviews 843
Adding and Replying to Discussion Comments 843
Ending Design Review Sessions 844
Saving Changes and Ending Reviews 845
Discarding All Changes and Ending Reviews 845
Opening an Existing Review 845
Exporting Reviews 850
Review Assistant Reference 851

10. Tools for Job Status and Monitoring 855

Job Monitor Tool 855
Displaying the Job Monitor Tool 856
Viewing Running Job Processes 856

Applying Job Process Controls 858
Managing Job Options 858
Creating Custom Job Class 861
Setting Up Independent Processes Distribution 863
Customizing the Job Name 863
Filtering Job Groups 864
Displaying a Netlist 864
Displaying the Output 864
Launching a Terminal 865
Adding Job Classes With xt::createJobClass Tcl 865
Sample Job Class for Design Rule Checking on Linux Systems 865
Sample Job Class for PrimeSim HSPICE Simulation on Solaris Systems 867
Adding Job Process Waiting With xt::wait Tcl 868
Sentry Monitor Library Tool 869
Watchdog Tool 869
Installing the Watchdog Tool 870
Watchdog Outputs 871
Database 871
Database Structure 871
Log Files 872
Watchdog Report Statistics 873
Query on Database 873

11. Customizing the Software and Workspaces 875
Defining Custom Workspaces 875
About Custom Workspaces 876
Creating Workspaces 877
Saving Workspaces 877
Loading Workspaces 878
Switching to Another Workspace 878
Renaming Workspaces 879
Deleting Workspaces 879
Preserving Workspace Geometries 879
Configuring Custom Windows 881
Planning the Custom Windows 881
Creating Custom Windows 881
Creating New Custom Menus and Toolbars 882

Adding Commands in Custom Menus and Toolbars 883
Deleting Menus, Menu Commands, and Toolbars 885
Saving Custom Windows for Future Sessions 885
Creating Dialog Boxes 886
Creating a Sample Dialog Box 886
Creating a Basic Dialog Box 887
Creating Elements for Collecting Input 888
Creating Boolean Input Check Boxes 888
Collecting File Input 889
Collecting Library Input 890
Collecting Input through a List 891
Creating Mutually Exclusive Input 892
Creating Number Input 893
Creating Text Input 893
Creating Grouping Elements 894
Creating Groups 895
Creating Inline Groups 896
Creating Tab Groups 896
Creating Checkable Groups 897
Creating Splitter Groups 898
Creating Push Buttons and Labels 899
Creating Push Buttons 900
Creating Labels 900
Creating Elements that Reconfigure 900
Creating Tables 900
Creating Dynamic Lists 901
Adjusting Custom Dialog Box Layouts 901
Adding Assistants and Other User Interfaces Using Tcl/Tk 902
Tcl Sample for Custom Graphic User Interface 904
Sample GUI for Creating a Symbol from a SPICE Netlist 905
Sample Preferences for Creating a Symbol from a SPICE Netlist 911

A. Integrating External Applications 914
Glossary of Terms 914
Handshaking with the Custom Compiler Environment 915
Walkup with the Custom Compiler Environment 917
Blocking and Non-Blocking IPC 918
Blocking Mode IPC 919

Non-Blocking Mode IPC 919
Recommendation 919
A Sample Integration 920
Using the Sample Integration 920
The External Application 921
The Interpreter 922
Opening a Design 922
Zoom 923
Probing a Net 924
Probing an Instance 925
Highlighting and Clearing Highlights 925
Creating a Rectangle 927
A Label to Modify 928
The Connection Status Frame 928
The Window Layout 928
Walkup Utilities 929
Sending Commands to the Custom Compiler Environment 931
The Handshaking Code 931
The Tcl Event Loop 933
Launching From the Custom Compiler Environment 933
Defining a Namespace 934
Adding a Menu and Actions 934
Action Callbacks 936
Job Callbacks 938

About This User Guide

This user guide covers instructions for using and customizing the Custom Compiler product environment, for both CAD managers and designers who use the Schematic Editor and Layout Editor products.

This preface includes the following sections:

- New in This Release
- Related Products, Publications, and Trademarks
- Conventions
- Customer Support
- Statement on Inclusivity and Diversity

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the Custom Compiler Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the Custom Compiler tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

https://solvnetplus.synopsys.com

You might also want to see the documentation for the following related Synopsys products:

Synopsys® Custom Compiler™

Synopsys® PrimeWave™ Design Environment

PrimeSim™ Continuum

PrimeSim™

PrimeSim™ Pro

PrimeSim™ SPICE

PrimeSim™ HSPICE®

PrimeSim™ XA

PrimeSim™ CCK

PrimeSim™ MOSRA

VCS® PrimeSim™ AMS

Synopsys® FineSim®

Synopsys® HSPICE®

Synopsys® VCS®

Synopsys® Verdi®

Synopsys® NanoSim®

Synopsys® WaveView™

Synopsys® Fusion Compiler™

Synopsys® IC Compiler™

Synopsys® IC Validator™

Synopsys® SolvNetPlus support site

Synopsys® StarRC™

OASIS® (Open Artwork System Interchange Standard)

Cadence® Virtuoso® Spectre®

Siemens® Calibre® platform

Siemens Calibre® Interactive™ software

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
`Courier`	Indicates syntax, such as `write_file`.
`Courier italic`	Indicates a user-defined value in syntax, such as `write_file` *`design_list`*

Feedback

Convention	Description
`Courier bold`	Indicates user input—text you type verbatim—in examples, such as `prompt> write_file top`
`Purple`	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as `include_enclosing = true | false`
[ ]	Denotes optional arguments in syntax, such as `write_file [-format fmt]`
...	Indicates that arguments can be repeated as many times as needed, such as *`pin1 pin2 ... pinN`*
|	Indicates a choice among alternatives, such as `low | medium | high`
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the **Edit** menu and choosing **Copy**.
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

https://solvnetplus.synopsys.com

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to https://solvnetplus.synopsys.com.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of noninclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Environment Overview

This overview introduces you to the Synopsys Custom Compiler environment. The chapter describes the environment products and explains when to use each product in the various design phases. This chapter also provides an overview of the software user interface and explains how to use Tcl (Tool Command Language) scripts to automate and customize the environment to meet your needs.

This chapter contains the following topics:

- Design Phases and Products to Use
- Software User Interface
- Common Graphical User Interface Operations
- Environment Management
- Custom Compiler Tcl
- Python in the Custom Compiler Environment
- Customization

Design Phases and Products to Use

The Custom Compiler environment uses visually assisted automation technologies to accelerate custom design tasks, reduce iterations, and enable reuse. In addition to an open architecture, the Custom Compiler environment bridges the gap between digital and custom design.

Table 1 lists the integrated circuit (IC) design phases and the Custom Compiler product to use at each design phase. For information about packaging designs, see IC Packaging Designs in the Custom Compiler Environment.

Feedback

Table 1 Design Phases and Products

Design Phase	Product	Description
Schematic Entry	Schematic Editor	Begin a design by placing and connecting its components in a schematic. Edit properties of components, add pins to define block interfaces for hierarchical designs, and verify the design for electrical and semantic correctness. In addition, text views (Verilog, VHDL) are supported through the Text Editor for language-based design and reuse. See the *Custom Compiler Schematic Editor and Text Editor User Guide* for instructions.
Simulation and Analysis	PrimeWave Design Environment	After schematic entry, you can generate a netlist and simulate the circuit. Perform netlisting from the **Home** tab using **Export > Netlist**. Use the PrimeWave Design Environment to define and work in a custom environment for netlisting, simulation, and plotting and viewing waveforms. From the PrimeWave Design Environment, you can also use the PrimeSim Continuum tools for circuit simulation and analysis. See the *PrimeWave Design Environment User Guide* for instructions. To perform only basic netlisting and simulation, see Exporting Design Data in Netlist Format for instructions.
Layout	Schematic Driven Layout (SDL) Symbolic Editor Layout Editor	After analysis, you can generate and develop the layout using the Schematic Driven Layout (SDL), the Symbolic Editor (SED), and the Layout Editor. The SDL product generates a design layout from a schematic. The SED product places multiple devices into a layout for device-level floorplanning that meets DRC requirements. The Layout Editor lets you further develop the design. See the *Custom Compiler SDL and Visually Assisted Automation Guide* for details. See the *Custom Compiler Symbolic Editor User Guide* for details. You can also use the Layout Editor to develop the layout. See the *Custom Compiler Layout Editor User Guide for Creating and Editing Objects* for details.

Feedback

Table 1 Design Phases and Products (Continued)

Design Phase	Product	Description
Routing	Layout Editor Custom Router	You can use the Layout Editor to create interconnects (or routes) manually. See the *Custom Compiler Layout Editor User Guide for Connecting Objects* for details. You can use the Custom Router to create interconnects (or routes) automatically or interactively for digital and mixed-signal nets at all levels of the chip design. For digital IC designs, you can pre-route sensitive nets using a set of custom routing options and continue with the Fusion Compiler, IC Compiler II, or IC Compiler tool to complete the physical implementation. You can also use the Custom Router to add shielding during the routing process or post route. You can choose from different shielding styles. In addition, you can set up shields such that they are shared between two adjacent signal routes. See the *Custom Compiler Custom Router User Guide* for details.
Co-Design	3DIC Compiler Fusion Compiler tool IC Compiler II tool IC Compiler Custom Compiler Environment	You can open Fusion Compiler, IC Compiler II, or IC Compiler designs in the Custom Compiler environment for digital and analog co-design flows. For example, you can use Custom Compiler products to design analog nets in a digital context, creating pre-routes before place-and-route. Then you can run place-and-route using the digital tool within the Custom Compiler environment. If the design was created in the Fusion Compiler, IC Compiler II, or IC Compiler tool, you can open the design in the Custom Compiler environment to add analog components. You can also import digital DRC errors into the Custom Compiler environment. See the *Custom Compiler and Fusion Compiler Co-design Assistant User Guide*, *Custom Compiler and IC Compiler II Co-design Assistant User Guide*, or Custom Compiler and IC Compiler Co-design Assistant User Guide for instructions. You can develop packaging designs using the 3DIC Compiler tool and the Custom Compiler tool. See the *Custom Compiler Packaging Designs User Guide* for instructions.
In-Design Error Checks	Layout Editor	While you are creating and editing the design, you can use the available in-design features to check for and correct design violations. • *DRD tool*: Interactively checks for violations in the entire design or in the viewport of your design. • *Via Checker*: Checks for missing or suboptimal vias in your design. • *Metal Density Analyzer*: Checks the metal density of your design by layer or by groups that you define. • *Color Decomposition Automation*: Automates the process in which colorable shapes across the hierarchy are assigned colors for multiple patterning. See the *Custom Compiler Design Checking and Physical Verification User Guide* for instructions.

Feedback

Table 1 Design Phases and Products (Continued)

Design Phase	Product	Description
In-Design Electrical Checks	EM Reporter Resistance Reporter Capacitance Reporter Voltage-Annotation Tools Shield Checker	A variety of electrical tools are available to permit you to check for integrated circuit performance and reliability issues during the design development process. Catching the issues early in the design phase can save time and mitigate costly iterations at the backend of the design cycle. See About Electrical Checking and Reporting for descriptions of the available electrical tools.
Physical Verification	Synopsys IC Validator Synopsys Hercules Siemens® Calibre® platform StarRC Custom Job	After you generate and develop the layout, you can verify that the layout was correctly designed by running design rule checking (DRC) and layout versus schematic (LVS) processes. You can choose from several DRC and LVS products integrated with the Custom Compiler environment. After DRC and LVS processing, you can perform layout parasitic extraction (LPE). **Note:** The DRC and LVS results must be error-free to perform parasitic extraction. Use the StarRC product and your LVS results to perform extraction. See the *Custom Compiler Design Checking and Physical Verification User Guide* for instructions. Physical verification also includes electrical checks. For information about checking and reporting electromigration, resistance, and shield coverage, see the Tools to Use for Electrical Checking and Reporting.
Post-Layout Simulation and Analysis	PrimeWave Design Environment	After layout parasitic extraction, you can perform post-layout simulation. See the *PrimeWave Design Environment User Guide* for instructions.

Software User Interface

The Custom Compiler environment provides a graphical user interface that lets you create and edit designs, run simulations, access in-design rule-checking and verification tools, monitor jobs, and review designs.

Feedback

This topic describes the main components of the Custom Compiler environment user interface:

- Home Tab
- Console Tab
- Tabs for Tools and Designs
- Shortcut Keys

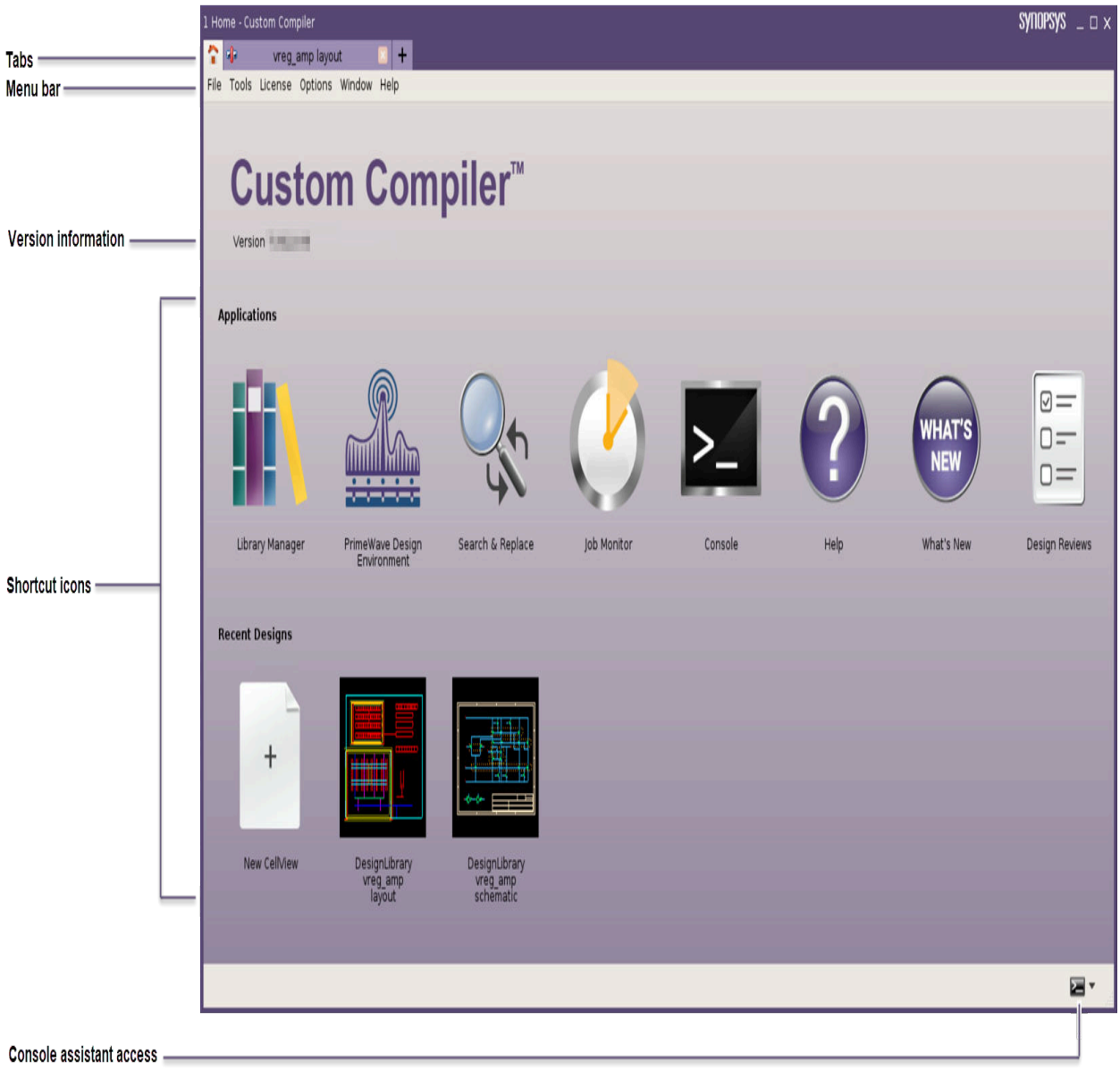
Home Tab

The **Home** tab is the starting point of the Custom Compiler environment. The tab displays the product version information and provides quick links to frequently accessed tools, assistants, and recently opened designs.

The **Home** tab:

- Displays automatically when you start a Custom Compiler session or open a new tab +.
- Is pinned and cannot be closed by default.
- Displays shortcuts that provide quick access to the following:
  - Library Manager
  - PrimeWave Design Environment
  - WaveView
  - Search & Replace
  - Job Monitor
  - Console
  - Design Reviews
  - Cellview creation dialog box
  - Recently opened designs

Feedback



The menu bar contains the following:

- File Menu
- Tools Menu
- License Menu
- Options Menu

- Window Menu
- Help Menu

See Also

- Pinning and Unpinning Tabs
- Console Assistant
- Customizing the Software and Workspaces

File Menu

The **Home** tab **File** menu contains commands that let you open, close, import, and export designs. You can also change the design mode to read-only and quit the Custom Compiler tool.

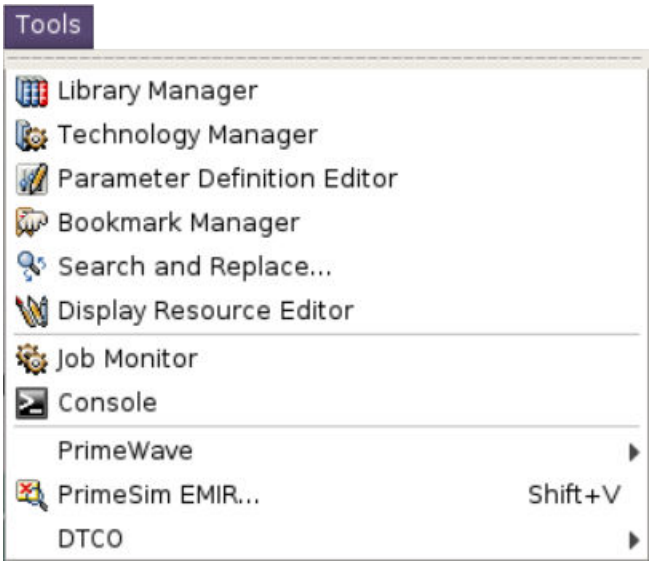
Command	Description
Open	Opens the **Open Design** dialog box, described in Opening Designs.
Close Designs	Opens the **Close Design** dialog box, in which you can select currently open designs to close. You are prompted to save any unsaved edits.
Refresh Designs	Opens the **Refresh Designs** dialog box, which lists all open, read-only, and out-of-date designs that have changed on disk since being opened in your session.
Make Designs Read-Only	Makes designs read-only.
Import	Lists the **Import** menu options, described in Import.
Export	Lists the **Export** menu options, described in Export.

Feedback

Command	Description
Quit	Opens the **Exit Custom Compiler** dialog box, described in Ending Sessions.

Tools Menu

The **Home** tab **Tools** menu lets you access design tools, such as the Library Manager, the Search and Replace tool, the Console tab, and the Job Monitor. You can also open the PrimeWave Design Environment and access the Reliability Analysis tools (EMIR and Static Res Check).

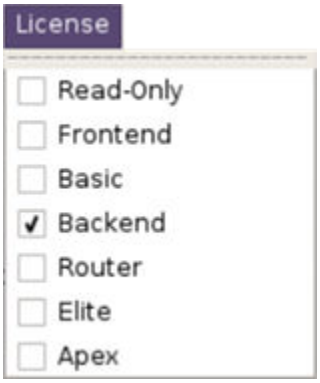


Command	Description
Library Manager	Opens the Library Manager in a new tab. See Library Manager Tool. Tcl preference: `dmShowLibraryManager`
Technology Manager	Opens the Technology Manager in a new tab. See Technology Manager Tool. Tcl command: `db::showManageTechnology`
Parameter Definition Editor	Opens the Parameter Definition Editor in a new tab. See Parameter Definition Editor Tool. Tcl command: `db::openParamDefEditor`
Bookmark Manager	Opens the Technology Manager in a new tab. See Bookmark Manager Tool. Tcl command: `de::showBookmarkManager`
Search and Replace	Opens the Search and Replace tool in a new tab. See Search and Replace Tool. Tcl command: `db::showFindReplace`

Command	Description
Display Resource Editor	Opens the Display Resource Editor in a new tab. See Display Resource Editor Tool. Tcl command: `dr::openDisplayResourcesEditor`
Job Monitor	Opens the Job Monitor in a new tab. See Job Monitor Tool. Tcl command: `xt::showJobMonitor`
Console	Opens the Console in a new tab. See Console Tab for details.
PrimeWave	Opens the PrimeWave Design Environment in a new tab when you choose **Single Testbench** or **Multiple Testbench**. Consult the *PrimeWave Design Environment User Guide* for details on using the PrimeWave Design Environment.
PrimeSim EMIR	Displays the PrimeSim EMIR user interface. See Reliability Verification.
DTCO	Displays the DTCO Setup and Run user interface.

License Menu

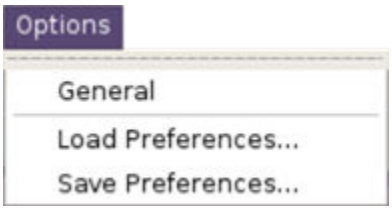
The **Home** tab **License** menu lets you choose the license to use. The Custom Compiler features that are available to you are based on the license that you choose.



Item	Description
<List of Licenses>	Displays the list of installed licenses. The licenses listed here vary depending on which license packages you have purchased. Consult your Synopsys representative for information about licensing. Licenses that are checked out show a check mark next to the license name. Clicking on a checked-out license releases that license and closes windows. Clicking on a license that is not checked out checks out and reserves that license for the duration of the session.

Options Menu

The **Home** tab **Options** menu contains the commands that let you specify design viewing and other preferences. For example, you can specify the font styles and type of keyboard shortcuts (bindkey settings). You can set the preferences individually, or load a saved set of preferences that were previously saved.

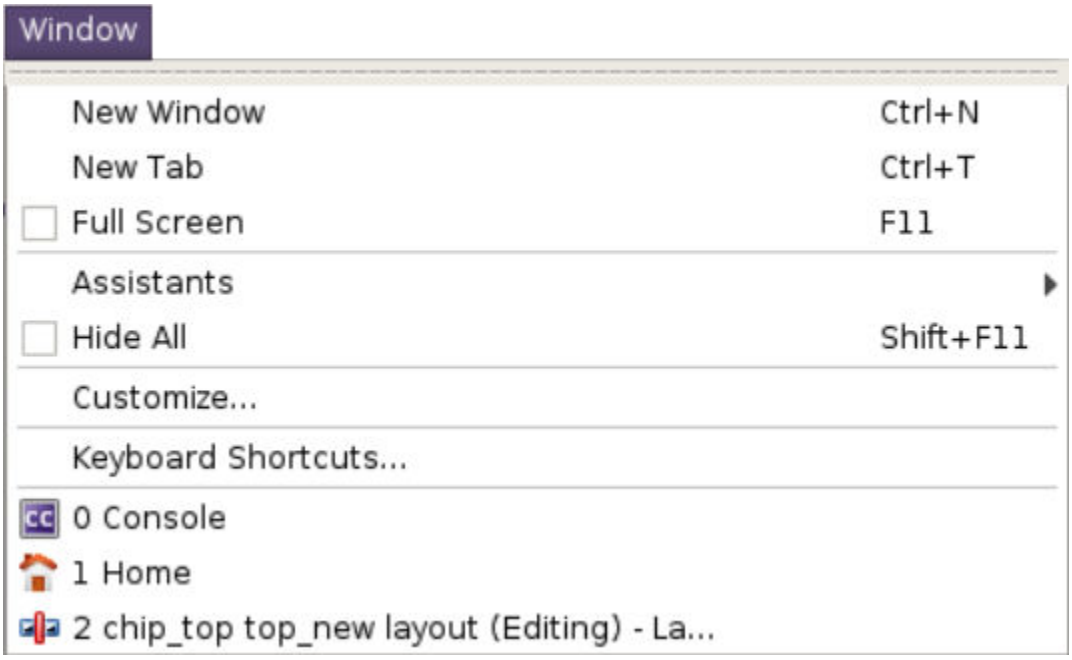


Command	Description
General	Opens the **General Options** dialog box, described in Setting General Options.
Load Preferences	Opens the **Load Preferences** dialog box, described in Loading Option Settings Saved as Preferences.
Save Preferences	Opens the **Save Preferences** dialog box, described in Saving Option Settings as Preferences.

Window Menu

The **Home** tab **Window** menu lets you open a new window or tab, display or hide assistants, customize the window settings, and add keyboard shortcuts.

Feedback

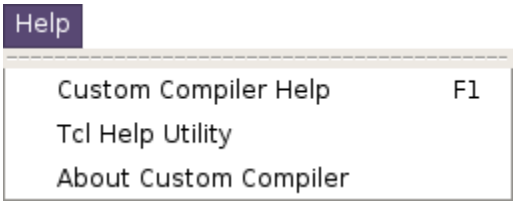


Command	Description
New Window	Opens a Custom Compiler session in a new window.
New Tab	Opens a new tab in the existing window. This tab automatically opens to the **Home** tab.
Full Screen	Expands the Custom Compiler window to full screen by hiding the window title bar. Shortcut key: F11.
Assistants	Lists the currently available assistants. See Tools for Designing for descriptions of Custom Compiler tools and assistants.
Hide All	Hides and displays the title bar, menu bar, status bar, and all toolbars and assistants within a window tab. Shortcut key: Shift+F11.
Customize	Opens the **Customize Window** dialog box.
Keyboard Shortcuts	Opens the **Keyboard Shortcuts** dialog box. See Customizing Keyboard Shortcuts.
<List of Open Tabs in the Current Window>	The **Window** menu includes a list of open tabs in the current window.

Feedback

Help Menu

The **Home** tab **Help** menu lets you access the Custom Compiler help, the Tcl Help Utility, and the product version information.



Command	Description
Custom Compiler Help	Displays the Custom Compiler help in a new tab.
Tcl Help Utility	Opens the Tcl Help Utility tab and displays the Custom Compiler Tcl syntax and descriptions in a graphical user interface. See Using the Tcl Help Utility.
About Custom Compiler	Opens the **About Custom Compiler** dialog box containing information about the Custom Compiler version number, copyright, legal notices, and Synopsys contact information.

Console Tab

The **Console** tab displays the product version information, error messages, and status information.

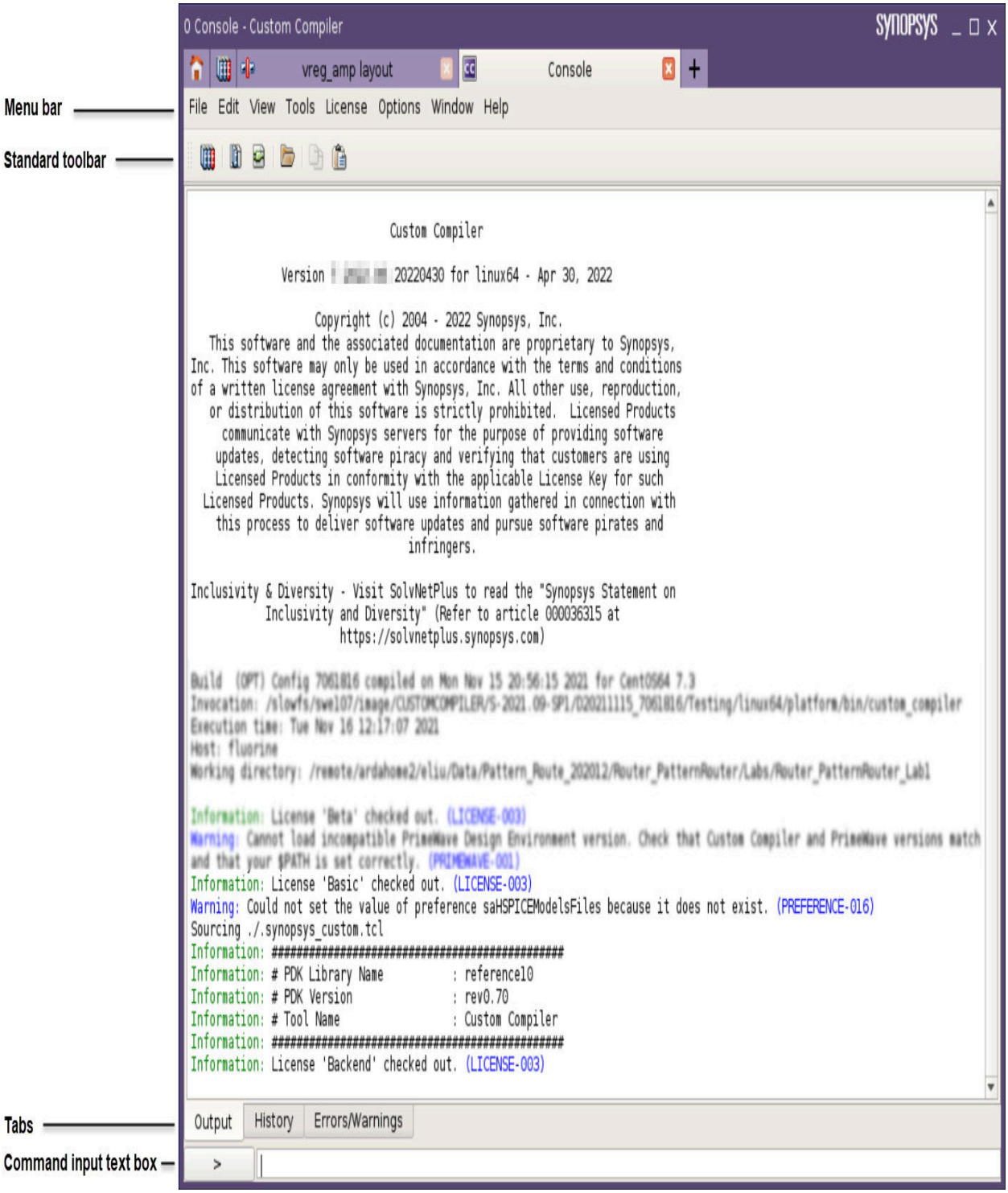
To display the **Console** tab, on the **Home** tab, click the **Console** shortcut icon. Alternatively, choose **Tools > Console**.

Feedback

1 Home - Custom Compiler
SYNOPSYS
File Tools License Options Window Help
Custom Compiler™
Version
Applications
Library Manager
PrimeWave Design Environment
Search & Replace
Job Monitor
Console
Help
What's New
Design Reviews
Recent Designs
New CellView
DesignLibrary vreg_amp layout
DesignLibrary vreg_amp schematic

The **Console** tab consists of the following components:

- Menu Bar
- Standard Toolbar
- Tabs
- Command Input Text Box


Menu bar
Standard toolbar
Tabs
Command input text box
0 Console - Custom Compiler
SYNOPSYS
vreg_amp layout
Console
File Edit View Tools License Options Window Help
Custom Compiler
Version 20220430 for linux64 - Apr 30, 2022
Copyright (c) 2004 - 2022 Synopsys, Inc.
This software and the associated documentation are proprietary to Synopsys, Inc. This software may only be used in accordance with the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, or distribution of this software is strictly prohibited. Licensed Products communicate with Synopsys servers for the purpose of providing software updates, detecting software piracy and verifying that customers are using Licensed Products in conformity with the applicable License Key for such Licensed Products. Synopsys will use information gathered in connection with this process to deliver software updates and pursue software pirates and infringers.
Inclusivity & Diversity - Visit SolvNetPlus to read the "Synopsys Statement on Inclusivity and Diversity" (Refer to article 000036315 at https://solvnetplus.synopsys.com)
Build (OPT) Config 7061816 compiled on Mon Nov 15 20:56:15 2021 for Cent0S64 7.3
Invocation: /slowfs/twel07/image/CUSTOMCOMPILER/S-2021.09-SP1/020211115_7061816/Testing/linux64/platform/bin/custom_compiler
Execution time: Tue Nov 16 12:17:07 2021
Host: fluorine
Working directory: /remote/ardahoe2/eliu/Data/Pattern_Route_202012/Router_PatternRouter/Labs/Router_PatternRouter_Lab1
Information: License 'Beta' checked out. (LICENSE-003)
Warning: Cannot load incompatible PrimeWave Design Environment version. Check that Custom Compiler and PrimeWave versions match and that your $PATH is set correctly. (PRIMEWAVE-001)
Information: License 'Basic' checked out. (LICENSE-003)
Warning: Could not set the value of preference saHSPICEModelsFiles because it does not exist. (PREFERENCE-016)
Sourcing ./.synopsys_custom.tcl
Information: ##
Information: # PDK Library Name : reference10
Information: # PDK Version : rev0.70
Information: # Tool Name : Custom Compiler
Information: ##
Information: License 'Backend' checked out. (LICENSE-003)
Output
History
Errors/Warnings
>

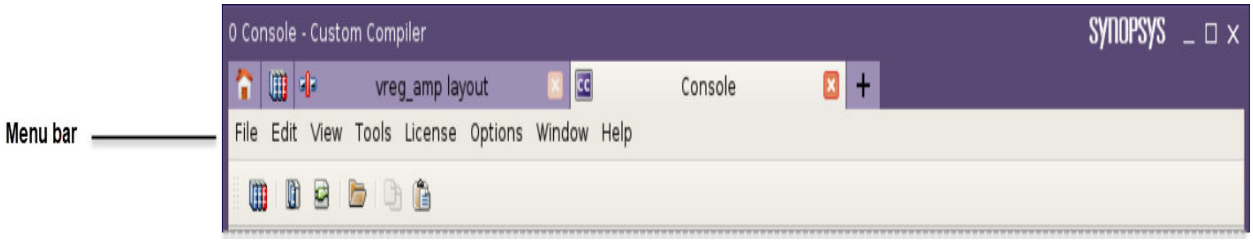
Feedback

See Also

- Console Assistant

Menu Bar

The **Console** menus contain mostly the same commands that are also available on the **Home** tab. Additional commands access the Abstract Generator and integrated simulation tools.



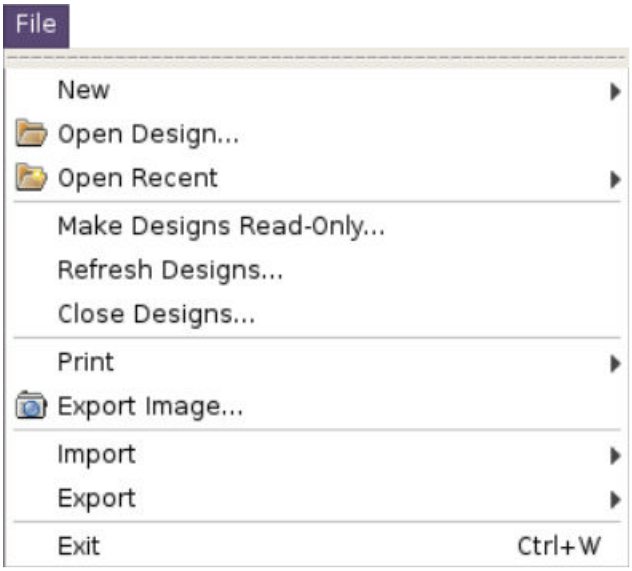
The following sections describe the Console menus.

- File Menu
- Edit Menu
- View Menu
- Tools Menu
- Console License Menu
- Options Menu
- Window Menu
- Help Menu

File Menu

The **Console** tab **File** menu contains commands that let you create new designs, open and close existing designs, import and export designs, change design modes to read-only, and refresh and apply updates to designs.

Feedback



Command	Description
New	Choose **Library** to open the **New Library** dialog box, described in Creating New Libraries. Choose **CellView** to open the **New CellView** dialog box, described in Managing Cellviews.
Open Design	Opens the **Open Design** dialog box, described in Opening Designs.
Open Recent	Lists all recent designs.
Make Designs Read-Only	Makes designs read-only.
Refresh Designs	Opens the **Refresh Designs** dialog box, which lists all open, read-only, and out-of-date designs that have changed on disk since being opened in your session.
Close Designs	Opens the **Close Design** dialog box, in which you can select currently open designs to close. You are prompted to save any unsaved edits.
Print	**Submit** opens the **Print** dialog box, described in Printing and Plotting Designs. **Queue** opens the print queue, described in Checking the Print Queue.
Export Image	Opens the **Export Image** dialog box, described in Exporting Design Images.
Import	Lists the **Import** menu options, described in Import.

Feedback

Command	Description
Export	Lists the **Export** menu options, described in Export.
Exit	Opens the **Exit Custom Compiler** dialog box, described in Ending Sessions.

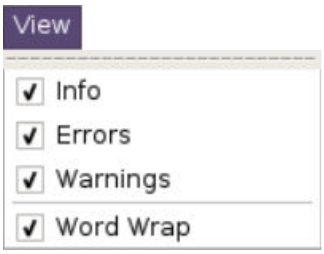
Edit Menu

The **Console** tab **Edit** menu lets you copy and paste selections in the command input text box.

Command	Description
Copy	Copies the current selection in the active pane to the clipboard.
Paste	Pastes the clipboard contents into the command line.

View Menu

The **Console** tab **View** menu lets you display or hide the Console tabs.



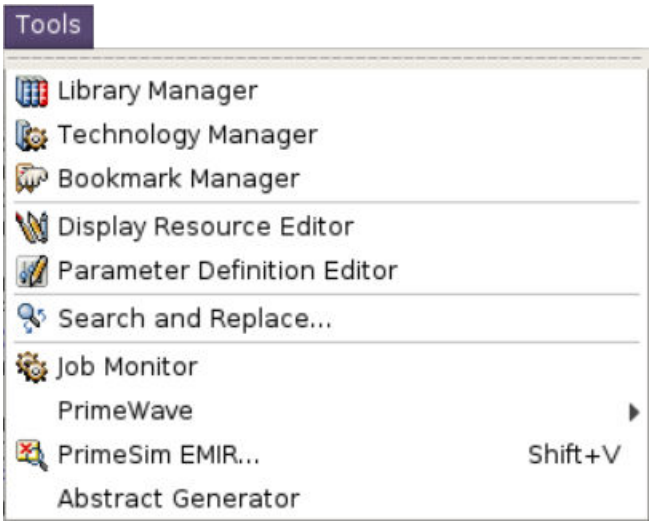
Command	Description
Info	Displays Custom Compiler informational messages in the **Output** tab. On by default. Tcl preference: `dbViewInfo`
Errors	Displays Custom Compiler error messages in the **Output** tab. On by default. Tcl preference: `dbViewErrors`

Feedback

Command	Description
Warnings	Displays Custom Compiler warning messages in the **Output** tab. On by default. Tcl preference: `dbViewWarnings`
Word Wrap	Controls whether the output in the various panes extends to the right of the window and is viewable using a horizontal scroll bar or whether the text automatically wraps to the next line. Tcl preference: `dbViewWordWrap`

Tools Menu

The **Console** tab **Tools** menu lets you access design tools, such as the Library Manager, the Search and Replace tool, the Console tab, and the Job Monitor. You can also open the PrimeWave Design Environment, access the Reliability Analysis tools (EMIR and Static Res Check), and the Abstract Generator.



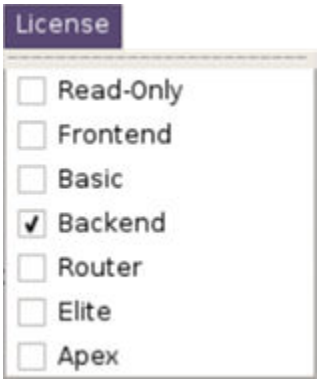
Command	Description
Library Manager	Opens the Library Manager in a new tab. See Library Manager Tool Tcl preference: `dmShowLibraryManager`
Technology Manager	Opens the Technology Manager in a new tab. See Technology Manager Tool Tcl preference: `db::showManageTechnology`
Bookmark Manager	Opens the Technology Manager in a new tab. See Bookmark Manager Tool Tcl preference: `de::showBookmarkManager`

Feedback

Command	Description
Display Resource Editor	Opens the Display Resource Editor in a new tab. See Display Resource Editor Tool Tcl preference: `dr::openDisplayResourcesEditor`
Parameter Definition Editor	Opens the Parameter Definition Editor in a new tab. See Parameter Definition Editor Tool Tcl preference: `db::openParamDefEditor`
Search and Replace	Opens the Search and Replace tool in a new tab. See Search and Replace Tool Tcl preference: `db::showFindReplace`
Job Monitor	Opens the Job Monitor in a new tab. See Job Monitor Tool Tcl preference: `xt::showJobMonitor`
PrimeWave	Choose **Single Testbench** or **Multiple Testbench** to open the PrimeWave Design Environment in a new tab. Consult the *PrimeWave Design Environment User Guide* for details on using the PrimeWave Design Environment.
PrimeSim EMIR	Displays the PrimeSim EMIR user interface. See Reliability Verification.
Abstract Generator	Opens the **Generate Abstract** dialog box, described in Displaying the Abstract Generator Options Dialog Box.

Console License Menu

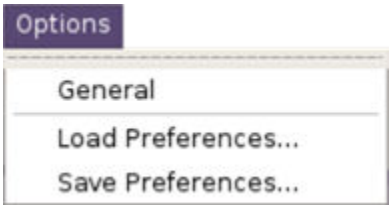
The **Console** tab **License** menu lets you choose the license to use. The Custom Compiler features that are available to you are based on the license that you choose.



Item	Description
<List of Licenses>	Displays the list of installed licenses. The licenses listed here vary depending on which license packages you have purchased. Consult your Synopsys representative for information about licensing. Licenses that are checked out show a check mark next to the license name. Clicking on a checked-out license releases that license and closes windows. Clicking on a license that is not checked out checks out and reserves that license for the duration of the session.

Options Menu

The **Console** tab **Options** menu contains the commands that let you specify design viewing and other preferences. For example, you can specify the font styles and type of keyboard shortcuts (bindkey settings). You can set the preferences individually, or load a saved set of preferences that were previously saved.

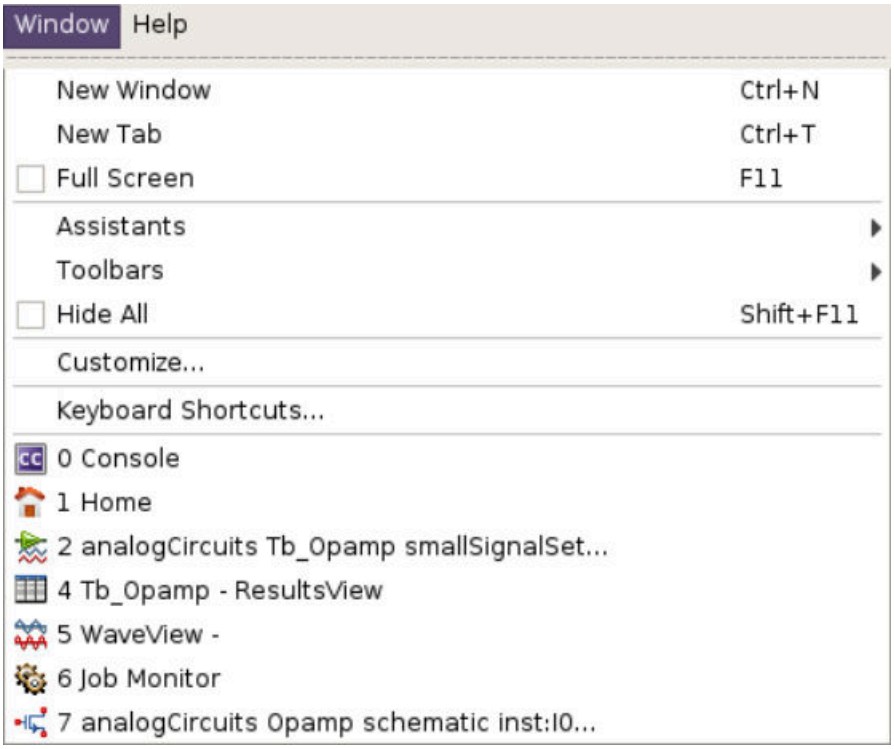


Command	Description
General	Opens the **General Options** dialog box, described in Setting General Options.
Load Preferences	Opens the **Load Preferences** dialog box, described in Loading Option Settings Saved as Preferences.
Save Preferences	Opens the **Save Preferences** dialog box, described in Saving Option Settings as Preferences.

Window Menu

The **Console** tab **Window** menu lets you open a new window or tab, display or hide assistants, custom the window settings, and add keyboard shortcuts.

Feedback

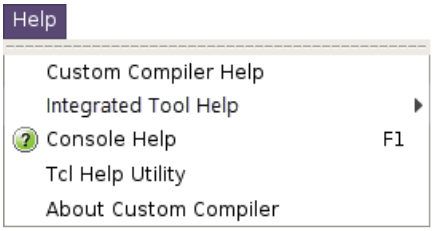


Command	Description
New Window	Opens a Custom Compiler session in a new window.
New Tab	Opens a new tab in the existing window. This automatically opens to the **Home** tab.
Full Screen	Expands the Custom Compiler window to full screen by hiding the window title bar. Shortcut key: F11.
Assistants	Lists the currently available assistants. See Tools for Designing, for descriptions of Custom Compiler tools and assistants.
Toolbars	Lists the currently available toolbars.
Hide All	Hides and displays the title bar, menu bar, status bar, and all toolbars and assistants within a window tab. Shortcut key: Shift+F11.
Customize	Opens the **Customize Window** dialog box.
Keyboard Shortcuts	Opens the **Keyboard Shortcuts** dialog box. See Customizing Keyboard Shortcuts.

Command	Description
<List of Open Tabs in the Current Window>	The **Window** menu includes a list of open tabs in the current window.

Help Menu

The **Console** tab **Help** menu lets you access the Custom Compiler help, the Tcl Help Utility, and the product version information. You can also access help for the integrated tools.

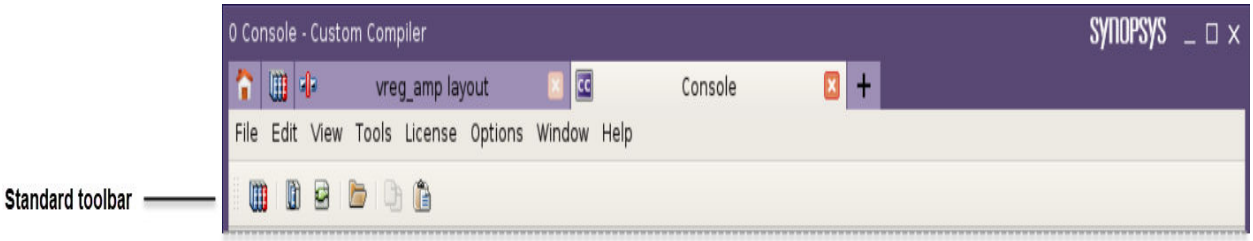


The following table describes the **Help** menu commands:

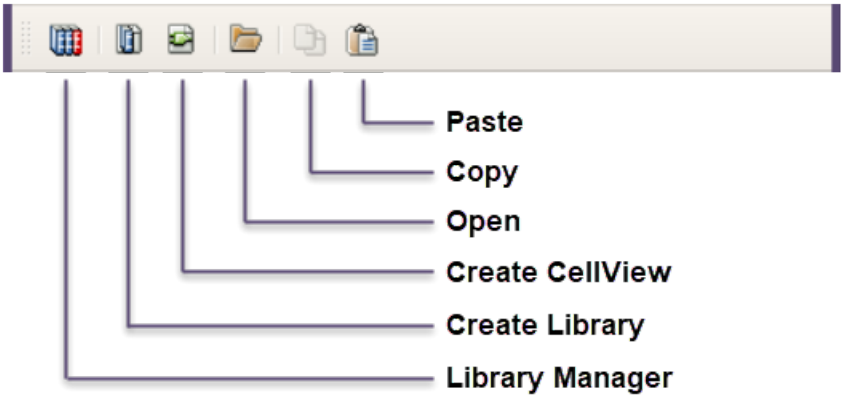
Command	Description
Custom Compiler Help	Opens a browser in a new tab with the online help documentation.
Integrated Tool Help	Provides links to supported simulator help sets.
Console Help	Opens a browser in a new tab to the help topic about the Console.
Tcl Help Utility	Opens the Tcl Help Utility tab and displays the Custom Compiler Tcl syntax and descriptions in a graphical user interface. See Using the Tcl Help Utility.
About Custom Compiler	Opens the **About Custom Compiler** dialog box containing information about the Custom Compiler version number, copyright, legal notices, and Synopsys contact information,

Standard Toolbar

The **Console** standard toolbar offers quick access to commonly used commands that let you open the Library Manager, create new libraries, create new cellviews, and copy and paste text in the command input text box.



The Console standard toolbar is displayed by default. If it is hidden, choose **Window > Toolbars > Standard** in the **Console** tab.



Button Icon	Description
Library Manager	Displays the Library Manager.
Create Library	Creates a library.
Create CellView	Creates a cellview.
Open	Browses to open a file.
Copy	Copies the currently selected text.
Paste	Pastes copied text to the current location.

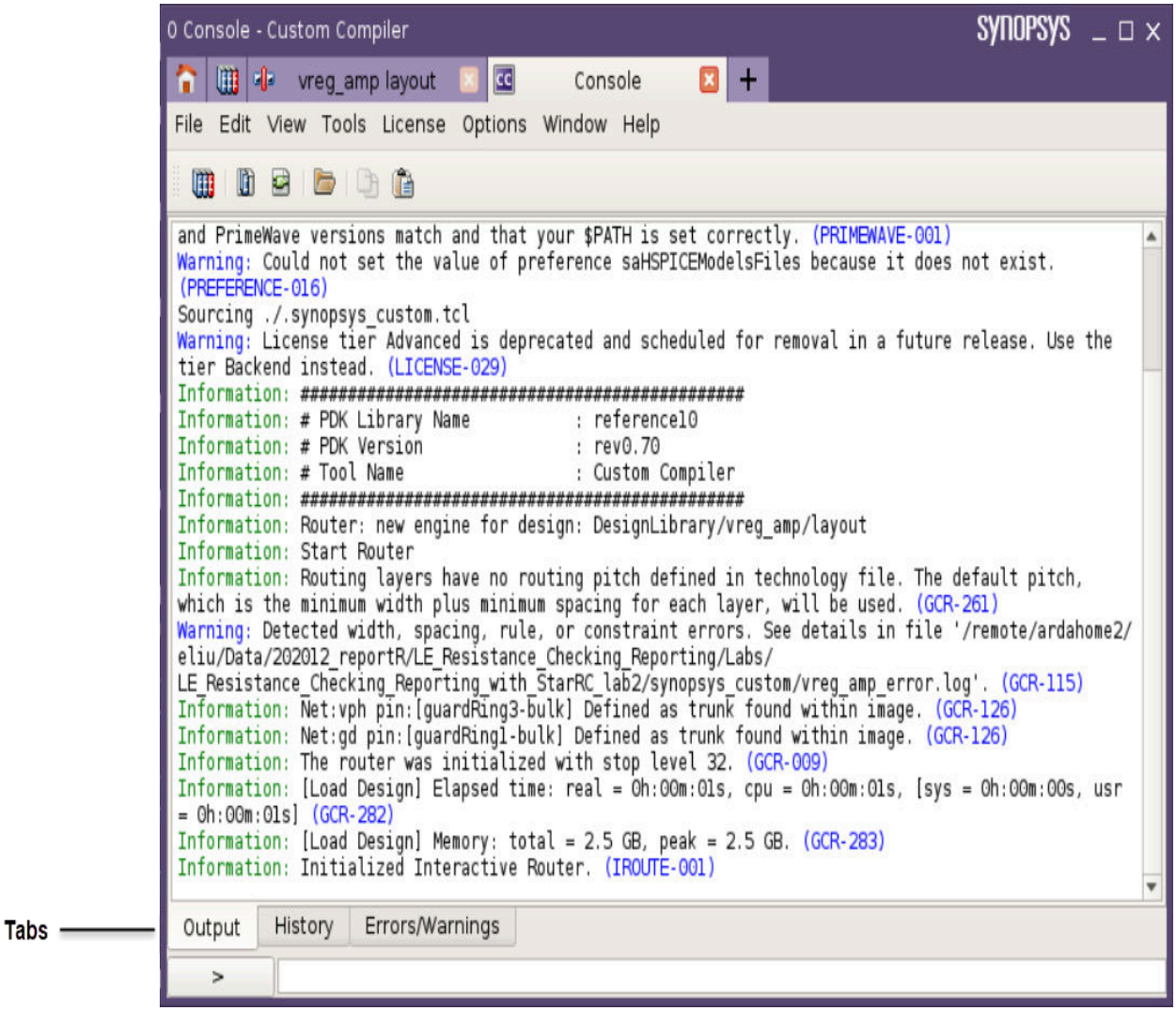
Tabs

The **Console** tab contains subtabs that categorize the following information: session log, history of executed commands and preferences, and error messages.

Feedback

The **Output** tab displays the Custom Compiler session log, which includes the following:

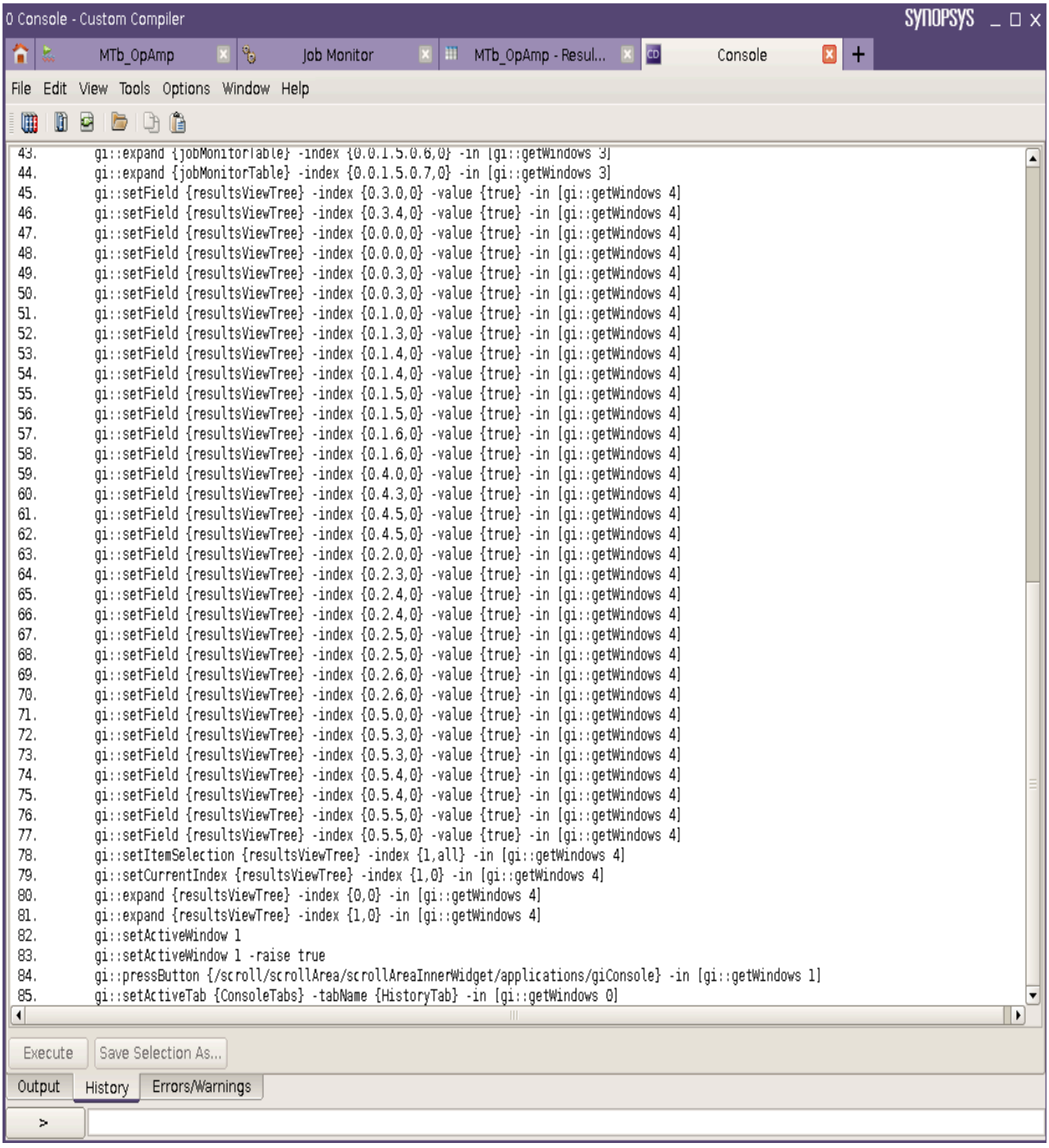
- Commands you typed in the command input text box.
- Output from running the commands that you typed in the command input text box.
- Error, warning, and information messages that result from running the commands.



The **History** tab shows only executed Custom Compiler commands and Tcl expressions in individual rows. The tab is populated with the contents of the *.history* file, which includes all commands executed in this session, including those that occur as a result of GUI inputs, such as mouse clicks, menu choices, and shortcut keys.

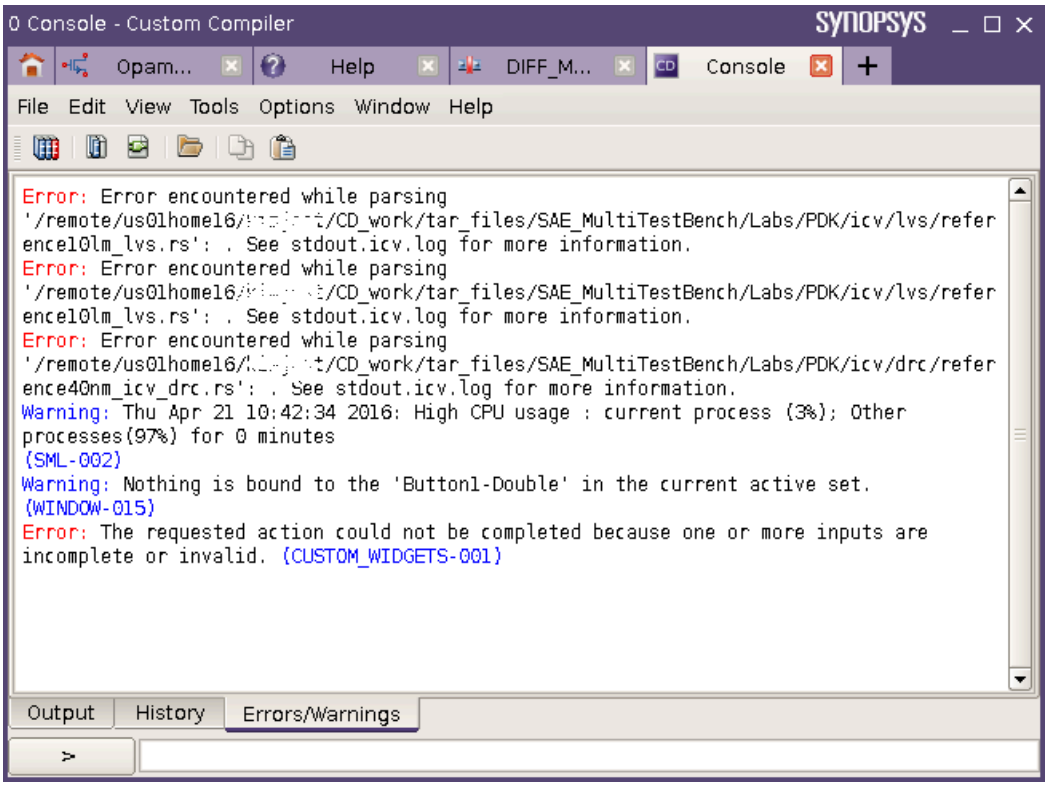
Feedback

- You can select a row or multiple rows, and click **Execute** to run the selected commands.
- You can select a row or multiple rows, and click **Save Selection As** to save the selection to a text file.



Feedback

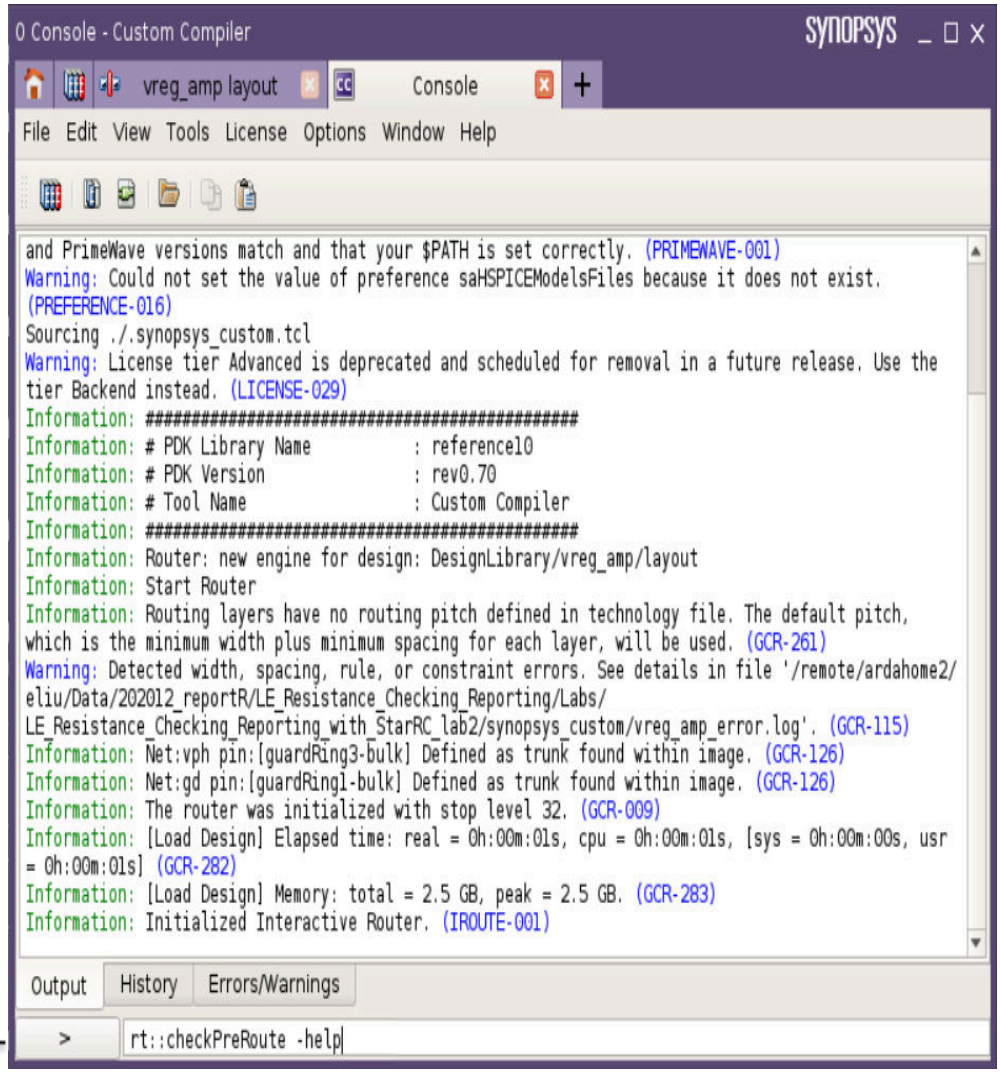
The **Errors/Warnings** tab shows only the error and warning messages that were generated for this particular Custom Compiler session. Errors are displayed in red text, and warnings are displayed in blue text.



Command Input Text Box

The **Console** command input text box accepts Tcl commands and canvas coordinates for design operations.

Feedback



Some Tcl commands accept relative and absolute coordinates for drawing operations as arguments. Instead of switching back and forth between the **Console** tab and the design editor tab when using drawing commands, you can enter the commands in the Console Assistant, which is displayed within the design editor tab.

If you want to run Python commands or expressions in the Custom Compiler environment, you can do so in the Console Assistant. See Python in the Custom Compiler Environment and Typing Python Commands in the Console assistant.

See Also

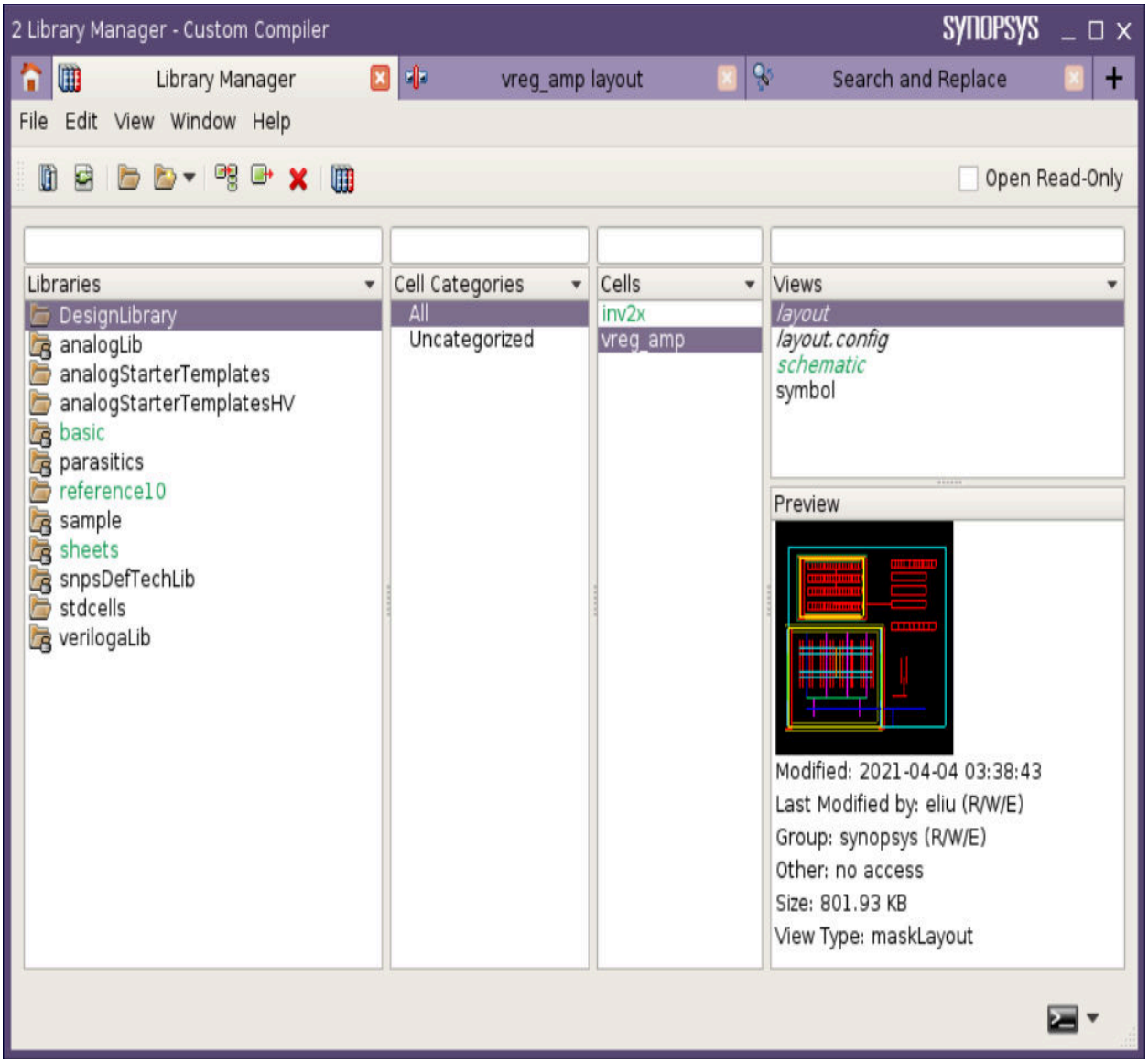
- Introduction to Custom Compiler Tcl

Feedback

Tabs for Tools and Designs

From the Home Tab , you can open various tools and designs. When you do so, a new tab for the tool or design opens in the window.

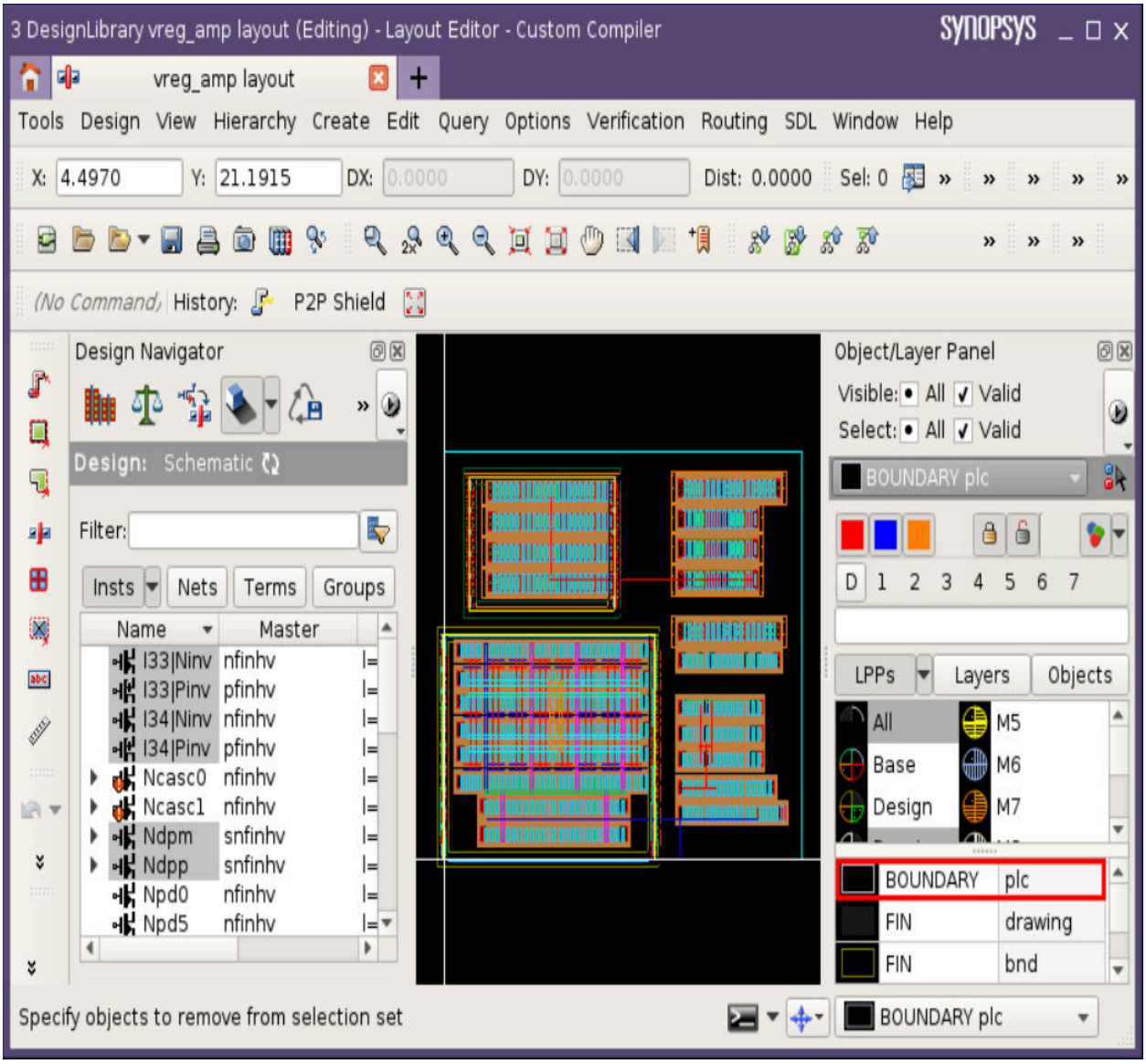
In the following example, the Library Manager tool is opened and displayed in a tab. Notice that other tools are also opened in tabs. For more information about the tools you use to assist in the design process, see Tools for Designing.



Feedback

When you open a schematic or layout design, it appears in a new tab. In the following example, a layout design named `vreg_amp layout` is opened and displayed in a tab. For more information about opening designs, see General Designing.

For information about the menus, commands, and toolbars available for specific tools or editors, see Design Phases and Products to Use.

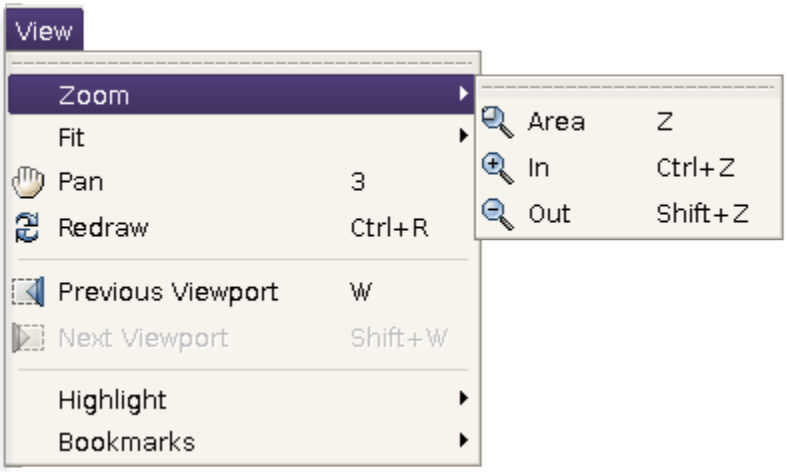


To return to the **Home** tab, click in the upper left corner of the window.

Shortcut Keys

Some commands have corresponding shortcut keys that you can use as an alternative to using the mouse. For example, you can press F1 to open the main page of the online help instead of choosing **Help** > **Help**.

Menu command shortcut keys are displayed in the menus next to the command names. The following example shows the shortcut keys for the **View** menu commands.



You can choose from different sets of shortcut keys. For more information, see Customizing Keyboard Shortcuts.

Common Graphical User Interface Operations

In the Custom Compiler environment, common graphical user interface operations are applicable across different assistants and other components.

This topic describes the following:

- Pinning and Unpinning Tabs
- Undocking and Docking Assistants
- Freezing Table Columns

Pinning and Unpinning Tabs

In the Custom Compiler window, you can pin tabs so that they remain persistent and readily accessible.

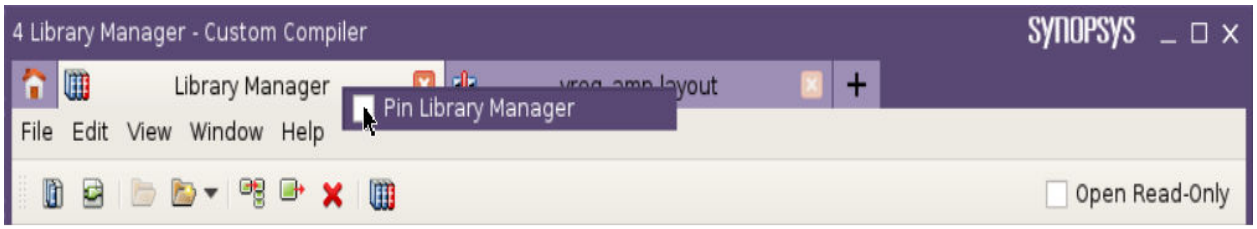
Feedback

Pinning Tabs

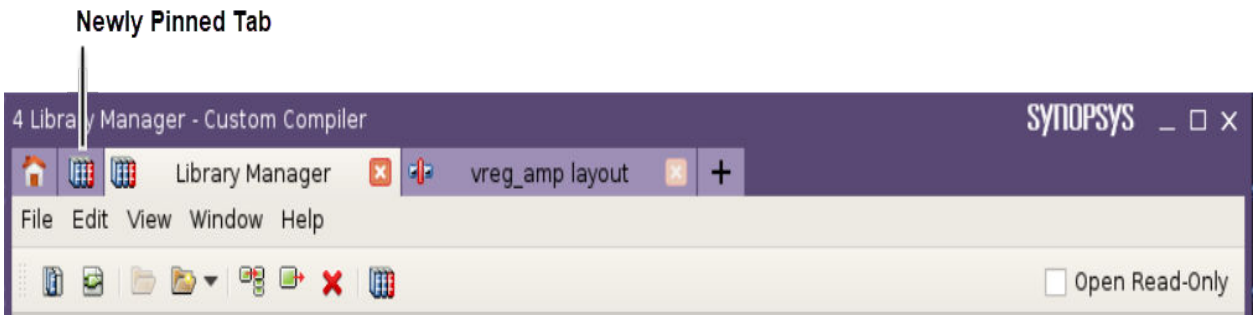
You can pin the Library Manager and Review Manager tabs.

To pin a tab:

1. Open the tool, such as the Library Manager.
2. Right-click the tab, and select **Pin <tool_name>**.



The tab becomes smaller and displays only the icon. The pinned tab appears to the right of the **Home** tab.



Unpinning Tabs

You can unpin tabs so that they are no longer accessible when you close them.

To unpin a tab:

► Right-click the pinned tab, and clear the **Pin <tool_name>** check box.

Undocking and Docking Assistants

You can undock assistants to move them to preferred positions in the Custom Compiler window. You can redock the assistants at any time to restore their default positions.

Feedback

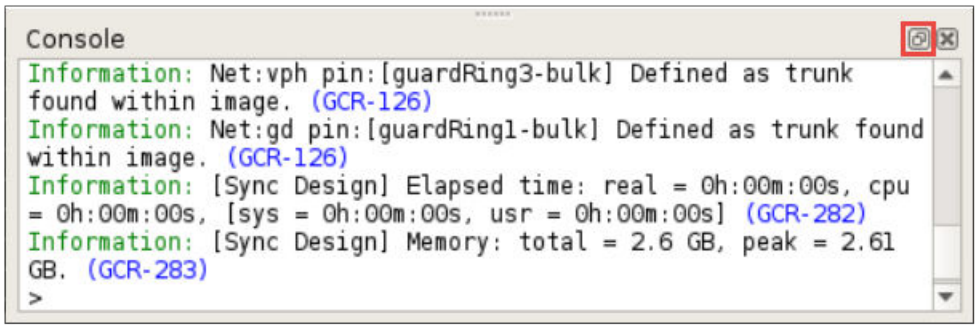
Undocking Assistants

To undock assistants:

1. Click in the assistant pane.

   Alternatively, click in the upper border of the assistant pane and drag it, or double-click the upper border of the assistant pane.

   The following example shows the location of the button in the Console assistant.



2. Move the assistant to its new position.

Docking Assistants

To dock assistants:

► Click in the assistant pane.

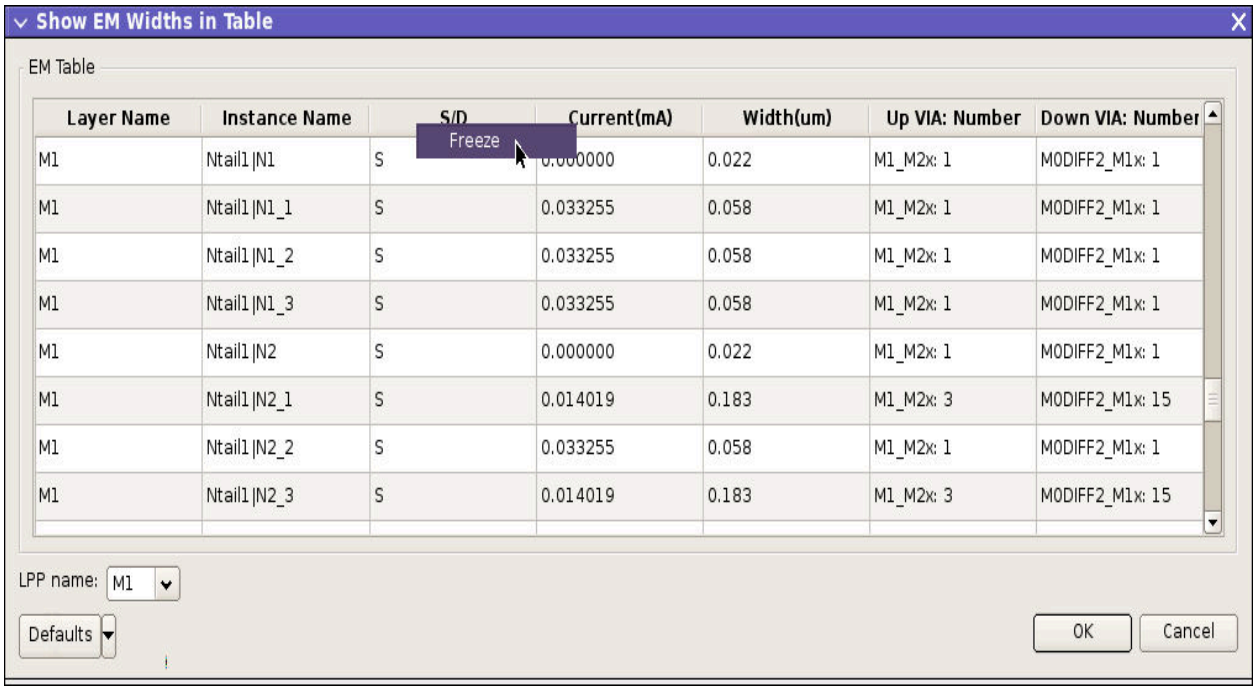
Freezing Table Columns

When viewing data in tables in the Custom Compiler graphical user interface, you have the option of locking (or freezing) some of the columns so that they remain visible while you scroll horizontally through the remaining columns.

Feedback

To freeze table columns:

► Right-click the column heading and choose **Freeze**.



For example, to freeze the left-most column only, right-click the heading at the top of the left-most column. To freeze the first three columns, right-click the heading at the top of the third column.

Environment Management

You can manage the environment by controlling session settings, creating a custom design work environment, managing display resources, managing the technology used for designs, and importing and exporting design data.

This topic describes the following:

- Session Settings
- Technology Data and Process Constraints
- Display Resources
- Instance Object Parameters
- Design Import and Export Requirements

- Job Process Monitoring
- Errors and Event Logs

Session Settings

You can configure session settings to create a standard startup process and initial design environment for both individual designers and groups.

The available session settings are:

- Environment variables
- General options, for example, cursor double-click speed, the designated text editor, and whether to apply infix mode
- Preferences that predefine input values for dialog box entry fields and design operations
- Preferences that specify whether windows, net selections, viewports, and command settings are synchronized across opened designs.
- Bindings (bindkeys, shortcut keys, key bindings) for shortcuts to keyboard and mouse operations

These are the ways you can control the application of settings:

- You can apply settings at different distribution scopes, from the current working directory to the project site.
- You can apply settings at startup to make them automatically available for every design session.

For details, see Defining Startup Settings and Defining Session Settings.

Technology Data and Process Constraints

You can manage the technology data and process constraints in design sessions.

Using the Technology Manager, you can:

- Add a technology database.
- Switch the technology referenced by a library to a different technology.
- Import technology data to add a technology database where one did not exist before, or to overwrite or merge imported data by compiling it into an existing technology database.

- Export technology data from a library to a new technology file.
- View the process constraints defined in the technology.

For details, see Technology Manager Tool. For more information about supplied technology files, see Technology and Display Resource Files in the *Custom Compiler Technology File and Display Resource Reference Manual*.

Display Resources

You can use display resources to determine the visual appearance of a design on different output devices, from monitors to plotters.

Using the Display Resource Editor, you can perform these operations:

- Define visual attributes such as colors, stipples, or line styles
- Define a display packet that represents a set of visual attributes
- Create a packet alias when you want to alter a packet's visual attributes from those listed in the technology file, but not overwrite them
- Load new, save changed, and merge two sets of display resources

For details, see Display Resource Editor Tool. For information about supplied display resource files, see Technology and Display Resource Files in the *Custom Compiler Technology File and Display Resource Reference Manual*.

Instance Object Parameters

Parameters are metadata associated with instance objects. Parameters differ from properties in that they can only be associated with instance objects. Additionally, the definition of the parameter cannot be created directly on an instance. Instead, the parameter definitions are inherited from data stored in the instance master's associated cell and library.

You can use the Parameter Definition Editor to access and edit the object parameters. For instructions, see Parameter Definition Editor Tool.

Design Import and Export Requirements

You can import and export design data in a variety of standard formats.

Import formats and data include:

- IC Compiler format
- GDSII Stream

- OASIS
- Library Exchange format (LEF)
- Design Exchange format (DEF)
- Netlist Text

Export formats include:

- IC Compiler format
- GDSII Stream
- OASIS
- Library Exchange format (LEF)
- Design Exchange format (DEF)
- Liberty File
- Netlist

For detailed instructions, see Import and Export.

For information about performing batch translation of design libraries to and from the Synopsys IC Compiler environment, see the *Custom Compiler and IC Compiler Co-Design Assistant User Guide* for details.

For more information about designing by using both the Custom Compiler environment and the IC Compiler product, see the *Custom Compiler and IC Compiler Co-Design Assistant User Guide* for details.

Job Process Monitoring

You can use the Job Monitor Tool to track the progress of running jobs, such as design importing and exporting operations, auto routing operations, and simulation progress.

You can view the process output using one of the following:

- Console Tab's **Output** tab. You can also use the **Errors/Warnings** tab to filter output to view errors and warnings.
- Session log file that is stored in the current working directory.
- Text Viewer Tool to display and dynamically update text output for all currently running job processes from the environment.

For processes that change or annotate designs, you can open the design as instructed for that process.

Errors and Event Logs

The Console displays all events and error information. For information about the Console, see Console Tab.

Custom Compiler Tcl

In addition to the graphical user interface, you can use the Custom Compiler Tcl interpreter to interact with the Custom Compiler environment and your designs.

- Executing Individual Tcl Commands
- Running Tcl Scripts
- Using the Tcl Help Utility

Executing Individual Tcl Commands

You can execute individual Tcl commands as follows:

- Type the Tcl command in the **Console** pane or in the command input text box in the **Console** tab, and then press Enter.

  In case you cannot remember the name of the command, you can list the command in the Console, select it, and then click **Execute**. For details about listing available Tcl commands, see the *Custom Compiler Tcl Reference Manual* for details.

- To repeat a command from this session, click the **History** tab at the bottom of the **Console** tab, select the command from the history list, and click **Execute**.
- To save a command operation from this session, select it in the history list, click **Save Selection As**, and choose a Tcl file.

You can also create, register, and use your own Tcl commands. Both CAD support groups and designers can use presentation layer (OPAL) Tcl commands to develop custom user interfaces supporting best-practice design processes. Commands in the Custom Compiler environment `gi` namespace create custom user interfaces.

Running Tcl Scripts

You can run Tcl scripts to automate Tcl command sequences. You can:

- Create a Tcl script. Open a text file and type the Tcl script.

  For information about the Tcl commands and preferences, see the *Custom Compiler Tcl Reference Manual* for details. To register new Tcl commands, use `de::createCommand`.

- Source a Tcl script. At the Console prompt, type:

  ```
  source [filePath] <script_name>.tcl
  ```

  You can enter the first few unique characters of the file name, and then press **Tab** to see the list of all available Tcl scripts in the directory.

  After sourcing the Tcl script, you can run it.

- Manually run a Tcl script. At the Console prompt, type:

  ```
  <namespace>::<command>
  ```

  If you are not familiar with the script, open the script file, and find the namespace declared at the top and follow that with the Tcl procedure you want to run.

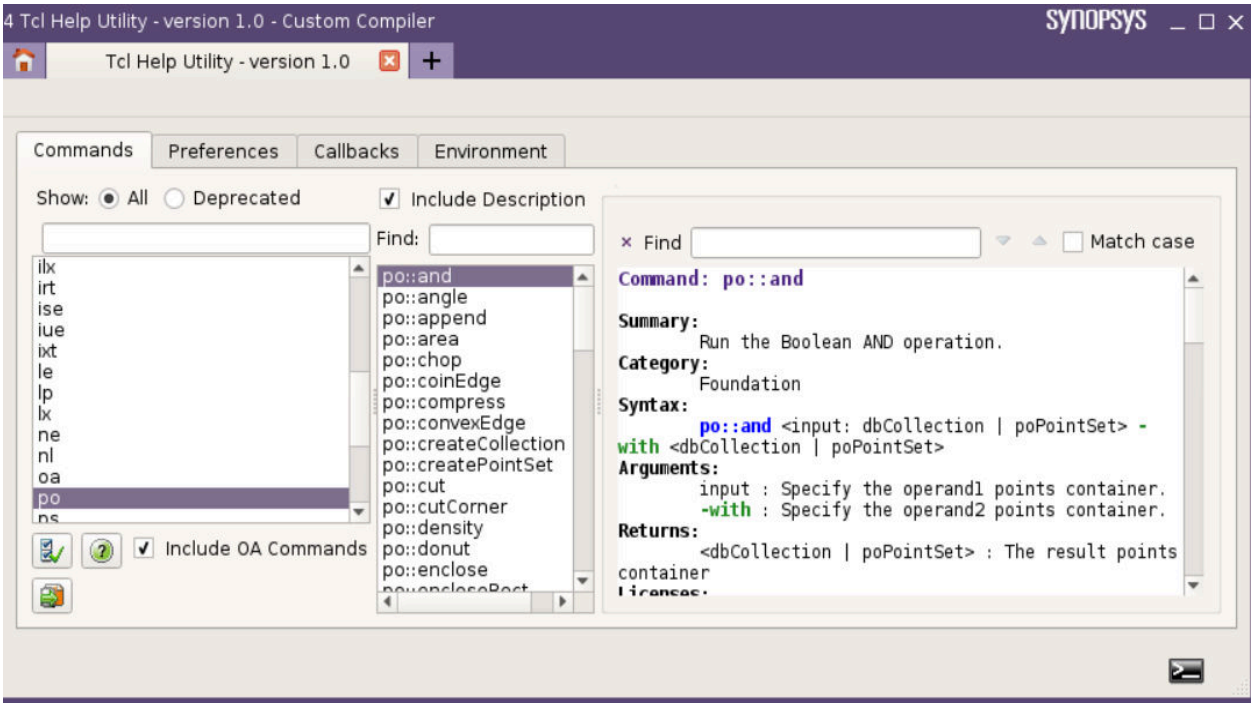
- Automatically run a Tcl script on startup. Store the script file in one of the startup directories at the distribution scope you prefer.

Using the Tcl Help Utility

You can use the **Tcl Help Utility** by selecting **Help** > **Tcl Help Utility** to enable the **Tcl Help Utility** window to view the following details of Tcl information.

	The Tcl Help Utility lets you control the display of Tcl commands and preferences. This video tutorial shows how to use the Tcl Help Utility.

Feedback

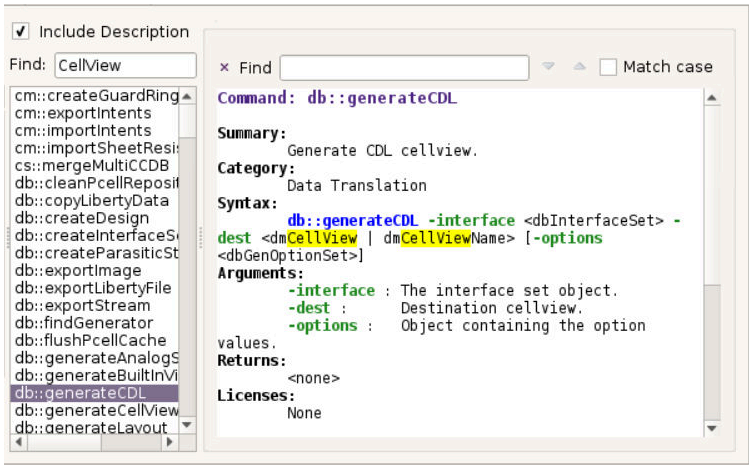
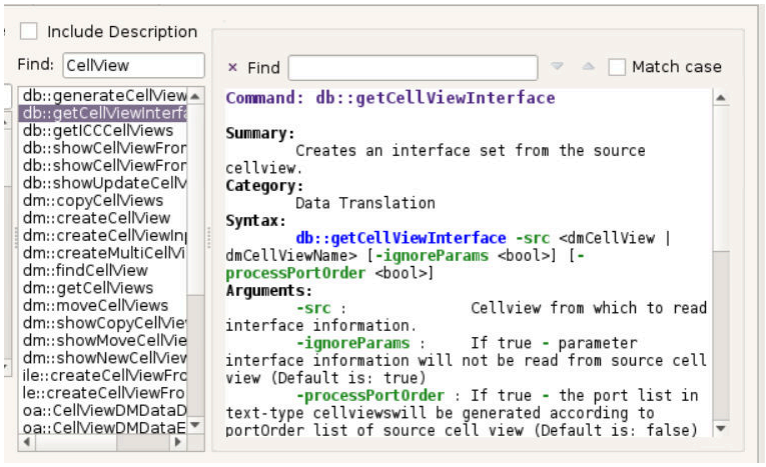


The utility let you control the display of Tcl commands and preferences.

Feedback

Table 2 describes the options available in the **Commands** tab.

Table 2 Commands Tab Options

Option	Description
Include Description	When this option is enabled the utility searches the string not only in command names, but also in their descriptions. Enabling **Include Description**:  Disabling **Include Description**: 
Show	Controls whether to display all commands or deprecated commands by **All** or **Deprecated**. When **Deprecated** is selected, only the commands list field is enabled to show the deprecated commands.
Find	Specifies the command name in the **Find** text field to find all the matching Tcl commands under the commands list field.
Select All	Click the button to select all commands category in the category field.

Feedback

Table 2 Commands Tab Options (Continued)

Option	Description
Export to Text File	Click the button to export the Tcl commands to a text file.
Export to XML	Click the button to export the Tcl commands to an XML file.
Include OA Commands	Controls whether to include oa commands in the utility.
Command Description Field	Displays the commands description. The **Find** text field filters the search information. The **Match Case** option controls whether to find the information with case-sensitive.

Table 3 describes the options available in the **Preferences** tab.

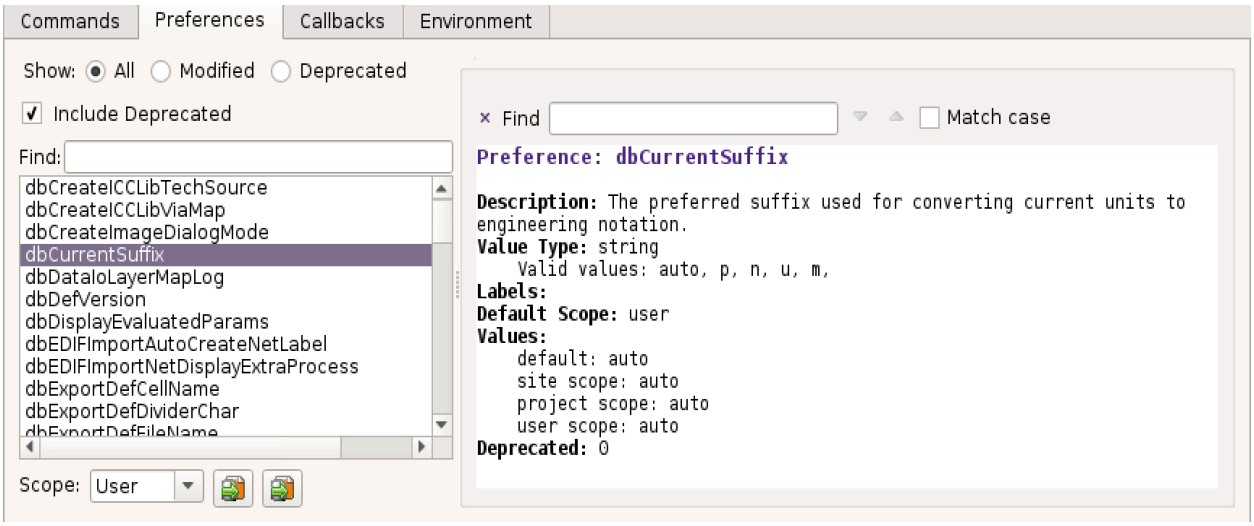


Table 3 Preferences Tab Options

Option	Description
Show	Controls whether to display the preferences by **All**, **Modified** or **Deprecated**.
Include Deprecated	Controls whether to include the deprecated preferences in the utility. This option is available when **All** and **Modified** is selected in **Show**.
Find	Specifies the preference name in the **Find** text field to find all the matching Tcl preferences under the preferences list field.
Scope	In the selection field, you can decide from which scope to read preferences to save in Tcl or XML formats.
Export to Tcl	Click the button to export the Tcl preferences to a Tcl file.

Feedback

Table 3 Preferences Tab Options (Continued)

Option	Description
Export to XML	Click the button to export the Tcl preferences to an XML file.
Preferences Description Field	Displays the preferences description. The **Find** text field filters the search information. The **Match Case** option controls whether to find the information with case-sensitive.

Table 4 describes the options available in the **Callbacks** tab.

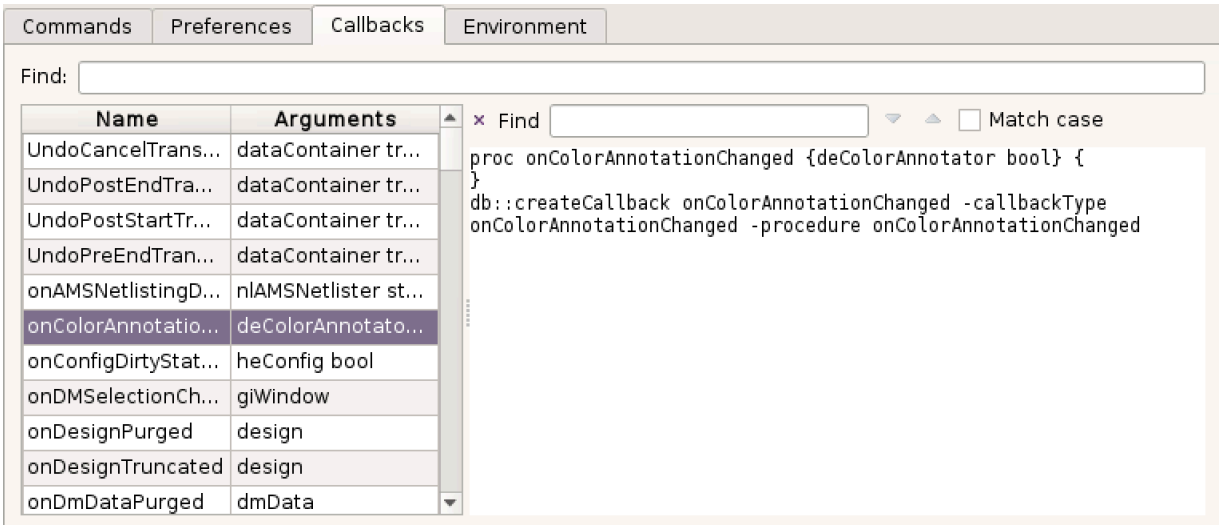


Table 4 Callbacks Tab Options

Option	Description
Find	Specifies the callback or arguments names in the **Find** text field to find all the matching Tcl callbacks under the callbacks table.
Name	Name of Tcl callbacks available in the platform.
Arguments	Arguments list of callbacks.
Callbacks Description Field	Displays the callbacks syntax. The **Find** text field filters the search information. The **Match Case** option controls whether to find the information with case-sensitive.

Feedback

Table 5 describes the tabs available in the **Environment** tab.

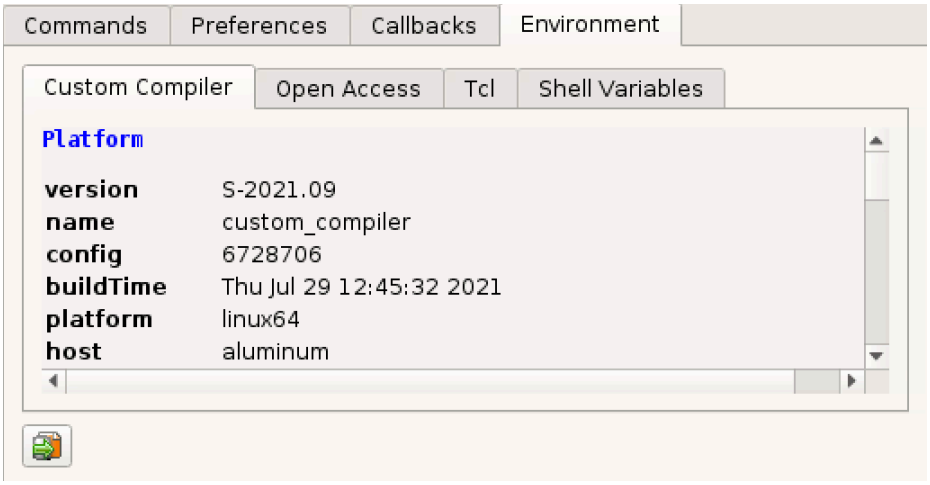


Table 5 Environment Tab Options

Option	Description
Custom Compiler	Shows the information about platform.
Open Access	Shows the information about OpenAccess.
Tcl	Shows the information about Tcl interpreter.
Shell Variables	Shows the linux shell variables.

Python in the Custom Compiler Environment

You can use the Python programming language in the Custom Compiler environment.

This topic describes the following:

- About Using Python in the Custom Compiler Environment
- How to Run Python Expressions and Scripts

About Using Python in the Custom Compiler Environment

In the Custom Compiler environment, you can use the Python programming language to interact with designs and create interoperable PCells.

Similar to using the Custom Compiler Tcl, you can run Python commands and scripts to interact with the Custom Compiler environment and your designs. See How to Run Python Expressions and Scripts.

Feedback

The Custom Compiler environment also supports using Python to specify a set of input values for PCells and instantiate those PCells in any OpenAccess design database. This version of PCells is called PyCells. See Creating Interoperable PCells Using the Python Language.

The supported Python versions for the different applications are as follows:

Application	Python Version
Python commands and scripts that interact with the Custom Compiler environment and designs	--python311 See How to Run Python Expressions and Scripts.
PyCells and user-defined devices (UDDs)	--python26 (default) You can use a different Python version by specifying the `--python<version>` argument in the `custom_compiler` startup command. See Starting Sessions and Setting Environment Variables.

If you use the `SYNOPSYS_CUSTOM_PYTHON_VERSION` environment variable or the `--python<version>` argument when starting the Custom Compiler tool, the setting applies only to PyCells and UDDs. The setting has no effect on the Python used in the Console assistant.

How to Run Python Expressions and Scripts

You can enter and run Python expressions and scripts using the following:

Method	See...
Custom Compiler Console assistant	Typing Python Commands in the Console assistant
Available Python interface for running Tcl scripts	• py::exec • py::eval
Available Tcl interface for running Python scripts	tcl.eval( )

The Custom Compiler environment includes oaScript 4.1 so that you can access the OA database using Python. See the `py::eval` topic in Tcl Interface for Running Python Scripts, Blocks, and Expressions.

Customization

You can customize the Custom Compiler environment for a seamless design experience. For example, you can:

- Add graphic user interface elements for individual design operations and integrated products.

  See Customizing the Software and Workspaces for instructions and Tcl Sample for Custom Graphic User Interface for Tcl samples.

- Customize access to process design kits.

  See Configuring PDKs for DRC, LVS, and PVE for instructions.

- Add access to additional design tools.

  See Integrating External Applications for instructions.

2

Getting Started

This chapter covers Custom Compiler environment design sessions that describes how to start and end sessions and define settings that apply at startup and apply defined setting at one of several design scopes, from your local directory to an entire project.

- Starting Sessions
- Ending Sessions
- Defining Startup Settings
- Defining Session Settings
- Restricting Design Access to Read-Only

When you are ready to design, see General Designing. To import data first, see Import.

Starting Sessions

You use the `custom_compiler` command to start the Custom Compiler tool. This topic describes how to use the command, the command syntax, and the available command arguments.

About the custom_compiler Command

The `custom_compiler` command starts the Custom Compiler tool and opens a design session. All arguments for the `custom_compiler` command are optional. You can specify arguments when you want to start the session with different settings. See custom_compiler Command Syntax and Arguments.

Feedback

Starting the Custom Compiler Tool

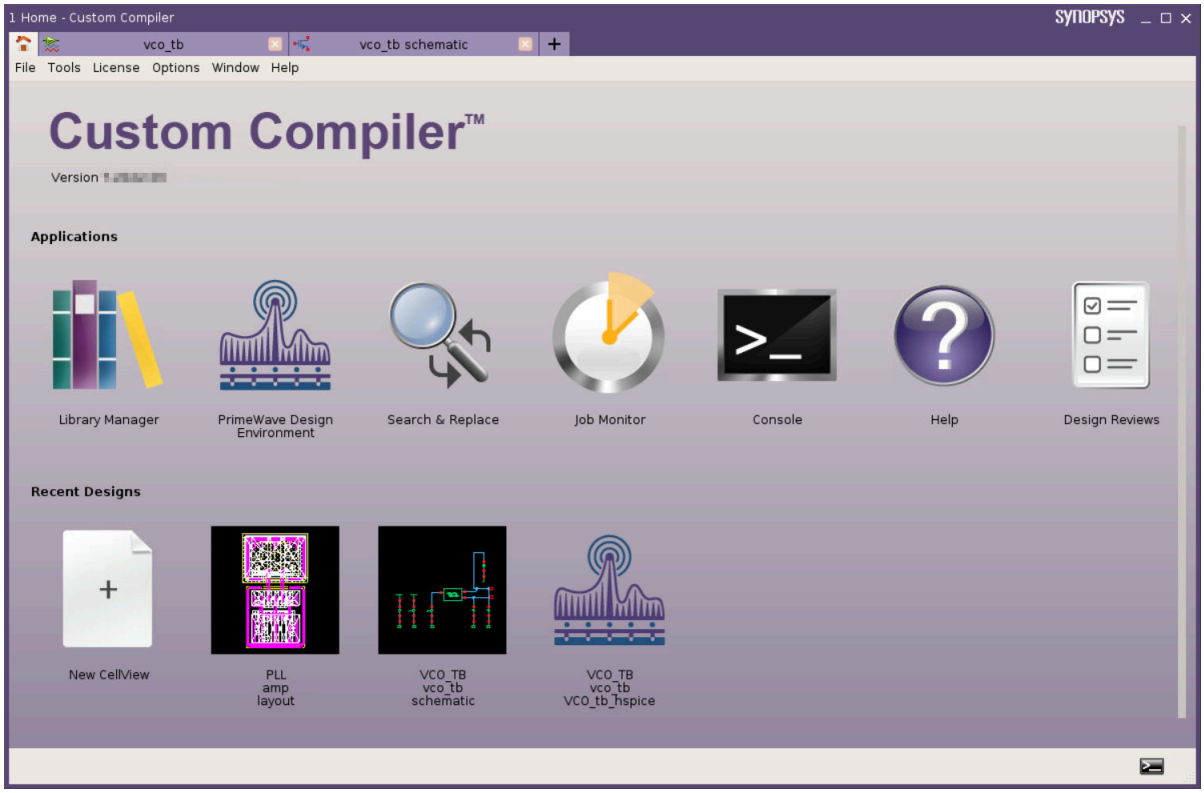
To start the Custom Compiler tool:

1. In the Linux terminal window, type the following at the command prompt:

   ```
   % custom_compiler &
   ```

   See custom_compiler Command Syntax and Arguments for the complete syntax and arguments.

   The Custom Compiler window opens and displays the Home Tab, which is pinned by default and cannot be closed.



2. To set design session settings, see Defining Session Settings.

custom_compiler Command Syntax and Arguments

The complete syntax for the `custom_compiler` command is as follows:

```
custom_compiler [-help] [-command <string>]
[-dynamicThreads <bool>] [-gui <bool>] [-scaleFactor <float>]
[-libDefFile <string>]  [-loadAll <bool>] [-localDir <string>]
```

Feedback

```
[-log <string>] [-numThreads <int>] [-replay <string>] [-tcl <string>]
[-python<version>] [-readOnly] [-container] [-batch]
```

Table 6 describes basic session startup command-line options.

Table 6 Session Startup Command-Line Arguments

Arguments	Description
`-help`	Reports the startup command usage statement and argument list.
`-command`	Runs the specified Tcl command after startup. If you specify `-command`, do not specify `-tcl`.
`-dynamicThreads`	Determines whether the number of threads will be modified depending on the load on the system.
`-gui`	Displays the graphical user interface or the text-based user interface. Set `-gui to true` to display the graphical user interface. If `$DISPLAY` is set, `-gui` is `true` by default. Set `-gui to false` to display the text-based user interface, and the Custom Compiler environment will prompt you for inputs and display outputs using a text-based user interface.
[-scaleFactor <float>]	Set High-DPI scale factor. (Default: 1.0)
`-libDefFile`	References a library definition file of libraries to use in this session. (Optional: Overrides the default `lib.defs` file).
`-loadAll`	To speed startup, Custom Compiler normally defers the initialization of some subsystems until a command is given that requires that subsystem. However, if the startup scripts that refer to data or objects are defined when a subsystem is initialized, such scripts will fail. In this case, you can set `-loadAll true` to ensure that all subsystems are initialized at startup. Note that this option slows down startup.
`-localDir`	Specifies the path that overrides the `SYNOPSYS_CUSTOM_LOCAL` environment variable.
`-log`	Specifies the file name or path to store log and command history files (overrides the defaults).
`-numThreads`	Sets the maximum number of threads to use in multithreaded parts of Custom Compiler. The default is either 8, or about three-fourth of the number of processors for machines with less than 12 processors.
`-replay`	Specifies the path to a log file to replay.
`-tcl`	Specifies a Tcl file to source after startup. If you specify `-tcl`, do not specify `-command`.

Feedback

Table 6 Session Startup Command-Line Arguments (Continued)

Arguments	Description
`--python<version>`	Specifies the Python version for PCells and user-defined devices (UDDs). For example: `--python26` Default version: Python26 See About Using Python in the Custom Compiler Environment. This argument has no effect on the Python version used in the Console assistant. For information about using Python in the Console assistant, see How to Run Python Expressions and Scripts and Typing Python Commands in the Console assistant.
`-readOnly`	Sets the permission of all designs to the read-only mode.
`-container`	Wrapper option that runs the Custom Compiler tool in a container. For more information about the container setup, see the Synopsys Containers page at https://www.synopsys.com/containers or the Synopsys Installation Guide page at https://www.synopsys.com/install.
`-batch`	Forces the release of licenses and terminates the Custom Compiler session at the end of the script. This argument requires that you also specify `-command` or `-tcl`, and is only applicable when you specify `-gui false`.

License Queuing

When a license is unavailable for check out, the Custom Compiler tool issues an error message. However, an alternate error-handling strategy is to wait for a license to become available. This feature is known as license queuing and is useful for periodic batch jobs.

With License Queuing, which works in the batch mode (`-batch`):

- The license is allocated to the process that first requests for a license.
- The application periodically polls to verify if the license status indicates successful check out.

Note:

License queuing is disabled by default in the GUI and non-GUI (`-gui false`) modes.

The GUI mode does not support license queuing.

Feedback

To enable license queuing in the non GUI mode, use one of the following methods:

- Use the `SYNOPSYS_CUSTOM_ENABLE_LICENSE_QUEUING` environment variable. See Example 1 and Example 2.

  Or

- Use the `custom_compiler -batch` command.

The following values can be specified with the `SYNOPSYS_CUSTOM_ENABLE_LICENSE_QUEUING` environment variable: `1`, `on`, `true`, `yes`, `0`, `off`, `false`, and `no`.

Example 1 Using SYNOPSYS_CUSTOM_ENABLE_LICENSE_QUEUING in the csh environment

```
setenv SYNOPSYS_CUSTOM_ENABLE_LICENSE_QUEUING TRUE
custom_compiler -gui false
```

Example 2 Using SYNOPSYS_CUSTOM_ENABLE_LICENSE_QUEUING in the bash environment

```
export SYNOPSYS_CUSTOM_ENABLE_LICENSE_QUEUING=1
custom_compiler -gui false
```

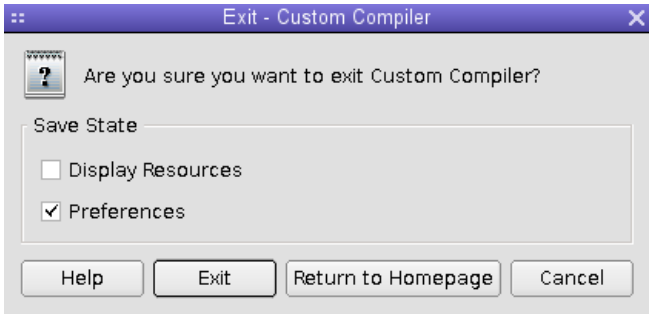
For information about the `custom_compiler` command, see custom_compiler Command Syntax and Arguments.

Ending Sessions

To end a Custom Compiler session, choose one of these options:

- Choose **File > Quit** from the **Home** tab to end the session and be prompted to save designs, preferences, bookmarks, and display resource changes.

  The Custom Compiler **Exit** dialog box opens.



Feedback

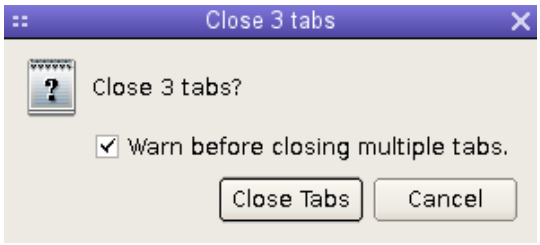
In the **Exit** dialog box, you can save current settings by choosing **Save State** and then using the check boxes to save workspaces, recent files, display resources, preferences, and bookmarks.

You can set `deExitPromptSave*` and `deExitSave*` Tcl preferences to specify which items the **Exit** dialog prompts and default **Save State** options, respectively.

- If a `deExitPromptSave<title>*` preference is true and the unsaved procedure returns true, the **Exit** dialog prompts to save <title>. The check box shown is set to the value of the `deExitSave*` preference.
- If prompting is disabled, via `deExitPromptSave*`, the tool either discards this state data or automatically saves it, depending on the value of the `deExitSave*` preference.

For information about the Tcl preferences, see the *Custom Compiler Tcl Reference Manual*.

- Click X on the Custom Compiler title bar. The **Close Tabs** warning dialog box opens.



Choose **Close Tabs** to exit Custom Compiler. To prevent the tool from warning you before closing multiple tabs, clear the **Warn before closing multiple tabs** check box.

- Alternatively, type `exit -force true` at the Console input line to end the session and discard all edits, bypass the **Exit** dialog box, and close all windows.

Defining Startup Settings

Define and apply startup settings to control the work environment for all product users. Use Tcl scripts and environment variables to apply settings at startup. To apply settings effectively, you also need to understand the installed directory structure.

These sections cover:

- Locating Common Files and Startup Directories
- Setting Environment Variables
- Viewing Installed Directories and Files

Feedback

Locating Common Files and Startup Directories

The Custom Compiler tool makes use of the following common files:

- `display.drf` and `display.tcl` — Can be placed in the current working directory, home directory, or any startup directory.
- `lib.defs` — Can be placed in the current working directory, home directory, or any startup directory.
- `.synopsys_custom.tcl` and `.synopsys_custom.py` — Can be placed in the current working directory, home directory, or any startup directory.
- `custom_compiler/init.tcl` — Can be placed in any startup directory.

  Note:
  The file is not read by standalone PrimeWave, OptoCompiler, or other Custom Infrastructure based applications.

For startup directories, the Custom Compiler tool searches: install, site, project, user, and local directories, *in that order*.

- Install refers to the auxx directory within the installation directory. To define more installation directories, they can be put in `SYNOPSYS_CUSTOM_INITPATH` (see Table 7).
- Site is optional, but can be specified by setting `SYNOPSYS_CUSTOM_SITE` (see Table 7).
- Project is optional, but can be specified by setting `SYNOPSYS_CUSTOM_PROJECT` (see Table 7).
- User defaults to `~/.synopsys_custom`, but can be changed by setting `SYNOPSYS_CUSTOM_USER` (see Table 7).
- Local defaults to `./synopsys_custom`, but can be changed with the `-localDir` command-line option (see Table 6) or by setting `SYNOPSYS_CUSTOM_LOCAL` (see Table 7). The Custom Compiler tool writes log files and application settings to this directory.

You can define a file named the same as any file within auxx. For example, if `<site>/layout_editor/actions.tcl` exists, it is sourced after `<install>/layout_editor/actions.tcl`.

Feedback

Setting Environment Variables

Set environment variables to create a custom environment at startup.

- Statement format for setting an environment variable for csh/tcsh shells:

```
setenv <environment_variable_name> <value>
```

- Statement format for setting an environment variable for bash/ksh shells:

```
export <environment_variable_name>=<value>
```

Table 7 and Table 8 list the supplied environment variables and their descriptions.

Table 7 Startup Directory Environment Variables

Environment variable	Location	Default	Description
SYNOPSYS_CUSTOM_INSTALL	install	<product_ installation_ directory>/auxx	Custom Compiler executable defines this variable
SYNOPSYS_CUSTOM_INITPATH	install	No default.	Defines custom installation directories. To specify multiple values, in csh/tcsh shells, separate values with a space (`pathname pathname pathname`). To specify multiple values in bash/ksh shells, separate values with a colon (`pathname:pathname:pathname`).
SYNOPSYS_CUSTOM_SITE	site	No default	Startup, Preferences
SYNOPSYS_CUSTOM_PROJECT	project	No default	Startup, Preferences
SYNOPSYS_CUSTOM_USER	user	~/.synopsys_custom	Startup, Preferences
SYNOPSYS_CUSTOM_LOCAL	local	<current_working_directory>/synopsys_custom	Startup, Preferences, Bindings, Display Resources, Log Files, Bookmarks, Favorite Devices, and so on. Custom Compiler typically saves files to this directory at exit and loads them back when it resumes.

Feedback

Table 8 Additional Environment Variables

Environment variable	Default	Description
EDITOR		Designates the text editor to offer to the designer. You can alternatively set the editor when you are Defining Session Settings.
HOME	~	Legacy files (`lib.defs, display.drf`), `.synopsys_custom.tcl` and `.synopsys_custom.py`

SNPS_BROWSER	firefox	Searches for the browser associated with the Synopsys Help browser variable, `SNPS_BROWSER`. The value is a browser executable with a full pathname.
		You can set `SNPS_BROWSER` using one of the following methods:
		• Use the appropriate command in the Linux terminal. For example, in the C shell, type:
		`setenv SNPS_BROWSER \`
		`/usr/bin/firefox`
		• In the Bourne, Korn, or Bash shell, type:
		`export SNPS_BROWSER \`
		`=/usr/bin/firefox`
		• Use the `deHelpViewer` Tcl preference. For example, in the Custom Compiler console, type:
		`db::setPrefValue \`
		`deHelpViewer -value \`
		`firefox`
		For more information, in any window choose **Help > Custom Compiler Help > Using Help**.

SNPS_CONTAINER	<unset>	Setting this variable before running the tool runs the Custom Compiler tool in a container. Alternatively, to run the tool in a container, use the `custom_compiler -container` command when you invoke Custom Compiler.
SYNOPSYS_CUSTOM_CRASH_DIR	synopsys_custom/stacks	(Optional) If specified, Custom Compiler copies crash information to the directory. You can send this information to Synopsys for improving the product.
SYNOPSYS_CUSTOM_DISABLE_RAMDISK	0	Disables RAM disk use so that the `SYNOPSYS_CUSTOM_RAMDISK` and `SYNOPSYS_CUSTOM_MINIMUM_RAMDISK_SPACE` variable settings have no effect.
SYNOPSYS_CUSTOM_MINIMUM_RAMDISK_SPACE		Sets the minimum RAM disk space requirement, in bytes. If the available RAM disk space is less than the minimum RAM disk space you specify, then RAM disk is not used.

Feedback

Table 8 Additional Environment Variables (Continued)

Environment variable	Default	Description
SYNOPSYS_CUSTOM_PLATFORM	Environment detected by product wrapper script at startup	Designates the machine type to run the Custom Compiler tool. The tool uses this value to load architecture-specific compiled libraries, for example, OpenAccess, Qt, and Tcl. Valid values for this environment variable are linux64 and suse64.
SYNOPSYS_CUSTOM_PYTHON_VERSION	2.6.2	Designates the version of the Python interpreter to initialize in the Custom Compiler tool. This setting also configures the Python version of PyCells that are rendered in the tool. Valid values for this environment variable are 2.5.1, 2.6.2, 3.8.6, and 3.11. **Note:** The Python version you specify using this environment variable does not affect the Python used in the Console. See Python in the Custom Compiler Environment.
SYNOPSYS_CUSTOM_RAMDISK	/dev/shm	RAM disk location used to reduce disk I/O operations for high-performance caching in different Custom Compiler functions.
TMPDIR	/tmp	Standard temporary directory.

Viewing Installed Directories and Files

View installed directories and files to see the installation hierarchy and to locate the files you can refer to when you customize the startup process for session management.

Table 9 shows the product installation directory structure.

Table 9 Custom Compiler Product Installation Directory Structure

Directory	Subdirectory	Startup Files	Description
bin/custom_compiler			Startup script that locates the environment-dependent binary and executes it.
auxx/			Environment-independent files loaded by Custom Compiler.
	custom_compiler/		Contains startup scripts
		init.tcl	Initializes the environment
	<package>/		Contains feature description and support files. Multiple packages are supplied.

Feedback

Table 9 Custom Compiler Product Installation Directory Structure (Continued)

Directory	Subdirectory	Startup Files	Description
		startup.tcl	Initializes this feature package
	help/		Documentation files for online Help system; includes PDFs
	fonts/		Application-specific fonts
	images/		Icon pixel-maps and images
	samples/		Common samples library
	tech/		Base technology file
	display.tcl		Base display.tcl file
contrib/			Sample scripts provided by Synopsys for customer use

Defining Session Settings

Define session settings to control general and specific system behaviors during design sessions. You can control settings for general options used in all sessions, such as the cursor double-click speed. You can control specific settings for only this session, for example, the values you enter in a dialog box for a process you want to run once. You can use the same dialog box settings in future sessions by saving them as dialog box *preferences*. In addition to dialog box preferences, the system supplies design preferences you can use for nearly every design operation. Use these design preferences by either entering them in the Console, or using them in scripts.

Note that some applications use nti_shell and pw_shell for background jobs. The startup files read by these processes is not the same, so you might need to put some customization in multiple files. For examples, see the custom calculator functions as described in the *PrimeWave Design Environment Expression Language Reference Manual*, Structuring the Custom Calculator Function Code.

These sections cover:

- Setting General Options
- Managing Option Settings from the Home Page
- Saving Dialog Box Settings as Preferences
- Specifying Design Preferences

Feedback

- Synchronizing Design Windows, Viewports, Net Selections, and Commands Across Opened Designs
- Customizing Keyboard Shortcuts

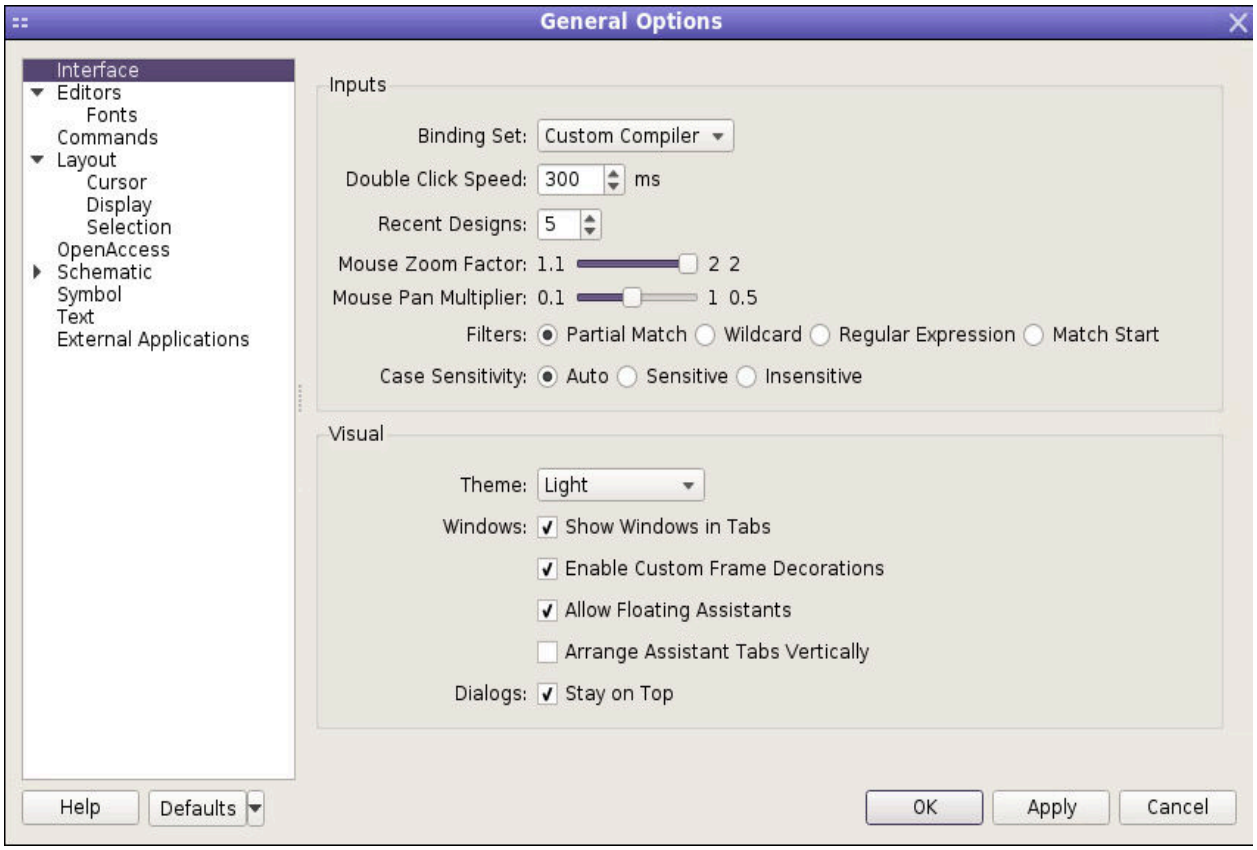
Setting General Options

The Custom Compiler general options designate the basic behavior of the interface. After you set the options, you can manage their application and distribution. See Managing Option Settings from the Home Page.

To set the general options:

1. From the **Home** tab, choose **Options > General**.

   The **General Options** dialog box opens with the **Interface** tab forward.



Feedback

2. Set the interface options:

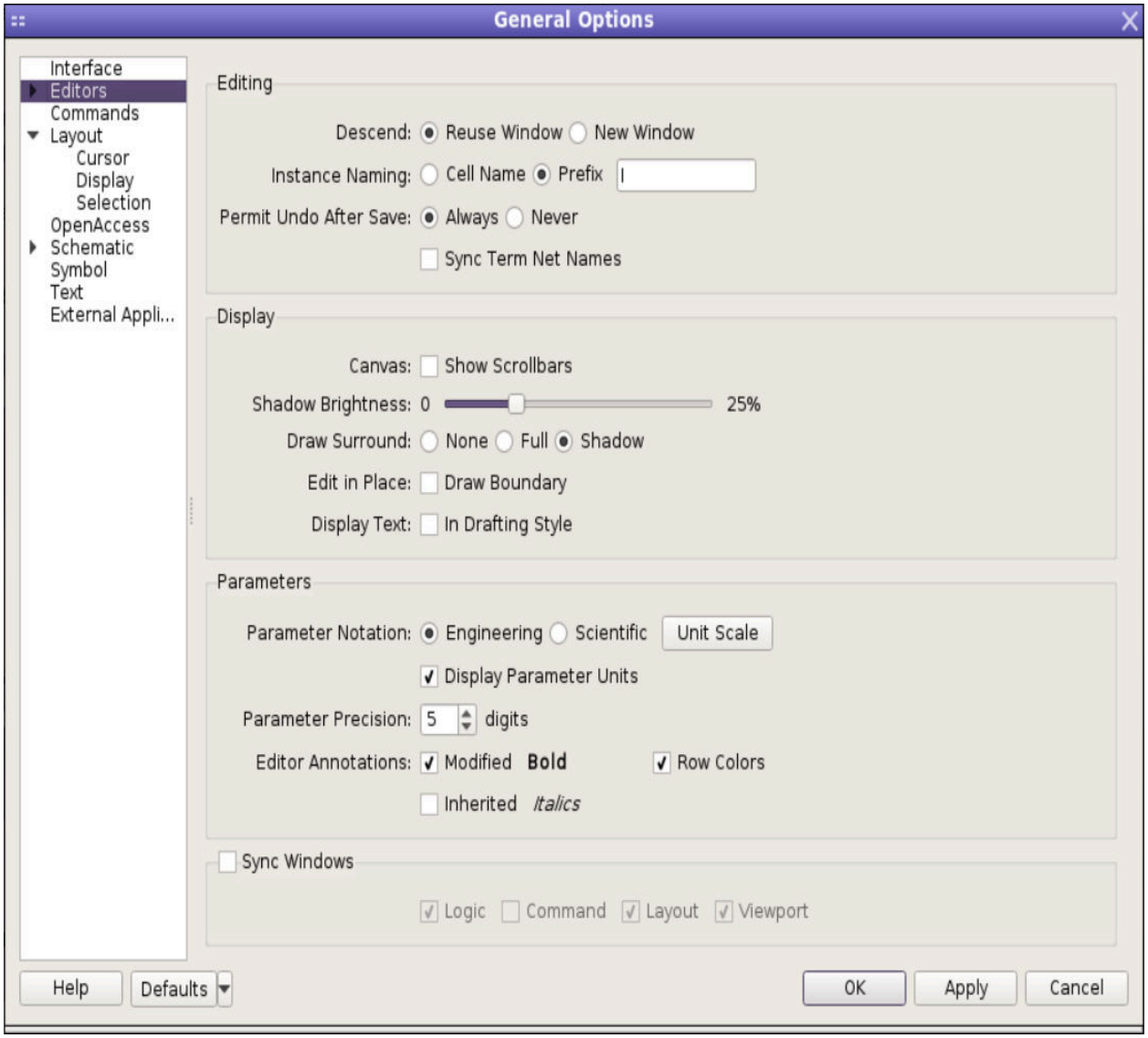
Option Group	Description
Inputs	**Binding Set** determines the set of key bindings (shortcut keys) for the session. Options are **Custom Compiler**, **IC Compiler**, **Laker**, and **Maestro**. Default is **Custom Compiler**. Tcl preference: `deBindingSet` **Double Click Speed** defines the time, in milliseconds, between mouse clicks to constitute a double-click. Default is 300. Tcl preference: `deDoubleClickTimeout` **Recent Designs** sets the number of recently opened designs to be listed under **Design > Open Recent**. Default is 5. Maximum is 25. **Mouse Zoom Factor** determines the zoom factor when zooming with the mouse wheel scroll. Valid values are between 1.1 and 2.0. Default is 2.0. **Mouse Pan Multiplier** determines the multiplier of viewport width or height by which to pan with the mouse wheel scroll. Valid values are between 0.1 and 1. Default is 0.5. **Filters** specifies the filtering method to use: ◦ **Partial Match** enables file-name matching from part of a pattern (for example, in the Design Navigator, entering **OU** filters for net names that include this two-character string). ◦ **Wildcard** enables wildcard matching (for example, **U*T** filters for net names with any character between the **U** and the **T**). ◦ **Regular Expression** enables pattern matching based on regExp syntax. See the *Custom Compiler Tcl Reference Manual* for a description of valid regular expressions. ◦ **Match Start** enables name-pattern matching based on the specified syntax. Tcl preference: `giFilterPatternStyle` **Case Sensitivity** options include: ◦ **Auto** makes a search case-insensitive unless it contains an uppercase letter, in which case the tool considers the search case-sensitive. ◦ **Sensitive** treats uppercase and lowercase letters as distinct. ◦ **Insensitive** treats uppercase and lowercase letters as equivalent.
Visual	**Theme** allows you to select the visual style of the interface. Choose between **Light**, **Gray**, or **Low Resource**. The **Low Resource** option is useful for low-bandwidth networks because it uses less bandwidth for graphics rendering in the GUI. Tcl preference: `giTheme`

Feedback

Option Group	Description
	Windows options include: **Show Windows in Tabs** allows you to revert to the legacy view when unchecked. **Enable Custom Frame Decorations** (on by default) controls whether to use the frame decorations in external desktops, such as KDE, including their drop-down window menus. Tcl preference: `giShowFrameDecoration` **Allow floating assistants** controls using assistants in floating windows. **Arrange Assistant Tabs Vertically** arranges the assistant tabs vertically. **Dialogs** includes the following option: **Stay on Top** controls whether to keep dialog boxes stay on top of their parent window. Tcl preference: `giDialogStayOnTop`

3. Click the **Editors** tab.

Feedback



4. Enter the design editing options:

Option Group	Description
Editing	**Descend** using **Reuse Window** replaces current window contents during descend operations. **New Window** opens each design instance master in a new window. **Instance Naming** applies automatic instance naming. **Cell Name** uses the instance master cell name as the prefix. **Prefix** applies the prefix you enter here to all new instances. Default is "I".

Feedback

Option Group	Description
	Permit Undo After Save enables either **Always** or **Never** undoing the save operation. The **Sync Term Net Names** option lets you update the terminal name when you modify the corresponding pin net name. The equivalent Tcl preference is `dbSyncTermNetNames`. The default is false.
Display	Enable **Show Scrollbars** (off by default) to show scrollbars in the design **Canvas**. **Shadow Brightness** controls how bright dimmed areas appear when using Shadow mode or when **Draw Surround** is **Shadow**. **Draw Surround** dims or hides the surrounding data when in the Edit in Place or Edit in Group mode. Select one of the following: ◦ **None**: Does not render objects that are surrounding the target edit hierarchy. The hidden objects are still functional (for example, snapping). ◦ **Full**: Renders all objects. ◦ **Shadow**: (Default) Dims objects surrounding the target edit hierarchy. The level of contrast depends on the **Shadow Brightness** setting. **Edit in Place** using **Draw Boundary** creates a boundary to edit the contents of the selected instance. **Display Text** using **In Drafting Style** always draws text objects from left to right or bottom to top regardless of the text drafting attribute.
Parameters	**Parameter Notation** with **Engineering** units displays real numbers in engineering units; with **Scientific** units, displays real numbers in Base 10. **Unit Scale** (available when **Parameter Notation** is **Engineering**) opens the **Parameter Units** dialog box, where you can adjust the parameter units for **Length**, **Resistance**, **Conductance**, **Capacitance**, **Inductance**, **Voltage**, **Current**, **Power**, **Time**, and **Frequency**. All of these are set to **auto** by default. **Display Parameter Units** (on by default) controls whether units are shown for parameters on the canvas or in the Property Editor. **Parameter Precision** sets the parameter decimal place limit for input forms and canvas notation. Minimum: 1. Maximum: 15. Default is 5. **Editor Annotations** controls displaying editor annotations with **Modified**, **Row Colors**, or **Inherited**. See Setting Editor Annotation Options.

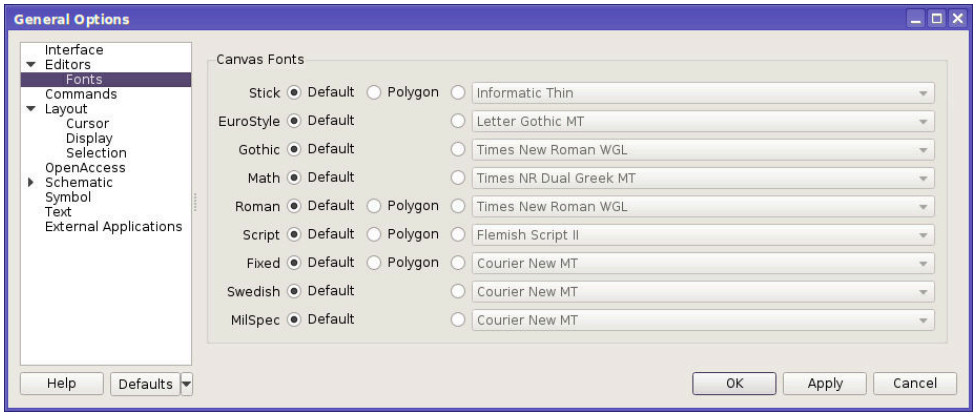
Feedback

Option Group	Description
Sync Windows	Synchronizes opened design windows, viewports, net selections, and command settings to facilitate design tasks. ◦ **Logic**: Synchronizes selected nets (Design Navigator selections) across opened designs. ◦ **Command**: Synchronizes active command settings across opened designs. ◦ **Layout**: Synchronizes window layouts across opened designs. ◦ **Viewport**: Synchronizes viewports across opened designs. Tcl preferences: `deSyncWindowLogicdeSyncWindowCommand`, `deSyncWindowLayout`, `deSyncWindowViewport`

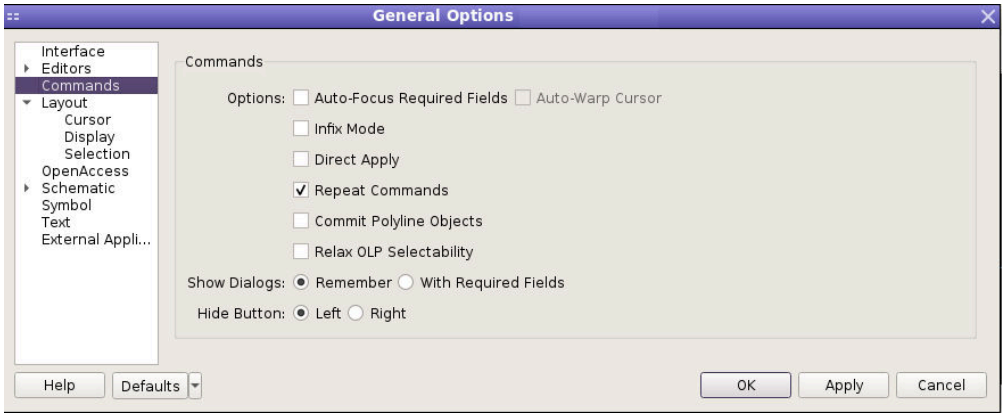
5. Click the **Fonts** tab within **Editors** and configure the fonts used for canvas rendering.

   To configure fonts to use for text and canvas labels including instance names, for each font preference (such as **Stick**) select an available font option. You have the following font options:

   - **Default** uses a predefined TrueType font.
   - **Polygon** uses a predefined polygon font.
   - The selection list of fonts allows you to select an available font supported by the tool.



6. Switch to the **Commands** tab.



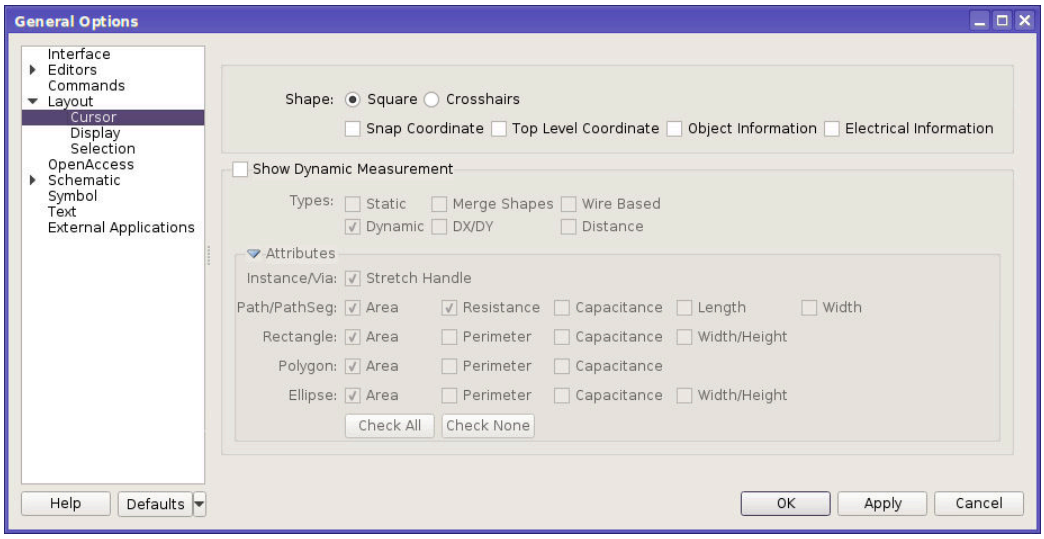
7. Enter the options for commands:

Option Group	Description
Commands	These **Options** control active field focus for interactive command input. **Auto-Focus Required Fields** automatically activates the first input field of the dialog box or toolbar on command start, allowing the cursor to remain active on the canvas. Key bindings are temporarily disabled. Setting **Auto-Warp Cursor** repositions the cursor to the middle of the active input field. **Infix Mode** interprets the current cursor coordinate point as the first-click location for commands requiring anchor points. Setting infix mode here enables it for all key bindings and shortcut commands. For help on using the infix mode, see Infix Mode in the *Custom Compiler Layout Editor User Guide*. **Direct Apply** lets you perform a quick and one-time execution for editing commands. See Direct Apply in the *Custom Compiler Layout Editor User Guide*. **Repeat Commands** causes commands to repeat in post-select mode but terminate in preselect mode. When unchecked, commands always terminate after a single cycle. On by default. **Commit Polyline Objects** automatically commits an editing or creating operation by the last click point when ending a command. Tcl preference: `leCommitPolyline` **Relax OLP Selectability** controls LE commands that can refer to unselectable LPP shapes. **Show Dialogs** controls the shown state of command dialog boxes. **Remember** restores the last shown state of a dialog box when starting the command. **With Required Fields** always starts the command without a dialog box.

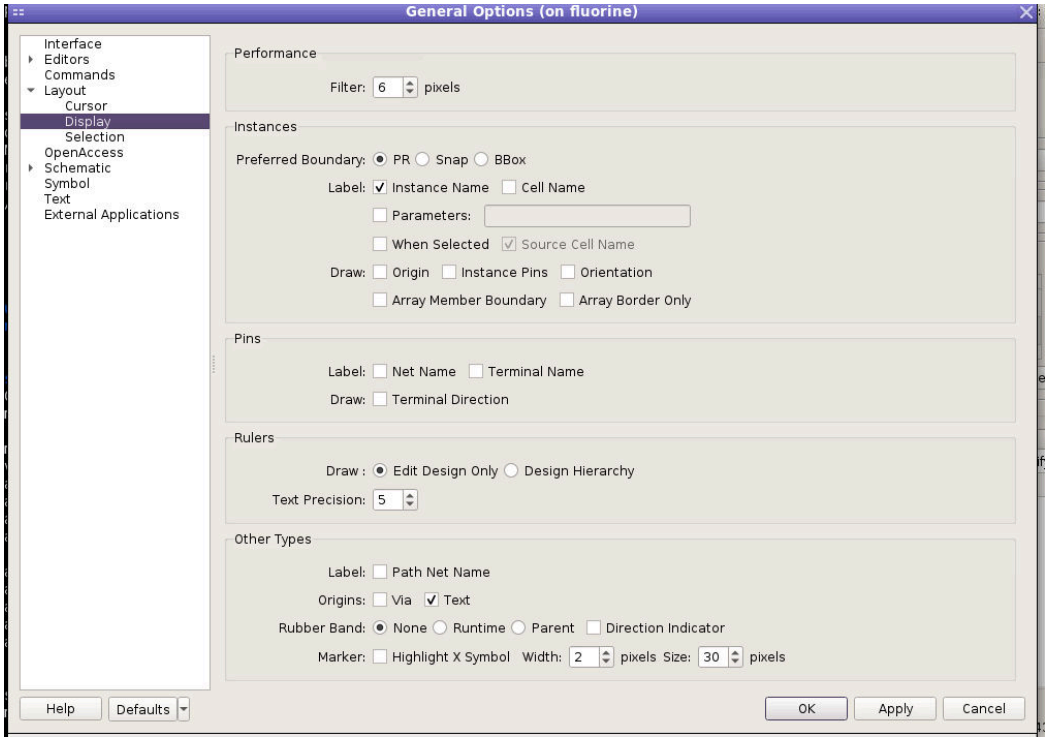
Feedback

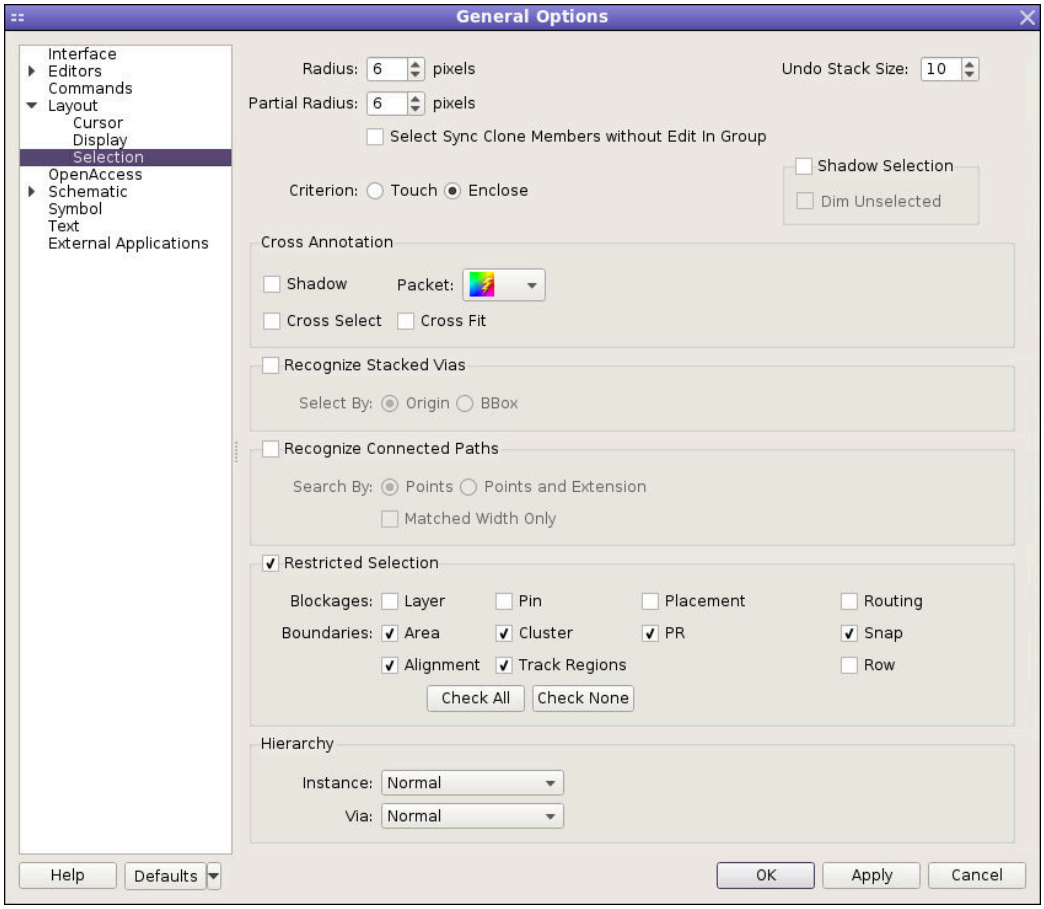
Option Group	Description
	Hide Button controls the **Hide** button position in command dialog boxes. **Left** (default) keeps the **Hide** button on the left of the dialog box. Setting **Right** repositions the **Hide** button to the right of the dialog box. Tcl preference: `deCommandDialogHidePosition`

8. Click the **Cursor**, **Display**, or the **Selection** tab within **Layout**. For information on layout options, see the *Custom Compiler Layout Editor User Guide for Creating and Editing Objects*.

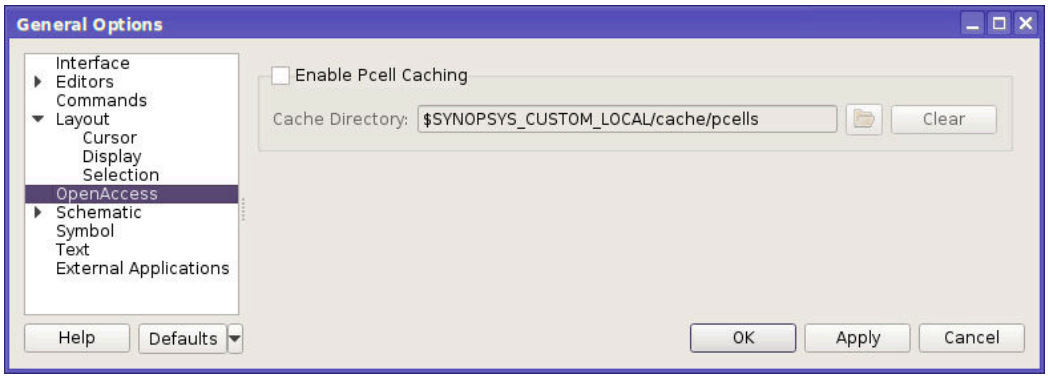


Feedback


General Options (on fluorine)
Interface
Editors
Commands
Layout
Cursor
Display
Selection
OpenAccess
Schematic
Symbol
Text
External Applications
Performance
Filter: 6 pixels
Instances
Preferred Boundary: PR Snap BBox
Label: Instance Name Cell Name
Parameters:
When Selected Source Cell Name
Draw: Origin Instance Pins Orientation
Array Member Boundary Array Border Only
Pins
Label: Net Name Terminal Name
Draw: Terminal Direction
Rulers
Draw : Edit Design Only Design Hierarchy
Text Precision: 5
Other Types
Label: Path Net Name
Origins: Via Text
Rubber Band: None Runtime Parent Direction Indicator
Marker: Highlight X Symbol Width: 2 pixels Size: 30 pixels
Help
Defaults
OK
Apply
Cancel



9. Click the **OpenAccess** tab.

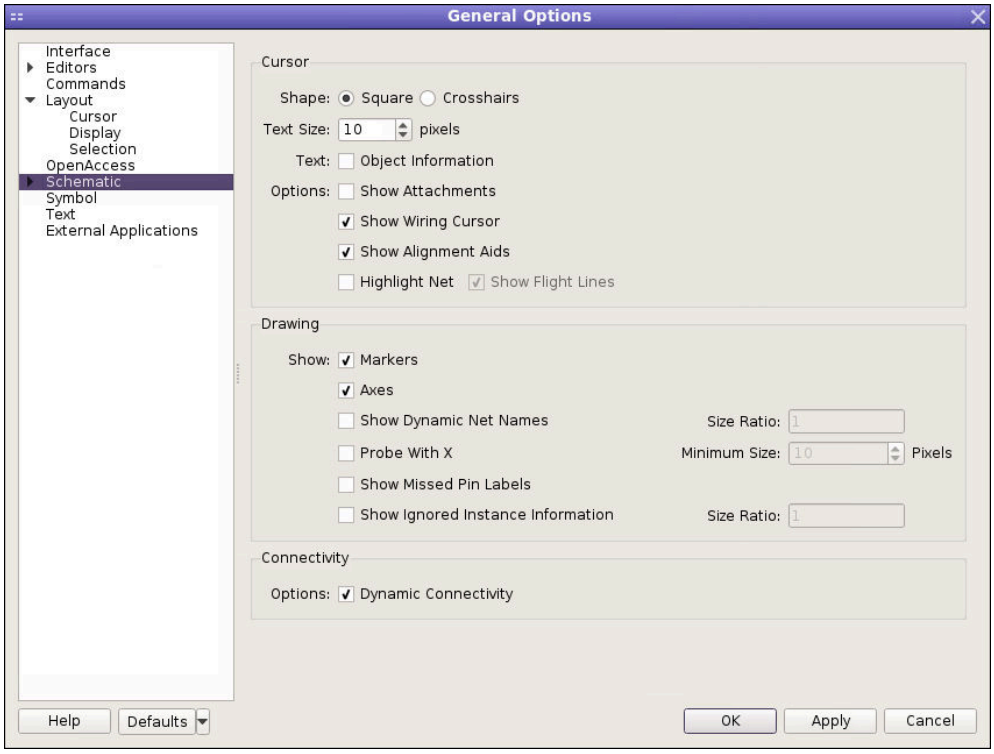


Feedback

10. Set the OpenAccess options:

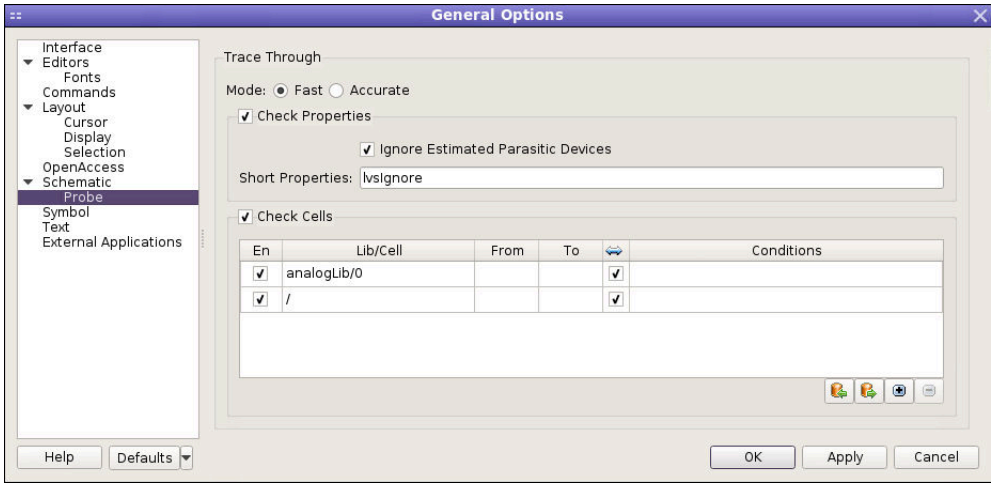
Option Group	Description
Enable Pcell Caching	Specifies a PCell cache for all supermasters. Click the **Enable Pcell Caching** check box to enable this feature. Use the **Cache Directory** field to specify, or browse to select the cache directory location. The default cache directory is $SYNOPSYS_CUSTOM_LOCAL/cache/pcells.

11. Click the **Schematic** tab to set the schematic options.



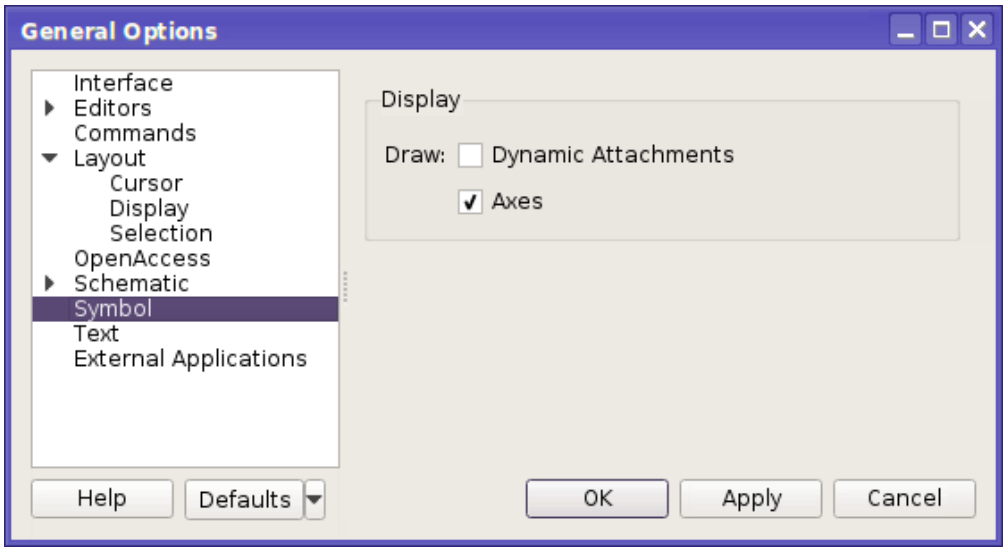
Switch to the **Probe** tab within **Schematic** and configure the options used for probing.

Feedback



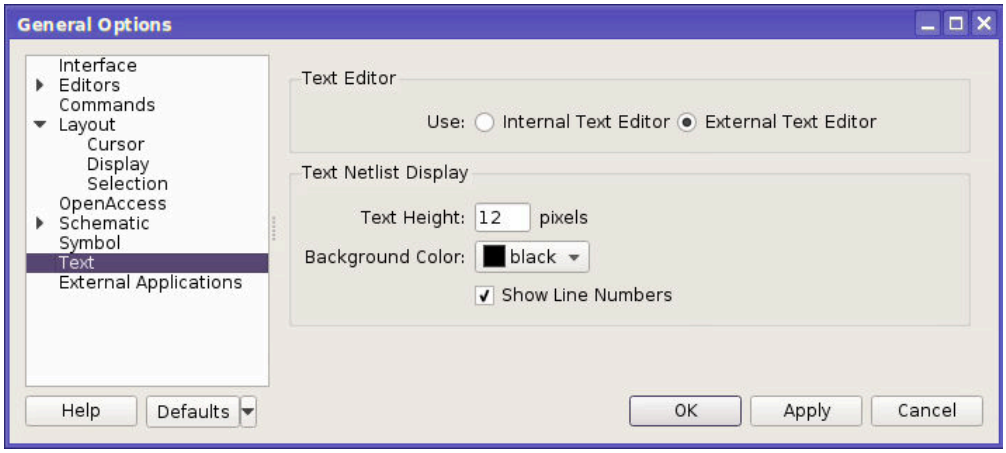
For information on schematic options, see the *Custom Compiler Schematic Editor User Guide*.

12. Click the **Symbol** tab. For information on symbol options, see the *Custom Compiler Schematic Editor and Text Editor User Guide*.

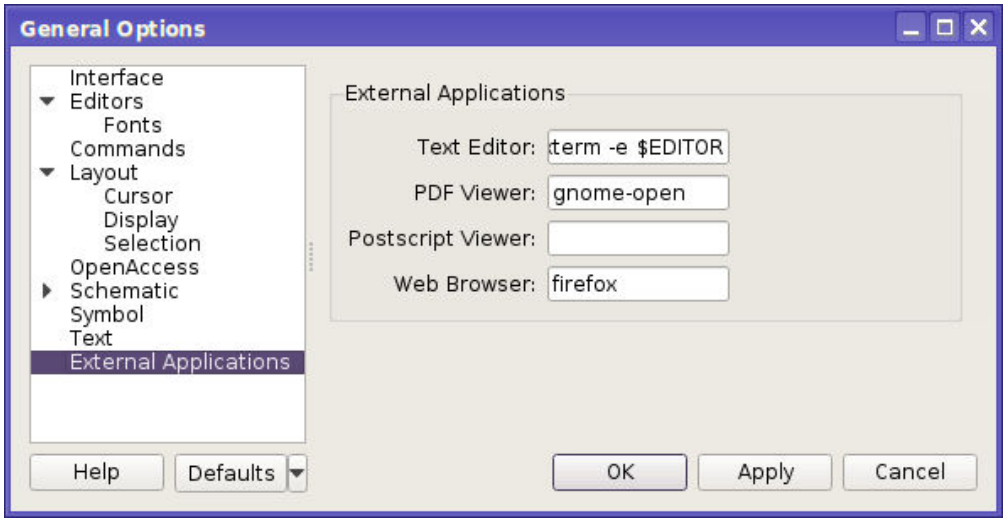


13. Click the **Text** tab. For information on Setting HDL Text Editor View Display Options, see *Custom Compiler Schematic Editor and Text Editor User Guide*.

Feedback



14. Click the **External Applications** tab.



15. Set the options for external applications:

Option Group	Description
External Applications	**Text Editor** specifies the text editor. Default: `xterm -e $EDITOR`. See Editing HDL Text Editor Views. Tcl preference: `deTextEditor`. **PDF Viewer** specifies the PDF viewer. Default: `gnome-open`. Tcl preference: `dbPDFViewer` **Postscript Viewer** specifies the postscript viewer. Tcl preference: `dbPostscriptViewer`

Feedback

Option Group	Description
	Web Browser specifies the web browser. Default: `firefox`. Tcl preference: `deHelpViewer`

16. (Optional) To save these settings as default values for this dialog box, choose **Defaults > Save**.
17. Click **OK**.

Managing Option Settings from the Home Page

You can save a design window's option settings as preferences and apply them at multiple scopes.

These sections cover:

- Saving Option Settings as Preferences
- Loading Option Settings Saved as Preferences

Saving Option Settings as Preferences

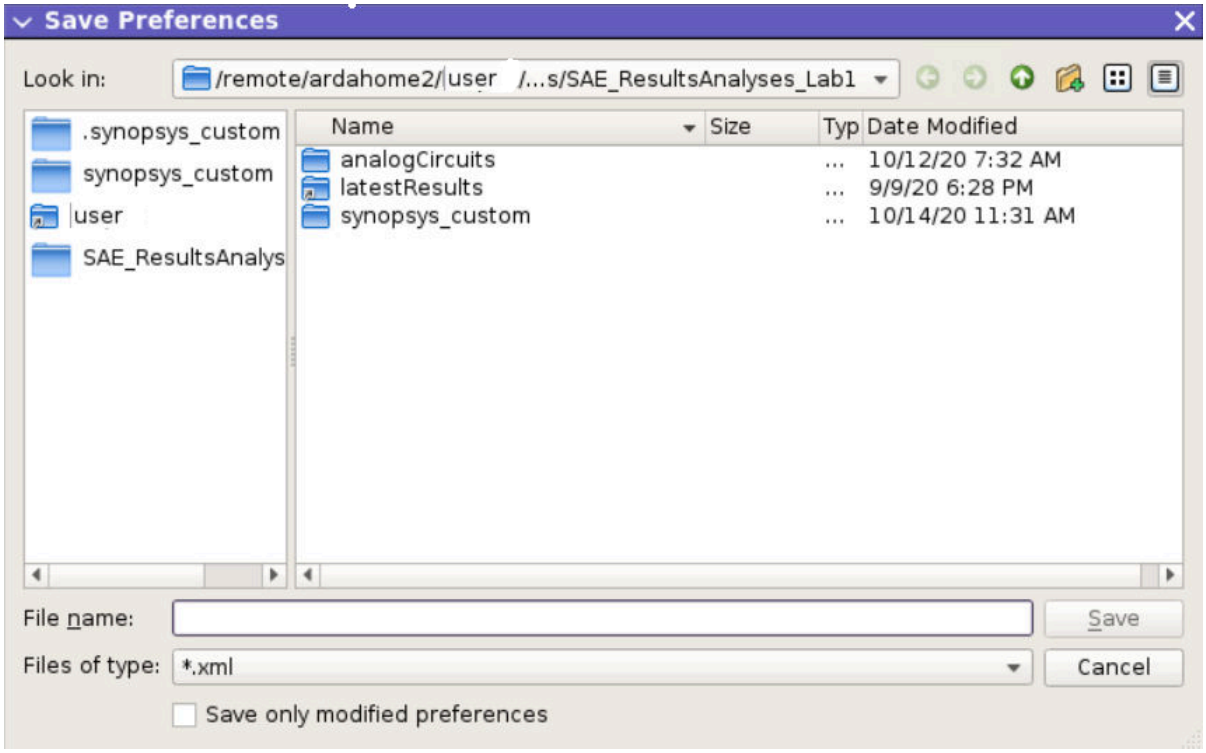
You can save option settings as preferences for use in future design sessions.

To save design options as preferences:

1. From the Home page or any design window, choose **Options > Save Preferences**.

   The **Save Preferences** dialog box opens.

Feedback



2. Browse to the required preferences file or type in a new **File name**.
3. (Optional) To save only changed options, click **Save only modified preferences**.
4. Click **Save**.

Loading Option Settings Saved as Preferences

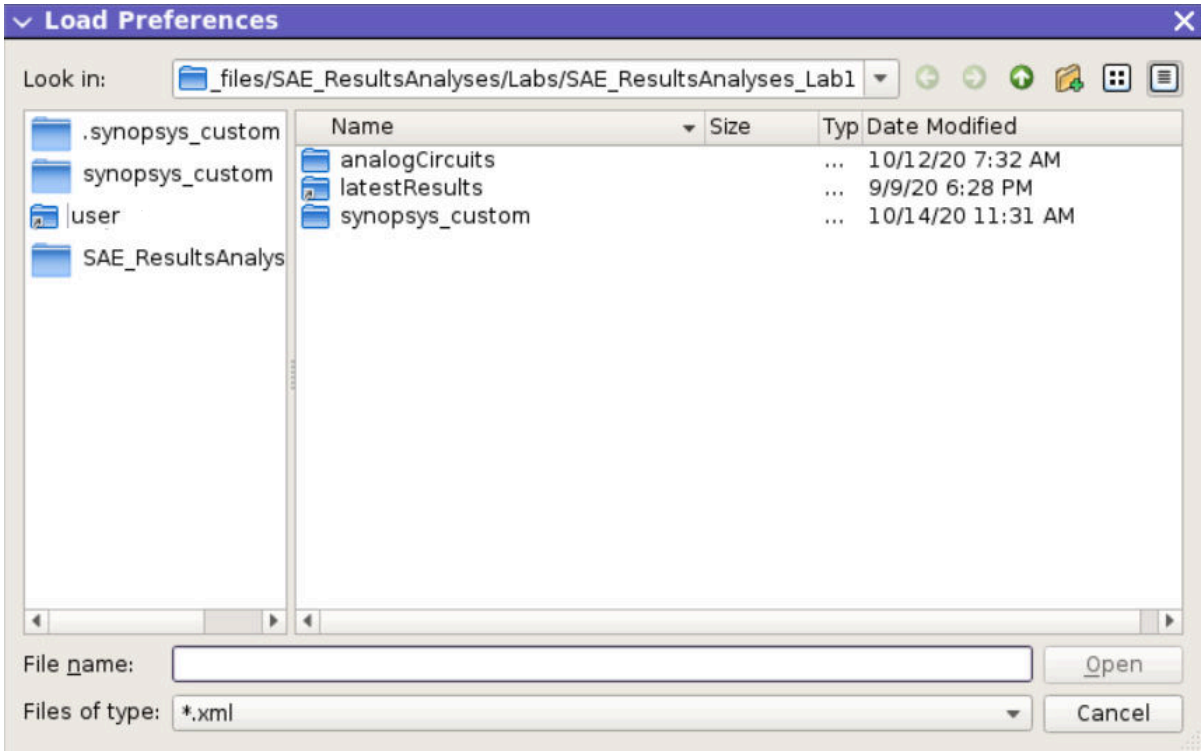
Load options saved as preferences to use settings from a previous session.

To load options saved as preferences:

1. From the Home page or any design window, choose **Options > Load Preferences**.

   The **Load Preferences** dialog box opens.

Feedback



2. Browse to the required preferences file.
3. Click **Open** to load the preferences file.

Saving Dialog Box Settings as Preferences

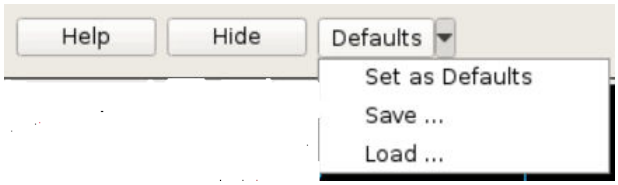
There are two ways to save dialog box settings as preferences.

- At the Current Location: (controlled by the `SYNOPSYS_CUSTOM_LOCAL` environment variable) At the end of a session, the system offers the option of saving dialog box settings from that session as preferences. If you choose to do this, the system applies these at the local location, which includes your current working directory when you were using the dialog box. At the next session, the system will offer these saved settings when you are working in the same directory.
- At User Scope: (home_dir.synopsys_custom.xml) If you want to save dialog box settings as preferences at user scope and greater scopes, you can choose to make the settings the default for that dialog box. The procedure that follows provides instructions for this task.

Feedback

To save dialog box settings as preferences:

1. Open the dialog box that offers the settings you want to save as defaults.
2. Enter the new settings you want to save.
3. From the **Defaults** menu (to the right of the **Defaults** button), choose **Set as Defaults**.



The system localizes and saves preferences in the `prefs.xml` file at the user scope.

For example, the preferences file that follows is saved at user scope:

`~/.synopsys_custom/prefs.xml`

The `prefs.xml` files are sourced at startup. However, you can choose **Options > Load Preferences** to load a different set of preferences. (See Loading Option Settings Saved as Preferences).

At session start, the system searches all scopes automatically in the order that follows. Where duplicate preference values are found, the value found at the last searched scope takes precedence. For example, if you have saved a `prefs.xml` file in your current working directory, its preferences always take precedence.

1. `SYNOPSYS_CUSTOM_INSTALL` (Not available for preference storage)
2. `SYNOPSYS_CUSTOM_SITE` (CAD support designation)
3. `SYNOPSYS_CUSTOM_PROJECT` (CAD support designation)
4. `SYNOPSYS_CUSTOM_INITPATH` (CAD support designation)
5. `SYNOPSYS_CUSTOM_USER` (The default value is `<$HOME>/.synopsys_custom`. The `prefs.xml` file in this location is where your preferences are stored and it is automatically loaded at startup).
6. `SYNOPSYS_CUSTOM_LOCAL` (The default value is `<current_working_dir>/synopsys_custom`. In this location is a `prefs.xml` file that is used to save preferences upon closing the Custom Compiler tool).

Specifying Design Preferences

While designing, you can use Tcl at the Console input line or in a Tcl script to dynamically apply a value to any design operation. See Console Tab.

Feedback

You can specify design preferences that control details of over 2000 design operations.

To specify design preferences:

1. Identify the preference whose value you want to set. The session log file records all settings from your previous session.

   For a complete list of available preferences for any Tcl category, see the *Custom Compiler Tcl Reference Manual* for details.

   To display all currently available preferences, you can use this Tcl script:

   ```
   proc listPrefs {{pattern "*"}} {
     db::createList [db::getAttr name -of [db::getPrefs $pattern]]
   }
   ```

   In the Tcl script, `db::getPrefs` returns the collection of preferences, and `db::foreach` loops through the collection and filters it by attribute.

   To use the script, copy it into a text file and execute the script through the Console.

2. Identify the default value for this preference and decide whether to apply a new value.
3. Decide which scope you want the new value to cover.
4. Set a new value using Custom Compiler Tcl `db::setPrefValue`.

   For example, by default, short detection is *on* in design layouts. To turn off short detection, you can set the value for the preference `leDetectShorts` to 0 using Tcl in the Console:

   ```
   db::setPrefValue leDetectShorts -value 0 -scope
       [de::getActiveContext]
   ```

   In this example, `db::setPrefValue` resets the `leDetectShorts` preference value from 1 (on by default) to 0 (off) and limits this setting to the scope that includes only the active design in the current layout window. (Leaving out the optional scope argument applies the setting globally).

   The setting remains valid when you push the design context into another level of hierarchy, and when you save it for the next session. The default setting applies again when you close this window and open another design layout.

Synchronizing Design Windows, Viewports, Net Selections, and Commands Across Opened Designs

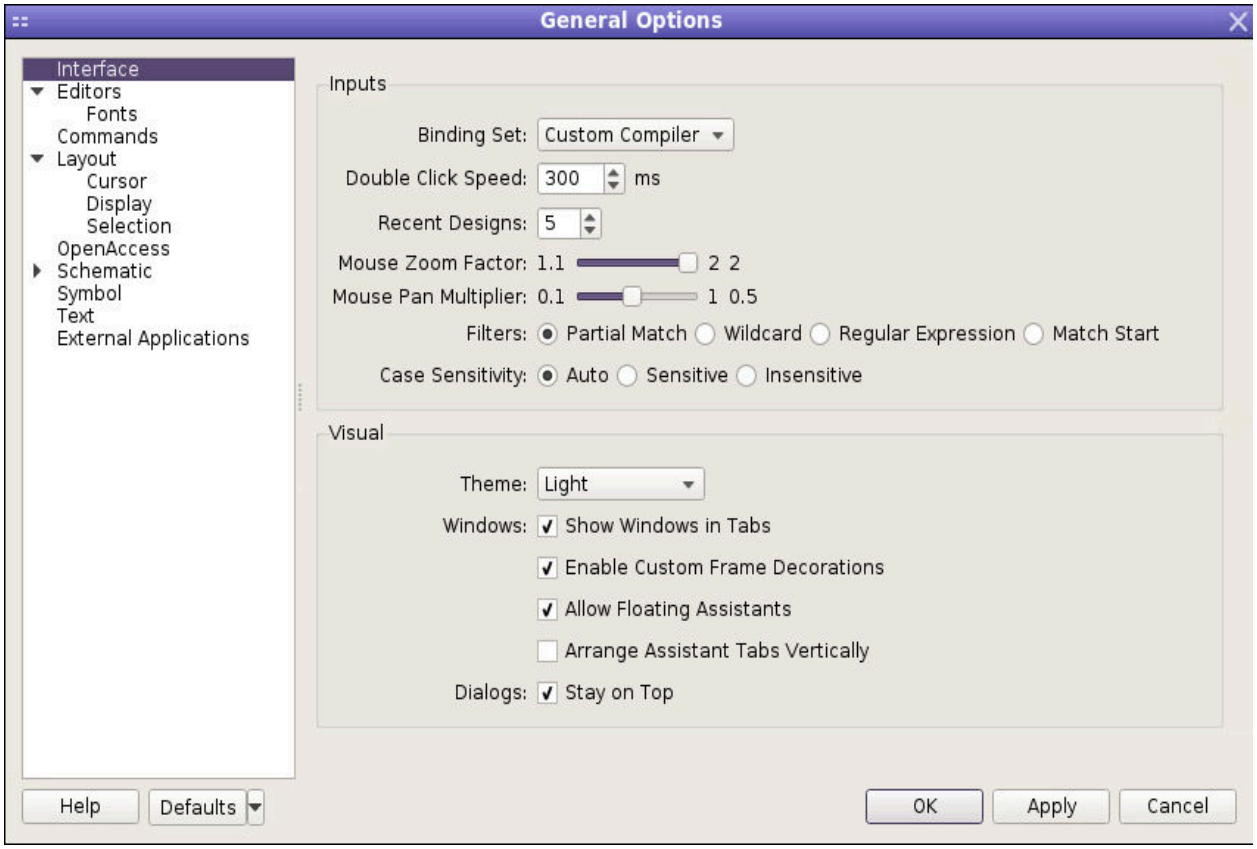
When you are working with multiple designs simultaneously, you can synchronize the opened design windows, viewports, net selections, and command settings to facilitate design tasks.

Feedback

To turn on and set up the synchronization:

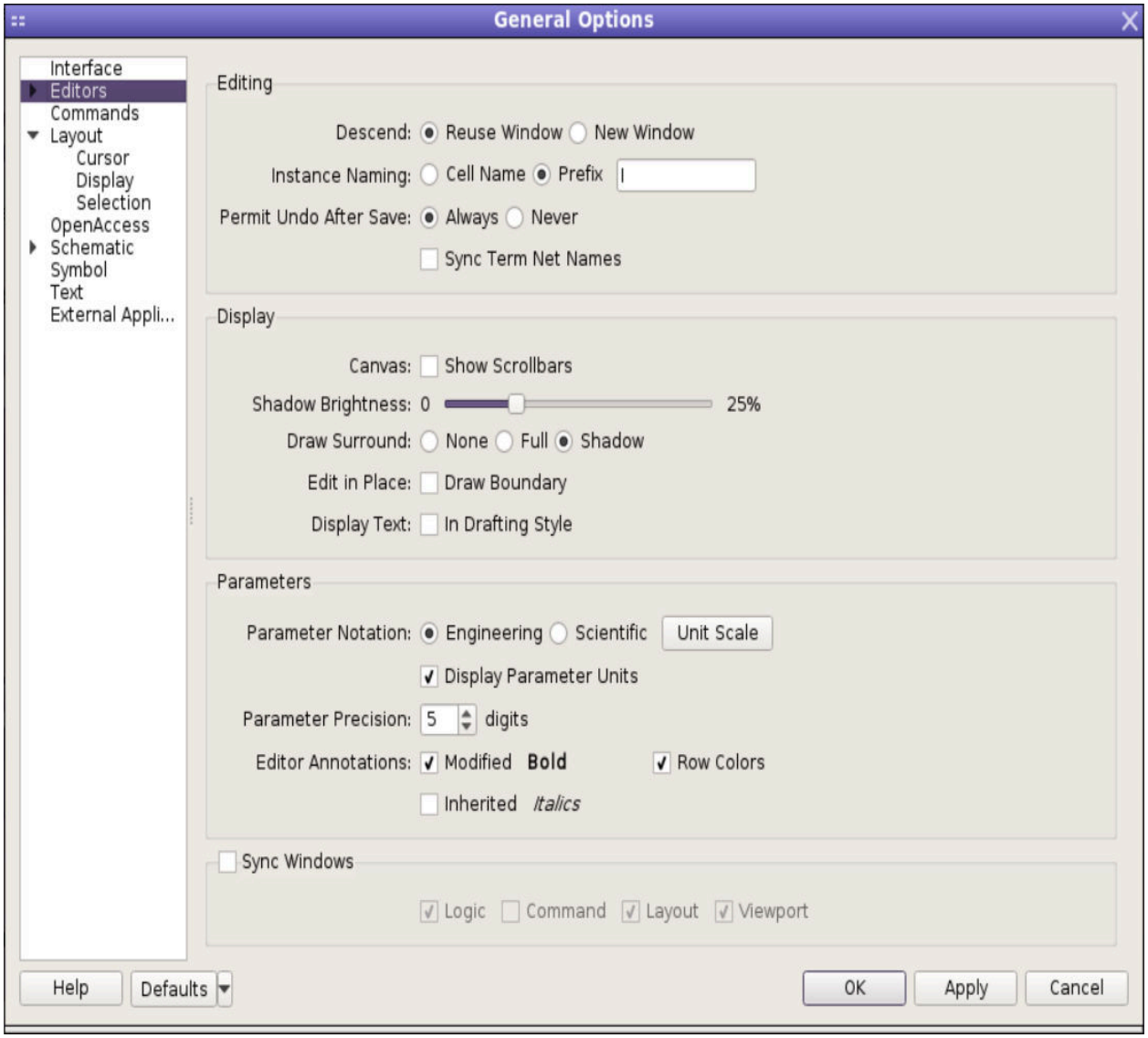
1. From the **Home** tab, choose **Options > General**.

   The **General Options** dialog box opens.



2. Click the **Editors** tab.

Feedback



3. Select **Sync Windows**, and select what you want to synchronize:
   - **Logic**: Synchronizes selected nets (Design Navigator selections) across opened designs.

     Tcl preference: `deSyncWindowLogic`
   - **Command**: Synchronizes active command settings across opened designs.

     Tcl preference: `deSyncWindowCommand`
   - **Layout**: Synchronizes window layouts across opened designs.

Feedback

Tcl preference: `deSyncWindowLayout`

- **Viewport**: Synchronizes viewports across opened designs.

  Tcl preference: `deSyncWindowViewport`

4. Click **OK**.

See Also

- Synchronizing Assistants
- Defining Startup Settings
- Setting General Options

Customizing Keyboard Shortcuts

You can create new and modify existing keyboard shortcuts to accommodate your preferences for accessing frequently used commands and Tcl procedures. In addition, you can import and export your keyboard shortcut specifications so that your preferences are preserved across systems.

This topic describes the following:

- About the Keyboard Shortcuts in the Custom Compiler Environment
- Selecting the Shortcut Key Binding Set
- Viewing Keyboard Shortcuts
- Creating and Editing Keyboard Shortcuts
- Creating Keyboard Shortcuts Using Tcl Commands
- Importing and Exporting Keyboard Shortcut Information

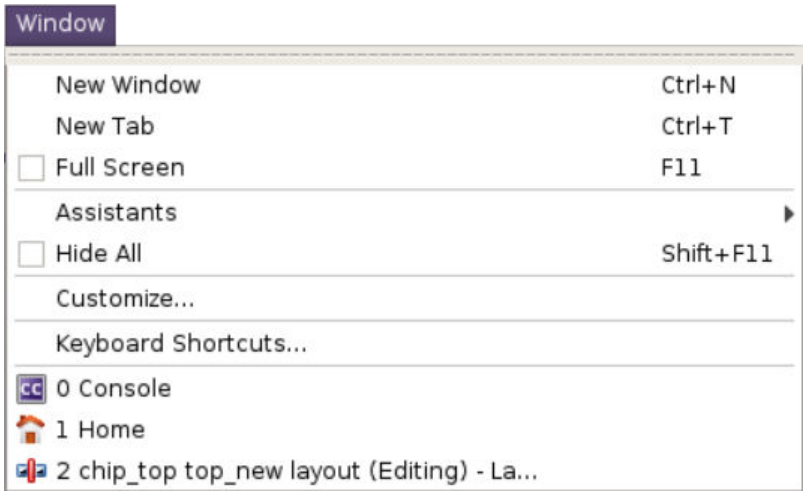
About the Keyboard Shortcuts in the Custom Compiler Environment

Keyboard shortcuts are keys that you press on the keyboard to quickly access commands or functions without going through menus or clicking buttons in the graphical user interface. Using shortcut keys can enable you to complete certain tasks faster, thus saving time and improving productivity.

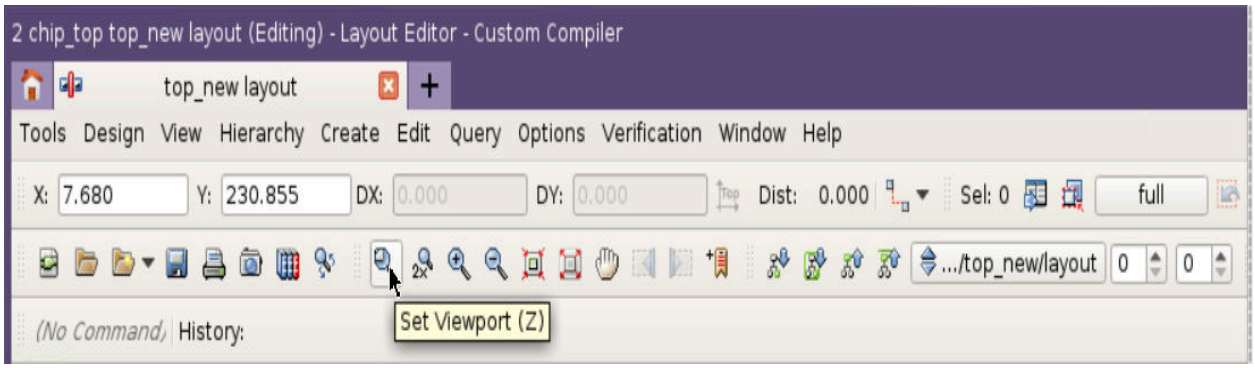
Feedback

Shortcut key information is displayed next to menu command and in the tooltips that appear when you hover over an icon in the Custom Compiler window. In the following examples:

- Shortcut keys for **New Window**, **New Tab**, **Full Screen**, and **Hide All** are displayed next to the commands.



- The tooltip displays the name of the button and the shortcut key for the button: Z.



To view the complete set of shortcut keys, see Viewing Keyboard Shortcuts.

You can select one of the following sets of keyboard shortcuts for use in the Custom Compiler environment. See Setting General Options.

- Custom Compiler
- IC Compiler
- Laker
- Maestro

Feedback

The Custom Compiler environment provides a default set of keyboard shortcuts. However, you can create new or modify the existing keyboard shortcuts. See Creating and Editing Keyboard Shortcuts and Creating Keyboard Shortcuts Using Tcl Commands.

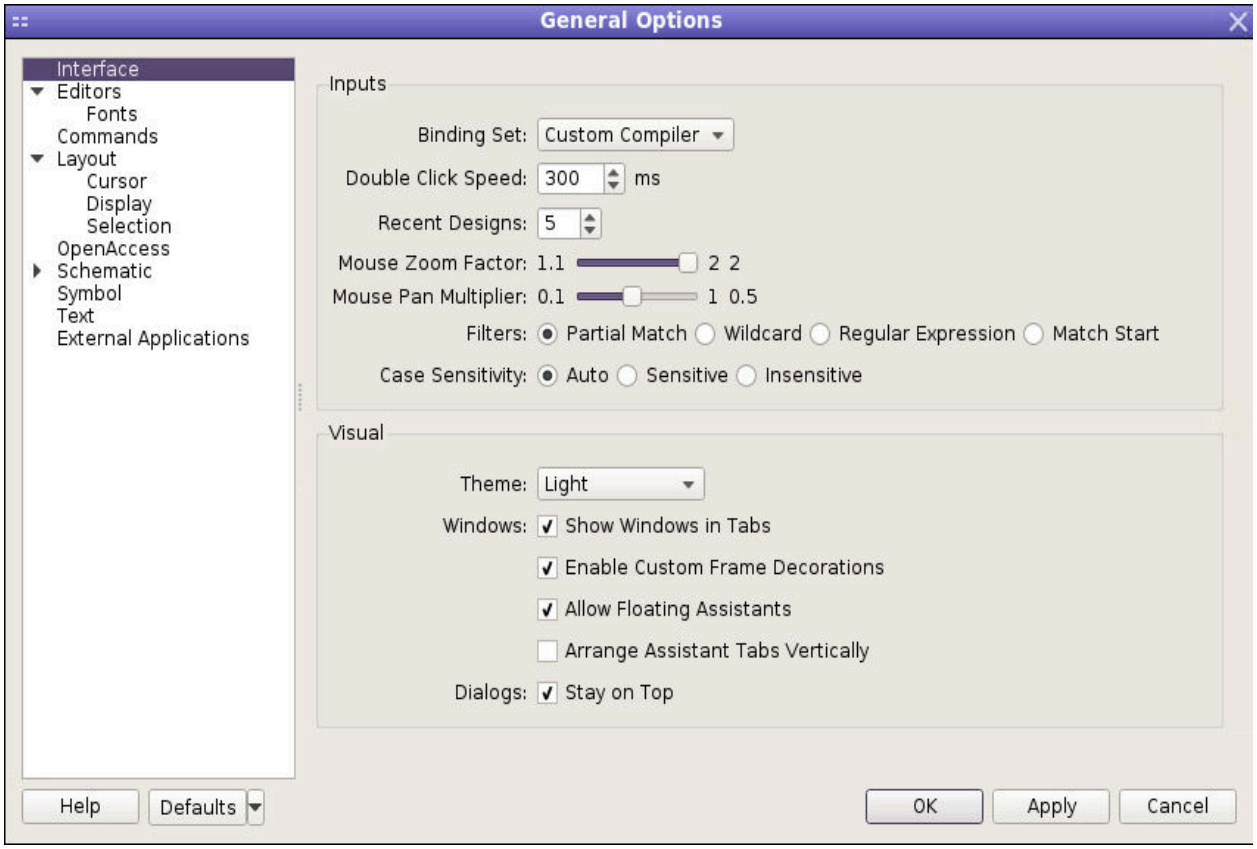
Selecting the Shortcut Key Binding Set

Shortcut keys are bound to commands and buttons. Different sets of shortcut key bindings are available in the Custom Compiler environment to facilitate analog and digital co-design flows or design migrations from third-party tools. This topic describes how to select the shortcut key binding sets to meet your preferences.

Selecting the Shortcut Key Binding Set Using the Graphical User Interface

To select your preferred shortcut key binding set using the graphical user interface:

1. From the **Home** tab, choose **Options > General**.



2. Under **Interface** options, choose the set from the **Binding Set** list:
   - Custom Compiler
   - IC Compiler

Feedback

- Laker
- Maestro

If you created custom binding sets, they would be available in the list.

3. (Optional) To save these settings as general options default settings, choose **Defaults > Save** and apply the settings to a scope.
4. Click **OK**.

See Also

- Defining Session Settings

Selecting the Shortcut Key Binding Set Using Tcl Commands and Preferences

To specify the active binding set using Tcl, use one of the following methods:

1. Use the `giBindingSet` preference, for example:

```
db::setPrefValue giBindingSet -value "Custom Compiler"
```

2. Use the `gi::getBindingSets` command:

```
db::createList [db::getAttr name -of [gi::getBindingSets]]
{Custom Compiler} {IC Compiler} Laker Maestro
```

See the *Custom Compiler Tcl Reference Manual* for details.

Viewing Keyboard Shortcuts

Available keyboard shortcuts are displayed next to menu commands or in button tooltips. You can also view the comprehensive list of keyboard shortcuts for all commands and user interface controls.

Note:

Available shortcut keys for the Layout Editor, Schematic Editor, the PrimeWave Design Environment, and SDL are listed in those tools' respective user guides.

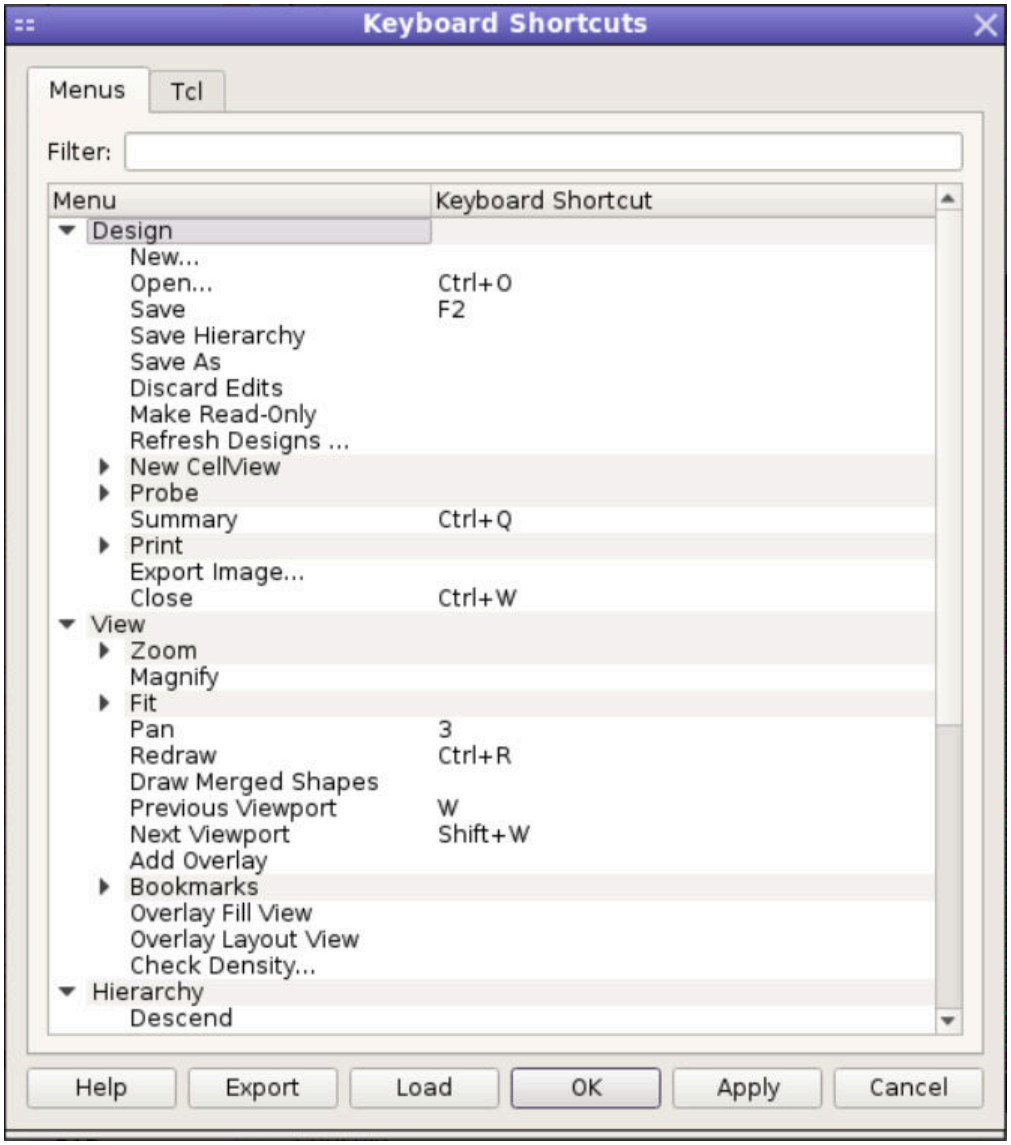
Viewing Keyboard Shortcuts Using the Graphical User Interface

To view available shortcut keys using the graphical user interface:

1. In the Custom Compiler window, choose **Window** > **Keyboard Shortcuts**.

   The **Keyboard Shortcuts** dialog box opens.

Feedback



The **Menus** tab has two columns. The **Menu** column shows menu titles and menu commands. Menu titles do not have keyboard shortcuts. The **Keyboard Shortcut** column shows the corresponding shortcut keys for the commands.

2. When you are finished viewing the shortcut key information, click **OK**.

Feedback

See Also

- Creating and Editing Keyboard Shortcuts
- Creating Keyboard Shortcuts Using Tcl Commands
- Importing and Exporting Keyboard Shortcut Information

Viewing Keyboard Shortcuts at the Source

View available shortcut keys at the directory source to identify shortcut keys associated with an editor or product module.

To view available shortcut keys at the directory source:

1. In the Custom Compiler environment product installation tree, change directories to this location:

   `<installation_directory>/auxx/<product_module>/bindings.tcl`

   where <product_module> is a design editor, such as `layout`.

   In some modules, `bindings.tcl` has a name related to the module name.

   There are several bindings files, including, for example, `layout_editor/icc_bindings.tcl`, `layout_editor/laker_bindings.tcl` and `layout_editor/maestro_bindings.tcl`.

2. Open one of the `bindings.tcl` file to see the list of bindings.

Viewing Keyboard Shortcuts Using Custom Compiler Tcl Procedures

You can also return binding sets with Tcl to see complete collections of shortcut keys, optionally filtered.

You can use a procedure that outputs the Custom Compiler environment shortcut key collection, filtered by attribute, into a file called `bindingFile.txt`.

To return the binding set in this manner, use the following Tcl procedure.

```
proc getBindings { } {
 set binds [gi::getBindings -set "Custom Compiler"]
 set fileName [open "bindingFile.txt" w]

 puts $fileName "Binding Set : Custom Compiler"
 puts $fileName "Total Bind keys defined : [db::getCount $binds]"
 puts $fileName "\n"

db::foreach bind $binds {
 ## To get different attributes for each binding, use
 ## db::listAttrs -of $bind
 puts $fileName "[db::getAttr event           -of $bind] \t \
  [db::getAttr action          -of $bind] \t \
```

Feedback

```
   [db::getAttr command         -of $bind] \t \
   [db::getAttr activeOperation -of $bind] \t \
   [db::getAttr activeObject    -of $bind] \t \
   [db::getAttr activeTool      -of $bind] \t "
 }
 close $fileName
}
```

The procedure performs these Tcl operations:

- `gi::getBindings` returns the Custom Compiler shortcut key collection.
- `db::foreach` loops through the collection and filters it by attribute.
- `db::listAttrs` returns the requested attributes for each shortcut key.

Alternatively, return the binding set using the `gi::saveBindings` command. For example:

1. Type the command:

   ```
   gi::saveBindings -filePath abc.xml
   ```

2. Type the executable:

   ```
   exec gedit abc.xml &
   ```

Creating and Editing Keyboard Shortcuts

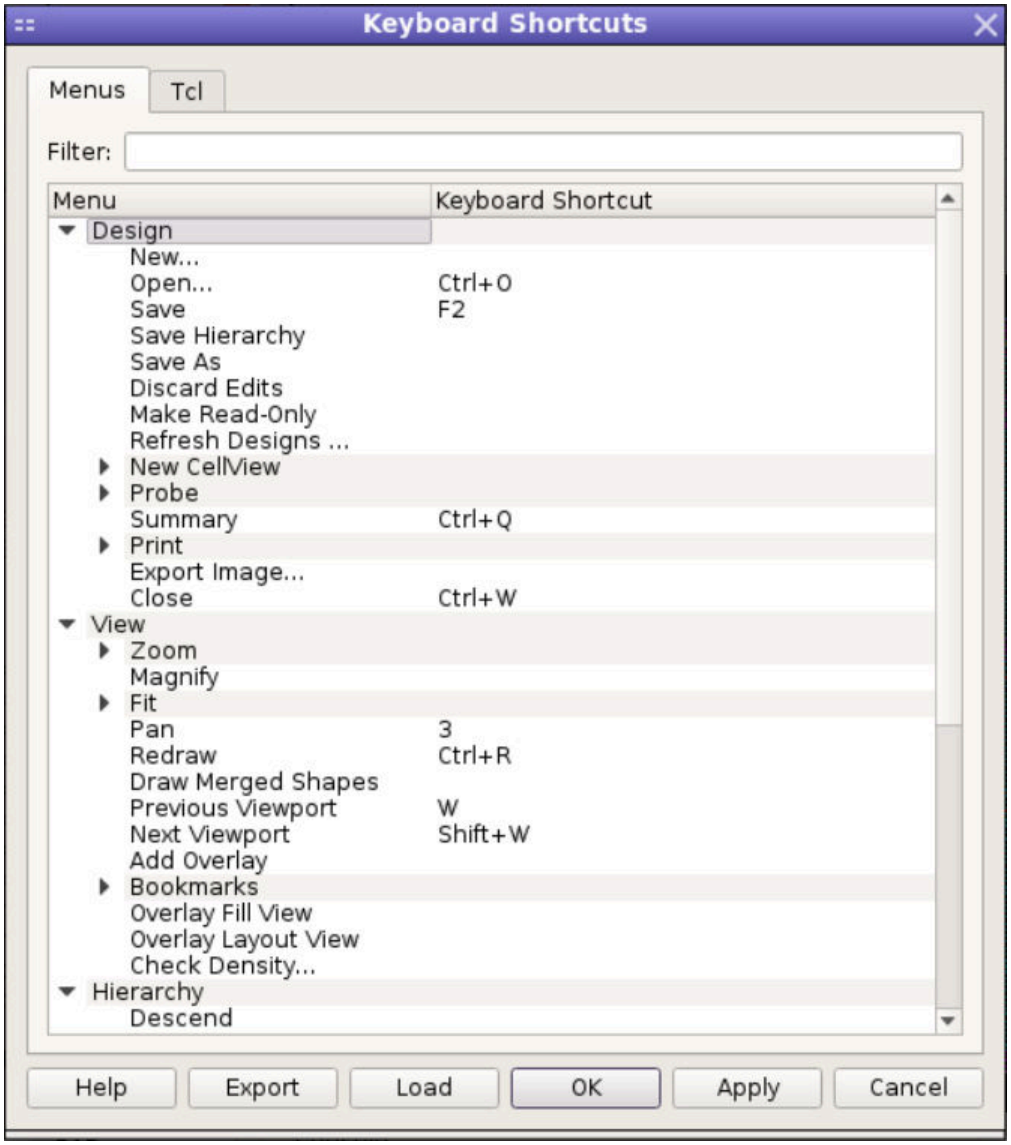
The Custom Compiler environment provides a set of default keyboard shortcuts. To meet your requirements, you can create new or edit existing keyboard shortcuts. This topic describes how to use the **Keyboard Shortcuts** dialog box to create and edit keyboard shortcuts.

To create new or modify existing keyboard shortcuts:

1. In the Custom Compiler window, choose **Window** > **Keyboard Shortcuts**.

   The **Keyboard Shortcuts** dialog box opens.

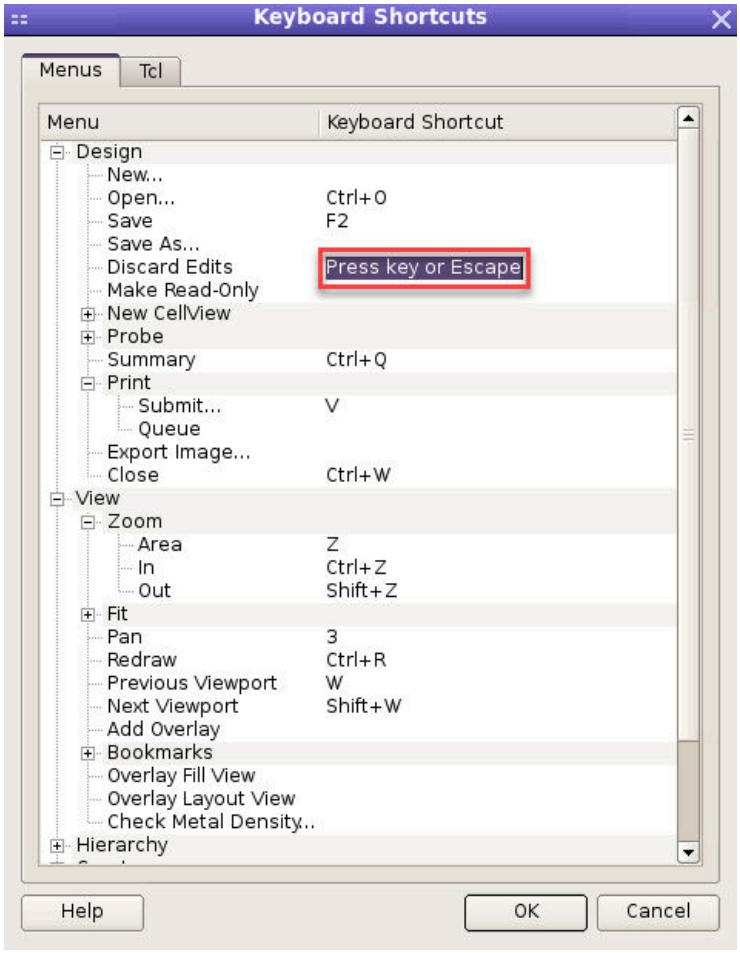
Feedback



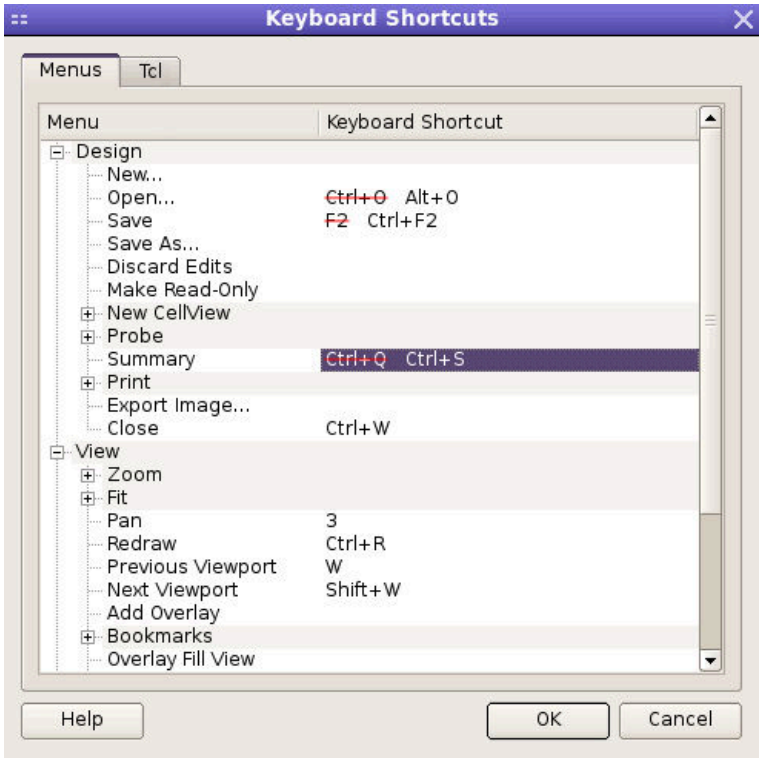
The **Menus** tab has two columns. The **Menu** column shows menu titles and menu commands. Menu titles do not have keyboard shortcuts. The **Keyboard Shortcut** column shows the corresponding shortcut keys for the commands.

2. Click a cell in the **Keyboard Shortcut** column. The text `<Press key or Esc>` appears.

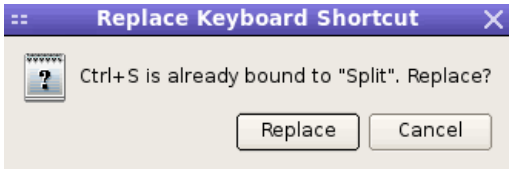
Feedback



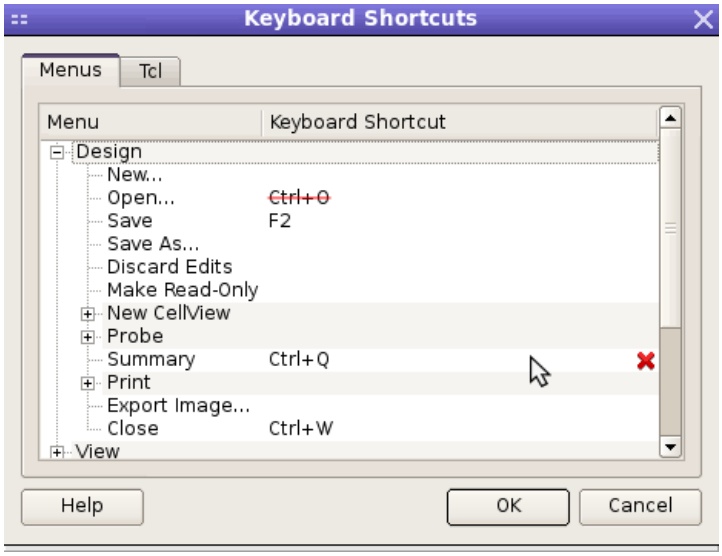
3. To create a new shortcut, press the required key or combination of keys on your keyboard. The names of the keys appear in the **Keyboard Shortcut** column.
4. To modify an existing shortcut, press the required key or combination of keys on your keyboard. The new key names appear and a red strike-through line appear over the old key names.



The **Replace Keyboard Shortcut** dialog box appears if the key combination you entered is already bound to another command. Click **Replace** to continue with your change, or click **Cancel** to return to the **Keyboard Shortcuts** dialog box and enter different shortcut keys.

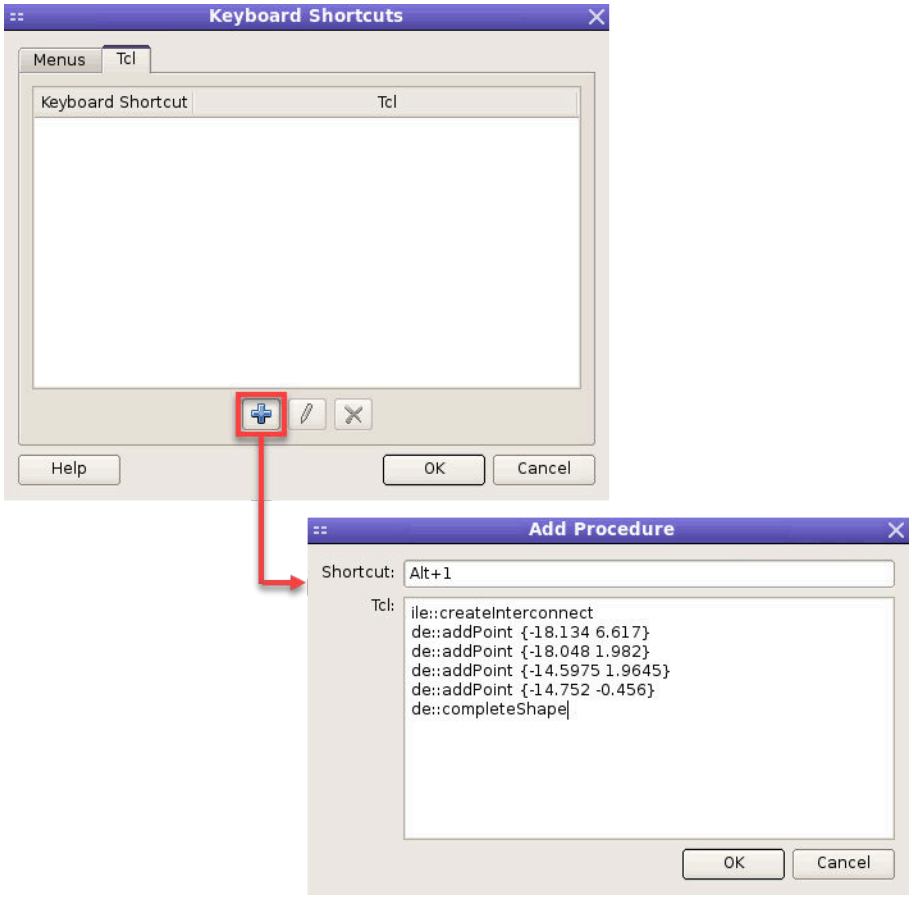


5. To remove an existing shortcut key binding, hover over the shortcut and click ✖. A red strikethrough line appears over the deleted shortcut.



6. To stop editing a shortcut key without saving any changes, press Esc.
7. Click the **Tcl** tab to assign a shortcut key to a Tcl procedure.
8. Click to open the **Add Procedure** dialog box. In the **Add Procedure** dialog box, enter your required shortcut key combination and Tcl procedure.

Feedback

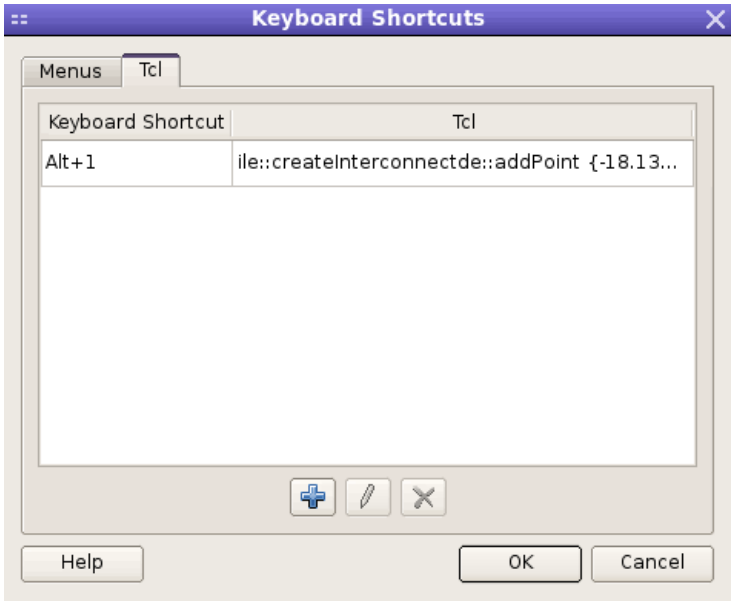


Note:

You can copy a procedure you have already performed from the Console history and edit it (if necessary) in the **Add Procedure** dialog box.

9. Click **OK** to save your changes. The new shortcut is entered in the **Tcl** tab.

Feedback



10. To edit a shortcut for a Tcl procedure, select the shortcut and click to open the **Add Procedure** dialog box again. Make your edits and click **OK**.
11. To delete a shortcut for a Tcl procedure, select the shortcut and click .
12. Click **OK** in the **Keyboard Shortcuts** dialog box to save your changes. Click **Cancel** to discard your changes.

Creating Keyboard Shortcuts Using Tcl Commands

Instead of using the graphical user interface, you can create custom keyboard shortcuts. This topic describes how to use Tcl commands to create custom keyboard shortcuts.

Creating Individual Shortcut Keys

Create individual shortcut keys to offer single keyboard and mouse shortcuts. Use Custom Compiler Tcl to create individual shortcut keys.

To create individual custom shortcut keys:

1. Open the **Console**.
2. Use the `gi::createBinding` Tcl command.

   The command creates a shortcut key within a binding set (or the active set if one is not specified). For syntax details, type `gi::createBinding -help` in the Console. For a usage description, see the *Custom Compiler Tcl Reference Manual*.

In the following example, `gi::createBinding` binds the keyboard sequence Alt+E to the edit-in-place command.

Feedback

```
gi::createBinding -event "Alt-e" -command {ide::descend -inPlace true}
```

In the following example, `gi::createBinding` binds the 7 key to a toggle operation for switching Infix mode on or off.

```
proc toggleInFix  {} {
    if {[db::getPrefValue deInFixMode]} {
         db::setPrefValue deInFixMode -value false
    } else {
         db::setPrefValue deInFixMode -value true
    }
}
gi::createBinding -event "7"  -command toggleInFix
```

See Also

- Console Tab

Creating Binding Sets

Create binding sets to offer a specified selection of bindings to designers.

To create binding sets:

1. Begin by either copying the XML file for an existing binding set and altering it, or creating a new set with shortcut keys you have defined.

   To create shortcut keys, see Creating and Editing Keyboard Shortcuts.

2.

3. In the **Console**, use the Custom Compiler Tcl commands listed in the following table.

   Refer to the examples that follow for assistance.

   For syntax, execute `<command_name> -help` in the **Console**. For a usage description, see the *Custom Compiler Tcl Reference Manual*.

Tcl Command	Usage
`gi::createBindingSet`	Creates and returns a new binding set.
`gi::loadBindings`	Loads bindings from an XML file into the specified set.
`gi::saveBindings`	Saves a collection of bindings to an XML file.

Examples

In this example, `gi::createBindingSet` creates a new binding set by copying shortcut key definitions from the Custom Compiler set.

Feedback

```
gi::createBindingSet "myBindingSet" -from "Custom Compiler"
```

In this example, `gi::loadBindings` loads shortcut keys from a `newBindings.xml` file into the Maestro binding set.

```
gi::loadBindings -filePath newBindings.xml -set Maestro
```

In this example, `gi::saveBindings` saves the shortcut key definitions from the Maestro binding set to a `maestro.xml` file.

```
gi::saveBindings [gi::getBindings -set Maestro] -filePath \
                  maestro.xml
```

4. After you create the binding set, you can make it available by Selecting the Shortcut Key Binding Set.

See Also

- Console Tab

Importing and Exporting Keyboard Shortcut Information

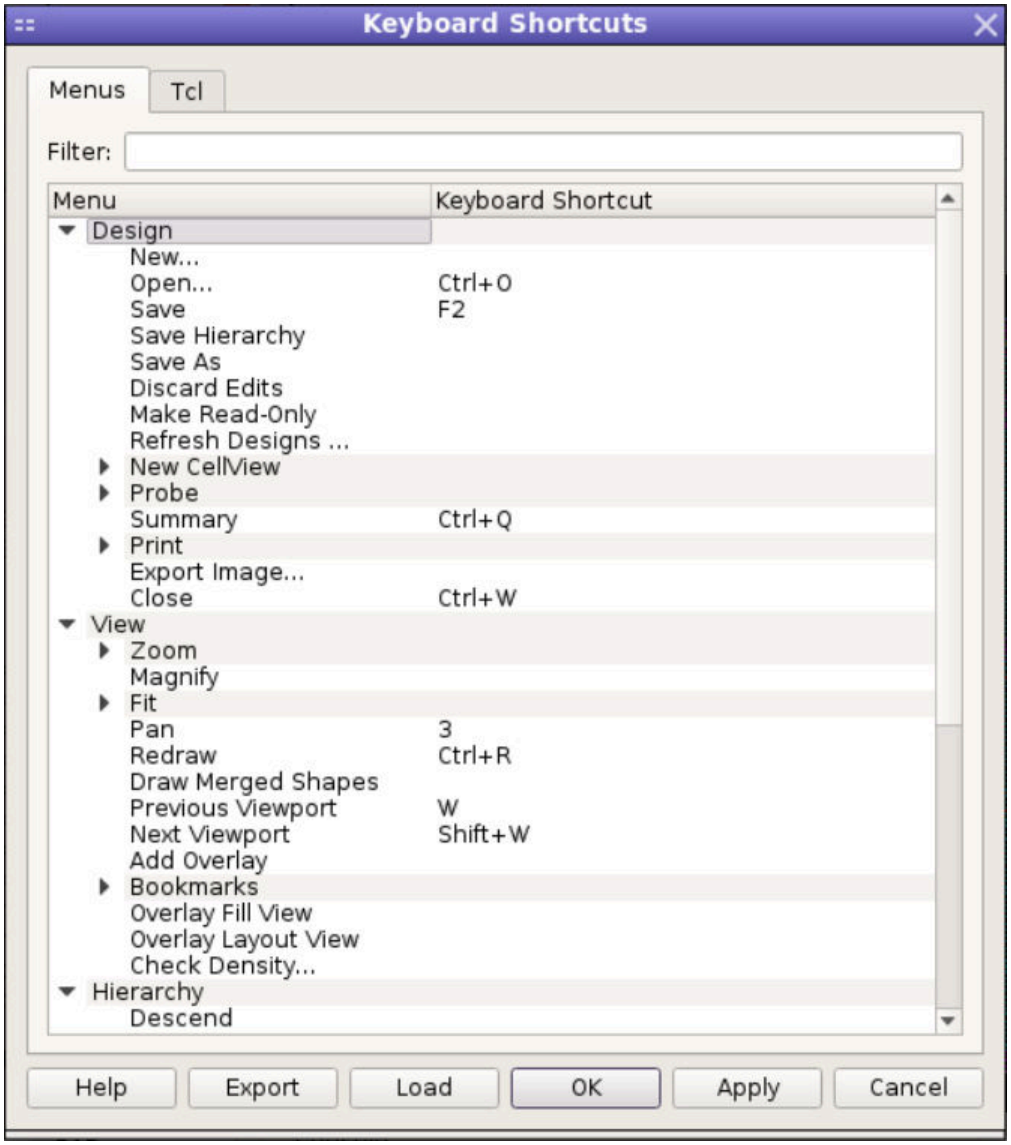
You can import and export your keyboard shortcut specifications so that your preferences are preserved across systems. This topic describes how to import and export the keyboard shortcut information.

To import or export keyboard shortcuts from a file:

1. In the Custom Compiler window, choose **Window** > **Keyboard Shortcuts**.

   The **Keyboard Shortcuts** dialog box opens.

Feedback



2. To import a file:

   a. Click **Load**.

   b. In the **Open** dialog box, locate the keyboard shortcuts file that you want to import, and click **Open**.

      The imported keyboard shortcut information appears in the dialog box.

Feedback

3. To export the keyboard shortcut information:
   a. Click **Export**.
   b. In the **Save As** dialog box, navigate to the target folder and type a name for the exported file.
   c. Click **Save**.

Restricting Design Access to Read-Only

You can restrict design access to read-only so that users can open and view them, but they cannot modify the designs. Use one of the following methods to make designs read-only:

Scope	Method
Designs in all libraries (entire Custom Compiler session)	When starting Custom Compiler, type the following at the Linux prompt: `custom_compiler -readOnly true` For more information, see custom_compiler Command Syntax and Arguments.
Designs in a specific library (entire Custom Compiler session)	Add the following statement in the lib.defs file: `ASSIGN <libName> libMode readOnly` For more information about lib.defs files, see Controlling Library Availability.
Specific design	When opening a design using the Library Manager, right-click the design's view name, and then choose **Open Read-Only**. For details, see Opening Designs. If you have already opened a design and want to make it read-only, see Changing Edit Modes.

By default, when libraries or designs are in the read-only mode, buttons and menu commands that let you modify the design are dimmed. If you want to hide the disabled buttons and menu commands, set the giHideAutoDisabled Tcl preference to `true`.

3

Supplying Mapping Files for GDSII Stream and OASIS Format Translations

To successfully import and export design data in GDSII Stream format, you need to supply mapping files to permit name conversions for cells, fonts, labels, layers, objects, properties, and references.

- Layer Mapping Files
- Object Mapping Files
- Cell Mapping Files
- Font Mapping Files (GDSII Stream Only)
- Label Mapping Files (GDSII Stream Only)
- Property Mapping Files (GDSII Stream Only)
- Property Mapping Files (OASIS Only)
- Reference Mapping Files (GDSII Stream Only)
- Text Mapping Files

Layer Mapping Files

The layer mapping file maps OpenAccess (OA) layer/purpose names to GDSII or OASIS layer number and data types. The file is used for both importing and exporting design data in the GDSII Stream and OASIS formats.

This section describes the following:

- Layer Mapping File Format and Contents
- OA, GDSII, and OASIS Mappings
- Mask Color and Color Lock Attributes
- Material and Mask Number Attributes
- Cut Size Attribute

- Mapping OpenAccess Objects in the Layer Mapping File
- Examples

For instructions on importing and exporting design data in the GDSII Stream format, see Importing Designs in GDSII Stream Format and Exporting Designs in GDSII Stream Format.

For instructions on importing and exporting design data in the OASIS format, see Importing Designs in OASIS Format and Exporting Designs in OASIS format .

Layer Mapping File Format and Contents

All layer mapping files are in the ASCII format. The syntax for each line in the file is as follows:

```
oaLayerName oaPurposeName gdsLayer# gdsDataType# [color:colorName
 [colorLock]] [Material [maskNumber | cutSize]]
```

Field	Description
oaLayerName	(Required) The name of the layer in the OA database. For more information, see OA, GDSII, and OASIS Mappings.
oaPurposeName	(Required) The purpose of the OA layer. For more information, see OA, GDSII, and OASIS Mappings.
gdsLayer#	(Required) The layer number in the GDS or OASIS. For more information, see OA, GDSII, and OASIS Mappings.
gdsDataType#	(Required) The data type number in the GDS or OASIS. The GDS or OASIS data type number can be a single number or a range of numbers. For example, the following are valid values for `gdsDataType#`: 1, 2, 5, or 0-255. For more information, see OA, GDSII, and OASIS Mappings.
color:colorName	(Optional) A mask color. The colorName must be preceded by the keyword `color`, followed by a colon (for example, color:red). The color names you supply must be defined in the technology file, or they must have the value `same`. If no color information is supplied in the file, the translator assumes that the shape on the layer has no color. For more information, see Mask Color and Color Lock Attributes.
colorLock	(Optional) The color lock status, which is only used if a colorName is specified, but it is not required. Valid values for colorLock are `locked` and `unlocked`. For more information, see Mask Color and Color Lock Attributes.

Field	Description
Material	(Optional) The layer function name to map to or from the OA database. The layer function name is typically defined in the technology file. If you are importing a design to a library that does not have the information in its technology database, you can supply the information in the `Material` field of the layer mapping file. For more information, see Mask Color and Color Lock Attributes.
maskNumber	(Optional) The mask number that specifies the manufacturing order of the layers used in the design. If the `Material` value is `cut`, the `maskNumber` field is not used.
cutSize	(Optional) The dimensions of the rectangular cut, `<dimension1>:<dimension2>`. The dimension values must be preceded by the keyword `CUTSIZE`, followed by a colon (for example, CUTSIZE:0.032:0.032). The `Material` field can be used to specify different mappings for cut rectangles of different sizes. If the `Material` value is `cut`, the cut dimensions can follow in the same field. For more information, see Cut Size Attribute.
techLayerAlias	(Optional) The alias of the layers, as defined in the `layerRules` rule of the technology file. • For imported files,all source layer objects in a design changes to corresponding alias layer • For exported files, all alias layer objects in a design changes to corresponding source layer

The layer mapping file can contain the following optional lines:

- Comment lines that begin with the following character: #. For example, a comment line can appear at the top of the file to label the fields.
- Blank lines. For example, you can add blank lines between sections of data. However, the translator ignores all blank lines in the file.

Feedback

OA, GDSII, and OASIS Mappings

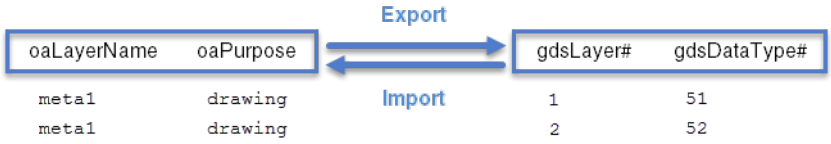
Each line in the file specifies the following mappings:

- During import, shapes on the GDS or OASIS layer number and GDS or OASIS data type are written to the given OA layer name and OA layer purpose.
- During export, shapes on the OA layer name and OA layer purpose are written to the given GDS or OASIS layer number and GDS or OASIS data type.

Each line must contain a unique pair of GDS or OASIS layer number and GDS or OASIS data type number. However, an OA layer name and OA layer purpose pair can appear multiple times in the file for the following reasons:

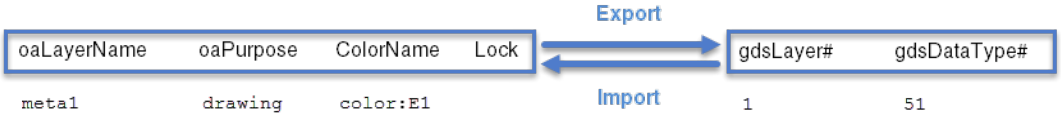
- A shape on a particular OA layer can be written multiple times in the GDSII or OASIS database.
- Because mappings can include color and cut size information, an OA layer and purpose can appear multiple times. A line applies only to shapes that match the particular attributes.

  In the following example, each of the two GDS layer number and GDS data type pairs is unique. However, the OA `metal1 drawing` layer is written two times in the GDS database.

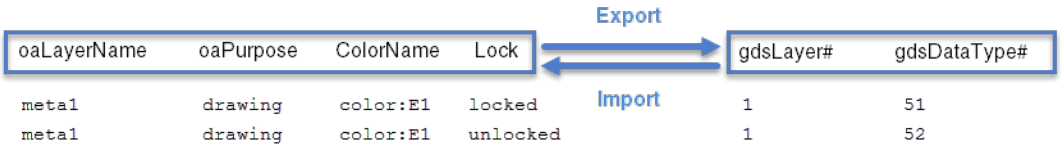


Mask Color and Color Lock Attributes

The optional mask color and color lock data is attributes of shapes that are on the layer described by each line. For any given color attribute, the color lock attribute is optional. In the following example, objects that are on the `metal1 drawing` layer have the color `E1`. Because the line does not have a color lock value, the mapping applies to `E1`-colored shapes on the `metal1 drawing` layer with either color lock status.



A color attribute can have one of two color status: `locked` or `unlocked`. In the following example, if shapes that are on the `metal1 drawing` layer have the color `E1` and the color is `locked`, they are mapped to GDS layer `1` and GDS data type `51`. If shapes that are on the `metal1 drawing` layer have the color `E1` and the color is `unlocked`, they are mapped to GDS layer `1` and GDS data type `52`.



When exporting a design, you can choose which lines are used to map colored shapes:

- Use only the lines that contain the `locked` and `unlocked` keywords. Ignore the lines that do not contain the `locked` and `unlocked` keywords.
- Use only the lines that do not contain the `locked` and `unlocked` keywords. Ignore the lines that contain the `locked` and `unlocked` keywords.

You can make this selection using the **Layer Map Colors** check box in the **Export Stream** dialog box. For more information about the **Export Stream** dialog box, see Exporting Design Data in GDSII Stream Format and Exporting Design Data in OASIS Format.

The following formats are available when you import and export GDSII Stream:

- `VIA1 drawing 1 0 cut CUTSIZE: 0.1: 0.1 mask1Color locked`
- `VIA1 drawing 1 0 mask1Color locked cut CUTSIZE: 0.1: 0.1`

Note:

The values of the `mask color` and `color lock` attributes can be placed before or after the `CUTSIZE` column.

The following format is available when you import and export OASIS:

```
VIA1 drawing 1 0 mask1Color locked cut CUTSIZE: 0.1: 0.1
```

Note:

The values of the `mask color` and `color lock` attributes must be placed before the `CUTSIZE` column.

Material and Mask Number Attributes

The layer mapping file might contain `Material` and `maskNumber` data if you are importing a design to a library that does not have the information in its technology database. If you selected the **AutoDetect Vias** option in the **Import Stream** or **Import OASIS** dialog box, the translator uses these attributes to detect vias during the import process. Doing so stores vias as oaVias instead of oaInsts in the OA database, thus ensuring compactness and a more accurate representation.

For more information about the **Import Stream** dialog box, see the Importing Design Data in GDSII Stream Format. For more information about the **Import OASIS** dialog box, see the Importing Design Data in OASIS Format.

Cut Size Attribute

If you are exporting a design, the `Material` field can contain cut size information in the following format:

```
cut CUTSIZE:<dimension1>:<dimension2>
```

where `cut` is the layer function name defined in the technology file.

The `dimension` values specify the dimensions of the rectangular cut, and the values should be floating numbers that specify the size in user units.

If the `Material` field contains cut information, the `maskNumber` field is not used.

Note:

The `cutSize` attribute is ignored during import.

Mapping OpenAccess Objects in the Layer Mapping File

PR Boundaries, Snap Boundaries, Area Blockages, and Layer Blockages objects can be exported to GDS and imported from GDS using the layer mapping file if the layer/purpose pairs shown in Table 10 are specified in the layer mapping file.

Table 10 Mapping OpenAccess Objects by Layer Purpose Pair

OpenAccess Object	Layer Name	Purpose Name	Keep Object Type with Property
PR Boundary	prBoundary	boundary	yes
Snap Boundary	snap	boundary	yes
Area Blockage	PinBorder	blockage	yes

Feedback

Table 10 Mapping OpenAccess Objects by Layer Purpose Pair (Continued)

OpenAccess Object	Layer Name	Purpose Name	Keep Object Type with Property
Layer Blockage	Any OA layer	blockage	yes

Note:

Object Mapping Files can work with this feature at the same time, but the object map has a higher priority than the layer mapping file.

Examples

In the following example, color and color lock attributes are provided for the `m1` layer. Red color-locked shapes on the `m1 drawing` layer are mapped using the first line. Blue color-unlocked shapes on the `m1 drawing` layer are mapped using the second line. E3 color-locked shapes on the `m1 drawing` layer are mapped using the third line.

```
# oaLayer #     oaPurpose     gds Layer#   gdsData Type#   colorName     colorLock

m1              drawing       1            50              color:red     locked

m1              drawing       2            51              color:blue    unlocked

m1              drawing       3            52              color:E3      locked
```

In the following example, cut size attributes are provided for the `v1` layer. Shapes that are `0.032` x `0.032` on the `v1` layer are mapped using the first line. Shapes that are `0.064` x `0.064` on the `v1` layer are mapped using the third line. Shapes that have other dimensions on the `v1` layer are mapped using the second line.

```
# oaLayer #     oaPurpose     gds Layer#   gdsData Type#   Material                maskNumber

V1              drawing       16           0               cut CUTSIZE:0.032:0.032

V1              drawing       16           1               cut

V1              drawing       16           15              cut CUTSIZE:0.064:0.064
```

Feedback

Object Mapping Files

This file specifies methods of mapping Boundaries and LayerBlockages between Stream shapes and OpenAccess databases.

This file is a text file that have a different number of fields for different initial keywords. Blank lines and lines starting with '#' are ignored. The first field is the object type name.

For example, with industry-compatible keywords:

```
# Boundary     Subtype   GDS Layer     GDS Data Type
Boundary       PR        27            4
Boundary       Snap      27            4

# Blockage     Subtype   oaLayer  GDS Layer GDS Data Type  Color
layerBlockage  routing   M1       191       31             color:red
layerBlockage  routing   M2       191       32             color:blue
layerBlockage  routing   M3       191       33             mask1color
layerBlockage  pin       M1       191       34             mask2color
layerBlockage  pin       M2       191       35
layerBlockage  pin       M3       191       36
```

For example, with first generation keywords (not supported for OASIS):

```
# Boundary       GDS Layer   GDS Data Type
  PRBoundary     27          4
# Blockage       oaLayer     GDS Layer   GDS Data Type
  LayerBlockage  Metal5       52          2
  LayerBlockage  *            72          2
```

Note:

Color can be specified as a layer mapping file for `layerBlockage` (with industry-compatible keywords only).

The tool supports to keep signal type in the object mapping file. The signal type can be mapped with the `Term` keyword. For example:

```
# Term     SigType   oaLayer   oaPurpose   GDS Layer GDS Data Type
# Term     scan      m0        drawing     230       240
# Term     power     m1        drawing     230       241
```

Cell Mapping Files

Note:

For OASIS, cell mapping files are supported by the export process but not the import process.

The cell mapping file maps cell and port names during translation.

The file can contain comment lines that begin with this character: #.

The wildcard character (*) is allowed for GDSII Stream import and export for library names, OA cell names, and cell names when importing or exporting GDSII Stream files. A prefix and postfix can be combined with a wildcard for OA cell name and cell name. Only one wildcard is allowed for same library name, OA cell name, or cell name.

The system ignores blank lines.

Create an ASCII file formatted as follows:

```
<library name> <OA cell name > <view name> <cell name>
```

For example:

```
#lib name  OA cell name   view name   cell name
Lib1  oa_cell1      layout      cell1
Lib2  oa_cell2      layout      cell2
```

Font Mapping Files (GDSII Stream Only)

The font mapping file maps font IDs to OpenAccess font names.

The file can contain comment lines that begin with this character: #.

The system ignores blank lines.

Create an ASCII file formatted as follows:

```
<font id> <OA font name>
```

For example:

```
#stream ID  OA font name
0    EuroStyle
1    Roman
2    Gothic
3    Script
```

Label Mapping Files (GDSII Stream Only)

The label mapping file is used by the translator to generate labels for OpenAccess objects.

The file also specifies which stream layers and data types should be used for the TEXT records that constitute each label.

The file is an ASCII files with either five or six white-space separated fields.

The file can contain comment lines that begin with this character: #.

The system ignores blank lines.

For example:

```
# Object     oaLayer    oaPurpose    GDS Layer    GDS Data Type    oaSigType
Term         Metal5     drawing         131          0
Term         Metal6     drawing         132          0
Net          Metal6     drawing         132          1
Term         Metal6     drawing         132          2              power
Net          Metal5     drawing         131          3              power
Net          Metal5     drawing         131          5              clock
```

Label mapping provides support for multiple pinfigs or vias using the `dbExportStreamLabelMapWithMultiPinFig` Tcl preference (false by default). By default, label mapping converts only one connected pinfig or via for the same term name or net name. When `dbExportStreamLabelMapWithMultiPinFig` is set to true, label mapping converts all connected pinfigs and vias for the same term name or net name. This preference corresponds to the option `labelMapWithMultiPinFig` in the exportStream binary. For example, assume there are three pins with same termName (a1), but on different layers (m1/drawing m2/drawing m3/pin), and the label mapping file defines "Term m2 drawing 10 0", "Term m1 drawing 8 0" and "Term m3 pin 12 0". The default label mapping generates only one label per shape because the three pins have the same termName (a1). If you use `dbExportStreamLabelMapWithMultiPinFig true`, label mapping generates three labels for three shapes.

Property Mapping Files (GDSII Stream Only)

The property mapping file defines mappings between property names in the GDSII file and property IDs and object types in OpenAccess.

These object types are allowed: Dot, Line, Path, PathSeg Pin, Polygon, Rect, Text, ArrayInst and ScalarInst.

The file can contain comment lines that begin with this character: #.

Create an ASCII file formatted as follows:

```
<prop id> <object type> <OA prop name>
```

For example:

```
# stream ID    object         name
   1            Rect         instP11
   2            Polygon      poly1
   3            Path         sigName
   4            PathSeg      pathSegNet
```

The following is an example of using the Pin object type during export:

```
#stream ID        object            property name
4                  Pin              accessDir,pinName,termName,direction
5                  ScalarInst       instName
```

where `accessDir`, `pinName`, `termName` and `direction` are reserved keywords for the attributes of the oaPin object. You can define multiple keywords simultaneously using a property index, separating each name with a comma. They cannot be defined in duplicate in different lines. `instName` is a reserved keyword for the object types `ScalarInst` and `ArrayInst` for exporting the names of instances.

Property Mapping Files (OASIS Only)

The property mapping file defines mappings between property names in the OASIS file and property IDs and object types in OpenAccess.

The following object types are allowed: Dot, Line, Path, PathSeg, Polygon, Rect, Text, ArrayInst, Pin and ScalarInst.

Note:
Pin is used for exporting OASIS files only.

To export specific properties of object types in OpenAccess to OASIS properties, you must name the attributes of object properties in the mapping file as an *<id, value>* pair. *<id>* and *<value>* are string values.

The file can contain comment lines that begin with this character: #.

Create an ASCII file formatted as follows:

```
<OA object type> <OA prop name>
```

For example:

```
  OA_object_type    OA_prop_name
        Rect            instP11
        Polygon         poly1
        Path            sigName
        Pin            accessDir,pinName,termName,direction
        ScalarInst     instName
```

The following is an example of using the Pin object type during export:

```
#OA_object_type   OA_prop_name
Pin                  accessDir,pinName,termName,direction
ScalarInst           instName
```

where `accessDir`, `pinName`, `termName` and `direction` are reserved keywords for the attributes of the oaPin object. You can define multiple keywords simultaneously using a

property index, separating each name with a comma. They cannot be defined in duplicate in different lines. `instName` is a reserved keyword for the object types `ScalarInst` and `ArrayInst` for exporting the names of instances.

Reference Mapping Files (GDSII Stream Only)

The reference mapping file maps every structure the system writes to the stream file and to the source OpenAccess design. This includes PCell submasters, standard and custom vias, and PCell custom vias. PCells are identified by their override parameters. Vias include their viaDef name and standard vias include the values for any override viaParams.

For example:

```
<!-- Created By exportStream G-2011.09 Started: Mon Mar 28 08:59:12 2012
 (Hostname: Host1) -->
<file-format version="1.0" name="synopsysExportStream">
  <exportStream>
    <options lib="ADFULG" cell="AD4FUL" view="layout"
      gds=remote/us01home10/user11/bugs/gdsprb/chklibdef.gds
      hierDepth=2 noOutputBlockages text=cdba ver=3 />
    <cellMappings>
      <mapping cellType="design" streamName="AD4FUL" lib="ADFULG"
 cell="AD4FUL" view="layout"/>
      <mapping cellType="design"
 streamName="p_4t_pcell_30136ea9a532e738c5494893f31a38a0"
        lib="generic90RF" cell="p_4t" view="layout">
            <param name="nf" type="intParam">2</param>
            <param name="w" type="floatParam">0.00000058</param>
            <param name="tapBottom"
 type="stringParam"><![CDATA[all]]></param>
            <param name="drawWell" type="booleanParam">1</param>
      </mapping>
      <mapping cellType="stdvia"
 streamName="via12_stdvia_f67b15b30b330a6024d3f1bd9610d2c4"
 techLib="ADFULG" viaDef="via12">
            <viaparams cutColumns="8" cutRows="2" layer1Enc="5.5,6.1"/>
      </mapping>
      <mapping cellType="customvia"
 streamName="via12_cust_pcell_8972b97befe9a7af324f4962de1704a0"
        lib="generic90RF" cell="via12_cust" view="layout"
 techLib="ADFULG" viaDef="via12">
            <param name="columns" type="intParam">2</param>
            <param name="rows" type="intParam">8</param>
      </mapping>
    </cellMappings>
  </exportStream>
</file-format>
```

Text Mapping Files

A text mapping file is used to specify how to map text between an OASIS or a GDSII stream file and an OA database, and to process the text for conversion in an OA database.

A text mapping file maps individual characters and delimiters in text labels. For example, during the import process, you can map the text label `IN[1]` to `IN<1>`. You must prepare the text mapping file before you import your design.

OASIS Text Mapping Files

The text mapping file syntax of each line for an OASIS file is as follows:

```
<cell-name> <layer-name> <layer-purpose> <text-mapping-string>
 <text-replacement-string>
```

Field	Description
cell-name	The name of the cell in OASIS.
layer-name	The OASIS layer name.
layer-purpose	The OASIS layer purpose.
text-mapping-string	The current text match, that is, the text in OASIS that is replaced when it is imported into the OA database.
text-replacement-string	The text that replaces the OASIS text during the import process.

Each field can be a regular expression with support to recall the regular expression matches. Empty lines and lines that begin with the hash (#) character are ignored. The first match found in the text mapping file is used during conversion.

The following is an example of an OASIS text mapping file:

```
# cell-name layer-name layer-purpose text-mapping-string  text-replacement-string
   cell1     11         0             IMP<0>               IMP\[0\]
   cell2     .+         0            ([^<]+)<([^>]+)>      \1[\2]
   cell3     .+         .+           (^[^\[]+)\[([^\]]+)\] \L\1\E<\2>
```

GDSII Stream Text Mapping Files

All text mapping files must be in the ASCII format. The syntax for each line is as follows:

```
<GDS cell name> <GDS layer #> <GDS data type> <Current text match>
<Replacement text>
```

Field	Description
GDS cell name	The name of the cell in GDS.
GDS layer #	The GDS layer number.
GDS data type	The GDS data type.
Current text match	The text in GDS that is replaced when it is imported into the OA database.
Replacement text	The text that replaces the GDS text during the import process.

In addition:

- The fields must appear in the order shown.
- All fields can contain string literals or regular expressions. When you provide a string literal, the software searches for the text that is identical to the one you provided. When you use regular expressions, you can search for text that match specific criteria. For example, you can use the regular expression `[0-9]+` to look for any integer. For information about writing regular expressions, consult a regular expression reference.
- The fields must be delimited by a space character. A space character that appears after the last delimiter is considered a part of the last regular expression or string literal.

In the following example:

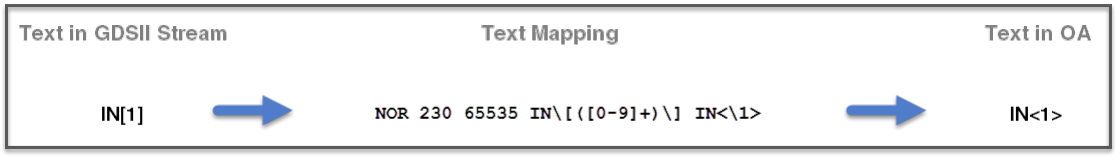
```
NOR 230 65535 IN\[([0-9]+)\] IN<\1>
```

- `NOR` is the GDS cell name. The text mapping applies only to the text in the NOR cell.
- `230` is the GDS layer number. During the import process, the software looks for GDS layer 230.
- `65535` is the GDS data type. During the import process, the software looks for GDS data type 65535, which typically maps to the drawing layer purpose.
- IN\[([0-9]+)\] is the regular expression that looks for `IN[ ]` and remembers any integer encountered between the square brackets.
- IN<\1> is the regular expression that replaces the square brackets, `[` and `]`, found by the previous regular expression and replaces them with angle brackets, `<` and `>`. In addition, the expression recalls the integer that was stored by the previous expression and places it between the angle brackets.

Figure 1 shows the result of the example text mapping.

Feedback

Figure 1 Mapping a Text Label from GDSII Stream to OA



4

Import

This chapter describes how to import designs and library data from different platforms through the Custom Compiler Console. The supported formats include IC Compiler, GDSII Stream, OASIS, LEF, DEF, Verilog (netlist text), and EDIF 200.

This chapter contains the following topics:

- Importing Design Data in GDSII Stream Format
- Importing Design Data in OASIS Format
- Importing Design Data in Library Exchange Format
- Importing Design Data in Design Exchange Format
- Importing Design Netlist Data as Text Views
- Importing Design Netlist Data for Schematic Generation
- Importing Design Data in EDIF 200 Format

To import technology data instead of design data, use the Technology Manager Tool.

Importing Design Data in GDSII Stream Format

Import design data in GDSII format by defining settings and options for the translation process to use.

Perform the import task after preparing mapping files. See Supplying Mapping Files for GDSII Stream and OASIS Format Translations.

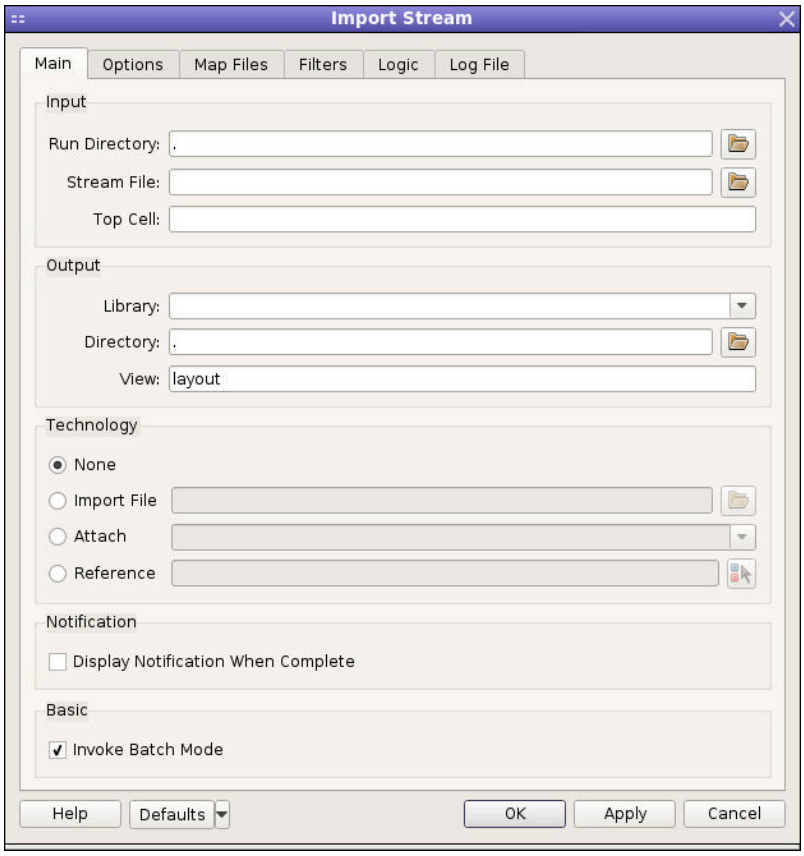
For information about Tcl commands to import design data in GDSII Stream format, see the *Custom Compiler Tcl Reference Manual* for details.

Feedback

To import design data in GDSII Stream format:

1. From the **Home** or **Console** tab, choose **File > Import > Stream** to open the **Import Stream** dialog box.

Figure 2 Import Stream Dialog Box: Main Tab



On the **Main** tab, enter settings as described in Table 11.

Table 11 Import Stream Settings (Main Tab)

Settings	Description
Input	**Run Directory** Directory you browse to select, or a new directory you create by entering its name. Leaving this field blank uses the current working directory. **Stream File** Stream file you browse to select. Click the folder icon to open the **Select Existing File(s)** dialog box. In the dialog box, the **Files of type** field is filtered by default for formats `.gds`, `.db`, `.strm`, `.Z`, and `.gz`. Select * from the drop-down menu to see all existing files.

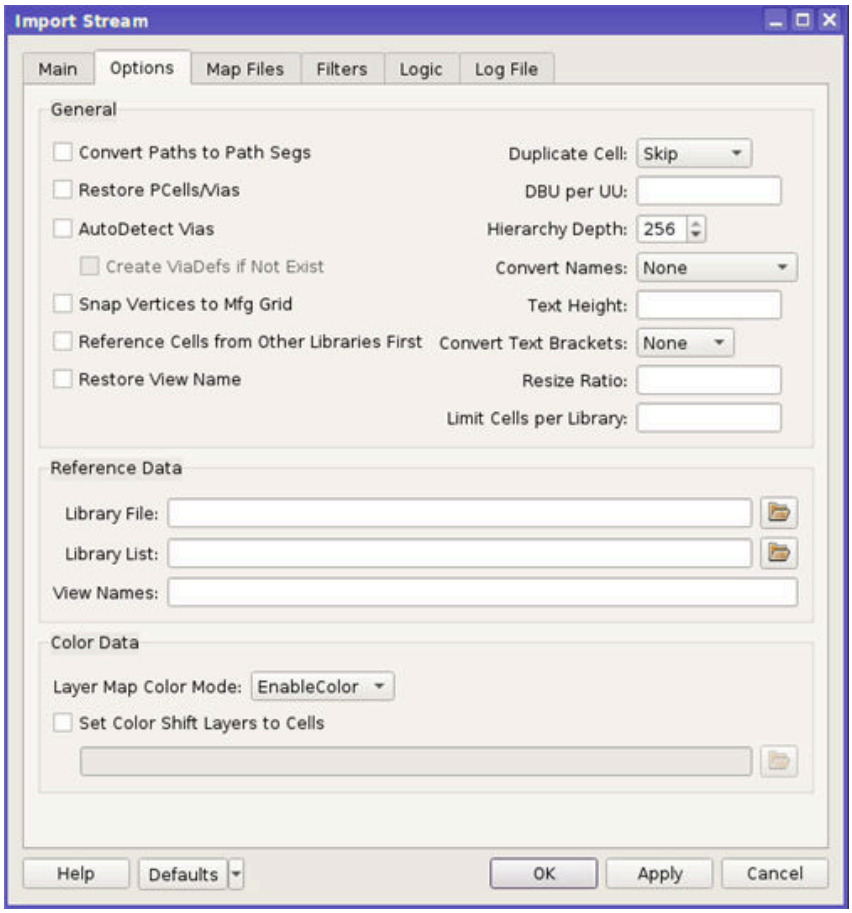
Feedback

Settings	Description
	Top Cell Cell to create and use for the import process.
Output	**Library** Target library for the Stream import. If the library is defined in the `lib.defs` file, Custom Compiler imports GDSII to this library by overwriting the existing library. If the library is not in the `lib.defs` file, Custom Compiler creates a new library and imports GDSII to this new library.
	View Name of the view to create; default is a layout view.
Technology	Choose **Import File**, **Attach**, or **Reference**. Default is **None**. You can change the selection of the technology group of options if the **Output Library** field is empty or a new library name is specified.
Notification	**Display Notification When Complete** Specifies whether to display a window with completion information after import. Off by default.
Basic	**Invoke Batch Mode** When selected (the default), runs importStream integrated with the Custom Compiler tool. Uncheck this option to run as a standalone utility.

2. Click the **Options** tab (Figure 3).

Feedback

Figure 3 Import Stream Dialog Box: Options Tab



On the **Options** tab, enter settings as described in Table 12.

Table 12 Import Stream Settings (Options Tab)

Settings	Description
General	**Convert Paths to Path Segs**: Imports two-point orthogonal and diagonal path records as `pathSeg` objects. Off by default. **Restore PCells/Vias**: Restores PCell, editable PCell, MCell, abutment, and via objects from the GDSII Stream file exported through **File > Export > Stream > Options > PCell > Keep** and **File > Export > Stream > Options > Keep Vias**. Off by default. When disabled, PCell, editable PCell, MCell, MPP, abutment, and via instances are translated into fixed layout instances after stream in and stream out.

Settings	Description
	AutoDetect Vias: Creates oaVias rather than instances for certain structures in the GDS files. Off by default. It requires either that you have an existing technology file defining the layer material types, or that the layer map file used during import has the material types specified for the layers. With that information it creates oaStdVias where a design follows the form that can be generated as an oaStdVia. In addition, if there is an existing tech file with oaCustomViaDefs then the translator creates oaCustomVias where the Stream file has references to structures that match the master of the oaCustomViaDef **Create ViaDefs if Not Exist**: Specifies whether to create via definitions if they do not exist in the tech database when detecting via objects during Stream import. Off by default. **Snap Vertices to Mfg Grid**: Snaps object vertices to manufacturing grid during Stream import. Off by default. **Limit Cells per Library:** Specifies the number of cells in a library during Stream import. Tcl preference: `dbImportStreamLimitCellsPerLib`. **Restore View Name**: Automatically restores the view name during import stream. If the names of cell or view conflict with the restore view name in the reference library, the tool retains the cell name in the GDS file. Off by default. **Reference Cells from Other Libraries First**: References cells from other libraries first if the same cell body name exists in both the input GDSII file and other libraries during the import process. Off by default. **Duplicate Cell**: Allows you to **Skip** (default), **Rename**, or **Overwrite** duplicate cells. In non-batch mode (when **Invoke Batch Mode** is not selected on the **Main** tab), the **Query** option becomes available. If **Query** is selected, during duplicate cell handling the **Action for Duplicate Cells** dialog box appears, prompting you to select how to handle the duplicated cell: **Overwrite**, **Rename**, or **Skip**. The dialog box also provides the option to treat all duplicate cells in the same way: **Overwrite All**, **Rename All**, or **Skip All**. **DBU Per UU**: The value for database units per user units. **Hierarchy Depth:** The depth of hierarchy to which to translate; default is 20 levels. **Convert Names**: Specifies translator conversion of cell and instance name text to either uppercase, lowercase, or none (no conversion) (default).

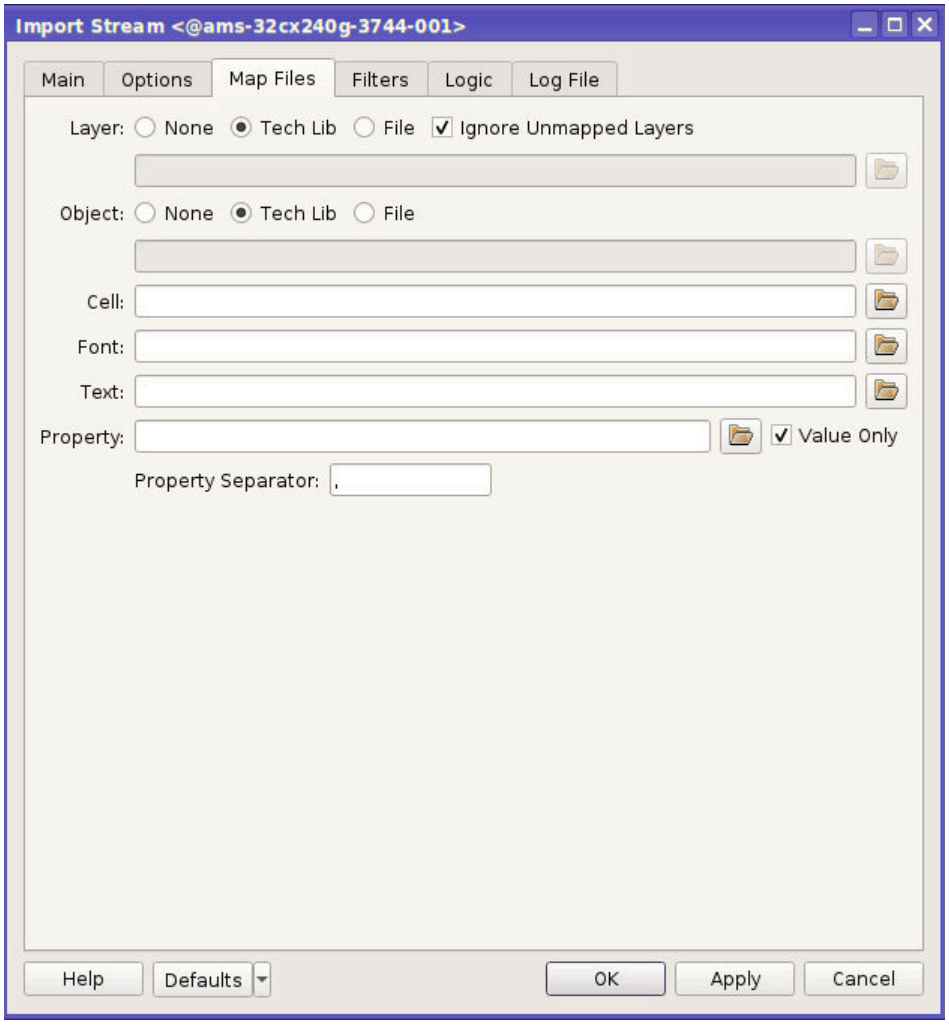
Feedback

Settings	Description
	Text Height: The height to assign to imported text, in user units. **Convert Text Brackets**: Specifies whether or how to convert text brackets during export: ◦ **None**: (default) Do not convert text brackets. ◦ **Angle**: Convert square text brackets, "[" and "]", to angle text brackets, "<" and ">". ◦ **Square**: Convert angle text brackets, "<" and ">", to square text brackets, "[" and "]". **Resize Ratio**: Specifies whether the translator resizes cells during import. The specified value is limited from 0.000001 to 100000. This option cannot be used with **Auto DetectVias** or **Restore PCells/Vias**.
Reference Data	**Library File**: The file you browse to select. This file has the names of reference libraries for the translation process to use when it binds cells referenced during import. **Library List:** The list of libraries with cells referenced in the design. Click the folder icon to open the **Import Stream Reference Library Browser**.
	◦ Sort from the libraries **Available** to the libraries **Selected** using the arrow buttons between the two areas.
	◦ Where there are many libraries in the selected list, you can use the arrow buttons at the right edge to scroll.
	View Names: The views of reference libraries to be used to bind cells referenced during import.
Color Data	**Layer Map Color Mode**: Specifies the layer mapping mode for the color attribute. Options are **EnableColor** (the default) and **DisableColor**. **Set Color Shift Layers to Cells**: When selected, browse to select the color shifting cell rename mapping file. Specifies if the tool sets the metal or cut color shiftable layers to the cells from the specified color shift mapping file. **Note:** If you do not specify a mapping file, the tool sets all color shiftable metal or cut color shiftable layers to the cells

3. Click the **Map Files** tab (Figure 4).

Feedback

Figure 4 Import Stream Dialog Box: Map Files Tab



On the **Map Files** tab, enter settings as described in Table 13.

Feedback

Table 13 Import Stream Settings (Map Files Tab)

Settings	Description
Layer	Select the required layer mapping mode: ◦ **None**: Do not use a layer mapping file for layer/purpose mapping. Instead, use the GDSII layer number and datatype for the OpenAccess layer number and purpose number. If the object type property is kept in the GDS file and the GDSII layer number and datatype are the same as the layer number and purpose number of a particular layer name and purpose name of OpenAccess object (see Table 9), then import as an OpenAccess object. ◦ **Tech Lib**: (default) Use the layer mapping files in the technology library, merging them if more than one exists. If there are duplicates, map the shapes in GDSII to the first matched OpenAccess layer name and purpose name. ◦ **File**: Use the layer mapping file you browse to select. This maps OpenAccess layer purpose pairs from the input file. In a mapping line where the material of type *cut* is provided, it can be followed by the keyword CUTSIZE followed by two colon-delimited floating numbers that represent the size of a cut rectangle in User Units. Where this is present, those cuts that match the given dimensions are written to the GDS file using that mapping, and not using any mapping provided without a CUTSIZE option. If **Ignore Unmapped Layers** is not selected and the GDSII layer number and data type is not matched in the layer mapping file, use the GDSII layer number and data type for OA layer number and purpose number. If particular layer names and purpose names for OpenAccess objects are used in the layer mapping file (seeTable 10), then import as an OpenAccess object. If **Ignore Unmapped Layers** is selected (default), the GDSII object is filtered out. For information about the layer mapping file, see Layer Mapping Files.
Object	Select the required object mapping mode: ◦ **None**: Do not use an object mapping file. ◦ **Tech Lib**: (default) Use the object mapping files in the technology library, merging them if more than one exists. If there are duplicates, map the shapes in GDSII to the first matched OpenAccess object. ◦ **File**: Use the object mapping file you browse to select. Specifies methods of mapping Boundaries and `LayerBlockages` between GDSII shapes and OpenAccess databases. For information about the object mapping file, see Object Mapping Files.
Cell	Cell mapping file you browse to select. Maps cell and port names. For information about the cell mapping file, see Cell Mapping Files.

Feedback

Settings	Description
Font	Font mapping file you browse to select. Maps font IDs to their OpenAccess names. For information about the font mapping file, see Font Mapping Files (GDSII Stream Only).
Text	Text mapping file you want to use during the import process. For information, see GDSII Stream Text Mapping Files.
Property	Property mapping file you browse to select. Select the **Value Only** option to import only property values. The check box is selected by default. For information about the property mapping file, see Property Mapping Files (GDSII Stream Only).
Property Separator	Character you specify to use as a separator in the property mapping file. Default is ".".

4. Click **OK**.

   The translator creates multiple libraries for Stream files with large numbers of cells. This avoids performance problems with large Linux directories.

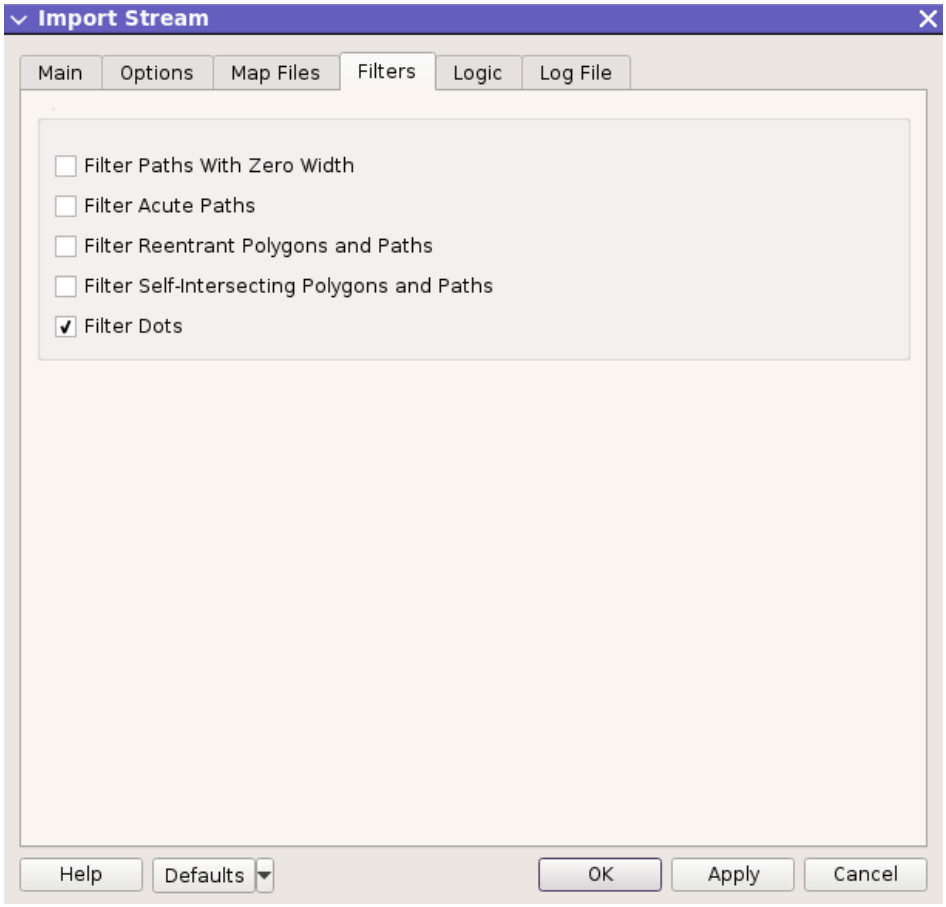
   For normal FileSys libraries, the translator limits each output library to contain a maximum of 20,000 cells, and then creates additional libraries for the remaining cells. Additional libraries have the same name with _n appended. For example, if the import is to the relLib library, then cells numbered 20,001 to 40,000 are created in a new library called relLib_1.

   In all cases where additional libraries are created on import, the new libraries are defined in the `lib.defs` file used by the import process. Any time the Stream file contains references to a cell that is placed in a different library, the instance in OpenAccess refers to the other library so that the design hierarchy remains intact.

5. Click the **Filters** tab (Figure 5).

Feedback

Figure 5 Import Stream Dialog Box: Filters Tab



On the **Filters** tab, enter settings as described in Table 14.

Table 14 Import Stream Settings (Filters Tab)

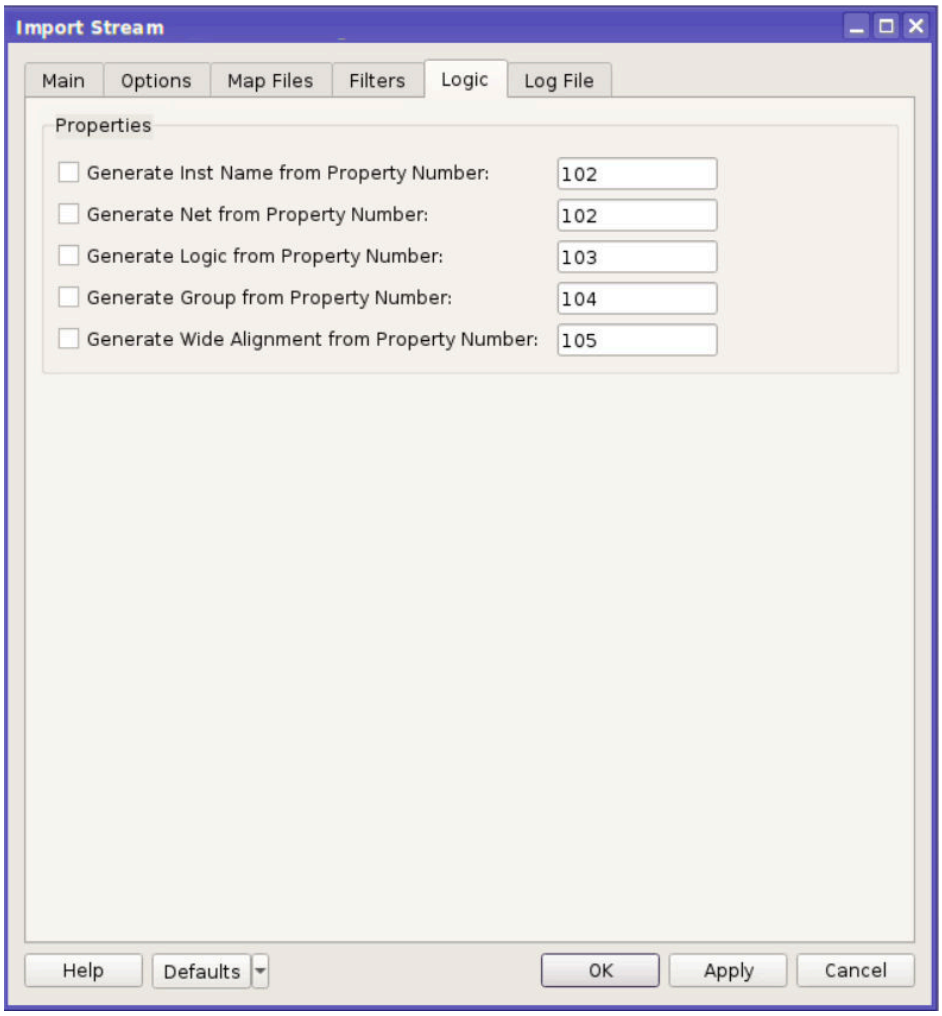
Settings	Description
Filters	**Filter Paths With Zero Width** specifies whether or not to ignore paths with zero width from the GDSII file. Tcl preference: `dbImportStreamFilterZeroWidth` **Filter Acute Path** specifies whether or not to ignore paths with acute angles from the GDSII file. Tcl preference: `dbImportStreamFilterAcutePaths` **Filter Reentrant Polygons and Paths** specifies whether or not to ignore reentrant polygons and paths from the GDSII file. Tcl preference: `dbImportStreamFilterReentrantPgns`.

Feedback

Settings	Description
	Filter Self-Intersecting Polygons and Paths specifies whether or not to ignore self-intersecting polygons and paths from the GDSII file. Tcl preference: `dbImportStreamFilterSelfIntersectingPgns` **Filter Dots** specifies whether or not to filter dots from the GDSII file, and is selected by default. Tcl preference: `dbImportStreamFilterDots`

6. Click the **Logic** tab (Figure 6).

Figure 6 Import Stream Dialog Box: Logic Tab



The property number fields are prepopulated and not editable if the selected GDSII file was created with a property number specified in the **Export Stream** dialog box. If

Feedback

the selected GDSII file was not created with a property number specified in the **Export Stream** dialog box, you can edit the property number for backward compatibility. On the **Logic** tab, enter settings as described in Table 15.

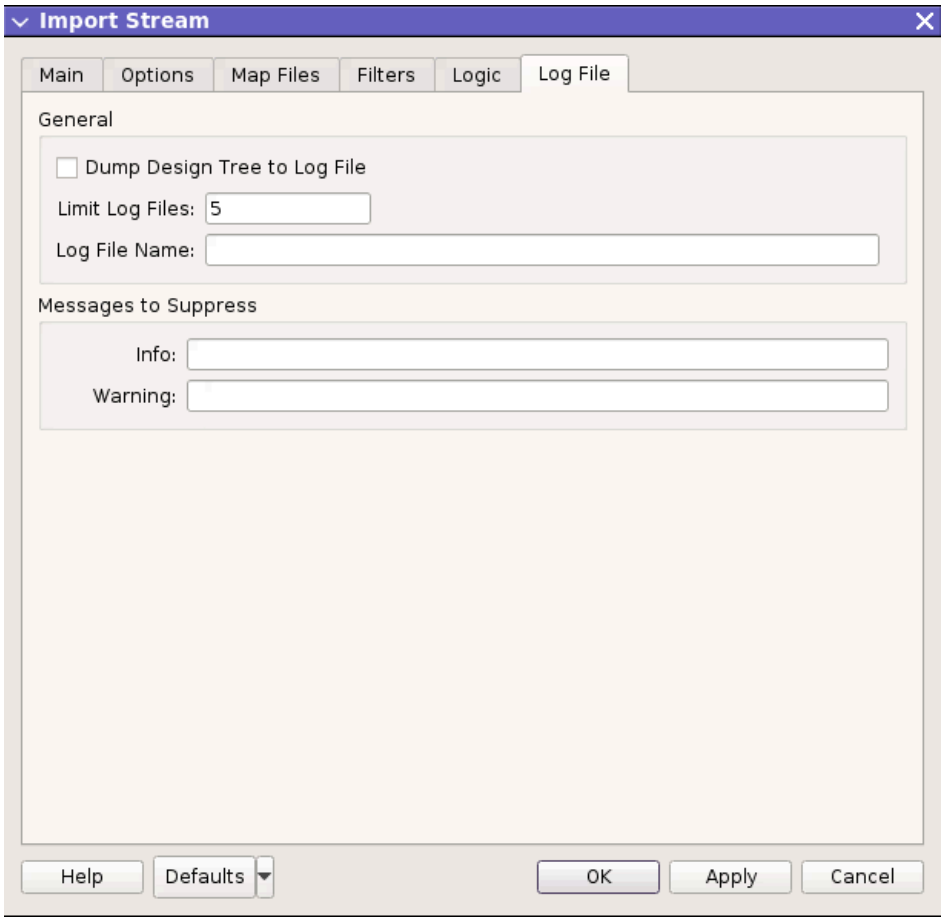
Table 15 Import Stream Settings (Logic Tab)

Settings	Description
Properties	**Generate Inst Name from Property Number** restores the instance name information from the GDSII file to the imported design using the specified GDSII property number. Specify the property number in the range of 1 to 127. The default is 102. Tcl preference: `dbImportStreamPropNumForInstName` **Generate Net from Property Number** restores the net name information from the GDSII file to the imported design using the specified GDSII property number. Specify the property number in the range of 1 to 127. The default is 102. Tcl preference: `dbImportStreamPropNumForNetName` **Generate Logic from Property Number** restores the logic information from the GDSII file to the imported design using the specified GDSII property number. Specify the property number in the range of 1 to 127. The default is 103. Tcl preference: `dbImportStreamPropNumForLogicName` **Generate Group from Property Number** restores the synchronous group and the figure group information (name, status and orientation) from the GDSII file to the imported design using a specified GDSII property number. Specify the property number in the range of 1 to 127. The default is 104. Tcl preference: `dbImportStreamPropNumForGroupName` **Generate Wide Alignment from Property Number** restores the logic information from the GDSII file to the imported design using the specified GDSII property number. Specify the property number in the range of 1 to 127. The default is 105. Tcl preference: `dbImportStreamKeepWideAlignment`

7. Click the **Log File** tab (Figure 7).

Feedback

Figure 7 Import Stream Dialog Box: Log File Tab



On the **Log File** tab, enter settings as described in Table 16.

Table 16 Import Stream Settings (Log File Tab)

Settings	Description
General	**Dump Design Tree to Log File**: Writes the design tree information to a log file during a Stream import. If the view name is not a layout, the option appends the view name after the cell name. Off by default. **Limit Log Files**: Limits the number of output log files. Default is 5. **Log File Name**: Specifies the name of the log file.

Feedback

Settings	Description
Messages to Suppress	**Info**: The list of informational messages to suppress during import.
	Warning: The list of warning messages to suppress during import.

The `onPostStreamExit` callback is triggered at the end of the **Import Stream** run.

The `importStream` command imports the design in GDSII Stream format.

Syntax

```
importStream -lib library -gds files [-ascii] [-DMAttributes attrList]
 [-DMSystem sysName] [-libDefFile file]
[-libPath path] [-shared] [-techDMAttributes attrList] [-techDMSystem
 sysName] [-techLib libName] [-techLibPath path]
[-techRefs libList] [-autoMapObjects propNum] [-cell name] [-cellMap
 file] [-compress] [-compressLevel zLevel]
-convertPathToPathSeg] [-convertTextBrackets angle|square][-dataModel
 version] [-dbuPerUU number] [-debugMsg]
[-detectVias] [-disableLibLocking] [-dumpDesignTree] [-duplicateCell
 skip|rename|overwrite] [-filterAcutePaths]
[-filterDots][-filterReentrantPgns] [-filterSelfIntersectingPgns]
 [-filterZeroWidth] [-fontMap file] [-hierDepth number]
[-ignoreBoxes] [-lakerLicense layout] [-layerMap fileList]
 [-layerMapColorMode enableColor|disableColor] [-layerMapLog]
[-layerMappingMode filter|map|off] [-logFile file] [-noInfo msgIds]
 [-noWarning msgIds] [-notCreateViaDefs]
[-notInternalizeShapes] [-numCellsInLibLimit limit] [-objectMap file]
 [-outputBBoxInfo] [-overwrite] [-propMap file]
[-propSeparator char] [-propertyInst number] [-propertyLogic number]
 [-propertyNet number] [-propertyWideAlignment number]
[-ramDiskChunkSize] [-ramDiskUtilityPath] [-refLibList file] [-refLibs
 libList] [-refOtherLibsFirst] [-refViews viewList]
[-resize scaleFactor] [-restorePcells] [-restoreVias]
 [-setColorShiftLayersToCells fileList useTech] [-si2ViaDetection]
[-snapToMfgGrid] [-templateFile file] [-textHeight height] [-textMap
 fileList] [-toLower] [-toLowerLabel] [-toUpper]
[-toUpperLabel] [-unmappedLayersUseStrmNums] [-useTechLayerMap]
 [-useTechObjectMap] [-viaMap fileList] [-view view]
[-h | -help] [-v] [-version]
```

For details of the `importStream` command, type `importStream -help`.

Feedback

Note:

When `-ascii` is specified, the following arguments are not used for importStream:

```
autoMapObjects, propMap, propSeparator, propertyInst,
 propertyLogic, propertyNet, propertyWideAlignment,
 restorePcells, restoreVias, viaMap
```

The format of `-ascii` when supporting Rectangle, Polygon, Path, Text objects:

```
#
#Sample: Stream ASCII dump file
#
#openLib("libname",DBUPerUU,"UserUnit");
# openCell("cellName1");
#  addRect(layer,dtype,x1,y1,x2,y2);
# closeCell("cellName1");
# openCell("cellName2");
#  addRect(layer,dtype,x1,y1,x2,y2);
#  addPoly(layer,dtype,nPoints)
#   (x1,y1) (x2,y2) (x3,y3) (x4,y4)
#   (x5,y5) (x6,y6) ......;
#  addPath(layer,dtype,width,style,nPoints,[bgnExt,endExt])
#   (x1,y1) (x2,y2) (x3,y3) (x4,y4)
#   (x5,y5) (x6,y6) ......;
#
 addText(layer,ttype,"textString",mag,x0,y0,orient,justify,fon
t);
#  addCellInst("masterName",x0,y0,angle,mirror,mag);
#
 addArrayInst("masterName",x0,y0,angle,mirror,mag,rows,columns,r
owSpacing,columnSpacing);
# closeCell("cellName2");
#closeLib("libName");
#
##Notes: ##
## orient = 1:R0 2:R90  3:R180 4:R270 5:MY 6:MYR90 7:MX 8:MXR90
```

Importing Design Data in OASIS Format

Import design data in OASIS format by defining settings and options for the translation process to use.

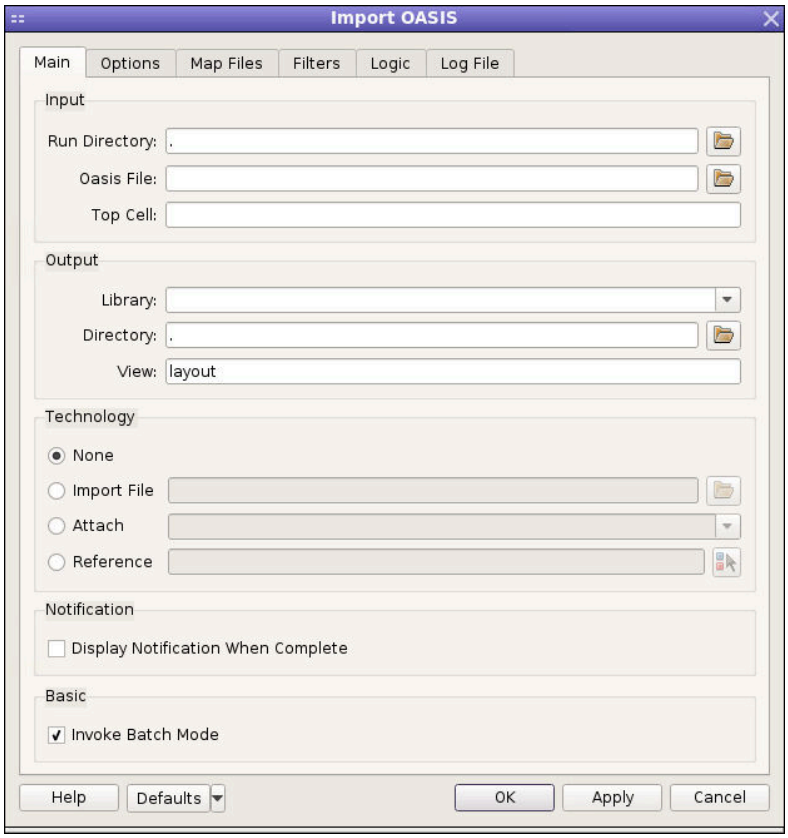
Perform the import task after preparing mapping files. See Supplying Mapping Files for GDSII Stream and OASIS Format Translations.

Feedback

To import design data in OASIS format:

1. From the **Home** or **Console** tab, choose **File > Import > OASIS** to open the **Import OASIS** dialog box.

Figure 8 Import OASIS Dialog Box: Main Tab

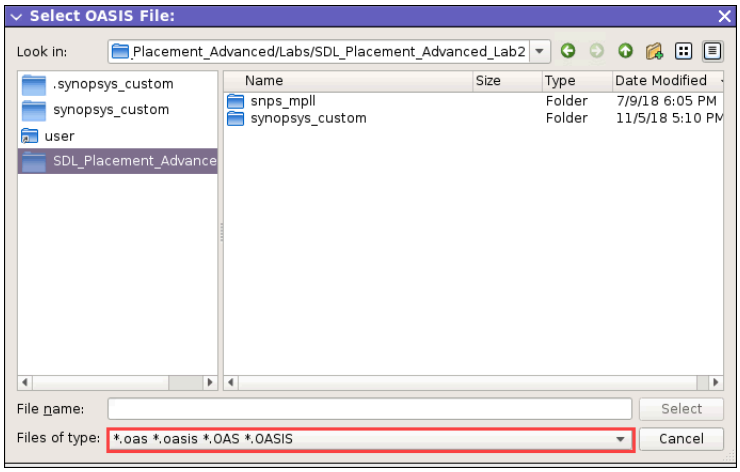


2. On the **Main** tab, enter settings as described in Table 17.

Table 17 Import OASIS Settings (Main Tab)

Settings	Description
Input	**Run Directory** Directory you browse to select. This directory must exist. Leaving this field blank uses the current working directory.

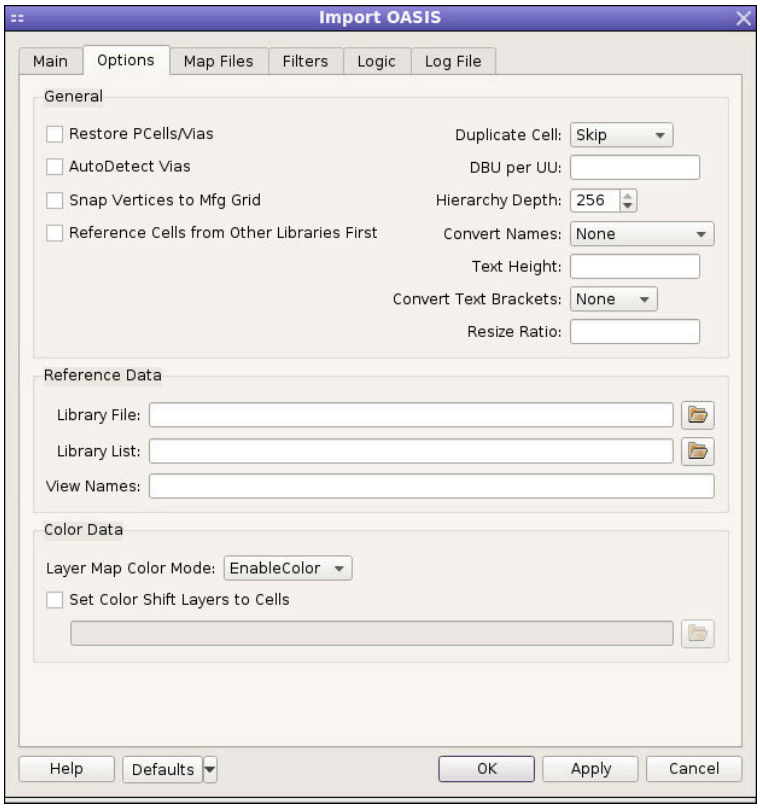
Feedback

Settings	Description
	OASIS File OASIS file you browse to select. Click the folder icon to open the **Select Existing File(s)** dialog box. To find OASIS files more quickly, select the ***.oas *.oasis *.OAS *.OASIS** filter in the **Files of type** box.  **Top Cell** Cell to create and use for the import process. If left blank, all cells in the OASIS file are translated. Specify a top cell in order to limit translation to the hierarchy under that cell, down to the depth specified by **Hierarchy Depth** (on the **Options** tab).
Output	**Library** Target library for the OASIS import; must be listed in the `lib.defs` file, or, if a new library is required, enter a new library name in the field. **View** Name of the layout view to create; the default name is "layout".
Technology	Choose **Import File**, **Attach**, or **Reference**. Default is **None**. You can change the selection of the technology group of options if the **Output Library** field is empty or a new library name is specified.
Notification	**Display Notification When Complete** Specifies whether to display a window with completion information after import. Off by default.
Basic	**Invoke Batch Mode** When selected (the default), runs importOasis integrated with the Custom Compiler tool. Uncheck this option to run as a standalone utility.

3. Click the **Options** tab in the **Import OASIS** dialog box (Figure 9).

Feedback

Figure 9 Import OASIS Dialog Box: Options Tab



4. On the **Options** tab, enter settings as described in Table 18.

Table 18 Import OASIS Settings (Options Tab)

Settings	Description
General	**Restore PCells/Vias**: Restores PCells, abutment information, and vias during import. Off by default. The restored information is from the source database and not affected by filtering of layer mapping file or color locks mode.

Feedback

Settings	Description
	AutoDetect Vias: Creates StdVias rather than instances for certain cells in the OASIS files. Off by default. It requires either that you have an existing technology file defining the layer material types, or that the layer map file used during import has the material types specified for the layers. With that information it creates StdVias where a design follows the form that can be generated as a StdVia. Searches for a StdViaDef that matches the via to be created. If a matching StdViaDef is not found, and if the technology file is writable, then a new StdViaDef is created. If the technology file is not writable, then the cell is written as a regular cell and referenced as an oaInst. **Snap Vertices to Mfg Grid**: Snaps objects vertexes to manufacturing grid during import. Off by default. **Reference Cells from Other Libraries First**: References cells from other libraries first if the same cell body name exists in both the input OASIS file and other libraries during the import process. Off by default. **Duplicate Cell**: Allows you to **Skip** (default), **Rename**, or **Overwrite** duplicate cells. In non-batch mode (when **Invoke Batch Mode** is not selected on the **Main** tab), the **Query** option becomes available. If **Query** is selected, during duplicate cell handling the **Action for Duplicate Cells** dialog box appears, prompting you to select how to handle the duplicated cell: **Overwrite**, **Rename**, or **Skip**. The dialog box also provides the option to treat all duplicate cells in the same way: **Overwrite All**, **Rename All**, or **Skip All**. **DBU Per UU**: The value for database units per user units. This field is activated only when you are creating a new library and that new library has its own technology (that is, is not attaching to another library's technology). **Hierarchy Depth**; The depth of hierarchy to which to translate; default is 20 levels. This option only applies if a **Top Cell** is specified. **Convert Names**: Specifies translator conversion of cell and instance name text to one of the following: ◦ **None**: (default) Do not convert names ◦ **Lower**: Lowercase ◦ **Upper**: Uppercase

Feedback

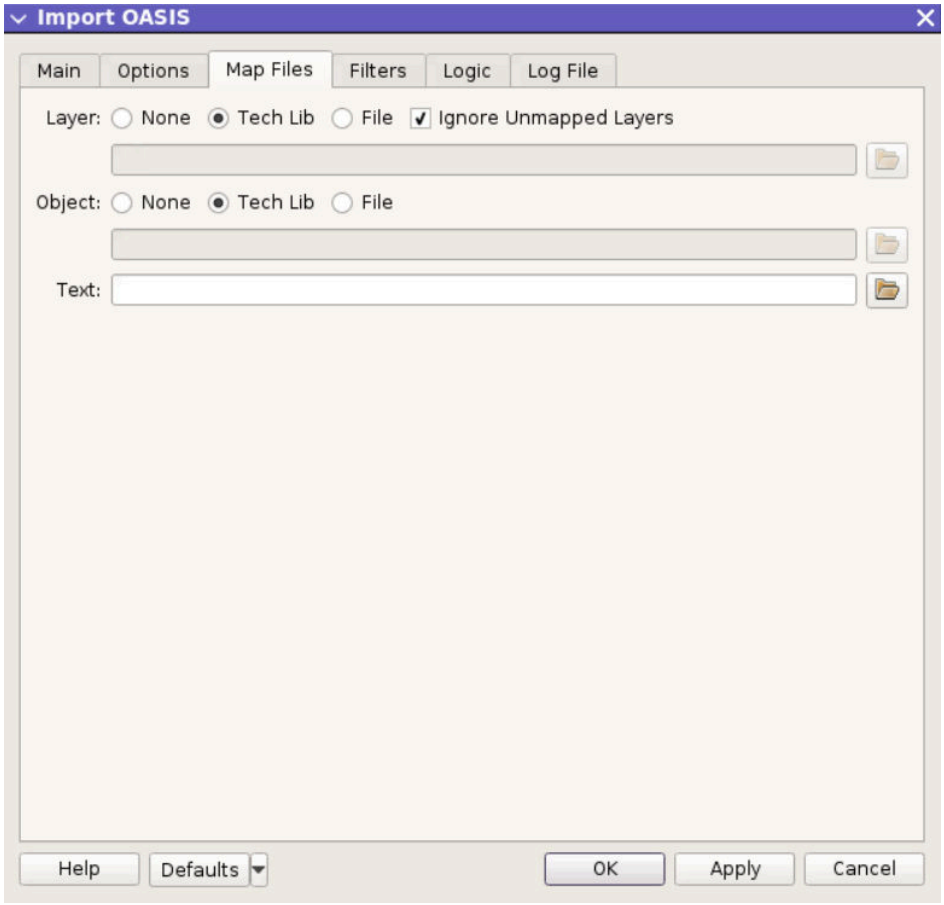
Settings	Description
	Text Height: The height to assign to imported text, in user units. **Convert Text Brackets**: Specifies whether or how to convert text brackets during import: ◦ **None**: (default) Do not convert text brackets. ◦ **Angle**: Convert square text brackets, "[" and "]", to angle text brackets, "<" and ">". ◦ **Square**: Convert angle text brackets, "<" and ">", to square text brackets, "[" and "]". **Resize Ratio**: Specifies whether the translator resizes cells during import. The specified value is limited from 0.000001 to 100000. This option cannot be used with **Auto DetectVias** or **Restore PCells/Vias**. Note: Import OASIS converts circles to ellipses.
Reference Data	**Library File**: The file you browse to select. This file has the names of reference libraries for the translation process to use when it binds cells referenced during import. **Library List** The list of libraries with cells referenced in the design. Click the **Pick Object** icon to open the **Import Oasis Reference Library Browser**.
	◦ Sort from the libraries **Available** to the libraries **Selected** using the arrow buttons between the two areas.
	◦ Where there are many libraries in the selected list, you can use the arrow buttons at the right edge to scroll.
	View Names: The views of reference libraries to be used to bind cells referenced during import.
Color Data	**Layer Map Color Mode**: Specifies the layer mapping mode for the color attribute. Options are **EnableColor** (the default) and **DisableColor**. Example: M1 drawing 3 0 M1 drawing 3 11 color:red locked M1 drawing 3 12 color:red unlocked When the **EnableColor** option is set, shapes on 3:11 are mapped to M1 drawing with color:red and unlocked.

Feedback

Settings	Description
	Set Color Shift Layers to Cells: When selected, browse to select the color shifting cell rename mapping file. Specifies if the tool sets the metal or cut color shiftable layers to the cells from the specified color shift mapping file. **Note:** If you do not specify a mapping file the tool sets all color shiftable metal or cut color shiftable layers to the cells

5. Click the **Map Files** tab in the **Import OASIS** dialog box (Figure 10).

Figure 10 Import OASIS Dialog Box: Map Files Tab



6. On the **Map Files** tab, enter settings as described in Table 19.

Feedback

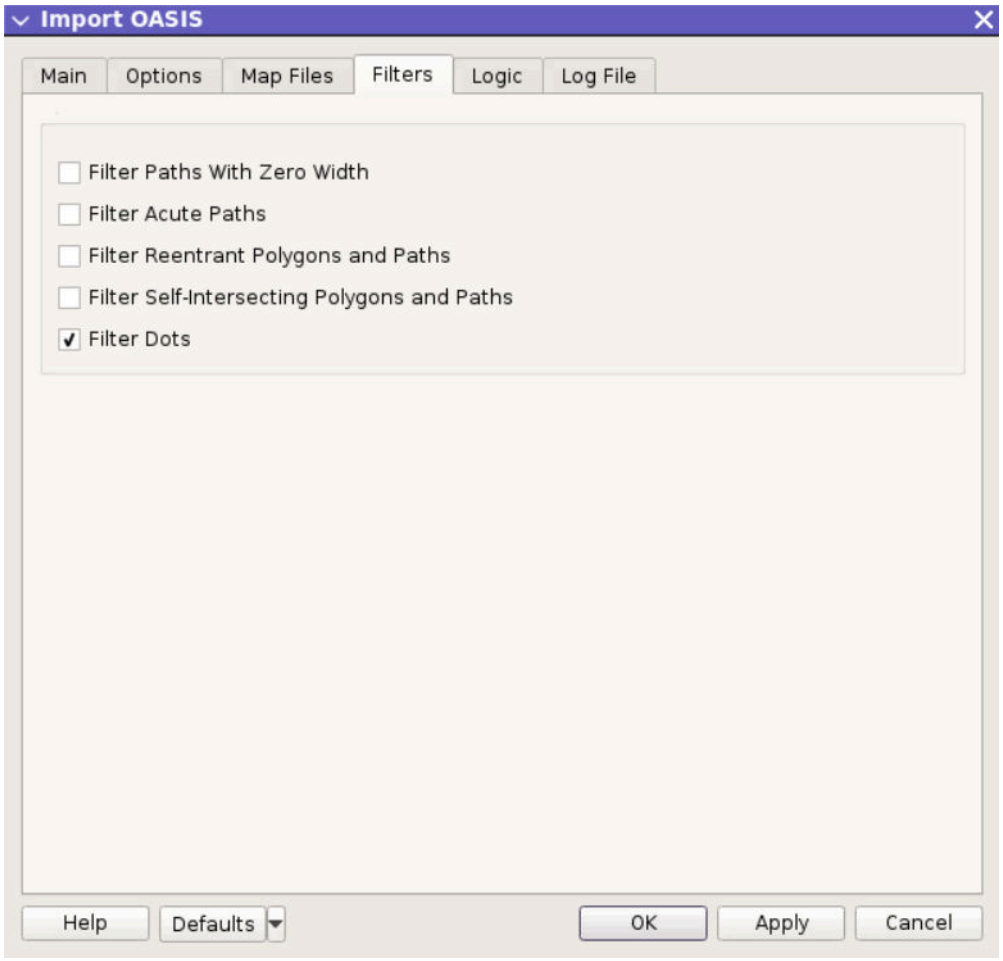
Table 19 Import OASIS Settings (Map Files Tab)

Settings	Description
Layer	Select the required layer mapping mode: ◦ **None**: Do not use a layer mapping file for layer/purpose mapping. Instead, use the OASIS layer number and datatype for the OpenAccess layer number and purpose number. ◦ **Tech Lib**: (default) Use the layer mapping files in the technology library, merging them if more than one exists. If there are duplicates, map the shapes in OASIS to the first matched OpenAccess layer name and purpose name. ◦ **File**: Use the layer mapping file you browse to select. The layer map file tells the translator how to map OASIS layer numbers and datatypes to OpenAccess layers and purposes. For layers that support mask colors it also determines the mask color of the shape. If **Ignore Unmapped Layers** is not selected and the GDSII layer number and data type is not matched in the layer mapping file, use the GDSII layer number and data type for OA layer number and purpose number. If particular layer names and purpose names for OpenAccess objects are used in the layer mapping file (see Table 10), then import as an OpenAccess object. If **Ignore Unmapped Layers** is selected (default), the GDSII object is filtered out. For information about the layer mapping file, see Layer Mapping Files.
Object	Select the required object mapping mode: ◦ **None**: Do not use an object mapping file. ◦ **Tech Lib**: (default) Use the object mapping files in the technology library, merging them if more than one exists. If there are duplicates, map the shapes in OASIS to the first matched OpenAccess object. ◦ **File**: Use the object mapping file you browse to select. Specifies methods of mapping Boundaries and `LayerBlockages` between OASIS shapes and OpenAccess databases. For information about the object mapping file, see Object Mapping Files.
Text	Specify the text mapping file you want to use during the import process. For information, see OASIS Text Mapping Files.

7. Click the **Filters** tab in the **Import OASIS** dialog box (Figure 11).

Feedback

Figure 11 Import OASIS Dialog Box: Filters Tab



8. On the **Filters** tab, enter settings as described in Table 20.

Table 20 Import OASIS Settings (Filters Tab)

Settings	Description
Filters	**Filter Paths With Zero Width**: Specifies whether or not to ignore paths with zero width from the OASIS file. Tcl preference: `dbImportOasisFilterZeroWidth` **Filter Acute Path**: Specifies whether or not to ignore paths with acute angles from the OASIS file. Tcl preference: `dbImportOasisFilterAcutePaths`

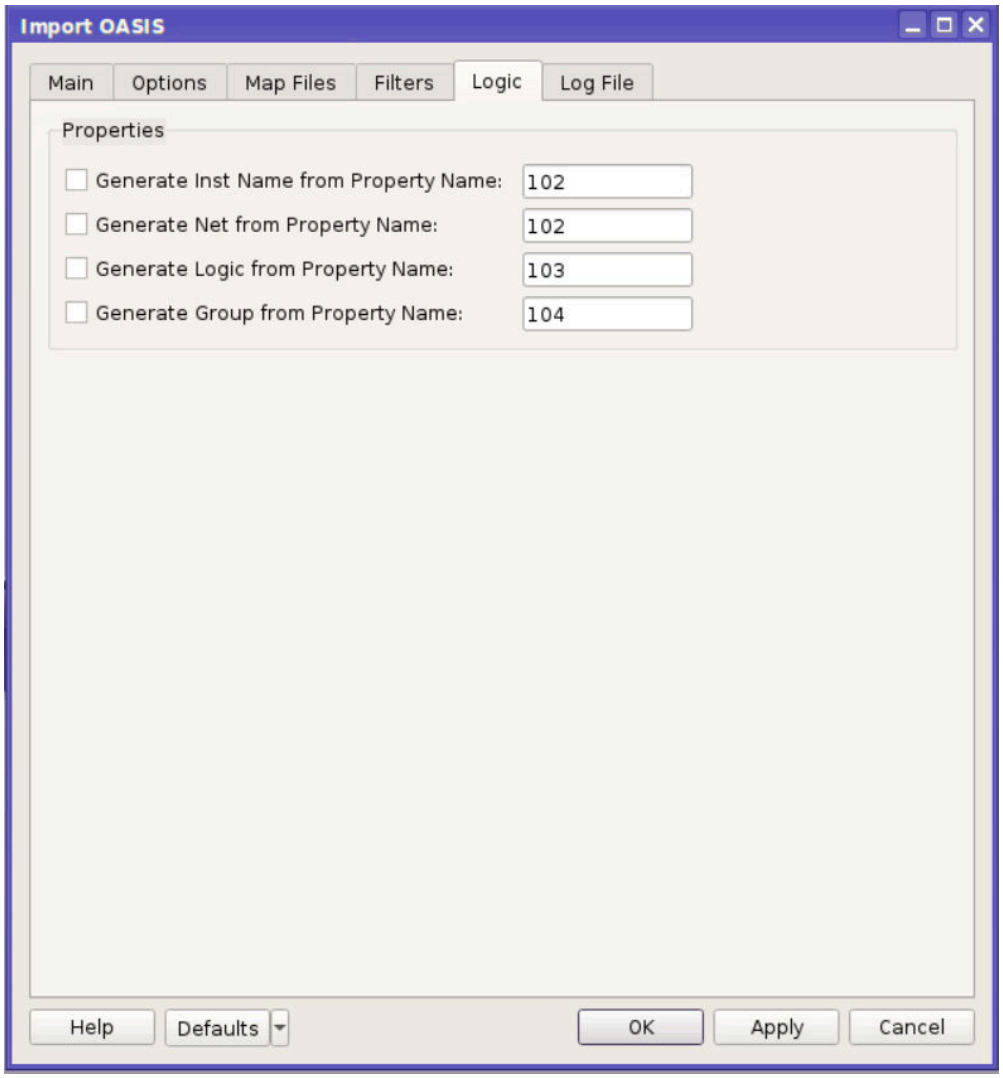
Feedback

Settings	Description
	Filter Reentrant Polygons and Paths: Specifies whether or not to ignore reentrant polygons and paths from the OASIS file. Tcl preference: `dbImportOasisFilterReentrantPgns`. **Filter Self-Intersecting Polygons and Paths**: Specifies whether or not to ignore self-intersecting polygons and paths from the OASIS file. Tcl preference: `dbImportOasisFilterSelfIntersectingPgns` **Filter Dots**: Specifies whether or not to filter dots from the OASIS file, and is selected by default. Tcl preference: `dbImportOasisFilterDots`

9. Click the **Logic** tab in the **Import OASIS** dialog box (Figure 12).

Feedback

Figure 12 Import OASIS Dialog Box: Logic Tab



10. On the **Logic** tab, enter settings as described in Table 21.

Table 21 Import OASIS Settings (Logic Tab)

Settings	Description
Properties	**Generate Inst Name from Property Name**: Restores the instance name from the OASIS file to the imported design using the specified OASIS property name. OASIS property names can be numbers or strings. The default is 102. Tcl preference: `dbImportOasisPropNameForInstName`

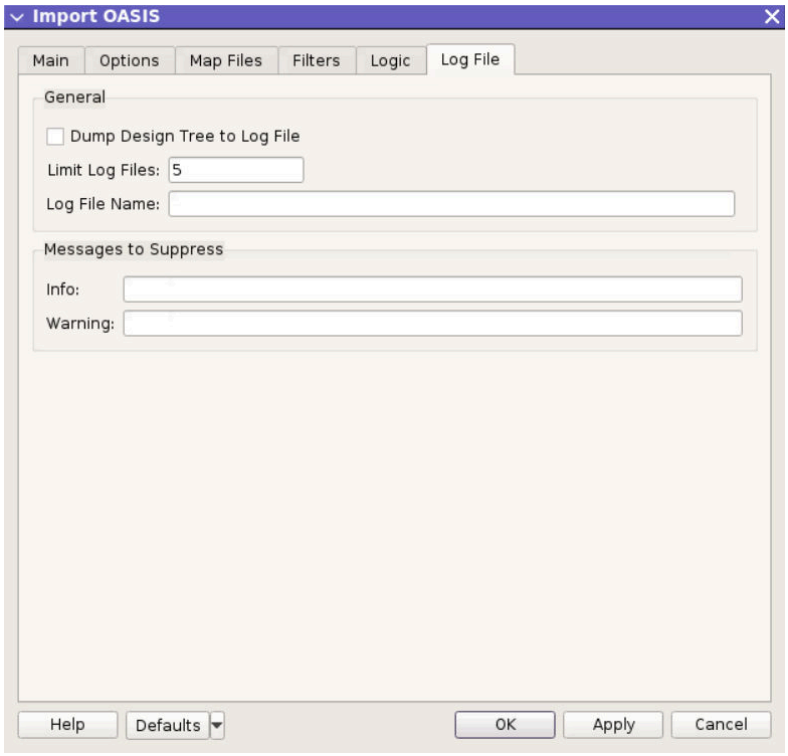
Feedback

Settings	Description
	Generate Net from Property Name: Restores the net name of shapes from the OASIS file to the imported design using the specified OASIS property name. OASIS property names can be numbers or strings. The default is 102. Tcl preference: `dbImportOasisPropNameForNetName` **Generate Logic from Property Name**: Restores the logic information of objects from the OASIS file to the imported design using the specified OASIS property name. For shapes, restores ◦ net name ◦ terminal name and type ◦ name, PlacementStatus, and AccessDir of pins For instances, restores ◦ instance name ◦ terminal name ◦ terminal net name OASIS property names can be numbers or strings. The default is 103. Tcl preference: `dbImportOasisPropNameForLogicName` **Generate Group from Property Name**: Restores the synchronous group and the figure group information (name, status and orientation) from the OASIS file to the imported design using a specified OASIS property number. Specify the property number in the range of 1 to 127. The default is 104. Tcl preference: `dbImportOasisPropNameForGroupName`

11. Click the **Log File** tab in the **Import OASIS** dialog box (Figure 13).

Feedback

Figure 13 Import OASIS Dialog Box: Log File Tab



12. On the **Log File** tab, enter settings as described in Table 22.

Table 22 Import OASIS Settings (Log File Tab)

Settings	Description
General	**Dump Design Tree to Log File**: Writes the design tree information to a log file during OASIS import. If the view name is not a layout, the option appends the view name after the cell name. Off by default. **Limit Log Files**: Limits the number of output log files. Default is 5. **Log File Name**: Specifies the name of the log file.
Messages to Suppress	**Info**: The list of informational messages to suppress during import. **Warning**: The list of warning messages to suppress during import.

13. Click **OK**.
14. The translator imports the design in OASIS format.

Feedback

Note:

When the OASIS file contains magnification factor, Import Oasis keeps the origin of the cell master, creates a new cell with magnification, and uses new cell for master of instance. The origin cell becomes as orphan cell. The new cell name is the origin name append with magnification, such as *cell1_0.25* (magnification is 0.25).

The `onPostStreamExit` callback is triggered at the end of the **Import OASIS** run.

The `importOasis` command imports the design in OASIS format.

Syntax

```
importOasis -lib arg -oasis arg [-libDefFile arg] [-libPath arg]
 [-techLib arg] [-techRefs arg] [-cell arg] [-compress] [-compressLevel
 arg (=1)] [-dbuPerUU arg (=0)] [-debugMsg] [-duplicateCellQuery]
 [-restorePcells] [-restoreVias] [-detectVias] [-notCreateViaDefs]
 [-internalizeShapes] [-hierDepth arg (=256)] [-useTechLayerMap]
 [-layerMap arg] [-convertTextBrackets arg] [-logFile arg] [-noInfo
 arg] [-noWarning arg] [-useTechObjectMap] [-objectMap arg] [-overwrite]
 [-duplicateCell arg (=5)] [-dumpDesignTree] [-propertyInst arg]
 [-propertyNet arg] [-propertyLogic arg] [-refLibList arg] [-refLibs arg]
 [-refViews arg] [-refOtherLibsFirst] [-textHeight arg (=0)] [-textMap
 arg] [-view arg (=layout)] [-resize arg (=1)] [-toUpper] [-toLower]
 [-snapToMfgGrid] [-unmappedLayersUseOasisNums] [-layerMapColorMode
 arg (=1)] [-setColorShiftLayersToCells arg] [-ramDiskChunkSize arg]
 [-ramDiskUtilityPath arg] [-filterZeroWidth] [-filterAcutePaths]
 [-filterReentrantPgns] [-filterSelfIntersectingPgns] [-filterDots]
 [-disableLibLocking] [-version] [-help]
```

For details of the `importOasis` command, type `importOasis -help`.

See Also

- Layer Mapping Files

Importing Design Data in Library Exchange Format

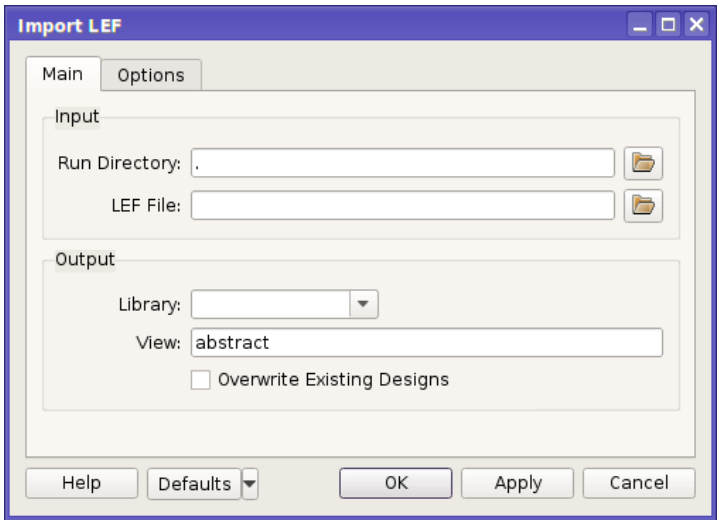
LEF format exchanges the library description, cells and their descriptions, and cell port information, excluding layout geometry information.

Feedback

To import design data in LEF format:

1. From the **Home** or **Console** tab, choose **File > Import > LEF**. The **Import LEF** dialog box opens.

Figure 14 Import LEF Dialog Box: Main Tab



2. On the **Main** tab, enter the settings as described in Table 23.

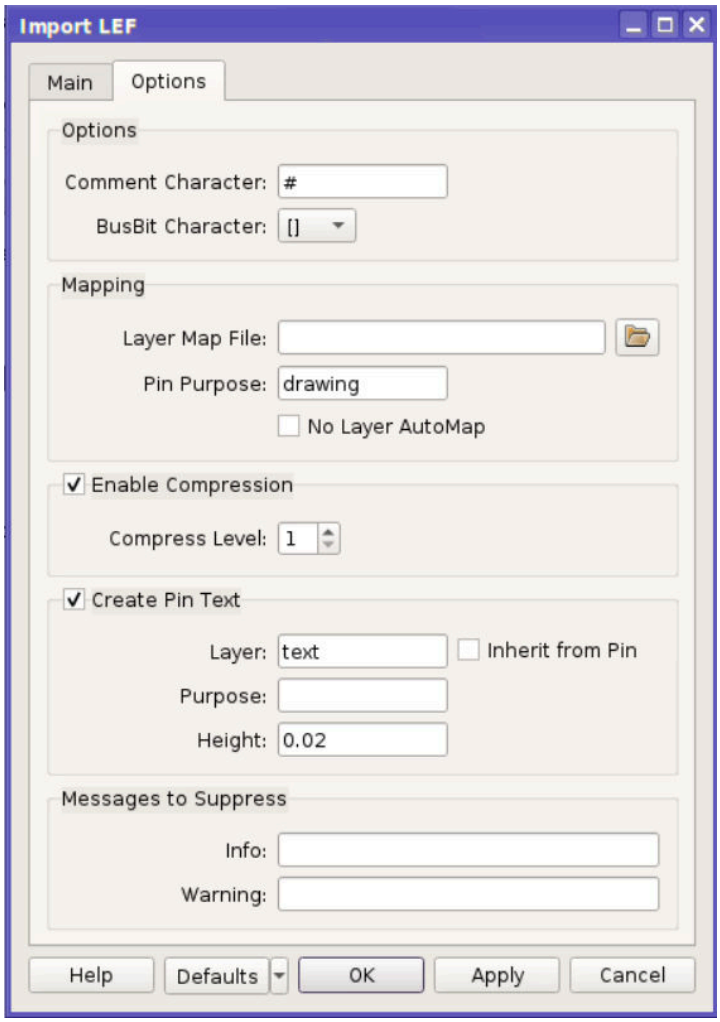
Table 23 Import LEF Settings (Main Tab)

Settings	Description
Input	**Run Directory** is the directory you browse to select that contains the LEF file. The default is the current directory. **LEF File** is the LEF file you browse to select (typical file name extension is `.lef`).
Output	**Library** specifies the destination library for the LEF import. **View** is the name of the view to create. The default is an abstract view. **Overwrite Existing Designs** replaces existing designs with the translated data. Off by default.

3. Click the **Options** tab (Figure 15).

Feedback

Figure 15 Import LEF Dialog Box: Options Tab



4. On the **Options** tab, enter settings as described in Table 24.

Table 24 Import LEF Settings (Options Tab)

Settings	Description
General	**Comment Character** is the character that introduces a comment line. The default is the pound sign (#). **BusBit Characters** lets you use either angle brackets <> or square brackets [] to define a bus.

Feedback

Settings	Description
Mapping	**Layer Map File** specifies the layer map file you wish to select. This is an ASCII format file that contains lines formatted with the following notations: <LEF Layer Name> <OA Layer#>. **Pin Purpose** inherits the purpose of the pin layer from the LEF file. Select **No Layer AutoMap** to only import shapes on layers that are in the layer map file. This file can also contain comments starting with the pound sign (#). Table 25 shows an example.
Enable Compression	Specifies if the translator should allow libraries to be compressed. Default is True. Tcl preference: `dbImportLefEnableCompression` **Compress Level** specifies compress level during LEF importing. Default is 1. Tcl preference: `dbImportLefCompressLevel`
Create Pin Text	A toggle that allows you to create pin text during import. On by default. **Layer** sets the specified layer for a label. The default is the text layer. **Purpose** inherits and changes the purpose of the pin layer from the label layer. **Inherit from Pin** inherits the label layer from the pin layer. **Height** sets the text height in user units.
Messages to Suppress	**Info** is the list of informational messages to suppress during import. **Warning** is the list of warning messages to suppress during import.

5. Click **OK**.

   The translator imports the design in LEF format.

The `importLef` command is also supported to import the design in LEF format.

Syntax

```
importLef -lib library -lef file [-DMAttributes attrList] [-DMSystem
 sysName] [-libDefFile file] [-libPath path] [-shared] [-techDMAttributes
 attrList] [-techDMSystem sysName] [-techLib libName] [-techLibPath
 path] [-techRefs libList] [-commentChar char (=#)] [-busBitChars
 chars (=[])] [-compress] [-compressLevel zLevel (=1)] [-dataModel
 version (=4)] [-defaultRuleName name (=LEFDefaultRouteSpec)] [-layerMap
 file][-layoutView viewName (=layout)] [-logFile file] [-noInfo msgIds]
```

Feedback

```
[-noWarning msgIds] [-noLayerAutoMap] [-overwrite] [-pinLabels]
[-templateFile file] [-textHeight height (=999)] [-textLayer name
(=text)] [-view viewName (=layout)] [-integratorLicense] [-help]
[-version]
```

For details of the `importLef` command, type `importLef -help`.

LEF Import Example

The following table illustrates how comment lines appear in layer map files.

Table 25 Library Exchange Format Import Example

#layer name	Layer number
Metal1	1
Via12	2
Metal2	3
Via23	4
Metal3	5

Importing Design Data in Design Exchange Format

To import design data in DEF format:

1. If necessary, import design data in LEF format. (See Importing Design Data in Library Exchange Format).
2. From the **Home** or **Console** tab, choose **File > Import > DEF**. The **Import DEF** dialog box opens.

Feedback

Figure 16 Import DEF Dialog Box: Main Tab

3. On the **Main** tab, enter the settings as described in Table 26.

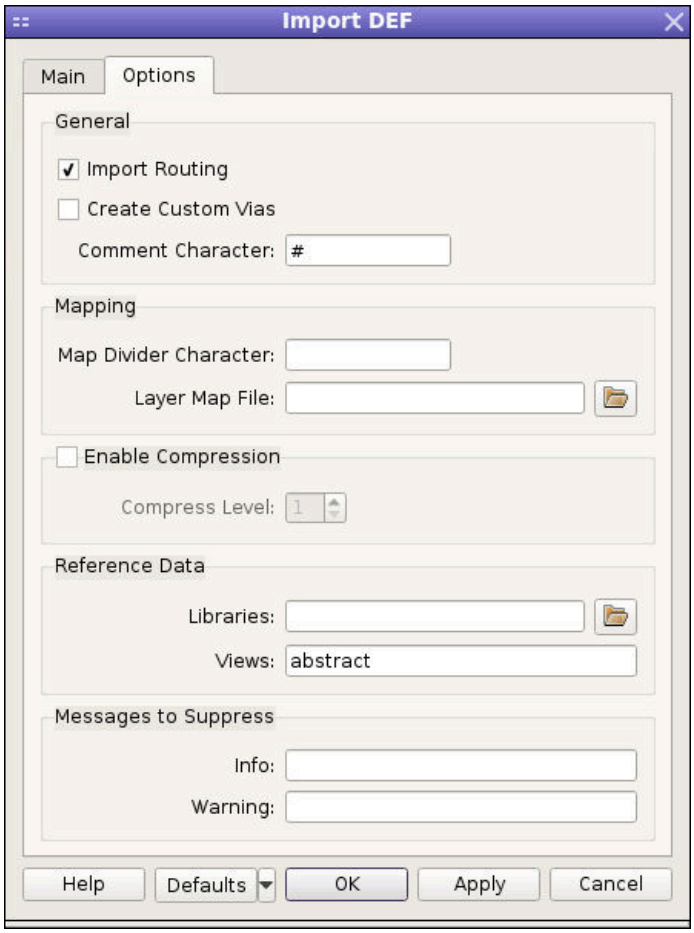
Table 26 Import DEF (Main Tab)

Settings	Description
Input	**Run Directory** is the directory you browse to select that contains the DEF file. The default is the current directory. **DEF File** is the DEF file you browse to select (typical file name extension is `.def`).
Output	**Library** specifies the destination library for the DEF import. **Cell** specifies the cell for the DEF import. **View** is the name of the view to create. The default is a layout view. **Overwrite Existing Designs** replaces existing designs with the translated data. On by default.

Feedback

4. Click the **Options** tab (Figure 17).

Figure 17 Import DEF Dialog Box: Options Tab



5. On the **Options** tab, enter settings as described in Table 27.

Table 27 Import DEF Settings (Options Tab)

Settings	Description
General	**Import Routing** allows the translator to import any routing data during translation, including the vias section. On by default. **Create Custom Vias** allows the translator to create and use oaCustomViaDefs instead of existing oaStdViaDefs for all vias in the DEF file. Off by default. **Comment Character** is the character that introduces the comment line. The default is the pound sign (#).

Feedback

Settings	Description
Mapping	**Map Divider Character** specifies a single character to be replaced in the instance and net names by the DEF DIVIDERCHAR. Preference: `dbImportDefMapDividerChar` **Layer Map File** specifies the layer map file. This is an ASCII format file that contains lines formatted with the following notations: <LEF Layer Name> <OA Layer#>. This file can also contain comments starting with the pound sign (#). Table 28 shows an example.
Reference Data	**Libraries** are libraries with cells referenced in the design.
Enable Compression	Specifies if the translator should allow libraries to be compressed. Default is False. Tcl preference: `dbImportDefEnableCompression` **Compress Level** specifies compress level during DEF importing. Default is 1. Tcl preference: `dbImportDefCompressLevel` **Views** are views of reference libraries to be used to bind cells referenced during import. The default is an abstract view.
Messages to Suppress	**Info** is the list of informational messages to suppress during import. **Warning** is the list of warning messages to suppress during import.

6. Click **OK**.

   The translator imports the design in DEF format.

The `importDef` command is also supported to import the design in DEF format.

Syntax

```
importDef -lib library -def file [-DMAttributes attrList] [-DMSystem
 sysName] [-libDefFile file] [-libPath path] [-shared] [-cell cellName]
 [-commentChar char (=#)] [-mapDividerChar mapChar] [-compress]
 [-compressLevel zLevel (=1)] [-dataModel version (=4)] [-layerMap file]
 [-logFile file] [-noInfo msgIds] [-noModHier] [-noRouting] [-noWarning
 msgIds] [-overwrite] [-refLibs libList] [-refViews viewList] [-techLib
 libName] [-techRefs libList] [-noLayerAutoMap] [-templateFile file]
 [-view viewName (=layout)] [-useCustomVias] [-help] [-version]
```

For details of the `importDef` command, type `importDef -help`.

Feedback

Import DEF Example

The following table illustrates how comment lines appear in layer map files.

Table 28 Design Exchange Format Import Example

#layer name	Layer number
Metal1	1
Via12	2
Metal2	3
Via23	4
Metal3	5

Note:

If the import design data in DEF format is written by the IC Compiler tool and the data contains metal fill shapes with `DataType=0`, the Custom Compiler tool automatically generates a new purpose fill for these shapes to distinguish from other shapes.

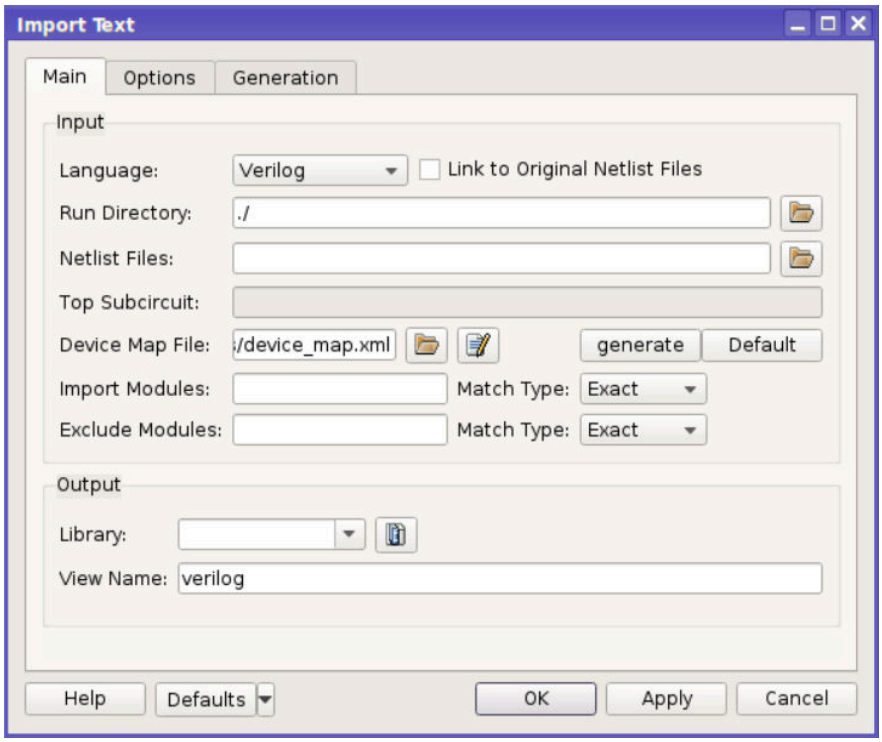
Importing Design Netlist Data as Text Views

Import design netlist data when you want to use text views for simulator inputs and other tasks. Available text netlist formats include SPICE, CDL, Verilog, Verilog-A, Verilog-AMS, SystemVerilog, and VHDL. You can optionally generate symbol views while importing netlist text. You can additionally generate schematic views (for all but VHDL text imports).

Feedback

To import design data as text views:

1. From the **Home** or **Console** tab, choose **File > Import > Netlist Text** to open the **Import Text** dialog box.



2. On the **Main** tab, enter settings as described in Table 29.

Table 29 Import Text Settings (Main tab)

Settings	Description
Input	**Language** is the behavioral modeling language you specify from the list: **SPICE**, **CDL**, **Verilog**, **Verilog-A**, **Verilog-Ams**, **SystemVerilog**, or **VHDL**. **Note:** ◦ You can use the PVA parser to import Verilog-A netlist files as text views. Set the `dbImportTextVerilogAParser` Tcl preference before you set the options in the **Import Text** dialog box to use the PVA parser. ◦ The following Information message appears in the console when you use the PVA parser `Information: Importing Verilog-A netlist text into OpenAccess database by PVA parser(NETLIST_PARSER-094)`

Feedback

Settings	Description

	Link to Original Netlist Files creates links to an external netlist view of the original files, rather than copying them.
	
	Run Directory specifies the directory for this import process to use. Browse to select the correct location.
	
	Netlist Files specifies the source netlist for this import process. Browse to select the file. Multiple files can also be selected.
	The *.gz file is also supported for Verilog, Verilog-AMS, or SystemVerilog formats.
	You can select a list of Verilog or VHDL files together by specifying the list of files in another file list. The content of the file lists must be one of the following:
	◦ Full paths to the individual files that need to be imported. For example:
	`<more sampleFullPath.vf>`
	`/remote/scratch/work/verFiles/dlatch_reset.v`
	`/remote/scratch/work/verFiles/up_counter.v`
	◦ Relative paths to the individual files with regard to the working directory where the Custom Compiler tool was invoked. For example:
	`<more sampleRelPath.vf>`
	`../../verFiles/dlatch_reset.v`
	`../../verFiles/up_counter.v`
	When **Language** is **Verilog**, `.vf` is the extension supported for the file list.
	When **Language** is **VHDL**, `.vhf` is the extension supported for the file list.
	
	Top Subcircuit works for CDL and SPICE formats only. Enter the name of a cell where top-level SPICE/CDL design data should be put. A name based off the imported file name is generated by default.
	
	Device Map File works for SPICE, CDL, and Verilog formats only.
	You can click any of the following buttons:
	◦ **Device Mapping File for Resolving Instance Masters** to designate the required device mapping source (to resolve instance masters to reference libraries) from a file.
	◦ **generate** to create the device map file from a reference library. See Generating Device Mapping File.
	Click **Edit Mapping File** to open the selected mapping file in a text editor for adding or modifying the mapping information.
	Click **Default** to load the sample provided with the installation hierarchy. This file can also be copied and edited to address any missing information. The default mapping file maps all devices to analogLib library cells.

Feedback

Settings	Description
	Import Modules lets you list module names or specify the name pattern for the modules you wish to import. **Match Type** can be **Exact**, **Reg Expr**, or **Wildcard**. **Exclude Modules** lets you list module names or specify the name pattern for the modules you wish to exclude from the import when files with multiple cells are imported. **Match Type** can be **Exact**, **Reg Expr**, or **Wildcard**. **Store Views by File Extension** displays when the **Language** selected is **Verilog**, **Verilog-Ams**, or **SystemVerilog**. This option is hidden for the remaining languages. When you select the **Store Views by File Extension** option, the **View Name** option is disabled. You can specify Verilog, Verilog-AMS, or System-Verilog netlist files to import at the same time in the **Netlist Files** box. The tool recognizes file format by file extension defined with the following preferences: ◦ `dbImportTextVerilogFileExtensions`: “v vp” ◦ `dbImportTextVerilogAMSFileExtensions`: “vams” ◦ `dbImportTextSystemVerilogFileExtensions`: “sv” The imported netlist with default view name is specified with the following preferences: ◦ `dbImportTextVerilogViewName`: verilog ◦ `dbImportTextVerilogAMSViewName`: verilogams ◦ `dbImportTextSystemVerilogViewName`: systemverilog
Output	**Library** specifies the destination library for this import operation, which you choose from the list. Click **Create Library** to open the **New Library** dialog box and create a new output library. **View Name** shows the default view name for this format. You can enter a different view name, if you prefer. The system applies the view name you enter here.

3. On the **Options** tab, enter settings as described in Table 30.

Feedback

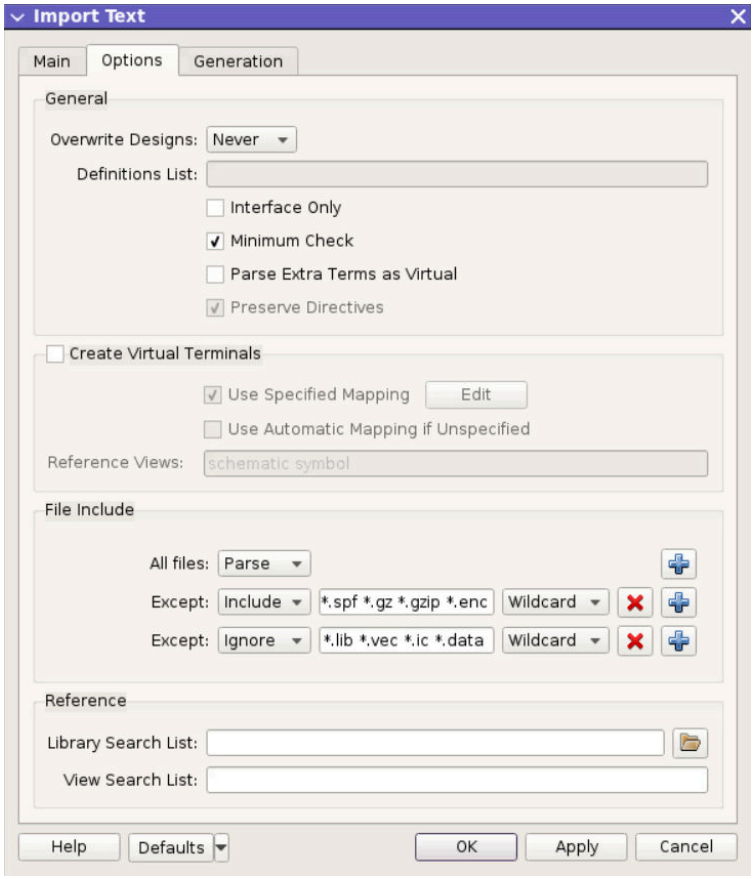


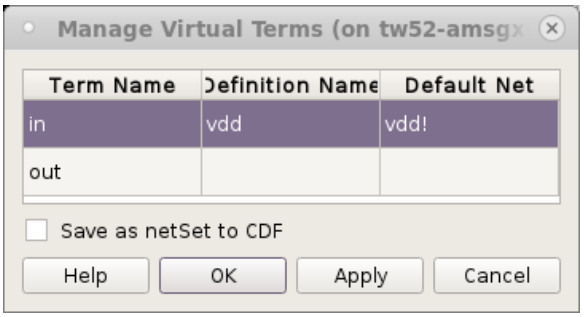
Table 30 Import Text Settings (Options Tab)

Settings	Description
General	**Overwrite Designs** specifies how the import process treats imported cellViews that conflict with existing cellViews. **Never** (the default) means that the existing cellView remains intact (no copy is created). **Always** means that existing cellviews are overwritten. (Verilog only) **Definitions List** specifies the list of define statements to be predeclared in each cellView. **Note:** The **Minimum Check** option is not available when you use the PVA parser **Interface Only** extracts only the interfaces (ports and parameters) of the imported modules.

Feedback

Settings	Description
	Minimum Check performs checking that ensures creation of shadow OA (a synchronized OpenAccess netlist design database) despite any errors occurring in the module body. Errors in the module definition or port list always result in a failure to create shadow OA. **Parse Extra Terms as Virtual** ensures that, when extracting internal representation for imported text views, terminals that are missed in the master design are considered virtual terminals and corresponding connections to virtual terminals are made. (Verilog only) **Preserve Directives** preserves Verilog compiler directives in the imported text views.
File Include	Specifies whether include files should be parsed, included, or ignored. The options are: ◦ **Parse**: Parses the include files and creates the cells for modules found in the include files. Include statements are not added to any module. For Verilog, effective directives are calculated and included for each module. ◦ **Include**: Adds all include statements. For Verilog, adds effective `'include`/`'define`/`pragma` statements into the text views created for each imported module. ◦ **Ignore**: The include files are parsed, but the modules declared inside the include files are not created. Include statements are not added to the modules. For Verilog, effective directives are calculated and included for each module. For SPICE and CDL, you can use the **Except** rows to create multiple **Parse**/**Include**/**Ignore** exceptions to the **All files** directive. Click to add an **Except** row under the current row, or to delete the current row. Match type for the exception can be **Exact**, **Reg Expr**, or **Wildcard** (the default). Exceptions are prioritized such that the lowermost condition is checked first; if there is no match, then the next condition up the list is checked, and so on. In other words, lower conditions have higher priority. Default exceptions are: ◦ Include files with extensions `.dspf`, `.gz`, `.gzip`, `.enc` and `.spf`. ◦ Ignore files with extensions `.data`, `.ic`, `.lib`, and `.vec`.
Create Virtual Terminals (SPICE/CDL only)	Select **Create Virtual Terminals** to enable the options for creating certain terminals in the subcircuit declaration as virtual.

Feedback

Settings	Description
	Use Specified Mapping allows you to specify manually the terminal names to be created as virtual. Click **Edit** to open the **Manage Virtual Term** dialog box and enter terminal name to definition name mapping (the default net name mapping entries); for example, terminal name > (connection definition, default net name). These are the parameters required for creating a connection definition for a terminal.  On import, the name of any terminal found in the subcircuit declaration is matched against the terminal names in the specified mapping table. If the mapping table has an entry for the terminal being looked up, it is created as virtual using the specified definition and default net name. The name matching is done according to the netlist type. If the netlist is case-insensitive, name matching is done case-insensitively and the connection definition is created with the specified parameter names in lowercase. If the netlist is case-sensitive, name matching is done case-sensitively and the connection definition is created as specified by the user. The table displays connection assignments and netSet CDF parameters. Clicking **OK** or **Apply** saves the modified definition name and default net as connection assignments. Selecting the **Save as netSet to CDF** checkbox and clicking **OK** or **Apply** saves the modified definition name and default net as netSet parameter.

Feedback

Settings	Description
	Use Automatic Mapping if Unspecified allows for the automatic determination/recognition of the terminals to be created as virtual, using the specified reference view list. If this option is selected (and **Use Specified Mapping** is unselected), the subcircuit terminal names are compared with the terminal names in the **Reference Views** (the first view on the list is considered first and each successive view is checked only if the preceding view does not exist). If the netlist contains a terminal that is not found in the **Reference Views** text field, it is created as virtual using the terminal name followed by bang (!) as the connection definition and the default net name. For example, for terminal vdd that is on the netlist but not in the **Reference Views**, a connection definition is created using the default mapping (vdd!, vdd!). If both **Use Specified Mapping** and **Use Automatic Mapping if Unspecified** are selected, to determine whether or not to create a terminal as virtual, its name is first looked up in the user-specified mapping table. If it is not found, automatic mapping is tried next.
Reference	**Library Search List** shows an ordered list of libraries for reference cells. By default all the existing libraries are considered, including the output library. **View Search List** lets you specify the views that are considered matches if a cell match is found. The system uses the library and view search lists to find reference cellViews for binding instances. The library list returns a match and the view list completes the match and sets the view name to be used. The default view is `symbol`, which is also the first priority. If the netlist file you import is in Verilog formats, like Verilog, Verilog-A, Verilog-AMS, and SystemVerilog, the system uses the following view search list: verilog, veriloga, verilogams, systemverilog, spice, cdl, ENTITY, vhdl.component If not, then uses the following view search list: spice, cdl, verilog, veriloga, verilogams, systemverilog, ENTITY, vhdl.component Tcl preferences: `dbImportTextVerilogViewName`, `dbImportTextVerilogAViewName`, `dbImportTextVerilogAMSViewName`, and `dbImportTextSystemVerilogViewName`. The target view name specified for the current import has the highest priority. For example, if you specify `verilog` as the view name for **Output** in the **Main** tab, then `verilog` has the highest priority in **View Search List**.

4. On the **Generation** tab, enter settings as described in Table 31.

Feedback

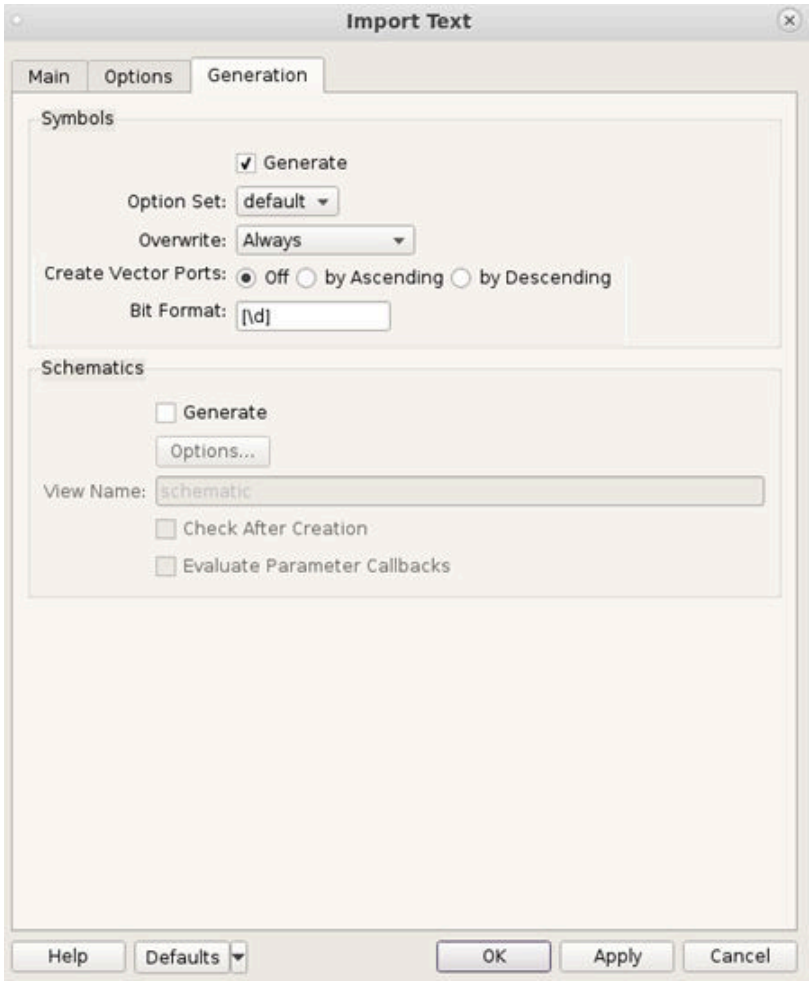


Table 31 Import Text Settings (Generation tab)

Settings	Description
Symbols	**Generate** generates a symbol view as part of the import process. Optional when generating a schematic that by definition generates symbols during schematic creation. But if child views do not exist, this option with **Overwrite** set to **Never** precreates symbols for text views with missing child views, allowing parent schematic views to be generated. **Option Set**: Choose the **Option Set** for the tool to use when creating a symbol view for each cell. See Generator Option Sets in the *Custom Compiler Schematic Editor and Text Editor User Guide* for details on available option sets. Choose when to **Overwrite**: **Never**, **Always**, or **Port Order Only**.

Feedback

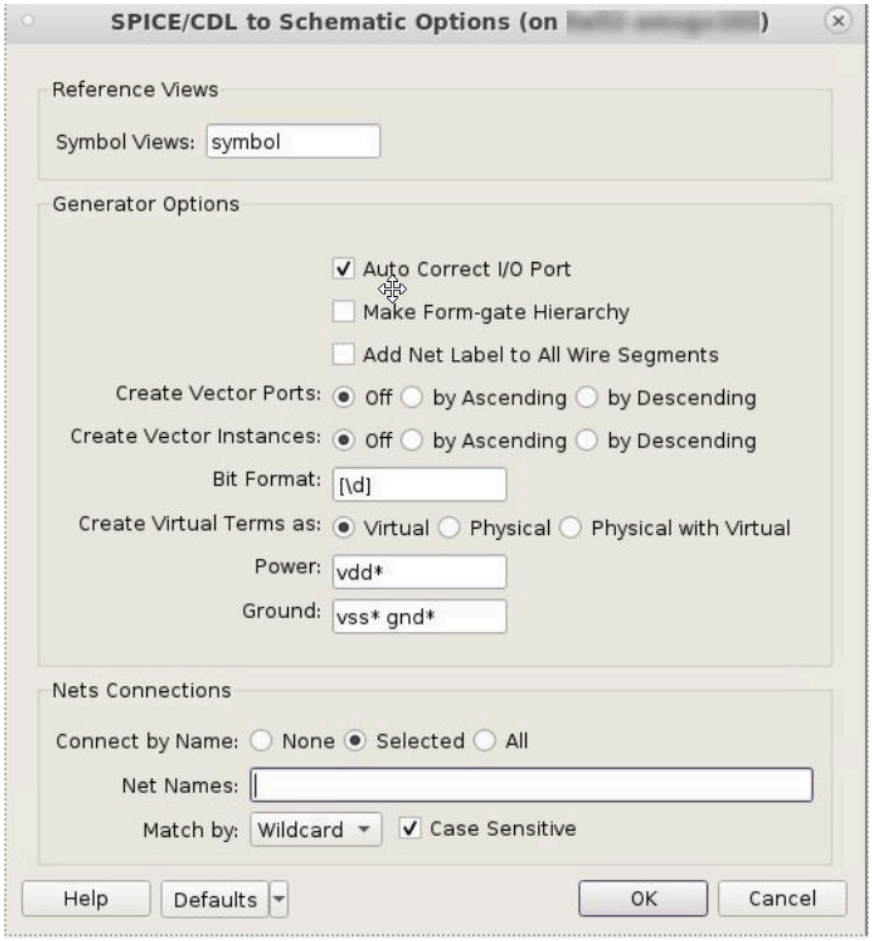
Settings	Description
	Create Vector Ports: Controls whether to merge vector ports when symbol generation. Choose **Off**, **by Ascending**, or **by Descending**. Default: Off **Bit Format**: The default bit format is [\d].
Schematics	(For SPICE, CDL, or Verilog views only). **Generate** generates schematic views from the imported text view. Click **Options** to generate schematics. See Step 5. **View Name** shows the default view name for generated schematic views. You can enter a different view name, as preferred. The system applies the view name you enter here. **Check After Creation** runs ERC/SRC checks after schematic creation. **Evaluate Parameter Callbacks** enables instance parameter callback evaluation during schematic generation.

5. Skip this step if you are not choosing to generate schematics.

   If you choose to generate schematics in the **Generation** tab, after clicking **Options** under **Schematics**, the **SPICE/CDL to Schematic Options** dialog box (for SPICE and CDL views) or the **Verilog to Schematic Options** dialog box (for Verilog view) appears.

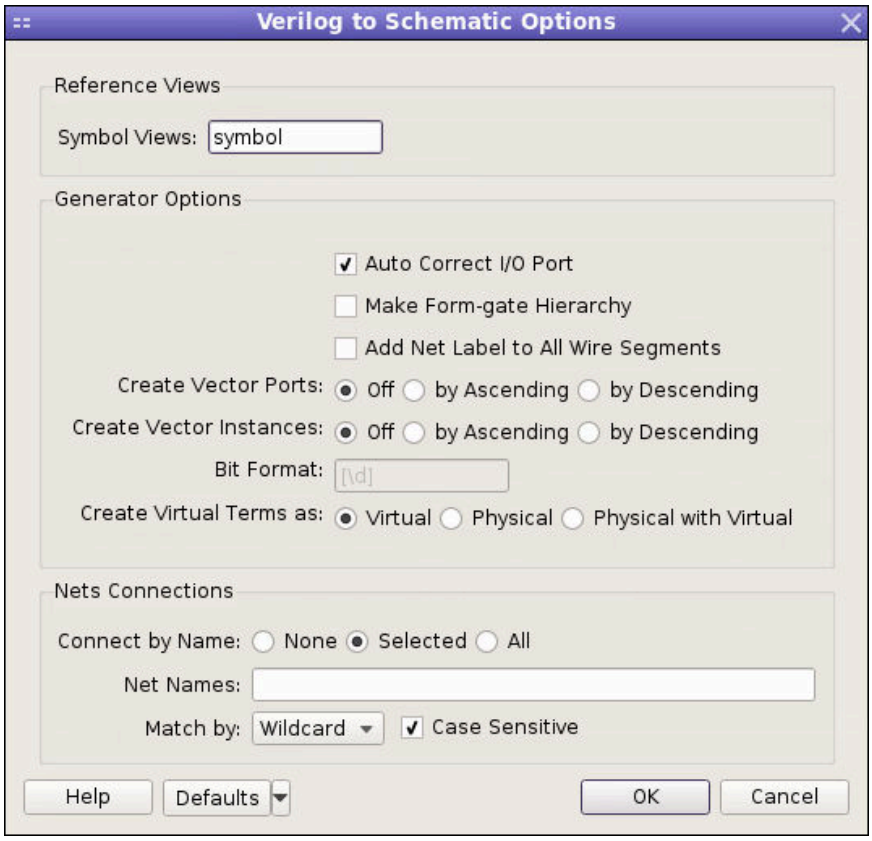
Feedback

Figure 18 SPICE/CDL to Schematic Options dialog box



Feedback

Figure 19 Verilog to Schematic Options dialog box



Enter settings as described in Table 32. Click **OK** in the **Schematic Options** dialog box when you are done.

Table 32 Schematic Options

Settings	Description
Reference Views	**Symbol Views** is the list of views the tool uses during schematic generation to find existing cells for binding (as opposed to creating) these views. Note that the **Option Set** and **Overwrite** values used for generating symbols during schematic generation are specified in the **Symbols** section of the **Import Text** dialog box's **Generation** tab. See Table 31.
Generator options	In SPICE, all ports are I/O, so the **Auto Correct I/O Port** option (on by default) attempts to correct the port direction if possible, which improves the overall look of the schematic/symbols.

Feedback

Settings	Description
	Make Form-gate Hierarchy attempts to recognize common gate structures (such as inverters, and, nand gates). If found, it creates hierarchical cells of these gates. **Add Net Label to All Wire Segments** adds net labels to each segment of a wire rather than just one segment. **Create Vector Ports** controls whether to merge the bus bit terminals to a bus by selecting one of the following options: **Off**, **by Ascending**, or **by Descending**. For example, `A<0>` and `A<1>` to `A<0:1>`. You can choose to sort bus bits in ascending or descending order. **Create Vector Instances** controls whether to merge Vector Bit instances to a Vector instance by selecting one of the following options: **Off**, **by Ascending**, or **by Descending**. The resulting Vector Instance bits order is controlled via the selected value. Instances from the source text view get merged to a Vector Instance only under the following conditions: ◦ The Instance is a Vector Bit instance. ◦ The base name should match for all bits. ◦ The placed master should match for all bits. ◦ Instance properties should match for all bits. **Bit Format** specifies the vector or bus bit notation format used in the source text. Target object's name is updated to match the tool's vector or bus bit notation. The name of objects (such as instances, terminals, and nets) from the source text matching to the specified bit format are placed. Valid values are `<\d>`, `[\d]` or any string ending by `\d`. For Verilog sources, this filed is disabled and the current value is ignored. The `[\d]` notation is forced to use for the specified value. It should either be enclosed in `[ ]` or `< >` brackets or as a suffix prepended by a delimiter, otherwise the value considered as invalid and ignored. For example, a format `#\d` matches all names like `Inst123#4`. The default is `[\d]` means any name like `Inst123[4]` is considered as a bit and changed to the tool's default bit notation. That is `Inst123<4>`. **Power** recognizes the nets as power net. **Ground** recognizes the nets as ground net.

Feedback

Settings	Description
	Create Virtual Terms as defines the virtual terminals found in the source text view are created in target schematic by selecting one of the following options: ◦ **Virtual**: Terminals from the source text view that have connection definition on them are not created in target schematic, instead the connection definition is copied to the corresponding net ◦ **Physical**: Terminals from the source text view that have connection definition on them are created in target schematic as regular terminals. ◦ **Physical with Virtual**: Terminals from the source text view that have connection definition on them are not copied as is (with the connection definition) to the target schematic.
Nets Connections	**Connect by Name** creates connections between nets by name for **Selected** nets, **All** nets, or **None** of the nets. **Net Names** is the list of patterns to match for net names when you choose **Selected** with **Connect by Name**. For example, select global nets, such as power and ground, to eliminate nets that connect to a large number of devices. Invalid patterns are ignored. **Match By**: Select the pattern syntax used in the **Net Names** text field. Valid values are **Wildcard** and **RegExp**. **Case Sensitive**: Controls the case sensitivity for net names matching patterns.

6. Click **OK**.

   The system imports the text and creates the designated views.

Generating Device Mapping File

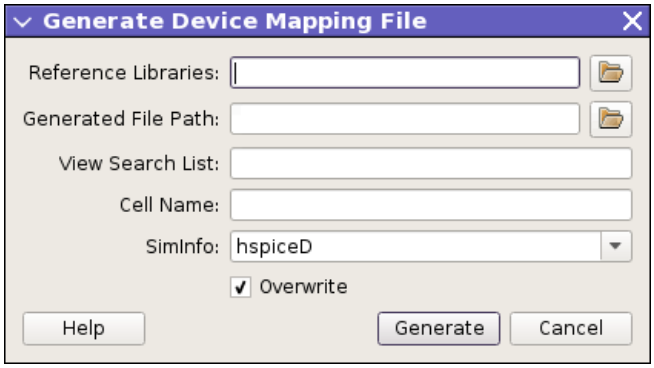
To create appropriate instances for netlist devices, an XML formatted device mapping file is required. The device mapping file is used mainly for determining the master cellview that should be used for instances, order of ports, and the correct instance property names and values, according to the master's CDF data.

The `$SUB` syntax is now supported in the CDL format for import netlist flow.

Feedback

To create a mapping file from a selected library, which can then be edited with additional required data:

1. On the **Import Text** dialog box, click **generate** to open the **Generate Device Mapping File** dialog box, as illustrated in the following figure.



   (Clicking **Generate** on the **Generate Schematic from Text** dialog box also opens the same **Generate Device Mapping File** dialog box).

2. Browse to select the **Reference Libraries** and the **Generated File Path** where the file resides. Enter the **View Search List** from which to generate the mapping data (the default is **symbol**), and the **Cell Name**, where cell name matching allows wildcard characters.
3. Specify which **SimInfo** section to be used for device mapping file template generation (the default is **hspiceD**).
4. (Optional) Select **Overwrite** to overwrite any existing mapping file.
5. Click **Generate** to generate the file. See Device Mapping File Format.

Device Mapping File Format

The device mapping file contains the following simple xml format:

```
<file-format version="1.0" name="synopsysDeviceMappingFile" >
     <deviceMappings><deviceMapping>
          <devType>devType</devType>
          <cellType>cellType</cellType>
          <models>modelname ...</models>
          <regModels>regModels</regModels>
          <portProp>portProp</portProp>
          <ports><{>p1<[portProp]> ...<}>
 <{>p2<[portProp]> ...<}> ...</ports>
          <portDefaultNets>pr1(n1) pr2(n2) ...</portDefaultNets >
          <propMatching>p1=v1 p2=v2 ...</propMatching>
          <lcv>libname/cellname/viewname</lcv>
```

Feedback

```
          <propMapping>p1<(v1, v2, ...)>=mp1<(mv1,
 mv2, ...)></propMapping>
          <stringParameters>p1[(p1v1, p2v2, ...)] p2(p2v1, p2v2, ...) ...
 <stringParameters>
          <appProps>appProps</appProps>
          </deviceMapping>
          ...
     </deviceMappings>
</file-format>
```

Each `deviceMapping` record can contain the following tags:

- devType
- cellType
- models
- regModels
- portProp
- ports
- portDefaultNets
- propMatching
- lcv
- propMapping
- stringParameters
- appProps

All tags are optional except `<devType>` and `<lcv>`. Note that only the cell part of the `lcv` field is required; the library and view names can be absent, which indicate that the `libSearchList` and `viewSearchList` property values are used to search for a cellview with the given cell name.

The values of the following tags are examined for determining the correct `lcv` to be used for a device: `devType`, `cellType`, `models`, `ports`, and `propMatching`. All the other fields are used to correctly map the device/instance parameter names and to create correct instance connections.

To find a matching `lcv`, the values of these tags are examined in the following order: `devType`, `cellType`, `models`, `ports`, `propMatching`. Matching to a device/instance

Feedback

declaration is done in the following steps, where each step does its filtering among the records which have been filtered after the previous steps:

1. Find the records having the `devType` of the given device.
2. Find those records which contain in their `models` field the name of the model specified in the device declaration. If no model has been specified in the device declaration or no matching record has been found, searching for a record with an empty model field, which is considered as a default mapping for that device type, is done.
3. For active elements for which the N or P type has been recognized from the netlist file, using the model card information or checking the model name against the N or P type patterns and a record with an empty model name has been found only, check also that the found cell's `cellType` matches the model type.
4. Check among the found records for those which have a `ports` field with port number corresponding to the number of nets in the device declaration.
5. For each record matching the preceding conditions, as the final matched record is considered the first one having `propMatching` conditions to which the parameters in the device declaration match.

If no matching device mapping is found for the device declaration, the device is considered as unbound. For instances for which no mapping is found the module name specified in the instance declaration is considered as a name for a master cell.

If a matching mapping record is found for an instance or a device, that instance is created with a master cellview specified in the `lcv` field. Instance connections are done according to the `ports` and `portDefaultNets` fields' values and the instance properties' creation is done according to the `propMapping` field's value. The ports with names given in the `ports` field are by order connected to given in the device declaration nets. The remaining ports are connected according to the connections specified in the `defaultPortNets` field.

For correct instance property creation, the `propMapping` field's value is used in the following way: parameters given in the device declaration for which the mapping data exists in the `propMapping` field, are mapped according to the specified mapped name and values. Note that for CDL devices, the names of the parameters which are netlisted with a proceeding `$` sign, should be specified in the device mapping file with `$` sign as well, for example: a capacitor's `$w` parameter following data can be specified in the `propMapping` and `propMatching` tags correspondingly as `$W=w` and `$w>2u`.

devType

`devType` for devices is the device name prefix for SPICE (R, C, L, W, U, and so on) and is the primitive name for Verilog (tranif0, tranif1, nand, and so on). For module/subckt instantiations, `devType` should be `X` for SPICE and `module` for Verilog. Subckt instantiations, which are actually subckt models, can be mapped using the `X<devType>` prefix, for example `XM`, `XC`, and so on. This allows to map the subckt model instantiations,

Feedback

with the X<device_prefix> instance name prefix, the same way as the real device instantiations.

The devType tag can contain more than one device type, to support identical mapping for same type devices and subcircuit models, for example: M XM" and R XR.

cellType

cellType is used for specifying the cellType of the mapped device. For active elements, it should indicate whether the device is of N or P type. Valid values for active elements are NMOS and PMOS (for MOSFET devices), NJFET and PJFET (for JFET devices), and NPN and PNP (for BJT devices). For all other SPICE devices, the cellType should be the same as the devType.

The cellType tag is used mainly for schematic generation purposes. For active elements, cellType is used to correctly map the devices recognized to be of N or P type. The N or P type is recognized from the model card if it exists. Otherwise if no model card exists, or the device instantiated a subckt model, the N or P type is recognized by matching the model name with a pattern. The pattern checks for the first symbol of the model/subkct name to see if it is N or P, if it is not then checks which of the N and P symbols appears more times in the name.

models

models contains the model names that can be specified in the device declarations. Note that this field can be empty indicating that the current lcv is the default one for the specified device type, and in case if no model name is specified in the device declaration or no lcv mapping is found for the specified model, the given default lcv is used.

For MOSFET, JFET, and BJT devices which can be of N or P type, different default mappings can be specified for N and P type elements. For this purpose, one of the following can be specified as the model name: nmos, pmos, njfet, pjfet, npn, pnp.

For module/subckt instantiations, models should contain the module/subckt name.

Examples

```
<deviceMapping>
 <devType>M </devType>
 <models>MM1 MM2</models>
 <lcv>myLib/nmos/symbol</lcv>
<deviceMapping>

<deviceMapping>
 <devType>M</devType>
 <models>pmos</models>
 <lcv>myLib/pmos1/symbol</lcv>
<deviceMapping>

<deviceMapping>
```

Feedback

```
 <devType>M</devType>
 <models> </models>
 <lcv>myLib/nmos2/symbol</lcv>
<deviceMapping>

<deviceMapping>
 <devType>X</devType>
 <models>cell1</models>
 <lcv>myLib/mycell/symbol</lcv>
<deviceMapping>
```

According to the previously mentioned device mapping records, the following device declarations are mapped to the following cells:

```
.model MM1 nmos
.model MM3 pmos
.model MM4 nmos

M1 n1 n2 n3 MM1 1 2 // maps to myLib/nmos/symbol
M3 n1 n2 n3 MM3 1 2 // myLib/pmos1/symbol
M4 n1 n2 n3 MM4 1 2 // myLib/nmos2/symbol as there is no mapping declared
 for MM4
M5 n1 n2 n3 MM5 1 2 // myLib/nmos2/symbol as there is no mapping declared
 for MM5
X1 n1 n2 n3 cell1  // myLib/mycell/symbol
X2 n1 n2 n3 cell2  // is not mapped, libSearchList and viewSearchList
 properties are used to find a cell with name cell2
```

regModels

`<regModels>` is used for adding regular expressions to match model names. The regular expression matching has lower priority than the exact model name matching which is done by the `<models>` tag. If a model name parsed matched both `<models>` and `<regModels>` tags, the `<models>` tag is honored if the ports and parameters matched.

You can define multiple patterns in a `<regModels> </regModels>` tag pair by separating each pattern using a space character. The regular expression pattern model matching is supported in both the legacy parser flow (TN library) and the new common parser flow (GNDB) for SPICE/CDL format. If a device mapping section in the device mapping file both have `<models>` and `<regModels>` tags, only the `<regModels>` tag is honored.

portProp

`portProp` is the name of the property, which is used for the PCells with generated number of ports. For example, for W, U, and S devices, the number of ports is not fixed and if a PCell is used as the master, the ports are generated according to property value like `portnumber`. And because the device declaration contains no such parameter, its value should be calculated from the number of net names given in the device declaration and certain port list pattern given in `ports`.

Feedback

ports

`ports` contains the list of ports to which the nets given in the device declaration should be connected by order.

Note that:

- The connected ports that should be connected to a net are given as a list. The port names in the *i-th* list are connected to the net which corresponds to the net on *i-th* position in the device declaration, for example, for PCells with a variable number of ports, if you want MOSFETs with 3 nodes to connect to a master with 4 terminals, specify the following port list meaning that the last net is connected both to ports S and B: `G D {S B}`.
- If the net given in some positions in the device declaration should not be connected to any net, an empty list can be specified instead of the port name, for example: for a MOSFET device, the list of ports can contain the following: `{} G D S`, meaning that the first name given in the device declaration should not be connected to any port, and the remaining 3 nets should be connected to G, D, and S ports correspondingly.
- Given port names can be either the names of existing ports in the cellview or the names of virtual terminals to which the nets from the device declaration should be connected. So any port name given in the list is considered as a terminal name, if such exists in the design, otherwise it is considered as a virtual terminal name to which the given net should be assigned. Use the `teParseExtraTermsAsVirtual` Tcl preference to decide whether extra port names should be considered as virtual terminals.
- For PCells, the ports of which are generated depending on property values (given in the `portProp` field), port names can be specified as a port name prefix followed by a bracket with the `portProp` name inside, for example, `{in[n]} {refin} {out[n]} {refout}` for w element. This indicates that the port names should be generated with the given prefix appended with indexes from 1 to *n*. Note that the value of *n* is actually calculated from the number of nets given in the device declarations. For example, for the preceding W element port pattern, the number of nets should be 2n+2 and if the device declaration contains 6 nets, n is calculated as 2, so the generated port names should be `in1 in2 refin out1 out2 refout`.
- If the `ports` field is empty, the portOrder/termOrder of the device is used for instterm connection. (If those properties are also empty, the instance remains unconnected).

portDefaultNets

`portDefaultNets` defines the default connections for ports with the following format: `pr1(n1) pr2(n2)`, where `pri` is the port name, and `ni` is the net that should connect to it if no other connection is specified for the port in the `ports` field.

For CDL netlists, this `portDefaultNets` tag can contain also port connections in the following format: `pi($param)`, which indicates that `pi` port should be connected to the net

Feedback

the name of which is given as a value of `$param` parameter. Note that *port-to-parameter* connections are considered as overriding for those made according to the `ports` field. This means that if the `ports` field contains `p1 p2 p3 p4` for a BJT element, and the `portDefaultNets` field contains value `p4($SUB)`, port `p4` is connected to the net the names of which is specified as a value of `$SUB` parameter in the BJT device declaration.

propMatching

To have the specified in the device mapping record `lcv` to be used as the master for the given device, `propMatching` defines the conditions to which the parameter values in the instance declaration should match.

`propMatching` can contain a set of conditions on more than one parameter represented as expressions with equal (`==`), not equal (`!=`), less (`<`), greater (`>`), less or equal (`<=`), and greater or equal (`>=`) operators. `propMatching` can also indicate matching by parameter existence or absence.

Note that more than one `propMatching` tag can be given in a device mapping record. To have matching with the case when a particular parameter is given in the device declaration, the parameter name should be specified in `propMatching`, and to indicate the opposite (the parameter is not given in the device declaration), a `!` sign should be set before the parameter name. For example: `w<2u`, `l>4u`, or `!CONVOLUTION ! FBASE !FMAX !RS` which indicates that the specified master should be chosen only for linear transistors, which exclude the usage of this cellview as the master for frequency dependent and skin effect resistors.

lcv

`lcv` contains the library, cell, and view names (separated by slashes) of the master cellview that should be used as a master for the device declaration which matches the data of all the previous fields. Note that if the library or view name is empty, `libSearchList` and `viewSearchList` values are used instead to search a given cell name in the listed libraries.

propMapping

`propMapping` contains the mapping of hspice parameter names and their values to corresponding parameter names and values for the given cell's CDF data. For example:

- `tranfun(SIN,PU)=srcType(SINE,PULSE)` means that SPICE parameter `tranfun` is mapped to CDF parameter `srcType` and its values `SIN` and `PU` are mapped correspondingly to values `SINE` and `PULSE`.
- `R=res` means that the R value of the SPICE device is saved in RES parameter of the OA instance.

Feedback

stringParameters

`stringParameters` contains the mapped names of string and enumeration type parameters for a given device.

The `stringParameters` tag is used for correct replacing of identifier names representing parameter names with `lineage("param_name")` expression in the instance parameter values. This is needed for correct schematic generation to make parameter evaluation look for the parameter value in upper levels of the hierarchy.

For string parameters, only the names should be listed, while for enumerated parameters, the tag can contain the parameter name followed by the supported enumeration values in brackets separated by commas, so that other string values passed to this parameter are considered as parameter names.

Also, non-string parameters that can have some string keyword values should be listed in this `stringParameters` tag. For example, keyword `infinity` for capacitor device's parameter `C`: `C(infinity)`.

appProps

`appProps` contains the name of parameters which should be created in OA as instance properties if specified in the device declaration, even if they are not valid parameters according to the SPICE documentation, does not exist in the CDF and are not specified in the `propMapping` field.

Device Mapping File Examples

The following lines show examples of `deviceMapping` records.

```
<deviceMappings>
<!-mapping Sinusoidal voltage sources to the vsin cellview of analoglib .
 -->
<deviceMapping>
     <devType> V </devType>
     <model> </model>
     <portProp></portProp>
     <ports> PLUS MINUS </ports>
     <propMatching> tranfun=sin </propMatching>
     <lcv> analogLib vsin symbol </lcv>
     <propMapping>
          tranfun(SIN)=srcType(Sine) dc=vdc acmag=acm acphase=acPhase
 vo=vo va=va td=td freq=freq q=q j=j
     </propMapping>
     <appProps>z</appProps>
</deviceMapping>

<!-mapping FET transistors (with 3 ports) with models named M1 and M2 to
 analoglib nmos (which is 3-port device with virtual terminal p_bulk),
```

```
 and the model names M1 and M2 are mapped as nmos1 and nmos2. The bulk is
 connected to the same net as the source. -->
<deviceMapping>
     <devType> M </devType>
     <model> M1 M2</model>
     <portProp></portProp>
     <ports> {G} {D} {S p_bulk} </ports>
     <propMatching> </propMatching>
     <lcv> analogLib nmos hspice </lcv>
     <propMapping>vds=Vds vgs=Vgs vbs=Vbs
        mname(M1,M2)=model(nmos1, nmos2)
     </propMapping>
     <appProps>a b</appProps>
</deviceMapping>

<!-mapping FET transistors (with 4 ports) with models named M3 and M4
 to analoglib nmos (which is 3-port device and has a virtual terminal
 p_bulk), and the model names M3 and M4 are mapped as nmos3 and nmos4.
 The bulk is connected to the same net as the source. -->
<deviceMapping>
     <devType> M </devType>
     <model> M3 M4</model>
     <portProp></portProp>
     <ports> {G} {D} {S} {p_bulk} </ports>
     <propMatching> </propMatching>
     <lcv> analogLib nmos hspice </lcv>
     <propMapping>vds=Vds vgs=Vgs vbs=Vbs l=l w=w
        mname(M3,M4)=model(nmos3, nmos4)
     </propMapping>
     <appProps>a b</appProps>
</deviceMapping>

<!-mapping High voltage FET transistors with models named HVP1 and HVP2
 to analoglib nmos4v, and the model names HVP1 and HVP2 are mapped as
 nmos1 and nmos2. -->
<deviceMapping>
     <devType> M </devType>
     <model> HVP1 HVP2</model>
     <portProp></portProp>
     <ports> {G} {D} {S} {B} </ports>
     <propMatching> </propMatching>
     <lcv> analogLib pmos4hv symbol </lcv>
     <propMapping>vds=Vds vgs=Vgs vbs=Vbs
        mname(HVP1, HVP2)=model(pmos1hv, pmos2hv)
     </propMapping>
     <appProps>a b</appProps>
</deviceMapping>

<!-mapping skin effect resistor to the myLib skin_effect_res symbol of
 myLib library,-->
<deviceMapping>
     <devType> R </devType>
     <model> </model>
```

```
      <portProp></portProp>
      <ports> {PLUS} {MINUS}</ports>
      <propMatching> rs </propMatching>
      <lcv> myLib skin_effect_res symbol </lcv>
      <propMapping> rs=RS </propMapping>
      <appProps></appProps>
</deviceMapping>

<!-mapping pdif resistor to the myLib resPdif symbol -->
<deviceMapping>
      <devType> R </devType>
      <model>pdifres pdres </model>
      <portProp></portProp>
      <ports> {PLUS} {MINUS}</ports>
      <propMatching> </propMatching>
      <lcv> myLib resPdif symbol </lcv>
      <propMapping> r=res mname=modelName </propMapping>
      <appProps></appProps>
</deviceMapping>

<!-mapping resistor with TC1>3 to the analogLib res symbol and
 settingmodelname to nwelRes -->
<deviceMapping>
      <devType> R </devType>
      <model> </model>
      <portProp></portProp>
      <ports> {PLUS} {MINUS}</ports>
      <propMatching> TC1>3 </propMatching>
      <lcv> analogLib res symbol </lcv>
      <propMapping> r=res </propMapping>
      <propDefaults> modelName(nwelRes) </propDefaults>
      <appProps></appProps>
</deviceMapping>

<!-mapping frequency dependent resistor to the myLib resFreq symbol -->
<deviceMapping>
      <devType> R </devType>
      <model> </model>
      <portProp></portProp>
      <ports> {PLUS} {MINUS}</ports>
      <propMatching> CONVOLUTION </propMatching>
      <lcv> myLib resFreq symbol </lcv>
      <propMapping> rs=RS </propMapping>
      <appProps></appProps>
</deviceMapping>

<!-mapping W device to analogLib mtline device. This is an example with
 the portProp usage, when the number of ports is variable. -->

<deviceMapping>
      <devType> W </devType>
      <model> </model>
      <portProp>n</portProp>
```

```
      <ports> {in[n]} {refin} {out[n]} {refout}</ports>
      <lcv> myLib skin_effect_res symbol </lcv>
      <propMapping>… </propMapping>
      <appProps></appProps>
</deviceMapping>

</deviceMappings>
```

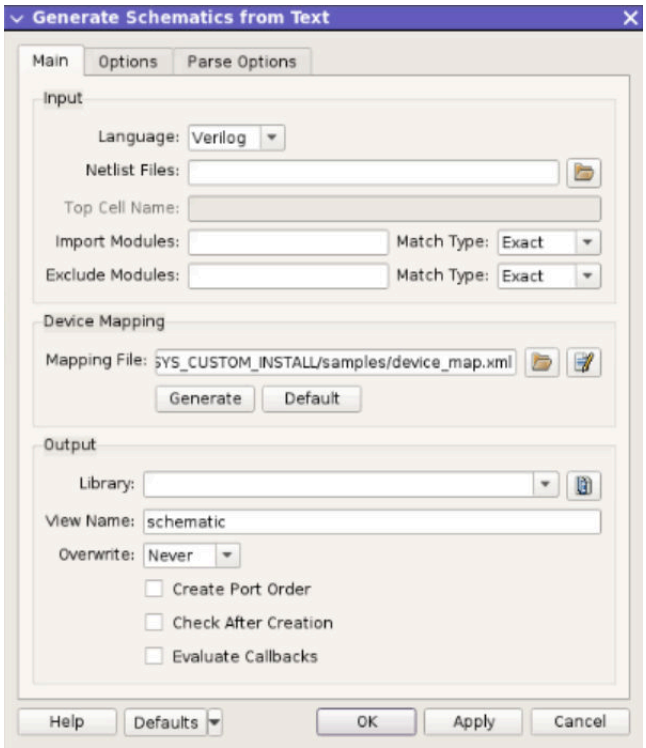
Importing Design Netlist Data for Schematic Generation

You can import design netlist data to generate schematics without creating text views. Available text netlist formats include SPICE, CDL, and Verilog. You can optionally generate symbol views while importing netlist text.

Compressed files that are supported by the Linux `zcat` command are supported by this flow. Compressed file support can also be enhanced using the `db::showGenerateSchematicsFromText::decompressFiles` procedure.

To import design data for schematic generation:

1. From the **Home** or **Console** tab, choose **File > Import > Schematic from Netlist** to open the **Generate Schematics from Text** dialog box.



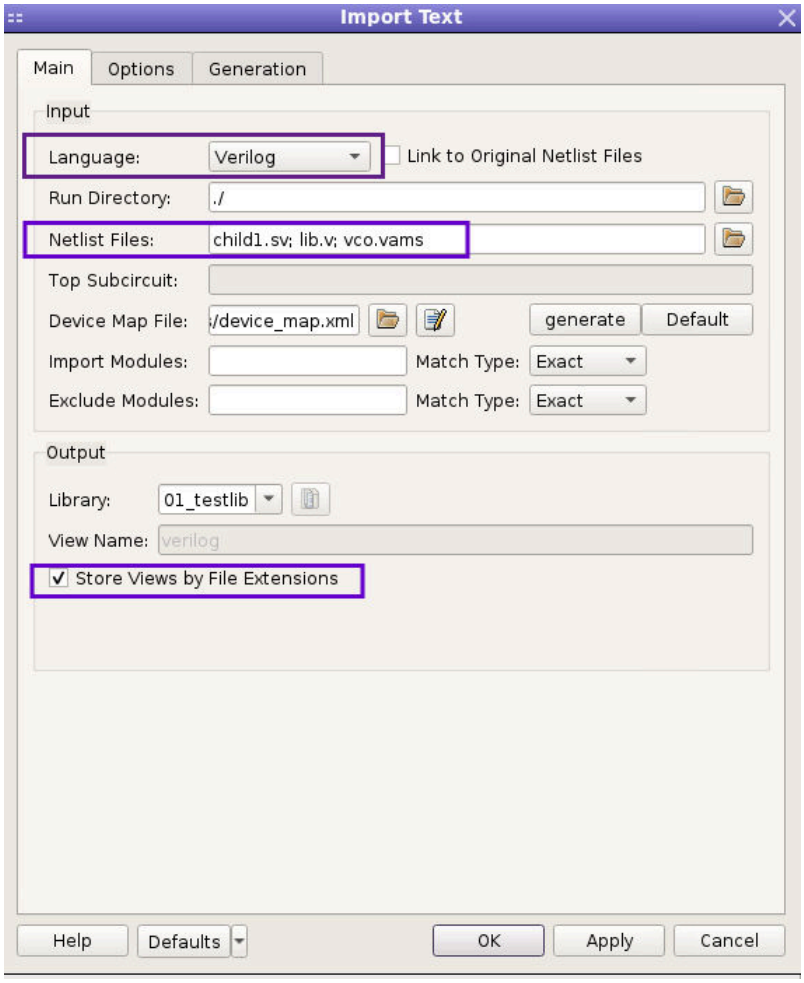
2. On the **Main** tab, enter settings as described in Table 33.

Feedback

Table 33 Generate Schematics From Text Settings (Main tab)

Settings	Description
Input	**Language** is the behavioral modeling language you specify from the list: **SPICE**, **CDL**, **Verilog**, **Verilog-A**, **Verilog-AMS**, **SystemVerilog**, or **VHDL**. **Netlist Files** specifies the source netlist for this import process. Browse to select the file. Multiple files can also be selected. **Top Cell Name** works for CDL and SPICE formats only. Enter the name of a cell where top-level SPICE/CDL design data should be put. A name based off the imported file name is generated by default. **Import Modules** lets you list module names or specify the name pattern for the modules you wish to import. **Match Type** can be **Exact**, **Reg Expr**, or **Wildcard**. **Exclude Modules** lets you list module names or specify the name pattern for the modules you wish to exclude from the import when files with multiple cells are imported. **Match Type** can be **Exact**, **Reg Expr**, or **Wildcard**.
Device Mapping	**Device Map File** works for SPICE, CDL, and Verilog formats only. You can click any of the following buttons: ◦ **Device Mapping File for resolving instance masters** to designate the required device mapping source (to resolve instance masters to reference libraries) from a file. ◦ **Generate** to create the device map file from a reference library. See Generating Device Mapping File. Click **Edit Mapping File** to open the selected mapping file in a text editor for adding or modifying the mapping information. Click **Default** to load the sample provided with the installation hierarchy. This file can also be copied and edited to address any missing information. The default mapping file maps all devices to analogLib library cells.
Output	**Library** specifies the destination library for this import operation, which you choose from the list. Click **Create Library** to open the **New Library** dialog box and create a new output library. **View Name** shows the view name of the schematic view that is generated. **Overwrite** specifies how the import process treats imported cellViews that conflict with existing cellViews. **Never** (the default) means that the existing cellView remains intact (no copy is created). **Always** means that existing cellViews are overwritten.

Settings	Description
	Create Port Order controls whether the portOrder property should be created in the generated schematic views. **Check After Creation** runs ERC/SRC checks after schematic creation. **Evaluate Callbacks** enables instance parameter callback evaluation during schematic generation.

Settings	Description
	Store Views by File Extension stores imported netlist with default view name specified with preference `dbImportText<Language>ViewName`. So, the Tcl preferences are `dbImportTextVerilogViewName` for Verilog, `dbImportTextVerilogAMSViewName` Verilog AMS, and `dbImportTextSystemVerilogViewName` for Syntem Verilog. This option appears when **Language** is set to **Verilog**, **Verilog-AMS**, or **SystemVerilog**. You can specify Verilog (*.v), Verilog-AMS (*.vams), and System-Verilog (*.sv) netlist files in the **Netlist Files** box to import them simultaneously. When this option is enabled, the **View Name** option is disabled. The tool recognizes file format by file extension defined with the preference `dbImportText<Langugage>FileExtensions`. So, the Tcl Preferences are `dbImportTextVerilogFileExtensions` for *.v, `dbImportTextVerilogAMSFileExtensions` for *.vams, and `dbImportTextSystemVerilogFileExtensions` for *.sv. 

Feedback

3. On the **Options** tab, enter settings as described in Table 34.

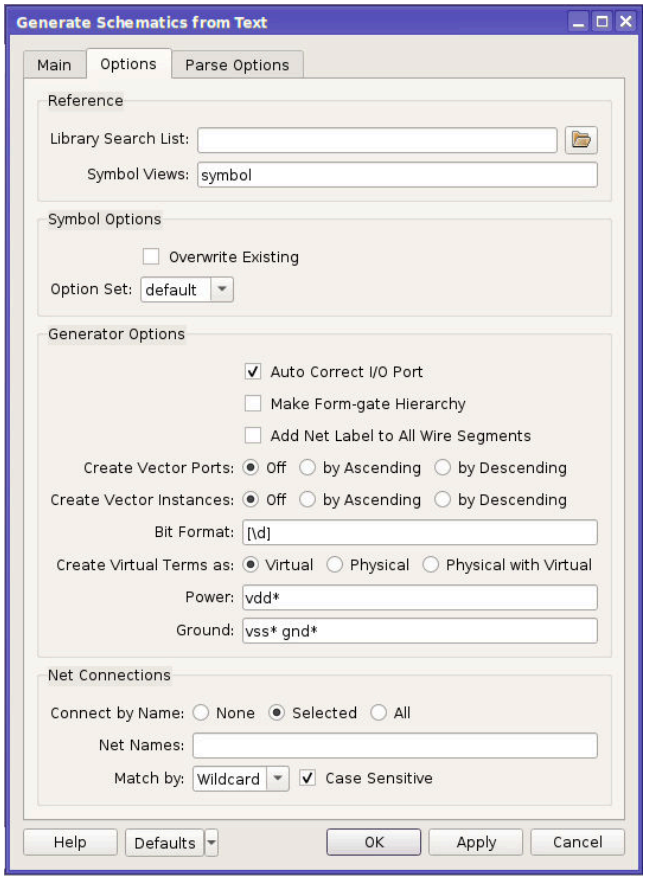


Table 34 Generate Schematics From Text Settings (Options Tab)

Settings	Description
Reference	**Library Search List** shows an ordered list of libraries for reference cells. By default all the existing libraries are considered, including the output library. **Symbol Views** is the list of views used during schematic generation to find existing cells for binding (as opposed to creating) these views.
Symbol Options	**Overwrite Existing** overwrites existing symbol views. **Options Set** is the options set for symbol generation.
Generator Options	In SPICE, all ports are I/O, so the **Auto Correct I/O Port** option (on by default) attempts to correct the port direction if possible, which improves the overall look of the schematic/symbols.

Settings	Description
	Make Form-gate Hierarchy attempts to recognize common gate structures (such as inverters, and, nand gates). If found, it creates hierarchical cells of these gates. **Add Net Label to All Wire Segments** adds net labels to each segment of a wire rather than just one segment. Enabled by default. Schematic Editor selects each island wire to create the net name label, so the generated results not only shows one net name on wire but each island has their net name label on the canvas. **Create Vector Ports** controls whether to merge the bus bit terminals to a bus by selecting one of the following options: **Off**, **by Ascending**, or **by Descending**. For example, `A<0>` and `A<1>` to `A<0:1>`. You can choose to sort bus bits in ascending or descending order. **Create Vector Instances** controls whether to merge Vector Bit instances to a Vector instance by selecting one of the following options: **Off**, **by Ascending**, or **by Descending**. The resulting Vector Instance bits order is controlled via the selected value. Instances from the source text view get merged to a Vector Instance only under the following conditions: ◦ The Instance is a Vector Bit instance. ◦ The base name should match for all bits. ◦ The placed master should match for all bits. ◦ Instance properties should match for all bits. **Bit Format** specifies the vector or bus bit notation format used in the source text. Target object's name is updated to match the tool's vector or bus bit notation. The name of objects (such as instances, terminals, and nets) from the source text matching to the specified bit format are placed. Valid values are `<\d>`, `[\d]` or any string ending by `\d`. For Verilog sources, this filed is disabled and the current value is ignored. The `[\d]` notation is forced to use for the specified value. It should either be enclosed in `[ ]` or `< >` brackets or as a suffix prepended by a delimiter, otherwise the value considered as invalid and ignored. For example, a format `#\d` matches all names like `Inst123#4`. The default is `[\d]` means any name like `Inst123[4]` is considered as a bit and changed to the tool's default bit notation. That is `Inst123<4>`.

Settings	Description
	Create Virtual Terms as defines the virtual terminals found in the source text view are created in target schematic by selecting one of the following options: ◦ **Virtual**: Terminals from the source text view that have connection definition on them are not created in target schematic, instead the connection definition is copied to the corresponding net ◦ **Physical**: Terminals from the source text view that have connection definition on them are created in target schematic as regular terminals. ◦ **Physical with Virtual**: Terminals from the source text view that have connection definition on them are not copied as is (with the connection definition) to the target schematic. **Power** recognizes the net as power net. **Ground** recognizes the net as ground net.
Net Connections	**Connect by Name** creates connections between nets by name for **Selected** nets, **All** nets, or **None** of the nets. **Net Names** is the list of patterns to match for net names when you choose **Selected** with **Connect by Name**. For example, select global nets, such as power and ground, to eliminate nets that connect to a large number of devices. Invalid patterns are ignored. **Match By**: Select the pattern syntax used in the **Net Names** text field. Valid values are **Wildcard** and **RegExp**. **Case Sensitive**: Controls the case sensitivity for net names matching patterns.

4. On the **Parse Options** tab, enter settings as described in Table 35.

Feedback

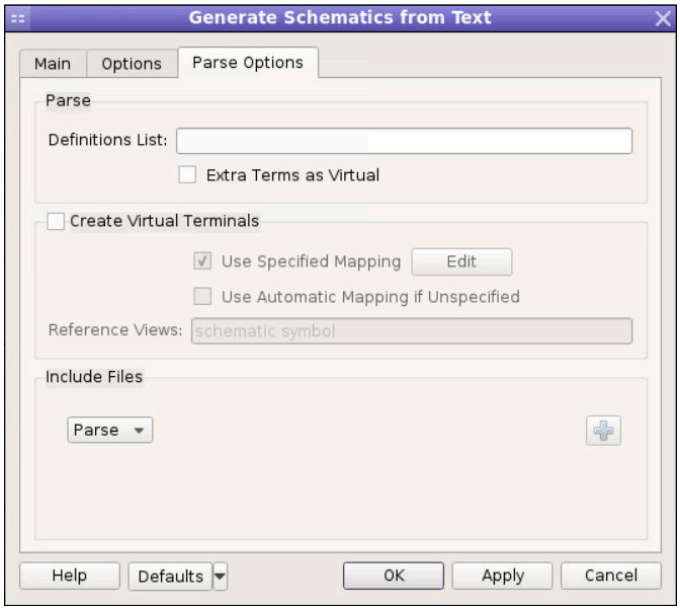
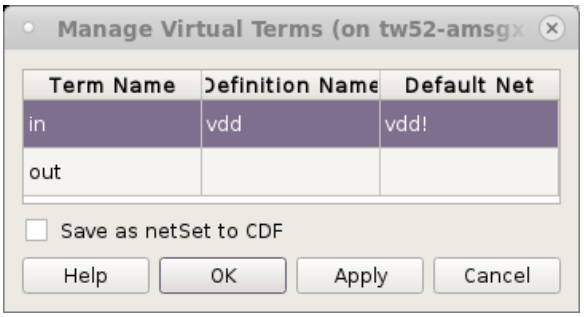


Table 35 Generate Schematics From Text Settings (Parse Options tab)

Settings	Description
Parse	(Verilog only) **Definitions List** specifies the list of define statements to be predeclared in each cellView. **Extra Terms as Virtual** ensures that, when extracting schematic designs from the netlist file, terminals that are missed in the master design are considered virtual terminals and corresponding connections to virtual terminals are made.
Create Virtual Terminals (SPICE/CDL only)	Select **Create Virtual Terminals** to enable the options for creating certain terminals in the subcircuit declaration as virtual.

Settings	Description
	Use Specified Mapping allows you to specify manually the terminal names to be created as virtual. Click **Edit** to open the **Term Mapping** dialog box and enter terminal name to definition name mapping (the default net name mapping entries); for example, terminal name > (connection definition, default net name). These are the parameters required for creating a connection definition for a terminal.  On import, the name of any terminal found in the subcircuit declaration is matched against the terminal names in the specified mapping table. If the mapping table has an entry for the terminal being looked up, it is created as virtual using the specified definition and default net name. The name matching is done according to the netlist type. If the netlist is case-insensitive, name matching is done case-insensitively and the connection definition is created with the specified parameter names in lower case. If the netlist is case-sensitive, name matching is done case-sensitively and the connection definition is created as specified by the user. **Use Automatic Mapping if Unspecified** allows for the automatic determination/recognition of the terminals to be created as virtual, using the specified reference view list. If this option is selected (and **Use Specified Mapping** is unselected), the subcircuit terminal names are compared with the terminal names in the **Reference Views** (the first view on the list is considered first and each successive view is checked only if the preceding view does not exist). If the netlist contains a terminal that is not found in the **Reference Views** text field, it is created as virtual using the terminal name followed by bang (!) as the connection definition and the default net name. For example, for terminal vdd that is on the netlist but not in the **Reference Views**, a connection definition is created using the default mapping (vdd!, vdd!). If both **Use Specified Mapping** and **Use Automatic Mapping if Unspecified** are selected, to determine whether or not to create a terminal as virtual, its name is first looked up in the user-specified mapping table. If it is not found, automatic mapping is tried next.

Feedback

Settings	Description
Include Files	Specifies whether all include files should be parsed or ignored. The options are: ◦ **Parse**: Parses the include files and creates the cells for modules found in the include files. Include statements are not added to any module. For Verilog, effective directives are calculated and included for each module. ◦ **Ignore**: The include files are parsed, but the modules declared inside the include files are not created. Include statements are not added to the modules. For Verilog, effective directives are calculated and included for each module. For SPICE and CDL, you can use the **Except** rows to create multiple **Parse**/**Ignore** exceptions to the **All files** directive. Click to add an **Except** row under the current row, or to delete the current row. Match type for the exception can be **Exact**, **Reg Expr**, or **Wildcard** (the default). Exceptions are prioritized such that the lowermost condition is checked first; if there is no match, then the next condition up the list is checked, and so on. In other words, lower conditions have higher priority. Default exceptions are: ◦ Include files with extensions `.dspf`, `.gz`, `.gzip`, `.enc` and `.spf`. ◦ Ignore files with extensions `.data`, `.ic`, `.lib`, and `.vec`.

5. Click **OK**.

   The system imports the text and generates the schematic.

Importing Design Data in EDIF 200 Format

EDIF is a common schematic format that is generated by many electronic CAD tools. Import design data as EDIF200 when you want to translate data files from an Electronic Design Interchange Format (EDIF) schematic to the Custom Compiler database.

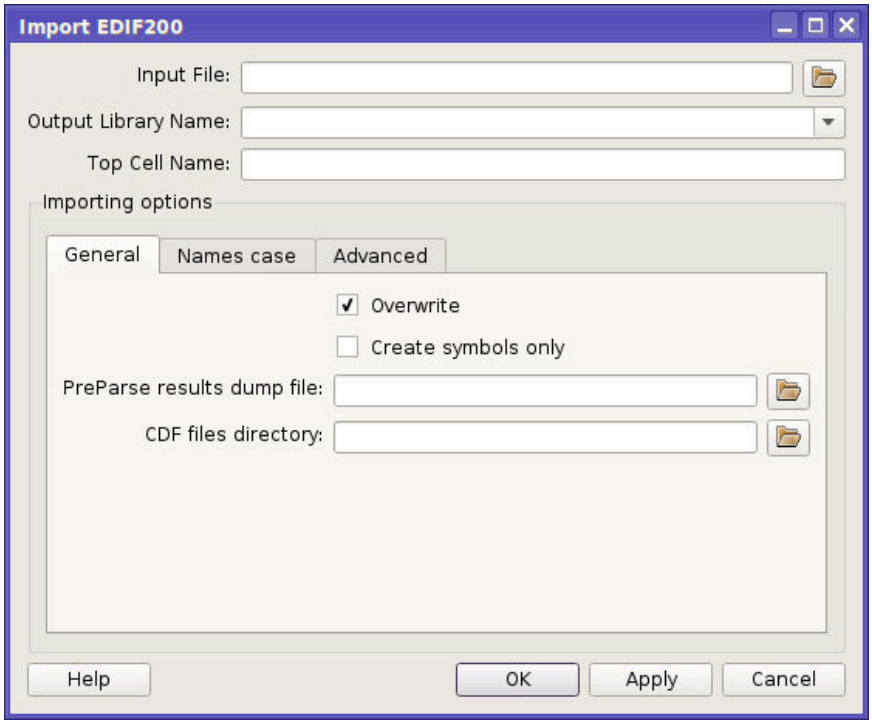
Feedback

Using the Import EDIF200 Dialog Box

To import design data as EDIF200:

1. From the **Home** or **Console** tab, choose **File > Import > EDIF200** to open the **Import EDIF200** dialog box.

Figure 20 Import EDIF200 Dialog Box: General Options



2. Enter settings as described in Table 36.

Table 36 Import EDIF200 Settings

Settings	Description
Input File	The schematic or netlist input design file. **Note:** Custom Compiler does not support importing EDIF files that define multipage schematics.

Feedback

Settings	Description
Output Library Name	The target design library. The EDIF format defines library names of cellviews being imported. If **Output Library Name** is not specified, Custom Compiler tries to create required libraries with names defined in EDIF file. The EDIF file could specify more than one library. If the **Output Library Name** is specified, all cellviews are created in the specified library regardless of their libNames defined in the EDIF file. In the case of conflicting cell names (for example, if the library being used already has such a cell), the tool renames the cell name using the format <LibNameFromEDIF>_cellName.
Top Cell Name	The top cell of the design to be translated. The EDIF file can contain many hierarchies, for example A > B > C and A1 > B1 > C1. If **Top Cell Name** is not specified, both hierarchies (6 cells) are imported. If, however, **Top Cell Name** is specified as A1, only the A1 > B1 > C1 hierarchy is imported. If **Top Cell Name** is specified as a mid-level cell name, such as B, Custom Compiler imports only the hierarchy starting from that mid-level cell, such as B > C.

3. On the **General** tab (Figure 20), enter settings as described in Table 37.

Table 37 Import EDIF200 Settings (General tab)

Settings	Description
Overwrite	Overwrites existing cell views after import. On by default.
Create symbols only	Imports only symbol views from the importing file, discarding everything else. This option can be used with **Output Library Name** to build a symbols library of the file being imported.
PreParse results log file	If a file path is defined, Custom Compiler does not import the specified EDIF file, but instead preparses it to collect the Library/Cell/View information of cellviews defined in the EDIF file. The complete list of Libraries/Cells/Views is output to the log file specified by the **PreParse results log file** option.
CDF files directory	The path to the directory in which to search a CDF file for a cell. If the EDIF file being imported has a cell with a cdfData property of string type defined, Custom Compiler tries to find a file of the name specified by this property in the directory specified by this option. If found, all parameters defined in that CDF file are appended to the cell.

Feedback

4. On the **Names case** tab (Figure 21), enter settings as described in Table 38. This dialog box controls the capitalization (case) of object names imported from the EDIF file.

Figure 21 Import EDIF200 Dialog Box: Names Case Options

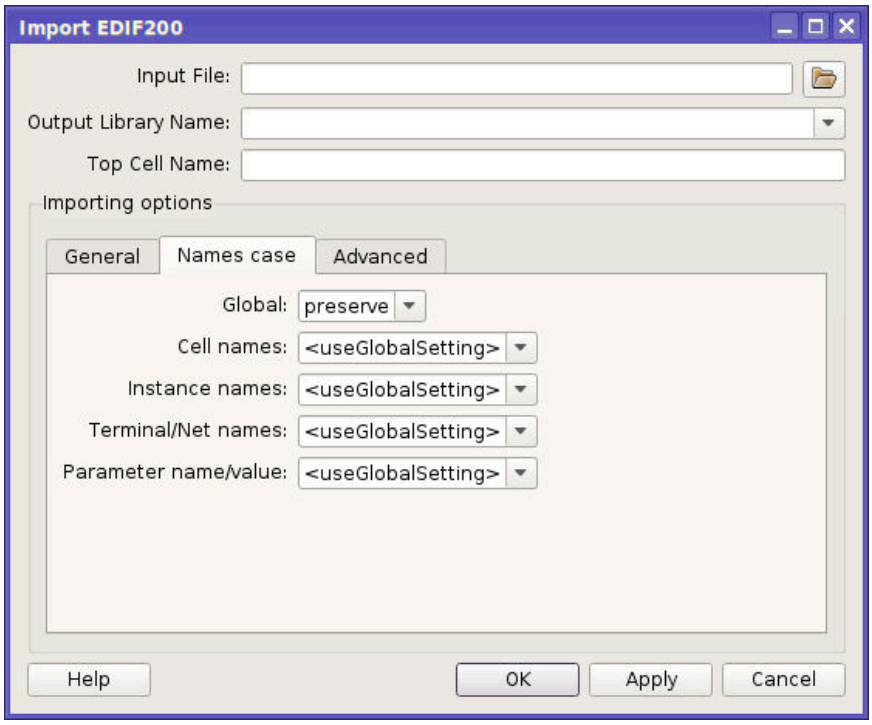


Table 38 Import EDIF200 Settings (Names Case Tab)

Settings	Description
Global	Globally defines the case setting for the design with one of the following options: **preserve** (the default), **upper**, or **lower**.
Cell names	Sets the case of cell names in the design individually. Options are **<useGlobalSetting>** (the default), **upper**, **lower**, or **preserve**. If turned on and then turned off again, the setting reverts to the global setting defined in the **Global** selection field.
Instance names	Sets the case of instance names in the design individually. Options are **<useGlobalSetting>** (the default), **upper**, **lower**, or **preserve**. If turned on and then turned off again, the setting reverts to the global setting defined in the **Global** selection field.
Terminal/Net names	Sets the case of terminal/net names in the design individually. Options are **<useGlobalSetting>** (the default), **upper**, **lower**, or **preserve**. If turned on and then turned off again, the setting reverts to the global setting defined in the **Global** selection field.

Feedback

Settings	Description
Parameter name/value	Sets the case of cell parameter names/values in the design individually. Options are **<useGlobalSetting>** (the default), **upper**, **lower**, or **preserve**. If turned on and then turned off again, the setting reverts to the global setting defined in the **Global** selection field.

5. On the **Advanced** tab (Figure 22), enter settings as described in Table 39.

Figure 22 Import EDIF200 Dialog Box: Advanced Options

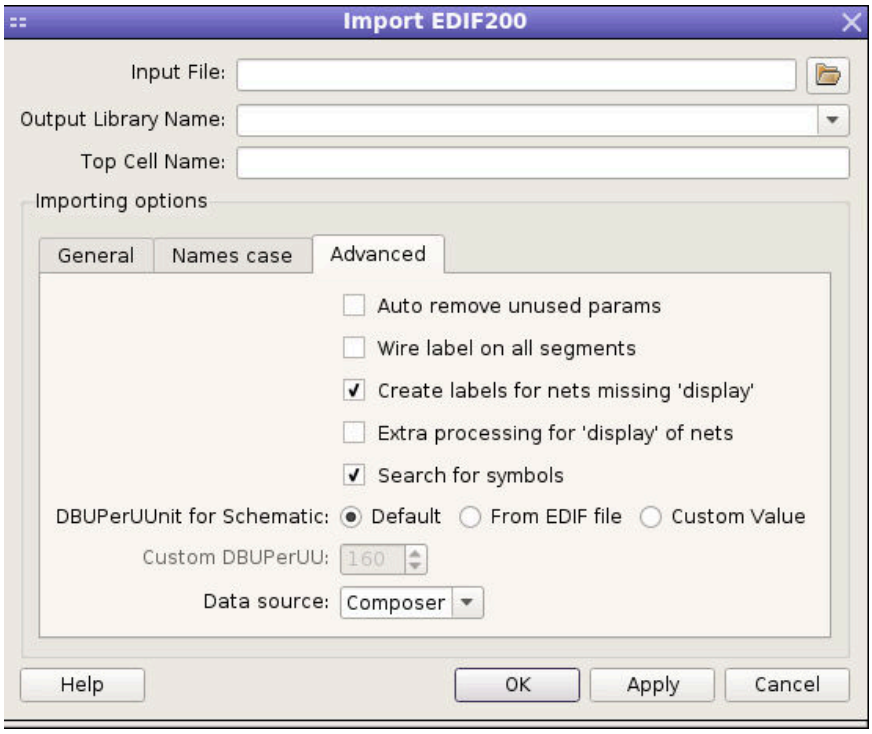


Table 39 Import EDIF200 Settings (Advanced Tab)

Settings	Description
Auto remove unused params	Automatically removes unused parameters. Off by default.
Wire label on all segments	Adds wireLabels to all wire segments of the created schematic view. Off by default.
Create labels for nets missing 'display'	On by default. Creates labels for nets that do not have an EDIF `display` entry. If not set, net labels are generated only based on the EDIF `display` data a net structure has.

Feedback

Settings	Description
Extra processing for 'display' of nets	Off by default. If true, the tool executes more accurate but slower processing to link labels with the closest wires. Tcl preference: dbEDIFImportNetDisplayExtraProcess
Search for symbols	Automatically searches all opened libraries for a symbol view having the same named cell and port matching. On by default.
DBUPerUUnit for Schematic	Controls where to get the value for DBUperUU scale. Options are **Default**, **From EDIF File**, or **Custom Value**. Tcl preference: dbEDIFImportSchematicDBUPerUUSource
Custom DBUPerUU	Specifies the custom value to use if the **DBUPerUUnit for Schematic** option is **Custom Value**. The valid value must be a positive integer. The default is 160. Tcl preference: dbEDIFImportCustomSchematicDBUPerUU
Data source	Specifies vendor-specific data semantics. Options are **Composer** (the default) or **ViewDraw**. Custom Compiler uses this option to correctly map array instance names, sub-bus net names and some parameter names/values. For example, if **Data source** is set to **ViewDraw** and a schematic defined in the EDIF file has a net name MyNet1, Custom Compiler tries to find a bus net with base name MyNet. If found, and the found bus includes bit #1, then the net name MyNet1 is mapped to MyNet<1>. This option is also used to set the array names' braces used in the source EDIF file. If set to **ViewDraw**, square brackets [] are treated as array names; otherwise, angle brackets <> are treated as array names.

6. Click **OK**.

   The system imports the file and creates the designated views.

5

Export

This chapter describes how to export design and library data using the Custom Compiler Console. The data can be exported to IC Compiler, GDSII Stream, OASIS, LEF, DEF, Liberty File, and netlist formats. To protect the proprietary information stored in your Tcl scripts, you can export it in bytecode format.

- Exporting Design Data in GDSII Stream Format
- Exporting Design Data in OASIS Format
- Exporting Design Data in Library Exchange Format (LEF)
- Exporting Design Data in Design Exchange Format (DEF)
- Exporting Design Data in Liberty Library File Format
- Exporting Design Data in Netlist Format

Note:

Exporting Designs and Libraries to the IC Compiler Tool is covered in the *Custom Compiler and IC Compiler Co-Design Assistant User Guide*.

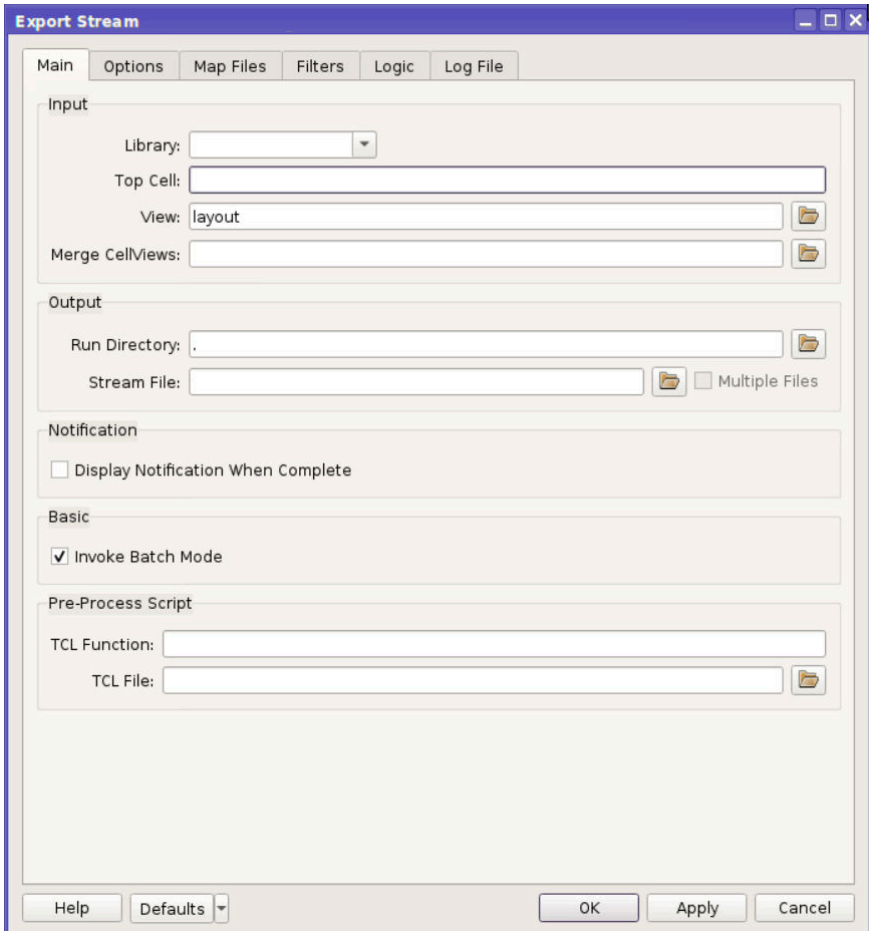
To export technology data instead of design data, use the Technology Manager Tool.

Exporting Design Data in GDSII Stream Format

To export design data in GDSII Stream format:

1. Perform any one of the following to open the **Export Stream** dialog box:
   - From the **Home** tab, choose **File > Export > Stream**.
   - From the toolbar in the Layout Editor, choose **Design > Export Stream**

Feedback

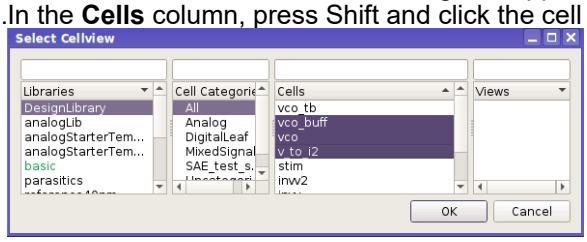


2. On the **Main** tab, enter settings as described in Table 40.

Table 40 Export Stream Settings (Main Tab)

Settings	Description
Input	**Library** contains the design designated for export. **Top Cell** is the top-level cell for export.

Feedback

Settings	Description
	View is the name of the view to export. The default is a layout view. To select multiple cell views simultaneously a. Click . The **Select Cellview** dialog box appears. b. In the **Cells** column, press Shift and click the cell names.  c. Click **OK**. The selected cell names appear in the **Cell** box. Each cellview name is separated by a space. **Merge Cellviews** opens the **Export Stream Merge CellViews Browser** dialog box when you click the folder icon. In the **Export Stream Merge Cellviews Browser** dialog box, choose available libraries/cells/views and use the arrow selectors to add them to the **Selected Cellviews** list to be merged during export.
Output	**Run Directory** is the directory you browse to select, or a new directory you create by entering its name. The default is the current directory. **Stream File** is the GDSII stream file you browse to select and use. Click the folder icon to open the **Select File** dialog box. In the dialog box, the **Files of type** field is filtered by default for formats `.gds`, `.db`, `.strm`, `.Z`, and `.gz`. Select * from the drop-down menu to see all existing files. **Multiple Files**: Creates a stream file for each cell defined in the **Top Cell** box in the **Input** section simultaneously. Tcl preference: `dbExportStreamMultipleFiles`
Notification	**Display Notification When Complete** specifies whether to display a window with completion information after export. Off by default.

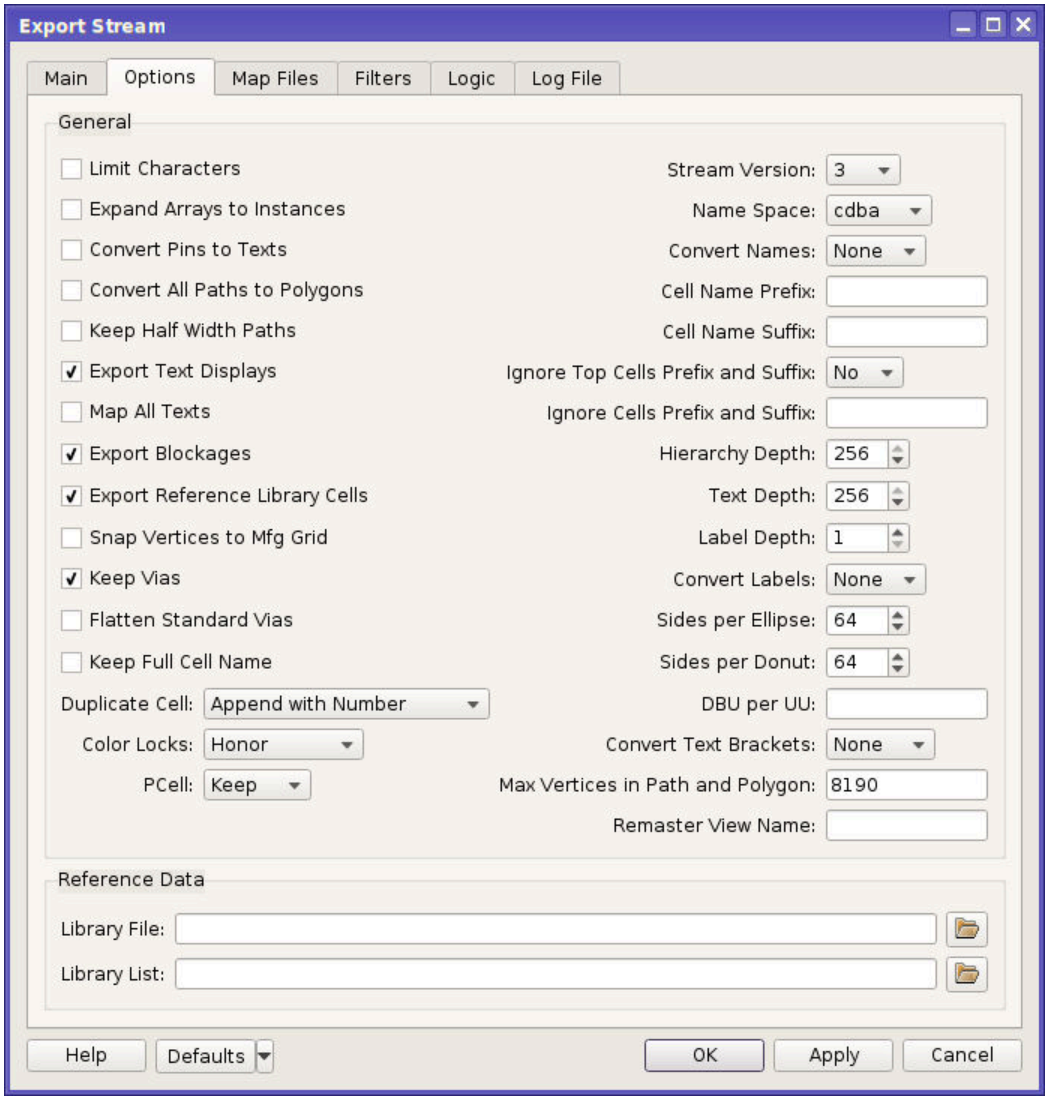
Feedback

Settings	Description
Basic	**Invoke Batch Mode** allows you to choose to run Export Stream with a standalone utility or with a version that is integrated with Custom Compiler. On by default.
Pre-Process Script	**TCL Function** is executed before translation, if specified. The returned values of the Tcl function are: 0 stops the export; 1 continues the export. **TCL File** is a Tcl file sourced during translation.

3. Click the **Options** tab in the **Export Stream** dialog box (Figure 23).

Feedback

Figure 23 Export Stream Dialog Box: Options Tab



4. On the **Options** tab, enter settings as described in Table 41.

Table 41 Export Stream Settings (Options Tab)

Settings	Description
General	**Limit Characters** specifies that the translator conforms to the 32-character limit on structure names. Off by default. **Expand Arrays to Instances** specifies whether to convert arrays to instances. Off by default.

Feedback

Settings	Description
	Convert Pins to Texts specifies whether to convert pin's term name to text. Each shape connected to the pin is used for the location and layer/purpose pair (LPP) of the created text. Off by default. **Convert All Paths to Polygons** imports two-point orthogonal and diagonal path records as `pathSeg` objects. Off by default. **Keep Half Width Paths** specifies whether the translator should keep half-width paths and not convert them to polygons. Off by default. **Export Text Displays**controls translation of `textDisplay` objects. **Map All Texts** specifies whether the translator should attempt to name map oaTexts and oaTextDisplays that are not oaNames. Off by default. **Export Blockages** controls export of blockage objects. Default is to export blockage objects (on). **Export Reference Library Cells** controls translation and output of cells referenced from a library other than the one specified by the Library argument. Default is that these cells are translated. **Snap Vertices to Mfg Grid** specifies whether the translator should snap the object vertices to manufacturing grid during export. Off by default. **Keep Vias** specifies whether the translator should write via information into the `.pcell` file. On by default. **Flatten Standard Vias** specifies if the exportStream translator should convert standard vias into their geometry. Off by default. **Keep Full Cell Name** specifies whether to keep the full name of the cells. When turned off, cell names are shortened during export. Off by default. **Keep View Name** specifies if the exportStream translator should keep the view name. If the view name is not a layout, the translator generates the view map with the `-gdsFileName` option to restore the view name. Off by default.

Settings	Description
	Duplicate Cell specifies what to do when the same cell name is found during export: ◦ **Append with Number**: (default) Append the duplicate cell name with a sequence number. ◦ **Append with Library Name**: Append the duplicate cell name with a source library name. ◦ **Skip Duplicate**: Skips the duplicate cell name when the same cell name is found during export stream. After duplicating the cell name, the tool appends the suffix with the view name if the view name is not a layout. However, if the cell name still duplicates even after appending the suffix with the view name, the **Duplicate Cell** option specifies that the translator should append the duplicate cell name with a sequence number during export.

Feedback

Settings	Description
	Color Locks specifies which mapping lines in the layer mapping file are used to map shapes with a mask color attribute. A color specification is present in the layer mapping file if the data type number is followed by the keyword `color` with a colon (:) and a color name. Optionally, this entry can be followed by one of the keywords `locked` or `unlocked`. `locked` refers to the anchor bit in the color value for the shape. Options for **Color Locks** are: ◦ **Ignore**: All colored shapes are mapped with the line that has neither keyword. ◦ **Strict**: Anchor bit is used to select either `locked` or `unlocked` from the layer map. ◦ **Full**: Mapping is compatible with Cadence® Virtuoso®, with the following behavior: The keyword `colorAnnotate` after the color specification in the layer map is used to ignore a whole line of layer mapping during GDSII Stream export and the same mapping line is handled during import. ◦ **Honor**: (default) Mapping is compatible with Cadence® Virtuoso®, with the following behavior: When the layer map does not have a color mapping rule in to match color shapes, the Custom Compiler tool tries to match the mapping rule without a color specified. ◦ **DisableColor**: Layer mapping lines with a color attribute in the layer mapping file are ignored. Any shape with color does not map those mapping lines with color attributes. Only mapping lines without a color attribute is used for layer mapping. Example: M1 drawing 3 0 M1 drawing 3 11 color:red locked M1 drawing 3 12 color:red unlocked When the **DisableColor** option is set, colored/colorless shapes on M1 are mapped to 3 0. ◦ **Lock Only**: The shapes of the colored layer that are not colored or are not locked, are filtered out during export. **Lock Only** is available only when you set the `dbExportStreamShowLockOnly` preference to `true`. The default setting is `false`. **PCell** options control conversion of PCells into their geometry. The default is **Keep**. The **None** option means no conversion. The **Flatten** option flattens PCells, editable PCells, and MCells, obeying the hierarchy limit. The **Keep** option preserves the information of PCells, editable PCells, MCells, and MPP in the GDS stream file, so that they can be restored during import using **Import > Stream > Options > Restore PCells**. Abutment information is stored in a side file and can also be restored during import.

Settings	Description
	Stream Version specifies the GDSII Stream format version for the translator to use. Rectangles are written as boundary regardless of which version is specified. **Name Space** is the namespace mapping for text displays: *cdba*, *native*, *lef*, *def*, *verilog*, *vhdl*, *spef*, *spice*, or *spf.* Default is *cdba*. **Convert Names** controls conversion of cell and instance names to either uppercase, lowercase, or no conversion. Default is no conversion. **Cell Name Prefix** specifies the string prefixed to every structure name written in the Stream file. The GUI validates that the string entered is a legal CDBA scalar name, and that it also contains no spaces. If left empty, no option is used. Note that this option does not exist in `oa2strm`. **Cell Name Suffix** specifies the string suffixed to every structure name written in the Stream file. The GUI validates that the string entered is a legal CDBA scalar name, and that it also contains no spaces. If left empty, no option is used. Note that this option does not exist in `oa2strm`. **Ignore Top Cells Prefix and Suffix** Specifies whether top cells listed in **Main > Input > Top Cell** should be added with a prefix or suffix. Default is No. **Ignore Cells Prefix and Suffix** Ignores the prefixes and suffixes of the cells listed in this field. Use blank space to separate cells. **Hierarchy Depth** is the number of hierarchy levels to translate. Default is 256. **Text Depth** specifies the depth to which the translator should export texts. Default is 256. **Label Depth** specifies the depth to which the translator provides labels. Default is 1. **Convert Labels** controls conversion of net and terminal names to either uppercase, lowercase, or no conversion. Default is no conversion. **Sides per Ellipse** specifies the number of edges used for ellipse to polygon conversion. Default is 64. **Sides per Donut** specifies the number of edges used for donut to polygon conversion. Default is 64.

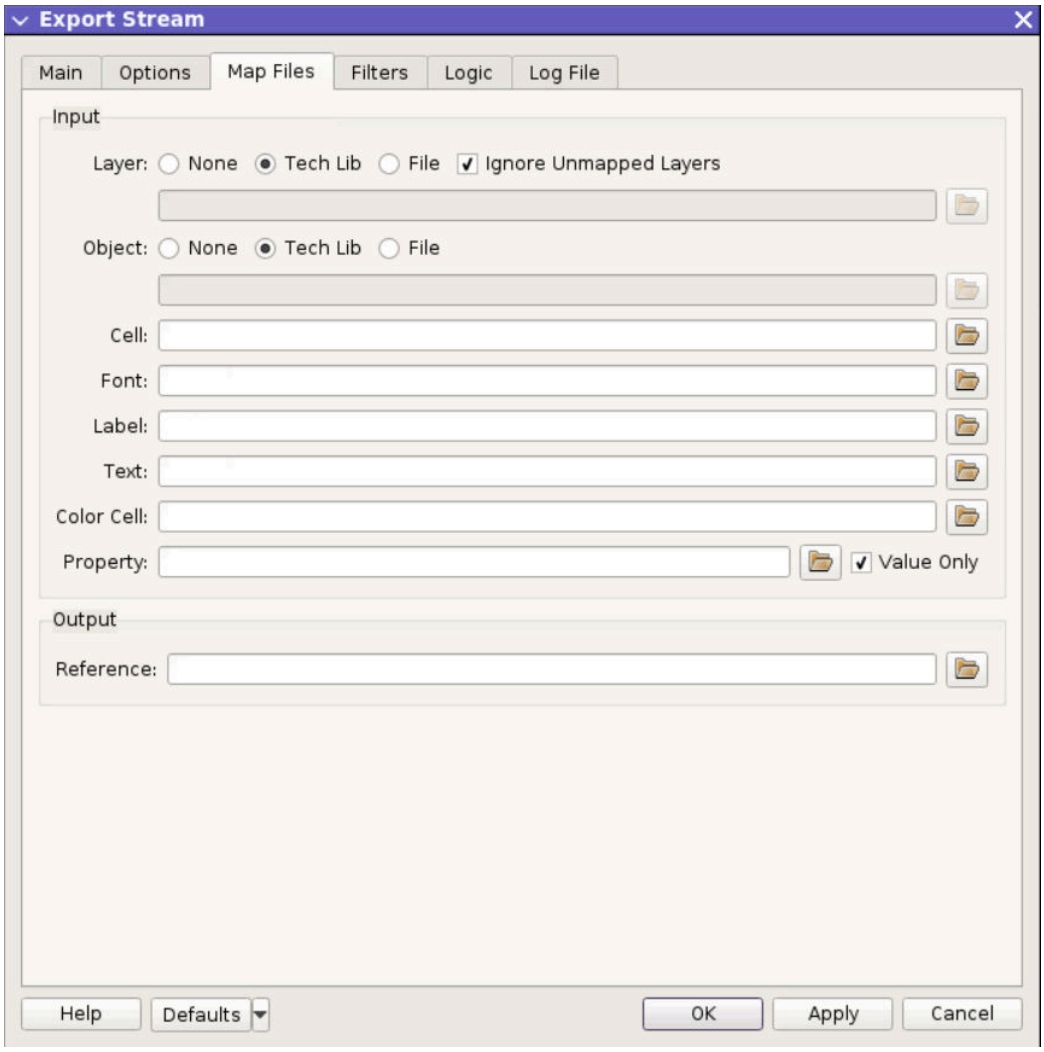
Feedback

Settings	Description
	DBU Per UU is the value for database units per user units. Used for scaling addition computations while translating OA database to GDSII without changing the source OA database. **Convert Text Brackets** specifies whether or how to convert text brackets during export: ◦ **None**: (default) Do not convert text brackets. ◦ **Angle**: Convert square text brackets, "[" and "]", to angle text brackets, "<" and ">". ◦ **Square**: Convert angle text brackets, "<" and ">", to square text brackets, "[" and "]". **Max Vertices in Path and Polygon** limits the maximal vertices number in path and polygon. Split it if vertices number is over the limit. Default is 8190. **Remaster View Name** changes the instance master view name from layout to the specified view name. For example, if instances are bound to abstract views and **Remaster View Name** is set to layout, all instances are bound with the layout view during the export stream process, if the layout view exists in the instance master's library. This option is applied in both the color shift and non-color shift flows. It is not applied for PCell and customer via.
Reference Data	**Library File** The file you browse to select. This file has the names of reference libraries for the translation process to use when it binds cells referenced during export. **Library List** The list of libraries with cells referenced in the design. Click the folder icon to open the **Export Stream Reference Library Browser**.
	◦ Sort from the libraries **Available** to the libraries **Selected** using the arrow buttons between the two areas.
	◦ Where there are many libraries in the selected list, you can use the arrow buttons at the right edge to scroll.

5. Click the **Map Files** tab in the **Export Stream** dialog box (Figure 24).

Feedback

Figure 24 Export Stream Dialog Box: Map Files Tab



6. On the **Map Files** tab, enter settings as described in Table 42.

Feedback

Table 42 Export Stream Settings (Map Files Tab)

Settings	Description
Input	**Layer** specifies the required layer mapping mode: ◦ **None**: Do not use a layer mapping file for layer/purpose mapping. Instead, use the OpenAccess layer number and purpose number for the GDSII layer number and datatype. You can also use the layer number and purpose number of particular layer names and purpose names of OpenAccess objects (see Table 10) for the GDS layer number and data type. ◦ **Tech Lib**: (default) Use the layer mapping files in the technology library, merging them if more than one exists. Maps the OpenAccess shapes to all the GDS layer numbers and datatypes found in the combined mapping file. ◦ **File**: Use the layer mapping file you browse to select. If **Ignore Unmapped Layers** is not selected and the OpenAccess layer number and purpose number are not matched in the layer mapping file, use the OpenAccess layer number and purpose number for the GDSII layer number and data type. You can also use the layer number and purpose number of particular layer names and purpose names of OpenAccess objects (see Table 10) for the GDS layer number and data type. If **Ignore Unmapped Layers** is selected (default), the OpenAccess shape/object is filtered out. **Object** specifies the required object mapping mode: ◦ **None**: Do not use an object mapping file. ◦ **Tech Lib**: (default) Use the object mapping files in the technology library, merging them if more than one exists. Maps OpenAccess objects to all the GDS layer numbers and datatypes found in the combined mapping file. ◦ **File**: Use the object mapping file you browse to select. Specifies methods of mapping Boundaries and `LayerBlockages` between GDSII Stream shapes and OpenAccess databases. **Cell** specifies the cell mapping file you browse to select. Maps cell and port names. **Font** specifies the font mapping file you browse to select. Maps font IDs to their OpenAccess Names. **Label** specifies the label mapping file you browse to select. **Text** specifies the text mapping file you browse to select. **Color Cell** specifies the color cell mapping file you browse to select. **Property** specifies the property mapping file you browse to select. Check **Value Only** to output only property values. On by default.

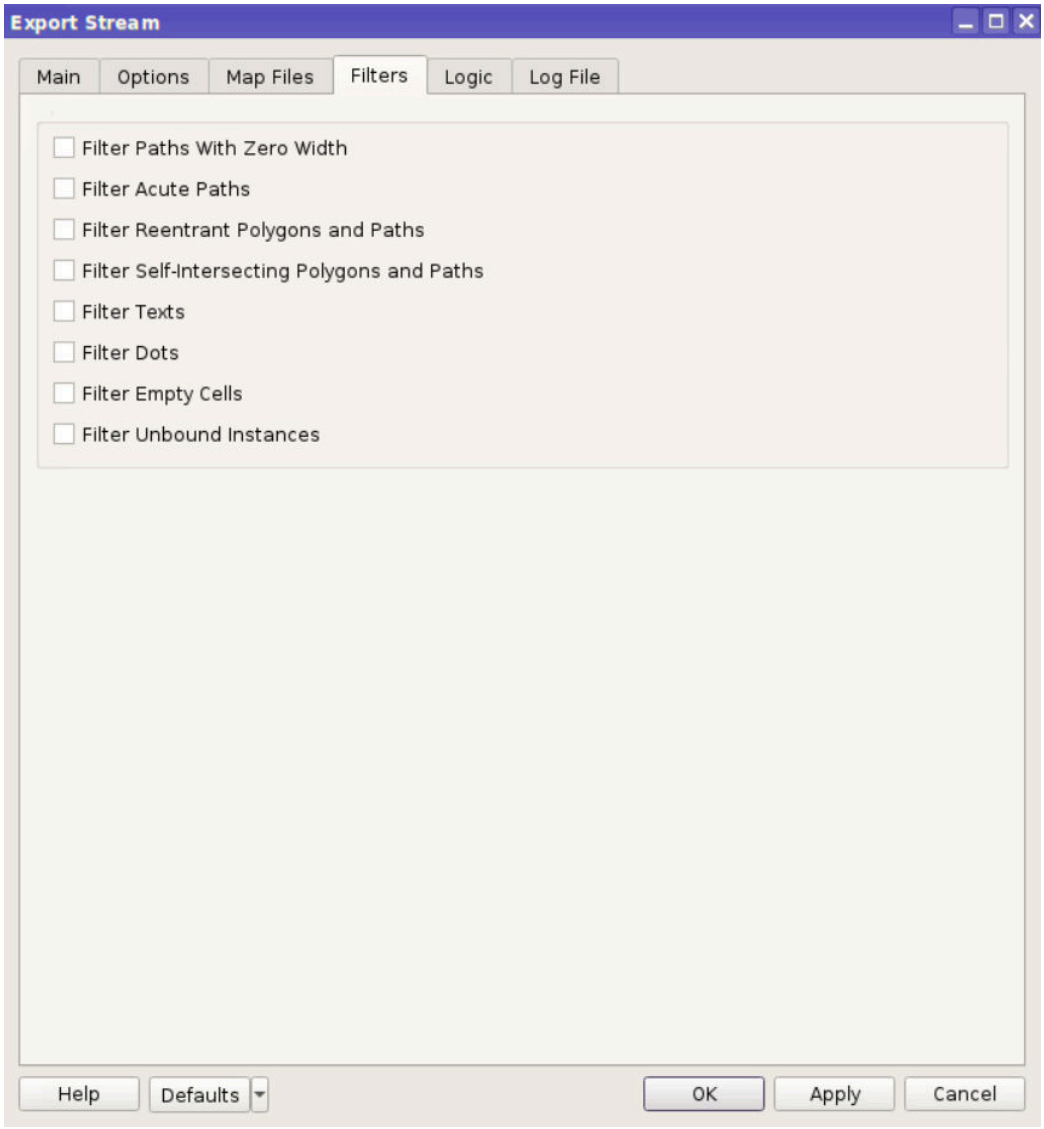
Feedback

Settings	Description
Output	**Reference** specifies the output reference mapping file. Specify a path to the file in the text box. This file is important when you export a GDSII file from Custom Compiler, run IC Validator DRC or LVS, and then cross-probe or highlight these violations to the Custom Compiler Layout Editor. In this flow, ICV reports violations with respect to the GDSII library/cell object context, but Custom Compiler must identify and probe the corresponding objects and coordinates in the OpenAccess lib/cell/view context. This reference mapping file stores the mapping relationship between the OpenAccess objects, (such as vias, PCells, lib/cell/view hierarchy, and so on) and their equivalent GDSII objects. Having this file ensures that you can cross-probe violations from a GDSII-based ICV run into Custom Compiler LE; this is the supported and recommended flow. Without this file, cross-probing might still be possible but is not guaranteed to work reliably. The browse icon is disabled if the current file path does not exist.

7. Click the **Filters** tab in the **Export Stream** dialog box (Figure 25).

Feedback

Figure 25 Export Stream Dialog Box: Filters Tab



8. On the **Filters** tab, enter settings as described in Table 43.

Table 43 Export Stream Settings (Filters Tab)

Settings	Description
Filters	**Filter Paths With Zero Width** specifies whether or not to filter paths with zero width. Tcl preference: `dbExportStreamFilterZeroWidth`

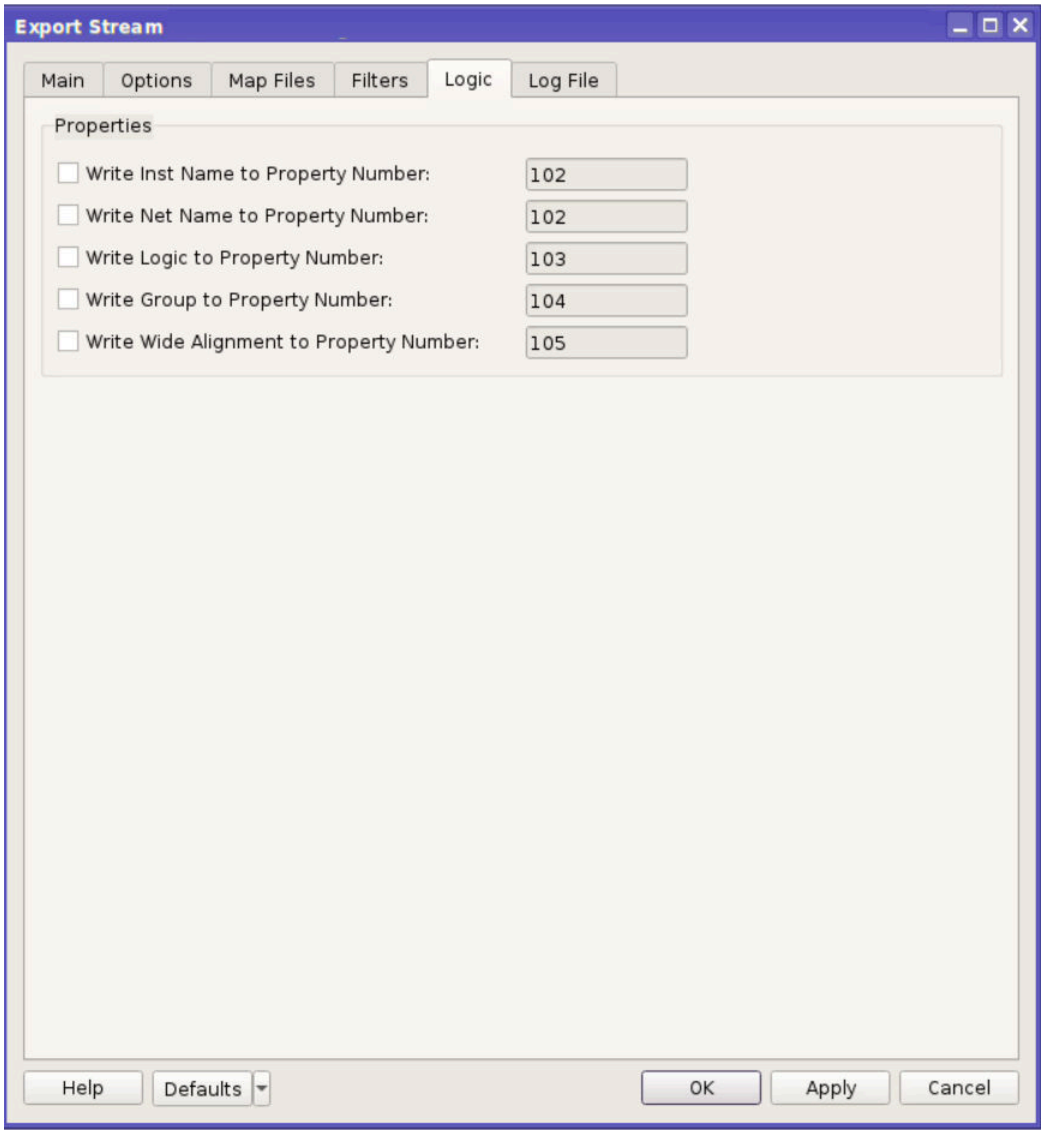
Feedback

Settings	Description
	Filter Acute Paths specifies whether or not to filter paths with acute angles. Tcl preference: `dbExportStreamFilterAcutePaths` **Filter Reentrant Polygons and Paths** specifies whether or not to filter paths with reentrant polygons and paths. Tcl preference: `dbExportStreamFilterReentrantPgns` **Filter Self-Intersecting Polygons and Paths** specifies whether or not to filter paths with self-intersecting polygons and paths. Tcl preference: `dbExportStreamFilterSelfIntersectingPgns` **Filter Texts** specifies whether or not to filter text. Tcl preference: `dbExportStreamFilterText` **Filter Dots** specifies whether or not to filter dots. Tcl preference: `dbExportStreamFilterDots` **Filter Empty Cells** specifies whether or not to filter the empty cells and the associated unbound instances. Tcl preference: `dbExportStreamFilterEmptyCells` **Filter Unbound Instances** specifies whether or not to filter unbound instances. Tcl preference: `dbExportStreamFilterUnboundInsts`

9. Click the **Logic** tab in the **Export Stream** dialog box (Figure 26).

Feedback

Figure 26 Export Stream Dialog Box: Logic Tab



10. On the **Logic** tab, enter settings as described in Table 44.

Table 44 Export Stream Settings (Logic tab)

Settings	Description
Properties	**Write Inst Name to Property Number** converts the instance name to the output GDSII file using the specified GDSII property number. Specify the property number in the range of 1 to 127. The default is 102. Tcl preference: `dbExportStreamPropNumForInstName`

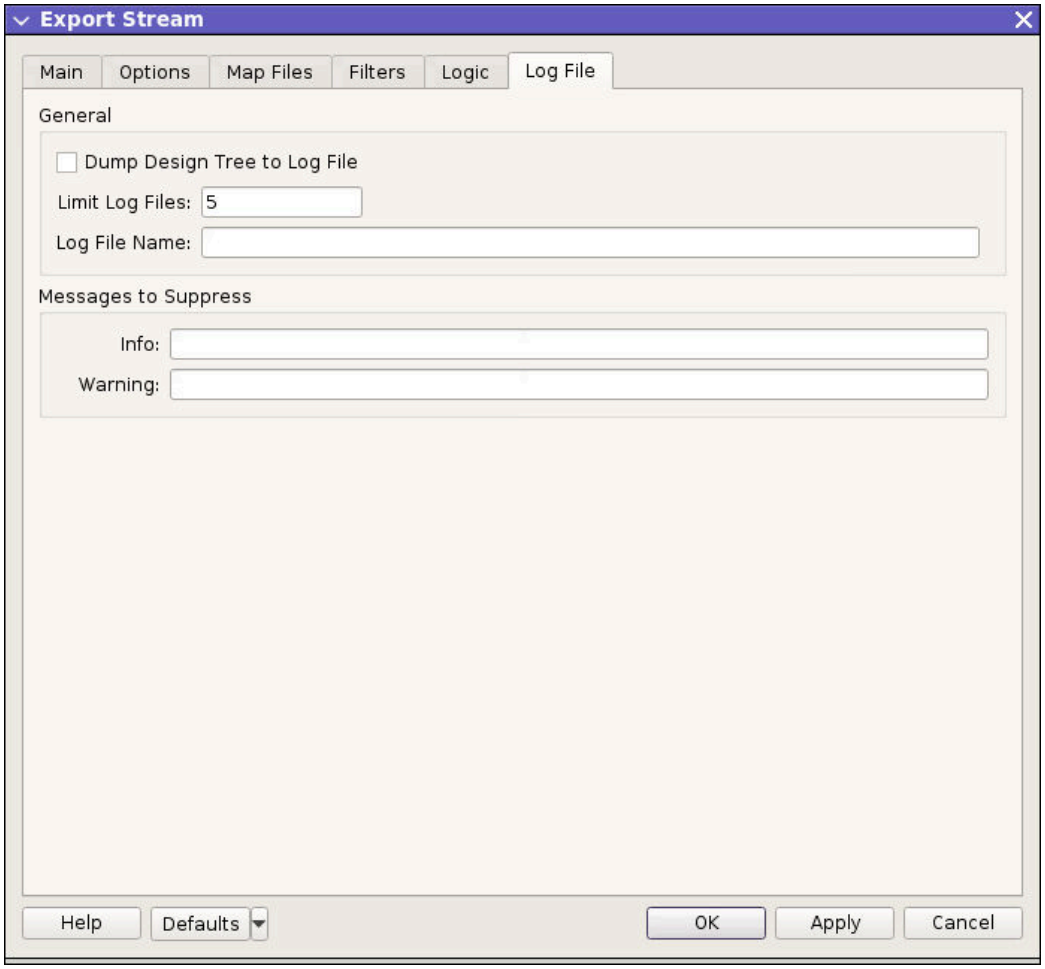
Feedback

Settings	Description
	Write Net Name to Property Number converts the net name to the output GDSII file using the specified GDSII property number. Specify the property number in the range of 1 to 127. The default is 102. Tcl preference: `dbExportStreamPropNumForNetName` **Write Logic to Property Number** converts the logic information to the output GDSII file using the specified GDSII property number. If the length of the property data is over 125 bytes, it is compressed and stored to property section of the output GDSII file. If the compressed data is also over 125 bytes, it is split into two or more properties. Specify the property number in the range of 1 to 127. The default is 103. Tcl preference: `dbExportStreamPropNumForLogicName` **Write Group to Property Number** converts the synchronous group and the figure group information (name, status and origin) to the output GDSII file using a specified GDSII property number. Specify the property number in the range of 1 to 127. The default is 104. Tcl preference: `dbExportStreamKeepGroupName` **Write Wide Alignment to Property Number** converts the wide alignment to the output GDSII file. Specify the property number in the range of 1 to 127. The default is 105. Tcl preference: `dbExportStreamKeepWideAlignment`

11. Click the **Log File** tab in the **Export Stream** dialog box (Figure 27).

Feedback

Figure 27 Export Stream Dialog Box: Log File Tab



12. On the **Log File** tab, enter settings as described in Table 45.

Table 45 Export Stream Settings (Log File Tab)

Settings	Description
General	The **Dump Design Tree to Log File** option writes the design tree information to a log file during a stream import. Off by default. **Limit Log Files** limits the maximum number of backup log files. Default is 5. **Log File Name** specifies the name of the log file.
Messages to Suppress	**Info** is the list of informational messages to suppress during export.

Feedback

Settings	Description
	Warning is the list of warning messages to suppress during export.

13. Click **OK**.

    The translator exports the design in GDSII Stream format.

    If you have set the `dbExportStreamCheckSave` preference to `true`, the **Save Design** dialog box appears if there are designs that are not saved. Where, the design can be based on the specified library, cell, view, and hierarchy.

Settings	Description
Save	Saves the design and continues exporting the design.
Ignore	Does not save the design but continues exporting the design.
Cancel	Does not save the design and cancels exporting the design.

The `onPostStreamExit` callback is triggered at the end of the **Export Stream** run.

The `exportStream` command exports the design in GDSII Stream format.

Syntax

```
db::exportStream -gds <string> -lib <string> [-batchMode <bool>]
 [-showResult <bool>] [-libDefFile <string>]
[-acutePathToPolygon <bool>] [-autoMapObjects <unsigned int>]
 [-blockageType <unsigned int>] [-cell [<string>]]
[-cellMap <string>] [-cellNamePrefix <string>] [-cellNameSuffix <string>]
 [-charLimit <bool>] [-colorMapping <enum>]
[-convertPinToText <bool>] [-convertNetFromPinToText <bool>]
 [-convertTextBrackets <enum>] [-dbuPerUU <int>]
[-donutNumSides <unsigned int>] [-dumpDesignTree <bool>] [-dumpRefLib
 <bool>] [-duplicateCell <enum>] [-ellipseNumSides <unsigned int>]
[-expandArrayInsts <bool>] [-filterAcutePaths <bool>] [-filterDots
 <bool>] [-filterEmptyCells <bool>] [-filterPlacementStatuses <string>]
 [-filterReentrantPgns <bool>]
```

Feedback

```
[-filterSelfIntersectingPgns <bool>] [-filterText <bool>]
 [-filterUnboundInsts <bool>] [-filterZeroWidth <bool>] [-flattenPcells
 <bool>]
[-flattenVias <bool>] [-fontMap <string>] [-hierDepth <int>]
 [-ignoreCellsAffix <string>] [-ignoreTopCellsAffix <bool>]
[-keepAbuts <bool>] [-keepFullCellName <bool>] [-keepViewName <bool>]
 [-keepHalfWidthPaths <bool>] [-keepPcells <bool>]
[-keepVias <bool>] [-labelDepth <unsigned int>] [-labelMap <string>]
 [-lakerLicense <string>] [-layerMap [<string>]]
[-logFile <string>] [-mapAllText <bool>] [-maxVertices <unsigned int>]
 [-mergeCellviews <string>] [-newMaxVerticesFunction <bool>]
[-newMaxVerticesRule <string>] [-noAutoBackupFile <bool>] [-noConvertPath
 <bool>] [-noExportRefLibCells <bool>]
[-noInfo [<unsigned int>]] [-noOutputBlockages <bool>]
 [-noOutputBoundaries <bool>] [-noOutputTextDisplays <bool>]
[-noRoundPathToPolygon <bool>] [-noWarning [<unsigned int>]]
 [-notExternalizeShapes <bool>] [-outputBBoxInfo <bool>]
[-sortLayer <bool>] [-objectMap <string>] [-pathToPolygon <bool>]
 [-propMap <string>] [-propValueOnly <bool>]
[-propertyInst <unsigned int>] [-propertyLogic <unsigned int>]
 [-propertyNet <unsigned int>] [-propertyVia <unsigned int>]
[-propertyWideAlignment <unsigned int>] [-propertyVia <unsigned
 int>][-rectAsBoundary <bool>] [-refLibList <string>]
[-refLibs [<string>]] [-referenceMap <string>] [-roundPathToPolygon
 <bool>] [-snapToMfgGrid <bool>] [-tclFile <string>]
[-tclFunction [<string>]] [-techLib <string>] [-text <string>]
 [-textDepth <int>] [-textHeight <enum>] [-textMap [<string>]]
[-toLower <bool>] [-toLowerLabel <bool>] [-toUpper <bool>] [-toUpperLabel
 <bool>] [-unmappedLayersUseOALayers <bool>]
[-useTechLayerMap <bool>] [-useTechObjectMap <bool>] [-ver <unsigned
 int>] [-view <string>]
```

For details of the `exportStream` command, type `exportStream -help`.

Note:

When `-ascii` is specified, the following arguments are not used for importStream:

```
autoMapObjects, keepPcells, keepSubMasters, keepVias,
 pcellFreeze, propMap, propValueOnly, propertyInst,
 propertyLogic, propertyNet, propertyWideAlignment, viaMap
```

The format of `-ascii`when supporting Rectangle, Polygon, Path, Text objects:

```
#
#Sample: Stream ASCII dump file
#
#openLib("libname",DBUPerUU,"UserUnit");
# openCell("cellName1");
#  addRect(layer,dtype,x1,y1,x2,y2);
# closeCell("cellName1");
# openCell("cellName2");
```

Feedback

```
#  addRect(layer,dtype,x1,y1,x2,y2);
#  addPoly(layer,dtype,nPoints)
#   (x1,y1) (x2,y2) (x3,y3) (x4,y4)
#   (x5,y5) (x6,y6) ......;
#  addPath(layer,dtype,width,style,nPoints,[bgnExt,endExt])
#   (x1,y1) (x2,y2) (x3,y3) (x4,y4)
#   (x5,y5) (x6,y6) ......;
#
 addText(layer,ttype,"textString",mag,x0,y0,orient,justify,fon
t);
#  addCellInst("masterName",x0,y0,angle,mirror,mag);
#
 addArrayInst("masterName",x0,y0,angle,mirror,mag,rows,columns,r
owSpacing,columnSpacing);
# closeCell("cellName2");
#closeLib("libName");
#
##Notes: ##
## orient = 1:R0 2:R90  3:R180 4:R270 5:MY 6:MYR90 7:MX 8:MXR90
## justify= 1:UpperLeft   2:CenterLeft 3:LowerLeft
 4:UpperCenter 5:CenterCenter
##          6:LowerCenter 7:UpperRight 8:CenterRight
 9:LowerRight
## font   = 0:stick 1:euroStyle 2:gothic 3:math
#
```

Ellipse is translated to polygon.

Note:

For information about Tcl commands to export design data in GDSII Stream format, see the *Custom Compiler Tcl Reference Manual* for details.

See Also

- Layer Mapping Files
- Supplying Mapping Files for GDSII Stream and OASIS Format Translations

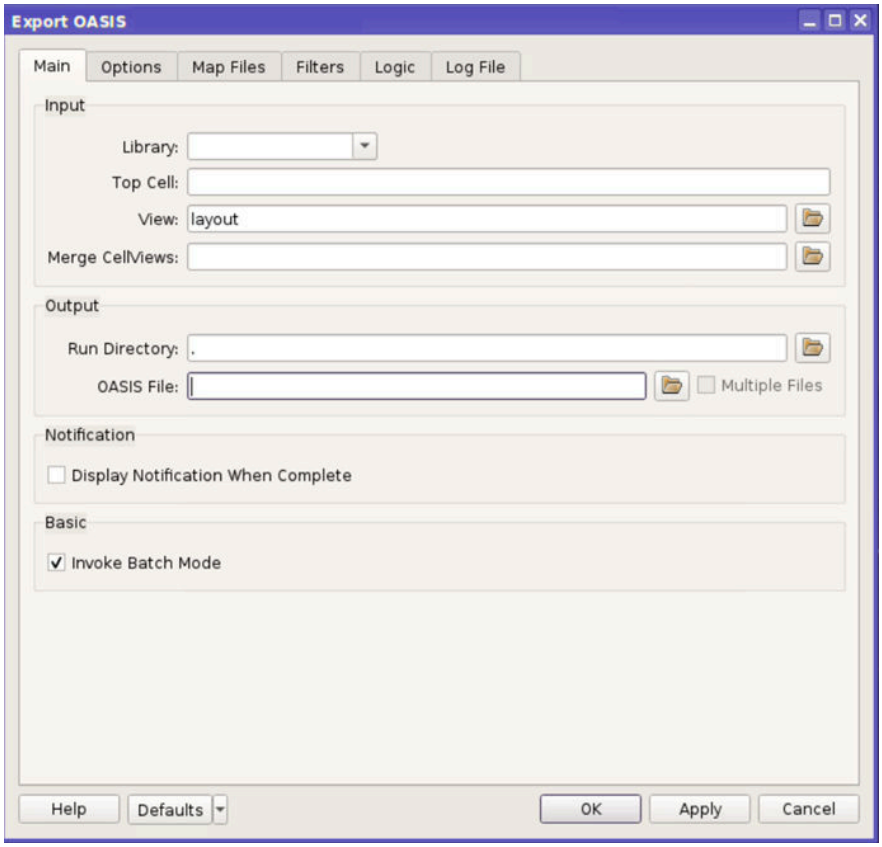
Exporting Design Data in OASIS Format

To export design data in OASIS format:

1. Perform any one of the following to open the **Export OASIS** dialog box:
   - From the **Home** tab, choose **File > Export > OASIS**.
   - From the toolbar in the Layout Editor, choose **Design > Export OASIS**

Feedback

Figure 28 Export OASIS Dialog Box: Main Tab

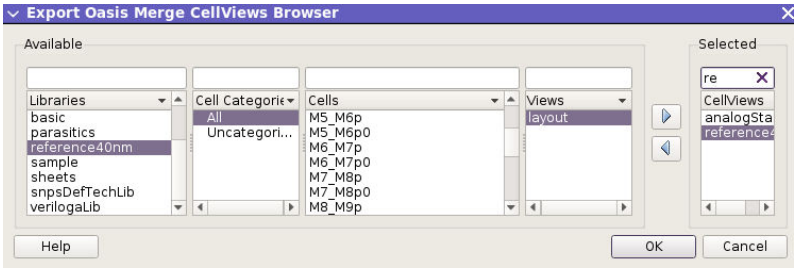
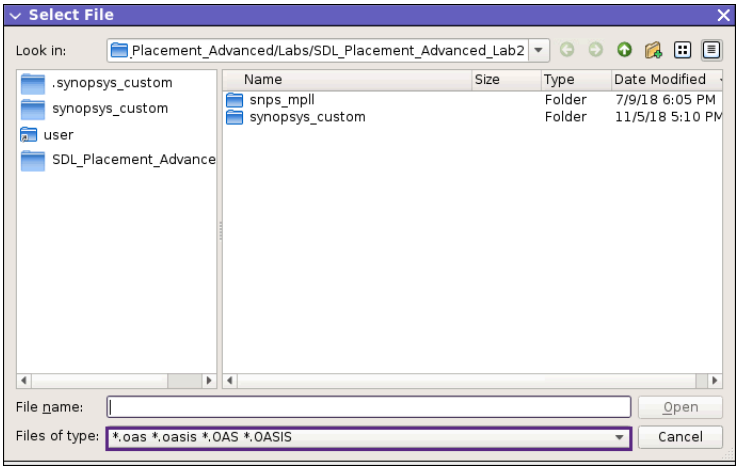


2. On the **Main** tab, enter settings as described in Table 46.

Table 46 Export OASIS Settings (Main Tab)

Settings	Description
Input	**Library** contains the design designated for export. **Top Cell** is the top-level cell for export. **View** is the name of the view to export.

Feedback

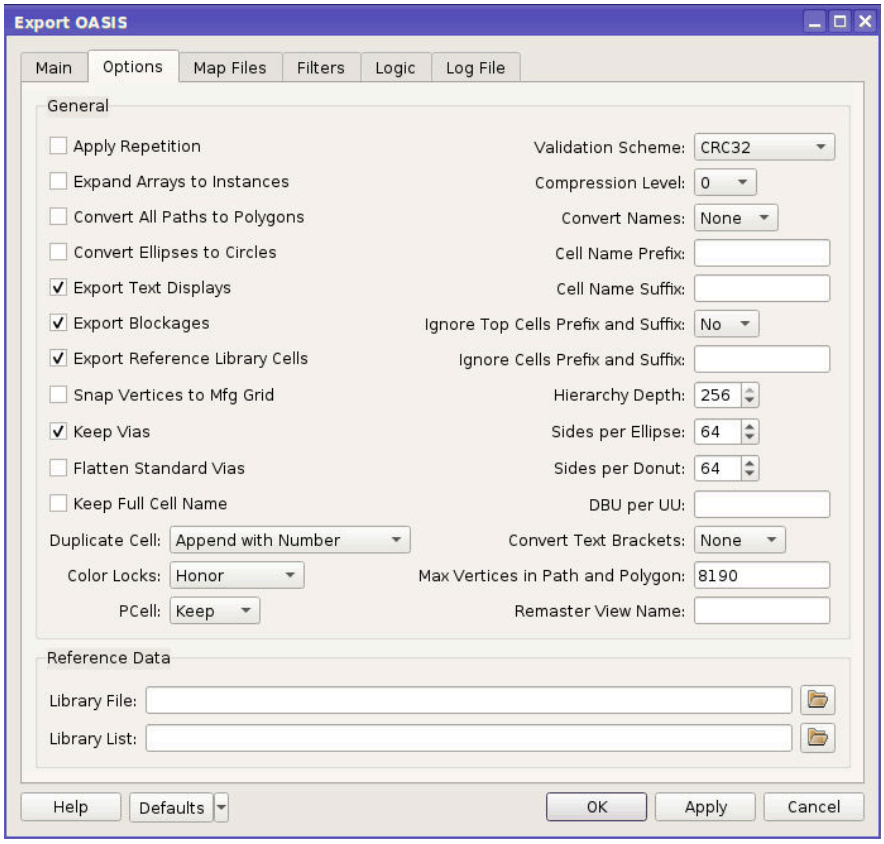
Settings	Description
	Merge CellViews is the field you use to specify the list of cell views to export, to be merged to the top cellview. The cellviews are separated with spaces. Each cellview is specified using the following format: libName/cellName/viewName. `Preference: dbExportOasisMergeCellviews` To generate the list of cellviews, click the browse button. The **Export Oasis Merge Cellviews Browser** dialog box opens. Select the cell views to merge, and click **OK**. 
Output	**Run Directory** is the directory you browse to select, or a new directory you create by entering its name. The default is the current directory. **OASIS File** is the OASIS file you browse to select and use. Click the folder icon to open the **Select File** dialog box. To find OASIS files more quickly, select the ***.oas *.oasis *.OAS *.OASIS** filter in the **Files of type** box.  **Multiple Files**: Creates an OASIS file for each cell defined in the **Top Cell** box in the **Input** section simultaneously. Tcl preference: `dbExportOasisMultipleFiles`

Feedback

Settings	Description
Notification	**Display Notification When Complete** When selected, the **Export OASIS Results** dialog box appears after export. Click **View Logs** to see the log file, or **OK** to exit the dialog box.
Basic	**Invoke Batch Mode** When selected (the default), runs exportOasis integrated with the Custom Compiler tool. Uncheck this option to run as a standalone utility.

3. Click the **Options** tab in the **Export OASIS** dialog box (Figure 29).

Figure 29 Export OASIS Dialog Box: Options Tab



4. On the **Options** tab, enter settings as described in Table 47.

Feedback

Table 47 Export OASIS Settings (Options Tab)

Settings	Description
General	**Apply Repetition** exports repeated instances, shapes, and text in --repetition mode so that they occupy less disk space. Off by default. **Expand Arrays to Instances** specifies whether to convert arrays to instances. Off by default. **Convert All Paths to Polygons** exports paths as a polygon describing the complete shape boundary rather than as a path object. Off by default. **Convert Ellipses to Circles** exports the conversion from circles to polygons. If this option is disabled, circles remain circles during export. Off by default. **Export Text Displays** controls translation of `textDisplay` objects. On by default. **Export Blockages** controls export of blockage objects. Default is to export blockage objects (on). **Export Reference Library Cells** controls translation and output of cells referenced from a library other than the one specified by the Library argument. Default is that these cells are translated. **Snap Vertices to Mfg Grid** specifies whether the translator should snap the object vertices to manufacturing grid during export. Off by default. **Keep Vias** specifies whether the translator should write via information into the `.pcell` file. On by default. **Flatten Standard Vias** specifies whether the translator should convert standard vias into their geometry. Off by default. **Keep Full Cell Name** specifies whether to keep the full name of the cells. When turned off, cell names are shortened during export. Off by default.

Feedback

Settings	Description
	Duplicate Cell specifies what the translator should do during export: ◦ **Append with Number**: (default) Append the duplicate cell name with a sequence number. ◦ **Append with Library Name**: Append the duplicate cell name with a source library name. ◦ **Skip Duplicate**: Skips the duplicate cell name when the same cell name is found during export stream. After duplicating the cell name, the tool appends the suffix with the view name if the view name is not a layout. However, if the cell name still duplicates even after appending the suffix with the view name, the **Duplicate Cell** option specifies that the translator should append the duplicate cell name with a sequence number during export.

Feedback

Settings	Description
	Color Locks specifies which mapping lines in the layer mapping file are used to map shapes with a mask color attribute. A color specification is present in the layer mapping file if the data type number is followed by the keyword `color` with a colon (:) and a color name. Optionally, this entry can be followed by one of the keywords `locked` or `unlocked`. `locked` refers to the anchor bit in the color value for the shape. Options for **Color Locks** are: ◦ **Ignore**: All colored shapes are mapped with the line that has neither keyword. ◦ **Strict**: Anchor bit is used to select either `locked` or `unlocked` from the layer map. ◦ **Full**: Mapping is compatible with Cadence® Virtuoso®, with the following behavior: The keyword `colorAnnotate` after the color specification in the layer map is used to ignore a whole line of layer mapping during GDSII Stream export and the same mapping line is handled during import. ◦ **Honor**: (default) Mapping is compatible with Cadence® Virtuoso®, with the following behavior: When the layer map does not have a color mapping rule in to match color shapes, the Custom Compiler tool tries to match the mapping rule without a color specified. ◦ **DisableColor**: Layer mapping lines with a color attribute in the layer mapping file are ignored. Any shape with color does not map those mapping lines with color attributes. Only mapping lines without a color attribute is used for layer mapping. Example: M1 drawing 3 0 M1 drawing 3 11 color:red locked M1 drawing 3 12 color:red unlocked When the **DisableColor** option is set, colored/colorless shapes on M1 are mapped to 3 0. ◦ **Lock Only**: The shapes of the colored layer that are not colored or are not locked, are filtered out during export. **Lock Only** is available only when you set the `dbExportOasisShowLockOnly` preference to `true`. The default setting is `false`. The **PCell** options control the conversion of PCells into their geometry: ◦ **None** means no conversion. ◦ **Flatten** flattens PCells. ◦ **Keep** specifies whether the translator should output PCell information for a later import operation. On by default. Starting with the P-2019.06 release, **Keep** also specifies whether the translator should output abutment information provided the Tcl preference, `dbExportOasisKeepAbutments`, is set to `true`.

Feedback

Settings	Description
	Validation Scheme specifies the validation scheme to use. Valid values are **None** for no validation, **CRC32** (default), or **CHECKSUM32**. **Compression Level** is the level of compression used when writing the file. 0 is the lowest and 9 is the highest level of compression. Default is 0. **Convert Names** Specifies translator conversion of cell and instance name text to one of the following: ◦ **None**: (default) Do not convert names ◦ **Lower**: Lowercase ◦ **Upper**: Uppercase **Cell Name Prefix** specifies a string that is prefixed to every structure name written in the stream file. The GUI validates that the string entered is a legal CDBA scalar name, and that it also contains no spaces. If left empty, no option is used. Note that this option does not exist in `oa2strm`. **Cell Name Suffix** specifies a string that is suffixed to every cell name. The GUI validates that the string entered is a legal CDBA scalar name, and that it also contains no spaces. If left empty, no option is used. **Ignore Top Cells Prefix and Suffix** Specifies whether top cells listed in **Main > Input > Top Cell** should be added with a prefix or suffix. Default is No. **Ignore Cells Prefix and Suffix** Ignores the prefixes and suffixes of the cells listed in this field. Use blank space to separate cells. **Hierarchy Depth** is the number of hierarchy levels to translate. Default is 20. **Sides per Ellipse** specifies the number of edges used for ellipse to polygon conversion. Default is 64. Note: If ellipses are not converted to polygons and the ellipses are circles and the diameters are even, they are exported as circles instead of polygons in the OASIS file. Cell/Layer Statistics show circles only if there are some circles in the OASIS file. **Sides per Donut** specifies the number of edges used for donut to polygon conversion. Default is 64. **DBU per UU** is an optional integer multiple or an integer divisor of the design's database units per user units that you specify for scaling while translating an OA database to OASIS without changing the source OA database. Default is empty. `Preference: dbExportOasisDbuPerUU`

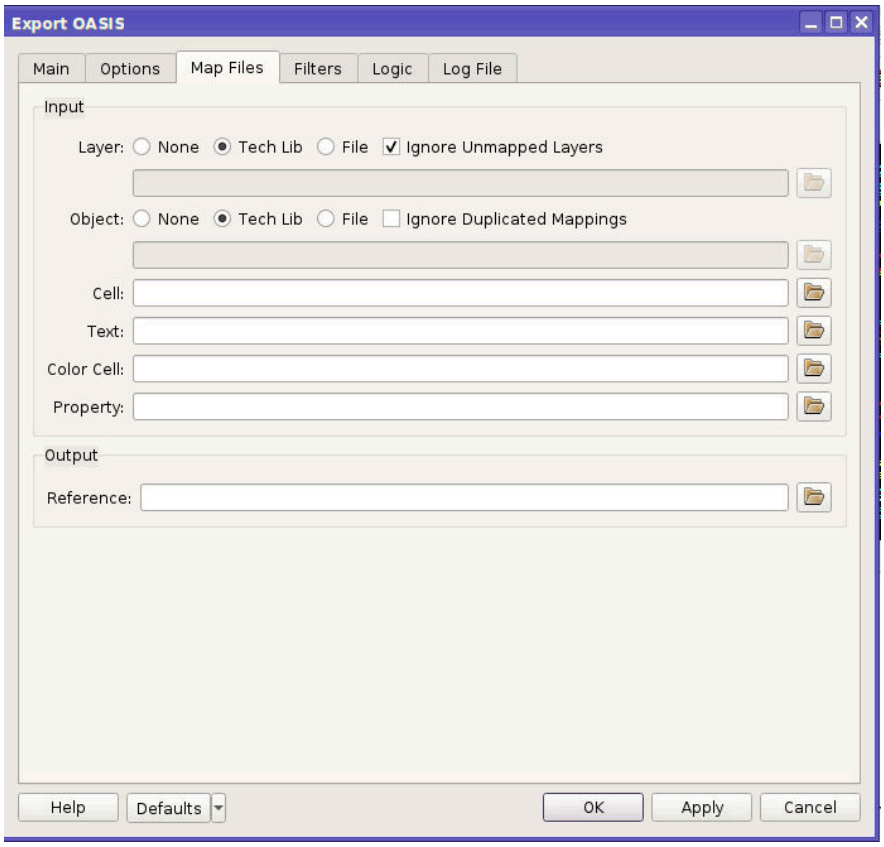
Feedback

Settings	Description
	Convert Text Brackets specifies whether or how to convert text brackets during export: ◦ **None**: (default) Do not convert text brackets. ◦ **Angle**: Convert square text brackets, "[" and "]", to angle text brackets, "<" and ">". ◦ **Square**: Convert angle text brackets, "<" and ">", to square text brackets, "[" and "]". **Max Vertices in Path and Polygon** limits the maximal vertices number in path and polygon. Split it if vertices number is over the limit. Default is 8190. **Remaster View Name** changes the instance master view name from layout to the specified view name. For example, if instances are bound to abstract views and **Remaster View Name** is set to layout, all instances are bound with the layout view during the export stream process, if the layout view exists in the instance master's library. This option is applied in both the color shift and non-color shift flows. It is not applied for PCell and customer via.
Reference Data	**Library File** The file you browse to select. This file has the names of the reference libraries to use to export referenced cells. **Library List** The list of reference libraries to use to export referenced cells. Click the folder icon to open the **Export OASIS Reference Library Browser**.

5. Click the **Map Files** tab in the **Export OASIS** dialog box (Figure 30).

Feedback

Figure 30 Export OASIS Dialog Box: Map Files Tab



6. On the **Map Files** tab, enter settings as described in Table 48.

Table 48 Export OASIS Settings (Map Files Tab)

Settings	Description
Input	
Layer	Select the required layer mapping mode: ◦ **None**: Do not use a layer mapping file for layer/purpose mapping. Instead, use the OpenAccess layer number and purpose number for the OASIS layer number and datatype. ◦ **Tech Lib**: (default) Use the layer mapping files in the technology library, merging them if more than one exists. Maps OpenAccess shapes to all the OASIS layer numbers and datatypes found in the combined mapping file. ◦ **File**: Use the layer mapping file you browse to select.

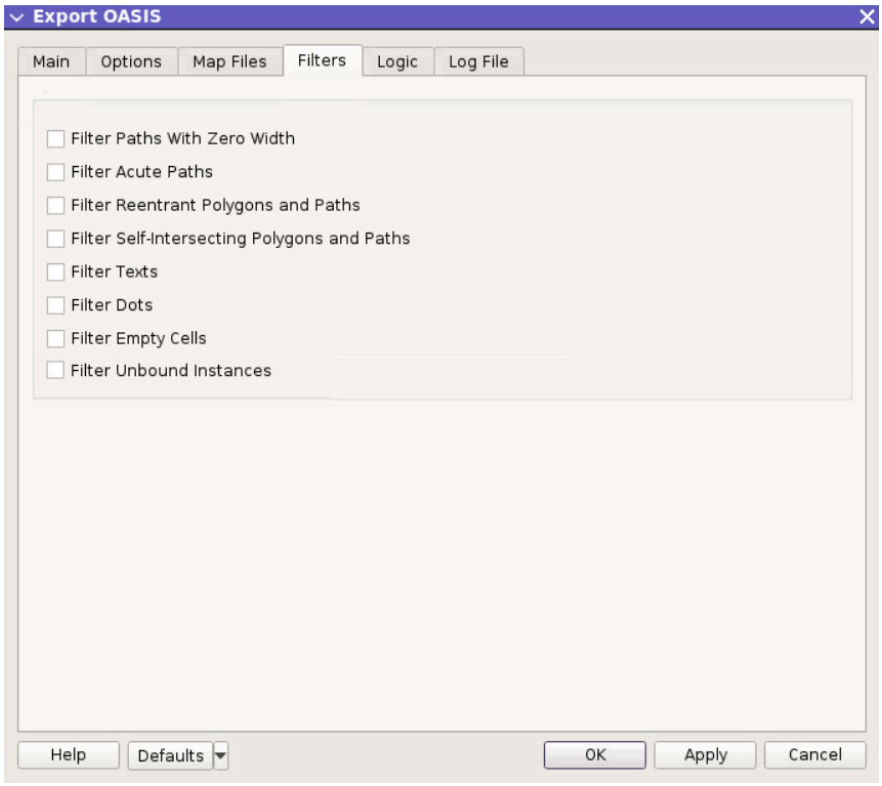
Feedback

Settings	Description
	If **Ignore Unmapped Layers** is not selected and the OpenAccess layer number and purpose number are not matched in the layer mapping file, use the OpenAccess layer number and purpose number for the OASIS layer number and data type. You can also use the layer number and purpose number of particular layer names and purpose names of OpenAccess objects (see Table 10) for the OASIS layer number and data type. If **Ignore Unmapped Layers** is selected (default), the OpenAccess shape/object is filtered out.
Object	Select the required object mapping mode: ◦ **None**: Do not use an object mapping file. ◦ **Tech Lib**: (default) Use the object mapping files in the technology library, merging them if more than one exists. Maps OpenAccess objects to all the OASIS layer numbers and data types found in the combined mapping file. ◦ **File**: Use the object mapping file you browse to select. Specifies methods of mapping Boundaries and `LayerBlockages` between OASIS shapes and OpenAccess databases. ◦ **Ignore Duplicated Mappings**: Specifies to ignore duplicated mappings for the keyword `Term` in the object mapping file.
Cell	Specifies the cell mapping file you browse to select. Maps cell and port names.
Text	Specifies the text mapping file you browse to select.
Color Cell	Specifies the color cell mapping file you browse to select.
Property	Specifies the property mapping file that defines mappings between property names in OASIS file and property IDs and object types in OpenAccess.
Output	
	Reference specifies an XML output reference map file that has the OA mapping for each structure of the OASIS file. Specify the path to the file in the text box.

7. Click the **Filters** tab in the **Export OASIS** dialog box (Figure 31).

Feedback

Figure 31 Export OASIS Dialog Box: Filters Tab



8. On the **Filters** tab, enter settings as described in Table 49.

Table 49 Export OASIS Settings (Filters Tab)

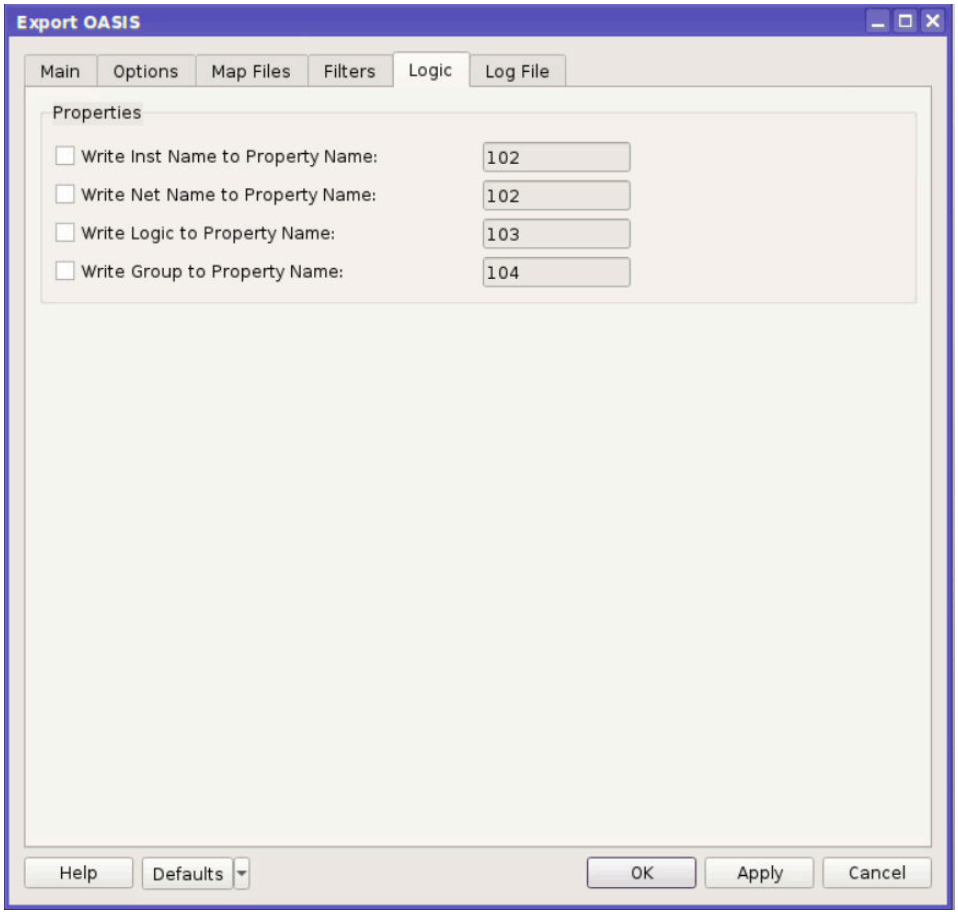
Settings	Description
Filters	**Filter Paths With Zero Width** specifies whether or not to filter paths with zero width. Tcl preference: `dbExportOasisFilterZeroWidth` **Filter Acute Paths** specifies whether or not to filter paths with acute angles. Tcl preference: `dbExportOasisFilterAcutePaths` **Filter Reentrant Polygons and Paths** specifies whether or not to filter paths with reentrant polygons and paths. Tcl preference: `dbExportOasisFilterReentrantPgns` **Filter Self-Intersecting Polygons and Paths** specifies whether or not to filter paths with self-intersecting polygons and paths. Tcl preference: `dbExportOasisFilterSelfIntersectingPgns` **Filter Texts** specifies whether or not to filter text. Tcl preference: `dbExportOasisFilterText`

Feedback

Settings	Description
	Filter Dots specifies whether or not to filter dots. Tcl preference: `dbExportOasisFilterDots` **Filter Empty Cells** specifies whether or not to filter empty cells and the associated unbound instances. Tcl preference: `dbExportOasisFilterEmptyCells` **Filter Unbound Instances** specifies whether or not to filter unbound instances. Tcl preference: `dbExportOasisFilterUnboundInsts`

9. Click the **Logic** tab in the **Export OASIS** dialog box (Figure 32).

Figure 32 Export OASIS Dialog Box: Logic Tab



10. On the **Logic** tab, enter settings as described in Table 50.

Feedback

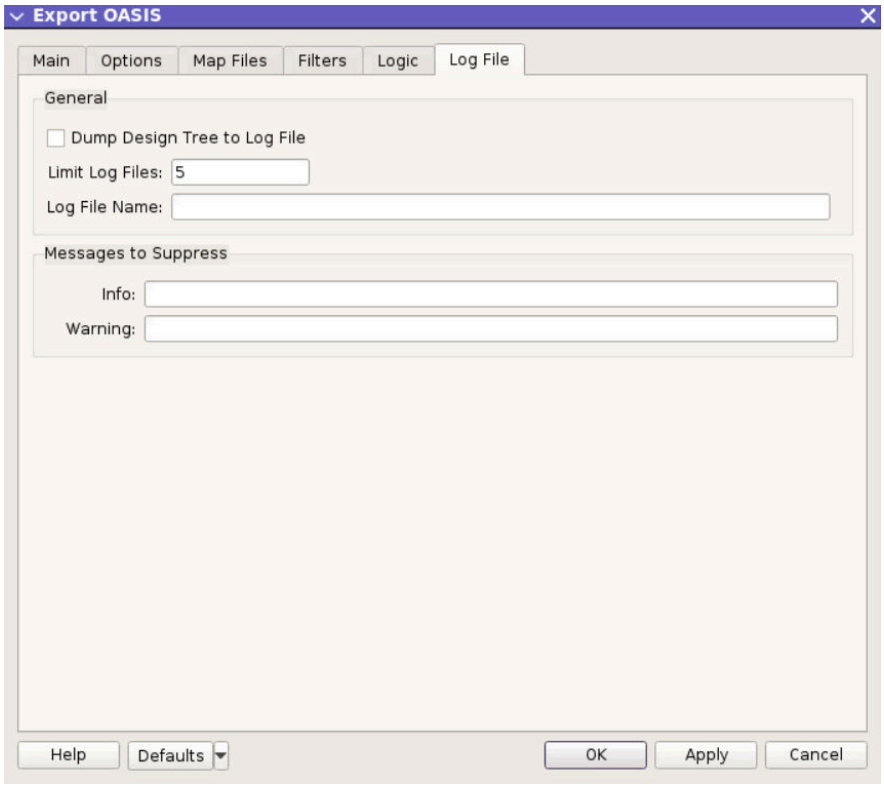
Table 50 Export OASIS Settings (Logic Tab)

Settings	Description
Properties	**Write Inst Name to Property Name** converts the instance name to the output OASIS file using the specified OASIS property name. OASIS property names can be numbers or strings. The default is 102. Tcl preference: `dbExportOasisPropNameForInstName` **Write Net Name to Property Name** converts the net name of shapes to the output OASIS file using the specified OASIS property name. OASIS property names can be numbers or strings. The default is 102. Tcl preference: `dbExportOasisPropNameForNetName` **Write Logic to Property Name** converts the logic information of objects to the output OASIS file using the specified OASIS property name. For shapes, converts ◦ net name ◦ terminal name and type ◦ name, PlacementStatus, and AccessDir of pins For instances, converts ◦ instance name ◦ terminal name ◦ terminal net name OASIS property names can be numbers or strings. The default is 103. Tcl preference: `dbExportOasisPropNameForLogicName` **Write Group to Property Name** converts the synchronous group and the figure group information (name, status and origin) to the output OASIS file using a specified OASIS property name. The default is 104. Tcl preference: `dbExportOasisKeepGroupName`

11. Click the **Log File** tab in the **Export OASIS** dialog box (Figure 33).

Feedback

Figure 33 Export OASIS Dialog Box: Log File Tab



12. On the **Log File** tab, enter settings as described in Table 51.

Table 51 Export OASIS Settings (Log File Tab)

Settings	Description
General	The **Dump Design Tree to Log File** option writes the design tree information to a log file during OASIS import. Off by default. **Limit Log Files** limits the maximum number of backup log files. Default is 5. **Log File Name** specifies the name of the log file.
Messages to Suppress	**Info** is the list of informational messages to suppress during export. **Warning** is the list of warning messages to suppress during export.

13. Click **OK**.

    The translator exports the design in OASIS format.

Feedback

If you have set the `dbExportOasisCheckSave` preference to `true`, the **Save Design** dialog box appears if there are designs that are not saved. Where, the design can be based on the specified library, cell, view, and hierarchy.

Settings	Description
Save	Saves the design and continues exporting the design.
Ignore	Does not save the design but continues exporting the design.
Cancel	Does not save the design and cancels exporting the design.

The `onPostStreamExit` callback is triggered at the end of the **Export OASIS** run.

The `exportOasis` command exports the design in OASIS format.

Syntax

```
exportOasis -lib library -out file [-libDefFile file] [-applyRepetition]
 [-autoMapObjects] [-cell cell] [-cellMap file]
[-cellNamePrefix prefix] [-cellNameSuffix suffix] [-colorCellMap
 fileList]
[-colorMapping ignore|strict|full|honor|allVias|lockOnly|disableColor]
 [-compress number] [-convertTextBrackets angle|square]
[-dbuPerUU number] [-donutNumSides number] [-dumpDesignTree]
 [-dumpRefLib] [-duplicateCell withNumber|withLibName|skip]
[-ellipseNumSides number] [-ellipseToCircle] [-expandArrayInsts]
 [-filterAcutePaths] [-filterDots] [-filterEmptyCells]
[-filterPlacementStatuses <string>] [-filterReentrantPgns]
 [-filterSelfIntersectingPgns] [-filterText] [-filterUnboundInsts]
 [-filterZeroWidth] [-flattenPcells]
[-flattenVias] [-hierDepth number] [-ignoreCellsAffix celllist]
 [-ignoreTopCellsAffix] [-keepAbuts] [-keepFullCellName]
[-keepPcells] [-keepSubMasters] [-keepVias] [-labelMap fileList]
 [-logFile file] [-maxVertices]
[-mergeCellviews file] [-newMaxVerticesFunction] [-newMaxVerticesRule
 SwapXY] [-noAutoBackupFile] [-noExportRefLibCells]
[-noInfo msgIds] [-noOutputBlockages] [-noOutputBoundaries]
 [-noOutputTextDisplays]
[-noUnmappedLayerWarning {layer1 purpose1} {layer2 purpose2} ...]
 [-noWarning msgIds] [-notExternalizeShapes] [-objectMap file]
```

```
[-pathToPolygon] [-pcellFreeze] [-propMap file][-propertyInst name]
 [-propertyLogic name]
[-propertyNet name][-refLibList file] [-refLibs libList] [-referenceMap
 file] [-remasterViewName viewName] [-snapToMfgGrid]
[-techLib library] [-textHeight mode] [-textMap fileList] [-toLower]
 [-toLowerLabel] [-toUpper] [-toUpperLabel]
[-unmappedLayersUseOALayers] [-useTechLayerMap] [-useTechObjectMap]
 [-validationScheme number] [-view view]
[-h|-help] [-v] [-version]
```

For details of the `exportOasis` command, type `exportOasis -help`.

See Also

- Layer Mapping Files
- Supplying Mapping Files for GDSII Stream and OASIS Format Translations

Exporting Design Data in Library Exchange Format (LEF)

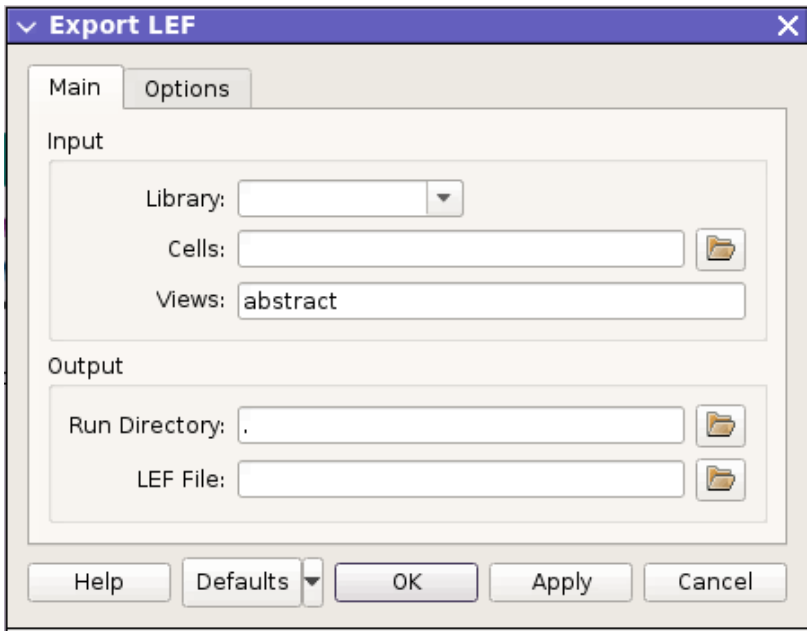
Library Exchange Format (LEF) is an ASCII data format that describes standard cell libraries. LEF files typically include a technology section (rules, definitions, and capacitance), a site extension section, and a macros section (cell information, pins, blockages, and so on). LEF is used with the design exchange format (DEF), which describes netlists and circuits, to fully represent the physical layout of designs.

Feedback

To export design data in Library Exchange Format (LEF):

1. From the **Home** or **Console** tab, choose **File > Export > LEF** to open the **Export LEF** dialog box.

Figure 34 Export LEF Dialog Box: Main Tab



2. On the **Main** tab, enter settings as described in Table 52.

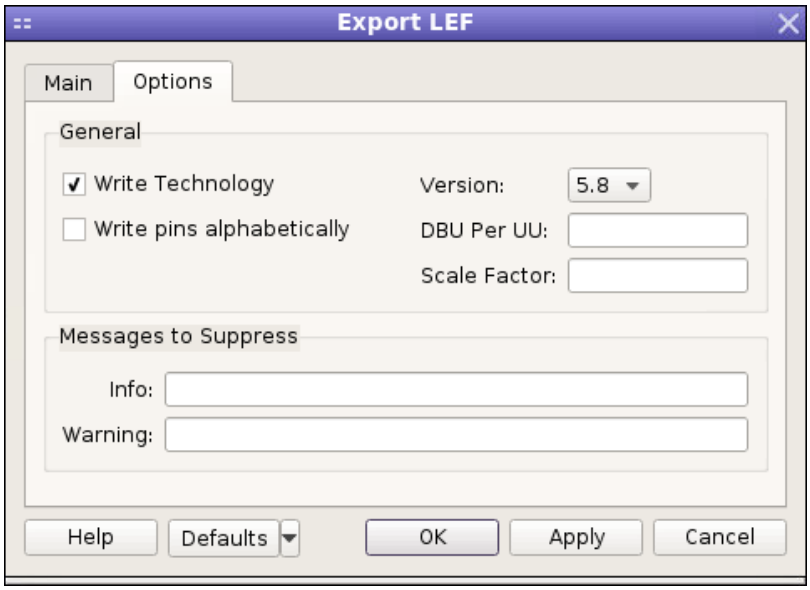
Table 52 Export LEF Settings (Main Tab)

Settings	Description
Input	**Library** specifies the design designated for export. **Cells** specifies the cell(s) to export. **Views** specifies the view(s) to export. The default is an abstract view.
Output	**Run Directory** is the directory you browse to select, or a new directory you create by entering its name. The default is the current directory. **LEF File** is the LEF file you browse to select and use.

3. Click the **Options** tab in the **Export LEF** dialog box (Figure 35).

Feedback

Figure 35 Export LEF Dialog Box: Options Tab



4. On the **Options** tab, enter settings as described in Table 53.

Table 53 Export LEF Settings (Options Tab)

Settings	Description
General	**Write Technology** specifies whether to include information from the technology database in the LEF file. Tcl preference: `dbExportLefWriteTechnology` **Write pins alphabetically** writes the pins from the abstract in alphabetical order. **Version** specifies the LEF file format version. You should select the latest version that is understood by the LEF readers in your flow. Default is 5.8. If you choose LEF version 5.8, mask number and DBU/UU information is included in the LEF file. Tcl preference: `dbExportLefVersion` **DBU per UU** overrides distance per database unit. Tcl preference: `dbExportLefDBUPerUU` **Scale Factor** scales cell coordinates. Tcl preference: `dbExportLefScaleFactor`
Messages to Suppress	**Info** is the list of informational messages to suppress during export. Tcl preference: `dbExportDefScaleFactor`

Feedback

Settings	Description
	Warning is the list of warning messages to suppress during export. Tcl preference: `dbExportDefScaleFactor`

5. Click **OK**.

   The translator exports the design in LEF format.

The `exportLef` command is also supported to export the design in LEF format.

Syntax

```
exportLef -lef <fileName> -lib <libraryName> [-libDefFile file] [-cells
 cells] [-logFile file] [-noInfo msgIds] [-noTech] [-noWarning msgIds]
 [-sortPins] [-templateFile file] [-ver version] [-integratorLicense]
 [-views views (=layout)] [-help] [-v] [-version]
```

For details of the `exportLef` command, type `exportLef -help`.

Examples

```
exportLef -lef lef_file_path -lib leaf_cells -cells AND2X1 -views
 abstract -ver 5.8 -libDefFile lib_defs_path -logFile exportLef.log
```

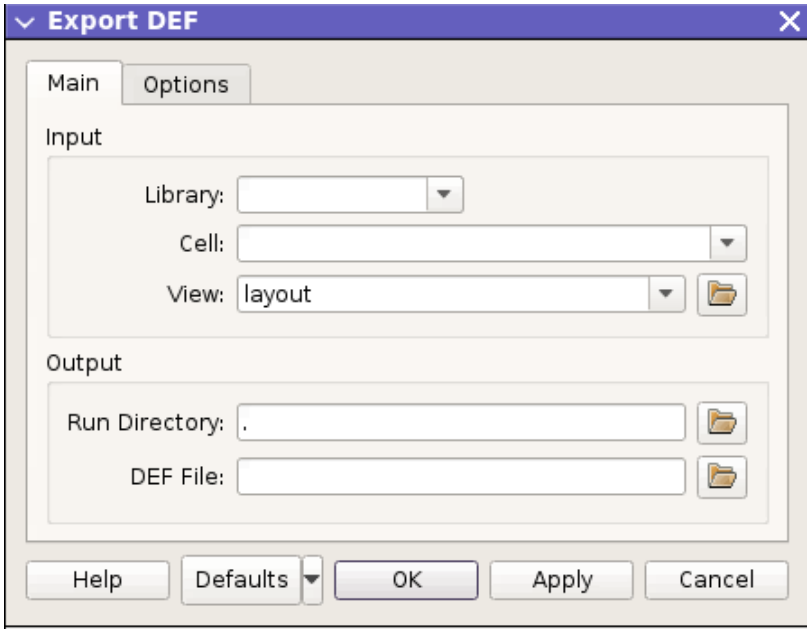
Feedback

Exporting Design Data in Design Exchange Format (DEF)

To export design data in Design Exchange Format (DEF):

1. From the **Home** or **Console** tab, choose **File > Export > DEF** to open the **Export DEF** dialog box.

Figure 36 Export DEF Dialog Box: Main Tab



2. On the **Main** tab, enter settings as described in Table 54.

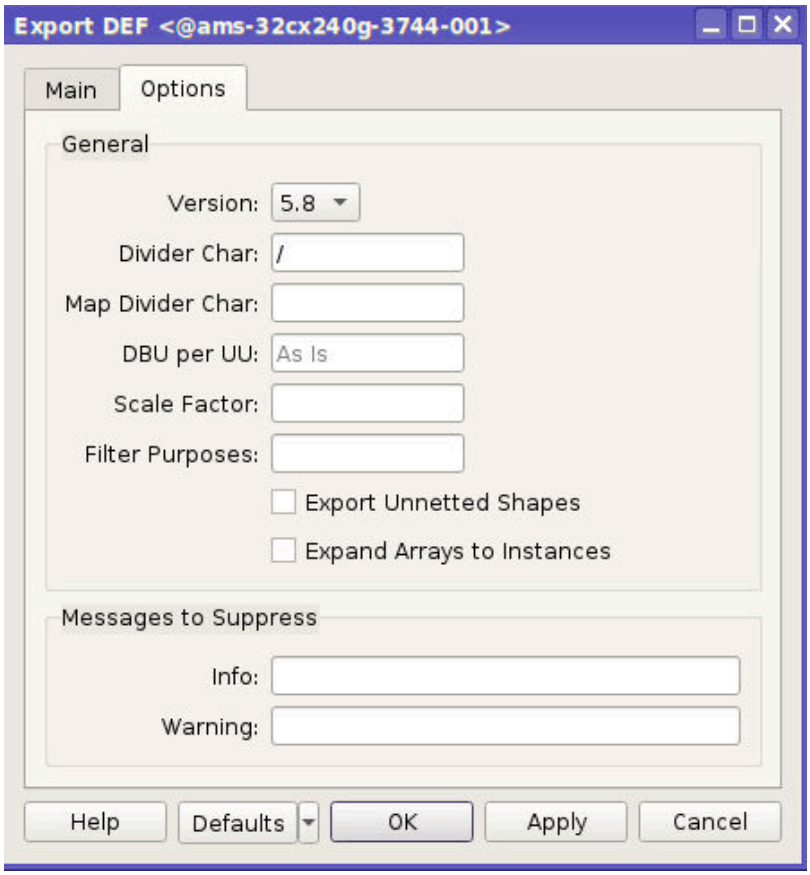
Table 54 Export DEF Settings (Main Tab)

Settings	Description
Input	**Library** contains the design designated for export. **Cell** is the name of the cell to export. **View** is the name of the view to export. The default is a layout view.
Output	**Run Directory** is the directory you browse to select, or a new directory you create by entering its name. The default is the current directory. **DEF File** is the DEF file you browse to select and use.

3. Click the **Options** tab in the **Export DEF** dialog box (Figure 37).

Feedback

Figure 37 Export DEF Dialog Box: Options Tab



4. On the **Options** tab, enter settings as described in Table 55.

Table 55 Export DEF Settings (Options Tab)

Settings	Description
General	**Version** specifies the DEF file format version. You should select the latest version that is understood by the DEF readers in your flow. Default is 5.8. Tcl preference: `dbExportDefVersion` **Divider Char** specifies the character used to express hierarchy when DEF names are mapped. Default is */*. Tcl preference: `dbExportDefDividerChar` **Map Divider Char** is equivalent to the `mapDividerChar` option in the `exportDef` batch command. Tcl preference: `dbExportDefMapDividerChar`

Feedback

Settings	Description
	DBU per UU overrides distance per database unit. Default is **As Is**. Tcl preference: `dbExportDefDBUOverride` **Scale Factor** scales design coordinates. Tcl preference: `dbExportDefScaleFactor` **Filter Purposes** filters shapes based on your preferences. Tcl preference: `dbExportDefFilterPurposes` **Export Unnetted Shapes** determines whether to export unnetted shapes to a DEF file. Tcl preference: `dbExportDefUnnettedShapes` **Expand Array Instances** determines whether to export arrays to instances Tcl preference: `dbExportDefExpandArrayInsts`
Messages to Suppress	**Info** is the list of informational messages to suppress during export. Tcl preference: `dbExportDefSuppressInfos` **Warning** is the list of warning messages to suppress during export. Tcl preference: `dbExportDefSuppressWarnings`

5. Click **OK**.

   The translator exports the design in DEF format.

The `exportDef` command is also supported to export the design in DEF format.

Syntax

```
exportDef -lib <libraryName> -cell <cellName> -views views (=layout) -def
 <fileName> [-libDefFile file] [-busbitChars busbitChars] [-dividerChar
 busbitChars] [-expandArrayInsts] [-filterPurposes][-logFile file]
 [-mapDividerChar mapChar] [-dbu dbu (=0)] [-noInfo msgIds] [-noWarning
 msgIds] [-ver version] [-help] [-v] [-version]
```

For details of the `exportDef` command, type `exportDef -help`.

Exporting Design Data in Liberty Library File Format

You can generate and export an output file in the Liberty Library format to provide input for synthesis tools.

For information about Liberty Data Editor, see *Custom Compiler and IC Compiler Co-Design Assistant User Guide*.

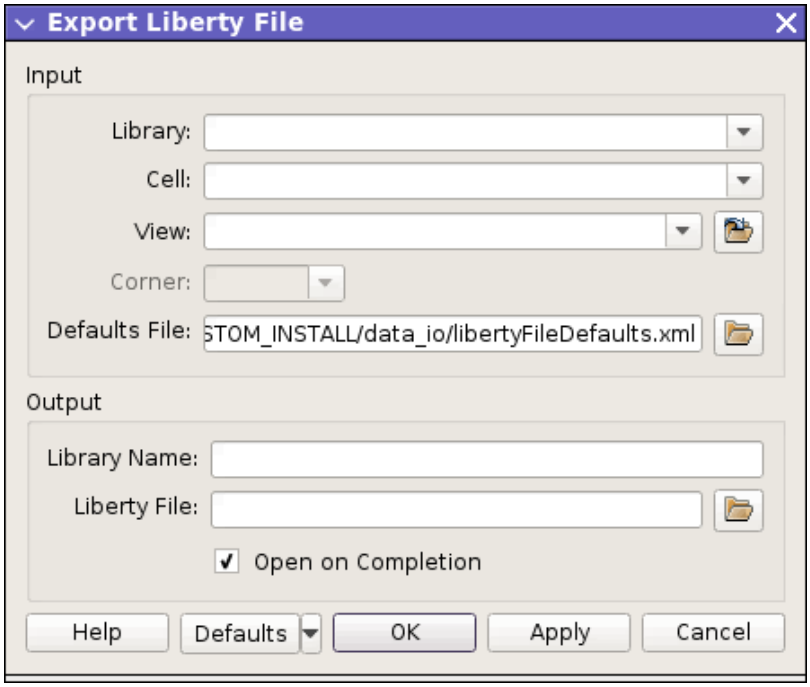
Feedback

To export a Liberty Library file:

1. From the **Home** or **Console** tab, choose **File > Export > Liberty** to open the **Export Liberty File** dialog box.

   You can also click **Export** in the **Edit Liberty Data** dialog box.

Figure 38 Export Liberty Dialog Box



2. Enter design information as described in Table 56.

Table 56 Export Liberty File

Settings	Description
Input	Specify the **Library** and **Cell**, and specify or browse to choose a **View** for the design. Note that If the **Export Liberty File** dialog box is called from the **Edit Liberty Data** dialog box, these fields are predefined and not visible. **Corner** is editable if you launched the dialog box from the console or the command line. If you launched the **Export Liberty File** dialog box from the **Edit Liberty Data** dialog box, the corner is predefined.

Feedback

Settings	Description
	Defaults File shows the XML format file containing the defaults you want to use. Accept the file name shown or browse to open a different one.
Output	**Library Name** is used in the exported text as a value of the tag "library" in the Liberty format. By default, it is composed from current OA library name and the corner name.
	Liberty File specifies the target file. Accept the Liberty file name shown or browse to open a different file of type `.lib` to overwrite with new output.
	Open on Completion opens the generated `.lib` file in a text editor.

3. Click **OK**.

   The tool generates the file as specified. To review the file, open the file in a text editor.

Exporting Design Data in Netlist Format

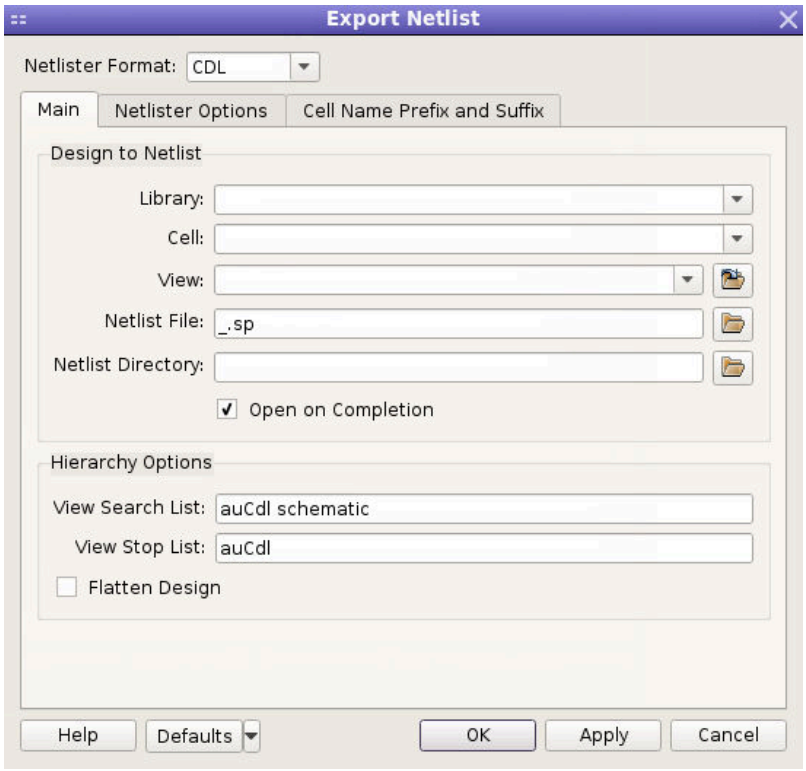
You can export design data in netlist format. You can do this without opening the design.

You can optionally perform flat netlisting with this procedure. The system flattens the design to a temporary cellView, optionally renames all instances and nets (except global and ported nets), and then netlists the flattened design.

Feedback

To export design data in netlist format:

1. From the **Home** or **Console** tab, choose **File > Export > Netlist** to open the **Export Netlist** dialog box.



2. Enter settings on the **Main** tab.

   (Optional) Use the `dbExportNetlist` preference to set the values in this dialog box. For information about using preferences, see Specifying Design Preferences.

   The following table describes settings on this **Main** tab.

Option	Description
Netlister Format	Choose a format from the list: **CDL**, **HSPICE**, **Spectre**, **VHDL**, **Verilog**, or **VerilogICC**. Tcl preference: `dbExportNetlistFormat`

Feedback

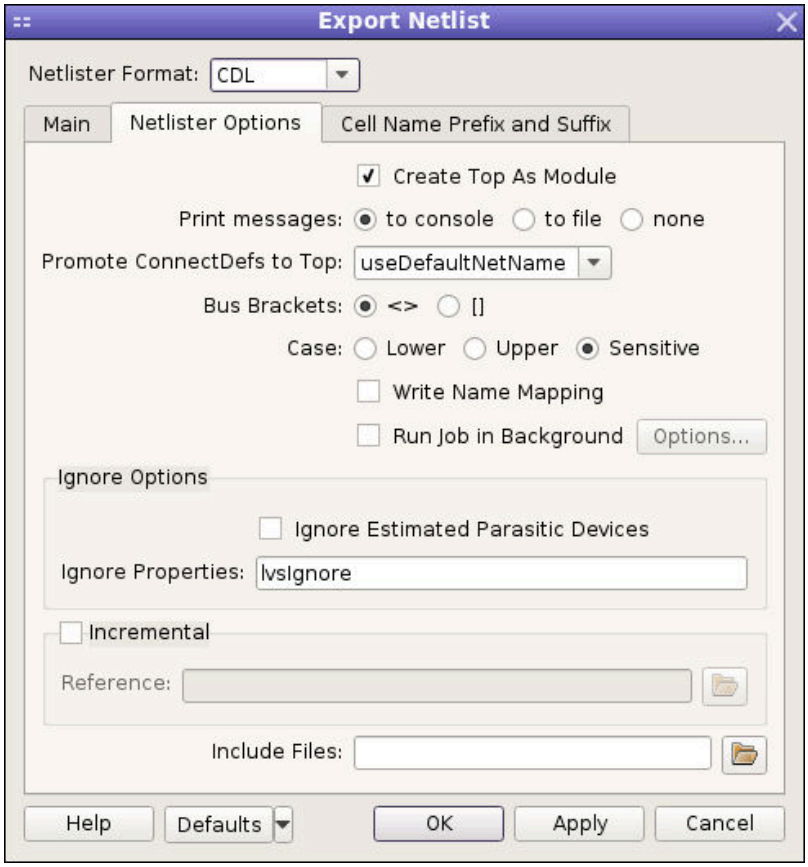
Option	Description
Design to Netlist	Specify the **Library** and **Cell**, and specify or browse to choose a **View** for the design. You can accept the current **Netlist File** for overwriting, browse to a different file, or enter the name of a new file. You can browse to select a **Netlist Directory**. **Open on Completion** (on by default) opens a text viewer window showing the contents of the exported netlist. Tcl preferences: `dbExportNetlistDir`, `dbExportNetlistFile`, `dbExportNetlistLCV`
Hierarchy Options	**View Search List** searches these views. Accept, or enter new values. Tcl preference: `dbExportNetlistViewSearchList` **View Stop List** stops the search at these views. Accept, or enter new values. Tcl preference: `dbExportNetlistViewStopList` **Flatten Design** flattens a hierarchical design before netlisting it. Enabling this feature adds a **Flattening Options** tab to accept additional settings. Tcl preference: `dbExportNetlistFlattenDesign`

Note:

- If a line in the Verilog netlist file exceeds 80 characters, ',' appears at the end of the line.
- For selecting multiple cellviews to export design in netlist format, see Selecting Multiple Cellviews

Feedback

3. Enter settings on the **Netlister Options** tab.



The following table describes settings on this **Netlister Options** tab.

Option	Description
Maximum Line Length	Maximum line length. The default is 80. Tcl preference: `dbExportNetlistMaxLength`
Indent Depth	Indent depth. The default is 4. Tcl preference: `dbExportNetlistIndentDepth`
Create Top As Module	Netlists the top design (the design root) as a module. Tcl preference: `dbExportNetlistCreateTopAsModule`
Sort Instance Headers	Sorts instance headers by **none** (default), **ascending**, or **descending**. Tcl preference: `dbExportNetlistSortInstanceHeaders`
Sort Instances	Sorts instances by **none** (default), **ascending**, or **descending**. Tcl preference: `dbExportNetlistSortInstances`

Feedback

Option	Description
Module Order	Specifies module ordering by **leaf first** (default), **top first**, **sort ascending**, or **sort descending**. Tcl preference: `dbExportNetlistModuleOrder`
Print messages	Prints message to the console (default), a file, or not at all.
Promote ConnectDefs to Top	The top-level terminal names for connectDefs are changed to match the default net names and valid top design net names are added to the top module terminal list in CDL netlist. Tcl preference: `nlCDLTopVirtualTerms`
Copy <netlister> File Content	Controls how text views are netlisted with one of these netlisters: CDL, HSPICE, or Spectre. Tcl preferences: `nlCDLIncludeCDLFile`, `nlHSPICEIncludeHSPICEFile`, `nlSpectreIncludeSpectreFile`

Port Connection	(CDL, Verilog, and VerilogICC only) Sets the port connection type using **byName** (default for VerilogICC) or **byOrder** (default for CDL and Verilog).
	Port connection type and its default might vary according to the currently selected **Netlist Format**. This option is hidden by default for CDL but it is shown by default for Verilog and VerilogICC.
	This option controls the way the instance terminals connection to the nets are netlisted in the instance declartion.
	For CDL,
	◦ When **byOrder** is set, the result looks like this:
	XVG2 **s3 s2** gainV
	*only the **net names** are declared*
	◦ When **byName** (works only for subcircuits) is set, the result looks like this:
	XVG2 gainV $**PINS Q=s3 D=s2**
	*both **terminal names** and **net names** with '=' are declared, after keyword "$PINS"*
	◦ When **byOrder** is set, the result looks like this:
	res R5(**net023**, **net040**);
	*only the **net names** are declared*
	◦ When **byName** is set, the result looks like this:
	res R5(**.MINUS**(**net023**), **.PLUS**(**net040**));
	*both **terminal names** with dot at start and **net names** in brackets are declared*
	Tcl preferences: `nlCDLPortConnectionType`, `nlVerilogICCPortConnectionType`, `nlVerilogPortConnectionType`

Bus Brackets	(CDL and HSPICE only) Allows you to choose between <> or [].
Case	(CDL and HSPICE only) Choose **Lower**, **Upper**, or **Sensitive**, which writes the resulting netlist text preserving the case used in the source design.

Feedback

Option	Description
Write Name Mapping	A `cnl` directory is created in the netlisting target directory to hold the saved name mapping, incremental netlisting data, intermediate netlist files, log, and so forth. If **Write Name Mapping** is off, the directory `cnl` is not created and a temporary directory is used instead (which is deleted after netlisting). The use of saved name mapping and incremental netlisting is therefore not available when **Write Name Mapping** is off. If **Write Name Mapping** is on but the `cnl` directory is not accessible to write, the temporary directory is used. Tcl preference: `dbExportNetlistWriteNameMapping`
Run Job in Background	(CDL, HSPICE, Spectre, VHDL, and Verilog only) Runs this job as a background process. To set job management options, click **Options** to open the **Job Management Options** dialog box.
Copy Text View Content	(Verilog only) Controls how text views are netlisted. When enabled, writes the contents of text views directly into the final netlist. When disabled (default), includes text views with an `.include` statement. Tcl preference: `nlVerilogIncludeVerilogFile`
Copy Text File to Target Folder	(Verilog only) Controls how the text views are included in Verilog. This option can be enabled only if **Copy Text View Content** is turned off. When enabled, copies the text file to the same folder of the target netlist file, and prints only the file name without path in an `.include` statement. When disabled (default), prints the full path of the text file located in the text view folder in an `.include` statement. Tcl preference: `nlVerilogCopyIncludedTextFile`
Ignore Options	**Ignore Estimated Parasitic Devices** disables inclusion of parasitics in the netlist. For information about ignoring estimated parasitic devices while tracing nets with probes, see the *Custom Compiler Schematic Editor and Text Editor User Guide*. **Ignore Properties** is a list of property names for shorting. Ignore property names are netlister-specific and controlled by the preference `nl<Simulator>ShortPropNames`. Ignore properties are used to determine whether an instance or a master should be shorted. If a device is shorted, the netlister looks in `nlShortExprPropNames` to determine *how* to short a device. For example, a typical two-terminal shorting description is `(short(PLUS MINUS))`, which tells the netlister to short the PLUS and MINUS terminals.

Feedback

Option	Description
Incremental	Enables incremental netlisting. If you choose this option, additionally browse to the **Reference** directory where the design was last netlisted.
Include Files	(CDL and HSPICE only) Adds include files to the netlist.

(Optional) **Maximum Line Length**, **Indent Depth**, **Sort Instance Headers**, **Sort Instances**, **Module Order**, **Port Connection** (shown by default for Verilog and VerilogICC), and **Copy <netlister> File Content** (where <netlister> can be CDL, HSPICE, or Spectre) are hidden by default when the following Tcl preference is set:

```
db::setPrefValue dbExportNetlistHiddenOptionsList -value
"maxLineLength indentDepth sortInstHeaders moduleOrder sortInsts
connectionTypeCDL copyNetlistFileContent"
```

- To make one of them visible, remove the associated value. For example, you can show the **Port Connection** option in the dialog box by removing the `connectionTypeCDL` value:

  ```
  db::setPrefValue dbExportNetlistHiddenOptionsList -value
  "maxLineLength indentDepth sortInstHeaders moduleOrder sortInsts"
  ```

- To make them all visible, set the following:

  ```
  db::setPrefValue dbExportNetlistHiddenOptionsList -value ""
  ```

(Optional) Assumed that you export a CDL netlist with the following settings on the **Main** tab when **Copy CDL File Content** is shown on the **Netlister Options** tab:

- **Design to Netlist**

  Library: topLib

  Cell: block1

  View: schematic

  Netlist File: topLib_block1.sp

  Open on Completion: on

- **Hierarchy Options**

  View Search List: auCdl schematic

  View Stop List: auCdl cdl

  where cdl is the text view name.

Feedback

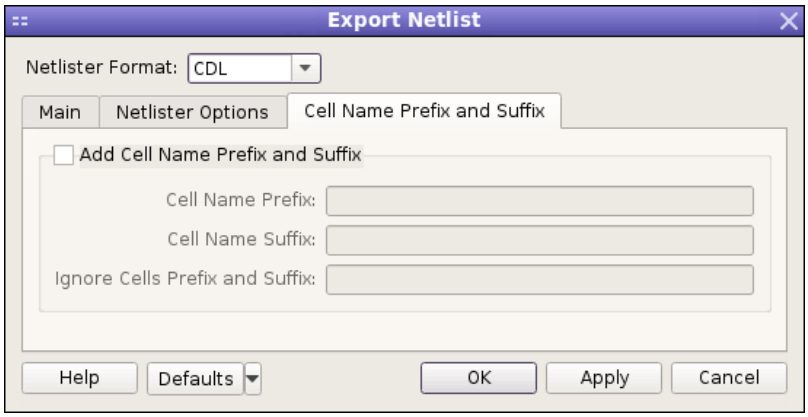
When `nlCDLIncludeCDLFile` is set to *false* (**Copy CDL File Content** is turned off), the netlister includes the cdl text view with an `.include` statement.

```
.subckt block1

XI142 net990 net991 net988 net989 block0

.ends block1

.include '/remote/project1/topLib/block0/cdl/netlist.cdl'
```

When `nlCDLIncludeCDLFile` is set to *true* (**Copy CDL File Content** is turned on), the netlister copies the contents of the cdl text view directly into the final netlist.

```
.subckt block1

XI142 net990 net991 net988 net989 block0

.ends block1

# Copy the CDL file content into the final netlist

.subckt block0 a b x y

*.PININFO a:I b:I x:O y:O

RR470 b y 1K $[RP]

RR469 a x 1K $[RP]

.ends block0
```

4. Enter settings on the **Cell Name Prefix and Suffix** tab.

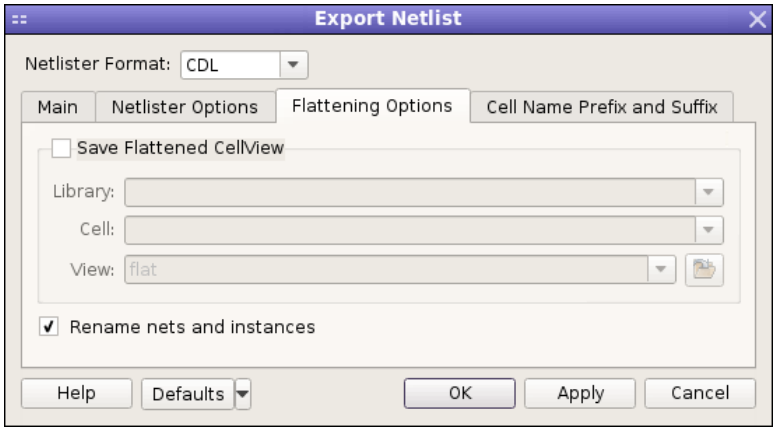


The following table describes settings on this **Netlister Options** tab.

Feedback

Option	Description
Add Cell Name Prefix and Suffix	When enabled, adds affixes to cell names with the following settings: **Cell Name Prefix** adds prefix for cell names in CDL netlisting. **Cell Name Suffix** adds suffix for cell names in CDL netlisting. **Ignore Cells Prefix and Suffix** specifies a space-separated list of cell name wildcards for which the prefix and suffix should not be added in CDL netlisting. Tcl preferences: `nlCDLAddCellNameAffix`, `nlCDLCellNamePrefix`, `nlCDLCellNameSuffix`, `nlCDLIgnoreCellsAffix`

5. (Optional) If you enabled the **Flatten Design** option on the **Main** tab to flatten the design, enter settings on the **Flattening Options** tab.



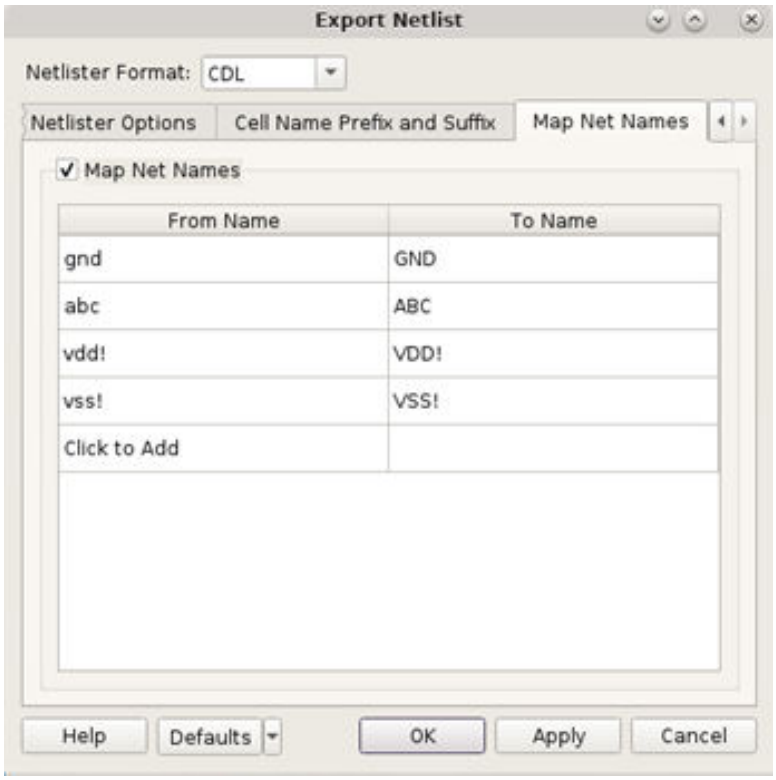
The following table describes settings on this **Flattening Options** tab.

Option	Description
Save Flattened CellView	When disabled (default), creates the flattened cellview on a temporary cellview, and then removes the temporary cellview after netlisting is completed. When enabled, specify the **Library**, **Cell**, and **View** to save the flatten cellview in a temporary cellview and then keeps the temporary cellview after netlisting is done. Tcl preference: `dbExportNetlistFlattenSaveCellView`

Feedback

Option	Description
Rename nets and instances	When enabled (default), renames all instances and most nets to random strings to obscure the design intent, and the nets and instances are renamed with combination of lowercase and uppercase. This option does not rename global nets and ported nets. However, when you set the `dbExportNetlistFlattenCaseInsensitiveRename` preference to `true`, the renamed nets and instances are generated only with lowercase. Tcl preference: `dbExportNetlistFlattenRename`

6. When exporting CDL netlist, an additional tab, the **Map Net Names** tab is shown.



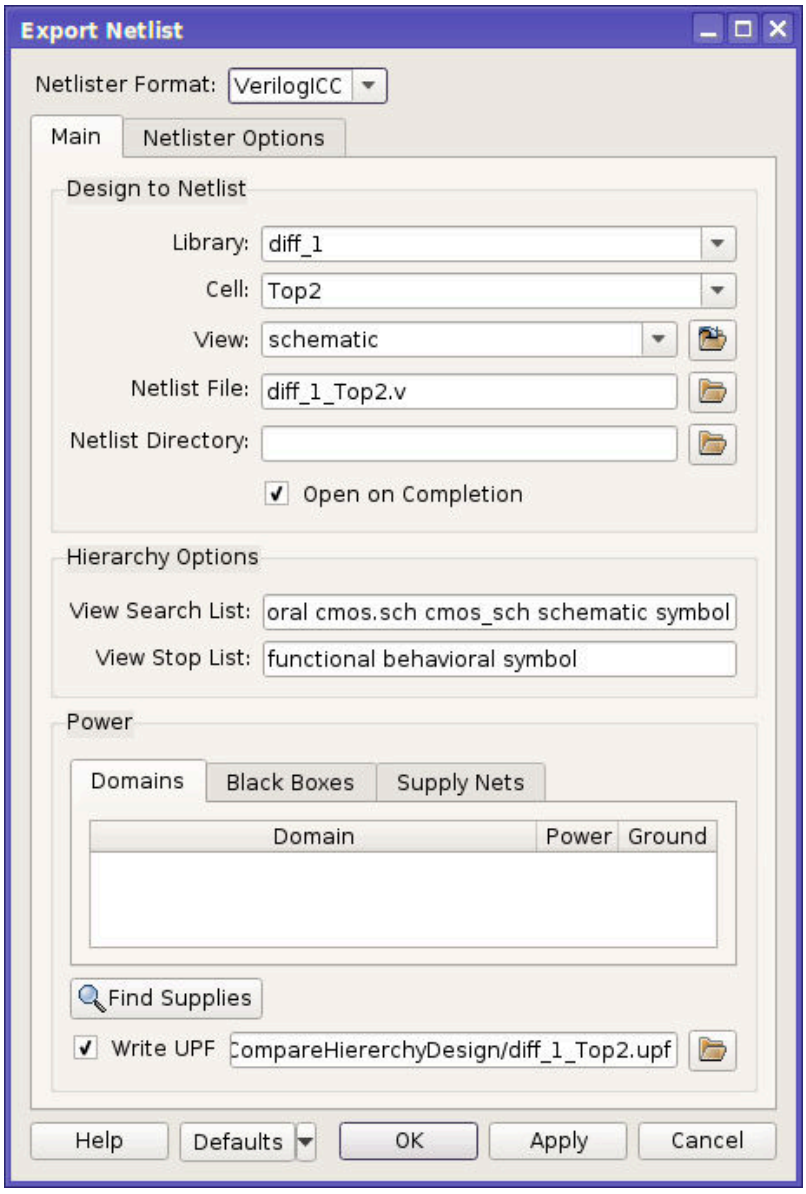
Define the pairs of `From Name` and `To Name` (source name and destination name, respectively) to be used in CDL netlisting. This tab is supported only if the `nlCDLSupportMapNetNames` preference is set to `true`. This table contains the source–destination names pairs according to the preference `nlCDLMapNetNamesData`, and can be edited.

Feedback

Note:

This specified mapping has lower priority than the mapping rules caused by the database (like, nets short) or netlisting settings (as in, names after uppercase or lowercase conversion).

7. For exporting in VerilogICC **Netlister Format**, use the **Power** section of the **Main** tab.



Feedback

a. To list the supply nets for a given view and netlist, click **Find Supplies**.

   If supply nets exist, the system lists them in the **Supply Nets** tab by **Net Name**, **ICC Name**, and **Supply Type**.

b. To optionally edit the IC Compiler name for a net, under **ICC Name**, click the net name, and enter text.

c. Consider netlist output options:

   To output one netlist that includes all supply nets, leave the **Write UPF** option unchecked.

   If you chose the **Multiple Supplies** option for place and route, you can output one netlist without supply nets and a second netlist in UPF format with supply nets. To do this, choose **Write UPF**.

   For UPF output, the system shows additional tabs for **Domains** and **Black Boxes**. Under **Domains**, you can edit a domain name by clicking the name and entering text.

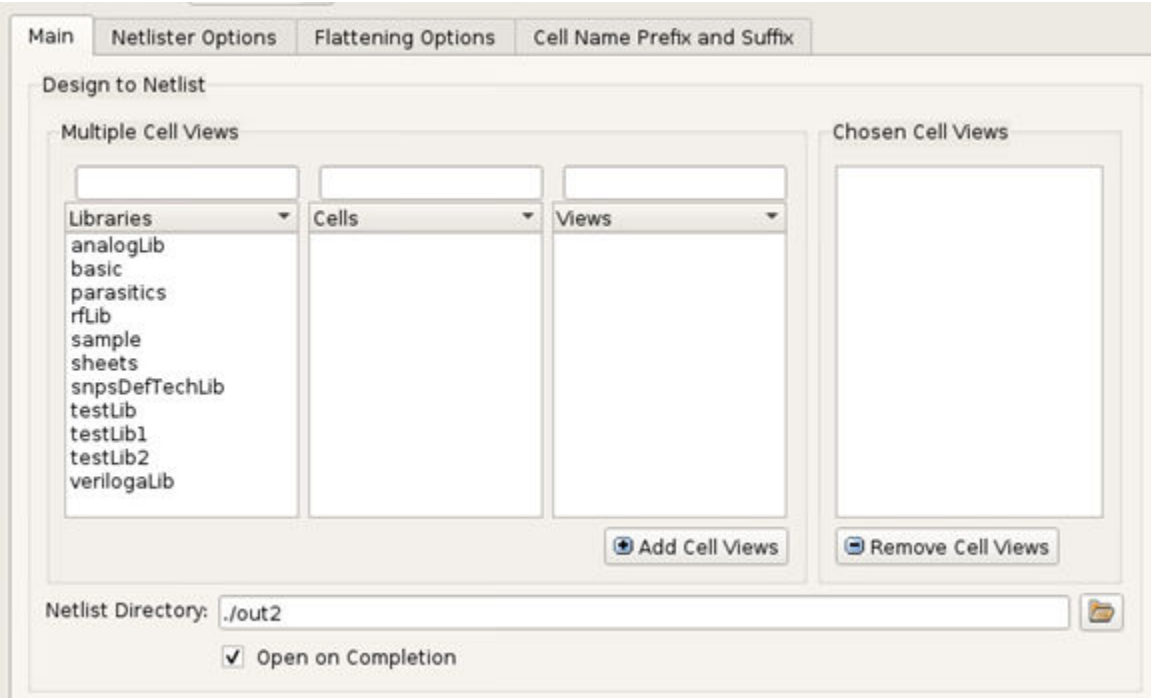
8. Click **OK**.

   The exported netlist file opens in the Text Viewer. See Text Viewer Tool.

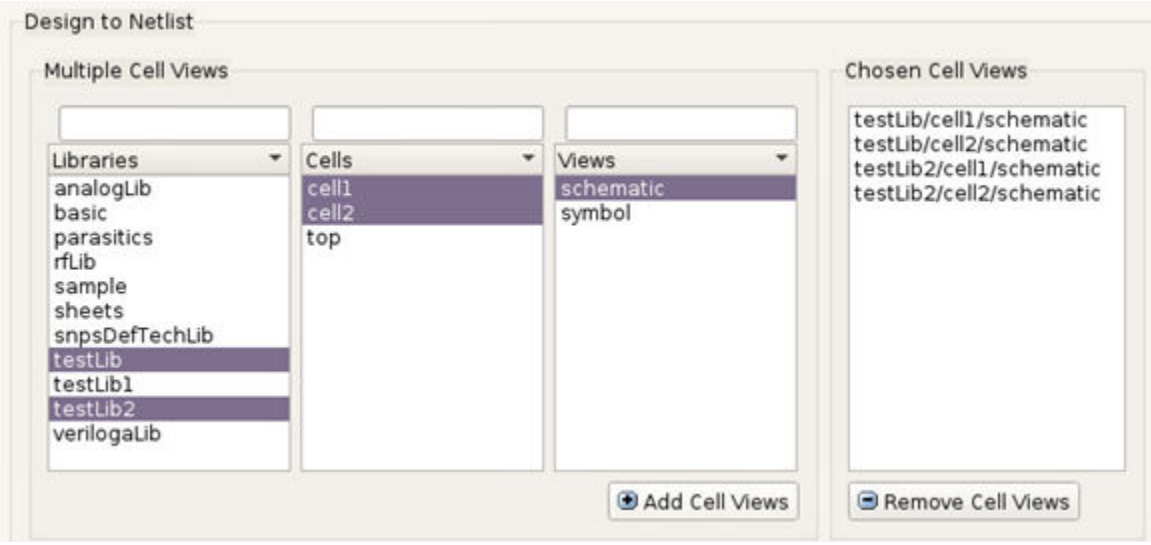
   If you also chose to write UPF output, the Text Viewer adds a second tab for the UPF file.

Selecting Multiple Cellviews

To select multiple cellviews, set the `dbExportNetlistMultipleLCV` preference to `true` (default: `false`). A new dialog box opens with the **Multiple Cell Views** panel in the **Design to Netlist** section instead of a single cellview selector, as shown in the following figure.



Select multiple **Libraries**, **Cells**, **Views** and click the **Add Cell Views** button to add all the selected cellviews to the **Chosen Cell Views** list, as shown in the following figure.



To remove some cellviews from the **Chosen Cell Views** list, select the cellviews in this list and click **Remove Cell Views** button.

The netlisting sessions for all chosen cellview are run in sequence and the corresponding message is shown in the Console.

The output folder is the same for all session, which is set in the **Netlist Directory** textbox in the **Main** tab, and the file name for each session is derived automatically according to the `dbExportNetlistFileNameFormat` preference.

Some design-related features are disabled in the multiple cellviews mode.

For example, **Power** panel for VerilogICC and temporary cellview selection for flatten netlist.

6

General Designing

This chapter describes how to create, open, navigate, edit, save, and run processes on designs. You can also view design and component summaries, and save your work. At any time, you can print and plot designs, and export design images. Close designs to complete each session.

This chapter contains the following topics:

- Creating Designs
- Opening Designs
- Navigating the Design Hierarchy
- Viewing the Design
- Using Filter Boxes
- Changing Edit Modes
- Reverting to Last-Saved Edits
- Saving Your Designs
- Displaying Design Summary Information
- Printing and Plotting Designs
- Exporting Design Data
- Exporting Design Images
- Closing Designs
- Configuring Custom Cells

Creating Designs

Create designs by creating cellViews from the OpenAccess database.

Feedback

To create a design:

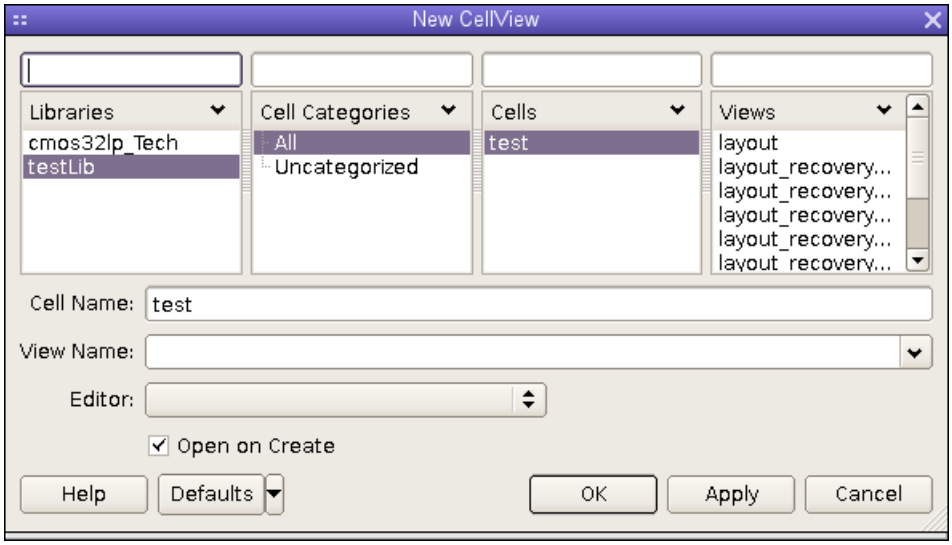
1. From the **Console** tab or the Library Manager, choose**File > New > CellView**.

   Alternatively, in the Layout Editor, choose **Design > New > CellView**.

   Note:

   The **Design > New > CellView** command is hidden if the design library is read-only or if you are in a read-only session.

   The **New CellView** dialog box opens.



2. In the **Libraries** list, choose the library to hold the cellView.

   You can filter the list of libraries by typing the library name in the empty box above the Libraries list. The system treats empty spaces on both ends of a string as wildcards.

3. Provide the following information for the new cellView:

CellView Information	Description
Cell Name	Name of the new cellView that you are creating.
View Name	Representation of the cell. For example, a cell can have a schematic view, a layout view, or a symbol view.
Editor	Product to use to open the new cellView.
Open on Create (check box)	Opens the new cellView when you click **OK**.

4. Click **OK**.

Feedback

The new design cellView opens in the specified product.

Creating a design creates a set of oaDesign objects in OpenAccess to contain design netlists, schematics, layouts and other types of device representations. The oaDesign objects hold the connectivity, geometry, hierarchy, parasitics, and floorplanning information for a design.

Opening Designs

Open a design cellView to view or edit it. Use the **Open Design** dialog box to also open DMFiles and associated application.

Note:

To improve the rendering performance for very large designs, set the Tcl preferences, `deEnableMouseMotionInterrupt` to `true` and `deRuntimeUpdateFrequency` to `500`.

These sections cover:

- From the Console Tab
- From an Editor Window
- From the Library Manager
- Opening a Bookmarked Design
- Responding to Warnings while Opening Designs
- Reopening Recently Opened Designs

Caution:

When opening legacy designs created in third-party tools, the Layout Editor automatically disables connectivity of routes that are not well formed. You can manually inspect the routes that are not well formed and decide whether you want to keep the routes or delete them.

Note:

If the design permission is set to read-only, some menu commands might be unavailable.

Feedback

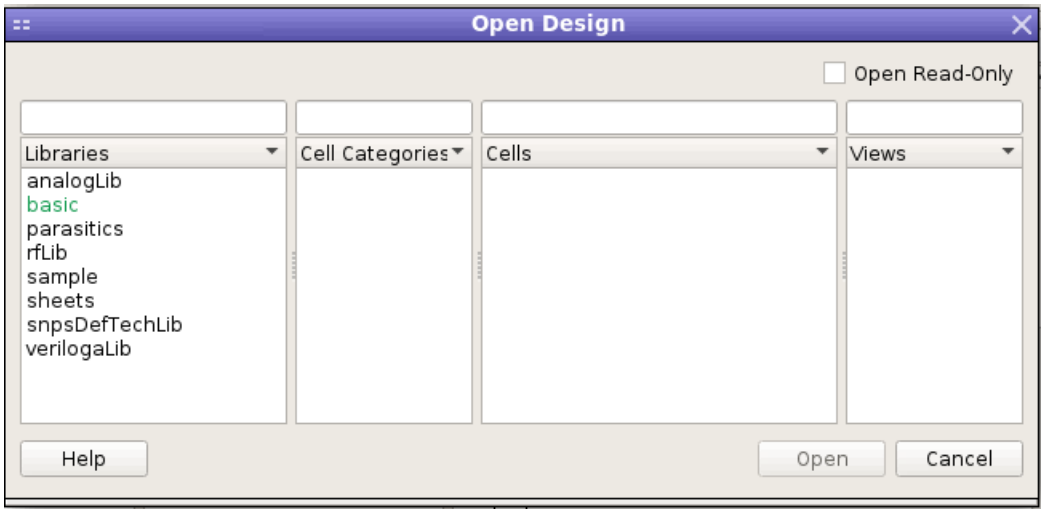
From the Console Tab

To open a design from the **Console** tab:

1. Choose **File > Open Design**.
2. Specify the names of the library, category (optional), cell, and view.

   Entering text in an input field at the top of a column filters the set of names offered. To return to the full list, delete text in the input field.

   The unique combination of library, cell, and view name describes a design.



3. Click **Open**.

   Alternatively, you can double-click the view name, or right-click and choose **Open**.

The design opens in the editor for this view.

Preference options apply. Active cellView level option settings also apply and take precedence.

From an Editor Window

To open a design from an editor:

1. Choose **Design > Open**.
2. Specify the names of the library, category (optional), cell and view.

   Entering text in an input field at the top of a column filters the set of names offered. To return to the full list, delete text in the input field.

Feedback

3. Click **Open**.

   Alternatively, you can double-click the view name, or right-click and choose **Open**.

The design opens in the editor for this view.

Preference options apply. Active cellView level preference settings also apply, and take precedence.

From the Library Manager

To open a design from the Library Manager:

1. In the Library Manager, specify the names of the **Libraries**, **Cell Categories** (optional), **Cells**, and **Views**.

   Entering text in an input field at the top of a column filters the set of names offered. To return to the full list, delete text in the input field.

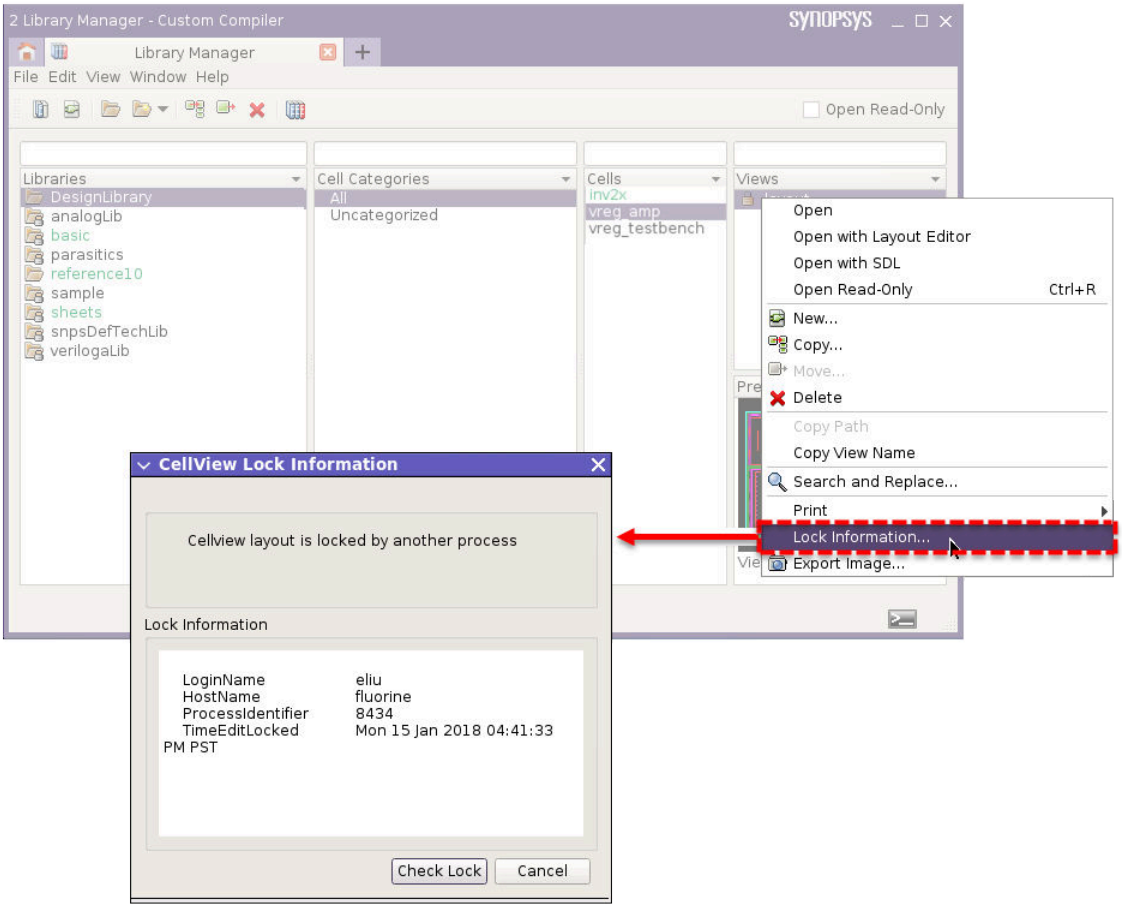
2. In the **Views** area, right-click the view name and select one of the following:
   - **Open**: Opens the design in the default view and in full-access mode. The default view is determined by the `dmDefaultView` Tcl preference setting.
   - **Open with Layout Editor**: Opens the design in the Layout Editor.
   - **Open with SDL**: Opens the design in SDL.
   - **Open Read-Only**: Opens the design in read-only mode.

The design opens in the editor for this view.

Preference options apply. Active cellview-level preference settings also apply, and take precedence.

If the cellview has a lock icon ( ), the cellview is currently opened in an editor or had been edited by a process that terminated abnormally. Right-click the cellview name and choose **Lock Information**. In the **CellView Lock Information** dialog box, you can:

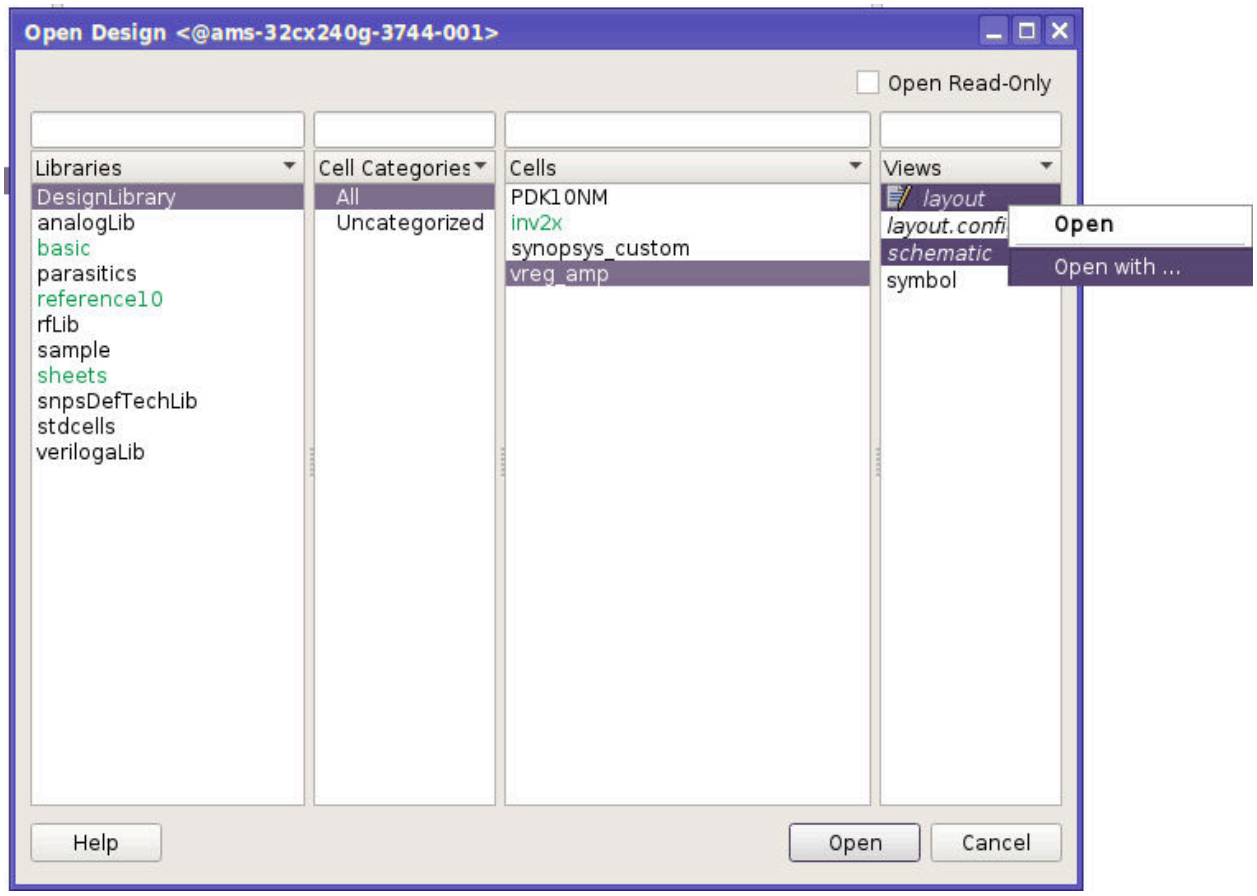
- Find out which user is currently editing the cellview.
- If the cellview had been edited by a process that terminated abnormally, click **Check Lock** to release the lock.


2 Library Manager - Custom Compiler
SYNOPSYS
Library Manager
File Edit View Window Help
Open Read-Only
Libraries
DesignLibrary
analogLib
basic
parasitics
reference10
sample
sheets
snpsDefTechLib
verilogaLib
Cell Categories
All
Uncategorized
Cells
inv2x
vreg_amp
vreg_testbench
Views
Open
Open with Layout Editor
Open with SDL
Open Read-Only Ctrl+R
New...
Copy...
Move...
Delete
Copy Path
Copy View Name
Search and Replace...
Print
Lock Information...
Export Image...
CellView Lock Information
Cellview layout is locked by another process
Lock Information
LoginName eliu
HostName fluorine
ProcessIdentifier 8434
TimeEditLocked Mon 15 Jan 2018 04:41:33 PM PST
Check Lock
Cancel

Feedback

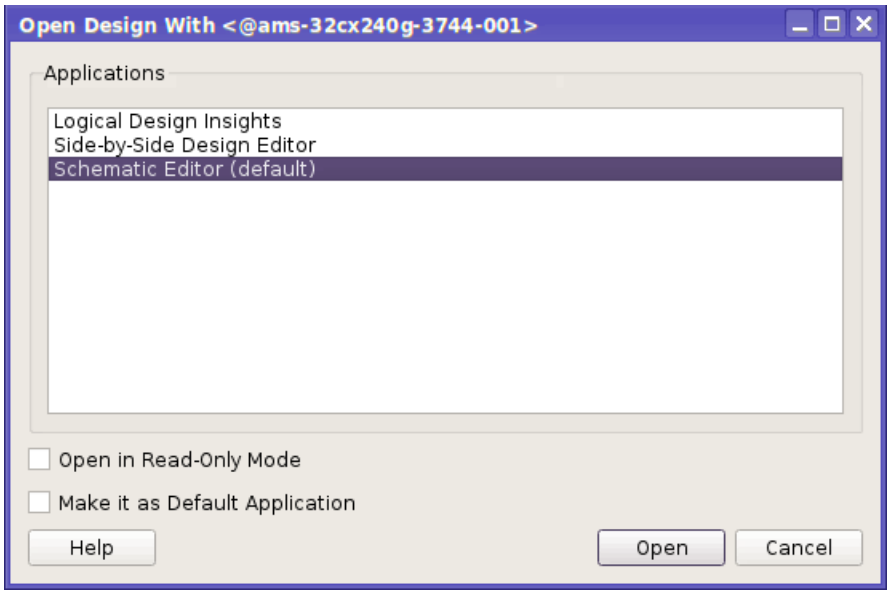
Opening Two Designs

1. In the Library Manager, choose **File > Open**. The **Open Design** dialog box opens.
2. Select the names of two views.
3. Right-click on any one of the selected view names and select **Open with**.



Feedback

The **Open Design With** dialog box opens:



4. Select the application of your choice in the **Applications** area from the list of the registered applications for the view type. The application in which the design is open is set as the default.
5. (Optional) Select the **Open in Read-Only Mode** option to open the designs in read-only mode.
6. (Optional) Select the **Make it as Default Application** option to set the selected application as the default.
7. Click **Open** to open the design in the selected application and to close the **Open Design With** dialog box.

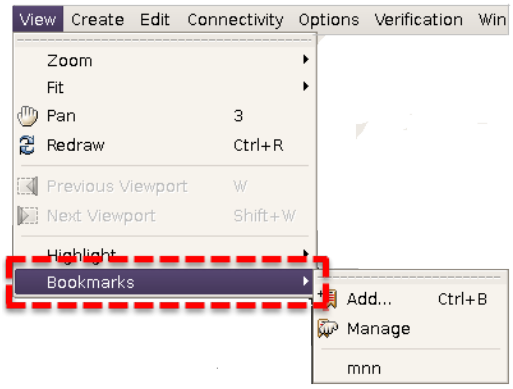
Opening a Bookmarked Design

Open a bookmarked design when you want to view a previously saved design data and its viewport combination.

To open a bookmarked design:

► In any design window, choose **View > Bookmarks >** *bookmark_name*.

Feedback



See Also

- Adding a Bookmark
- Bookmark Manager Tool

Responding to Warnings while Opening Designs

Respond to warnings while opening designs to correct problems the system detects when it scans the design before it opens.

To respond to warnings:

1. Begin by Opening Designs.

   For schematic and symbol views, the system scans the entire placed hierarchy. For layouts, the system scans the placed hierarchy down to the view depth.

2. If the system detects problems, a warnings dialog box lists them.

   Two types of warnings can occur:

   - Missing references, in both schematics and layouts.

     These are references from the design at the current view depth to instance and via objects not at located this view depth. The system sees these as unbound to the design. Their definitions are not found in the current technology file.

     To address the underlying problem, check to make sure the design uses the correct technology library, or add the object to the current technology library.

     Unbound instances occur when the corresponding OpenAccess instance header (oaInstHeader) points to a non-existent master design. Unbound vias occur when an OpenAccess via definition (oaViaDef) is not found, or the master design for an OpenAccess custom via definition (oaCustomViaDef) is not found.

- Out-of-date multi-part paths (MPPs) in layouts.

  An MPP definition differs between the design and the technology file.

3. Choose an option.
   - For missing references, you can visually identify the affected objects by adding them to the selected set: Enable **Select Missing References**.
   - For out-of-date MPPs, you can update them: Choose **Update MPPs**.
4. Click **OK**.

Reopening Recently Opened Designs

Reopen a design by choosing it from the accumulated list of recently opened designs.

To reopen a design:

► In any design window, choose **Design > Open Recent**, and select the design by its library, cell, and view.

Navigating the Design Hierarchy

Navigate designs to descend to varying levels of device representation. You can descend one instance at a time, and you can navigate using alternate methods.

These sections cover:

- Navigating One Instance at a Time
- Navigating Using Alternate Methods

Navigating One Instance at a Time

Navigate one instance at a time to descend directly into the design. For information on using the `ide::descend` Tcl command, see Design Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

The navigation commands of the Layout Editor and Schematic Editor are cascaded in different menus, but they work similarly in different contexts.

Layout Editor	Schematic Editor or Symbol Editor
Hierarchy > Descend	**Design > Hierarchy > Descend Edit**
-	**Design > Hierarchy > Descend Read**

Feedback

Layout Editor	Schematic Editor or Symbol Editor
-	**Design > Hierarchy > Descend Out of Context**
Hierarchy > Edit in Place	**Design > Hierarchy > Edit in Place**
Hierarchy > Edit in Group	-
Hierarchy > Return	**Design > Hierarchy > Return**
Hierarchy > Return to Level	-
Hierarchy > Return to Top	**Design > Hierarchy > Return to Top**

To navigate a design one instance at a time:

1. To descend to the next level away from the root for editing, choose the **Descend** or **Descend Edit** command and then select an instance. The **Descend** toolbar appears.

   Note:

   The **Descend** or **Descend Edit** command is hidden if the design library is read-only or if you are in a read-only session.

   The **Descend** toolbar contains different options in the Layout Editor, Schematic Editor, and Symbol Editor.

   The following figure illustrates the **Descend** toolbar for Layout Editor.

   The toolbar contains the following options:

Option	Description
New Window	Descends in a new window. Tcl preference: `deDescendLayoutNewWindow`
Open Mode	When **Auto** (default) is selected, determines the descend mode automatically. When **Edit** or **Read-Only** is selected, enables Edit or Read-Only mode for the descended instance. Tcl preference: `deDescendMode`
Descend Lineage	Specifies a lineage during descend if more than one choice is available. Tcl preference: `deDescendLineage`

Feedback

Option	Description
Shadow	Turns on Shadow mode to increase the contrast between the selected lineage and the remaining objects during descend.

The following figure illustrates the **Descend** toolbar for Schematic Editor and Symbol Editor.

The toolbar contains the following options:

Option	Description
Descend	Specifies if the target descend type is **Instance** (default), **Group** (figure group), or **Both**. Tcl preference: `deDescendObjectType`
Edit In Place	Enables the Edit In Place mode. This option is ignored if **New Window** is selected because it takes precedence. Tcl preference: `deDescendInPlace`
New Window	Descends in a new window. Mutually exclusive with **Edit In Place**. Tcl preference: `deDescendNewWindow`
Open Mode	When **Auto** (default) is selected, determines the descend mode automatically. When **Edit** or **Read-Only** is selected, enables Edit or Read-Only mode for the descended instance. Tcl preference: `deDescendMode`
Prompt for View	Prompts for view. Tcl preference: `deDescendDialog`

2. (Schematic or symbol view only) To descend to the next level away from the root for reading in the schematic, choose **Design > Hierarchy > Descend Read** and select an instance.
3. (Schematic or symbol view only) To descend and also switch the view, choose **Design > Hierarchy > Descend Out of Context**, select an instance, and choose the view.
4. To edit the instance in the same window, choose **Edit In Place** from the appropriate menu. The **Descend** toolbar appears.

   The **Descend** toolbar for layout views looks slightly different than the one previously described. From this toolbar, you can set the target descend type as **Instance** (default),

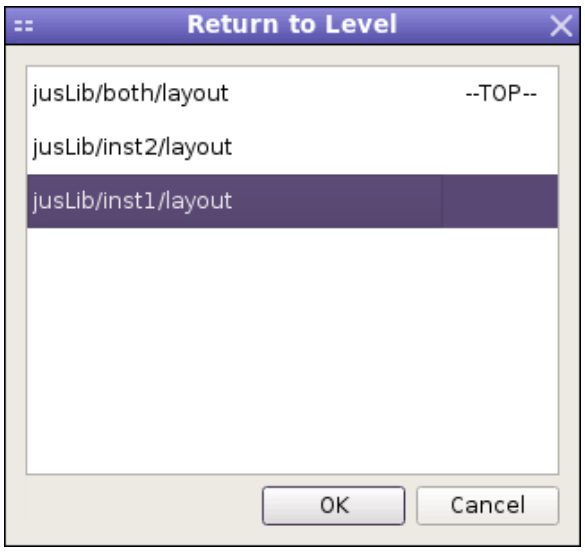
Feedback

Group (figure group), or **Both**. The **Group** option corresponds to the **Hierarchy > Edit In Group** command.

If the instance is in an instance array, complete the operation by additionally choosing a component instance.

For typical descent operations, the *edit* design and the *view* design are set to the new design context, which is the instance's switched master. For edit in place operations, only the edit design is modified and set to the instance's placed master.

5. To reverse the last descent, choose **Return** from the appropriate menu.
6. (Layout view only) To return to a specific level, choose **Hierarchy > Return to Level**. The **Return to Level** dialog box appears.



7. To return to the design root level, choose **Return to Top** from the appropriate menu.

Navigating Using Alternate Methods

Use alternate navigation methods to have additional control while navigating.

- If you want to see all instances or masters in a hierarchy and select the descent level from the list, use the Hierarchy Navigator. See Hierarchy Navigator Assistant.
- If you want to configure a custom hierarchy that reflects the design, use the Hierarchy Editor. See Hierarchy Editor Tool.
- If you want to navigate by net name, use the Design Navigator. See Design Navigator Assistant.

Viewing the Design

When you edit a design, you can use canvas menu options to zoom to different scales, fit the image size to an area, pan across the design, redraw after making changes, overlay fill views, and switch viewports.

At any time, you can also bookmark a design data and viewport combination. See Adding a Bookmark.

- Zooming
- Magnifying
- Peeking
- Fitting
- Panning
- Redrawing Designs
- Switching to Previous or Next Viewports
- Interrupting Rendering and Layout Edits with Ctrl+C

Zooming

You can zoom to magnify the design view, changing the viewport scale by the ratio of viewport pixels to database units (DBUs).

- To magnify a canvas area, choose **View > Zoom > Area**.

  Then use the mouse to set one point and rotate the mouse wheel or drag the mouse to adjust the scale.
- To increase magnification of the canvas area, choose **View > Zoom > In**, and rotate the mouse wheel forward.
- To reduce current magnification of a canvas area, choose **View > Zoom > Out**, and rotate the mouse wheel in reverse.

Alternatively, you can zoom in or out of the canvas area using your mouse.

- To zoom in, right-click and drag the cursor over the canvas area you would like to magnify.
- To zoom out, hold Shift as you right-click and drag the cursor over the canvas area you would like to demagnify.

Feedback

Magnifying

The **Magnify** command allows you to magnify portions of a design without zooming in on them.

To magnify a design:

1. In a design, choose **View > Magnify** or click the **Show magnified design** button .
2. Select the **Size** of the magnify window. Options are **Small**, **Medium**, and **Large**.
3. Choose whether to **Zoom** in on an area of the design in the magnify window or **Fit** the magnified area into the design window. **Zoom** options are **2x**, **3x**, **5x**, and **10x**. Fit options are **150 px/uu**, **250 px/uu**, **400 px/uu**, and **600 px/uu**.
4. To lock the placement of the peek window, check the **Position** option and choose **Auto**, **Near Cursor**, **Top Left**, **Top Right**, **Bottom Left**, or **Bottom Right**.
5. In the design, hover the cursor over a portion of the design to view its details in the magnify window. Click the canvas to view that portion of the design fitted or zoomed into the design window.
6. Click the **Magnify** button again to return to the original view of the design.
7. Press Esc to exit the command.

Peeking

Note:

Peek functionality is only available for schematic designs.

The **Peek** command allows you to visually descend into a design without actually navigating lower in the design's hierarchy.

To peek into a design:

1. In a design, choose **View > Peek** or click the **Peek into master design** button .
2. Select the **Size** of the peek window. Options are **Small**, **Medium**, and **Large**.
3. To lock the placement of the peek window, check the **Position** option and choose **Auto**, **Top Left**, **Top Right**, **Bottom Left**, or **Bottom Right**.
4. In the design, hover the cursor over an instance to peek into its hierarchy.

5. To cycle through all available views in the peek window, use the shortcut keys Alt+P (for previous) and Alt+N (for next).
6. Press Esc to exit the command.

Alternatively, you can activate the **Peek** command by hovering over an instance and pressing the T shortcut key. When using the shortcut key, your most recent settings determine the window size and position.

Fitting

You can fit a design viewport to specified extents to magnify the image size.

- To fit the viewport to the extent of the design, choose:

  View > Fit View in a design schematic.

  View > Fit > View in a design layout.

- To fit the viewport to the extent of an Edit-in-Group mode figure group, choose:

  View > Fit Edit in a design schematic.

  View > Fit > Edit in a design layout.

- To fit the viewport to the set of selected objects, choose:

  View > Fit to Selected in a design schematic.

  View > Fit > Selected in a design layout.

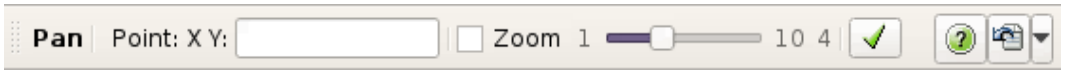
- In design layouts (only), to leave the center point unchanged and fit the viewport to the finest available grid, choose **View > Fit Grid**.

Panning

You can pan to move the viewport to a different position on the canvas without changing the scaling factor. Use panning to view parts of the image not visible in the current window view.

To pan a design:

1. In a design, choose **View > Pan**. The **Pan** toolbar appears.



Feedback

You can use the coordinate inputs along with an optional zoom level to pan to coordinate. The **Pan** toolbar includes the following options:

Settings	Description
Point X Y	The point input is a space delimited point (2 floating-point number). You may enter any of the following formats: ◦ x y ◦ x,y ◦ (x y) ◦ (x,y) To aid cut/paste from a coordinate file, the value entered for panning removes the following characters when the point field focus loses focus: `"{}():,`
Zoom	When disabled (default), the zoom level is disabled. Apply requires both X and Y coordinates to be valid values in user units within the allowed coordinate space. When enabled, you can specify the zoom level on the toolbar, or you can still press and slide the cursor on the canvas. Tcl preference: `dePanZoom`
Apply	Applies the changes.
Defaults	Resets all of the options to your Custom Compiler environment default values.

2. Press and slide the cursor on the canvas, or use the arrow keys.

Alternatively, you can pan a design by holding Ctrl as you right-click and drag the cursor around the canvas.

Redrawing Designs

You can redraw a design to show recent changes.

To redraw a design:

► In a design window, choose **Edit > Redraw**.

Switching to Previous or Next Viewports

You can switch viewports to move between design contexts.

The system saves the viewport for each open design in the context and restores former viewports upon ascending after descending into a hierarchy.

Feedback

You can switch to the next and previous viewport.

- To switch context to the previous viewport, choose **View > Previous Viewport**.
- To switch context to the next viewport, choose **View > Next Viewport**.

Interrupting Rendering and Layout Edits with Ctrl+C

You can use the shortcut key Ctrl+C to interrupt a rendering process or an active command taking a long time to execute. If the application is not in either of these states, the shortcut key Ctrl+C triggers the **Copy to Clipboard** command.

To interrupt a create instance array operation:

1. In any design layout view, choose **Create > Instance**.

   The **Instance** toolbar opens.

2. Press F3 to bring up the **Create Instance** dialog box.
3. Choose an instance from the required library, cell, and view, and then specify the number of columns and rows to create a 500x500 instance array.
4. (Optional) Pressing Ctrl+C now activates the **Copy to Clipboard** command.

   Press Esc to end the **Copy to Clipboard** command and return to the **Create Instance** command to continue your work.

5. Move back to the canvas and click on the required location to place the instance array.
6. Use the shortcut key Ctrl+C to stop the operation if it takes too long to wait.

   The **Create Instance** command exits after the interrupt event.

Using Filter Boxes

Filter boxes are available in assistants and tools such as the Design Navigator, Object/Layer Panel, and Library Manager. The filter box filters the list directly below the text box to find the closest match, thus you can organize lists and locate items quickly.

These sections cover:

- Filtering Lists
- Displaying Selections in Filter Boxes

Feedback

Filtering Lists

You can use filter boxes to specify which items are displayed in an item list.

To filter lists:

- In the filter box, enter the full or partial name of the item or object you want to select using a simple string match, a glob-style pattern, or a regular expression which is controlled by the **Pattern Matching** option in the **General Options** dialog box. For help on setting the **Pattern Matching** option for filter boxes, see Setting General Options.

  When you enter a partial word, the list displays all the items that contain the partial word.

For example, to find all route-related objects in a list of object types in the Object/Layer Panel, first click the **Objects** tab, and then enter `route` in the filter box.

The tool treats empty spaces on both ends of any string as wildcard characters. All searches or auto fills are case insensitive by default. However, if at least one uppercase letter is included, case sensitivity appears.

To clear a filter box, click X as shown in the following figure.

Displaying Selections in Filter Boxes

You can set the Library Manager filter boxes to display single-selected items from a list. This feature is controlled by the `dmShowSelectedInFilter` Tcl preference.

Note:

Using this feature changes the behavior of the Library Manager filter boxes. While item names are displayed in the filter box, the item list remains unfiltered.

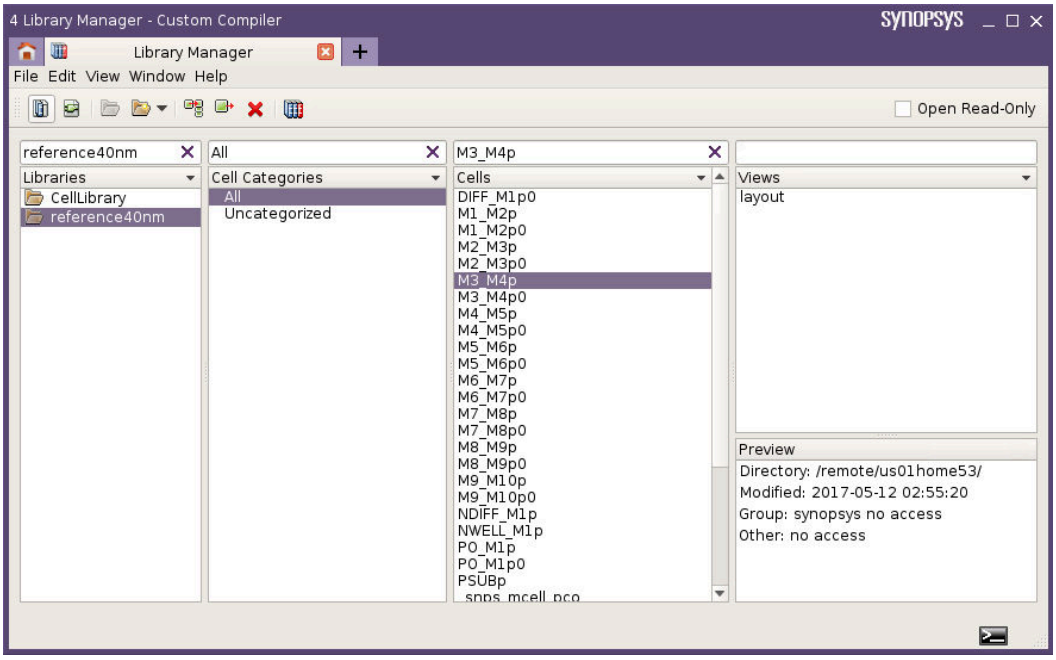
To display selection in filter boxes:

- Set the `dmShowSelectedInFilter` Tcl preference to `true`, as shown in the following example:

  ```
  db::setPrefValue dmShowSelectedInFilter -value 1
  ```

  The tool displays single-selected objects in the Library Manager filter boxes.

Feedback



Changing Edit Modes

Change the edit mode to switch the editing context between read-only and write-enabled.

To change the edit mode:

- In a design window, choose **Design > Make Read-Write** or **Design > Make Read-Only**.

If the design is currently write-enabled, you can save recent changes before switching to read-only mode.

For other ways to restrict editing of designs, see Restricting Design Access to Read-Only.

Reverting to Last-Saved Edits

Discard edits when you want to revert the design to the last saved state.

Complete this single step:

- In any design window, choose **Design > Discard Edits**.

Alternatively, you can revert the design to any previous state in this session by using the Transaction History assistant. See Transaction History Assistant.

Feedback

Saving Your Designs

These sections cover:

- Saving Designs
- Saving Designs With Hierarchy
- Saving Designs With New Names
- Saving Designs With Schematic and Config Views
- Saving Hierarchical Design Layouts
- Checking and Saving Individual Design Schematics
- Checking and Saving Designs Represented as Text
- Checking and Saving All Designs in Libraries
- Saving Designs With Backup
- Automatically Saving Designs
- Retrieving Your Designs

Saving Designs

Save a design to retain edits made after the last save operation.

Complete this single step:

- Choose **Design > Save**.

The system saves all design views opened from this context, committing all in-memory changes to disk, and storing a record of all undo operations performed after the last save operation.

Saving Designs With Hierarchy

Save a design with hierarchy to retain edits made after the last save operation.

To save a design hierarchy:

- Choose **Design > Save Hierarchy**.

The system saves all open cells down the hierarchy without prompting. This feature is available only in the layout view.

Feedback

Saving Designs With New Names

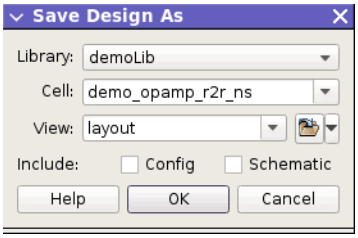
Save a design with a new name to create an additional version with the most recent changes.

To save a design with a new name:

1. In the design editor, choose **Design > Save As**.
2. To save this design in a different library, or as a different cell or view, select options from the Library, Cell, and View lists.
3. Click **Save**.

Saving Designs With Schematic and Config Views

Save a design with a new name to create an additional version with the most recent changes and also create a schematic view and a config view for SDL design.



To save a design with a schematic view and config view:

1. In the design editor, select **Tools** > **SDL**.
2. In the design editor, choose **Design** > **Save As**.
3. To save this design in a different library, or as a different cell or view, select options from the Library, Cell, and View lists. To save this design with config view and schematic view, select the **Config** or **Schematic** options. See `lxSaveAsCopyConfig` and `lxSaveAsCopySchematic` in Schematic Driven Layout Tcl in the *Custom Compiler Tcl Reference Manual*.
4. Click **OK**.

Saving Hierarchical Design Layouts

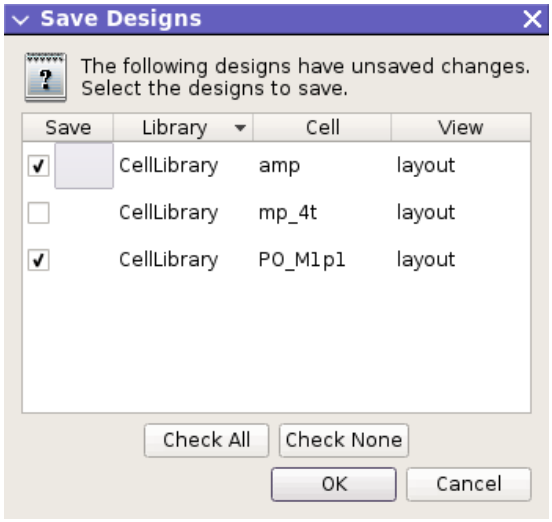
The **Save Designs** dialog box lets you choose which designs to save in a modified hierarchy. To show the dialog box when saving, set the `dePromptSaveHier` preference to `true`.

Feedback

To save hierarchical design layouts:

1. From the top-level design, choose **Design > Save**.

   The **Save Designs** dialog box opens if at least one lower-level design is modified.



2. Select the designs you want to save.
3. Click **OK**.

Checking and Saving Individual Design Schematics

You can check and save a schematic to confirm its compliance with electrical and semantic rules.

To check and save a design:

► In the design schematic, choose **Design > Check and Save** or click **Check and Save** on the **Standard** toolbar. When **Check and Save** is clicked, the `ise::check` and `de:save` commands are executed, which runs checks, creates markers, and ensures the design is saved. The Save icon without a tick ( )indicates that the design is saved.

If no errors are found, the Console reports information noting that no rule violations were found.

If errors are found, a dialog box opens listing the number of errors, and the Marker Browser tool opens to assist with locating them. You can choose to **Save** anyway, or you can click **Cancel**, and fix the errors before checking and saving again.

Feedback

Checking and Saving Designs Represented as Text

To check and save a design's HDL format:

► In the text editor, click **Check and Save** on the **Standard** toolbar to perform syntax checking and cross view checks before saving the design.

If no errors are found, the Console reports information noting that no warnings were found.

Checking and Saving All Designs in Libraries

You can check and save all schematic designs in libraries to validate them for compliance with electrical and semantic rules. This operation completes by saving all designs in the library. It is useful after importing design libraries exported from other tools.

To check and save libraries:

1. Open the Library Manager.

   From the **Console** tab or any design window, choose **Tools > Library Manager** or click the **Library Manager** button.

2. Choose **Edit > Check and Save Libraries**.

3. Specify check and save settings as described in Table 57.

Table 57 Library Check and Save Settings

Settings	Description
Force Checks	Controls whether a design will first be checked to see if the rules need to be executed.
Check Only	Checks designs, but does not save them.
Remove all Markers	Removes markers set by all tools.
Prompt for Save	Prompts to save the design when errors are found, and lists the number of errors. Otherwise, saves automatically.
Views	To include specific views in the check and save operation, enter them here, and leave the **Exclude** check box unchecked.
Exclude	To exclude specific views from the check and save operation, choose the **Exclude** option, and enter the views by name in the **Views** box.

4. Specify libraries. You can filter the list, and use the buttons to specify libraries.

Feedback

- To select all listed libraries, click **Check All**.
- To deselect all libraries, click **Check None**.
- To hide the libraries you have not selected, click **Hide Unchecked**.

5. (Optional) To see a preview of the selected list of libraries and designs, click **Preview**. You can further filter the set of designs to check, and optionally click **Run** to run the check and save operation.
6. After settings are complete, click **Run** to run the library check and save operation.

Saving Designs With Backup

You can save your designs and create multiple copies of your designs at different stages of the design.

During a particular session, you can specify the number of backup files for you to view. To enable the generation of backup files, use the `deMaxBackupCount` Tcl preference and specify the maximum number of backup files to be generated, as shown in the following example:

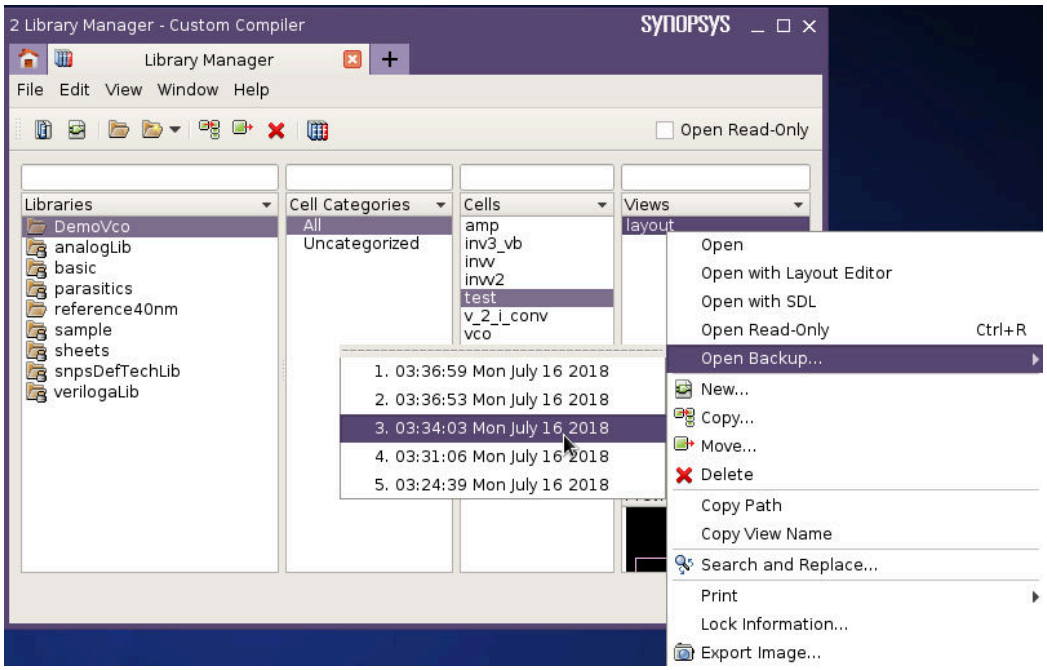
```
db::setPrefValue deMaxBackupCount -value 5
```

Each time you save your design, the tool saves the current updates in the cellView and generates a backup file of the previously saved design up to the maximum specified number. The default is 0, that is, no backup file is generated.

Feedback

To view the backup files:

1. From the Library Manager, right-click the selected view and choose **Open Backup**.



2. The backup files are displayed with their timestamps.

   Click the selected backup file to open. To retain the contents of the backup file, you must save it. Backup files are not retained after the current session.

The backup file format is compatible with the Laker file format. You can also open these files in Laker.

Automatically Saving Designs

You can automatically save your designs at specified intervals to minimize data loss caused by unforeseen events, such as, you might accidentally close your design without saving the updates.

To enable automatic saving, set the `dePeriodicSaveInterval` Tcl preference and specify the time interval between automatic save operations in minutes, as shown in the following example:

```
db::setPrefValue dePeriodicSaveInterval -value 1
```

The default is 0, that is, automatic saving is not enabled. The tool automatically saves your design at the specified time intervals, if the design is modified. The tool, however, retains only the latest automatically saved copy of the design.

Feedback

In case of any eventuality, you can retrieve the automatically saved design. See Retrieving Your Designs.

Retrieving Your Designs

You can recover your designs in case of an unexpected termination of the tool, such as due to a power outage. When you start a new session after such an event, you can access the latest manually-saved file (regular), retrieve the latest automatically-saved file (autosave) or the latest unsaved file before the tool terminated (recovery).

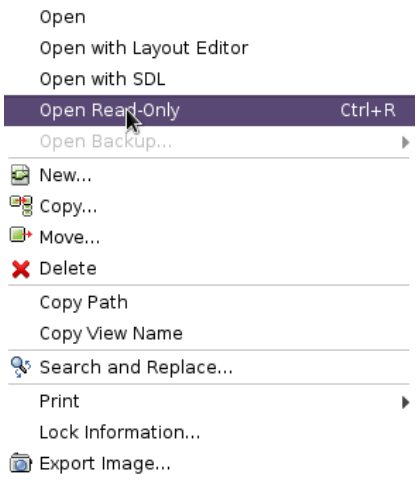
To view and open the available regular, autosave, or recovery files:

1. Optionally set the Tcl preference `deRecoveryFileType` to `regular`, `autosave`, or `recovery`, as shown in the following example:

   ```
   db::setPrefValue deRecoveryFileType -value recovery
   ```

2. In the Library Manager, select the name of the **Libraries**, **Cell Categories** (optional), **Cells**, and **Views**.

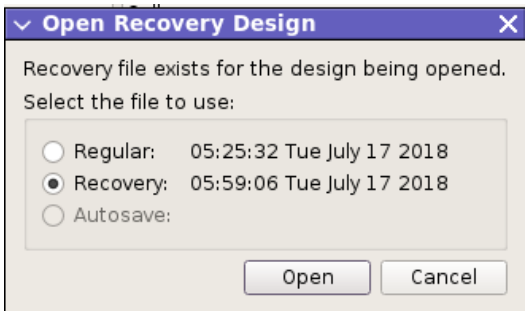
   Right-click the selected view and choose **Open Read-Only** or double-click the selected view to directly open the **Open Read-Only** dialog box.



3. The **Open Read-Only** dialog box displays the available files with the latest timestamps. When you set the Tcl preference `deRecoveryFileType` to `regular`, `autosave`, or `recovery`, the **Open Read-Only** dialog box appears with the corresponding file

Feedback

selected. To open a different file, click the corresponding radio button and choose **Open**.



The autosave and recovery files open in the read-only mode. You must save these to overwrite the regular design. The autosave and recovery file formats are compatible with the Laker file format. You can also open these files in Laker.

Displaying Design Summary Information

The **Design Summary** command generates a formatted list of OpenAccess objects contained in the active window's oaDesign.

This section describes the following topics:

- Generating a Design Summary Report in the Layout Editor
- Generating a Design Summary Report in the Schematic Editor

Generating a Design Summary Report in the Layout Editor

To generate a design summary report for layout designs:

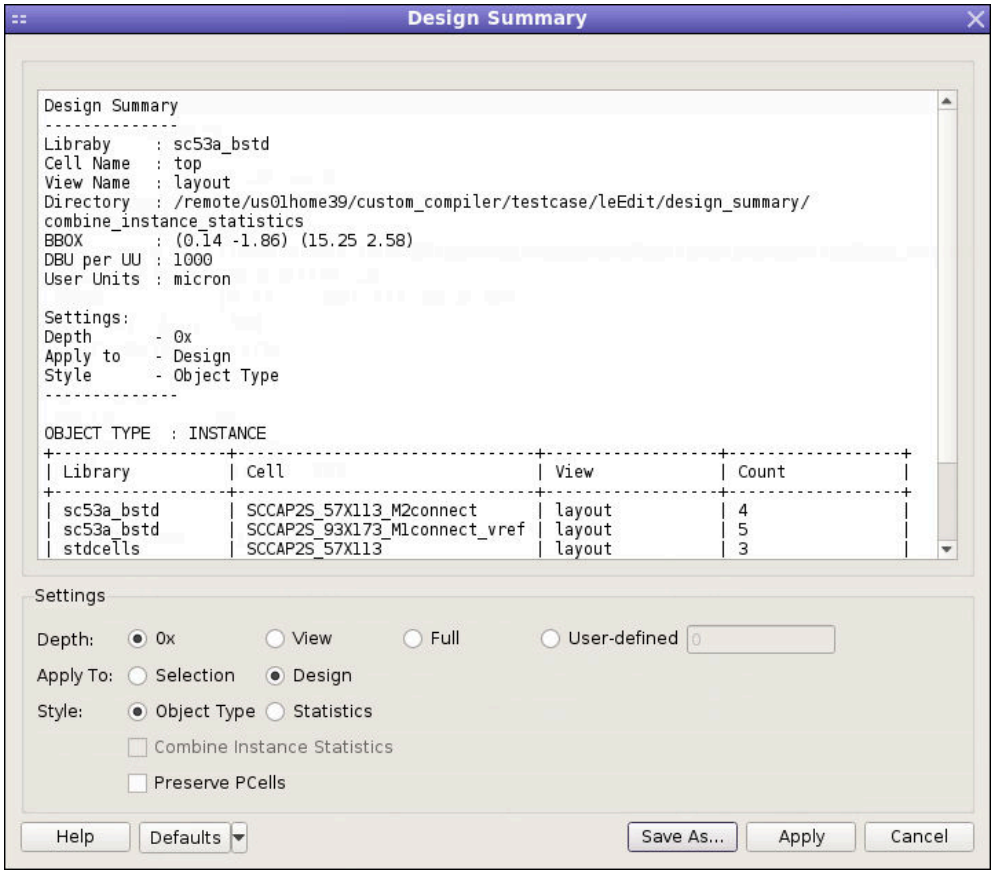
1. In the Layout Editor, choose **Design > Summary**.

   The **Design Summary** dialog box opens.

   For information on using the le::designSummary Tcl command, see Layout Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

Feedback

2. To view the design summary, click **Apply**.



The **Design Summary** dialog box shows a summary about the design and objects, including object count, object type, layer purpose pair (LPP), and logical information. In addition, you get the basic information of the current cell view, such as its library name, cell name, view type, working directory of the design, the lower-left and upper-right points of the bounding box, the value for database units (DBU) per user units (UU), user units, and other associated information.

3. Use the following options to reset the design information you want to display:

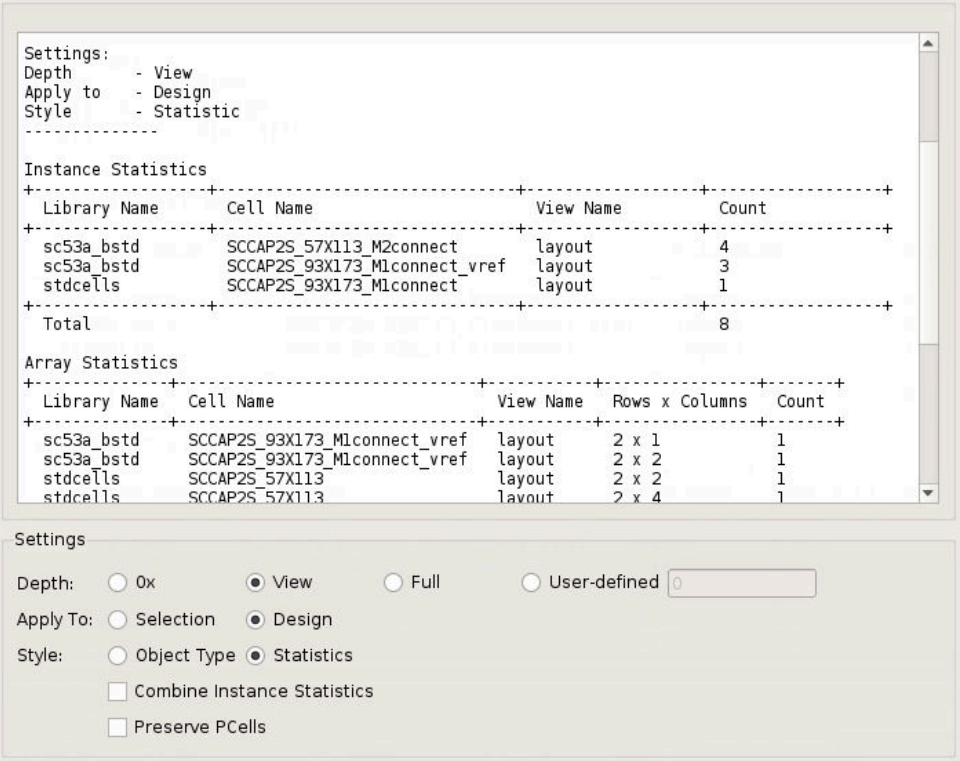
Settings	Description
Depth	Specifies a viewing level for the content. ◦ **0x** shows the information at the top view level. ◦ **View** shows the information at the current viewing level. ◦ **Full** shows the information at the maximum view level. ◦ **User-defined** shows the information at a specified view level. Tcl preference: `leDesignSummaryDepth`

Feedback

Settings	Description
Apply To	Specifies the mode to summarize all cell objects or selected objects. ◦ **Selection** displays the summary of selected objects. ◦ **Design** displays the summary of all cell objects. Tcl preference: `leDesignSummaryApplyTo`
Style	Combines statistics of instances and array instances. Specifies the summary format by choosing one of the following: ◦ **Object Type** (default) generates a formatted list of OpenAccess objects. ◦ **Statistics** generates a formatted list of statistics for the following object types: Shape, Instance, Via, Device, Boundary, Ruler/Coordinate Marker, Array Instance, and Blockage. Tcl preference: `leDesignSummaryStyle`
Combine Instance Statistics	Combines statistics of instances and array instances. Tcl preference: `leDesignSummaryCombineInstanceStatistics`
Preserve PCells	When selected, do not look down into PCells when getting the design summary. Tcl preference: `leDesignSummaryPreservePcells`
Save As	Click this button to save the summary information into a file. The default file name is *CustomCompilerSummary.log*.
Apply	Click this button to refresh the summary information in the dialog box.

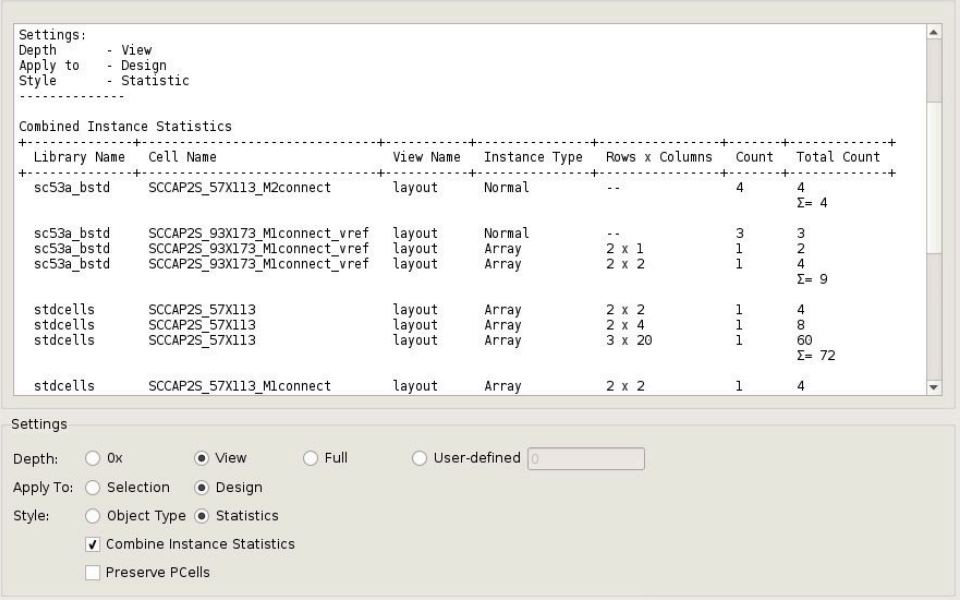
4. (Optional) Change the viewing level to **View**, choose **Statistics** for **Style**, and then click **Apply**.

   The summary shows statistics of instances and array instances as follows.



5. (Optional) Enable **Combine Instance Statistics** and click **Apply**.

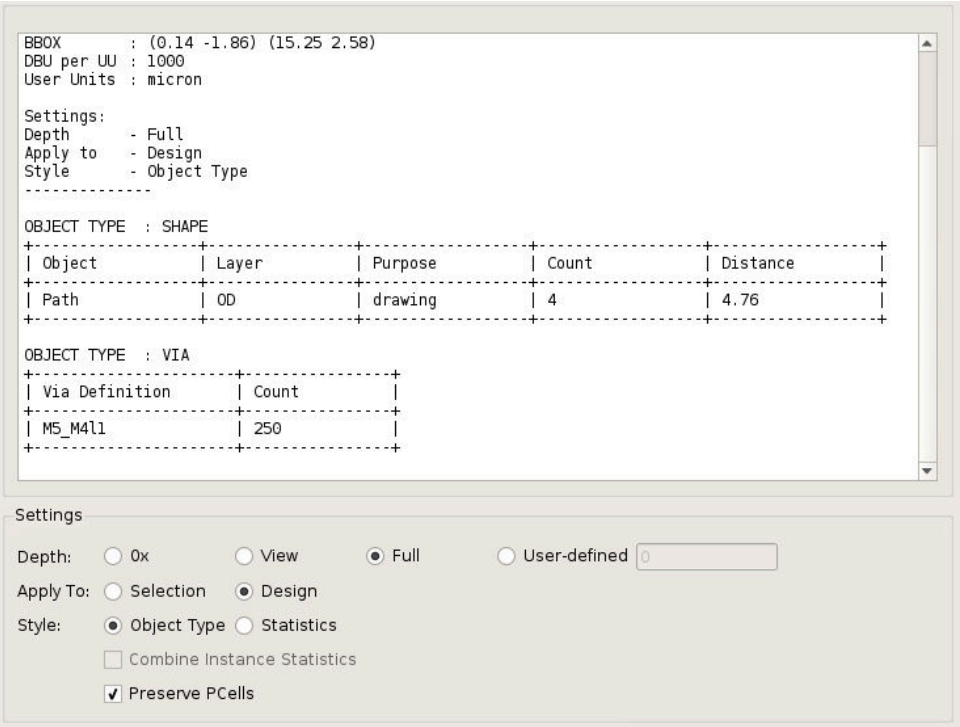
   The summary shows combined statistics of instances and array instances as follows.



Feedback

6. (Optional) Change the viewing level to **Full**, choose **Object Type** for **Style**, enable **Preserve PCells**, and then click **Apply**.

   The summary shows the results as follows.



7. Press Esc to end the command.

Generating a Design Summary Report in the Schematic Editor

This section covers the following:

- Generating a Schematic Design Summary Report
- About Cell Mapping File
- Specifying Result Type for Schematic Summary
- Consolidating Schematic Summary Results

Generating a Schematic Design Summary Report

To generate a design summary report for schematic designs:

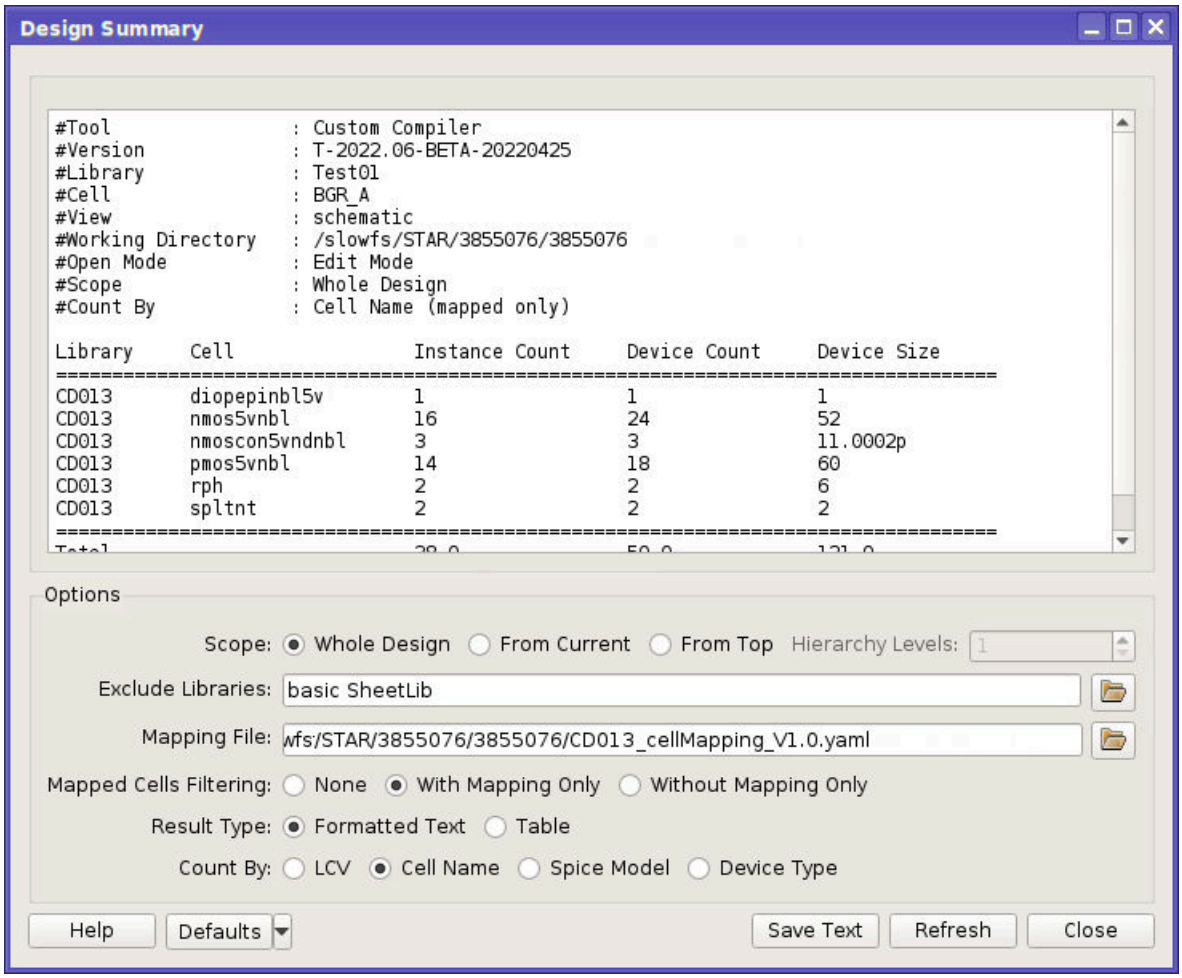
1. In the Schematic Editor, choose **Design > Summary**.

   This opens the **Design Summary** dialog box.

Feedback

For information on using the se::populateDesignSummary Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

2. To view the design summary, click **Refresh**.



From the **Design Summary** dialog box, you will get a summary about the design, including the information of instances within the specified scope, such as the library name, cell name, view type, working directory of the design, open mode of the design, design scope, hierarchical level, and other associated information.

Feedback

3. Use the following options to reset the design information you want to display:

Settings	Description
Scope	Specifies the scope for hierarchy traversing by one of the following: ◦ **Whole Design** traverses the complete hierarchy from top to bottom. ◦ **From Current** starts traversing the hierarchy from the current level of the hierarchy (parent editor window) and up to the specified number of levels. ◦ **From Top** starts traversing the hierarchy from the top of the hierarchy (parent editor window) and up to the specified number of levels. Tcl preference: `seDesignSummaryScope`
Hierarchy Levels	Specifies the depth of the hierarchy to process. Tcl preference: `seDesignSummaryCrossHierarchyLevel`
Exclude Libraries	Specifies the list of libraries the cells from which are ignored together with their lower hierarchy. Tcl preference: `seDesignSummaryExcludedLibraries`
Mapping File	Specifies the cell mapping file to refer for evaluating values of columns for an instance. See About Cell Mapping File. Tcl preference: `seDesignSummaryMappingFile`
Mapped Cells Filtering	Sets processed cells filtering based on mapping file entries. ◦ **None**: Do not filter any cells in the mapping file. ◦ **With Mapping Only**: Counts instances with mapping data. ◦ **Without Mapping Only**: Counts instances without any mapping data. This option is enabled only when you specify a mapping file and the results are not consolidated by LCV. Tcl preference: `seDesignSummaryMappedCellsFiltering`
Result Type	Sets the type of results by one of the following: ◦ **Formatted Text** (default): Shows results in plain text formatted to columns. Click the **Save Text** button to save the results in a plain text file. The default file name is *CCSESummary.log*. ◦ **Table**: Shows results in table view, which lets you select a column based on what the results can be sorted. Click the **Save CVS** button to save the results in a CVS formatted text file. The default file name is *CCSESummary.cvs*. See Specifying Result Type for Schematic Summary. Tcl preference: `seDesignSummaryResultType`

Feedback

Settings	Description
Count By	Specifies how to consolidate the results by one of the following: ◦ **LCV** (default): Counts instances by the instances' master library, cell, and view (LCV). ◦ **Cell Name**: Counts instances by the cell name of instances. ◦ **Spice Model**: Counts instances by the `model` parameter value of instances. ◦ **Device Type**: Counts instances by the device type of instances. Tcl preference: `seDesignSummaryCountBy` See Consolidating Schematic Summary Results.
Close	Click this button to close the dialog box and end the command.

About Cell Mapping File

The cell mapping file helps to evaluate values for columns of an instance. The file defines the following parameters:

- *LCV Pattern* is the master placed by the instance to match. When it matches, the instances used the specified expression. Patterns follow a top to bottom priority and uses the pattern that matches first. If none of them matches, the expression dependent column values for an instance display default empty values.

  Patterns support wildcard syntax. Each pattern can consist of 3 sub patterns separated by `/` to use for library, cell, and view names accordingly. If a `/` separator is not found, the specified pattern applies to the cell name.

  For example, a pattern `INV*` matches to all cellviews with cell name that start with `INV`, regardless of library and view names. A pattern `stdLib*/INV*/*` matches to all cellviews from libraries that start with `stdLib` and cell names that start with `INV`. Use either none or two `/` separators.

  Sub expressions can be *any string* or empty values, like `myLib/INV*/` or `/cell*/symbolN*`).

- *Device Type* is the hard-coded constant value with no evaluation. You use it as a type of device for the instance.

- *Spice Model* is a simple SEL expression that starts with `[@` and ends by `]`. It evaluates the value for space model name for an instance. *Spice Model* can also be a hard-coded value as a regular string.

- *Extracted Parameters* are optional in the mapping file. This part refers to a list of space separated parameter names to store values in the database. The

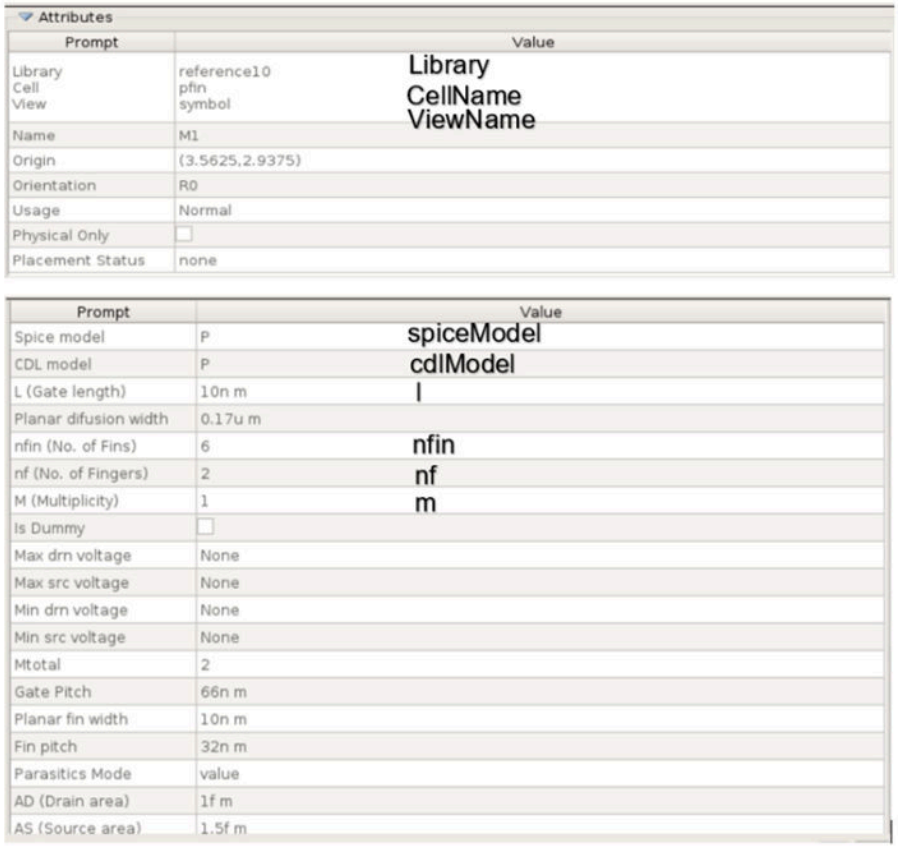
Feedback

`se::populateDesignSummary` Tcl command creates a corresponding column in the table for each unique parameter name.

- *Expression1, Expression2*, are the expressions to calculate custom value for the instance of specific master cell. The expression can be:
  - A hard-coded value (a simple string convertible to a number).
  - A SEL expression to evaluate for an instance.

  For an instance matching to a pattern in cell mapping, only the specified extracted parameter values are filed and other parameter columns are left empty. A pattern item of `CellMapping` can have many sub items, in case if a cell is mapped to other different devices with values specified for them. The *Extracted Parameters* expression of sub items can have a list of pairs with parameter name and predefined values but can be left empty as well.

Consider the example of MOS device. Set the parameters as follows:



Feedback

Check for the following details in the corresponding map file:

Version: 1.0

patterns:

- -LCVPattern: LCV Pattern
- Device Type: Any_mark_name_as_summary_category
- Spice Model: [@cdf_parameter of spice model]
- Extracted Params: cdf_parameter_list
- Expression1: Expression_created_by_parameters
- Expression2: Expression_created_by_parameters

The cell mapping file is an YAML formatted text file defining values for the pattern and corresponding expressions. For example:

```
Version: 1.0
patterns:
- LCVPattern: pfin
  Device Type: pfin
  Spice Model: "[@spiceModel]"
  Extracted Params: spiceModel cdlModel m nf nfin
  Expression1: "[@m]"
  Expression2: "[@m] * [@nf]"
- LCVPattern: nfin
  Device Type: nfin
  Spice Model: "[@spiceModel]"
  Extracted Params: spiceModel cdlModel m nf nfin
  Expression1: "[@m]"
  Expression2: "[@m] * [@nf]"
- LCVPattern: nfincaphv
  Device Type: nfincaphv
  Spice Model: "[@spiceModel]"
  Extracted Params: spiceModel cdlModel m nf nfin
  Expression1: "[@m]"
  Expression2: "[@m] * [@nf]"
- LCVPattern: rppwl
  Device Type: rppwl
  Spice Model: "[@spiceModel]"
  Extracted Params: spiceModel cdlModel m l w
  Expression1: "[@m]"
  Expression2: "[@m] * [@l] * [@w]"
```

Feedback

Use the following map file to generate a report by Device Type:

```
#Tool                : Custom Compiler
#Version             : W-2024.09-SP1-4
#Library             : demoPLL_ref10
#Cell                : mpll_top
#View                : schematic
#Working Directory   : /remote/cdesigner5/user/yuesheng/CCSE/NewCCDir/Labs/CC_FinFET_Fro
#Open Mode           : Edit Mode
#Scope               : Whole Design
#Count By            : Device Type

Device Type     Instance Count     Device Count     Device Size
=============================================================
nfin_test       6949               7.861k           26.787k
nfincap         16                 16               320
nfincaphv       68                 136              952
nfinhv          2485               2.631k           5.911k
pfin_test       5729               7.249k           13.349k
pfinhv          2313               2.651k           8.519k
rppwl           403                403              0.6545670000n
snfin           114                85               152
snfinhv         349                349              349
spfin           75                 67               85
spfinhv         273                263              503
=============================================================
Total           18.774k            21.7110k         56.92700000k
```

Use the following map file to generate a report by Spice Model Name, available in CDF data as [@spiceModel].

Feedback

```
#Tool               : Custom Compiler
#Version            : W-2024.09-SP1-4
#Library            : demoPLL_ref10
#Cell               : mpll_top
#View               : schematic
#Working Directory  : /remote/cdesigner5/user/yuesheng/CCSE/NewCCDir/Labs/CC_FinFET_Fro
#Open Mode          : Edit Mode
#Scope              : Whole Design
#Count By           : Spice Model

Library         Spice Model     Instance Count     Device Count     Device Size
=============================================================================
reference10     N               6949               7.861k           26.787k
reference10     NHV             2485               2.631k           5.911k
reference10     P               5729               7.249k           13.349k
reference10     PHV             2313               2.651k           8.519k
reference10     nfincap         16                 16               320
reference10     nfincaphv       68                 136              952
reference10     rppwl           403                403              0.6545670000n
=============================================================================
Total                           17.963k            20.9470k         55.83800000k
```

Specifying Result Type for Schematic Summary

To set the result type of a design summary report:

1. In the Schematic Editor, choose **Design > Summary**.

   This opens the **Design Summary** dialog box.

2. Select **Formatted Text** for **Result Type**.

   Select **LCV** for **Count By**. See Consolidating Schematic Summary Results.

   Click **Refresh** to get the summary report, which is shown in plain text formatted to columns.

Feedback

```
#Tool                : Custom Compiler
#Version             : T-2022.06-DEV-20220214
#Library             : A1
#Cell                : pll
#View                : schematic
#Working Directory   : /remote/work/data/designSummary_testing/spf_examples
#Open Mode           : Edit Mode
#Scope               : Whole Design
#Count By            : LCV

Library          Cell              View          Count
============================================================
reference        p_4t              symbol        160
reference        n_4t              symbol        151
basic            vss               schematic     132
basic            vdd               schematic     129
sheets           A5Sheet           symbol        98
reference        rnpoly            symbol        10
A1               tg                schematic     9
A1               inv               schematic     8
A1               inv3_vb           schematic     6
A1               invv              schematic     6
A1               tgl               schematic     6
basic            noConn            symbol        6
```

3. Select **Table** for **Result Type**. From this table, you can select a column based on what the results can be sorted.

	Library	Cell	View	Count
1	reference	p_4t	symbol	160
2	reference	n_4t	symbol	151
3	basic	vss	schematic	132
4	basic	vdd	schematic	129
5	sheets	A5Sheet	symbol	98
6	reference	rnpoly	symbol	10
7	A1	tg	schematic	9
8	A1	inv	schematic	8
9	A1	inv3_vb	schematic	6

4. When you are finished, press Esc or click **Close** to end the command.

Feedback

Consolidating Schematic Summary Results

To consolidate the results in a schematic summary report:

1. In the Schematic Editor, choose **Design > Summary**.

   This opens the **Design Summary** dialog box.

2. Select **Formatted Text** for **Result Type**. See Specifying Result Type for Schematic Summary.

   Select **LCV** for **Count By**.

   Click **Refresh** to get the summary report, showing the results in plain text formatted to columns and counting instances by their master library, cell, and view (LCV).

```
#Count By            : LCV

Library       Cell     View       Count
=========================================
analogLib     nmos     symbol     1
analogLib     pmos     symbol     1
=========================================
Total                             2
```

3. (Optional) Select **Cell Name** for **Count By**.

   Click **Refresh** to get the summary report, showing the results in plain text formatted to columns and counting instances by their cell name.

```
#Count By            : Cell Name

Library       Cell     Instance Count     Device Count     Device Size
=============================================================================
analogLib     nmos     1                  1                1
analogLib     pmos     1                  1                1
=============================================================================
Total                  2                  2                2
-----------------------------------------------------
```

4. (Optional) Select **Spice Model** for **Count By**.

   Click **Refresh** to get the summary report, showing the results in plain text formatted to columns and counting instances by their `model` parameter value.

```
#Count By            : Spice Model

Library     Spice Model     Instance Count     Device Count     Device Size
=============================================================================
```

5. (Optional) Select **Device Type** for **Count By**.

   Click **Refresh** to get the summary report, showing the results in plain text formatted to columns and counting instances by their device type.

Feedback

```
#Count By            : Device Type

Device Type     Instance Count     Device Count     Device Size
=================================================================
```

6. When you are finished, press Esc or click **Close** to end the command.

Printing and Plotting Designs

Print a design to view it on paper or as plotted output. You can choose to print or plot the design, export it as a PNG, JPEG, or BMP image (see Exporting Design Images), or create an Adobe PostScript, PDF, or PDF With Links file for later printing.

The Job Monitor tracks printing processes. See the Job Monitor Tool.

You can submit print requests, check the current print queue, and cancel print requests.

- Submitting a Print Request
- Checking the Print Queue
- Canceling a Print Request

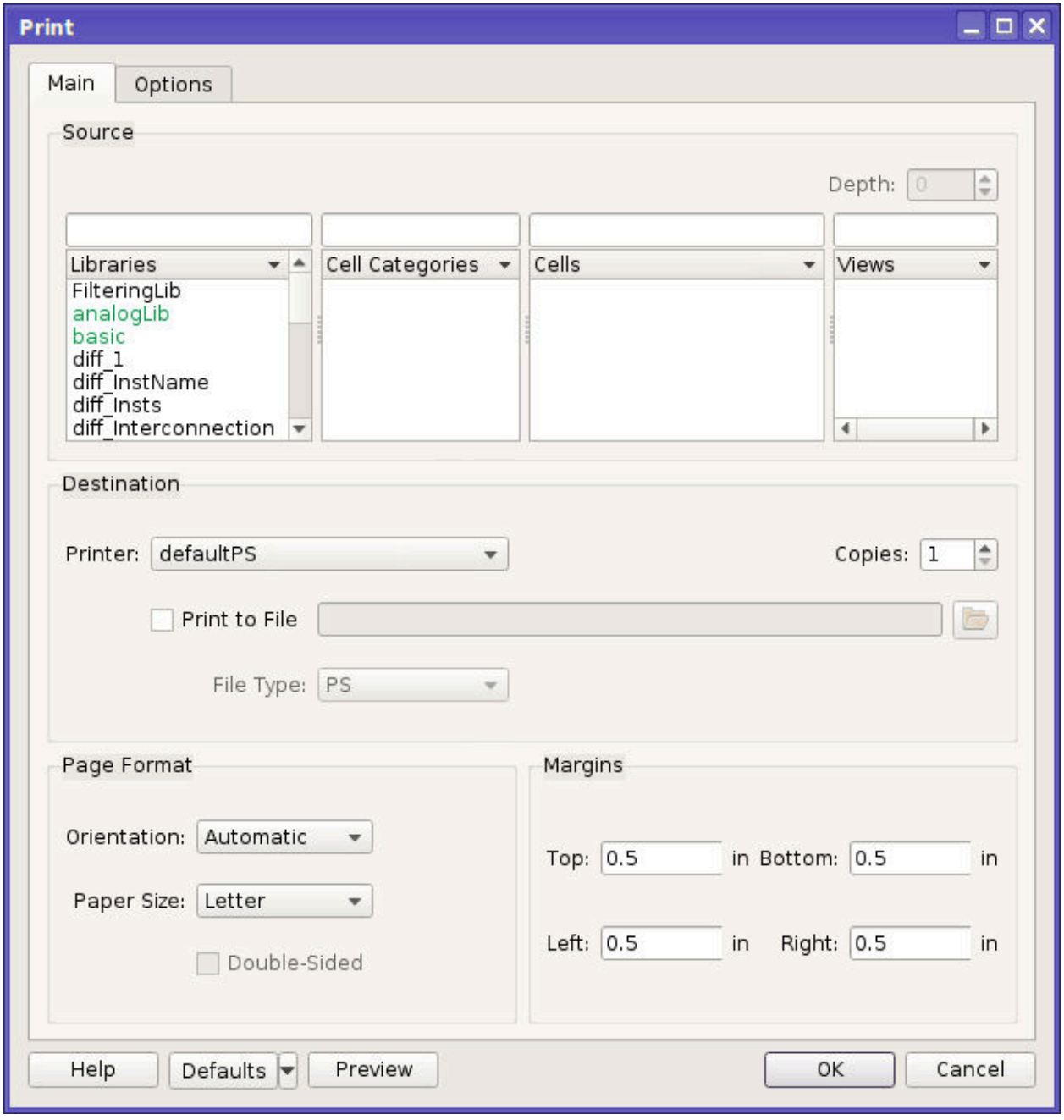
To set up a printer, use the db::createPrinter Tcl command.

Submitting a Print Request

Submit a print request to add it to the queue.

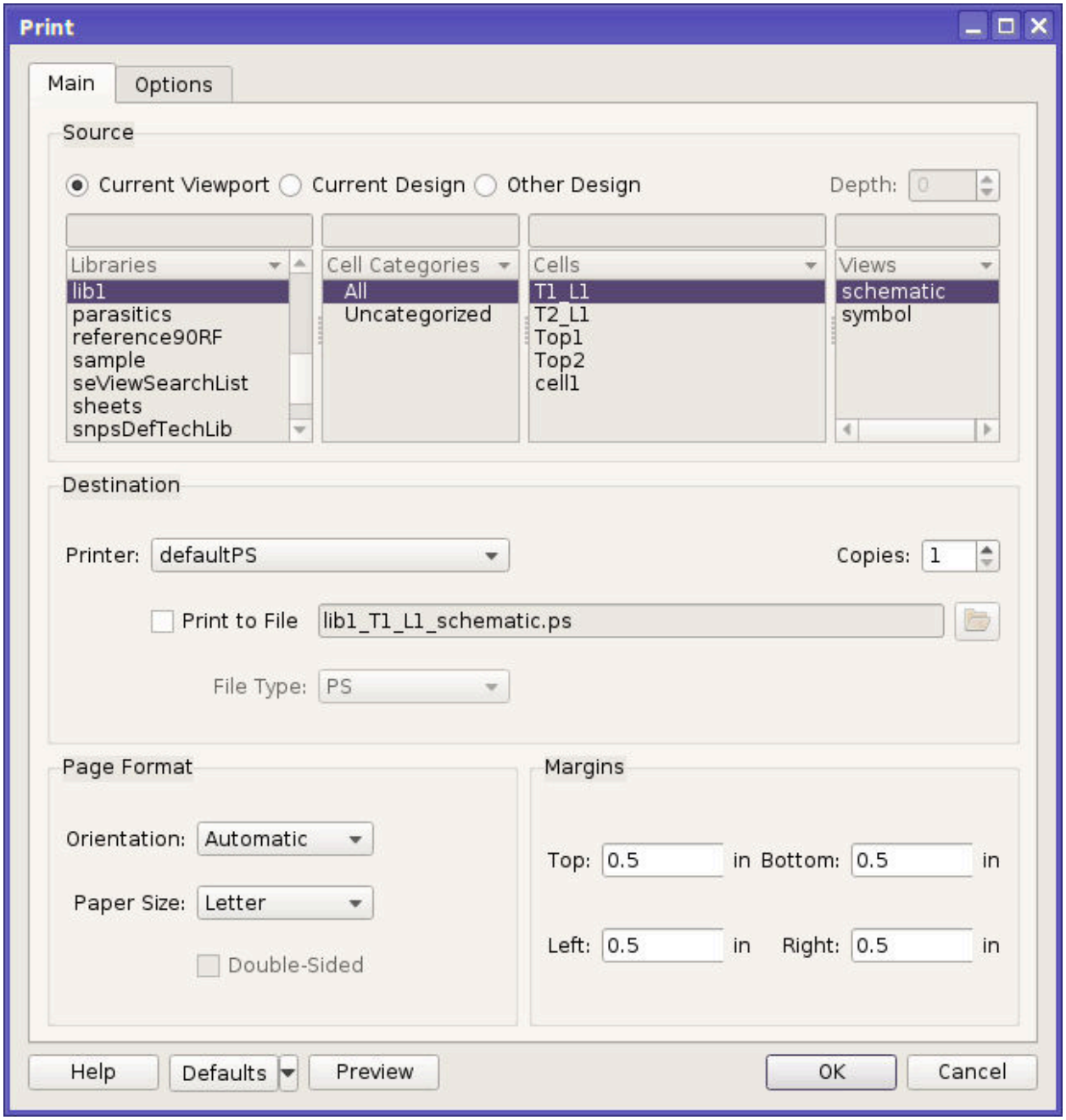
Feedback

1. From the **Console** tab or the Library Manager, choose **File > Print > Submit**.



From a design window, choose **Design > Print > Submit**. Use this option when you want to print or plot the contents of the current design viewport.

Feedback



2. Specify a source cellview.

   If the **Source** option is not already selected, you can directly choose a library, optional category, cell, and view.

Feedback

From the design view, by default, the cellview in the current viewport is selected. To choose a different cellview, deselect the **Current Viewport** option and specify the image by library, optional cell category, cell, and view.

From the **Console** tab, a current viewport cellview is not available. Specify the image by library, optional cell category, cell, and view.

3. Specify print options.

   Table 58 lists basic print options.

Table 58 Print Options

Option	Description
Source	Determines the data source for the print operation.
Current Viewport	(Design only) Prints or plots all design data within the viewport, including view depth, highlighting, flight lines, rulers, and the grid.
Current Design	(Design only) Prints and plots the entire design. When **Depth** is greater than 0, generates hierarchical prints when the hierarchical references are controlled based on a config or a viewlist. A config is used to control the hierarchical output of a print when printing a config cellview directly. A viewlist is used to control the hierarchical output when a cellview other than a config cellview is printed. The viewlist controls which view to traverse into when following the placed instance masters of the design.
Other Design	Lets you select one or more different cellviews at a time for printing.
Destination	Sets the output format.
Printer	Plots on the printer you specify from the list. The default printer is **defaultPS**.
Copies	Choose a number of copies.
Print to File	For Adobe PostScript, PDF, and PDF with Links outputs, choose and enter, or browse for a file name.
File Type	Select the **PS**, **PDF**, or **PDF with Links** file type to print to. In **Hierarchy Mode**, the printed instances should have links to the master page if **PS** or **PDF with Links** type is selected. This option is available only when **Print to File** is enabled.
Page Format	

Feedback

Option	Description
Orientation	The orientation choices include **Portrait**, **Landscape**, or an **Automatic** option that rotates those pages whose aspect ratio is greater than one by 90-degrees to more fully use the aspect ratio of the paper. The default is **Automatic**.
Paper Size	Choose the paper size that corresponds to the specified printer or plotter. The default is **Letter**.
Double-Sided	When enabled, allows printing plots or prints on both sides of each page.
Margins	Sets the border width on all sides of the printed or plotted image.

4. (Optional) For display appearance, scaled size, annotation, and hierarchy options, additionally specify advanced print options, and click **OK**.

   Table 59 lists advanced print options, which can be found in the **Options** tab under the **Print** dialog box.

Feedback

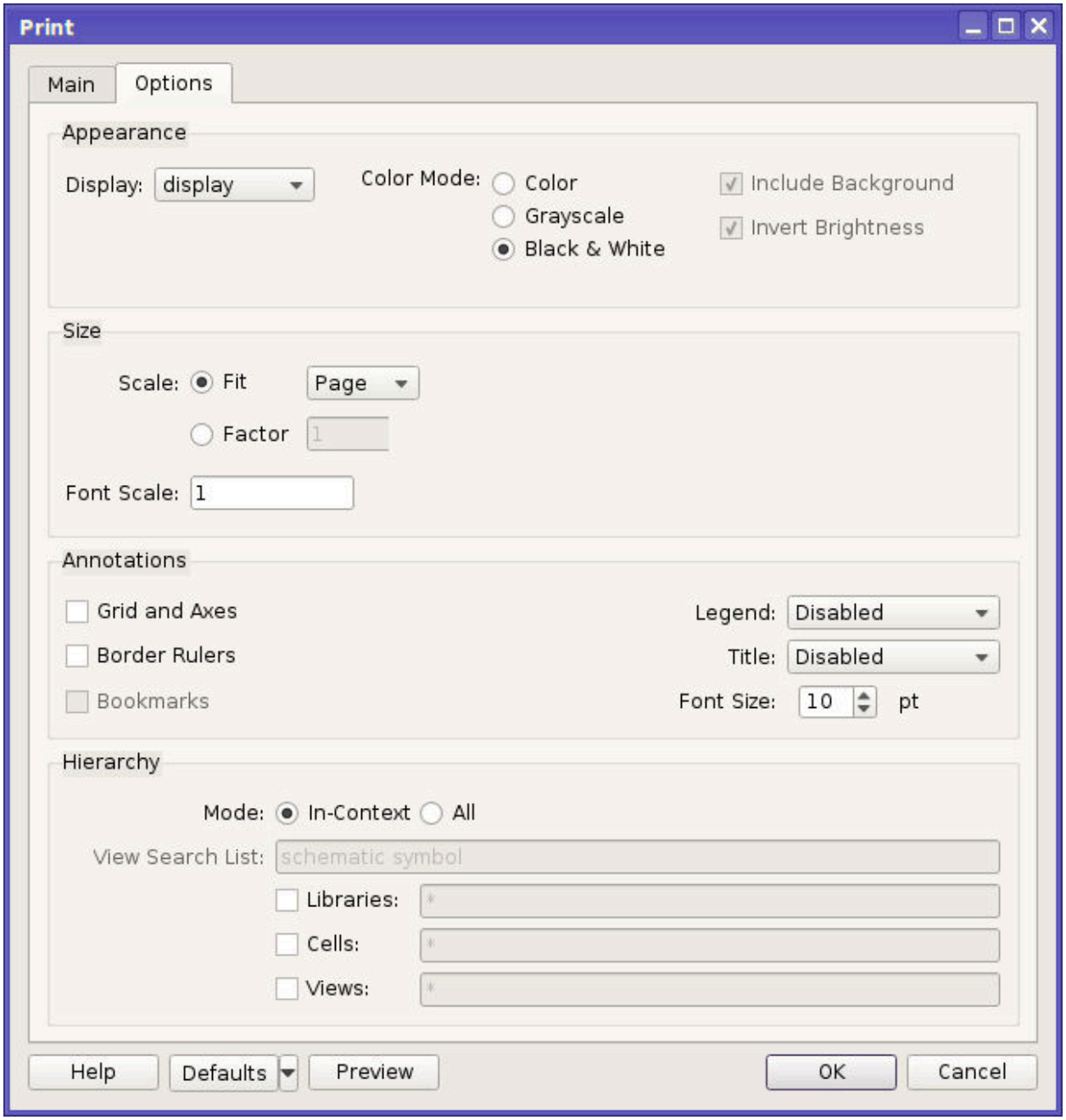


Table 59 Advanced Print Options

Option	Description
Appearance	

Feedback

Option	Description
Display	Sets the printer type from display resources.
OLP Group	If available, offers the option to set an object layer panel group.
Color Mode	Renders the output in **Color**, **Grayscale**, or **Black & White**.
Include Background	Renders the background color transparent.
Invert Background	Prints light-colored shapes darker and dark-colored shapes to lighter. When this is enabled and **Include Background** is disabled, a design can be printed to white paper, with a white background, using a display designed for a black background.
Size	
Scale	Fits the image to a page by width or height, or by a factor you set.
Font Scale	Scales the font size for all characters, including any annotation fonts.
Annotations	
Grid and Axes	Adds the background grid and orientation axes.
Border Rulers	Adds rulers around the boarder of the design.
Bookmarks	Adds bookmarks in the printed PDF output.
Legend	Adds a legend at the left, right, top, or bottom, or on a separate page.
Title	Adds a title to the bottom border with the library, cell, and view, the timestamp of the last save operation, and the user ID of the person outputting the data.
Font Size	Sets the font (default: **Arial**) size for the border, rulers, and legend, if used (see **Font Scale** to set the font size in proportion to the image size).
Hierarchy	
Mode	**In-Context** prints only the placed, or switch view. **All** prints all views of the master cellView.
View Search List	The viewlist limits the number of hierarchy levels printed when a config view is not used. **Filter** limits printed output to pattern-matched names for libraries, cells, and views you specify. Enter lists with spaces between entries.

5. (Optional) To view the specified output before printing, click **Preview**.

Feedback

6. Click **Print**.

   In the confirmation dialog box, you can make changes before submitting the job.

Checking the Print Queue

Check the print queue to view current print requests.

To check the print queue:

1. Begin by opening the queue window.

   From the Console, choose **File > Print > Queue**.

   From a design window, choose **Design > Print > Queue**.
2. Choose a printer.
3. View the queue.

You can sort print requests by rank, owner, job, file, or total size.

Canceling a Print Request

Cancel a print request to delete it from the queue.

To cancel a print request:

1. Open the queue window:

   From the Console, choose **File > Print > Queue**.

   From a design window, choose **Design > Print > Queue**.
2. Select a print request.
3. Choose **Edit > Cancel Print Job**.

Exporting Design Data

You can export the design data in GDSII and OASIS format by defining settings and options for the translation process to use.

- To export design data in GDSII Stream format, choose **Design > Export Stream**. The **Export Stream** dialog box opens.

  To set the options in the dialog box, see Exporting Design Data in GDSII Stream Format

Feedback

- To export design data in OASIS format, choose **Design > Export OASIS**. The **Export OASIS** dialog box opens.

  To set the options in the dialog box, see Exporting Design Data in OASIS Format

Exporting Design Images

Export design images to produce output files in various image formats.

To export a design image:

1. In any design window, choose **Design > Export Image**.

   Alternatively, from the **Console** tab choose **File > Export Image**.

2. Specify export options.

   Table 60 describes the Export Image options.

Table 60 Export Image Options

Option	Description
Source	**Current Viewport** prints or plots all design data within the current viewport, including highlighting, flight lines, rulers, and the grid. **Show Cell Categories** adds the category column. **Depth** specifies the deepest level to output, with 0 outputting the top level only.
Destination	**File**: Enter, or browse to find a file to overwrite. **Format**: PNG, JPEG, or BMP image.
Appearance	**Display** sets the printer type from display resources. **OLP Group**, if available, offers the option to set an object layer panel group. **Grayscale** renders the plot in grayscale instead of color or black and white. **Omit Background** renders background color transparent.

   From the design view, by default, the cellview in the current viewport is selected.

   From the Console, a current viewport cellview is not available. Specify the image by library, optional cell category, cell, and view.

3. Enter, or browse to choose a destination file name.

4. Choose an image format, PNG (lossless compression), JPEG (lossy compression), or BMP (standard bitmap).

Feedback

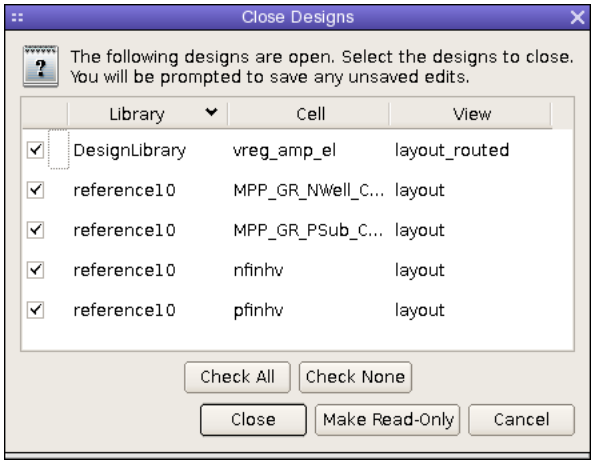
5. For layouts, optionally specify the image appearance by named display type or Object/Layer Panel group (see the Object/Layer Panel Assistant).
6. Click **Export**.

Closing Designs

Close a design to remove it from view in the current editing session, and to close all open files for the current context. To retain changes, save the design before closing it.

To close a design:

1. In the Custom Compiler **Home** tab, choose **File > Close Designs**. The **Close Design** dialog box opens and displays the list of all open designs. If a design is not the parent file within its cellview, the file name appears within parentheses next to the view name.



2. Select the designs you want to close.
3. To change the selected designs to the read-only mode, click **Make Read-Only**.
4. Click **Close**.

   If any of the designs are currently open in an editor, follow the instructions in the prompt dialog box to save the changes.

See Also

- Checking and Saving Individual Design Schematics

Feedback

Configuring Custom Cells

This topic describes the custom cell configuration syntax used in the Custom Compiler tool.

The custom cell configuration is set on `lxCustomCellsConfig`, which is a library CDF parameter. The tool reloads the `lxCustomCellsConfig` setting every time after the CDF parameter changes. If the overlay library scheme is applied, the CDF parameter reads from the overlay library.

Grouping Custom Cells by Tags

The Custom Cells Grouping feature allows you to categorize design instances into groups based on user-defined tags, such as lcv, or parameter names. This feature is useful for organizing and managing large designs by grouping similar instances together, which can simplify downstream processing, analysis, and reporting.

Benefits:

- Flexible Grouping: Group instances by any combination of lcv, or parameter values.
- Customizable Criteria: Define multiple grouping criteria to suit your design requirements.
- Improved Organization: Easily manage and analyze grouped instances for design reviews or further processing.
- Automatic Handling: Instances not matching specified criteria can be grouped as 'others' or discarded, based on your configuration.

Argument	Description
CustomCellsGrouping	Specifies grouping criteria to categorize instances into different groups by tags. It supports multiple values to define different grouping criteria. If `CustomCellsGrouping` is not specified, instances are separated into individual groups by each instance. Instances which are discarded in the previous rule are moved to the next rule.

Feedback

Argument	Description
tags	Specifies the attribute to use for grouping. You can select either lcv or a parameter name. The tool groups instances based on the order of tag definitions. • Required if `CustomCellsGrouping` is specified • Value type: list • Valid values: lcv or parameter name. • The tool does not issue warnings or errors if an instance does not have the specified parameter.
category	Defines how to group items based on tag values. Each tag value can be associated with a list of lists. Use the keyword `others` to group cells that do not match any specified tag value. If `others` is not specified, cells that do not match any tag value are excluded from all groups under this grouping criterion. • Error: If a tag is not specified in the tags list but is specified in the category, the tool ignores that tag and does not group instances by it. • Error: If the value type is not a list of lists or a list, the tool ignores the tag and does not group instances by it. • Error: If a tag name is specified without a corresponding value (for example, specifying lcv without providing values), the tool reports an error. • Warning: If `others` is specified along with other values (for example, [others, 3n]), the syntax is incorrect. The tool treats this as if only `others` is specified. • Warning: If only `others` is specified for a tag, the tool groups all instances into a single group for the tag. • Warning: If the value is empty, the tool ignores the tag and does not group instances by it.

Example 3 Grouping Custom Cells

```
CustomCellsGrouping:
    -
        tags: [lcv, Custom_Cell_Type, pnType, l]
        category:
            lcv:
                - ["*", custom_cell, "*"]
                - ["*", custom_cmos_cell, "*"]
            Custom_Cell_Type:
```

Feedback

```
                - ["*_130_*", "*_260_*"]
                - "*_280_*"
                - others
            l:
                - 3n
    -
        tags: [lcv]
        category:
            lcv:
                - ["*", cap_cell, "*"]
```

7

Assistants for Designing

This chapter describes how to use Custom Compiler design assistants to facilitate design development in the current window.

This chapter contains the following topics:

- Deciding Which Assistant to Use
- Arranging Assistant Tabs
- Synchronizing Assistants
- Clipboard Assistant
- Console Assistant
- Constraint Editor Assistant
- Design Navigator Assistant
- Device Palette Assistant
- Error Viewer Assistant
- Hierarchy Navigator Assistant
- Object Filter Assistant
- Object/Layer Panel Assistant
- Probe Assistant
- Property Editor Assistant
- Selection Table Assistant
- Track Pattern Assistant
- Track Region Assistant
- Transaction History Assistant

Feedback

Deciding Which Assistant to Use

Table 61 lists assistants in alphabetical order, and describes how each tool assists with designing. Choose an assistant when you want to use its capabilities.

Table 61 Design Assistants

Assistant	Design Development
AMS Visualization Assistant	Annotates the analog, digital, and mixed signal partitioning of a design to aid in debugging and analysis. Uses view-based or simulation-based partitioning in the Custom Compiler Schematic Editor or a text editor. For information, see AMS Visualization Assistant in the *Custom Compiler Schematic Editor and Text Editor User Guide*.
Clipboard Assistant	Shows thumbnails of available clipboards when pasting objects in the Layout Editor, Schematic Editor, and Symbol Editor windows.
Console Assistant	Enters precision coordinates and Tcl commands.
Constraint Editor Assistant	Creates and applies design constraints. (Use the Constraint Manager Tool to manage process constraints).
Design Navigator Assistant	Navigates designs by component, assigns flight line colors, applies shadow mode; operates on nets, nets groups, and synchronized clone figure groups.
Device Label Editor Assistant	Controls settings for device labeling. For information, see Device Label Editor Assistant in the *Custom Compiler Schematic Editor and Text Editor User Guide*.
Device Palette Assistant	Automates repetitive instance placement.
Error Viewer Assistant	Shows and browses the resulting violations from physical verification tools such as DRD on the canvas.
Estimated Parasitics Assistant	Adds estimated parasitic devices to a schematic design. For information, see Estimated Parasitics Assistant in the *Custom Compiler Schematic Editor and Text Editor User Guide*.
Hierarchy Navigator Assistant	Navigates designs by instance or master name.

Feedback

Table 61 Design Assistants (Continued)

Assistant	Design Development
Liberty Data Editor	Controls settings and values for a Liberty library file. See Liberty Data Editor in the *Custom Compiler Co-Design Assistant User Guide*.
Marker Browser Assistant	Lists and browses to marked design violations in designs. For information, see Marker Browser Assistant in the *Custom Compiler Schematic Editor and Text Editor User Guide*.
Net Tracer Assistant	Probes nets in designs. For information, see Net Tracer Assistant in the *Custom Compiler Schematic Editor and Text Editor User Guide*.
Object Filter Assistant	Controls settings for precision canvas selection.
Object/Layer Panel Assistant	Controls settings that display built-in and user layer purpose pair sets (LPPs) and make canvas objects visible and selectable.
Pattern Route Assistant	Lets you create routes using a chosen route pattern. See Pattern Routing.
Probe Assistant	Displays, cycles through, and deletes probes. (Use the **Add Probe** command to add probes).
Property Editor Assistant	Creates, and edits, properties, edits parameters, and edits the values of attributes; controls connectivity and connection assignments; displays instance terminal connectivity.
Creating, Editing, and Managing Design Reviews	Lets you review designs and track reviews from within the Custom Compiler tool.
Selection Table Assistant	Lists objects that are selected in the design.
Template Manager Assistant	Allows you to find matches for template cells that already have layout information associated with them (for example, a set of layout shapes/polygons), or that have constraint information associated with them. For information, see Template Manager Assistant in the *Custom Compiler Schematic Editor and Text Editor User Guide*.
Topology Assistant	Lets you set topology viewing options. See Topology Routing.

Feedback

Table 61 Design Assistants (Continued)

Assistant	Design Development
Track Region Assistant	Creates and modifies track regions for correct-by-construction designs.
Track Pattern Assistant	Defines and applies track patterns for correct-by-construction design.
Transaction History Assistant	Lists committed database transactions; reverts designs to previous states in the current session.
UDD Assistant	Creates PyCells and defines operations and parameters for UDD design. See Using the UDD Assistant.
VDRC Assistant	Lets you edit voltage annotations and set up voltage heat map views. See Annotating Voltage Data for VDRC.

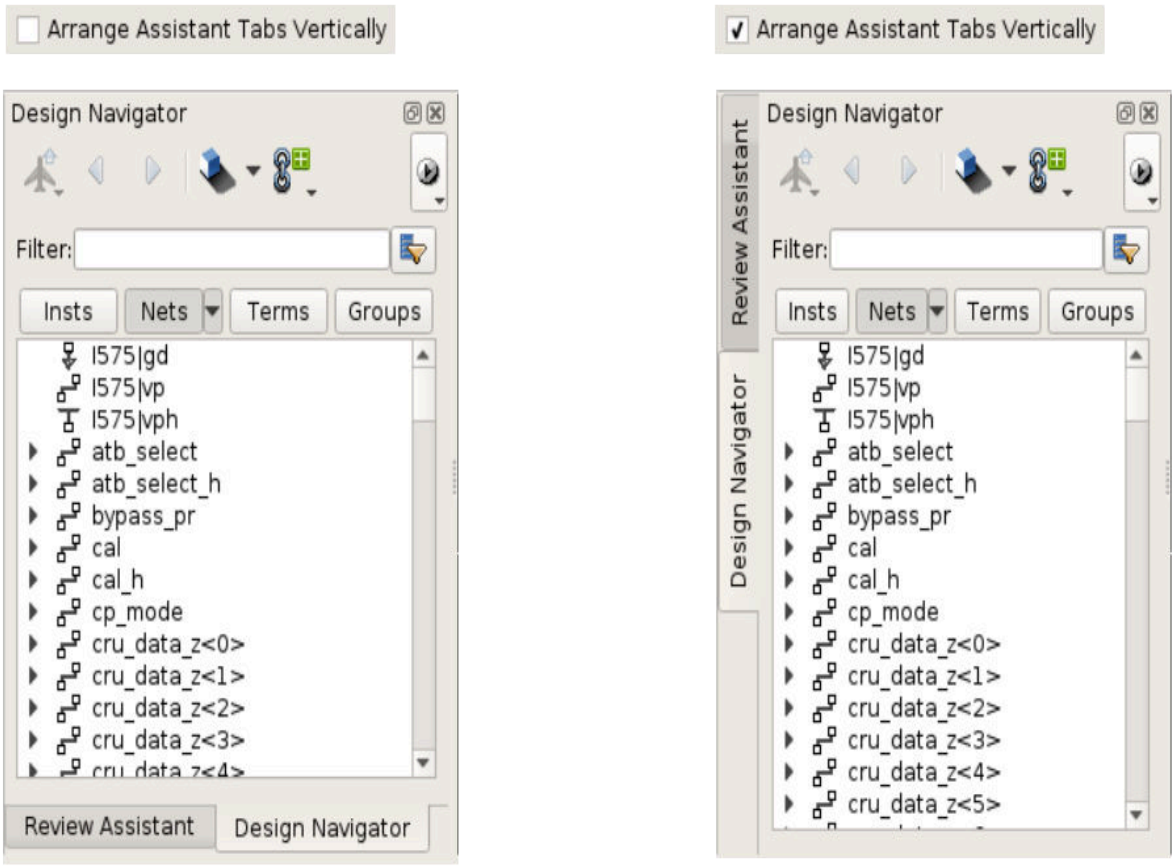
Arranging Assistant Tabs

Assistants are displayed as tabs within the design editor window. By default, they are arranged horizontally. You have the option of displaying them vertically.

To change how the assistant tabs are arranged in the window:

1. In the design editor window, choose **Options > General**.
2. In the **General Options** dialog box, select the **Interface** category.
3. In the **Visual** section, do one of the following:
   - Clear the **Arrange Assistant Tabs Vertically** check box to arrange the tabs horizontally. This is the default setting.
   - Select **Arrange Assistant Tabs Vertically** to arrange the tabs vertically.

Feedback



4. Click **OK**.

Synchronizing Assistants

This topic describes the following:

- About Assistant Synchronization
- Annotation Toolbar

About Assistant Synchronization

To facilitate design tasks, the Custom Compiler environment automatically synchronizes many of the interactions between the assistants and the canvas. For example, when you select a net in the Design Navigator, the shapes that belong to that net are automatically selected on the canvas.

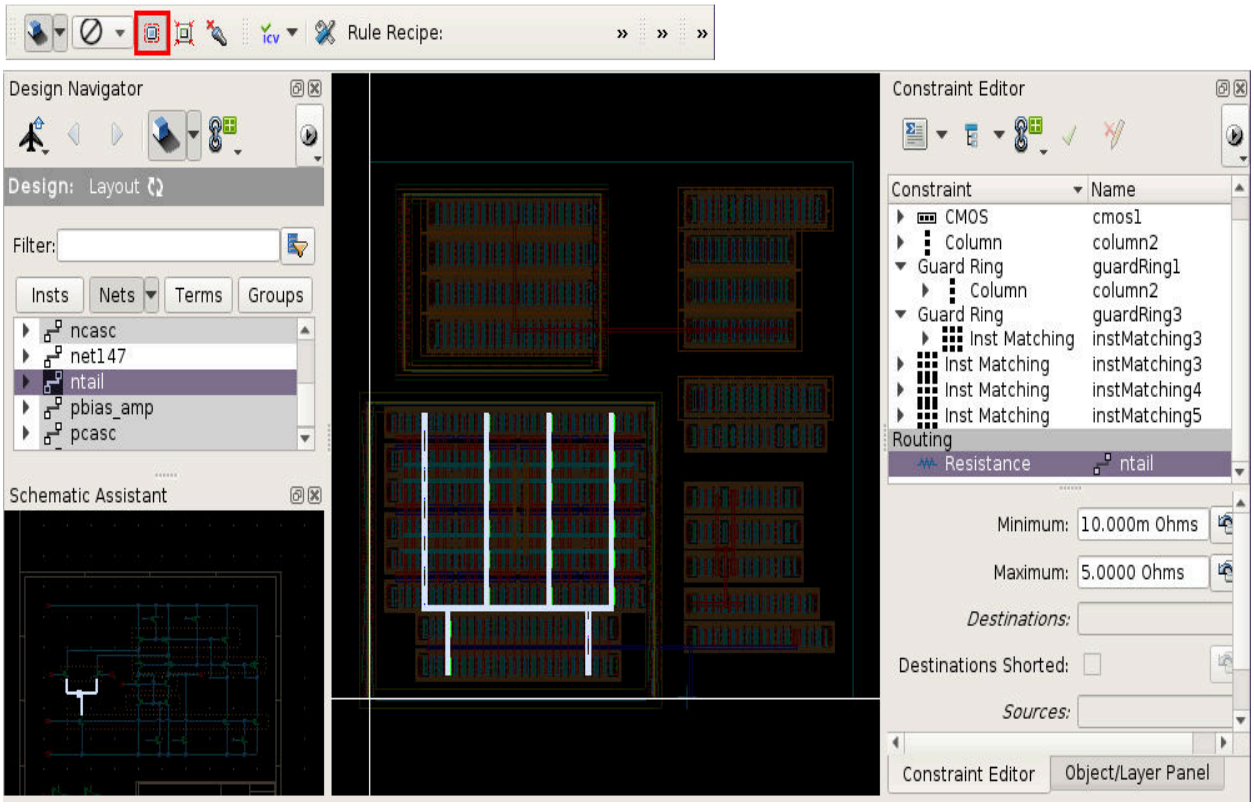
Feedback

Show me an example of assistant synchronization in SDL.

To further facilitate your design tasks, you can enable **Cross Selection** on the Annotation Toolbar so that actions in one of the following are automatically synchronized across the others:

- Constraint Editor
- Design Navigator
- Schematic Assistant (in the Layout Editor)
- Layout canvas
- Placement Gallery

As the following example shows, selecting a net in the Design Navigator selects and highlights the corresponding component in the Schematic Assistant and the shapes on the layout canvas. The design-level constraint that is associated with the net is also selected in the Constraint Editor. In the example shown, the Shadow Mode is turned on, and the selection highlight color is white.



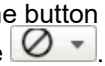
The Annotation toolbar also provides other functions to assist you with your design tasks. See Annotation Toolbar.

The Error Viewer can also be synchronized with the Design Navigator. For example, if you select a violation that is associated with a net, the net is simultaneously selected in the Design Navigator. If you select a violation that is associated with an instance, the instance is simultaneously selected in the Design Navigator. To set up the synchronization, see Error Viewer Assistant.

During any of the synchronizations, you can enable the Shadow mode to optimize the selection viewing. See Annotation Toolbar.

Annotation Toolbar

The **Annotation** toolbar consists of the following:

Button	Description
	Turns on or turns off the Shadow Mode. Turning on the Shadow Mode increases the visual contrast between the selected items and the surrounding design. Click the arrow and move the slider to increase or decrease the contrast. The Shadow Mode control is also available in the Design Navigator Assistant.
	Specifies the selection highlight color. Click the button to choose a highlight color. To turn off highlighting, choose  . The selection highlight color is also available in the Design Navigator Assistant.
	Turns on or turns off Cross Selection. When Cross Selection is on, items you select in the Design Navigator, Schematic Assistant, Constraint Editor, or the layout canvas automatically selects the corresponding item in the other assistants. See About Assistant Synchronization.
	Fits the selected item within the viewport in the Layout Editor, Schematic Editor, and Placement Gallery.
	Removes the highlighting in the assistants and on the Layout Editor canvas.

Clipboard Assistant

The Clipboard assistant shows thumbnails of available clipboards when you paste objects in the Layout Editor, Schematic Editor, and Symbol Editor windows.

Feedback

	The Clipboard assistant shows thumbnails of available clipboards when you paste objects in the Layout Editor, Schematic Editor, and Symbol Editor windows. Instead of using the default Recent clipboard library, you can also paste the clipboard contents from different clipboard libraries to the current editor window. This video tutorial shows how to use Clipboard assistant.

This topic describes the following:

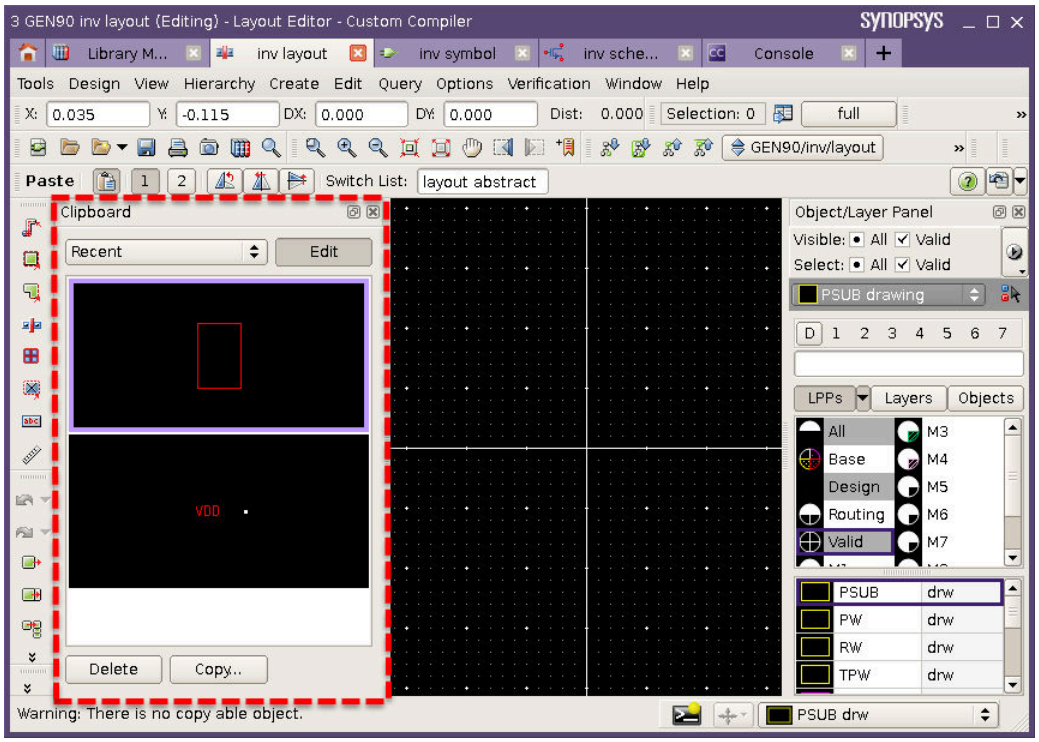
- Displaying the Clipboard Assistant
- Using the Clipboard Assistant
- Selecting the Clipboard Library
- Opening and Closing the Clipboard Library
- Copying Selected Clipboards
- Deleting Selected Clipboards
- Reordering Selected Clipboards

Feedback

Displaying the Clipboard Assistant

To display the Clipboard assistant:

1. In any layout, schematic, or symbol view, choose **Window > Assistants > Clipboard**. The Clipboard assistant pane appears on the left side of the window.



   If you have already executed Clipboard commands, the clipboard contents are listed in the assistant. Otherwise, the assistant is empty and begins recording the command operations.

2. You can undock the Clipboard assistant pane using one of these methods:
   - Click and drag it to a required location.
   - Click **Dock** (next to **Close**).

Using the Clipboard Assistant

Clicking on a thumbnail starts the **Paste** command for the current editor. If the **Paste** command is already running, it switches to the specified clipboard.

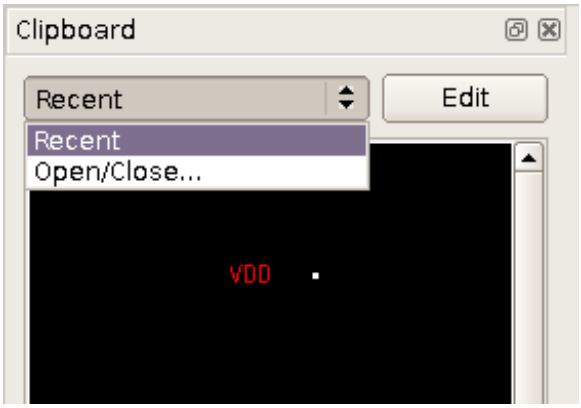
If the **Paste** command is running, the clipboard being pasted is drawn with a purple outline, as shown in the following figure.

Feedback

The thumbnail with the purple outline is always the same clipboard as that indicated by the toggle button on the **Paste** toolbar. Otherwise, no outline is drawn around any thumbnail.

Selecting the Clipboard Library

By default, the Clipboard assistant saves the last 10 clips in a Recent clipboard library, which is saved under the `<install_dir>/synopsys_custom/deClipboard/Recent` directory. Recent clips are persistent across tool sessions.



Opening and Closing the Clipboard Library

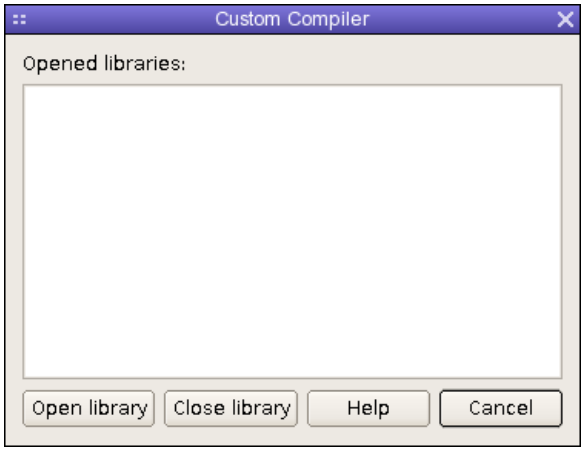
Instead of using the default Recent clipboard library, you can also paste the clipboard contents from different clipboard libraries to the current editor window.

Feedback

Changing libraries does not affect the Paste command. To paste a clipboard from a different clipboard library, simply click in the clipboard in another library and move your cursor to the canvas to place the objects to a required location.

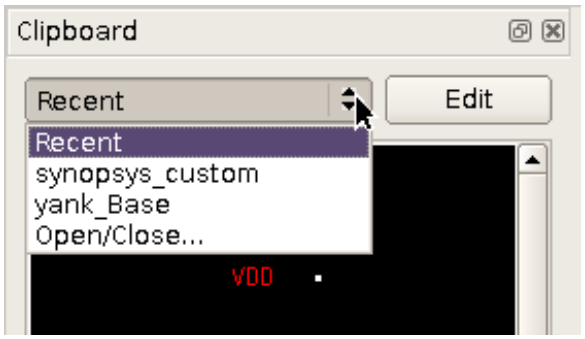
To open clipboard libraries:

1. Begin by opening the Clipboard assistant.
2. On the assistant pane, click the clipboard library list and then choose **Open/Close** to open the following dialog box.



3. Click **Open library** to choose the clipboard library you want to view.
4. Click **Cancel** to close the dialog box and return to the assistant pane.

   The selected clipboard libraries (for example: synopsys_custom and yank_Base) are now shown in the clipboard library list.



To close a clipboard library:

1. Begin by opening the Clipboard assistant.
2. On the assistant pane, click the clipboard library list and then choose **Open/Close** to open the dialog box.

3. Click **Close library** to choose the clipboard library you want to close.
4. Click **Cancel** to close the dialog box and return to the assistant pane.

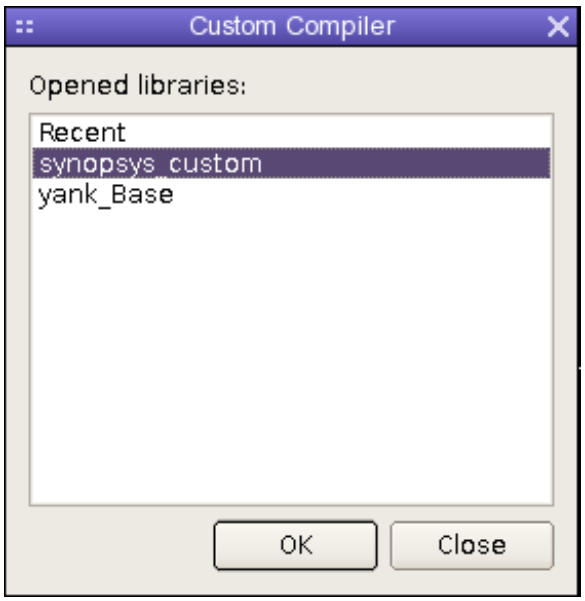
   The selected clipboard library is now removed from the clipboard library list.

Copying Selected Clipboards

To copy selected clipboards from an opened clipboard library to another:

1. Begin by opening the Clipboard assistant.
2. On the assistant pane, choose the Recent clipboard library and then click **Edit** to modify the clipboard contents.
3. Select the required clipboard and then click **Copy**.

   (Optional) To select more than one clipboard in the pane, use Ctrl-click to select each clipboard individually. You can alternatively use Shift-click or drag-left to select clipboards in the pane.
4. Choose an opened clipboard library (for example: synopsys_custom) from the following dialog box and then click **OK**.



   The selected clipboard from the Recent clipboard library is now copied to the synopsys_custom clipboard library.

Deleting Selected Clipboards

To delete selected clipboards from an opened clipboard library:

1. Begin by opening the Clipboard assistant.
2. On the assistant pane, choose the Recent clipboard library and then click **Edit** to modify the clipboard contents.
3. Select the required clipboard and then click **Delete**.

   (Optional) To select more than one clipboard in the pane, use Ctrl-click to select each clipboard individually. You can alternatively use Shift-click or drag-left to select clipboards in the pane.

Reordering Selected Clipboards

Clipboards are shown in order from the recently copied to the least recently copied. The same order is also shown on the **Paste** toolbar in the Layout Editor, Schematic Editor, and Symbol Editor windows.

To reorder selected clipboards in an opened clipboard library:

1. Begin by opening the Clipboard assistant.
2. On the assistant pane, choose the Recent clipboard library and then click **Edit** to modify the clipboard contents.
3. Select the clipboard you want to move.

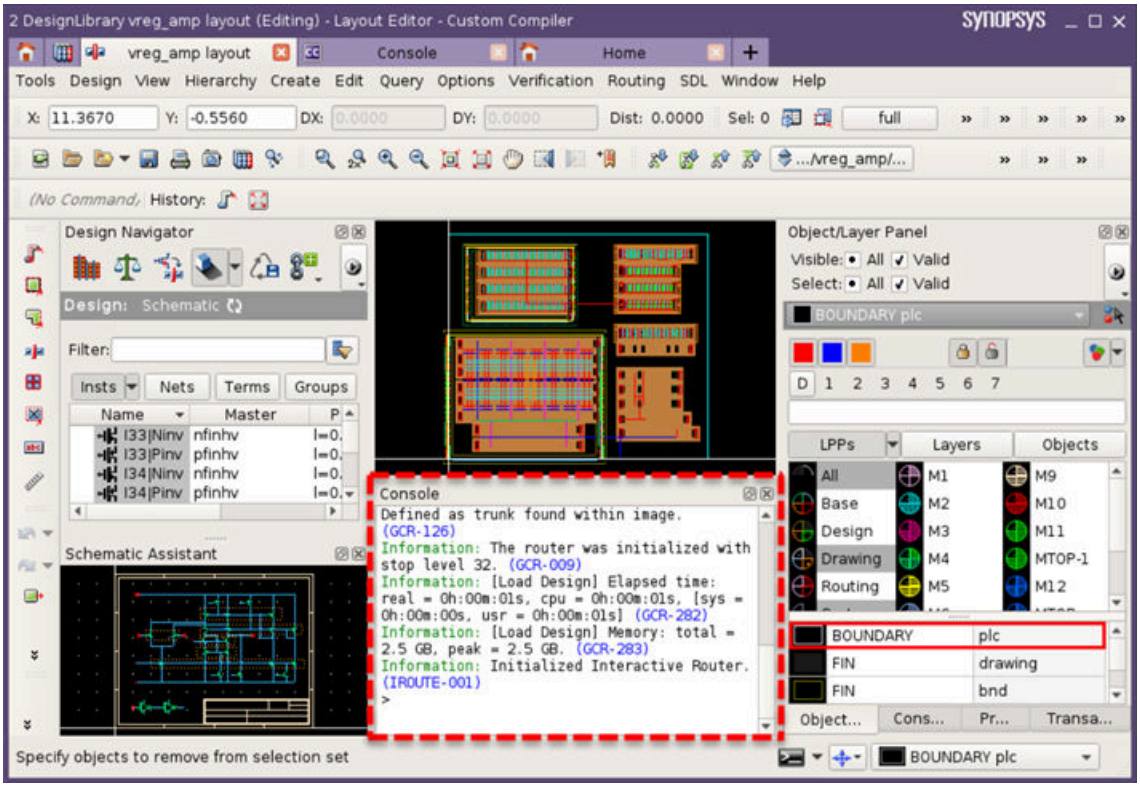
   (Optional) To select more than one clipboard in the pane, use Ctrl-click to select each clipboard individually. You can alternatively use Shift-click or drag-left to select clipboards in the pane.
4. Drag and drop the selected clipboard to the required position.

   The clipboard library and the associated Paste toolbar are updated immediately after reordering.

Console Assistant

The contents of the Console assistant is identical to the contents of the Console Tab. The Console assistant provides a convenient way to display logs, errors, and Tcl information from within a tab, such as the layout design tab.

Feedback



This section describes the following:

- Displaying the Console Assistant
- Viewing New Messages Without Displaying the Console Assistant
- Displaying Console Message Tooltips Automatically
- Viewing the Session Log Information
- Viewing the Command History
- Viewing Only Error and Warning Messages
- Typing Tcl Commands and Canvas Coordinates
- Typing Python Commands

See Also

- Undocking and Docking Assistants
- Console Tab

Feedback

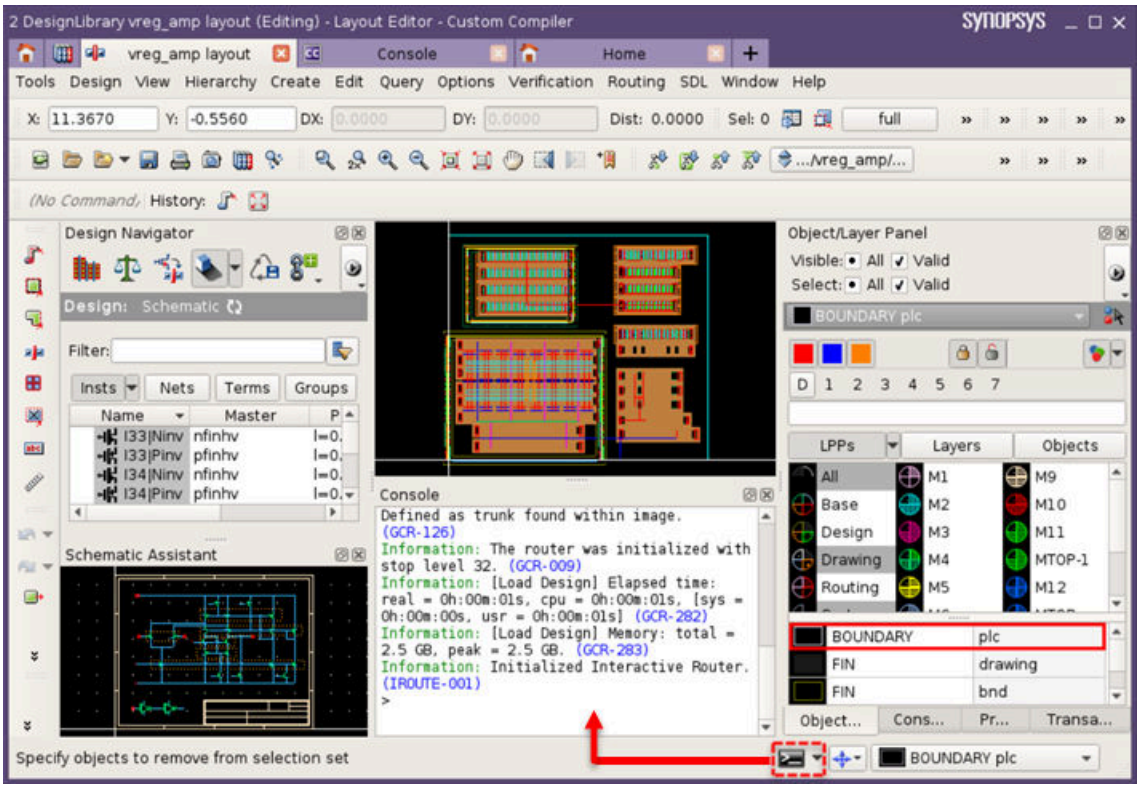
Displaying the Console Assistant

By default, the Console assistant is hidden in the tab so that you have the largest possible workspace for other assistants. However, you can display the Console assistant at any time when troubleshooting or when entering Tcl or Python commands.

To display the assistant, use one of the following methods in the current tab:

- Choose **Window > Assistants > Console**.
- Click .
- Press Ctrl+Space.

The assistant appears at the bottom of the current tab.



See Also

- Undocking and Docking Assistants
- Console Tab

Feedback

Viewing New Messages Without Displaying the Console Assistant

When the assistant is hidden, indicators notify you of new unviewed messages that are printed in the Console. You have the option of viewing the new messages without opening the assistant.

Indicators on the button are:

Indicator	Description
	New unviewed error messages. The number on the red dot indicates the number of unviewed error messages.
	New unviewed warning messages.

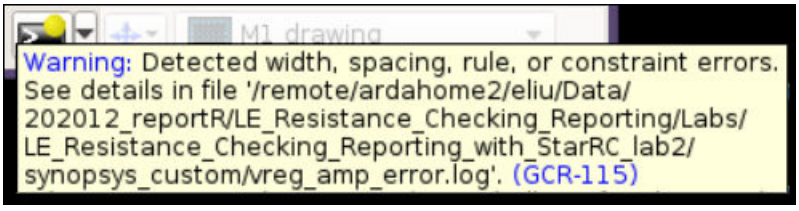
The indicators are removed after you open the assistant.

To read the unviewed messages without opening the assistant:

► Use one of the following methods:

- Automatic: Turn on the Console **Tooltips** option so that the unviewed messages appear in a tooltip automatically as they are printed to the Console assistant. See Displaying Console Message Tooltips Automatically.
- Manual: Rest the pointer on or .

  A tooltip appears briefly and displays the unviewed information, warning, and error messages.



Displaying Console Message Tooltips Automatically

When new messages are printed to the hidden Console assistant, you have the option of displaying the newly printed messages (in tooltips) without any action on your part. The automatic display lets you view the messages without having to pause your work.

By default, new messages printed to the hidden Console assistant appear automatically in tooltips. This topic describes how to use the graphical user interface setting to turn on

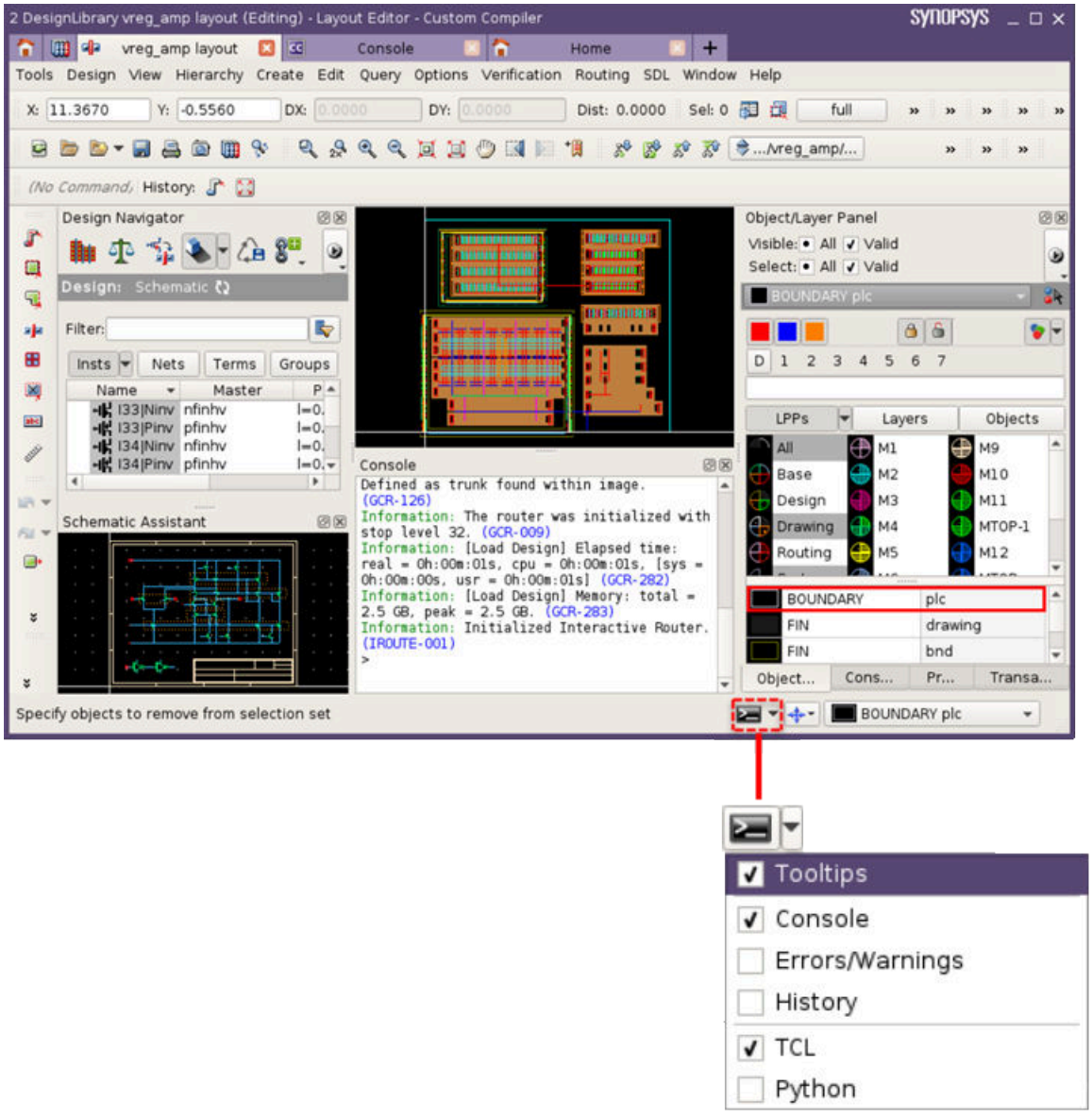
Feedback

and off the automatic tooltip display. To change the default so that the setting is applied at startup, set the `giShowConsolePopups` Tcl preference in the `.synopsys_custom.tcl` file. See Defining Startup Settings.

To automatically display or hide newly printed messages in a tooltip:

► In the current tab, click and:

- Select **Tooltips** to display the message tooltips automatically.
- Clear the **Tooltips** check box to hide the message tooltips.



Feedback

Viewing the Session Log Information

By default, the Console assistant displays the Custom Compiler session log. This view is equivalent to the **Output** tab in the Console tab.

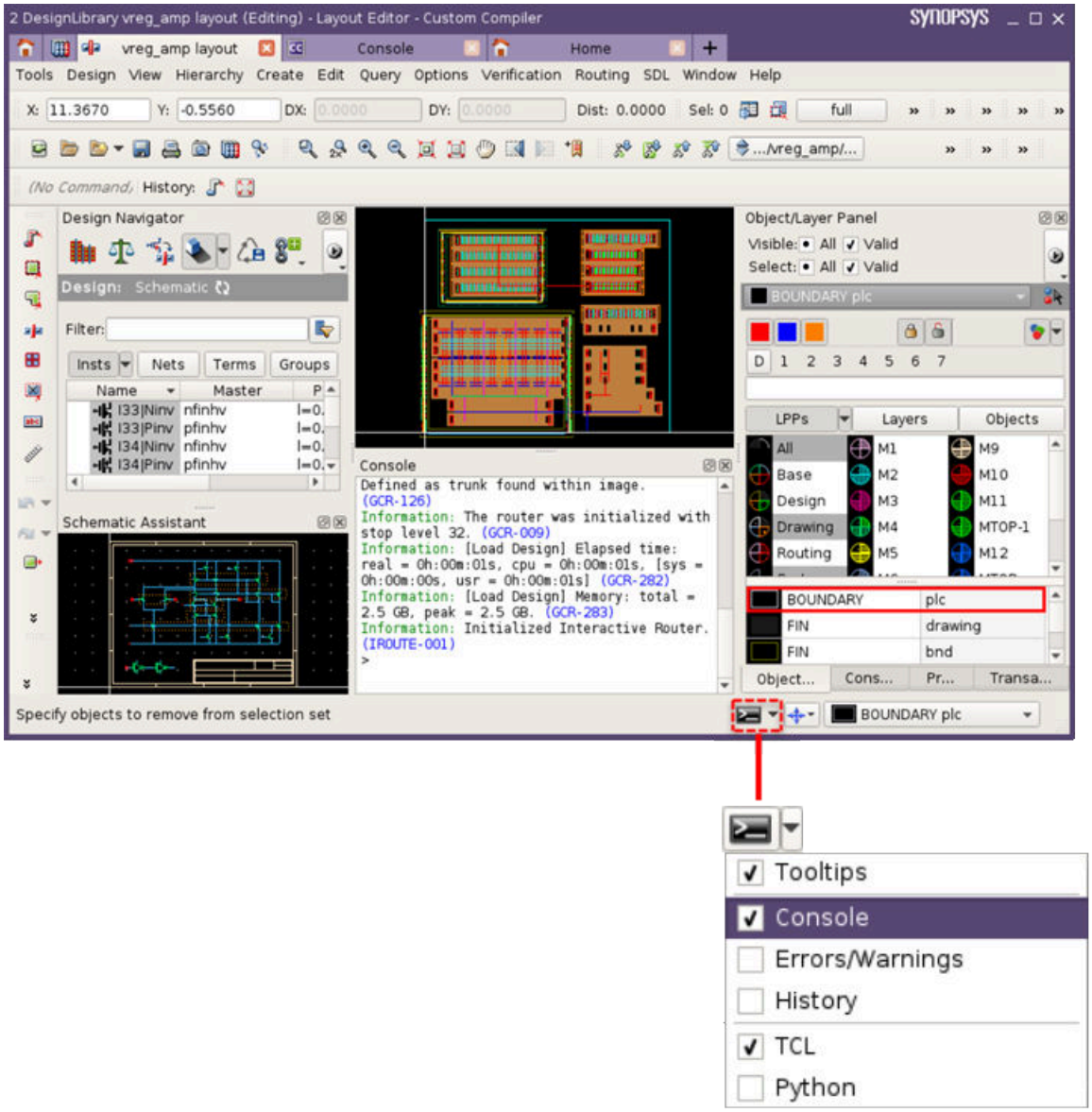
The Custom Compiler session log includes:

- Commands you typed in the command input text box.
- Output from running the commands that you typed in the command input text box.
- Error, warning, and information messages that result from running the commands.

Feedback

If the session log information is hidden, you can display it as follows:

► In the current tab, click and select **Console**.



Viewing the Command History

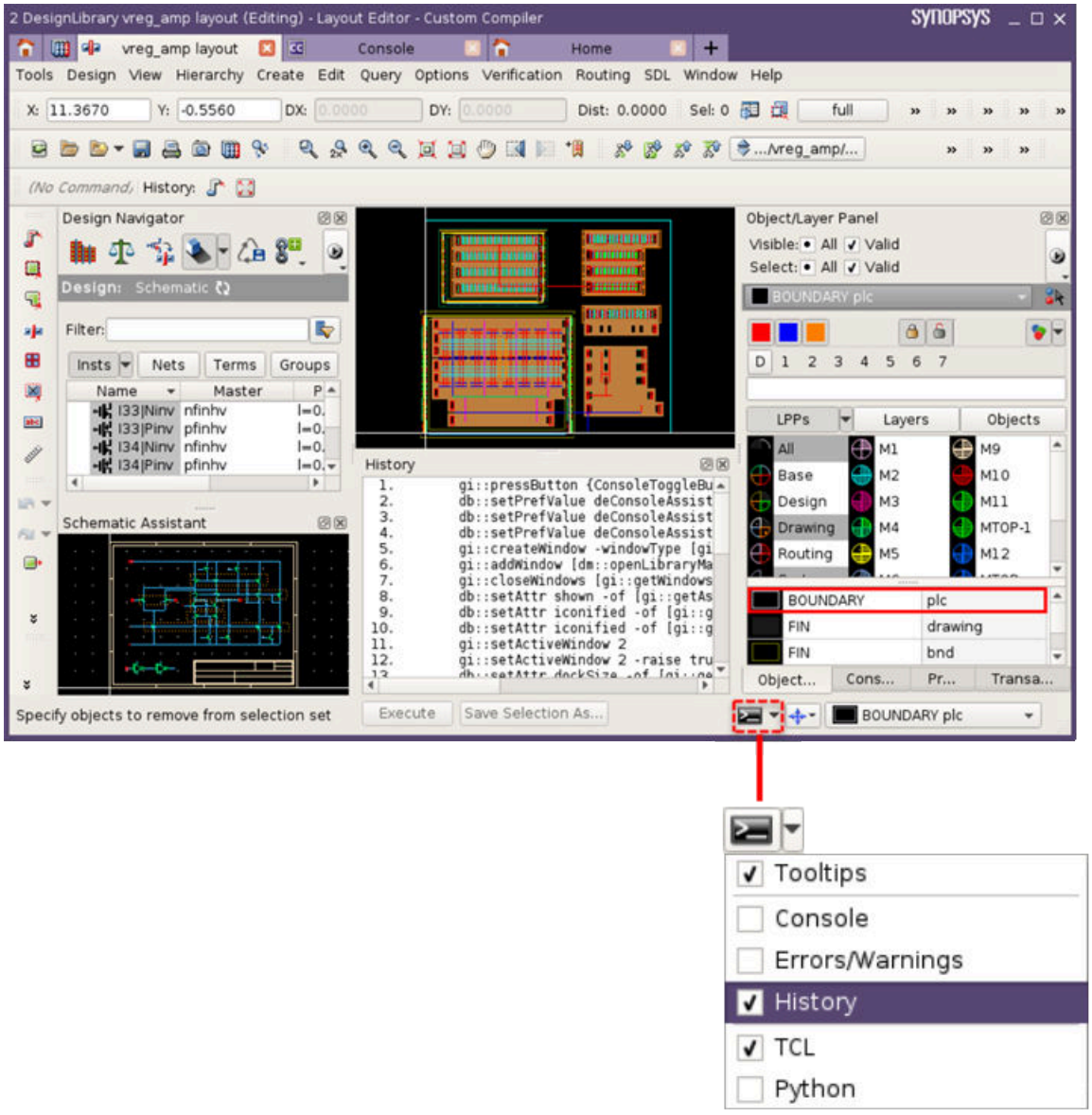
You can view the list of commands and actions that were run during the current Custom Compiler session. The list, which is the contents of the *.history* file, can facilitate

Feedback

troubleshooting. This view in the Console assistant is equivalent to the **History** tab in the Console tab.

To view the list of commands and actions that were run during the current session:

► In the current tab, click and select **History**.



The Console assistant refreshes and displays the list of commands and actions from the current session. For details about the list and the operations you can perform, see Console tab.

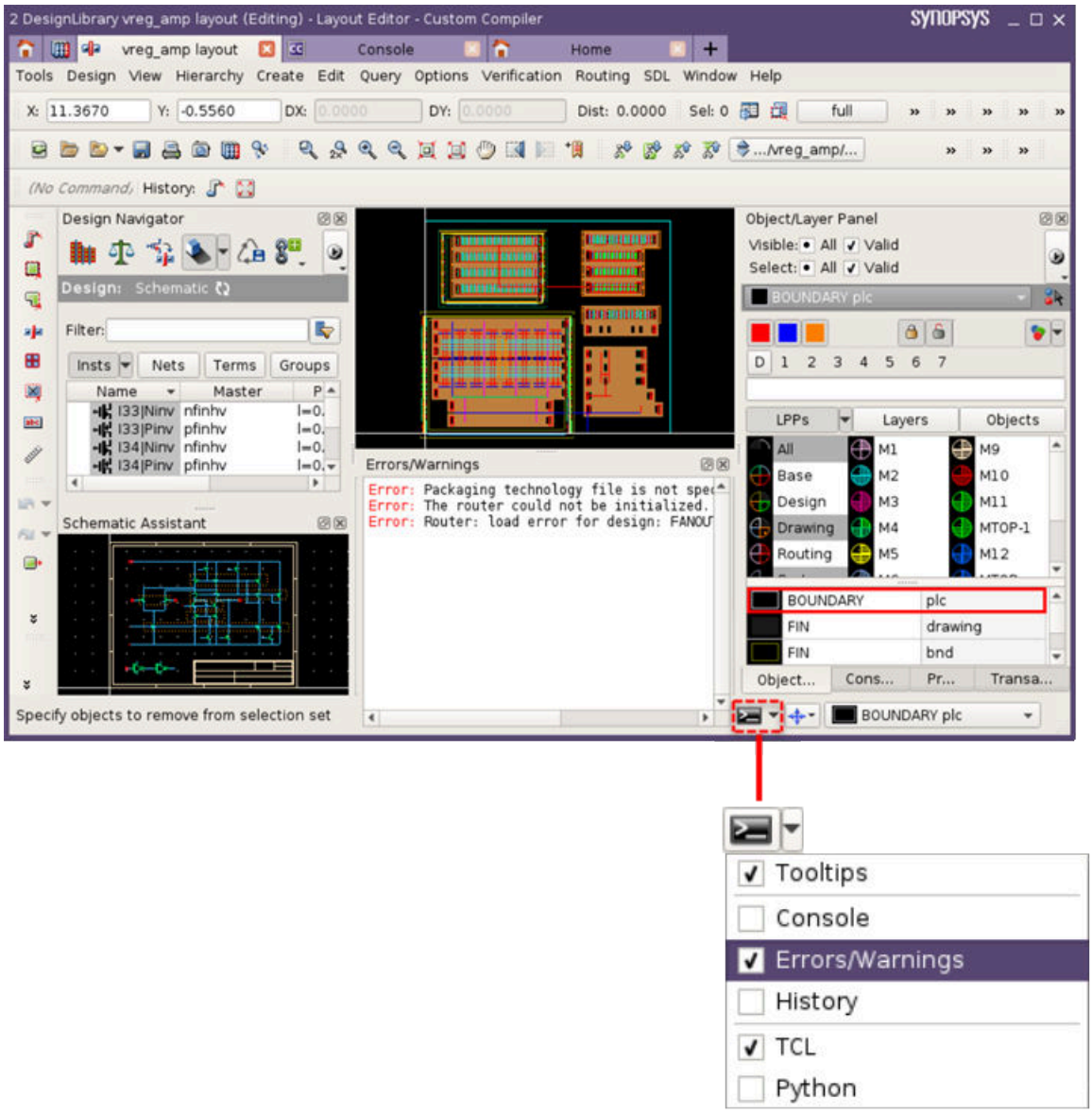
Feedback

Viewing Only Error and Warning Messages

To facilitate troubleshooting, you have the option of viewing only the error and warning messages that appear in the Console assistant. This view is equivalent to the **Errors/Warnings** tab in the Console tab.

To view only error and warning messages in the Console assistant:

► In the current tab, click and select **Errors/Warnings**.



Feedback

See Also

- Console Tab

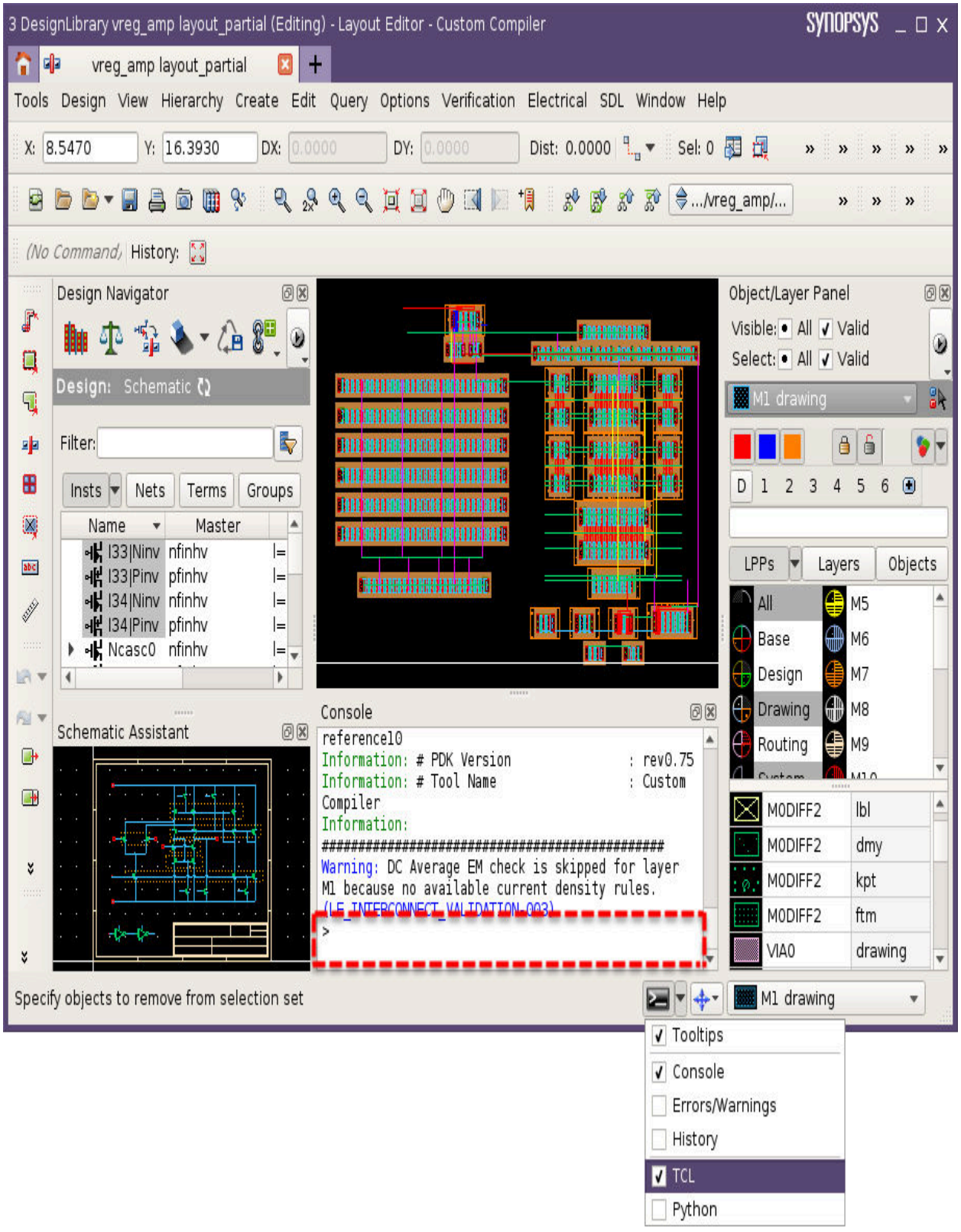
Typing Tcl Commands and Canvas Coordinates

Just as in the Console Tab, you can type Tcl commands and canvas coordinates in the Console assistant.

To enter Tcl commands and canvas coordinates in the Console assistant:

1. Display the Console assistant.
2. Click and select **TCL**.
3. Type the Tcl commands at the **>** prompt.

Feedback


3 DesignLibrary vreg_amp layout_partial (Editing) - Layout Editor - Custom Compiler
SYNOPSYS
vreg_amp layout_partial
Tools Design View Hierarchy Create Edit Query Options Verification Electrical SDL Window Help
X: 8.5470 Y: 16.3930 DX: DY: Dist: 0.0000 Sel: 0
.../vreg_amp/...
(No Command) History:
Design Navigator
Design: Schematic
Filter:
Insts Nets Terms Groups
Name Master
I33|Ninv nfinhv
I33|Pinv pfinhv
I34|Ninv nfinhv
I34|Pinv pfinhv
Ncasc0 nfinhv
Schematic Assistant
Console
reference10
Information: # PDK Version : rev0.75
Information: # Tool Name : Custom Compiler
Information:
Warning: DC Average EM check is skipped for layer M1 because no available current density rules.
(LE_INTERCONNECT_VALIDATION-003)
>
Object/Layer Panel
Visible: All Valid
Select: All Valid
M1 drawing
D 1 2 3 4 5 6
LPPs Layers Objects
All Base Design Drawing Routing
M5 M6 M7 M8 M9
MODIFF2 lbl
MODIFF2 dmy
MODIFF2 kpt
MODIFF2 ftm
VIA0 drawing
Specify objects to remove from selection set
M1 drawing
Tooltips
Console
Errors/Warnings
History
TCL
Python

Feedback

Some Tcl commands accept relative and absolute coordinates for drawing operations as arguments. Use the following guidelines when entering coordinates:

- For relative coordinates, the first coordinate defines a width and height without respect to the origin, and the second coordinate locates the shape at a point with respect to the origin. Enter relative coordinates following this convention: <float> <float>.
- For absolute coordinates, the first coordinate defines a point with respect to the origin, and the second coordinate determines the width and height with respect to the first point. Enter absolute coordinates following this convention: =<float> <float>.

For example, you can create a rectangle where the first point is absolute (such as, =4 5), and the second point is relative (such as, 2 3), and the system commits an oaRect with a bounding box of {4 5} {6 8}.

See Also

- Console Tab
- Typing Python Commands

Typing Python Commands

If you prefer to use Python commands in the Custom Compiler environment, you can do so in the Console assistant. To run supported Python commands, see Tcl Interface for Running Python Scripts, Blocks, and Expressions.

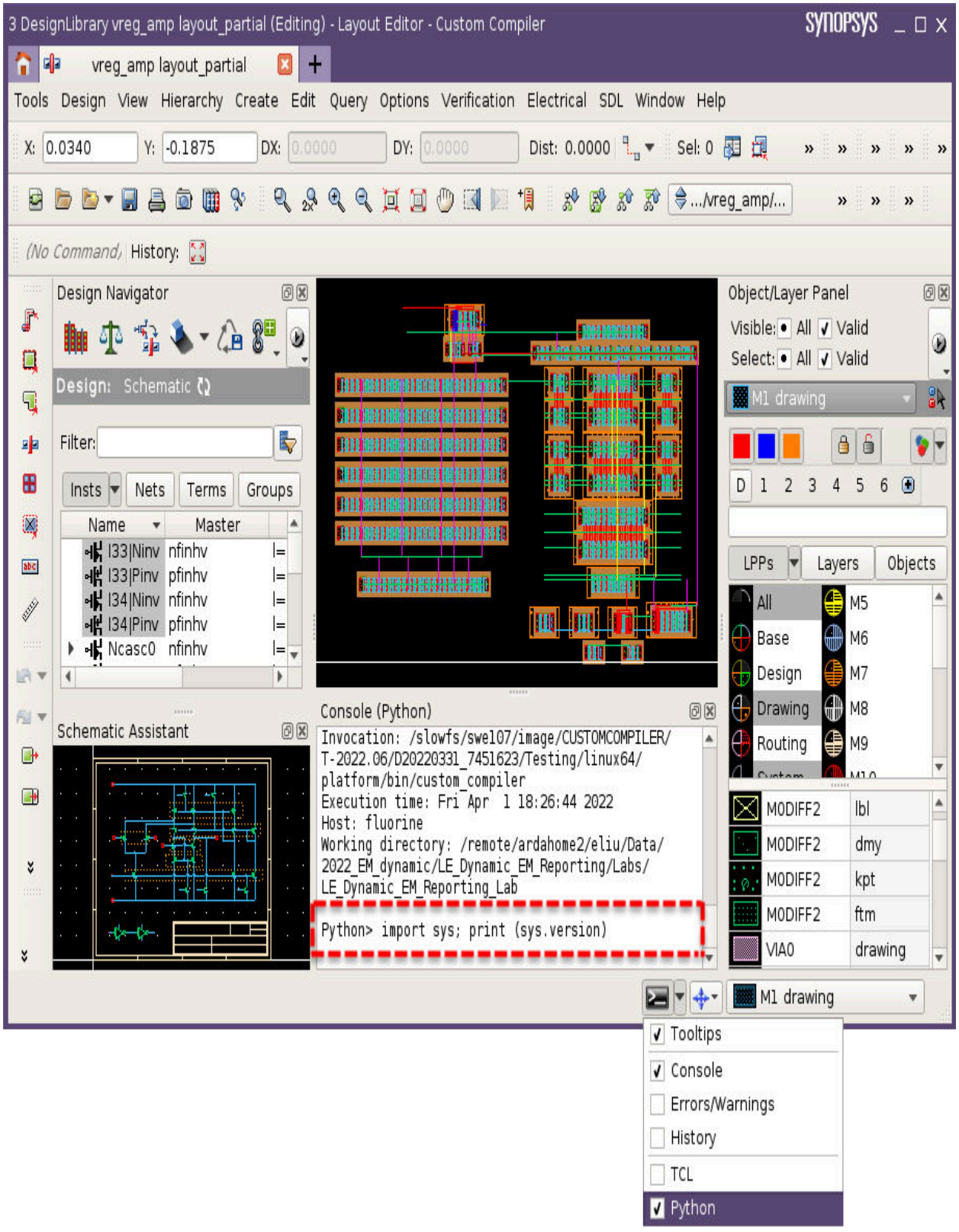
To enter Python commands in the Console assistant:

1. Display the Console assistant.
2. Click and select **Python**.

Feedback

3. Type the Python commands at the **Python >** prompt.

Feedback


3 DesignLibrary vreg_amp layout_partial (Editing) - Layout Editor - Custom Compiler
SYNOPSYS
vreg_amp layout_partial
Tools Design View Hierarchy Create Edit Query Options Verification Electrical SDL Window Help
X: 0.0340 Y: -0.1875 DX: DY: Dist: 0.0000 Sel: 0
.../vreg_amp/...
(No Command) History:
Design Navigator
Design: Schematic
Filter:
Insts Nets Terms Groups
Name Master
I33|Ninv nfinhv
I33|Pinv pfinhv
I34|Ninv nfinhv
I34|Pinv pfinhv
Ncasc0 nfinhv
Schematic Assistant
Console (Python)
Invocation: /slowfs/swe107/image/CUSTOMCOMPILER/T-2022.06/D20220331_7451623/Testing/linux64/platform/bin/custom_compiler
Execution time: Fri Apr 1 18:26:44 2022
Host: fluorine
Working directory: /remote/ardahome2/eliu/Data/2022_EM_dynamic/LE_Dynamic_EM_Reporting/Labs/LE_Dynamic_EM_Reporting_Lab
Python> import sys; print (sys.version)
Object/Layer Panel
Visible: All Valid
Select: All Valid
M1 drawing
D 1 2 3 4 5 6
LPPs Layers Objects
All Base Design Drawing Routing
M5 M6 M7 M8 M9
MODIFF2 lbl
MODIFF2 dmy
MODIFF2 kpt
MODIFF2 ftm
VIA0 drawing
M1 drawing
Tooltips
Console
Errors/Warnings
History
TCL
Python

Feedback

For example, you can type the following in the Console to source a Python script file (layoutExample.py):

```
Python> exec(open('layoutExample.py').read())
```

The layoutExample.py file contains the following:

```
# In Custom Compiler, the following command can be used to source a
 Python file in the Python Console
#     exec(open('layoutExample.py').read())
import oa

# Create a cell in the library
lib        = oa.oaLib.find("TESTLIB")
maskLayout = oa.oaViewType.get("maskLayout")
design     = oa.oaDesign.open("TESTLIB",
                              "cell1",
                              "layout",
                              maskLayout,
                              "w")

# Get the top block or create it
topBlock   = design.getTopBlock()
if topBlock == None:
    topBlock = oa.oaBlock.create(design)

# Get technology information
tech     = oa.oaTech.find(lib)
mgrid    = tech.getDefaultManufacturingGrid()
dbuPerUU = tech.getDBUPerUU( maskLayout )
um       = dbuPerUU

# Add prBoundary
pa = oa.oaPointArray(4)
pa.append((0   , 0))
pa.append((1*um, 0))
pa.append((1*um, 1*um))
pa.append((0   , 1*um))
prBoundary=oa.oaPRBoundary.create(topBlock,pa)

# Add an instance
blockDomainVisibility = oa.oaBlockDomainVisibility("inheritFromTopBlock")
inst = oa.oaScalarInst.create(topBlock,
                                  "reference10",
                                  "nfin",
                                  "layout",
                                  "W2",
                                  oa.oaTransform(),
                                  oa.oaParamArray(0),

 oa.oaBlockDomainVisibility("inheritFromTopBlock"),
                                  oa.oaPlacementStatus("placed"))
```

Feedback

```
# Create nets
net1 = oa.oaScalarNet.create(topBlock,"net1")
net2 = oa.oaScalarNet.create(topBlock,"net2")

# Create design terminal
termType = oa.oaTermType("input")
term     = oa.oaScalarTerm.create(net1, "net1", termType)

# Add a net to an instTerm
instTerm = oa.oaInstTerm.create(net1, inst, oa.oaName("D"))
instTerm = oa.oaInstTerm.create(net2, inst, oa.oaName("S"))

# Create a path
truncate = oa.oaEndStyle("truncate")
segStyle = oa.oaSegStyle(int(0.01*um), truncate, truncate, 0, 0)

polyLayer      = oa.oaLayer.find(tech, oa.oaString("POLY")).getNumber()
m1Layer        = oa.oaLayer.find(tech, oa.oaString("M1"  )).getNumber()
m2Layer        = oa.oaLayer.find(tech, oa.oaString("M2"  )).getNumber()
drawingPurpose = oa.oaPurpose.find(tech,
 oa.oaString("drawing")).getNumber()
pathSeg        = oa.oaPathSeg.create(topBlock,
                                     polyLayer, drawingPurpose,
                                     (int(-0.5*um), 0),
                                     (int(-0.5*um), 1*um),
                                     segStyle)

# Creata polygon
pa = oa.oaPointArray(4)
pa.append((0   , 0))
pa.append((1*um, 0))
pa.append((1*um, 1*um))
pa.append((0   , 1*um))
polygon=oa.oaPolygon.create(topBlock,m1Layer,drawingPurpose,pa)

# Creata rectangle
rect=oa.oaRect.create(topBlock,m2Layer,drawingPurpose,(int(2*um),int(2*um
),int(2.5*um),int(2.5*um)))

oa.oaDesign.save(design)
```

See Also

- Tcl Interface for Running Python Scripts, Blocks, and Expressions
- Python Interface for Running Tcl Scripts
- Typing Tcl Commands and Canvas Coordinates

Constraint Editor Assistant

Constraints are rules that designs must follow to ensure correct layout fabrication and operation. The constraints are either global or design-specific. You use the Constraint Manager to view and manage the global constraints. You use the Constraint Editor to create and manage design-specific constraints.

This topic describes the Constraint Editor and covers the following:

- Constraint Types and Sources
- About the Constraint Editor
- Displaying the Constraint Editor
- Viewing Constraints
- Importing and Exporting Design-Specific Constraints
- Constraint Notes

For the descriptions of design-specific constraints, including how to create them, see the following:

- Placement constraints: See the *Custom Compiler SDL and Visually Assisted Automation Guide* and the *Custom Compiler Schematic Editor and Text Editor User Guide*.
- Routing constraints: See the *Custom Compiler Custom Router User Guide* and the *Custom Compiler Schematic Editor and Text Editor User Guide*.

For information about the Constraint Manager, see Constraint Manager Tool.

Constraint Types and Sources

The Custom Compiler environment interacts with the following constraint types:

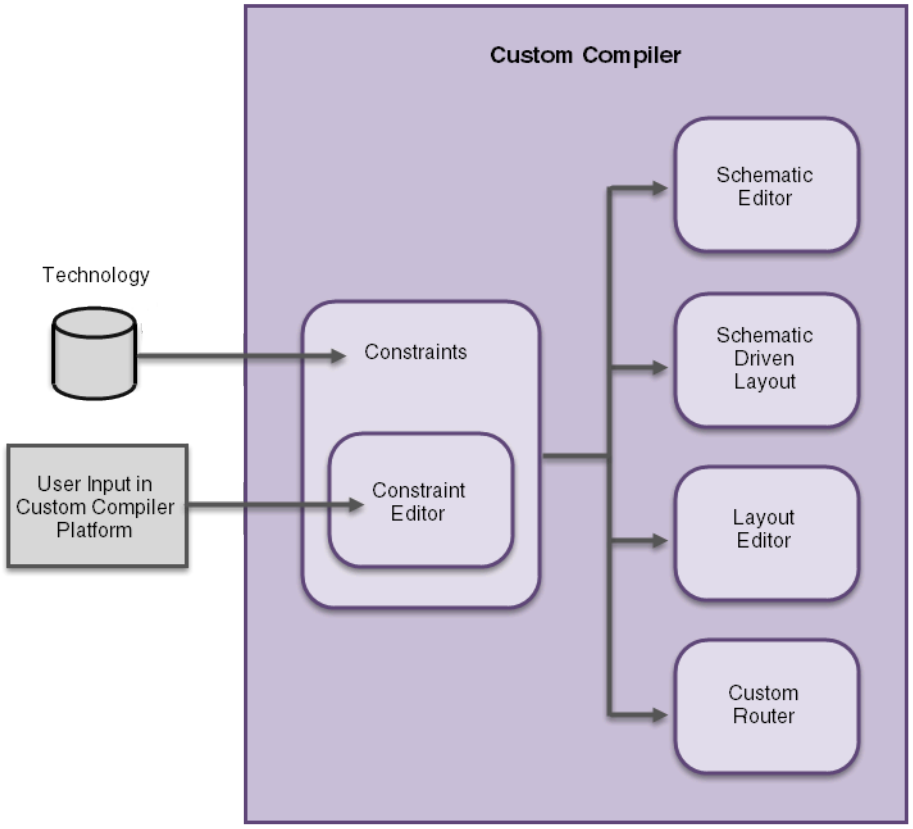
Constraint Type	Description	Source
Process	A set of global rules and constraints that all designs must follow during layout and routing, such as spacing rules and clearance rules. For information about the rules and constraints in the technology database, see the *Custom Compiler Technology and Display Resource File Reference Manual*.	Technology database

Feedback

Constraint Type	Description	Source
Design-specific	Constraints that the Layout Editor, Schematic Editor, and Custom Router must follow instead of or in addition to the process rules in the technology database. The design-specific constraints further restrict process constraints. You cannot set design-specific constraints that violate process constraints.	Constraint Editor

Figure 39 summarizes the concept of the different constraint sources and the input into the various Custom Compiler products.

Figure 39 Sources of Design Rules Used by Custom Compiler Tools



The design-specific constraints that are created and managed using the Constraint Editor are in one of the following categories:

- **Placement**: Constraints that control the placement of objects. For example, you can specify how objects are aligned. Some Placement constraints are created or updated automatically. For example, when you use the Symbolic Editor to realize instances from layout or schematic in the Matching mode, any existing Instance Matching constraint is automatically updated. To set or modify Placement constraints, see the *Custom Compiler SDL and Visually Assisted Automation Guide* and the *Custom Compiler Schematic Editor and Text Editor User Guide*.
- **Routing**: Constraints that control routing operations and results. For example, you can specify valid routing layers, the minimum width of each routing layer, preferred direction for each layer, and shielding styles. To set or modify Routing constraints, see the *Custom Compiler Custom Router User Guide* and the *Custom Compiler Schematic Editor and Text Editor User Guide*.

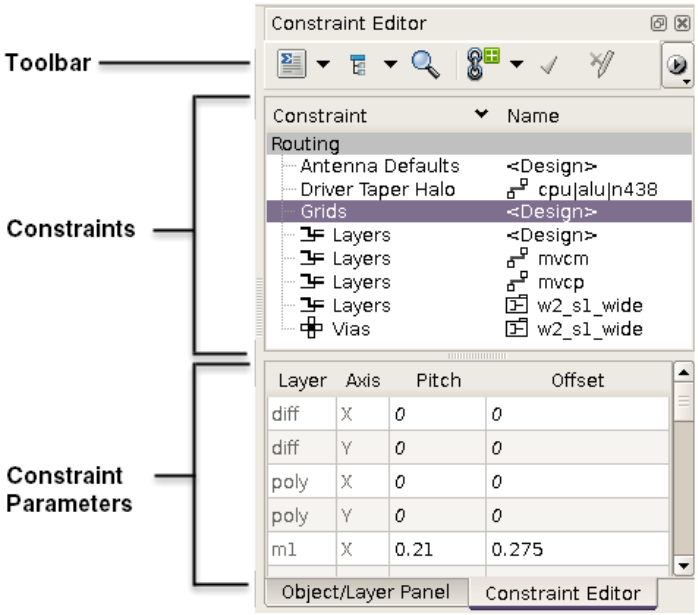
About the Constraint Editor

You can use the Constraint Editor to:

- Display the list of existing design-specific constraints.
- Edit existing design-specific constraints.
- Create new design-specific constraints.
- Import or export design-specific constraints.

Feedback

The Constraint Editor consists of the following:

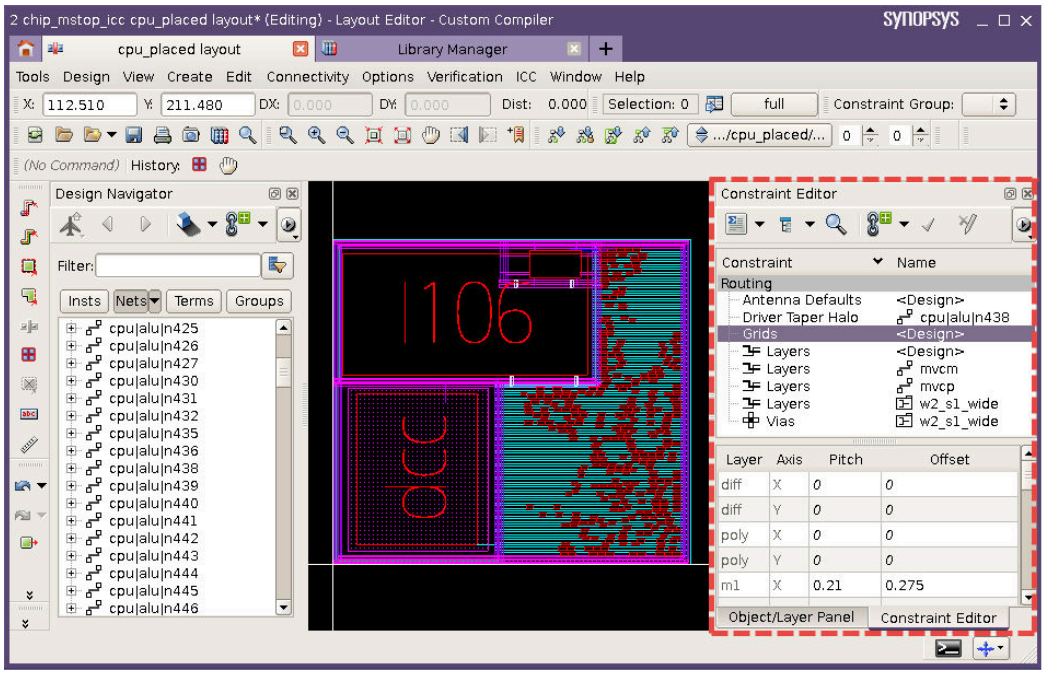


Feedback

Displaying the Constraint Editor

To display the Constraint Editor:

► In the Layout Editor or Schematic Editor window, choose **Window > Assistants > Constraint Editor**. The Constraint Editor appears on the right side of the window.



Viewing Constraints

In the Constraint Editor, you can filter and display the constraints of interest, expand all or collapse all constraints, and clear any object or net highlighting in the canvas.

The following topics describe:

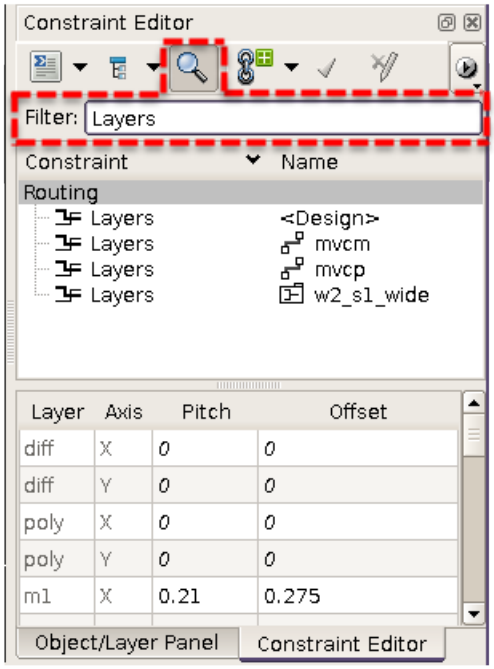
- Using Common View Features
- Customizing Views and Setting View Options
- Viewing and Correcting Invalid Constraint Settings

Feedback

Using Common View Features

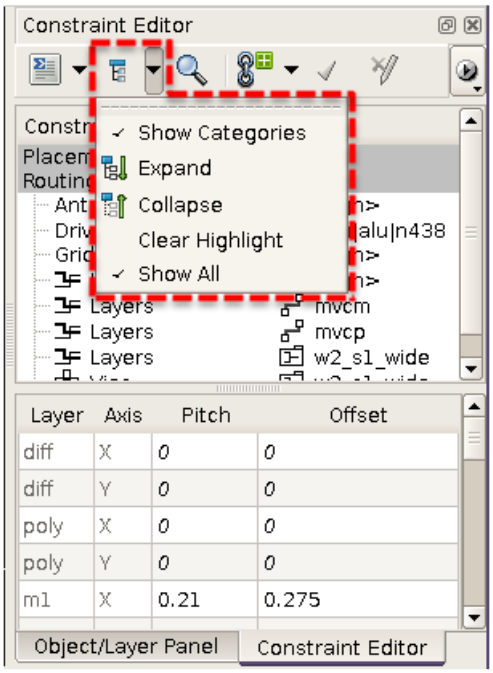
To use the common view features:

1. Make sure the Constraint Editor is displayed. In the Layout Editor or Schematic Editor window, choose **Window > Assistants > Constraint Editor**.
2. To filter the list of constraints, on the Constraint Editor toolbar, click **Show/Hide Filter Input** ( ), and then type the name of the filter expression to use in the **Filter** box.



Feedback

3. To select other view options, on the Constraint Editor toolbar, click **View** ( ), and then select one of the following commands:



Command	Description
Show Categories	Shows or hides the two design constraint categories: Placement and Routing. (On by default). Select **Show Categories** to display the category labels in the **Constraint** list. The constraints are listed under the appropriate categories alphabetically. If you unselect **Show Categories**, the category labels are removed from the **Constraint** list, and all constraints appear in a single list alphabetically.
Expand	Expands all constraints in the list so that you can see the associated nets or instances under each constraint.
Collapse	Collapses all constraints to hide the associated objects under the constraints.
Clear Highlight	Removes any highlighting on the canvas. For information about highlighting, see Customizing Views and Setting View Options.
Show All	Displays all parent and child constraints. When you clear the selection, the Constraint Editor displays only parent constraints.

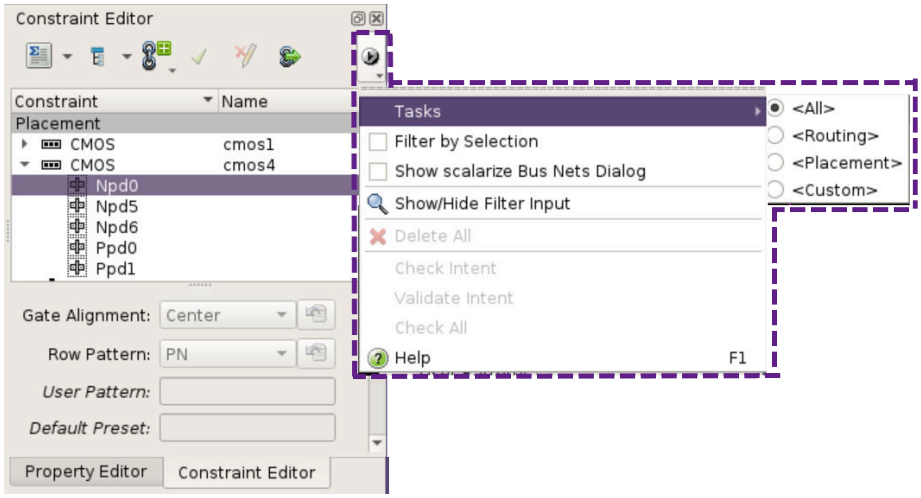
Feedback

Customizing Views and Setting View Options

You can create customized views and set other view options. For example, you can create a custom constraint category so that only the constraints that belong in the category are displayed.

To customize views and set view options:

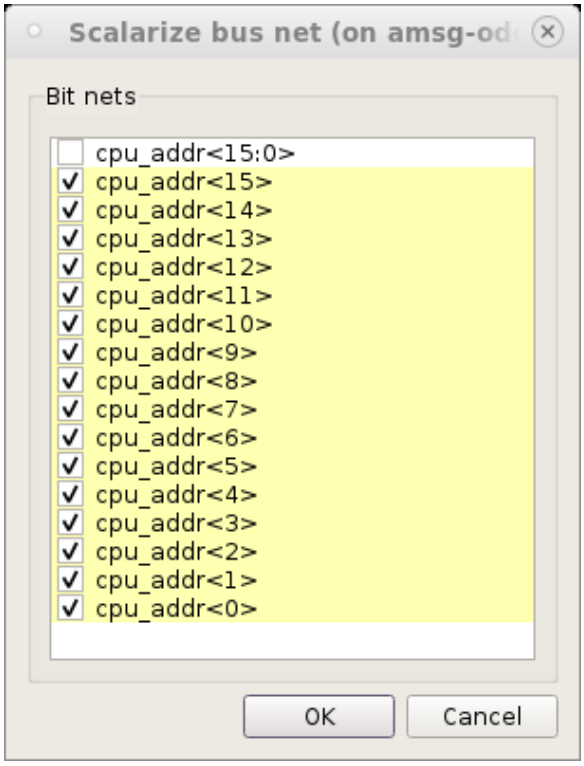
1. In the Layout or Schematic Editor window, display the Constraint Editor by choosing **Window > Assistants > Constraint Editor**.
2. On the Constraint Editor toolbar, click . The custom view menu appears.



3. Choose one of the following commands:

Command	Description
Tasks	Displays the selected category in the Constraint list: ◦ **All**: Displays all constraints in their categories. ◦ **Routing**: Displays the Routing constraints only. ◦ <custom category>: Displays the constraints that meet the criteria you set. For example, you can create a new category called Matching to display only wire matching constraints. To create a custom category, type the following Tcl commands in the Console: ◦ `cm::createTask`: Creates and registers a new category in the Tasks list. ◦ `cm::getTasks`: Returns the set of constraints. The following Tcl example creates a category called Match in the Tasks list and returns the list of constraints that have Match in their names. `cm::createTask MatchConst -title "Matching" -intents [cm::getIntentDefs -filter {%name=~/Match/}]`

Feedback

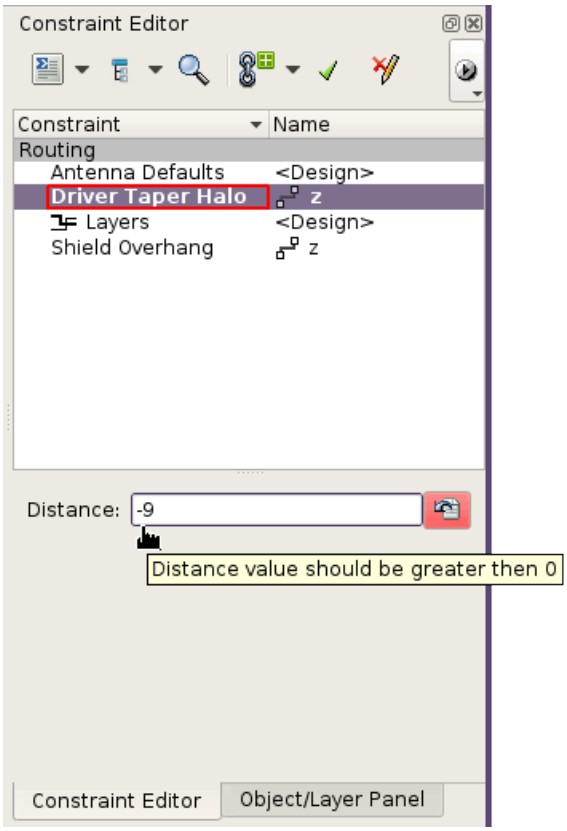
Command	Description
Filter by Selection	Enables selection filtering.
Show scalarize Bus Nets	Enable to show the **Scalarize bus nets** dialog box. Constraints can be created on selected bits using the **Scalarize bus nets** dialog box, as shown in the following figure: 
Zoom	Automatically zooms to the objects or nets of the selected constraint in the canvas.
Highlight	In the canvas, dims the objects and nets that are not associated with the constraint. Selected constraints are highlighted.
Delete All	Deletes all constraints in the Constraint list.
Help	Opens the help topic about the Constraint Editor.

Viewing and Correcting Invalid Constraint Settings

When creating or editing constraints, a parameter field becomes red if you enter an invalid value. In the example shown, the Driver Taper Halo constraint has an invalid **Distance** value.

Feedback

To determine the cause of the error, rest the pointer on the field. A tooltip appears and explains how to fix the error. In the following example, the tooltip explain that you must enter a **Distance** value that is greater than 0.



Importing and Exporting Design-Specific Constraints

You can export the design-specific constraints to create backup copies without performing a system-wide backup. You can use the backup copy to restore the constraints in case they become damaged or lost.

You use the following methods to back up and restore constraints:

- Importing Design-Specific Constraints
- Exporting Design-Specific Constraints

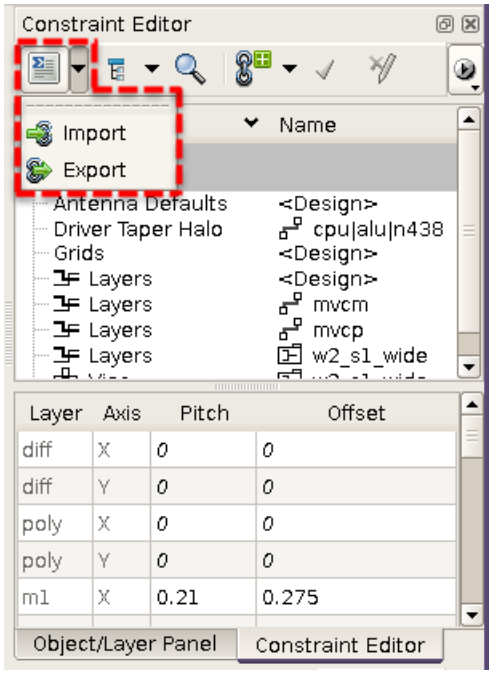
Importing Design-Specific Constraints

You can import design-specific constraints as part of the data recovery process.

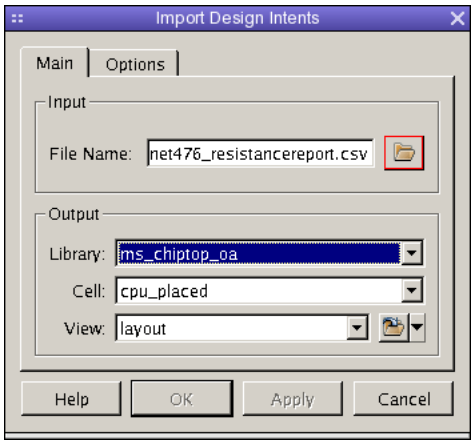
Feedback

To import constraints:

1. In the Constraint Editor, choose **File (**) **> Import**.



The **Import Design Intents** dialog box opens.

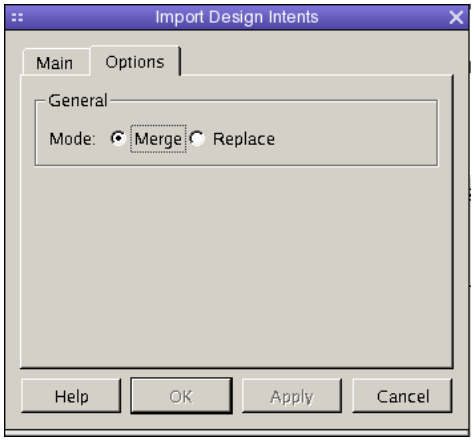


2. In the **Main** tab, supply the following information:

Selection	Description
Input: File Name	The name and directory of the constraint file. The file has the CSV file type.

Selection	Description
Output: Library, Cell, View	The library, cell, and view to be associated with the constraints.

3. In the **Options** tab, select one of the following:



Selection	Description
Merge	Imports the specified constraints and keeps existing constraints. If an existing constraint has the same name, it is replaced by the newly imported constraint.
Replace	Removes all existing constraints, and then Imports the specified constraints.

4. When you are finished, do one of the following:
   - Click **Apply** to import the constraints and remain in the **Import Design Intents** dialog box.
   - Click **OK** to import the constraints, and then return to the Constraint Editor.

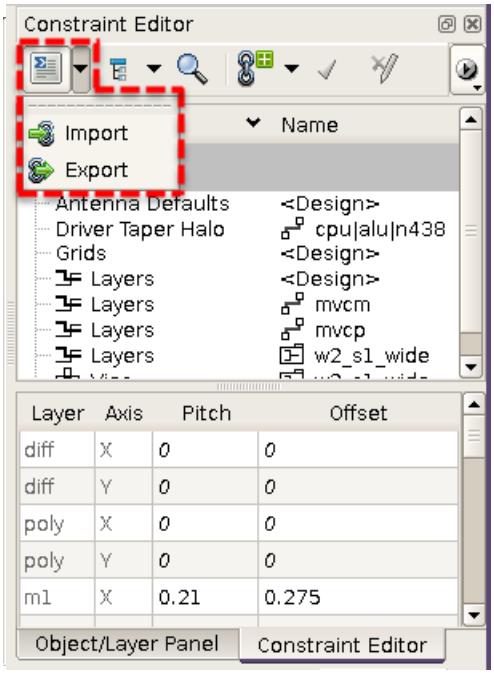
Exporting Design-Specific Constraints

You can import the design-specific constraints as part of the data recovery process.

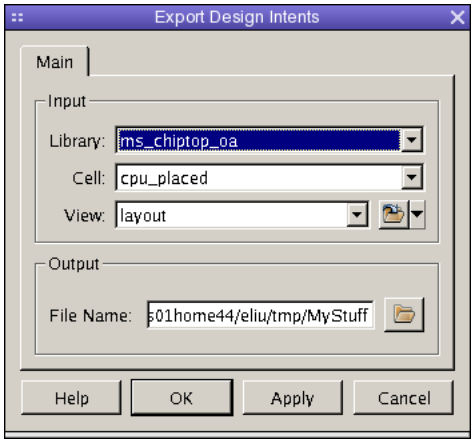
Feedback

To import the constraints:

1. In the Constraint Editor, choose **File (** **) > Export**.



The **Export Design Intents** dialog box opens.



2. Supply the following information:

Selection	Description
Input: Library, Cell, View	The library, cell, and view that are associated with the constraints.

Feedback

Selection	Description
Output: File Name	The name and directory of the exported constraint file.

3. When you are finished, do one of the following:
   - Click **Apply** to export the constraints and remain in the **Import Design Intents** dialog box.
   - Click **OK** to export the constraints, and then return to the Constraint Editor.

Constraint Notes

You can associate note objects to constraints using the **Constraint Editor Assistant**.

The association can be done either to a new intent during creation or to an existing intent. These operations are available only for Schematic view designs and when `seUseConstraintCellView` is false. If `seUseConstraintCellView` is true and the intent has an associated note object in the Schematic design, you can see the appropriate visual feedback in both Schematic and Layout views indicating the existence of the associated note object.

Creating Intent With Notes

During intent creation, if there is a valid note object in the selection set, then the selected note object is associated with the newly created `oaConstraint`. The tool examines the selection set for existence of the `itkNote` object and associates with a constraint.

Note:
Only one `itkNote` object can be associated with a constraint. However, you can also add the `itkNoteShape` object, which is optional. Thus, as a valid selection for note creation is considered to be either a single `itkNote` object or `itkNote` and `itkNoteShape` objects (in addition to intent member objects). This operation is controlled using the preference `cmConstraintEditorAttachNotesOnCreate`. If the value is false, then the operation is ignored. The intent is created in the usual way but note creation is skipped.

Adding or Removing Notes

When the `itkNote` object without any intent association is selected in the Schematic design, the `cmAddNote` action is displayed in the **Intent** context-sensitive menu of the **Constraint Editor**. If the action is completed, the note object is attached to `oaConstraint` and the transaction is registered in Transaction History.

Feedback

If the intent has an associated note object, then `seRemoveNote` is displayed in the **Intent** context-sensitive menu of the **Constraint Editor**. If the action is completed, the note object is detached from `oaConstraint` and the transaction is registered in Transaction History.

Note:
This operation affects only the note and the association of the intent objects.

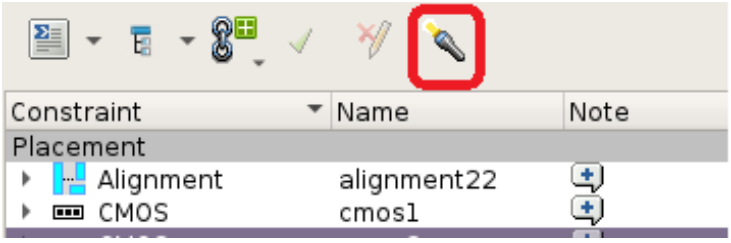
Data Model

The association between `itkNote(oaText)` (and auxiliary `itkNoteShape`) object and the related `oaConstraint` is implemented by creating an `oaGroup`. The group name is formatted using specific descriptive prefix appended with index to make it unique in edit design. When the note is detached from the intent, the corresponding `oaGroup` is destroyed. The described data model implies that a note object can be associated only with intent types that are represented with a single constraint. Evidently these intent types should be supported in the Schematic design.

GUI Changes

The associated note objects are reflected in various ways. In the **Constraint Editor**, the **Note** column is displayed to indicate that the intent has a note object. Every intent associated with note objects has a corresponding icon to indicate the same. This column exists both in Schematic and Layout designs when preference `seUseConstraintCellView` is false.

Highlight Notes is added to the **Constraint Editor** toolbar to highlight all design note objects that have association with a constraint. This button is available only in Schematic View.



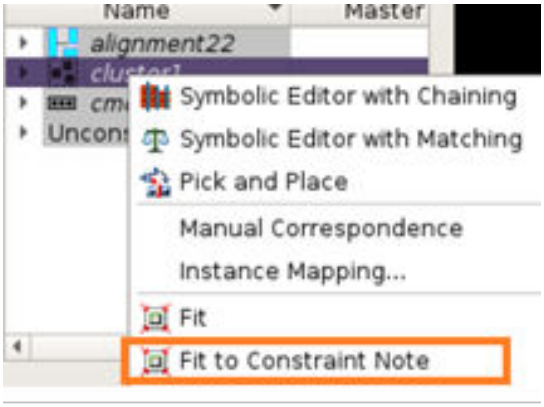
Cross selection related with note object behaves in a similar manner as with intent members, particularly when the intent is selected in **Constraint Editor**. The associated note is highlighted on a canvas along with intent members and viceversa. If the note object is selected on the canvas, then the associated intent is selected in **Constraint Editor**.

SDL Flow and Layout Design

The note object association is also reflected in **Layout Design**. A constraint Boolean parameter `intent_note` is maintained to indicate that owner constraint object has an associated note in **Schematic Design**.

During SDL flow, the transfer parameter is copied onto Layout Design. This parameter only maintains note association and does not affect parameter matching logic like reporting mismatch and so on.

The `lxBrowserFitToNote` action is displayed in the **Constraint** section of Design Navigator that can be used to fit schematic assistant to the associated note object of the constraint.



Design Navigator Assistant

The Design Navigator lists the nets, figure groups, and terminals in your design. The assistant also offers features that facilitate design editing tasks.

This topic describes the following:

- About the Design Navigator
- Displaying the Design Navigator Assistant
- Switching Between the Layout and Schematic Design Modes
- Locating Components
- Renaming Components
- Manipulating Components
- Placing Design Areas in Shadow
- Setting Flight Line Display Options
- Showing and Hiding Flight Lines From the Canvas
- Working With Nets
- Working With Figure Groups

Feedback

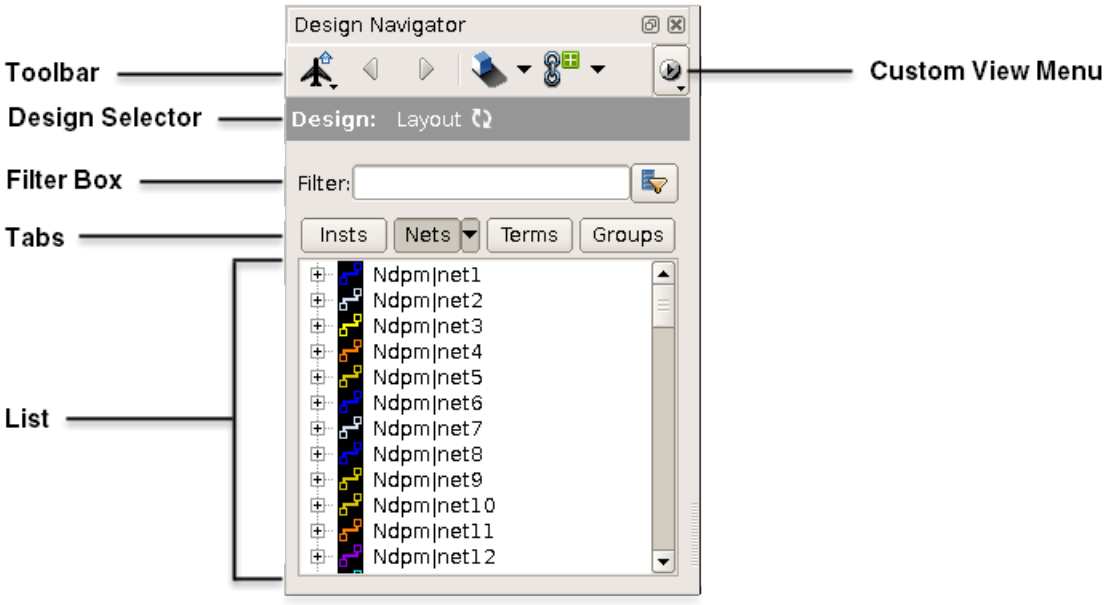
About the Design Navigator

You use the Design Navigator to:

- Locate and select instances, nets, terminals, figure groups, and synchronous groups in the design
- Rename instances, nets, and groups
- Place design areas or non-selected objects in shadow
- Display and hide flight lines
- Manage flight line colors
- Work with nets
- Work with groups of nets and figures
- Synchronize cloned figure groups you have created with the Layout Editor
- Display the Schematic assistant and browse the schematic

To create and clone figure groups, see the *Custom Compiler Layout Editor User Guide*. To control the area selection of figure groups, see Controlling Area Selections of Figure Groups. For more information about the Schematic assistant and Design Navigator with logic information, see Browsing the Schematic Using the Design Navigator in the *Custom Compiler SDL and Visually Assisted Automation Guide*.

The Design Navigator consists of the following:



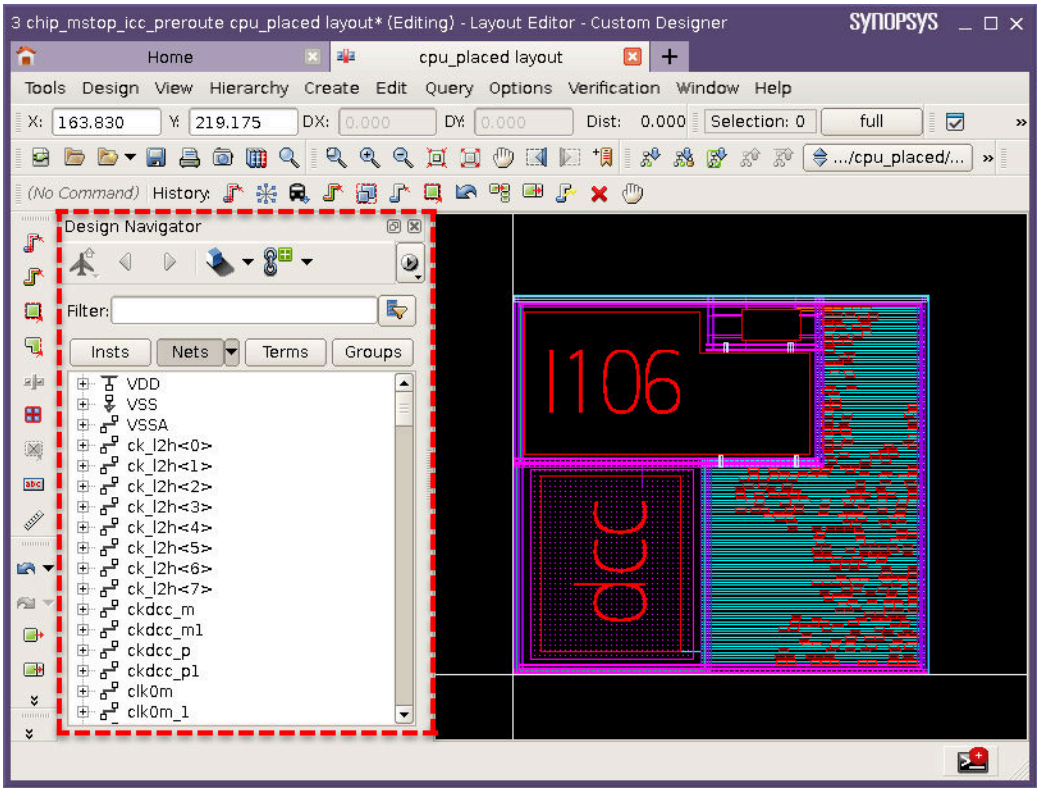
Feedback

Displaying the Design Navigator Assistant

To display the Design Navigator assistant, use one of the following methods:

- From any layout or schematic design, choose **Window > Assistants > Design Navigator**.
- From a design layout where you are drawing routes, click the **Net to Route** button on the Auto Route command options toolbar.
- From any command options toolbar where you see the **Design Navigator** button, click it:

By default, the Design Navigator appears on the left side of the design window. You can undock and move it by dragging it to another location.



Switching Between the Layout and Schematic Design Modes

In the Design Navigator, you can switch between the physical (layout) and logical (schematic) information. This feature is available only if you used the Schematic-Driven

Feedback

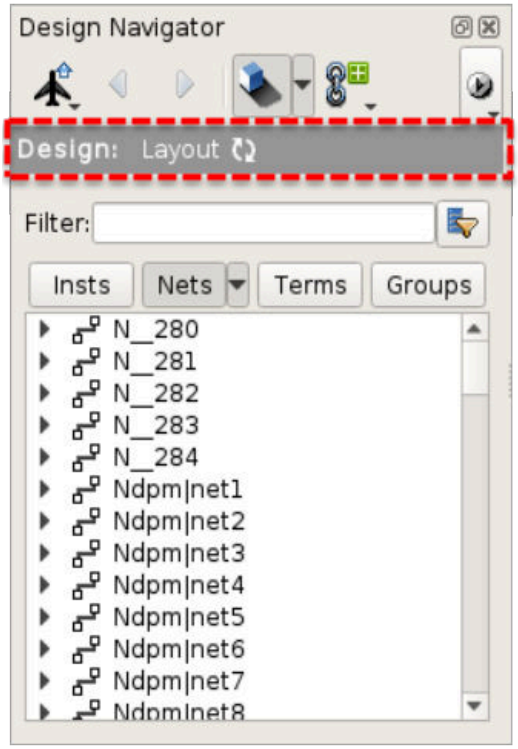
Layout (SDL) tool to establish a correspondence between a design schematic and a design layout.

To switch between the physical and logical information of a design:

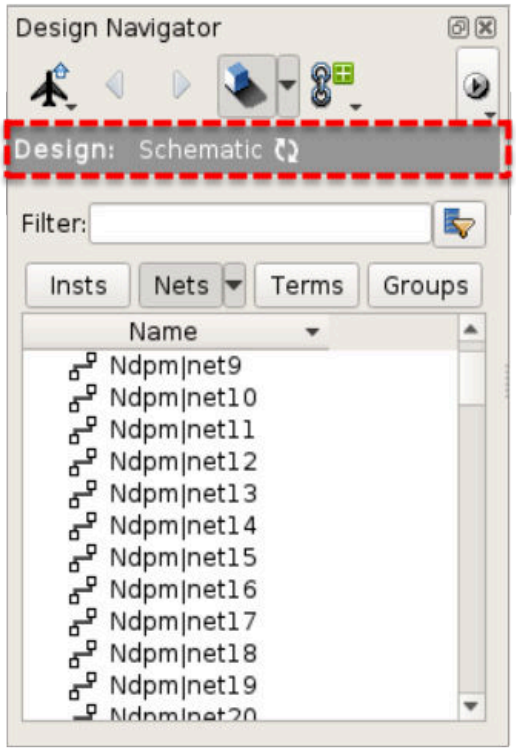
► In the Design Navigator, click one of the following:

- **Design: Layout**: Switches to the physical (Layout) mode.
- **Design: Schematic**: Switches to the logical (schematic) mode.

In Schematics



In Layout



See Also

- Browsing the Schematic Using the Design Navigator

Locating Components

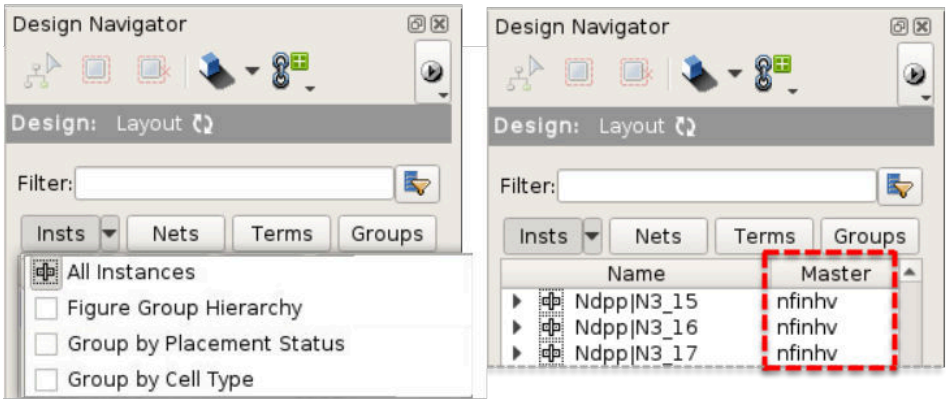
Using the Design Navigator, you can locate instances, nets, and terminals in your design. If some components are grouped, you can locate the groups in the design. After you locate components, you can work with them. See Manipulating Components.

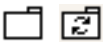
Feedback

To locate design components:

1. Make sure the Design Navigator is open. If it is not, choose **Window > Assistants > Design Navigator**.
2. In the Design Navigator, click one of the following tabs:

Tab	Symbols	Description
Insts		Displays the list of instances in the design. When you click the **Insts** tab, a menu becomes available and you can choose to display different components, depending on whether you are currently viewing the design schematic or layout. For example, when viewing layouts, the available components include **All Instances**, **Figure Group Hierarchy**, **Group by Placement Status**, and **Group by Cell Type**, **Instance by Master**, **Instance Hierarchy**, or **Master Hierarchy** when available. **Note:** **All Instances** is set as the default. When displaying **All Instances**, the corresponding master cell names are displayed in the **Master** column. When the master cell type is via and when the `heDesignNavigatorViaAsInstance` Tcl preference is set to `true`, instances as vias are not listed.



Tab	Symbols	Description
Nets		Displays the list of nets in the design. When you click the **Nets** tab, a menu becomes available and you can choose to display **All Nets**, **Unrouted Nets**, **Nets by Group**, or **Nets by Signal Type**. Note that when displaying Nets by Group, any nets not contained in a group are placed in the **Ungrouped** list. When the master cell type is via and when the `heDesignNavigatorViaAsInstance` preference is set to `true`: ◦ Actions to select, deselect, and delete net shapes apply to instances as vias if they have the instance terminal on the net. ◦ Logically, cross-selects nets when instances as vias are selected and the instance has the instance terminal on the net.
Terms		Displays the list of terminals in the design. To search for specific terminals by name, you can alternatively use the Search and Replace Tool.
Groups		Displays the list of figure groups or synchronous groups in the design.
Reference Arrow		Switches by reference to the next tab and displays components connected to the selected component. To see the reference arrow, rest the cursor on any component name. To switch to the next tab, click the reference arrow that appears. For example, clicking the **Nets** tab displays the list of nets and their connected terminals. Clicking the reference arrow to the right of a terminal name switches your view to the **Terminals** tab and list for that net. You can navigate farther through the terminals using their reference arrows, or you can return to the list of nets by clicking the **Nets** tab again.

3. (Optional) Filter the number of items initially shown in any list by entering unique characters in the **Filter** box.

   For example, to find all ground nets in a list of nets, first click the **Nets** tab, and then enter **VSS** in the **Filter** box.

   The tool treats all searches or auto fills as case insensitive, by default. However, if at least one uppercase letter is included, case sensitivity appears.

Feedback

To quickly delete the text in the **Filter** box, click the X button:

4. When you locate the list of components you want to work with, highlight the component names in the Design Navigator.

   Use the keyboard and the mouse.

   - To highlight one name, click the name.
   - To highlight a contiguous group, Shift-select the starting and ending names.
   - To highlight multiple individual names, Ctrl-select one at a time.
   - Alternatively, you can select a shape or shapes in the design, which highlights the shape or shapes by name in the Design Navigator.

5. (Optional) Filter the list to show only the currently highlighted names by clicking the **Show Only Selected Items** button:

6. With the list of components you want to use now highlighted, and optionally filtered, select them in the design.

   You can use the following toolbar buttons to select components in a design.

Button	Tooltip	Action
	Fit to Design Object	Fits the design viewport to the component whose name is selected in the list.
	Select in Design	For components highlighted in the list, selects them in the open design.
	Deselect in Design	Deselects currently selected components in the open design.

7. Use the Design Navigator to act on the selected components.

   When you are ready to act on the selected components, see Manipulating Components.

Renaming Components

You can rename the following components in the Design Navigator:

- instances
- nets
- groups

To rename a component:

1. Double-click the instance, net, or group to rename.
2. Enter the new name.
3. Press Enter.

If the new name already exists in the design, the renaming is not allowed, and a message is displayed in the Console.

Manipulating Components

You can manipulate components you have located through the Design Navigator. You can select the components in the design. You can navigate with them, run reports on them, and create and manage them in groups.

Prepare for this procedure by Locating Components.

To work components using the Design Navigator, right-click the component name or names, and choose an option from the menu that appears.

The tables that follow list menu options for each component type.

If a menu command is available on a toolbar, the table shows the icon for the toolbar button.

- Table 62 lists options for the **Instances** menu.
- Table 63 lists options for the **Nets** menu.
- Table 64 lists options for the **Terminals** menu.

Table 62 Design Navigator Instances Menu

Button	Menu Option	Description
	Fit	Zooms to this instance, and fits it to the viewport.

Feedback

Table 62 Design Navigator Instances Menu (Continued)

Button	Menu Option	Description
	Descend	Descends to this instance in the design.
	Select in Design	Selects this instance in the design.
	Deselect in Design	Deselects this instance in the design.
	Select Connected	Selects this instance, and its connected **Islands**, **Branches**, or **Nets**.
	Deselect Connected	Deselects this instance, and its connected **Islands**, **Branches**, or **Nets**.
	Placement Status	Shows, and allows adjustment of, the placement status for this instance to **None**, **Unplaced**, **Placed**, **Fixed**, or **Locked**. With multiple instances selected where the placement status varies, no status is shown.
	Extract Parasitics	Performs parasitic extraction on this instance.
	Delete Instances	Deletes this instance or instances from the design.

Feedback

Table 62 Design Navigator Instances Menu (Continued)

Button	Menu Option	Description
	Select Instance View Select Occurrence View	Sets the instance or occurrence view for multiple instances. Right-click on the instance to view the options. Both the menus provide the following options: • **<open>**: Allows to set open rules for the selected instances. When you set the view name to <open>, the selected instance is bound to open view. • View names are listed based on the set open rules for the selected instances. The view names include all the views in the selected instance master cell, as follows: ◦ Shows all the view names in the list. For example, if the selected instances are I0 and I1, ▪ I0 master is Cell1/symbol, this cell has view1, view2 ▪ I1 master is Cell2/symbol, this cell has view1, view3 The view name lists the following views: ▪ <open> always exists ▪ view1 is enabled as it exists in both I0 and I1 master cell (cell1 and cell2) ▪ view2 is disabled as it exists in cell1 only and not in cell2 ▪ view3 is disabled as it exists in cell2 only and not in cell1 Click the enabled view name to set selection rule. ◦ Disables the view name if the view name does not exist in the master cell of all the selected instances. ◦ Enables the view only if the view name exists in the master cell of all the selected instances. Indicated by *italic* and **bold** style to show that the selected instances are bound to the view. Both the menus are disabled: • When you do not link schematic design in the hierarchy configuration (config view) • If the config view is read-only • If one of the selected instance is set in overlay config view
	Placement	Creates the placement constraint you choose from the list of available constraints for this net. For information about placement constraints, see the *Custom Compiler Layout Editor User Guide* for details.

Feedback

Table 63 Design Navigator Nets Shortcut Menu

Button	Menu Operation	Description
	Fit	Zooms to this net, and fits it to the viewport.
	Select Shapes in Design	Selects these net shapes in the design.
	Deselect Shapes in Design	Deselects these net shapes in the design.
	Select Instances	Selects instances connected to this net.
	Delete Net Shapes	Deletes all shapes on all selected nets that are not members of pins.
	Set Signal Type	Sets net signal type to **Signal**, **Power**, **Ground**, **Clock**, **Tie-Off**, **Tie-Hi**, **Tie-Lo**, **Analog**, **Scan**, or **Reset**.
	Set Flight Line	Sets flight lines with options **Hidden**, **Persistent**, or **Shown**, and also reconfigures their color. See Setting Flight Line Display Options. When **Hidden** is selected, no figures on net are highlighted.
	Add to Group	Adds the selected net or nets to a net group. For details, see Creating Net Groups, and Adding Nets to Net Groups.
	Remove from Group	Removes this net from its group.
	Trim Nets	Trim paths and dangling branches. For details, see Trimming Wires in the *Custom Compiler Layout Editor User Guide for Connecting Objects*.
	Report Nets	Generates a net report. For details, see Reporting Information about Nets.
	Report Resistance	Reports resistance on a net, or between source and destination pins or coordinates on a net. For details, see Reporting Resistance for Selected Path (Pin-to-Pin, Terminal-to-Terminal, Instance-to-Instance, Point-to-Point) in the *Custom Compiler Electrical Checking and Reporting User Guide*.
	Report EM	Reports electromigration. For details, see Reporting Electromigration in the *Custom Compiler Electrical Checking and Reporting User Guide*.

Feedback

Table 63 Design Navigator Nets Shortcut Menu (Continued)

Button	Menu Operation	Description
	Report Shield	Reports shield coverage. For details, see Reporting Shield Coverage in the *Custom Compiler Electrical Checking and Reporting User Guide*.
	Check Resistance	Checks resistance on a net. For details, see Checking Resistance in the *Custom Compiler Electrical Checking and Reporting User Guide*.
	Check EM	Checks electromigration. For details, see Checking Electromigration in the *Custom Compiler Electrical Checking and Reporting User Guide*.
	Check Shield	Checks shield coverage. For details, see Checking Shield Coverage in the *Custom Compiler Electrical Checking and Reporting User Guide*.
	Routing	Creates the routing constraint you choose from the list of available constraints for this net. For information about routing constraints, see the *Custom Compiler Custom Router User Guide*.

Table 64 Design Navigator Terminals Shortcut Menu

Button	Menu Operation	Description
	Fit	Zooms to this terminal, and fits it to the viewport.
—	Placement Status	Sets the placement status for the selected terminals: • **None**: The placement of the terminal pins is not yet set. • **Unplaced**: The terminal pins are placed arbitrarily, and the placement should be changed. The Layout Editor cannot locate unplaced pins. • **Placed**: The terminal pins have been placed, and the placement can be changed. • **Fixed**: The placement of the selected terminal pins cannot be changed by automatic processes. However, you can manually change the placement. • **Locked**: The placement of the selected terminal pins cannot be changed by automatic or manual methods.
	Select Shapes in Design	Selects these terminal shapes in the design.
	Deselect Shapes in Design	Deselects these terminal shapes in the design.

Feedback

Table 64 Design Navigator Terminals Shortcut Menu (Continued)

Button	Menu Operation	Description
	Delete Pins	Deletes the pins on this terminal.

Placing Design Areas in Shadow

You can place design areas in shadow for additional visual contrast between selected nets, and the surrounding design. To increase contrast between selected objects and the surrounding design, set the `leShadowSelection` Tcl preference to true.

To place design areas in shadow:

1. Begin by opening the Design Navigator assistant. From any design layout, choose **Window > Assistants > Design Navigator**.
2. Locate a net either in the list or by selecting it directly on the canvas.

   For more information about locating nets, see Locating Components.
3. On the Design Navigator toolbar, click (Shadow Mode).

   Shadow mode increases the contrast between the nets, and the surrounding design, allowing for more targeted net viewing.
4. To adjust the contrast, click the arrow and move the slider up or down.

Setting Flight Line Display Options

Flight lines identify missing connections between physical objects on the same net in analog designs. You can set the display options to show or hide the flight lines and assign colors to the flight lines.

Before you set the display options, make sure the `leDisplayFlightlines` Tcl preference is set to `true`.

Feedback

To set the flight line display options:

1. In the Design Navigator, click the **Nets** tab.
2. Enable the Connectivity Engine as follows:
   a. Choose **Options > Design**. The **Layout Design Options** dialog box opens.
   b. Click the **Connectivity** tab, and then select **Connectivity Engine**.
   c. Click **OK** to close the dialog box.

   Notice that in the Design Navigator, the list of nets shows current flight line color settings.
3. In the Design Navigator **Nets** list, select the nets for which you want to apply the flight line display settings.

   Alternatively, you can select nets by selecting shapes, vias, or instances (instance terminals). When you select objects on the canvas, the nets associated with the objects are selected.
4. On the Design Navigator toolbar, click **Set Flight Line**. The Flight Line menu appears. The menu contents depend on whether you selected the **Show Manual** or **Show Selected** option in the **Layout Design Options** dialog box.

Flight Line Show Option	Flight Line Menu	
Show Selected		◦ **Hidden**: Does not display the flight lines. ◦ **Persistent**: Always displays flight lines regardless of whether you select the nets. ◦ **Shown**: Dynamically displays or hides the flight lines when you select or deselect the nets in the Design Navigator or netted objects on the canvas. On the canvas, you can select 0x shapes that are on nets, 0x and 1x shapes on pins, 0x vias on nets, and all instance terminals for 0x instances. ◦ <color>: Assigns the selected color to the flight lines. Choose (random color) to automatically assign different colors to adjacent flight lines. Tcl preference: `leFlightlineSyncSelection` = `true`

Flight Line Show Option	Flight Line Menu	
Show Manual		◦ ⊘: Does not display flight lines. ◦ <color>: Assigns the selected color to the flight lines. Choose (random color) to ensure that neighboring flight lines are assigned different colors. Tcl preference: `leFlightlineSyncSelection` = `false`

If the **Connectivity Engine Mode** in the **Layout Design Options** dialog box is **Netlist**, the `leFlightlineSyncSelection` Tcl preference is set to `false`, and all flight lines are hidden by default.

If a net's signal type is power or ground, the flight line is hidden by default.

5. From the Flight Line menu, choose the display setting and the color to assign to the selected nets.

   For example, you can select a set of nets on the canvas, and then use the Flight Line menu to assign the orange color to those nets.

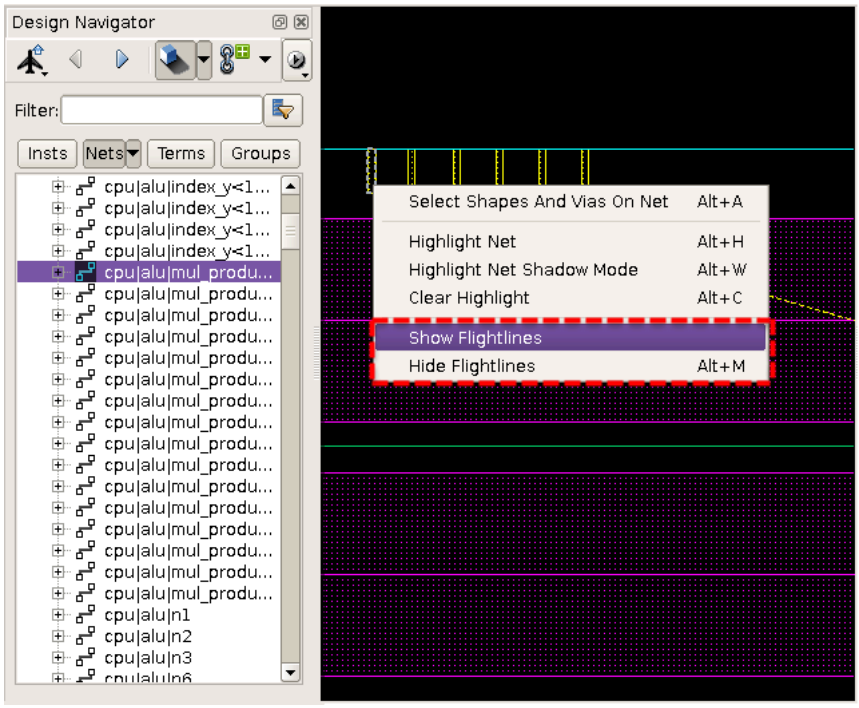
   Nets for which flight lines are not enabled are shown in black.

Showing and Hiding Flight Lines From the Canvas

To show or hide flight lines while working with shapes on the canvas:

1. Make sure you set the flight line display options in the Design Navigator. See Setting Flight Line Display Options.
2. Right-click the shapes, paths, or instances, and then choose either **Show Flight Lines** or **Hide Flight Lines**.

Feedback



When flight lines are displayed, you can view flight lines that run between centers of instances that do not have pins.

Working With Nets

You can work with nets by generating net detail and net resistance reports, creating objects with net names, creating net groups, and creating constraints.

These sections cover:

- Before You Start
- Adding Nets to Net Groups
- Creating Net Groups
- Viewing Grouped Nets
- Setting Signal Types
- Viewing Information About Nets
- Viewing Open Nets

- Viewing Shorted Nets
- Reporting Information about Nets

For information about using the Design Navigator to check and report wire resistance and electromigration, see the *Custom Compiler Electrical Checking and Reporting User Guide*. For information about creating net constraints for routing, see the *Custom Compiler Custom Router User Guide*.

Before You Start

Net information is based on logical and physical connections between objects.

When working with nets in design layouts, be sure to turn on the Connectivity Engine. See Enabling the Connectivity Engine.

The Design Navigator shows the list of nets and the objects that are on the nets. You can assign nets to objects by:

- Adding Nets to Schematic Objects
- Adding Nets to Layout Objects

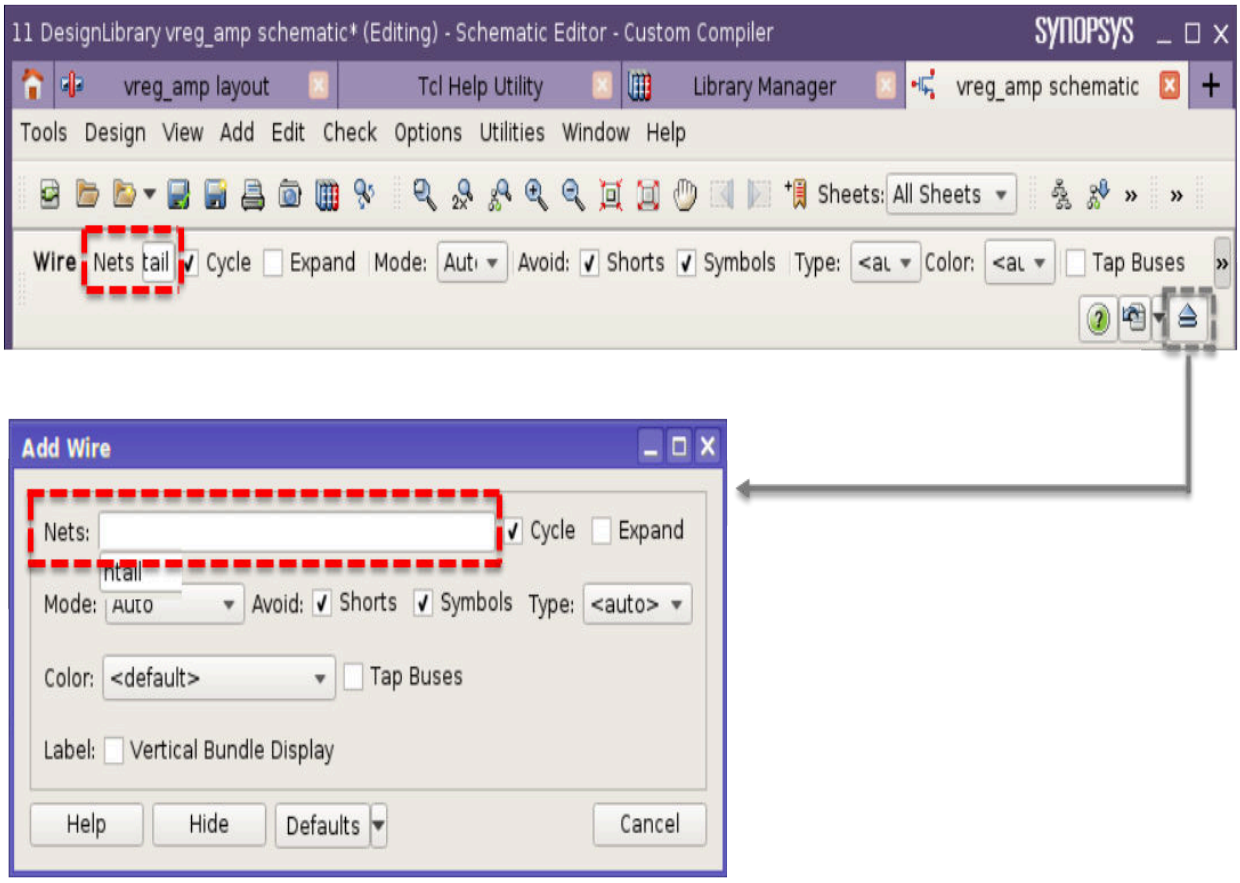
See Also

- About Connectivity

Adding Nets to Schematic Objects

Adding net names to schematic objects associates the objects with connections in the design.

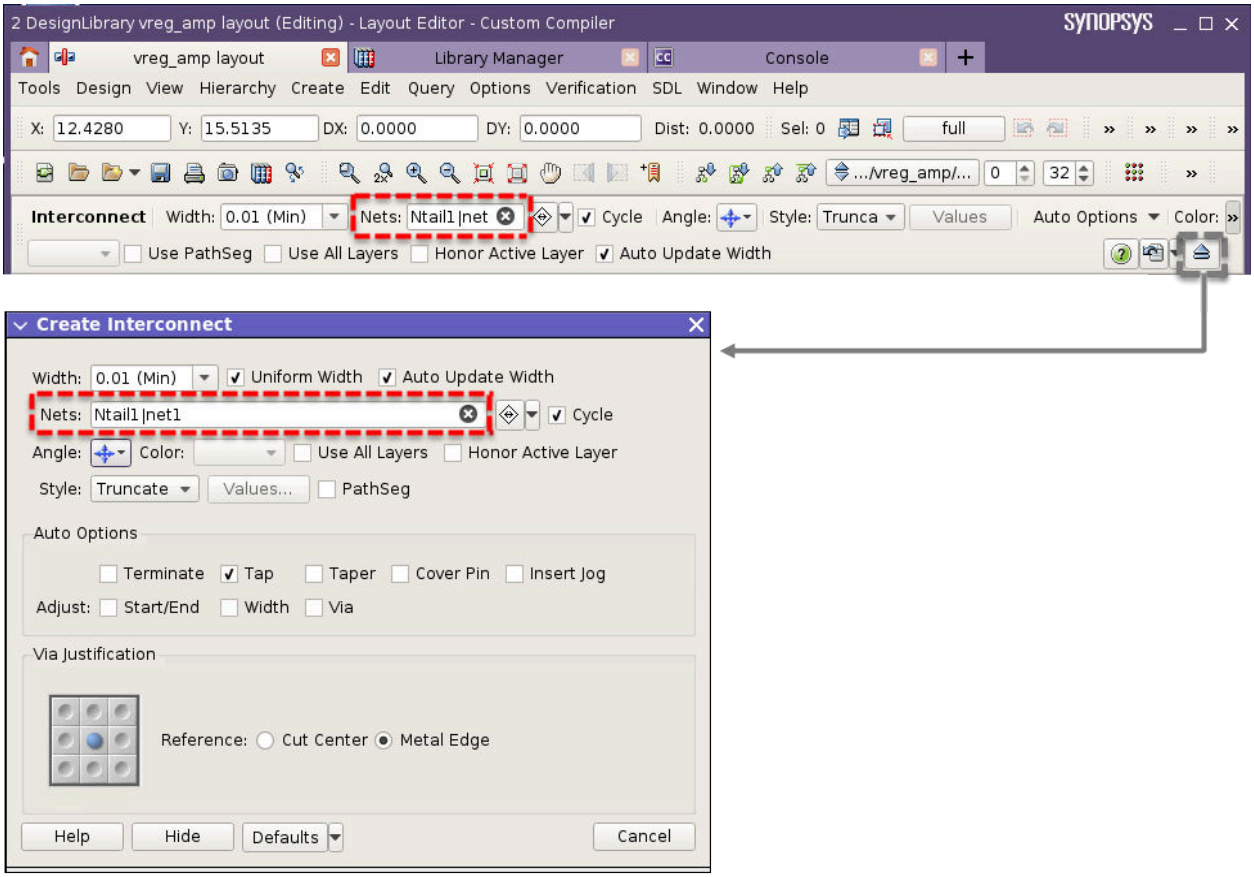
You can add nets to the following schematic objects during schematic creation: wire, route, and by-name connector. For example, when using the **Add Wire** command, you can list one or more **Nets** on the **Wire** toolbar or in the **Add Wire** dialog box.



Adding Nets to Layout Objects

Adding net names to layout objects associates the objects with connections in the design.

You can add nets to layout objects (such as interconnect paths or path segments, topology objects, buses, paths, rectangles, polygons, and ellipses) during layout creation. For example, when using the **Create Interconnect** command, you can list one or more **Nets** on the **Interconnect** toolbar or in the **Create Interconnect** dialog box.



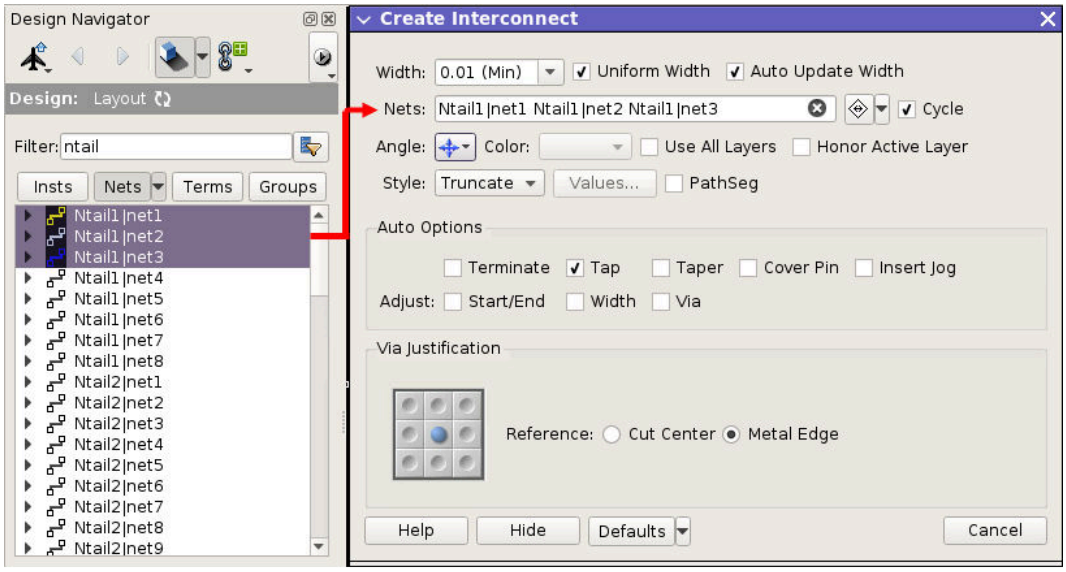
Instead of typing the net names manually, you can automatically add the nets selected in the Design Navigator to the Layout Editor command **Nets** box.

To automatically add Design Navigator net selections to the Layout Editor command Nets box:

1. Set the `leAutoPickUpNamesFromDN` Tcl preference to true.
2. In the Design Navigator, select the nets.
3. In the Layout Editor, run one of the following commands. The net selections in the Design Navigator automatically appear in the **Nets** box for the command.

Feedback

- Create Bus
- Create Interconnect
- Create Pin



If you modify the net selection in the Design Navigator, you can synchronize the changes and update the list of **Nets** in the Layout Editor command:

1. On the command toolbar or dialog box, click **Synchronize** . The **Nets** list update to match the selections in the Design Navigator.
2. To change the order of the nets, click , and choose one of the following:
   - **Ascending Order**: Sorts the net names in alphanumeric ascending order.
   - **Descending Order**: Sorts the net names in alphanumeric descending order
   - **Selection Order**: Keeps the order in which the nets were selected in the Design Navigator.

   The net synchronization is available for the following Layout Editor commands:
   - Create Ellipse
   - Create Polygon
   - Create Rectangle
   - Create Pin (by Shape)

- Create Interconnect
- Create Pin
- Create Via (Auto mode)
- Reassign Net
- Create Plane Boundary

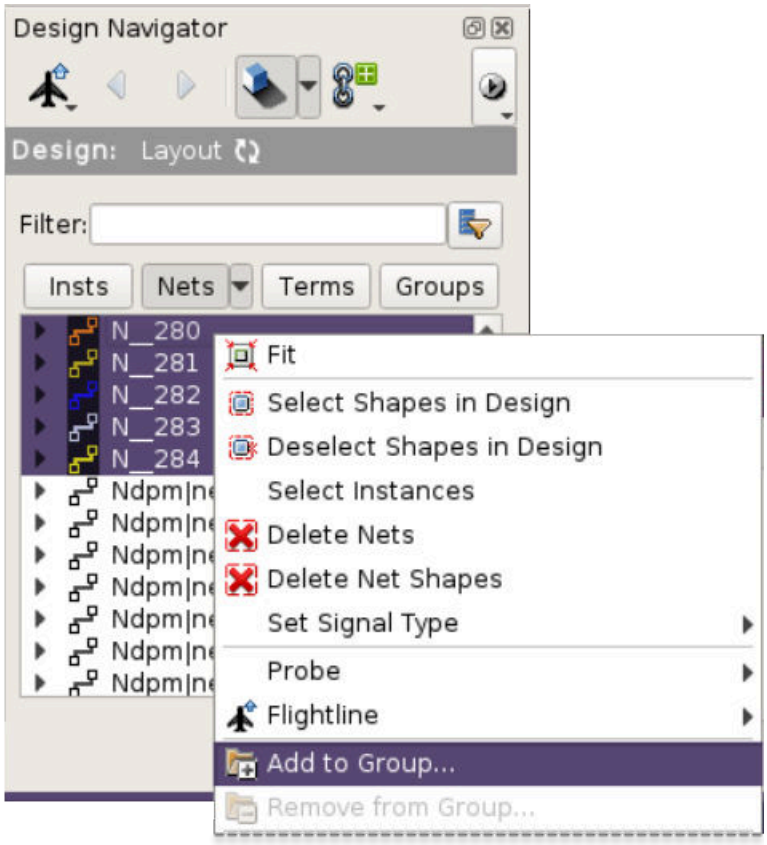
Adding Nets to Net Groups

You can use the Design Navigator to add nets to existing net groups.

To add nets to net groups:

1. From any design schematic or layout, choose **Window > Assistants > Design Navigator**.
2. Click the **Nets** tab to list the nets in the opened design.
3. In the **Nets** list, select the names nets you want to add to a group.
4. Right-click the selected nets and choose **Add to Group**.

Feedback



5. In the **Add to Group** dialog box that opens, locate the group, using the **Filter** box, if necessary.
6. Select the group by its name.
7. Click **Add to Group**.

   The dialog box closes. The selected nets are added to the specified group.
8. To confirm the new additions to the group, see Viewing Grouped Nets.

Creating Net Groups

You can create net groups to organize them and to facilitate net selection during schematic and layout operations.

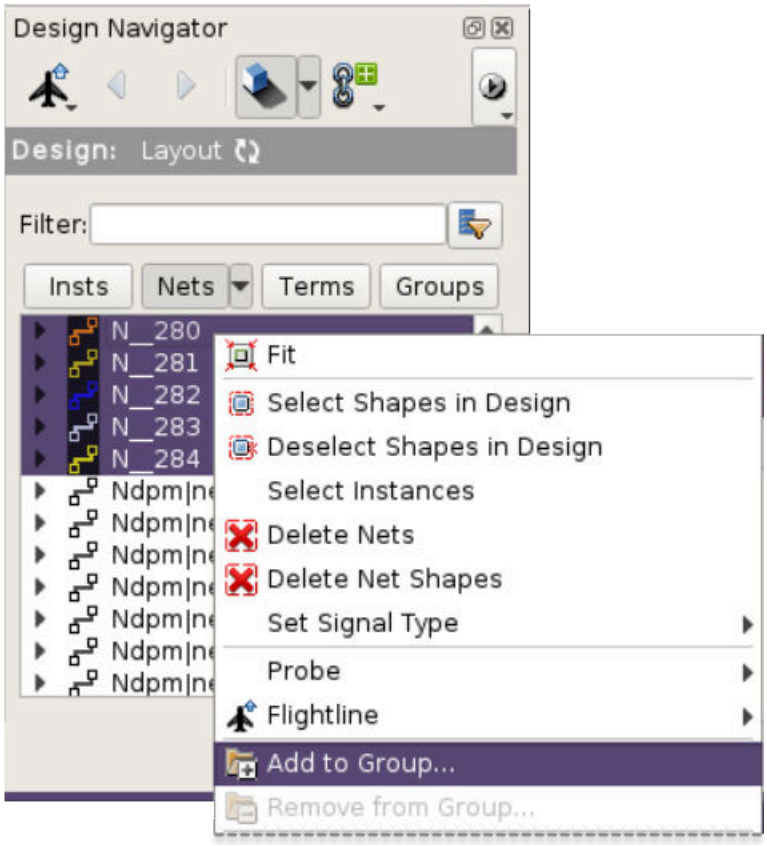
To create net groups:

1. From any design schematic or layout, choose **Window > Assistants > Design Navigator**.
2. In the Design Navigator assistant, click the **Nets** tab to show the list of nets.

Feedback

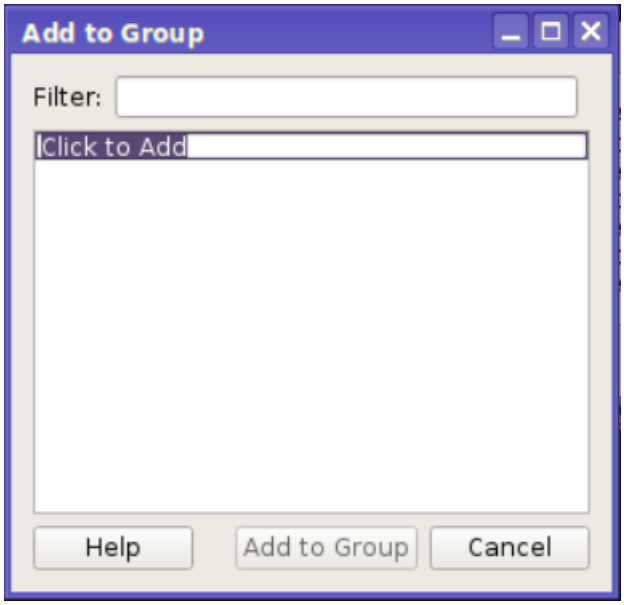
You can navigate to the list of nets you want to group. To do this, see Locating Components.

3. In the **Nets** list, select the net or nets by name.
4. On the highlighted name or names, right-click and choose **Add to Group**.



5. In the **Add to Group** dialog box, click **Click to Add** and enter a name for the group. Press **Enter**.

Feedback



6. Select the new net group and click the **Add to Group** button to close the dialog box.

See Also

- Adding Nets to Net Groups

Viewing Grouped Nets

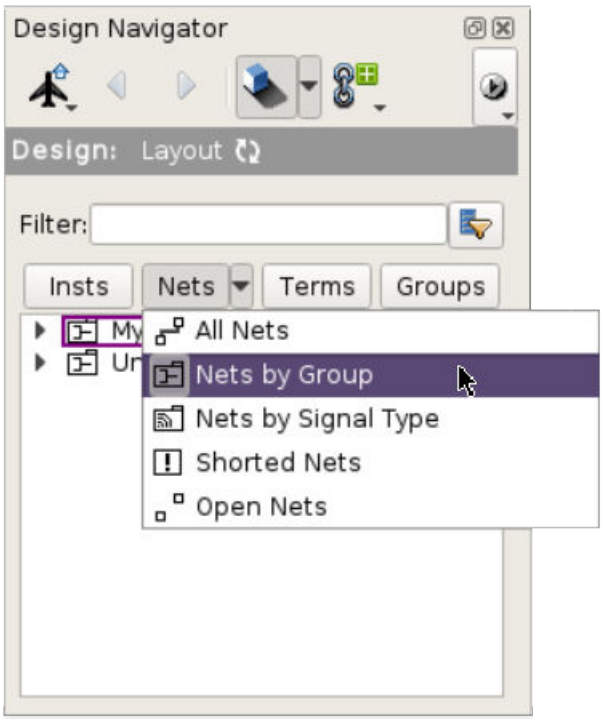
The Design Navigator lets you filter the **Nets** list so that you can view only the grouped nets. These filtered views allow you to quickly locate nets of interest.

To view only grouped nets:

1. From any design schematic or layout, choose **Window > Assistants > Design Navigator**.
2. Click the **Nets** tab to list the nets in the opened design.
3. On the **Nets** tab, choose **Nets by Group**.

   The list of net groups appear.

Feedback



4. Expand the group name.

   The name of the nets you added to the group appear in the list.

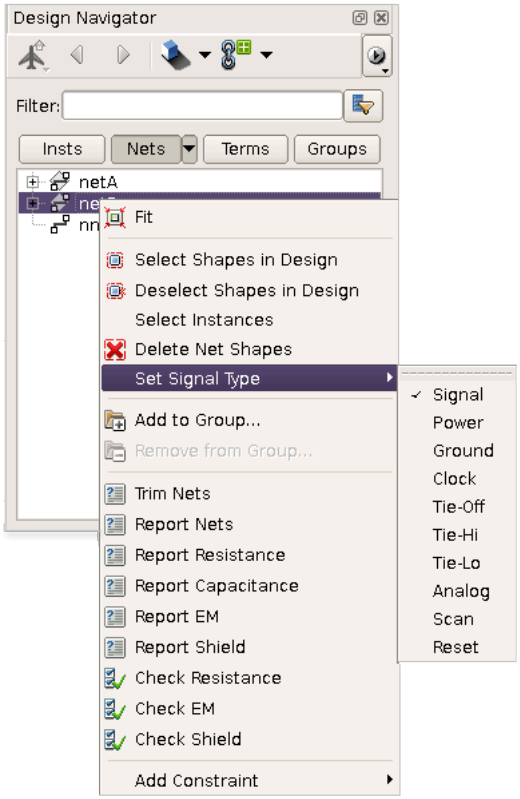
Setting Signal Types

To facilitate the editing of net information, you can set or change the net signal type in the Design Navigator.

Feedback

To set or change the net signal type:

► In the Design Navigator **Nets** tab, right-click the net name, and then choose **Set Signal Type**.



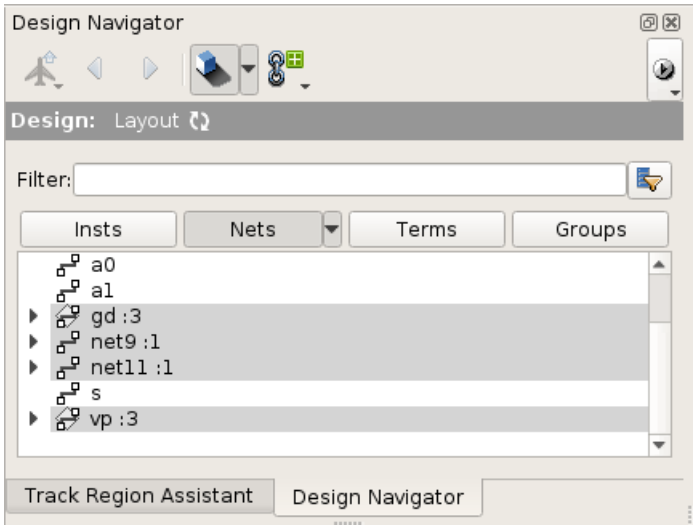
Viewing Information About Nets

The Design Navigator provides visual indicators to notify you of unrouted nets and unconnected pins. Having visual indicators can help you quickly locate issues in the design.

Feedback

The Design Navigator Nets tab provides the following visual indicators:

- Gray background behind a net name indicates that the net is not routed.
- For nets with a gray background, values next to the net name show the number of pins or instPins that are not connected. To display the number of unconnected pins, set the `heDesignNavigatorShowExtendedFlightlineInfo` Tcl preference to true.



Viewing Open Nets

Open nets are nets that are not yet routed. You can display the list of open nets for targeted viewing and troubleshooting.

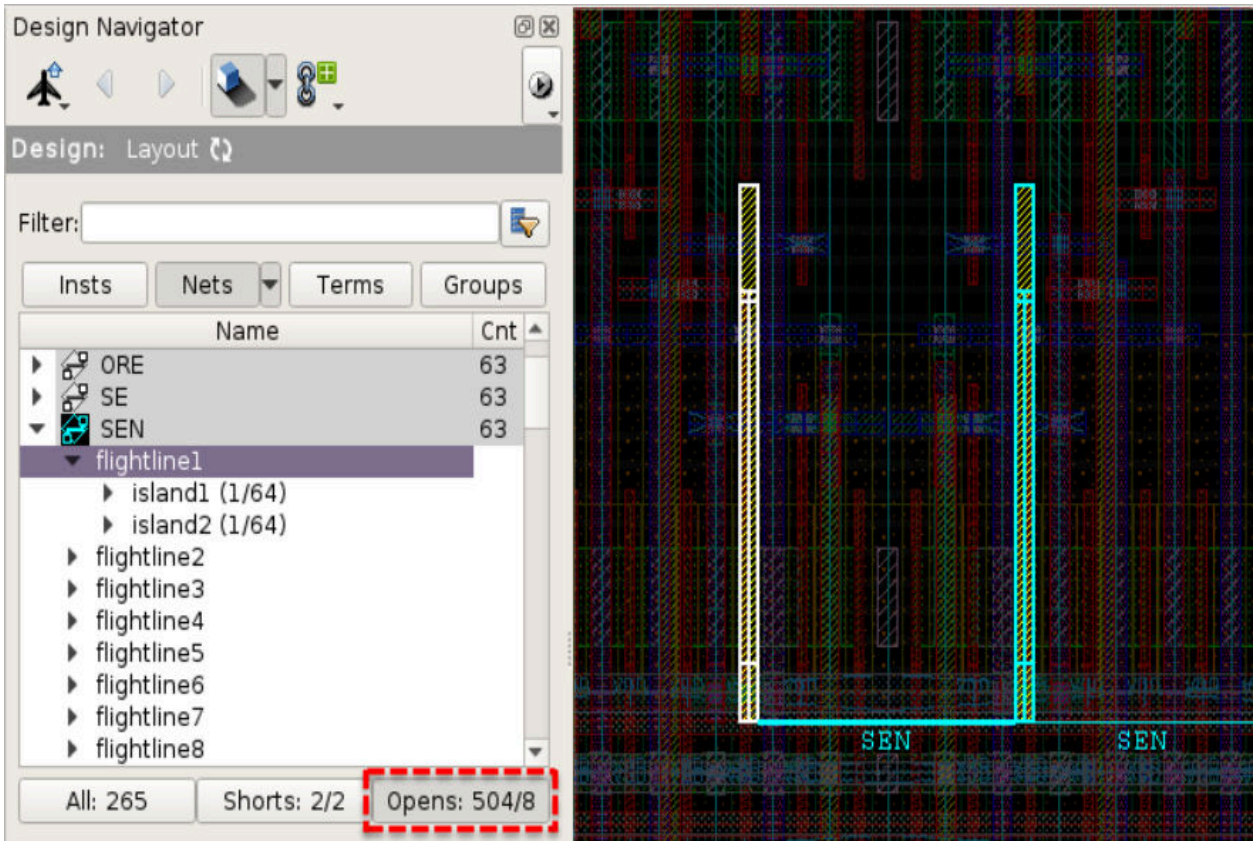
To display only the list of open nets:

1. In the Design Navigator, click the **Nets** tab, and then click the **Opens: n/m** tab.

   The **Nets** tab refreshes and displays only open nets.

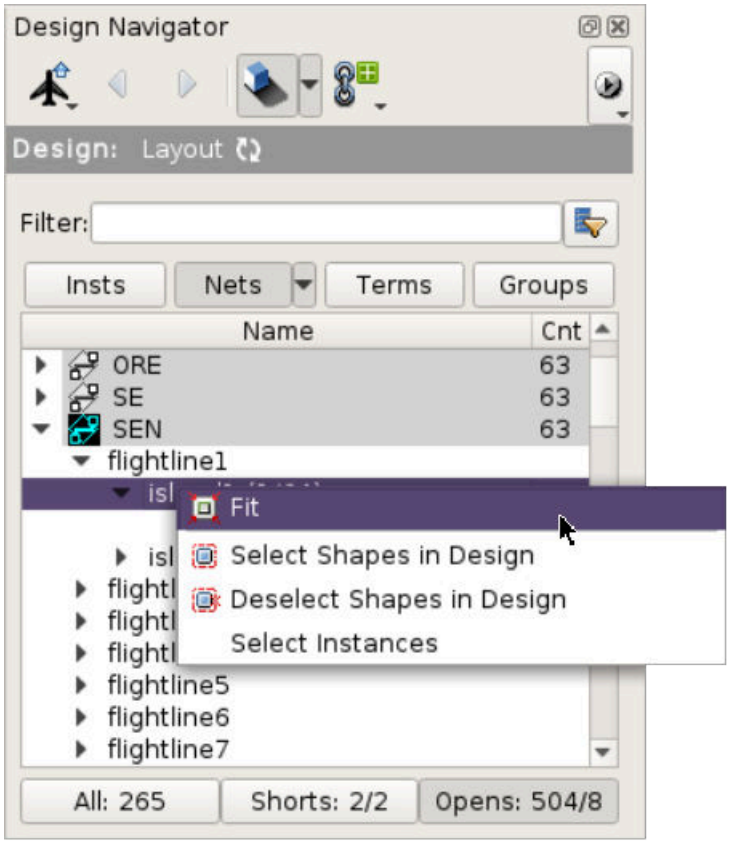
   - The total number of opens and open nets are displayed next to the net name.
   - Under each net name is the list of flight lines and associated Connectivity Islands.
   - Under each island is the list of terminals or instance terminals.

Feedback


Design Navigator
Design: Layout
Filter:
Insts
Nets
Terms
Groups
Name
Cnt
ORE 63
SE 63
SEN 63
flightline1
island1 (1/64)
island2 (1/64)
flightline2
flightline3
flightline4
flightline5
flightline6
flightline7
flightline8
All: 265
Shorts: 2/2
Opens: 504/8
SEN
SEN

Feedback

2. (Optional) To zoom in to the location of the open net, right-click the open-net name in the Design Navigator and choose **Fit**.



See Also

- Error Viewer Assistant

Viewing Shorted Nets

Shorted nets are nets that contain short circuits. You can display the list of shorted nets for targeted viewing and troubleshooting.

A short circuit occurs when path segments on the same layer but belonging to different nets touch, causing the nets to connect. You can view short circuits using one of the following methods:

Method	Description
Design Navigator	Provides a high-level and quick view of shorts by net, layer, and object.

Feedback

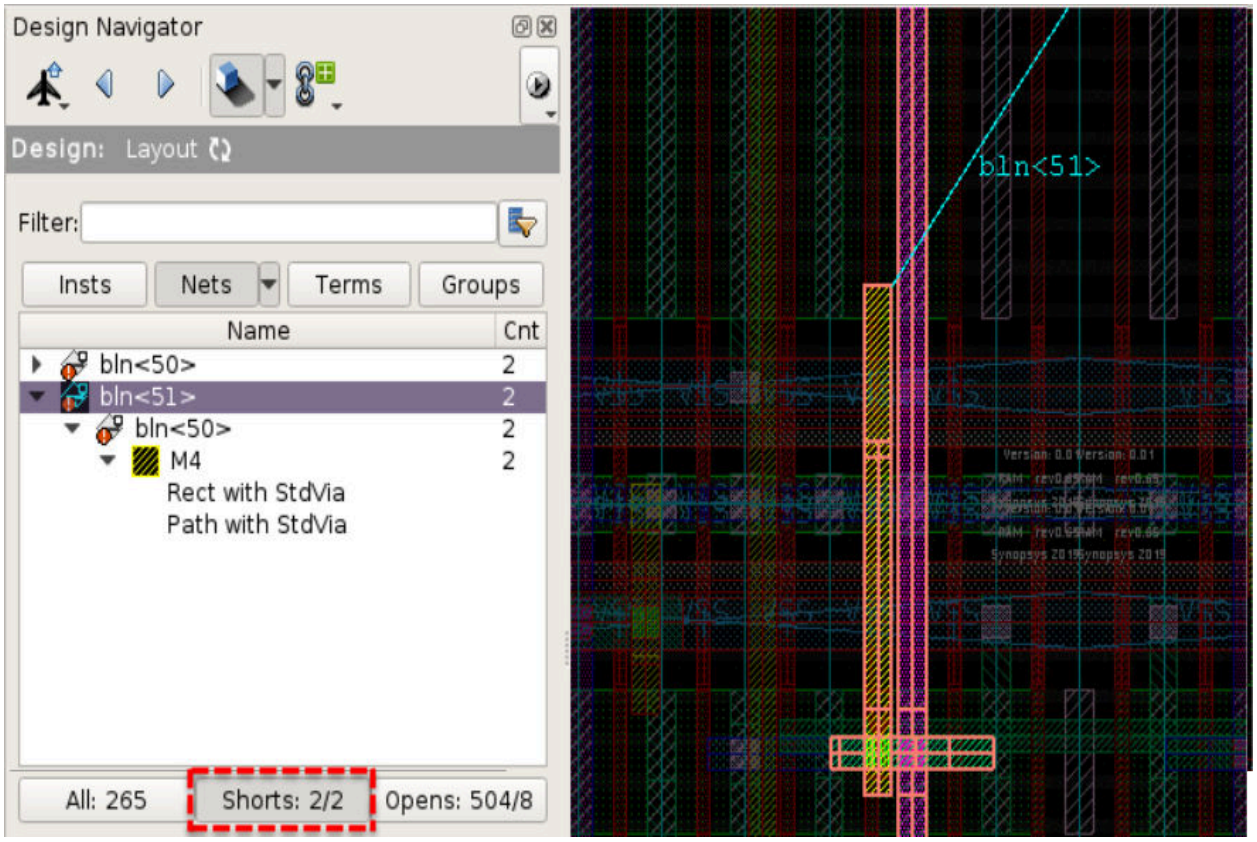
Method	Description
Error Viewer	Provides details about each short.

This topic describes how to use the Design Navigator to view shorts in design layouts. For detailed views of shorts, see Error Viewer.

To display the list of shorted nets:

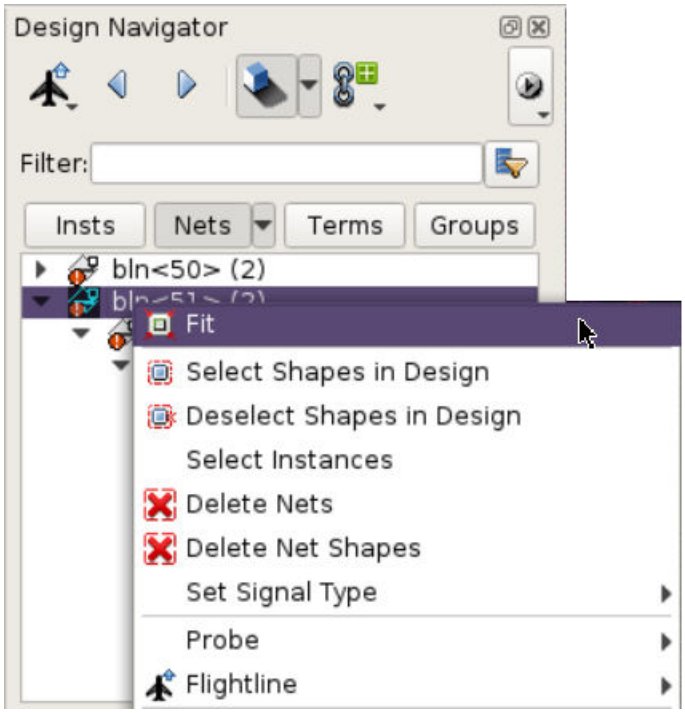
1. In the Design Navigator, click the **Nets** tab, and then click **Shorts**.

   The **Nets** tab refreshes and displays only shorted nets. The total number of shorts and shorted nets appear next to the net name. In the following example, the bln net has two shorts and two shorted nets.



2. Expand the shorted net name and view the details (layer, path shapes, and via shapes).

3. To zoom in to the location of the short, right-click the short in the Design Navigator and choose **Fit**.



To troubleshoot short circuits, see Reporting and Locating Short Circuits.

See Also

- Error Viewer Assistant

Reporting Information about Nets

You can report information about nets in a design to collect this information in one place. The system shows this report in the **Console** tab.

Use the Design Navigator to report information about nets.

To report information about nets in a design:

1. Begin by opening the Design Navigator assistant. From any design layout, choose **Window > Assistants > Design Navigator**.
2. In the Design Navigator, locate the net.

   For more information about locating nets, see Locating Components.
3. On the net name, right-click, and choose **Report Nets**.

4. In the Console **Output** tab, view the report details.

   For more than 100 nets, the report is generated as a file in the CSV format instead of in the **Console** tab with the name, reportNets.csv, under the synopsys_custom directory.

   For a single net, the report lists:

   - Routed net length
   - Routed net length on each valid routing layer
   - Manhattan net length
   - Ratio of routed net length over Manhattan net length
   - Evidence of the use of non-foundry rules for `minWidth`, `minSpacing`, `validRoutingLayers`, `validRoutingVias`, and `minNumCut`

   For a collection of nets, the report lists the information for a single net plus this additional information:

   - Via usage: the total number of vias, the number of vias between adjacent valid routing layers, redundant via rate
   - Number of completely, and partially, routed nets, and unrouted nets
   - Number of nets that use non-foundry rules, expressed as a percentage of all reported nets
   - Optionally reported nets over a user-specified ratio of routed over Manhattan net length
   - Optionally reported nets longer than a user-specified length threshold

Working With Figure Groups

You can use the Design Navigator to work with figures (or shapes) that you have grouped.

You can also use the Design Navigator to work with figure groups you have *cloned* to create multiple identical figure groups. You can synchronize the cloned figure groups. When synchronized, edits to figures in one group automatically apply to figures in all peer groups of this cloned group, synchronizing design development.

You can maintain clone group synchronization to easily apply editing operations that would normally be repetitive, and you can also break synchronization when you want to work with a cloned figure group individually. Editing the elements of the cloned group in other tools will break synchronization. However, you can synchronize them again in the Layout Editor tool.

Feedback

To create figure groups and synchronous clone figure groups, use the Layout Editor. See Grouping Objects.

This topic describes the following:

- Displaying the List of Figure Groups
- Filtering the List of Figure Groups
- Selecting and Unselecting Figure Groups
- Operating on Figure Groups
- Showing Synchronous Clone Groups
- Adding Figure Groups to Synchronous Groups
- Breaking Synchronization of Cloned Groups
- Synchronizing Cloned Groups to Peers

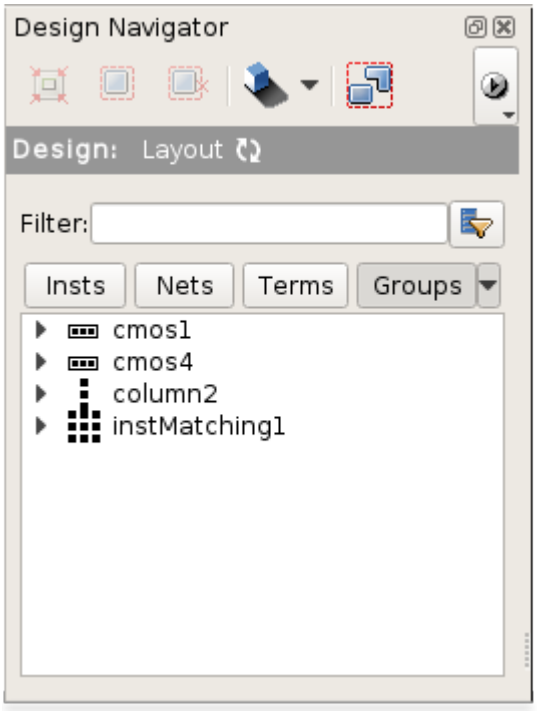
Displaying the List of Figure Groups

You can display the list of figure groups, including cloned figure groups, by name in the Design Navigator. After you display the list of figure groups, you can perform various operations by group.

To display the list of figure groups:

1. Display the Design Navigator. See Displaying the Design Navigator Assistant.
2. In the Design Navigator, click the **Groups** tab. The list of existing figure groups appear.

Feedback



3. (Optional) Click the Design Navigator toolbar and choose one of the following commands:
   - **Expand All**: Expands all figure groups to show subgroups or underlying components.
   - **Collapse All**: Hides the subgroups or underlying components from view.

Filtering the List of Figure Groups

After displaying the existing figure groups in the design, you can filter the list to show only the groups of interest.

Use one of the following methods to filter the figure groups in the Design Navigator:

- Displaying Cloned Synchronous Figure Groups Only
- Filtering Figure Groups by Name
- Displaying Only Selected Figure Groups

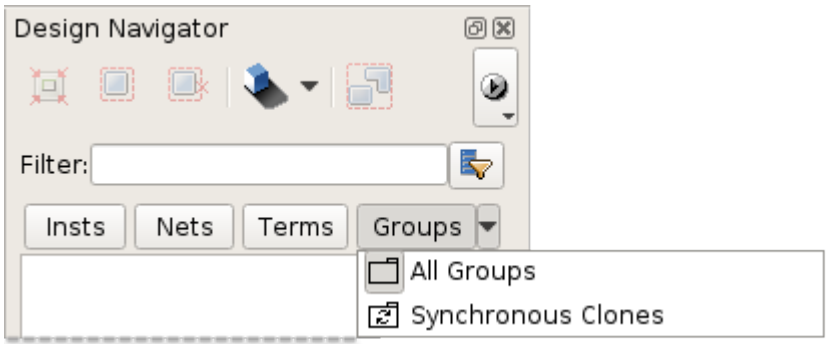
Displaying Cloned Synchronous Figure Groups Only

The Design Navigator **Groups** tab menu lets you display all groups or cloned synchronous figure groups only.

Feedback

To use the **Groups** tab menu to filter the figure groups list:

► From the **Groups** tab menu, choose one of the following options:



Option	Description
All Groups	Lists all figure groups in the open design that are not components of custom elements such as MPPs and rulers.
Synchronous Clones	Lists cloned, synchronized figure groups (only) in the open design. Additionally shows groups with broken synchronization.

Filtering Figure Groups by Name

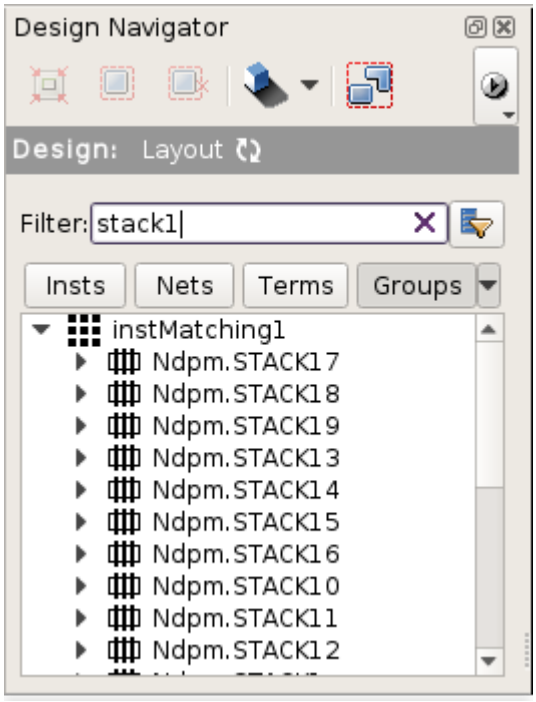
If the names of the groups you want to list share one or more unique identifying characters (for example, they all begin with the letter U), you can use the **Filter** box to enter the common characters.

Before you start, specify the filtering method to use in the **General Options** dialog box. See Setting General Options.

To display the list of figure groups that have common characters in their names:

1. With the **Groups** tab displayed in the Design Navigator, type one or more characters in the **Filter** box.

   As you start to type, the Groups tab updates and shows only the figure group names that match the filter (characters) you provided. The filter also searches lower-level group names.



2. To scroll to the matching names in lower-level figure groups:
   - Press Enter repeatedly to scroll to the next name in the list.
   - Press Shift+Enter repeatedly to scroll to the previous name in the list.

Displaying Only Selected Figure Groups

► (Optional) After Selecting and Unselecting Figure Groups, to display only the selected figure groups in the Design Navigator and hide all other groups, click **Filter Selected** .

Click **Filter Selected** again to restore the list and unhide the non-selected figure groups.

Selecting and Unselecting Figure Groups

After you display the list of figure groups, you can select one or more groups for editing operations. You can also unselect figure groups.

This topic describes the following:

- Selecting Figure Groups
- Unselecting Figure Groups

Feedback

Selecting Figure Groups

To select one or more figure groups:

1. To select all figure groups in the list, click the Design Navigator toolbar and choose **Select All**.
2. To select individual figure groups in the list:
   - To only select one group, click its name.
   - To select a contiguous sequence, Shift-select the names of the starting, and ending, groups in the sequence.
   - To select multiple individual groups, Ctrl-select each one by name.

   The Design Navigator highlights the names of your selected groups in the list.

Unselecting Figure Groups

To unselect all figure groups (regardless of how many you selected):

► On the Design Navigator toolbar, click and choose **Deselect All**.

Operating on Figure Groups

You can operate on figure groups you have listed, and selected by name, in the Design Navigator. The assistant offers actions from the **Design Navigator** toolbar, and also through menus.

To operate on figure groups:

1. Begin by Displaying the List of Figure Groups.
2. Act on the figure groups selected in the list.

   You can use the toolbar buttons. You can additionally use the menus available when you right-click selections from the list.
   - Table 65 describes actions you can perform using Design Navigator toolbar buttons.
   - Table 66 describes the Design Navigator icons that also have a menu.
   - Table 67, Table 68, and Table 69 describe the available menus.

Feedback

Table 65 Design Navigator Toolbar Buttons and their Actions

Button	Tooltip	Action
	Fit to Design Object	Fits the design viewport to the group whose name is selected in the list.
	Select in Design	For groups selected in the list, also selects them in the open design.
	Deselect in Design	Deselects currently selected groups in the open design.
	Highlight Peer Group	Highlights peers in a synchronous clone group, and their internal connectivity, applying the color you choose here, in the open design. For more information about synchronous clone groups, see Showing Synchronous Clone Groups.
	Shadow Mode	Activates, and deactivates, the shadow mode, which places surrounding design areas in shadow for easier viewing.
	Edit in Group	Activates, and deactivates, the edit-in-group feature, which descends into the selected group for editing in place. For more information about editing figures in groups, see Editing Shapes in Figure Groups in the *Custom Compiler Layout Editor User Guide for Creating and Editing Objects*.

Table 66 Design Navigator Symbols and their Menus

Icon	Description	Menu
	Figure Group	Table 67
	Synchronous Clone in a Cloned Figure Group	Table 67
	Clone Not in synchronization in a Cloned Figure Group	Table 67
	Synchronous Group	Table 68
	Instance	Table 69

Feedback

Table 67 Design Navigator Figure Group Menu

Figure Group Menu	Description
Fit, Edit in Group, Select in Design, Deselect in Design	See Table 65 for these toolbar actions.
Add to Sync Group	(Requires Layout Advanced) Adds selected figure groups to an existing or new synchronous group. If a figure group already belongs in a synchronous group, the command moves the figure group from the current synchronous group to the newly designated synchronous group.
Break Sync Clone	When synchronous groups are present, breaks synchronization for the selected clone group or groups. For more information, see Breaking Synchronization of Cloned Groups.
Sync to Peers	(Requires Layout Advanced) Within a synchronous group, make the peer figure groups look like the selected figure group. For more information, see Synchronizing Cloned Groups to Peers.
Make Sync Peers	(Requires Layout Advanced) Removes the selected figure group from its synchronized group, and then add it to a new synchronous group where it is the sole member of the group. The newly created synchronous group has the system default name, SG_n.
Ungroup	Deletes the selected group or groups. Objects in the group remain in the design.

Note:

As noted in Table 67, some of the **Figure Group** commands are available only if you choose **Tools > Enable Layout Advanced**.

Table 68 Design Navigator Synchronous Group Menu

Synchronous Group Menu	Description
Fit, Select in Design, Deselect in Design	See Table 65 for these toolbar actions.
Delete	Deletes the selected group or groups from the cloned group.

Feedback

Table 69 Design Navigator Instance Menu

Instance Menu	Description
Descend	Adjusts the hierarchy level to show the selected instance
Fit, Select in Design, Deselect in Design	See Table 65 for these toolbar actions.
Select Connected	Selects any connected islands, branches, or nets in the design
Deselect Connected	Deselects any connected islands, branches, or nets in the design
Placement Status	Shows, and allows assignment of, placement status for this instance: **None**, **Unplaced**, **Placed**, **Fixed**, or **Locked**. Where placement status varies among a set of selected instances, the tool does not indicate a status.
Delete Instances	Deletes instances from the design that are selected in the Design Navigator list. Ignores instances that are visible but not available for editing.
Select Routes to Instance	Selects all the routes (paths and vias) connected to the terminals of the selected instance

Showing Synchronous Clone Groups

You can show synchronous clone groups for an open design through the Design Navigator. Additionally, you can highlight group connectivity.

To create synchronous clone groups, see instructions on working with figure groups in the *Custom Compiler Layout Editor User Guide for Creating and Editing Objects*.

To show synchronous clone groups:

1. Begin by opening the Design Navigator.

   (From a design layout open for editing, choose **Window > Assistants > Design Navigator Assistant**).

2. In the Design Navigator with an open design containing synchronous clone groups, view the groups.

a. In the Design Navigator toolbar, click the **Groups** button.

   By default, the system shows all groups. This icon indicates a standard figure group:

b. From the **Groups** button menu (to the right of the **Groups** button), choose **Synchronous Clones**.

3. In the list of synchronous clone groups, view status.

   Synchronous clone groups appear as group icons with circling arrows:

   Clones out of synchronization with their peer group show a red **x**:

   To synchronize unsynchronized clone groups, see Synchronizing Cloned Groups to Peers.

4. Show in the design connectivity in a synchronous clone group:

   In the toolbar, click the **Highlight Peer Group** button, and choose a highlight color to apply from the drop-down menu. The color yellow has been chosen here.

   The system applies the selected color to group names in the list of synchronous clones. The system also highlights connectivity within cloned groups on the canvas, using this color.

Adding Figure Groups to Synchronous Groups

The two ways you can add a figure group to a synchronous group are:

- Using the Add to Sync Group Command
- Using the Make Sync Peers Command

Caution:

The **Add to Sync Group** command and the **Make Sync Peers** command are available only if you choose **Tools > Enable Layout Advanced**.

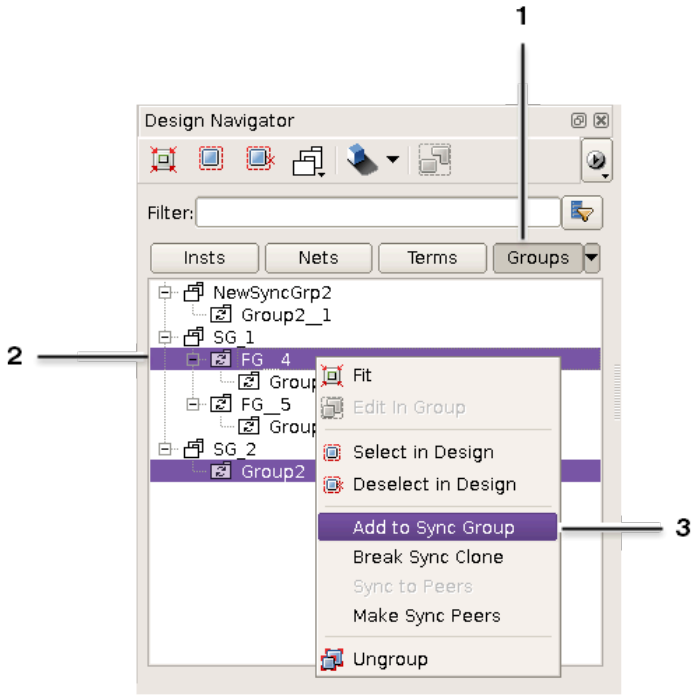
Using the Add to Sync Group Command

In the Design Navigator, you can use the **Add to Sync Group** command to add selected figure groups to an existing or new synchronous group. If a figure group already belongs in a synchronous group, the command moves the figure group from the current synchronous group to the newly designated synchronous group.

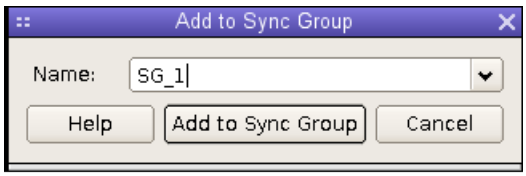
Feedback

To add figure groups to a synchronous group:

1. In the Design Navigator, display the list of figure groups. For instructions, see Displaying the List of Figure Groups.
2. Select the figure groups.
3. Right-click the selected groups, and then choose **Add to Sync Group**.



The **Add to Sync Group** dialog box opens.



4. Do one of the following:
   - In the **Name** list, select an existing synchronous group.
   - In the **Name** box, type a name for the synchronous group you want to create.
5. Click **Add to Sync Group**. The Design Navigator Groups tab refreshes and displays the new groupings.

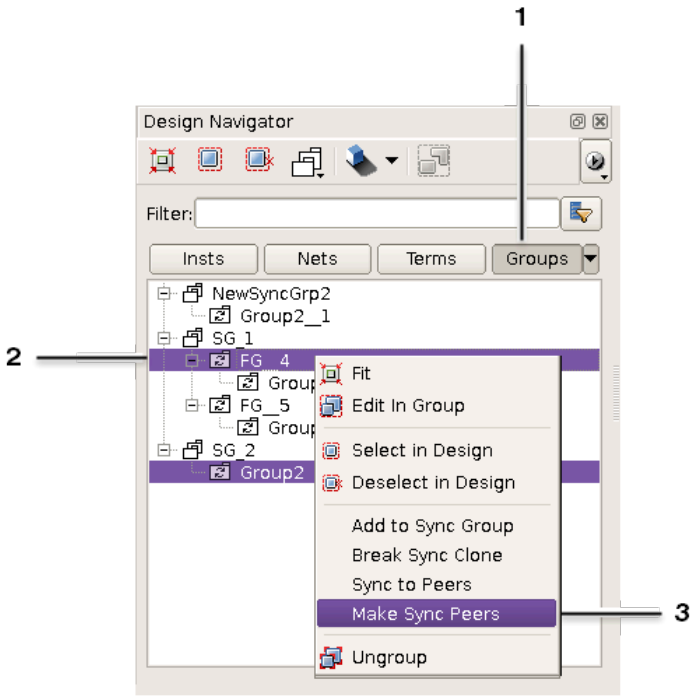
Feedback

Using the Make Sync Peers Command

In the Design Navigator, you can use the **Make Sync Peers** command to break selected figure groups from an existing synchronous group and add them to a new synchronous group. The new synchronous group is assigned a default name.

To use the **Make Sync Peers** command:

1. In the Design Navigator, display the list of figure groups. For instructions, see Displaying the List of Figure Groups.
2. Select the figure groups.
3. Right-click the selected groups, and then choose **Make Sync Peers**.



The selected figure groups are moved to the newly created synchronization group, which appears in the Groups tab with a system-assigned default name.

Breaking Synchronization of Cloned Groups

You can break figure group synchronization manually, or you can break the synchronization through editing of synchronized clone elements when you transfer the design to another tool, and back again.

You can break synchronization of cloned groups when you want to work with their figure groups individually.

To break synchronization in synchronous cloned groups:

1. In the Design Navigator, display the list of figure groups. For instructions, see Displaying the List of Figure Groups.
2. Display the list of synchronous clone groups. To do this:
   a. In the **Design Navigator** toolbar, click **Groups**.
   b. In the menu that appears, choose **Synchronous Clones**.
3. Select the name of the cloned group whose synchronization you want to break, right-click, and choose **Break Sync Clone**.

   You can now edit this group without automatically applying the edits to all clones of this group.

   To synchronize the groups again, see Synchronizing Cloned Groups to Peers.

Synchronizing Cloned Groups to Peers

You can synchronize cloned groups to their peers again after breaking the synchronization. This option is only available for figure groups that have been created by cloning.

To synchronize cloned groups to peers:

1. In the Design Navigator, display the list of figure groups. For instructions, see Displaying the List of Figure Groups.
2. Locate the cloned group that is no longer synchronized ( ).
3. Select the cloned group name, right-click, and choose **Sync to Peers**.

   The system synchronizes this group to its cloned peers.

   You can now edit this group, and automatically apply the edits to all clones of this group.

Device Palette Assistant

You can use the Device Palette assistant to drag-and-drop commonly placed devices onto the design canvas. You can create, and categorize, sets of devices on the Device Palette. (Devices that do not require added parameters can be used). You can place these device instances directly from the palette to the design canvas.

Feedback

These sections cover:

- Displaying the Device Palette Assistant
- Adding Favorite Devices to the Device Palette
- Placing Devices from the Palette
- Adjusting Device Palette Contents

Displaying the Device Palette Assistant

To display the Device Palette assistant:

► In a layout or schematic view, choose **Window > Assistants > Device Palette**.

The assistant docks along the window edge. Undock it by clicking, and dragging it.

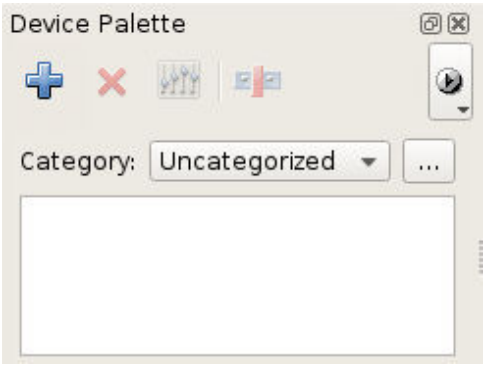
Adding Favorite Devices to the Device Palette

You can add favorite devices to the palette to create a collection of devices and make them available directly from the palette.

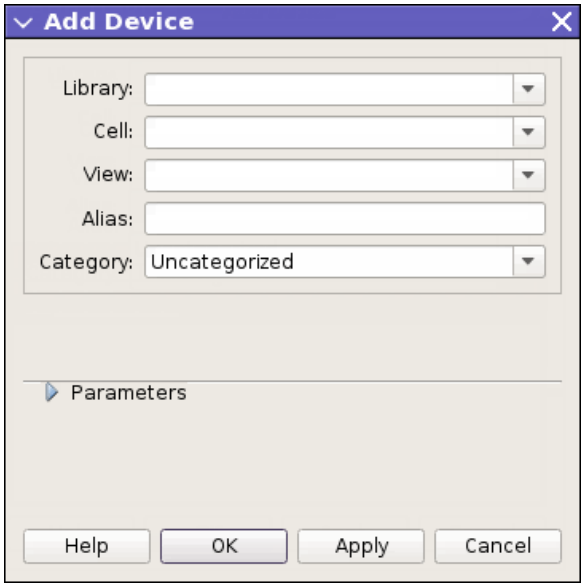
Use the Device Palette assistant to add favorite devices to the palette.

To add a favorite device to the palette:

1. Begin by opening the Device Palette assistant. In a layout or schematic view, choose **Window > Assistants > Device Palette**.



2. On the device palette, click the **+** button on the toolbar. The **Add Device** dialog box appears.
3. Specify or browse to find the library, cell, and view.



The palette allows referencing of a master of any view type from either the layout or schematic editor. However, device favorites you add here appear in the device palette for this editor only.

4. (Optional) Accept the suggested alias, or enter a new alias.

   Aliases provide an additional level of classification, after categories, for the devices in the palette.

5. From the list, choose a category or enter a new category for the palette to offer.

   A new category you create here will include this favorite device.

6. (Optional) Parameters of the device is collapsed at startup. You can click the button to expand the Parameters table.

7. Click **OK** to close the **Add Device** dialog box.

Editing Favorite Devices in the Device Palette

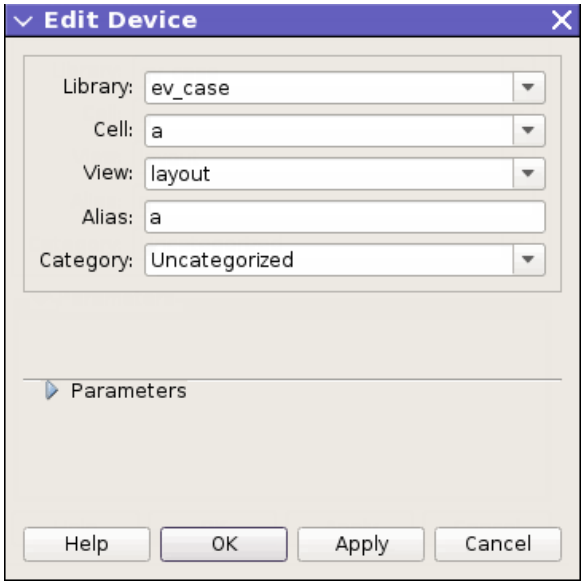
You can edit favorite devices in the palette.

To edit a favorite device to the palette:

1. Begin by opening the Device Palette assistant. In a layout or schematic view, choose **Window > Assistants > Device Palette**.

2. On the device palette, click the button on the toolbar. The **Edit Device** dialog box appears.

3. Specify or browse to change the library, cell, and view as needed.



4. (Optional) Parameters of the device is collapsed at startup. You can click the button to expand the Parameters table.
5. Click **OK** to close the **Edit Device** dialog box.

Placing Devices from the Palette

You can place a device from the device palette to quickly repeat the instance creation process for favorite devices. Use the Device Palette assistant to place commonly used devices in a design.

To place a device from the palette:

1. Begin by opening the Device Palette assistant. In a layout or schematic view, choose **Window > Assistants > Device Palette**.
2. From the **Device Palette** toolbar, choose the category.
3. From the palette list, use one of the three methods.

Feedback

- Select the device by name, and drag it onto the design canvas.
- Double-click the device name, fine tune instance creation options, and place the instance.
- Select the device by name, click **Create Instance**, fine tune instance creation options, and place the instance.

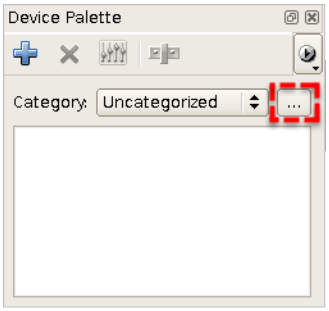
When the palette places a device instance, it also adds it to the **Recently Placed** category for additional quick access.

Adjusting Device Palette Contents

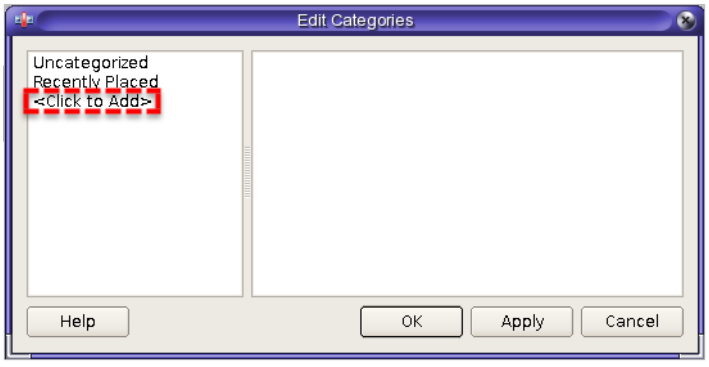
You can adjust device palette contents by adding or deleting categories, deleting device favorites, and copying device favorites to new categories.

To adjust device palette contents:

1. Begin by opening the Device Palette assistant. In a layout or schematic view, choose **Window > Assistants > Device Palette**.
2. To the right of the Category list box, click the **Browse** button (...).



- Add a category by choosing **<Click to Add>** and entering the category name.



- Delete a category or device favorite (by its alias) by selecting it and clicking the red **x**.

  The **Recently Placed** and **Uncategorized** categories cannot be deleted. However, you can delete device favorites from any category where they are listed.

- Copy a favorite (by its alias) to another category by selecting it by name, and dragging it to the destination category.

3. Click **OK**.

Error Viewer Assistant

You can use the Error Viewer assistant to show and browse the resulting violations from physical verification tools such as DRD on the canvas.

The Error Viewer assistant lists all violations in the marker tree and marker table pane, and shows the detailed information for each marker in the marker detail pane.

These sections cover:

- Opening the Error Viewer Assistant
- Using the Error Viewer Assistant in the Schematic Editor
- Managing the Marker Tree
- Viewing Marker Categories
- Viewing Error Markers
- Viewing Error Markers Hierarchically
- Manipulating Error Markers
- Sorting Error Markers
- Deleting Error Markers
- Customizing Marker Information to Display
- Showing Tooltips of Markers
- Customizing the Error Viewer Layout Style
- Viewing Markers in Shadow
- Managing Highlight Colors
- Highlighting Markers When Selecting

- Highlighting With an X Symbol
- Setting Zoom Mode after Highlighting
- Setting Highlight Actions
- Specifying the Maximum Number of Markers to Display
- Clearing All Highlights When Closing Error Viewer
- Selecting First Item of Marker Table
- Setting Filter Type for Categories
- Loading Calibre Result
- Loading RedHawk Result
- Loading Generic Format File
- Exporting to a Text File
- Exporting Waived List

Opening the Error Viewer Assistant

Open the Error Viewer assistant to work with design violations (represented by oaMarker objects).

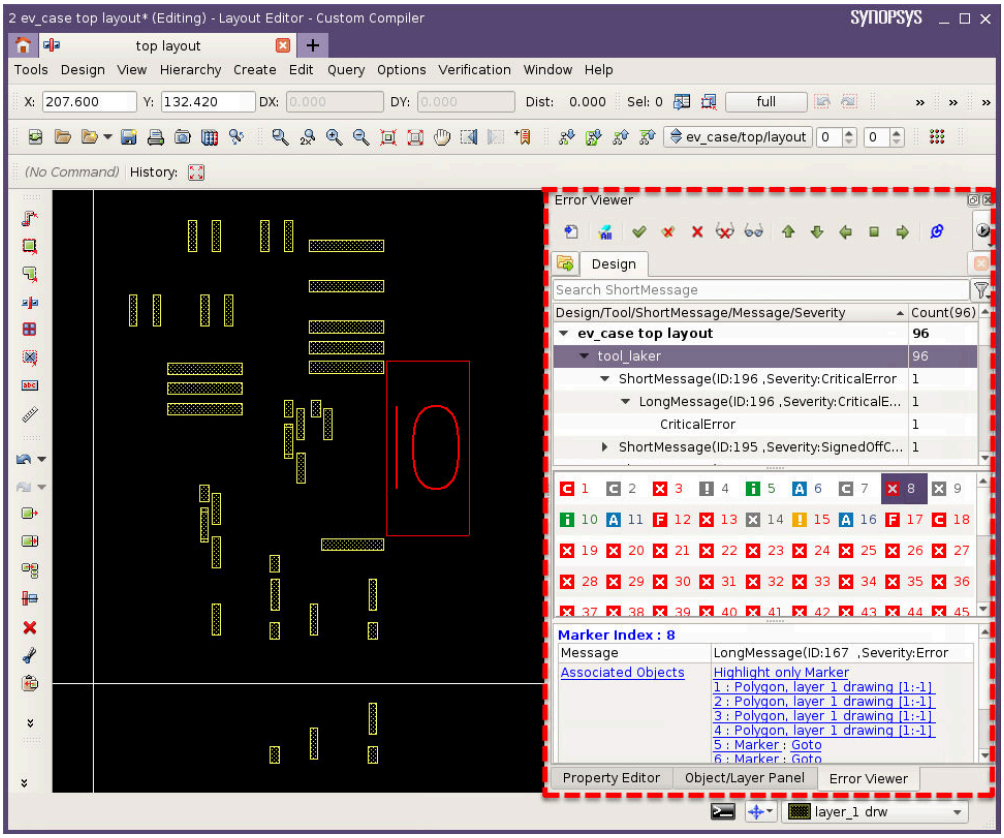
To open the Error Viewer assistant, use one of these methods:

- In a design, choose **Window > Assistants > Error Viewer**.
- Use the shortcut key Shift+E that is bound in the Laker binding set. For information about binding set selections, see Customizing Keyboard Shortcuts.

The Error Viewer assistant pane appears on the right side of the window.

Feedback

Figure 40 Error Viewer Assistant



You can undock the Error Viewer assistant pane using one of these methods:

- Click and drag it to a required location.
- Click **Dock** (next to **Close**).

When redocking the floating pane to the window, you can use one of the following methods:

- Click and drag it to the design window, or use the docking button.
- Click **Pin Dock Widget** (off by default) on the toolbar to only allow docking from the docking button.

Using the Error Viewer Assistant in the Schematic Editor

Starting from W-2024.09 release, the Error Viewer assistant is the default error viewer in the Schematic Editor. Which means, the default of the `seUseErrorViewer` preference is `true`.

Feedback

To run ERC or SRC checks on a read-only schematic design using the Error Viewer assistant to view the error markers, perform the following steps.

1. Disable the **Create Violation Markers in Designs** option in the Schematic Editor **Check Options** dialog box.
2. Choose **Window > Assistants > Error Viewer** to open the assistant and use the features described in the following sections.

To use Marker Browser Assistant as the error viewer in the Schematic Editor, set the `seUseErrorViewer` preference as `false` by typing `db::setPrefValue seUseErrorViewer -value false` in the Console.

Managing the Marker Tree

The marker tree shows marker categories along with the number of markers and the color used for categories and markers.

The marker tree consists of the following columns: Category, Count, and Color.

- In the Category column, markers can be classified by the following categories:

Category	Description
Design	Shows the name of the opened design. When the **Hierarchy** option is enabled, the design name in bold represents the current opened design while the design name in plain represents the design at other hierarchy levels. See Viewing Error Markers Hierarchically for more details.
Tool	Shows the tool that verifies the violations. This information is derived from the oaMarker attributes.
Severity	Shows the severity type of violations. This information is derived from the oaMarker attributes. Severity types are fixed values as defined by OpenAccess: ◦ Error ◦ Warning ◦ Signed Off Error ◦ Information ◦ Annotation ◦ Fatal Error ◦ Critical Error ◦ Acknowledge Warning ◦ Signed Off Critical Error
ShortMessage	Shows the short message of markers. The short message is set as the rule ID, for example: m1.s.1. If the rule is converted from a constraint, the constraint ID is used.

Feedback

Category	Description
Message	Shows the long message of markers. The long message is set as the description of the rule set. For example, the minimum spacing rule description can be *Minimum metal space in MINP direction (horizontal)*. If such information is not found, the tool composes a name from the original content of the rule or constraint. If the violation marker relates to spacing or width, the Error Viewer shows an actual distance and a needed distance at the beginning of the message field, as shown in the following figure, to ease the effort required for violation fixing.

- The Count column shows the number of markers in the corresponding category. The Error Viewer assistant shows the Count column by default. The Error Viewer displays the column title as `Count(%d)` where `%d` indicates the number of markers in the corresponding category. This column is not resizable.

  For more information on how to show the column, see Manipulating Error Markers.

- The Color column shows the mapping highlight color for each category. The Error Viewer assistant hides the Color column by default. This column is not resizable.

  This column is available only when the **By Category** option is enabled for highlight colors in the **Error Viewer Options** dialog box.

  For information on how to assign colors to highlighted categories or markers, see Managing Highlight Colors.

  For more information on how to show the column, see Manipulating Error Markers.

These sections cover:

- Displaying Category Levels in the Marker Tree
- Configuring the Marker Tree Contents
- Wildcard Matching in the Display Filter

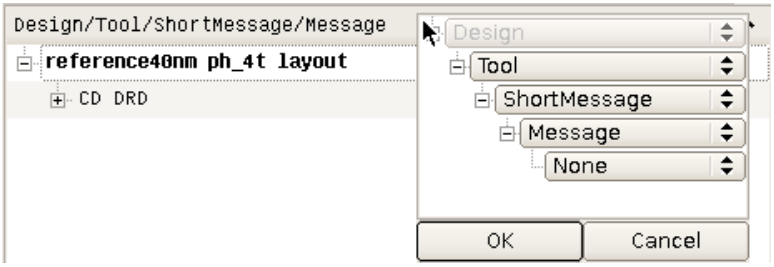
Displaying Category Levels in the Marker Tree

By default, markers are classified in the Category column by the following sequence of display levels: Design, Tool, ShortMessage, and Message.

Feedback

To change the display level of categories:

1. Begin by opening the Error Viewer assistant. In a design, choose **Window > Assistants > Error Viewer**.
2. From the Category column, right-click the column header to show the category menu.



3. Click the drop-down list to the right of the categories to change the selected display levels.

   Click **OK** to commit the change. The Category column is refreshed and the marker tree is collapsed.

   The category level you defined in the marker tree affects the **Filter Display** dialog box contents. See Configuring the Marker Tree Contents for more information.

4. (Optional) Choose the arrow buttons next to the marker category to expand the current node's subtree and see the underlying markers in the category.

   Double-click the marker category to expand or collapse specific levels.

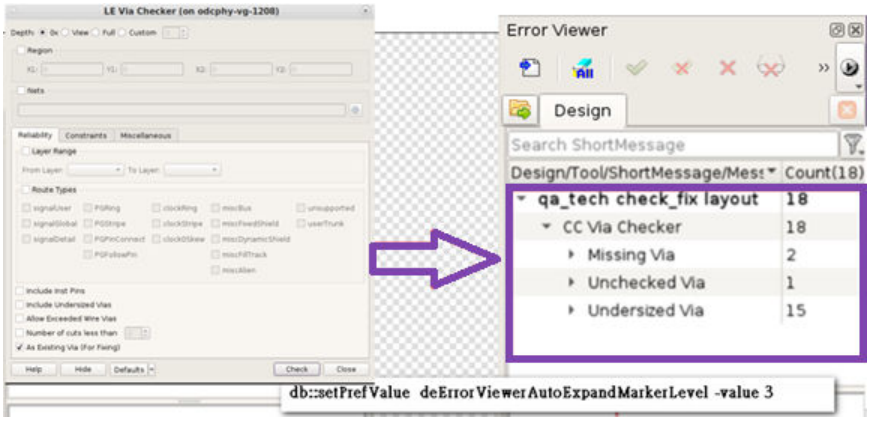
   For more information on showing or hiding category levels in the marker tree, see Manipulating Error Markers.

Automatically Expanding Marker Levels in Error Viewer

The **Error Viewer** in Custom Compiler provides a centralized interface for reviewing and managing error markers generated by various tools, such as Via Checker. Automatically expanding marker levels after loading error data streamlines the review process, allowing you to quickly access relevant error categories without manual navigation.

You can control the default expansion level of the marker tree in the **Error Viewer** using the `deErrorViewerAutoExpandMarkerLevel` preference. This setting determines how many levels of the marker tree are expanded automatically when error markers are loaded. The default is 2.

Feedback



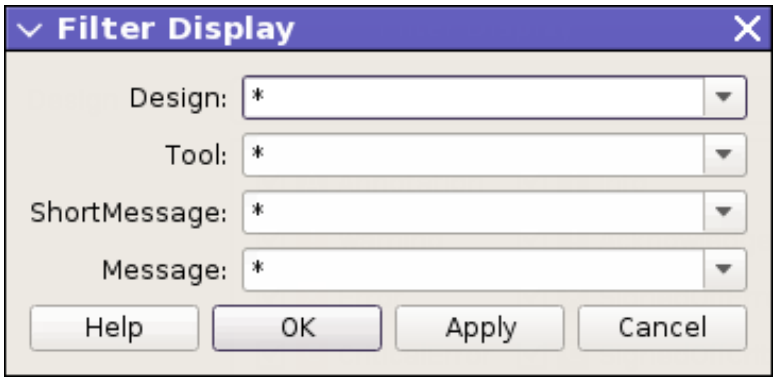
Configuring the Marker Tree Contents

You can focus on the data that you want to focus on by using a display filter.

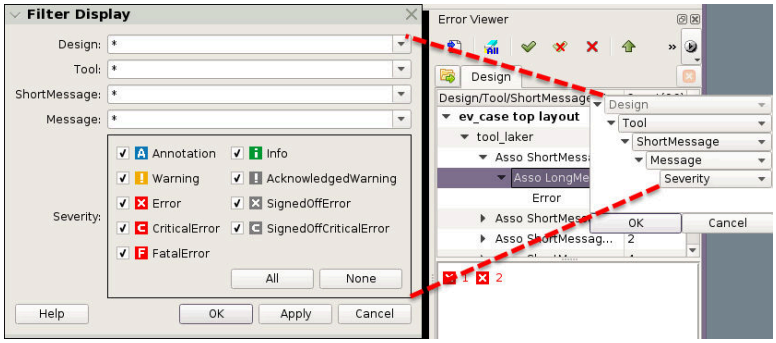
To configure the marker tree contents with single or multiple categories:

1. Begin by opening the Error Viewer assistant. In a design, choose **Window > Assistants > Error Viewer**.
2. Click the toolbar menu and choose **Options**. In the **Error Viewer Options** dialog box, set **Filter Type** as **Multiple Category**, and then click **OK**. See Setting Filter Type for Categories for more information.
3. Click the toolbar menu and choose **Filter Display**.

   The **Filter Display** dialog box appears.

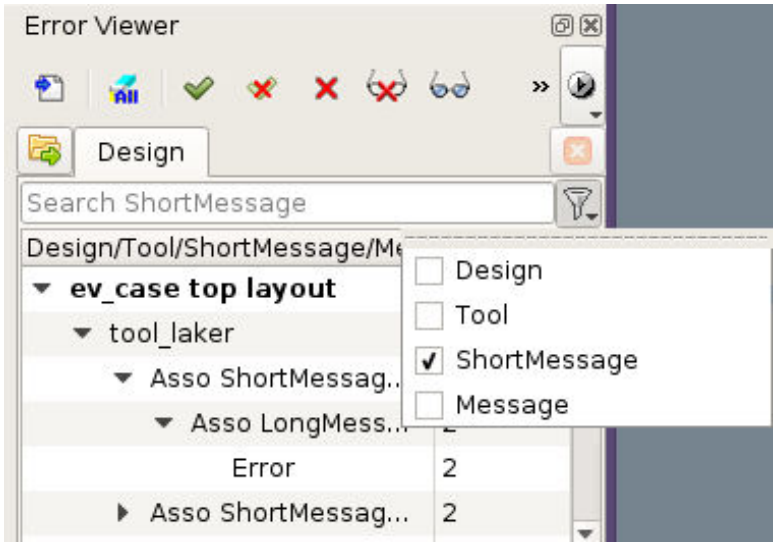
   Contents of the **Filter Display** dialog box vary when you change the category level in the marker tree, as shown in the following figure. See Displaying Category Levels in the Marker Tree for more information.



By default, the tool uses a wildcard character * to show all of the violations in the marker tree.

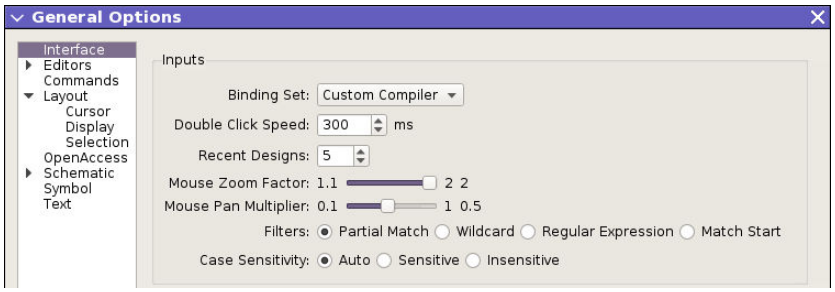
4. (Optional) Click **None** to deselect all severity types. Choose any of the severity types you want to display.
5. Click **OK** to commit the change. The marker tree is refreshed and collapsed.
6. Click the toolbar menu and choose **Options**. In the **Error Viewer Options** dialog box, set **Filter Type** as **Single Category**, and then click **OK**. See Setting Filter Type for Categories for more information.

   Now you can only filter violations from one of the available categories: Design, Tool, ShortMessage (default), and Message.



Feedback

You may enter the required filter string in the filter text box, which corresponds to **Filters** and **Case Sensitivity** settings in the **General Options** dialog box. See Setting General Options for more information.



Wildcard Matching in the Display Filter

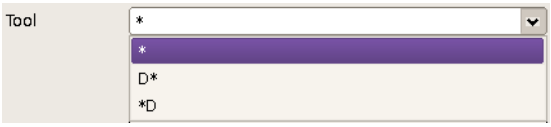
You can use the wildcard character * when typing a filter string in the text box, for example: G*. Click **Apply** to refresh the contents of the marker tree.

Characters	Functions	Examples
`*`	Matches any sequence of zero or more characters.	`t*t` matches `test` or `tonight` but not `tactic`.
`?`	Matches any single character. This is the same as "." in full regular expression (regrexps).	`m?n` matches `man`, `men` or `min` but not `moon`.
`[CHARS]`	Matches any character in square brackets. If `CHARS` contains a sequence of the form a-b, any character between a and b, inclusive, may match.	`*a[ch]` matches all strings that end with either `ac` or `ah`.
`\`	Use the backslash character to escape a single character.	If string var equals to `var="abcd\ef*g?h"` • `*\?*` matches string that contains `?`, so it matches `var`. • `*[\]*` matches string that contains `\`, so it matches `var`. • `*[12]*` matches string that contain `1` or `2`, so it does not match `var`.

The tool saves the filter history (Figure 41) so that you can easily choose from previously used filters.

Feedback

Figure 41 Filter History



Viewing Marker Categories

You can view and highlight the categories of markers in the marker tree and the design.

To view and highlight the categories of markers:

1. Begin by opening the Error Viewer assistant. In a design, choose **Window > Assistants > Error Viewer**.
2. From the marker tree, choose the design and click **Highlight Previous Category** and **Highlight Next Category** on the toolbar to navigate to the previous or next marker category shown in the marker tree and also highlight the associated objects on the canvas.
3. (Optional) Click **Clear Highlight** to clear highlights of markers and their associated objects at the current design level.

Viewing Error Markers

You can view and highlight error markers in the marker tree and the design, and also show more detailed messages associated with the highlighted markers.

To view and highlight error markers:

1. Begin by opening the Error Viewer assistant. In a design, choose **Window > Assistants > Error Viewer**.
2. From the marker tree, choose the design or any severity type of a category (for example: CriticalError).

   (Optional) From the marker table pane, choose a marker of the selected design or category.
3. Click **Highlight Previous Marker** and **Highlight Next Marker** on the toolbar to navigate to the previous or next marker relative to the selected category and highlight the marker and its associated object on the canvas.
4. (Optional) Click **Highlight Selected Marker** to highlight the selected marker and its associated object on the canvas. You can find the longer message of the highlighted marker in the lower region of the Error Viewer.

Feedback

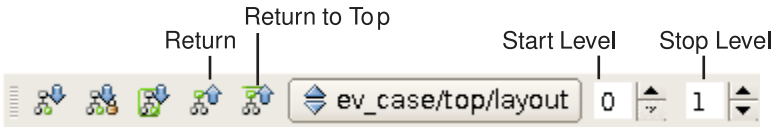
5. (Optional) To show more than one marker or a marker category, use Ctrl+click to select each marker individually.
6. (Optional) Click **Clear All Highlight** to clear highlights of markers and their associated objects at the current design level.

Viewing Error Markers Hierarchically

By default (hierarchy disabled), the marker tree displays only the markers in the current edit design. When the **Hierarchy** option is on, you can view all markers from the top design hierarchy to the hierarchy that are visible down to the view depth.

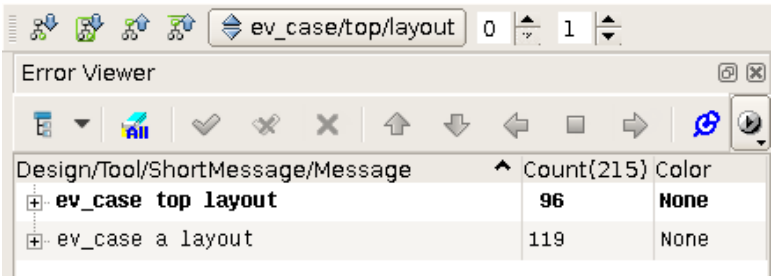
To view error markers from the top design hierarchy to down:

1. Begin by opening the Error Viewer assistant. In a design, choose **Window > Assistants > Error Viewer**.
2. Click the toolbar menu and choose **Hierarchy**. For more information about toolbar menu options, see Manipulating Error Markers.
3. Set the stop level on the **Hierarchy** toolbar to 1.



You can specify the start and stop levels from the **Design Options** dialog box (see Setting Layout Design Options) or from the **Start Level** and **Stop Level** boxes on the Hierarchy toolbar.

The Error Viewer updates the marker tree and shows the design name of the current opened design in bold text to distinguish it from designs (shown in plain text) at other hierarchy levels.



You can use the options available when you right-click selections from the list.

Feedback

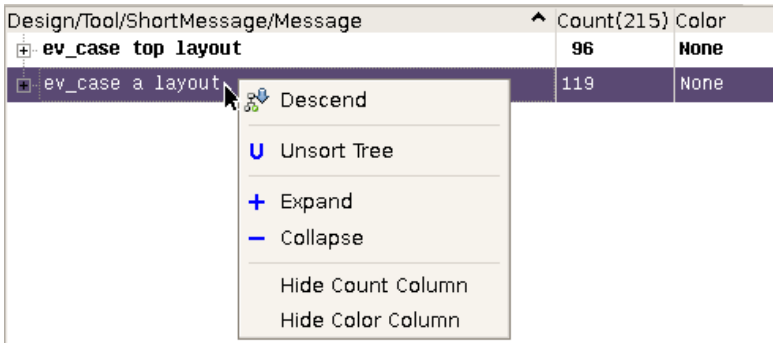


Table 70 describes the Error Viewer options menu in the marker tree for descending design levels.

Table 70 Error Viewer Options Menu in Marker Tree for Descending Design Levels

Error Viewer Menu	Description
Descend	Moves down to a lower hierarchy level to let you view and highlight the markers at the visible depth.
Unsort Tree	Restores the marker tree order to its initial state.
Collapse	Collapses the selected categories.
Expand	Expands the selected categories.
Hide Count Column	Hides the Count column from the marker tree.
Hide Color Column	Hides the Color column from the marker tree.

4. Right-click the design shown in plain text and choose **Descend**.

   The Error Viewer updates the marker tree and shows the design name in bold.

   Pushing and popping through the hierarchies causes the display of the marker tree to update, always showing the markers down to the visible depth.

5. Act on the markers selected in the marker tree or the marker table pane. For more information, see Manipulating Error Markers.

6. Click the **Return** button on the **Hierarchy** toolbar to return to the upper level of the design hierarchy.

   (Optional) Click the **Return to Top** button on the **Hierarchy** toolbar to return to the top level.

Feedback

Manipulating Error Markers

You can manipulate error markers you have selected in the Error Viewer. The Error Viewer offers actions from the Error Viewer toolbar buttons and toolbar menu, and also through the options menu (the right-click selections).

To manipulate error markers in the marker tree:

1. Begin by opening the Error Viewer assistant. In a design, choose **Window > Assistants > Error Viewer**.
2. Act on the error markers selected in the marker tree.

   You can use the toolbar buttons and toolbar menu. You can additionally use the options available when you right-click selections from the list.

   - Table 71 describes actions you can perform using the Error Viewer toolbar buttons.

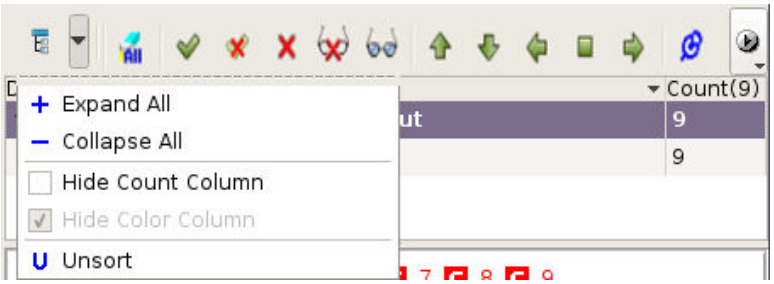


Table 71 Error Viewer Toolbar Buttons and their Actions

Button	Tooltip	Action
	Tree View	▪ Configures the marker tree with these options: ▪ **Expand All**: Expands all levels of the marker tree to view detailed information. ▪ **Collapse All**: Closes all levels of the marker tree to hide all information. ▪ **Hide Count Column**: Hides the Count column. ▪ **Hide Color Column**: Hides the Color column. ▪ **Unsort**: Restores the marker tree order to its initial state. See Sorting Error Markers.
	Clear All Highlights	Clears all highlights at the current design level.
	Sign Off Marker	Changes the severity type of selected markers to SignedOffError. This only works for severity types Error, Critical Error, and Warning. This button is disabled if the current design is read-only.
	Unsign Off Marker	Reverses the Sign Off Marker action.

Feedback

Button	Tooltip	Action
	Delete Marker	Removes the selected markers from the design database and also removes the highlights. The marker tree is refreshed after deleting the markers on the canvas and the marker tree. This button is disabled if the current design is read-only.
	Hide	Hides selected markers. This button is only supported if markers exist in edit design and does not support overlay view. Hierarchy mode is not supported.
	Show Selected Only	Only displays selected markers. This button is only supported if markers exist in edit design and does not support overlay view. Hierarchy mode is not supported.
	Highlight Previous Category	Navigates to the previous marker category shown in the marker tree and also highlights the associated objects on the canvas. See Viewing Error Markers. For more information about setting the highlight actions of buttons, see Setting Highlight Actions.
	Highlight Next Category	Navigates to the next marker category shown in the marker tree and also highlights the associated objects on the canvas. See Viewing Error Markers. For more information about setting the highlight actions of buttons, see Setting Highlight Actions.
	Highlight Previous Marker	Navigates to the previous marker relative to the selected category and highlight the associated object on the canvas. See Viewing Error Markers. For more information about setting the highlight actions of buttons, see Setting Highlight Actions.
	Highlight Selected Marker	Highlights the selected markers and their associated objects on the canvas. See Viewing Error Markers. For more information about setting the highlight actions of buttons, see Setting Highlight Actions.
	Highlight Next Marker	Navigates to the next marker relative to the selected category and highlight the associated object on the canvas. See Viewing Error Markers. For more information about setting the highlight actions of buttons, see Setting Highlight Actions.
	Pin Dock Widget	Only allows docking from this button. (Off by default).

Feedback

- Table 72 describes actions you can perform using the Error Viewer toolbar menu .

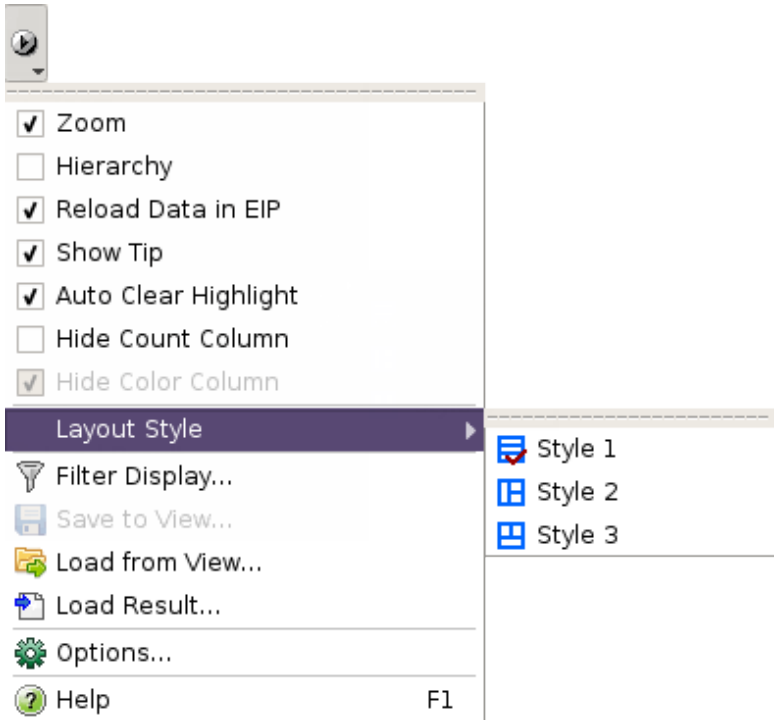


Table 72 Error Viewer Toolbar Menu Options

Error Viewer Menu Option	Description
Zoom	Zooms to each marker and fits it to the viewport when highlighted, regardless of whether they are currently visible on the canvas. (On by default). You can turn off the visibility of all markers from the Object/Layer Panel and then filter and highlight only the markers you want to see on the canvas. Tcl preference: `deErrorViewerZoom`

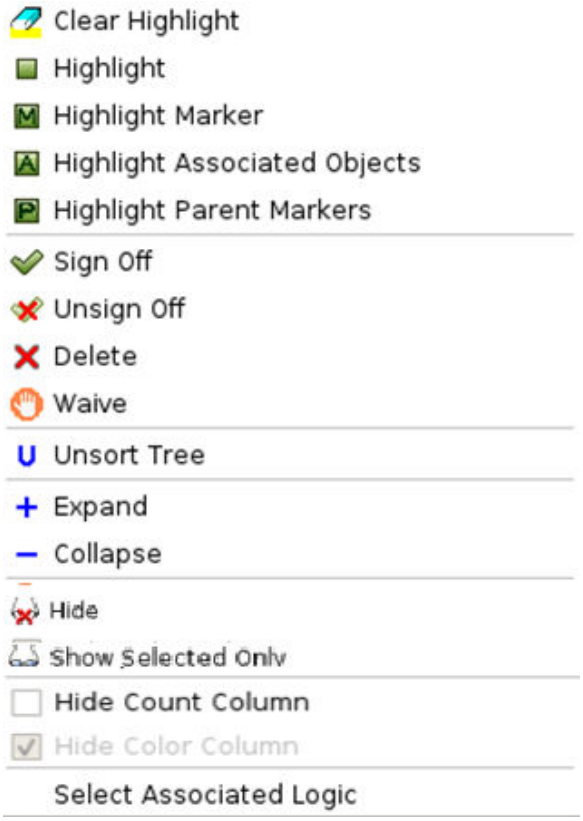
Feedback

Error Viewer Menu Option	Description
Hierarchy	By default (hierarchy disabled), the marker tree displays only the markers in the current edit design. When the **Hierarchy** option is on, the marker tree lists all markers from the top design's hierarchy to the hierarchies that are visible down to the view depth. (Off by default). For more information about viewing and highlighting markers at different hierarchy levels, see Viewing Error Markers Hierarchically. Tcl preference: `deErrorViewerHierarchy`
Reload Data in EIP	When enabled (default), refreshes data after changing the context to Edit-in-Place (EIP). When disabled, if you change the context to EIP, the Design tab is change, as shown in the following figure; the data is not reloaded and it is the data of the upper context. Tcl preference: `deErrorViewerReloadEIP`
Show Tip	Shows a tooltip containing the tool information, the severity type, and the short and long messages when the pointer hovers over a marker on the canvas. See Showing Tooltips of Markers. Tcl preference: `deErrorViewerShowTip`
Auto Clear Highlight	Automatically clears the highlight of the last marker and its associated objects on the canvas before the new highlight action is triggered. (On by default). Tcl preference: `deErrorViewerAutoClearHighlight`
Hide Count Column	Hides the Result column from the marker tree.
Hide Color Column	Hides the Color column from the marker tree.
Layout Style	Applies a layout style (**Style 1**, **Style 2**, or **Style 3**) to the assistant. See Customizing the Error Viewer Layout Style.
Filter Display	Configures the marker tree contents by filtering the display level of categories. See Configuring the Marker Tree Contents.
Save to View	Saves an overlay design to view.
Load from View	Loads an overlay design from view.

Feedback

Error Viewer Menu Option	Description
Load Result	Loads the third-party result file in Calibre ASCII format.
Options	Configures the Error Viewer options. See the following for more information: ▪ Managing Highlight Colors ▪ Viewing Markers in Shadow ▪ Highlighting Markers When Selecting ▪ Highlighting With an X Symbol ▪ Setting Zoom Mode after Highlighting ▪ Setting Highlight Actions ▪ Specifying the Maximum Number of Markers to Display ▪ Clearing All Highlights When Closing Error Viewer ▪ Selecting First Item of Marker Table ▪ Setting Filter Type for Categories

◦ Table 73 describes the Error Viewer options menu in the marker tree.

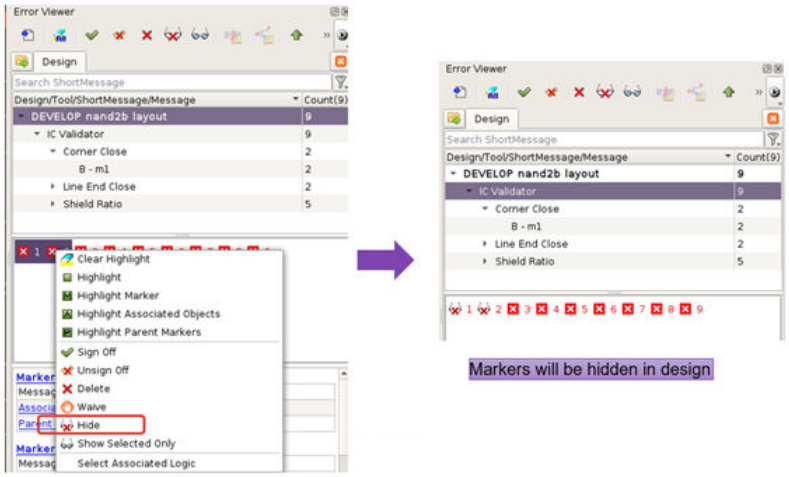


Feedback

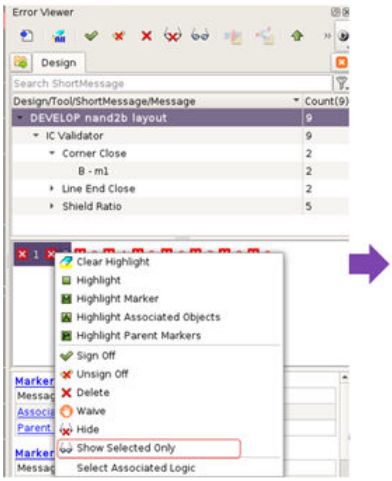
Table 73 Error Viewer Options Menu in the Marker Tree

Error Viewer Menu	Description
Clear Highlight	Clears the highlight of the selected marker and the highlight of its associated objects.
Highlight	Highlights all markers in the selected categories on the canvas.
Highlight Marker	Only highlights the selected markers on the canvas.
Highlight Associated Objects	Only highlights the associated objects of the selected markers on the canvas.
Highlight Fixing Guides	Only highlights the selected fixing guides on the canvas. This appears only when the markers are from DRD.
Highlight Parent Markers	Only highlights the parent markers of the selected markers on the canvas.
Sign Off	Changes the severity type of highlighted markers to SignedOffError. This only works for severity types Error, Critical Error, and Warning. This button is disabled if the current design is read-only.
Unsign Off	Reverses the Sign Off Marker action.
Delete	Removes selected markers from the marker tree and the design. The marker tree is refreshed after deletion. This button is disabled if the current design is read-only.
Perform Via Checker Fix	Performs via checker fix on a design with existing constraints violations. This button is disabled in the context-sensitive menu if at least one supported marker is not selected.
Waive	Marks waive rules for the selected markers. When you mark **Waive** for a selected marker, Error Viewer assigns a boolean property and moves the markers to the **Waived_List** tab. **Note:** There is only one **Waived_List** tab for every design.

Feedback

Error Viewer Menu	Description
Unwaive	Unmarks waive rules for the selected markers. When you right-click a marker and click **Unwaive**, Error Viewer moves the marker back to the original tab from the **Waived_List** tab. **Note:** ▪ The **Waived_List** tab is not deleted even when there is no marker added. ▪ If you remove any tab, all the corresponding markers are also removed from the **Waived_List** tab. Tcl preference: `deErrorViewerDeleteWaiveTogether` (default: true)
Unsort Tree	Restores the marker tree order to its initial state.
Collapse	Collapses the selected categories.
Expand	Expands the selected categories.
Hide	Hides selected markers in the table and in the design. For example, see the following figure 

Feedback

Error Viewer Menu	Description
Show Selected Only	Shows only the selected markers. This is only supported if markers exist in the edited design. 
Hide Count Column	Hides the Count column from the marker tree.
Hide Color Column	Hides the Color column from the marker tree.
Select Associated Logic	When using the Error Viewer in the Layout Editor, you can right-click a violation in the marker tree or marker table and choose this command to highlight and cross-select associated objects. For example, if you select a violation that is associated with a net, the net is also selected in the Design Navigator. If you select a violation that is associated with an instance, the instance is also selected in the Design Navigator. The cross-selection feature honors the global **Shadow** mode setting.

- Table 74 describes the Error Viewer options menu in the marker table pane.

Table 74 Error Viewer Options Menu in the Marker Table Pane

Error Viewer Menu	Description
Clear Highlight	Clears the highlight of the selected marker and the highlight of its associated objects.
Highlight	Highlights the selected marker and its associated objects on the canvas.
Highlight Marker	Only highlights the selected marker on the canvas.
Highlight Associated Objects	Only highlights the associated objects of the selected marker on the canvas.
Highlight Fixing Guides	Only highlights the selected fixing guides on the canvas. This appears only when the markers are from DRD checking.
Highlight Parent Markers	Only highlights the parent markers of the selected markers on the canvas.
Sign Off	Changes the severity type of highlighted markers to SignedOffError. This only works for severity types Error, Critical Error, and Warning. This button is disabled if the current design is read-only.
Unsign Off	Reverses the Sign Off Marker action.

Feedback

Error Viewer Menu	Description
Delete	Removes the selected marker from the marker tree and the design. The marker tree is refreshed after deletion. See Deleting Error Markers. This button is disabled if the current design is read-only.
Waive	Marks waive rules for the selected markers.

3. Act on the marker detail pane.

   The marker detail pane appears at the lower part of the Error Viewer assistant showing the detailed information of selected markers (up to 20 markers per page).

Feedback

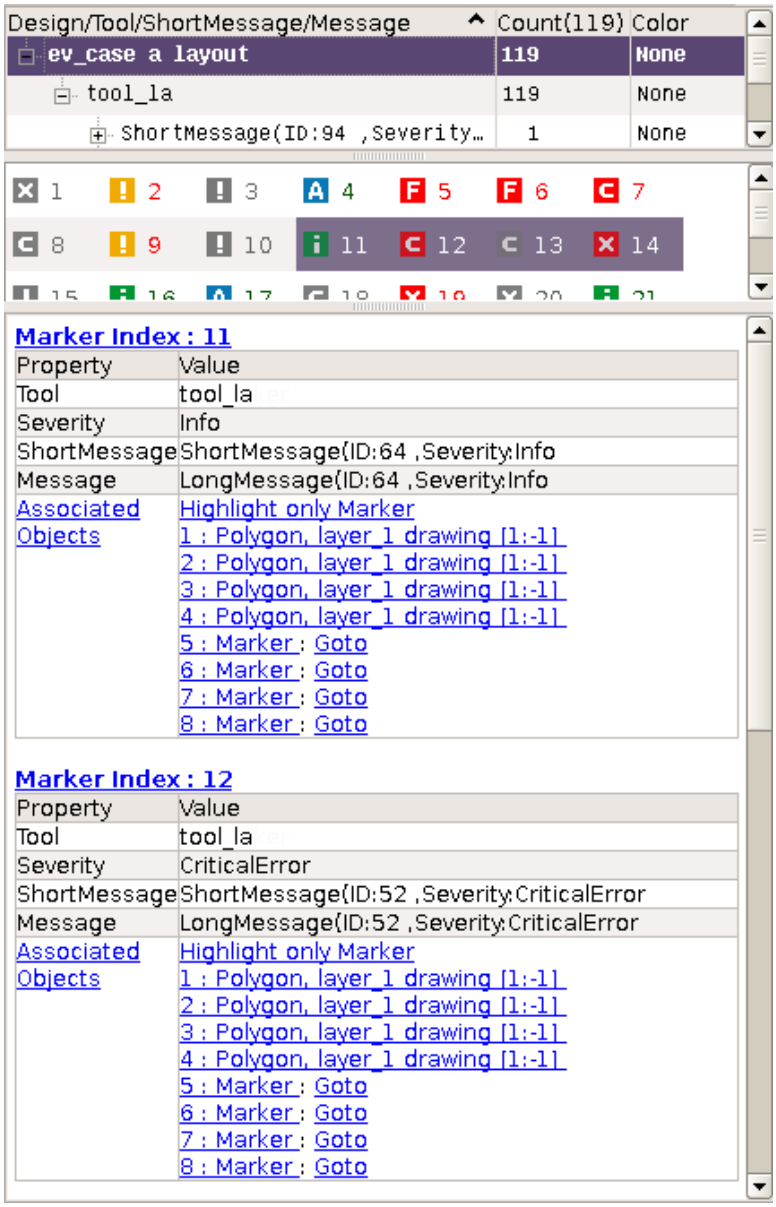


Table 75 describes the information of selected markers.

Feedback

Table 75 Marker Detail Pane

Information	Description
Marker Index	Shows the index number of selected markers. Click this link to highlight the selected marker and its associated objects on the canvas. This behaves the same as **Highlight** in the marker table pane.
Property and value	Shows the properties and values of the selected marker by tool, severity type, short message, and long message.
Associated Objects	Shows individual objects along with net information (if any) and links to highlight them on the canvas. Click the **Associated Objects** link to highlight only the associated objects of the selected marker on the canvas. This behaves the same as **Highlight Associated Objects** in the marker table pane. Click the **Highlight only Marker** link to highlight only the selected markers on the canvas. This behaves the same as **Highlight Marker** in the marker table pane. Click the link of each individual object to highlight the associated object of the marker on the canvas. **Note: Associated Objects** are not available when using the Error Viewer on an overlay design in the Schematic Editor.
Fixing Guides	Shows the information of fixing guides, which are the markers from DRD checking. Fixing guides help you fix violations by providing guidance on the edge of candidate design objects, the distance required to fix, and the direction to fix. Click the **Fixing Guides** link to highlight only the fixing guides of the selected marker on the canvas. This behaves the same as **Highlight Fixing Guides** in the marker table pane. Click the **Highlight only Marker** link to highlight only the selected markers on the canvas. This behaves the same as **Highlight Marker** in the marker table pane. Click the link of each individual marker to highlight the fixing guide of the marker on the canvas. Click the **Goto** link to select the parent marker in the marker tree and table panes, and also to highlight the marker on the canvas. Click the **Back** link (next to the Marker Index) to jump to the previous marker.

Feedback

Information	Description
Parent Makers	Shows the information of parent markers, which are the markers associated with the selected markers. Click the **Parent Markers** link to highlight only the parent markers of the selected marker on the canvas. This behaves the same as **Highlight Parent Markers** in the marker table pane. Click the **Highlight only Marker** link to highlight only the selected markers on the canvas. This behaves the same as **Highlight Marker** in the marker table pane. Click the link of each individual marker to highlight the parent marker of the marker on the canvas. Click the **Goto** link to select the parent marker in the marker tree and table panes, and also to highlight the marker on the canvas. Click the **Back** link (next to the Marker Index) to jump to the previous marker.
Click Here to Display Next 20 Info	Click this link to show the information of the remaining markers when more than 20 markers are selected.

4. Act on the shortcut menu on the canvas.

   The shortcut menu appears when you right-click on a marker on the canvas.

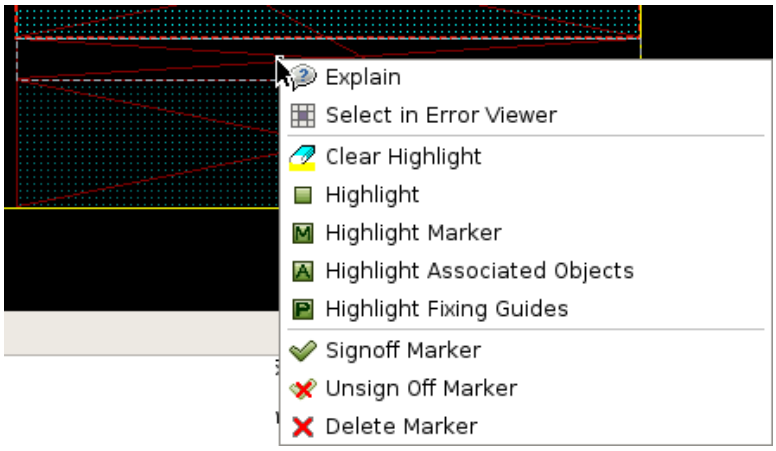


Table 76 describes the information of selected markers.

Feedback

Table 76 Shortcut Menu Options

Option	Description
Explain	Shows the long message of the selected marker in a separate **Explain** dialog box.
Select in Error Viewer	Highlights the selected marker on the canvas and also selects the marker in the Error Viewer.
Clear Highlight	Clears the highlight of the selected marker and the highlight of its associated objects.
Highlight	Highlights the selected marker and its associated objects on the canvas.
Highlight Marker	Only highlights the selected marker on the canvas.
Highlight Associated Objects	Only highlights the associated objects of the selected marker on the canvas.
Highlight Fixing Guides	Only highlights the selected fixing guides on the canvas. This appears only when the markers are from DRD checking.
Highlight Parent Markers	Only highlights the parent markers of the selected markers on the canvas.
Signoff Marker	Changes the severity type of highlighted markers to SignedOffError. This only works for severity types Error, Critical Error, and Warning. This button is disabled if the current design is read-only.
Unsign Off Marker	Reverses the Sign Off Marker action.
Delete Marker	Removes the selected marker from the marker tree and the design. The marker tree is refreshed after deletion. See Deleting Error Markers. This button is disabled if the current design is read-only.

Sorting Error Markers

You can sort error markers in descending (the default) or ascending order to easily access the ones you want to browse.

To sort error markers in the marker tree:

1. Begin by opening the Error Viewer assistant. In a design, choose **Window > Assistants > Error Viewer**.
2. From the Category column, click the column header to sort markers in the marker tree.

   (Optional) Click the **Tree View** button menu and choose **Unsort** to restore the marker tree order to its initial state.

Deleting Error Markers

You can delete more than one error marker in the marker tree and remove the highlight of the marker from the canvas.

To delete error markers:

1. Begin by opening the Error Viewer assistant. In a design, choose **Window > Assistants > Error Viewer**.
2. In the Error Viewer, choose the marker you want to delete.

   (Optional) To select more than one marker or a marker category, use Ctrl+ click to select each marker individually.

   (Optional) To select a range of markers or marker categories, use Shift-click or drag-left to quickly select them.
3. Click **Delete Marker** to remove the selected marker(s) from the hierarchical tree and the design.

   The tool clears the highlight of markers on the canvas and also refreshes the marker tree.

Customizing Marker Information to Display

You can now customize viewing the detailed information of the error markers that you have selected in the marker pane.

To customize the detailed information of a marker:

1. Begin by opening the Error Viewer assistant. In a design, choose **Window > Assistants > Error Viewer**.
2. In the Error Viewer, right-click the marker for which you want to customize the detailed information.
3. Check the information items to display from the menu, as shown in the following figure.

Feedback

The selected marker displays detailed information for only the information items that you have selected.

Showing Tooltips of Markers

You can show a tooltip containing the tool information, the severity type, and the short and long messages when hovering the pointer over a marker on the canvas.

To show the tooltip of a marker:

1. Begin by opening the Error Viewer assistant. In a design, choose **Window > Assistants > Error Viewer**.
2. Click the toolbar menu and make sure that **Show Tip** is on.
3. On the canvas, hover the pointer over the marker you want to check.

   When there are overlapped error markers, the tooltip uses the format `(num1/num2)` where `num1` indicates the index and `num2` indicates the total number of markers.

Feedback

Use shortcut keys 1 and 2 to cyclically show the tooltip of each marker.

Customizing the Error Viewer Layout Style

Instead of displaying the Error Viewer with the default layout style, you can change the layout with an alternate style. The tool provides the following layout styles: Style 1 (the default), Style 2, and Style 3.

To apply another layout style:

1. Begin by opening the Error Viewer assistant. In a design, choose **Window > Assistants > Error Viewer**.
2. Click the toolbar menu and choose **Layout Style > Style 2** to apply Style 2 to the layout.

Viewing Markers in Shadow

You can view design areas in shadow for additional visual contrast between selected markers and the surrounding design.

► To view design areas in shadow:

Click the **Cross Shadow** button from the design layout view to turn on the Shadow Mode. Click the arrow and move the slider to increase or decrease the contrast.

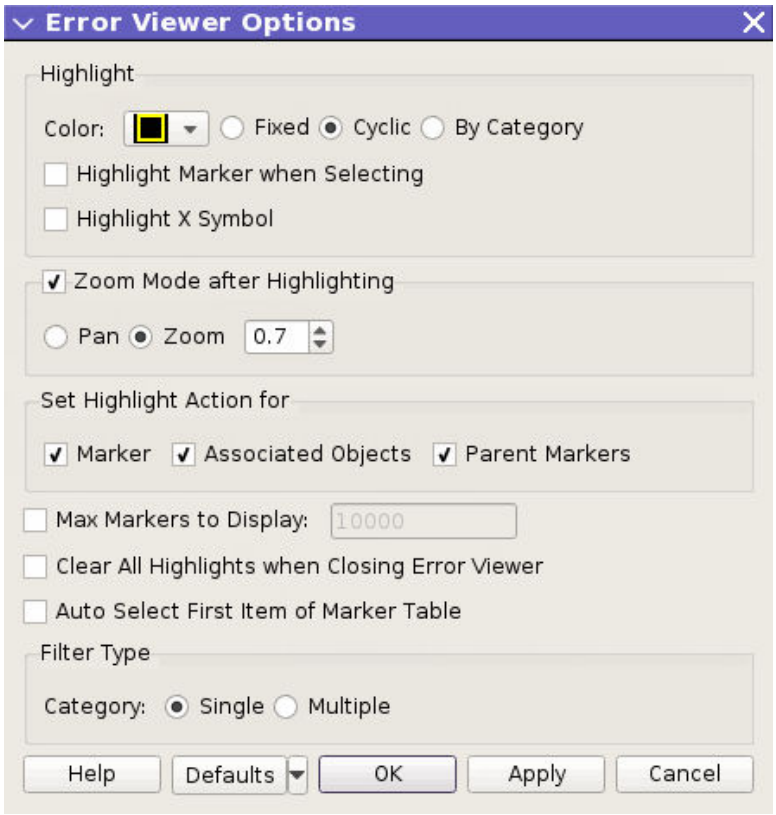
Managing Highlight Colors

You can manage highlight colors of markers and their associated objects to control their visual display in a design, and also assign highlight colors for different categories.

Feedback

To set highlight colors:

1. Begin by opening the Error Viewer assistant. In a design, choose **Window > Assistants > Error Viewer**.
2. Click the toolbar menu and choose **Options** to open the **Error Viewer Options** dialog box.



By default, the tool assigns colors cyclically through a recurring sequence within predefined colors. Table 77 describes predefined colors and color packets to highlight markers and their associated objects. For more information on how to view or edit color packets, see Display Resource Editor Tool.

The tool draws the outline of highlighted markers with solid color lines, and also draws the outline of their highlighted objects with dashed color lines.

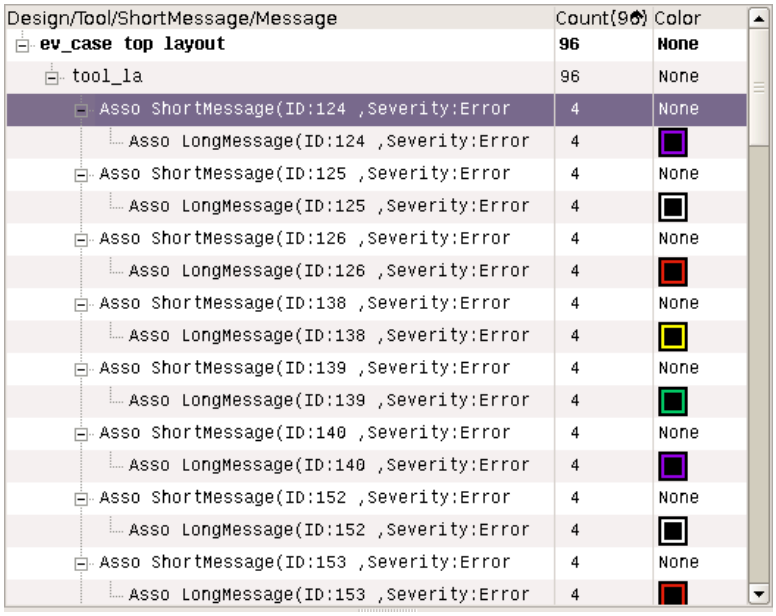
Feedback

Table 77 Predefined Colors and Color Packets for Highlights

Predefined Color	Color Packets (Solid Lines) to Highlight Markers	Color Packets (Dashed Lines) to Highlight Associated Objects
	highlight1	highlight1_dashed
	highlight2	highlight2_dashed
	highlight3	highlight3_dashed
	highlight4	highlight4_dashed
	highlight5	highlight5_dashed
	highlight6	highlight6_dashed
	highlight7	highlight7_dashed
	highlight8	highlight8_dashed
	highlight9	highlight9_dashed
	highlight10	highlight10_dashed

3. (Optional) From the **Error Viewer Options** dialog box, choose **Fixed** and pick a color from the list. The tool highlights markers and their associated objects in the same color.
4. From the **Error Viewer Options** dialog box, choose **By Category** to set different colors for different categories.

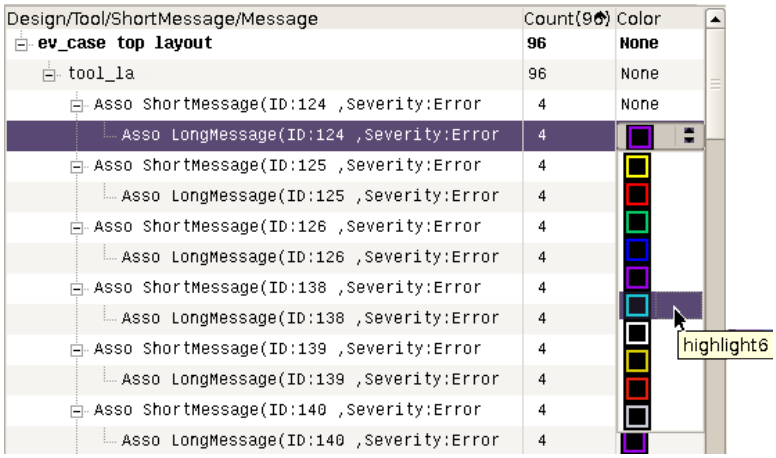
Feedback



5. Click **OK** to commit the change. You can find the Color column showing in the marker tree pane.

   By default, the tool only assigns colors to each lowest level of categories cyclically through a recurring sequence within predefined colors, described in Table 77.

6. From the Color column, double-click to choose a preferred color for the selected category.



   Changing the color of any upper levels of categories disables the color of underlying markers in the hierarchical tree.

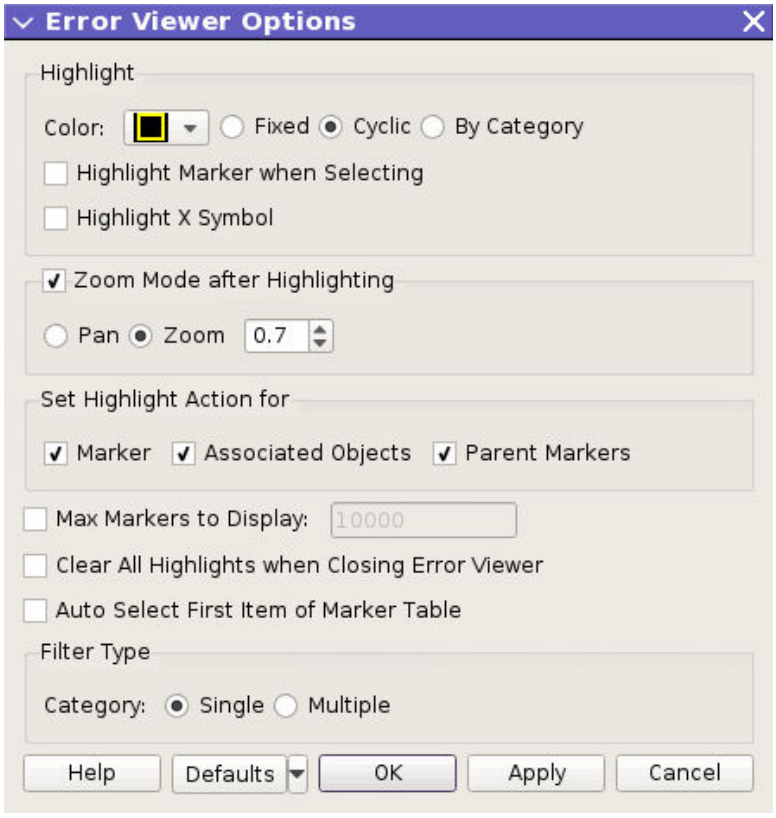
Feedback

Highlighting Markers When Selecting

You can highlight error markers and their associated objects on the canvas when selecting markers in the marker table pane.

To highlight markers and their associated objects on the canvas when selecting markers:

1. Begin by opening the Error Viewer assistant. In a design, choose **Window > Assistants > Error Viewer**.
2. Click the toolbar menu and choose **Options** to open the **Error Viewer Options** dialog box.



3. From the **Error Viewer Options** dialog box, turn on the **Highlight Marker when Selecting** option.
4. Click **OK** to save the changes.
5. In the marker table pane, choose the marker you want to highlight.

Feedback

(Optional) To select more than one marker, use Ctrl+click to select each marker individually.

(Optional) To select a range of markers, use Shift+click or drag-left to quickly select them.

Highlighting With an X Symbol

You can highlight markers by placing an X symbol at the center of the marker's bounding box.

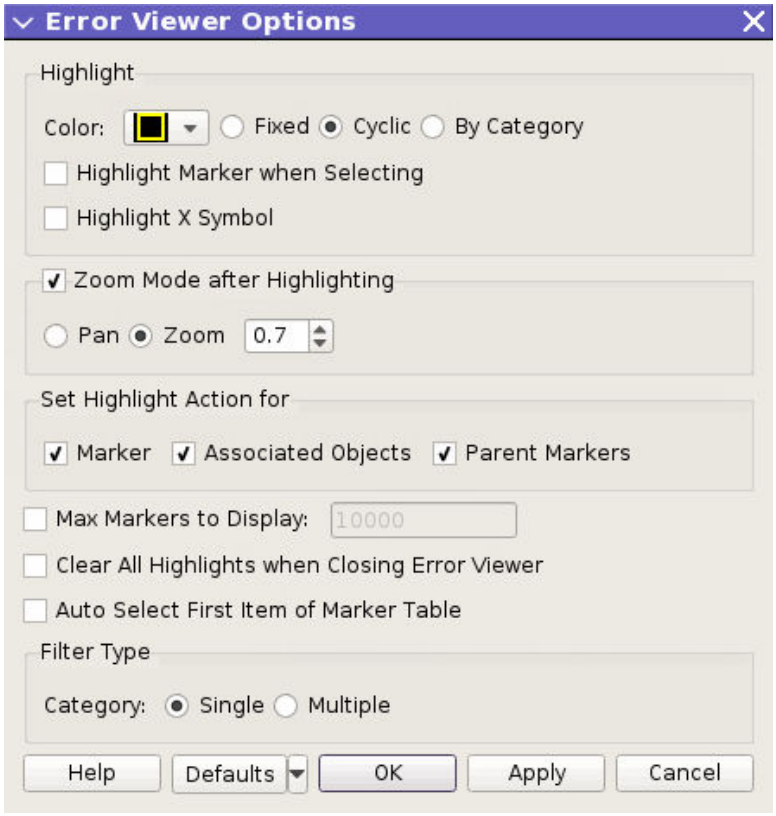
Highlighting without X symbol

Highlighting with X symbol

To highlight markers with an X symbol:

1. Begin by opening the Error Viewer assistant. In a design, choose **Window > Assistants > Error Viewer**.
2. Click the toolbar menu and choose **Options** to open the **Error Viewer Options** dialog box.

Feedback



3. From the **Error Viewer Options** dialog box, turn on the **Highlight X Symbol** option.
4. Click **OK** to save the settings.

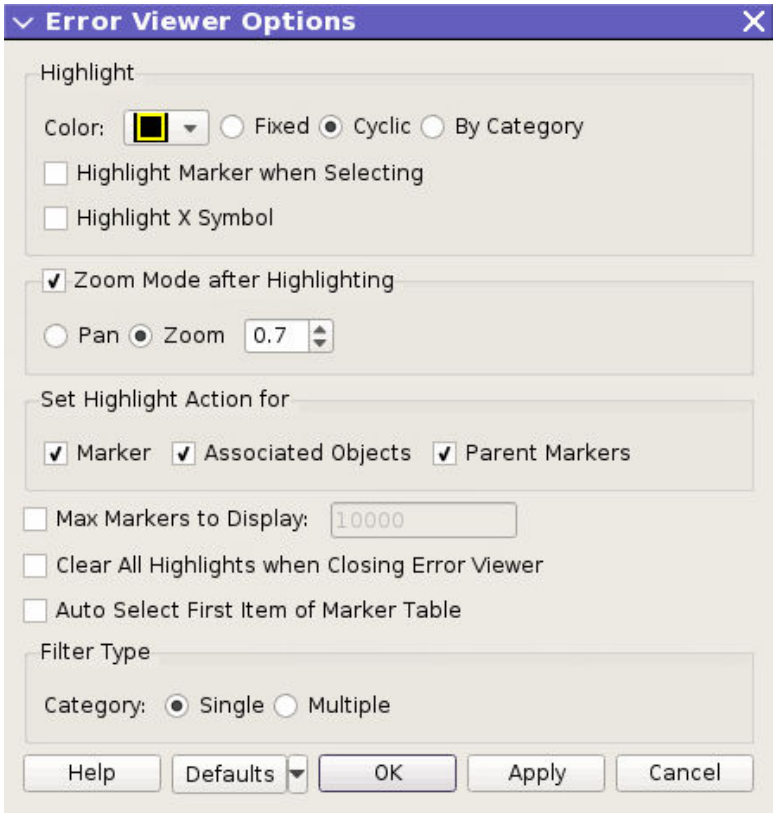
Setting Zoom Mode after Highlighting

You can zoom to the newly highlighted objects after enabling the **Zoom Mode after Highlighting** option.

To set the option:

1. Begin by opening the Error Viewer assistant. In a design, choose **Window > Assistants > Error Viewer**.
2. Click the toolbar menu and choose **Options** to open the **Error Viewer Options** dialog box.

Feedback



3. From the **Error Viewer Options** dialog box, ensure that the **Zoom Mode after Highlighting** option is enabled so that you can set the following options:

   When **Pan** is selected, you can pan to the newly highlighted objects without changing the zoom ratio of the canvas

   When **Zoom** is selected, you can zoom to the newly highlighted objects with the specified zoom ratio. By default, the zoom ratio is 0.7.

4. Click **OK** to save the settings.

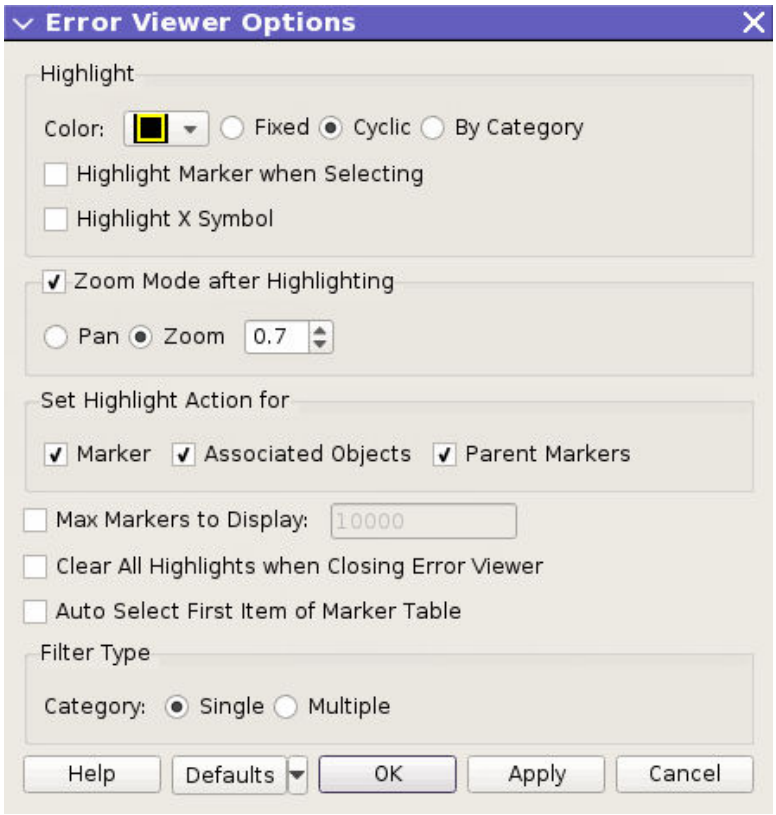
Setting Highlight Actions

You can set the highlight actions to highlight markers, the associated objects of markers, fixing guides, or parent markers.

Feedback

To set the option:

1. Begin by opening the Error Viewer assistant. In a design, choose **Window > Assistants > Error Viewer**.
2. Click the toolbar menu and choose **Options** to open the **Error Viewer Options** dialog box.



3. From the **Error Viewer Options** dialog box, go to the **Set Highlight Action for** section. By default, the tool highlights markers, the associated objects of markers, and fixing guides when **Marker**, **Associated Objects**, and **Fixing Guides** are enabled.

   To highlight only markers, disable all the options and then enable **Marker**.

   To highlight only the associated objects of markers, disable all the options and then enable **Associated Objects**.

   To highlight only fixing guides, disable all the options and then enable **Fixing Guides**.

   To highlight only parent markers, disable all the options and then enable **Parent Markers**.

4. Click **OK** to save the settings.

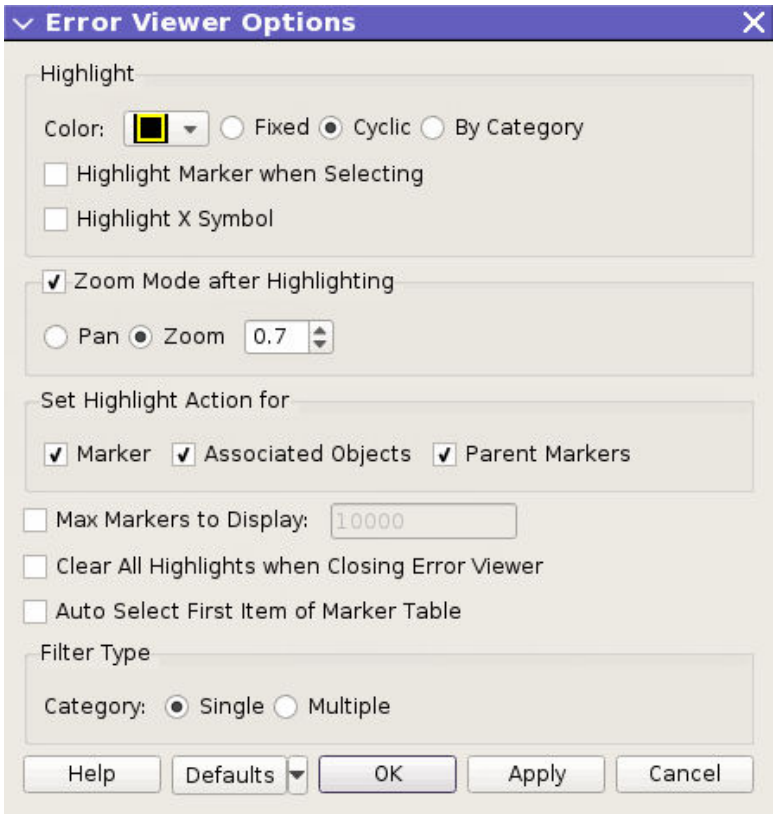
Feedback

Specifying the Maximum Number of Markers to Display

You can specify how many markers you want to display in the marker table pane.

To set the maximum number of markers to display:

1. Begin by opening the Error Viewer assistant. In a design, choose **Window > Assistants > Error Viewer**.
2. Click the toolbar menu and choose **Options** to open the **Error Viewer Options** dialog box.



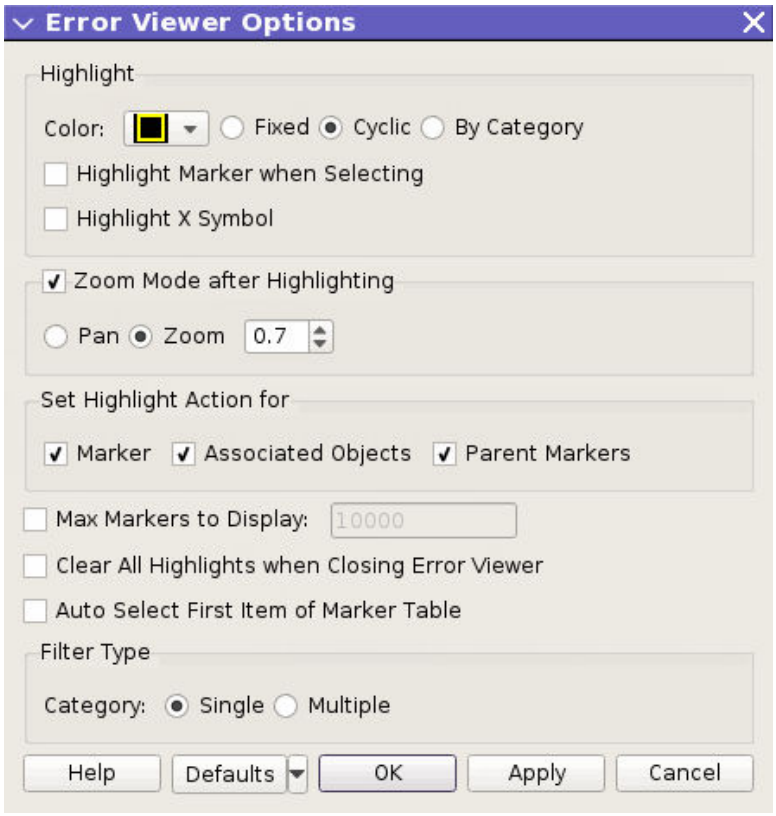
3. From the **Error Viewer Options** dialog box, turn on the **Max Markers to Display** option. By default, the maximum number is 10000.
4. Click **OK** to save the settings.

Feedback

Clearing All Highlights When Closing Error Viewer

To clear all highlights when closing the Error Viewer:

1. Begin by opening the Error Viewer assistant. In a design, choose **Window > Assistants > Error Viewer**.
2. Click the toolbar menu and choose **Options** to open the **Error Viewer Options** dialog box.



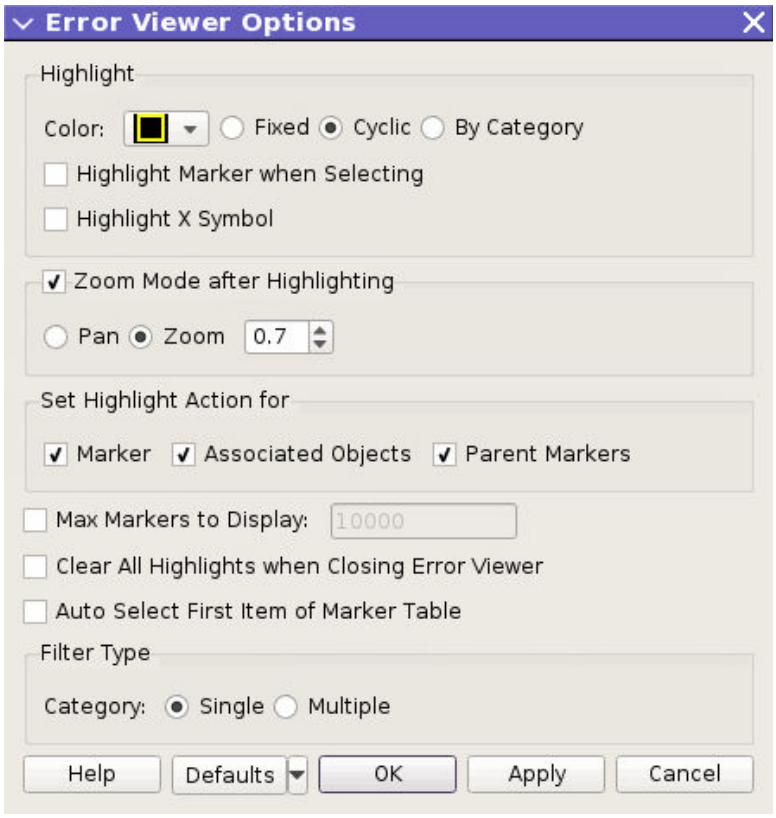
3. From the **Error Viewer Options** dialog box, turn on the **Clear All Highlights when Closing Error Viewer** option.
4. Click **OK** to save the settings.

Feedback

Selecting First Item of Marker Table

To automatically select the first item of the marker table when you change the selection in the marker tree:

1. Begin by opening the Error Viewer assistant. In a design, choose **Window > Assistants > Error Viewer**.
2. Click the toolbar menu and choose **Options** to open the **Error Viewer Options** dialog box.



3. From the **Error Viewer Options** dialog box, turn on the **Auto Select First Item of Marker Table** option.

   The associated Tcl preference is `deErrorViewerAutoSelectMarkerTable`.

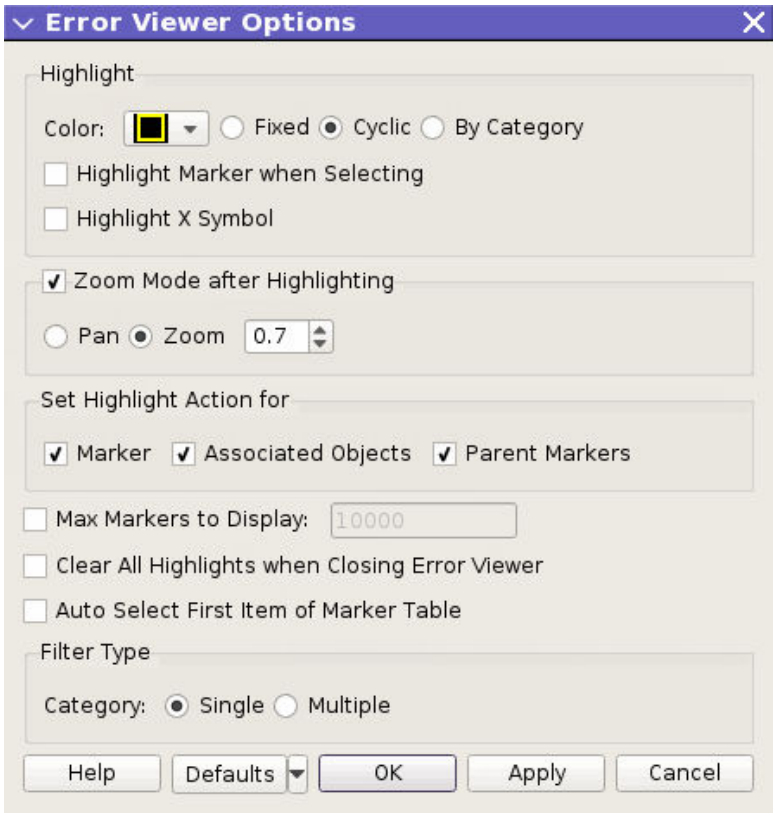
4. Click **OK** to save the settings.

Feedback

Setting Filter Type for Categories

To set the category filter type:

1. Begin by opening the Error Viewer assistant. In a design, choose **Window > Assistants > Error Viewer**.
2. Click the toolbar menu and choose **Options** to open the **Error Viewer Options** dialog box.



3. From the **Error Viewer Options** dialog box, go to the **Filter Type** section, choose **Single** (default) to show single category, or choose **Multiple** to allow showing multiple categories by the **Filter Display** option.

   The associated Tcl preference is `deErrorViewerFilterType`.

4. Click **OK** to save the settings.

   Note:

   Filter Display is available in the toolbar menu only when you set **Filter Type** as **Multiple Category** in the **Error Viewer Options** dialog box. See Configuring the Marker Tree Contents for more information.

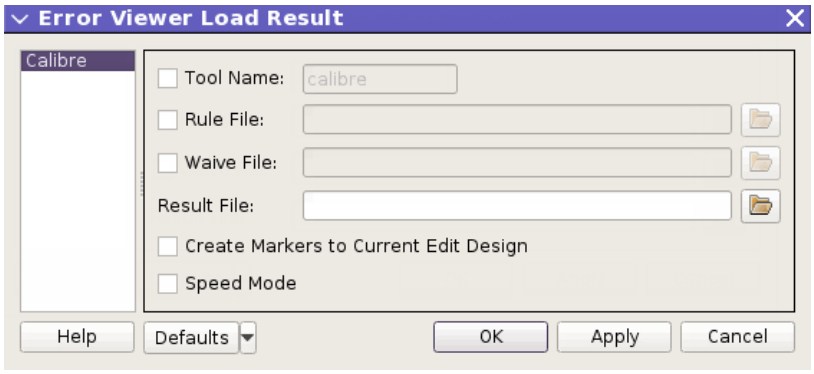
Loading Calibre Result

Note:

This feature is not available when using the **Error Viewer** assistant on an overlay design in the Schematic Editor.

To load a third-party result file into the Error Viewer:

1. Begin by opening the Error Viewer assistant. In a design layout, choose **Window > Assistants > Error Viewer**.
2. Click the toolbar menu and choose **Load Result** to open the **Error Viewer Load Result** dialog box.



3. From the **Error Viewer Load Result** dialog box, specify necessary files with the options in the following table.

Table 78 Load Result Options

Option	Description
ToolName	When enabled, specifies the tool name of the result file. The tool name can be set to an oaMarker's tool name. When disabled (default), the default tool name is **Calibre**.
Rule File	When enabled, specifies the rule file of the result file. (Off by default).

Feedback

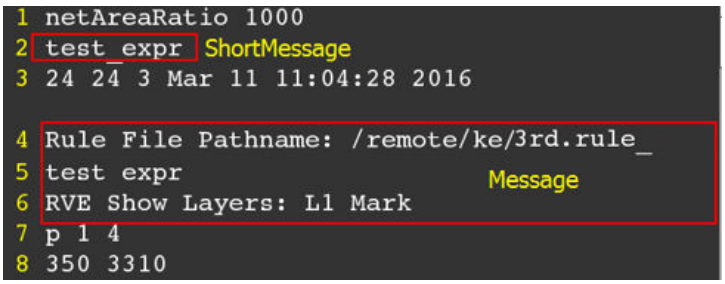
Option	Description
Waive File	When enabled, specifies the waive file. (Off by default).
Result File	Specifies the result file.
Create Markers to Current Edit Design	When enabled, creates markers to the current edit design. When disabled (default), creates markers to the overlay design.
Speed Mode	Used for loading large Calibre result files over 50MB.

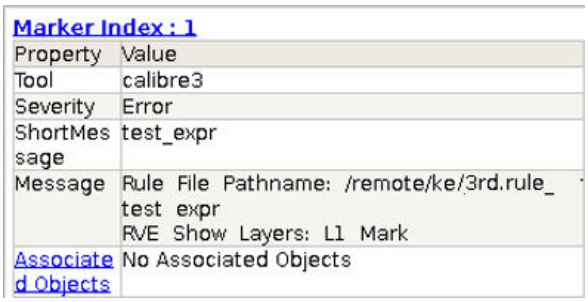
4. Click **OK** to save the settings and load the result file.

   After loading the result file, the tool creates markers in the current working cellview. The severity for each created marker is set as *Error Severity*. If the marker is from the waive file, the severity is set as *AcknowledgedWarning*.

   The rule type in the result file is mapped as the marker's ShortMessage attribute.

   The explanation strings in the result file are mapped as the marker's Message attribute if no rule file is specified or no corresponding rule information in the rule file. If a rule file is specified and there is a corresponding rule information of a rule type, the rule information of the rule type is mapped as the marker's Message attribute.





Feedback

Loading RedHawk Result

The **Error Viewer** parses the result file and creates as `oaMarkers` in the current working cellview.

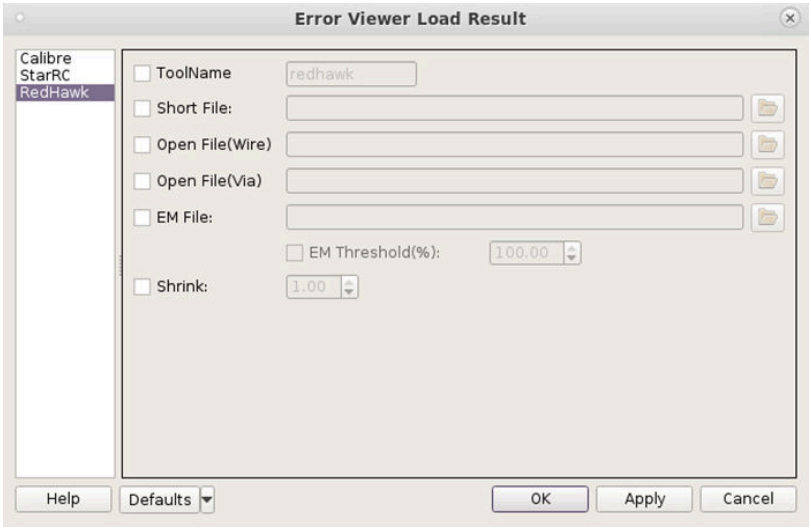


Table 79 Load Result Options

Option	Description
ToolName	When enabled, specifies the tool name of the result file. The tool name can be set to an `oaMarker's` tool name. When disabled (default), the default tool name is **RedHawk**.
Short File	When enabled, specifies a short file for short error information. (Off by default).
Open File (Wire)	When enabled, specifies open file for wire. (Off by default).
Open File (Via)	When enabled, specifies open file for via and net. (Off by default).
EM File	When enabled, reports the EM violations. (Off by default).
EM Threshold(%)	Error marker information is displayed if this value is greater than the user-specified value. Valid range is 10%~1000% (Default is 100%.)
Shrink	If user specified, the coordinates are converted to Shrink rate. Valid range is 0.01~1.0 (Default is 1.0).

Feedback

The following TcL preferences are introduced:

- `deErrorViewerRedHawkToolName`
- `deErrorViewerRedHawkToolNameValue`
- `deErrorViewerRedHawkShortFile`
- `deErrorViewerRedHawkShortFileText`
- `deErrorViewerRedHawkOpenWireFile`
- `deErrorViewerRedHawkOpenWireFileText`
- `deErrorViewerRedHawkOpenViaFile`
- `deErrorViewerRedHawkOpenViaFileText`
- `deErrorViewerRedHawkEMFile`
- `deErrorViewerRedHawkEMFileText`
- `deErrorViewerRedHawkShrink`
- `deErrorViewerRedHawkShrinkValue`
- `deErrorViewerRedHawkEMThreshold`

Loading Generic Format File

The **Error Viewer** does not support export/import of a text file, but only supports the **Save to view** feature. However, this feature is disabled during co-design. To be able to share errors to debug and avoid accidental deletion of errors in the **Error Viewer**, the tool allows conversion of different error marker formats into a generic format so that the **Error Viewer** can import this text file.

Feedback

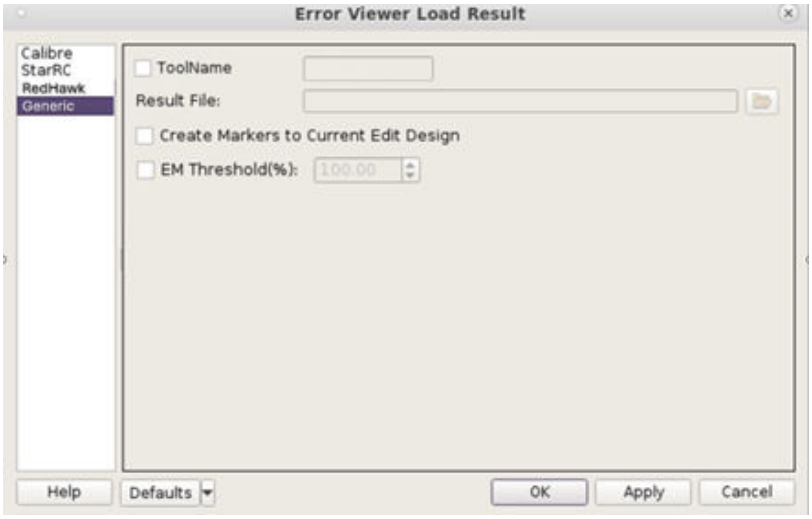


Table 80 Load Result Options - Generic

Option	Description
ToolName	When enabled, specifies the tool name of the result file. The tool name can be set to an oaMarker's tool name. When disabled (default), the default tool name is **Generic**.
Result File	When enabled, specifies the result file. It is in YAML format.
Create Markers to Current Edit Design	When enabled, creates markers to the current edit design. When disabled (default), creates markers to the overlay design.
EM Threshold(%)	Used for the **RedHawk** tool. Error marker information is displayed if this value is greater than user-specified value. The valid range is 10% ≦ x ≦ 1000%#Default is 100#). If this option is turned on, but error marker data is not `em_ratio` attribute, then error marker ignores to filter this error marker data.

The following Tcl preferences are introduced:

- `deErrorViewerExportGenericFile`
- `deErrorViewerGenericEMThreshold`
- `deErrorViewerGenericEMThresholdValue`
- `deErrorViewerGenericLoadFile`

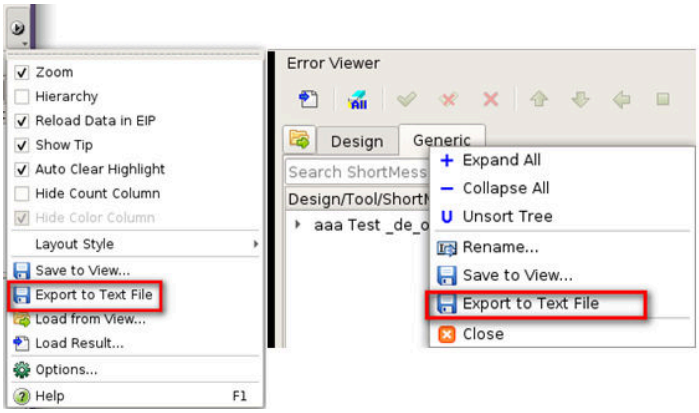
Feedback

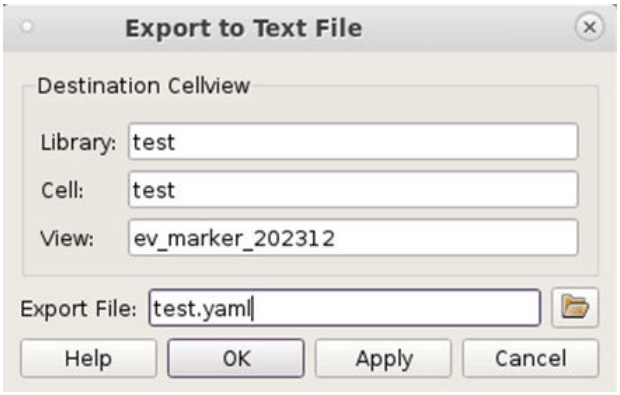
- deErrorViewerGenericLoadFileText
- deErrorViewerGenericToolName
- deErrorViewerGenericToolNameValue

Exporting to a Text File

You can export normal or overlay design to a text file and load it onto **Error Viewer**.

You can invoke the **Export to Text File** function in the following manner.





After exporting, **Error Viewer** loads the text file to new overlay or current design and then displays it in **Error Viewer**.

Table 81 Load Result Options - Generic

Option	Description
Library Cell	Specifies the destination library/cell/view. The default is the library name of current design, cell name, and, view name.

Feedback

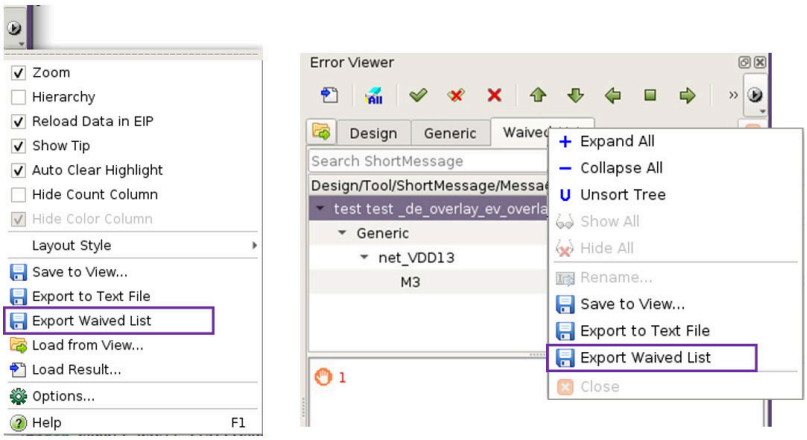
Table 81 Load Result Options - Generic (Continued)

Option	Description
View	
Export File	Specifies the file name to export. For an existing file, a number is appended to the old file name like `filename.$index`. The default is `$viewname.yaml`.

Tcl preference: `deErrorViewerExportGenericFile`.

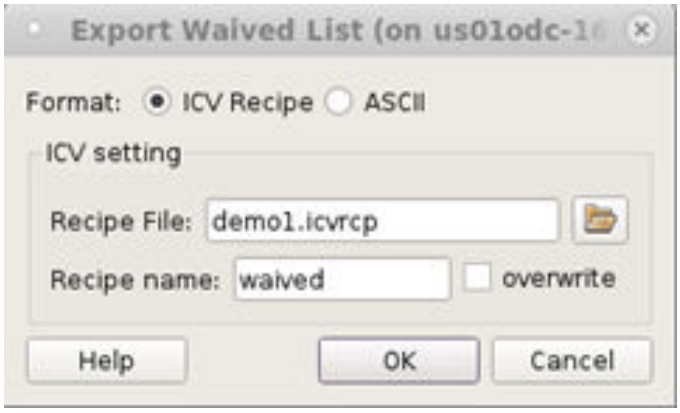
Exporting Waived List

You can export the **Waived_list** tab to a text file using **Export Waived List** option.



The Export Waived List has the following two formats:

- ICV recipe



Feedback

- ASCII

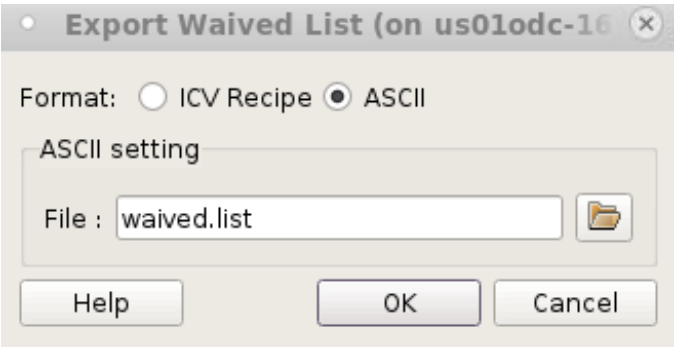


Table 82 Export Waived List

Option	Description
Recipe File	Specify a source recipe file or new file. When the file is not writable, **Error Viewer** displays message and stops. When the file is empty, **Error Viewer** exports the entire legal format.
Recipe name	Specifies the recipe name. For an existing file, a number is appended to the old file name like `name.$index`, If the overwrite option is Off, The default is **waived**.
overwrite	Overwrites the name existed in recipe file. The default is Off.
File name	Specifies the File name. For an existing file, a number is appended to the old file name like `filename.$index`, the default is **waived.list**.

The following TcL preferences are introduced:

- - `deErrorViewerExportRecipeFile`
- - `deErrorViewerExportRecipeName`
- - `deErrorViewerExportRecipeIsOverwrite`
- - `deErrorViewerExportRecipeFileType`
- - `deErrorViewerExportWaviedList`

Hierarchy Navigator Assistant

You can use the Hierarchy Navigator assistant to navigate and work in the hierarchy, by instance or master name.

- Displaying the Hierarchy Navigator Assistant
- Setting Navigation by Instance
- Setting Navigation by Master
- Exporting Design Hierarchy
- Resolving Context Issues
- Searching Instances
- Showing Quick Access Menu

Displaying the Hierarchy Navigator Assistant

To display the **Hierarchy Navigator** in a design layout or schematic view, choose **Window > Assistants > Hierarchy Navigator**.

The Hierarchy Navigator shows the hierarchy tree structure. The root of the hierarchical tree is the top library, cell, view (LCV). Each parent node contains the names of the child instances. Instances in the design are highlighted when they are selected in the Hierarchy Navigator.

Use the `heNavigatorSortOrder` Tcl preference to sort instances in alphabetical order or to place the cells with no hierarchy at the bottom of the hierarchy.

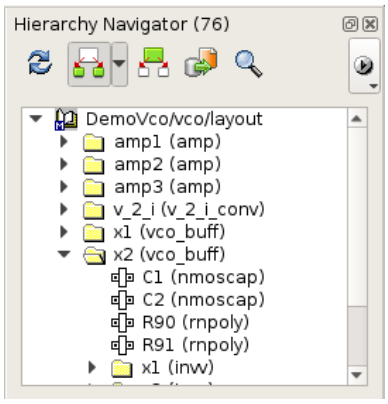
Setting Navigation by Instance

You can set navigation by instance to work with instances only.

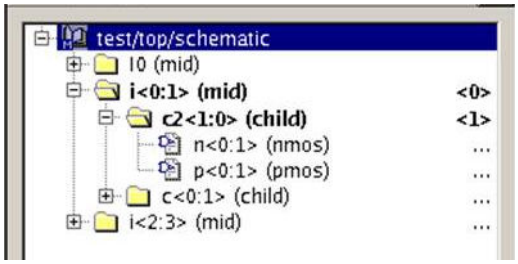
Feedback

To set navigation by instance:

1. Open the **Hierarchy Navigator** assistant. In a design layout or schematic view, choose **Window > Assistants > Hierarchy Navigator**.



2. In the toolbar, click **Hierarchy by Instance** to see the hierarchy of each occurrence of an instance in the design. The **Hierarchy Navigator** also specifies the individual bit index into a vector instance to descend into when navigating by instance. The selected index is displayed in a second column as shown in the following figure:



The following information is displayed in the second column based on the instances:

- The **Design Editor** descends into the selected index. The column shows the index, represented in angle brackets, of any vector instance in the descend path.
- Vector instances that are not in the descend path: Shows the current default or ellipses (…) if there is no current default index.
- Other instances: Shows no information in the index. The text in the left column spans both columns.

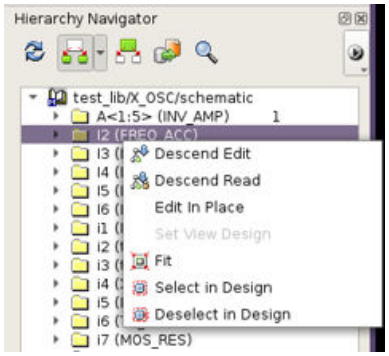
Click the index to show an in-line combination box with all indexes of the available vector instance. It is possible to select an index with the mouse or keyboard. You can type the number of the index for the cursor to automatically scroll and select the index from the list.

Feedback

To change the descend path in the **Design Editor** without prompting or redecorating the window, change the selected index in the descend path. The design is redrawn if the evaluation of any text objects depends on the descend path.

You can show and change the index of a vector instance that is not in the descend path after the column is displayed. The default index for the next descend operation changes (without descending into the instance) when the index of a vector instance that is not in the descend path changes. This index serves as the default for all descend operations, including the `ide::descend` preference, the `de::descend` preference, and by double-clicking within the **Hierarchy Navigator**.

3. Double-click the instance to descend into the selected instance. Right-click on an instance to show the following menu:



From this menu, select the following option to perform edits:

- Edit or view the designs in the hierarchy by choosing **Descend Edit** or **Descend Read**
- Edit the instance in place by choosing **Edit in Place** (available when the edit and the selected view designs match).
- Zoom to the selected master by choosing **Fit**. If the selected master does not exist within the edit design, **Fit** zooms to its parent master within the edit design.
- Set a design as the view design by choosing **Set View Design** (available when the edit and the selected view designs match).
- Select or deselect the instance in the canvas by choosing **Select in Design** or **Deselect in Design**.

Setting Navigation by Master

You can set navigation by master to work with master instances only.

Feedback

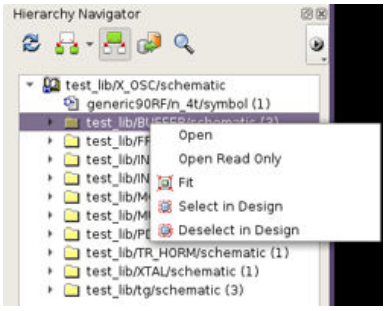
To navigate by master:

1. Open the Hierarchy Navigator assistant. In a design layout or schematic view, choose **Window > Assistants > Hierarchy Navigator**.
2. In the toolbar, click the **Hierarchy by Master** .

   To display the list of masters used in the design, and their hierarchy. The Hierarchy by Master mode shows the structure of the design.

   When many instances of a single master are used in a design, only one row is shown in the tree, and the instance count is shown between brackets if the master is not root.

In the design hierarchy list, right-click on a master to show the following menu:



From this menu, select the following option to perform edits:

- Open the master in the **Hierarchy Editor** or **Design Editor** option by choosing **Open** or **Open Read Only**.
- Zoom to the selected master by choosing **Fit**. If the selected master does not exist within the edit design, **Fit** zooms to its parent master within the edit design.
- Select or deselect the master in the canvas by choosing **Select in Design** or **Deselect in Design**. If the selected master does not exist within the edit design, it selects or deselects the parent master within the edit design.

The PCell submasters are not displayed as separate masters in the Hierarchy by Master display mode.

While in the Hierarchy by Master mode, you can open the design in a new editor window by double-clicking a master. However, it is not possible to change the design editor's context from the **Hierarchy by Master** display mode.

Exporting Design Hierarchy

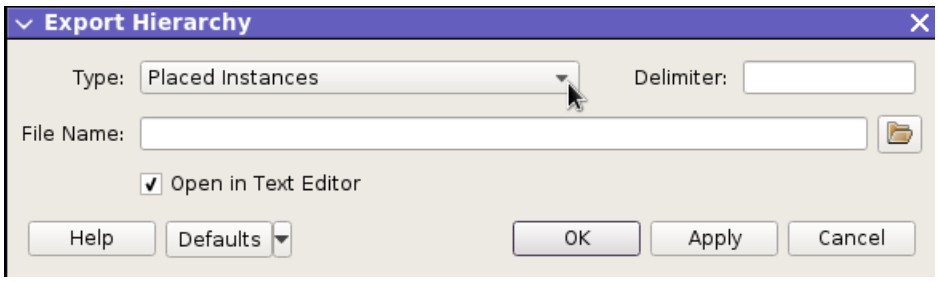
You can export the design hierarchy structure to a text file.

Feedback

To export the design hierarchy:

1. Begin by opening the Hierarchy Navigator assistant. In a design layout or schematic view, choose **Window > Assistants > Hierarchy Navigator**.
2. In the toolbar, click **Export Hierarchy** .

   This opens the **Export Hierarchy** dialog box.



   To select the type of hierarchy structure to export use the drop-down list with **Type**.

   In the text file with the exported hierarchy, the default delimiter between the instance names and their paths is a single space. To use any other delimiter, such as an underscore (_), specify it in the **Delimiter** tab. To view the file with the exported hierarchy structure in the Text Viewer tool, select **Open in Text Editor**. Click **OK** to open the file.

Resolving Context Issues

There are three types of context issues that can occur while working with designs.

You can use the Hierarchy Navigator to resolve the following issues.

- Invalid Contexts: If the context of the current edit or view design becomes invalid (due to change in the parent lineage), the Hierarchy Navigator prunes all the submodules of invalid instances from the tree except instances in the lineage of the current edit context.
- Out of context lineages: Instances can be pushed out-of-context while working with the design. The Hierarchy Navigator indicates the out-of-context instances by a red color font.
- Invalid Placed/Switched Masters: If the placed or switched master does not exist, the instance/master is displayed in a red color font. If the master name is visible, the placed master is displayed instead of the switched master.

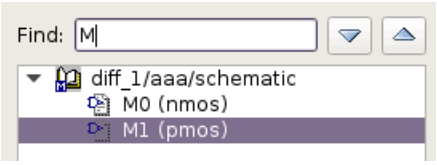
Feedback

Searching Instances

To find instances in the design hierarchy:

1. Begin by opening the Hierarchy Navigator assistant. In a design layout or schematic view, choose **Window > Assistants > Hierarchy Navigator**.
2. In the toolbar, click **Show Search Line** , which shows or hides the **Find** text field.

   Type an initial of the instance name you want to locate, for example:

   Filtering adheres to the **Find** preference setting of the **General Options** dialog box for partial match, wildcard, regular expression, and match start. See Setting General Options.

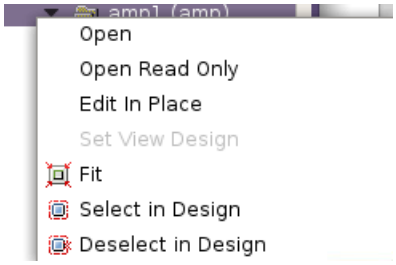
   Click **Next** and **Previous** to cycle through search results.

Showing Quick Access Menu

The Hierarchy Navigator allows you to access frequently used commands when you right-click an instance.

To show the menu:

1. Open the Hierarchy Navigator assistant. In a design layout or schematic view, choose **Window > Assistants > Hierarchy Navigator**.
2. In the design hierarchy list, right-click on an instance to show the following menu:

   From this menu, you can

- Open the selected instance in another window by choosing **Open** or **Open Read Only**.
- Edit the selected instance directly in the same window by choosing **Edit in Place**.
- Zoom to the selected instance by choosing **Fit**. If the selected instance does not exist within the edit design, **Fit** zooms to its parent instance within the edit design.
- Select or deselect the instance in the canvas by choosing **Select in Design** or **Deselect in Design**.

Object Filter Assistant

You can use the Object Filter assistant to control the way the system responds to the selection of specific object types. The Object Filter works with schematic and symbol views.

If you are working in an area of a schematic where you cannot select a wire due to the interference of bounding boxes, you can use the Object Filter to turn off bounding box object selection, and more easily allow wire selection.

You can also use the Object Filter during area selection, for example, in situations where you might want to select a group of instances, and exclude wires, and other objects, from the selection set.

This topic describes the following:

- Opening the Object Filter Assistant
- Filtering Schematic Objects
- Filtering Symbol Objects

Opening the Object Filter Assistant

Open the Object Filter to work with object selection settings in schematics and symbols.

To open the Object Filter:

- In the schematic or symbol, choose **Window > Assistants > Schematic Object Filter** or **Symbol Object Filter**.

The assistant docks along the window edge. Undock it by clicking and dragging it.

Filtering Schematic Objects

You can filter schematic objects, and control their selectability.

Feedback

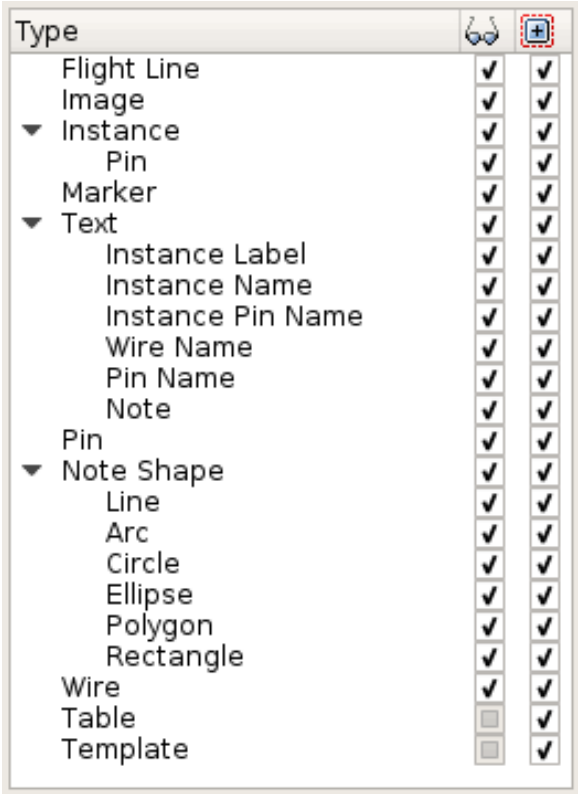
To filter schematic objects:

1. Begin by opening the Object filter assistant. In the schematic editor, choose **Window > Assistants > Schematic Object Filter**. (Alternatively, in a design view, choose **Edit > Select > Schematic Object Filter**).
2. Choose an option:

   To enable or disable individual selectability for an object, click its check box.

   To enable or disable selectability for all objects, click the buttons ( ) at the top.

The Schematic Object Filter controls selectability and visibility for these object types:



Filtering Symbol Objects

You can filter symbol objects to limit, and control their selectability.

Feedback

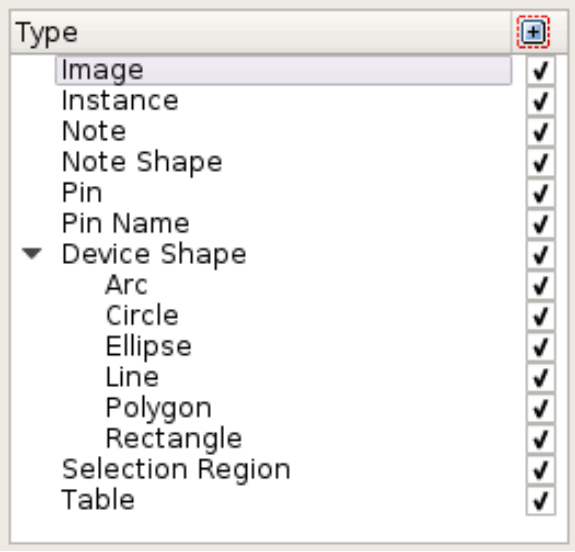
To filter symbol objects:

1. Begin by opening the Object filter assistant. In the symbol editor, choose **Symbol Object Filter**. (Alternatively, in a design view, choose **Edit > Select > Symbol Object Filter**).
2. Choose an option:

   To enable or disable individual selectability for an object, click its check box.

   To enable or disable selectability for all objects, click the buttons at the top.

The Symbol Editor Object Filter controls selectability and visibility of these object types:



Object/Layer Panel Assistant

You can use the Object/Layer Panel (OLP) in layouts to:

- Locate layer purpose pairs (LPPs), layers, and objects
- Display multiple built-in and user-defined LPP sets
- Create, edit, or remove user-defined LPP sets
- Make LPP sets, active LPP, and object types visible and selectable
- Filter LPPs, layers, objects, and mask color
- Sort the list of active LPPs
- Display more LPPs and LPP sets at a time when using Compact View

- Toggle between using design settings and technology settings
- Control via and instance selections
- Control the visibility and selectability of objects
- Redraw the canvas after changes are made within the OLP
- View the display packets associated with LPPs
- Reload the display resource file

Each design has an Object/Layer Panel. Settings you make apply immediately, and are stored in the technology database.

This topic describes the following:

- Opening the Object/Layer Panel Assistant
- Manipulating the Object/Layer Panel
- Locating LPPs, Layers, and Objects
- Displaying Color Annotations
- Displaying Presets
- Working With LPPs
- System LPPs
- Working With Layers
- Working With Objects
- Sharing OLP Settings
- Controlling Area Selections of Figure Groups
- Working With Display Resources
- Setting OLP Options

	The Object/Layer Panel opens automatically with design layout views. This video tutorial shows how to display,select and manage LPPs using the various options available for layers and objects.

Feedback

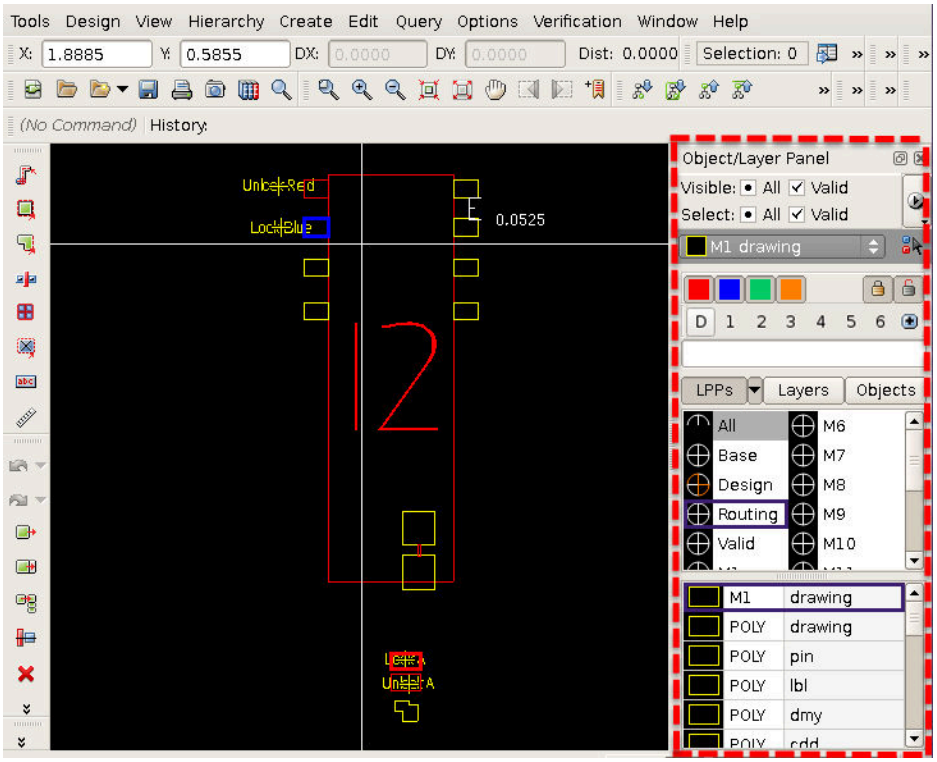
Opening the Object/Layer Panel Assistant

The Object/Layer Panel opens automatically with design layout views. You can also open it manually as follows:

In a design layout view, choose **Window > Assistants > Object/Layer Panel**.

The Object/Layer Panel docks on the right side of the design window. Undock it by clicking and dragging it to a required location.

Figure 42 Object/Layer Panel Assistant



Manipulating the Object/Layer Panel

The Object/Layer Panel offers actions from the Object/Layer Panel toolbar and also through the toolbar menu.

To manipulate the Object/Layer Panel:

1. Begin by opening the Object/Layer Panel assistant. In a design layout view, choose **Window > Assistants > Object/Layer Panel**.
2. Act on the objects or LPPs in the Object/Layer Panel.

Feedback

You can use the toolbar boxes and toolbar menu.

Table 83 describes actions you can perform using Object/Layer Panel toolbar boxes.

Table 83 Object/Layer Panel Toolbar Boxes and their Actions

Selection	Action
Visible: ● All ☑ Valid	Set the visibility for all objects or LPPs or only those that are valid.
Select: ● All ☑ Valid	Set the selectability for all objects or LPPs or only those that are valid.

You can use the `leOLPShowActionsToolbar` Tcl preference to display or hide the **Visible** and **Select** options.

Table 84 describes actions you can perform using the Object/Layer Panel toolbar menu .

Table 84 Object/Layer Panel Toolbar Menu Options

Menu Option	Description
OLP Options	Opens the Object Layer Panel dialog box. See Setting OLP Options.
Make Visible LPPs Selectable	Makes all currently visible LPPs selectable.
Display Resource Editor	Opens the Display Resource Editor tool for the current design. See Working With Display Resources for help using this option.
Reload Display	Reloads the modified display resource file to the Object/Layer Panel. See Working With Display Resources for help using this option.

Locating LPPs, Layers, and Objects

Using the Object/Layer Panel, you can use the filter box to locate LPPs, layers, and objects in the **LPPs**, **Layers**, and **Objects** tabs separately. The filter applies to the current tab. When switching tabs, the filter applies to the new tab.

Feedback

To locate LPPs, layers, and objects:

1. Begin by opening the Object/Layer Panel assistant. In a design layout view, choose **Window > Assistants > Object/Layer Panel**.
2. In the Object/Layer Panel, click one of the following tabs:
   - **LPPs** tab that displays the list of LPPs in the design
   - **Layers** tab that displays the list of layers in the design
   - **Objects** tab that displays the list of objects in the design

   When switching the **LPPs**, **Layers**, and **Objects** tabs, the filter applies to the tab you are currently in.
3. (Optional) To view more LPPs at once, you can resize the Object/Layer Panel by adjusting its width so that the LPPs are listed in several visible columns.
4. (Optional) From the toolbar menu , make sure that the **Show LPP Filter** option is on.

   The Object/Layer Panel shows the filter box by default. You can deselect **Show LPP Filter** to hide the filter.
5. In the filter box, type the full or partial text or an expression to search for the items of interest. See Using Filter Boxes for help using this filter.

   When locating LPPs or layers, use a space to apply filtering to the layer name and purpose name, for example: m* dr* matches LPP metal1 drawing (which is the closest match), and other LPPs starting with layer name *m* and purpose *dr*.

   When finding route related objects in a list of object types, first click the **Objects** tab, and then type `route` in the filter box.
6. With the list of components you want to use now optionally filtered, select them in the Object/Layer Panel.
7. Use the Object/Layer Panel to act on the selected components.

   When you are ready to act on the selected components, see the following sections:
   - Working With LPPs
   - Working With Layers
   - Working With Objects

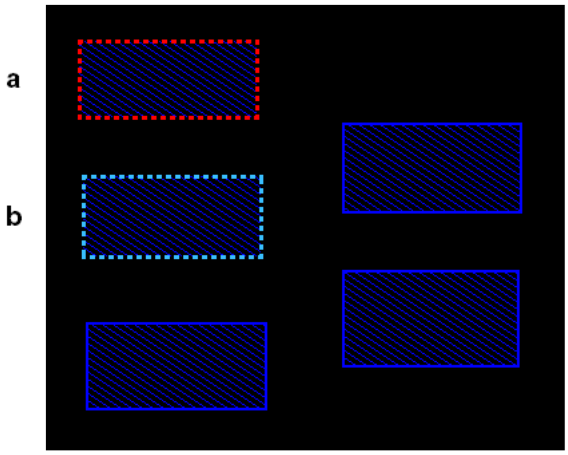
Feedback

Displaying Color Annotations

You can display or hide color annotations in your design. By default, color annotations appear as colored outlines surrounding shapes to indicate the colors that are assigned to the shapes. However, you can change how the annotations appear using the **Packets** settings in the Display Resource Editor. For more information, see Display Resource Editor Tool.

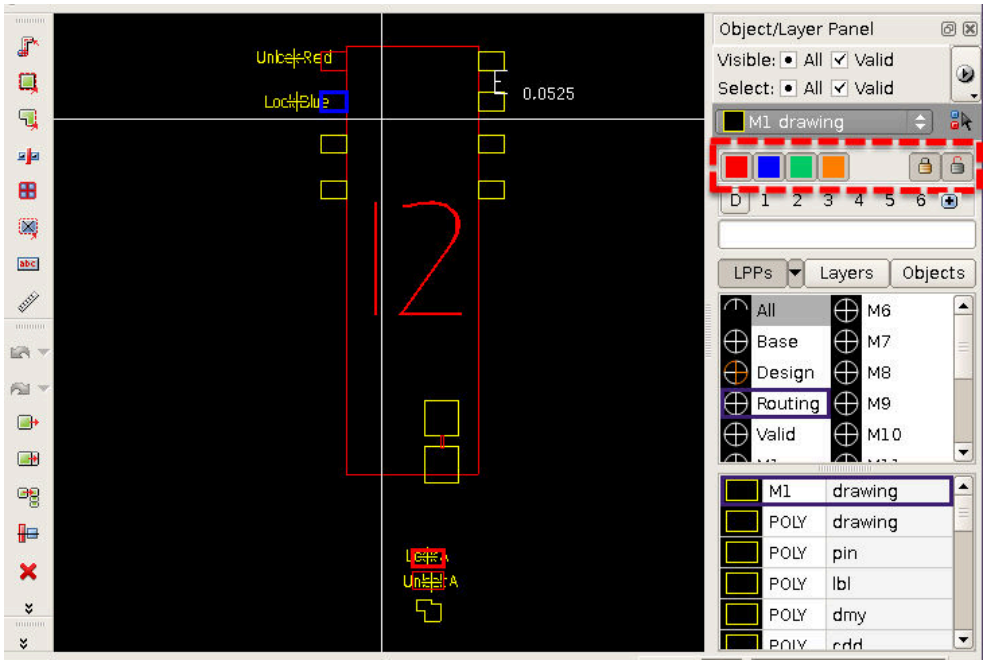
In Figure 43, color annotations are displayed for shapes a and b using the default annotation settings. Shape a has a red outline, indicating that it is assigned color1. Shape b has a blue outline, indicating that it is assigned color2. The remaining shapes in this example are not annotated.

Figure 43 Color Annotations Displayed for Shapes a and b



To display color annotations:

1. Begin by opening the Object/Layer Panel assistant. In a design layout view, choose **Window > Assistants > Object/Layer Panel**.
2. Click the toolbar menu choose **Show Mask Color Filter** to show the **Color** toolbar at the top of the Object/Layer Panel.



3. Click one of the following to display color annotations:

Button	Description
	Displays annotations of shapes that are assigned the Red color.
	Displays annotations of shapes that are assigned the Blue color.
	Displays annotations of shapes that are assigned the Green color.
	Displays annotations of shapes that are assigned the Same color.
	Displays only the color annotations of shapes whose colors are locked.
	Displays only the color annotations of shapes whose colors are not locked.

For information on how to assign colors to shapes, see the *Custom Compiler Layout Editor User Guide for Creating and Editing Objects* for details.

Feedback

Displaying Presets

You can display or hide the **Presets** toolbar.

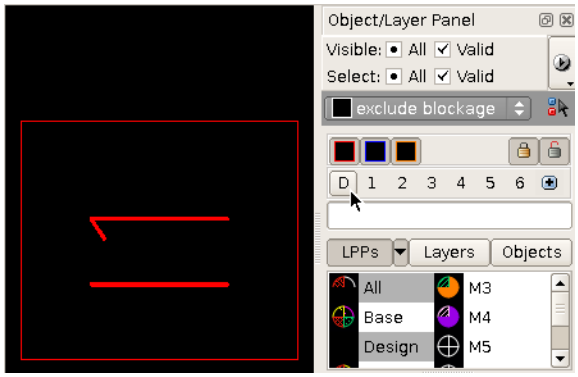
A preset is a named collection of state information in the Object/Layer Panel which can be restored en masse. A preset contains:

- Selected LPP sets and their relationship
- LPP visibility, selectability, and fill status
- Object filter visibility and selectability status

A preset can be created from the current design state using the `de::createOLPPreset` Tcl command. It is not possible to create one programmatically outside of manipulating the design and then capturing its state.

To display a preset status information:

1. Begin by opening the Object/Layer Panel assistant. In a design layout view, choose **Window > Assistants > Object/Layer Panel**.
2. Click the toolbar menu choose **Show Presets** to show the **Presets** toolbar at the top of the Object/Layer Panel.

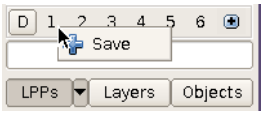


   On the **Presets** toolbar, **D** stands for Default, and it is the default setting of LPP list (or selected LPPs) in the design that is opened. The preset number are user-saved LPP selection settings that apply globally. Click the button to add more preset number.

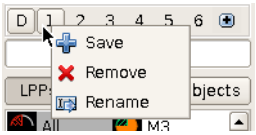
3. You can set your own LPP selection settings and then right-click one of the numbered preset buttons to save the settings.

   Right-clicking preset button shows a **Save** button to save the user-defined LPP selection settings.

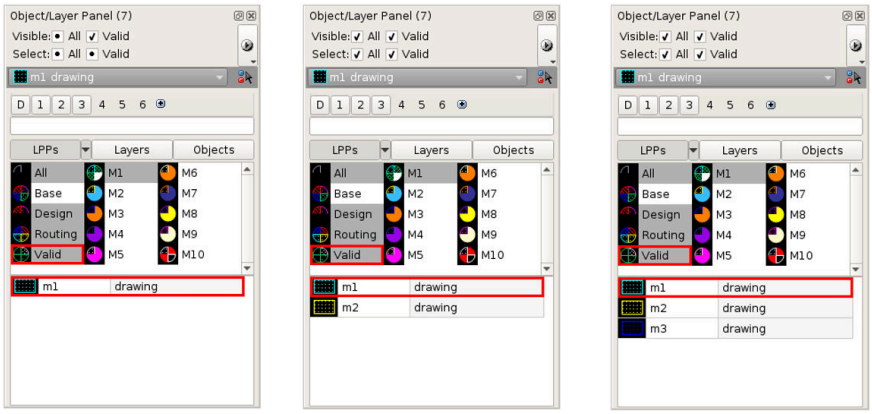
Feedback



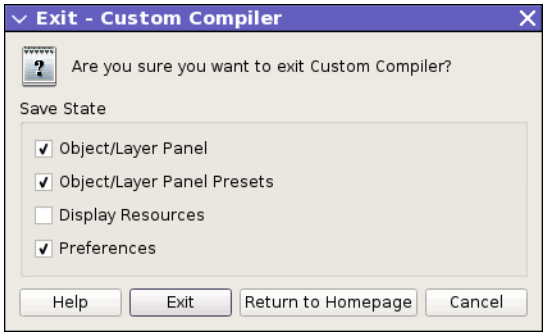
You can remove or rename an existing user-saved LPP selection settings preset by right-clicking the numbered button for the user-saved preset.



You can activate multiple presets at a time by saving multiple numbered buttons. You can merge an active preset with the current LPP settings by clicking the corresponding numbered button while pressing the Shift key. To delete or subtract this preset setting from the current LPP settings, click the corresponding numbered button while pressing the Ctrl key. You must click the least-numbered button to merge or delete.



Presets in the **Object** tab are saved and loaded through the application state interface defined by `deOLPPresets.xml`. You will be prompted to save the preset settings with the **Object/Layer Panel Presets** option when closing the software.



Feedback

Working With LPPs

In the **LPPs** tab, you can:

- Display multiple built-in and user-defined layer LPP sets

  LPP sets are sorted alphanumerically. LPP sets with no default LPP are shown first, followed by a small separator, and then the LPP sets which have default LPPs. The LPP sets with default LPPs are sorted in mask order of the default LPPs.

- Create, edit, or remove user-defined LPPs
- Make active LPPs visible and selectable
- Show abbreviations and stream layer numbers of LPPs
- Change the drawing priority of an LPP
- Show or hide LPPs with specified purposes
- Sort the list of active LPPs
- Control the visibility and selectability of boundary objects

The **LPPs** tab consists of two panes. The upper pane shows the built-in LPP sets and the user-defined LPP sets. The lower pane shows the contained LPPs of a selected LPP set.

The following sections cover:

- Using LPPs Tab Menu
- Creating User-Defined LPP Sets
- Editing User-defined LPP Sets
- Removing User-defined LPP Sets
- Setting the Active Layer Purpose Pair
- Making LPP Sets Visible and Selectable
- Showing Abbreviations and Stream Layer Numbers
- Changing the Drawing Priority of a Layer Purpose Pair
- Sorting Layer Purpose Pairs
- Controlling the Visibility and Selectability of Boundary Objects

Feedback

Using LPPs Tab Menu

To use the **LPPs** tab menu:

1. Begin by opening the Object/Layer Panel assistant. In a design layout view, choose **Window > Assistants > Object/Layer Panel**.
2. Click the **LPPs** tab and then click the tab menu. Table 85 describes the LPPs menu options.

Table 85 LPPs Tab Menu Options

LPPs Menu Option	Description
New Set	Creates a new user-defined LPP set. See Creating User-Defined LPP Sets for help creating your own LPP sets. This set is always available.
Edit LPP Set	Available only when a user-defined LPP set is selected and it is not a built-in LPP set. See Editing User-defined LPP Sets for help editing user-defined LPP sets.
Remove LPP Set	Available only when one or more LPP sets are selected and none of them are system-defined. See Removing User-defined LPP Sets for help removing LPP sets.
Hide Valid	Hides the selected LPP set in the **LPPs** tab.
Show All LPP Sets	Displays all LPP sets, including those previously hidden, in the **LPPs** tab.
AND Selected Sets	Filters LPPs that are used in the current design by making an intersection of two or more selected LPP sets. This option works for all LPP sets including system-reserved and user-defined LPP sets. You can also type the following line in the console to show the intersection of selected sets (for example: lpp_set1 and lpp_set2) on the Object/Layer Panel: `db::setPrefValue leLPPSet -value "lpp_set1 * lpp_set2"`
Show LPP Sets	Displays the upper pane of LPP Sets in the **LPPs** tab.
Abbreviate Names	Displays LPPs using their technology-defined abbreviated layer names. (On by default). Tcl preference: `leLPPAbbreviations`

Feedback

LPPs Menu Option	Description
Show Stream Layer Numbers	When enabled, the tool shows the stream layer and data type numbers in a tooltip, rather than showing the OpenAccess layer and purpose numbers, if the layer mapping file is found. If the layer mapping file is not found, the tool checks if the purpose is system reserved and if there is a mapping for the LPP and then uses it as the stream layer or data type number. Tcl preference: `leLPPShowStreamLayerNumbers`
Compact View	Shows more LPP sets and LPPs at a time. By default, the tool uses a 24 pixel row height for each table in the Object/Layer Panel. When enabled, a 20 pixel row height is used instead. Tcl preference: `leOLPCompactView`
Show/Hide Purposes	Displays a **Show/Hide Purposes** dialog box with a list of purposes used by LPPs.
Sort	Sorts the LPPs in the lower pane by **Priority** (default), **Alphanumeric**, and **Layer Number**. See Sorting Layer Purpose Pairs for help sorting the LPPs. Tcl preference: `leSortLPPs`

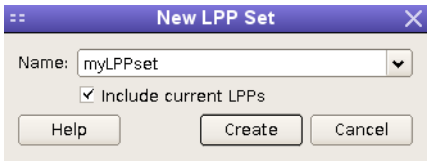
Creating User-Defined LPP Sets

You can create your own LPP sets in the **LPPs** tab.

To create a user-defined LPP set:

1. Begin by opening the Object/Layer Panel assistant. In a design layout view, choose **Window > Assistants > Object/Layer Panel**.
2. Choose an existing LPP set, for example: *M1*.
3. Click the **LPPs** tab menu and choose **New Set**.

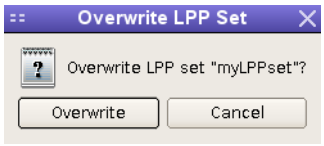
   The **New LPP Sets** dialog box appears.
4. Enter *myLPPset* in the **Name** box, and then enable the **Include current LPPs** option to include the currently valid LPPs; otherwise, the new LPP set is empty. In this case, the new LPP set includes the valid LPPs from the M1 set.



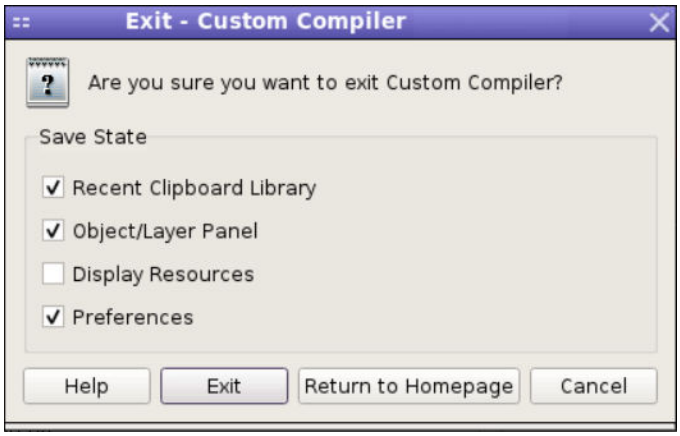
Other than the built-in LPP sets, current sets are listed in the **Name** list so that they can be replaced if needed.

5. Click **Create** to create the new LPP set, *myLPPset*.

   If the LPP set already exists, the tool shows an **Overwrite LPP Set** dialog box to confirm overwriting it.



   New user-defined LPP sets are saved to the `./synopsys_custom/lppSets.tcl` file if you choose to save the Object/Layer Panel state when exiting the Custom Compiler tool. The system automatically loads the file on next session.



   You can start editing the newly created LPP set in the LPP pane if needed.

Editing User-defined LPP Sets

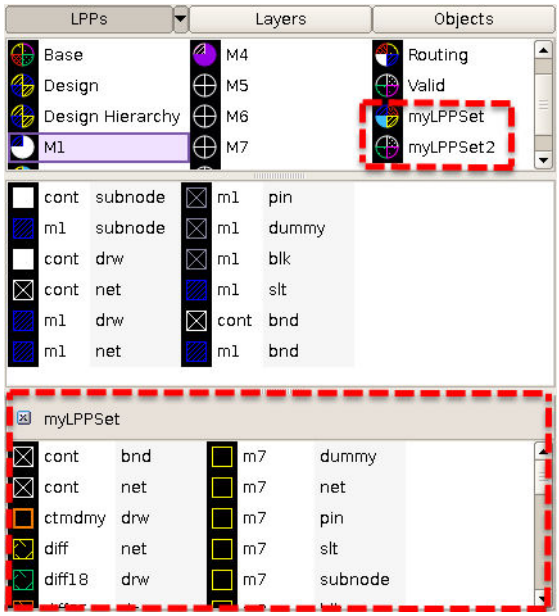
You can edit user-defined LPP sets in the **LPPs** tab.

To edit a user-defined LPP set:

1. Begin by opening the Object/Layer Panel assistant. In a design layout view, choose **Window > Assistants > Object/Layer Panel**.
2. Choose the user-defined LPP set you want to edit, for example: *myLPPset*.
3. Click the **LPPs** tab menu and choose **Edit <setName>**.

   The tool replaces <setName> with *myLPPSet* and displays its contained LPPs in a new pane.

Feedback



4. Select a built-in LPP set or another user-defined LPP set, and then drag and drop the required LPPs from the selected set to *myLPPset*.

   (Optional) To select or deselect more than one LPP set, use Ctrl-click to select or deselect each LPP individually.

   (Optional) To select multiple LPPs, use Shift+click or drag-left to quickly select them.

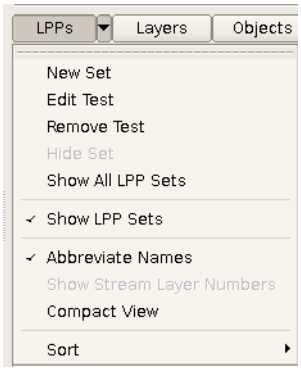
5. Click the close button after editing. This updates the *myLPPSet* contents.

Removing User-defined LPP Sets

You can remove user-defined LPP sets from the **LPPs** tab.

To remove a user-defined LPP set:

1. Begin by opening the Object/Layer Panel assistant. In a design layout view, choose **Window > Assistants > Object/Layer Panel**.
2. Choose the user-defined LPP set you want to remove, for example: *myLPPset*.
3. Click the **LPPs** tab menu and choose **Remove <setName>**.



The tool replaces <setName> with *myLPPset* and displays its contained LPPs in the lower pane.

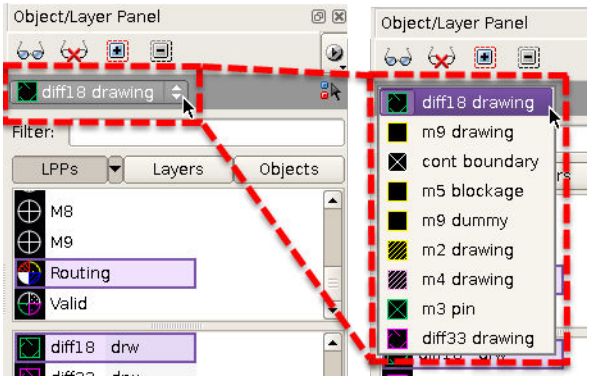
The **Remove LPP Set** dialog box appears.

4. Click **Remove** to confirm removing the selected LPP set.

Setting the Active Layer Purpose Pair

You can set the active layer purpose pair (LPP) to make it the target for layout operations.

The active LPP is indicated by name at the top of the Object/Layer Panel. Click the arrow next to the active LPP indicator and choose from a list of the last ten active LPPs.



To set the active LPP:

1. Begin by opening the Object/Layer Panel assistant. In a design layout view, choose **Window > Assistants > Object/Layer Panel**.
2. Click the **LPPs** tab.

   (Optional) To view more LPPs at once, you can resize the Object/Layer Panel by adjusting its width so that the LPPs are listed in several visible columns.

Feedback

3. Scroll the list to locate the LPP, or filter the list by entering characters in the **Filter** box. See Locating LPPs, Layers, and Objects for help locating the LPP.
4. (Optional) From the toolbar menu , make sure that **Auto Redraw** is enabled. This option automatically redraw the canvas objects when changes are made within the Object/Layer Panel.
5. Select the LPP by name.

   This sets the active LPP and makes it visible and selectable. There is always only one active LPP.

   Note:

   The active layer purpose pair (LPP) must be visible as you create objects. For more information about using active LPP on creation, see Using Active Layer Purpose Pair in the *Custom Compiler Layout Editor User Guide for Creating and Editing Objects*.

   (Optional) To select or deselect more than one LPP, use Ctrl-click to select or deselect each LPP individually.

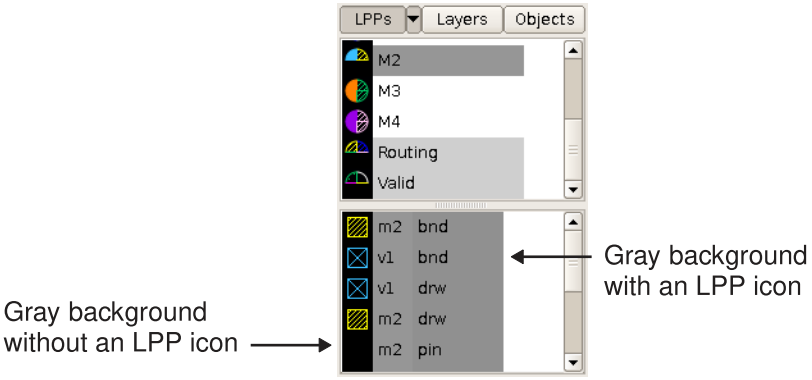
   (Optional) To select multiple LPPs, use Shift-click or drag-left to quickly select them.
6. From the toolbar menu, choose any of the following options:
   - **Make Visible LPPs Selectable**: Makes all currently visible LPPs selectable.
   - **Keep Active LPP Selectable**: Makes the active LPP selectable.
   - **Keep Active LPP Visible**: Makes the active LPP visible.

   (Optional) Use Shift+right-click to make an LPP under the cursor position visible and other LPPs invisible. Next click makes the LPP invisible and other LPPs visible.

   (Optional) Use Ctrl+middle-click to toggle the stipple visibility of an active LPP.

Feedback

The Object/Layer Panel shows each LPP with an icon and indicates the visibility and selectability settings of each LPP as follows:



- Gray background indicates that the LPP is set to be unselectable
- LPP icon disappears if the LPP is set to be invisible (and also unselectable)

Making LPP Sets Visible and Selectable

You can make LPP sets visible and selectable to facilitate the design process. You can also enable and disable LPP fill patterns.

By default, the PR boundary, snap boundary, and global wide alignment grids are listed in the **Objects** tab when you select the **Design** LPP set. To control the visibility and selectability of these boundary objects along with the LPPs, see Controlling the Visibility and Selectability of Boundary Objects.

To control the visibility and selectability of LPP sets:

1. Begin by opening the Object/Layer Panel assistant. In a design layout view, choose **Window > Assistants > Object/Layer Panel**.
2. Click the **LPPs** tab.

   (Optional) To view more LPP sets at once, you can resize the Object/Layer Panel by adjusting its width so that the LPP sets are listed in several visible columns.
3. (Optional) From the toolbar menu , make sure that **Auto Redraw** is enabled. This option automatically redraw the canvas after making changes.
4. Select the LPP set, either a built-in LPP set or a user-defined LPP set, you want to apply.

Feedback

Note:

The active layer purpose pair (LPP) must be visible as you create objects. For more information about using active LPP on creation, see Using Active Layer Purpose Pair in the *Custom Compiler Layout Editor User Guide*.

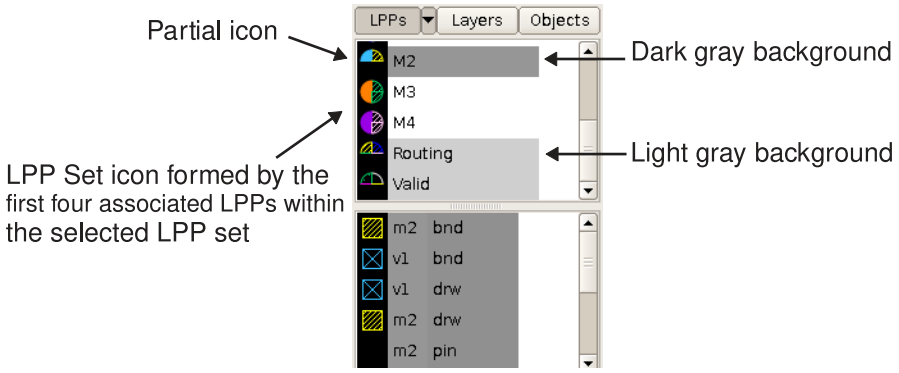
(Optional) To select or deselect more than one LPP set or LPP, use Ctrl+click to select or deselect each LPP set or LPP individually.

(Optional) To select multiple LPP sets or LPPs, use Shift+click or drag-left to quickly select them.

5. Right-click and apply the operation of your choice to the selected LPP set:
   - Set the selected LPP set(s) to be visible or invisible.
   - Enable or disable fill. Disabling fill ignores the fill style defined in the associated packet, and renders the packet with a color outline, but no fill pattern. This action does not change the resource definition for the packet.
   - Set the selected LPP set(s) to be selectable or unselectable.
   - Remove (deselect) the currently selected LPP set(s).

The Object/Layer Panel shows each LPP set with an icon constructed from the packets of the first four LPPs within the LPP set (Figure 44), and indicates the visibility and selectability settings of each LPP set as follows:

Figure 44 LPP Set Icons and Visibility/Selectability Status



- LPP Set icon depicts the packets of the first four LPPs within the LPP set whose purpose is one of these defined in the `leOLPDefaultPurposeList` preference. If fewer than four LPPs remain, they are repeated until the circle is complete. Fill style, outline, and fill colors are all depicted in the pie slice (Figure 44).

  Full LPP Set icon indicates that all the LPPs within the LPP set are fully visible.

Feedback

Partial LPP Set icon indicates that the LPP set contains invisible LPPs.

LPP Set icon disappears if all the LPPs within the LPP set are completely invisible.

- LPP sets depict the selectable state with an alternate background color.

  White background indicates that all the LPPs within the LPP set are selectable.

  Light gray background indicates that the LPP set contains unselectable LPPs.

  Dark gray background indicates that all the LPPs within the LPP set are completely unselectable.

Showing Abbreviations and Stream Layer Numbers

To show abbreviations and stream layer numbers:

1. Begin by opening the Object/Layer Panel assistant. In a design layout view, choose **Window > Assistants > Object/Layer Panel**.
2. From the **LPPs** tab menu, make sure that **Abbreviate Names** is on.
3. Hover over an LPP. The tool shows a tooltip that contains the full layer name and purpose name (abbreviated or unabbreviated), the layer and purpose numbers, and the layer and purpose descriptions.

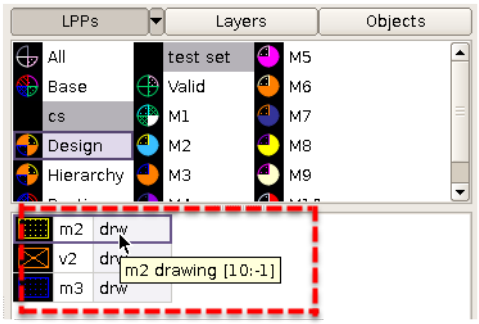
   Note:

   The purpose description is displayed only if it is specified in the `.tech` file.

   For example, a tooltip may contain the text *metal1 layer description \n drawing purpose description [35:-1]*.

   Note:

   If the preference `leLPPShowStreamLayerNumbers` is true, and a stream mapping file is available, the layer and purpose numbers are automatically mapped.



4. Click the **LPPs** tab menu and choose **Show Stream Layer Numbers**. If a stream mapping file is available, the tool automatically maps the layer and purpose numbers and shows them in a tooltip.

Changing the Drawing Priority of a Layer Purpose Pair

To change the drawing priority of an LPP, relative to other LPPs:

1. Begin by opening the Object/Layer Panel assistant. In a design layout view, choose **Window > Assistants > Object/Layer Panel**.
2. Drag one or more selected LPPs and drop toward the location where they should move.

Sorting Layer Purpose Pairs

You can sort all LPPs to easily access the ones you need frequently.

To sort LPPs:

1. Begin by opening the Object/Layer Panel assistant. In a design layout view, choose **Window > Assistants > Object/Layer Panel**.
2. From the toolbar menu , choose **Sort LPPs**, and then choose a sorting operation:
   - **Priority** lists LPPs in priority drawing order.
   - **Alphanumeric** lists the LPPs in this order.
   - **Layer Number** lists the LPPs by layer, and purpose number as defined in the technology database.

Controlling the Visibility and Selectability of Boundary Objects

Setting the `leOLPLPPsForBoundaries` Tcl preference to *true* allows you to list boundary objects in the **LPPs** tab and remove them from the **Objects** tab.

Feedback

You can sort all LPPs to easily access the ones you need frequently.

To view and select the boundary objects in the **LPPs** tab:

1. Begin by opening the Object/Layer Panel assistant. In a design layout view, choose **Window > Assistants > Object/Layer Panel**.
2. Click the **LPPs** tab.
3. When the preference is set, the Object/Layer Panel assistant lists the following objects in the **LPPs** tab when you select the **Design** LPP set:
   - PR boundary: Listed as `{prBoundary boundary}`
   - Snap boundary: Listed as `{snap boundary}`
   - Global Wide alignment grid: Listed as `{grid object}`

   The mentioned LPPs are in the **Design** LPP set if the current edit design contains corresponding objects.

   When listed in the **LPPs** tab, the boundary objects no longer appear in the **Objects** tab in the Object/Layer Panel.

System LPPs

A system layer-purpose pair (LPP) is a layer-purpose pair that is owned by the system, not by the design or technology.

System LPPs have the following characteristics:

- Do not appear on the **Object/Layer Panel**.
- Not listed on the Layer-Purpose Pair Selector.
- Not returned by the `de::getLPPs` command unless you specify `-includeSystem` is `true`.
- Cannot be modified by any user interface control for layer-purpose pairs, including `Set All Visible` and `Set All Selectable`.
- Cannot be set as active using commands such as `de::setActiveLPP`.

To designate a layer-purpose pair as a system LPP, you need to set the system attribute on the `deLPP` object.

The `deLPP` object represents a layer purpose pair (LPP) in a design. You can use this object to modify the validity, selectability, and visibility attributes as shown in the **Object/Layer Panel**. You can obtain a `deLPP` object using the `de::getLPPs` command.

Feedback

Note:

You cannot destroy a `deLPP` object.

For more information, see the following Tcl commands:

- de::createLPPSet
- de::getlpps
- de::setActiveLPP

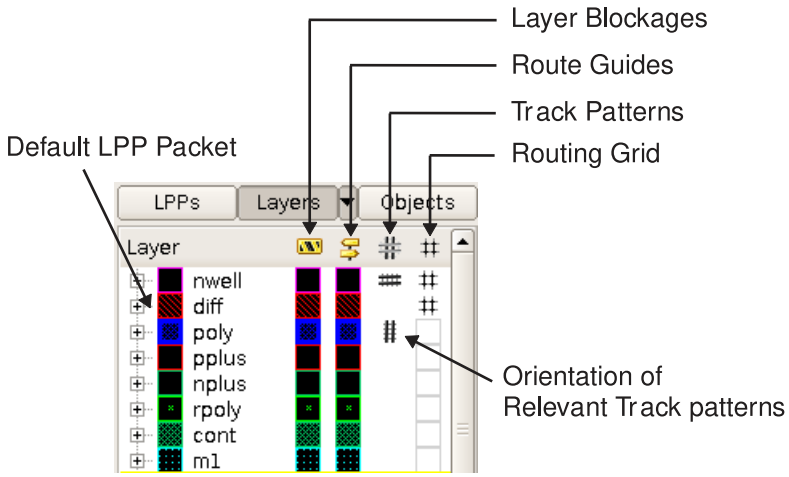
Working With Layers

You can use the **Layers** tab to change the visibility and selectability of layers as well as layer blockages, route guides, track patterns, and the routing grid.

Setting the `leOLPShowLayersTab` Tcl preference to *true* allows you to display the **Layers** tab in the Object/Layer Panel.

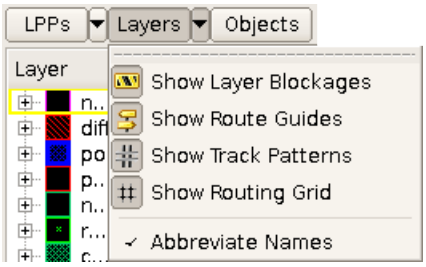
To work with layers:

1. Begin by opening the Object/Layer Panel assistant. In a design layout view, choose **Window > Assistants > Object/Layer Panel**.
2. Click the **Layers** tab.
3. View the columns for layer blockages, route guides, track patterns, and the routing grid.



Show and hide columns by choosing them from the **Layers** tab menu.

Feedback



4. To change whether route guides, routing grids, layer blockages, or track patterns appear on this layer, click one of the columns described in Table 86.

   Use the column headings to toggle options:

   - Left-click to make the layer active.
   - Middle-click to toggle visibility.
   - Right-click to toggle selectability.

   (Optional) To select or deselect more than one layer, use Ctrl-click to select or deselect each layer individually.

   (Optional) To select multiple layers, use Shift-click select them.

   Table 86 describes the settings you can adjust for each layer, by column.

Table 86 Layers Tab Menu Options

Layers Menu Option	Description
Show Layer Blockages	Shows the Layer Blockages column in the **Layers** tab on a per-layer basis, and indicates whether the layer blockage is visible (white background) or invisible (gray background). Tcl preference: `leOLPShowLayerBlockages`
Show Route Guides	For use only with Fusion Compiler and IC Compiler designs, shows the Route Guides column in the **Layers** tab on a per-layer basis, and indicates whether the route guide is active (white background) or inactive (gray background) for this layer. Tcl preference: `leOLPShowRouteGuides`
Show Track Patterns	Shows the Track Patterns column in the **Layers** tab on a per-layer basis, and indicates whether the track pattern is active (white background) or inactive (gray background). Tcl preference: `leOLPShowTrackPatterns`

Feedback

Layers Menu Option	Description
Show Routing Grid	Shows the Routing Grid column in the **Layers** tab on a per-layer basis, and indicates whether the routing grid is active (white background) or inactive (gray background). Tcl preference: `leOLPShowRoutingGrid`
Abbreviate Names	Displays LPPs using their technology-defined abbreviated layer names. Tcl preference: `leLPPAbbreviations`

For any given layer, view and optionally click the row and its columns to change its elements, expanding the tree listing as necessary to view associated LPPs.

A white row background indicates that this layer and all aspects of it are visible. A gray background indicates that this layer is invisible. A light gray background indicates a mixture of invisible and visible layer aspects.

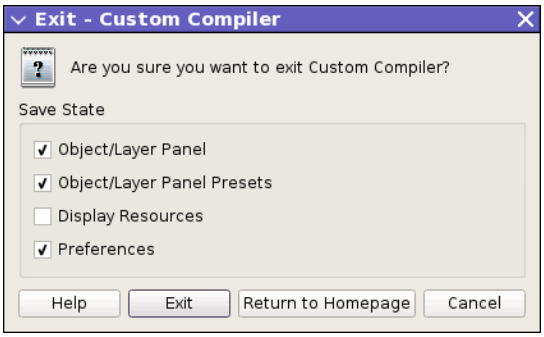
The system shows visibility settings on the **Layers** tab that you have made on the **LPPs** tab. It also copies visibility settings you adjust on the **Layers** tab to the **LPPs** tab.

Working With Objects

You can control the set of objects you want to be visible and selectable to more easily access the design elements you want to change.

Visible objects are selectable when both the LPP and object type and are set to *selectable*. You can manipulate selectable objects with editing commands such as **Delete** or **Move**.

Object type settings in the **Object** tab are saved and loaded through the application state interface defined by `deOLPObjectType.xml` at startup. You will be prompted to save the settings with the **Object/Layer Panel** option when closing the software.



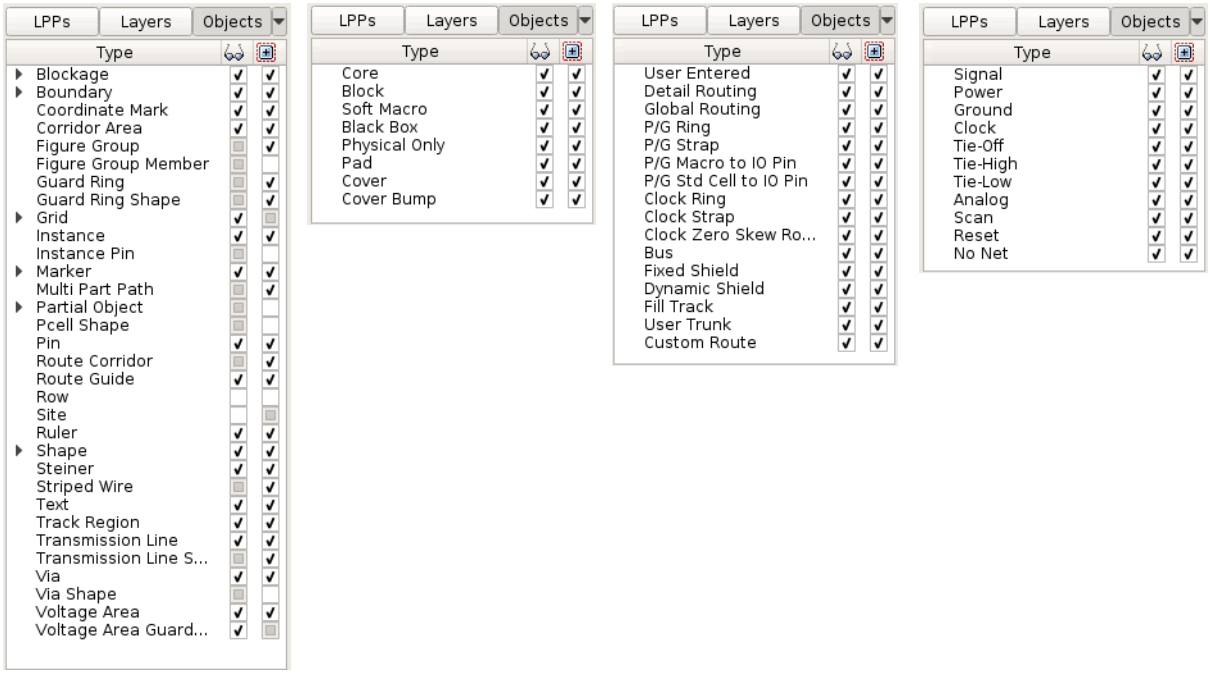
By default, the PR boundary, snap boundary, and wide alignment grids are listed in the **Objects** tab. To control the visibility and selectability of these boundary objects along with the LPPs, see Controlling the Visibility and Selectability of Boundary Objects.

Feedback

To work with objects:

1. Begin by opening the Object/Layer Panel assistant. In a design layout view, choose **Window > Assistants > Object/Layer Panel**.
2. Click the **Objects** tab menu and choose one of the following options:
   - **Object Types**: Shows visible and selectable objects by object types.
   - **Cell Types**: Shows visible and selectable objects by cell types.
   - **Route Types**: Shows visible and selectable objects by route types.
   - **Signal Types**: Shows visible and selectable objects by signal types.

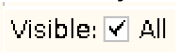
   The following figure illustrates the **Objects** tab contents when switching types (from left to right): Object Types, Cell Types, Route Types, and Signal Types.



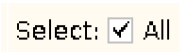
3. Act on the objects.

Feedback

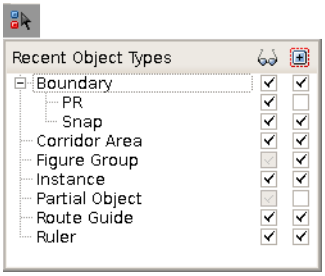
- Set individual objects to be visible by check marking them by name in the Visibility column.
- Set all objects to be visible by selecting the **Visible** box on the toolbar:

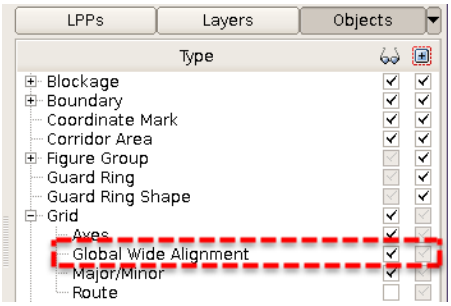
- Set individual objects to be selectable by check marking them by name in the Selectability column.
- Set all objects to be selectable by selecting the **Select** on the toolbar:

- Use the quick object selector icon to select from the list of the ten most recently selected objects.



For example, to display global fin grids, display **Object Types**, expand **Grid**, and then select **Visibility All** for **Global Wide Alignment**.



Sharing OLP Settings

By turning on the **Share OLP Settings** option, the Object/Layer Panel automatically synchronizes the filters of LPPs and object types between designs which share a common technology.

Design Settings are defined as an ordered list of LPPs with their validity, selectability, visibility, and fill settings, together with a set of object types with their visibility and selectability settings and the active LPP. One design setting exists for each design opened by the design editor.

Feedback

Interactive commands may further restrict the selection filters by providing additional per-type, per-LPP, and per-object selection filters that are applied in addition to the design's selectability registry. Thus, before selecting an object, both the design and active interactive command's filter set must be queried. Inactive commands in the command stack do not affect the selectability.

When a design is opened, the technology may be modified. Typically this happens when an LPP is encountered in the design which does not exist within the technology. In this case, the LPP is added to the technology by the technology manager with some default settings. The Object/Layer Panel is not responsible for updating the technology but it is responsible for showing LPPs added with this method.

By default, modifications made within the Object/Layer Panel are reflected back in only the settings for the edit design. However, by turning on the **Share OLP Settings** option or by setting the `deShareOLPSettings` preference to *true*, these changes are automatically applied to all designs which share a common technology.

By turning off the **Share OLP Settings** option, you can modify multiple design settings when the designs are explicitly linked with a linked design context.

To enable synchronization to other design contexts, the linked design context must have the `leSyncOLP` capability applied. This capability indicates to the Object/Layer Panel that the edit design of the linked design context should be modified at the same time as the edit design is modified.

Note:

This is a one-way synchronization.

This does not require that the two designs share the exact same technology. The Object/Layer Panel attempts to apply changes to the same-named LPPs in the opposite design and ignores any errors.

The `leOLPAutoRedraw` preference is supported for redrawing all affected design contexts.

When a design is opened, depending on the `deShareOLPSettings` preference value, the design's OLP settings are read from either another design or the design's referenced technology or the appropriate Technology Settings (if `deShareOLPSettings` preference value is *true*). When using Design Settings and descending into a design which was not formerly open from another design with the same technology, the parent's OLP settings are copied to the newly opened design. These settings are not copied back to the parent on return, but only used as defaults for the newly opened design.

Feedback

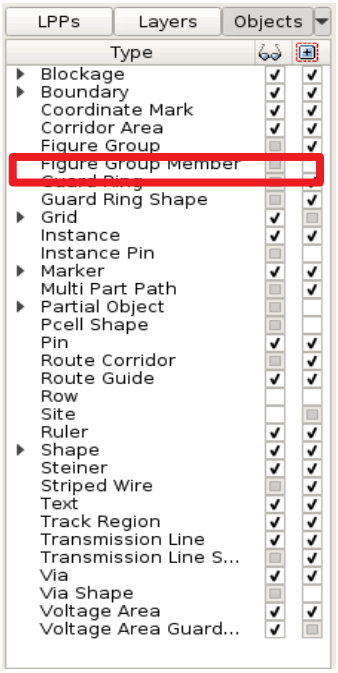
Controlling Area Selections of Figure Groups

You can specify whether the Layout Editor selects all synchronous peer members or just the parent object of a figure group during an area selection. You can specify your preference in the Object/Layer Panel **Objects** tab.

In the Object/Layer Panel **Objects** tab, locate **Figure Group Member**, and then:

- Select (**Selectability**) if you want to select all synchronous peer members during an area selection.
- Clear the (**Selectability**) check box to select only the parent object of a figure group during an area selection.

By default, (**Selectability**) is not selected.



Caution:

Commands that modify the selection can cause members to become desynchronized. To maintain synchronization, you must edit in group when making modifications.

Feedback

Working With Display Resources

From the Object/Layer Panel, you can open the Display Resource Editor tool for the current design to view and edit the display packets associated with LPPs and then reload the display resource file settings to the Object/Layer Panel.

To work with the display resources:

1. Begin by opening the Object/Layer Panel assistant. In a design layout view, choose **Window > Assistants > Object/Layer Panel**.
2. Click the toolbar menu and choose **Display Resource Editor** to open the Display Resource Editor tool for the current design.

   See Display Resource Editor Tool for help using the tool.
3. Click the toolbar menu and choose **Reload Display Resource** to reload the display resource file settings to the Object/Layer Panel.

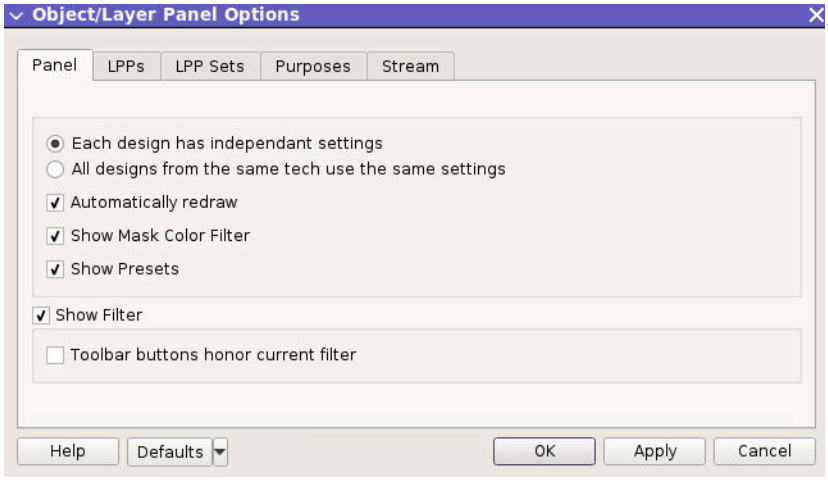
Setting OLP Options

From the Object/Layer Panel, you can open the Object/Layer Panel Options dialog box for the current design to view and edit the Object/Layer Panel settings.

To enter the **Object/Layer Panel** option settings:

1. Open the **Object/Layer Panel** assistant. In a design layout view, choose **Window > Assistants > Object/Layer Panel**.
2. Click the toolbar menu and choose **OLP Options**. The **Object/Layer Panel Options** dialog box opens in the **Panel** tab.

Feedback



3. Enter the **Panel** options as described in Table 87.

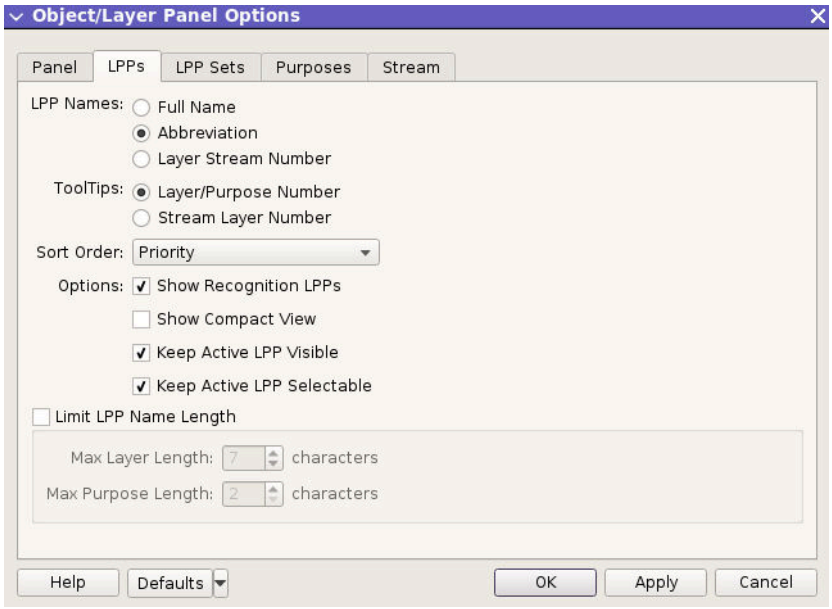
Table 87 OLP Options Panel Settings

Setting Option	Description
Each design has independent settings	Applies the OLP settings to individual designs
All designs from the same tech use the same settings	Applies the OLP settings to all designs from the same technology
Automatically redraw	Toggles the option to automatically redraw the canvas objects when changes are made within the Object/Layer Panel. (On by default). Preference: `leOLPAutoRedraw`
Show Mask Color Filter	Toggles the display of the **Color** toolbar in the Object/Layer Panel. (On by default). See Displaying Color Annotations for help using this option. Preference: `leOLPShowColorToolbar`
Show Presets	Toggles the display of the **Presets** toolbar in the Object/Layer Panel. (On by default). See Displaying Presets for help using this option. Preference: `leOLPShowPresets`
Show Filter	Displays the **Filter** box in the Object/Layer Panel. (On by default).

Feedback

Setting Option	Description
Toolbar buttons honor current filter	Controls whether the Set All Selectable and Set All Visible boxes honor the valid and filtered LPPs or Object Filters, depending on the current tab. (Off by default). Preference: `leOLPButtonsHonorFilter`

4. Switch to the **LPPs** tab.



5. Enter the **LPPs** options as described in Table 88.

Table 88 OLP Options LPPs Settings

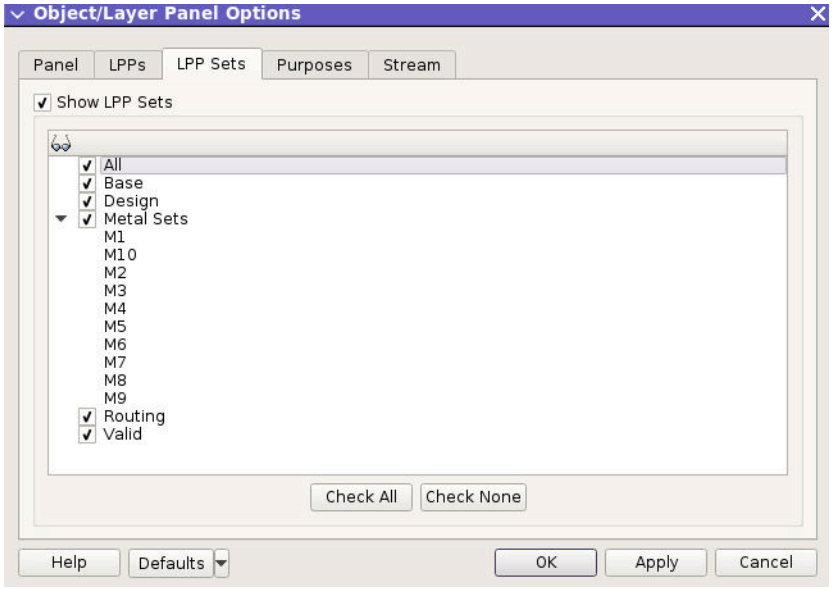
Setting Option	Description
LPP Names	
Full Name	Displays LPPs using their full names. (Off by default).
Abbreviation	Displays LPPs using their technology-defined abbreviated layer names. (On by default). Tcl preference: `leLPPAbbreviations`
Layer Stream Number	Displays LPPs using their layer stream number names. (Off by default).
ToolTips	

Feedback

Setting Option	Description
Layer/Purpose Number	If the layer mapping file is not found, the tool checks if the purpose is system reserved and if there is a mapping for the LPP and then uses it as the stream layer or data type number. **Note:**
Stream Layer Number	The tool shows the stream layer and data type numbers in a tooltip, rather than showing the OpenAccess layer and purpose numbers, if the layer mapping file is found. Tcl preference: `leLPPShowStreamLayerNumbers`
Sort Order	Sorts the LPPs in the lower pane by **Priority** (default), **Alphanumeric**, and **Layer Number**. See Sorting Layer Purpose Pairs for help on sorting the LPPs. Tcl preference: `leOLPSortLPPs`
Options	
Show Recognition LPPs	Toggles the display of layers defined as color recognition layers. (On by default). Preference: `leShowRecognitionLpps`
Show Compact View	Shows more LPP sets and LPPs at a time. By default, the tool uses a 24 pixel row height for each table in the Object/Layer Panel. When enabled, a 20 pixel row height is used instead. Tcl preference: `leOLPCompactView`
Keep Active LPP Visible	Makes the active LPP visible. Preference: `leOLPKeepActiveLPPVisible`
Keep Active LPP Selectable	Makes the active LPP selectable. Preference: `leOLPKeepActiveLPPSelectable`
Limit LPP Name Length	Limits the number of characters of LPP names using **Max Layer length** and **Max Purpose Length**.

6. Switch to the **LPP Sets** tab.

Feedback



7. Select the **Show LPP Sets** check box to display the built-in LPP sets as shown in Table 89.

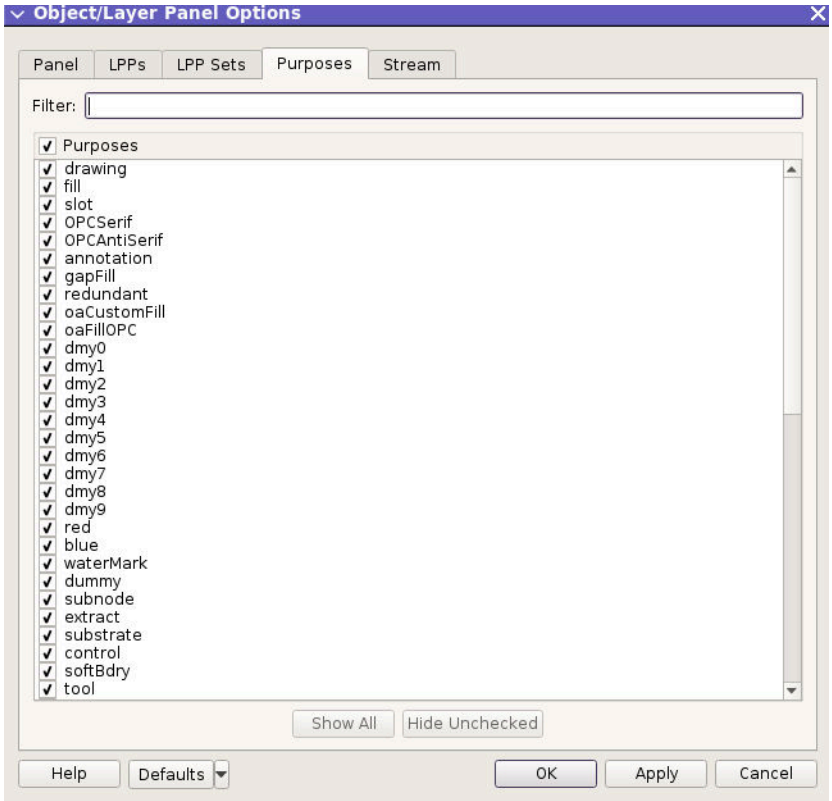
Table 89 Built-in LPP Sets

Built-in LPP Set	Description
All	Contains all purposes of all LPPs in the design's technology. The default LPP is the first contained LPP.
Base	Contains the base layers that typically exist within devices, rather than between devices. This includes all LPPs from the technology whose material is one of the following: nWell, pWell, nDiff, pDiff, nImplant, pImplant, poly, diffusion, and contactlessMetal. The default LPP is undefined.
Design	Contains all LPPs used in the view design. When performing a descend edit/read or Edit-In-Place (EIP), the Object/Layer Panel changes its contents to show all LPPs used in the open child design. The default LPP is undefined.

Feedback

Built-in LPP Set	Description
Metal Sets (M1...Mn)	Contains all LPPs on the metal layer as well as the cut layers which connect directly to this metal layer. One LPP set exists for each technology metal layer, as indicated by the material type on the layer object. The name of the set is the full (unabbreviated) name of the metal layer, with the first letter capitalized to match other sets. The default LPP is identified by matching the purpose against `leOLPDefaultPurposeList`.
Routing	Contains all purposes of all routing layers that are used for the current design. Only valid routing layers are displayed. The default LPP is undefined.
Valid	Contains all LPPs that are valid in the technology. This is the default set in all new designs. The default LPP is undefined.

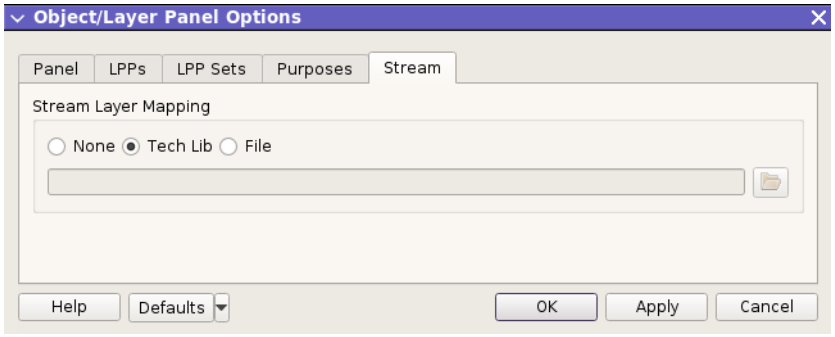
8. Switch to the **Purposes** tab.



9. The Object/Layer Panel does not show the LPPs whose purpose is listed in the `leOLPHidePurposes` preference. To customize the display of LPPs in the Object/Layer Panel, you can show or hide LPPs with specified purposes.

   You can use the **Filter:** filter box to locate the LPPs. Click **Show All** to display all purposes in the dialog box. Click **Hide Unchecked** to hide the unchecked purposes in the dialog box.

10. Switch to the **Stream** tab.



11. Select the **Stream Layer Mapping** file, if available, using **None**, **Tech Lib** (default), or **File**.

Probe Assistant

You can use the Probe assistant to view probes in the design or in a currently selected set of objects. This assistant displays probes visually through object highlighting. You can use the Probe assistant to find and cycle through probed objects, to sort the list of probed objects, and to view text details for each one.

This topic describes the following:

- Opening the Probe Assistant
- Cycling Through Probed Objects
- Deleting Probes
- Viewing Probes in Shadow
- Viewing Probed Object Details

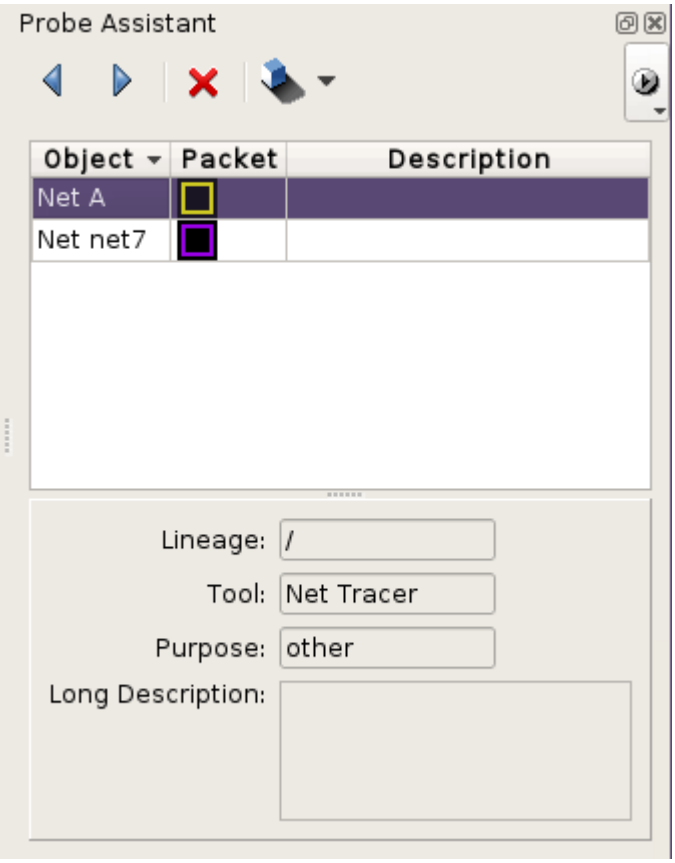
Opening the Probe Assistant

Open the Probe assistant to find and cycle through probes and their objects in any probed design context.

Feedback

To open the Probe assistant:

► In any design, choose **Window > Assistants > Probe Assistant**.



Probes attach to objects both when designers add them manually and when design analysis processes add them automatically.

The assistant docks along the window edge. Undock it by clicking and dragging it.

Cycling Through Probed Objects

You can cycle through probed objects in a design to view them individually.

To cycle through probed objects:

1. Begin by opening the Probe assistant. In any design, choose **Window > Assistants > Probe Assistant**.
2. In the list of probes in the Probe assistant, select a probe by name.

Feedback

The assistant browses to see the selected probe. If the **Zoom** option is checked in the toolbar menu, the assistant also zooms in to where it is.

3. To cycle through all probed objects in the design or selected set, click the toolbar arrows.

Deleting Probes

You can select probes and then remove them from your design.

To delete probes from the design canvas:

1. Begin by opening the Probe assistant in the design context with probes you want to view. In any design, choose **Window > Assistants > Probe Assistant**.
2. In the list of probes in the Probe assistant, select the probe by name.
3. In the Probe assistant toolbar, click the **Delete** button below or press the Delete key.

Viewing Probes in Shadow

You can view design areas in shadow for additional visual contrast between selected probes and the surrounding design.

1. To view design areas in shadow:

   Click the **Shadow** button on the Probe assistant toolbar. Click the arrow and move the slider to increase or decrease the contrast.

2. (Optional) Set the `deProbeHighlightShadow` Tcl preference with the following value for probing operations:
   - `true`: Turns on the Shadow mode to increase the contrast between the objects you are probing and the remaining objects.
   - `net`: Turns on the Shadow mode to increase the contrast between the nets you are probing and the remaining nets.
   - `false`: Turns off the Shadow mode.

Feedback

Viewing Probed Object Details

You can view probed object details to obtain information about object names, packet colors, and short descriptions entered when the probes were created.

To view probed object details:

1. Begin by opening the Probe assistant in the design context with probes you want to view. In any design, choose **Window > Assistants > Probe Assistant**.
2. In the list of probes in the Probe assistant, select a probe by name.

   The system browses to the design where the probed object is, and displays details about this probe. If the **Zoom** option is checked in the toolbar menu, the assistant also zooms to fit the object.

   The Probe assistant shows the lineage path to the probed object, the tool associated with this probe (signifying how the probe was created), and the purpose, and long description, entered when this probe was created.
3. To sort by object, packet, or description, select the column titles.
4. Click the toolbar menu to see the available viewing options.

Menu Item	Action
Auto Refresh	Automatically refreshes the design context. Tcl preference: `deProbeAutoRefresh`
Zoom	Zooms to each probe when selected. (On by default). Tcl preference: `deProbeAssistantZoomToProbe`
Hierarchy	By default (hierarchy disabled), the table displays only the probes visible in the current editor context. When the **Hierarchy** option is on, the table lists all probes in the design hierarchy. (Off by default). Tcl preference: `deProbeAssistantHierarchy`
Filter by Selection	Shows probes filtered by selection in the Probe Assistant. Tcl preference: `deProbeAssistantFilterBySelection`

Feedback

Menu Item	Action
Type	Filters the probe list by object type. Check the object types you want to include from **All**, **None**, **Cellview**, **Instance**, **Instance Terminal**, **Net**, **Path**, and **Terminal**. Tcl preferences: `deProbeAssistantShowCellViewType`, `deProbeAssistantShowInstanceTerminalType`, `deProbeAssistantShowInstanceType`, `deProbeAssistantShowNetType`, `deProbeAssistantShowPathType`, `deProbeAssistantShowTerminalType`
Tools	Filters the probe list by tool. Tcl preference: `deProbeAssistantTools`
Help	Opens the online help documentation for this topic.

Property Editor Assistant

You can use the Property Editor assistant to create and edit properties, parameters, and attribute values of selected objects. You can also view connection assignments, instance terminal connectivity, and attributes for colored shapes in advanced node designs.

- About the Property Editor
- Setting Editor Annotation Options
- Displaying the Property Editor
- Opening the Properties Dialog Box
- Viewing Design Object Properties
- Editing and Deleting Design Properties
- Creating New Design Properties
- Enabling Zoom Operations
- Adjusting Parameters in the Selected Set
- Resetting to Default Parameter Values
- Instance Terminal Region
- Property Editor Toolbar

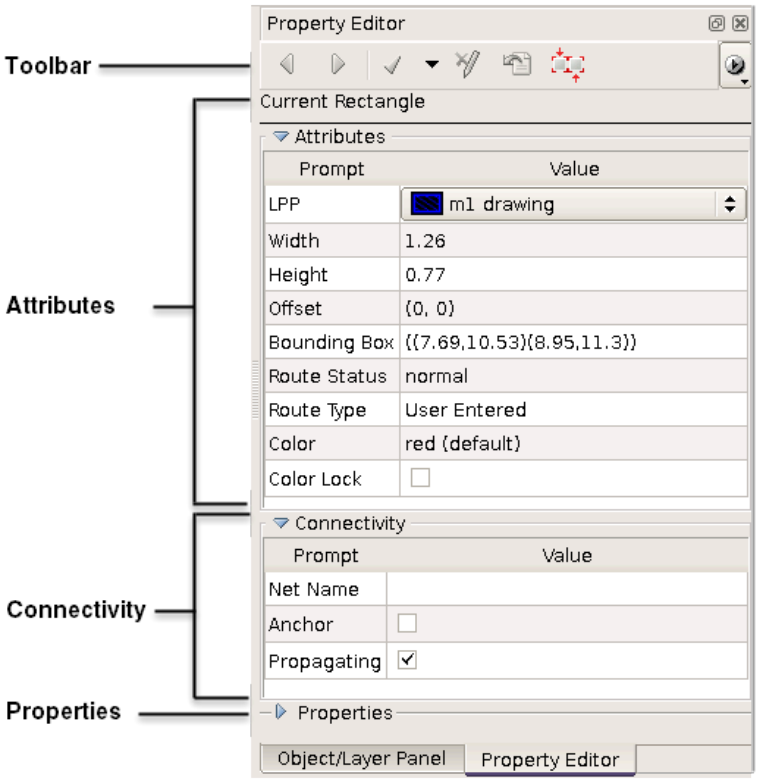
About the Property Editor

You can use the Property Editor to:

- Display the properties of selected objects.
- Edit the properties and parameters of selected objects.
- Create new properties to be associated with the selected objects.

Feedback

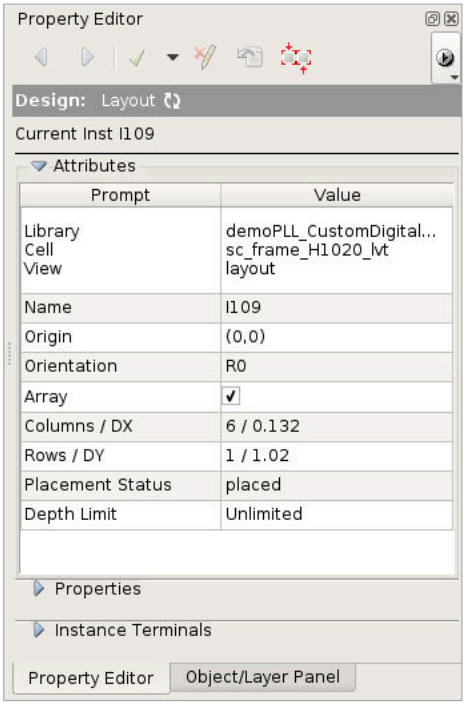
The Property Editor consists of the following:



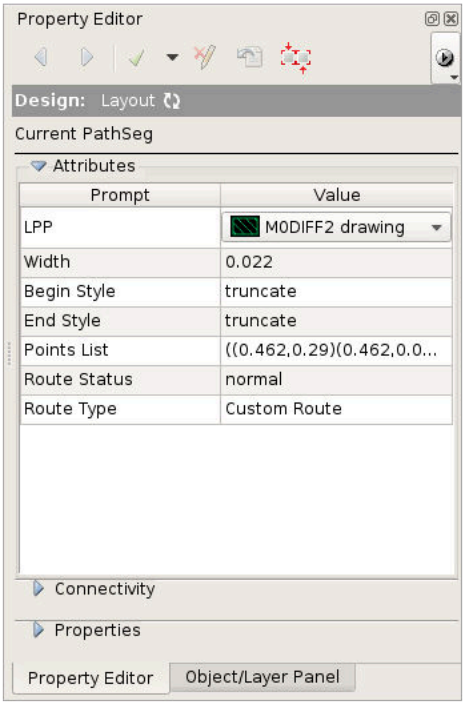
The contents of the Property Editor depends on the objects you select in the design. For example, if you select an instance, the properties of the instance are displayed. If you select a path segment, the properties of the path segment are displayed.

Feedback

Instance



Path Segment



For more information, see Viewing Design Object Properties. For the descriptions of the toolbar buttons, see Property Editor Toolbar.

Setting Editor Annotation Options

The attributes, parameters, and properties in the Property Editor can have different font colors and styles to indicate the following:

Font Color/Style	Description
Gray	The value is read-only.
Italic	The value is inherited. For example, PCell instances might have inherited values defined by the PCell master.
Bold	The value was modified but not yet applied.

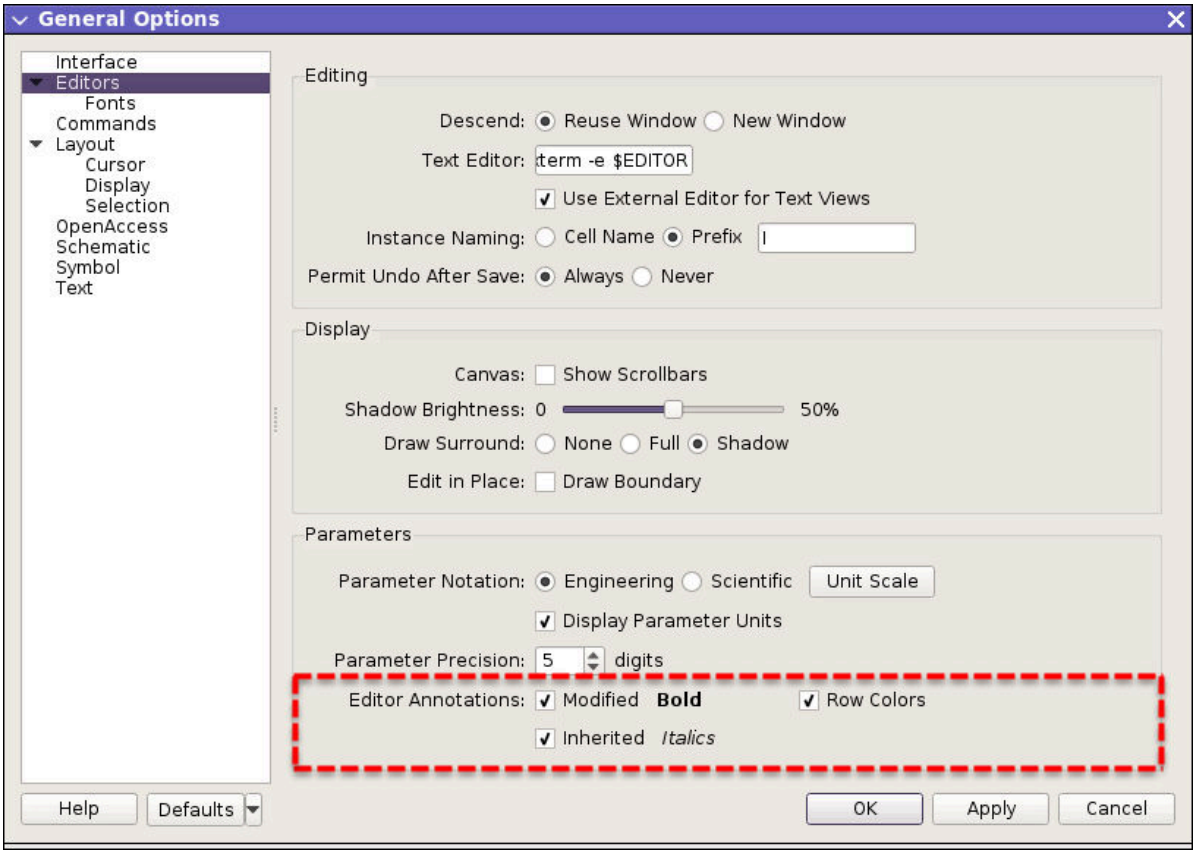
To turn on visual indicators for inherited and modified values:

1. In the design editor, choose **Options > General**.
2. In the **General Options** dialog box that opens, click **Editors**.

Feedback

3. In the **Editor Annotations** section, select the visual indicator options that you want to turn on.

Option	Description
Modified: Bold	Displays modified values in bold font. Tcl preference: `dbPropertyEditorAnnotateModified`
Inherited: Italics	Displays inherited values in italic font. Tcl preference: `dbPropertyEditorAnnotateInherited`
Row Color	Displays rows in alternating colors (white and gray). Tcl preference: `dbShowParamRowColors`



Feedback

Displaying the Property Editor

You can view the object properties using either the Property Editor or the **Properties** dialog box. The Property Editor and the **Properties** dialog box contain the same information. The main differences are:

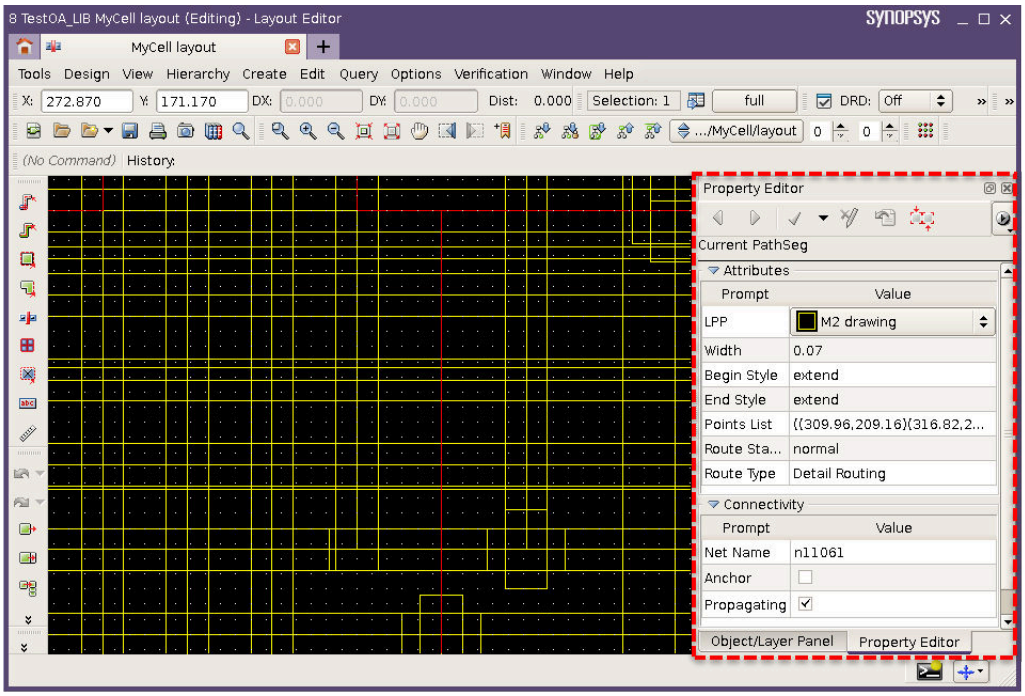
- The Property Editor presents information in the same pane.
- The **Design Properties** dialog box presents information in different tabs. See Opening the Properties Dialog Box.

To display the Property Editor:

► In any design window, choose **Window > Assistants > Property Editor**.

The Property Editor appears on the right side of the Layout Editor. You can undock it by dragging it to a different location.

Notice that the text above the **Attributes** section indicates the object you selected. In the following example, the text above the **Attributes** section indicates that a path segment is selected in the design.



Feedback

Opening the Properties Dialog Box

You can view the object properties using either the Property Editor or the **Properties** dialog box. The Property Editor and the **Properties** dialog box contain the same information. The main differences are:

- The **Properties** dialog box presents information in different tabs in an independent dialog box.
- The Property Editor presents information in the same pane. By default, the pane appears on the right side of the Layout Editor. See Displaying the Property Editor.

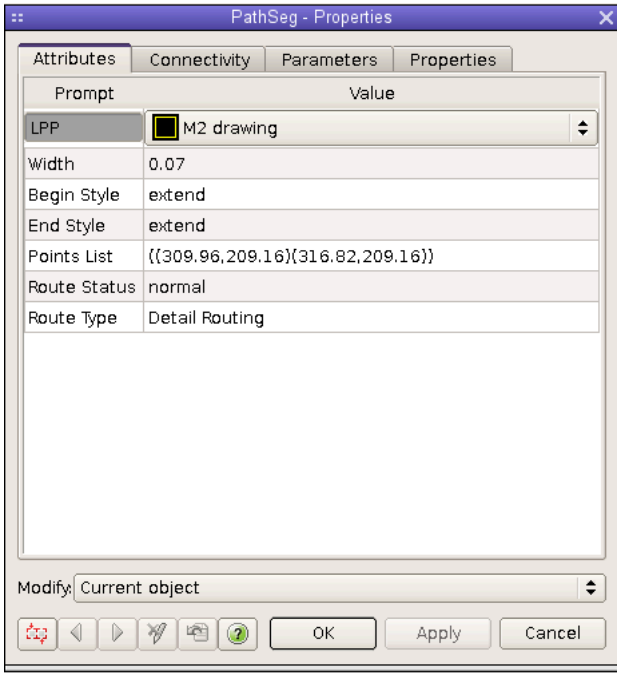
To open the Properties dialog box on a one-time basis:

1. In the Layout Editor, choose **Window** > **Console**.
2. In the **Console** pane, type the following at the prompt:

   ```
   > de::showProperties
   ```

   The **Properties** dialog box opens. Notice that the text in the title bar indicates the object you selected. In the following example, the title bar indicates that a path segment is selected in the design.

   If you click **Show Multiple Item View** ( ), the **Common** text appears next to the selected objects on the title bar.



Feedback

To configure the Layout Editor so that the **Window** > **Assistants** > **Property** Editor command opens the **Properties** dialog box instead of the Property Editor by default:

1. Use a text editor to open the `~/.synopsys_custom.tcl` file.
2. Add the following line in the file:

   ```
   gi::createBinding -event q -command de::showProperties -windowType
   leLayout
   ```

   Instead of `leLayout`, you can specify `seSchematic` or `seSymbol`.

Viewing Design Object Properties

You can view the properties of selected design objects. While viewing the properties, you can edit, delete, or create new properties. See Editing and Deleting Design Properties.

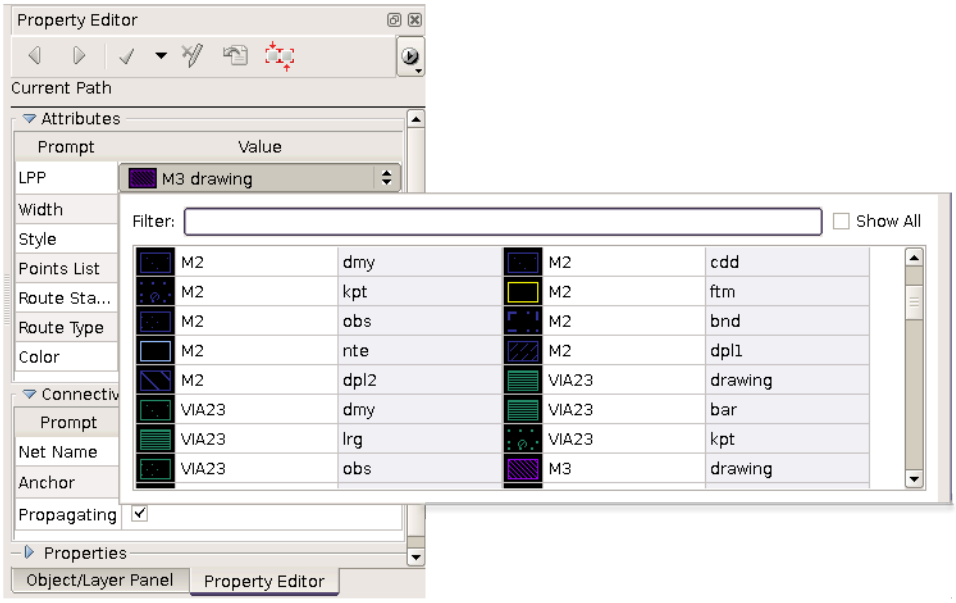
Before you start, set the editor annotations if you want visual indicators for the source or status of the property values. See Setting Editor Annotation Options.

	The video demonstrates how to select multiple schematic instances and edit their attributes in the **Property Editor** in the Custom Compiler environment.

To view object properties:

1. Begin by Displaying the Property Editor or Opening the Properties Dialog Box.
2. Select one or more objects on the canvas. To see properties for the design itself, click anywhere on the canvas.

   If you select pre-colored shapes, the Property Editor lists their color attributes and indicates whether the color is a color anchor or has been propagated from another shape.
3. In the Property Editor, review details available for the currently selected set.
   - If you selected one object, information about that object is displayed. To see additional details about that object's properties (or, for example, its parameters or attributes if that information is available), expand each section.
   - For some objects that are associated with layers, you can filter and search for the layer of interest.



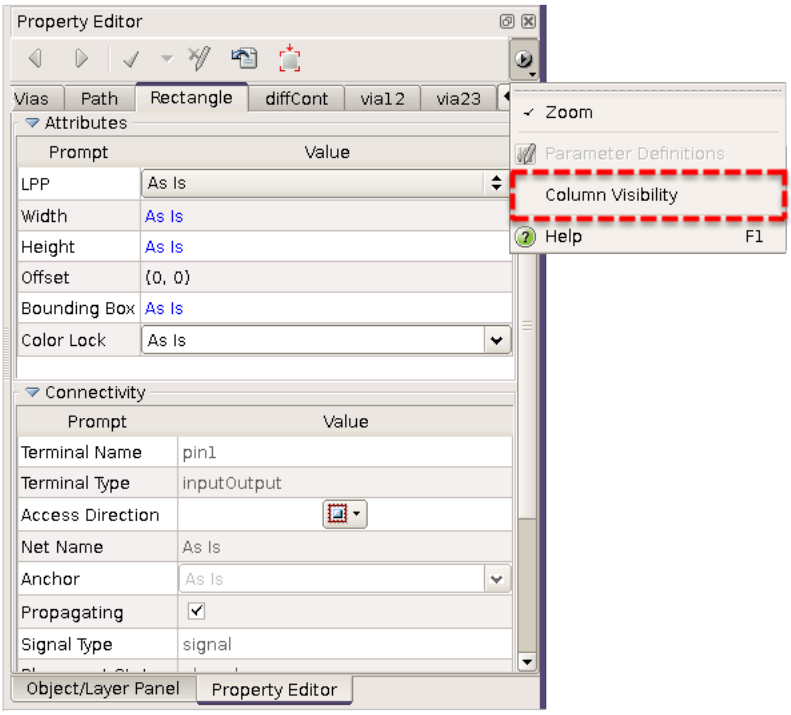
- If you selected a group of objects, you can cycle through them using the **Show Previous Object** and **Show Next Object** arrows on the assistant's toolbar. The assistant shows details for one object at a time, and simultaneously delineates the object's shape on the canvas.
- If you selected a group of objects, and you want to use tabs for quicker access to details about each one, click **Show Multiple Item View** ( )on the assistant's toolbar.

  If the `dePropertyEditorShowCommonAllRefs` Tcl preference is set to `true`, then the Property Editor will display the selected PMOS and NMOS instance parameters and properties in the **All Instances** or **All Vias** tab. You can make changes to these instances in the same tab instead of in different tabs.
- If you want to expand the scope of the selected set to include additional items of this object type, choose an expanded scope from the black double-arrow button menu on the assistant's toolbar.

  The assistant shows the currently applied scope below the assistant toolbar, lists the number of objects in the selected set, and shows the current object's position in the set's order.

4. (Optional) Display or hide details about the selected set. To do this, click , and then choose **Column Visibility**.

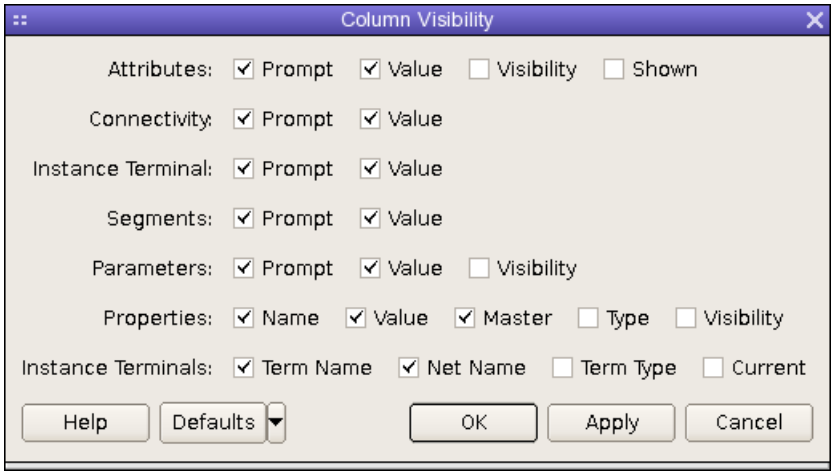
Feedback



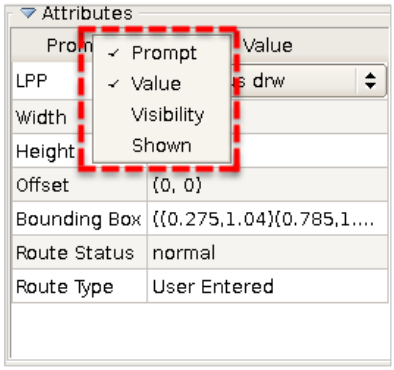
In the **Column Visibility** dialog box that appears, select the information you want to display in the Property Editor.

Note:

The contents of the **Column Visibility** dialog box depend on whether you are viewing the design in the Layout Editor, Schematic Editor, Symbolic Editor, or Netlist Editor. In the following example, the design is viewed in the Layout Editor. A subset of the contents is available when you view designs in the Schematic Editor, Symbolic Editor, and Netlist Editor.



Instead of using the **Column Visibility** dialog box, you can right-click the column heading area, and then choose the information you want to display.



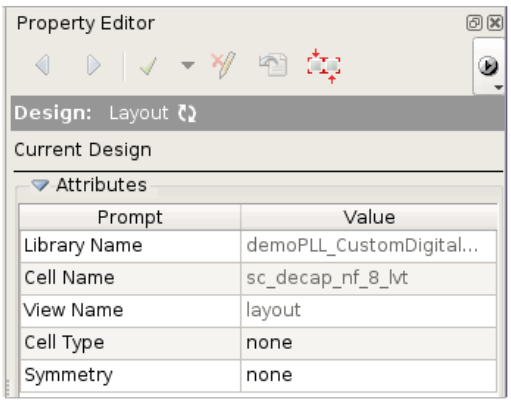
Editing and Deleting Design Properties

Before you start, set the editor annotations if you want visual indicators for the source or status of the property values. See Setting Editor Annotation Options.

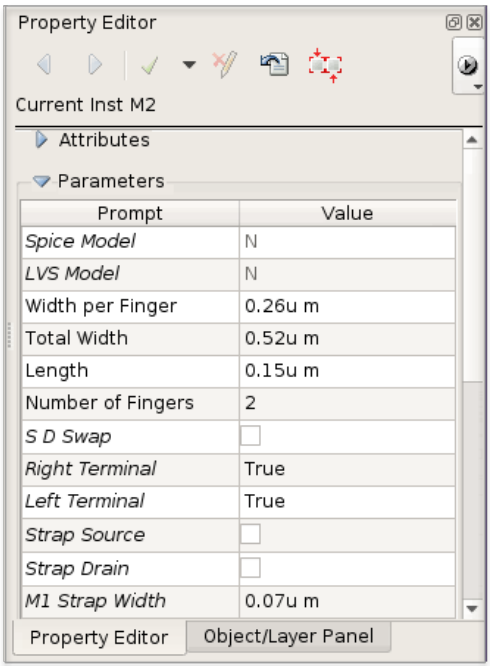
To edit and delete design properties:

1. Begin by Displaying the Property Editor or Opening the Properties Dialog Box.
2. On the canvas, select the object whose properties you want to edit or delete.

   Values that are in gray text are read-only. You cannot edit them. In the following example, the Library Name, Cell Name, and View Name are read-only, so the names are gray.



If the Editor Annotation is turned on, Attributes and Parameters that are italicized are inherited values. In the following example, Inst M2 has inherited several Parameters values from its master cell, such as Spice Model, LVS Model, and so on.



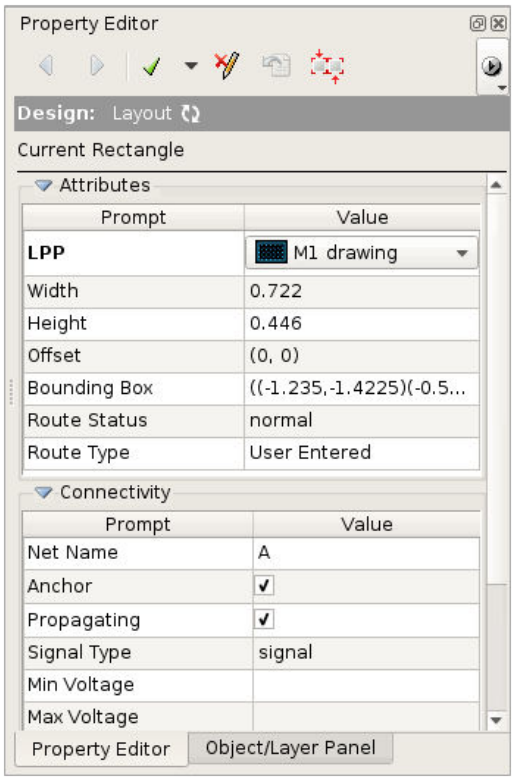
3. Edit the attribute or parameter values. For assistance in showing properties, see Viewing Design Object Properties.

   - To edit a value, type the value or text, select from the menu, or change the check box selection. To move to and edit the next value, press Tab. To edit parameter definitions, see Adjusting Parameters in the Selected Set.

     If the Editor Annotation is turned on, after you edit the values, attributes, and parameters, the property names appear in bold font to indicate that they have

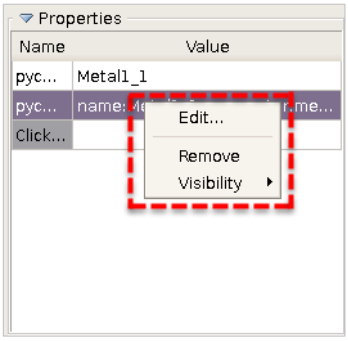
Feedback

been modified but not yet applied. In the following example, the LPP selection was modified and not yet applied.



- To delete a property, select its row, right-click, and choose **Remove**.

  The tool draws a line striking through the entry. When you apply the changes, the tool removes this listing altogether.



- To add new properties now, see Creating New Design Properties.

4. (Optional) Adjust edits before saving the final changes:
   - To return all values to the default, on the Property Editor toolbar click **Reset to Defaults**.
   - To discard the last change, move the cursor to the design window, and choose **Edit > Undo**.
   - To discard all changes, on the Property Editor toolbar click **Discard Changes**.
5. Click **Apply Changes** ( ) when you are done.

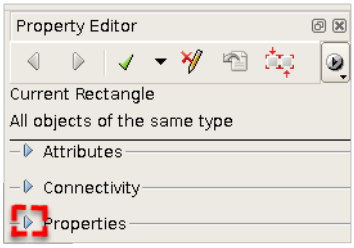
   The tool validates the entries, and reports any errors it finds, by outlining the text boxes with a red border. Move the cursor over an invalid entry to see a tool tip for correcting the error. Adjust entries to create a valid property.
6. (Optional) Discard these applied changes by moving the cursor to the design window, and choosing **Edit > Undo**.

Creating New Design Properties

You can create new design properties to automatically associate them with objects in a selected set.

To create a new design property:

1. Begin by Displaying the Property Editor or Opening the Properties Dialog Box.
2. In the Property Editor, expand the **Properties** region of the assistant using the small arrow button.

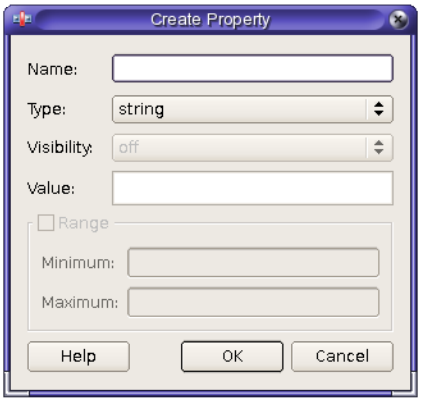


3. Create the property.
   - To create only one property, and its value, single-click the **Click-to-Add** text box. Enter the name and the value. Click the **Apply Changes** button (a green check mark) at the top of the assistant.

     This completes the procedure to create a property.

Feedback

- To create a property with additional details, double-click the **Click-to-Add** text box, which opens a dialog box. (Alternatively, you can right-click this text box, and choose **Add** to open the same dialog box).



4. Define the property you are creating as described in Table 90.

Table 90 Create Property

Definition	Description
Name	Name for this new property that does not conflict with an existing property name
Type	One of these property types: application, Boolean, double, enumeration, float, hierarchical, integer, string, or time
Visibility	Choose **On** to show this property in the Property Editor.
Value	Initial value for this property. You can edit values later. (See Editing and Deleting Design Properties).
Range	For double, float, integer, and time property types, optionally set a range by choosing the range option, and defining the minimum and maximum values for the range. For the enumeration property type, optionally set a range by entering a list of values.

5. Click **OK**.

   The tool validates the entries, and reports any errors it finds, by outlining the text boxes with a red border. Move the cursor over the invalid entry to see a tool tip for correcting the error. Adjust entries to create a valid property.

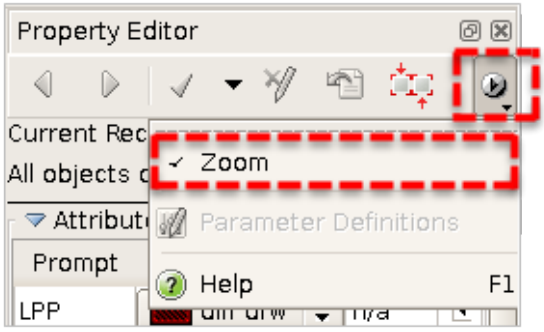
Feedback

Enabling Zoom Operations

You can enable zoom operations to conveniently scale the selected set.

To enable zoom operations:

1. Begin by Displaying the Property Editor or Opening the Properties Dialog Box.
2. From the Property Editor toolbar menu, choose the **Zoom** option.



   For zoom operation details, see Zooming.

3. (Optional) Cycle through the selected set.

   The tool zooms to magnify each object.

   For more information about viewing properties in the design, see Viewing Design Object Properties.

4. (Optional) Disable automatic zooming by deselecting the **Zoom** option in the Property Editor toolbar menu.

Adjusting Parameters in the Selected Set

You can navigate from the Property Editor assistant to the Parameter Editor and adjust parameters for the currently selected set.

To adjust parameters in the currently selected set:

1. Begin by Displaying the Property Editor or Opening the Properties Dialog Box.
2. In the design, select one or more objects.
3. From the Property Editor toolbar menu, choose **Parameter Definitions**.

   The Parameter Definition Editor opens.

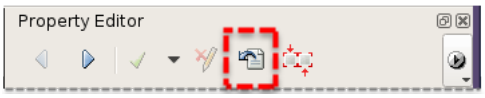
   For more information, see Parameter Definition Editor Tool.

Feedback

Resetting to Default Parameter Values

To reset the parameters to their default values:

► On the Property Editor toolbar, click **Reset to Defaults**.



You can customize **Reset to Defaults** so that specific parameters do not revert to their default values. Use the following CDF parameter to specify the parameters that should not be reverted when resetting to default values in the Property Editor:

CDF Parameter	Description
dbResetParamsToIgnore	(String) List of parameters that should not revert to their default values, but their callback procedures will run after the reset operation. The order of the callback is determined by the order the parameters appear in the list.

The default values for vias and multi-part paths (MPPs) are from the StdViaDef and leMPP definitions, respectively and not from any corresponding CDF parameter default. The default values for instances and custom vias are from the CDF parameters first, and if no such parameter exists then the default is obtained from the OA database (SuperMaster oaParamArray).

Instance Terminal Region

The Instance Terminal editing region has a table for editing (layouts only) and viewing the instance terminals of an instance. The default configuration only presents the only the **Term Name** and **Net Name** columns. Additionally, only for layouts, the default configuration also presents the **Current** column. A context-sensitive menu allows hiding and showing the **Term Name, Net Name, Term Type, Anchor**, and **Current** columns for instance terminals

Column	Description
Term Name	Displays the instance terminal names.
Net Name	Displays the name of the net connected to the named instance terminal.
Term Type	Displays the terminal type.

Feedback

Column	Description
Anchor	(Layout only) Displays the anchor value, which associated with instance terminal. If there is no associated net with a terminal, this column with check box is visible, empty, and dimmed. Only when a net is entered in the **Net Name** column, the check box in the **Anchor** column becomes editable.
Current	(Layout only) Displays the **Average DC**, **Average AC**, **Peak**, and **AC RMS** as the four sub columns of the **Current** column with each cell having the same behavior as the Current widget's cells.

Property Editor Toolbar

The Property Editor toolbar contains the following:

Button	Description
	Displays the properties of the previous or next selected object. The buttons are available only if you selected .
	Applies the changes you made.
	Allows you to choose which properties to display: the current object, objects of the same type, objects in the selected set, objects of the same type in the selected set, or objects on the same layer-purpose pair.
	Discards changes made since the last time you saved.
	Resets to default values.
	Displays the properties of the objects you selected in one of the following modes: Single-item mode: Displays the properties of each object in the selection individually. To see the properties of the next item in the selection, click one of the two arrow buttons on the toolbar ( ). Multi-item mode: Displays the properties of each object in the select in a tab. To see the properties of the next item, click the tab.

Feedback

Button	Description
	Allows you to set the viewing options: • **Remove Properties on Remaster**: Removes all properties not associated with CDF/OA parameters when remastering the instance.* • **Reset to Defaults on Remaster**: Resets all CDF parameters to defaults when remastering the instance.* • **Zoom**: Allows you to magnify the area of interest. • **Parameter Definitions**: Displays the Parameter Definition Editor Tool. • **Parameter Filter**: (Schematic Editor only) Displays the Parameter Filter dialog box, where you can filter parameters relevant to a given application. See Filtering Parameters in the *Custom Compiler Schematic Editor and Text Editor User Guide*. • **Column Visibility**: Allows you to choose the information to display in the Property Editor. • **Help**: Displays the Property Editor help topic.

* Instances can only be remastered if they are listed in the `dbAllowedRemasterCells` CDF parameter, or if the CDF parameter is not defined for the library. See Parameter Definition Editor.

Selection Table Assistant

The Selection Table assistant lists objects that are selected in the design.

These topics cover:

- Opening the Selection Table Assistant
- Showing Columns
- Viewing Parameters
- Viewing Figure Group Members
- Showing Objects in Shadow
- Fitting Viewport to Selected Objects

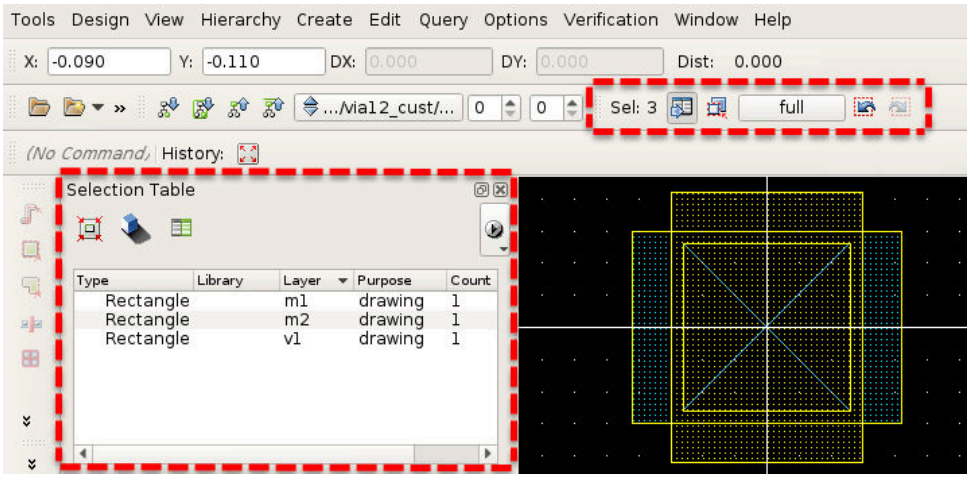
Opening the Selection Table Assistant

To open the Selection Table assistant, use one of these methods:

- From any design layout or schematic, choose **Window > Assistants > Selection Table**.
- From the **Selection** toolbar, click the **Selection Table** button next to the selection count . See Selection Toolbar.

Feedback

By default, the Selection Table appears on the left side of the design window. You can undock and move it by dragging it to another location.



	The Selection Table Assistant lists the selected objects in the design. It provides options to view parameters of the selected objects and the selected figure groups with group members. This video tutorial shows how to select objects in the **Selection Table**.

Showing Columns

The Selection Table consists of a single table which shows the current selection set according to the enabled columns in the **Select Attributes** dialog box.

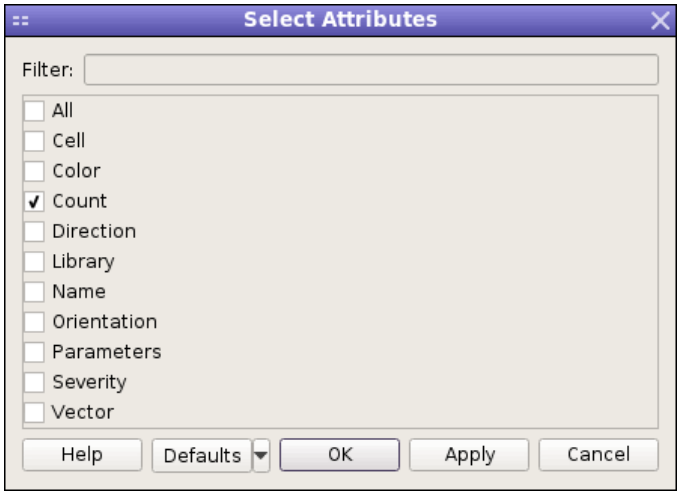
To choose columns you want to show:

1. Begin by opening the Selection Table assistant. From any layout or schematic design, choose **Window > Assistants > Selection Table**.
2. Select the objects on the canvas, or use shortcut key Ctrl+A to select all objects in the design.

   The Selection Table shows the selected objects after your selection.
3. (Optional) To enable the **Filter** for attribute selection, check **Show Filter** from the Selection Table toolbar menu.
4. From the Selection Table toolbar or the Selection Table toolbar menu , click the **Attributes** button .

   The following figure shows the **Select Attributes** dialog box for a layout design.

The following figure shows the **Select Attributes** dialog box for a schematic design.



Feedback

The **Select Attributes** dialog box opens and shows a list of attributes that you can select as columns, as listed in the following table.

Column	Description for Layout Types	Description for Schematic Types
All	All object types.	All object types.
Area	The area, which is calculated from the polygonal boundary, for paths, path segments, rectangles, polygons, blockages, and ellipses.	-
Array	Number of rows and columns of array instances.	-
Cell	Instance cell name.	Instance cell name.
Color	Color name for shapes and blockages.	Color name for tables, wires, wire names, pins, note shapes, connection definition labels, and instance labels.
ColorLock	The color lock of selected shapes or standard vias is set.	-
Count	Total number of objects, regardless of partially selected parts, matching the row.	Total number of objects, regardless of partially selected parts, matching the row.
Cut Color	Cut color for standard vias.	-
Definition	Definition name for MPPs and vias.	-
Direction	-	Direction
Layer	Layer name for shapes, standard vias, and blockages. For vias, this is a cut layer.	-
Layer1Color	Layer1 color for standard vias.	-
Layer2Color	Layer2 color for standard vias.	-
Length	Length for paths and the total path length for path segments.	-
Library	Referenced library name.	Referenced library name.
MaxVoltage	Maximum voltage of nets, applied to shapes and vias on net.	-
Member	Name of the object type.	-

Feedback

Column	Description for Layout Types	Description for Schematic Types
MinVoltage	Minimum voltage of nets, applied to shapes and vias on net.	-
Name	Name of instances, pins, figure groups, area boundary, and text labels.	Name of tables, wires, note shapes, connection definition labels, instance labels, and text labels.
Orientation	Orientation for text labels, instances, and vias.	Orientation for wire names, note shapes, connection definition labels, and instance labels.
Origin	Origin.	-
Parameters	Parameter names of instances and array instances with values, as defined in the **Select Parameters** dialog box.	Parameter names of instances and vectored instances with values, as defined in the **Select Parameters** dialog box.
PhysicalOnly	Indicates whether selected instances are physical only.	-
Purpose	Purpose name for shapes.	-
Severity	Severity of oaMarker.	-
Type	Semantic object types.	Semantic object types.
Usage	Usage of instance, either normal or dummy.	-
Vector	-	Vector names.
Width	Width for paths and path segments.	-

5. Check the attributes to display as columns in the selection table. Check **All** to select/deselect all columns.

Viewing Parameters

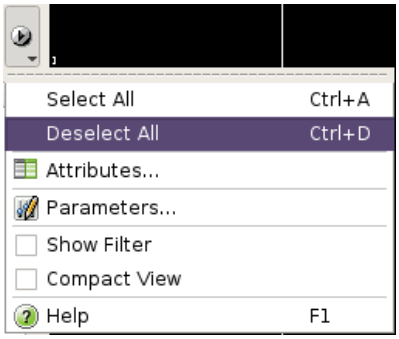
To view parameters of selected objects in the Selection Table:

1. From any design layout or schematic, choose **Window > Assistants > Selection Table**.
2. Select the objects on the canvas, or use the shortcut key, Ctrl+A, to select all objects in the design.

   The Selection Table shows the selected objects after your selection.

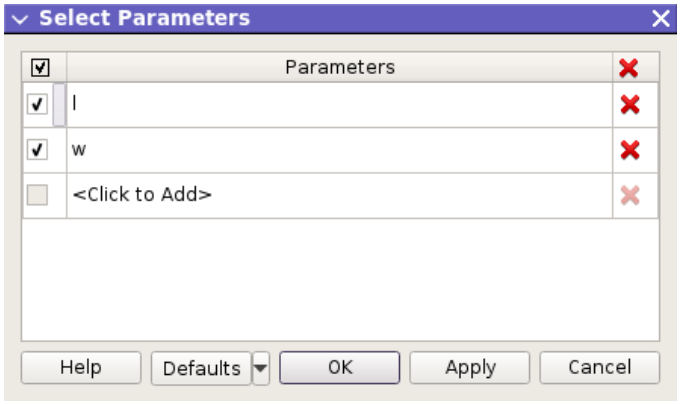
Feedback

3. Click the Selection Table toolbar menu and choose **Parameters**.



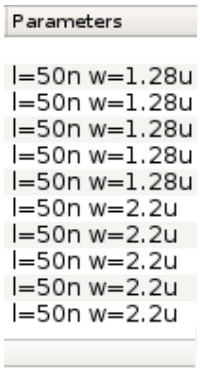
The **Select Parameters** dialog box opens.

4. In the **Select Parameters** dialog box, enter the **Parameters** to view.



5. In the Selection Table, check the **Parameters** column to update and view the parameter values.

You can now view the parameter names with values, as defined in the **Select Parameters** dialog box.



Feedback

Viewing Figure Group Members

You can view selected figure groups with group members in the Selection Table. For information about figure groups, see Working With Figure Groups.

To view the members of selected figure group objects in the Selection Table:

1. From any design layout or schematic, choose **Window > Assistants > Selection Table**.
2. Select the figure group objects on the canvas.

   The Selection Table shows the selected figure groups.

Type	Name
FigGroup	FG1
FigGroup	FG2
FigGroup	FG3
FigGroup	FG4
FigGroup	FG5

3. From the console, set the `deSelectionTableShowFigGroupMembers` Tcl preference to `true`, as shown:

   ```
   db::setPrefValue deSelectionTableShowFigGroupMembers -value true
   ```

   When the **Members** column is checked, the Selection Table is updated to show the members of the selected figure groups.

Feedback

Type	Member	Name
FigGroup		FG1
	Rectangle	
	Path	
FigGroup		FG2
	Path	
	Rectangle	
FigGroup		FG3
	Polygon	
	Polygon	
FigGroup		FG4
	Path	

If a figure group is a synchronized clone, then the members are disabled, that is, they cannot be selected. For example, figure group FG2, is a synchronized clone in the following figure.

Type	FigGroup Member	Name
FigGroup		FG1
	Rectangle	
	Path	
FigGroup		FG2
	Path	
	Rectangle	

If a figure group is selected, then the members of the figure group cannot be selected. Similarly, if a figure group member is selected, then the figure group cannot be selected.

4. (Optional) To see the selected figure group objects in a tree view, set the `deSelectionTableTreeView` Tcl preference to `true`.

Showing Objects in Shadow

For more targeted viewing, you can show selected objects in shadow for additional visual contrast between selected objects and the surrounding design.

Feedback

To show selected objects in shadow:

1. Begin by opening the Selection Table assistant. From any design layout, choose **Window > Assistants > Selection Table**.
2. Select the objects on the canvas.

   The Selection Table shows the selected objects after your selection.
3. From the Selection Table toolbar, click the **Shadow selection** button .

   The tool displays the selected objects brighter than surrounding objects on the canvas.

Fitting Viewport to Selected Objects

Fitting the design viewport to objects selected in the table allows for a quick and focused viewing of selected objects on the canvas.

To fit the design viewport to objects selected in the table:

1. Begin by opening the Selection Table assistant. From any design layout or schematic, choose **Window > Assistants > Selection Table**.
2. Select the objects on the canvas.

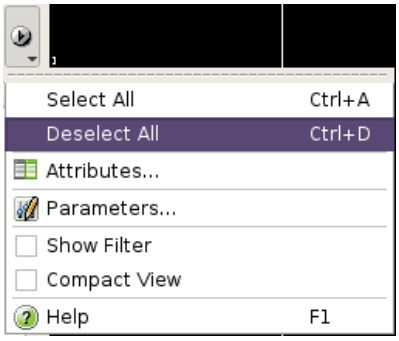
   The Selection Table shows the selected objects after your selection.
3. From the Selection Table toolbar, click the **Fit selection** button .
4. Select the objects in the table.

   (Optional) To select more than one object in the table, use Ctrl-click to select each row individually. You can alternatively use Shift-click or drag-left to select rows in the table.

   (Optional) Click the Selection Table toolbar menu and choose **Select All** to select all rows in the table.

   (Optional) Click the Selection Table toolbar menu and choose **Deselect All** to clear the selection of all rows in the table.

Feedback



The tool fits the design viewport to the selected objects after your selection.

Selecting any row in the table replaces the contents of the selection set with the figures that correspond to that row. The Selection Table is not updated when the selection is changed by clicking within the table.

Track Pattern Assistant

This topic describes the following:

- About the Track Pattern Assistant
- Displaying the Track Pattern Assistant
- Viewing Track Patterns
- Creating and Editing Track Patterns
- Copying Track Patterns
- Renaming and Deleting Track Patterns
- Creating and Editing Track Pattern Definitions
- Assigning New Definitions

About the Track Pattern Assistant

You can use the Track Pattern assistant to enable correct-by-construction layout design for advanced process nodes, such as those containing FinFETs. Using track patterns can reduce design time.

When you design with active track patterns, objects snap to the nearest track during create, copy, move, and stretch operations. In addition, the Layout Editor displays only relevant candidate tracks as you create, copy, move, or stretch objects, even if **Visibility** is turned off for those tracks.

Feedback

You need to define track patterns before you can use the Track Pattern assistant to apply them. A track pattern definition is a set of horizontal or vertical tracks that indicate available routing resources, that is, both the positions and the number of routing tracks.

To define track patterns for your designs, you can use one of the following methods:

- Use the Track Pattern assistant.

  See Creating and Editing Track Pattern Definitions.

- Add track pattern definitions directly to the technology file.

  See the section about `trackPatternDefs` in the *Custom Compiler Technology File and Display Resource Reference Manual*.

  To import an updated technology file after adding track pattern definitions, see Importing Technology Data.

  Contact your Synopsys Support representative for additional assistance.

- Use Tcl to add track pattern definitions.

  See the section covering the `le::createTrackPatternDef` command in the *Custom Compiler Tcl Reference Manual*.

  Contact your Synopsys Support representative for additional assistance.

Displaying the Track Pattern Assistant

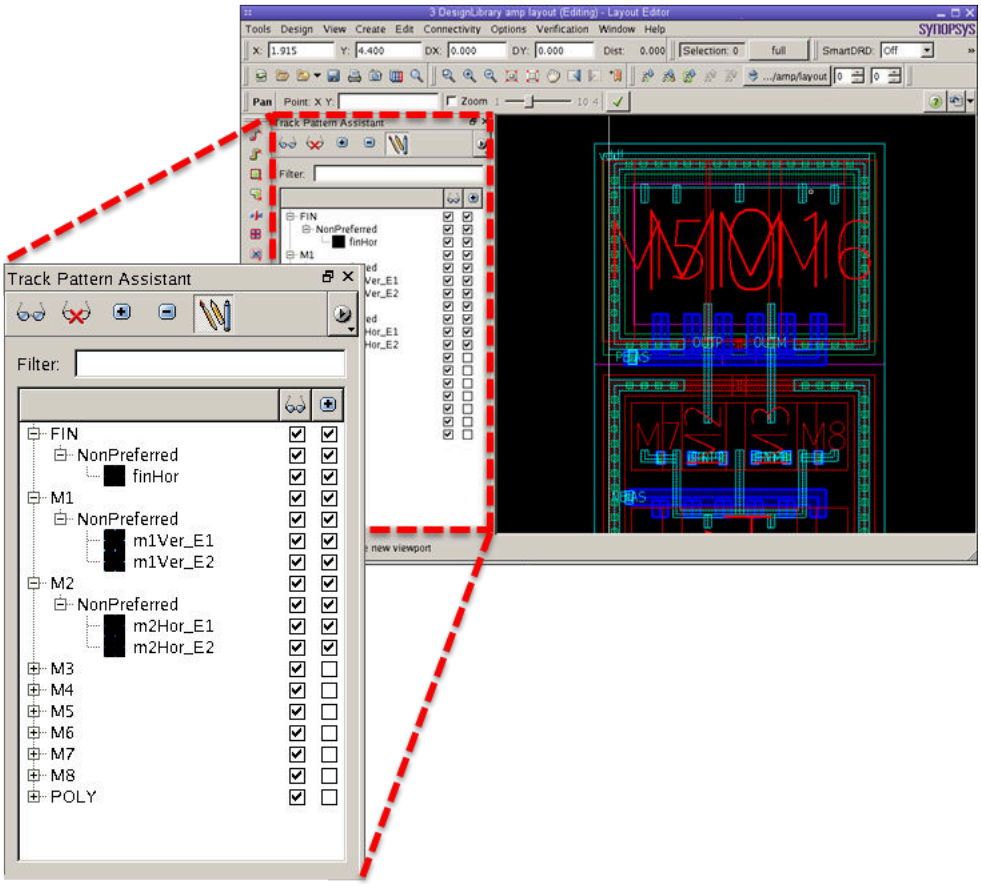
To display the Track Pattern assistant:

► In the Layout Editor window, choose **Window > Assistants > Track Pattern Assistant**. The Track Pattern assistant pane appears on the left side of the window. You can undock the pane by dragging it to a required location.

  The Track Pattern assistant shows a list of track patterns for the current design, in alphabetical order by associated layer name, preferred sub-node direction (vertical

Feedback

or horizontal), visibility, and active state. If a layer does not have an associated track pattern, the system does not list the layer.



Viewing Track Patterns

You can view track patterns that are set up and defined for your designs. You can set them to be visible or invisible, and active or inactive, either individually or all at once. In addition, if your design's technology contains mask color definitions, you have the option of displaying the track patterns' assigned colors.

To view track patterns:

1. In a design layout, display the Track Pattern assistant. In the Layout Editor window, choose **Window > Assistants > Track Pattern Assistant**.
2. To limit the list of tracks displayed in the Track Pattern assistant, type characters in the **Filter** box. Any name that matches the characters you typed will be displayed.

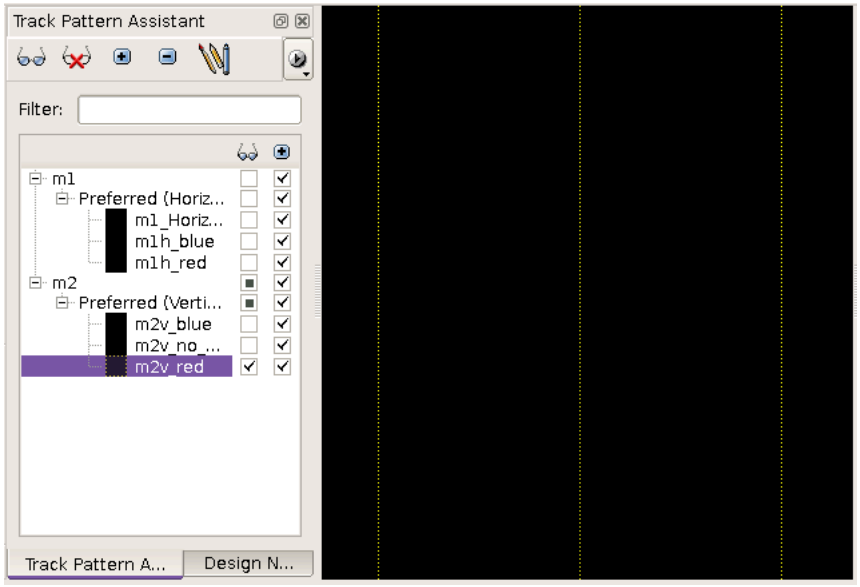
Feedback

If the **Filter** box is not displayed, click , and then choose **Show Filter**.

3. To show individual track patterns, select **Visibility** for those track patterns. To hide the track patterns, clear the **Visibility** check box.

   Note:

   The Layout Editor displays relevant candidate tracks as you create, copy, move, or stretch objects, even if **Visibility** is turned off for those tracks. To turn off the automatic display of candidate tracks, set the `leDisplayInCommandTrackPatterns` Tcl preference to `false`.

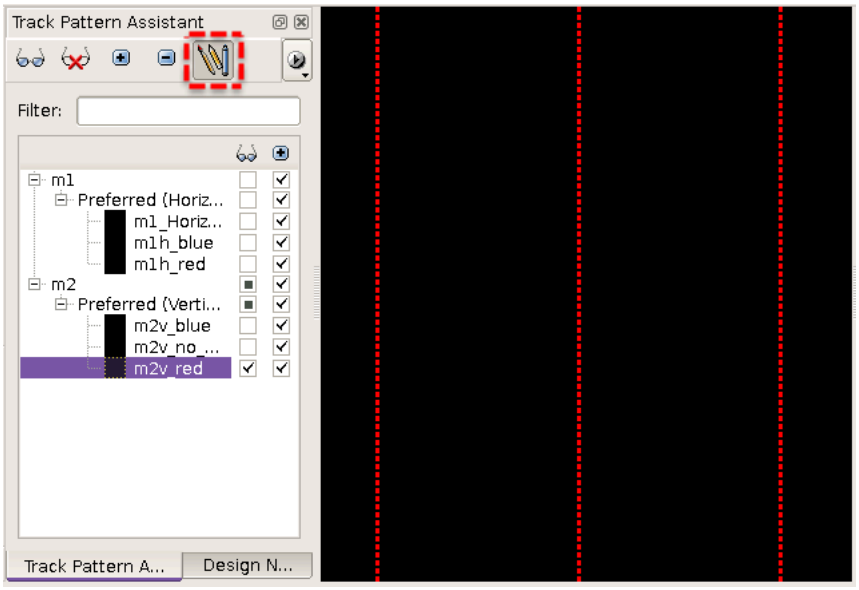


   In the following example, the m2 layer vertical red tracks are set to be displayed on the canvas. Note that the tracks' assigned color is not displayed.

4. To display a track pattern's assigned color, on the **Track Pattern** toolbar, click **Toggle track color rendering**. The tracks' assigned color displays. (To display the assigned color, the track pattern must be visible. See step 3).

   In the following example, **Toggle track color rendering** is turned on for the m2 layer vertical red tracks. Notice that the tracks are colored Red.

Feedback



5. To show or hide all track patterns in the list, click one of the following Track Pattern toolbar buttons, respectively: .

6. To make a track pattern active, select **Activity**. To make a track pattern inactive, clear the **Activity** check box.

   Caution:

   If you want the Layout Editor to display relevant candidate tracks as you create, copy, move, or stretch objects, make sure those track patterns are active. To turn off the automatic display of candidate tracks, set the `leDisplayInCommandTrackPatterns` Tcl preference to `false`.

7. To make all track patterns in the list active or inactive, click one of the following **Track Pattern** toolbar buttons, respectively: .

8. To switch the sorting order from layer (default) to alphanumeric, on the Track Pattern toolbar, click , and then choose **Sort > Alphanumeric**.

   To return to the default order, click , and then choose **Sort > Layer Number**.

9. To show the list of non-preferred tracks, from the toolbar menu, choose **Show Non-Preferred Tracks**.

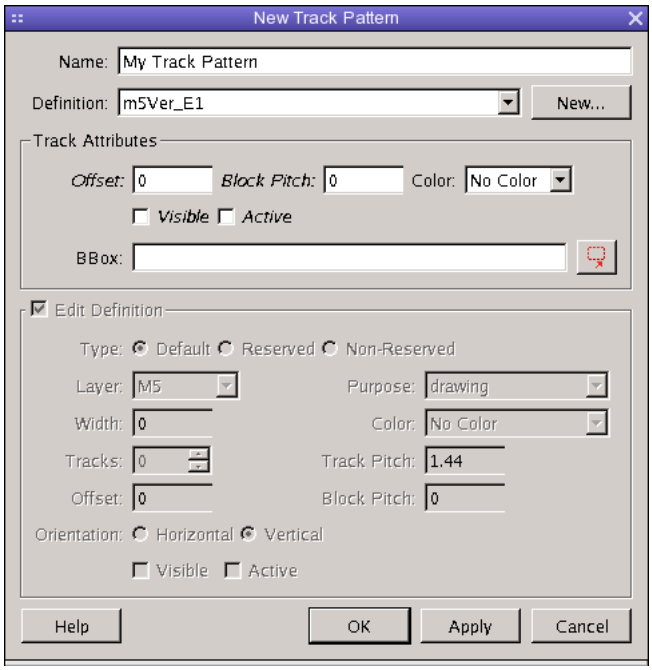
Feedback

Creating and Editing Track Patterns

To create and edit track patterns:

1. Open the Track Pattern assistant. In the Layout Editor window, choose **Window > Assistants > Track Pattern Assistant**.
2. On the **Track Pattern Assistant** toolbar, click , and then choose one of the following commands:
   - New: Opens the **New Track Pattern** dialog box and lets you create a new track pattern.
   - Edit: Opens the **Edit Track Pattern** dialog box and lets you edit an existing track pattern.

   The contents of the **New Track Pattern** and **Edit Track Pattern** dialog boxes are identical.



3. Provide or revise the track pattern information in the dialog box:

Table 91 Track Pattern Settings (New or Edit)

Setting	Description
Name	Name to assign to this track pattern. No spaces are allowed.

Feedback

Setting		Description
Definition		Applies the track pattern definition you choose from the list box. To create a new definition, click **New**, and then follow the instructions in Creating and Editing Track Pattern Definitions.
Track Attributes	Offset	Overrides the applied track pattern definition with a new offset from origin you enter here.
	Block Pitch	Overrides the applied track pattern definition with a new block pitch you enter here.
	Color	Assigns a color to the track pattern. The available color names must be defined in the technology file. **Color** is available only if you selected a track pattern definition for a layer that is colorable.
	Visible	Displays this track pattern by default.
	Active	Makes this track pattern active by default.
	Bbox	Bounding box within which you want the tool to create tracks, generated by the coordinate set you define when you click the button and draw the box area.

4. (Optional) If you want to edit an existing track pattern definition as you create or edit the track pattern, see Creating and Editing Track Pattern Definitions.
5. Click **OK**.

   If you created a new track pattern, to view it, locate the layer you assigned it to in the **Track Pattern** list and expand the tree view for this layer.

Copying Track Patterns

You can copy existing track patterns to add to the number of track patterns available for use.

To copy track patterns:

1. Open the Track Pattern assistant. In the Layout Editor window, choose **Window > Assistants > Track Pattern Assistant**.
2. From the list of track patterns, select one by name.
3. Right-click and choose **Copy**.

Feedback

4. In the **Copy Track Pattern** dialog box, enter a name for the new copy.
5. (Optional) Adjust settings.

   Table 91 lists track pattern settings you can adjust.

   Creating and Editing Track Pattern Definitions lists track pattern definition settings you can adjust if you want to change the currently associated track pattern definition.

Renaming and Deleting Track Patterns

You can rename existing track patterns. You can also delete track patterns from the set available for this design.

To rename or delete track patterns:

1. Open the Track Pattern assistant. In the Layout Editor window, choose **Window > Assistants > Track Pattern Assistant**.
2. From the list of track patterns, select one by name.
3. Right-click and choose an option:
   - To rename the track pattern, choose **Rename** and enter a new name here.
   - To delete the track pattern, choose **Delete**.

   Choosing **Edit > Undo** reverses either of these operations.

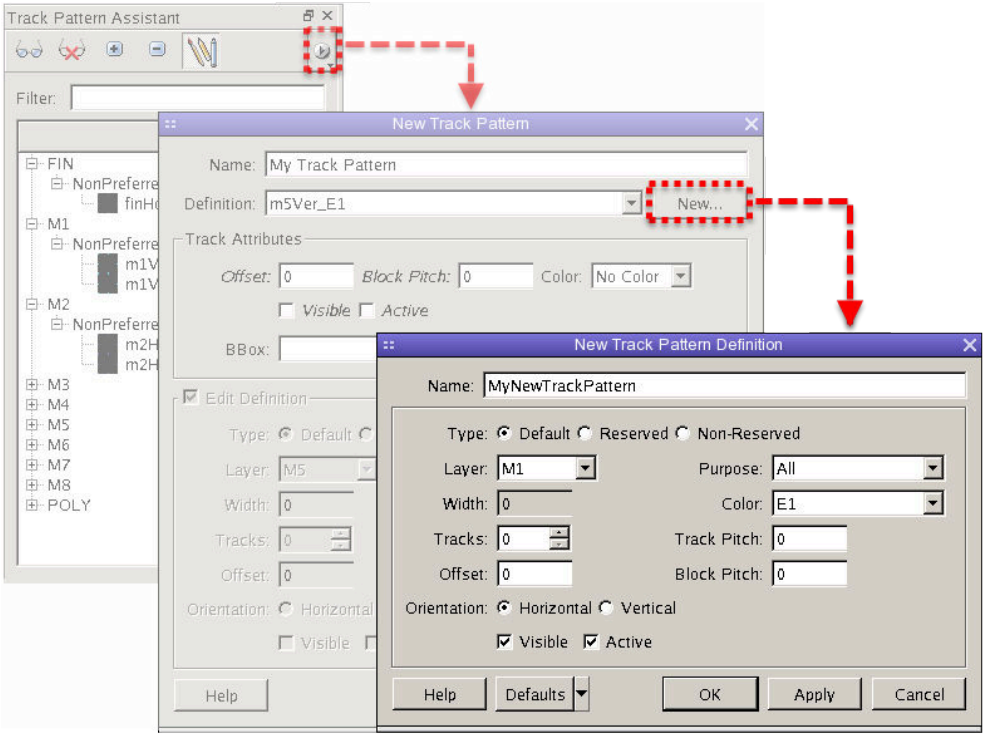
Creating and Editing Track Pattern Definitions

You can use the Track Pattern assistant to create and edit track pattern definitions. The values that you specified in the technology file appear as default values in the Track Pattern assistant.

Feedback

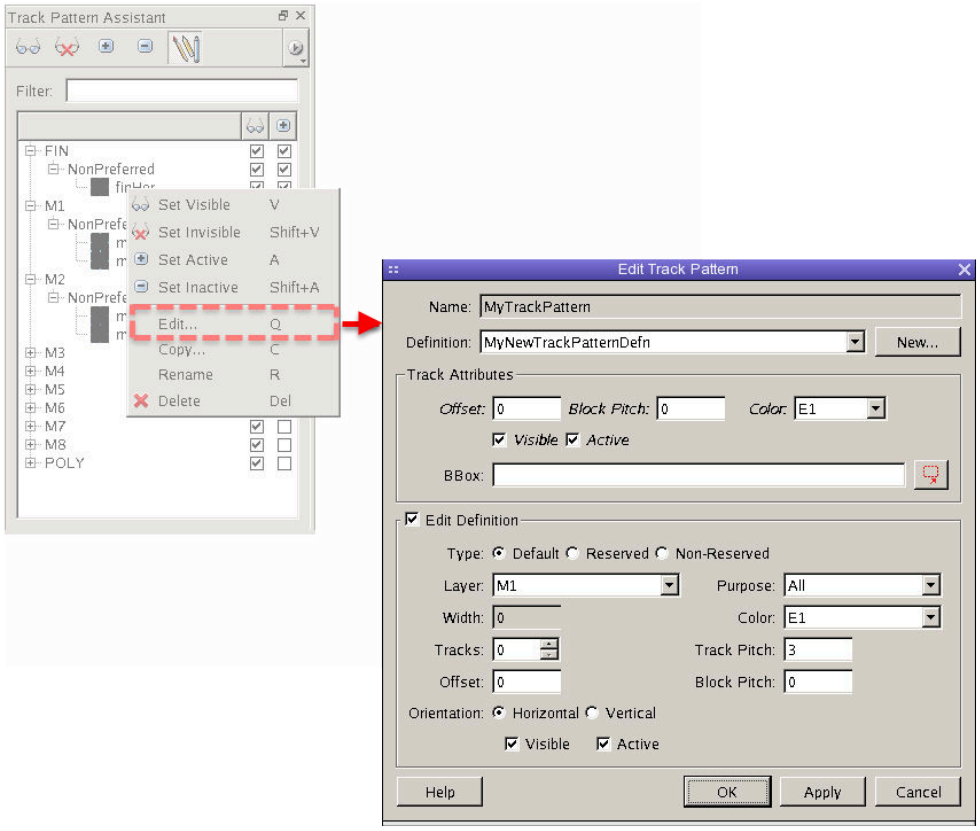
To create and edit track pattern definitions using the Track Pattern assistant:

1. Open the Track Pattern assistant. In the Layout Editor window, choose **Window > Assistants > Track Pattern Assistant**.
2. Do one of the following:
   - To create a new track pattern definition, on the **Track Pattern Assistant** toolbar, click , and then choose **New**. In the **New Track Pattern** dialog box that appears, click **New**. The **New Track Definition** dialog box opens.



Feedback

- To edit an existing track pattern definition, in the **Track Pattern Assistant**, right-click the track pattern name, and then choose **Edit**. The **Edit Track Pattern** dialog box opens.



The contents of the **New Track Pattern Definition** dialog box and the **Edit Definition** area of the **Edit Track Pattern** dialog box are identical.

Feedback

3. Enter or revise the existing track pattern definition settings:

Setting	Description
Type	◦ Default: Default tracks have no width requirements. Wires of any width can route on them, with the wider wires consuming (or blocking) more available area. ◦ Reserved: Reserved tracks have a width parameter. When the preference `leReservedTrackWidthCheck` is set to false, the track operates like a default track, and any wire can route on the tracks. The wire's width does not have to match the track's width. When the preference is set to true, only wires with non-default-rule nets can route on the tracks, and the wires must match the width of the reserved tracks. ◦ Non-Reserved: Non-Reserved tracks have a width parameter, and any wire placed on the tracks must comply with the width of the tracks.
Layer	The layer to which the track pattern definition applies.
Purpose	The purpose of the layer you selected.
Width	The width of the track path, in micrometers.
Color	The color of the tracks. The available color names must be defined in the technology file. **Color** is available only if you selected a layer that is colorable. The default selection is the value specified in the technology file.
Tracks	The number of tracks in the pattern.
Track Pitch	The distance between individual tracks.
Offset	The offset of the pattern from the origin.
Block Pitch	The distance between the first track of a block and the first track of the next block.
Orientation	◦ Horizontal: The orientation of the tracks. Select **Horizontal** if you want the tracks to run along the *x*-axis. ◦ Vertical: The orientation of the tracks. Select **Vertical** if you want the tracks to run along the *y*-axis.
Visible	The option to display or hide the track pattern.
Active	The option to let shapes snap to the track during create, copy, move, and stretch operations.

4. Click **OK** to save the changes.

Assigning New Definitions

You can assign a new definition to a track pattern by choosing it from the list of available definitions.

To assign a new track pattern definition:

1. Open the Track Pattern assistant. In the Layout Editor window, choose **Window > Assistants > Track Pattern Assistant**.
2. In the list, select the track pattern by name.
3. Right-click and choose **Edit**.
4. In the **Edit Track Pattern** dialog box, from the **Definition** list box, choose a new definition to assign.
5. Click **OK**.

Track Region Assistant

The Track Region Assistant provides a graphical user interface that facilitates the creation and editing of track regions.

This topic describes the following:

- About Track Regions
- About the Track Region Assistant
- Before You Start
- Defining Track Presets
- Displaying the Track Region Assistant
- Setting Track Region Assistant Options
- Creating Track Region PCells
- Creating Track Regions
- Creating a Global Track Region
- Creating Tracks
- Creating Track Group Definitions
- Creating Track Region Definitions
- Deleting Track Regions

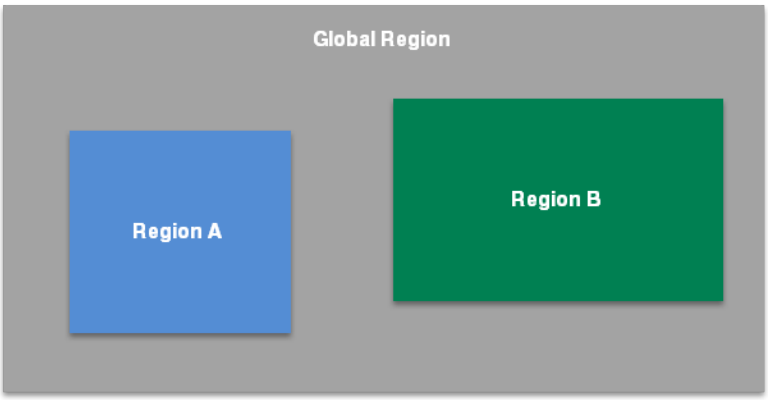
Feedback

- Editing Tracks
- Editing Track Group Definitions
- Editing Track Region Definitions
- Editing Track Region Properties
- Filtering the List of Track Regions
- Displaying and Hiding Track Regions on the Canvas
- Making Track Regions, Track Groups, and Individual Tracks Active

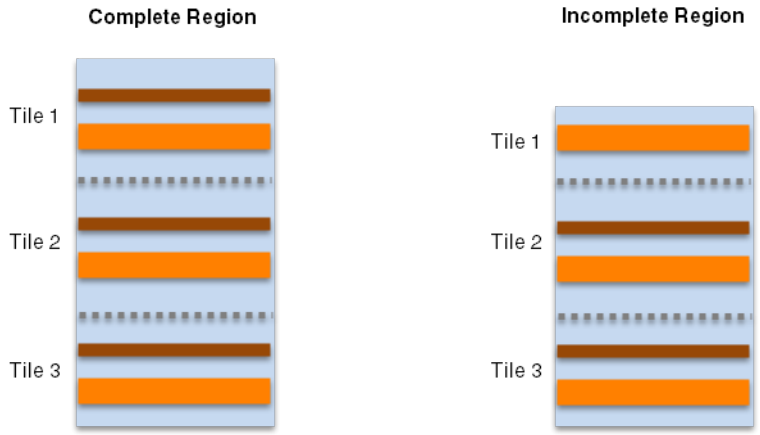
	The video demonstrates an example of how to define track regions and track groups.

About Track Regions

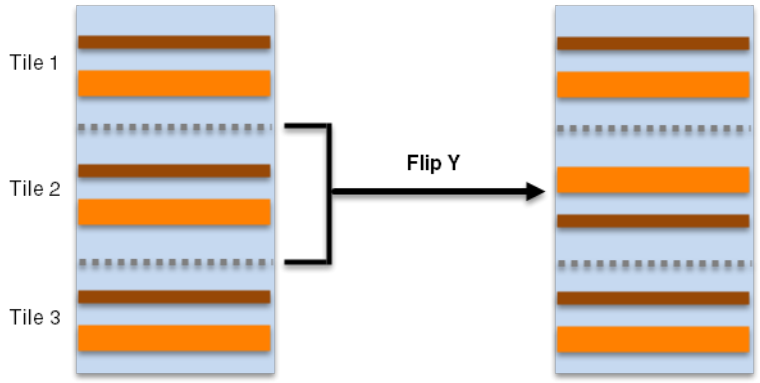
Tracks ensure that interconnects and routes in advanced node designs, such as those containing FinFETs, are properly placed and correct by construction. You can create multiple track regions to accommodate different requirements within the same design. Each track region consists of one or more tiles. Each tile consists of a set of tracks.



A complete region consists of an integer number of tiles. An incomplete region consists of one or more incomplete tiles. In the following example, the incomplete region has an incomplete tile in the Y direction (tile 1 is incomplete in the Y direction).



Within a region, you can specify that alternating tiles flip along the X or Y direction. In the following example, the second tile is flipped vertically (in the Y direction).



Note:

If you create track regions, the Layout Editor will ignore existing track patterns in the design.

When you design with active track regions, shapes and objects snap to the nearest track during create, copy, move, and stretch operations. In addition, the Layout Editor displays only relevant candidate tracks as you place interconnects and routes, even if **Visibility** is turned off for those tracks.

About the Track Region Assistant

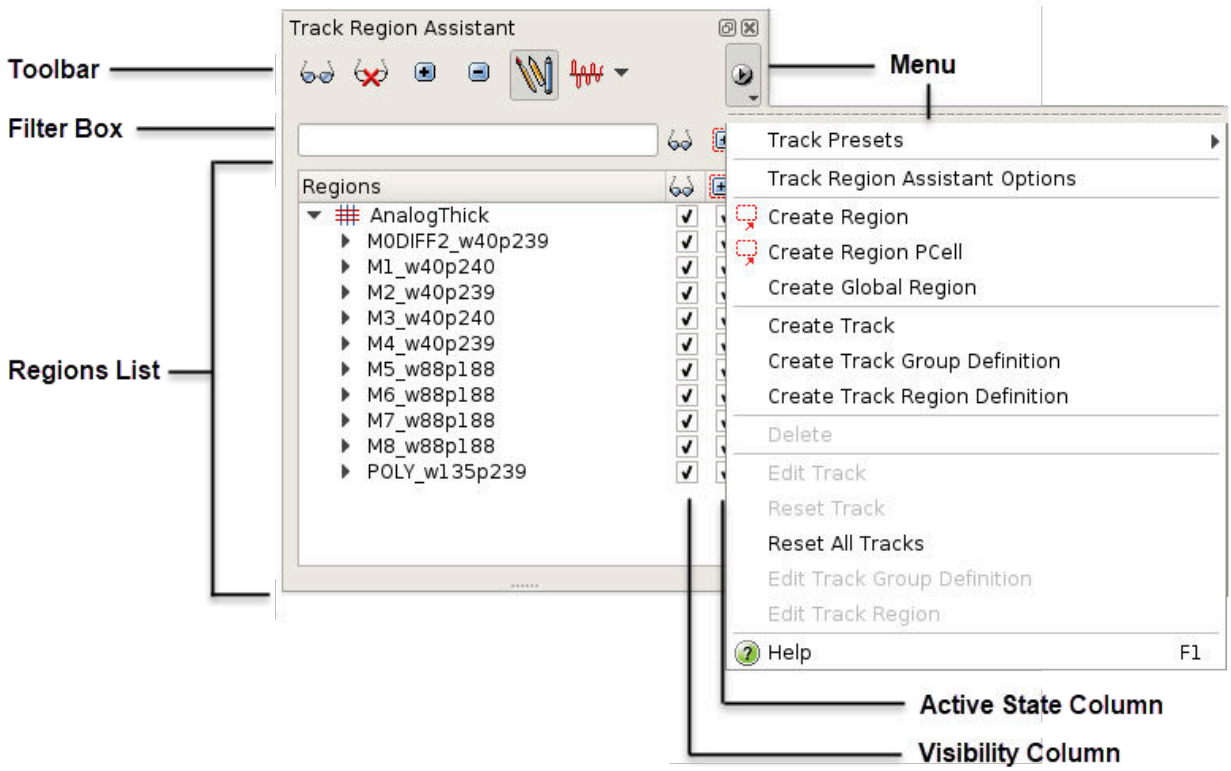
You can use the Track Region assistant to:

- Create, edit, and delete local ( ) and global ( ) track regions.
- Display or hide existing track regions on the canvas.

Feedback

- Set tracks and regions to the Active mode so that shapes snap to the tracks.
- Change signal types for specific tracks.
- Select preset track region definitions.

The Track Region Assistant (TRA)) consists of the following:



Grouping Track Grids and Grid Groups

You can group track grids and grid groups that share the same color and width to a single track group.

Using this option, you can:

- Define grids with the same color and width in `grids()`
- Define tracks with the same color and width in `tracks()`
- Group those tracks in a single group in `groups()`

This group applies to a single `trackGroup`, and all tracks/grids in the group share the specified color and width.

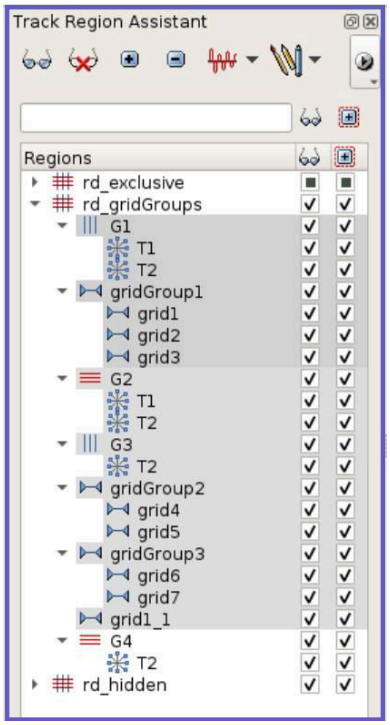
Feedback

This approach allows you to logically group together tracks (and their associated grids) that have matching visual and physical properties, and have them apply to a single `trackGroup`.

In the TRA, the track grids are displayed as separate rows at the same level as group definitions. Grid groups appear at the same level as individual grids and use the same icon. Grid groups are expandable; expanding a grid group displays its underlying grids, which can be managed together. In the TRA, grid groups are positioned between group definitions and individual grids.

The following images display the grouping of the track grids and grid groups:

Figure 45 Region > Group > Track



Feedback

Figure 46 RegionDef > Group > Track

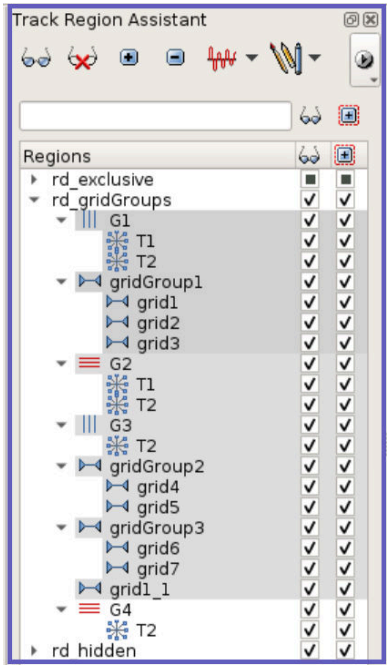
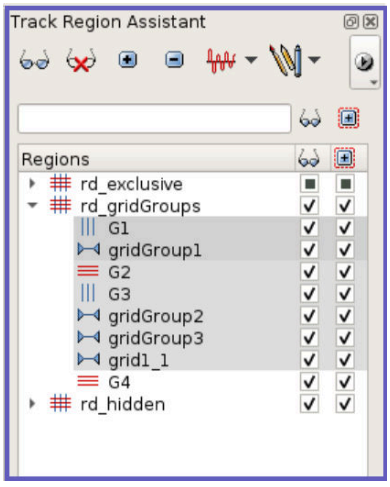


Figure 47 Region > Group



Feedback

Figure 48 Region > Layer > Track

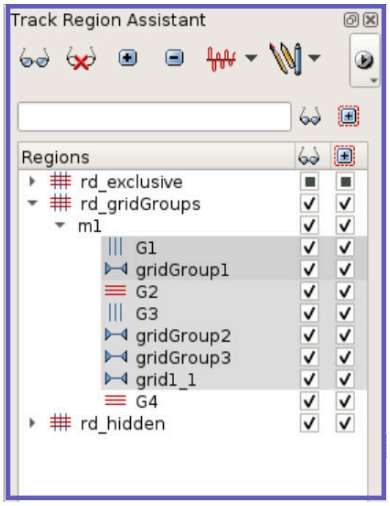
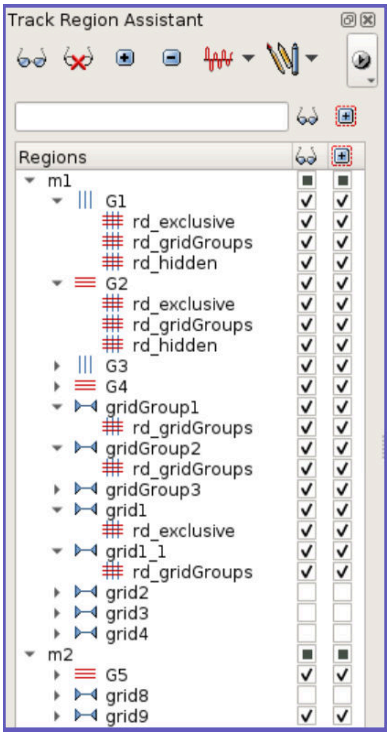
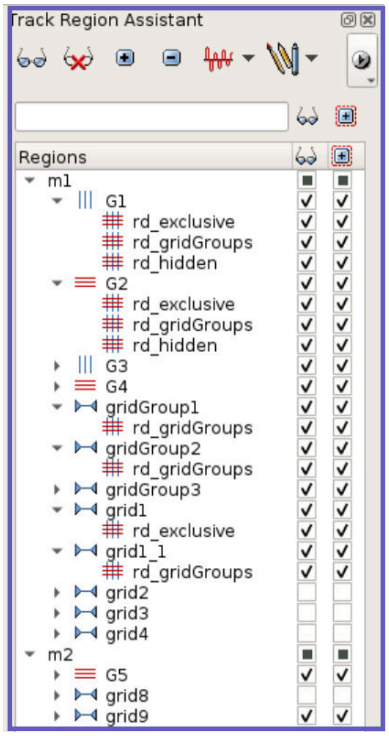


Figure 49 Layer > Group > Region



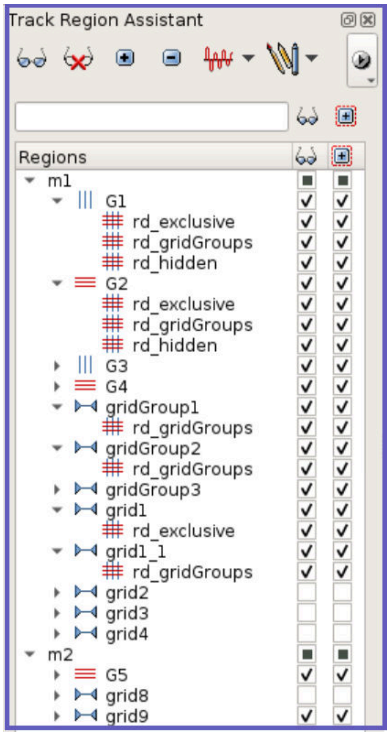
Feedback

Figure 50 Layer > Region > Group



Feedback

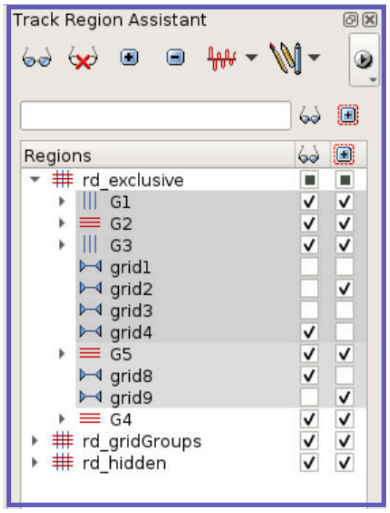
Figure 51 Group > Region > Track



Exclusive Track Grids

In the TRA, for exclusive track grids, only one grid can be active or visible at a time.

Figure 52 Exclusive Track Grids



Feedback

When you enable one grid, all other grids in the exclusive group are automatically disabled.

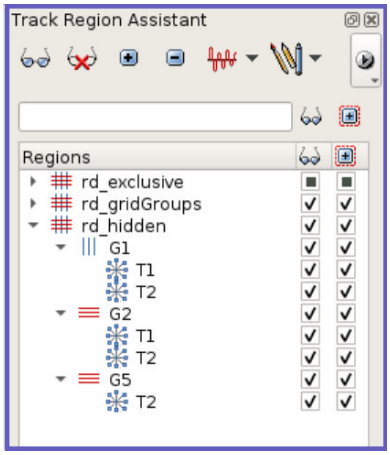
The active and visible states are managed separately and operate independently.

When you open a design or create a region, the active and visible states of exclusive grids are initialized based on their definitions. The exclusivity logic is enforced only when you change grid states using the TRA. If you change the visible or active state of a grid using Tcl commands, it does not affect the states of other grids in the same exclusive group.

Hidden Tracks and Group Definitions

An object is not displayed in the TRA if the hidden attribute of a track or group definition is set to true, which cannot be controlled from the assistant. The `Set all tracks visible/active` command does not affect hidden objects.

You can access these objects using the Tcl commands.



Examples

Feedback

Before You Start

Some required and optional tasks should be completed before you start to create and edit track regions in the Track Region Assistant.

Make sure that the technology database has one or more track region definitions specified in the trackDefs Layer Rules section. For information about modifying the technology database, see the *Custom Compiler* Technology and Display Resource File Reference Manual.

The track region definitions that are available for global and local track region creation are filtered by the `leTrackRegionAllowedDefNames` Tcl preference. If you do not see the definition when creating track regions, check the Tcl preference setting.

If you defined grids to specify permissible starting (low) and ending (high) locations for interconnects and routes, you can optionally set the `leTrackRegionAssistantShowTrackGrids` Tcl preference to show (true) or hide (false) the grid information in the Track Region Assistant. To change how the grids are displayed on the layout canvas (line or arrow), set the `leRenderTrackGridsMethod` Tcl preference. When `leRenderTrackGridsMethod` is `line`, you can specify the line style, such as the color or thickness, by editing the **trackGridLines** display packet. See Creating and Editing Display Packets and Packet Aliases.

If you want to use design- or project-based preset track regions instead of track regions defined in the PDK or foundry technology, see Defining Track Presets.

Defining Track Presets

Some designs or projects require that you use tracks from different track regions. To address the requirement, you can define a group of tracks (track presets) that are in different track regions. Doing so lets you access and use the tracks in the different regions without having to enable each region individually.

To define a track preset:

1. Create a preset file named deTRAPresets.txt. The file should be in the ASCII format and should contain the following information for each track preset:

```
presetName1 {
    {regionDef1/grouDef1/track1 <visibleState> <activeState>}
    ....
    {regionDefN/grouDefN/trackN1 <visibleState> <activeState>}
}
```

   For example:

```
preset1 {
    {r1/g1/t1 0 1}
```

Feedback

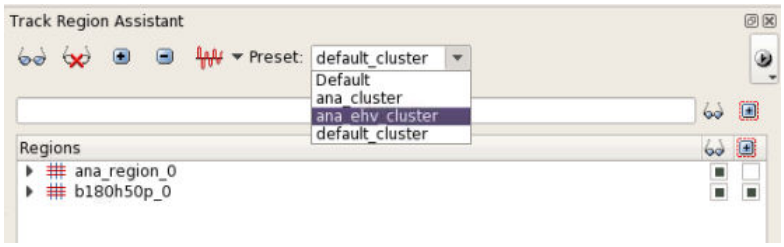
```
    {r1/g1/t2 1 0}
    {r1/g2/t1 1 1}
    {r3/g2/t2 0 0}
}
preset2
{
    {r1/*         true    false}
    {r2/*/t2      false   1}
    {r3/g4/t2     1       1}
}
```

2. Set the `leTrackPresetsFilePath` Tcl preference to specify the path to the preset file. The default value is:

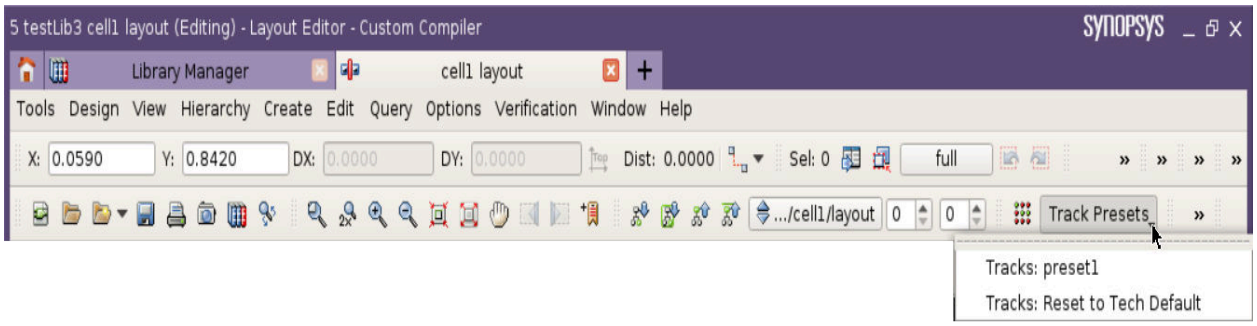
   `$SYNOPSYS_CUSTOM_LOCAL/deTRAPresets.txt`

3. (Optional) Set the `leTrackDefaultSet` Tcl preference to specify the default preset, the preset that is applied when the design is opened.

4. (Optional) To select a non-default track preset, open the **Track Region Assistant** and select from the **Preset** list.



Alternatively, select the preset from the Layout Editor **Advanced** toolbar.



See Also

- About the Track Region Assistant
- Displaying the Track Region Assistant

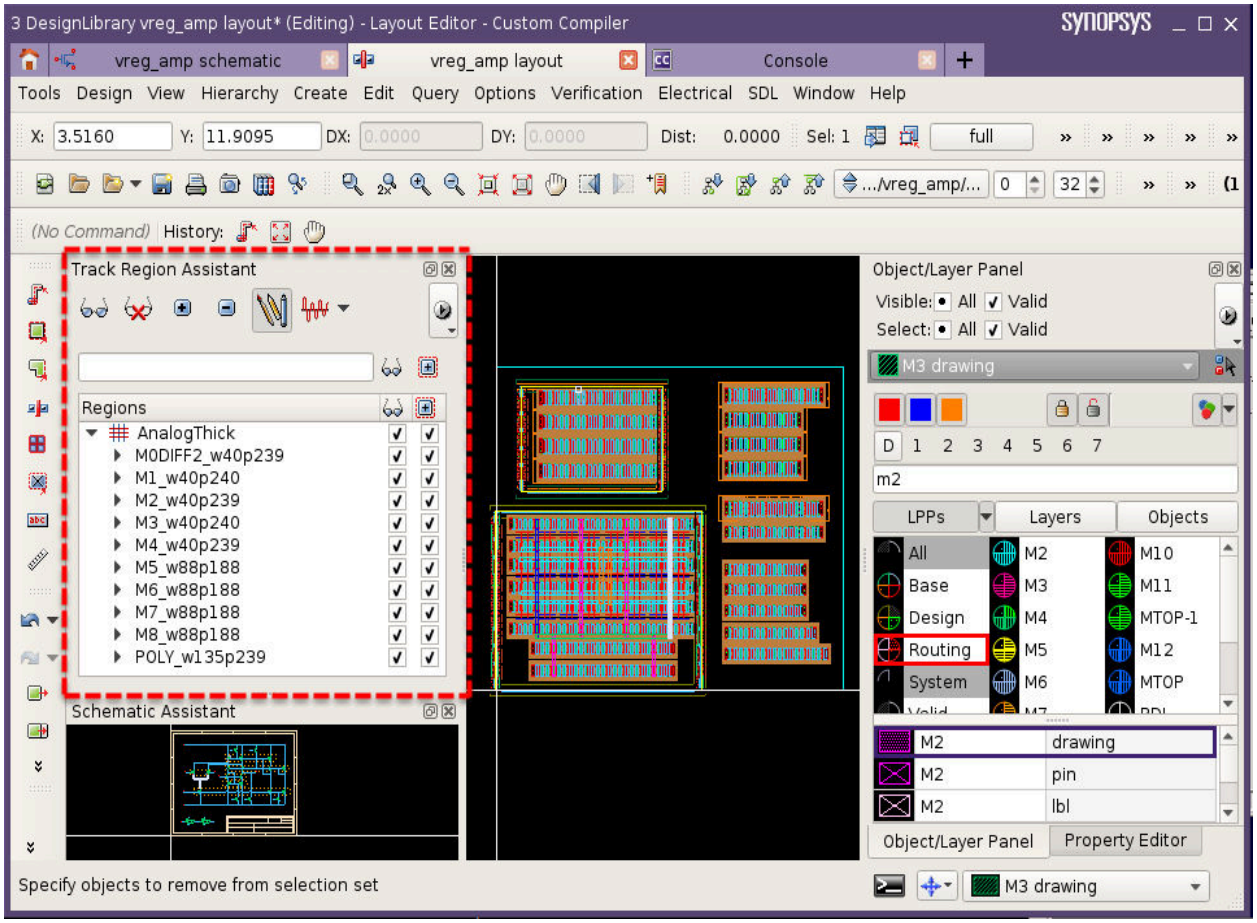
Feedback

- Setting Track Region Assistant Options
- Creating Track Region PCells

Displaying the Track Region Assistant

To display the Track Region assistant:

► In the Layout Editor, choose **Window > Assistants > Track Region Assistant**. The Track Region assistant appears on the left side of the Layout Editor.



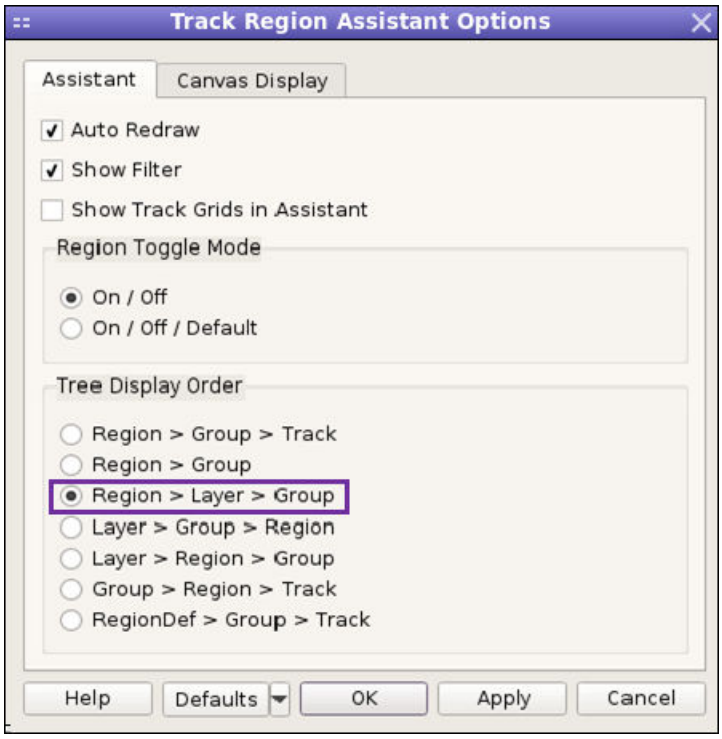
Setting Track Region Assistant Options

From the Track Region assistant, you can open the Track Region Assistant Options dialog box for the current design to view and edit the Track Region Assistant settings.

Feedback

To enter the Track Region options settings:

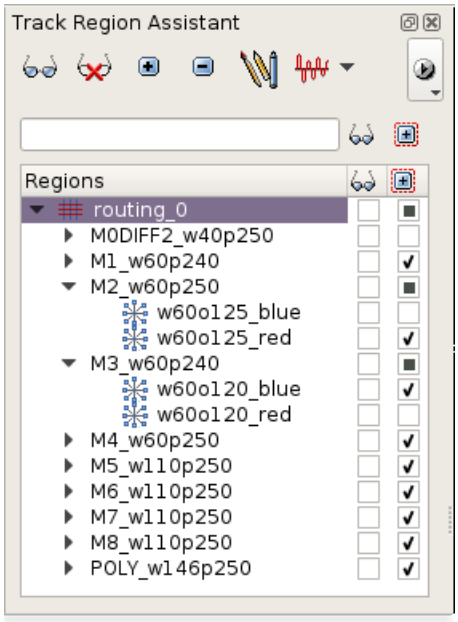
1. Open the Track Region assistant. In a design layout view, choose **Window > Assistants > Track Region Assistant**.
2. Click the toolbar menu and choose **Track Region Assistant Options**. The **Track Region Assistant Options** dialog box opens.



3. On the **Assistant** tab, select the options:

Option	Description
Auto Redraw	Automatically redraws the design on the canvas after you make changes in the Track Region assistant. On by default.
Show Filter	Displays the active track and visibility filter box.

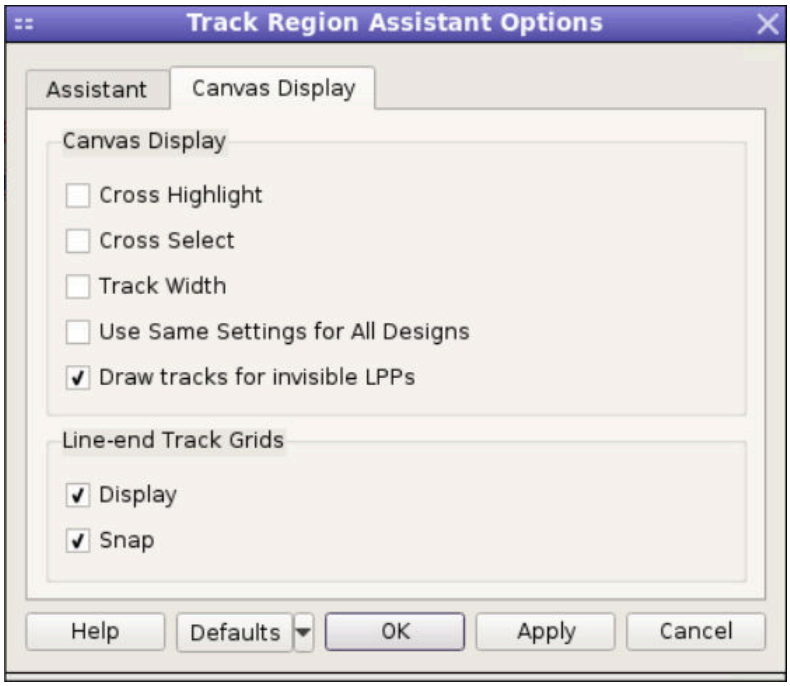
Feedback

Option	Description
Show Track Grids in Assistant	(If track grids are defined) Displays the indicators for low- (starting) and high (ending) points of the grid. See Setting Up Track Grids.
Region Toggle Mode	Specifies the track state options displayed in the assistant: ◦ **On / Off**: On ☑ and Off ☐ only. ◦ **On / Off / Default**: On ☑, Off ☐. and Default ■. The Default ■ state indicates that tracks are using the setting already defined in definition (leTrack) or are using the setting from region-level override if exists. 

Feedback

Option	Description
Tree Display Order	Displays the list of existing track regions in the **Regions** list using one of the following orders: ◦ **Region > Group > Track**. On by default. ◦ **Region > Group** ◦ **Region > Layer > Group** ◦ **Layer > Group > Region** ◦ **Layer > Region > Group** ◦ **Group > Region > Track** ◦ **RegionDef > Group > Track**

4. On the **Canvas Display** tab, select the options:



Option	Description
Cross Highlight	Turns on cross highlight.
Cross Select	Turns on cross selection.
Track Width	Sets the track width displayed on the canvas using the tracks() -width specification. If the track type is set to `default`, the Layout Editor uses the layer `minWidth` specification in the technology. If you do not choose the command, the tracks are drawn using simple lines.

Feedback

Option	Description
Use Same Settings for All Designs	Applies the same **Visibility** and **Activity** settings across all opened designs in the current session. This option eliminates the need to select the settings in each design. The following requirements must be met: ◦ The designs must belong to the same library. ◦ The track names, group definition names, and track region names must be the same across the designs. ◦ The tracks (`leTrack`) and track group (`leTrackGroupDef`) must be defined in the technology file. See Displaying and Hiding Track Regions on the Canvas and Making Track Regions, Track Groups, and Individual Tracks Active. Tcl preference: `leTrackRegionShareSettings`
Draw Tracks for Invisible LPPs	Displays tracks on the layout canvas even if the associated layers (LPP) for the tracks are not visible. Displaying tracks without displaying layers can be useful for troubleshooting designs. For example, a via connects the M1 and M2 shapes, and you want to check that the via and the M2 shape are aligned on the track. You can to hide the M1 shape and display the track to have a better view of the via, M2 shape, and the track. Tcl preference: `leRenderTracksOnInvisibleLPPs`
Display	If tracks have grids, displays them on canvas.
Snap	Snaps segment point to track grids

5. When you are finished, click **OK**.

See Also

- Associating Track Region Definitions With Alternate Foundry Constraint Groups

Creating Track Region PCells

A track region PCell is an editable PCell that contains a set of track tiles. See About Track Region PCells.

Feedback

You can create track region PCells using one of the following methods:

- **Create > Rectangle** command, which requires that you use the Object/Layer Panel to select a track boundary LPP, such as the system-reserved **border track** LPP.
- Track Region Assistant **Create Region PCell** command, which automatically detects and uses the available track boundary LPPs.

This topic describes how to use the Track Region Assistant **Create Region PCell** command. For instructions on using the **Create** > **Rectangle** command, see Creating and Editing Track Region PCells.

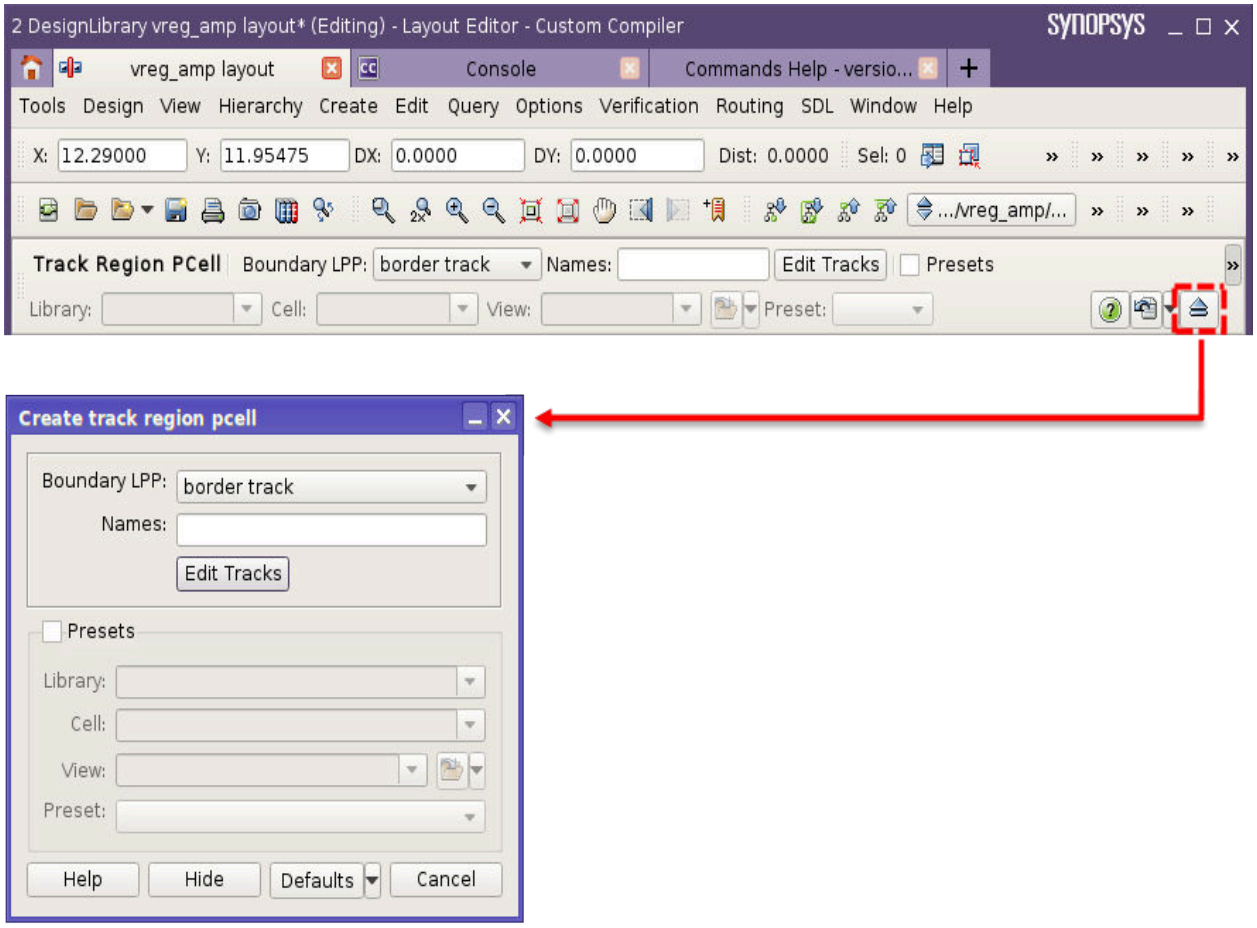
To create a new track region PCell using presets, see Defining Track Presets and Creating Track Region PCells Using Presets.

To use the Track Region Assistant to create track region PCells:

1. (To use presets) In the Track Region Assistant, click , and choose **Track Presets**.
2. In the Track Region Assistant, click , and choose **Create Region PCell**.

   The **Track Region PCell** toolbar appears. To display additional options, click
   Eject. The **Track Region PCell** dialog box opens.

Feedback



3. On the toolbar or in the dialog box, type the **Names** of the new track region PCells. A name is applied to each track region PCell that you create consecutively while the command is active.

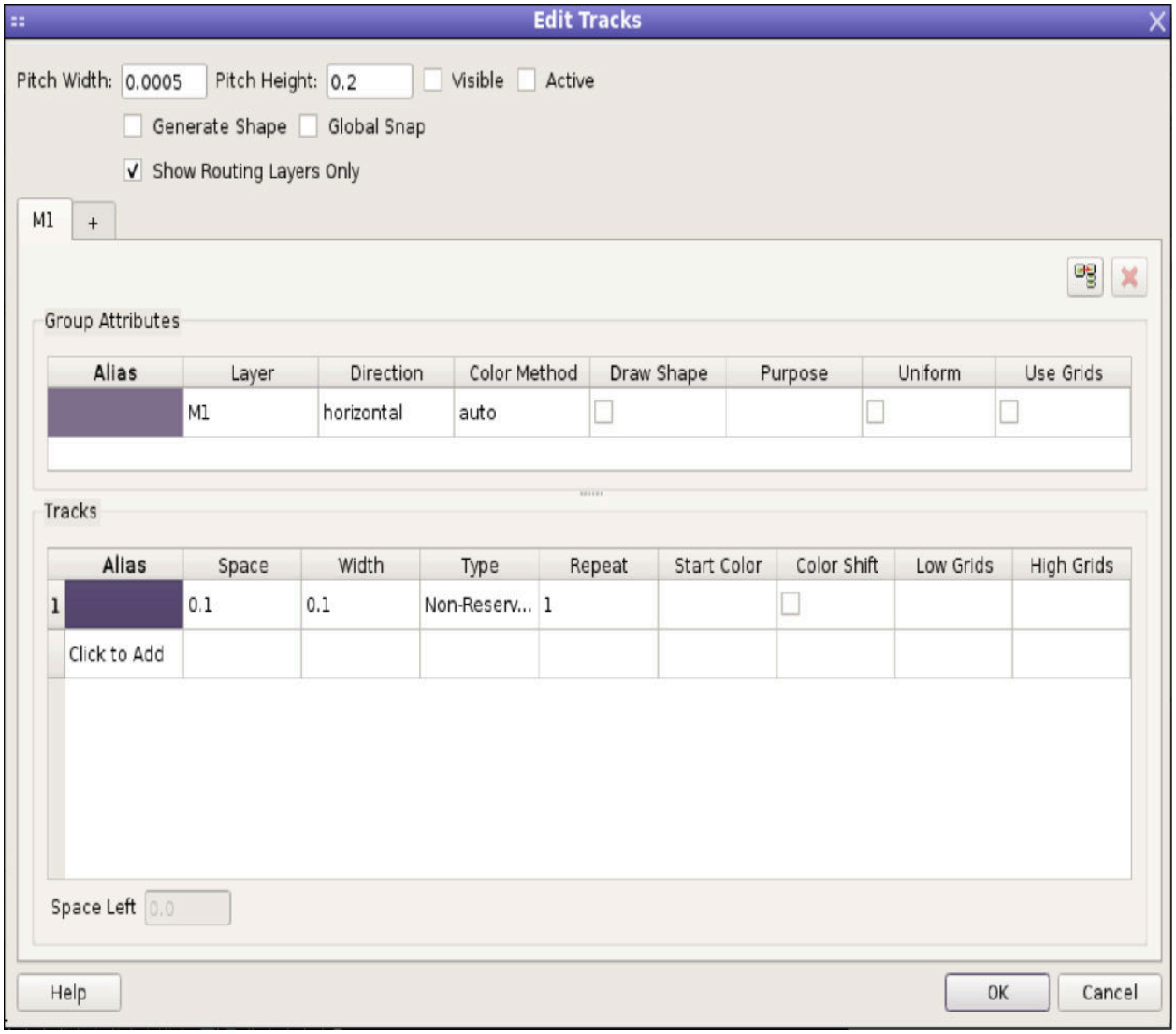
   Note:

   The names appear in the Preset list so that you can reuse the track region PCell definitions later. See Creating Track Region PCells Using Presets.

4. Click **Edit Tracks**. The **Edit Tracks** dialog box appears.

   At the top of the dialog box are general options that apply to the entire track region PCell. Below the general parameters are tabs that contain data for tracks. Each tab represents a track group. Within each tab are data for individual tracks.

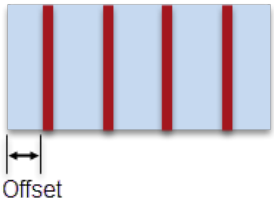
Feedback



5. In the tab, set the following in the **Layer** table (the first table). Required fields have a red background.

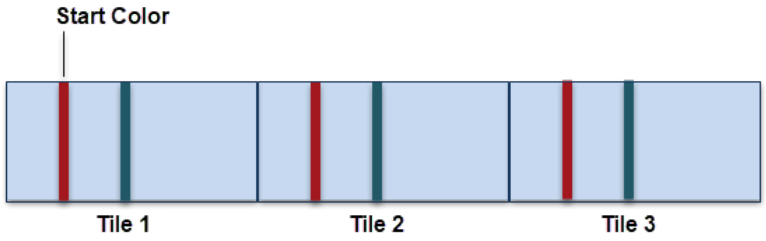
Option	Description
Alias	The Alias names that are applied to the attribute, `displayName` of `trackGroupDef` and `track`. Tcl preference: `leTrackPcellUseAliasName`
Layer	The layer on which the tracks are created.
Direction	The direction of the tracks: **Horizontal** or **Vertical**.

Feedback

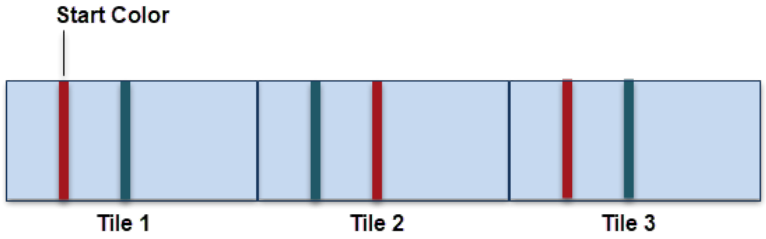
Option	Description
Color Method	The colors of the tracks: ◦ **auto**: Automatically assigns the color. ◦ **manual**: Lets you manually assign the color. ◦ **no color**: Does not assign any color.
Draw Shape	Draws physical shapes inside the PCell master to represent the tracks. Tcl Preference: `leTrackPcellDrawShape`
Purpose	The corresponding purpose of the layer you chose.
Uniform	Specifies whether the properties of a group of tracks are uniform. In the uniform mode, each row in a tab provides the definition for a set of tracks in the tile. The set of tracks have the same track width and spacing. In the example shown, one row (or one track definition) results in the set of four uniform tracks in the tile. In the non-uniform mode, each row in a tab defines a single track. In the example shown, three rows in the same tab results in three tracks with different widths and spacing. If you want a track to appear more than once in the tile, set the **Repeat** option in the Track Parameters Table. **Uniform**  Same: o Track Widths o Track-to-Track Spacing **Non-uniform** Different: o Track Widths o Track-to-Track Spacing
Use Grids	Creates tracks in intervals, with starting points and ending points. If you select **Use Grid**, you must also specify the Low Grids and High Grids. See Before You Start for information about track grids.

6. In the tab, set the following in the track parameters table (the second table in the tab). The parameters that are displayed depend on whether you specified **Uniform** tracks in Step 5. Required fields have a red background.

Feedback

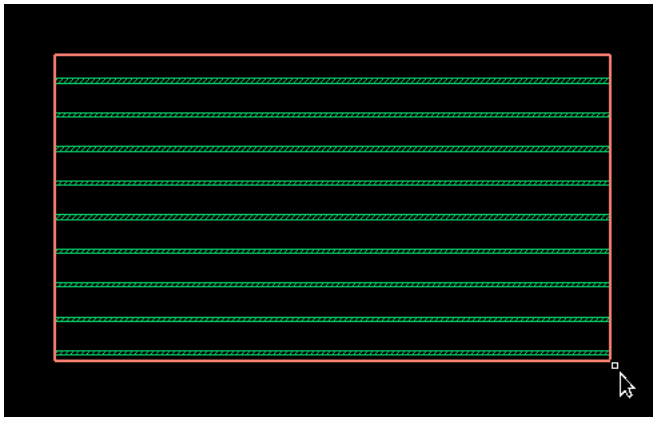
Option	Description
Alias	The Alias names that are applied to the attribute, `displayName` of `trackGroupDef` and `track`. Tcl preference: `leTrackPcellUseAliasName`
Offset (Uniform only)	Distance between the first track and the PCell boundary. The offset always measured from the left or bottom side of the PCell.
Width	Track width.
Space	Distance between adjacent tracks.
Type	Type of track that paths will snap to: ◦ **default**: Wire snapping has no wire width requirements. Wires of any width can snap to the track. with wider wires consuming more available area. ◦ **reserved**: Wire snapping depends on the `leReservedTrackWidthCheck` Tcl preference setting. If the preference is set to `false`, the track operates like a default track, and wires of any width can route on the tracks. When the preference is set to `true`, only wires with non-default-rule nets can route on the tracks, and the wires must match the width of the reserved tracks. ◦ **nonReserved**: Only wires that have the same width as the tracks can snap to the tracks. Tcl Preference: `leTrackPcellTrackType`
Repeat (Non-uniform only)	Number of times to repeat the track. When the tracks are uniform, the repeat value is automatically calculated, so you do not need to specify it.
Start Color	Color of the first track in the tile. In the following example, Start Color is red. 

Feedback

Option	Description
Color Shift	Cycles the Start Color in subsequent tiles. In the following example, the available colors are red and blue. Start Color is red, so blue becomes the First Color in tile 2. 
Low Grids	Specifies the starting point information for gridded tracks: <offset> <lowPoint1> ... <lowPoint*n*> The first value is the offset distance for the grid starting points. The subsequent values are the starting points for the grid. Add a character space between the values. You must select **Use Grids** before setting this option.
High Grids	Specifies the ending point information for gridded tracks: <offset> <highPoint1> ... <highPoint*n*> The first value is the offset distance for the grid ending points. The subsequent values are the ending points for the grid. Add a character space between the values. You must select **Use Grids** before setting this option.
Space Left (Non-uniform only)	(Read-only) Displays the amount of space left in the region after adding the tracks you defined. You can use the information to determine if you can add more tracks in the region.

7. To add a new tab, click the **+** tab.
8. Repeat Step 5 through Step 7 to define new track groups.
9. When you are finished, click **OK**.
10. On the layout canvas, draw the track region PCell.

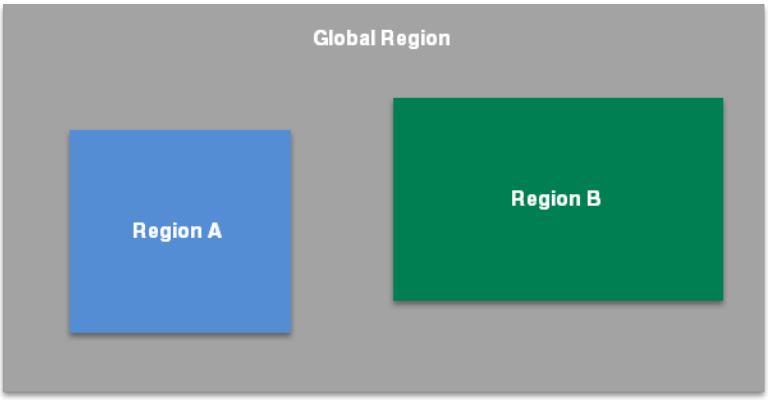
Feedback



11. To edit track region PCells, see Editing Track Region PCells.

Creating Track Regions

You can use the Track Region assistant to create new track regions, such as Regions A and B shown in the example. For instructions on creating a global track region, see Creating a Global Track Region.

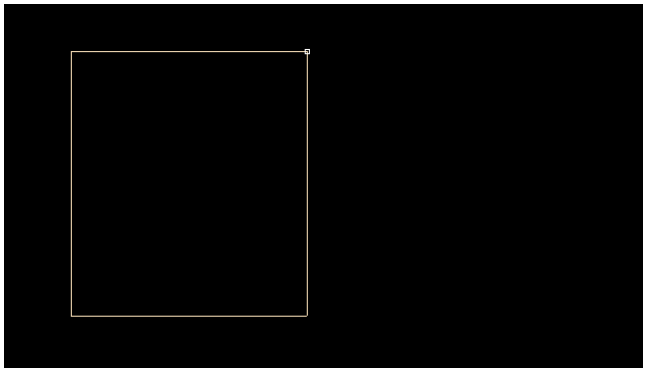


This section describes how to use the Track Region assistant to create track regions. You can also use the `le::createTrackRegion` Tcl command to create track regions. For information, see the *Custom Compiler* Tcl Reference Manual.

To create new track regions:

1. From the Track Region menu ( ), choose **Create Region**.
2. On the canvas, draw a rectangular box at the required location in the design. An outline of the region is displayed on the canvas.

Feedback



To view the tracks in the region, see Displaying and Hiding Track Regions on the Canvas.

Creating a Global Track Region

You can use the Track Region assistant to create a global track region that covers the entire design. For instructions on creating track regions within the global region, see Creating Track Regions.

Caution:

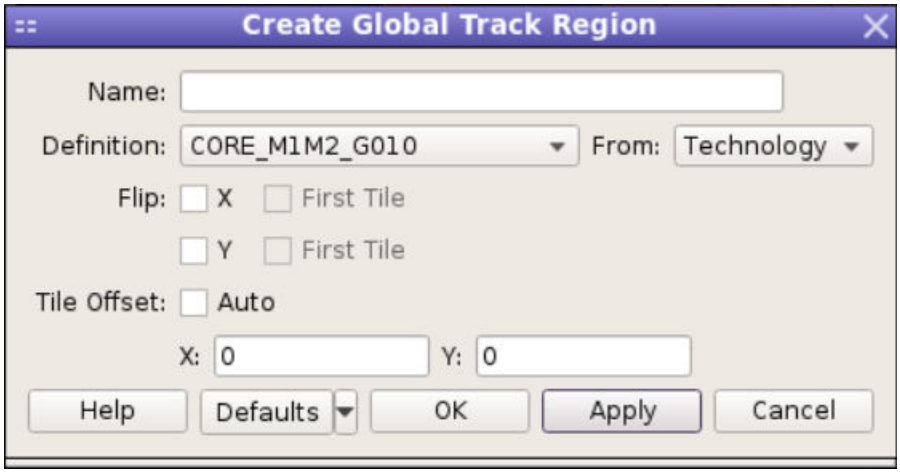
Each design can contain only one global track region. Creating a new global track region overwrites the existing global track region.

	The video demonstrates how to create a global track region.

Feedback

To create a global track region:

1. From the Track Region menu ( ), choose **Create Global Region**. The **Create Global Track Region** dialog box opens.



2. Set the following:

Setting	Description
Name	The name of the global track region. You can provide a unique name, or you can use the system-generated unique name.
Definition	The track region definition to use to create the global region. The available definitions are specified in the technology. For information, see the *Custom Compiler* Technology and Display Resource File Reference Manual.
Flip	Alternating tiles in the region are flipped. Select the flip pattern: ◦ **X**: Flips alternating tiles in the x-direction only. Start the flips with the second tile in the x-direction. ◦ **Y**: Flips alternating tiles in the y-direction only. Start the flips with the second tile in the y-direction. ◦ **X First**: Start the flips with the first tile in the x-direction. ◦ **Y First**: Start the flips with the first tile in the y-direction.

Feedback

Setting	Description
Tile Offset	The offset distance of the first tile. The offset distance is calculated using one of the following methods: ◦ **Auto**: Automatically calculates the offset distance from the bottom-left corner of the PR boundary. ◦ **X** and **Y**: Lets you manually specify the X and Y distance between the first tile and the bottom-left corner of the global track region. Track grids on perpendicular tracks should also apply tile offset. For example, grids on vertical tracks should apply tile offset Y. Grids on horizontal tracks should tile offset X.

3. When you are finished, click **Create**. The global track region is created.

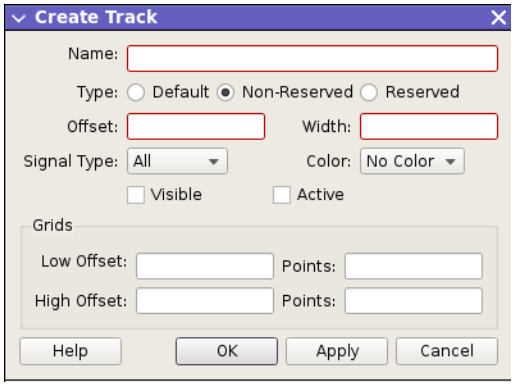
   To view the tracks in the global track region, see Displaying and Hiding Track Regions on the Canvas.

Creating Tracks

You can use the Track Region assistant to create tracks. You can also use the `le::createTrack` Tcl command to create tracks. For information, see the *Custom Compiler* Tcl Reference Manual.

To create a new track:

1. From the Track Region menu ( ), choose **Create Track**. The **Create Track** dialog box opens.



2. Set the following options:

Setting	Description
Name	The name of the track. You can provide a unique name, or you can use the system-generated unique name.

Feedback

Setting	Description
Type	The track type. **Default**, **Non-Reserved**, or **Reserved**.
Offset	Offset of the track from the region tile lower left.
Width	Width of the track. Disabled for **Default** Type.
Signal Type	The signal type from the following list: All, Power, Ground, Clock, Signal, Tie Off, Tie High, Tie Low, Analog, Scan, Reset. This option is not available in the **Edit Track** dialog box. **Note:** Saving the design will save the signal type selection.
Color	Color of the track. Default is **No Color**. Hidden if technology does not support color.
Visible	Default visible state of the track.
Active	Default active state of the track.
Grids	
Low	**Offset**: Offset distance for grid start points. **Points**: Start points of the grid.
High	**Offset**: Offset distance for grid endpoints. **Points**: Endpoints of the grid.

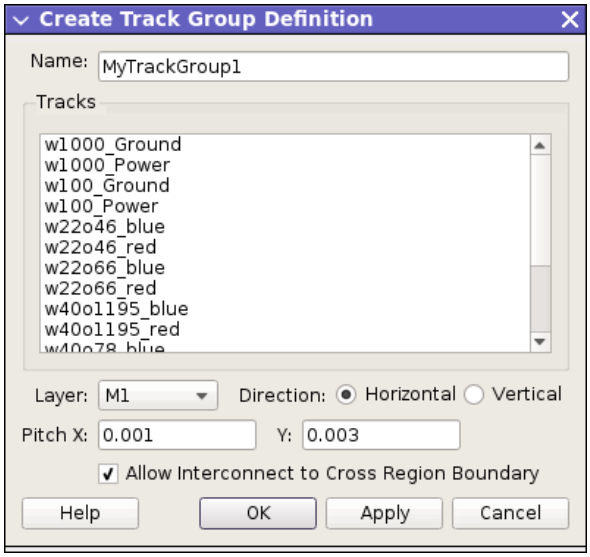
Creating Track Group Definitions

You can use the Track Region assistant to create track group definitions. You can also use the `le::createTrackGroupDef` Tcl command to create track group definitions. For information, see the *Custom Compiler* Tcl Reference Manual.

Feedback

To create a new track:

1. From the Track Region menu ( ), choose **Create Track Group Definition**. The **Create Track Group Definition** dialog box opens and displays a list of tracks.



2. From the list of tracks, select the tracks to include in the group. Then, set the following options:

Setting	Description
Name	The name of the track group.
Layer	All tracks in the group definition are on this layer.
Direction	Direction of the tracks, **Horizontal** or **Vertical**.
Pitch	Size of the group, in the **X** and **Y** directions. You can specify 0 for X pitch if the track direction is horizontal, and 0 for Y pitch if it is vertical.
Allow Interconnect to Cross Region Boundary	Permits interconnects to cross global and local track regions. See Crossing Track Regions.

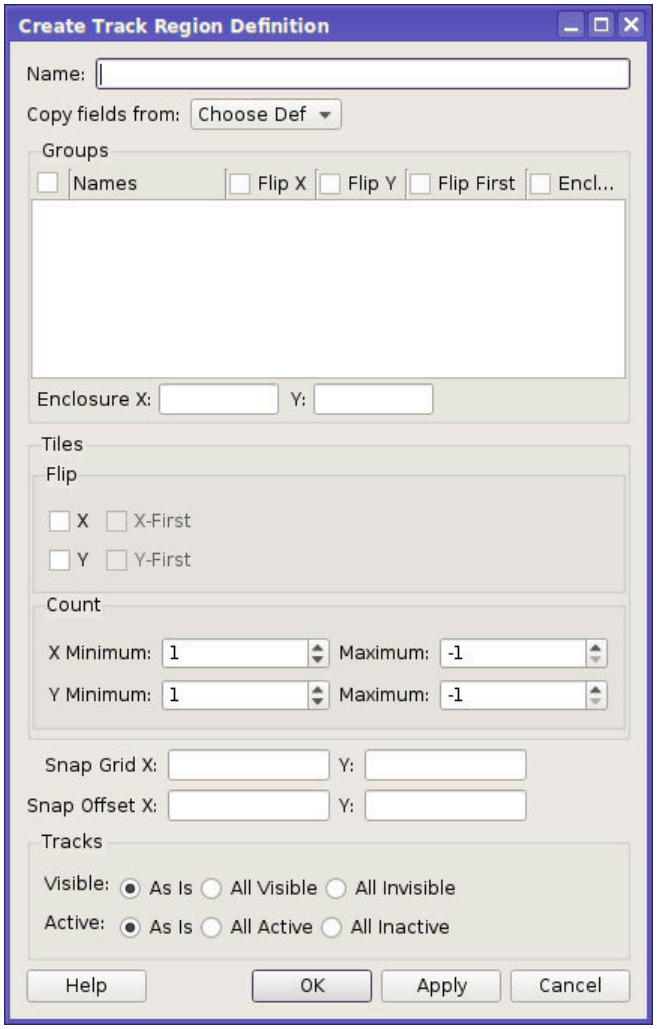
Creating Track Region Definitions

You can use the Track Region assistant to create track region definitions. You can also use the `le::createTrackRegionDef` Tcl command to create track group definitions. For information, see the *Custom Compiler* Tcl Reference Manual.

Feedback

To create a new track:

1. From the Track Region menu ( ), choose **Create Track Region Definition**. The **Create Track Region Definition** dialog box opens and displays a list of track groups.



2. From the list of track **Groups**, select the track groups to include in the track region. Then, set the following options:

Setting	Description
Name	The name of the track region.
Copy fields from	Allows you to copy and modify an existing track region definition instead of creating a similar track region definition from scratch.

Setting	Description
Groups	List of track groups.
Enclosure X, Y	Minimum X and Y distance between the region boundary and the first and last track in the region.
Flip X	Determines if tiles in the region are flipped about the x-axis.
Flip Y	Determines if tiles in the region are flipped about the y-axis. Alternating tiles in the region are flipped.
Tile Count	
X	**Minimum**: Minimum number of least common multiple groups repeated in the x-direction. **Maximum**: Maximum number of least common multiple groups repeated in the x-direction.
Y	**Minimum**: Minimum number of least common multiple (LCM) times groups repeated in the y-direction. **Maximum**: Maximum number of least common multiple (LCM) times groups repeated in the y-direction.
Grid Groups	Associates track grids with track groups in track region definitions. The specified groups' layer and grids' layer should be the same.
Tile Offset	Specifies the offset for the tiles in the track region. Line-end grids on the perpendicular tracks should also apply the offset.
Snap Grid	Snap grid in **X** and **Y** directions.
Snap Offset	Track region offset distance from the design axes.
Tracks	
Visible	Select the visibility state of the tracks.
Active	Select the activity state of the tracks.

Deleting Track Regions

To delete a track region:

1. In the **Regions** list, select the track region you want to delete.
2. From the Track Region menu ( ), choose **Delete**. The track region name is removed from the **Regions** list.

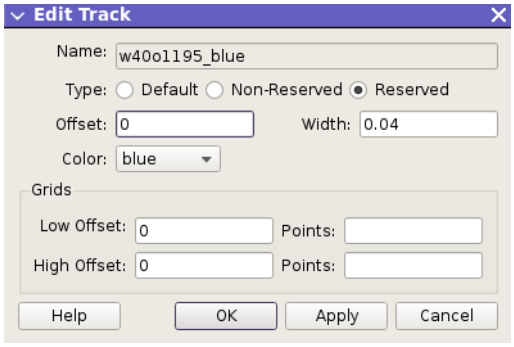
Feedback

Editing Tracks

You can use the Track Region assistant to edit tracks. You can also use the `le::editTrack` Tcl command to create tracks. For information, see the *Custom Compiler* Tcl Reference Manual.

To edit a track:

1. From the Track Region menu ( ), choose **Edit Track**. The **Edit Track** dialog box opens.



2. You can now edit the track options described in Creating Tracks, except **Signal Type**.

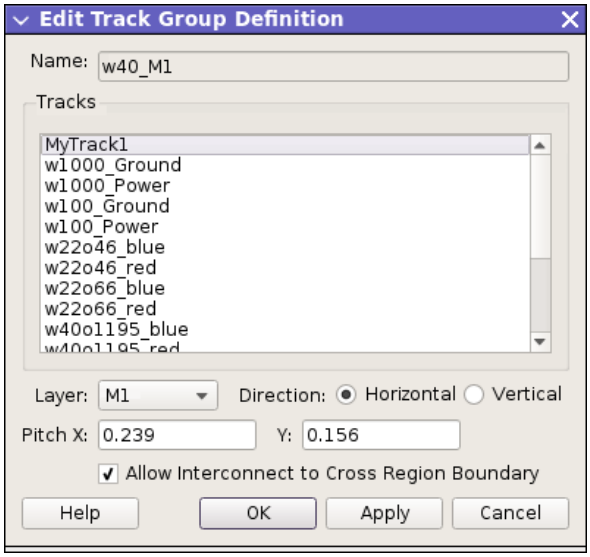
Editing Track Group Definitions

You can use the Track Region assistant to edit track group definitions. You can also use the `le::editTrackGroupDef` Tcl command to create track group definitions. For information, see the *Custom Compiler* Tcl Reference Manual.

Feedback

To edit a track:

1. From the Track Region menu (), choose **Edit Track Group Definition**. The **Edit Track Group Definition** dialog box opens.



2. Edit the track group definition options described in Creating Track Group Definitions.

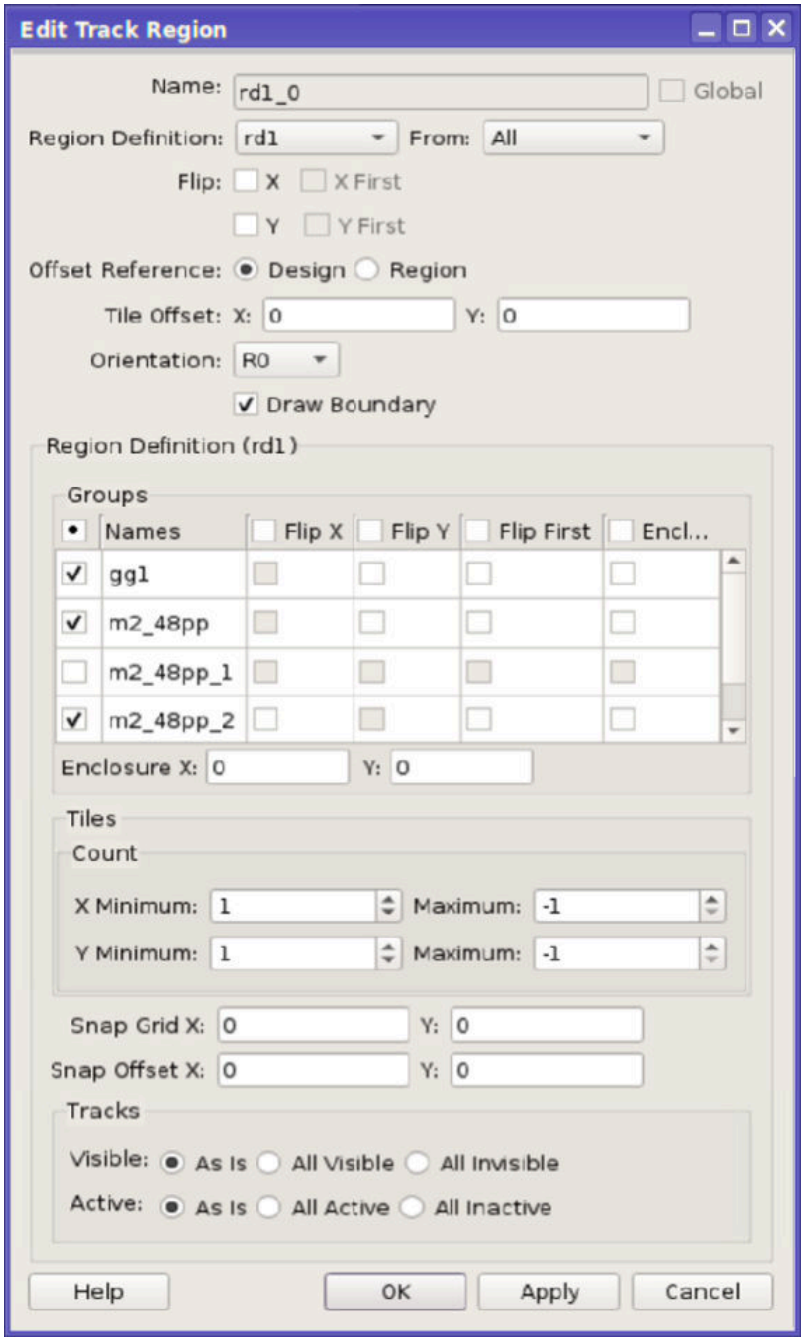
Editing Track Region Definitions

You can use the Track Region assistant to edit track region definitions.

Feedback

To edit a track:

1. From the **Track Region** menu ( ), choose **Edit Track Region Definition**. The **Edit Track Region Definition** dialog box opens.



2. Edit the track region definition options.

Feedback

See Creating Track Region Definitions.

3. **Orientation**: Specify the orientation of the region. Only **R0**, **R180**, **MX**, and **MY** are the valid values and the default is **R0**.

   This orientation attribute does not affect the boundary of track region. The tracks are transformed according to the orientation of the track region.

   Note:

   - Both Track Region and Global Track Region support Orientation.
   - Tiles (and groups) are constructed from right to left and from top to bottom for a **R180** region; from right to left and from bottom to up for an **MY** region; from left to right and from top to bottom for an **MX** region.
   - Tile offset is based on the upper-right/lower-right/upper-left point of the boundary for **R180/MY/MX** regions respectively (when track offsets are relative to the region boundary).
   - The orientation is **R0** for an existing track region already created in the design.

4. **From**: Filter the region definitions from design, technology, or both (technology and design). Valid values are **Design**, **Technology**, and **All**.

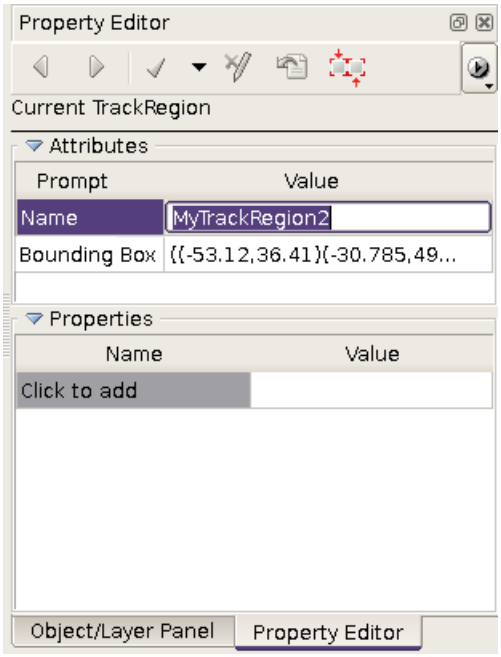
Editing Track Region Properties

You can change the name and the dimensions of a selected track region using the Property Editor.

To edit the track region properties:

1. Display the Property Editor. For instructions, see Displaying the Property Editor.
2. On the canvas, select the track region. The properties of the selected track region appear in the Property Editor.

Feedback



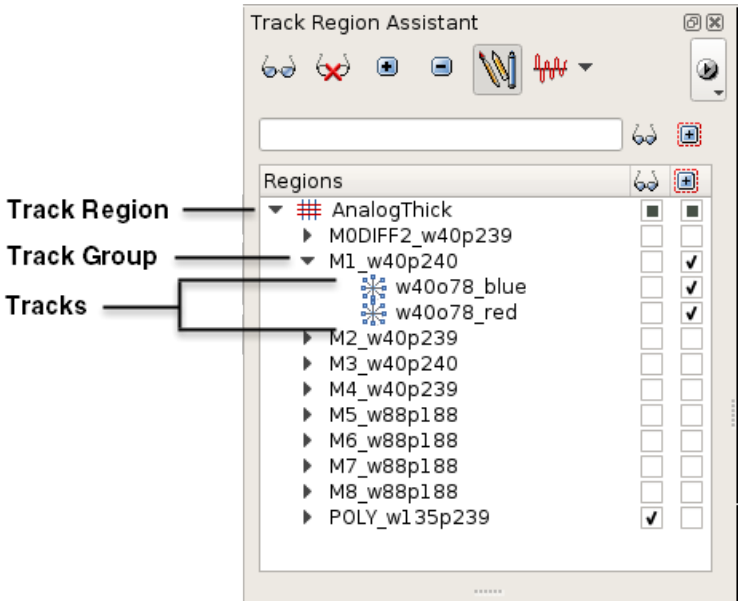
3. In the Property Editor, locate the Attributes section, and then edit the following:

Attribute	Description
Name	The track region name.
Bounding Box	The locations (x, y) of the top-left corner and bottom-right corner of the track region.

Filtering the List of Track Regions

The Track Region assistant displays the list of existing track regions in the **Regions** list. You can expand each track region name to view the track groups and individual tracks within the group.

Feedback



To filter the list of track regions, type the name of the region in the filter box. To display only active or visible tracks with the name you provided, click **Show only visible tracks** or **Show only active tracks** . To display tracks of specific signal types, click **Filter tracks by signal types** , and select the signals of interest.

Displaying and Hiding Track Regions on the Canvas

You can display and hide track regions, track groups, or individual tracks. You can also choose to render or not render the track color.

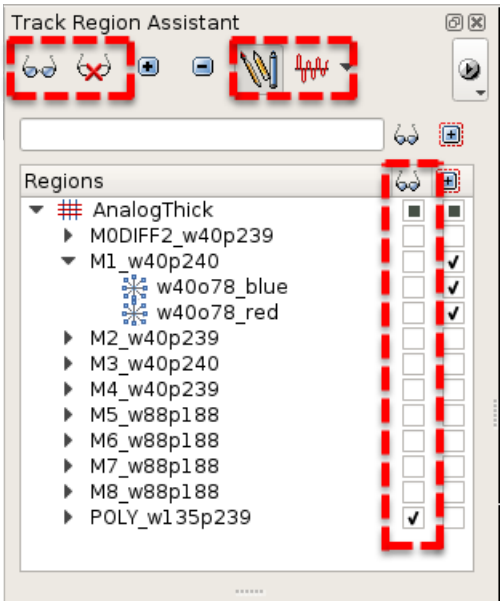
Note:

Saving the design will save the Visibility states.

Feedback

To display or hide track regions or toggle the color rending:

1. Filter the list of track regions. See Filtering the List of Track Regions.
2. Click or select one of the following:



Button or check box	Description
(on the toolbar)	Displays all track regions.
(on the toolbar)	Hides all track regions.
	Displays (☑) or hides (☐) the required track region, track group, or individual tracks. **Note:** The Layout Editor displays relevant candidate tracks as you create, copy, move, or stretch objects, even if **Visibility** is turned off for those tracks. To turn off the automatic display of candidate tracks, set the `leDisplayInCommandTrackPatterns` Tcl preference to `false`.
(on the toolbar)	Toggles track rendering. When enabled, track colors are displayed. When disabled, tracks are visible but do not have colors.

Feedback

Making Track Regions, Track Groups, and Individual Tracks Active

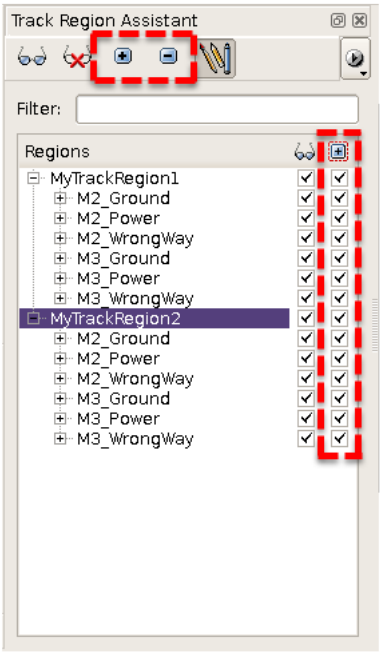
To ensure that shapes snap to tracks, you must make the tracks active.

Note:

Saving the design will save the Activity states.

To make a track region, a track group, or individual tracks active:

1. Filter the list of track regions. See Filtering the List of Track Regions.
2. Click or select one of the following:



Button or check box	Description
(on the toolbar)	Activates all track regions.

Feedback

Button or check box	Description
(on the toolbar)	Deactivates all track regions.
	Activates ( ) or deactivates ( ) specific track regions, track groups, or individual tracks. **Important:** If you want the Layout Editor to display relevant candidate tracks as you create, copy, move, or stretch objects, make sure those track regions are active. To turn off the automatic display of candidate tracks, set the `leDisplayInCommandTrackPatterns` Tcl preference to `false`.

Transaction History Assistant

The Transaction History assistant lists all the command operations that you performed in reverse chronological order. You can use the list to track the operations you performed. You can also use the list to return the design to any previously committed state in this design session.

The Transaction History assistant stores up to 256 command operations per design. When the maximum number of commands is reached, the assistant releases the oldest record so that it can add a new record.

In an edit-in-place context, the Transaction History assistant displays the list of commands of the design that is being edited in place.

The list of commands in the Transaction History assistant is not retained across Custom Compiler sessions. When you close a design, the list in the Transaction History assistant is cleared.

By default, the list of commands in the Transaction History assistant is retained after a Save design operation. To clear the list after a Save design operation, use the deUndoClearOnSave Tcl preference.

- Displaying the Transaction History Assistant
- Using the Transaction History List

Feedback

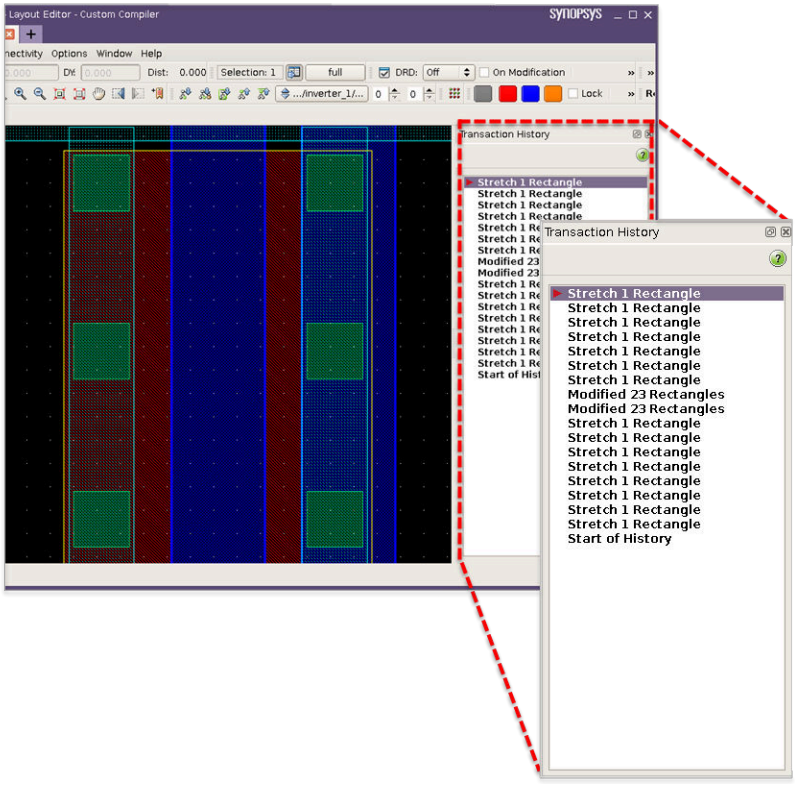
	The Transaction History assistant lists all the command operations that you performed in reverse chronological order. This video tutorial shows how to display the Transaction History assistant and use the Transaction History List to retrace design operations.

Displaying the Transaction History Assistant

To display the Transaction History assistant:

► In any layout, schematic, or symbol view, choose **Window > Assistants > Transaction History**. The assistant appears on the right side of the window.

If you have already executed commands, those commands are listed in the assistant. Otherwise, the assistant is empty and will begin recording the command operations.



If you are using scripts, transactions involving direct calls to the OpenAccess Tcl API are marked with the title "unnamed".

Only operations that change data is recorded as transactions. The process of saving is transparent; saving records no transaction. (The save operation changes the on-disk data, but not the in-memory data).

Feedback

You can use this list to return the design to any previously committed state in this design session.

Using the Transaction History List

The following table lists the operations you can perform in the Transaction History assistant.

To:	Perform this action:
Undo a command operation	Press U. This undoes the transaction.
Redo a command operation	Press Shift-U. This reexecutes the transaction. You can use the redo operation to step forward through a transaction history sequence one command at a time.
Return to a preceding state	Select the transaction after it.
Jump to a later state	Select a later transaction.
Take design development in a new direction from any state	Jump to that state, and then execute new commands. (Caution: The system erases all succeeding transactions from the previous history list).
Save a version of the design that matches a state	Return to the state. Then save the design.

8

Tools for Designing

This chapter describes how to use Custom Compiler design tools to facilitate design development. Design tools include editors and features that operate independently.

This chapter contains the following topics:

- Deciding Which Tool to Use
- Bookmark Manager Tool
- Constraint Manager Tool
- Display Resource Editor Tool
- Hierarchy Editor Tool
- Library Manager Tool
- Parameter Definition Editor Tool
- Search and Replace Tool
- Technology Manager Tool
- Text Viewer Tool
- Design Technology Co-optimization Tool

Deciding Which Tool to Use

The following table lists tools in alphabetical order, and describes how each tool helps with designing. Choose a tool when you want to use its capabilities.

Table 92 Design Tools

Tool	Design Development
Bookmark Manager Tool	Sets new bookmarks, and loads and saves existing bookmarks.

Feedback

Table 92 Design Tools (Continued)

Tool	Design Development
Constraint Manager Tool	Manages process constraints. (Use the Constraint Editor Assistant to create and apply design constraints). For information about placement constraints and the Constraint Manager, see the *Custom Compiler SDL and Visually Assisted Automation Guide* for details. For information about routing constraints, see the *Custom Compiler Custom Router User Guide* for details.
Display Resource Editor Tool	Assigns display resources; creates display colors, stipples, and line styles; loads and merges display resource files; defines packets and packet aliases.
Hierarchy Editor Tool	Controls the number of instances edited at one time.
Liberty Data Editor	Controls settings and values for a Liberty library file. See the *Custom Compiler and IC Compiler Co-Design Assistant User Guide* for details.
Library Manager Tool	Creates and manages libraries, cells, cellViews, and cell categories.
Parameter Definition Editor Tool	Displays, defines, and sets values for parameters.
Search and Replace Tool	Finds and optionally replaces components in a design.
Technology Manager Tool	Imports and exports technology data, technology files; resets referenced technologies.
Text Viewer Tool	Displays text output for all running jobs that produce it.

Bookmark Manager Tool

The Bookmark Manager controls bookmarks.

This topic describes the following:

- Adding a Bookmark
- Displaying the Bookmark Manager Tool
- Managing Bookmarks

Feedback

	The video demonstrates an example of how to use the Bookmark Manager.

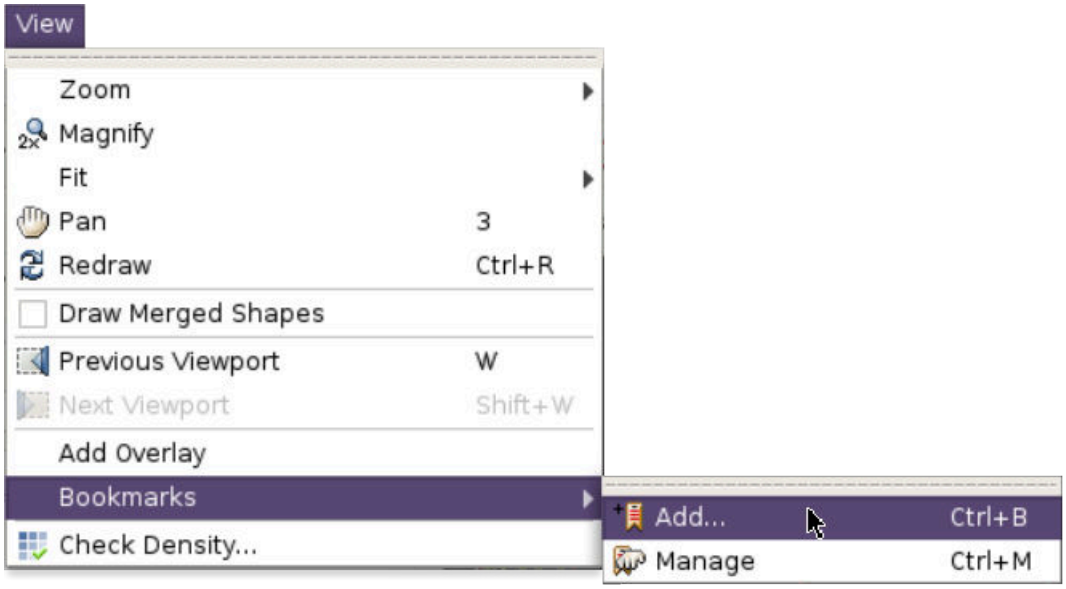
Adding a Bookmark

You can add a bookmark to save the association between this viewport and this design context. You can add bookmarks to any open design.

The Bookmark Manager does not need to be open to add a bookmark.

To add a bookmark:

1. In the design window displaying the context and viewport you want to bookmark, choose **View > Bookmarks > Add**.



   (Alternatively, you can use the key binding Ctrl+B).

2. Enter a bookmark name using a single unbroken string of characters.
3. Click **Create**.

The tool creates the bookmark, and adds it to the list of bookmarks accessible through any editor.

Feedback

Displaying the Bookmark Manager Tool

Display the Bookmark Manager to load, save, delete, or rename bookmarks.

To display the Bookmark Manager:

- From the **Console** tab, choose **Tools > Bookmark Manager**.
- From a design window, choose **View > Bookmarks > Manage**.

Managing Bookmarks

You can manage bookmarks to control stored design data and viewport combinations.

These sections cover:

- Loading Bookmarks
- Saving Bookmarks
- Deleting Bookmarks
- Renaming Bookmarks

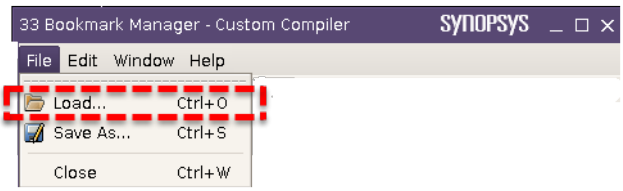
(To add a bookmark, see Adding a Bookmark).

Loading Bookmarks

You can load bookmarks to make them available for use in this session. By default, bookmarks stored in the current working directory file `bookmarks.xml` loaded automatically on session start. The `SYNOPSYS_CUSTOM_LOCAL` environment variable controls this operation.

To load bookmarks:

1. Begin by displaying the Bookmark Manager Tool. From the **Console** tab, choose **Tools > Bookmark Manager**.
2. In the Bookmark Manager, choose **File > Load**.



Feedback

3. Browse to choose the correct XML file.
4. Click **Open**.
5. When you are done, choose **File > Close**.

Saving Bookmarks

You can save bookmarks to retain them for future sessions.

Use the Bookmark Manager to save bookmarks. To add a bookmark, see Adding a Bookmark.

To save bookmarks:

1. Begin by opening the Bookmark Manager Tool. From the **Console** tab, choose **Tools > Bookmark Manager**.
2. In the Bookmark Manager, choose **File > Save As**.
3. Enter a new file name, or browse to choose a bookmark XML file you want to overwrite.
4. Click **Save**.
5. When you are done, choose **File > Close**.

Deleting Bookmarks

You can delete bookmarks to remove them from use.

Use the Bookmark Manager to delete a bookmark.

To delete bookmarks:

1. Begin by opening the Bookmark Manager Tool. From the **Console** tab, choose **Tools > Bookmark Manager**.
2. In the Bookmark Manager, select the bookmark.
3. Choose **Edit > Delete**. (Alternatively, you can use the bindkey Ctrl+D).
4. When you are done, choose **File > Close**.

Renaming Bookmarks

You can rename a bookmark to change the name of a stored design and viewport combination.

Use the Bookmark Manager to rename a bookmark.

Feedback

To rename a bookmark:

1. Begin by opening the Bookmark Manager Tool. From the **Console** tab, choose **Tools > Bookmark Manager**.
2. In the Bookmark Manager, select the bookmark.
3. Choose **Edit > Rename**.
4. Enter the new name.
5. When you are done, choose **File > Close**.

Constraint Manager Tool

Constraints are rules that designs must follow to ensure correct layout fabrication and operation. The constraints are either global or design-specific. Global constraints are defined in the technology database, but you can use the Constraint Manager to view and manage these constraints. You use the Constraint Editor to create and manage design-specific constraints.

This topic describes the following:

- Constraint Types and Sources
- About the Constraint Manager
- Displaying the Constraint Manager
- Adding and Deleting Process Constraint Parameters
- Grouping a Set of Process Constraints
- Specifying User-Defined Constraint Groups
- Enabling Alternate Foundry Constraints
- Constraint Manager Reference

See Also

- Technology and Display Resource Files
- Constraint Editor Assistant

Feedback

Constraint Types and Sources

Different constraint types work together in the Custom Compiler environment to ensure correct-by-construction designs while providing some flexibility to accommodate special requirements.

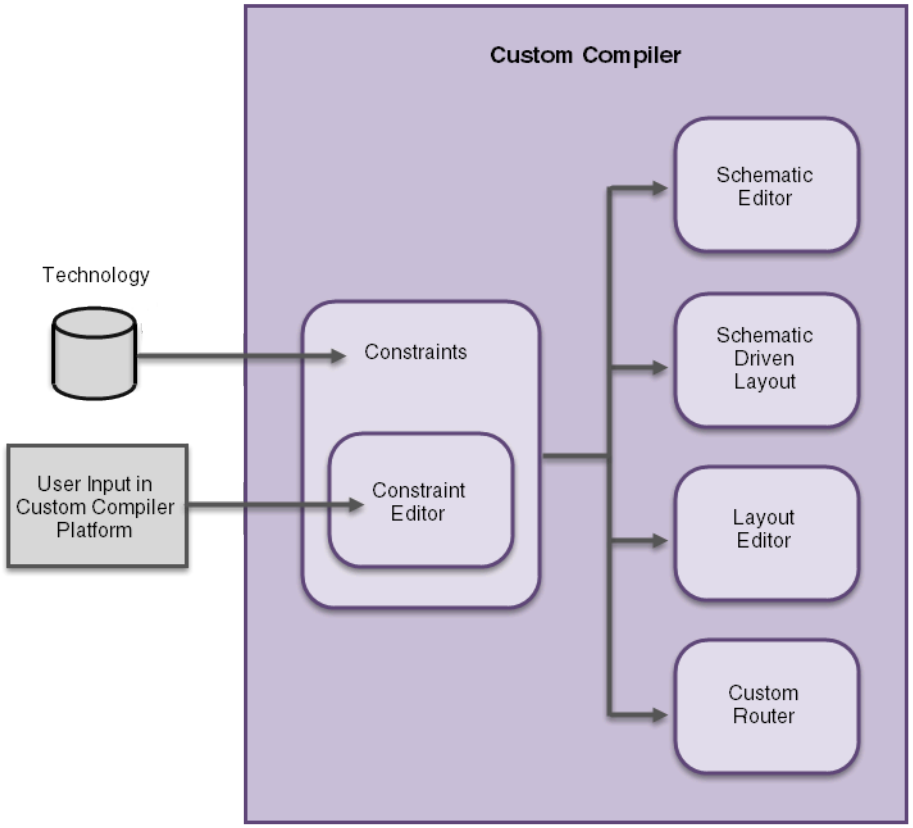
The Custom Compiler environment interacts with the following constraint types:

Constraint Type	Description	Source
Process	A set of global rules and constraints that all designs must follow during layout and routing, such as spacing rules and clearance rules. For information about the rules and constraints in the technology database, see the Custom Compiler Technology and Display Resource File Reference Manual.	Technology database
Design-specific	Constraints that the Layout Editor, Schematic Editor, and Custom Router must follow instead of or in addition to the process rules in the technology database. The design-specific constraints further restrict process constraints. You cannot set design-specific constraints that violate process constraints.	Constraint Editor Assistant

Feedback

Figure 53 summarizes the concept of the different constraint sources and the input into the various Custom Compiler products.

Figure 53 Sources of Design Rules Used by Custom Compiler Tools



About the Constraint Manager

Although process constraints are stored in the technology database, you can use the Constraint Manager to access the technology data.

The Constraint Manager is a graphical user interface that lets you:

- Display the process constraints and default design constraints.
- Create, edit, and delete default constraints in a design or technology database.
- Create and manage process constraint groups.
- Create and manage user-defined constraint groups.

For information about process constraints in the technology database, see the *Custom Compiler Technology File and Display Resource Reference Manual*.

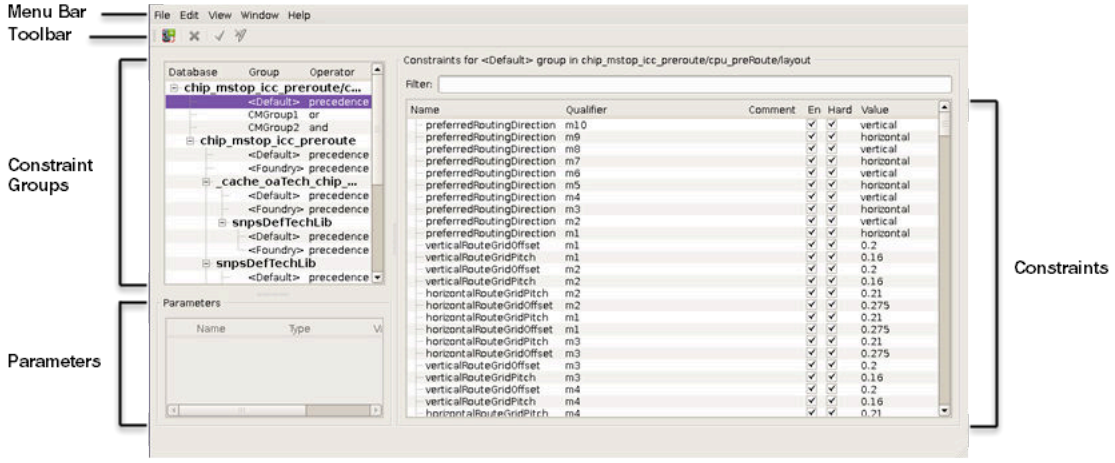
Feedback

Displaying the Constraint Manager

You can access the Constraint Manager from the Layout Editor. This section describes how to display the Constraint Manager. You can also display the Constraint Manager using the cm::showConstraintManager Tcl command.

To display the Constraint Manager:

1. Open the layout view of a design.
2. Choose **Tools > Constraint Manager**. The Constraint Manager opens.



The Constraint Manager consists of the following:

Component	Description
Menu Bar	Contains menus of commands. For more information, see Constraint Manager Menus.
Toolbar	Displays button commands. For more information, see Constraint Manager Toolbar.
Constraint Groups	Displays list of constraint groups. If you select a constraint group, the list of constraints that belong in the group appears in the Constraints pane. To group a set of constraints, see Grouping a Set of Process Constraints.
Constraints	Displays the list of constraints associated with the selected constraint group.
Parameters	Lists the parameters associated with the constraint you select in the Constraint pane. To add or delete parameters, see Adding and Deleting Process Constraint Parameters.

Feedback

Adding and Deleting Process Constraint Parameters

Although process constraint parameters are set in the technology database, you can adjust them using the Constraint Manager.

This topic describes the following:

- Adding Parameters
- Deleting Parameters

Adding Parameters

To add a parameter to an existing constraint:

1. In the Constraint Group pane, select the required constraint group.

   The lists of associated constraints appear in the Constraint pane.
2. In the Constraints pane, select the constraint in which you want to add parameters.
3. In the Parameters pane, click **Click to add**. A row in the Parameters table becomes editable.
4. In the **Name** box, type a name for the new parameter.
5. In the **Type** list, choose the data type.
6. In the **Value** box, type a value to assign to the new parameter.
7. On the toolbar, click **Commit** ( ).

   The text font changes from bold to plain to indicate that the new parameter is associated with the constraint.

Deleting Parameters

To delete a parameter from an existing constraint:

1. In the Constraint Group pane, select the required constraint group.

   The lists of associated constraints appear in the Constraint pane.
2. In the Constraints pane, select the constraint whose parameters you want to delete.
3. In the Parameters pane, select the parameter you want to delete. The row in the table is highlighted.

Feedback

4. On the toolbar, click **Delete** ( ). A strikethrough mark appears over the parameter type and value.
5. On the toolbar, click **Commit** ( ). The parameter is deleted.

Grouping a Set of Process Constraints

You can group a set of constraints within the design scope instead of having to add or update constraints within the technology scope. You can create constraint groups using one of the following:

- Technology file
- Constraint Manager

Regardless of the method you use, the constraint group you create is available for selection in the **Constraint Group** toolbar.

This section describes how to create constraint groups using the Constraint Manager. For instructions on how to create groups in the technology file, see the *Custom Compiler™ Technology and Display Resource File Reference Manual*. For information about the **Constraint Group** toolbar and constraint group selection, see Specifying User-Defined Constraint Groups.

To group a set of constraints:

1. Open the **Constraint Manager**. In the layout view of a design, choose **Tools > Constraint Manager**.
2. Choose **File > New Group**. Alternatively, on the Constraint Manager toolbar, click **New Group** ( ).

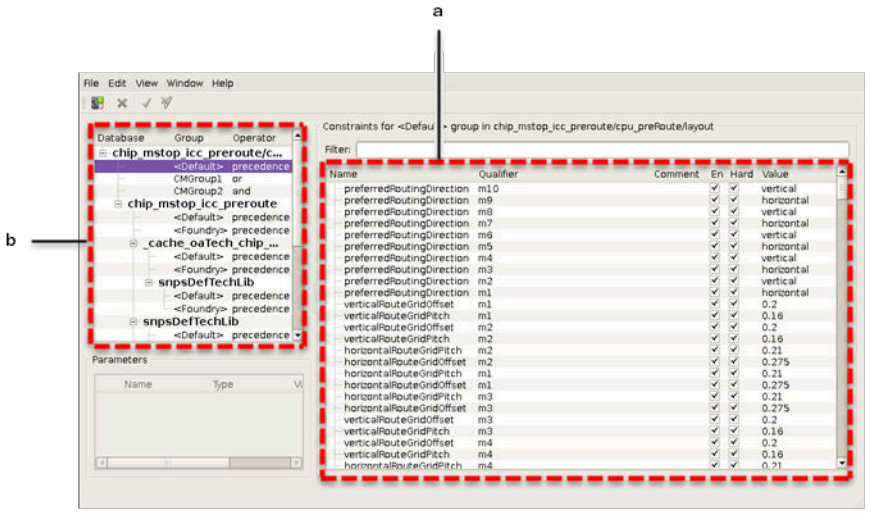
   The **Create Constraint Group** dialog box opens.

Feedback

3. Set the following options:

Option	Description
Database	(Optional) The database to associate with the constraint group. Choose one of the following: ◦ **Design**: The design in which the new constraint group is created. ◦ **Tech: <lib_name>**: The technology libraries associated with the active design.
Name	The name of the new constraint group.
Operator	The logical operator to apply to the constraint group. Choose one of the following: ◦ **precedence**: Uses the first found constraint. ◦ **and**: All hard constraints must be met. ◦ **or**: At least one of the hard constraints must be met.

4. When you are finished, click OK to save the changes and close the dialog box. The newly created group appears in the Constraint Manager.
5. To add constraints to the constraint group, Ctrl+drag a constraint from the Constraints list (a) to the newly created constraint group (b).



Specifying User-Defined Constraint Groups

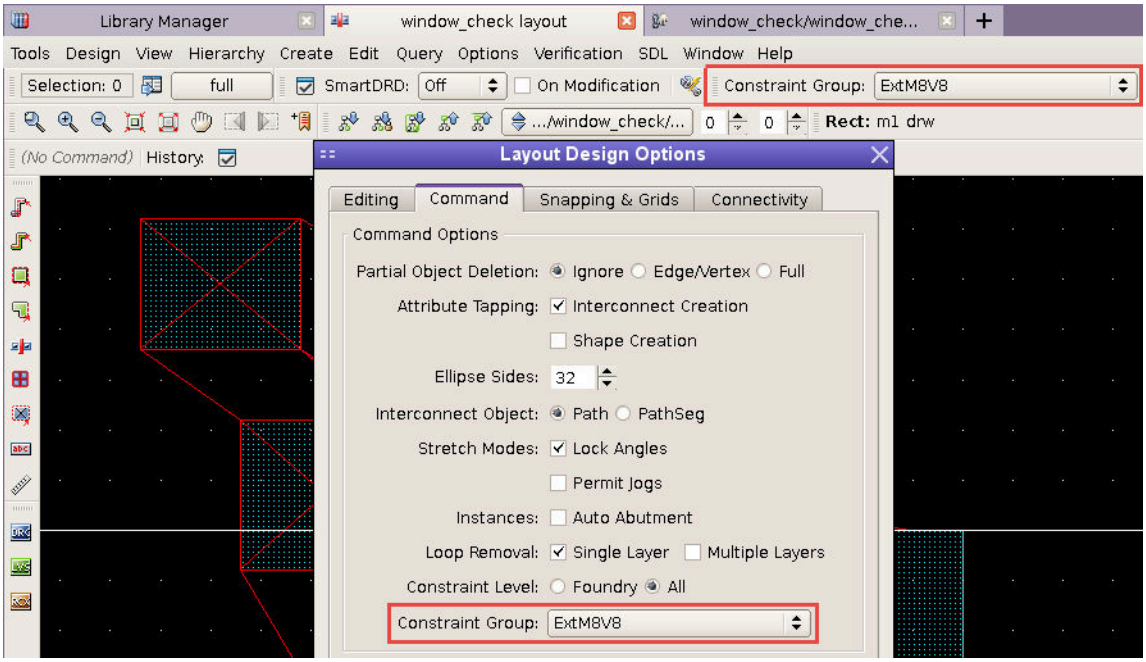
You can use Tcl preference `leUserDefinedConstraintGroups` to specify a list of user-defined constraint groups. When defined, the **Command** tab of **Layout Design Options** dialog box and the **Constraint Group** toolbar (Figure 54) shows candidate user-defined constraint groups in a **Constraint Group** list. When not defined or the list is empty, the

Feedback

Constraint Group list shows the constraint groups defined in the design and technology library.

Figure 54 shows the **Constraint Group** list on the **Constraint Group** toolbar and the **Layout Design Options** dialog box.

Figure 54 Constraint Group List



From the **Constraint Group** list, you can specify which user-defined constraint group to be searched first in the Constraint Manager. The selected constraint group is referenced by Tcl preference `leUserDefinedConstraintGroup`.

Note:

When you set the **Constraint Level** to **Foundry**, the tool ignores the **Constraint Group** value. As a result, the **Constraint Group** field in the **Layout Design Options** dialog box and in the toolbar is disabled.

Enabling Alternate Foundry Constraints

By default, foundry rules are applied when you work with designs in the Custom Compiler environment. However, you can specify alternate foundry constraints that replace existing foundry rules. For example, if the existing foundry constraint group consists of analog rules, you can enable a constraint group containing digital-specific rules.

Feedback

This section describes the following:

- Workflow for Enabling Alternate Foundry Constraints
- Before You Start
- Defining Alternate Foundry Constraint Groups
- Applying the Alternate Foundry Rules in the Design
- Associating Track Region Definitions With Alternate Foundry Constraint Groups

Workflow for Enabling Alternate Foundry Constraints

This topic presents the basic workflow for enabling alternate foundry constraints.

Step		See...
1	Review the requirements and considerations.	Before You Start
2	Define the alternate foundry constraint group in the technology file.	One of the following methods: • Choosing the Constraint Group Using the Alternate Foundry Toolbar • Setting the leTechFoundryConstraintGroups Tcl Preference
3	Choose and apply the alternate foundry rules in the design.	Applying the Alternate Foundry Rules in the Design

Before You Start

Before setting up and using alternate foundry constraints, review the requirements and considerations to optimize the implementation.

The alternate foundry constraint groups must meet the following requirements and considerations:

- The alternate foundry constraint groups must contain a complete set of design rules.
- User-defined constraint groups have priority over alternate foundry constraint groups.
- Cut class, color, and wide-alignment rules must still be defined in the original foundry technology.
- You can specify the original foundry technology with the alternate constraint groups.
- You can specify incremental technology databases. Each technology within the technology graph can have its own alternate foundry constraint group.

Feedback

See Also

- Constraint Groups Section
- Grouping a Set of Process Constraints

Defining Alternate Foundry Constraint Groups

You can define alternate rules for your designs instead of applying the default foundry constraints. Doing so lets you use the relevant rules for specific applications.

To designate a constraint group as an alternate foundry constraint group:

► In the Constraint Groups Section of the technology file, make sure the constraint group is defined with the `-alternate true` argument.

See Also

- Grouping a Set of Process Constraints
- Workflow for Enabling Alternate Foundry Constraints
- Before You Start
- Applying the Alternate Foundry Rules in the Design

Applying the Alternate Foundry Rules in the Design

While editing your design layout, you can choose the alternate foundry constraint group that is searched first in the Constraint Manager. This topic describes how to choose the alternate foundry constraint group.

This topic describes the two methods you can use to apply alternate foundry rules in the design:

- Choosing the Constraint Group Using the Alternate Foundry Toolbar
- Setting the leTechFoundryConstraintGroups Tcl Preference

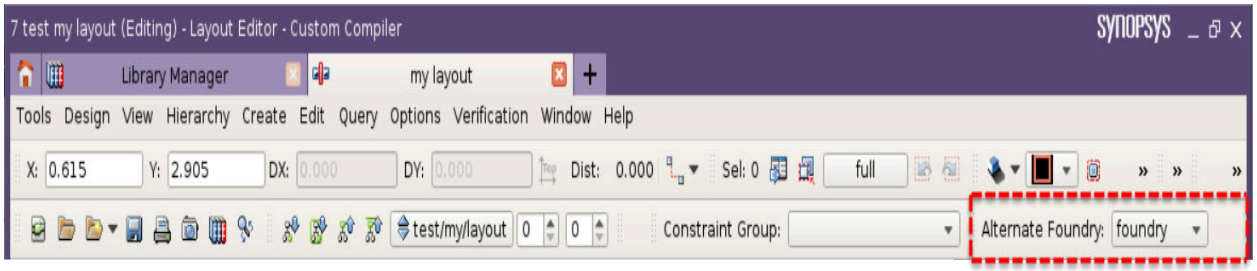
Choosing the Constraint Group Using the Alternate Foundry Toolbar

To choose the alternate foundry constraint group:

1. In the Layout Editor, choose **Window > Toolbars > Alternate Foundry**.

   The **Alternate Foundry** toolbar appears.

Feedback



2. Choose the constraint group from the **Alternate Foundry** list.

Setting the leTechFoundryConstraintGroups Tcl Preference

To set the `leTechFoundryConstraintGroups` Tcl preference:

► Type the following in the Custom Compiler Console:

```
db::setPrefValue leTechFoundryConstraintGroups -value "<groupName>"
```

See Also

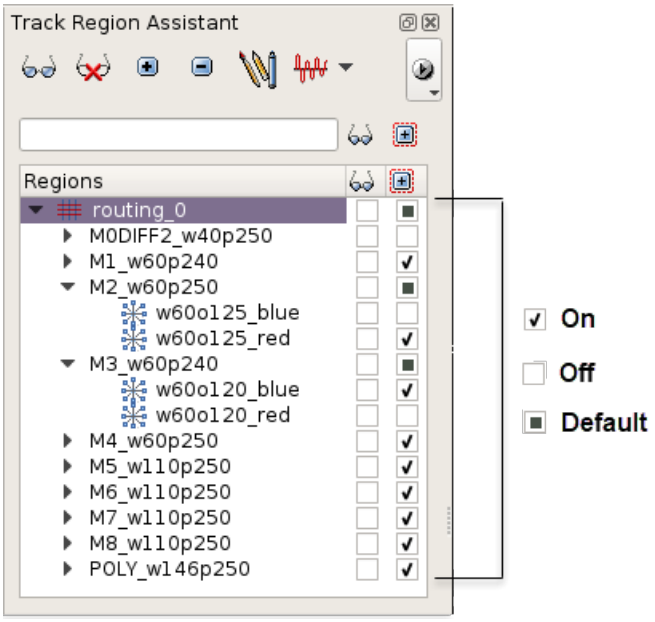
- Workflow for Enabling Alternate Foundry Constraints
- Before You Start
- Defining Alternate Foundry Constraint Groups

Associating Track Region Definitions With Alternate Foundry Constraint Groups

In the Track Region Assistant, in addition to turning on or off the active state of individual tracks, you can associate track region definitions with the foundry or alternate foundry constraint groups.

In this case, when the design is opened and when the `leTechFoundryConstraintGroups` Tcl preference value changes, the active and visible states of the tracks are automatically set based on the foundry or alternate foundry in use.

Feedback



To associate track region definitions with alternate foundry constraint groups:

► Define the `foundryConstraintGroups` attribute in the track region definitions that you want to associate with alternate foundry constraint groups. See trackDefs.

See Also

- Setting Track Region Assistant Options

Constraint Manager Reference

This topic provides a quick reference of the Constraint Manager. For instructions on how to add and delete constraint parameters, group constraints, and specify user-defined constraint groups, see the relevant topics in Constraint Manager Tool.

This topic describes the following:

- Constraint Manager Menus
- Constraint Manager Toolbar

Feedback

Constraint Manager Menus

The menu bar contains the following:

Menu	Commands
File	• **New Group**: Opens the **Create Constraint Group** dialog box. See Grouping a Set of Process Constraints. • **Commit**: Saves the modifications you made to the active constraint group. • **Discard**: Reverts to the previously saved settings for the active constraint group. • **Close**: Closes the Constraint Manager.
Edit	• **Delete**: Deletes selected objects.
View	*Mode*: Controls the organization of parameters in the constraint view tree. The options are: • **Standard** (Default): Displays the list of constraints by constraint name. • **Hierarchical**: Displays the list of constraints by hierarchy. • **Group By Rule**: Displays the list of constraints by rules.
Window	• **Toolbars > Standard**: Displays or hides the Constraint Manager toolbar. • **Workspaces**: Lets you create new workspaces. For information about workspaces, see Starting Sessions. • **Customize**: Lets you display and hide menus and toolbars. You can select and display the customized interface next time you return to the Constraint Manager.
Help	Opens the Help.

Constraint Manager Toolbar

The toolbar contains the following:

Button	Description
 New Group	Opens the **Create Constraint Group** dialog box. See Grouping a Set of Process Constraints.
 Delete	Deletes selected objects.
 Commit	Saves modifications made to the active constraint group.

Feedback

Button	Description
Discard	Reverts to the previously saved settings for the active constraint group.

Display Resource Editor Tool

This topic describes the following:

- About the Display Resource Data
- Displaying the Display Resource Editor
- Managing Display Definitions
- Creating and Editing Display Packets and Packet Aliases
- Creating Colors, Stipples, and Line Styles
- Managing Display Resources
- Commonly Used Display Packets

About the Display Resource Data

Display resources control how everything appears on the canvas in the design editors, such as the color of the canvas background, the colors of objects, the colors and styles of lines, and how colored objects are filled. Display resource data is stored in the `display.tcl` file. See Display Resource File.

Note:

The Custom Compiler environment also supports third-party display resource data files in the DRF format (`display.drf`).

The Custom Compiler default `display.tcl` file is in the installation directory. If your company, project, or individual designers have different display preferences, you can optionally use overriding `display.tcl` files in the directories listed in Table 93. You can point to the directories using the corresponding environment variables. The startup process loads all of these files in the order shown. The last-loaded file overrides the previous file. For example, the specifications in the user-specific `display.tcl` file always overrides the settings in the project-specific `display.tcl` file.

Feedback

Table 93 Locations of display.tcl Files

Locations of display.tcl	Environment Variable
Installation directory (auxx directory)	$SYNOPSYS_CUSTOM_INSTALL
Directory for site-specific data (optional)	$SYNOPSYS_CUSTOM_SITE
Directory for project-specific data (optional)	$SYNOPSYS_CUSTOM_PROJECT
Directories for custom setup	$SYNOPSYS_CUSTOM_INITPATH
Directory for user-specific data	$SYNOPSYS_CUSTOM_USER
User's home directory, where compatible third-party product files are stored	$HOME
Current working directory	$PWD
Directory for data that are saved when you exit the Custom Compiler tool and reloads them when you restart	$SYNOPSYS_CUSTOM_LOCAL

If the $SYNOPSYS_CUSTOM_USER environment variable is not defined, the startup process will search the $HOME/synopsys_custom directory. If the $SYNOPSYS_CUSTOM_LOCAL environment variable is not defined, the startup process will search the $PWD/synopsys_custom directory. For details about setting up the environment variables, defining the startup sessions, and the different directories, see Defining Startup Settings.

If a directory contains a `display.drf` file and a `display.tcl` file, both will be loaded in the respective order. Because the `.tcl` file is loaded later, its content overrides the content in the `.drf` file.

You can modify the display resource data using one of the following methods:

- Using the Displaying the Display Resource Editor.
- Editing the `display.tcl` files. See the Display Resource Tcl chapter in the *Custom Compiler Tcl Reference Manual*.

Displaying the Display Resource Editor

The Display Resource Editor is the interface you use to view and modify the data in the `display.tcl` files.

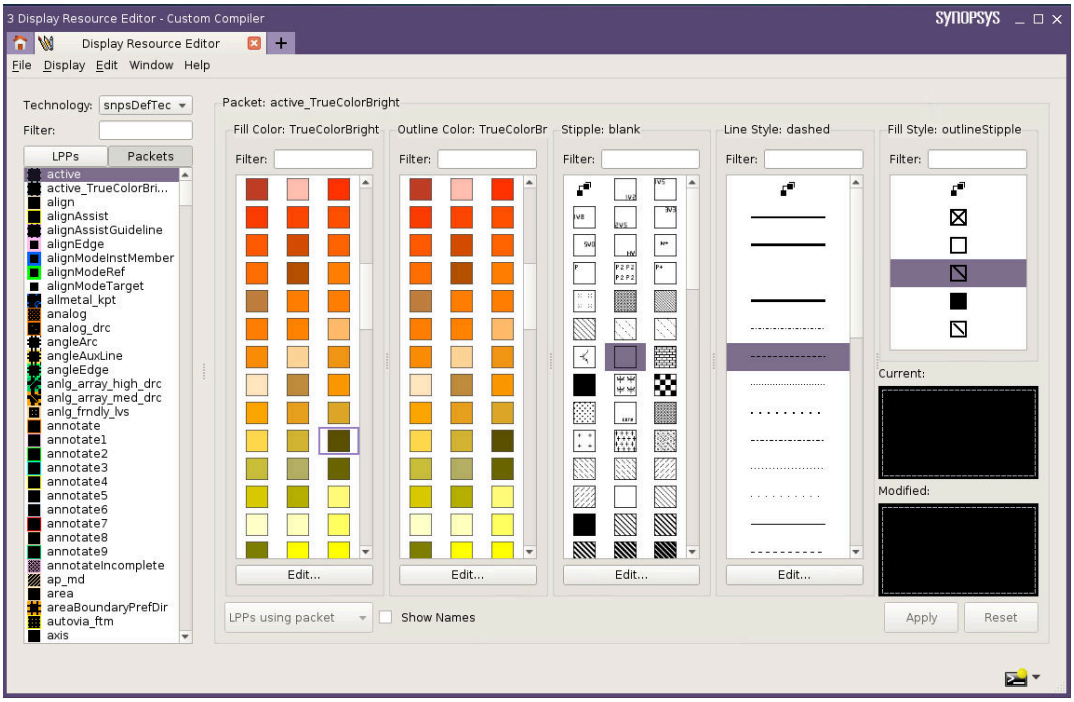
Feedback

To display the Display Resource Editor:

► From the **Home** tab or a design layout view, choose **Tools > Display Resource Editor**.

To display the Display Resource Editor using the `dr::showDisplayResourceEditor` Tcl command, type the command in the Console tab or Console pane.

In the Display Resource Editor, you can select the layer-purpose pair (LPP) or the packet on the left. The associated display attributes appear on the right.



Area Name	Description
Technology	Includes all the libraries that have local technology databases. When selecting a library in the list, then the technology file is opened. Only available when enabling the Display Resource Editor from the console.
Filter	Filtering to reduce the number of items to visually search in the browser, which supports multiple selection.
LPPs/Packets tabs	Lists the defined LPPs or display packets. See Commonly Used Display Packets.

Feedback

Area Name	Description
Packet	Contains the attributes of the selected LPP or display packet: • **Fill Color**: Color that is used to fill an object or area. • **Outline Color**: Color of the line that encloses an object or area. • **Stipple**: Pattern that is used to fill an object or area. • **Line Style**: Style of line. Examples of line styles are solid lines, dotted lines, and dashed lines. • **Fill Style**: Style used to fill an object or area. Examples of fill styles are solid and outline (an outline encloses the object or area, but there is no fill).
LPPs using Packet	Lists the LPPs that are using the selected packet.
Show Names	Displays the names of attributes.

Managing Display Definitions

Because a single display definition might not be optimal for all designs, you can create different display definitions to accommodate the different designs. You can also create different display definitions to showcase different aspects of the same design.

This section describes the following:

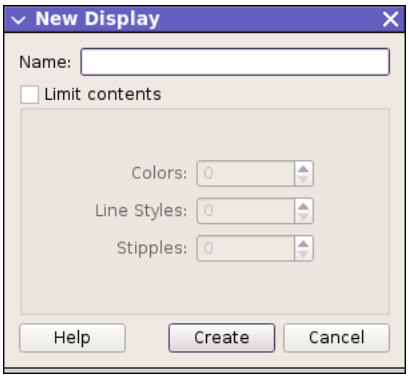
- Creating Display Definitions
- Editing Display Attributes
- Renaming Display Definitions
- Deleting Display Definitions

Feedback

Creating Display Definitions

To create a display definition:

1. In the Display Resource Editor, choose **Display > New**.
2. In the **New Display** dialog box:
   a. Type a unique **Name** for the new display definition.
   b. (Optional) Select **Limit contents** if you want to restrict the maximum number of **Colors**, **Line Styles**, and **Stipples** in the definition.



3. When you are finished, click **Create**. The name of the newly created display definition appears in the **Display** menu.
4. To display the newly created display definition, choose **Display** > <displayDefinitionName>.
5. To edit the display attributes, see Editing Display Attributes.

Editing Display Attributes

Each display definition consists of a list of LPPs, display Packets, and corresponding display attributes (line color, line style, fill style, and so on).

To edit a display attribute:

1. In the Display Resource Editor, select the **LPP** or **Packet**.

   When a single packet or LPP is selected, the **Packet: <name>** is displayed. When nothing is selected, **Packet Attributes** is displayed. When multiple items are selected, **Multiple Packets** is displayed.

2. In the **Packets** area on the right side of the dialog box, choose different attribute values from the **Fill Color**, **Outline Color**, **Stipple**, **Line Style**, and **Fill Style** lists.

Feedback

If you want to add new or modify existing attribute values, click **Edit**, and then create or modify an attribute value. When you are finished, you should see the new or modified attribute value in the dialog box.

3. To reset any value to its previous state, click **Reset**.
4. To change the attributes of LPP/Packet, click **Apply**.

Renaming Display Definitions

Note:

All display definitions, except the default definition, are available for renaming.

To rename a display definition:

1. In the Display Resource Editor, choose the display definition you want to rename in the **Display** menu.
2. Choose **Display > Rename**.
3. In the **Rename Display** dialog box, type a new name.
4. Click **Rename**. You should see the new name in the **Display** menu.

Deleting Display Definitions

To delete a display definition:

1. In the Display Resource Editor, choose the display definition you want to delete in the **Display** menu.
2. Choose **Display > Delete.**
3. In the **Confirm Delete Display** dialog box, click **OK** to proceed with the delete operation.

Creating and Editing Display Packets and Packet Aliases

You can create display packets to define a set of attributes for association with a display definition. You can also create packet aliases to enable adjustment of display attributes without affecting their definitions in the technology file.

- Creating Packets
- Creating Packet Aliases
- Using Active Packet Aliases

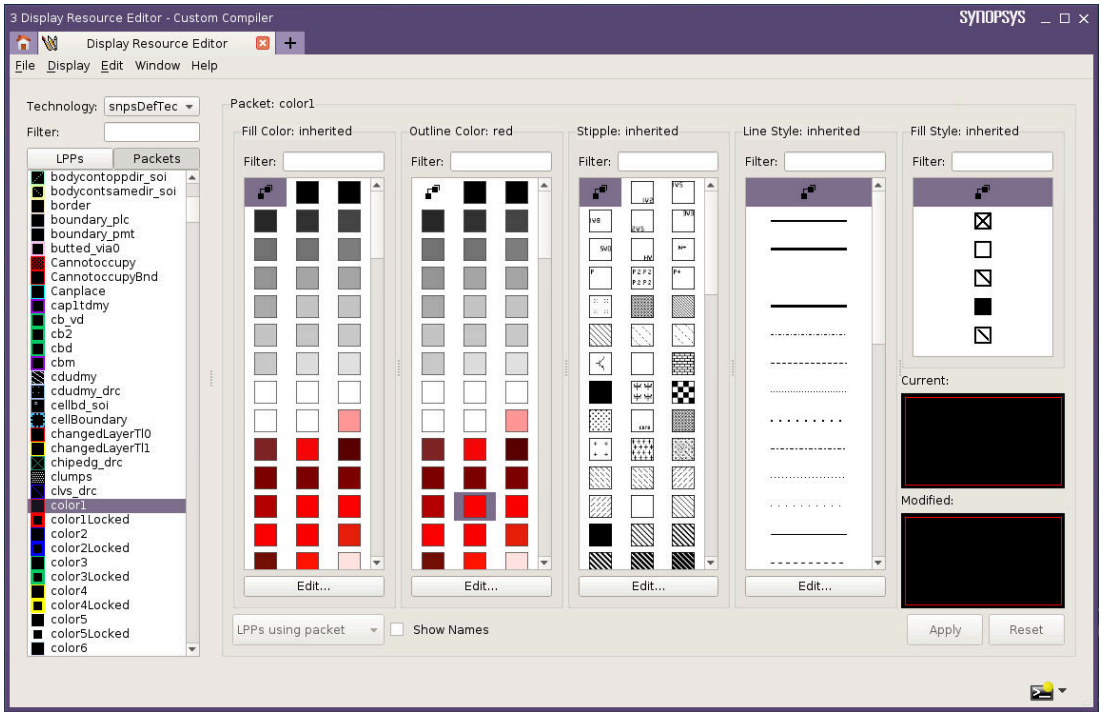
Feedback

Creating Packets

You can create packets to define sets of visual attributes for design objects. Use the Display Resource Editor to create a packet.

To create a packet:

1. In the Display Resource Editor, click **Packets**.



2. (Optional) In the **Packets** list, choose a packet whose attributes are similar to the one you want to create.
3. Choose **Edit > New Packet**.
4. Enter a packet name.
5. Edit attributes for this packet.

   For any attribute you want to change, choose a predefined value from the menu.

   To additionally alter the visual definition for this value, click **Edit**, make changes, and click **OK**.
6. When you are done, choose **File > Close**.

Feedback

Creating Packet Aliases

You can create a packet alias to modify attributes for an existing packet without changing its packet definition in the technology file.

Use the Display Resource Editor to create a packet alias.

To create a packet alias:

1. In the Display Resource Editor, choose a packet.
2. Choose **Edit > New PacketAlias**.
3. Enter a **PacketAlias** name.
4. When you are done, choose **File > Close**.

Using Active Packet Aliases

The active object is displayed using the active display packet alias. By default, this alias points to an `active_TrueColorBright` display packet. This packet has no stipple, dashed line style, and color (RGB) 166 176 186. For a path, it appears as follows:

Normal:

Active:

The following table denotes a mapping of object filter types to packet alias for the layout editor only. If the `drDisplay` display object contains a packet alias matching an entry in the table then the active figure highlight will use the specified packet alias in lieu of the active packet alias.

For example:

If the packet alias `leActiveObjectShape` exists then the shape activation will use this packet alias to highlight shapes. If the packet alias does not exist then active shape will use packet alias active.

leObjectFilter	Packet Alias
leBlockage	leActiveObjectBlockage
leBoundary	leActiveObjectBoundary
leCoordinateMark	leActiveObjectCoordinateMark

Feedback

leObjectFilter	Packet Alias
leCorridorArea	leActiveObjectCorridorArea
leFigGroup	leActiveObjectFigGroup
leFigGroupMember	leActiveObjectFigGroupMember
leGuardRing	leActiveObjectGuardRing
leGuardRingShape	leActiveObjectGuardRingShape
leInstance	leActiveObjectInstance
leInstanceHierarchy	leActiveObjectInstanceHierarchy
leInstancePin	leActiveObjectInstancePin
leMarker	leActiveObjectMarker
leMpp	leActiveObjectMpp
lePartialObject	leActiveObjectPartialObject
lePcellShape	leActiveObjectPcellShape
lePin	leActiveObjectPin
leRouteCorridor	leActiveObjectRouteCorridor
leRouteGuide	leActiveObjectRouteGuide
leRow	leActiveObjectRow
leRuler	leActiveObjectRuler
leShape	leActiveObjectShape
leSteiner	leActiveObjectSteiner
leStripedWire	leActiveObjectStripedWire
leText	leActiveObjectText
leTrackRegion	leActiveObjectTrackRegion
leTransmissionLine	leActiveObjectTransmissionLine
leTransmissionLineShape	leActiveObjectTransmissionLineShape
leVia	leActiveObjectVia

Feedback

leObjectFilter	Packet Alias
leViaShape	leActiveObjectViaShape
leVoltageArea	leActiveObjectVoltageArea

Creating Colors, Stipples, and Line Styles

Create colors, stipples, and line styles to provide additional attributes for display packets to use.

These sections cover:

- Creating Fill and Outline Colors
- Creating Stipple Patterns
- Creating Line Styles

Creating Fill and Outline Colors

You can create a fill and outline colors by defining their display names, RGB (red, green, blue) luminosity values, and optionally, a blink setting.

Use the Display Resource Editor to create fill and outline colors.

To create fill or outline color:

1. In the Display Resource Editor, in the section for the **Fill Color** or **Outline Color** attribute, click **Edit.**
2. Click **Add**.
3. Enter a color name.
4. Adjust the RGB values by moving the cursor over the continuous tone bitmap, or choose by number.
5. To cause layers with this color to blink, click **Blink**.
6. Click **OK**.
7. When you are done, choose **File > Close**.

Creating Stipple Patterns

You can create a stipple pattern for fills by defining its display name, and a bitmap pattern, in pixels.

Use the Display Resource Editor to create stipple patterns.

Feedback

To create a stipple pattern:

1. In the Display Resource Editor, in the section for the **Stipple** attribute, click **Edit**.
2. Click **Add**.

   All displays support the built-in X stipple. This stipple is not defined as a bitmap, and it can be referenced in any packet definition. The X stipple causes an 'X' of fixed line width to be drawn from corner to corner in all rectangles, and rectangular polygons, leaving other shapes unfilled.

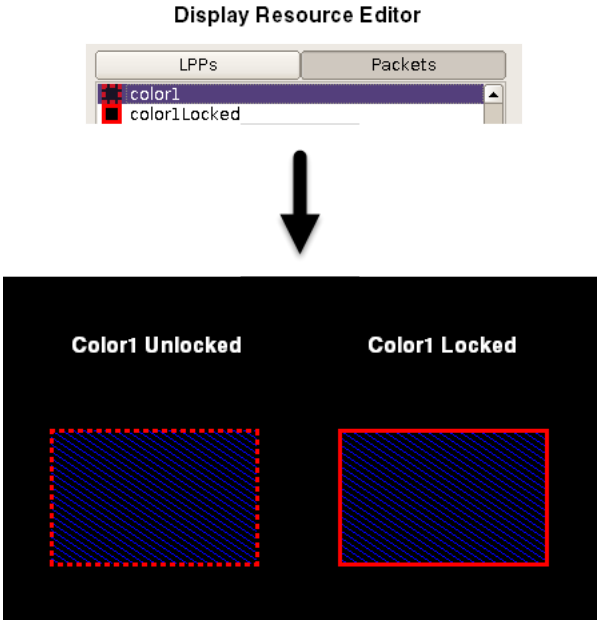
   You can override the built-in X stipple in any display by creating a bitmap-based stipple named 'X' in that display.
3. Enter a stipple name.
4. Choose a base pattern size in number of pixels.
5. Use the cursor to select pixels for the base pattern.

   To create a pattern with a dark background, define the fewer-pixel portion, and click **Invert**.
6. When you are done, choose **File > Close**.

Creating Line Styles

You can use the Display Resource Editor to create line styles. You might want different line styles to create visual distinctions between similar objects. For example, by default, the line styles for color*n* and color*n*Locked are different so that you can visually distinguish locked colors from unlocked colors on the canvas. With this visual aid, you do not need to use the Property Editor to check the color lock status.

Feedback



You can create a line style by defining its display name, a line width in pixels, and a one-dimensional bit pattern (1s and 0s) of any length.

To create a line style:

1. In the Display Resource Editor, in the section for the **Line Style** attribute, click **Edit**.
2. Click **Add**.
3. Enter a line style name.
4. Choose a pattern length, and a line width.
5. Use the cursor to select pixels for the pattern.

   (Optional) To create a pattern with a dark background, define the fewer-pixel portion, and click **Invert**.
6. When you are done, choose **File > Close**.

Managing Display Resources

The changes you make to the display definitions or display data is temporarily stored in memory. If you want to keep the changes for future use, you must save the changes.

This section describes the following:

- Saving Display Resource Data
- Loading Display Resource Files

Feedback

- Reloading Last-Saved Display Resource Files
- Reloading Default Display Resource Files
- Merging Display Resource Files

Saving Display Resource Data

You can save modified display resource data from the current session to a `display.tcl` or `display.drf` file.

Caution:

If you want Custom Compiler to automatically load the display resource data during startup, name the file `display.tcl` or `display.drf` and save it in one of the directories listed in About the Display Resource Data.

To save new or modified display resource data to a display file:

1. In the Display Resource Editor, choose **File > Save As**. The **Save Display Resource** dialog box opens.
2. Select the location where you want to save the file.
3. Type the **File name**.
4. Choose a file type: Tcl or DRF.
5. (Optional) To save only the data that were modified in the current design session or since the last Reload operation, select **Modifications Only**.
6. Click **Save**.

Loading Display Resource Files

During startup, Custom Compiler automatically loads the `display.tcl` or `display.drf` files as described in About the Display Resource Data. However, if you have a different display data file you want to use, you can load it using the Display Resource Editor.

To load a display resource file:

1. In the Display Resource Editor, choose **File > Load**.
2. In the Load Display Resource File dialog box, locate and select the `display.tcl` or `display.drf` file.
3. Click **Load**.

Reloading Last-Saved Display Resource Files

In case you want to revert to the last-saved or last-loaded display data, you can use the **File > Reload Display** command. Doing so overwrites any modifications you made

Feedback

that you have not yet saved. If you want to keep the modifications, see Saving Display Resource Data.

When you reload the last-saved display resource file, Custom Compiler loads the `display.tcl` files in the following directories, in the order shown:

Locations of display.tcl	Environment Variable
Installation directory (auxx)	$SYNOPSYS_CUSTOM_INSTALL
Directory for site-specific data	$SYNOPSYS_CUSTOM_SITE
Directory for project-specific data	$SYNOPSYS_CUSTOM_PROJECT
Directories for custom setup	$SYNOPSYS_CUSTOM_INITPATH
Directory for user-specific data	$SYNOPSYS_CUSTOM_USER
User's home directory, where compatible third-party product files are stored	$HOME
Current working directory	$PWD
Directory for data that are saved when you exit Custom Compiler and reloads them when you restart	$SYNOPSYS_CUSTOM_LOCAL

To reload the last-saved display resource data:

1. In the Display Resource Editor, choose **File > Reload Display**. The **Discard Display Resource Edits** dialog box opens.
2. Click **Overwrite** to continue. The Resource Editor loads all the display.tcl or display.drf files in the order described in About the Display Resource Data.

Reloading Default Display Resource Files

In case you want to revert to the original display data that were provided with the product, you can use the **File > Reload Default** command. Doing so allows you to start from the original display setup.

When you reload the last-saved display resource file, Custom Compiler loads the `display.tcl` files in the following directories, in the order shown:

Locations of display.tcl	Environment Variable
Installation directory (auxx)	$SYNOPSYS_CUSTOM_INSTALL
Directory for site-specific data	$SYNOPSYS_CUSTOM_SITE

Feedback

Locations of display.tcl	Environment Variable
Directory for project-specific data	$SYNOPSYS_CUSTOM_PROJECT
Directories for custom setup	$SYNOPSYS_CUSTOM_INITPATH
Directory for user-specific data	$SYNOPSYS_CUSTOM_USER
User's home directory, where compatible third-party product files are stored	$HOME

To reload the original display resource files that were provided with the product:

1. In the Display Resource Editor, choose **File > Reload Default**. The **Discard Display Resource Edits** dialog box opens.
2. Click **Overwrite** to continue. The Resource Editor loads all the original display.tcl or display.drf files in the order described in About the Display Resource Data.

Merging Display Resource Files

You can combine the data from multiple display resource files into one display resource file. Note that when you list the files to merge, the file order determines the overriding sequence (the last file overrides all preceding files).

To merge display resource from multiple files:

1. In the Display Resource Editor, choose **File > Merge**.
2. In the **Merge Display Resource Files** dialog box, click **Add**.

3. In the **Select Source Files** dialog box, locate and select the .tcl or .drf file, and then click **Open**.
4. Repeat step 3 to add additional files.
5. To change the order of the files, in the **Merge Display Resource Files** dialog box, select a file in the **Source** list and click **Move Up** or **Move Down**. Be aware that each file overrides the file that precedes it.
6. To remove a file from the list, select the file in the **Source** list and click Remove.
7. In the **Destination** area, click to select the location for the merged data file, type a **File name**, and click **Save**.
8. Choose the **Format** for the destination file (**TCL** or **DRF**). If you choose **Auto**, the format will be determined automatically, based on the filename extension you specified in step 7. If you did not specify a filename extension in step 7, the newly merged file will be in the TCL format by default.
9. (Optional) To prevent the data from being loaded into memory now, clear the **Load Merged Data** check box.
10. Click **Merge**.

Feedback

Commonly Used Display Packets

The following table lists the commonly used display packets. If you want to add new, repurpose existing, or change the names of the display packets, see Creating and Editing Display Packets and Packet Aliases.

Note:

Make sure the object's or feature's visibility is enabled, or the set of conditions that permit them to be rendered are enabled.

Display Packet Name	Description
active	Active object
alignAssist	Using the Align Assist indicator
alignModeRef	Alignment reference indicator
alignModeTarget	Alignment target indicator
annotate	cdsParam labels
annotate7	Instance label (cdsname())
annotate8	cdsTerm labels
axis	Design axis
background	Canvas background
blockage	Blockages The display renderer will use the packet assigned to the LPP blockage. If that packet does not exist, the renderer will default to this generic blockage packet.
color1–6	Color annotations for unlocked multipattern color assignments
color*n*Locked	Color annotations for locked multipattern color assignments
colorSame	Color annotation for unlocked same color assignment
colorSameLocked	Color annotations for locked same color assignments
coordinate_mark	Coordinate mark

Feedback

Display Packet Name	Description
defaultPacket	Default display settings for shapes if the associated LPP is assigned to a packet that does not exist
device	Symbol shapes
deviceLbl	Device label ([@instanceName])
FinGrid	Wide alignment grids (fins in advanced node designs) The display renderer uses this packet to display all global wide alignment grids. Local wide alignment grids use LPP-specific packets. If that packet does not exist, the renderer will default to this FinGrid packet.
fixingGuide fixingGuide1–9	Fixing guides, the markings on the canvas that indicate the violations and provide suggested fixes during in-design rule checking. The commonly used fixingGuide packets are: • fixingGuide–Spacing • fixingGuide2–Enclosure • fixingGuide5–Width For details about fixing guides, see Fixing Violations with Guidance.
highlight1–30	Highlighted objects
hilite1	Current object in the Property Editor (Single Item View)
instance	Instance objects
instanceLbl	Library, cell, view labels ([@libName], [@cellName], [@viewName])
major_grid	Major grid
marker<severity>	Markers that indicate the severity of the error. See Error Viewer Assistant.
minor_grid	Minor grid
net_flightline	Flight lines between net's physical components
net_highlight1-2	Highlighted nets
pin	Schematic and symbol pins
pinLbl	Schematic and symbol pin label

Feedback

Display Packet Name	Description
prBoundary	PR boundary
probe1-10	Probes
row	Row objects
rowSite	Sites in rows
rubberBand rubberBand2	Rubber band
ruler	Linear ruler
ruler0–9	Circular ruler and dimension mark
select1	Selected objects
snap	Snap boundary
SR_HighlightCurrent	Selected objects in the Results pane in the Search and Replace tool
SR_HighlightFound	Objects found by the Search and Replace tool
text	Schematic shapes and labels
trackPatternDefaultNonPrefDir	(Track pattern) Default track type in non-preferred direction if `leTrackPatternColor` is `false`. If the preference is `true`, the display renderer will use the color1 packet.
trackPatternDefaultPrefDir	(Track pattern) Default track type in preferred direction if `leTrackPatternColor` is `false`. If the preference is `true`, the display renderer will use the color2 packet.
trackPatternNonReservedNonPrefDir	(Track pattern) Non-reserved track type in non-preferred direction if `leTrackPatternColor` is `false`. If the preference is `true`, the display renderer will use the color3 packet.
trackPatternNonReservedPrefDir	(Track pattern) Non-reserved track type in preferred direction if `leTrackPatternColor` is `false`. If the preference is `true`, the display renderer will use the color4 packet.
trackPatternReservedNonPrefDir	(Track pattern) Reserved track type in non-preferred direction if `leTrackPatternColor` is `false`. If the preference is `true`, the display renderer will use the color5 packet.

Feedback

Display Packet Name	Description
trackPatternReservedPrefDir	(Track pattern) Reserved track type in preferred direction if `leTrackPatternColor` is `false`. If the preference is `true`, the display renderer will use the color6 packet.
wire	Schematic wires
wireLbl	Schematic wire label
y0–9Flt	Static flight lines
y0–9Flt_dyn	Dynamic flight lines

Hierarchy Editor Tool

The Hierarchy Editor tool controls the number of instances edited at one time. Use this tool to design with hierarchy configurations.

This topic describes the following:

- Displaying the Hierarchy Editor
- Creating Hierarchy Configurations
- Editing Hierarchy Configurations
- Controlling Viewed Information
- Hierarchy Editor Menus
- Hierarchy Editor Toolbar

	The Hierarchy Editor tool controls the number of instances edited at one time. Use this tool to design with hierarchy configurations. This video tutorial shows how to use the Hierarchy Editor..

Displaying the Hierarchy Editor

You can create hierarchy configurations (config views) to work with a subset of selected instances in a design schematic. Hierarchy configurations limit the view depth. This controls the extent of the design the system evaluates in analysis operations. The Hierarchy Editor defines, manages, and maintains configuration views.

You use the Hierarchy Editor to work with hierarchy configuration (config) views.

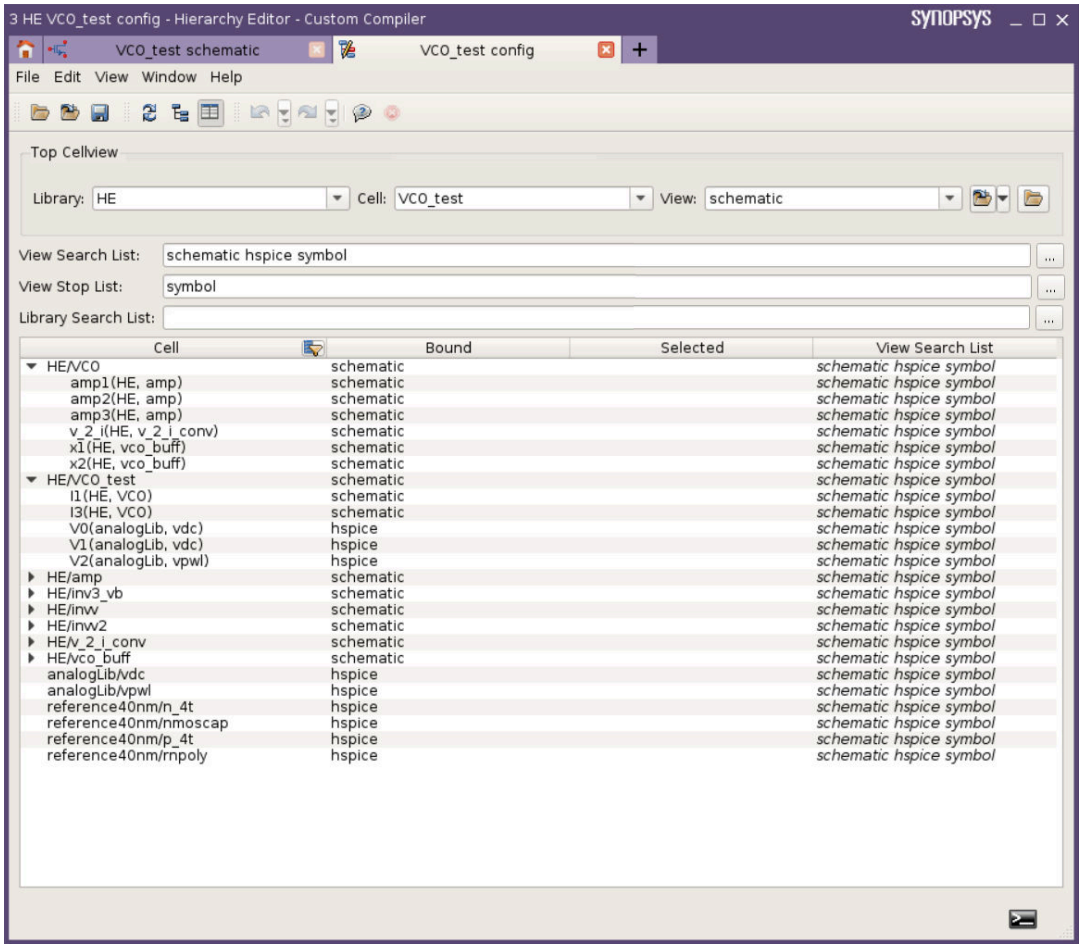
Feedback

The config cellview determines the switched master for each placed instance in a design. A switched master uses rules to control which instance view netlisters and other tools use in the design they process.

The config cellview also stores a reference to the root cellview of the hierarchy and all additional rules necessary to traverse (or bind) the design.

To open the Hierarchy Editor:

► Open the config view of a schematic and choose **Tools > Hierarchy Editor**. The Hierarchy Editor opens in a new tab.



Creating Hierarchy Configurations

Create hierarchy configurations by creating config cellviews. Through a config view, you can explore the design, set proper views for netlisters, and edit the design with concurrent visibility into the associated hierarchy.

Feedback

Create config views from the source of your choice:

- Creating Configuration Views From an Existing Design
- Creating Configuration Views as New CellViews

Creating Configuration Views From an Existing Design

Create a config view from an existing design to use this library, cell, and view combination as the switched master.

1. Begin by opening the Hierarchy Editor. Open the config view of a schematic, and choose **Tools > Hierarchy Editor**.
2. Open the Hierarchy Editor: Choose **Tools > Hierarchy Editor**.
3. Confirm the **Top Cellview** or choose a new design.
4. (Optional) If you did not supply a template, define the **View Search List** by entering view names.

   A view search list is an ordered list of view names that determines the switched master. You can set the default using the `heEditorDefaultViewSearchList` preference.

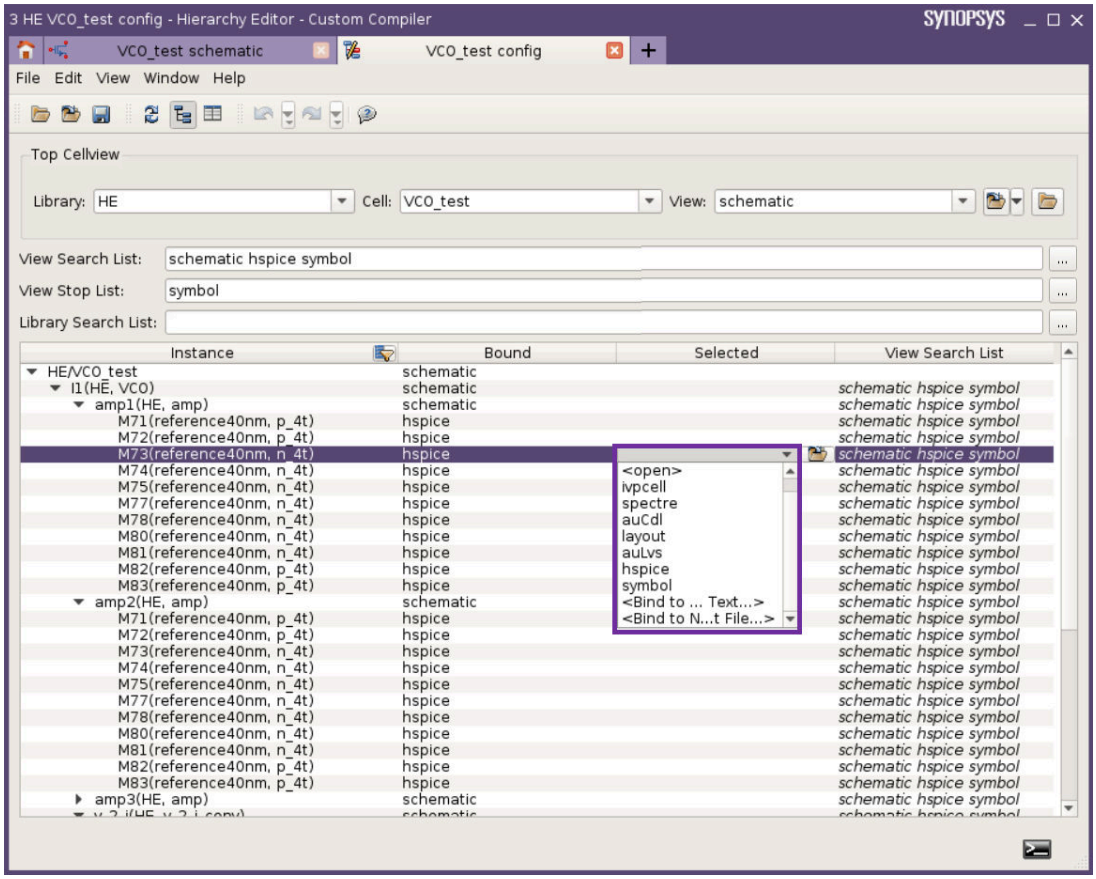
   The views found are governed by the search list rules you are defining. If a cell does not contain any of the views listed in the view search list and view selection is not set, the view found is listed as *unbound* in red text.

   After the **View Search List** is added, the configuration for the top cellview is created.

   To view the hierarchy, and see the view found for each instance, expand the root of the tree.
5. (Optional) For an explanation of the rule governing a view found, select an instance in the tree and choose **Edit > Explain** (or right-click the instance and choose **Explain**). (See Explaining Found Hierarchy Views).
6. (Optional) If you did not supply a template, define a **View Stop List** by entering names of views to define the lowest descent level for that leaf. You can set the default using the `heEditorDefaultViewStopList` preference.

   Alternatively, allow the tool to determine stop levels where it encounters views holding no instances.
7. (Optional) You can change the view to which an instance is bound by clicking in the **Selected** field and selecting from the drop-down menu. You can select multiple instances to change at one time. (See Selecting and Clearing Scoped Views).

Feedback

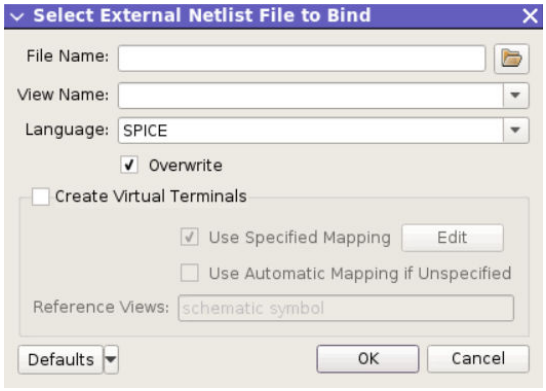


When you modify the view rule in the drop-down menu,

- The occurrence of the scoped view is cleared, and the instance scoped view is set.
- A reminder message appears:
  - Instance scoped view changes to “ABC” and the occurrence of the scoped view is cleared
  - The occurrence of the instance scoped view is cleared

8. (Optional) You can change the view to which an occurrence is bound by clicking in the **Selected** field and selecting from the drop-down menu. Selecting a view adds an occurrence rule to the configuration associated with the design editor.
9. (Optional) To bind the instance (or multiple instances, if selected) or occurrence to an external netlist, choose **<Bind to Netlist Text>** to open the **Select External Netlist File to Bind** dialog box. Select a **File Name**, **View Name**, and **Language**.

Feedback



Check **Overwrite** to overwrite the cellView, otherwise a warning is issued and the Hierarchy Editor does not overwrite the view or set the binding.

(SPICE/CDL only) Check **Create Virtual Terminals** to enable the options for creating certain terminals in the subcircuit declaration as virtual. See Table 30 in the topic Importing Design Netlist Data as Text Views for details on these options.

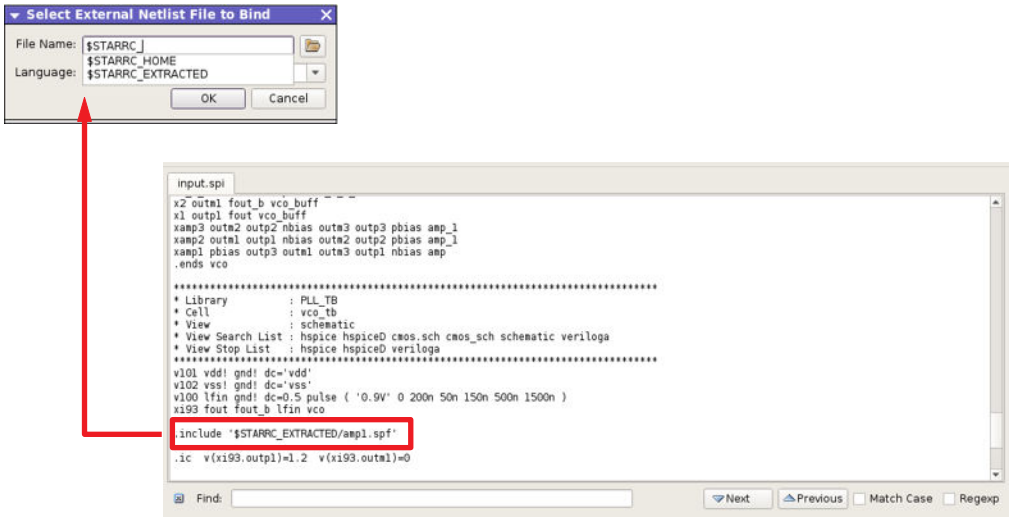
Click **OK** to close the dialog box.

10. (Optional) To bind the instance (or multiple instances, if selected) or occurrence to an external netlist file without importing to the OA database, choose **<Bind to Netlist File>** to open the **Select External Netlist File to Bind** dialog box. Select a **File Name** and **Language**.

    Note:

    When you select **<Bind to Netlist File>**, the path last browsed becomes the default path. Set the `heExternalNetlistFileBrowsePath` preference to use this default path.

Feedback



As seen in the preceding example, you can use environment variables in the netlist file path; the tool automatically lists available environment variables. When selecting netlist files, the tool automatically detects the netlist format from the file name extension.

Click **OK** to close the dialog box.

11. Choose **File > Save**.

Creating Configuration Views as New CellViews

Create a config view from a new cellView to define the switched master as you create the cellView.

1. Open the Library Manager: **Tools > Library Manager**.
2. Create the cellView: **File > New > CellView**.
3. Select the library, cell, and the Hierarchy Editor.
4. Choose the view name config or enter a different name.
5. Click **OK**.

   The hierarchy is populated from the root cellview. (After you open the Hierarchy Editor, and define the view search and view stop lists, the hierarchy expands to display the tree corresponding to the top level of the design).

6. Save the current Configuration cellView: **File > Save (Needed)**.

Feedback

7. Save the unchanged Configuration cellView: **File > Save**.

   Note:

   The **Save** icon displays an asterisk on the upper-right corner when there are unsaved changes.

Editing Hierarchy Configurations

Edit hierarchy configuration by working with config views.

- Undoing and Redoing Configuration View Modifications
- Explaining Found Hierarchy Views
- Importing Attributes From Existing Configuration Views
- Specifying a Template for a Hierarchy Configuration View
- Using Hierarchy View Name Groups
- Setting Hierarchy Editor Options
- Searching Configuration Views
- Filtering Configuration Views on Libraries or Cells

Undoing and Redoing Configuration View Modifications

Undo or redo config modifications to reverse or reactivate changes. (To reverse or reactivate general session operations, use the Transaction History Assistant).

Begin by opening the Hierarchy Editor. Open the config view of a schematic, and choose **Tools > Hierarchy Editor**.

- To undo the last config modification, in the Hierarchy Editor choose **Edit > Undo** or click the button in the toolbar. In the toolbar, you can select from a list of recent modifications by clicking the Down Arrow.
- To redo the last config modification, in the Hierarchy Editor choose **Edit > Redo** or click the button in the toolbar. In the toolbar, you can select from a list of recent modifications by clicking the Down Arrow.

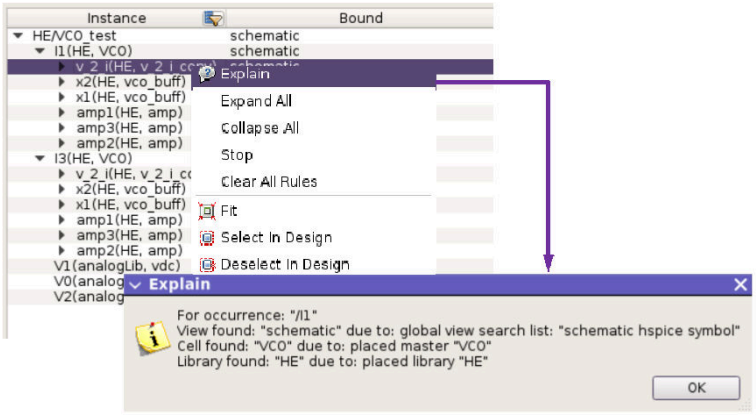
Explaining Found Hierarchy Views

Explain found views to learn the rule that determines how a view is found for this configuration.

Feedback

1. Begin by opening the Hierarchy Editor. Open the config view of a schematic, and choose **Tools > Hierarchy Editor**.
2. To explain the view found for a cellview or instance, position the cursor in the Bound column, right-click the mouse, and choose **Explain** from the menu.

   Alternatively, you can select the cell or instance first, and then choose **Edit > Explain**.



Importing Attributes From Existing Configuration Views

Import attributes from existing config views to easily supply them to a current config view. You can import the top library/cell/view combination, stop lists, global view search lists, open view lists, view name groups, and selection rules.

1. Begin by opening the Hierarchy Editor. Open the config view of a schematic, and choose **Tools > Hierarchy Editor**.
2. In the Hierarchy Editor, choose **Edit > Import**.
3. Specify the source config cellview.
4. Check mark the attributes to import.
5. Click **Import**.

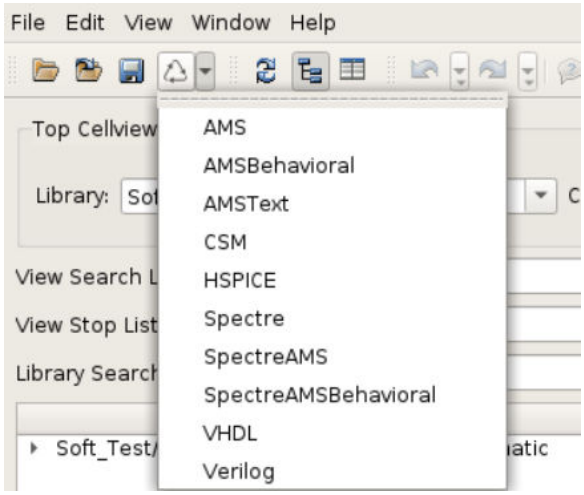
Specifying a Template for a Hierarchy Configuration View

Specify a template for a config view to set the view search and view stop list for the current configuration from predefined settings associated with a tool.

1. Begin by opening the Hierarchy Editor. Open the config view of a schematic, and choose **Tools > Hierarchy Editor**.
2. In the Hierarchy Editor, choose **Edit > Use Template**, and select a simulator by name.

Feedback

3. (Optional) Supply a template by choosing **Edit > Use Template**, and selecting a tool name, or by clicking the **Use Template** button.



Templates are available for AMS, AMSBehavioral, AMSText, CSM, HSPICE, Spectre, SpectreAMS, SpectreAMSBehavioral, VHDL, and Verilog. Supplying a template populates the design cell list and view search and stop lists.

Using Hierarchy View Name Groups

View Name Groups define aliases for commonly used lists of View Names. For example, you can group the view name list VERILOG VLG VLG2 VHDL into a group named DIGHDL. View name groups are referenced by prepending a percent character (‘%’) before the View Name Group.

You can add, edit, (or delete) view name groups to change view name groups associated with a config view.

To use hierarchy view name groups:

1. Begin by opening the Hierarchy Editor. Open the config view of a schematic, and choose **Tools > Hierarchy Editor**.
2. In the Hierarchy Editor, choose **Edit > View Name Groups**.
3. Choose an option:

   To add a new group, in the dialog box add an empty row (if necessary) by right-clicking the mouse, and choosing **Add** from the menu. In the group name column, enter the name of the group. In the view list column, specify the view search list.

   To edit a name or view list, select the corresponding field, and edit.

   To delete a name group, select the row, and choose **Delete**.

Feedback

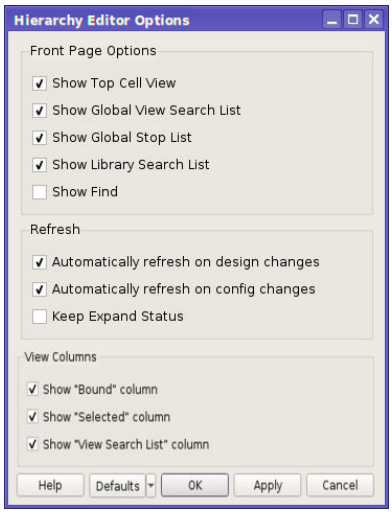
The system validates all changes and, for invalid edits, reports errors.

4. Click **OK**.

Setting Hierarchy Editor Options

Set Hierarchy Editor options to control which information you can view in the Hierarchy Editor. Setting options here makes them available for selection through the Hierarchy Editor View menu.

1. Begin by opening the Hierarchy Editor. Open the config view of a schematic, and choose **Tools > Hierarchy Editor**.
2. In the Hierarchy Editor, choose **Edit > Options**.



3. Set options as described in Table 94.

Table 94 Hierarchy Editor Options

Option	Description
Front Page Options	Show Top Cell View
	Show Global View Search List
	Show Global View Stop List
	Show Library Search List
	Show Find
Refresh	Automatically refresh on design changes

Feedback

Option	Description
	Automatically refresh on config changes
	Automatically cross select instances on design
	Keep Expand Status
View Columns	Show "Bound" Column
	Show "Selected" Column
	Show "View Search List" Column
Defaults	Sets the options to their default.

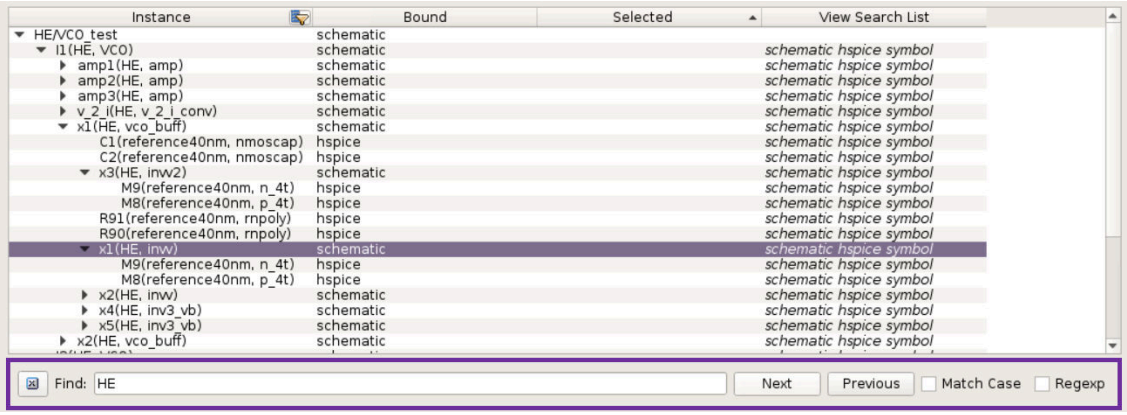
4. Click **OK**.

Searching Configuration Views

You can search in config views by instance name, library name, and cell name.

To search a config view:

1. Press Ctrl+F or choose **View > Find**. The **Find** window appears at the bottom of the Hierarchy Editor.



2. Enter a search term in the **Find** window. Press Enter, or use the **Next** and **Previous** buttons to scroll through the found items.
3. To close the **Find** window, click .

Filtering Configuration Views on Libraries or Cells

You can hide selected libraries and cells from the configuration view.

Feedback

To filter a configuration view:

1. Click . The **Filter** dialog box appears.

2. Select the **Library** (and **Cell**, if required) on which to filter.
3. Click to add a filter row under the current row, or to delete a filter row.
4. Click **OK** to apply the filter(s) and close the dialog box.

Controlling Viewed Information

Control viewed information to work more easily with a configuration.

- Synchronizing Design and Hierarchy Data
- Enabling Automatic Cross Selection Between Hierarchy Editor and Design Editor
- Showing Hierarchy View Search and Stop Lists
- Showing Hierarchy Tree or Cell Views
- Showing and Hiding Vector Bits
- Selecting the View or Library
- Viewing Unbound Cell or Instance Information
- Selecting and Clearing Scoped Views

Synchronizing Design and Hierarchy Data

Synchronize data by refreshing the design on demand or automatically, or by refreshing the config view automatically.

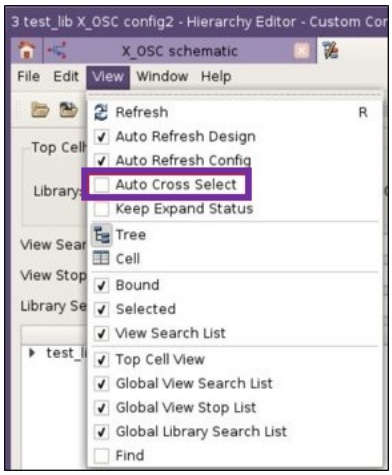
Feedback

Begin by opening the Hierarchy Editor. Open the config view of a schematic, and choose **Tools > Hierarchy Editor**.

- To synchronize the config view with the design on demand, choose **View > Refresh** in the Hierarchy Editor.
- To update the visible tree or cell hierarchy after each design change, choose **View > Auto Refresh Design** in the Hierarchy Editor.
- To update the visible tree or cell hierarchy after each config change, choose **View > Auto Refresh Config** in the Hierarchy Editor.

Enabling Automatic Cross Selection Between Hierarchy Editor and Design Editor

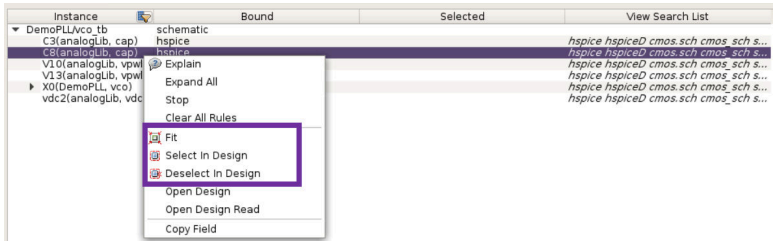
1. Begin by opening the Hierarchy Editor. Open the config view of a schematic, and choose **Tools > Hierarchy Editor**.
2. To enable automatic cross selection between Hierarchy Editor and Design Editor, choose **View > Auto Cross Select** in the Hierarchy Editor.



3. Show, and adjust the top cellview displayed, by choosing **View > Top Cell View**.

Feedback

4. Right-click on any instance to choose:
   - **Fit**: Zooms to the selected instance or instances in the latest opened window with the same configuration.
   - **Select in Design**: Selects the selected instance or instances in the latest opened window with the same configuration.
   - **Deselect in Design**: Deselects the selected instance or instances in the latest opened window with the same configuration.



Showing Hierarchy View Search and Stop Lists

Show hierarchy search and stop lists in the Hierarchy Editor.

1. Begin by opening the Hierarchy Editor. Open the config view of a schematic, and choose **Tools > Hierarchy Editor**.
2. Specify the information to show.
   - Show, and adjust the top cellview displayed, by choosing **View > Top Cell View**.
   - Show the search and stop lists by choosing **View > Global View Search List**, and **View > Global View Stop List**.
   - Show the library search list (useful for schematic driven layout design) by choosing **View > Global Library Search List**.

Showing Hierarchy Tree or Cell Views

In the Hierarchy Editor, you can show the tree or cell view of a design.

Feedback

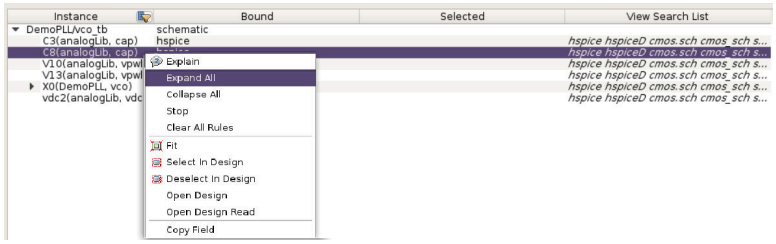
To show the tree or cell view:

1. Begin by opening the Hierarchy Editor. Open the config view of a schematic, and choose **Tools > Hierarchy Editor**.
2. In the Hierarchy Editor, choose a view:
   - Choose **View > Tree** (or click **Tree**) to list the complete hierarchical instance tree. Instance rules apply here.
   - Choose **View > Cell** (or click **Cell**) to list each cellview in the design. Cell-based rules apply here. The cell-based hierarchy view lets you bind all instances of a selected cell.

   The tree view is the default view. You can set the cell view as the default using the following Tcl command:

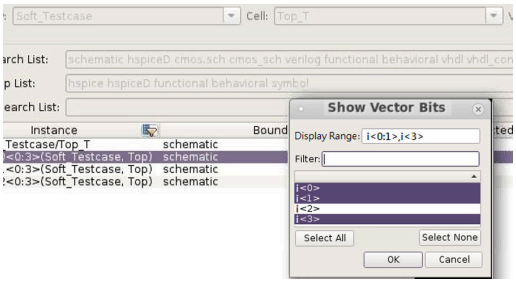
   ```
   db::setPrefValue heViewType -value cell
   ```

3. Expand or collapse the hierarchy by right-clicking in the **Instance** column and choosing **Expand All** or **Collapse All**. (Select more than one instance row to expand or collapse all selected instances).



Showing and Hiding Vector Bits

From the **Show Vector Bits** dialog box, you can expand vector bits by input bit range.



Select the display bits from the list or input the display range by updating the text field. The default display range value is the full vector range of this vector instance.

Choose **OK** to show the vector bits on the same level as the vector instance. These items are in gray text. The bits range number shows on the vector instance item if the display

Feedback

range is not the full bit range of this vector. After **Show Vector Bits**, this vector item is not allowed to expand, and the arrow icon is hidden.

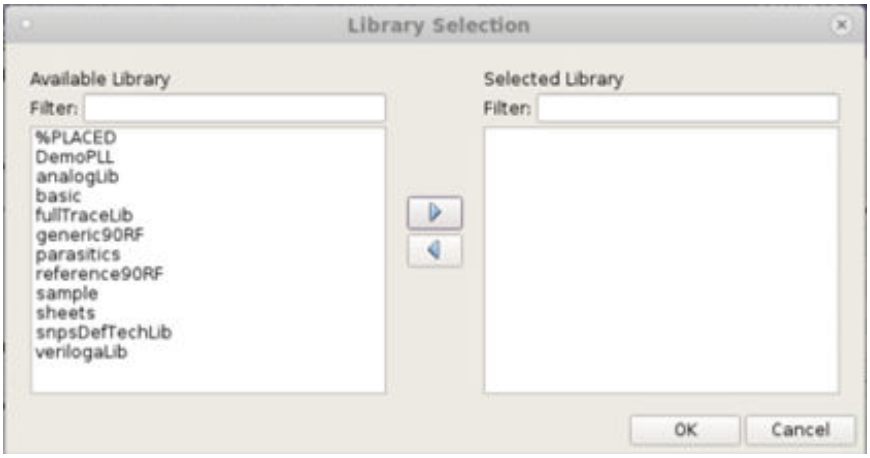
When a vector shows bits on the list, **Hide Vector Bits** is enabled, and you can hide the implicit bits without any rule in the list. If a bit contains any rule, it cannot be hidden.

Selecting the View or Library

You can select either the **View Selection** or the **Library Selection** dialog boxes to select the following lists:

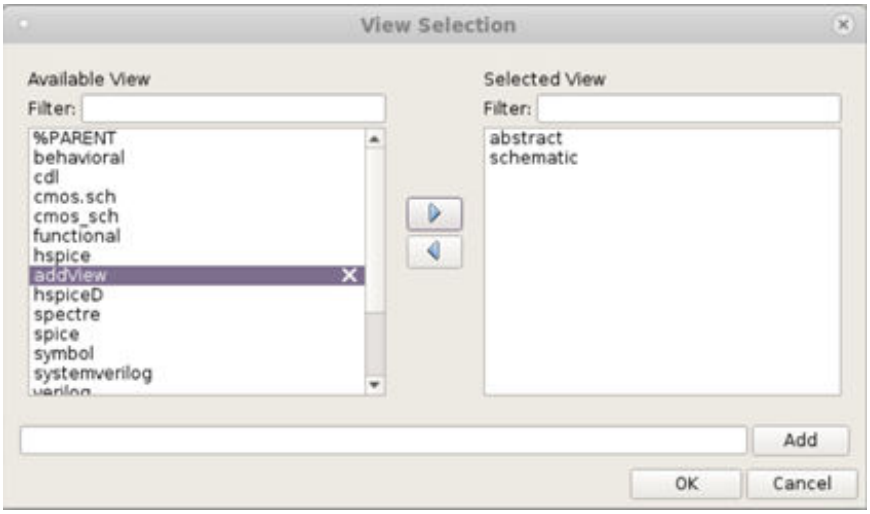
- View search list
- View stop list
- Library search list

In the **Library Selection** dialog box, the opened libraries and keyword, %PLACED are listed in the **Available Library**. When you choose a library and click , the selected library is moved to the **Selected Library** list. The order of the available and selected library lists can be reordered by dragging and dropping the files. The library search list is filled according to the order of the selected library list, and you can filter the case-sensitive libraries.



The **View Selection** dialog box shows only commonly used views in the **Available View** list. Therefore, you can add views by typing them in the provided field, and separating them by space. If the views you add are set to the view search or stop list, they are added to the `heEditorAvailableViewList` preference, which is loaded the next time you launch the dialog box. You can remove a view from the available view list by clicking x beside the view. The preference is then updated. However, if the view remains in VSL or VTL, the preferences are not be updated.

Feedback



Viewing Unbound Cell or Instance Information

If there are unbound cells or instances in the config view, the **Show Unbound Information** button is enabled in the toolbar. (If the button is not enabled, there are no unbound cells or instances in this config view).

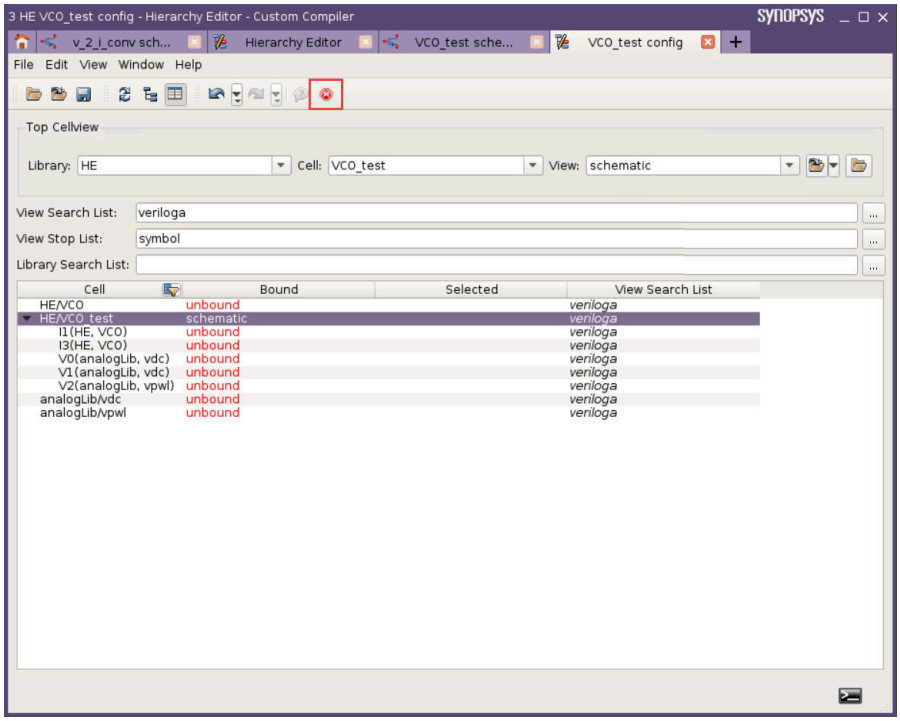
The **Show Unbound Information** button is available when:

- You are opening the config view for the first time.
- You have just refreshed the view (using **View > Refresh**).
- You have just switched from the tree view to the cell view.

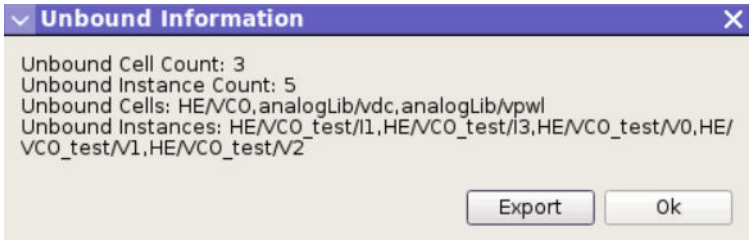
Feedback

To view unbound cell or instance information:

1. Select a cell (in the cell view) or instance (in the tree view) and click the **Show Unbound Information** button.



The **Unbound Information** dialog box appears.



2. (Optional) Click **Export** to save the unbound information to a text file.
3. Click **OK** to exit the dialog box.

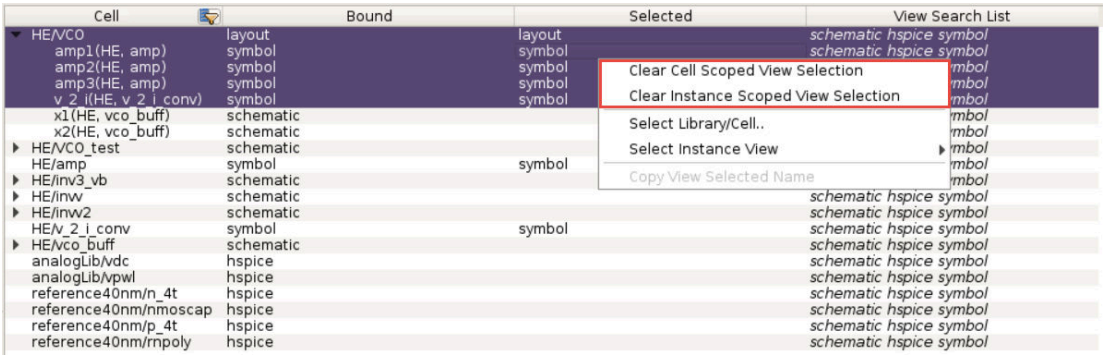
Feedback

Selecting and Clearing Scoped Views

To change the scoped view selection rule for instances or cells:

1. In either a tree or cell view, select an instance or a cell (or multiple instances/cells) and right-click in the **Selected** column.
2. Choose the required view from the menu that appears to set the selection rule.

   The configuration view and the design are automatically updated because the **Auto Refresh Design** and **Auto Refresh Config** options under the **View** menu are enabled by default.
3. Remove the scoped view selection rule (for cells or instances) by selecting an instance or a cell and right-clicking in the **Selected** column and choosing **Clear Cell Scoped View Selection** or **Clear Instance Scoped View Selection** from the menu. (You can select multiple instances/cells on which to perform this operation).



Hierarchy Editor Menus

The following tables describe the Hierarchy Editor menus:

- File Menu
- Edit Menu
- View Menu
- Window Menu
- Help Menu

Table 95 File Menu

Menu	Command	Description
File	New	Opens the **New CellView** dialog box. For more information, see Creating Designs.

Feedback

Table 95 File Menu (Continued)

Menu	Command	Description
	Open	Opens the **Select CellView** dialog box.
	Open Recent	Displays a list of recently opened cellviews. These can be opened in the Schematic Editor by clicking on their names.
	Open Top Cellview	Opens the top cellview design in the Schematic Editor.
	Export Hierarchy to Text File	Opens the **Export Hierarchy** dialog box.
	Export Netlist	Opens the **Export Netlist** dialog box with the config cell view of the Hierarchy Editor window.
	Save	Saves the config view.
	Save As	Opens the **Save As** dialog box.
	Discard Edits	Discards edits you made to the current config view.
	Make Read-Only	Makes the current config view read-only. If there are unsaved edits, you are prompted to save or discard them before making the config view read-only.
	Close	Closes the Hierarchy Editor.

Table 96 Edit Menu

Menu	Command	Description
Edit	Undo	Undoes the most recent modification. See Undoing and Redoing Configuration View Modifications.

Feedback

Table 96 Edit Menu (Continued)

Menu	Command	Description
	Redo	Redoes the most recent modification. See Undoing and Redoing Configuration View Modifications.
	Explain	Opens the **Explain** dialog box, which explains the rule that determines how the view is found for this configuration. See Explaining Found Hierarchy Views.
	Import	Imports attributes from another config view. See Importing Attributes From Existing Configuration Views.
	Use Template	Specifies a template for a config view to set the view search and view stop list for the current configuration from predefined settings associated with a tool. See Specifying a Template for a Hierarchy Configuration View.
	View Name Groups	Opens the **Edit View Name Groups** dialog box. See Using Hierarchy View Name Groups.
	Options	Opens the **Hierarchy Editor Options** dialog box. See Setting Hierarchy Editor Options.

Table 97 View Menu

Menu	Command	Description
View	Refresh	Synchronizes the config view with the design. See Synchronizing Design and Hierarchy Data.
	Auto Refresh Design	Updates the visible tree or cell hierarchy after each design change. See Synchronizing Design and Hierarchy Data.
	Auto Refresh Config	Updates the visible tree or cell hierarchy after each config change. See Synchronizing Design and Hierarchy Data.
	Auto Cross Select	Enables automatic cross selection between Hierarchy Editor and Design Editor. This is only visible when switching to tree pane. When checked, it cross selects any instance when selection changes in both Hierarchy Editor and Design Editor. See Synchronizing Design and Hierarchy Data
	Keep Expand Status	Keeps the expanded status of the visible tree or cell hierarchy after each rule change.

Feedback

Table 97 View Menu (Continued)

Menu	Command	Description
	Tree	Displays the instance-based hierarchy tree view. See Showing Hierarchy Tree or Cell Views.
	Cell	Displays the cell-based hierarchy view. See Showing Hierarchy Tree or Cell Views.
	Bound	Displays the **Bound** column.
	Selected	Displays the **Selected** column.
	View Search List	Displays the **View Search List** column.
	Top Cell View	Displays the **Top Cell View** information at the top of the Hierarchy Editor.
	Global View Search List	Displays the global **View Search List** information at the top of the Hierarchy Editor.
	Global View Stop List	Displays the global **View Stop List** information at the top of the Hierarchy Editor.
	Global Library Search List	Displays the global **Library Search List** information at the top of the Hierarchy Editor.
	Find	Displays the **Find** window.

Table 98 Window Menu

Menu	Command	Description
Window	New Window	Opens a new Custom Compiler window.
	New Tab	Opens a new Custom Compiler **Home**tab.
	Full Screen	Makes Hierarchy Editor full screen.
	Assistants	Contains the **Console** command which displays the **Console** pane.
	Toolbars	Displays the toolbars if checked.
	Hide All	Hides all menus.

Feedback

Table 98 Window Menu (Continued)

Menu	Command	Description
	Customize	Opens the **Customize Window** dialog box so that you can add or remove menus and toolbars to create a customized work environment. See Configuring Custom Windows.
	Keyboard Shortcuts	Opens a dialog box that displays keyboard shortcuts for certain commands. For more information, see Customizing Keyboard Shortcuts.
	<window_name>	At the bottom of the Window menu, the names of the open windows are displayed. You can choose the name of the window that you want to display.

Table 99 Help Menu

Menu	Command	Description
Help	Use System Web Browser	Opens the Custom Compiler help in an external system browser rather than the browser built in to the Custom Compiler environment.
	Custom Compiler Help	Displays the Custom Compiler help in a new tab.
	Hierarchy Editor Help	Opens the Hierarchy Editor help in a different tab or in an external system browser, depending on whether or not **Use System Web Browser** is checked.
	About Custom Compiler	Opens the **About Custom Compiler** dialog box containing information about the Custom Compiler version number, copyright, legal notices, and Synopsys contact information.

Hierarchy Editor Toolbar

The toolbar contains the following:

Table 100 Hierarchy Editor Toolbar

Button	Description
Open	Opens the **Select CellView** dialog box.

Feedback

Table 100 Hierarchy Editor Toolbar (Continued)

Button	Description
Open Top Cellview	Opens the top cellview design in the Schematic Editor.
Save	Saves the config view.
Use Template	Supplies a template.
Refresh	Synchronizes the config view with the design. See Synchronizing Design and Hierarchy Data.
Tree	Displays the complete hierarchical instance tree.
Cell	Displays each cellview in the design.
Undo	Undoes the most recent modification. Select from a list of recent modifications by clicking the Down Arrow. See Undoing and Redoing Configuration View Modifications.
Redo	Redoes the most recent modification. Select from a list of recent modifications by clicking the Down Arrow. See Undoing and Redoing Configuration View Modifications.
Explain	Opens the **Explain** dialog box, which explains the rule that determines how the view is found for this configuration. See Explaining Found Hierarchy Views.
Show Unbound Information	Opens the **Unbound Information** dialog box, which provides a summary of information about unbound cells. This option is only available if there are unbound cells or instances in the config view. Choose **Export** to save the information to a text file. See Viewing Unbound Cell or Instance Information.

Feedback

Library Manager Tool

This topic describes the following:

- About the Library Manager
- Displaying the Library Manager
- Showing and Hiding Libraries and Cells
- Viewing Library Attributes
- Opening a Terminal From the Directory of a Library
- Displaying or Hiding Cellview Version Information
- Displaying or Hiding the Library and Cell File Browser
- Copying Library, Cell, and Cellview Names
- Copying Library, Cell, and Cellview Paths
- Creating New Libraries
- Editing Library Definitions
- Creating Library Categories
- Controlling Library Availability
- Compressing and Uncompressing Libraries
- Modifying Component Description Format (CDF) Overlays
- Managing Individual Libraries
- Managing Cells
- Managing Cellviews
- Managing Duplicate Name Conflicts
- Managing Properties
- Using Cell Categories
- Post-processing MCells
- Library Manager Menus
- Library Manager Toolbar

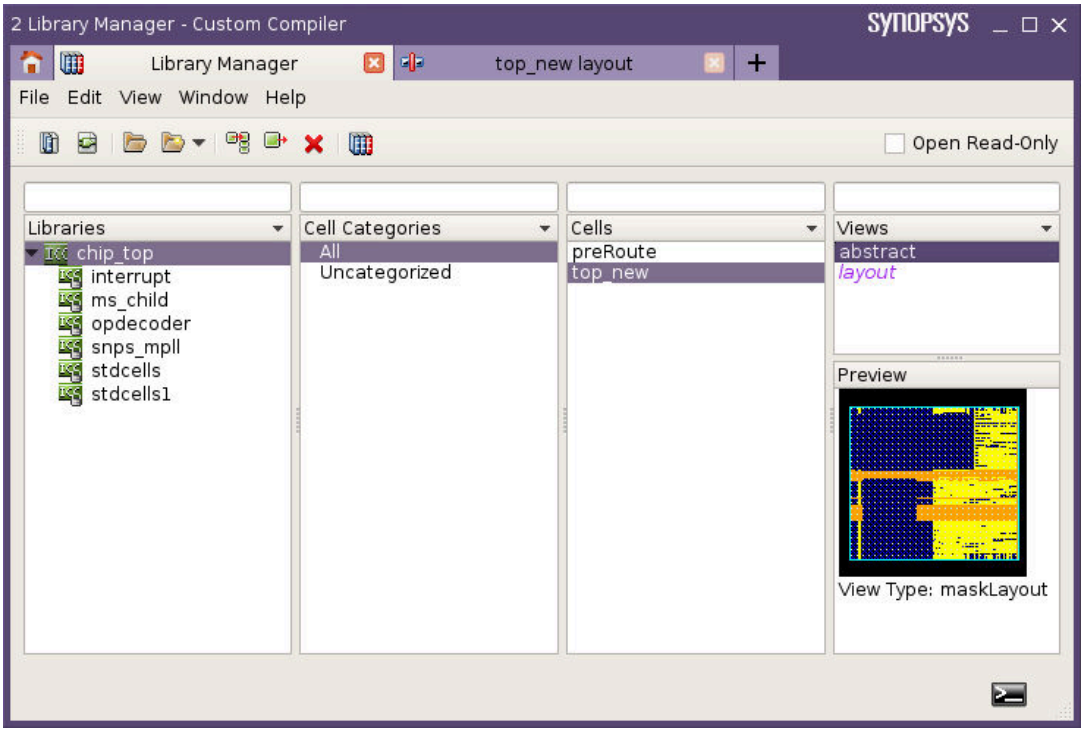
Feedback

About the Library Manager

Every design in the Custom Compiler environment has the following components:

Design Component	Description
Library	The collection of data that comprise a whole or partial design that you create. Common data, or reference libraries, contain master copies of cells that you can reference in multiple designs.
Cell	Building blocks of the designs.
View	Representations of a cell. For example, a cell can have a schematic view, a layout view, or a symbol view.

The Library Manager displays information about the designs and lets you manage libraries, cells, and views for the design process.



Feedback

For example:

- The **Libraries** pane uses icons to distinguish between library sources. Library names without icons are Custom Compiler libraries. Fusion Compiler, IC Compiler II, and IC Compiler libraries have the following icon associations:

  Fusion Compiler

  IC Compiler II library

  IC Compiler library

  The list of libraries and the category of libraries displayed in the Library Manager are defined in the library definitions (`lib.defs`) file. The file maps the libraries to their physical locations on the computer disk. You can view and edit the files using the Library Definitions Editor. See Editing Library Definitions and Creating Library Categories.

- You can use the **Cell Categories** pane to simplify and organize large lists of cells.
- When you choose a library, cell, or view, the **Preview** pane shows the directory corresponding to your selection or a thumbnail view of the cellview, the date and time it was last modified, the user who last modified the cellview, the owner of the directory, and the read/write permissions.

  By default, the **Preview** pane appears under the Views pane. To move it so that is beneath all the other panes, choose **View > Display Preview Horizontally** or set the `dmPreviewPosition` Tcl preference to `bottom`.

- When you choose the library, cell, and view, a thumbnail of the design displays in the **Preview** pane to provide visual confirmation.

  To update preview, choose **Edit > Update Preview**, or right-click a library, cell, or cellview and select **Update Preview** . The corresponding Tcl command is `dm::updatePreviews`.

- The **Views** pane uses the icon to distinguish between designs created in the Custom Compiler environment and in third-party products. (For the icon to display, you must set the `dmMarkSavedDesigns` Tcl preference to `true`).
- Colored cell and view names indicate that they are already open and are in either the edit mode (purple) or read-only mode (green).

By default, the **Views** pane is displayed. If you prefer to display a customized pane instead, you can hide the **View** pane. To hide the **Views** pane, set the `dmHideViews` Tcl preference to `true`. To create a customized pane, see Tools for Designing.

Feedback

See Also

- Displaying or Hiding Cellview Version Information
- Displaying or Hiding the Library and Cell File Browser

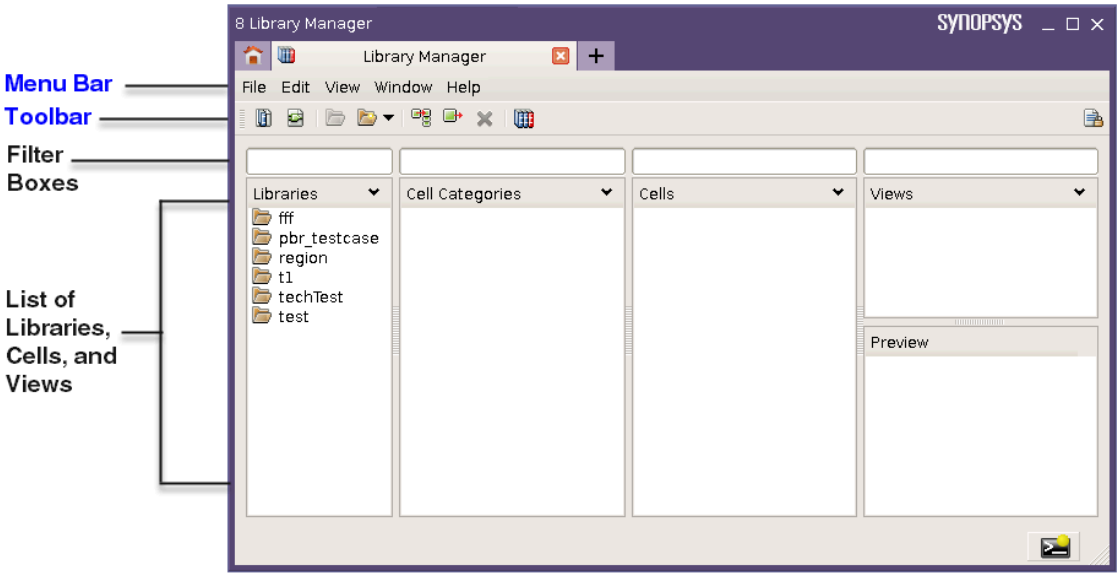
Displaying the Library Manager

The Library Manager lets you view and manage libraries. You can open multiple Library Manager windows, as needed.

To display the Library Manager, use one of these methods:

- From the **Console** tab or any open design window, choose **Tools > Library Manager**.
- In any design window toolbar, click the **Library Manager** button.

The Library Manager consists of the following:



Component	Description
Menu Bar	Contains menus of commands. For more information, see Library Manager Menus.
Toolbar	Displays button commands. For more information, see Library Manager Toolbar.

Feedback

Component	Description
Filter Boxes	Filters the list directly below the text box to find the closest match. You can use wildcards (*) or partial text. When you provide a partial word, the filter lists all the names that contain the partial word. **Cell Categories**, **Cells**, and **Views** cannot be filtered if no library is selected or if multiple libraries are selected.
List of Libraries, Cells, and Views	**Library**, **Cell Categories**, **Cells,** and **Views** display a hierarchical list of items that can be filtered by the filter boxes.

The Library Manager appears and displays the list of libraries. The Library Manager also lists read-only libraries. If you select a read-only library in the **Library** list:

- **Open Read-Only** in the upper right corner of the window is disabled.
- The design you select in the read-only library opens as read-only.
- Some **File** and **Edit** commands are disabled.
- The **Edit > Move** dialog box does not let you select read-only libraries as the source.
- The **Edit > Copy** dialog box does not let you select read-only libraries as the destination.
- User-interface operations, such as drag-and-drop actions, are disabled.

You can optionally open a second Library Manager window by clicking **New Library Manager** at the top of the first Library Manager window.

You can pin the Library Manager using the context menu item **Pin Library Manager**, available by right-clicking on the Library Manager tab.

Showing and Hiding Libraries and Cells

You can show and hide libraries and cells in the Library Manager. Hiding libraries and cells can be useful if the PDK contains too many libraries and cells, and you only want to display those that are used in your design.

This topic describes the following:

- Before You Start
- Showing and Hiding Libraries
- Showing and Hiding Cells

Feedback

Before You Start

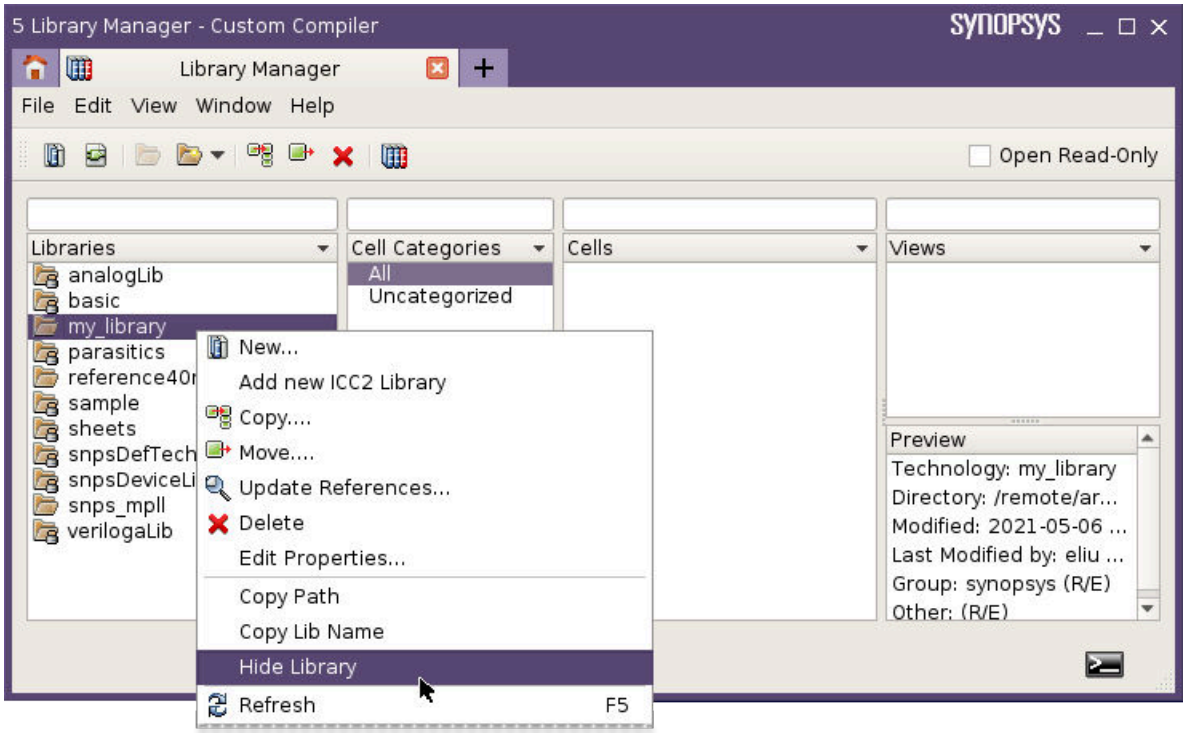
To show or hide the top-level library, you must have permission to modify the lib.defs file. To show or hide library cells, you must have write access to *.Cats files and their directories.

The `shown` attribute of the `dmCell` object works independently of the show and hide procedures in the Library Manager. For example, if you want to show a cell but the `shown` attribute is false, you must change it to true in addition to using the show procedure in the Library Manager as described in this topic.

Showing and Hiding Libraries

To hide a library:

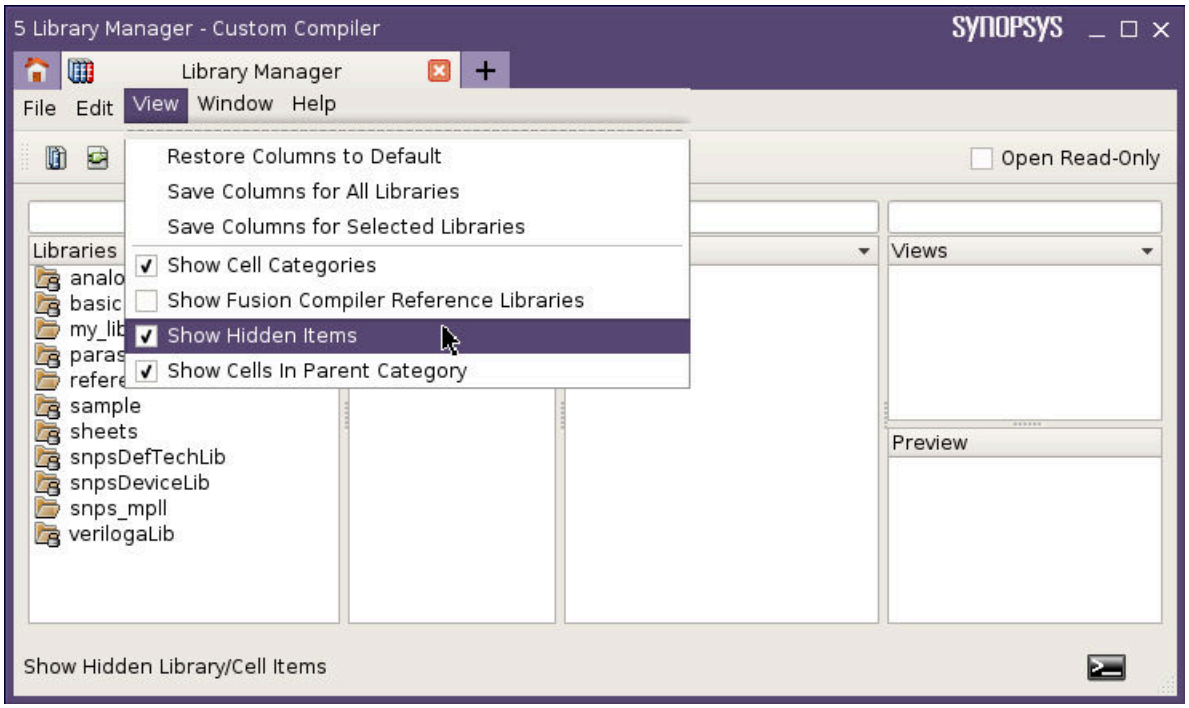
► In the Library Manager, right-click the library name and choose **Hide Library**.



Feedback

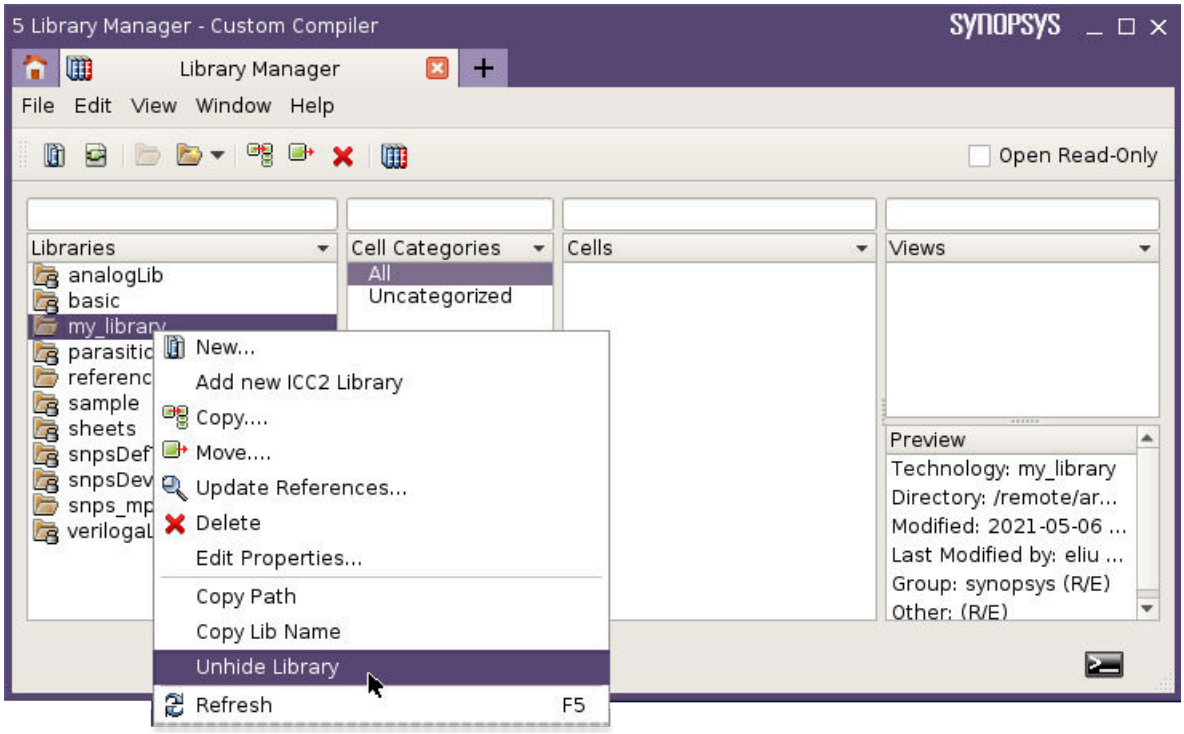
To show a hidden library:

1. In the Library Manager, choose **View** > **Show Hidden Items** so that the hidden library name is visible.



Feedback

2. Right-click the library name, and choose **Unhide Library**.

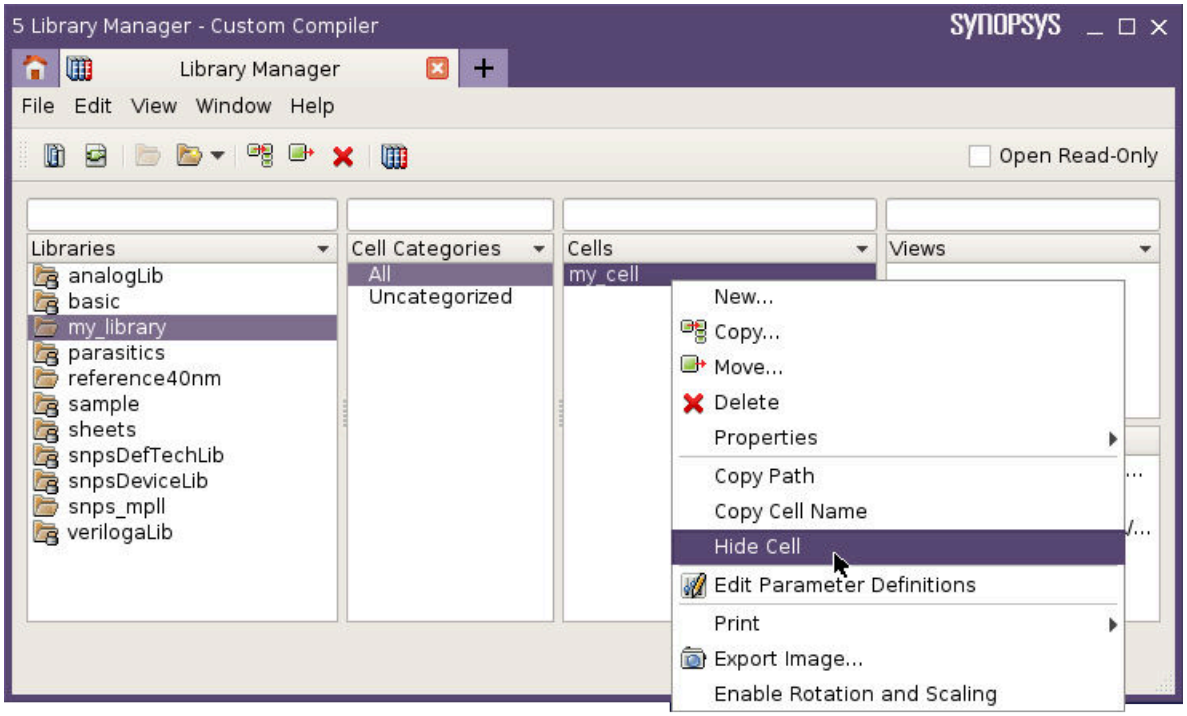


Feedback

Showing and Hiding Cells

To hide a library cell:

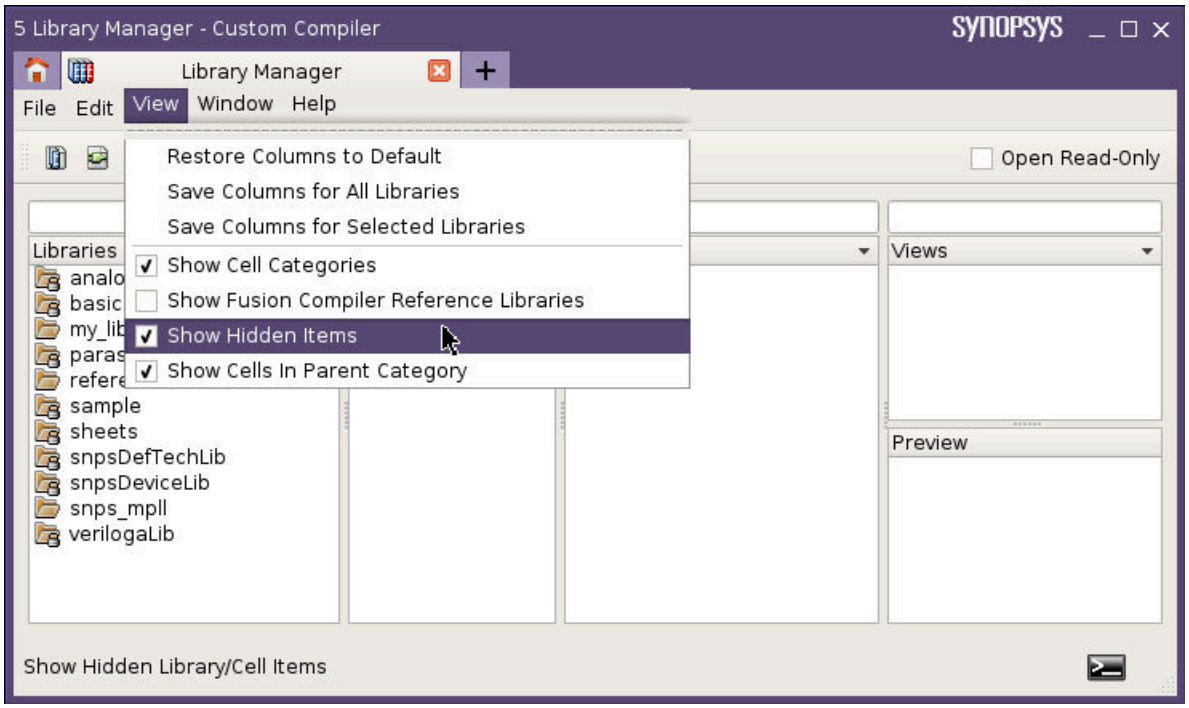
► In the Library Manager, right-click the cell name and choose **Hide Cell**.



Feedback

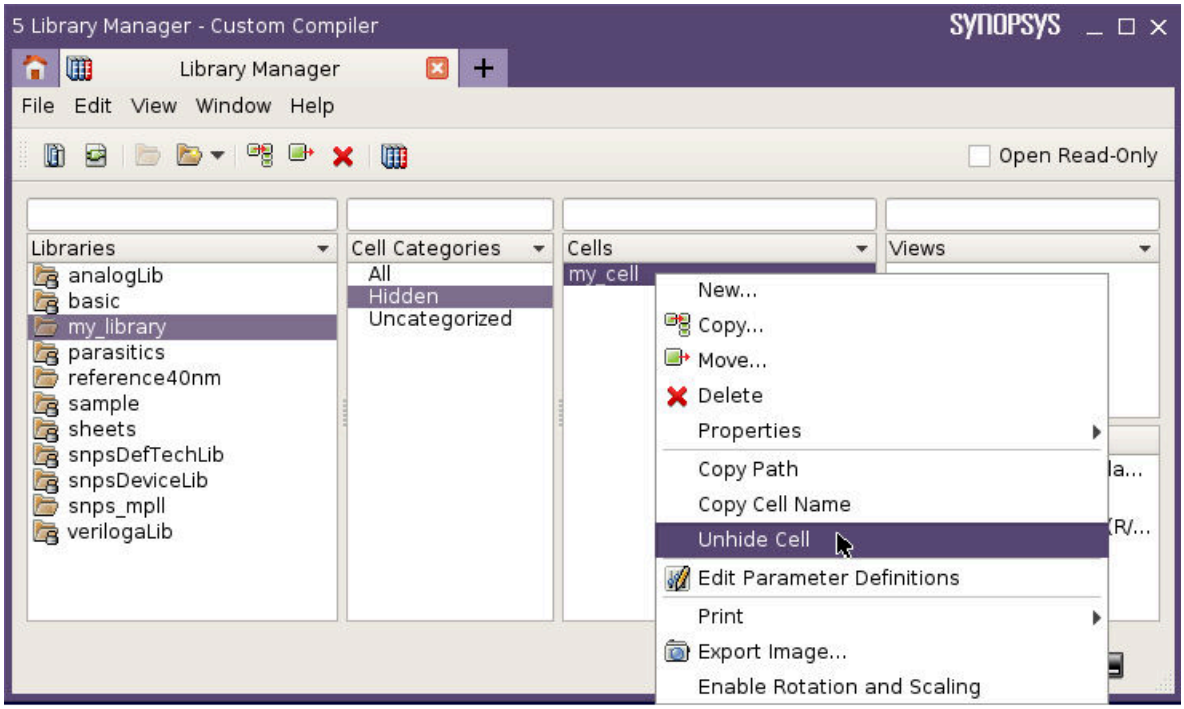
To show a hidden library cell:

1. In the Library Manager, choose **View** > **Show Hidden Items** so that the hidden cell name is visible.



Feedback

2. Right-click the cell name, and choose **Unhide Cell**.



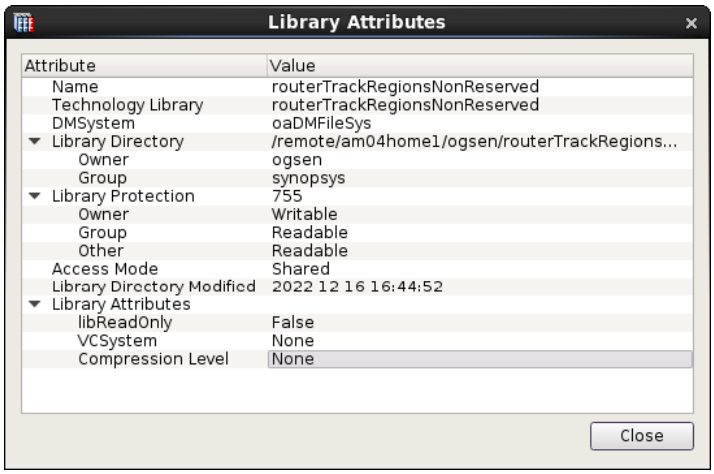
Viewing Library Attributes

You can view library attributes to see details about one library. You can copy (using Ctrl+C) and paste the text you see outside of the Custom Compiler environment.

Feedback

To view library attributes:

1. In the Library Manager, select the library name, right-click, and choose **Library Attributes**. The **Library Attributes** dialog box opens.



2. View the library attributes. The following table describes the information shown in the **Library Attributes** dialog box:

Attribute	Description
Name	Library name in the CDBA namespace
Name mapped for `lib.defs`	Library name in the Linux namespace
Technology Library	Name of the library containing the OpenAccess technology for this library, if found. Red font indicates that either an OpenAccess library is not found or the techLibName property for the current library is not found.
DMSystem	Data Management system type, for example oaDMFileSys; oaLib::getDMSystemName() returns this information
Library Directory	Full path to this library; oaLib::getFullPath() returns this information
Library Protection	Access permission: Writable, Readable, Accessible, and No Access for Owner, Group, and Other
Access Mode	Either Shared, Nonshared, or ReadOnly; oaLib::getMode() returns this information

Feedback

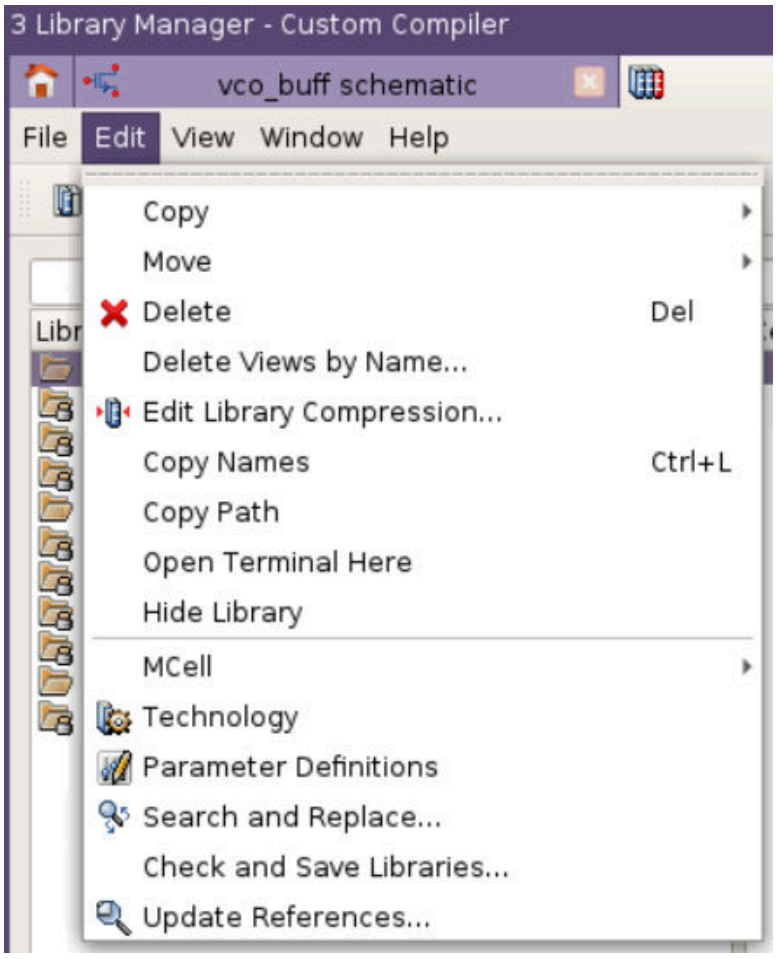
Attribute	Description
Library Directory Modified	Last modification year, month, date, and time
Library Attributes	General attributes, if available; The following Compression Level values indicate if the library is compressed using oaZip: ◦ None: Indicates that the library is not compressed. ◦ 1-9: Indicates that the library is compressed.

Opening a Terminal From the Directory of a Library

The Library Manager lets you open a terminal from the directory of the selected library.

You can open the terminal with one of the following methods:

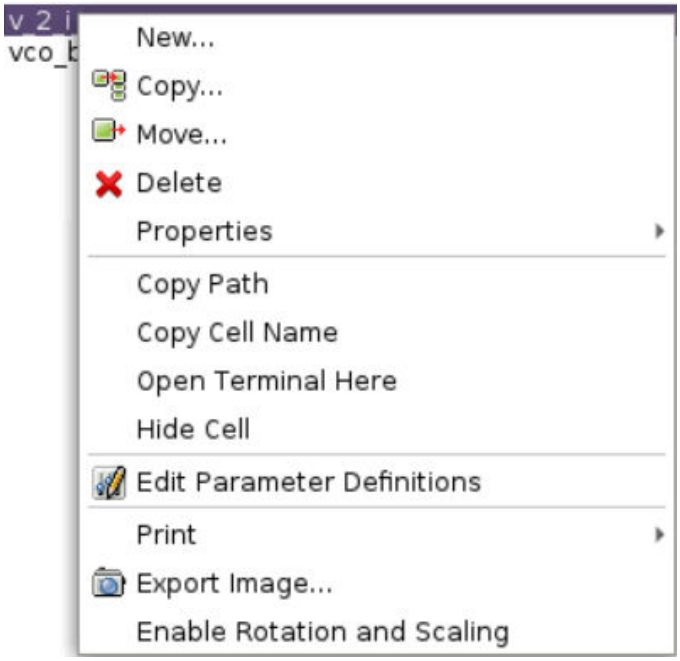
- Choose **Edit > Open Terminal Here**.



Feedback

- Right-click on a library, cell, or cellView and click **Open Terminal Here**.

  If the selected cell or cellView belongs to a combined library, the path is resolved to its real library. The `xt::execTerminal` command can be configured using the `xtTerminalCommand` preference, which defaults to `xterm`. If the current session exits, the `xt::execTerminal` command still be active.



In the Library Manager, if you select

- DMFiles in the design library and cell directories, the **Open Terminal Here** command uses the container path
- An oaDMTurbo library, the **Open Terminal Here** command uses the library path only, even if the selection is a cell, cellView, or DMFile

Displaying or Hiding Cellview Version Information

In the Library Manager, you can display or hide the cellview version information.

Displaying Cellview Version Information

Important:

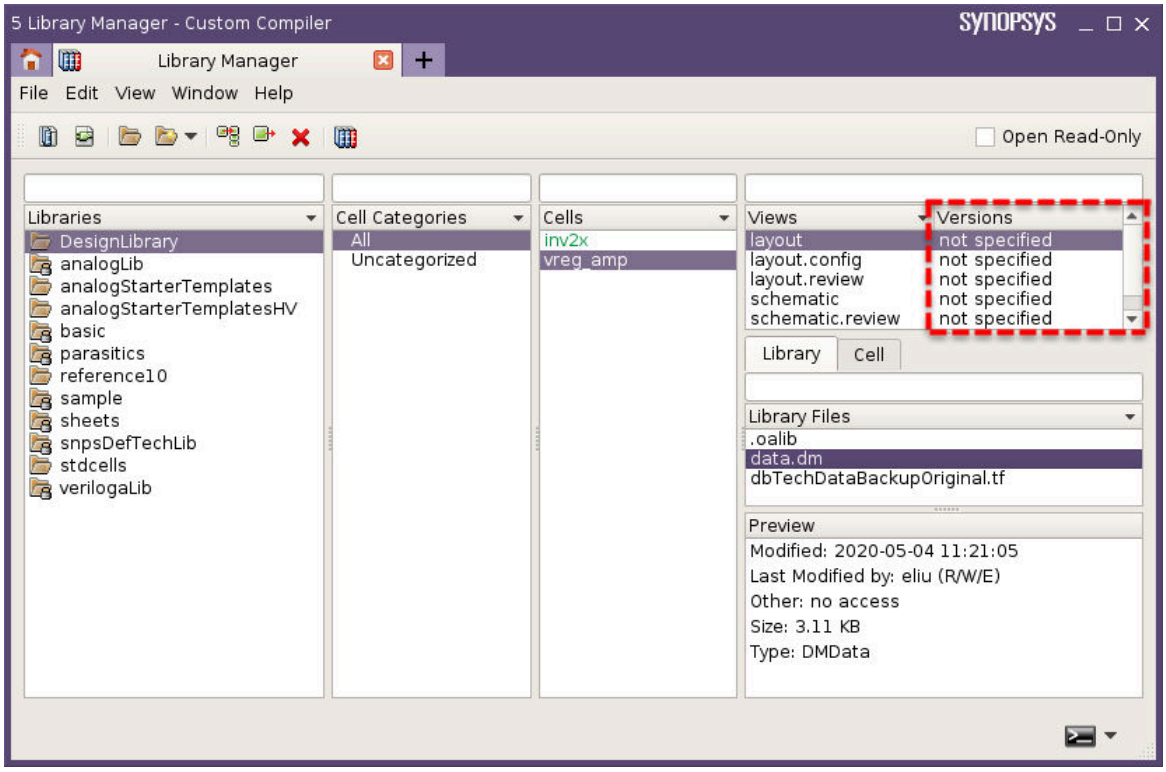
Before you start, make sure a version control system is registered. See your system administrator.

Feedback

To display the cellview version information:

► In the Library Manager, choose **View > Show Versions**.

The **Versions** column appears in the **Views** section.



See Also

- Displaying the Library Manager
- Displaying or Hiding the Library and Cell File Browser

Hiding Cellview Version Information

To display the cellview version information:

► In the Library Manager, click the **View** menu, and deselect **Show Versions**.

The **Versions** column is removed from the **Views** section.

Feedback

Displaying or Hiding the Library and Cell File Browser

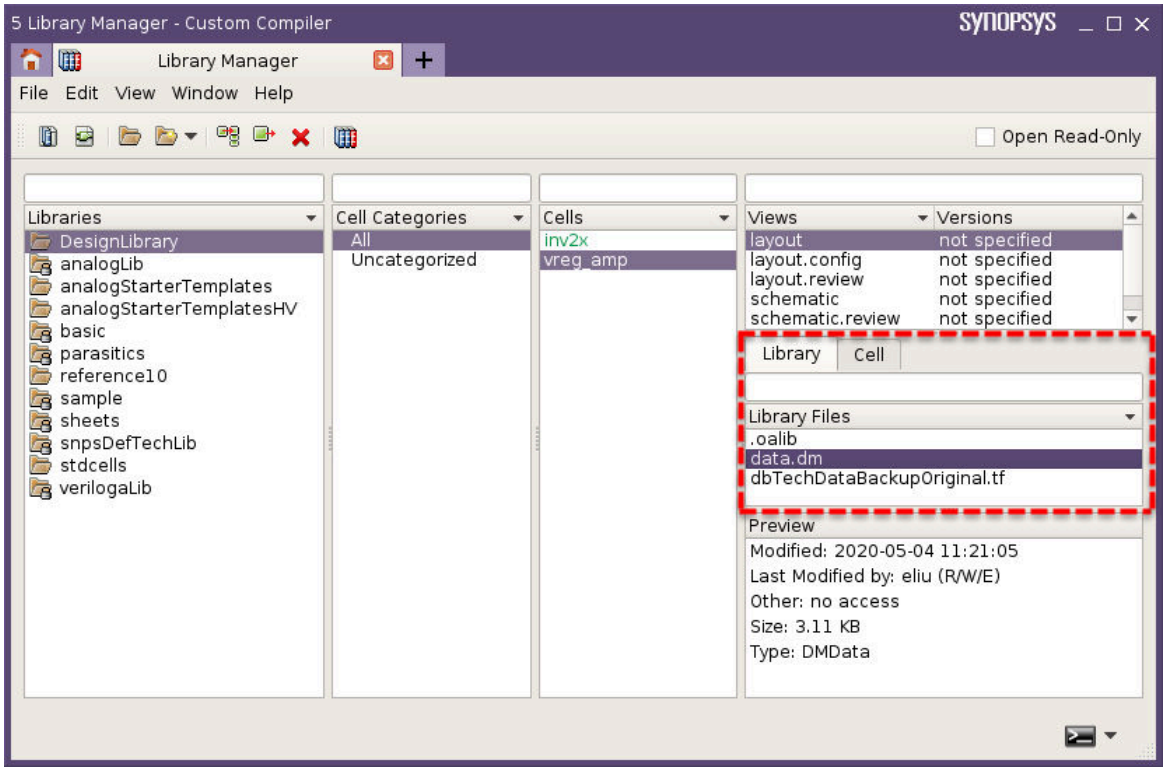
In the Library Manager, you can display or hide the list of DMFiles in the design library and cell directories. Displaying the DMFiles in the library and cell directories lets you view the associated files along side the library/cell/views. By default, the list of files are hidden.

Displaying the Library and Cell File Browser

To display the library and cell file browser:

► In the Library Manager, choose **View > Show File Browser**.

The file browser section appears. The **Library** tab lists the files that are in the design library directory. The **Cell** tab lists the files that are in the cell directory.



See Also

- Copying Library, Cell, and Cellview Names
- Copying Library, Cell, and Cellview Paths

Feedback

Hiding the Library and Cell File Browser

To hide the library and cell file browser:

► In the Library Manager, click the **View** menu, and deselect **Show File Browser**.

The file browser section is removed.

Copying Library, Cell, and Cellview Names

You can copy library, cell, and cellview names to make the names available for use outside of the Custom Compiler environment.

The methods you can use to copy the library, cell, or view names are:

- From the Libraries, Cells, or Views Sections
- From the File Browser

From the Libraries, Cells, or Views Sections

To copy the library, cell, and cellview names inclusively:

1. In the Library Manager, select the name you want to copy.

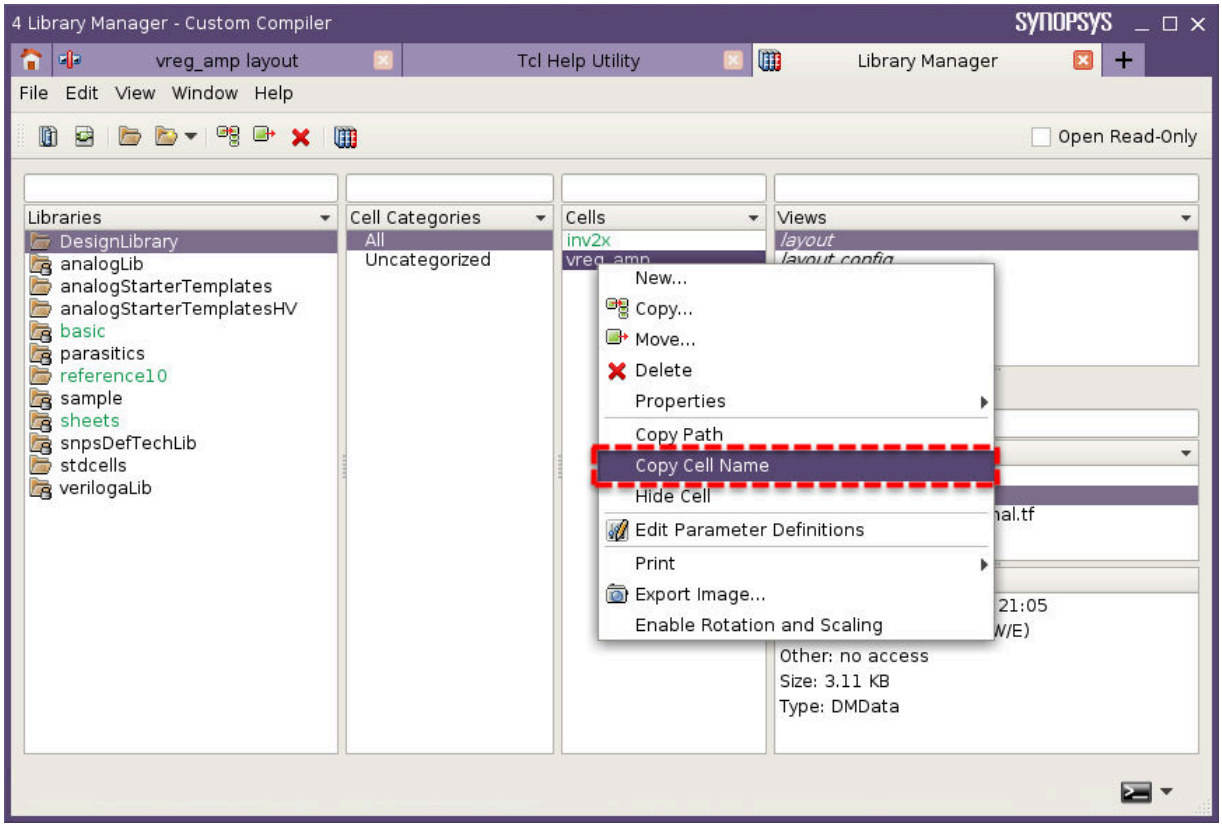
   A single name selection can extend to include the library, cell, and cellview. Categories are not copied.

2. Choose **Edit > Copy Names** or use the Ctrl+L keyboard shortcut.
3. Paste the copied text into a local terminal window or file.

To copy the library, cell, and cellview names independently:

1. In the Library Manager, right-click the name you want to copy. For example, if you want to copy the cell name, right-click the cell name.
2. Choose the corresponding command:
   - **Copy Lib Name**
   - **Copy Cell Name**
   - **Copy View Name**

Feedback



3. Paste the copied text into a local terminal window or file.

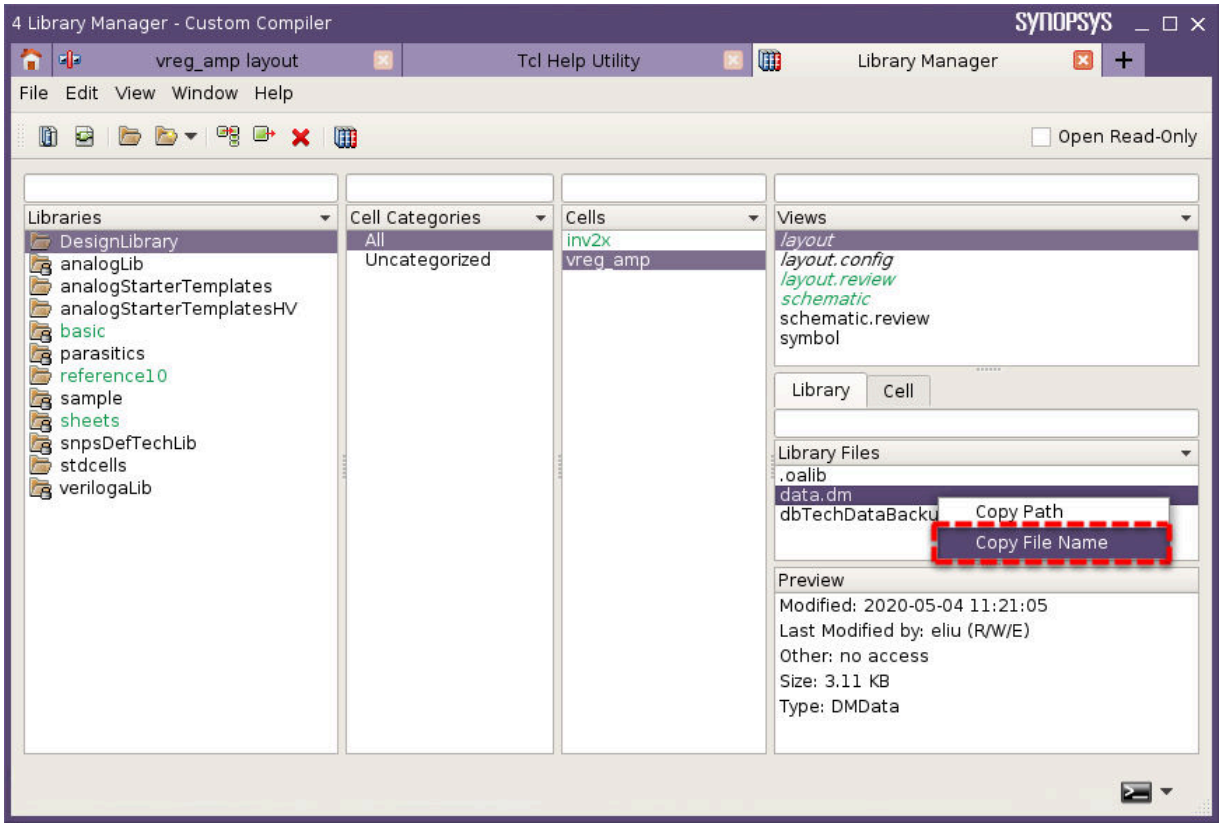
From the File Browser

In the Library Manager, you can display or hide the list of DMFiles in the design library and cell directories. If the DMFiles are displayed in the file browser, you can copy the DMFile names.

To copy the DMFile name from the file browser section:

1. In the Library Manager, right-click the file name in the **Library** or **Cell** tab.
2. In the menu that appears, choose **Copy File Name**.

Feedback



3. Paste the copied text into a local terminal window or file.

See Also

- Displaying or Hiding the Library and Cell File Browser

Copying Library, Cell, and Cellview Paths

You can copy the path of a library, cell, or cell view to view or access data in that folder. You can copy the path while in the Library Manager.

The methods you can use to copy the library, cell, or view paths are:

- From the Libraries, Cells, or Views Sections
- From the File Browser

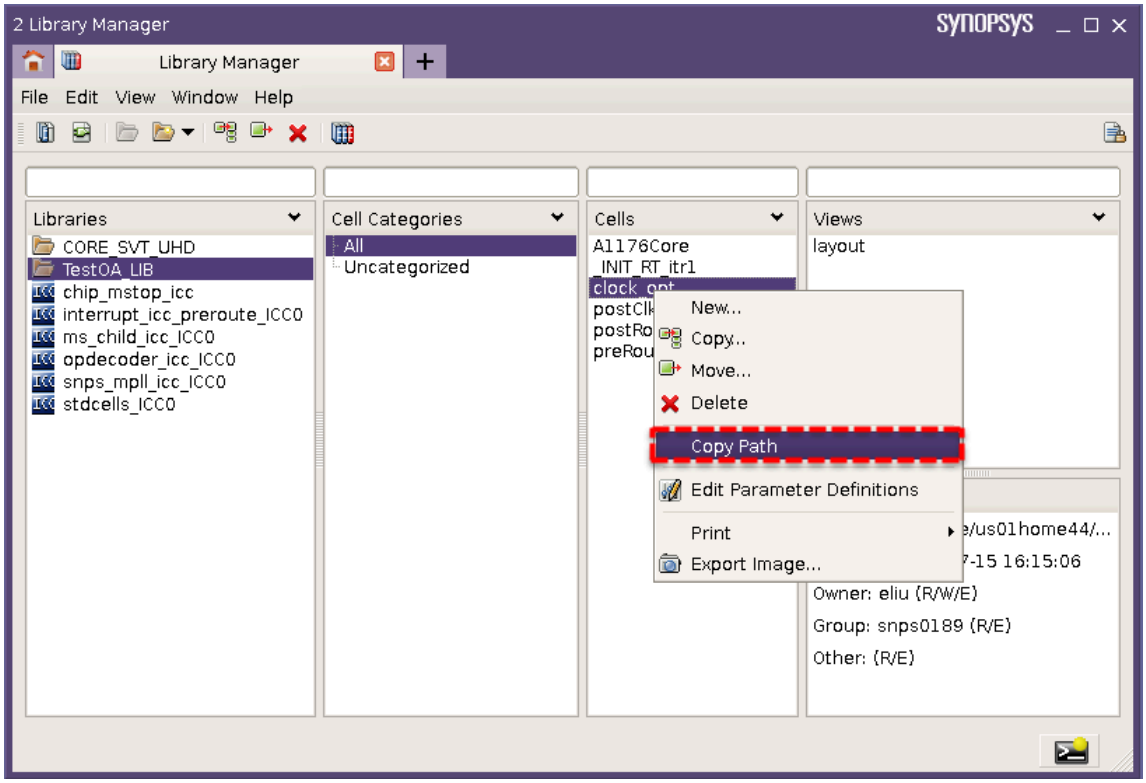
Feedback

From the Libraries, Cells, or Views Sections

To copy the path of a library, cell, or cell view from the Libraries, Cells, or Views sections:

1. In the Library Manager, right-click the **Library**, **Cell**, or **View** name.
2. In the menu that appears, choose **Copy Path**.

   If you select a cell or view and the library from an OpenAccess library (oaDMTurbo) library, the path of an oaDMTurbo library is copied.



3. Paste the path where you need the information.

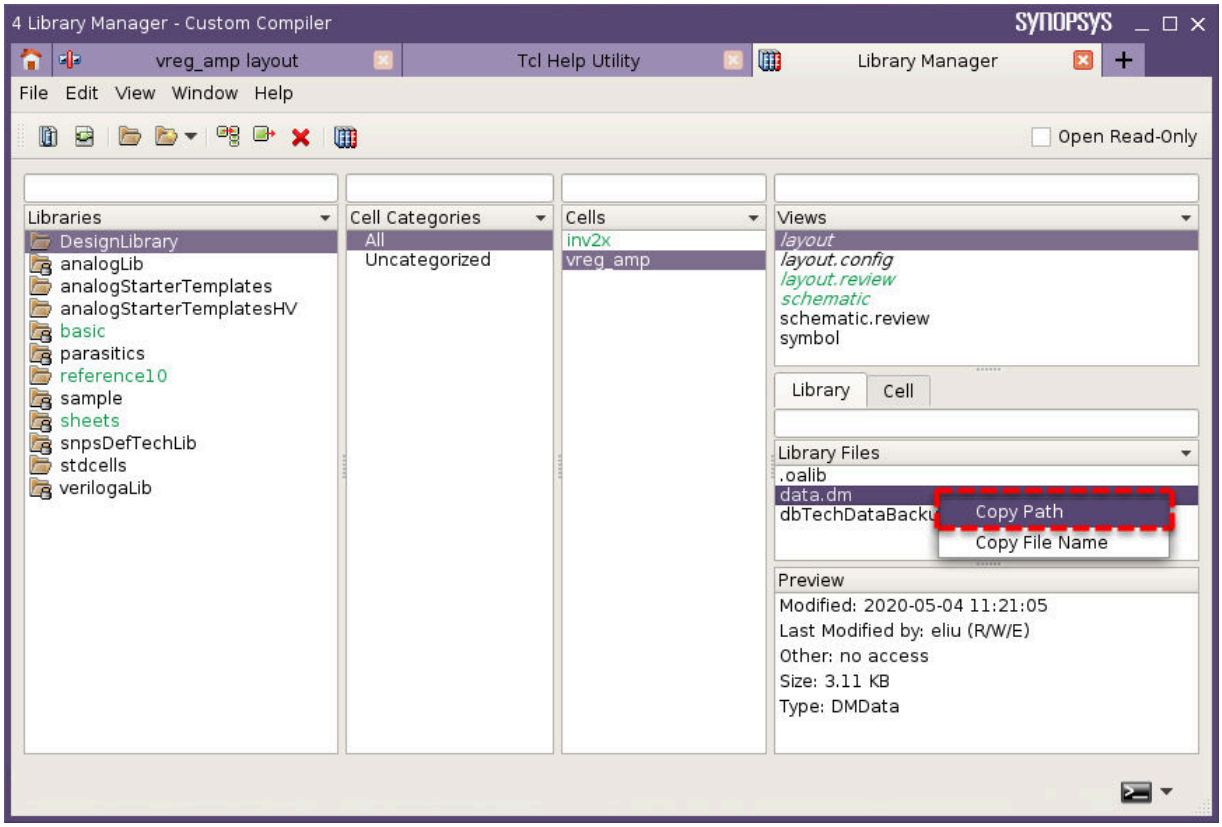
From the File Browser

In the Library Manager, you can display or hide the list of DMFiles in the design library and cell directories. If the DMFiles are displayed in the file browser, you can copy the DMFile paths.

To copy the DMFile path from the file browser section:

1. In the Library Manager, right-click the file name in the **Library** or **Cell** tab.
2. In the menu that appears, choose **Copy Path**.

Feedback



3. Paste the path where you need the information.

See Also

- Displaying or Hiding the Library and Cell File Browser

Feedback

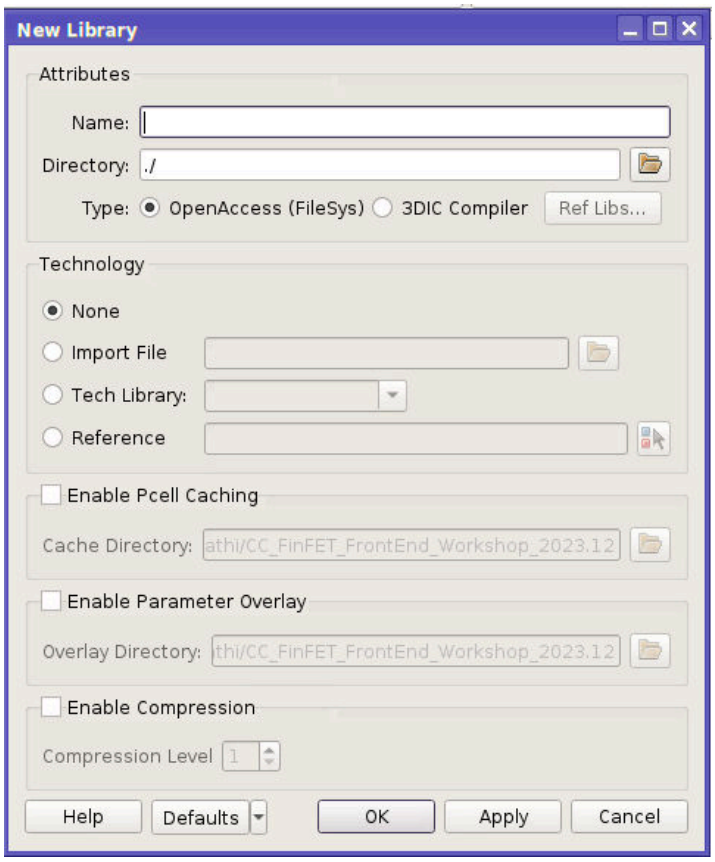
Creating New Libraries

You can create a new library to define a logical collection of data representing a complete design or some portion of it. Additionally, you can create libraries to define common data shared among several designs. A library is the top organizational level of design data.

To create a new library:

1. In the Library Manager:
   - Select any location in the Libraries list, right-click the mouse, and choose **New**.
   - From the **File** menu, choose **New > Library**.
   - From the **Toolbar**, click **Create Library** .

   The **New Library** dialog box opens.



2. Type the library **Name**.
3. Browse to choose a **Directory** to hold the library.

Feedback

The system stores this directory in the current library definition file. The stored path includes both the parent directory you entered and the library directory.

4. Select the library **Type**:
   - **OpenAccess (FileSys)** (Default)
   - **3DIC Compiler**
   - **Fusion Compiler**
   - **ICC2**
   - **ICC**
5. Set the options for the **OpenAccess** library.

   For the description of Fusion Compiler options, see Preparing Design Libraries for Fusion Compiler co-designing. For the description of ICC2 options, see Preparing Design Libraries for IC Compiler II co-designing. For the description of ICC options, see Preparing Libraries for IC Compiler Co-Designing.

Settings	Description
Technology	**None** creates a new library using the process-neutral technology file supplied with the environment. **Import File** uses the technology file you import by browsing to choose it. **Attach** uses the technology from the library you select from the list box. **Reference** uses an ordered list of open libraries to be incrementally referenced from the technology of the new library. Alternatively, the browse button invokes the **Technology References** dialog box, which lists all the currently open libraries with technology databases.
Enable Pcell Caching	Specifies a PCell cache to be used for supermasters in the library, and adds an ASSIGN statement to the library definition file with the cache directory location.
Enable Parameter Overlay	Specifies that component description format parameters can be overlaid, and adds an ASSIGN statement to the library definition file with the component description format file location.

Feedback

Settings	Description
Enable Compression	Select a value from the drop-down list to compress a new library. If the `dmEnableLibraryCompression` preference is set to `true`, the new library is compressed on creation.

6. Click **OK**.

Editing Library Definitions

The two ways you can edit `lib.defs` files are:

- Using the Library Definitions Editor
- Using a Text Editor

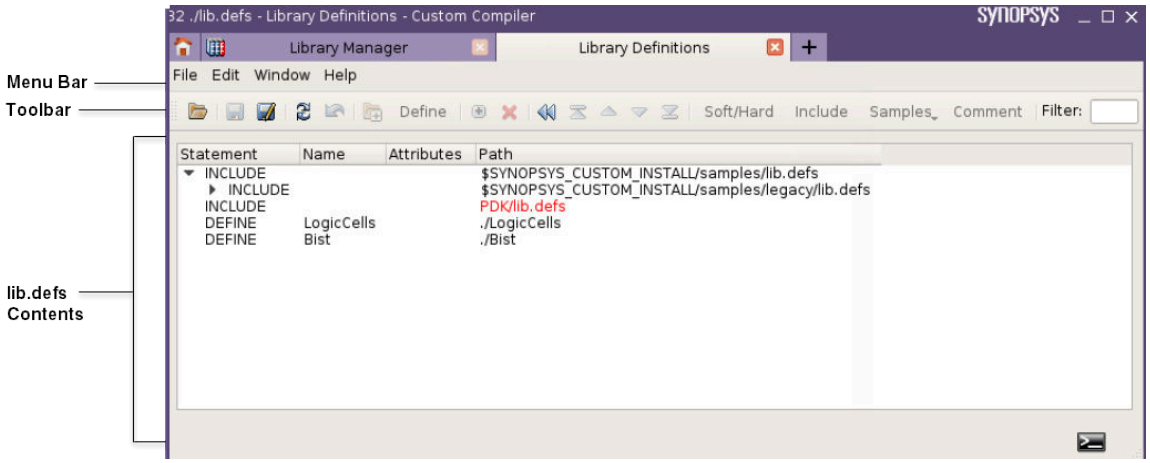
Using the Library Definitions Editor

Instead of using a text editor to modify library definition files (`lib.defs`), you can use the Library Definition Editor to view and edit the contents of the files while in the Library Manager. For example, you might want to specify the library availability to offer appropriate libraries for design sessions. For more information, see Controlling Library Availability.

To edit library definitions:

1. In the Library Manager, choose **File** > **Library Definitions Editor**. The **Library Definitions** tab appears.

   Notice that the editor consists of a menu bar, a toolbar, and a pane that displays the contents of the selected `lib.defs` file. By default, the `lib.defs` file in the current directory is displayed.



Feedback

2. To view a different `lib.defs` file, choose **File** > **Open Definitions File**, locate the file, and then click **Open**. The contents of the selected file display.
3. Edit the contents:

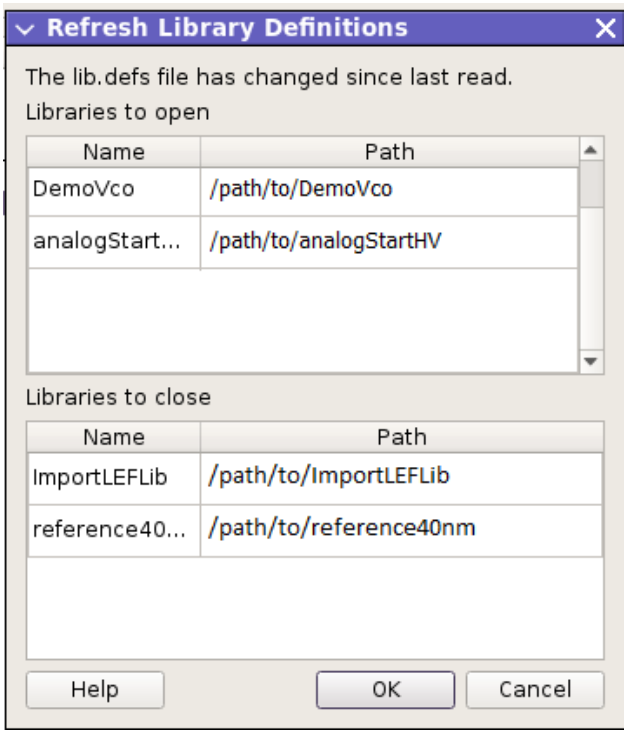
If you want to...	Then...
Add a new line	Select the line below which you want to add the new line, and then click **Add New Row** on the toolbar. An empty line is added.
Delete a line	Select the line, and then click Delete Current Row.
Edit an existing line	Click the line you want to edit, and then do the following: ◦ Type a statement on the line. For more information about the statements you can add, see Controlling Library Availability. ◦ Click **Define** on the toolbar to select and define a library. ◦ Click **Soft/Hard** to specify whether the inclusion of the library definition is optional (soft include) or required (hard include). ◦ Click **Include** on the toolbar to select and include a library definition. If the file does not yet exist in the directory you specified, the line appears in red text. ◦ Click **Samples** to include a Synopsys sample definitions file with the following options: **Legacy** or **<Major version>**. If there is no existing lib.def, by default, new lib.defs adds INCLUDE for <major version> automatically. ◦ Click **Comment** to convert a line into a comment. Click **Comment** again to uncomment the line.
Undo a change you made	Click Undo last edit.
Move an existing line up or down	Select the line, and then click one of the following: ◦ : Moves the line up a row. ◦ : Moves the line down a row.

4. If you want to edit an included library definition, double-click the file name in the DEFINE or INCLUDE statement. The Layout Editor opens a new **Library Definitions** tab and displays the contents of the selected file.
5. If you want to revert to the previously saved `lib.defs` content, click **Refresh from Disk** .

Feedback

6. When you are finished, click **Save** to save the changes.

   If the `lib.defs` file content has changed, the **Refresh Library Definitions** dialog box opens and shows the list of libraries to open and close as a result of changes made to the `lib.defs` file (including dependencies) since the last time it was read. Click **OK** to proceed, or **Cancel** to ignore the changes.



   If the `lib.defs` file content has changed, the **Refresh Library Definitions** dialog box opens and shows the list of libraries to open and close as a result of changes made to the `lib.defs` file (including dependencies) since the last time it was read. Click **OK** to proceed, or **Cancel** to ignore the changes.

Using a Text Editor

To edit the `lib.defs` file directly using a text editor:

1. Locate the `lib.defs` file.
2. Edit and save the file in your text editor. For information on statements you can use in the file, see Controlling Library Availability.
3. In the Library Manager, choose **File > Refresh** or press F5.

Feedback

The **Refresh Library Definitions** dialog box opens and shows the list of libraries to open and close as a result of changes made to the `lib.defs` file (including dependencies) since the last time it was read. Click **OK** to proceed, or **Cancel** to ignore the changes.

Setting the Path to Display

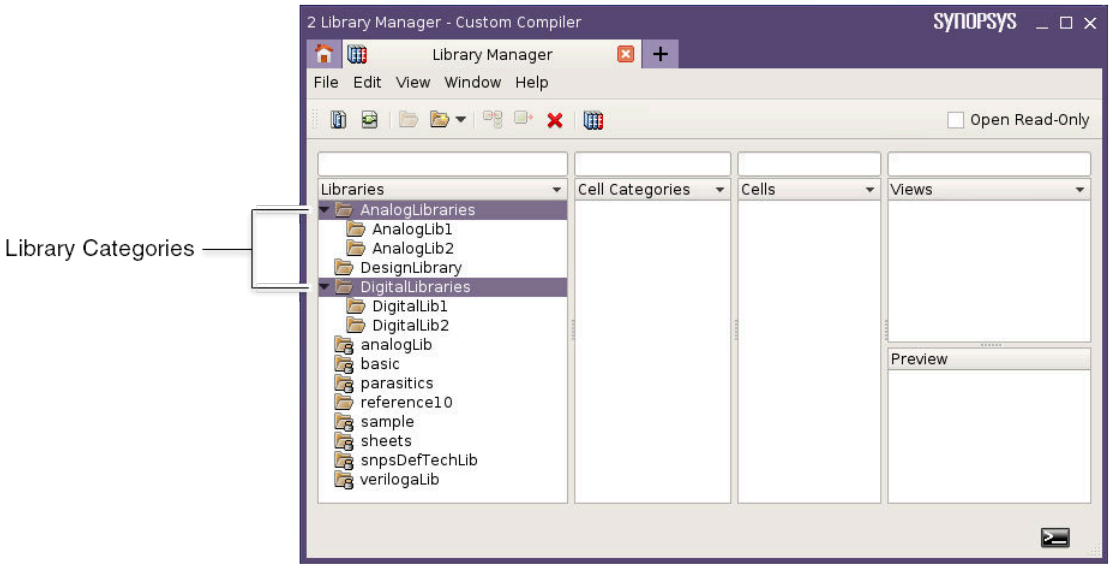
In the Library Definition editor, you can show either the absolute path or relative path to the `lib.defs` location. Use the **Use Absolute Paths** option in the **Edit** menu of the Library Definition window.

When the **Use Absolute Paths** option is selected, the absolute path to the `lib.defs` location is displayed. When the **Use Absolute Paths** option is unselected (default), the relative path to the `lib.defs` location is displayed.

You can also set the path using the `dmLibDefEditorUseAbsolutePaths` preference. If `true`, displays absolute path. If `false`, displays relative path. Default: `false`.

Creating Library Categories

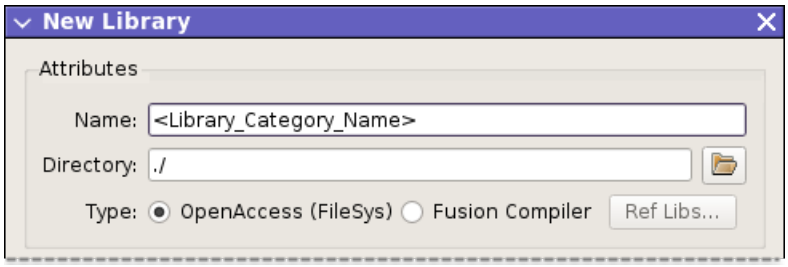
If you have large collections of libraries, you can create categories to better organize and manage the collections in the Library Manager. For example, for co-design flows, having categories for analog libraries and digital libraries allows you to quickly find and select designs of interest.



Feedback

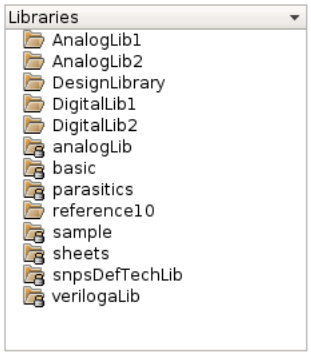
To define categories for existing design libraries:

1. In the Library Manager, choose **File** > **New** > **Library** to create a new library category.
2. In the **New Library** dialog box, type the name of the library category, and click **Apply**. (You do not need to set the remaining options in the dialog box).

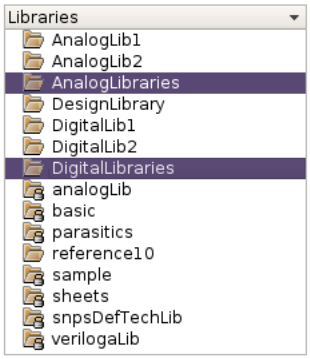


The new library category you created appears in the Library Manager at the same level as the existing libraries.

Before

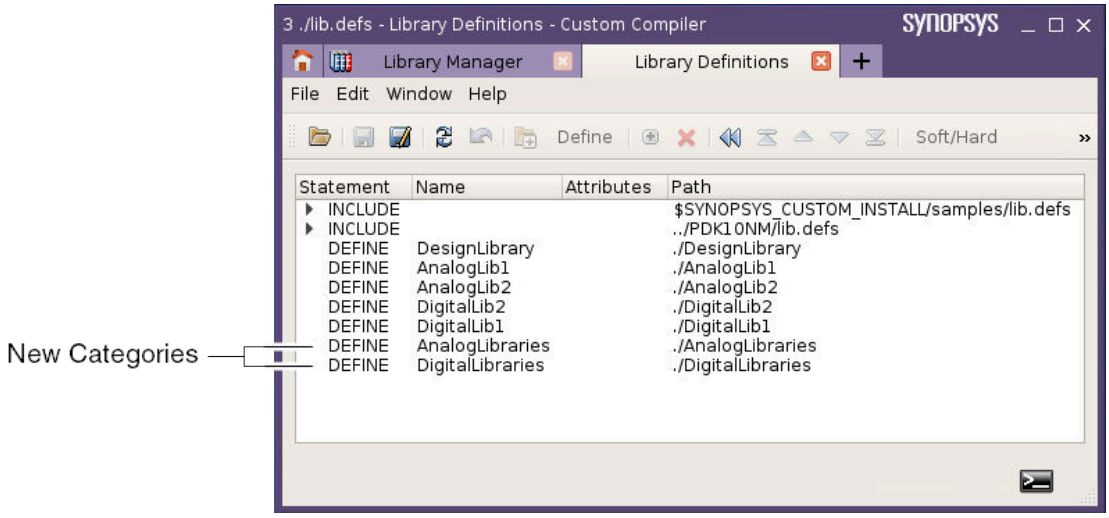


After



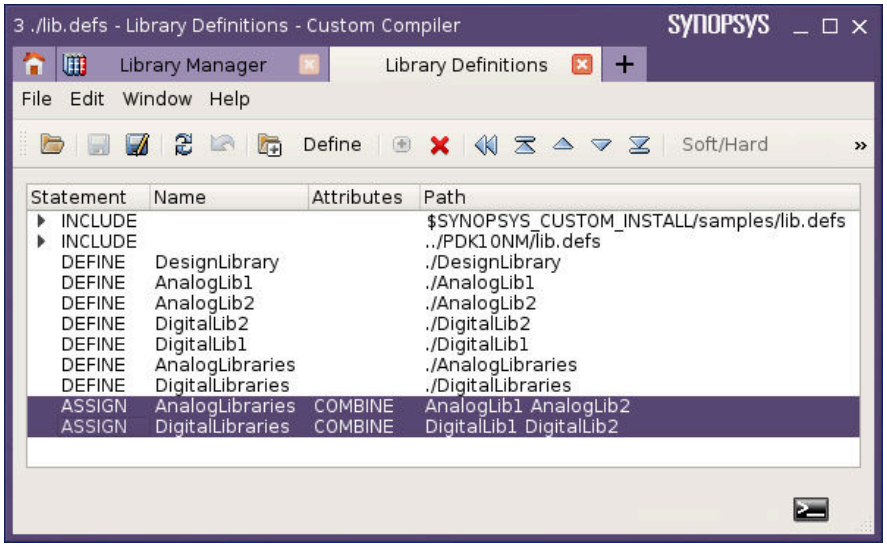
3. Repeat Step 2 to create additional categories.
4. (If you will edit the lib.defs file in a text editor in Step 5) When you are finished, quit the Custom Compiler tool.
5. Open the lib.defs file using either the Library Definitions Editor or a text editor. See Editing Library Definitions. Confirm that the newly created library categories appear in the file.

Feedback



6. Add `ASSIGN` statements with the `COMBINE` attribute to group libraries within categories. See Controlling Library Availability for details about the `ASSIGN` statement and `COMBINE` attribute.

   In the following example, an `ASSIGN` statement combines two analog libraries in the AnalogLibraries category, and another `ASSIGN` statement combines two digital libraries in the DigitalLibraries category.

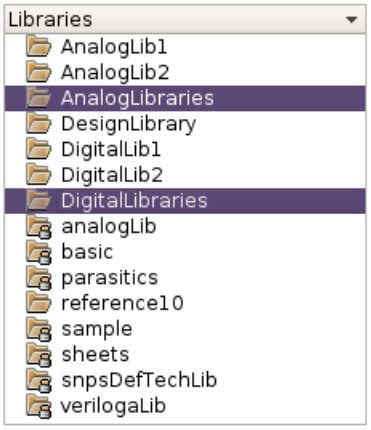


7. (If you edited the lib.defs file in a text editor in) When you are finished, save the lib.defs file and restart the Custom Compiler tool.

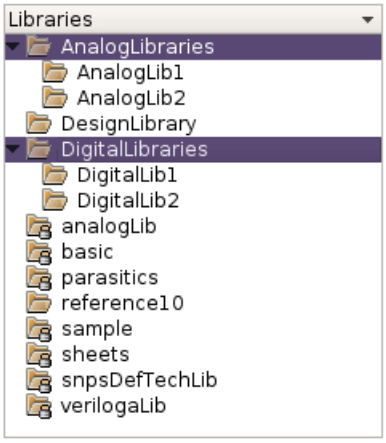
Feedback

8. In the Library Manager, confirm that the libraries are grouped under the correct categories.

Before



After



Controlling Library Availability

You can control the libraries that are availability for design sessions. You can specify availability for libraries created using the Custom Compiler environment, and you can adjust availability for libraries whose data was imported.

To control library availability, use library definition files (`lib.defs`).

This section describes the following for controlling availability for reference libraries used by integrated tools.

- Specifying Library Availability
- Combining Libraries
- Hiding Libraries
- Loading Libraries Automatically

Specifying Library Availability

Specify general library availability by editing the library definition file. You can use this file to add, change, and remove library availability.

You can use the library definition file to directly define paths to libraries. Additionally, you can also use it to indirectly define paths by including references to additional library definition files.

Feedback

The default version of this file is named `lib.defs`. However, different names are allowed. (To specify a library definition file with a different name at session start, use the `-libDefsFile` command line option, and the filePath to the library location).

To edit a library definition file:

1. Display the Library Definitions Editor. See Editing Library Definitions.
2. Edit the file using valid definition statements in Table 101. For editing instructions, see Editing Library Definitions.

Table 101 Library Definition File Statements

Statement	Description
`ASSIGN <library_name> <attribute_name> <attribute_value>`	Assigns one of the following attributes such as `libMode` or `dbOverlayPath` to a listed library. For example, create an overlay CDF (Component Description Format) using a statement such as: `ASSIGN libName dbOverlayPath overlay/path`
`DEFINE <library_logical_name_or_label> <path_to_the_library>`	Defines a library, and its library path. The library's logical name is in the Linux name space, therefore, it might appear differently in the Cdba name space that the Custom Compiler environment uses to display the name. The path is not name-mapped, and appears just as it would in a Linux shell command.
`INCLUDE <path_to_library_definition_ file>`	Includes in this library definition file the libraries listed in another library definition file. Add this statement for each library definition file whose library definitions you want to include. When this statement is used, and the file is not found, the system generates an error.
`SOFTINCLUDE <path_to_the_included_file>`	Adds other library definitions. When this statement is used, and the file is not found, the system ignores it.
`UNASSIGN <libraryName> <attribute_name>`	Removes this attribute.
`UNDEFINE <libraryName>`	Removes a previously defined library.

Note:

The `SOFTINCLUDE`, `UNASSIGN`, and `UNDEFINE` statements are Synopsys extensions to the OpenAccess standard library definition syntax. Files

Feedback

that use these statements are not guaranteed to work with non-Synopsys OpenAccess based tools when they are run outside of this environment. When translators and integrated tools run from within the Custom Compiler environment, the system uses a temporary copy of the library definition file that replaces these statements.

When library names include characters other than letters and numbers, note that these can appear differently in the definition file and the Library Manager. This is due to name mapping, which enables these names to be recognized by multiple environment features. When this occurs, use the Library Manager to both create the library and automatically add its access instead of adding access manually.

3. Save the file.

Combining Libraries

To combine the contents of two or more libraries, use the COMBINE attribute with the ASSIGN command. In the following example, the contents of libA and libB are listed under libC in the Library Manager. The instances still belong to their original libraries.

```
ASSIGN libC COMBINE libA libB
```

The following limitations apply to the COMBINE attribute:

- You cannot use the attribute with Fusion Compiler or IC Compiler II libraries.
- If a cell exists in more than one of the libraries, the cell listed in the combined library represents only the first occurrence of this cell.
- The **Copy > Cell** and **Move > Cell** commands do not permit groups of cells in different physical libraries.
- A cell that is copied into a combined library will be placed in the top-level library (`libC` in the example shown).
- You cannot copy or move cells in libraries that are members of the combined library.

Hiding Libraries

After you combine libraries, you might want to hide the libraries so that the cells do not appear in more than one place in the Library Manager.

To hide libraries such that they do not appear in the Library Manager, use the INVISIBLE attribute with the ASSIGN command. The following example hides `mylib` so that it does not appear in the Library Manager:

```
ASSIGN mylib INVISIBLE
```

Feedback

Loading Libraries Automatically

Libraries can have an autoload file to automatically load the libraries at startup. You can also disable the autoload file before sourcing a particular library by assigning the `loadAutoLoadFile` attribute to `false`. To disable loading of the autoload file for a library defined in the `lib.defs` file:

```
ASSIGN <libName> loadAutoLoadFile false
```

Note:

The attribute only affects one library and not its hierarchy. If you need to disable autoLoad.file for its technology library too, then each library must have a separate `ASSIGN` statement.

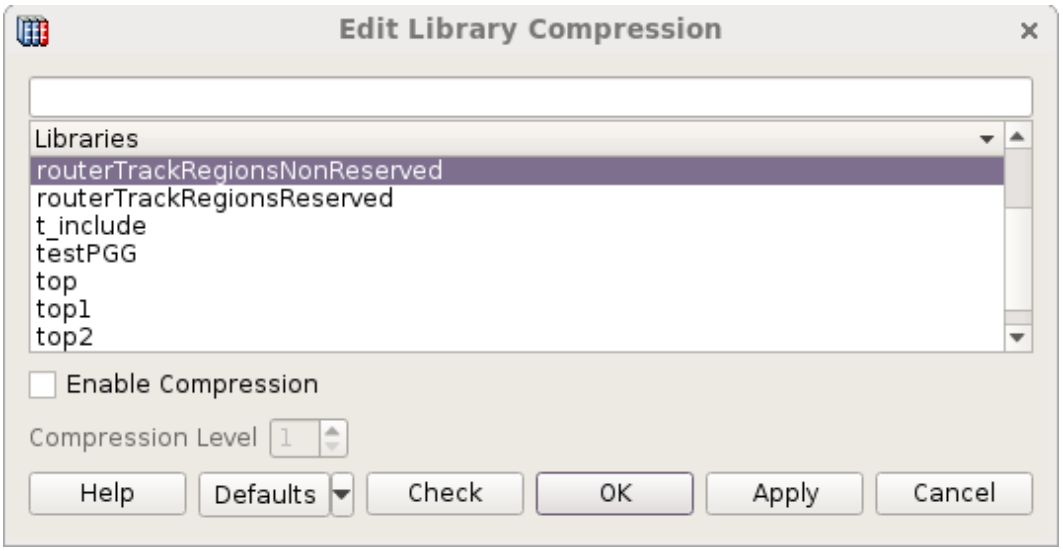
Compressing and Uncompressing Libraries

The Library Manager tool uses the oaZip method to compress libraries. This type of compressing the libraries allows you to save the disk space and to view or edit designs in the compressed library.

Note:

Supports compressing of OpenAccess filesys (oaDMFileSys) and OpenAccess turbo (oaDMTurbo) libraries only and does not support compressing of NDM and IC Compiler libraries.

The **Edit Library Compression** dialog box allows you to compress, uncompress, or check the current compression state of a library.



Feedback

To compress or uncompress the library:

► In the **Edit Library Compression** dialog box,

- Select the **Enable Compression** check box and click either **OK** or **Apply** to compress the selected library.
- Do not select the **Enable Compression** check box and click either **OK** or **Apply** to uncompress the selected library.
- Click the **Check** button to check the current compression state of the selected library. The **Check** button is independent from the **Enable Compression** check box selection.

Defining Compression Level

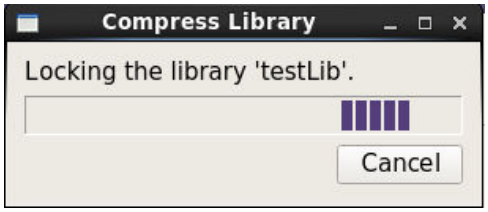
You can define the compression level to compress the library with the `dmLibraryCompressionLevel` preference. The default is `1`. Valid values are 1-9.

You can compress the library using the Edit Library Compression action under the Library CSM and Edit menu. The action is enabled only if a library is selected.

Preparing Libraries for Compressing

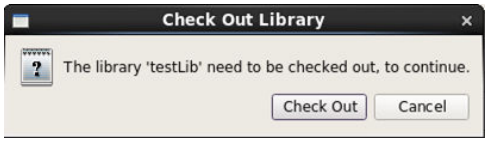
While preparing the library for compressing, various dialog boxes might appear. This helps you to take action, if needed, when prompted. Listing a few checks that occur while preparing for compressing, and you might not see all of them based on your environment:

- Designs in read mode are closed without notification.
- Checking for the locked libraries using the library lock.



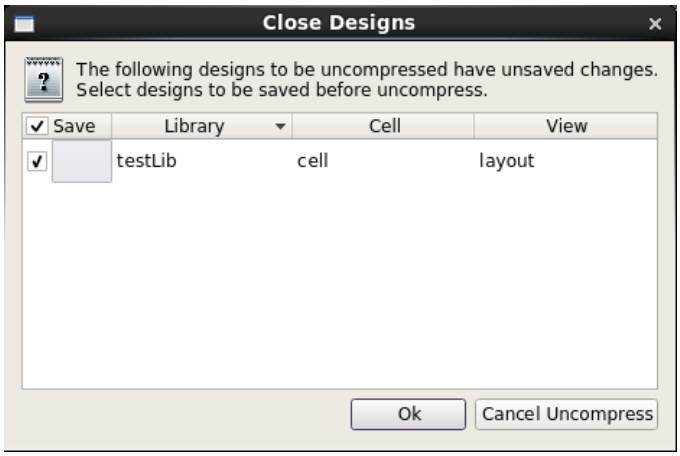
  If the library is locked, the **Compress Library** dialog box appears for 5 seconds. Then, interrupts the compression process if the library is not unlocked in the other session.
- Checking data.dm file for the lock state.
- Checking for locks in all cellviews under the library.
- Checking the file permissions for data.dm and cellviews.

  If the library is managed by a version control system and has checked in data.dm or any cellviews, it performs necessary checks. If anything should be checked out for compressing the library, the **Check Out Library** dialog box appears.



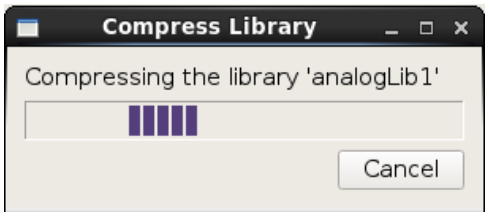
Click the **Check Out** button to check out the data.dm file of the library, which also checks out all the checked in cellviews in the library. If check out process fails, the library is not compressed. If you click **Cancel**, the compression operation is not performed.

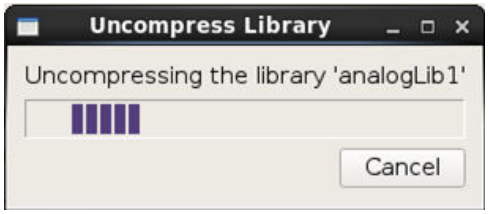
- Checking for open designs and prompts to save and close.



If the library contains open designs, the **Close Designs** dialog box appears. If you click **Cancel Uncompress**, compressing of the library is interrupted.

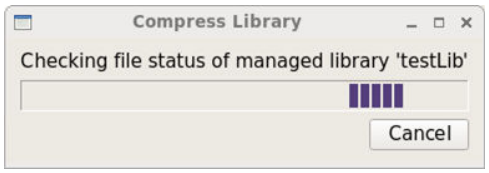
- Compressing and uncompressing the library. The following dialog boxes appear to show the progress.

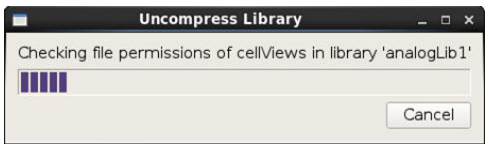




- Checking file status and permission of cell views

Feedback





Modifying Component Description Format (CDF) Overlays

You can use the library definition file (`lib.def` file) to modify PDK CDF definitions and netlisting behavior for a library without changing the original or base PDK. This method might be useful when the library is read-only (for example, foundry supplied PDKs) or if you do not want to directly modify the library.

To modify CDF overlays, add ASSIGN statements with the following attribute or keyword in the `lib.def` file:

ASSIGN Attribute or Keyword	Description
dbOverlayPath	Attribute that specifies one or more CDF overlays. When specifying more than one CDF overlay: • Add quotation marks to enclose the overlay names. • Use space delimiters. The modifications are applied based on the order in which the overlay names are specified. The last overlay listed has the highest priority and is the default. Parameter reordering is based on the overlay with the highest priority. If that overlay does not contain all the parameters, the parameter reorder information in the base overlay is used.
dbOverlayPath:append	Keyword for appending to existing overlays.

The Custom Compiler tool automatically synchronizes the CDF overlay modifications with the Parameter Definition Editor and saves the changes to the overlay library.

The library does not need to exist. If the specified path does not exist, the library is automatically created.

Example of modifying a single CDF overlay:

```
ASSIGN foundryPDK dbOverlayPath ./path/to/foundryPDKOverlay
```

Feedback

Example of modifying a multiple CDF overlays:

```
ASSIGN myLib dbOverlayPath "myLibOverlay3 myLibOverlay2 myLibOverlay1"
```

In the multiple CDF overlay example, `myLibOverlay1` has the highest priority, and `myLibOverlay3` has the lowest priority.

For information on how to view multiple overlays and edit the parameters in Parameter Definition Editor, see Viewing CDF Overlay Parameters and Editing CDF Overlay Parameters.

Managing Individual Libraries

You can manage individual libraries by copying, moving, and deleting them. After making changes, update library name references to allow recognition of changes from referenced libraries.

This topic describes the following:

- Copying Libraries
- Moving Libraries
- Deleting Libraries
- Updating Library Name References
- Updating Library References

Copying Libraries

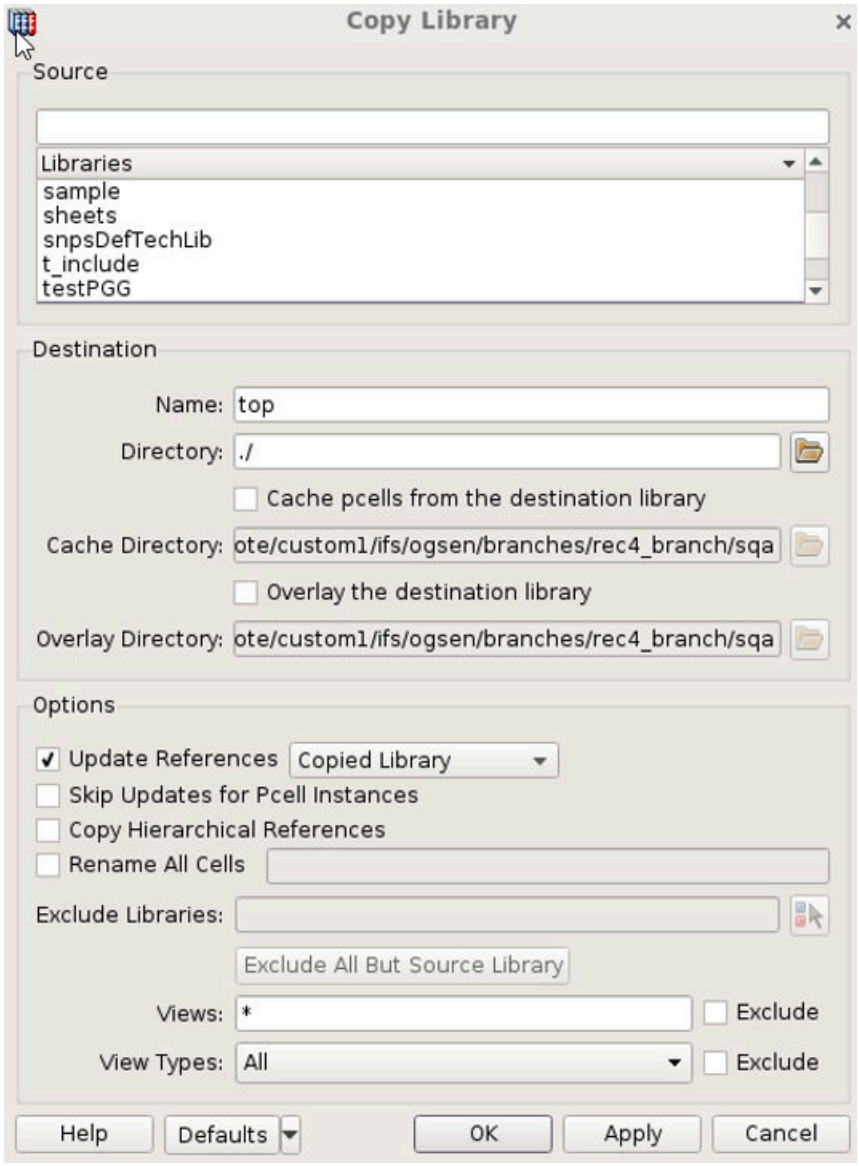
You can copy a library to create a new library with identical contents.

To copy a library:

1. In the Library Manager, specify the library to copy using one of these methods:
   - From the Libraries list, select the library name, right-click, and choose **Copy**.
   - From the **Edit** menu, choose **Copy > Library**, and filter the list to select the library.
   - Make sure the library name is selected in the Libraries list. From the **Toolbar**, click **Copy** .

   The **Copy Library** dialog box opens.

Feedback



2. In the **Source** section, choose the source library to copy. Filter the list to specify the source library or directly select the library name.
3. In the **Destination** section:
   a. Type the **Name** to assign to the copied library.
   b. Specify the **Directory** of the copied library. Accept the default, or browse to select a directory.

Feedback

c. Select **Cache pcells from the destination library** to save evaluated PCells in a cache directory for use in later sessions. If you choose this option, browse to select the **Cache Directory** location.

d. Select **Overlay the destination library** to specify that an overlay directory can be used to overlay CDF parameters for cells in the destination library. If you choose this option, accept the default, or browse to select an **Overlay Directory**.

4. In the **Options** section:

   - Select **Update References** to update the designs in the copied library so that all design references to copied cellviews point to the new location. You can specify the scope of the updates: **Copied Library** or **All Writable Libraries**. See Updating Library Name References to modify external references. See also Updating Library References.

   - Select **Skip Updates for Pcell Instances** to skip PCell evaluation after the update process.

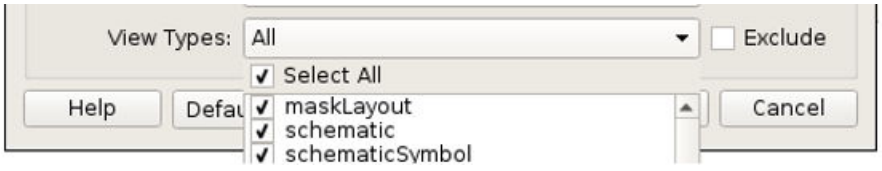
     Because PCell evaluation is a time-consuming operation, you can use this option to skip it, especially for hierarchical copy operations. If you skip the evaluation, the PCells might not be displayed correctly when you first open the designs.

   - Select **Copy Hierarchical References** to copy the hierarchy underneath each cell in the library by descending the hierarchy, and copying the master of each instance.

     To control where the descent stops, enter a list of library names in the **Exclude Libraries** text box. (Descent stops at a reference to any cellview within any of these libraries). To facilitate data entry, click **Exclude All But Source Library**, and keep or the list that populates the text box.

     To copy only certain views, enter them in the **Views** text box. To exclude the specified view, select **Exclude**.

     To copy cellview based on the view type name, select the cellview view type name from the **View Types** list. To exclude the specified cell view type name, select **Exclude**. The default is **Select All**. The view type filtering applies to the copied library and to the cellview references from the copied hierarchy.



   - Select **Rename All Cells** to use new names for the copied cells in the library. Specify the new name for the cells in the box.

Feedback

You can use name patterns to rename the cells in the library. The name patterns you can use include %L (original library name), %C (original cell name), %N (a unique number in the system), and %% (the % character).

For example, `%C_a` uses the original cell name (`%C`) and adds the `_a` suffix.

5. Click **OK**.

If the source library contains a technology database, the copy also receives it. If the source library points to an external technology library, the copy points to the same technology library, and the system generates this notice: `<library name> references an external technology file that exists in <library name>`.

If conflicts occur, the **Cell Conflicts** dialog box opens and lets you decide how to resolve the conflicts.

See Also

- Managing Duplicate Name Conflicts
- Copying Library, Cell, and Cellview Names
- Copying Library, Cell, and Cellview Paths

Moving Libraries

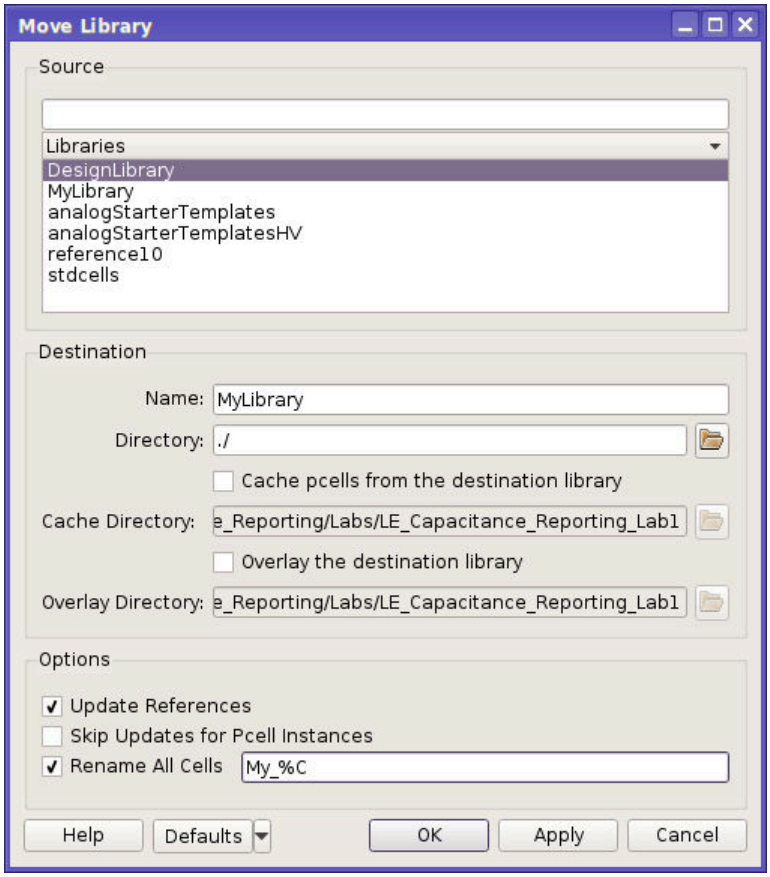
You can move a library to a different location to provide a different point of access to its contents.

To move a library:

1. In the Library Manager, specify the library to move using one of these methods:
   - From the Libraries list, select the library name, right-click the mouse, and choose **Move**.
   - From the **Edit** menu, choose **Move > Library**, and filter the list to select the library.
   - Make sure the library name is selected in the **Libraries** list. From the **Toolbar**, click the **Move** button.

Feedback

2. In the **Move Library** dialog box that opens, specify the following settings:



Settings	Description
Source	The source library to move. Filter the list to specify the source library or directly select the library name.
Destination	**Name** The name for the moved library. This is a required field. To use the original library name, type %L. You can add prefixes and suffixes to the original library name. For example, to add a suffix to the original library name, type *%L_<suffix>*. **Directory** The directory to contain the moved library. Accept or browse to select a directory. **Cache pcells from the destination library** saves evaluated PCells in a cache directory for use in later sessions. If you choose this option, browse to select the **Cache Directory** location. This option is not available for Fusion Compiler or IC Compiler libraries.

Feedback

Settings	Description
	Overlay the destination library overlays the destination library with the contents of the source library. If you choose this option, accept or browse to select an **Overlay Directory**.
Options	◦ **Update References**: Updates the designs in the moved library so that all design references to moved cellViews point to the new location. Only the references within the moved library are updated. External references from other libraries are not modified. See Updating Library Name References to modify external references. See also Updating Library References. ◦ **Skip Updates for PCell Instances**: Does not update PCell instances during the move process. ◦ **Rename All Cells**: Renames the moved cells to the specified name. You can add prefixes and suffixes to the original cell names. For example, to add a prefix to the original cell name, type *<prefix>_%C*. **Copy Hierarchical References** copies the hierarchy underneath each cell in the library by descending the hierarchy and copying the master of each instance. To control where descent stops, enter a list of library names in the **Exclude Libraries** text box. (Descent will stop at a reference to any cellView within any of these libraries). To copy only certain views, enter them in the **Include Views** text box.

3. Click **OK**.

   If you have open designs with unsaved modifications in the source library, the system prompts you to save those designs. After you save the designs, the system closes any designs that remain open in the source library.

Deleting Libraries

You can delete a library to completely remove it from the disk. The operation also removes the library listing from the library definition file.

To delete Fusion Compiler libraries, see Deleting Fusion Compiler Libraries. To delete IC Compiler II libraries, see Deleting IC Compiler II Libraries.

Caution:

You cannot undo this operation.

To delete a library:

1. In the Library Manager, from the Libraries list, select the library.
2. Delete the selected library using one of these methods.

Feedback

- Right-click the mouse, and choose **Delete**.
- From the **Edit** menu, choose **Delete** .
- Make sure the library name is selected in the Libraries list. From the **Toolbar**, click **Delete** .

3. Click **Delete** to complete the operation.

Updating Library Name References

You can update library name references to change all the references from a library to name a different library. This is useful after you replace or rename a library.

Copying or moving libraries updates internal references for designs. However, the system does not update external references. External references are those that are in other libraries. This operation updates these external references.

To update library name references:

1. In the Library Manager, choose **Edit > Update Library Name**.
2. Enter the name of the old library whose references should be replaced.
3. From the list, choose the new library to supply new references.
4. Select any additional libraries to search to find additional references to change.
5. Click **Update**.

This update process updates instances in OpenAccess designs whose master has the old library name, and also updates custom vias in a technology database that is local to a searched library. Where the master of the custom via uses the old library name, it also updates the reference to use the technology database in the old library.

Updating Library References

After you rename, replace, or move a library, you can update references to the library so that associations remain intact.

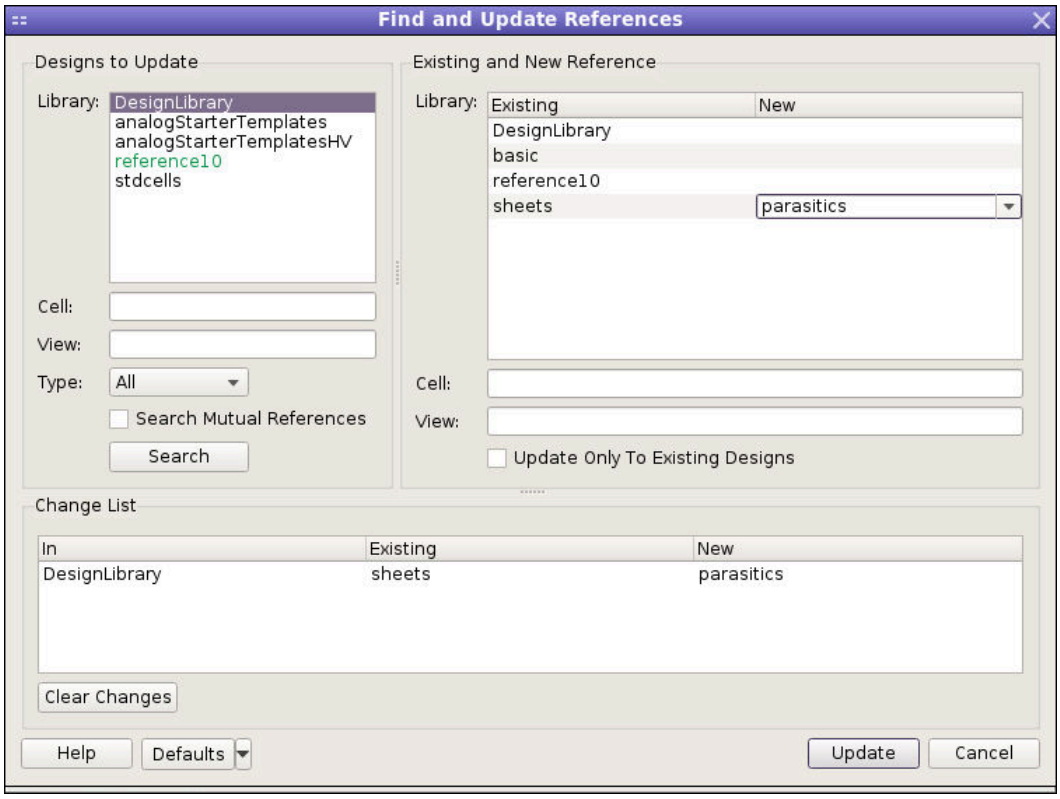
Use one of the following methods to update the library reference:

- Select the **Update References** option when you are in the **Copy Library** or **Move Library** dialog box. For information, see Copying Libraries and Moving Libraries.
- Choose **Edit** > **Update References** command in the Library Manager. The command lets you update references without having to copy or move the library. You can also set up multiple reference library updates, and then run the multiple updates simultaneously.

Feedback

To update the library reference using the **Edit** > **Update References** command:

1. In the Library Manager, choose **Edit** > **Update References**. The **Find and Update References** dialog box opens.

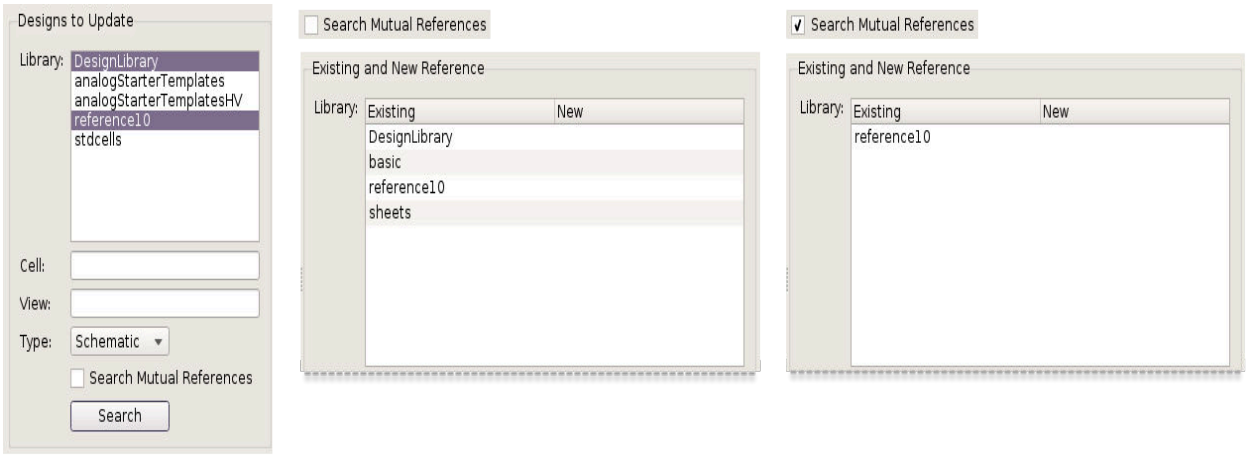


2. In the **Designs to Update** section:

   a. Choose one or more libraries from the **Library** list. You can further filter the set of designs by specifying the **Cell**, **View**, and view **Type**.

   b. Select **Search Mutual References** if you want to include only reference libraries that are common among the selected designs.

      Tcl preference: `dmSearchMutualReferences`



c. Click **Search**.

3. In the **Existing and New Reference** section, choose a **New** reference library to replace the **Existing** reference library.

   As soon as you choose a new reference library, the change appears in the **Change List** section. This allows you to review the changes you set up before you update the reference. Any new change you make to the same design library reference replaces the existing change in the **Change List**.

   You can further filter the references to be updated by specifying the **Cell** and **View**.

4. Select **Update only to existing designs** to update the library reference only if the same names exist in the new library. If the names do not exist in the new library, the cells will retain their original references.

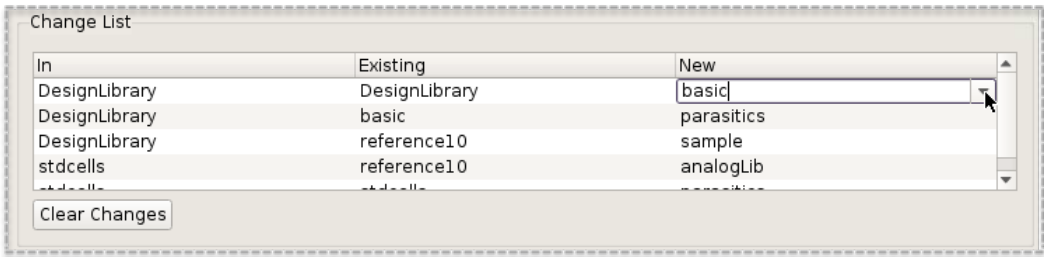
   Tcl preference: `dmUpdateOnlyToExisting`

5. Repeat steps 3 through 4 to specify additional library reference changes.

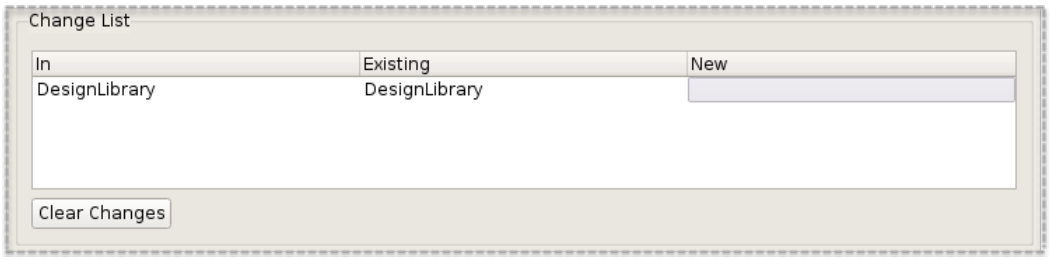
   For each design library, you can set up multiple reference library changes, and then update all of them simultaneously.

6. In the **Change List** section, review all the changes you have set up so far.

   If necessary, you can correct updated references. To do this, in the **New** column, select the new reference library. The **Update** command will apply the newly assigned reference instead.



To skip a change, select the blank line in the **New** column.



To start over, click **Clear Changes** to remove all changes from the **Change List** and **Existing and New Reference** sections. The original search results are retained.

7. Repeat steps 2 through 6 to change the references of other design libraries.

   All the changes you set up are displayed in the Change List section. You can review them, and then update them all simultaneously.

8. Click **Update**. The update operation starts.

   If there are open designs, the **Save Designs** dialog box appears. Like other Library Manager operations (copy and move), related designs that are open in the read mode are closed implicitly before the update operation starts.

9. When you are finished, click **Cancel** to return to the Library Manager.

Managing Cells

You can manage cells by creating, copying, moving, and deleting them.

This topic describes the following:

- Creating New Cells
- Creating New and Reusing Cell Hierarchies
- Copying a Hierarchy of Cells
- Moving Cells

Feedback

- Deleting Cells
- Enabling Rotation and Scaling

Creating New Cells

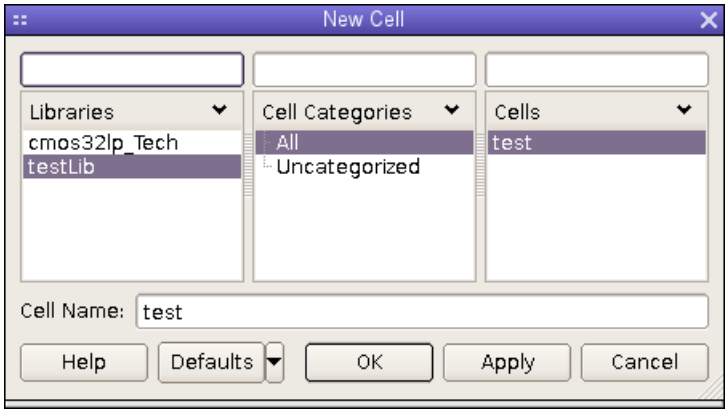
You can create a cell in a library to define a logically separate component or device in a design. Creating a cell creates a cell container within the given library. A single cell describes an individual building block of a chip or system.

Use the Library Manager to create cells.

To create a cell:

1. In the Library Manager, start to create the cell using one of these methods.
   - From any location in the Cells list, right-click the mouse, and choose **New**.
   - From the **File** menu, choose **New > Cell**.
2. In the **New Cell** dialog box that opens:
   - Confirm that the selected library is the correct target library to hold this cell, or select a different library.
   - Select an existing cell on which to base the new cell.
   - Specify a cell name by editing the name shown or by entering a new name.

     If you specify the exact name of an existing cell, this operation deletes the contents of the existing cell, and creates an empty cell. Additionally, if there are any open designs in that cell, the tool closes them.



3. (Optional) To add the new cell to a category as you create it, choose from the **Cell Categories** list.

Feedback

4. When you are finished, click **Apply** to create the new cell in the destination library.
5. Click **OK** to close the dialog box.

Creating New and Reusing Cell Hierarchies

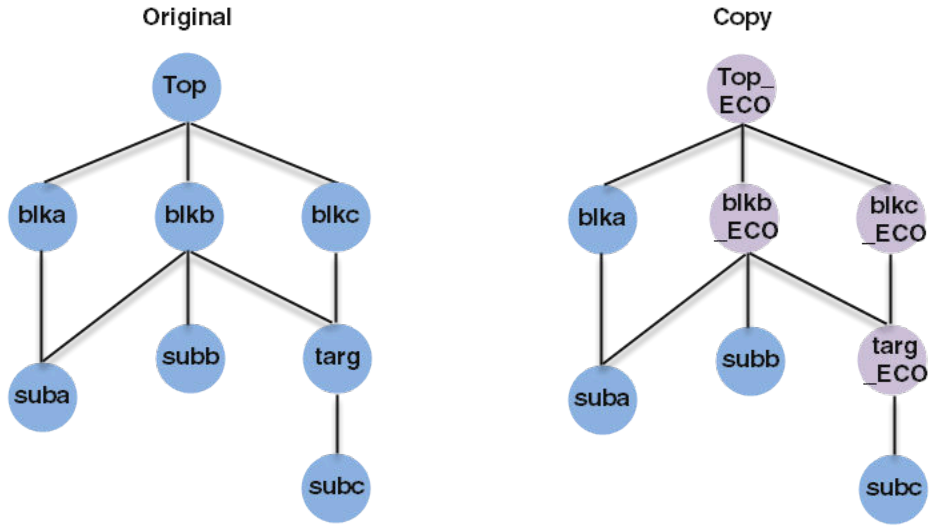
You can copy an existing hierarchy of cells, and rename the copy to create a new hierarchy of cells.

You can use the **Edit** > **Copy** > **Unique** command to:

- Create a copy of a target cell (or leaf cell) and all cells that reference it.
- Rename the copied leaf cell and all cells that reference it so that they can be modified without affecting the originals.
- Retain references to all other cells in the original hierarchy, thus reducing the number of duplication cells in the system.

In the following example, the target cell and the top cell are identified for the **Copy** > **Unique** operation. During the copy operation, the Top cell, target cell, and all cells that reference the target cell are copied and renamed so that they can be modified without affecting the originals. References to all other cells in the original hierarchy are retained.

If you were to open the original Top cell, you will see the original target cell, and all the original cells that reference the target cell. If you were to open the Top_new cell, you will see targ_new, blkb_new, and blkc_new. You will also see blka, blkb, blkc, and the other cells that were not copied.

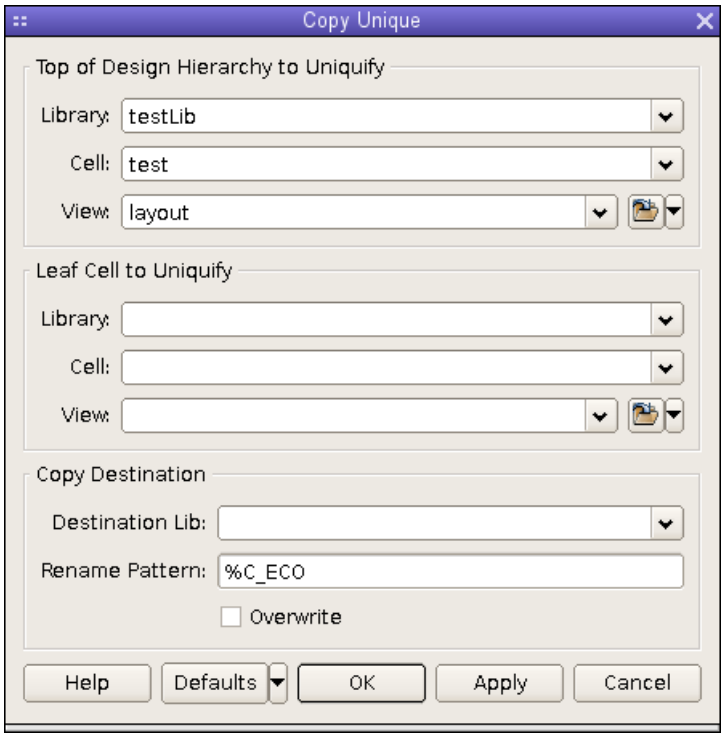


If you want to duplicate the entire cell hierarchy, use the **Edit** > **Copy** > **Cell** command. See Copying a Hierarchy of Cells.

Feedback

To create new and reuse cell hierarchies:

1. In the Library Manager, choose **Edit** > **Copy** > **Unique**. The **Copy Unique** dialog box opens.



2. In the **Top of the Design Hierarchy to Uniquify** area, specify the top-level cell you want to copy by choosing the **Library**, **Cell**, and **View**.
3. In the **Leaf Cell to Uniquify** area, specify the referenced cell in the hierarchy that you want to copy by choosing the **Library**, **Cell**, and **View** of the referenced cell.
4. In the **Copy Destination** area, set the following:

Setting	Description
Destination Lib	The target library that will contain all the renamed cells in the uniquified hierarchy.
Rename Pattern	The prefix or suffix to append to the existing cell name. %C is the existing cell name. To append a suffix to the existing cell's name, type `%C_<suffix>`. For example, suppose the existing cell name is MyCell. If you type `%C_copy1` in the **Rename Pattern** box, the top cell and all leaf cell names will have the _copy1 suffix.

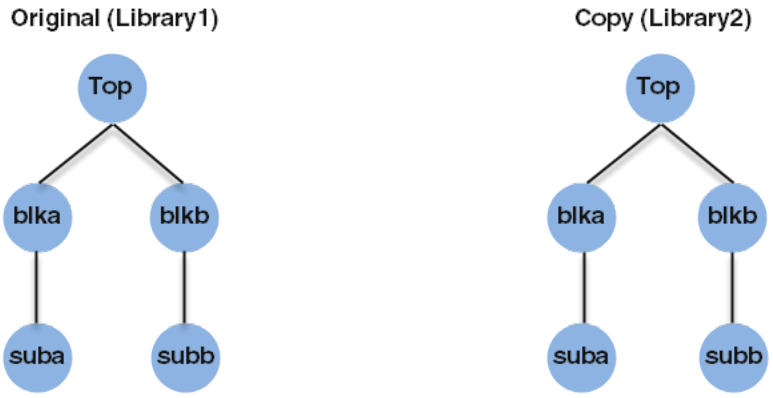
Feedback

Setting	Description
Overwrite	The option to overwrite existing cell copies if name conflicts are encountered during the copy operation. Clear the check box if you want to skip the copy operation when a conflict is encountered.

5. When you are finished, click **OK** to create the copy and close the **Copy Unique** dialog box.

Copying a Hierarchy of Cells

You can copy a cell and its hierarchy to create a duplicate version of it. You can copy cells from one library to another with or without changing their names. You use the Library Manager to copy cells.



This topic describes the following:

- Copying One or More Cells Without Changes
- Copying Individual Cells With Changes

See also Creating New and Reusing Cell Hierarchies.

Copying One or More Cells Without Changes

You can copy cells without changes to create duplicate versions in a different library.

The new versions retains their names, maintain their positions within any cell categories, and update their references. (To change these aspects of a cell while copying it, see instead Copying Individual Cells With Changes).

Feedback

To copy cells without changes:

1. In the Library Manager Cells list, press Ctrl, and use the mouse to select one or more cell names.
2. Drag the name or names toward the destination library.
3. At the destination, drop the names in place, and release Ctrl.

   If you do not use the Ctrl key, the system moves the cells instead of copying them.

   If the system detects cells with duplicate names in the destination library, the **Conflicts** dialog box opens. See Managing Duplicate Name Conflicts.

The Lib Name Field's Dependency from Source and Destination library Selection Change

When no direct edits have been made to the Destination frame's **Cell Name** field, since the dialog box was first shown, then the Cell Name field track changes to the selection in the source Cellslistview as follows:

- If the new selection is a single cell, then the selected cell name is entered into the **Cell Name** field.
- If the new selection has multiple cells then the **Cell Name** field is cleared, and disabled.
- After edits are made to the **Cell Name** field, this tracking stops.
- Tracking resumes, if the **Cell Name** field value is emptied, or matches to a cell name from source selector by selecting a cell from destination cells list, or by changing **Cell Name** filed value manually

  If the value in **Cell Name** field doesn't match to any cell in destination cells list, the selection in destination cells list is cleared.

  The **Lib Name** field is initially seeded from source library selector's selection. If the field is not empty a selection change of source library is not set. Any selection change in destination library selector is reflected on **Lib Name** field.

  The value of **Lib Name** field can be entered also manually. If the value in **Lib Name** field does not match to any library in destination libraries list (note that read-only libraries are not listed in the list,and they also should not be possible to use as destination), the selection in destination libraries list is cleared.

  The following prompt is to show if the destination library doesn't exist: "The destination library '%s' doesn't exist. Do you want to create it and continue?", where the **Create** button will create the library and continue copy process, and **Cancel** will cancel the operation, by returning the control to parent dialog box.

Feedback

Copying Individual Cells With Changes

You can copy an individual cell with changes to create an additional version of it while changing some cell aspects. As you create the new version, you can choose to change the cell name, its position in cell categories, and its references.

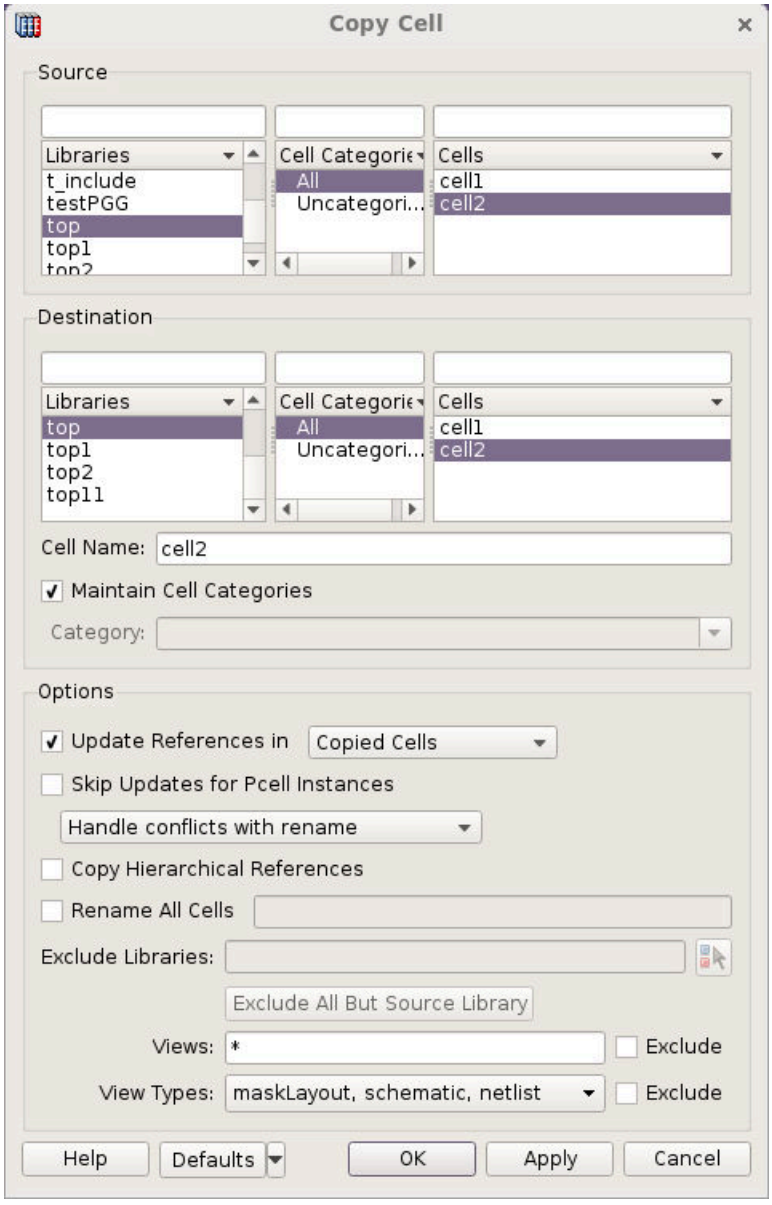
To copy an individual cell with changes:

1. In the Library Manager, specify the cell to copy using one of these methods:
   - From the Cells list, select the cell name, right-click the mouse, and choose **Copy**.
   - From the **Edit** menu, choose **Copy > Cell**, and filter the list to select it.
   - Make sure the cell name is selected in the Cells list. From the **Toolbar**, click **Copy** .

   The **Copy Cell** dialog box opens.

Feedback

2. In the **Copy Cell** dialog box that opens, specify the following settings:



Setting	Description
Source	The source cell to copy. Filter the list to specify the source cell, or directly select the cell name. To view cell categories during this procedure, choose **Show Cell Categories**.

Feedback

Setting	Description
Destination	**Cell Name** The name for the copied cell at its destination. Filter the list to select a base name or directly enter one. If the new selection has multiple cells then the **Cell Name** field is cleared, and disabled. **Maintain Cell Categories** inserts copied cells into corresponding categories at their destination, creating categories if necessary. Alternatively, you can place the cell in a new category by deselecting this option, and choosing a category from the list.
Options	**Update References** lets you choose to update references to only **Copied Cells**, all cells in the **Destination Library**, or all cells in **All Writable Libraries**. **Skip Update for PCell Instances** excludes PCells in the update process. You can choose how to resolve destination conflicts during the copy operation: ◦ **Handle conflicts with rename**: Opens the **Cell Conflicts** dialog box to let you rename the selected cells. ◦ **Handle conflicts by selecting best cell**: Opens the **Cell Selection** dialog box and lets you choose the cells to copy to the destination, and skip the cells if conflicts exist. **Copy Hierarchical References** copies the hierarchy underneath the cell by descending the hierarchy, and copying the master of each instance. To control where descent stops, enter a list of library names in the **Exclude Libraries** text box. (Descent stops at a reference to any cellView within any of these libraries). To copy only certain views, enter them in the **Include Views** text box. To copy cellview based on the view type name, select the cell view type name from the **View Types** list. To exclude the specified cell view type name, select **Exclude**. The default is **Select All**. The view type filtering applies to the copied cellviews and to the cellview references from the copied hierarchy. **Rename All Cells** lets you use new names for the copied cells in the library. Specify the new name for the cells in the box. You can use name patterns to rename the cells in the library. The name patterns you can use include %L (original library name), %C (original cell name), %N (a unique number in the system), and %<character> (one or more characters). For example, `%C_a` uses the original cell name (`%C`) and adds the `_a` suffix.

3. When you are finished, click **Apply**.

Feedback

If the system detects cells with duplicate names in the destination library, the **Conflicts** dialog box opens. See Managing Duplicate Name Conflicts.

4. Click **OK** to close the **Copy Cell** dialog box.

The destination library now contains the copied cells with changes.

Moving Cells

You can use the Library Manager to rename a cell or move it to a different library. You can move cells from one library to another with or without changing their names and other details.

This topic describes the following:

- Moving One or More Cells Without Changes
- Moving Individual Cells With Changes

Moving One or More Cells Without Changes

You can move one or more cells to relocate them while maintaining cell categories and references. The moved cells retain their names during the process.

To move one or more cells without changes:

► In the Library Manager, select the cell names, and drag them onto the destination library name in the **Libraries** list.

If the system detects cells with duplicate names in the destination library, the **Conflicts** dialog box opens. See Managing Duplicate Name Conflicts.

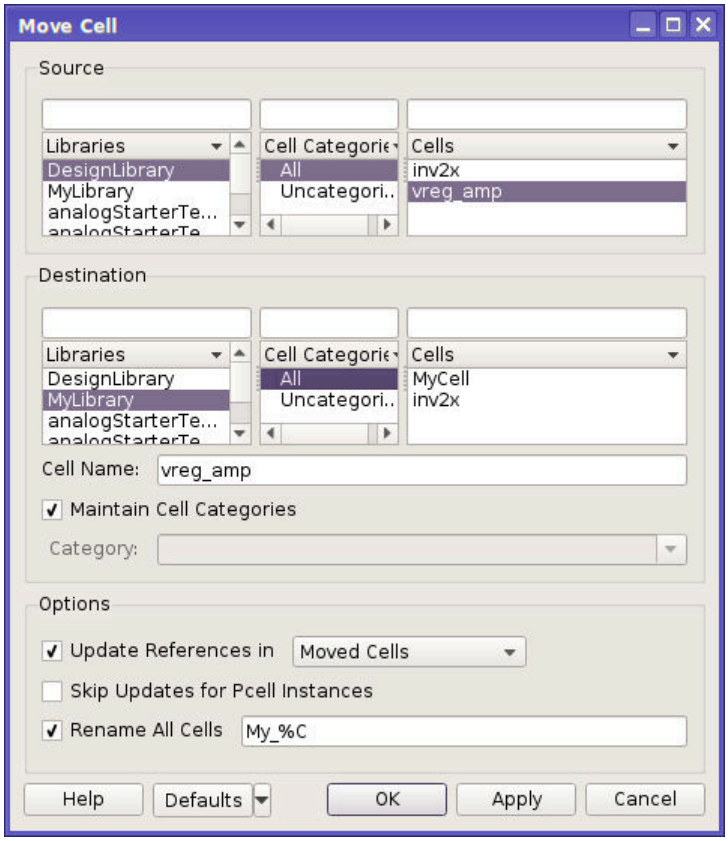
Moving Individual Cells With Changes

You can move individual cells to relocate them while adjusting category positions, and references. In the process, you can rename the cells.

Feedback

To move individual cells with changes:

1. In the Library Manager, specify the cell to move using one of the following methods:
   - From the Libraries list, select the cell name, right-click the mouse, and choose **Move**.
   - From the **Edit** menu, choose **Move > Cell**, and filter the list to select the cell.
   - Make sure the cell name is selected in the Cells list. From the **Toolbar**, click the **Move** button.
2. In the **Move Cell** dialog box that opens, specify the following settings:



Setting	Description
Source	The source cell to move. Filter the list to specify the source cell, or directly select the cell by name. To view cell categories during this procedure, choose **Show Cell Categories**.
Destination	**Cell Name** The name for the moved cell at its destination. Filter the list to select a base name or directly enter one.

Feedback

Setting	Description
	Maintain Cell Categories inserts moved cells into their original categories at their destination, creating categories if necessary. To place the moved cells in a new category, clear the check box, and choose a different **Category**.
Options	◦ **Update References**: Updates references to only **Moved Cells**, all cells in the **Destination Library**, or all cells in **All Writable Libraries**. ◦ **Skip Updates for PCell Instances**: Does not update PCell instances during the move process. ◦ **Rename All Cells**: Renames the moved cells to the specified name. You can add prefixes and suffixes to the original cell names. For example, to add a prefix to the original cell name, type *<prefix>_%C*.

3. When you are finished, click **Apply** to start the move operation.

   If the system detects cells with duplicate names in the destination library, the **Conflicts** dialog box opens. See Managing Duplicate Name Conflicts.

4. Click **OK** to close the **Move Cell** dialog box.

Deleting Cells

You can delete a cell to completely remove it from the disk.

Caution:

You cannot undo this operation. After deleting the selected cells, you cannot retrieve the cell contents.

To delete a cell:

► In the Library Manager, delete the cell using one of the following methods:

- Select the cell by name, right-click the mouse, and choose **Delete**.
- Select the cell by name, and choose **Edit > Delete**.
- Make sure the cell name is selected in the Cells list. From the **Toolbar**, click **Delete** .

The destination library no longer contains the cell and its files are deleted.

Enabling Rotation and Scaling

Some OpenAccess designs, such as flat panel designs, require rotation (by an angle of any integer value) and magnification of instance masters. The Library Manager tool can

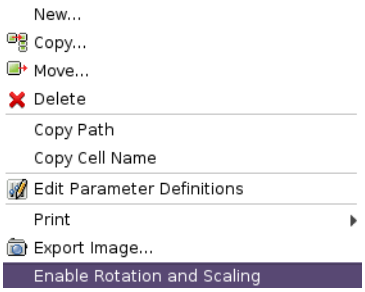
Feedback

enable rotation by any integer value and scaling (magnification) by extending a static instance into an extended instance.

To enable rotation and scaling:

► In the Library Manager tool, enable rotation and scaling using one of these methods:

- Select the cell by name, right-click the mouse, and choose **Enable Rotation and Scaling**.



- Select the cell by name, select **layout** from the CellViews list, right-click the mouse, and choose **Enable Rotation and Scaling**.

Managing Cellviews

You can manage cellviews by creating, copying, moving, and deleting them.

This topic describes the following:

- Creating Cellviews
- Copying Cellviews
- Moving Cellviews
- Deleting Cellviews

Creating Cellviews

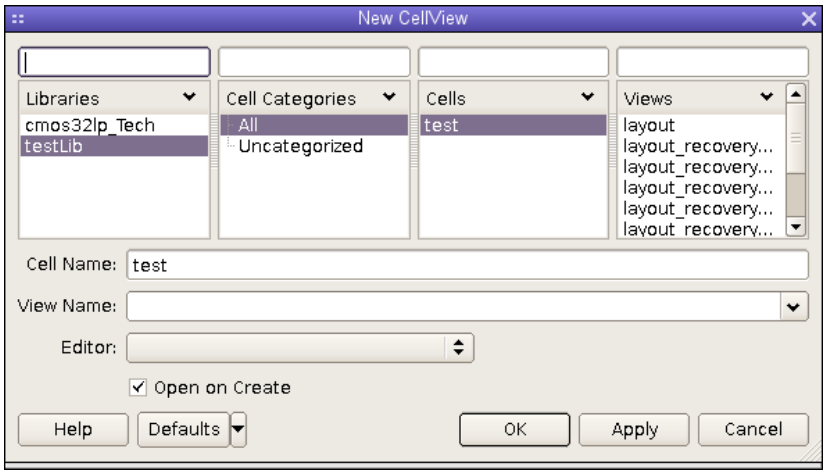
You can create a cellview to hold one design representation of an individual cell. Creating a cellview creates a container for a single representation of the cell.

Use the Library Manager to create cellviews.

Feedback

To create a cellview:

1. In the Library Manager, create the cellview using one of these methods.
   - From any location in the cellviews list, right-click and choose **New**.
   - From the **File** menu, choose **New > CellView**.
   - From the Toolbar, choose **Create CellView** .
2. In the **New CellView** dialog box that opens, specify cellview creation details for its library, optional cell category, cell, and view. Select each item directly or filter the lists.



3. Enter a **Cell Name** for the new cellview.

   To overwrite the cellview shown, accept the name shown.
4. (Optional) To use a different view, select or enter the **View Name**.
5. (Optional) To associate a different editor from the ones shown, choose it from the **Editor** list.
6. (Optional) If you want to start creating the design using the new cellview now, click **Open on Create**.
7. When you are finished, click **Apply** to create the new cellview in the specified library.
8. Click **OK** to close the **New Cellview** dialog box.

Copying Cellviews

You can copy a cellview to create an additional version of it in another library. You can copy cellviews from one library to another with or without changing their names, and other details.

Feedback

This topic describes the following:

- Copying One or More Cellviews Without Changes
- Copying Individual Cellviews With Changes

Copying One or More Cellviews Without Changes

You can copy cellviews without changes to create additional versions in a different library. New versions retain their names, maintain their positions within cell categories, and update their references. To change a cellview while copying it instead, see Copying Individual Cellviews With Changes.

To copy cellviews without changes:

1. In the Library Manager **CellViews** list, press Ctrl and use the mouse to select one or more cellview names.
2. Drag the name or names toward the destination cell.
3. At the destination, drop the names in place, and release the Ctrl key.

   If you do not use the Ctrl key, the system moves the cellviews instead of copying them.

   If the system detects cellviews with duplicate names in the destination library, the **Conflicts** dialog box opens. See Managing Duplicate Name Conflicts.

Copying Individual Cellviews With Changes

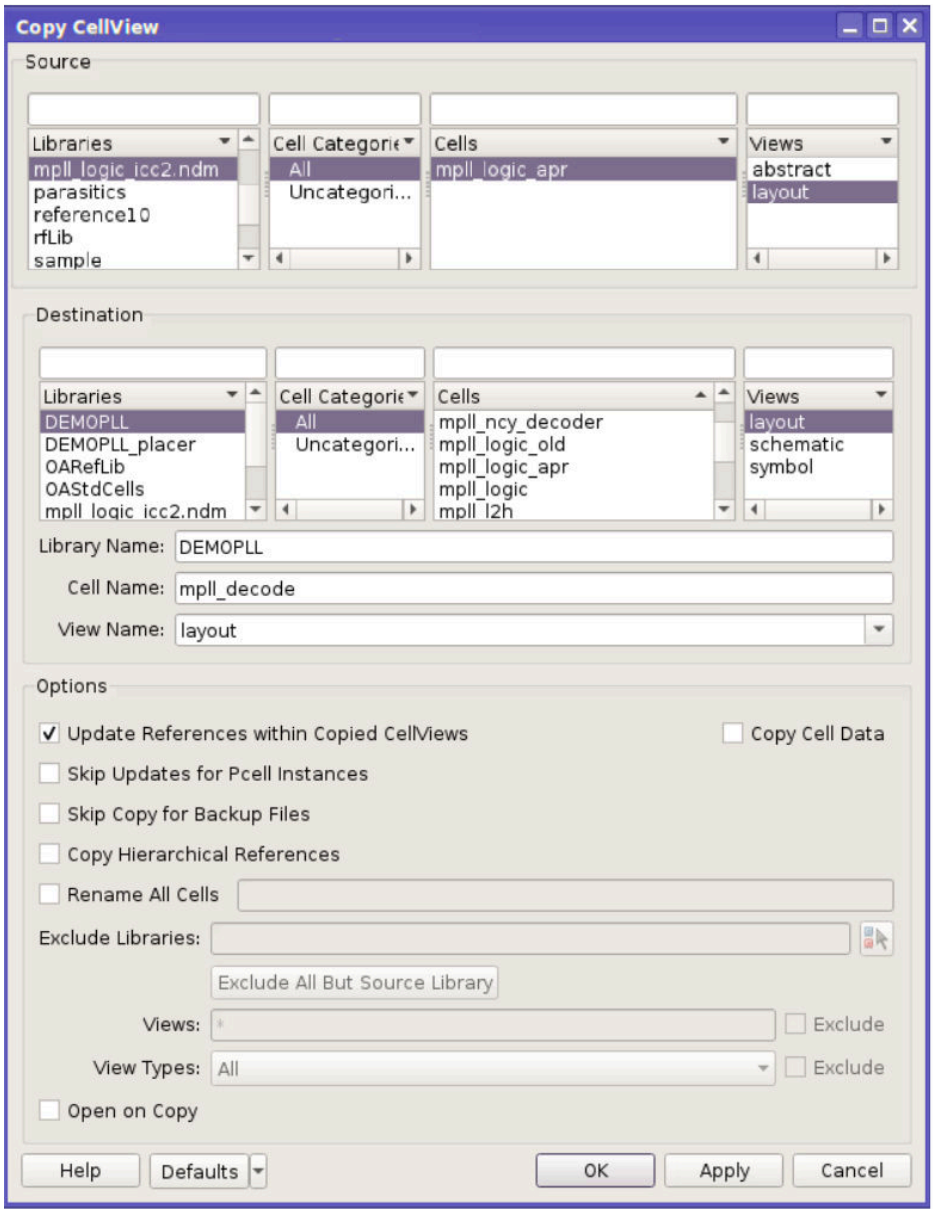
You can copy an individual cellview with changes to create an additional version of it while changing some cellview aspects. As you create the new version, you can choose to change the cellview name, whether to maintain its position in cell categories or update references, and to copy hierarchical references.

To copy an individual cellview with changes:

1. In the Library Manager, specify the cellview to copy using one of these methods:
   - From the cellviews list, select the cell by name, right-click the mouse, and choose **Copy**.
   - From the **Edit** menu, choose **Copy > CellView**, and filter the list to select it.
   - Make sure the cellview is selected in the Views list. From the **Toolbar**, click the **Copy** button.

Feedback

2. In the **Copy CellView** dialog box that opens, specify the following settings:



Setting	Description
Source	The source cellview to copy. Filter the list to specify the source cell or directly select the cellview by name. To view cell categories during this procedure, choose **Show Cell Categories**.
Destination	**Cell Name**: The cell name for the copied cellview at its destination. Filter the list to select a base name or directly enter one.

Feedback

Setting	Description
	View Name: The view name for the copied cellview at its destination.
Options	**Update References within Copied CellViews** causes all design references to instantiated cellviews to point to the copied location. Only references within the copied cellviews are updated. **Skip Updates for Pcell Instances** excludes the selected cellviews during reference updates. **Copy Hierarchical References** copies the hierarchy underneath the cellview by descending the hierarchy, and copying the master of each instance. To control where descent stops, enter a list of library names in the **Exclude Libraries** text box. (Descent stops at a reference to any cellview within any of these libraries). To copy only certain views, enter them in the **Include Views** text box. To copy cellview based on the view type name, select the cell view type name from the **View Types** list. This list is available only when you select **Copy Hierarchical References**. To exclude the specified cell view type name, select **Exclude**. The default is **Select All**. The view type filtering applies to the copied cellviews and to the cellview references from the copied hierarchy. **Rename All Cells** lets you use new names for the copied cellviews in the library. Specify the new name for the cells in the box. You can use name patterns to rename the cells in the library. The name patterns you can use include %L (original library name), %C (original cell name), %N (a unique number in the system), and %<character> (one or more characters). For example, `%C_a` uses the original cell name (`%C`) and adds the `_a` suffix. **Open on Copy** opens the copied cellview when you click **OK**. **Copy Cell Data** copies properties assigned to the cell from the source cellview to the destination cellview, and overwrites any conflicting data that exists in the destination cellview.

3. When you are finished, click **Apply**.

   If the system detects a cellview with a duplicate name in the destination library, the **Conflicts** dialog box opens. See Managing Duplicate Name Conflicts.

4. Click **OK** to close the **Copy CellView** dialog box.

Feedback

Moving Cellviews

You can use the Library Manager to rename a cellview or move it to another library with or without changing cellview details.

This topic describes the following:

- Moving One or More Cellviews Without Changes
- Moving Individual Cellviews With Changes

Moving One or More Cellviews Without Changes

You can move one or more cellviews without changes to automatically maintain categories, and update cellview references during the operation.

To move cellviews without changes:

► In the Library Manager, select one or more cellviews by name, and drag them onto a different cell in the **Cells** list.

If the system detects cellviews with duplicate names in the destination library, the **Conflicts** dialog box opens. See Managing Duplicate Name Conflicts.

Moving Individual Cellviews With Changes

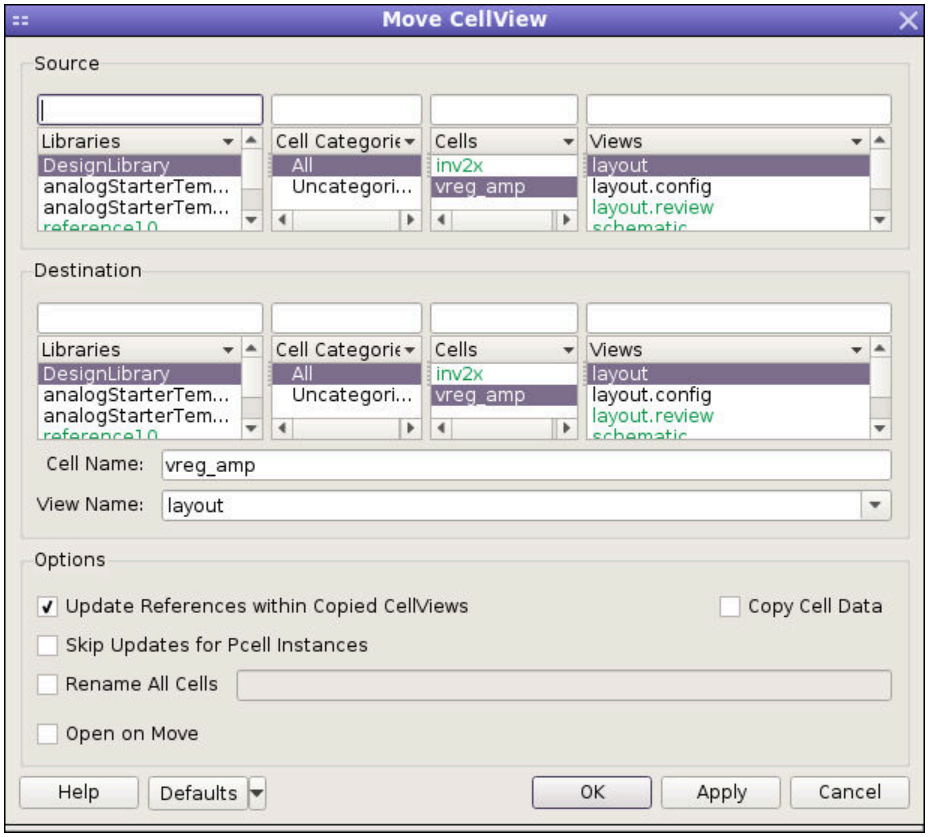
You can move individual cellviews to rename them or relocate them to new cells while adjusting category positions, and references.

To move a cellview with changes:

1. In the Library Manager, specify the cellview to move using one of these methods:
   - From the **Libraries** list, select and right-click the cellview, and choose **Move**.
   - Choose **Edit > Move > Cellview**, and filter the list to select the cellview.
   - Make sure the cellview is selected in the **Views** list. From the **Toolbar**, click **Move** .

Feedback

2. In the **Move Cellview** dialog box, specify the settings:



Settings	Description
Source	The source cellview to move. Filter the list to specify the source cellview or directly select the cellview by name. To view cell categories during this procedure, choose **Show Cell Categories**.
Destination	**Cell Name**: The cell name for the moved cellview at its destination. Filter the list to select a base name or directly enter one.

Feedback

Settings	Description
	View Name: The view name for the moved cellview at its destination.
Options	◦ **Update References within Copied CellViews**: Causes all design references to be instantiated cellviews to point to the new cellview location. Only references within the moved cellviews are updated. ◦ **Copy Cell Data** copies properties assigned to the cell from the source cellview to the destination cellview, and overwrites any conflicting data that exists in the destination cellview. ◦ **Skip Updates for Pcell Instances**: Does not update PCell instances during the move process. ◦ **Rename All Cells**: Renames the moved cells to the specified name. You can add prefixes and suffixes to the original cell names. For example, to add a prefix to the original cell name, type *<prefix>_%C*. ◦ **Open on Move**: Opens the moved cellview when you click **OK**.

If the system detects a cellview with a duplicate name in the destination library, the **Conflicts** dialog box opens. See Managing Duplicate Name Conflicts.

See Also

- Managing Cellviews

Deleting Cellviews

You can delete cellviews by deleting them from a library or deleting them from the disk.

This topic describes the following:

- Deleting Cellviews from a Library
- Deleting Cellviews From Disk

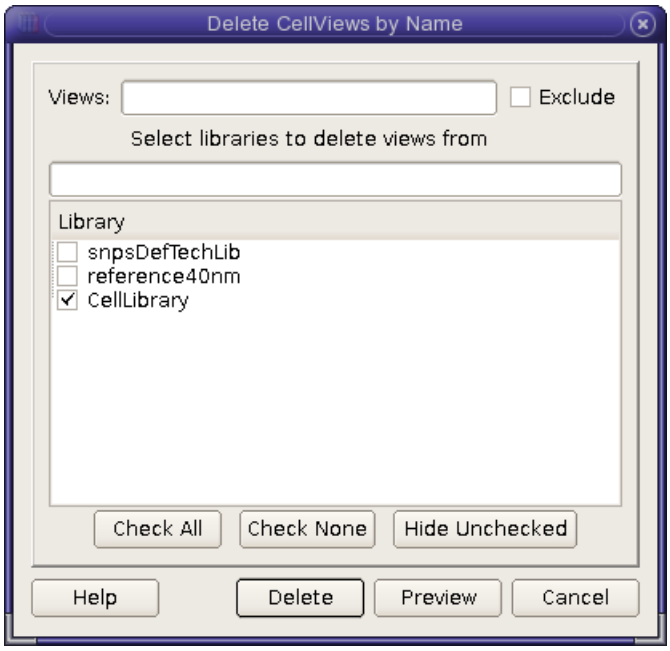
Deleting Cellviews from a Library

You can delete a cellview from a library by deleting its view list name.

Feedback

To delete a cellview from a library:

1. In the Library Manager, choose **Edit > Delete Views by Name**.



2. Specify the libraries from which to delete the view or views.

   Use the check boxes to specify libraries. You can optionally filter the list.

3. At the bottom of the dialog box, enter the exact names for views to designate for deletion from all selected libraries.

   Separate each name with a space. In order to prevent inadvertent deletions, partial names, and wildcard characters, are not accepted here.

4. (Optional) Click **Preview**.

5. Click **Delete**.

The system deletes all designated views, closing them first if they are open, and reports the results of the operation.

Deleting Cellviews From Disk

You can delete a cellview from disk to completely remove it from the system.

Caution:

Select with care. You cannot undo this operation.

After this operation completes there is no method within this environment to retrieve the contents of the deleted cellview.

Feedback

To delete a cellview from disk:

► In the Library Manager, delete the cellview using one of the following methods:

- Select the cellview by name, right-click the mouse, and choose **Delete**.
- Select the cellview by name, and choose **Edit > Delete**.
- Select the cellview by name, and click **Delete** .

The cellview files are no longer on the disk.

Managing Duplicate Name Conflicts

You can manage duplicate name conflicts when copying or moving cells, and cellViews, by refining and confirming your selections.

Destination conflicts can occur when a cell or view with the same name already exists in the destination.

Source conflicts can occur when you have chosen to copy hierarchical references, and the system finds multiple source cells or cellViews with the same name.

When name conflicts occur, the **Cell Conflict** dialog box opens and lets you decide how to resolve them.

To manage duplicate name conflicts:

1. View the list of duplicate items identified by the system.
2. Refine, and confirm, your selections.
   - If you are moving or copying cells, and you want to select individual cellViews for the operation, click **Manage Views**.
   - To continue to copy or move all items, click **Check All**.
   - To copy or move only those items without duplicate names in the destination location, where applicable, click **Check Not in Dest**.
   - To deselect all items, click **Check None**.
3. In the **Conflicts** dialog box, click **Copy**, **Move**, or **Overwrite** to confirm your selections, and complete the operation.

Managing Properties

Library Manager provides editor for Library, Cell, and cellview properties kept in the corresponding data.dm files.

Feedback

This topic describes the following:

- Opening the Context-Sensitive Menu to Manage Properties
- Opening Library Properties
- Copying Cell Properties
- Opening a File in the File Browser

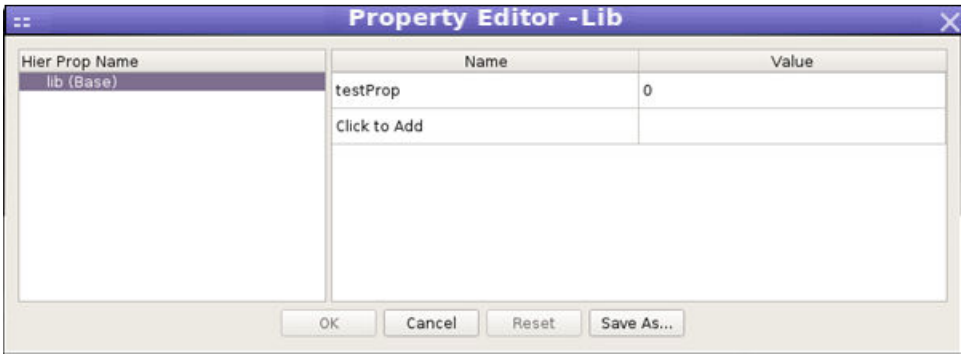
Opening the Context-Sensitive Menu to Manage Properties

Properties can be opened directly from Library, Cell, cellview, or dmFile context-sensitive menus as shown in the following table.

Context	Opening the Context Sensitive Menu
Library	Right-click a Library and select **Edit Properties**.
Cell	Right-Click a Cell and select **Properties > Copy** or **Properties > Edit**.
View	Right-click a Library and select **Edit Properties**.
File	Right-click a File and select **Open** or **Open Read-Only**.

Opening Library Properties

Right-click a Library and select **Edit Properties** to open the **Property Editor** dialog box.



The Property editor contains the following options:

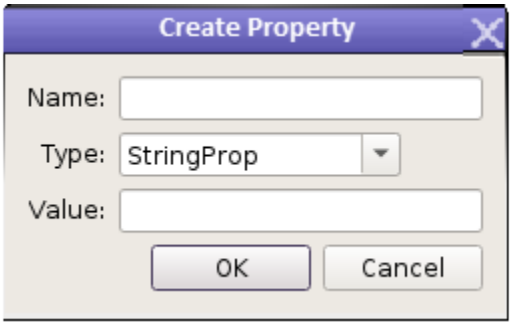
Label	Name
Hier Prop Name	heirProps
Name	propTable

Feedback

Label	Name
Value	propTable

Creating a Property

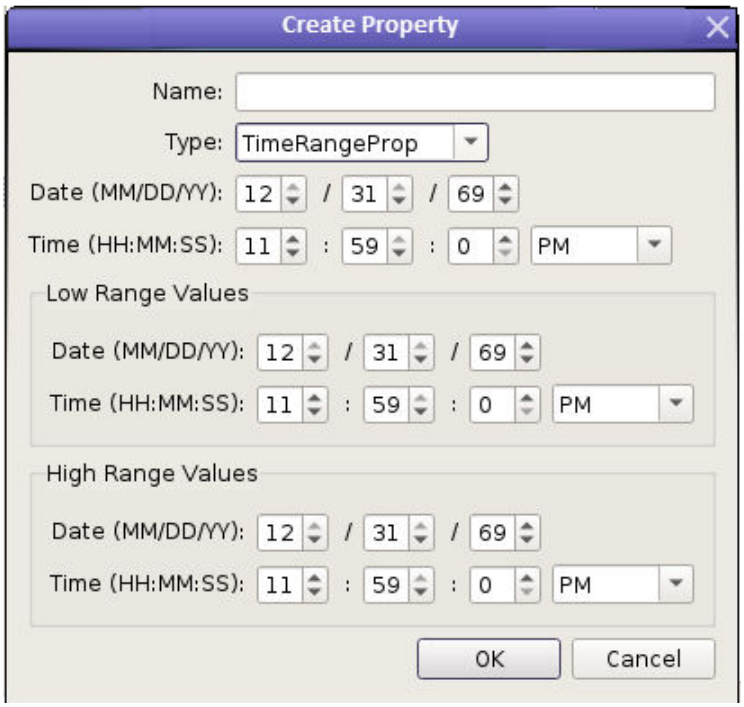
Click to Add to open the **Create Property** dialog box to define the new property name, its type, and the value.



The following property types are supported:

- StringProp
- BooleanProp
- IntProp
- IntRangeProp
- FloatProp
- FloatRangeProp
- DoubleProp
- DoubleRangeProp
- AppProp
- HierProp
- TimeProp
- TimeRangeProp

The **Create Property** dialog box has predefined layouts for each property type. The following figure is an example of the **Create Property** dialog box for the **TimeRangeProp** type.

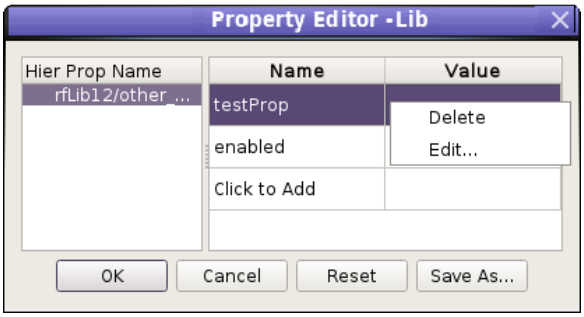


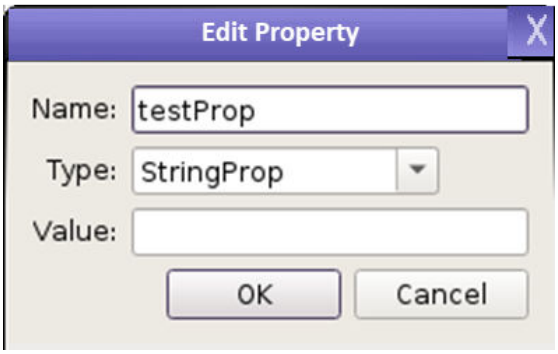
Set the required options and click **OK** to create the property. This property get listed in the Property Editor table.

If the library is managed by version control, and the file is not checked out, you are prompted to check out and continue editing.

Editing a Property

In the **Property Editor** dialog box, right-click a **Name** or **Value** and select **Edit** to open the **Edit Property** dialog box.





You can edit the **Name**, **Type**, or **Value**, then click **OK** to save the changes.

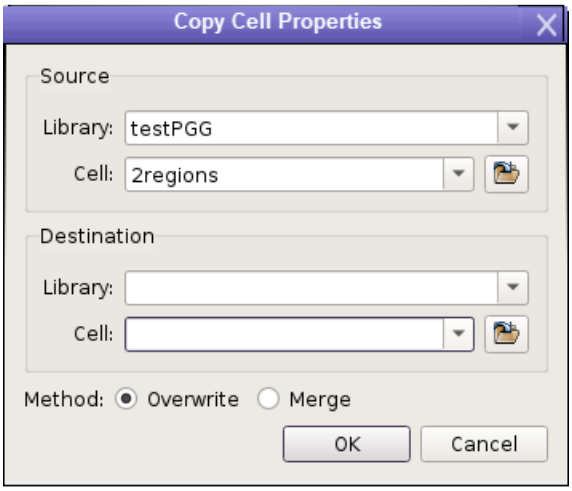
Deleting a Property

In the **Property Editor** dialog box, right-click a **Name** or **Value** and select **Delete** to delete a property.

Copying Cell Properties

The option allows to copy data.dm file from one library hierarchy to another. If destination file exists, then there to conflict resolution methods, either by overwriting the copied file, or by merging properties from the source file to the destination file.

Right-Click a Cell and select **Properties > Copy** to open the **Copy Cell Properties** dialog box.



The **Copy Cell Properties** dialog box contains the following options:

Group	Option Label	Option Name	Description
Source	Library	NA	Select the source library to copy from.

Feedback

Group	Option Label	Option Name	Description
	Cell	sourceCell	Select the source cell to copy from.
Destination	Library	NA	Select the destination library to copy to.
	Cell	destCell	Select the destination cell to copy to.
Method	Overwrite	overwrite	Overwrites the copied cell properties in the destination. The corresponding preference is `dmPropertyUpdateMethod`.
	Merge	merge	Merges the copied cell properties with the destination. The corresponding preference is `dmPropertyUpdateMethod`.

Click **OK** to copy cell properties based on the method you chose.

The corresponding Tcl command is `dm::showCopyCellProperties`.

Opening a File in the File Browser

Library Manager now supports opening any dmDMFile, through file browser, including files that do not belong to a cellview. Whenever a file does not have a default application associated with it, the operating system's default application is used for the file type.

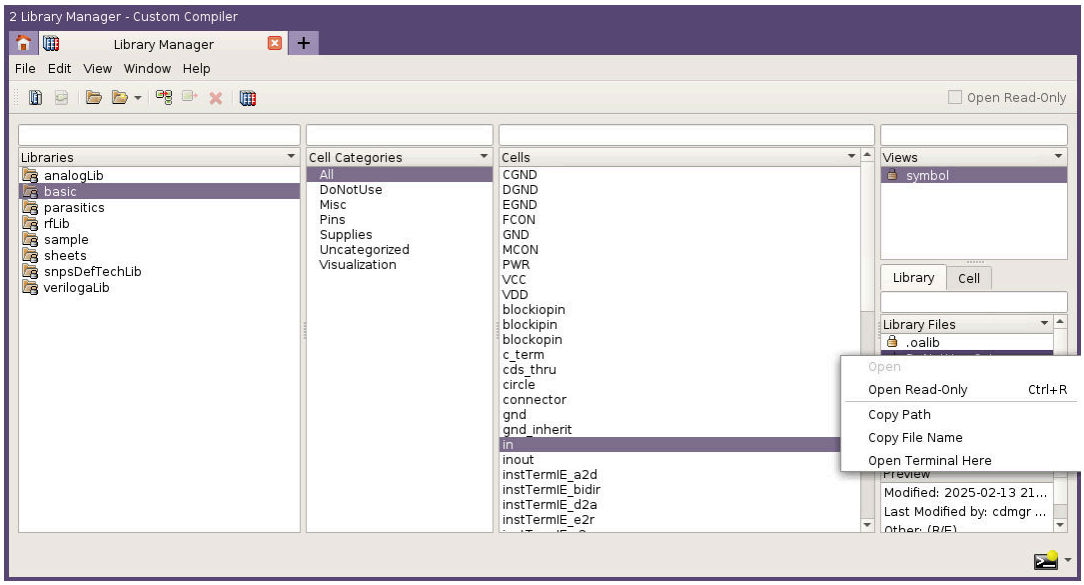
Custom application associations can be created by wildcard file type using the corresponding Tcl commands `de::open` and `de::createAssociation`.

Feedback

To open a file:

► In the file browser, right-click the file and select **Open**. The file opens in the associated default application.

However, to open the file in read-only, right-click the file and select **Open Read-Only**.



Using Cell Categories

You can use cell categories to classify cellviews for organized use. Category groupings can contain cells, and their cellviews, and other cell categories. Any cell can belong to one or more cell categories, or no category.

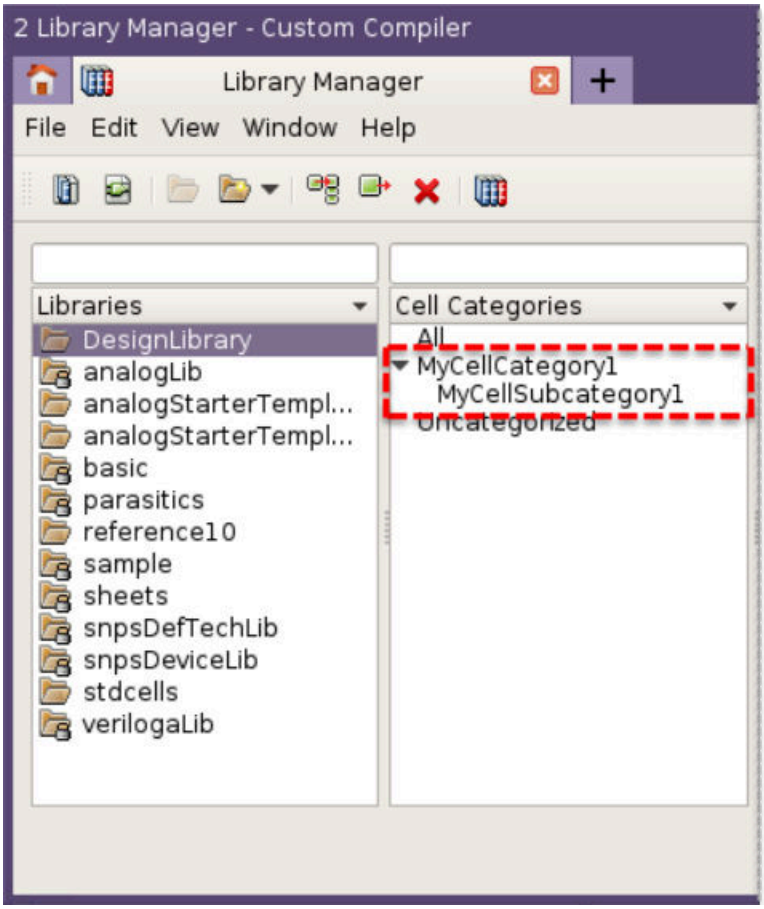
This topic describes the following:

- About Cell Categories and Subcategories
- Viewing Cell Categories and Subcategories
- Creating New Cell Categories
- Managing Cell Categories

About Cell Categories and Subcategories

Cell categories and subcategories allow you to group and organize cellviews.

Feedback



By default, cells in subcategories are displayed in their parent category. When a cell is in a subcategory, it is displayed when you select the parent category in the Library Manager.

Database		**Library Manager**	
Parent_A • **Subcategory_A**	Cell1	**Parent_A** • Subcategory_A	Cell1

If a cell is in multiple subcategories, their mutual parent category shows only one instance of the cell in the Library Manager. The parent itself can also have an instance of the cell.

Database		**Library Manager**	
Parent_A • Subcategory_A • Subcategory_B		**Parent_A** • Subcategory_A • Subcategory_B	Cell1

Feedback

Database		Library Manager
Parent_A • **Subcategory_A** • Subcategory_A	Cell1	
Parent_A • Subcategory_A • **Subcategory_B**	Cell1	

If you use the `dm::deleteFromCellCategory` command to remove a cell and the cell is in multiple subcategories, the parent category continues to display the cell until the last instance is removed from all its subcategories.

Database		Library Manager	
Parent_A • Subcategory_A • Subcategory_B		**Parent_A** • Subcategory_A • Subcategory_B	
Parent_A • **Subcategory_A** • Subcategory_A			
Parent_A • Subcategory_A • **Subcategory_B**			

If you cleared the **View > Show Cells In Parent Category** command check box, cells are displayed only under their direct parent category. This method is useful when organizing cells in cell categories.

Database		Library Manager	
Parent_A • Subcategory_A • Subcategory_B		**Parent_A** • Subcategory_A • Subcategory_B	
Parent_A • **Subcategory_A** • Subcategory_A	Cell1		
Parent_A • Subcategory_A • **Subcategory_B**	Cell1		

Feedback

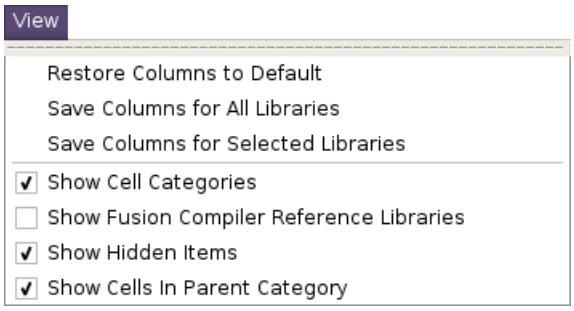
Viewing Cell Categories and Subcategories

You can view categories to see the way cells are classified. Libraries group cells in one category. Custom categories offer an additional level of cell classification.

Use the Library Manager to view cell categories and subcategories.

To view cell categories:

1. In the Library Manager, choose **View > Show Cell Categories**.



2. To display cells in their parent categories, make sure **View > Show Cells in Parent Category** is selected (default).

   Clear the **Show Cells In Parent Category** check box if you want to display cells under their direct parent category only. This mode is useful when you are trying to organize cells in cell categories. After organizing the cells into their subcatgories, you can select **Show Cells In Parent Category** to view the results.

3. Select a library.

   The **Cell Categories** column shows custom categories created by designers and their CAD support group. The categories with all cells, and uncategorized cells, are maintained by the Custom Compiler environment.

See Also

- About Cell Categories and Subcategories

Creating New Cell Categories

You can create new cell categories when you want to organize a group of cells.

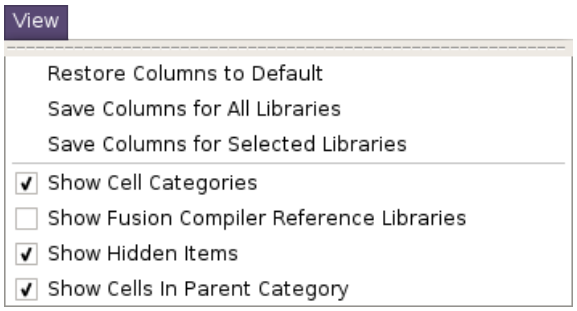
Use the Library Manager to create new cell categories.

After you create the category by name, you can add cells to the category by Managing Cell Categories.

Feedback

To create cell categories:

1. In the Library Manager, choose **View > Show Cell Categories**.



2. Begin to create the category.
   - To create a top-level category for this library, position the cursor on the white space at the bottom of the category list.
   - To create a subcategory, position the cursor over the name of the parent category.
3. Right-click and choose **New**.
4. Enter a unique name for this category.

   Duplicate names generate a warning message.
5. Click **OK.**

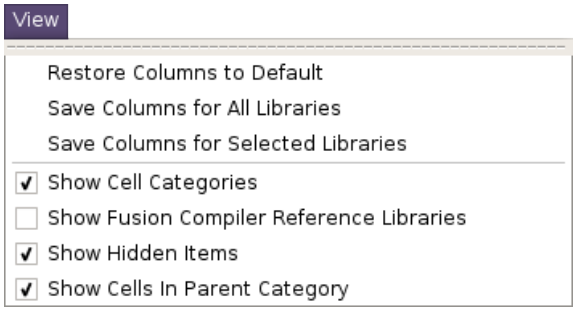
Managing Cell Categories

You can manage cell categories to organize, and classify, cells.

Use the Library Manager to manage cell categories.

To manage cell categories:

1. In the Library Manager, choose **View > Show Cell Categories**.



Feedback

2. Manage categories:
   - To rename a category, position your cursor on the name of the category, right-click, choose **Rename**, enter a new name, and click **Rename**.
   - To move a category under another category, select its name, and drag and drop it onto the name of the target category.
   - To move cells from one category to another, expand the source category so that its contents are visible in the **Cells** column. From the **Cells** column, drag cells you want to move to their new category in the **Cell Categories** column.
   - To delete a category, select its name, right-click, and choose **Delete**.
   - To move a cell out of any category, you can optionally move it to the **Uncategorized** category.

Post-processing MCells

You can import, export, and refresh MCells with the Library Manager.

These sections cover:

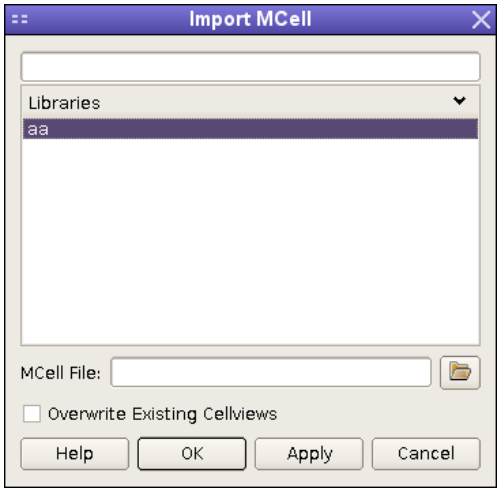
- Importing MCells
- Exporting MCells
- Refreshing MCells

Importing MCells

You can import MCells by parsing a specified MCell definition file into the MCell superMaster design.

1. Begin by opening the Library Manager. From the **Console** tab or any open design window, choose **Tools > Library Manager**.
2. In the Library Manager, choose to **Edit > MCell > Import**.

   The **Import MCell** dialog box appears.



3. Choose the target library from the **Libraries** pane.
4. Specify the MCell definition file in the **MCell File** text field or click the **Browse** button to choose the file. The default file extension is **.mcdef*. This is controlled by the Tcl preference `dbImportMCellFileName`.

   In the definition file, multiple MCells can be defined at the same time. For information, see Custom Compiler Application Notes.
5. (Optional) Enable the **Overwrite Existing Cellviews** option to overwrite any existing cellview with the new-defined cellview if any new-defined superMaster's cell name and view name are in conflict with the existing cellview.
6. (Optional) Enable the **Load CDF** option to load the CDF parameters of defined MCells.

   To load the CDF parameters defined in files stored in a particular directory, specify the directory name.

   To load the CDF parameters defined in a particular file (with the naming format, *library name-cell name*.tcl), specify the file name.

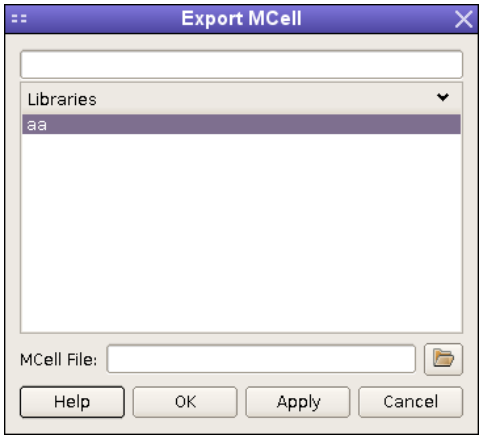
Exporting MCells

You can export all MCell superMasters in the specified library to an output file through an **Export MCell** dialog box. This output file can be used as the MCell definition file when importing MCells.

1. Begin by opening the Library Manager. From the **Console** tab or any open design window, choose **Tools > Library Manager**.
2. In the Library Manager, choose to **Edit > MCell > Export**.

   The **Export MCell** dialog box appears.

Feedback



3. Choose the required library from the **Libraries** pane.
4. In the **MCell File** text field, specify a file name for the MCell definition file to be exported, or click the **Browse** button to choose the file. The default file extension is **.mcdef*. This is controlled by the Tcl preference `dbExportMCellFileName`.

   If the specified file name already exists, you will be prompted to confirm overwriting the existing file.

   To export the CDF parameters of all the MCells defined in the selected library to a directory, select **Export CDF** and specify the directory name. The CDF parameters for each MCell are exported as a file with the naming format, *library name-cell name.tcl*.

   In the output file, the sequence of MCell superMasters, layer definitions, and the rules might be changed and might be different from the imported MCell definitions file.

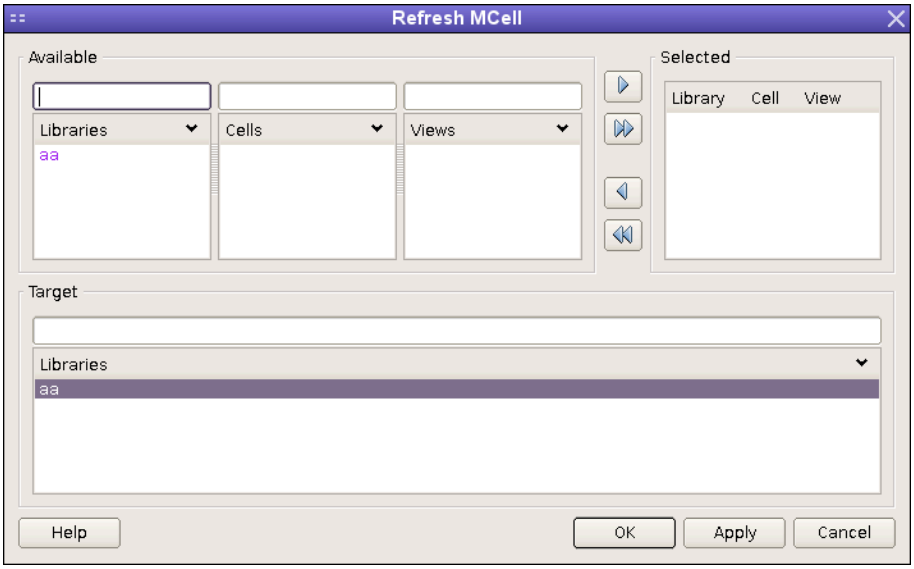
   For information, see MCell Definition File in *Custom Compiler Layout Editor User Guide for Creating and Editing Objects*.

Refreshing MCells

You can use the **Edit > MCell > Refresh** command to get the latest subMaster layout with the up-to-date superMaster and evaluator.

In the **Refresh MCell** dialog box, you can specify the superMasters of MCells and editable PCells and the target design libraries. The tool refreshes the recovery and version parameters for MCells and editable PCells, and saves the MCell instance and editable PCell instances and vias in the target design libraries with specified superMasters.

Feedback



When multiple users work on the same design, the Custom Compiler tool checks whether MCell SuperMasterSignature is changed and reports an error message if it fails to create or edit the edited MCells when reloading MCell definitions.

You can use the `leMGRDefaultParam` Tcl preference to calculate the default parameters for MCell guard rings when importing MCell definitions.

When you update MCell definitions locally, use **Edit > MCell > Refresh** to update the layout of MCells.

When other users update the MCell definitions, load the new MCell definitions from disk to update the content in memory using one of the following methods:

- Use the following Tcl command to reload:

  ```
  db::loadMCells -libName $mcellSuperMasterLibName
  ```

- Exit Custom Compiler and restart the tool.

Library Manager Menus

This topic provides a quick reference of the Library Manager menus:

- File Menu
- Edit Menu
- View Menu

Feedback

- Window Menu
- Help Menu

File Menu

The Library Manager **File** menu contains the following commands:

Command	Description
New	Contains the **Library**, **Cell**, and **CellView** commands. For more information, see Creating New Libraries, Creating New Cells, and Creating Cellviews.
Open	Opens the **Open Design** dialog box. For more information, see From the Console Tab.
Open Recent	Displays a list of recently opened files. These can be opened by clicking on their names.
Add Fusion Compiler Library	Opens the **Add Existing Fusion Compiler Library** dialog box so that you can add Fusion Compiler design libraries to the Custom Compiler environment.
Create Fusion Compiler Design	Opens the **Create Fusion Compiler** dialog box for Developing Digital Blocks from Verilog Files in co-design processes.
Add ICC Library	Opens the **Add Existing ICC Library** dialog box so that you can add IC Compiler design libraries to the Custom Compiler environment.
Update ICC Reference Libraries	Updates ICC Reference Libraries to reflect most recent changes.
Add ICC2 Library	Opens the **Add Existing ICC2 Library** dialog box so that you can add IC Compiler II design libraries to the Custom Compiler environment. See the *Custom Compiler* and IC Compiler II Co-Design Assistant User Guide for details.
Create ICC2 Design	Opens the **Create ICC2 Design** dialog box for Developing Digital Blocks from Verilog Files in co-design processes.
Expand Schematic	Opens the **Expand Schematic** dialog box. For more information, see Expand Schematic.
Generate Layout Pins	Generates multiple pins at the library level for layouts. See Generating Multiple Pins Simultaneously.
Import	Imports design data in a particular format. See Import.
Export	Exports design data in a particular format. See Export.
Print	For more information, see Printing and Plotting Designs.

Feedback

Command	Description
Export Image	Opens **Export Image** dialog box. For more information, see Exporting Design Images.
Refresh	Reverts to previously saved `lib.defs` content by refreshing from disk, or opens the **Refresh Libraries Definitions** dialog box when `lib.defs` content has changed.
Library Definitions Editor	Opens the **Library Definitions Editor.** For more information, see Editing Library Definitions.
Close	Closes the Library Manager.

Edit Menu

The Library Manager **Edit** menu contains the following commands:

Command	Description
Copy	Opens the appropriate **Copy** dialog box based on whether a library, cell, or cellView is selected. If nothing is selected, opens the **Copy Library** dialog box.
Move	Opens the appropriate **Move** dialog box based on whether a library, cell, or cellView is selected. If nothing is selected, opens the **Move Library** dialog box.
Delete	Deletes the item from the disk.
Delete Views by Name	Opens the **Delete CellViews by Name** dialog box. For more information, see Deleting Cellviews.
Edit Library Compression	Opens the **Edit Library Compression** dialog box. For more information, see Compressing and Uncompressing Libraries.
Copy Names	Copies selected item's name for use outside of the Custom Compiler environment. For more information, see Copying Library, Cell, and Cellview Names.
Copy Path	Copies the path of a library, cell, or cell view to view or access data in that directory. For more information see Copying Library, Cell, and Cellview Paths.
Open Terminal Here	Opens terminal in the directory of the selected container. see Opening a Terminal From the Directory of a Library.
Hide Cell	Hides selected library or cell. (To unhide the library or cell, choose **View > Show Hidden Items**).

Feedback

Command	Description
Import in Background	Imports the digital design data in the background when you add the Fusion Compiler or IC Compiler II design. If the digital design is greater than 1.2 GB, the co-design assistant displays a message and suggests that you use this command to import the digital design data in the background. If the import operation is in progress, this command changes to **Stop Background Import**. The command is unavailable if the digital design is already imported. The equivalent command, **Import in Background**, is available in the shortcut menu when you right-click a **Library**, **Cell**, or **View** name in the Library Manager. Tcl preference: `dmImportCellViewsInBackground`
MCell	For more information, see Post-processing MCells.
Technology	Opens the **Technology Manager**. For more information, see Technology Manager Tool.
Parameter Definitions	Opens the **Parameter Definitions Editor**. For more information, see Parameter Definition Editor Tool.
Search and Replace	Opens **Search and Replace**. For more information, see Search and Replace Tool.
Check and Save Libraries	Opens the **Check and Save Libraries** dialog box. For more information, see Checking and Saving All Designs in Libraries.
Update Library References	Opens the **Update Library References** dialog box. For more information, see Updating Library Name References.

View Menu

The Library Manager **View** menu contains the following commands:

Command	Description
Restore Columns to Default	Resets the widths of the panes in the Library Manager to the default widths.
Save Columns for All Libraries	Saves the current pane width settings for all libraries.
Save Columns for Selected Libraries	Saves the current pane width settings for only the selected libraries.
Show Cell Categories	Shows cell categories when checked.

Feedback

Command	Description
Show ICC2 Reference Libraries	Displays the associated IC Compiler II reference libraries. Use this command after you add the IC Compiler II libraries to the Custom Compiler environment.
Show Fusion Compiler Reference Libraries	Displays the associated Fusion Compiler reference libraries. Use this command after you add the Compiler Compiler libraries to the Custom Compiler environment.
Show Hidden Items	Displays hidden libraries. See Showing and Hiding Libraries and Cells.
Show Cells in Parent Category	Displays cell subcategories created under parent categories. See Viewing Cell Categories and Subcategories.
Display Preview Horizontally	Displays the **Preview** pane horizontally at the bottom of the Library Manager.
Show File Browser	Displays the list of DMFiles in the design library and cell directories.
Keep Filter Text on Selection Change	Retains the filter text you specified in the Library Manager.
Show Cells in Parent Library	Shows cells from child libraries of the COMBINE library under the parent (top) library.

Window Menu

The Library Manager **Window** menu contains the following commands:

Command	Description
New Library Manager	Creates a new instance of the Library Manager.
Full Screen	Makes Library Manager full screen.
New Window	Opens a new Custom Compiler window.
New Tab	Opens a new **Home** tab.
Assistants	Contains the **Console** command which displays the **Console** pane.
Toolbars	Displays the toolbar if **Standard** is checked.
Hide All	Hides all menus.
Customize	Opens the **Custom Window** dialog box so that you can add or remove menus and toolbars to create a customized work environment. See Configuring Custom Windows.

Feedback

Command	Description
Keyboard Shortcuts	Opens a dialog box that displays keyboard shortcuts for certain commands. For more information, see Customizing Keyboard Shortcuts.
<window_name>	At the bottom of the Window menu, the names of the open windows are displayed. You can choose the name of the window that you want to display.

Help Menu

The Library Manager **Help** menu contains the following commands:

Command	Description
Custom Compiler Help	Displays the Custom Compiler help in a new tab.
Library Manager Help	Opens the Library Manager help in a different tab or in an external system browser depending on whether or not **Use System Web Browser** is checked.
Tcl Help Utility	Displays the **Tcl Help Utility** tab. See Using the Tcl Help Utility.
About Custom Compiler	Opens the **About Custom Compiler** dialog box containing information about the Custom Compiler version number, copyright, legal notices, and Synopsys contact information.

Library Manager Toolbar

The toolbar contains the following:

Table 102 Library Manager Toolbar

Button	Description
Create Library	Opens the **New Library** dialog box. For more information, see Creating New Libraries.
Create CellView	Opens the **New CellView** dialog box. If a cell is selected, the **Libraries**, **Cell Categories**, and **Cells** categories are automatically selected in the dialog box. For more information, see Creating Cellviews.
Open CellViews	Opens the currently selected cellView. This button can only be used when a cellView is selected.

Feedback

Table 102 Library Manager Toolbar (Continued)

Button	Description
Open Recent	Displays a list of recently opened files. These can be opened by clicking on their names.
Copy	Opens the appropriate **Copy** dialog box based on whether a library, cell, or cellView is selected. If nothing is selected, opens the **Copy Library** dialog box.
Move	Opens the appropriate **Move** dialog box based on whether a library, cell, or cellView is selected. If nothing is selected, opens the **Move Library** dialog box.
Delete	Deletes the selected item from the disk.
New Library Manager	Creates a new instance of the Library Manager.

Feedback

Parameter Definition Editor Tool

You can use the Parameter Definition Editor to display, define, and set values for parameters.

Parameters are metadata associated with instance objects. Parameters differ from properties in that they can only be associated with instance objects. Additionally, the definition of the parameter cannot be created directly on an instance. Instead, the parameter definitions are inherited from data stored in the instance master's associated cell and library.

CAD support personnel and library developers can define parameters to serve the requirements of output tools.

This topic describes the following:

- Displaying the Parameter Definition Editor Tool
- Viewing Parameters
- Adding Parameters
- Adjusting Parameters
- Editing Components Parameters
- Viewing CDF Overlay Parameters
- Editing CDF Overlay Parameters
- Parameter Definition Editor Menus
- Parameter Definition Editor Toolbar

Displaying the Parameter Definition Editor Tool

To display the Parameter Definition Editor, use one of the following methods:

- From the Custom Compiler Home tab, choose **Tools > Parameter Definition Editor**.
- From the Library Manager, choose **Edit > Parameter Definitions**.
- In the Library Manager, right-click a selected library or cell and choose **Edit Parameter Definitions**.
- In the Schematic Editor, press Q or choose **Edit > Properties > Assistant** or **Window > Assistants > Property Editor** to bring up the Property Editor, from which you choose **Parameter Definitions** from the toolbar menu .

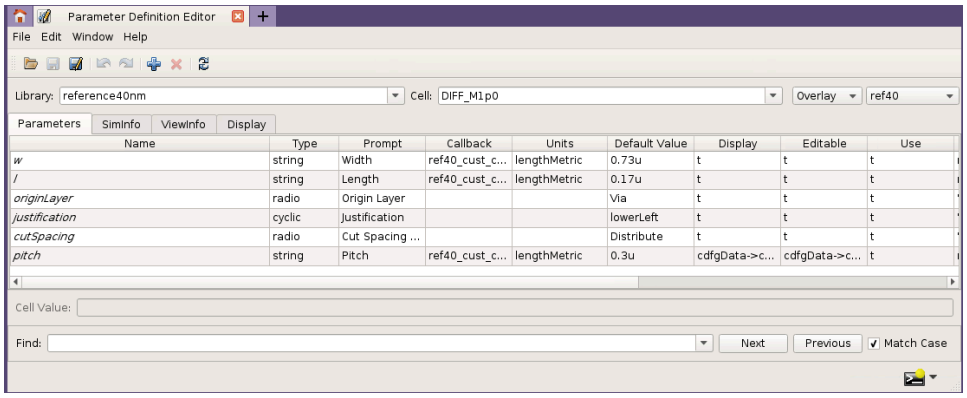
Feedback

Viewing Parameters

You can view parameters in the Parameter Definition Editor to begin to work with them.

To view parameters:

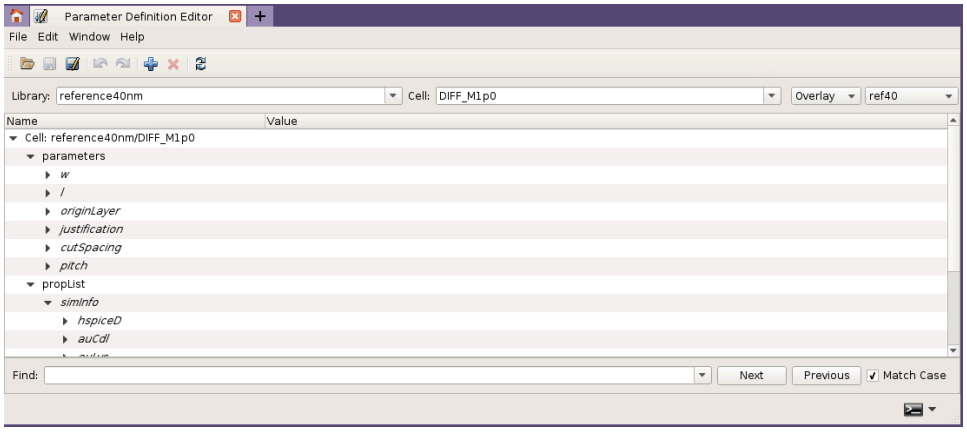
1. In the Parameter Definition Editor, choose **Library** or **Cell**.



2. The table view contains the **Parameters** tab and other tabs of the propList section (**SimInfo**, **ViewInfo**, and **Display**). Switch tabs to view the parameter information. **Cell Value** is provided to display the value of selected column.

   To edit and view the details of IxComponentType, see Editing Components Parameters.

3. The table view can be switched to tree mode by setting the `dbEnableTableBasedParamDefEditor` preference. For example: `db::setPrefValue dbEnableTableBasedParamDefEditor -value false`



   Scroll to view the parameters.

4. To find a parameter, enter the parameter name in the **Find** box, and click **Next** or **Previous**.

   If the **Find** box is not shown, choose **Edit > Find** to display it.

Adding Parameters

You can add a parameter to an object by defining its type, and optionally setting its value.

To add a parameter:

1. In the Parameter Definition Editor, choose **Edit > Add Parameter**. Alternatively, click .
2. Type the name of the parameter and press **Enter**. The new parameter is added to the list.
3. Define each parameter aspect by left-clicking next to the notation.

   Define the type, prompt, callback, units, default value, display, whether editable, use, parseAsPEL, parseAsNumber, storeDefault, and propList.
4. Choose **File > Save**.

Adjusting Parameters

You can adjust parameters to change values set for them.

To adjust parameters:

1. In the Parameter Definition Editor, view the tree display.
2. To rearrange the order of the parameters, drag a parameter to its new position in the list.
3. To rename a parameter, double-click the parameter name, and type the new name.

To modify parameters using Component Description Format (CDF) overlays, see Modifying Component Description Format (CDF) Overlays.

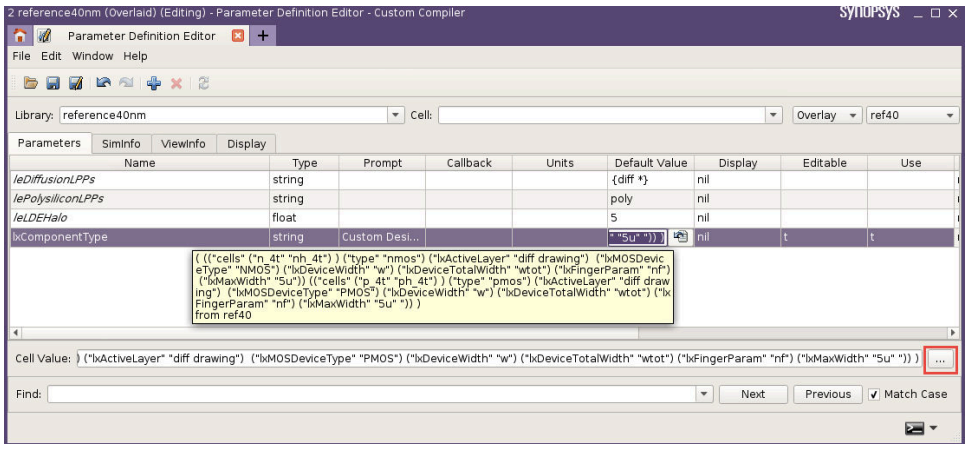
Editing Components Parameters

You can view and edit the values of the lxComponentType parameters in the **Edit Components** dialog box.

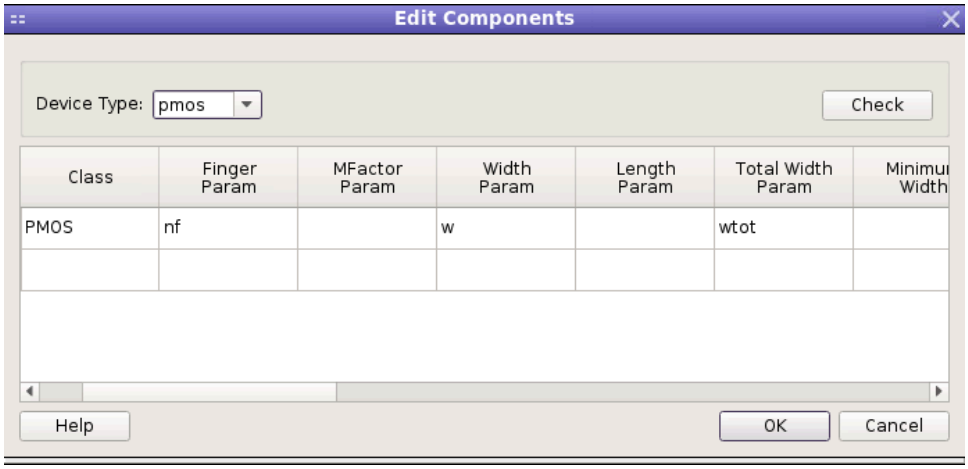
Feedback

To view and edit the values of the lxComponentType parameters:

1. Click the **Default Value** column from lxComponentType.
2. The ... button appears next to the **Cell Value** field.



3. Click the ... button to enable **Edit Components** dialog box. The table is categorized the components information by **Device Type**. The **Check** button parses and checks the components from the library into the **Components Report** dialog box.



Feedback

Viewing CDF Overlay Parameters

If a Component Description Format (CDF) overlay library exists, the Parameter Definition Editor automatically displays the **Overlay** option. If more than one CDF overlay is available, the list of CDF overlays is displayed. The order of the overlays is determined by the `ASSIGN dbOverlayPath` statements in the `lib.defs` file. See Modifying Component Description Format (CDF) Overlays.

Single CDF Overlay

Multiple CDF Overlays

To view overlay library parameters:

► In the Parameter Definition Editor, select **Overlay**.

Selecting **Overlay** displays the available overlays in the Parameter Definition Editor. The overlays are ordered from the highest to the lowest priority. The default active overlay is the overlay with the highest priority.

If you select **Base**, only the base CDF is displayed in the editor.

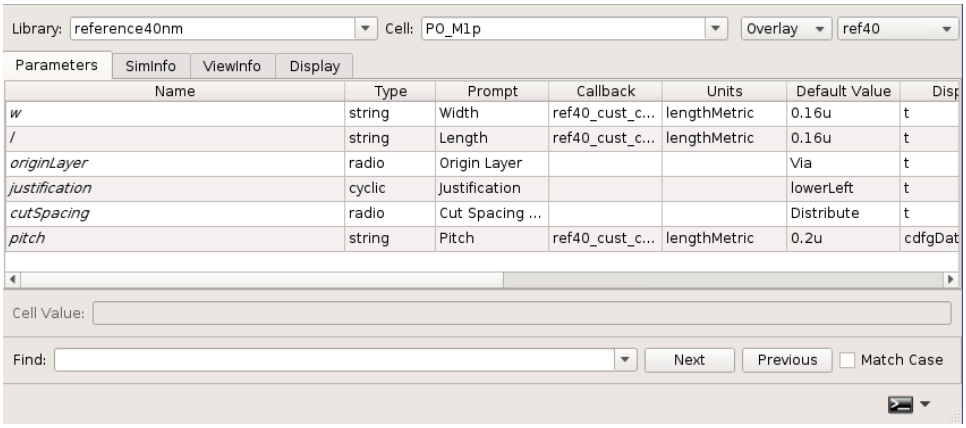
If you select **Effective**, parameters of effective level, all overlays, and the base CDF level are displayed in the read-only mode. **Effective** is in the highest precedence level, supersedes **Base** and **Overlay**. The tool allows to change CDF data dynamically in the **Effective** level. All the **Effective** level information is cleared after closing the tool.

When overlays are displayed in the Parameter Definition Editor:

- *Italicized* text indicates the base parameter values.
- Non-italicized text indicates overlays that override base parameter values.
- **Bold** text indicates edited parameter values.

The following example shows an overlay in use. The *w* parameters are italicized to indicate that they are values from the base library. The model parameters, which are not italicized, contain overlay values that override the *w* parameters. The **defValue** model parameter is in bold font because it has been edited.

Feedback


Library: reference40nm
Cell: PO_M1p
Overlay
ref40
Parameters
SimInfo
ViewInfo
Display
Name
Type
Prompt
Callback
Units
Default Value
Disp
w
string
Width
ref40_cust_c...
lengthMetric
0.16u
t
l
string
Length
ref40_cust_c...
lengthMetric
0.16u
t
originLayer
radio
Origin Layer
Via
t
justification
cyclic
Justification
lowerLeft
t
cutSpacing
radio
Cut Spacing ...
Distribute
t
pitch
string
Pitch
ref40_cust_c...
lengthMetric
0.2u
cdfgDat
Cell Value:
Find:
Next
Previous
Match Case

Feedback

Editing CDF Overlay Parameters

You can use the Parameter Definition Editor to edit the parameters of one or more Component Description Format (CDF) overlays.

To edit overlay library parameters:

1. In the Parameter Definition Editor, select **Overlay**.

   Selecting **Overlay** displays the available overlays in the Parameter Definition Editor. The overlays are ordered from the highest to the lowest priority. The default active overlay is the overlay with the highest priority.

   Single CDF Overlay

   Multiple CDF Overlays

   If you select **Base**, only the base CDF is displayed in the editor.

   If you select **Effective**, parameters of effective level, all overlays, and the base CDF level are displayed in the read-only mode. You can use the `db::getParamDefs` and `db::setNetlistInfo` commands to edit the CDF data under the **Effective** level. For examples:

Example 4

```
[db::getParamDefs $param_name -of $cell -level memory]
db::setAttr param.defvalue -value $new_value
```

Example 5

```
db::setNetlistInfo termOrder -data $lib -info simInfo -netlister
 hspiceD -level memory -value $new_value
```

2. (Multiple overlays only) In the **Overlay** list, select the overlay you want to edit.

   The CDF data for the selected overlay and all overlays with lower priority appear in the editor. Data of overlays with higher priority are not displayed.

Feedback

For example, if Overlay1 has the highest priority and Overly3 has the lowest priority, then:

- If you select Overlay, parameters of all overlays and the base CDF are displayed in priority order.
- If you select Overlay2, parameters of Overlay2, Overlay3, and the base CDF are displayed in priority order.
- If you select Overlay3, parameters of Overlay3 and the base CDF are displayed in priority order.

3. Edit the overlay parameters.

   When **Overlay** is selected, you can edit the parameters of the selected overlays. When **Base** is selected, you can edit the base CDF data only.

Parameter Definition Editor Menus

The following tables describe the Parameter Definition Editor menus:

- Table 103
- Table 104
- Table 105
- Table 106

Table 103 File Menu

Menu	Command	Description
File	Open	Opens the **Open Parameter Definitions** dialog box.
	Save	Saves the parameter definitions.
	Save As	Opens the **Save Parameter Definitions** dialog box.
	Discard Edits	Opens the **Discard Changes** dialog box to discards edits you made to the current parameter definitions.

Feedback

Table 103 File Menu (Continued)

Menu	Command	Description
	Make Read-Only	Makes the parameter definition editor read-only. If there are unsaved edits, you are prompted to save or discard them before making the parameter definition editor read-only.
	Close	Closes the Parameter Definition Editor.

Table 104 Edit Menu

Menu	Command	Description
Edit	Undo	Undoes the most recent modification.
	Redo	Redoes the most recent modification.
	Cut	Cuts the selected parameter value.
	Copy	Copies the selected parameter value.
	Paste	Pastes the copied or cut parameter value.
	Add Parameter	Adds a new parameter definition to the library. See Adding Parameters.
	Remove Parameter	Removes the selected parameters definition from the **Parameters** tab.
	Find	Enables or disables the **Find** field.

Feedback

Table 104 Edit Menu (Continued)

Menu	Command	Description
	Update Parameters	Synchronizes parameter definitions of cell with the parameters of its schematic and text views.

Table 105 Window Menu

Menu	Command	Description
Window	New Window	Opens a new Custom Compiler window.
	New Tab	Opens a new Custom Compiler **Home** tab.
	Full Screen	Makes Parameter Definition Editor full screen.
	Assistants	Contains the **Console** command which displays the **Console** pane.
	Toolbars	Displays the toolbars if checked.
	Hide All	Hides all menus.
	Customize	Opens the **Customize Window** dialog box so that you can add or remove menus and toolbars to create a customized work environment. See Configuring Custom Windows.
	Keyboard Shortcuts	Opens a dialog box that displays keyboard shortcuts for certain commands. For more information, see Customizing Keyboard Shortcuts.
	<window_name>	At the bottom of the Window menu, the names of the open windows are displayed. You can choose the name of the window that you want to display.

Table 106 Help Menu

Menu	Command	Description
Help	Custom Compiler Help	Displays the Custom Compiler help in a new tab.
	Parameter Definition Editor Help	Opens the Parameter Definition Editor help in a different tab or in an external system browser.
	Tcl Help Utility	Opens Tcl Help Utility. See Using the Tcl Help Utility.

Feedback

Table 106 Help Menu (Continued)

Menu	Command	Description
	About Custom Compiler	Opens the **About Custom Compiler** dialog box containing information about the Custom Compiler version number, copyright, legal notices, and Synopsys contact information.

Parameter Definition Editor Toolbar

The toolbar contains the following:

Table 107 Hierarchy Editor Toolbar

Button	Description
Open	Opens the **Open Parameter Definitions** dialog box.
Save	Saves the parameter definitions.
Save As	Opens the **Save Parameter Definitions** dialog box.
Undo	Undoes the most recent modification.
Redo	Redoes the most recent modification.
Add Parameter	Adds a new parameter definition to the library. See Adding Parameters.
Remove Parameter	Removes the selected parameters definition from the **Parameters** tab.

Feedback

Table 107 Hierarchy Editor Toolbar (Continued)

Button	Description
Update Parameters	Synchronizes parameter definitions of cell with the parameters of its schematic and text views.

Search and Replace Tool

You can use the Search and Replace tool to locate components in your designs. After locating components, you can use this tool to replace their associated attribute values, properties, and parameters.

	This video tutorial shows an example of how to use the Search and Replace tool.

This topic describes the following:

- Displaying the Search and Replace Tool
- Setting Search and Replace Options
- Setting the Design Root
- Setting Search Criteria for Attributes and Properties
- Searching for Design Components
- Browsing Component Search Results
- Refining Component Search Results
- Checking Whether a Property Exists on an Instance
- Finding Instances With the Default Value of a Given Property
- Finding Instances With a Specific Property Value
- Finding Instances With a Property Not Equal to a Specific Value
- Finding Instances With a Property Value in a Given Range
- Finding Paths or Path Segments in Layouts

Feedback

- Finding Text on Layer Shapes in Layouts
- Finding Text Objects in Layouts
- Finding Vectored Instances
- Finding Via Vector Values
- Finding Vias in Layouts
- Getting Total Number of Instances Used in a Design
- Replacing Component Aspects Using Search Results
- Replacing Scientific Notation With Equivalent Value
- Replacing Property Values

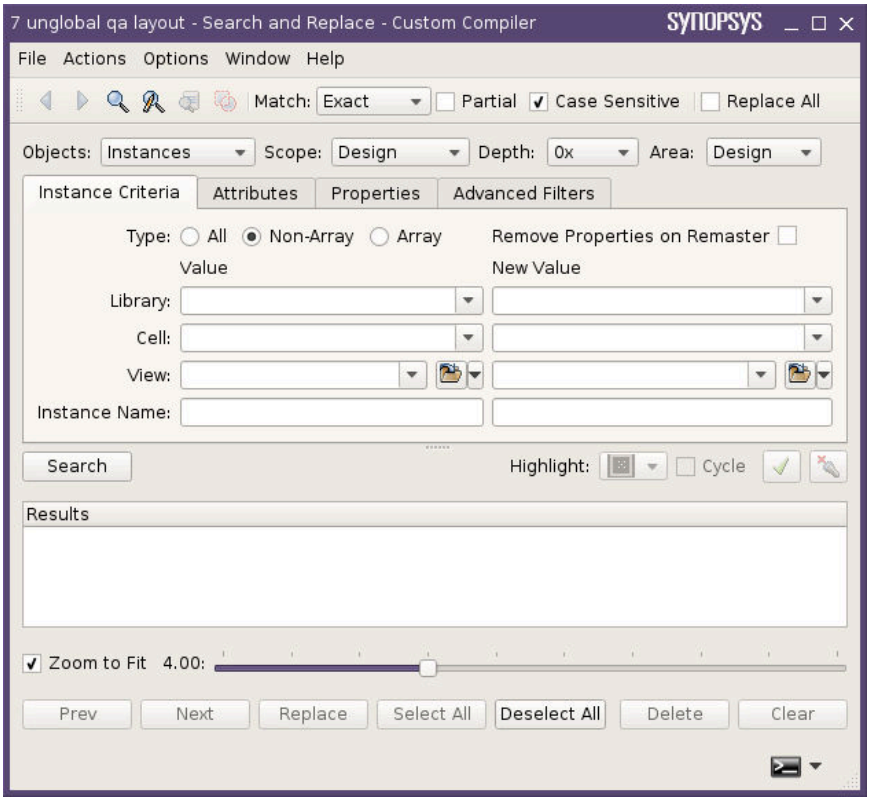
Displaying the Search and Replace Tool

To display the Search and Replace tool, use one of the following methods:

- From the Console, choose **Tools > Search and Replace**.
- From the Library Manager, choose **Edit > Search and Replace**.
- From the Layout Editor, choose **Query > Search and Replace**.
- From the Schematic Editor, choose **Edit > Search and Replace**.
- From the Symbol Editor, choose **Edit > Search and Replace**.
- From the Text Editor, click **Search and Replace** on the **Standard** toolbar.

The **Search and Replace** dialog box displays. The content of tabs in the dialog box are the same.

Feedback



The default search scope depends on where you displayed the Search and Replace tool. To change the scope, see Setting the Design Root.

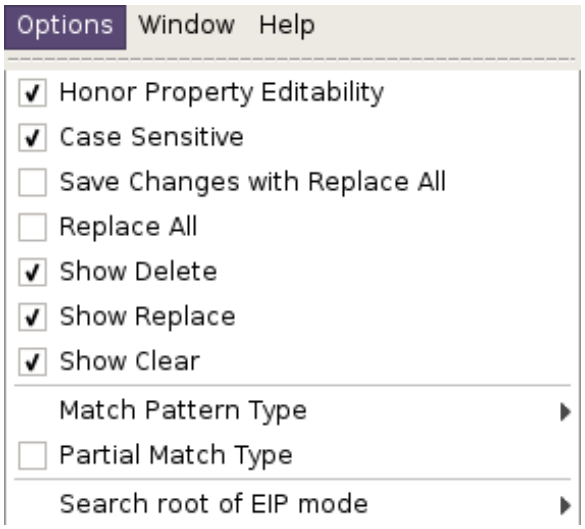
Setting Search and Replace Options

There are two general search and replace options. You can make replacement processes ignore properties that cannot be edited. You can make all searches case-sensitive.

Feedback

To set general search and replace options:

► In the Search and Replace tab or dialog box, choose one of the following **Options** commands:



- **Honor Property Editability** ignores properties that are not editable in a search and replace operation. By default, this option is enabled, and properties that are not editable are ignored.
- **Case Sensitive** performs all searches in a case-sensitive manner. This option does not affect replacement values, which are applied exactly as you enter them. By default, this option is enabled, and case is evaluated in a search operation.
- **Save Changes with Replace All** is disabled by default, and when enabled, saves changes made by the **Replace All** operation.
- **Replace All** is disabled by default, and when enabled, selects the **Replace All** option in the **Search and Replace** dialog box.
- **Show Delete**, **Show Replace**, and **Show Clear** are enabled by default, and when disabled, hides the **Delete**, **Replace**, and **Clear** buttons in the **Search and Replace** dialog box.
- **Match Pattern Type** determines if the search operation should use **Exact** matching, glob-style **Wildcard** matching, or if the input should be interpreted as a regular expression (**Reg Expr**). See Searching for Design Components.
- **Partial Match Type** determines if the pattern matching should be partial or full. When disabled, the pattern match is considered to have succeeded only if the pattern matches the entire target string. when enabled, the pattern match is

Feedback

considered to have succeeded if it matches any substring of the target string. See Searching for Design Components.

- **Search root of EIP mode** determines which cellview, either **Edit** (default) or **View**, to use from the parent design editor window as a search root cellview.

Setting the Design Root

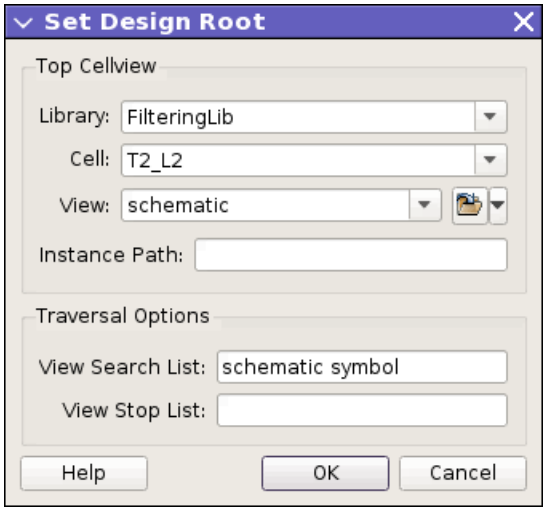
When you display the Search and Replace tool from the **Console** tab, the search scope defaults to the library level. You can set a design root to limit the scope to a single design or design hierarchy for search and replace operations.

If you opened the Search and Replace tool from a design editor, the search scope defaults to the design level. To use a different top design for search and replace operations, set a design root.

To display the Search and Replace window with no design root set automatically, from the **Console** tab choose **Tools > Search and Replace**. From here, you can independently set any scope and search options. You do not need to open a design first.

To set the design root:

1. In the Search and Replace window, choose **File > Design Root**. The **Set Design Root** dialog box opens.



2. Enter the top design (for a new design root) by its library, cell, and view.
3. Enter traversal options to determine the extent of any searches from this top design.
4. Click **OK**.

Feedback

Setting Search Criteria for Attributes and Properties

When searching instances and vias, you can filter objects based on attributes, properties, and value comparisons. All value comparisons are done on the value stored in the database.

To set the search criteria:

1. On the **Attributes** tab, click to add an attribute in the **Attribute Name** column. You can chose an attribute from one of the following: Area, Array, Columns, Delta X, Delta Y, Depth Limit, Fixed, Frozen, Height, Orientation, Origin, Physical Only, Placement Status, Rows, Soft Macro, Usage, or Width.

   Alternatively, on the **Properties** tab, click to add a property in the **Property Name** column.

   (Optional) Use the Tab key to navigator the entries in a row.

2. Enter the operator in the **Operator** column.

   For example, to check to see if a parameter exists on an object, you can use the `=~` operator with a value of `".*"`.

   The following table lists operators for objects evaluated in attribute and property searches.

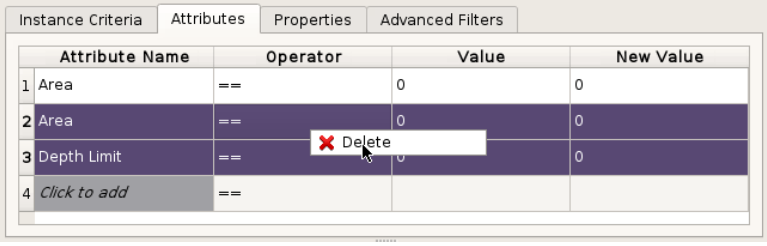
Name	Symbol	Finds design objects where the attribute or property value ...
Equality	==	Is equal to the specified value. By default, when you select an attribute or a property, the operator is set to the default value of `==`.
Inequality	!=	Is not equal to the specified value.
Less Than	<	Is less than the specified value.
Greater Than	>	Value is greater than the specified value.
Less Than or Equal	<=	Value is less than or equal to the specified value.
Greater Than or Equal	>=	Value is greater than or equal to the specified value.
Pattern Match	=~	(**Properties** tab only) Matches the specified regular expression. The pattern matching style is controlled by the preference `dbRegexp`.

Feedback

Name	Symbol	Finds design objects where the attribute or property value ...
Pattern does not match	!~	(**Properties** tab only) Does not match the specified regular expression. The pattern matching style is controlled by the preference `dbRegexp`.
Value does not have a local override	isDefault	(**Properties** tab only) Value is the default value: returns *true*. Note that this returns *false* if the effective value is set by a local value, regardless of whether the value matches the default.
Value does have a local override	isNotDefault	(**Properties** tab only) Effective value is set by a local override: returns *true*.

3. Enter the current attribute or property value you want to find in the **Value** column.
4. (Optional) In the **New Value** column, enter the replacement value if you want this attribute or property value to be replaced.

   Alternatively, you can enter <Delete> in this column to delete the found attribute or property value and reset it to the default value.
5. (Optional) Select one or more rows in the attributes or properties table, right-click and choose **Delete** to remove the selected rows from the table.



Searching for Design Components

You use the Search and Replace tool to find design components. The tool recognizes design components as OpenAccess objects. Therefore, when you perform a search, you are searching for OpenAccess objects.

Using the search results, you can replace attribute values, properties, and parameters associated with objects either individually or as a batch operation.

Feedback

To search for design components:

1. On the **Standard** toolbar in the **Search and Replace** dialog box, choose the **Match** type.

Match Type	Description
Exact	Finds exact matches only.
Reg Expr	Finds patterns that are regular expressions.
Wildcard	Uses wildcard matching that conforms to IEEE standard 1003.1-2001 sections 2.13.1 and 2.13.2 describing patterns to match a single character and multiple characters. The symbol **$** accepts a single character wildcard match.
Partial	Additionally finds any subset of a complete string.
Case Sensitive	Performs all searches in a case-sensitive manner. This option is a shortcut to **Options > Case Sensitive** described in Setting Search and Replace Options.
Replace All	This option is disabled by default. This option is a shortcut to **Options > Replace All** described in Setting Search and Replace Options.

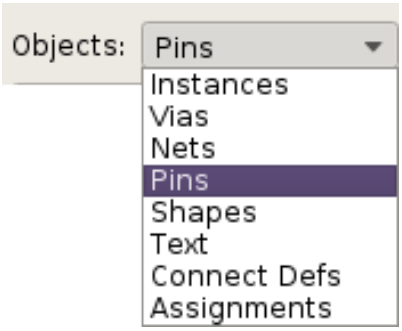
2. Choose **Scope** to control the extent of the search:

Scope	Description
Design	Searches all objects in the open design (offered when you open this feature from a design).
Viewport	Searches all objects in the viewport.
Region	Searches a user-defined polygon or rectangle. When the search scope is **Region**, you choose **Polygon** or **Rectangle** from the toolbar. For polygons, you can select the **Angle** mode. You can also select the search **Criteria** from the toolbar: touch/intersect or enclose. For the intersect selection criteria type, the objects that intersect the specified region are also searched. For the enclose criteria type, only objects that are fully enclosed in the specified region are searched.

Feedback

Scope	Description
Design Hierarchy	Searches the design hierarchy for the open design (offered when you open this feature from a design). When the search scope is **Design** and the hierarchy depth is **Levels**, you have the option of enabling **Show Occurrences** (next to the **Search** button) to view results for hierarchical searches by their occurrences, and you can also set the start/stop levels. See Getting Total Number of Instances Used in a Design.
Selection	Searches the current selection set.
Library	Searches the library you specify here by choosing from the list.
All Libraries Writable Libraries	Searches all available writable libraries. Only writable libraries are selectable in the **Search Scope** list.

3. Choose **Objects** to specify the design component:



4. The object tab may vary depending on the object type you select from **Objects**. Set additional criteria to narrow the search, and provide replacement values.

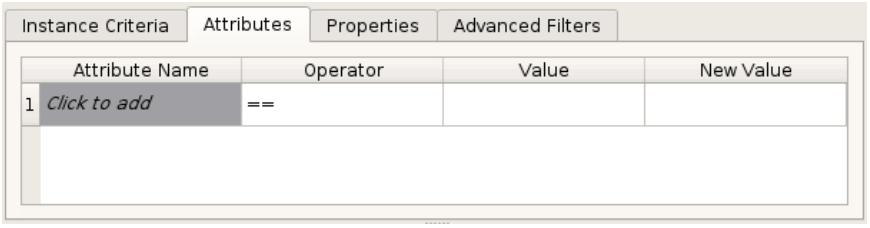
Search Object Type	Description
Instances	In the **Instance Criteria** tab, finds instances by the instance criteria you specify: **Library**, **Cell**, **View**, or **Instance Name**. To search for instances where the **Placed master is unbound** (missing from the technology file) or where a **Replacement master exists**, choose **Advanced Filters**, and specify one or both of these options. To additionally search for vias by their attributes, see Step 5. To additionally search for vias by their properties, see Step 6.

Feedback

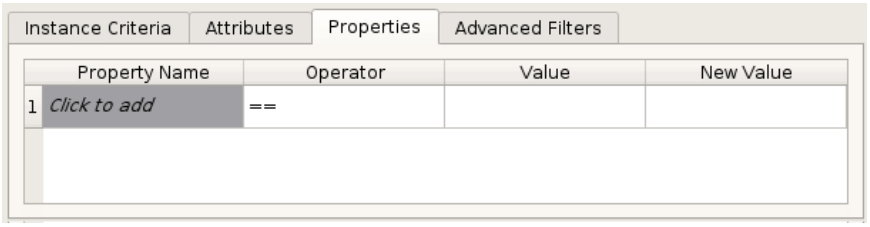
Search Object Type	Description
Instance Arrays	In the **Instance Criteria** tab, finds instance arrays by the **Library**, **Cell**, **View**, or **Instance Name** you specify. To optionally filter results based on object attributes of rows, columns, delta x, or delta y, choose one of these options, specify an operator, and choose a value. You can obtain the object attribute value to enter here by selecting an object, and viewing its attributes in the Property Editor. To include instance arrays where the **Placed master is unbound**, or where a **Replacement master exists**, choose **Advanced Filters**, and select one or both of these options. To additionally search for vias by their attributes, see Step 5. To additionally search for vias by their properties, see Step 6.
Vias	In the **Via Criteria** tab, finds vias by their via definition names or net names. To search for unnetted vias, turn on **Find only unnetted vias**. **Net Name** is disabled when this option is turned on. To replace only vias where the given replacement via definition exists, choose the **Advanced Filters** tab, and specify the **Require replacement via definition exists** option. To additionally search for vias by their attributes, see Step 5. To additionally search for vias by their properties, see Step 6.
Nets	In the **Net attributes** tab, finds nets by net name, and whether or not the net is global. Optionally filters the list of nets by shape or LPP.
Pins	In the **Pin Attributes** tab, finds pins (as pin figures) by their terminal name, terminal type, or net name. Optionally filters by object type and LPP, or by master library, cell, and view.
Shapes	In the **Layout Shape Criteria** tab, finds layout shapes by net name, LPP, and object type. To search for unnetted shapes, turn on **Find only unnetted shapes**. **Net Name** is disabled when this option is turned on.
Text	Finds text objects by text, LPP, font, rotation, justification, and height.
Connection Defs	Finds connection definitions by their names, or the names of the associated default nets.
Assignments	Finds connection assignments by name, a new name you specify here, or net, or visibility.

Feedback

5. In the **Attributes** tab (for instances and vias only), specify attributes for instances, instance arrays, and vias, and provide replacement values. See Setting Search Criteria for Attributes and Properties about operators for objects evaluated in property searches.



6. In the **Properties** tab, specify properties for instances, instance arrays, and vias, and provide replacement values. See Setting Search Criteria for Attributes and Properties about operators for objects evaluated in property searches.



7. (Optional) In the **Advanced Filters** tab, select one or both of the following criteria:
   - **Placed master is unbound**
   - **Replacement master exists**
8. Do one of the following to start the search:
   - To search without making replacements, click the **Search** button.

     (The **Find Objects** button on the toolbar also performs this operation).
   - To search and automatically replace found values, on the toolbar click **Find and replace all objects**.
9. View process status on the Status bar (at the lower window edge).

   (Optional) To halt the search process and retain results found up to this point, press Ctrl+C.

   When the search process finishes, the results appear in the Results area.

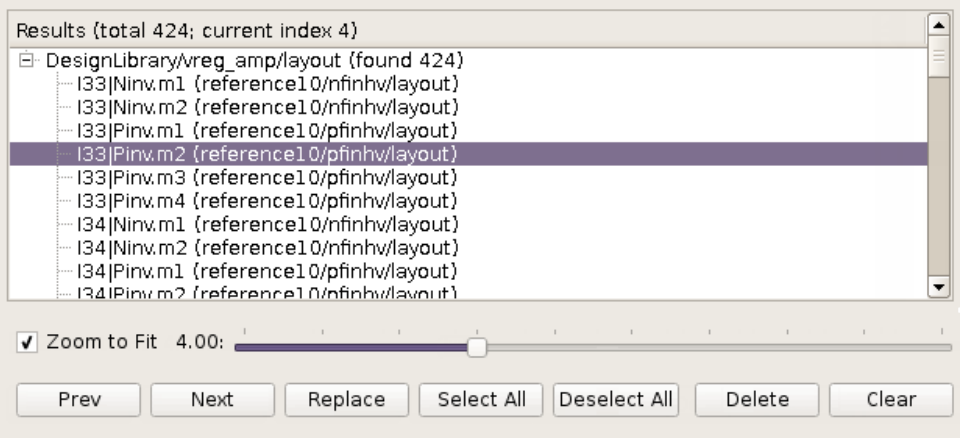
Feedback

Browsing Component Search Results

To browse component search results:

1. In the **Results** browser, select the result you want to view.

   Selecting a result highlights the component on the canvas and, in layouts, also applies the current **Zoom to Fit** setting. For schematics, single-click items to highlight them, and double-click to apply the current **Zoom to Fit** setting.



2. To view the previous result, click **Prev**. To view the next result, click **Next**.
3. (Optional) Set the following viewing options:

Options	Description
Zoom to Fit	Fills the viewport with the image of the selected item, adjusted by the zoom level you specify.
Replace	Replaces existing values with new values for the currently selected items. The tool indicates successful replacement by changing item listings to italic type. For additional information, see Replacing Component Aspects Using Search Results.
Select All	Selects all items in the results. You can use this option to select all objects for batch replacement of any specified attribute values, properties, or parameters. If you set the `dbFindAutoSelectInEditor` Tcl preference to `true`, **Select All** also adds the selected result components to the selection set in the design editor.
Deselect All	Deselects all items in the results.
Delete	Deletes the selected object from the design.

Feedback

Options	Description
Clear	Removes the selected item or items from the search results.
From the **Actions** menu, choose **Add to Selection**	Adds all items in the search results to the selected set, and shows the selection set on the canvas.

4. (Optional) Use one of the following methods to modify the selected design component:

Method	Description
Right-click and choose **Select**	If all selected items are in the same design, selects their design objects in the viewport.
Right-click and choose **Deselect**	If selected items are in the same design, deselects their design objects in the viewport.
Right-click and choose **Collapse**	Collapses expanded tree nodes in the current selection.
Right-click and choose **Expand**	Expands collapsed tree nodes in the current selection.
Right-click and choose **Edit in Place**	Opens the selected design object for editing in place.
Right-click and choose **Fit**	Fits the selected design object to the current viewport.
Right-click and choose **Descend**	If all selected items are in the same design, descends to their location in the hierarchy.
Right-click and choose **Copy > Text**	Copies the text labels for selected items for later pasting into a text file.
Right-click and choose **Copy > Library Name**	Copies the library name attribute.
Right-click and choose **Copy > Cell Name**	Copies the cell name attribute.
Right-click and choose **Copy > View Name**	Copies the view name attribute.

Refining Component Search Results

You can refine component search results to find a subset of items within an existing set of search results.

Feedback

To refine search results:

1. Obtain search results by Searching for Design Components.
2. At the top of the Search and Results tab or dialog box, adjust search criteria values for searching within these results.

   For example, you can refine a search you made previously for text labels on LPPs (see Searching for Design Components) to now locate individual LPPs in the search results by layout shape. To do this, change the search type to **Shapes**, select **Any LPP** from the list, and choose **text** as the **Object Type**.
3. On the toolbar, click the **Refine Current Result** button, or choose **Actions > Refine Result**.

   The tool performs a new search within the existing search results, and lists a new set of results.

Checking Whether a Property Exists on an Instance

You can use the Search and Replace tool to determine whether a property exists on an instance.

The example that follows illustrates this process.

Assume that you have a design that contains instances with the model property, and you want to find all the instances having that property.

1. In the **Properties** tab, define the **Property Name** as *model*.
2. Define the **Operator** as =~. (Setting Search Criteria for Attributes and Properties lists operators for objects evaluated in property searches).
3. Use *.** for the **Value**.
4. Click **Search**.
5. In the **Results** browser, review the list of found instances.

Finding Instances With the Default Value of a Given Property

You can use the Search and Replace tool to find instances that have the given property set at its default value.

The example that follows illustrates this process.

Assume that you have a design that contains instances with the wtot property, and you want to find all the instances having the default value of that property.

Feedback

1. In the **Properties** tab, define the **Property Name** as *wtot*.
2. Define the **Operator** as *isDefault*. (Setting Search Criteria for Attributes and Properties lists operators for objects evaluated in property searches).

   Alternatively, if you want to search for instances having a non-default value of a given property, define the **Operator** as *isNotDefault*.
3. Click **Search**.
4. In the **Results** browser, review the list of found instances.

Finding Instances With a Specific Property Value

You can use the Search and Replace tool to find instances that have the given property set at a specific value, or greater than or less than a specific value.

The example that follows illustrates this process.

Assume that you want to find all instances named n_4t2 from the library ref9 that have a w property greater than 1u.

1. In the **Instance Criteria** tab, define the **Library Name** as *ref9* and the **Cell Name** as *n_4t2*.
2. In the **Properties** tab, define the **Property Name** as *w*.
3. Define the **Operator** as >=. (Setting Search Criteria for Attributes and Properties lists operators for objects evaluated in property searches).
4. Use *1u* for the **Value**.
5. Click **Search**.
6. In the **Results** browser, review the list of found instances.

Finding Instances With a Property Not Equal to a Specific Value

You can use the Search and Replace tool to find instances with a property that is not set at a specific value.

The example that follows illustrates this process.

Assume that you want to find all instances named n_4t2 from the library ref9 that have a w property not equal to 3u.

1. In the **Instance Criteria** tab, define the **Library Name** as *ref9* and the **Cell Name** as *n_4t2*.
2. In the **Properties** tab, define the **Property Name** as *w*.

Feedback

3. Define the **Operator** as *!=*. (Setting Search Criteria for Attributes and Properties lists operators for objects evaluated in property searches).
4. Use *3u* for the **Value**.
5. Click **Search**.
6. In the **Results** browser, review the list of found instances.

Finding Instances With a Property Value in a Given Range

You can use the Search and Replace tool to find instances that have the given property within a range of values by defining more than one search criteria for that property.

The example that follows illustrates this process.

Assume that you want to find all instances named n_4t2 from the library ref9 that have a w property between 0.5u and 3.5u.

1. In the **Instance Criteria** tab, define the **Library Name** as *ref9* and the **Cell Name** as *n_4t2*.
2. In the **Properties** tab, define the **Property Name** as *w*.
3. Define the **Operator** as >=. (Setting Search Criteria for Attributes and Properties lists operators for objects evaluated in property searches.)
4. Use *0.5u* for the **Value**.
5. Click row 2 to add another search criteria.
6. Define the **Property Name** as *w* again.
7. Define the **Operator** as <=.
8. Use *3.5u* for the **Value**.
9. Click **Search**.
10. In the **Results** browser, review the list of found instances.

Finding Paths or Path Segments in Layouts

You can find paths or path segments (pathSegs) on layer shapes in layouts by filtering their width attribute or property.

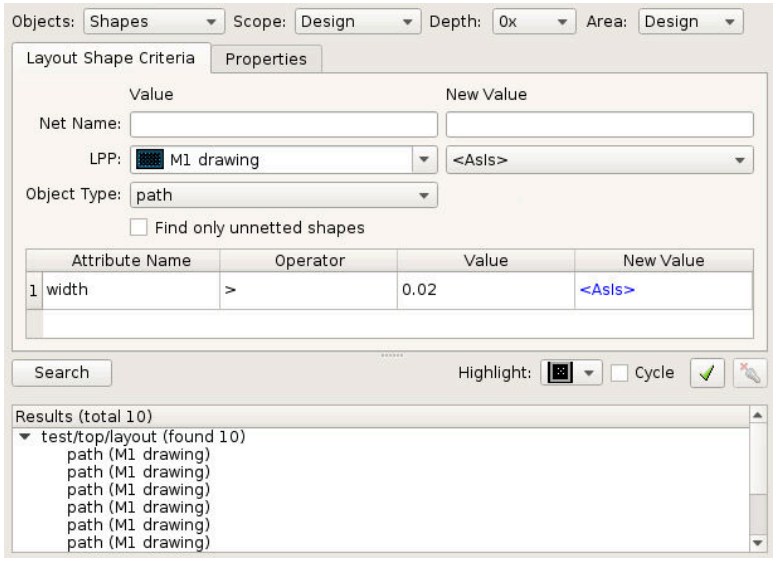
The example that follows illustrates this process.

Assume that you want to find paths or path segments on LPP m1 drawing.

Feedback

1. In the **Layout Shape Criteria** tab, use the search criteria that follows:
   - LPP: **m1 drawing**
   - Object Type: **path**
2. Define the **Operator** as > for the width attribute. (Setting Search Criteria for Attributes and Properties lists operators for objects evaluated in attribute searches).
3. Enter *0.02* for the **Value**.

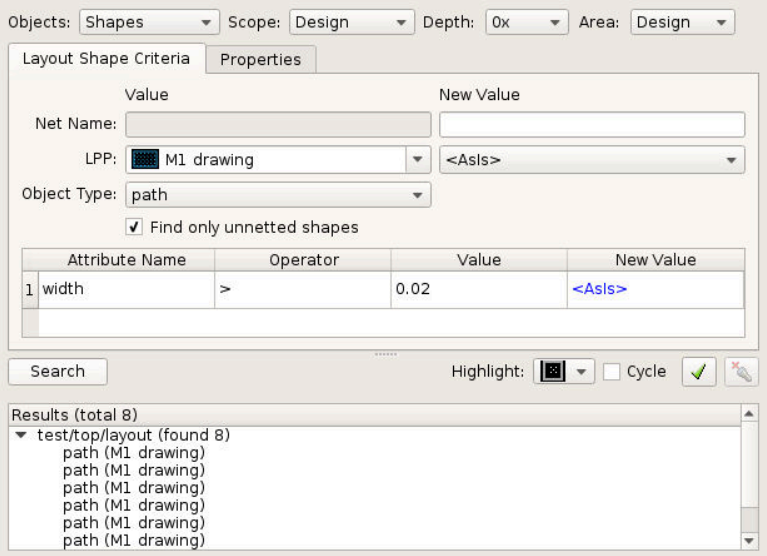
   Alternatively, if you want to search for all paths or path segments in the layout, leave **Value** empty.
4. Click **Search**.
5. In the **Results** browser, review the list of found paths or path segments.



6. (Optional) Enable **Find only unnetted shapes** to search for paths without net name. **Net Name** is disabled when this option is turned on.

   Click **Search**.

   In the **Results** browser, review the list of found paths or path segments.



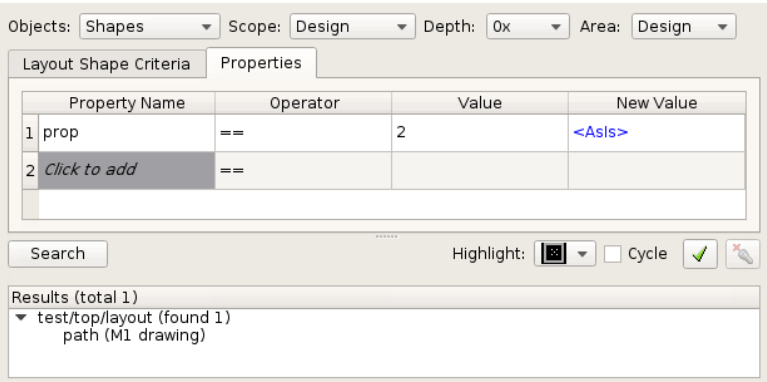
7. (Optional) Switch to the **Properties** tab.

   Define the **Property Name** as *prop*.

   Enter *2* for the **Value**.

   Click **Search**.

   In the **Results** browser, review the list of found paths or path segments.



Finding Text on Layer Shapes in Layouts

You can find text on layer shapes in layouts by searching for text display (textDisplay) objects. This is useful when you have added text to pins to identify the pins to integrated tools.

The example that follows illustrates this process.

Feedback

Assume that you want to find text on any LPP whose layer name begins with the letter m.

To do this, you use the search criteria that follows:

Directly enter *m** into the **LPP** box instead of using the list box.

- Match: **Wildcard**
- Search Objects: **Shapes**
- LPP: **m***
- Object Type: **textDisplay**

As a variation, you can find text on any LPP whose layer name begins with the letter m, and whose purpose name begins with the letter p.

To do this, use the search criteria that follows:

Directly enter *m* p** into the **LPP** box instead of using the list box.

- Match: **Wildcard**
- Search Objects: **Shapes**
- LPP: **m* p***
- Object Type: **textDisplay**

Finding Text Objects in Layouts

You can find text objects within a specified design scope in layouts.

The example that follows illustrates this process.

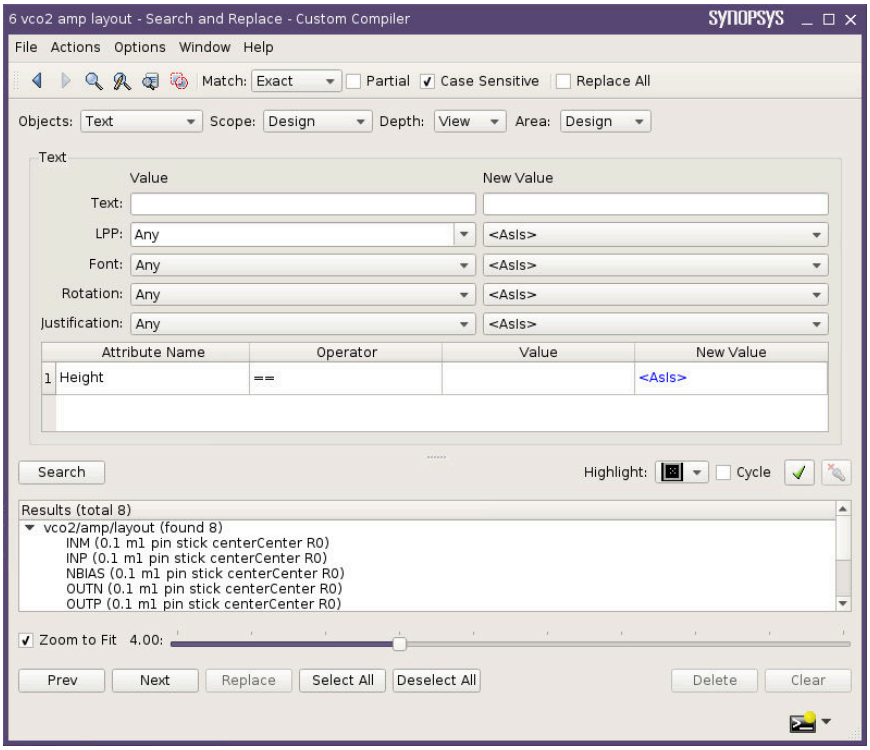
Assume that you want to find text objects within the current design.

To do this, you use the search criteria that follows:

- Match: **Exact**
- Search Objects: **Text**
- Scope: **Design**
- Depth: **View**
- Area: **Design**

After clicking **Search**, the **Results** pane shows the matching text objects with information such as font height, LPP, font, justification, and orientation.

Feedback



Finding Vectored Instances

You can use the Search and Replace tool to find vectored instances.

The example that follows illustrates this process.

Assume that you have a design that contains instances with these names, and you want to find various vectored instances within it:

- R3
- M17<0.4> (generic9/p_4t2/symbol)M18<0.4> (generic90RF/p_4t/symbol)
- M27<0.4> (generic9/p_4t2/symbol)
- M34 (generic9/p_4t2/symbol)

To search for vectored instances (example):

1. (Optional) Open the design.

   (You can perform search-and-replace operations with no design open. To do this, start by setting the design root to the design you want to search. See Setting the Design Root).

Feedback

2. Open the Search and Replace tool.

   From the Console, choose **Tools > Search and Replace**. Alternatively, from the Library Manager, choose **Edit > Search and Replace**.

3. Input search criteria as shown in the following table.

Search to Find	Input	Returns	Match Type
Instance with this name	R3	R3	Exact
Vectored Instances	M.+<.+>	M17<0.4> (generic9/p_4t2/symbol) M18<0.4> (generic9/p_4t2/symbol) M27<0.4> (generic9/p_4t2/symbol)	Regular expression
Instances with M and 7 separated by any character	M*7	M17<0.4> (generic9/p_4t2/symbol) M27<0.4> (generic9/p_4t2/symbol)	Wildcard
Instances whose names begin with M followed by any characters	M$	M17<0.4> (generic9/p_4t2/symbol) M18<0.4> (generic9/p_4t2/symbol) M27<0.4> (generic9/p_4t2/symbol) M34 (generic9/p_4t2/symbol)	Single Character Wildcard

4. Click **Search**.

5. In the **Results** browser, review the list of found instances.

Finding Via Vector Values

You can use the Search and Replace tool to find via vector values, and additionally replace them with new values. An example illustrates this process.

The example that follows illustrates this process.

Assume that you want to search for an oaVia with a cutSpacing value of (0.2, 0.3) and replace it with a cutSpacing value of (0.4, 0.5). For a successful search to find this two-value search entry, the search criteria notation must include the parentheses.

When you use this process in practice, open the Property Editor to identify values for input criteria.

To search for, and replace, an oaVia vector value:

1. (Optional) Open the design.

   (You can perform search-and-replace operations with no design open).

Feedback

2. Open the Search and Replace tool. From the Console, choose **Tools > Search and Replace**. Alternatively, from the Library Manager, choose **Edit > Search and Replace**.
3. Specify a search scope of **Design**.
4. Specify a search type of **Vias**.
5. On the **Via Criteria** tab, choose a via definition value of **via12**.
6. On the **Properties** tab, enter search criteria.
   a. Enter the property name **cutSpacing**.
   b. Accept the default operator of ==.
   c. Enter a **Value** of *(0.2, 0.3)*, including the parentheses.
   d. Enter a **New Value** of *(0.4, 0.5)*, including the parentheses.
7. Click **Search**.
8. In the **Results** browser, select the via whose vector value you want to replace. (If you searched for only one oaVia, this is the only via listed).
9. Click **Replace**.

   The tool replaces the cutSpacing value for this oaVia.

Finding Vias in Layouts

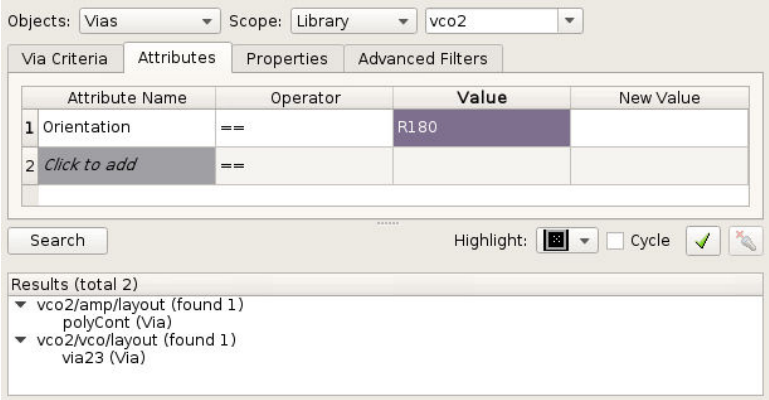
You can find vias in layouts by filtering their attributes and other options.

The example that follows illustrates this process.

1. In the **Via Criteria** tab, leave each option as default.

   (Optional) Specify the via type by **All**, **ViaDefs**, or **ViaVariants**.
2. In the **Attributes** tab, define the **Operator** as == for the Orientation attribute. (Setting Search Criteria for Attributes and Properties lists operators for objects evaluated in attribute searches).
3. Choose **R180** for the **Value**.
4. Click **Search**.
5. In the **Results** browser, review the list of found vias.

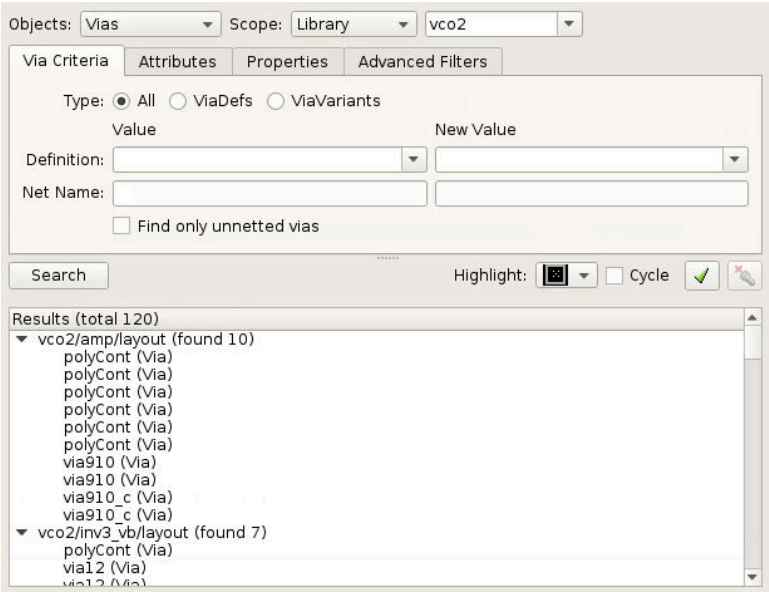
Feedback



6. (Optional) Delete the Orientation attribute. Switch to the **Via Criteria** tab and enable **Find only unnetted vias** to search for vias without net name. **Net Name** is disabled when this option is turned on.

   Click **Search**.

   In the **Results** browser, review the list of found vias.



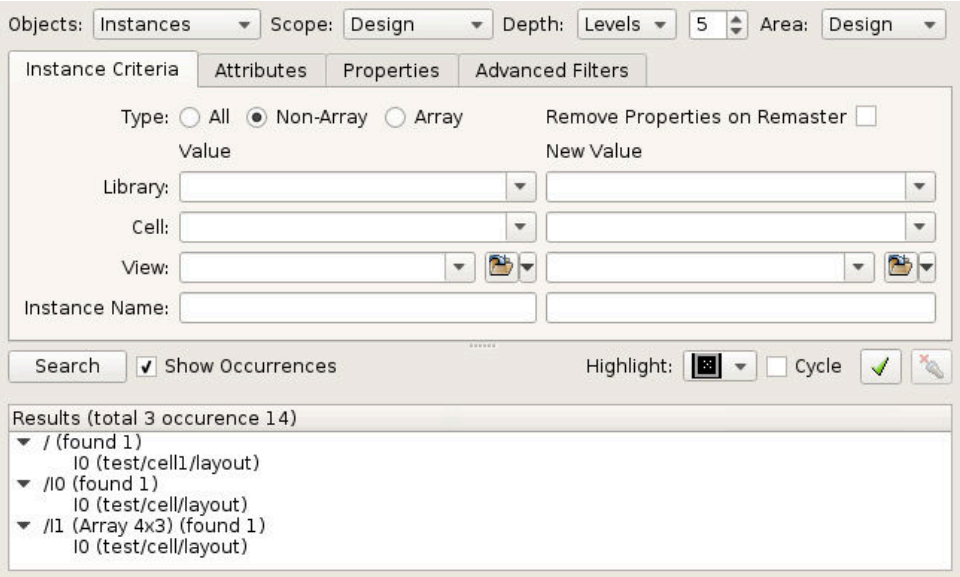
Getting Total Number of Instances Used in a Design

By turning on the **Show Occurrences** option, you can show the total number of instances used in a design.

Feedback

To get the number of instances used in a design:

1. Set up the search by Searching for Design Components.
2. Specify a search scope of **Design**.
3. Specify a search type of **Instances**.
4. Specify a hierarchy depth of **Levels**.
5. Specify a stop level at **5**.
6. Specify a search area of **Design**.
7. Turn on **Show Occurrences** and then click **Search**.
8. In the **Results** browser, review the list of found instances in your design.



Replacing Component Aspects Using Search Results

You can replace attribute values, properties, and parameters. You can make replacements while performing a search, or you can review the search results, and then make replacements.

To replace component aspects using search results:

1. Set up the search by Searching for Design Components.
2. In the **Instance Criteria** and **Properties** tabs, review the values available for replacement.

Feedback

3. In the **New Value** column, enter the replacement values.
4. Perform replacement operations:
   - To replace values for currently selected objects, click **Replace**.
   - To replace values as a batch operation for all items in this scope, click **Select All** and then click **Replace**.

   The tool indicates successful replacement by changing item listings to italic type.
5. (Optional) Save all designs modified by a replacement operation (including designs not currently shown) by choosing **File > Save**.

Replacing Scientific Notation With Equivalent Value

You can use the Search and Replace tool to replace regular expressions with Perl-style substitution formats. The grouping metacharacters `()` are supported, allowing for the extraction of the parts of a string that match. For each grouping, the part that matches inside the `()` goes into the special variables `$1`, `$2` , and so forth. For example, the pattern `(\S+)Lib$` can be used to match `analogLib` or `myLib`. Using the substitution pattern `$1Lib2` would produce `analogLib2` and `myLib2` respectively.

The example that follows illustrates this process.

Assume that you have a design that contains instances with the w property, and you want to find all the instances of that property having `u` at the end, and replace that value with `10e-6`.

1. In the **Properties** tab, define the **Property Name** as *w*.
2. Define the **Operator** as =~. (Setting Search Criteria for Attributes and Properties lists operators for objects evaluated in property searches).
3. Use *(.+)u* for the **Value**.
4. Use *$1e-6* for the **New Value**.
5. Click **Search**.
6. In the **Results** browser, review the list of found instances in which the value has been replaced as specified.

Feedback

Replacing Property Values

This topic describes the following:

- Replacing Values Within Search Results
- Replacing Values With Value Multiplied by a Constant

Replacing Values Within Search Results

You can enter multiple search criteria in each tab to specify what to search, and then what to replace in the results of that search.

The example that follows illustrates this process.

Assume that you want to find all instances named n_4t2 from the library ref9 that have a *w* property greater than 2u. Then, within that group, you want to replace any instance of the *nf* property with the number 4.

1. In the **Instance Criteria** tab, define the **Library Name** as *ref9* and the **Cell Name** as *n_4t2*.
2. In the **Properties** tab, define the **Property Name** as *w*.
3. Define the **Operator** as >= (greater than or equal to). (Setting Search Criteria for Attributes and Properties lists operators for objects evaluated in property searches).
4. Enter *2u* for the **Value**.
5. Click row 2 to add another search criteria.
6. Define the **Property Name** as *nf*.
7. Define the **Operator** as =~.
8. Enter *.** for the **Value**.
9. Enter *4* for the **New Value**.
10. Click **Search**.
11. In the **Results** browser, review the list of found instances.

Replacing Values With Value Multiplied by a Constant

You can also replace the value of a property with its current value multiplied by a constant.

The example that follows illustrates this process.

Assume that you want to find all instances named p_4t2 from the library ref9 that have a w property less than or equal to 3u, then replace that with the value multiplied by 4.

Feedback

1. In the **Instance Criteria** tab, define the **Library Name** as *ref9* and the **Cell Name** as *p_4t2*.
2. In the **Properties** tab, define the **Property Name** as *w*.
3. Define the **Operator** as <=.
4. Enter *3u* for the **Value**.
5. Enter *3*$w* for the **New Value**.
6. Click **Search**.
7. In the **Results Browser**, review the list of found instances.

Technology Manager Tool

Manage technologies through the Custom Compiler Technology Manager. Use the Technology Manager to create design library access to the correct technology database for the current design objective. The technology database is located either inside the design library or referenced via inheritance through another library that has the technology you want to use. If referenced, the referenced library must exist first, otherwise, the technology is considered unbound.

This topic describes the following:

- Displaying the Technology Manager Tool
- Considering Technology Management Options
- Changing the Technology Referenced by a Design Library
- Importing Technology Data
- Exporting Technology Data
- Adding a Reference
- Deleting a Reference
- Saving a Modified Technology Database
- Reverting Changes to a Modified Technology Database

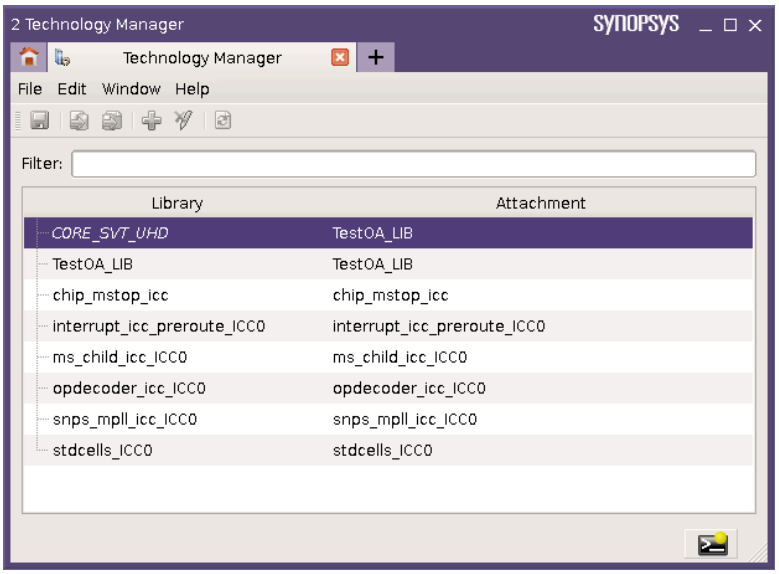
Feedback

Displaying the Technology Manager Tool

You can display the Technology Manager using one of the following methods:

- From the **Console** tab, choose **Tools > Technology Manager**.
- From the Library Manager, choose **Edit > Technology**.

The Technology Manager appears and displays the list of libraries.



The Technology Manager will also list read-only libraries. However, for read-only libraries:

- The **File** menu commands will be disabled to prevent you from importing, exporting, reverting, and saving changes.
- The **Edit** menu commands will be disabled to prevent you from adding references and deleting references. If only a parent reference is read-only, then the commands are disabled for the specific parent reference only.
- User-interface operations, such as drag-and-drop actions, will be disabled.
- The **Attachment** selection is disabled for the library.

Considering Technology Management Options

Consider your options given these possible situations:

- If you have a technology file (without a techArray), you can create a library that uses this technology. In this case, use the Library Manager to create the library. Then use the Technology Manager to either compile the technology file into it, or to reference

Feedback

another library with the technology you want to use. Alternatively, you can attach the available default technology file. For all technology options, use the Technology Manager. See Importing Technology Data.

- If you have a technology file that includes a techArray, the techArray defines the required technology libraries to reference and these libraries need to now exist. In this case, you can use the Technology Manager to import the technology from the technology file. See Importing Technology Data.
- If you have a library, you can import technology into it. Additionally, if you have manually edited a technology file, you can import it into a new or existing library, creating a technology database for that library that matches the technology described in the file. Use the Technology Manager. See Importing Technology Data.
- From any library with attached technology data, you can export the technology data for use in additional libraries or to edit for customizing. Use the Technology Manager. See Exporting Technology Data.

Changing the Technology Referenced by a Design Library

You can change the technology referenced by a library to use a different technology for the library.

If the technology was compiled into the library, the referenced library name and the design library name are shown as matching in the Technology Manager.

1. Begin by opening the Technology Manager. From the **Console** tab, choose **Tools > Technology Manager**. Alternatively, from the Library Manager, choose **Edit > Technology**.
2. In the Technology Manager, locate the design library by name, click the technology library currently referenced, and choose a different library from the list.

   If a referenced library is read-only, the list does not appear.

   Libraries whose referenced technology has been changed appear in italic type. Libraries associated with incremental technology are identified by the "(i)" notation after the reference name.

To reverse this operation, repeat Step 2, and choose the previously used technology library.

Importing Technology Data

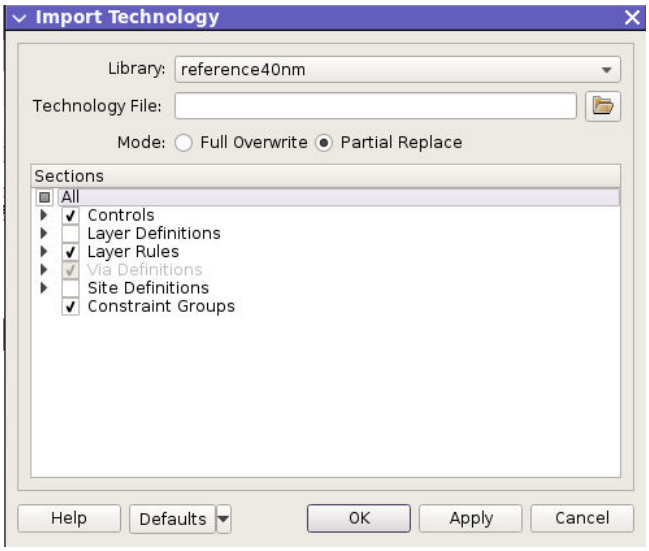
You can import technology data to add a technology database where one did not exist before, or to overwrite or merge imported data by compiling it into an existing technology database.

Feedback

To import technology data:

1. Begin by opening the Technology Manager. From the **Console** tab, choose **Tools > Technology Manager**. Alternatively, from the Library Manager, choose **Edit > Technology**.

2. In the Technology Manager, choose **File > Import** or click **Import** in the toolbar. The **Import Technology** dialog box opens.



Note:

For read-only libraries, the **File > Import** command is disabled to prevent you from modifying the library.

3. Verify that the correct technology **Library** is chosen. The imported technology data will be stored in the chosen technology library.

4. Browse to find the technology file whose technology you want to import.

5. Choose the import mode:

   To replace the currently defined technology for this library (effectively purging the library's technology) with the technology in the file you are importing, choose **Full Overwrite**.

   To partially replace the currently defined technology for this library with the technology in the file you are importing, choose **Partial Replace** and select the files to replace.

6. Click **OK**.

Feedback

Libraries with names in bold letters signify those with changed technologies. You can choose **File > Revert** to reverse any import operation not yet saved. This enables you to change technologies for this session only.

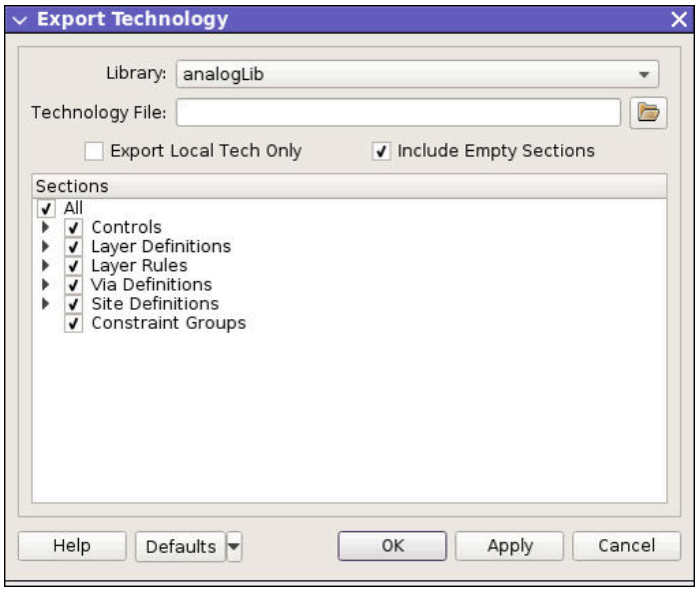
7. Click **Save** to save the imported technology file to disk.

Exporting Technology Data

You can export technology data to transfer the data from a library to an existing or new technology file. You can use this process to back up your technology database or to give someone else a copy of the technology you use. You can also use this process when you want to manually edit the technology file for customizing.

1. Begin by opening the Technology Manager. From the **Console** tab, choose **Tools > Technology Manager**. Alternatively, from the Library Manager, choose **Edit > Technology**.

2. In the Technology Manager, choose **File > Export** or click **Export** in the toolbar. The **Export Technology** dialog box opens.



Note:

For read-only libraries, the **File > Export** command is disabled.

3. Choose the technology **Library** from which you want to export the technology data.

4. Enter a technology file name or browse to find the technology file to receive the exported technology.

Feedback

5. Select **Export Local Tech Only** to limit the data exported from the library to newly created, incremental technology data. Do not select this option if you want to write all technology from the library to the file. See Export Incremental Technology Data.
6. Select **Include Empty Sections** if you want the exported file to contain comment lines that show the syntax of sections even if they do not contain any data. If you decide to use the sections later, you can type data directly in the file without researching the syntax.
7. Click **OK**.

Adding a Reference

You can add a technology database reference into the selected technology database.

1. Begin by opening the Technology Manager. From the **Console** tab, choose **Tools > Technology Manager**. Alternatively, from the Library Manager, choose **Edit > Technology**.
2. In the Technology Manager, select a library item.
3. Click **Add Reference** in the toolbar. A drop-down list of valid technology databases appears under the selected library item.
4. Click a technology database in the list to add it as a reference.
5. Alternatively, you can drag library item from the **Library** list and drop it into another library item to create a new sub-item. The technology database of the dragged library will be referenced to the technology database of the dropped library.

Deleting a Reference

You can delete a technology database reference from the selected technology database.

1. Begin by opening the Technology Manager. From the **Console** tab, choose **Tools > Technology Manager**. Alternatively, from the Library Manager, choose **Edit > Technology**.
2. In the Technology Manager, select a library item.
3. Click **Delete Reference** in the toolbar.

Feedback

Saving a Modified Technology Database

You can save a modified technology database.

1. Begin by opening the Technology Manager. From the **Console** tab, choose **Tools > Technology Manager**. Alternatively, from the Library Manager, choose **Edit > Technology**.
2. Add or delete references, as described in Adding a Reference and Deleting a Reference.
3. Click **Save** in the toolbar.

Reverting Changes to a Modified Technology Database

You can revert any changes you made to a technology database and have not saved.

1. Begin by opening the Technology Manager. From the **Console** tab, choose **Tools > Technology Manager**. Alternatively, from the Library Manager, choose **Edit > Technology**.
2. Add or delete references, as described in Adding a Reference and Deleting a Reference.
3. Click **Revert** in the toolbar.

Text Viewer Tool

The Text Viewer tool displays and dynamically updates text output for all currently running job processes from the environment. (If you are looking for information about text views of designs, and not text views of job processes, see instead Editing HDL Formats With the Text Editor in the *Custom Compiler Schematic Editor and Text Editor User Guide*).

This topic describes the following:

- Displaying the Text Viewer
- Viewing Job Process Text
- Printing Files From the Text Viewer

Feedback

Displaying the Text Viewer

The Text Viewer opens automatically for many processes. You can also open the Text Viewer any time you want to view process output that generates text, such as log files, and other text files.

- To display the Text Viewer from the Console, execute the Tcl command `xt::openTextViewer` at the command input line.
- To display the Text Viewer to view the output files of a running job, open the Job Monitor, select the job, and then choose **Job > View Output**. Alternatively, double-click on a job's status message, such as FAILED or FINISHED.

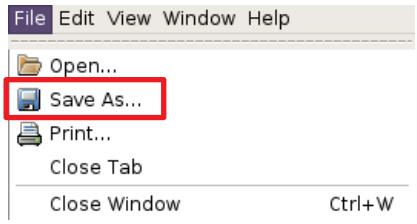
Viewing Job Process Text

You can view text output for all job processes with the Text Viewer.

If it is not already open, begin by Displaying the Text Viewer.

In the Text Viewer, choose an option:

- To view different files currently opened in the text viewer, choose between the tabs by name.
- To open a text file for viewing, choose **File > Open**, and browse to choose a file.
- To scroll to the end of a file, use the keyboard end key.
- To display line numbers in the text file, select **View > Line Number**
- To save a text file with a different name, open it, and then choose **File > Save As**.



- To enable searching, choose **Edit > Find**, and enter text and search options at the bottom of the window.

- To switch off the polling function, choose **View > Poll**.

  By default, the tool polls for new processes generating text output, and dynamically updates the text as it outputs.

Feedback

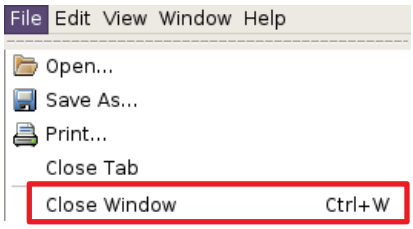
- To view Console command output in the Text Viewer, you can generate the output, and redirect it.

  For example, this statement redirects a Console Help request for `gi::createBinding` help to a file, and opens it in the Text Viewer:

  ```
  xt::openTextViewer -text [gi::createBinding -help]
  ```

  From here, you can save the output as a file.

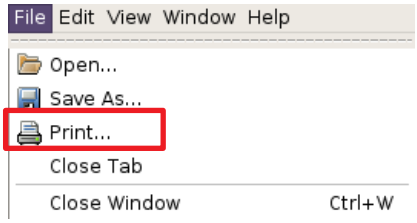
- To close the Text Viewer, choose **File > Close Window**.



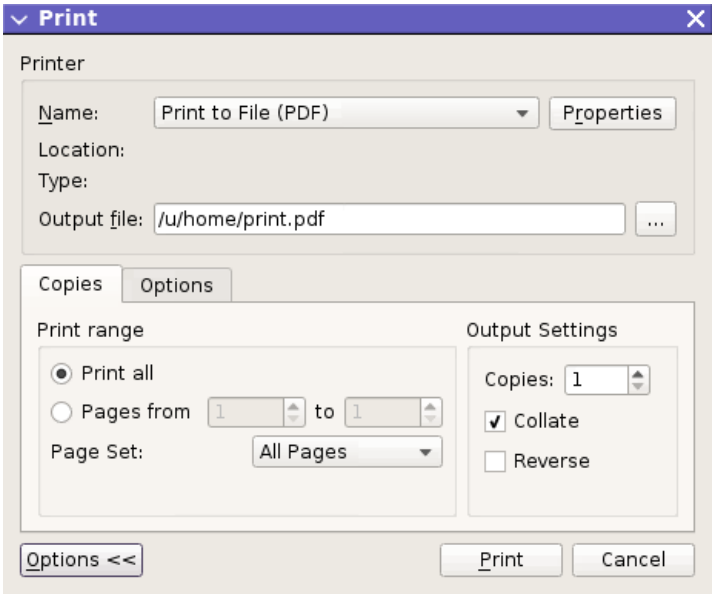
Printing Files From the Text Viewer

You can print the text of an output file to a PDF from the Text Viewer.

1. Choose **File > Print**.



   The **Print** dialog box opens.



2. Select your print options and click **Print**.

Design Technology Co-optimization Tool

This topic describes the following:

- About the Design Technology Co-optimization
- Workflow for Design Technology Co-optimization
- Preparing for Design Technology Co-optimization
- Setting Up the DTCO Parameters
- Running Transistor Modeling
- Running Interconnect Characterization
- Running Cell Parasitic Extraction
- Running Digital Implementation
- Creating a Split
- Comparing Split Results
- Monitoring Computations in Job Monitor

Feedback

About the Design Technology Co-optimization

The Design Technology Co-optimization (DTCO) is a feature to aim at a team of engineers who work on making technology and design choices to improve overall design metrics typically during modifications to a process, at the start of new node or a derivative of an existing node. The DTCO solution is a set of features that can be used to define a flow for evaluating choices in process technology, standard cell architecture, and their impact on design. This feature:

- Helps a team assess the impact of the choices on design by providing a unified environment for experimentation and validation.
- Smartly manages the preferences and settings for complex designs.
- Creates and manages multiple process splits for the design with the design collaterals and sample design data to compare the splits for designers.
- Examines individual purpose from full DTCO flow.

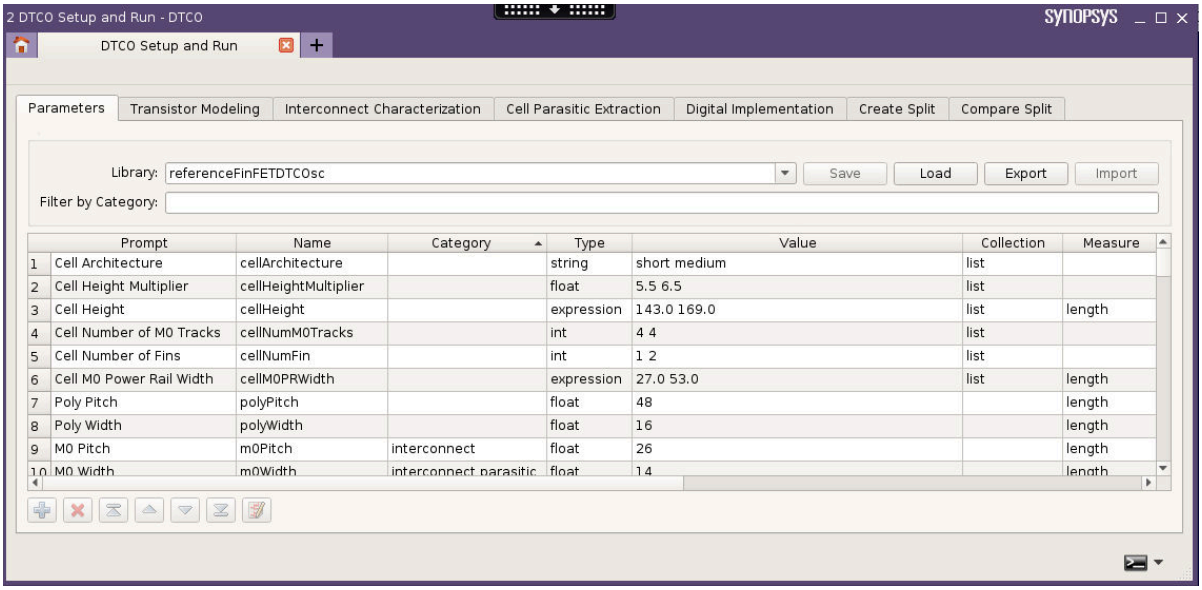
The Design Technology Co-optimization (DTCO) flow provides multiple tools to let you conducts repetitive process experiments in a semi-automated way.

Feedback

Workflow for Design Technology Co-optimization

Before you start the Design Technology Co-optimization (DTCO), see Preparing for Design Technology Co-optimization.

After the preparations, proceed as follows from the **DTCO Setup and Run** dialog box:



Step		See...
1	Specify **Parameters**.	Setting Up the DTCO Parameters
2	Specify and run **Transistor Modeling** for baseline computations.	Running Transistor Modeling
3	Specify and run **Interconnect Characterization** for baseline computations.	Running Interconnect Characterization
4	Specify and run **Cell Parasitic Extraction** for baseline computations.	Running Cell Parasitic Extraction
5	Specify and run **Digital Implementation** for baseline computations.	Running Digital Implementation
6	After specifying the baseline computations, design new parameters from original data by using **Create Split**.	Creating a Split
7	Perform computations in Step 2 to 5 for the split.	

Feedback

Step		See...
8	After getting several designs, run **Compare Split** to find the differences.	Comparing Split Results

Feedback

Preparing for Design Technology Co-optimization

The software tool requirements are:

Tool	Version
Custom Compiler	U-2023.03
SPX	2020.09
HSPICE	2020.12-SP2-2
Sentaurus	2020.09a
StarRC	2019.12-SP4
Raphael FX	2020.09a
Fusion Compiler	2020.09-SP6
IC WorkBench Edit/View Plus	2019.12-SP1

Before running the DTCO, load the tools if needed. For example:

```
module load spx/2020.09
module load hspice/2020.12-SP2-2
module load sentaurus/2020.09a
module load starrc/2019.12-SP4
module load raphael_fx/2020.09a
module load fc/2020.09-SP6
module load icwbev_plus/2019.12-SP1
```

See SolvNetPlus article #2256166 Custom Compiler Tools Compatibility Table.

The **DTCO** > **Setup and Run** option is available under the **Tools** menu when the environment is set correctly.

Feedback

1 Home - Custom Compiler
SYNOPSYS
File
Tools
License
Options
Window
Help
Library Manager
Technology Manager
Parameter Definition Editor
Bookmark Manager
Search and Replace...
Display Resource Editor
Job Monitor
Console
PrimeWave
PrimeSim EMIR...
Shift+V
DTCO
Setup and Run
piler™
Library Manager
PrimeWave Design Environment
WaveView
Search & Replace
Job Monitor
Console
Help
What's New
Design Reviews
DTCO Setup & Run
Recent Designs
Setup and Run

Feedback

Setting Up the DTCO Parameters

You can use the **Parameters** tab to display, define, import and export values for DTCO parameters.

All DTCO parameters are stored with the library in its data.dm file. If a parameter affects the layout of any cell in the library, then that becomes a CDF parameter. A DTCO parameter can be dependent on one or more DTCO parameters. This dependency is captured by a Tcl expression sting as the value and has a Type called `expression`. A DTCO parameter is independent if the value is not a Tcl expression string and a Type other than `expression`.

This topic describes the following:

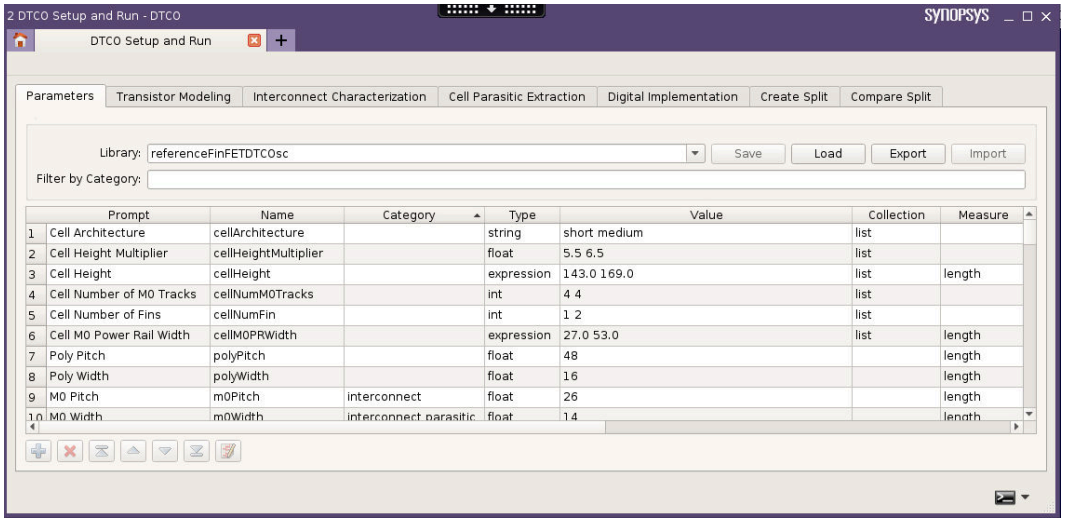
- Displaying and Filtering the DTCO Parameters
- Adding and Editing DTCO Parameters
- Exporting and Importing DTCO Parameters
- Loading DTCO Parameters
- Saving DTCO Parameters

Displaying and Filtering the DTCO Parameters

You can view the DTCO parameters in the **Parameters** tab to begin to work with them.

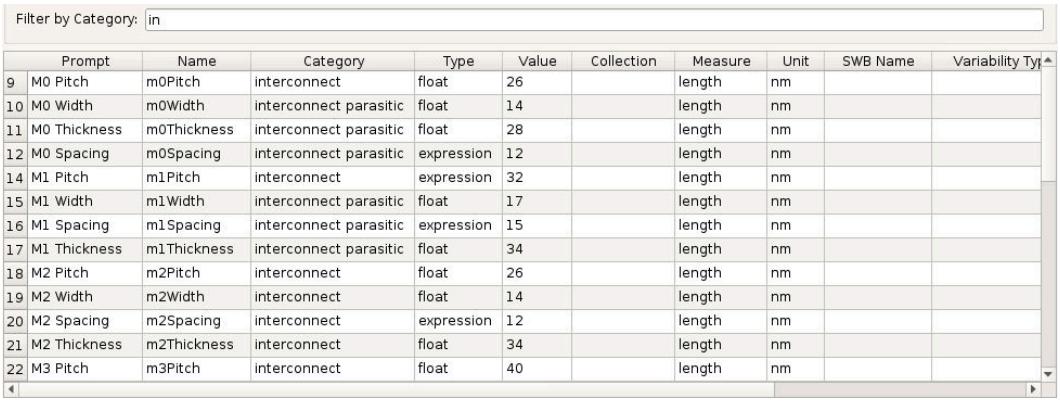
To view parameters:

1. In the **Parameters** tab, choose library in the **Library** selection field. In a DTCO flow, all the DTCO parameters are defined, created, and stored in the reference library.
2. Choose **Import** (Exporting and Importing DTCO Parameters) or **Load** (Loading DTCO Parameters) to import the stored parameters.
3. All the parameters associated with the library are displayed in the parameters table.



The parameters table shows the following parameter information: **Prompt**, **Name**, **Category**, **Type**, **Value**, **Collection**, **Measure**, **Unit**, **SWB Name**, **Variability Type**, **Var Value**, and **Layout Parameter Name**. To edit the parameter value, see Adding and Editing DTCO Parameters.

You can filter the parameters table by **Category** column. To filter the parameters table, specify the string in the **Filter by Category** text field. After specifying the string in the text field, the tool lists the DTCO parameters that match the filter criteria.

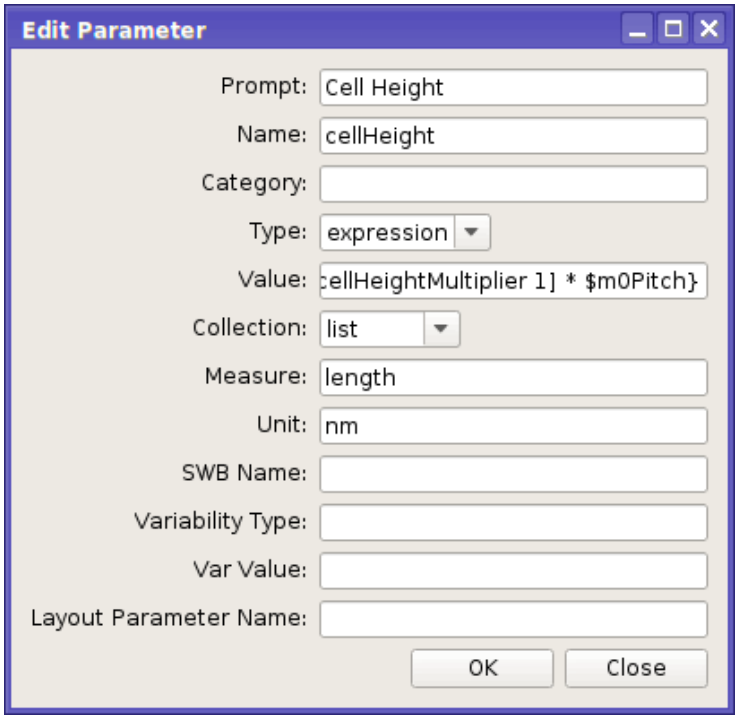


Feedback

Adding and Editing DTCO Parameters

To edit DTCO parameters in the parameters table:

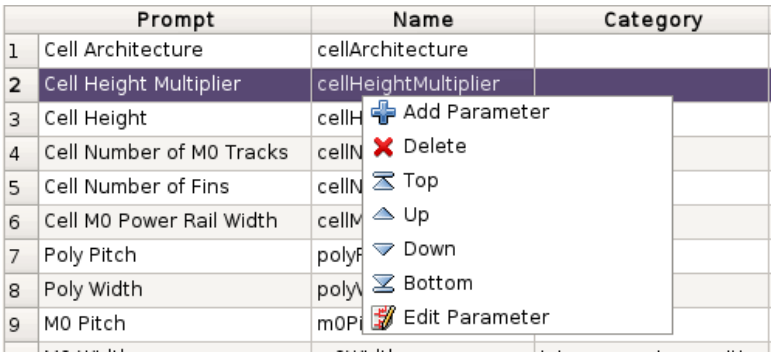
1. Double-click any column in the parameter table to display the **Edit Parameter** dialog box. Or click the button to enable the **Edit Parameter** dialog box.



2. Edit the parameters in the **Edit Parameter** dialog box and click **OK** to implement the changes.

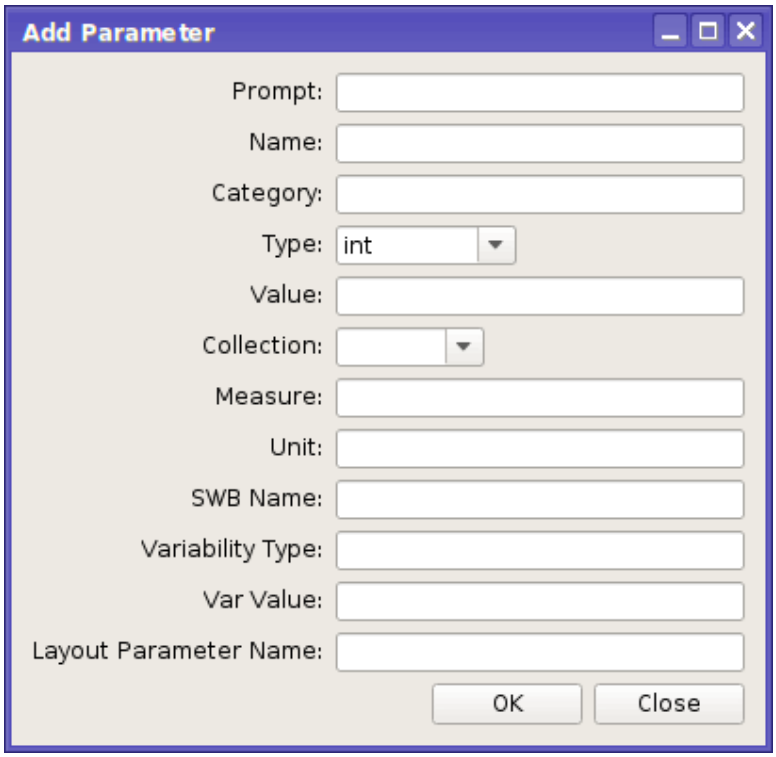
To add DTCO parameters in the parameters table:

1. Right-click the parameter table to display the menu as below.



Feedback

Or click the button to enable the **Add Parameter** dialog box.



2. Specify the parameter information in the **Add Parameter** dialog box.
3. Click **OK** to add new parameters.

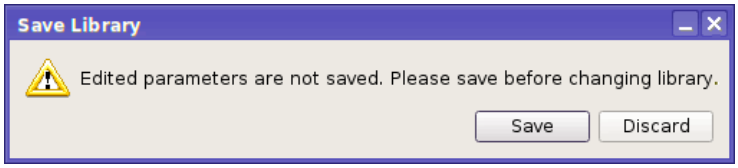
   The parameters that have been changed and added appear in red with italic type.

	Name	Category	Type	Value	Collection
1	cellArchitecture		string	short medium	list
2	*cellHeightMultiplier*		*float*	*5.5 3*	*list*
3	cellHeight		expression	143.0 78	list
4	cellNumM0Tracks		int	4 4	list
5	*cellNumFin*		*int*	*1 1*	*list*
6	cellM0PRWidth		expression	27.0 -38	list
7	polyPitch		float	48	
8	polyWidth		float	16	
9	*test*		*int*	*123*	

4. Save the design after editing. See Saving DTCO Parameters.

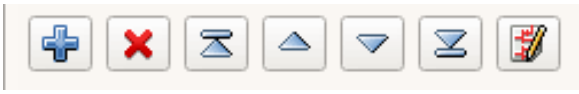
Feedback

If switching the library without saving the changes, the **Save Library** dialog box is displayed to save or discard the changes.



Parameters Toolbars

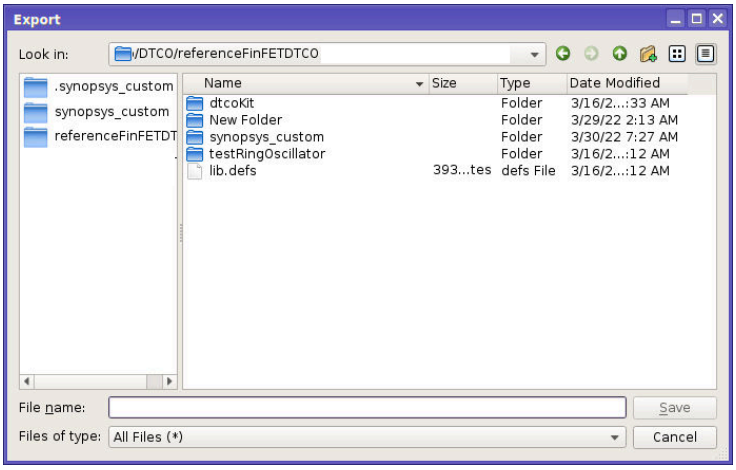
Parameters toolbars let you quick edit the parameter table. Performs add parameter, delete, move, and edit the parameters in the table.



Icon	Description
	Click this button to add new parameters in the **Add Parameter** dialog box.
	Click this button to delete the selected row.
	Click this button to move the selected row to the start of the table.
	Click this button to move the selected row up.
	Click this button to move the selected row down.
	Click this button to move the selected row to the end of the table.
	Click this button to edit the parameter in the **Edit Parameter** dialog box.

Exporting and Importing DTCO Parameters

You can export the DTCO parameters to a YAML file by choosing the **Export** option. The **Export** dialog box is displayed to specify the library.



The exported parameters information can be edited by text editor.

Here is the example of YAML file:

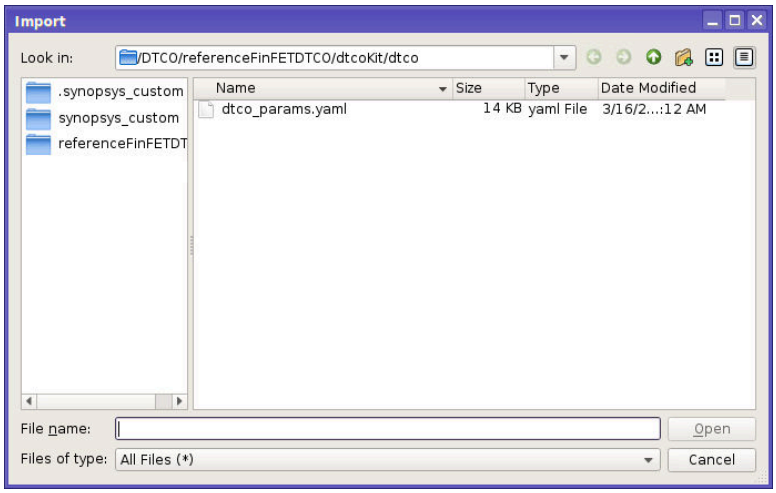
```
# DTCO Parameters
---
Version: 0.0
Parameters:
  cellArchitecture:
    prompt: Cell Architecture
    type: string
    collection: list
    value: short medium tall
    layoutParameterName: cellStyle
  cellHeightMultiplier:
    prompt: Cell Height Multiplier
    type: float
    collection: list
    value: 5.5 6.5 7.5
  cellHeight:
    prompt: Cell Height
    type: expression
    collection: list
    value:
      - "[lindex $cellHeightMultiplier 0] * $m0Pitch"
      - "[lindex $cellHeightMultiplier 1] * $m0Pitch"
      - "[lindex $cellHeightMultiplier 2] * $m0Pitch"
    measure: length
    unit: nm
```

When the YAML file is created in the associated library, you can import the parameters saved in the YAML file.

Feedback

To import a YAML file:

1. Choose **Import** to display the **Import** dialog box.



2. Choose the YAML file in the associated library in the **Import** dialog box.
3. The parameters are displayed in the parameters table.

Loading DTCO Parameters

To load the stored parameters:

1. Select a library in the **Library** selection field.
2. Choose **Load** to load the data.dm associated with the library from disk.
3. The stored parameters are displayed in the parameter table.

Saving DTCO Parameters

You can save the edited or imported DTCO parameters by choosing the **Save** option. The parameters are saved in a data.dm file associated with the library. Running any computation flows from other DTCO tabs would only consume the saved parameters or values.

Feedback

Running Transistor Modeling

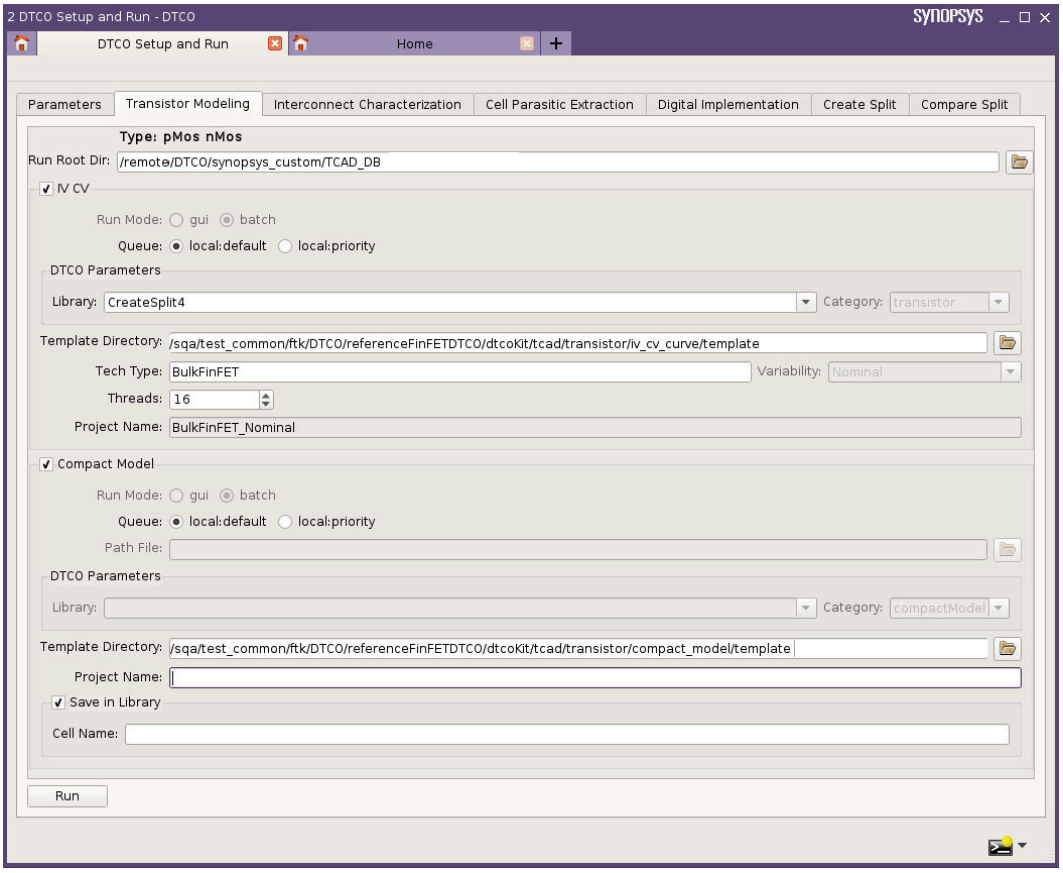
This feature creates a spice model using the TCAD tools. To generate a compact spice mode, you need IV CV data for the transistor geometry and its corresponding material description.

Note:

The transistor modeling flow does not need any layout design.

In the transistor modeling flow, you can choose to run:

- Only the IV CV data generation.
- Or only the Compact Model generation.
- Or both IV CV data and Compact Model generations.



Feedback

To create transistor modeling:

1. Specify the directory in the **Run Root Dir** field where files related to IV CV curve and the Compact Model generation are stored.
2. Enable **IV CV** to select IV CV curve generation or disable **IV CV** to ignore the IV CV curve generation.

   To run the IV CV curve generation, specify the following settings:

Option	Description
Run Mode	Specifies one of the following modes: ◦ gui: Select this option to launch the Sentaurus Workbench Graphical User Interface. The Transistor Modeling need to be run manually. ◦ batch: Select this option to run in the background and status can be checked in Job Monitor.
Queue	To select the Queue name where the computation would be performed.
DTCO Parameters	**Library**: Specifies the library to load the DTCO parameters for the transistor modeling. **Category**: When the library of DTCO parameters is specified, the parameters in the **transistor** category are used.
Template Directory	Specifies the directory contains the files for Sentaurus TCAD tool to run the transistor modeling.
Tech Type	Specifies the technology name or label.
Variability	The **Nominal** option is specified.
Threads	Specifies the number of threads.
Project Name	This filed is automatically populated by **Tech Type** and **Variability**.

3. Enable **Compact Model** to select Compact Model generation or disable **Compact Model** to ignore the Compact Model generation.

Feedback

To run the Compact Model generation, specify the following settings:

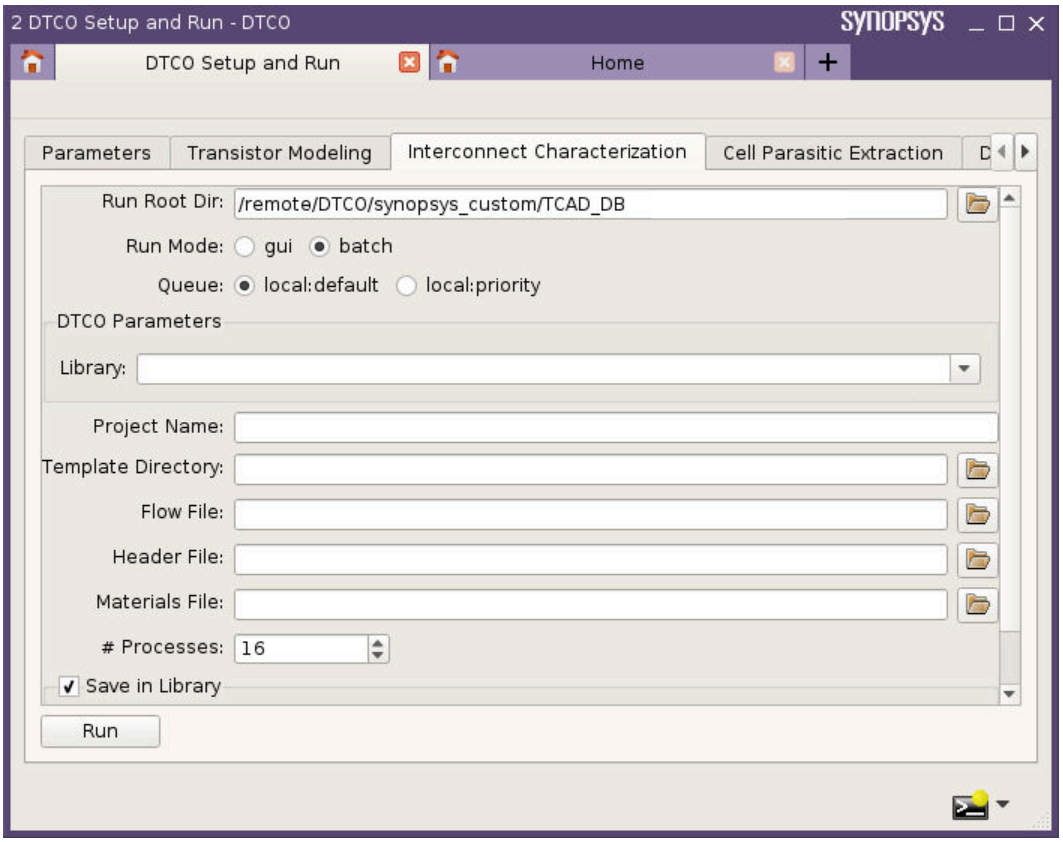
Option	Description
Run Mode	Specifies one of the following modes: ◦ gui: Select this option to launch the Sentaurus Workbench Graphical User Interface. The Transistor Modeling need to be run manually. ◦ batch: Select this option to run in the background and status can be checked in Job Monitor.
Queue	To select the Queue name where the computation would be performed.
Path File	This field is enabled when **Compact Model** is checked and **IV CV** is not checked. Specifies the file which contains information for IV CV run. If **IV CV** is checked, the path information is passed from IV CV run to Compact Model run.
DTCO Parameters	**Library**: Specifies the library to load the DTCO parameters for compact model generation. If **IV CV** is checked, the library of DTCO parameters is passed from IV CV run to Compact Model run. **Category**: When the library of DTCO parameters is specified, the parameters in the **compactModel** category are used.
Template Directory	Specifies the directory contains the files for Sentaurus TCAD tool to run the compact model generation.
Project Name	The final spice file of Compact Model run is stored with the Project Name directory in the Run Root Directory.
Save in Library	Enables the option to save a cell view of the final spice model in the DTCO Library with specified **Cell Name**.

4. Click **Run** to process the transistor modeling flow.

Feedback

Running Interconnect Characterization

The interconnect characterization flow builds an integration with the TCAD tools to create interconnect structures, simulate through the process flow, run field extraction and create an ITF file. The ITF is compiled with grdgenxo (StarRC) to prepare a NXTGRD file which can be used in the digital implementation flow.



To create or update a NXTGRD file for parasitic extraction in the implementation flow, specify the following options and fields:

Option	Description
Run Root Dir	Specify the directory where files related to Interconnect Characterization generation are stored.

Feedback

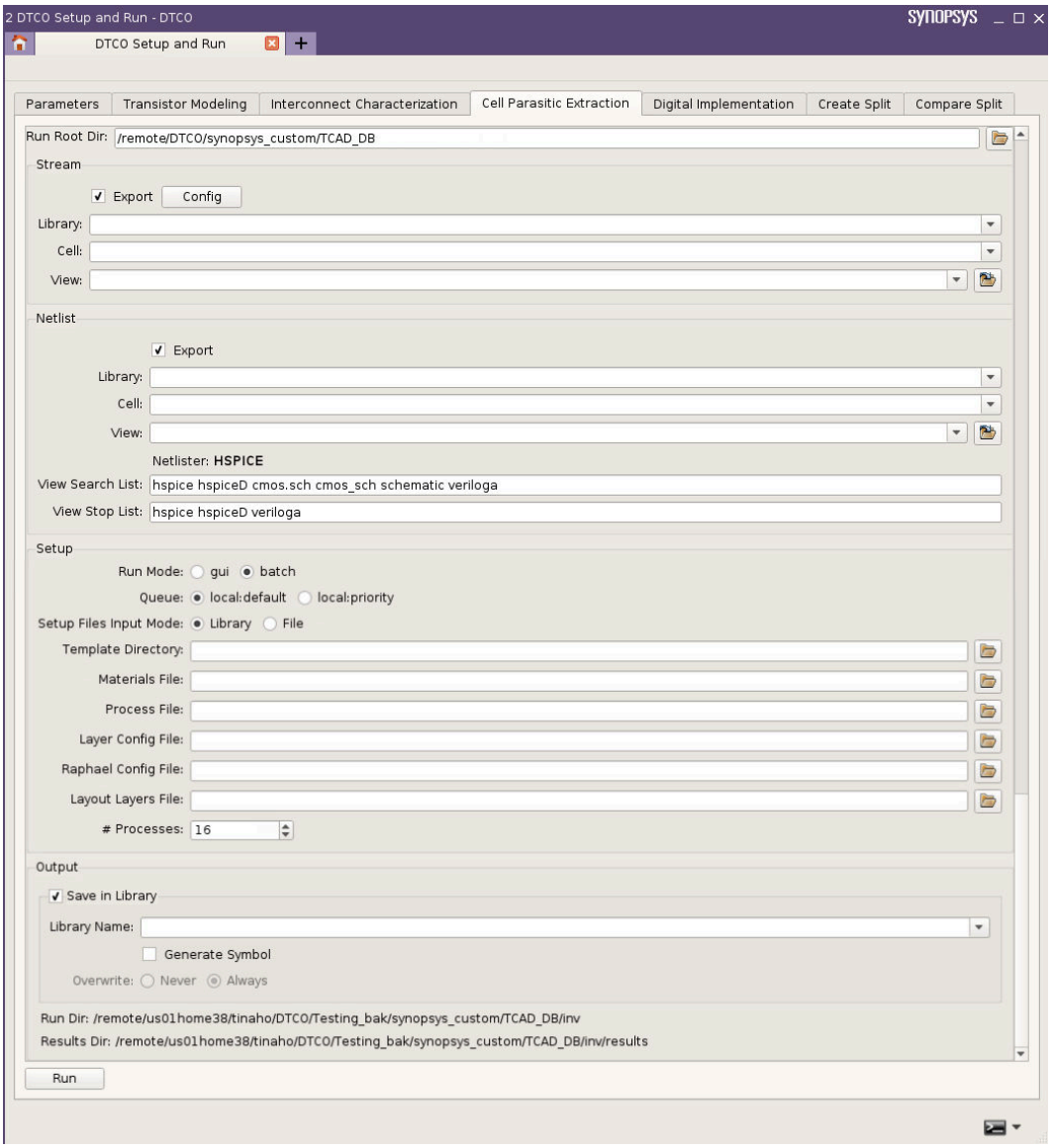
Option	Description
Run Mode	Specify the Run Mode by selecting one of the following options: • **gui**: Select this option to manually run the Interconnect Characterization in the Sentaurus Workbench graphical user interface. • **batch**: Select this option to run the Interconnect Characterization in the background and check the result in Job Monitor.
DTCO Parameters	Specify the **Library** to load the DTCO parameters for Interconnect characterization.
Project Name	When specifying the **Project Name**, the directory with project name is created in **Run Root Dir** to run the Interconnect Characterization.
Template Directory	Specifies the directory contains the files for Sentaurus TCAD tool to run the Interconnect Characterization.
Flow File	Specifies the file contains the layer information with correct layer order.
Headers File	Specifies the file contains the information (technology, foundry, process node, process version, process type, process corner, and temperature) regarding the ITF header.
Materials File	Specifies the file contains information (dielectrics, conductors, bulk resistivity model parameters, bulk resistivity, and temperature dependence) about the physical properties specific to the material needs to be simulated in the Sentaurus Workbench.
Processes	Specifies the number of jobs to be launched to run the Interconnect Characterization.
Save in Library	The output of Interconnect characterization (NXTGRD, ITF) files are placed in interconnect_results directory. If this option is enabled and **Cell Name** is provided, the tool creates cell views of NXTGRD and ITF files in the **DTCO Parameters Library**.

To generate the output of Interconnect characterization, click **Run** to process the implementation flow.

Feedback

Running Cell Parasitic Extraction

After completing the layout for a standard cell, you can create the post layout extraction netlist to be used for library characterization. The extraction includes the following information. An important intermediate step between the standard cell layout and extraction is to ensure that the GDS that is generated for extraction using Raphael-FX has all the necessary layers for the process steps.



Feedback

To create or update the post layout extraction netlist, specify the following options and fields:

Option	Description
Run Root Dir	Specifies the directory where files related to the Cell Parasitic Extraction are stored.
Steam	
Export	When this option is enabled, specify the standard cell **Library**/**Cell**/**View** of layout. The tool exports GDS stream before running the cell parasitic extraction. You can click the **Config** button to display the **Export Stream** dialog box. See **Export** > **Exporting Design Data on GDSII Stream Format** in Custom Compiler Environment User Guide If **Export** is disabled, specify the GDS stream file (*.GDS) and cell name to run the cell parasitic extraction.
Netlist	
Export	When this option is enabled, specify the standard cell **Library**/**Cell**/**View** of schematic. The **HSPICE Netlister** is used for the given **View Search List** and **View Stop List** to generate the netlist file. If **Export** is disabled, specify the name of the netlist file (*.spi | *.lvs) to run the cell parasitic extraction.
Setup	
Run Mode	Specifies the **Run Mode** by selecting one of the following options: • **gui**: Select this option to manually run the Cell Parasitic Extraction in the Sentaurus Workbench graphical user interface. • **batch**: Select this option to run the Cell Parasitic Extraction in the background and check the result in Job Monitor.
Queue	Specifies the Queue name where the computation would be performed.

Feedback

Option	Description
Setup Files Input Mode	Parasitic extraction setup is typically specified through a prefs.xml file. If **Library** is specified, the setup files are picked from Stream/Netlist Library libpath/prefs.xml. The preference is Stream first and next is netlist. If **File** is specified, the setup files are picked from runDir/prefs.xml. If both export stream and export netlist are disabled, the mode switches to **File**.
Template Directory	Specifies the directory contains the files for Sentaurus TCAD tool to run the Cell Parasitic Extraction.
Materials File	Specifies the file contains information (dielectrics, conductors, bulk resistivity model parameters, bulk resistivity, and temperature dependence) about the physical properties specific to the material needs to be simulated in the Sentaurus Workbench.
Process File	Specifies a binary file describing the process which is provided as TCAD tool setup.
Layout Config File	Specifies a file contains layer name mappings provided as TCAD tool setup.
Raphael Config File	This file contains user settings for Process Explorer (meas_RC) and Raphael used for FinFET flow.
Layout Layers File	Specifies a file contains layer numbers.
# Processes	Specifies the number of jobs to run the Cell Parasitic Extraction.
Output	
Save in Library	If this option is enabled, the output of Cell Parasitic Extraction files are placed in RunRootDirectory/CellName for a standard cell.

To generate the output of Cell Parasitic Extraction, click **Run** to process the extraction flow.

Feedback

Running Digital Implementation

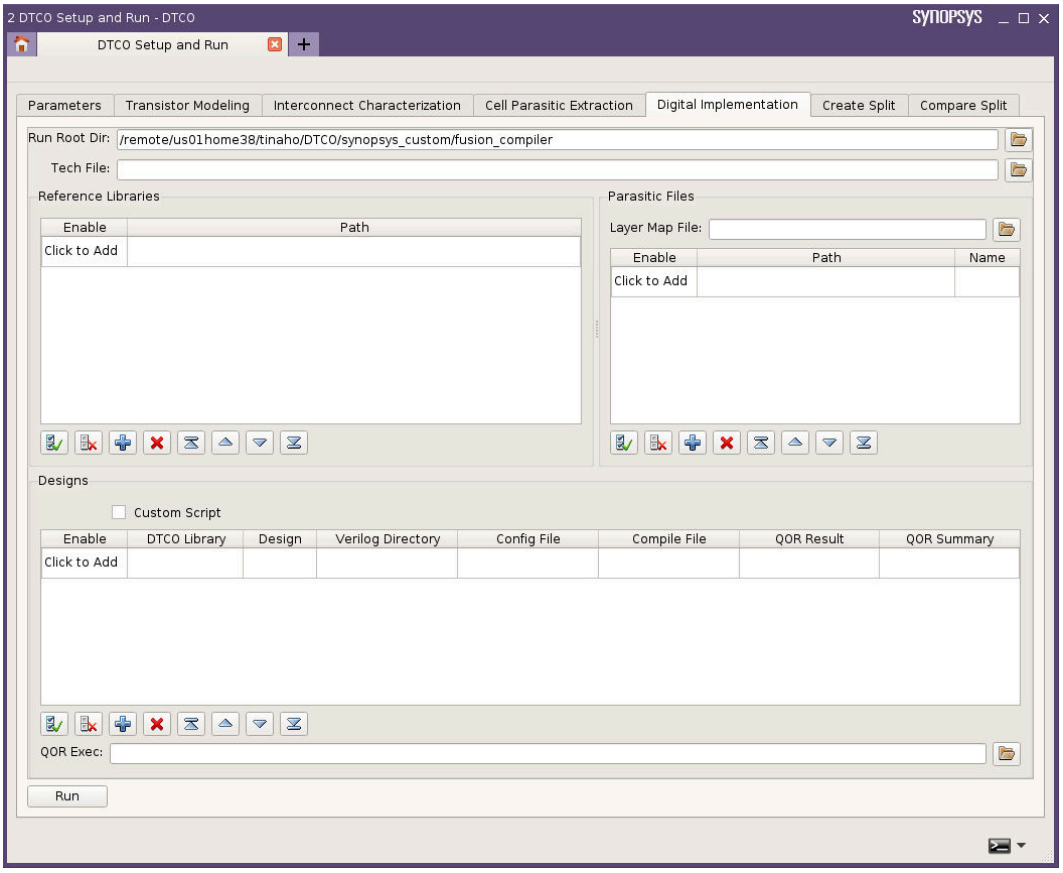
The Digital Implementation defines a set of sample digital designs to be run through the implementation flow. The step abstracts the notion of implementation and lets you define scripts that achieve the result. You can specify the following information in this tab:

- Fusion Compiler Technology File
- NDM Libraries (timing, power and physical information related to standard cells)
- Parasitic Files (TLU+ or NXTGRD)

Two methods are provided for the optimization: a simplified but less flexible step or a fully customized script by using Custom Script option.

1. Simplified Digital Implementation Flow: In this flow, you specify the library and design name, directory path where all verilog files are available, a config file, a compile file, QoR directory, and QoR result file. The tool generates a simple script to run these into an ‘fc_shell’, and then invoke the QoR script on the results generated by Fusion Compiler. It requires users to put specific setup and place and route tcl flows in the config file and compile file in exchange for simple flow.
2. Custom Script Flow: A production digital implementation flow relies on other automation elements, such as makefiles, grid management, environment variables. In this flow, you specify the library and design name, and a custom script. The script is an entry point for you to setup and manage the complexities of a production flow. The tool passes rest of the arguments filled in the **Digital Implementation** tab as a YAML file to this script. The script may parse and get values from this YAML file.

The output of Fusion Compiler is analyzed to generate a YAML file which contains relevant key-value pairs for comparing in **Compare Split**. See Comparing Split Results.



To run the Digital Implementation, specify the following options and fields:

1. Specify the directory in the **Run Root Dir** field where files related to Digital Implementation are stored.
2. Specify a file contains technology-specific information such as names, characteristics for each metal layer, and design rules in the **Tech File** field. This information are required to route a design.
3. Specify NDM libraries that contain the timing, power and physical information related to standard cells in the **Reference Libraries** section. You can add the reference library by clicking **Click to Add**. You can enable the library in the **Enable** column to specify what libraries are reference.

Feedback

You also can use the toolbars to adjust the **Reference Libraries** table. The table includes the following options:

Table 108 Digital Implementation Toolbars

Option		Description
	Check Selected	Lets you enable the selected libraries to be reference libraries.
	Uncheck Selected	Lets you disable the selected libraries to be reference libraries.
	Add File	Lets you add a new row and open the **Select Library** dialog box to specify the reference library in the created row.
	Delete Selected	Lets you delete the selected row.
	Move Top	Lets you move the selected row to the start of the table.
	Move Up	Lets you move the selected row up.
	Move Down	Lets you move the selected row down.
	Move Bottom	Lets you move the selected row to the end of the table.

4. Specify the mapping file in the Layer Map File field. The mapping file contains physical layer mapping information between the input database and the specified NXTGRD file.

   (Optional) Specify NXTGRD or TLU+ files that contain the modeled layers of a circuit. You can add the files by clicking **Click to Add** and enable the file in the **Enable** column to specify what files are used. You can use the toolbar to adjust the **Parasitic Files** table. See Table 108.

5. In the **Designs** section, enable the **Custom Script** option to specify the script which contains the information to run the Fusion Compiler in the **Designs** table. Or disable the **Custom Script** option to specify RTL, Config files, Compile files and output QOR(Result) directory in the **Designs** table.

Feedback

When disabling the **Custom Script** option, the **Designs** table includes the following fields:

Option	Description
Enable	Lets you enable the files to run for digital flows.
DTCO Library	Lets you specify the library name for running digital flows.
Design	Lets you specify the design name for running digital flows.
Verilog Directory	Lets you specify the Design Verilog file which is input to the Fusion Compiler.
Config File	Lets you specify the file that contains information about NDM library configuration MCMM setup, optional floor plan and place route setup.
Compile File	Lets you specify the file that contains commands to run Fusion Compiler and place the results in QOR (result) directory.
QOR Result	Lets you specify the directory to store the Fusion Compiler results.
QOR Summary	Lets you specify the YAML file that contains summary of the digital run, generated by **QOR Exec**.

You can add the files by clicking **Click to Add**. The added files can be used by selecting the checkboxes in the **Enable** column. You can use the toolbar to adjust the **Designs** table. See Table 108.

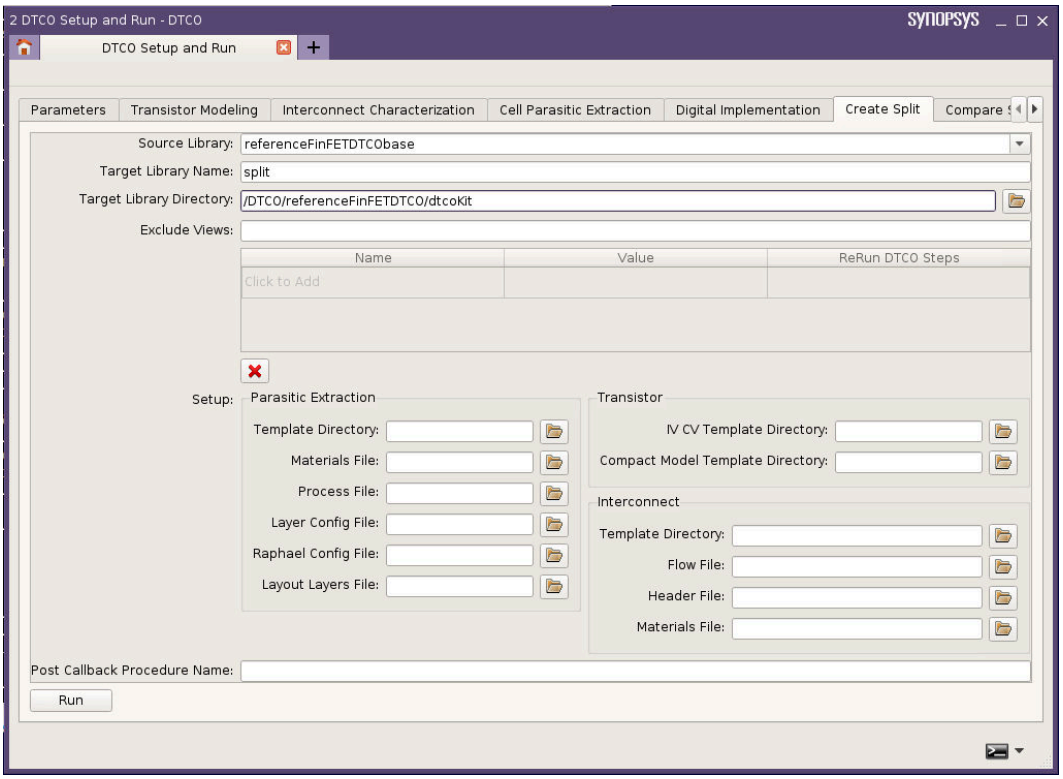
6. Specify the directory in the **QOR Exec** field. The Quality of Result exec file parses the QOR result directory and constructs the QOR.yaml file (see Figure 55) used for Compare Split. See Comparing Split Results.
7. Click **Run** to process the Digital Implementation flow.

Feedback

Creating a Split

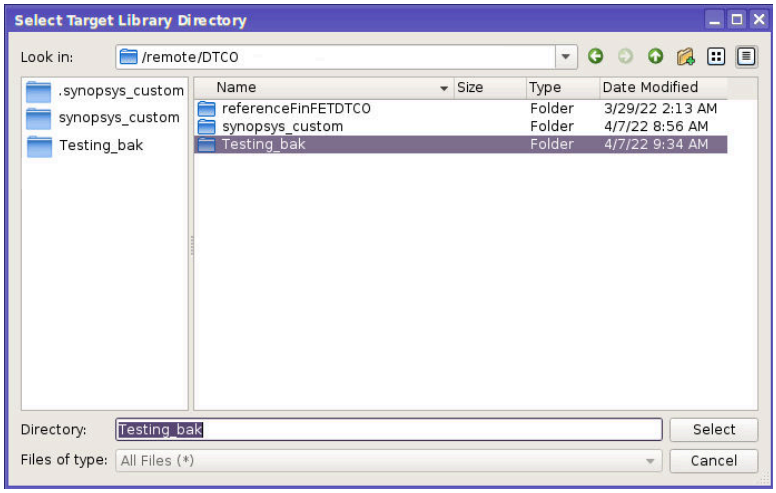
The tool picks a set of independent DTCO parameters whose values are changed from current nominal values. Only independent DTCO parameters are candidates for participating in a Split.

After specifying a set of updated values for DTCO parameters, you can specify a new target library from a process **Create Split** which is created by clicking the **Run** button. This creates a copy of all cells that designs in the source library with the target library name and remaster instances to come from the target library. All cells in the source library (even if they are in other libraries) are copied to the target library.



To create a split from DTCO parameters:

1. Specify the source library to be copied in the **Source Library** selection field.
2. Specify the target library name in the **Target Library Name** text field. It is important to name the Target Library with indicative name which tells about DTCO parameters that have been changed in the corresponding split.
3. Click the **Browse** button in the **Target Library Directory** text field to specify the target library directory from the **Select Target Library Directory** dialog box.



4. (Optional) View names specified in the **Exclude Views** text field are not copied during a split operation.
5. Using **Click to Add** to add new row in the DTCO parameter table.
6. Add DTCO parameters by clicking the arrow from each row in the DTCO parameter table.

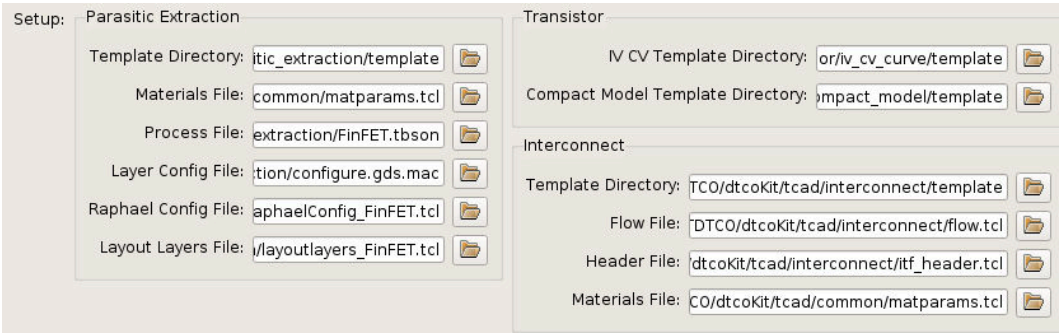
The table contains the following columns and toolbar:

Option	Description
Name	Selects one or more independent DTCO parameters from the source library by clicking the arrow.
Value	Displays the values of parameters when the parameters are specified. New values can be added or edited.

Feedback

Option	Description
ReRun DTCO Steps	Based on the modified parameters, this column reruns steps affected by the changes to the DTCO parameter. This information relies on the **Category** attribute of the DTCO parameter with the correct values.
	Deletes the selected row.

7. If the split involves changes to the setups to the TCAD process simulations that are not exposed as DTCO parameters, then specifies the information to be updated for **Parasitic Extraction**, **Transistor Modeling**, and **Interconnect Characterization** in the **Setup** section. The values entered for various files are stored in the corresponding source library and the corresponding xml file (libraryName/tcad_prefs.xml).



8. (Optional) The **Post Callback Procedure Name** is provided to execute a callback after completing Create Split process. The callback procedure has a signature of source library name, target library name and list of DTCO parameters with name, old value, and new value for each parameter that was specified into the Create Split dialog box.
9. Click **Run** to split to a new library which copied all cells from the source library.

Feedback

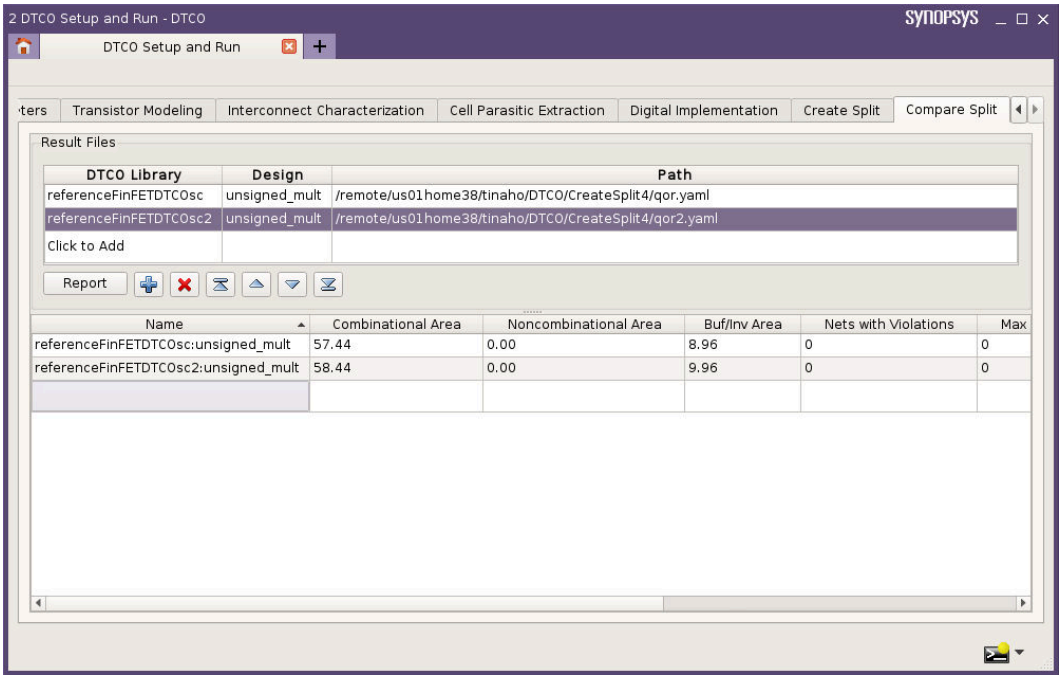
Comparing Split Results

The **Compare Split** function compares the results of QOR summary files and baseline data. The Digital Implementation flow generates a result file (QOR summary) in a simple YAML format of key-value pairs. See Running Digital Implementation.

Figure 55 Example of QOR Summary File

```
---
"Combinational Area": "52.46"
"Noncombinational Area": "9.88"
"Buf/Inv Area": "12.63"
"Nets with Violations": "0"
"Max Trans Violations": "0"
"Max Cap Violations": "0"
"Dynamic Power": "2.43e+05 nW"
"Leakage Power": "1.89e+04 nW"
"WNS": "0.0"
"TNS": "0.00"
...
```

QOR summary files from different runs in the Digital Implementation flow are compared.



Feedback

To compare the QOR summary files:

1. Add the QOR summary files from different runs in the Digital Implementation flow to be compared in the **Result Files** section by clicking **Click to Add**. Each summary file is listed in a row in the table.

   To edit the table, use the following options:

Option		Description
	Add File	Lets you add a new row and open the **Select File** dialog box to specify the summary file in the created row.
	Delete Selected	Lets you delete the selected row.
	Move Top	Lets you move the selected row to the start of the table.
	Move Up	Lets you move the selected row up.
	Move Down	Lets you move the selected row down.
	Move Bottom	Lets you move the selected row to the end of the table.

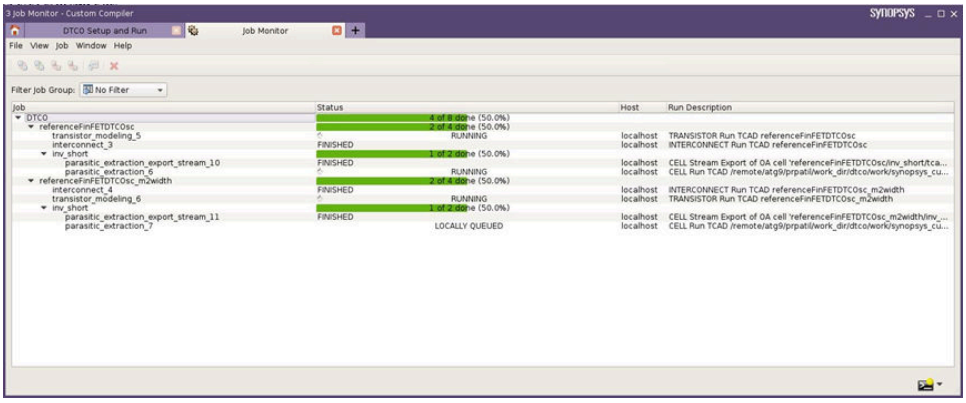
2. To compare all the QOR summary files, click the **Report** button. The compared result is generated in the table.

   The table shows the following information: **Name**, **Combinational Area**, **Noncombinational Area**, **Buf/Inv Area**, **Nets with Violations**, **Max Trans Violations**, **Max Cap Violations**, **Dynamic Power**, **Leakage Power**, **WNS**, and **TNS**.

Feedback

Monitoring Computations in Job Monitor

For multiple splits being created and computed on concurrently, each job submission are named, inspected, and managed from Job Monitor. Job Monitor is opened by choosing **Tools** > **Job Monitor**. DTCO jobs are organized under the **DTCO** category and the library names are listed under subcategories.



9

Design Reviews

The Custom Compiler design review system allows you to review design schematics and layouts, provide feedback, respond to feedback, and track review status. The built-in system enables collaborations between design and layout teams.

This chapter contains the following topics:

- About the Design Review System
- Workflows for Design Reviews
- Setting Up Design Review Policies
- Setting Up Automatic Review Notifications
- Creating, Editing, and Managing Design Reviews
- Opening an Existing Review
- Exporting Reviews
- Review Assistant Reference

About the Design Review System

The Custom Compiler design review system consists of the Review Assistant and the Review Manager. The Review Assistant lets you enter and respond to feedback. The Review Manager lets you browse the design libraries and locate existing reviews of interest.

This topic describes the following:

- Review Assistant
- Review Manager
- Review Data Structure

Feedback

- Review Database
- Review Process

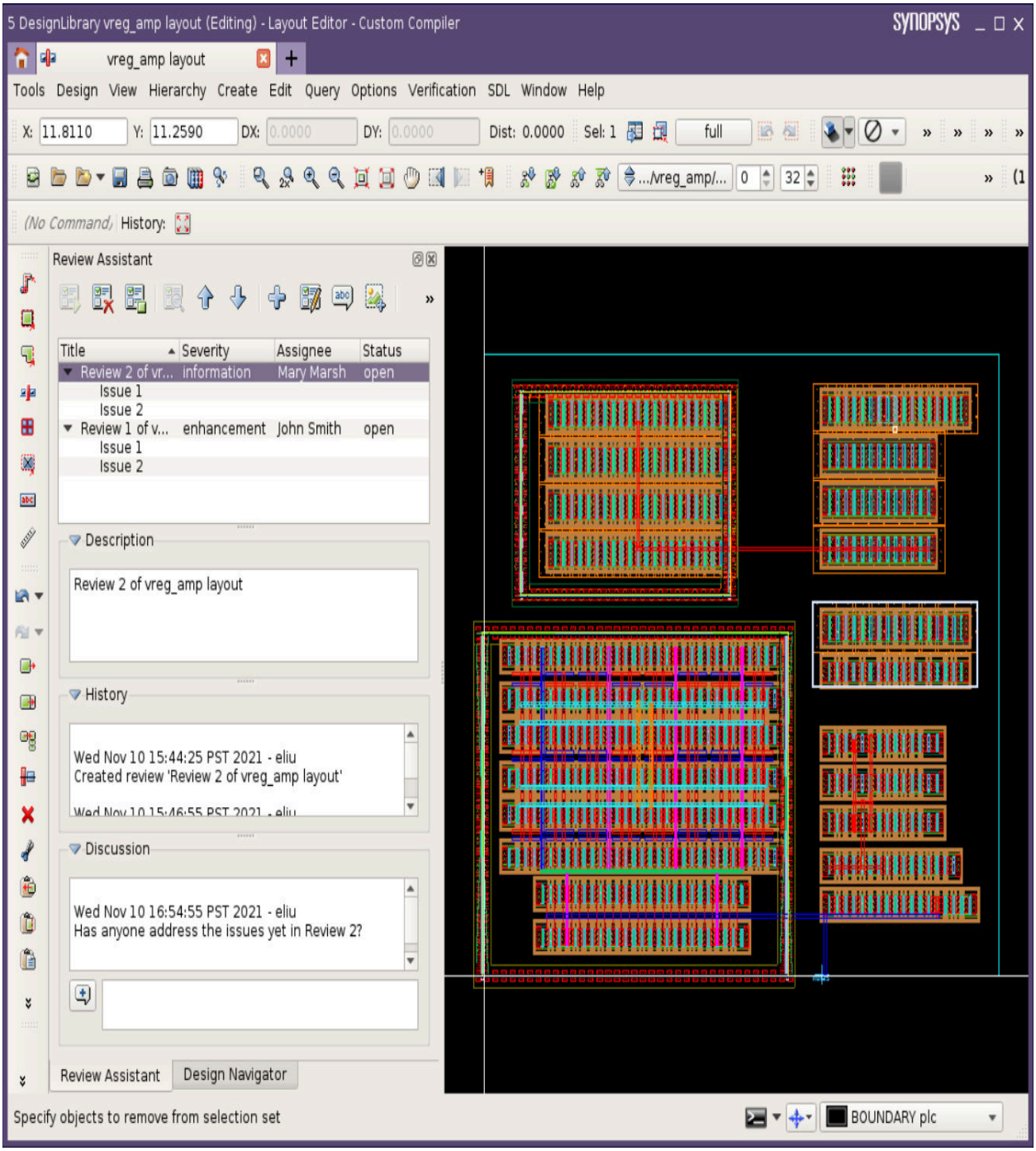
Review Assistant

The Review Assistant is the graphical user interface that lets you create and edit reviews.

You use the Review assistant to:

- Start, end, or delete reviews while in the Schematic Editor, Symbolic Editor, or Layout Editor.
- Enter review feedback without editing the design. The feedback can consist of text, annotations, and marked screen captures.
- Respond to the review feedback.
- Modify and track the status of the review feedback.

The Review Assistant supports entering and viewing comments in languages, including Chinese, Korean, and Japanese. To enable this support, start the `ibus-setup` executable on your computer and follow the instructions.



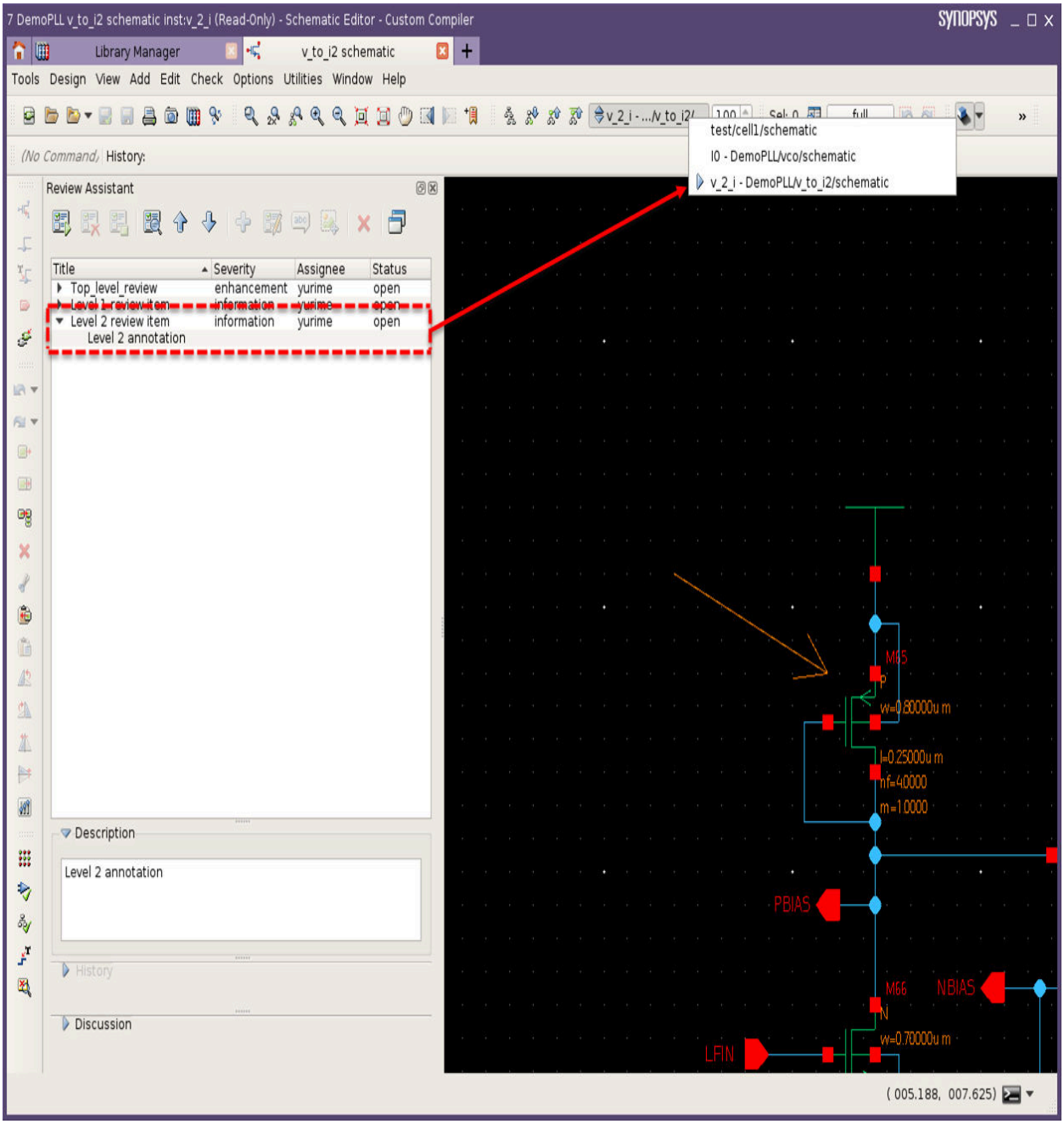
Review comments created on different hierarchy levels of the design can all be accessed from the Review Assistant.

Feedback

- The Review Assistant displays the hierarchy levels up to the **Stop Level**.
- To facilitate design reviews, the review comments in the assistant are synchronized with the annotations in the design. When reviewing layouts, double-clicking the level_n review item in the assistant descends in place. When reviewing schematics, double-clicking the level _n review item descends to level_n.

  In the schematic example shown, double-cilcking the Level 2 annotation in the assistant descends to design level 2 and displays the annotations on the canvas.
- If reviews on different hierarchy levels have the same name, the assistant displays the review name and merges the annotations from both levels.

Feedback



Review Manager

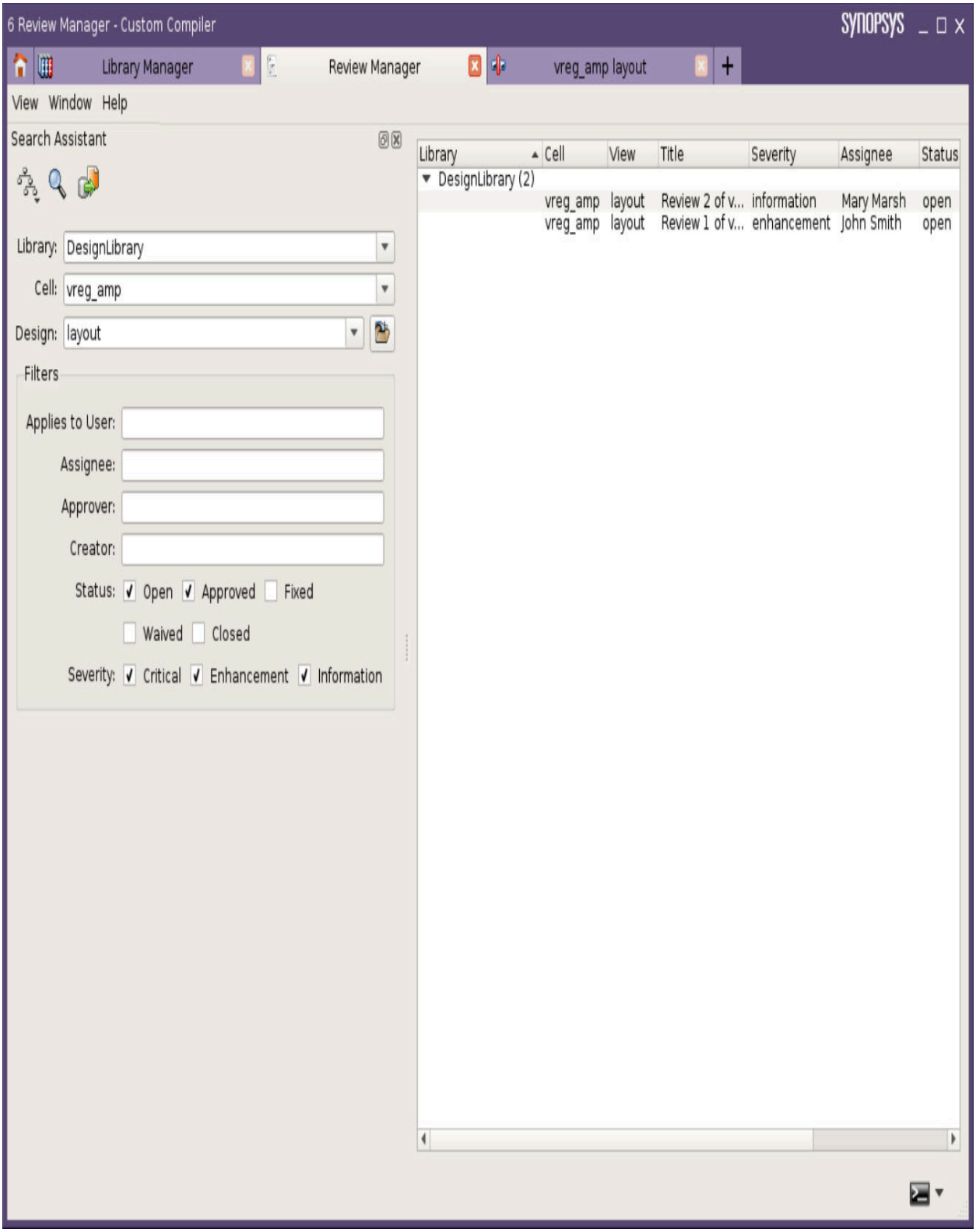
The Review Manager is the graphical user interface that lets you search for reviews of interest.

Feedback

You use the Review Manager to:

- Search for review of interest among design libraries. Available filters let you narrow the search.
- Browse the list of existing reviews for selected designs.

Feedback


6 Review Manager - Custom Compiler
SYNOPSYS
Library Manager
Review Manager
vreg_amp layout
View Window Help
Search Assistant
Library: DesignLibrary
Cell: vreg_amp
Design: layout
Filters
Applies to User:
Assignee:
Approver:
Creator:
Status: Open Approved Fixed
Waived Closed
Severity: Critical Enhancement Information
Library Cell View Title Severity Assignee Status
DesignLibrary (2)
vreg_amp layout Review 2 of v... information Mary Marsh open
vreg_amp layout Review 1 of v... enhancement John Smith open

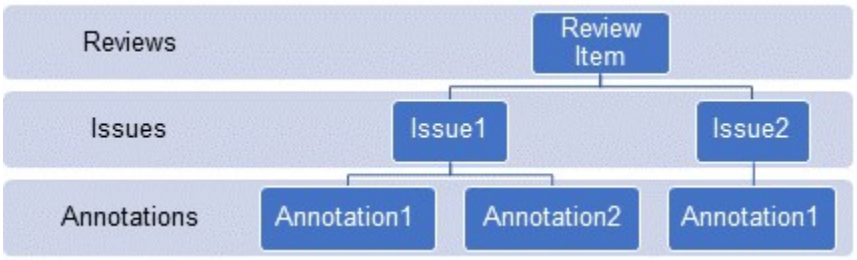
Feedback

Review Data Structure

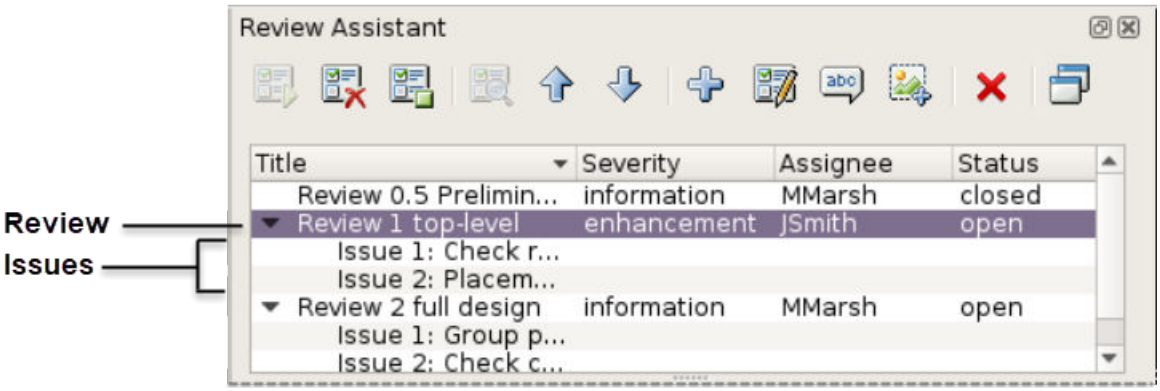
The Custom Compiler design review consists of reviews, issues, and annotations. Understanding the structure helps you determine how you set up the review policies and how to organize the reviews.

In the Custom Compiler environment, a design can have multiple reviews. Each review has a description, an associated severity, and multiple issues.

An issue can consist of a set of annotations or an image. When you create an issue, the canvas state and the set of annotations or images are preserved.



All review items are stored in a review database and are accessed using the Review Assistant.



Review Database

The design review data is stored in a review database that is closely associated with the corresponding design. Understand the review database and its relationship with the design can help you quickly navigate and locate reviews of interest.

The review database stores a set of reviews for a given cellview (schematic, symbol, or layout), and is automatically generated when you initiate a review for the cellview. The review database is also a cellview with a hierarchical OpenAccess (OA) database for storage. The review cellview exists in the same oaCell as the cellview that is reviewed.

Feedback

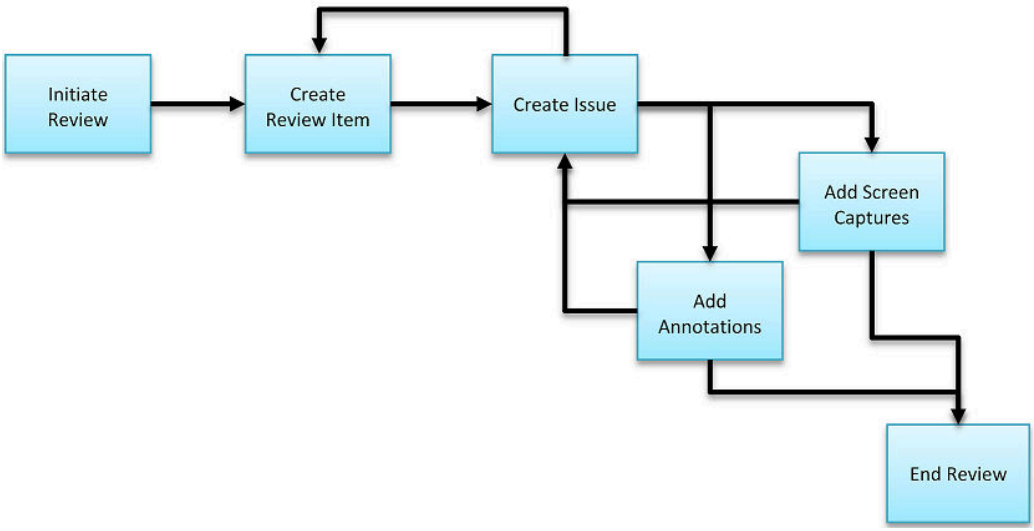
For easy identification of the review cellview, it has the same name as the reviewed cellview with an appended suffix (Tcl preference: `dcReviewCellViewSuffix`). The default suffix is *.review*. For example, the review cellview for *lib1/cell1/layout* is *lib1/cell1/layout.review*.

Review Process

The Custom Compiler design review process is based on the Review Data Structure.

The following figure shows a simplified design review process that uses the Review Assistant. A reviewer initiates the review process (opens the review database for write operations), creates one or more review items, and adds one or more issues (each with annotations and screen shots) for each review item. See Review Data Structure.

The reviewer then ends the review process, that is, saves and changes the review database to a read mode.



The review assignee then views the review items and examines each issue. The assignee can add comments to each review item and issue, and view the already existing comments. Each review item can then be marked approved, waived, fixed, or closed to conclude the specific review item.

See Also

- Workflows for Design Reviews

Feedback

Workflows for Design Reviews

The design review process consists of two main tasks: setting up and maintaining reviews, and conducting reviews.

This topic provides the following:

- Workflow for Setting Up Design Reviews
- Workflow for Conducting Reviews

Workflow for Setting Up Design Reviews

Before design reviews can be started, administrators should set up design review policies to define review users and permissions. In addition, if you can optionally set up automatic email notification so that members and approvers are aware of new or modified reviews.

The following table shows the workflow steps for setting up the design reviews.

Step		See...
1	Set up the design review policy for your organization.	Setting Up Design Review Policies
2	(Optional) Set up automatic review notifications.	Setting Up Automatic Review Notifications

Workflow for Conducting Reviews

You use the Review Assistant to conduct reviews. The following table shows the workflow steps for design reviews.

	The Review Assistant in the Custom Compiler environment is designed to facilitate the design review process. The Review Assistant lets you provide comments about layouts, respond to feedback, and track review status. This video tutorial shows how to start reviews, create and edit a review, add annotations in a review, add and reply to comments, and export the review data using the Review Assistant.

Step		See...
1	Open a design.	Opening Designs

Feedback

Step		See...
2	Display the Review Assistant.	Displaying the Review Assistant
3	Start a review session.	Starting Design Review Sessions
4	Create and edit a review.	• Creating New Reviews • Editing Reviews
5	Mark up the design.	• Adding Annotations in a Review • Editing, Moving, and Deleting Annotations • Adding Images to a Review • Displaying Annotations and Images • Hiding Annotations • Deleting Issues and Reviews
6	Start or reply to discussions.	Adding and Replying to Discussion Comments
7	End the review session.	Ending Design Review Sessions
8	(Optional) Export the review data.	Exporting Reviews

After reviews are created, you can search for and open existing reviews. You use the Review Manager to search for existing reviews. See Opening an Existing Review.

See Also

- About the Design Review System
- Review Assistant Reference

Setting Up Design Review Policies

Design review policies enforce rules implemented by your organization. The policies define users and their permission levels for various design review activities.

This topic describes the following:

- About Design Review Policies
- Default Design Review Policy
- Creating Design Review Policies
- Modify Existing Design Review Policies

About Design Review Policies

A design review policy defines review users and review permissions as follows:

Policy Component	Description
Users	Review users can be one of the following: • Creator: Creates the review. • Assignee: Is assigned an issue. • Member: Is a participant in the review. • Approver: Approves the proposed modifications.
Permissions	Review users can have permission to: • Create issues, add review annotations in the design, and add images in the review comments. • Modify issues, review annotations in the design, and images in the review description. • Delete issues, review annotations in the design, and images in the review comments.

Default Design Review Policy

A default design review policy is provided, but it is not activated. The policy is defined as follows:

Operation	Description
Create	Review users can create issues, add review annotations in the design, or add images in the review comments.
Modify	Review users can modify issues, review annotations in the design, or images in the review comments.

Feedback

Operation	Description
Delete	Review users can delete reviews or issues within a review. Review users cannot delete review annotations or images.

To activate the default design review policy, run the `dc::setPolicy` Tcl command as follows:

```
dc::setPolicy default
```

Creating Design Review Policies

You can create your own design review policies for your organization. To create a new set of design review policies, use the `dc::addPolicy` Tcl command.

Modify Existing Design Review Policies

To modify an existing design review policy, use the `dc::addPolicy` Tcl command to overwrite the existing policy. For example, to modify the default design review policy, you can use the `dc::addPolicy` command as follows:

```
dc::addPolicy default -canCreateProc newCanCreate
 -canDeleteProc newCanDelete -canModifyProc newCanModify
```

Setting Up Automatic Review Notifications

Design review members and approvers can be notified of any new or modified review input. This topic describes how to set up automatic notifications.

To set up the automatic review notification:

1. Create a Tcl procedure for sending email notifications.

   In the Tcl procedure:

   - Use the `sendmail` Linux command, or an equivalent method for email transfers.
   - Include `to`, `subject`, and `body` variables.

   Examples:

   ```
   proc sendmail {to subject body}
   ```

   ```
   proc custom_notification {to subject body}
   ```

Feedback

2. Set the `dcReviewNotificationProc` Tcl preference to specify the procedure you created in step 1.

   For example:

   ```
   db::setPrefValue dcReviewNotificationProc -value send_email_proc
   ```

3. When you create reviews, make sure you define the **Approver(s)** and **Member(s)** in the **Create Review** dialog box.

   Only the defined approvers and members receive email notifications. Email notifications are automatically sent when you end reviews.

See Also

- Creating New Reviews
- Editing Reviews
- Ending Design Review Sessions

Creating, Editing, and Managing Design Reviews

You use the Review Assistant to create new reviews, enter and respond to design review feedback, track review status, and end reviews in the Schematic Editor, Symbolic Editor, and Layout Editor.

This section describes the following:

- Displaying the Review Assistant
- Starting Design Review Sessions
- Creating New Reviews
- Copying Reviews
- Editing Reviews
- Adding Annotations in a Review
- Editing, Moving, and Deleting Annotations
- Adding Images to a Review
- Displaying Annotations and Images
- Hiding Annotations
- Deleting Issues and Reviews

Feedback

- Adding and Replying to Discussion Comments
- Ending Design Review Sessions

	Review Assistant facilitates design reviews by allowing you to create, edit, and manage reviews, add annotations and images, track review statuses, and collaborate through discussions. It supports hierarchical navigation, synchronization with design annotations, and exporting reviews to HTML for external sharing.

Displaying the Review Assistant

You can display the Review Assistant in the Schematic Editor, Symbolic Editor, or Layout Editor.

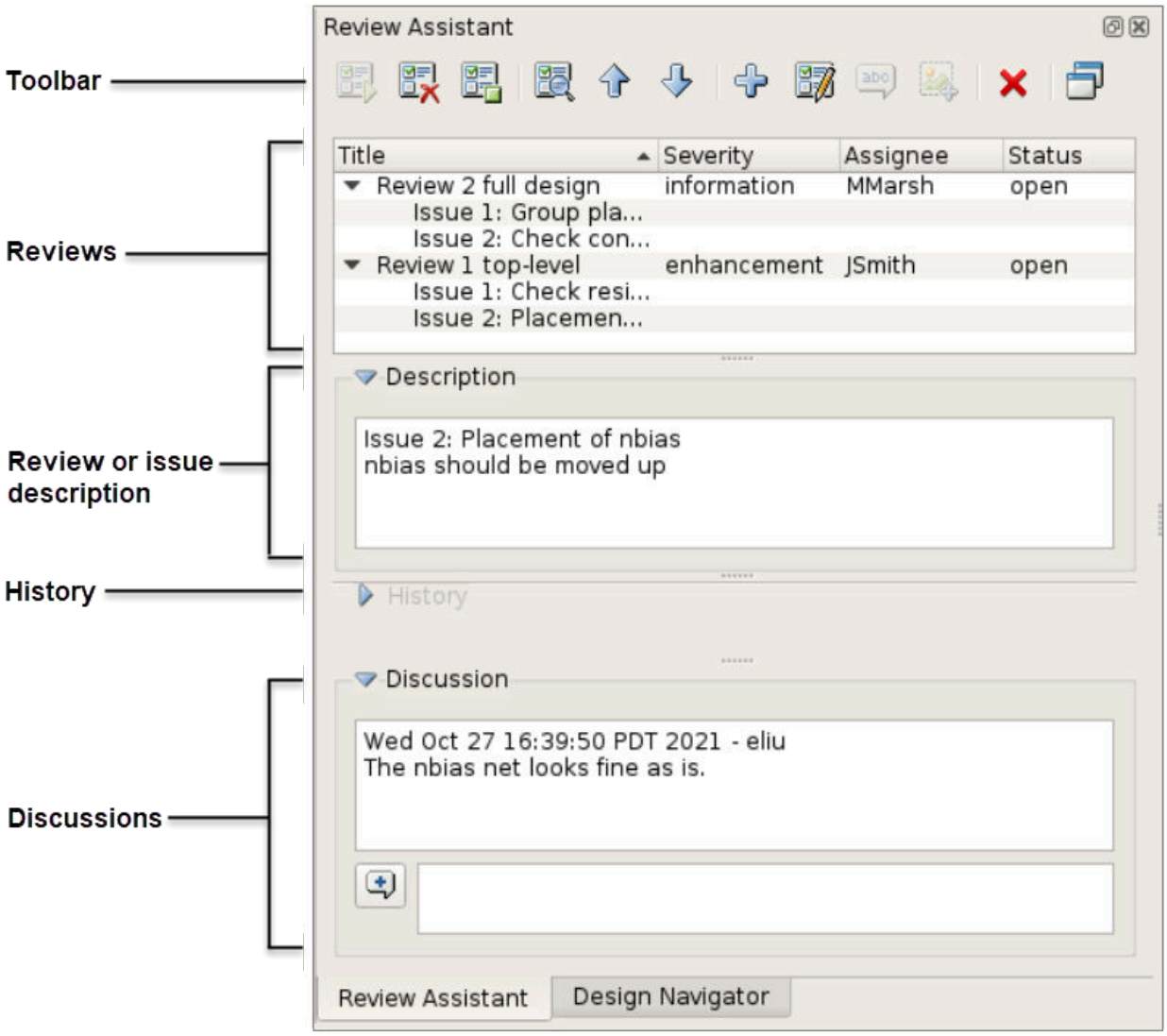
To display the Review assistant:

► In the Schematic Editor, Symbolic Editor, or Layout Editor, choose **Window > Assistants > Review Assistant**.

The assistant appears on the left side of the design editor tab and lists existing reviews, if any.

The following diagram shows the components of the Review Assistant. For the quick reference, see Review Assistant Reference.

Feedback



See Also

- About the Design Review System
- Creating, Editing, and Managing Design Reviews
- Opening an Existing Review

Starting Design Review Sessions

Starting design reviews in the Review Assistant enables functions that permit you to add and edit reviews and issues.

Feedback

To start a design review:

► On the Review Assistant toolbar, click **Start Review**.

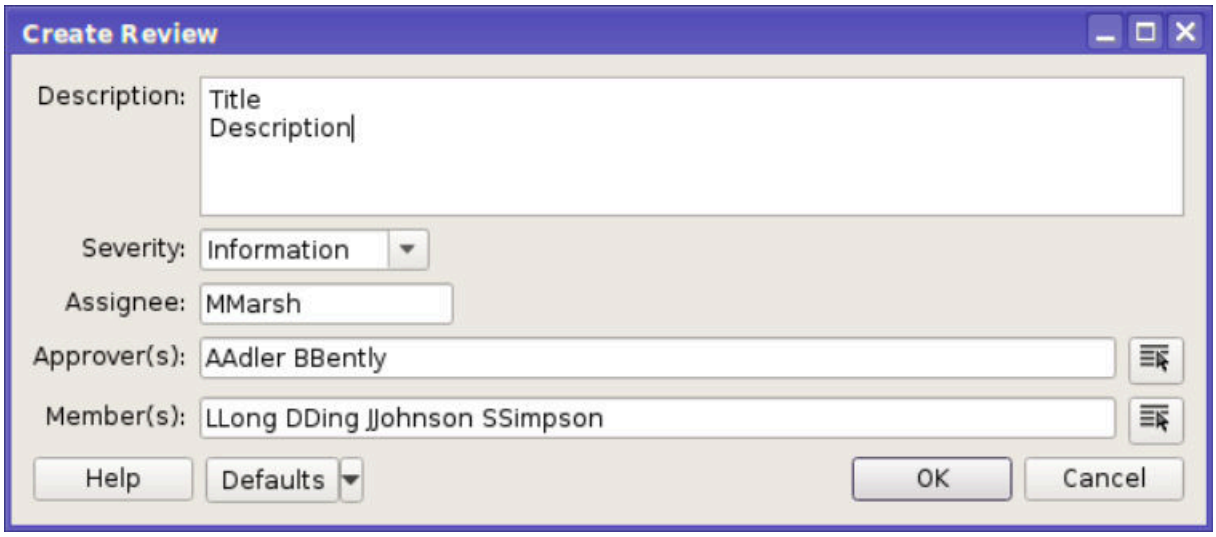
As soon as you start a review, a review cellview (.review) is automatically generated.

Creating New Reviews

To create a review for a design:

1. On the Review Assistant toolbar, click **New Review**.

The **Create Review** dialog box opens.



2. In the **Description** box:
   a. Type the review title on the first line. The title appears in the **Title** column in the Review Assistant.
   b. Type a description for the review on subsequent lines. The description appears in the **Description** section of the Review Assistant.

3. Choose the **Severity** of the review comments:

Argument	Description
Enhancement	New features to be added to the existing design.
Critical	Defects that must be fixed.
Information	Notes to the designers or team.

4. Specify the **Assignee**, the recipient of the review.
5. Specify the list of review **Approver(s)**, users who can set the review as **Approved**, **Closed**, or **Waived**.

   Type the space-separated list of user names in the **Approver(s)** box. Alternatively, click , and select the user names.

   You must define approvers if you want approvers and review members to automatically receive notifications when you add or modify reviews.
6. Specify the review **Member(s)**, users who are notified of the review but are not required to approve.

   Type the space-separated list of user names in the **Member(s)** box. Alternatively, click , and select the user names.

   You must define members if you want approvers and review members to automatically receive notifications when you add or modify reviews.

See Also

- Setting Up Automatic Review Notifications
- Adding Annotations in a Review
- Editing Reviews

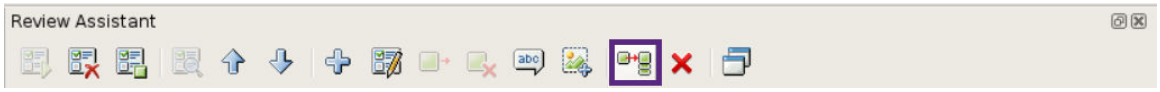
Copying Reviews

After you add a design review, you can copy the same review.

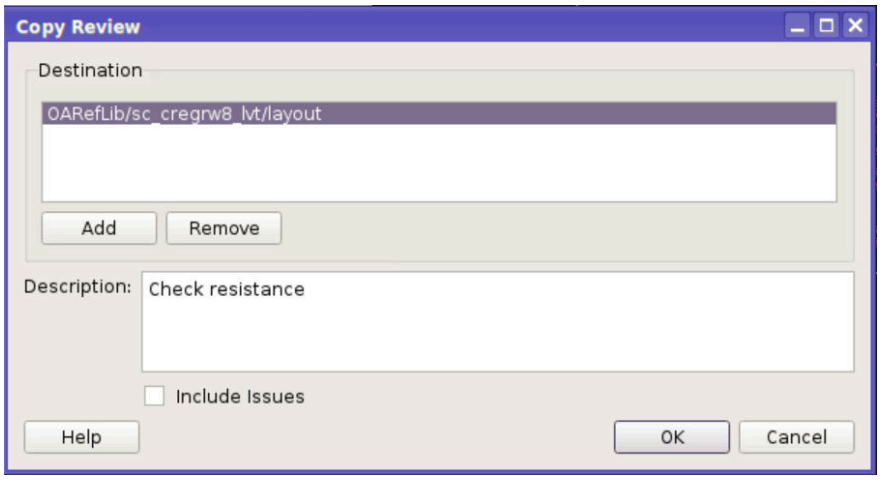
Feedback

To copy a review:

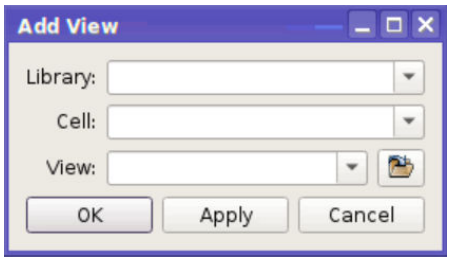
1. In the Review Assistant, select the review, and click **Copy Review**.



The **Copy Review** dialog box opens.



2. To add one or more library, cell, views to for review, click **Add** in the **Destination** section. The **Add View** dialog box opens.



3. Enter the **Library, Cell, View** and click **OK** or **Apply**.

   The library, cell, view appears in the **Destination** section in the **Copy Review** dialog box.

4. (Optional) To delete a destination from a library, cell, view from the **Destination** section, select the library, cell, view and click **Remove**.

5. The description of the copied design review appears in the **Description** box. You can edit the description, if needed.

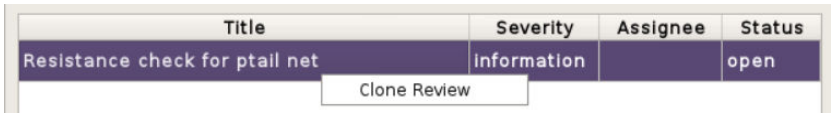
6. Select the **Include Issues** box to copy the design review annotations associated with the review. By default, this box is not selected.
7. Click **OK** or **Apply.** The copied review appears in the Review Assistant table.

Cloning Reviews

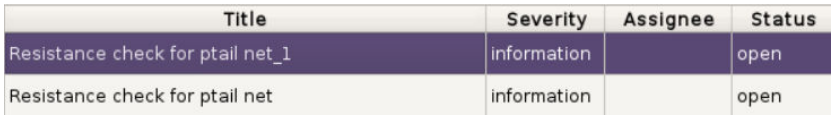
You can clone an existing design review from the Review Assistant. The cloned review contains the same information as the reference review.

To clone a review:

1. In the table right-click on the **Title** of the review, and select the **Clone Review** option.



2. The cloned review appears in the first row of the table.



See Also

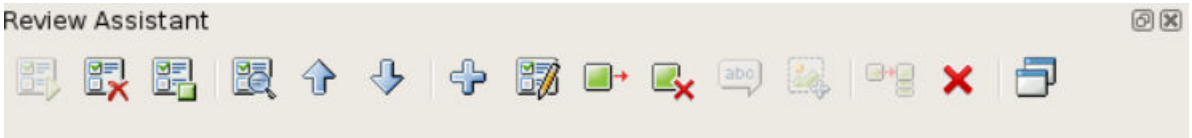
- Creating New Reviews
- Editing Reviews
- Deleting Issues and Reviews

Editing Reviews

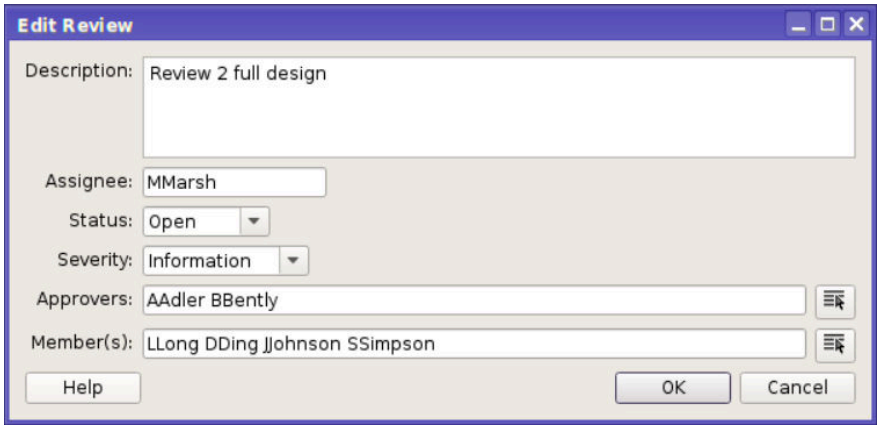
After you add a review, you can edit it to set the review status or change the original review information, such as the title, description, assignee, or approvers.

To edit a review:

1. In the Review Assistant, select the review, and click **Edit Review**.



The **Edit Review** dialog box opens.



2. Choose the review **Status** to indicate the current state of the selected review:

Argument	Description
Open	The review is in progress.
Approved	The review is finished and all issues have been addressed.
Fixed	The review issues have been fixed.
Waived	The review is no longer applicable or under consideration.
Closed	Reviewers should no longer view or add issues or comments.

3. Set or modify the remaining options as described in Creating New Reviews.

See Also

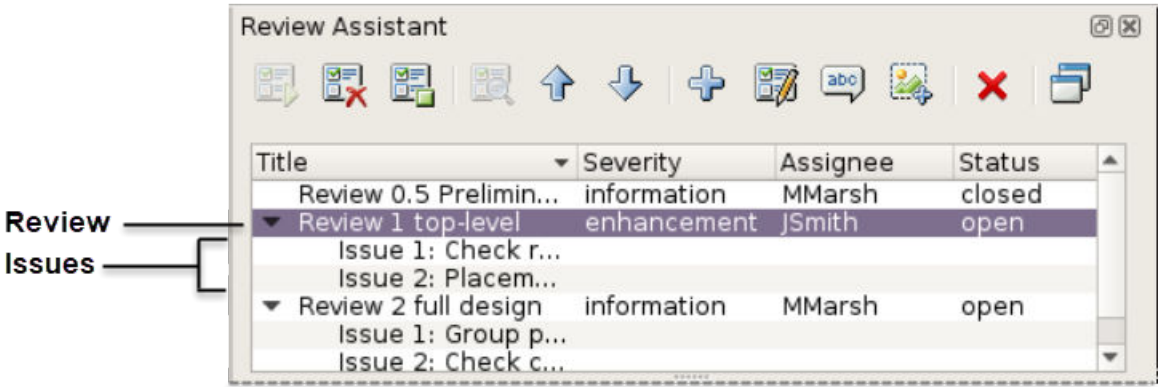
- Adding Annotations in a Review
- Editing, Moving, and Deleting Annotations
- Ending Design Review Sessions

Adding Annotations in a Review

You can add annotations in the design to let designers know the locations of issues. Annotations include text callouts, arrows, ellipses, and so on. The annotations are overlays and do not alter the original design.

When you add review annotations, you are creating issues that are stored as part of the review in the Review Assistant. Each issue can contain multiple annotations. Keep this organization in mind when you create review annotations.

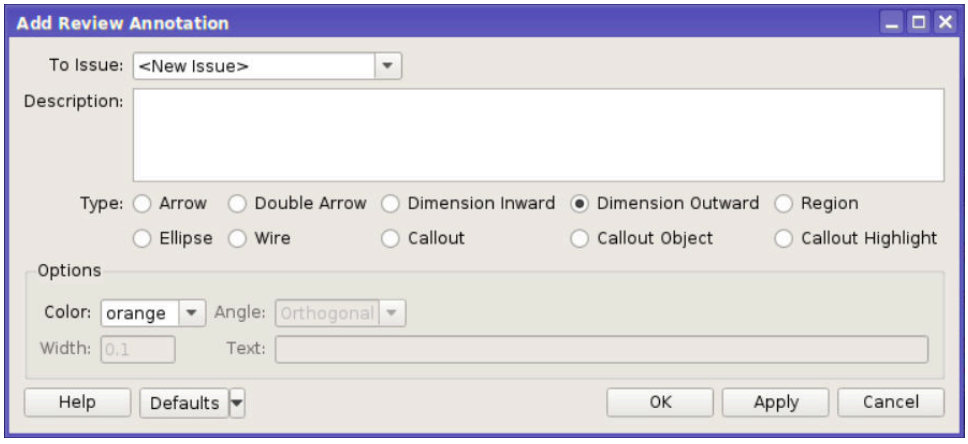
Feedback



To add annotations in a review:

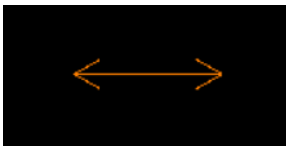
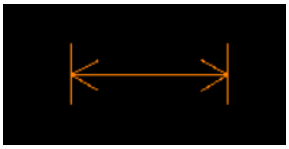
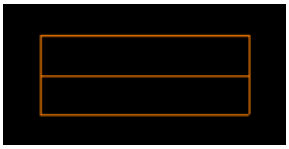
1. In the Review Assistant, select the review.
2. On the Review Assistant toolbar, click **Add Annotation**.

The **Add Review Annotation** dialog box opens.

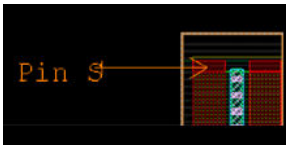
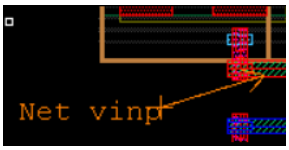


3. In the **Description** box:
   a. Type the issue title on the first line. The title appears in the **Title** column in the Review Assistant.
   b. Type a description for the issue on subsequent lines. The description appears in the **Description** section of the Review Assistant.
4. Select the **Type** of annotation you want to add.

Feedback

Argument	Description
Arrow	
Double Arrow	
Dimension Inward	
Dimension Outward	
Region	
Ellipse	
Wire	
Callout	Requires the **Text** option.

Feedback

Argument	Description
Callout Object	
Callout Highlight	

5. In the **Options** section, set the following as applicable:

Setting Option	Description
Color	Color of the annotation objects.
Angle	Angle of the **Wire** annotation: **Orthogonal**, **Diagonal**, or **Any Angle**.
Width	Width of the **Wire** annotation.
Text	Text to use for the **Callout** annotation type.

6. On the canvas, add the selected annotation: Click the starting and ending points for the annotation.

   For ellipses, the starting point is the center of the ellipse.

7. To add more annotations, repeat steps 4 through 6.
8. When you are finished, click **OK** in the dialog box.

See Also

- Editing, Moving, and Deleting Annotations
- Adding Images to a Review

Feedback

Editing, Moving, and Deleting Annotations

You can edit, move and delete design review annotations from the Review Assistant:

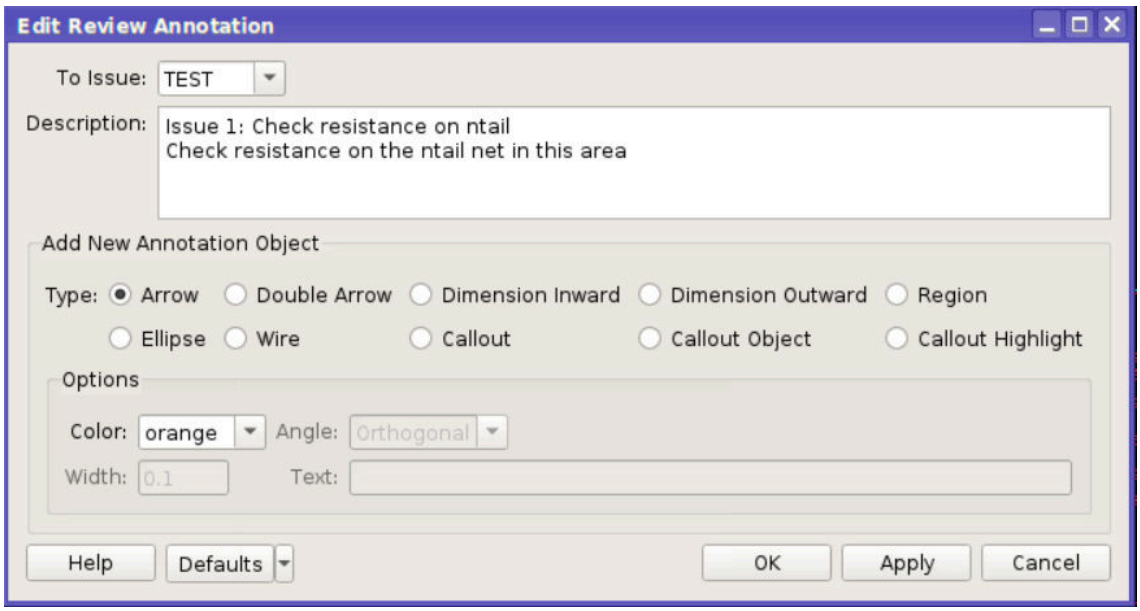
- Editing Annotations
- Moving Annotations
- Deleting Annotations

Editing Annotations

To edit the design review annotations:

1. Select the issue, and select the **Edit Issue** ( ) option.

   The **Edit Review Annotation** dialog box opens.



2. Modify any of the settings. For the descriptions of the options, see Adding Annotations in a Review.

Feedback

Moving Annotations

You can move design review annotations on the canvas. To move one or more annotations:

1. On the canvas, select the annotations you want to move. You can area-select multiple annotations.
2. Perform any one of the following:
   - Select the **Move Annotation** ( ) option.
   - Press the following keys in the keyboard:
     - For schematic designs, press Shift+m
     - For layout designs, press m
   - Enter the `idc::move` command in the console.
3. In the canvas, drag the annotation to the new location.

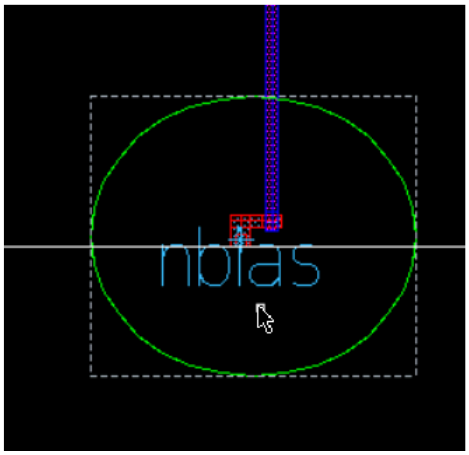
Feedback

Deleting Annotations

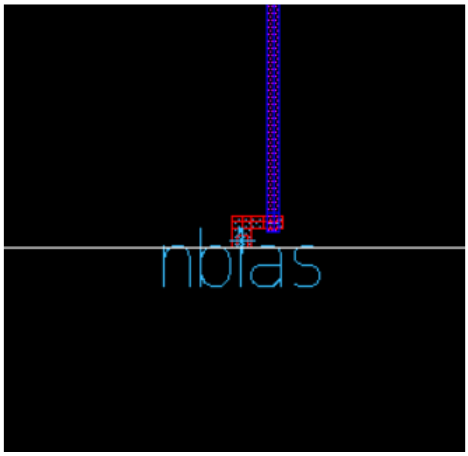
You can delete the design review annotations that are no longer needed. To delete one or more annotations:

1. On the canvas, select the annotations you want to delete. You can area-select multiple annotations.
2. Perform any one of the following:
   - Select the **Delete Annotation** ( ) option.
   - Press the Delete key on the keyboard.
   - Enter the `idc::delete` command in the console.

Select annotation



Annotation is deleted



See Also

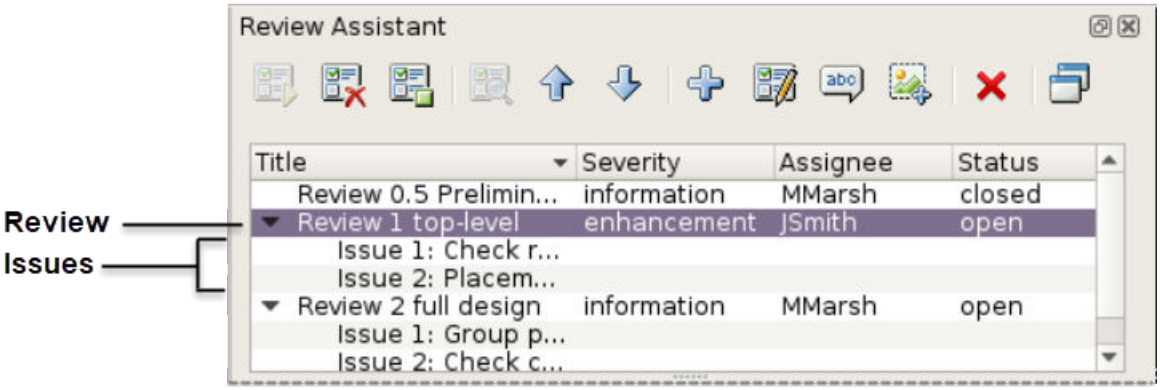
- Adding Annotations in a Review
- Displaying Annotations and Images
- Editing Reviews
- Adding Images to a Review

Adding Images to a Review

You can add a screen capture (image) of the canvas to your review to clarify your review comments.

Feedback

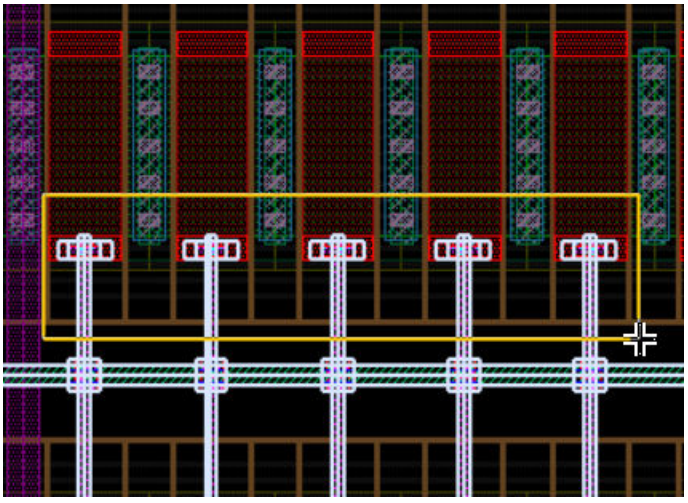
When you add images, you are creating issues that are stored as part of the review in the Review Assistant. Each issue contains only one image.



To add an image to the review:

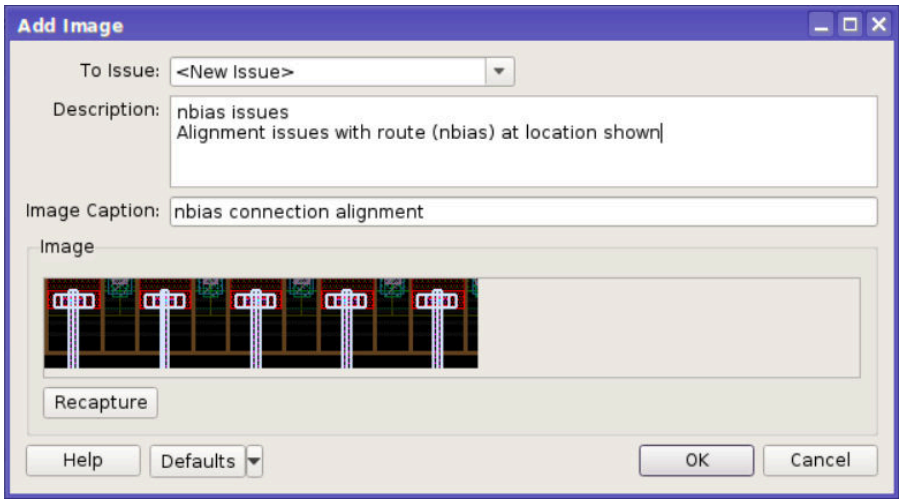
1. In the Review Assistant, select the review or issue.
2. On the Review assistant toolbar, click **Add Image** ( ).

3. Specify the design region for the screen capture:
   - To capture the entire viewport, click anywhere on the canvas.
   - To capture a specific area, draw a rectangle around the area of interest on the canvas.



Feedback

The **Add Image** dialog box opens and displays the image you just captured.



4. If the image looks unsatisfactory and you want to retake the screen capture, click **Recapture**.

   The new image appears in the dialog box.

5. In the **Description** box:

   a. Type the review title on the first line. The title appears in the **Title** column in the Review Assistant.

   b. Type a description for the review on subsequent lines. The description appears in the **Description** section of the Review Assistant.

6. In the **Image Caption** box, type a short summary of the image.

7. When you are finished, click **OK**.

   The new image (issue) appears in the Review Assistant.

Displaying Annotations and Images

If annotations and images are hidden or not visible on the canvas, you can use the **Show Issue** command to display them.

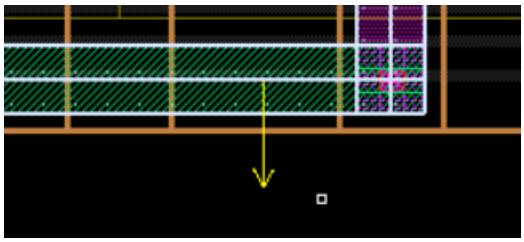
Feedback

To display annotations and images associated with an issue:

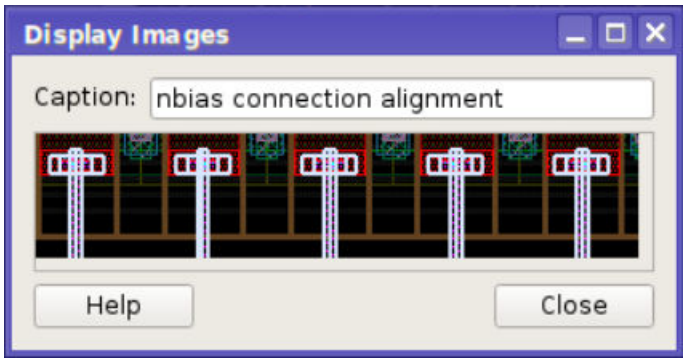
► In the Review Assistant, do one of the following:

- Double-click the issue.
- Select the issue, and click **Show Issue** on the Review Assistant toolbar.

If you selected an issue that contains annotations, the annotations appear on the canvas.



If you selected an issue that contains an image, the **Display Images** dialog box opens.



Hiding Annotations

You can hide annotations to view the design in its original state and add new annotations.

Feedback

To hide existing annotations that are currently displayed on the canvas:

1. In the Review Assistant, select the review.
2. On the Review Assistant toolbar, click **Restore Window**.

See Also

- Displaying Annotations and Images

Deleting Issues and Reviews

You can delete reviews or issues that are no longer needed.

To delete an issue or a review:

1. In the Review Assistant, select the issue or review.
2. On the Review Assistant toolbar, click **Delete Selected** ( ).

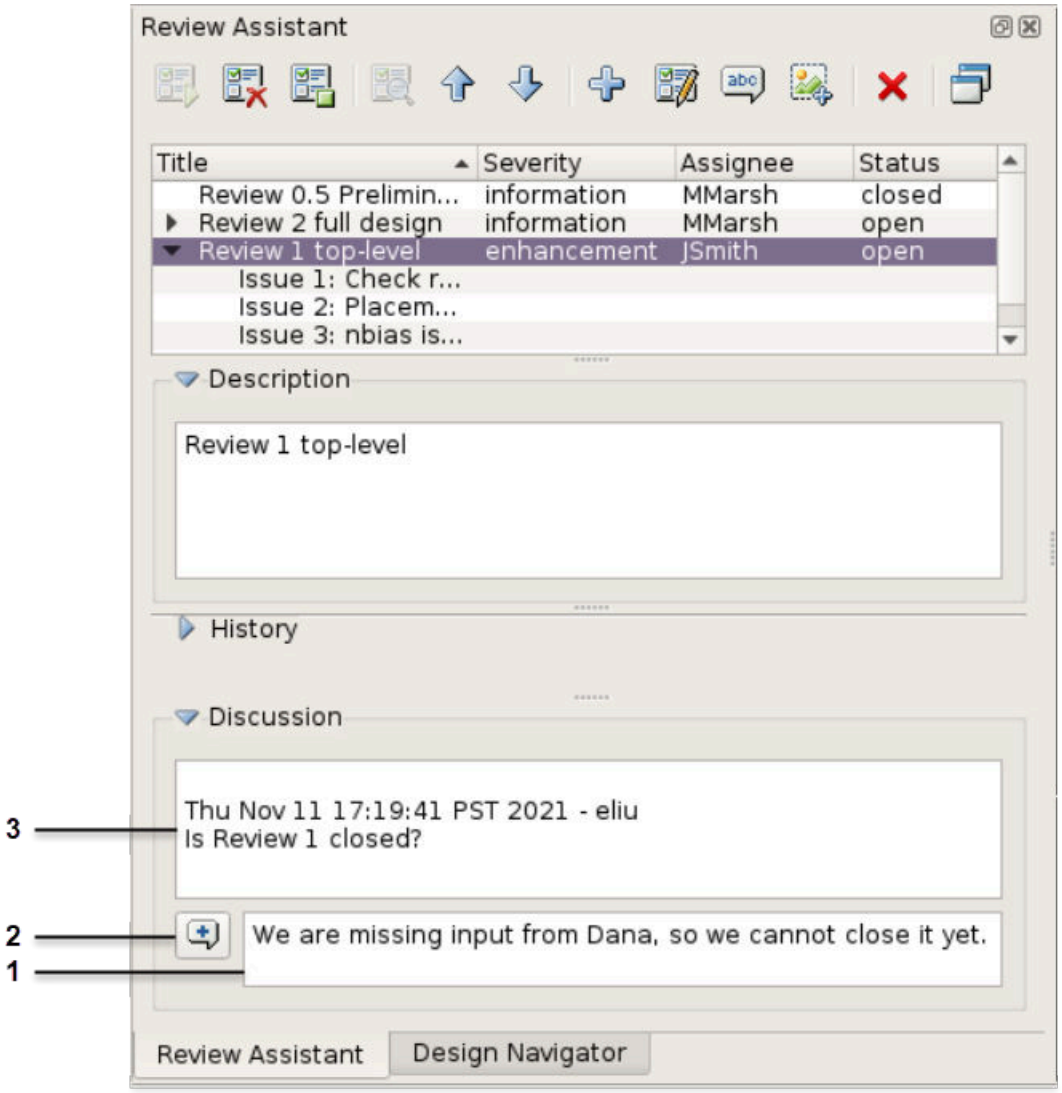
Adding and Replying to Discussion Comments

You can add or reply to comments during design reviews to help communicate questions or highlight issues for other reviewers to discuss.

To add or reply to comments in a review:

1. In the Review Assistant, locate the **Discussion** section and type your comments in the second text box.
2. Click **Add comment to discussion** .
3. View the comment you entered.

Feedback



Ending Design Review Sessions

When you are finished creating reviews or issues, you can end the review session to save the changes. Ending the review session and saving the changes allow other reviewers to see your input. You can also discard all changes made during the current review session and end the review.

This topic describes the two ways to end reviews:

- Saving Changes and Ending Reviews
- Discarding All Changes and Ending Reviews

Feedback

Saving Changes and Ending Reviews

To save all changes made and end the review session:

► On the Review Assistant toolbar, click **End Review**.

The new reviews and issues you created during the review session are saved, and the session closes. If automatic review notification was set up, approvers and members are automatically notified by email.

See Also

- Setting Up Automatic Review Notifications

Discarding All Changes and Ending Reviews

To discard all changes made during the current review session and end the session:

► On the Review Assistant toolbar, click **Discard Review**.

All reviews and associated issues created during the current review session are discarded, and the session closes.

Opening an Existing Review

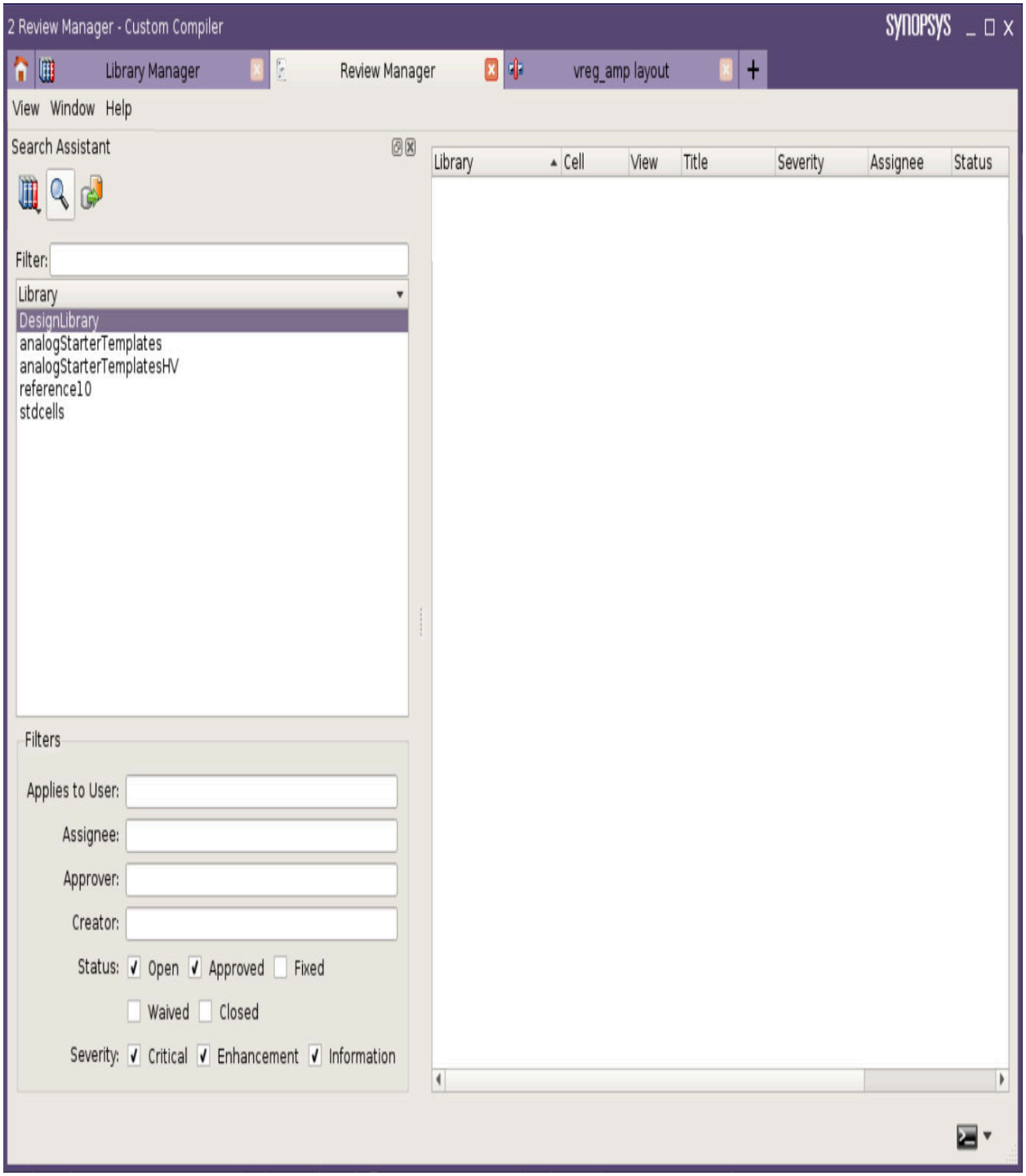
You use the Review Manager to locate an existing review or issue. After you locate it, you can open it in the Review Assistant.

To open a review:

1. In the Library Manager, double-click the design review cellview (.review).

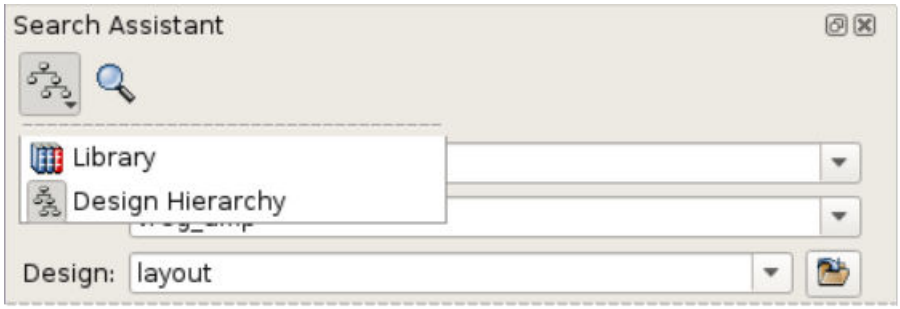
   The Review Manager opens.

Feedback


2 Review Manager - Custom Compiler
SYNOPSYS
Library Manager
Review Manager
vreg_amp layout
View Window Help
Search Assistant
Filter:
Library
DesignLibrary
analogStarterTemplates
analogStarterTemplatesHV
reference10
stdcells
Filters
Applies to User:
Assignee:
Approver:
Creator:
Status: Open Approved Fixed
Waived Closed
Severity: Critical Enhancement Information
Library
Cell
View
Title
Severity
Assignee
Status

Feedback

2. In the **Search Assistant** area, click **Search Type**, and choose one of the following:

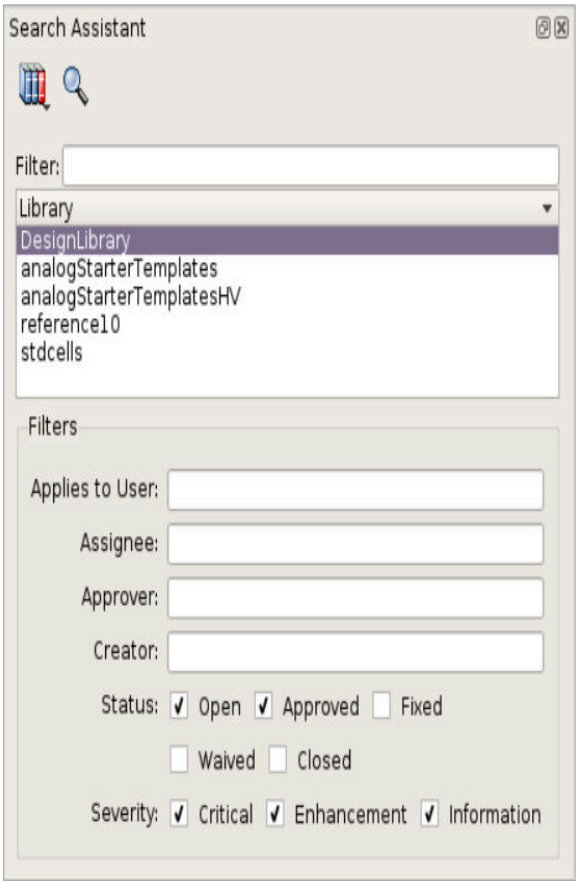


Argument	Description
Library	Lists the design libraries.
Design Hierarchy	Lets you specify the Library, Cell, and View names. You can also set addition search filters to narrow the search results.

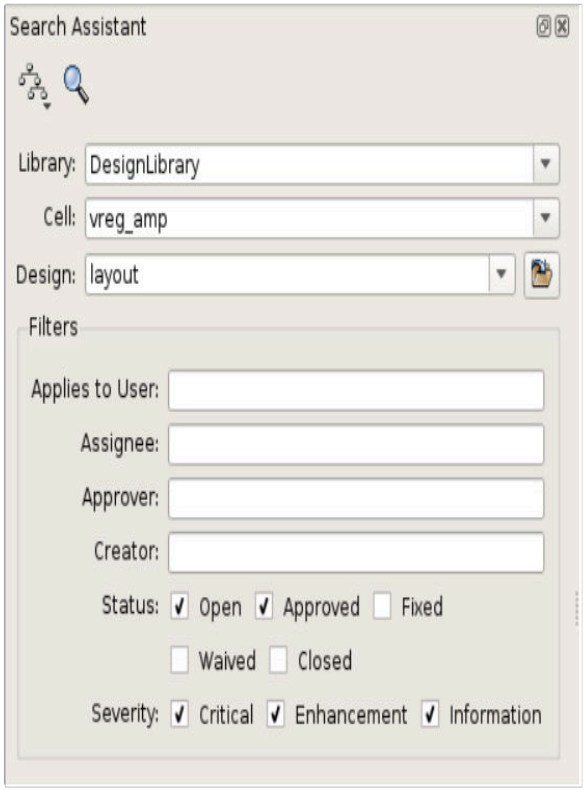
3. Set the search criteria:
   - If you selected the **Library** search type, select the design library, and optionally specify the additional filters.
   - If you selected the **Design Hierarchy** search type, specify the **Library**, **Cell**, and **View** names, and optionally specify the additional filters.

Feedback

Library



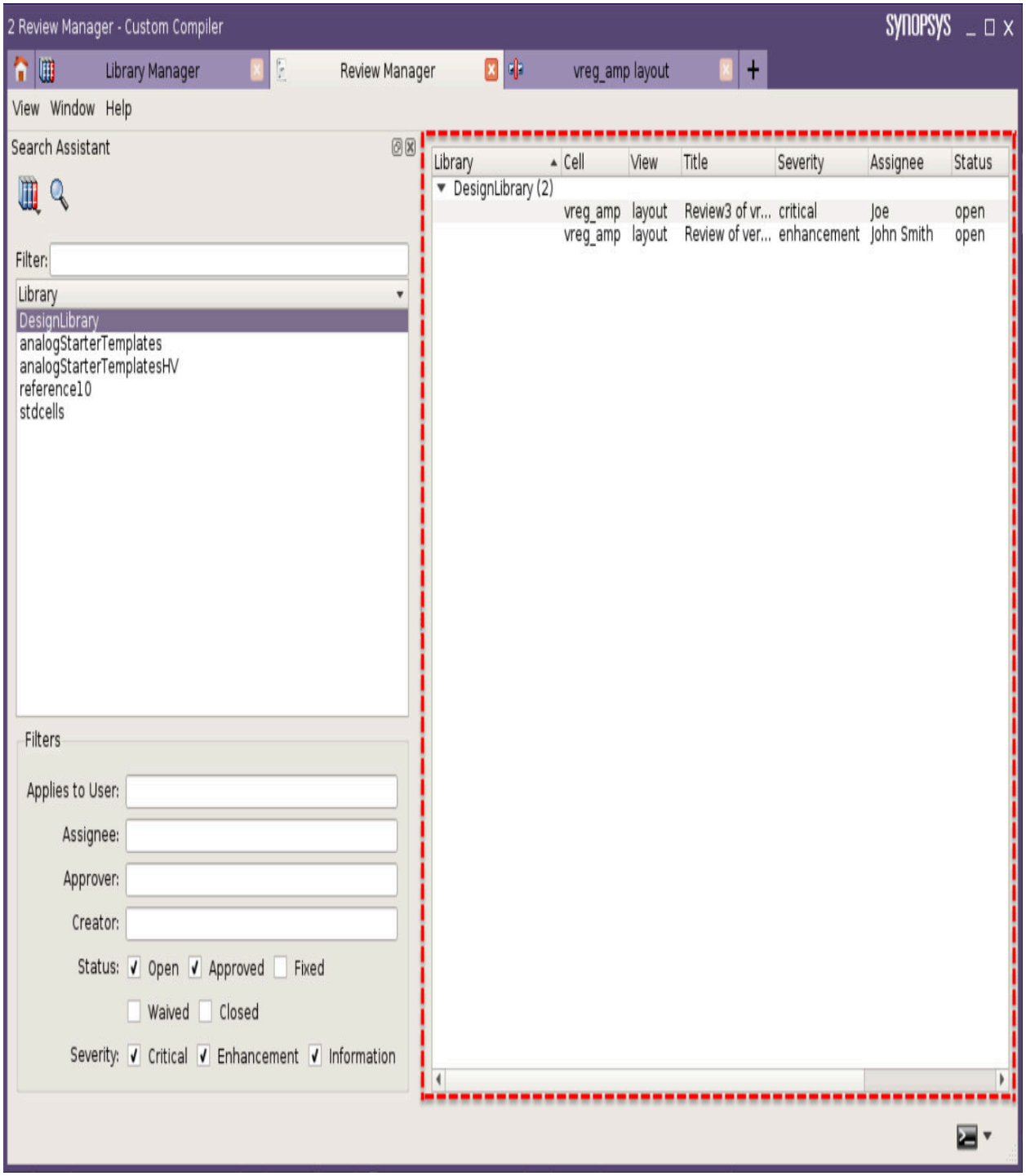
Design Hierarchy



4. Click Search .

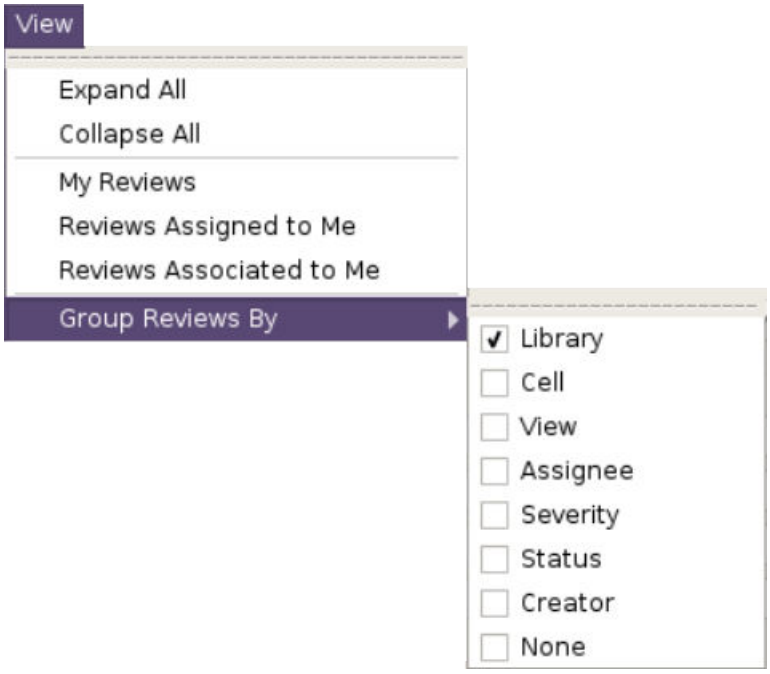
   The list of reviews appears.

Feedback



5. To further filter the results, click **View** and choose a command.

Feedback



6. To open a review or issue, double-click the review name or issue name.

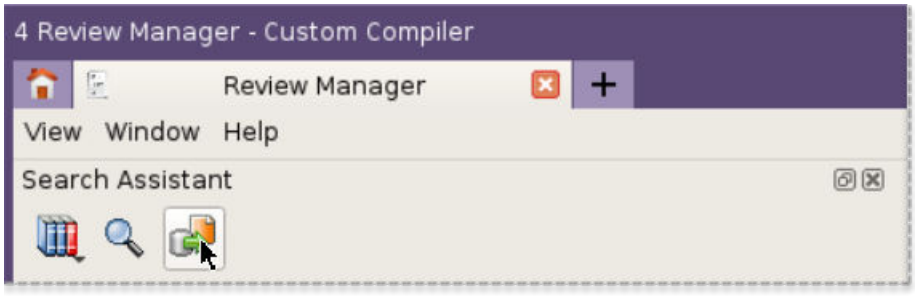
   The review or issue opens in the Review Assistant.

Exporting Reviews

You can export specified reviews to an HTML file to view the design review information without having to run the Custom Compiler tool.

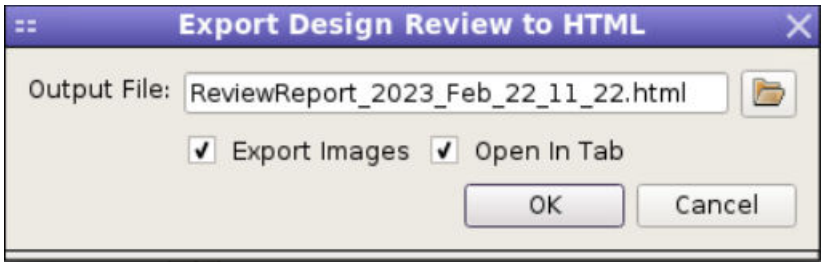
To export reviews:

1. Open the review that you want to export using the Review Manager. See Opening an Existing Review.
2. On the **Search Assistant** toolbar, click **Export Review**.



Feedback

The **Export Design Review to HTML** dialog box opens.



3. In the **Output File** box, specify the directory and name for the exported file. If you do not specify a directory, the review data is exported to <current_working_directory>/synopsys_custom/reports.
4. Select **Export Images** to export the corresponding images from the review.
5. Select **Open in Tab** to open the HTML file containing the exported review data in a new tab in the Custom Compiler window.
6. Click **OK**.

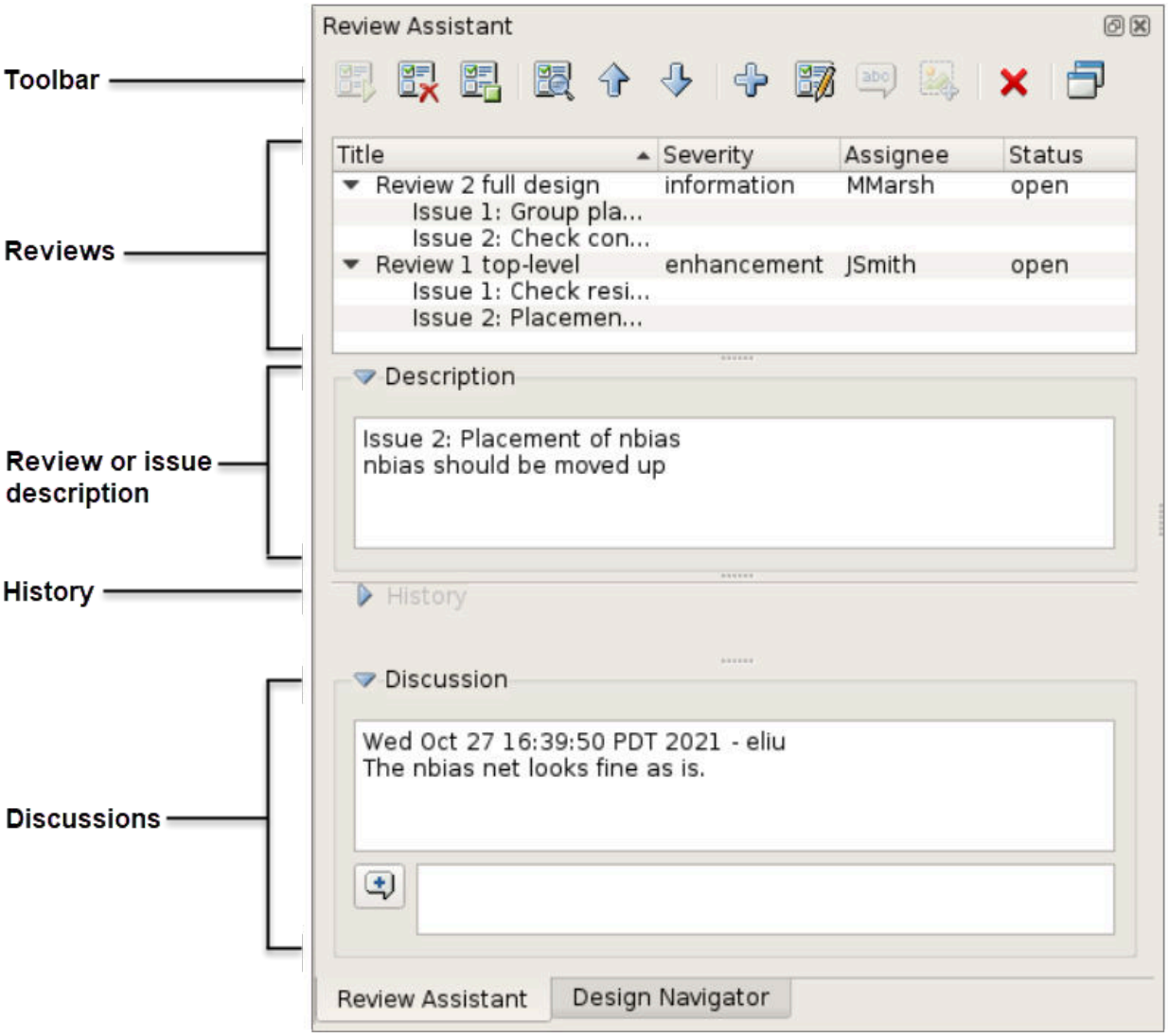
   The review data is exported to an HTML file in the default output file directory or in the directory you specified. If you selected Open in Tab, the HTML file opens immediately in a separate tab in the Custom Compiler window.

Review Assistant Reference

This topic provides a quick reference to the Review Assistant components.

Feedback

The Review Assistant consists of the following:



Component	Description
Toolbar	Contains commands that let you start and end reviews, edit and delete reviews, add annotations, add images, and so on.
Reviews	Lists open reviews (or reviews that have not yet ended). Under each review is the list of issues that reviewers have added.
Review or issue description	Displays the description of the review or issue you selected in the Reviews section.

Feedback

Component	Description
History	Displays the log of events in the Review Assistant that show when reviews or issues were created and who created them.
Discussions	Lets you add and respond to comments about a selected review.

The toolbar consists of the following:

Button	Description
Start Review	Enables the review functions so that you can add and edit reviews and issues.
Discard Review	Deletes all reviews associated with the design.
End Review	Changes the review to the read-only mode. You can select reviews and issues to view annotations, but you cannot add or edit reviews or issues.
Show Issue	Zooms in to the location of the issue or annotation.
Show Previous Issue	Displays the previous issue in the review.
Show Next Issue	Displays the next issue in the review.
New Review	Creates a new review in the design.
Edit Review/Issue	Lets you edit the selected review or issue.

Feedback

Button	Description
Add Annotation	Lets you add annotations to the selected review. When you add an annotation, you are creating an issue in the review.
Add Image	Adds a screen capture of the viewport to the issue.
Delete Selected	Deletes the selected review or issue.
Restore Window	Hides the review annotations so that you can view the original design.

10

Tools for Job Status and Monitoring

This topic describes how to use monitoring tools with Custom Compiler to track and improve the tool performance.

This chapter contains the following topics:

- Job Monitor Tool
- Sentry Monitor Library Tool
- Watchdog Tool

Job Monitor Tool

Use the Job Monitor to track running job processes.

By default, the Job Monitor tracks batch processes. Interactive process tracking is also available. (See Viewing Running Job Processes).

In addition to using the Job Monitor to track jobs, you can use Tcl preferences to add *job classes* that combine multiple grid settings into one setting. You can also use Tcl preferences to add job process waiting to allow scripts to finish without interruption.

This topic describes the following:

- Displaying the Job Monitor Tool
- Viewing Running Job Processes
- Applying Job Process Controls
- Managing Job Options
- Filtering Job Groups
- Displaying a Netlist
- Displaying the Output
- Launching a Terminal

Feedback

- Adding Job Classes With xt::createJobClass Tcl
- Adding Job Process Waiting With xt::wait Tcl

Displaying the Job Monitor Tool

Open the Job Monitor to view current session's batch and interactive processes.

To open the Job Monitor:

► In any design window, choose **Tools > Job Monitor**.

Viewing Running Job Processes

You can view job processes as they run with the Job Monitor.

To view a running job process:

1. Begin by opening the Job Monitor Tool. In any design window, choose **Tools > Job Monitor**.
2. In the Job Monitor, choose **View**.
3. In the **View** menu, toggle Job Monitor View Options. The system adds and removes tracking columns as directed.

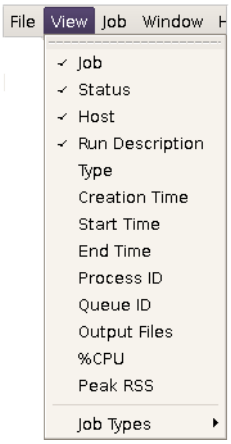
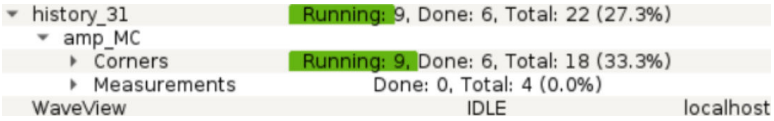


Table 109 describes the Job Monitor view options.

Table 109 Job Monitor View Options

Option	Description
Job	Displays a tree of job groups and jobs, with leaf nodes for each running application.

Feedback

Option	Description
Status	Displays the status of the process, such as FINISHED, QUEUED, RUNNING, or KILLED. Progress feedback for the job is displayed as a bar graph along with the percentage completed. In addition, the status text is color-coded as follows: ◦ FAILED - bright red ◦ KILLED - dark red ◦ Others - black (default text color) Double-click the status to open the Text Viewer and view associated output files. It also shows the status of the jobs Running, Done, Total (% of Done) while simulation is running for different sections like History, Corners, MonteCarlo, Measurements, and so on. 
Host	Name of host associated with this job, with local host noted if the job is running on this system.
Run Description	Displays the description associated with the job.
Type	The job type (group, batch, or interactive).
Creation Time	Specifies the job creation time.
Start Time	Time job was initiated.
End Time	Time job ends.
Process ID	ID as listed on the host.
Queue ID	The grid identifier associated with the job (only applies if the job was launched via SGE or LSF).
Output Files	Expected output files from this job as defined when the job was created.
%CPU	CPU usage of the job.
Peak RSS	Peak resident memory of the job.
Job Types	Toggles between the display of batch and interactive jobs.

4. You can choose to show interactive job processes in addition to batch job processes by choosing **View > Job Types**, and toggling the **Interactive** option. Alternatively, use the `xtJobMonitorShowInteractiveJobs` preference.
5. Choosing **File > Close** closes the Job Monitor.

Feedback

Applying Job Process Controls

You can apply job process controls that suspend, resume, kill, and delete jobs.

To apply job process controls:

1. Begin by opening the Job Monitor Tool. In any design window, choose **Tools > Job Monitor**.
2. In the Job Monitor, choose **Job**.
3. Select the job by name.

   (This step is not required when you are deleting all completed jobs)
4. Choose one of the job process controls.

   Table 110 describes the job process controls.

Table 110 Job Process Controls

Option	Description
Suspend	Halts this process
Resume	Resumes the halted process
Kill Selected Jobs	Ends this process
Kill Selected Jobs and All Remaining Jobs	Ends this process and any remaining processes
View Output	Displays output in the Text Viewer
Delete	Removes this job
Delete All Completed	Removes records of all completed jobs

5. Choosing **File > Close** closes the Job Monitor.

Managing Job Options

You can manage job options to control batch processing limits, and to optionally set a queue engine. You can optionally specify a job class, which assigns queue engine resources based on a predefined job type.

Feedback

To manage job options:

1. Begin by opening the Job Monitor Tool. In any design window, choose **Tools > Job Monitor**.
2. In the Job Monitor, choose **File > Options**.
3. Choose job management options.

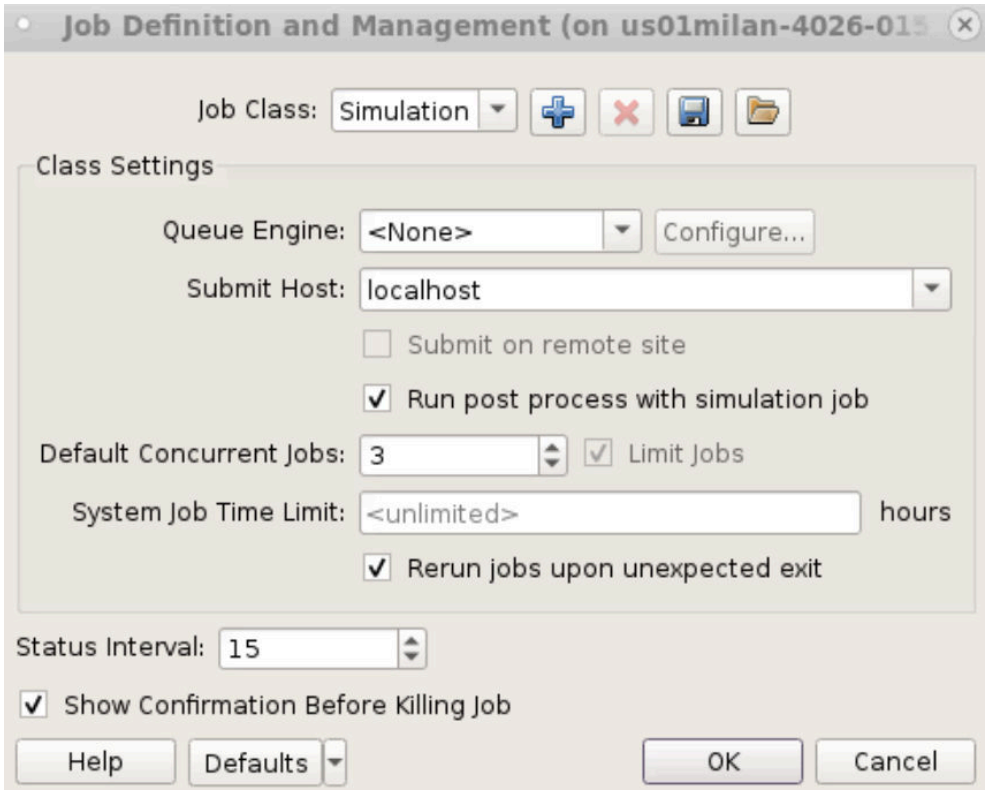


Table 111 describes the job management options.

Table 111 Job Management Options

Option	Description
Job Class	Specifies a setting that automatically includes an analysis process, a queue engine, whether concurrent jobs are allowed, and the number of allowable concurrent jobs. CAD Support teams can add custom job classes. For details, see Adding Job Classes With xt::createJobClass Tcl.

Feedback

Option	Description
Class Settings	**Queue Engine** sets the distributed computing engine for batch processing to SGE (Sun Grid Engine), LSF (Load Sharing Facility), NetworkComputer, SLURM, no queue engine, or custom engine. To set additional options for an engine, click **Configure**, and choose **Configuration** options. (Engine configuration information is provided in SolvNetPlus article #000031229 "Setting Up the Computer Network") **Note:** You must be logged in to SolvNetPlus for the link to connect directly to the article. If you are prompted to log in to SolvNetPlus upon clicking the link to the article, log in, then click the link again to reach the article. **Submit Host** is the host where the jobs are launched. Choose **Submit on remote site** if this job class uses a remote (WAN) grid. **Limit Jobs** limits the concurrent jobs in this job class to the number you set in the **Concurrent Jobs** box. **Concurrent Jobs** sets the number of allowable jobs to process at one time, based on the job class you have set above. (You can create custom job classes. See Adding Job Classes With xt::createJobClass Tcl).
Status Interval	Sets the number of seconds between each queue engine status poll.
Show Confirmation Before Killing Job	Allows you to control whether or not the **Kill Jobs** prompt appears before you end a job.
Run post process with simulation job	Enables immediate measurement calculation in same remote worker after the simulation is finished. This helps in improving performance. This check box is enabled by default. When this check box is enabled, the **Measurement Jobs** section of the **Setup > Simulator** dialog box is disabled. However, when this check box is disabled, the **Measurement Jobs** section is enabled with the **Share Resource with Simulation** check box enabled by default.
Rerun jobs upon unexpected exit	Enable to rerun the jobs when user finds the jobs terminated within the grid due to unexpected reasons like terminated worker. This check box is enabled by default.

If you want to limit jobs to control license usage, and you do not have access to a queue engine for distributed processing, you can set the number of concurrent jobs to 1. This effectively creates a queue.

Feedback

4. Click **OK**.
5. Choosing **File > Close** closes the Job Monitor.

Note:

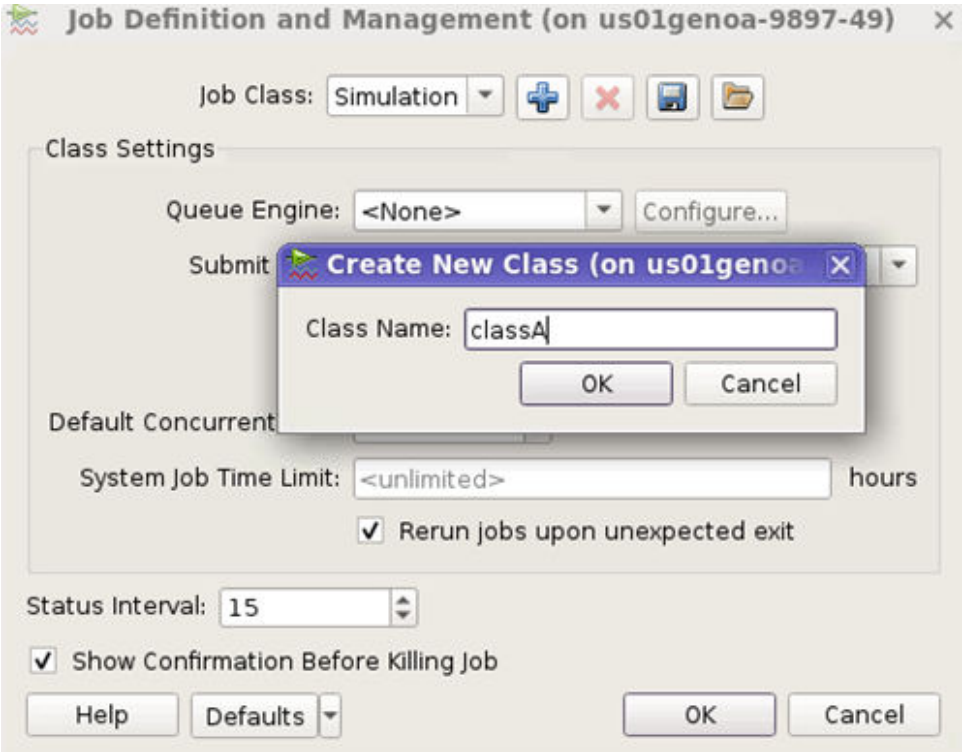
See SolvNetPlus article #000031229 "Setting Up the Computer Network" for more details on configuring the **Job Management Options** dialog box.

Creating Custom Job Class

PrimeWave has two job classes by default – Simulation and SAEShell. These are defined to simulate within a local host assuming there is no grid available and can allow maximum of 3 and 2 concurrent jobs respectively. In a custom environment, the simulation jobs are usually outsourced to a grid containing machines with high processing capability. Some of the popular Queue Engines supported in PrimeWave Design Environment are SGE, LSF, SLURM, and Network Computer.

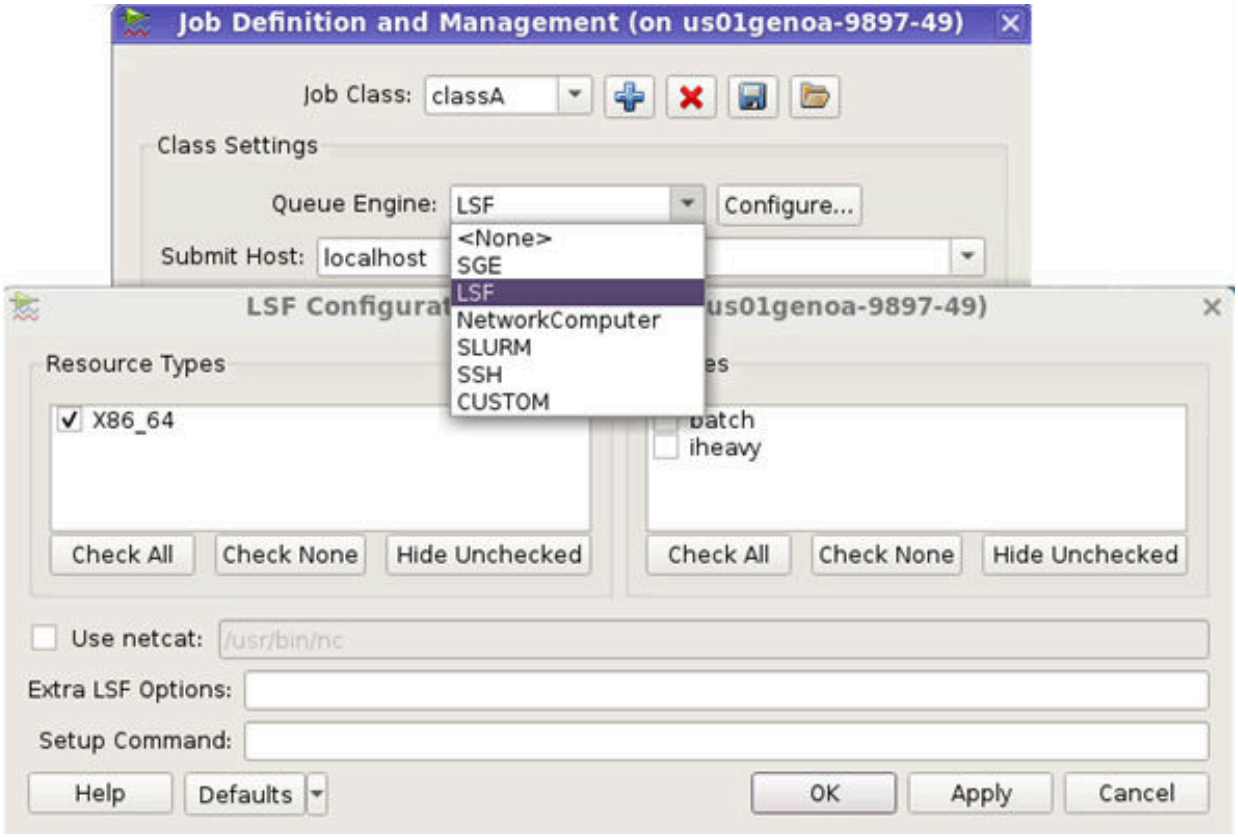
To setup a new job class:

1. Click **+** beside the job class. Add the name of the custom job class you want to add.



2. For a newly added class, select a relevant **Queue Engine** available in your network. Click **Configure** to see the additional options related to the selected queue engine. For example, when LSF is selected, the available queues are displayed.

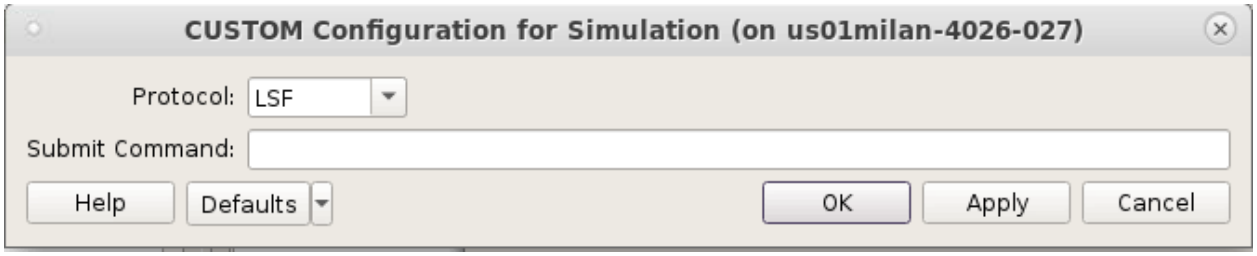
Feedback



3. Use the check box to mention the feasible queues to which the simulation job can be submitted. Additional LSF flags and options can be added to the **Extra LSF Options** field. If the grid machine requires any pre-simulation settings, the complete shell command top prepares the environment before executing the job submit command and can be added to the **Setup Command** field.
4. The newly created job class appears in the **Job Class** drop-down menu, sorted alphabetically. To use the job class for the current testbench, select it from **Setup > Simulator > Simulation Jobs**. The **Job Class** drop-down displays all available job classes, including the newly added ones. For more information, see .
5. If you want to distribute a job with customized submit command or customized grid protocol, use CUSTOM queue engine to setup the job class. If the environment supports a custom queue engine that is not in the earlier list, you can select **CUSTOM**

Feedback

in the drop-down menu. The relevant protocol can be chosen from the drop-down menu. The **Submit Command** can be added to the configuration.

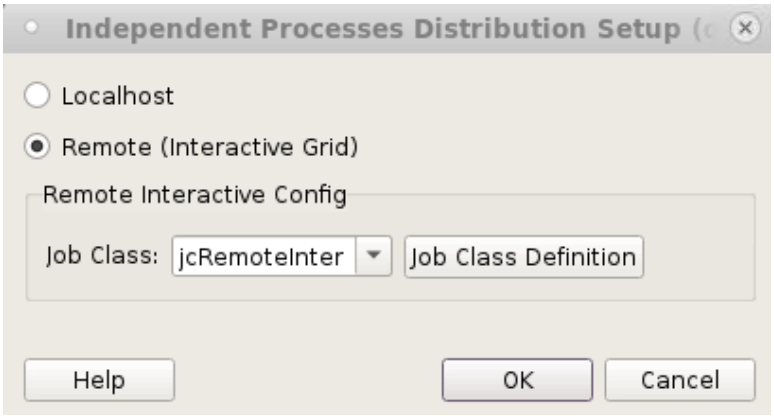


Setting Up Independent Processes Distribution

PrimeWave processes like WaveView, Results Viewer, and Master Calculation Engine can be distributed to a remote grid with an interactive grid job class.

To distribute the processes to remote:

1. Open the **Independent Processes Distribution Setup** dialog box by selecting **Tools > Independent Processes Distribution.**
2. Select the **Remote (Interactive Grid)** option.
3. From the **Job Class** drop-down, select the existing job class to be used for the remote distribution.
4. If you want to create a new job class to be used for these PrimeWave process, click **Job Class Definition** to open the **Job Definition and Management** dialog box for setting up a new job class.
5. Click **OK**.



Customizing the Job Name

You can customize the job name when launching simulations to the grid engine. The procedure `::sa::run::createJobArrayName` should be used to override the default

Feedback

implementation in which the format <testbenchName_simulatorName> is used for the job name.

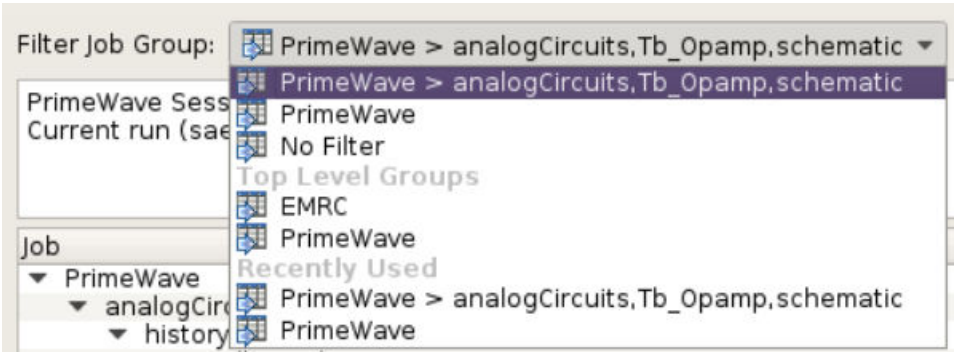
```
namespace eval ::sa::run {
proc createJobArrayName { tb }
{ set tbName [db::getAttr tb.name] set simName
[db::getAttr tb.simulator]
return "${tbName}_${simName}" }
}
```

Filtering Job Groups

You can filter jobs that are visible in the Job Monitor, for example, to change from a specific PrimeWave Design Environment session to a more general one. The Job Monitor also remembers the previous filters so you can go back and forth between filtered views.

Select the job group you want to filter from the **Filter Job Group** menu.

Figure 56 Filter Job Group



You can also change the current filter to a selected job group by right-clicking on a job group in the job tree and choosing **Filter to this job group** from the menu.

Displaying a Netlist

You can display a netlist from the Job Monitor. To display the netlist, select a specific iteration, right-click to display the menu, and choose **View Netlist**. The netlist opens in the Text Viewer on a new tab.

Displaying the Output

You can display the output from the Job Monitor. To display the output, select a **Job**, right-click to display the menu, and choose **View Output**. The output log files open in the Text Viewer on a new tab.

Feedback

Launching a Terminal

You can launch a terminal from the Job Monitor. To launch a terminal, select a **Job**, right-click to display the menu, and choose **Launch Terminal**. A new terminal opens.

Adding Job Classes With xt::createJobClass Tcl

You can create *job classes* that combine settings for individual analysis tools and system environment grids into one setting option.

For example, if you want PrimeSim HSPICE simulation jobs to run on Solaris-64 machines via LSF, and to limit the number of concurrent jobs to 10, you can create a job class that contains these settings.

In use, designers can open the Job Monitor, and select this job class to apply the settings.

This documentation provides two script samples that create custom job classes. To use them, choose the one that most closely matches your requirements, adjust values to create an exact match, and implement the script. You can also write an original script using the supplied Tcl commands and preferences.

This section covers Tcl samples:

- Sample Job Class for Design Rule Checking on Linux Systems
- Sample Job Class for PrimeSim HSPICE Simulation on Solaris Systems

Sample Job Class for Design Rule Checking on Linux Systems

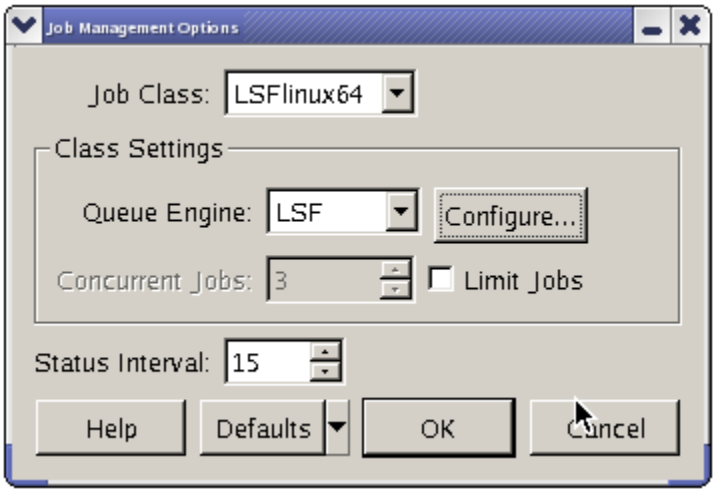
This Tcl sample configures PrimeSim XA and ICV tool DRC jobs to run on Linux-64 machines via LSF, without limiting the number of concurrent jobs.

You can implement this job class using the sample code that follows.

In use, designers can specify this job class by opening the Job Monitor, choosing **File Options**, and selecting **LSFlinux64** as shown in Figure 57.

Feedback

Figure 57 Sample Job Class for Design Rule Checking on Linux Systems




```
# Job Class for Design Rule Checking on Linux Systems# For reference,
 this sample lists available job class options.# General options:
#     JobQueueEngine
#     JobLimit
#     JobLimitEnabled
# LSF-specific options:
#     LSFQueues
#     LSFResources
#     LSFExtraOptions
# SGE-specific options:
#     SGEProjects
#     SGESelectedProject
#     SGEExtraOptions
#
# Configure XA and ICV DRC jobs to run on
# linux-64 machines via LSF with no limiting.
#
set lsflnx [xt::createJobClass LSFlinux64]
db::setPrefValue [xt::getJobClassOptionPrefName $lsflnx \
   -option "JobQueueEngine"] -value "LSF"
db::setPrefValue [xt::getJobClassOptionPrefName $lsflnx \
   -option "JobLimitEnabled"] -value false
db::setPrefValue [xt::getJobClassOptionPrefName $lsflnx \
   -option "LSFResources"] -value "SOL64 0 X86_64 1"
xt::addJobClassPrefix XA -jobClass $lsflnx
xt::addJobClassPrefix icv_drc -jobClass $lsflnx
#
```

See Also

- Sample Job Class for PrimeSim HSPICE Simulation on Solaris Systems
- Adding Job Classes With xt::createJobClass Tcl

Feedback

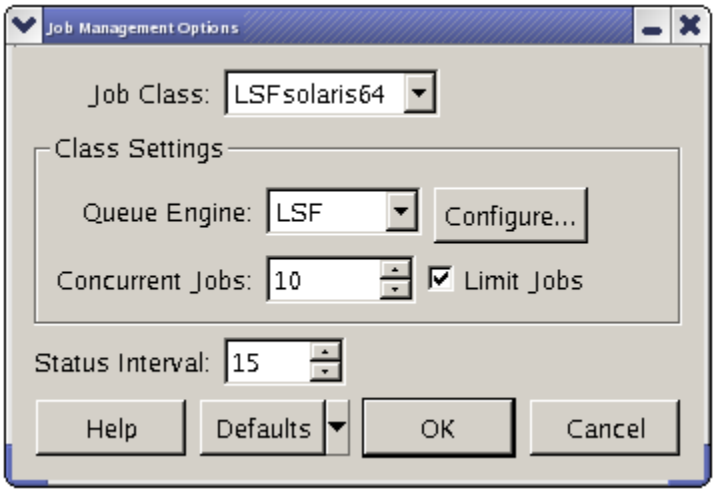
Sample Job Class for PrimeSim HSPICE Simulation on Solaris Systems

This Tcl sample configures PrimeSim HSPICE simulation jobs to run on Solaris-64 machines via LSF, and limits the number of concurrent jobs to 10.

You can implement this job class using the sample code that follows.

In use, designers can specify this job class by opening the Job Monitor, choosing **File Options**, and selecting **LSFsolaris64** (Figure 58).

Figure 58 Sample Job Class for PrimeSim HSPICE Simulation on Solaris Systems




```
# Job Class for PrimeSim HSPICE Simulation on Solaris Systems# For
 reference, this sample lists available job class options.
#
# General options:
#     JobQueueEngine
#     JobLimit
#     JobLimitEnabled
# LSF-specific options:
#     LSFQueues
#     LSFResources
#     LSFExtraOptions
# SGE-specific options:
#     SGEProjects
#     SGESelectedProject
#     SGEExtraOptions
#
# Configure PrimeSim HSPICE jobs to run on solaris-64 machines
# via LSF, limited to 10 jobs at a time.
#
set lsfsol [xt::createJobClass LSFsolaris64]
db::setPrefValue [xt::getJobClassOptionPrefName $lsfsol \
```

Feedback

```
   -option "JobQueueEngine"] -value "LSF"
db::setPrefValue [xt::getJobClassOptionPrefName $lsfsol \
   -option "JobLimit"] -value 10
db::setPrefValue [xt::getJobClassOptionPrefName $lsfsol \
   -option "JobLimitEnabled"] -value true
db::setPrefValue [xt::getJobClassOptionPrefName $lsfsol \
   -option "LSFResources"] -value "SOL64 1 X86_64 0"
xt::addJobClassPrefix HSPICE -jobClass $lsfsol
#
```

See Also

- Sample Job Class for Design Rule Checking on Linux Systems
- Adding Job Classes With xt::createJobClass Tcl

Adding Job Process Waiting With xt::wait Tcl

In Tcl scripts, you can add job process waiting, which blocks user interaction until currently running jobs finish. Job process waiting is useful when your script needs to use the results of any currently running job as input for its next operation.

To add job process waiting to a Tcl script:

1. Identify the job.
   - If you created the job using the `xt::createJob` command, use the returned job object.
   - If you want to add waiting to a standard system job process, use the job as listed in the Job Monitor.

     To find the job, run the process once using the graphic user interface. Then, in the Job Monitor **Job** column, take note of the process name.

     For example, to add waiting to the IC Compiler export process, choose **Export > To ICC**, and run the process. In the Job Monitor **Job** column, note that the Job Monitor identifies the process as **dbExportICC**.
2. Using the identified job, add the `xt::wait` command to the Tcl script at the line where you want the script to wait.

   In the example that follows, `xt::getJobs` gets all IC Compiler export jobs. It uses xt::wait to cause the system to wait for these jobs to finish before executing the next Tcl command. While the system waits, it blocks user input. After the jobs finish, the script continues.

   The system individually names each job process by appending the underscore character (_) and adding incremental numbering. For instance, the first dbExportICC

Feedback

job is dbExportICC_1 and the fifteenth is dbExportICC_15. The example uses the wildcard character (*) to capture all dbExportICC jobs.

```
# wait for any dbExportICC jobs to finish
db::foreach xtJob [xt::getJobs dbExportICC*] {
 xt::wait $xtJob
}
```

3. Implement your Tcl script.

   For details about running Tcl scripts on demand, see Running Tcl Scripts.

   For information about making Tcl scripts available from the installation tree, see Defining Startup Settings.

Sentry Monitor Library Tool

The Sentry Monitor Library is a computer and network monitoring tool that reports error messages for computer and network-related performance issues while you are running a Synopsys software product.

See Synopsys Sentry Monitor Library Error Messages.

Note:

You must be logged in to SolvNetPlus for the earlier link to connect directly to the article. If you are prompted to log in to SolvNetPlus upon clicking the link to the article, log in, click the link again to reach the article.

The Sentry Monitor Library monitors and reports the following types of issues:

- High CPU use (host and processes)
- High memory use (physical memory, cache, and swap)
- Poor network conditions (packet transmission issues, host latency)

Watchdog Tool

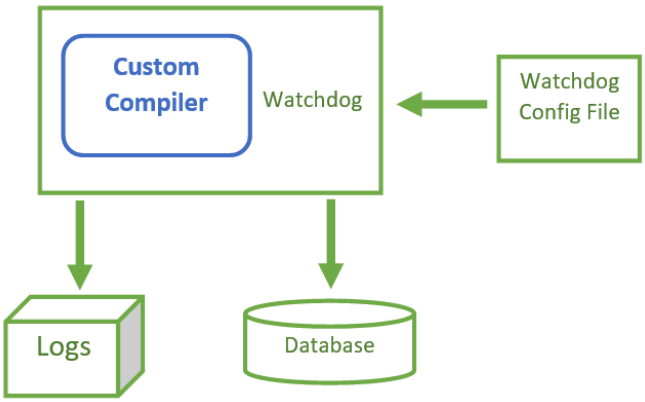
The Watchdog tool provides information to assist Design and CAD teams in analyzing tool usage and quality for their users. The main functionality of the Watchdog tool is to collect statistical information about Custom Compiler sessions. This version of the Watchdog tool can also copy Custom Compiler log files (only software failures or all logs) to a central location. By dividing the total session open time by the number of failures, you can calculate the failure rate of the Custom Compiler tool. This data can also be used to analyze common failures and work with your Synopsys representative to determine root causes and resolve them more quickly. Overall, the Watchdog tool can help increase the stability of the Custom Compiler tool.

Feedback

The Watchdog tool uses the process ID of the Custom Compiler session. It is attached to the Custom Compiler session and starts a collection of statistics and machine information.

The Watchdog tool writes the collected data into a database and copies the log file after the session is ended.

The collection of statistics and copying of log files can be configured in the Watchdog config file.



This topic describes the following:

- Installing the Watchdog Tool
- Watchdog Outputs
- Watchdog Report Statistics

Installing the Watchdog Tool

To install the Watchdog tool:

1. Create a directory to collect all watchdog statistic data including log files.

   Note:

   All users should have read-write access to the created directory.

   The directory should be available to all users and from all machines (that is, a shared location).

2. Add the following environment variable:

   bash:

   ```
   export SYNOPSYS_CUSTOM_WATCHDOG_DIR=<Directory from 1st step>
   ```

Feedback

csh:

```
setenv SYNOPSYS_CUSTOM_WATCHDOG_DIR <Directory from 1st step>
```

3. Create a `watchdog.cfg` configuration file in the `$SYNOPSYS_CUSTOM_WATCHDOG_DIR` directory.

   The configuration file should contain the following:

   ```
   [Watchdog]
   collectStatistics = True/False
   collectLogs = True/False
   collectNonCrashLogs = True/False
   ```

   The preceding options control the following features of the Watchdog tool:

   - `collectStatistics` - Enables collection of statistical data in the `$SYNOPSYS_CUSTOM_WATCHDOG_DIR/data.sqlite` database.
   - `collectLogs` - Enables copying of software failure logs only into the Watchdog statistics directory (from step 2).
   - `collectNonCrashLogs` - Works with the `collectLogs` option and enables copying non-failure logs, as well.

The Watchdog tool runs in the background for each Custom Compiler session after you complete the preceding installation procedure.

Watchdog Outputs

The Watchdog tool saves all statistical information into a database and copies log files into a central location. Both features can be disabled and enabled in the configuration file.

Database

The Watchdog tool collects all statistic information for all users in one SqLite database named `data.sqlite`.

Database Structure

Statistics data for each version of the Custom Compiler tool is collected in separate tables in the database.

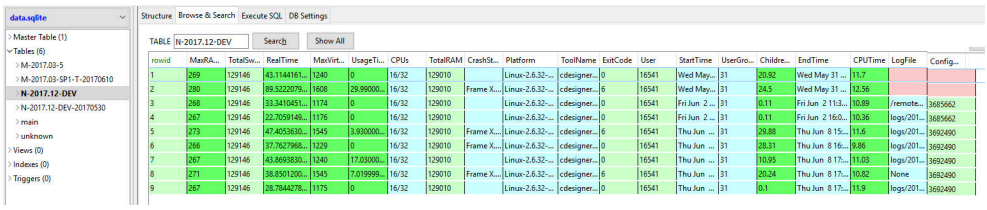
Each table has the following columns:

- Maximum RAM and Virtual Memory usage
- CPU Time for parent and children processes (children processes are external processes run via the Custom Compiler tool)

Feedback

- User usage time based on command execution frequency
- Start and End Times (Real time)
- Crash stack and exit code
- Copied log file location (nothing if feature disabled)
- Machine information:
  - Total RAM
  - Total Swap
  - CPU count
  - Platform
- User information:
  - User ID
  - User Group
- Tool version information:
  - Tool name
  - Config number

Database example:



Log Files

All copied log files can be found in the root `logs` directory located in the Watchdog output directory. The log files are copied into `yyyy/month/dd` directories corresponding to the date of copy.

Log directory example:

```
Logs
 +-2023
 |  +-Dec
 |     +-26
 |     |  +-<copied log files>
 |     +-27
```

Feedback

```
|     |  +-<copied log files>
|     +-29
|        +-<copied log files>
+-2023
   +-Jan
   |  +-04
   |  |  +-<copied log files>
   |  +-08
   |  |  +-<copied log files>
   |  +-21
   |     +-<copied log files>
   +-Feb
      +-01
         +-<copied log files>
```

Watchdog Report Statistics

The `wreport.py` utility can be used to generate high-level report from watchdog database. It is in the watchdog directory (for example, `<CC install dir>/auxx/watchdog/wreport.py`).

The usage of the utility is the following:

```
wreport.py [-r <release name>] -d <database location>
```

Use `-h(--help)` for help.

If the release option is not set, the utility reports for all existing releases.

The following is a simple example of `wreport.py` output:

```
<Release name>
 Total Sessions   : 97
 Crashed Sessions : 6
 Unique Crashes   : 4
 Users            : 7
 CPU Time         : 4.865 hours of cpu usage
 Elapsed Time     : 117.879 hours tool was up
 Usage Time       : 4.745 hours tool actively used
 MTBC             : 0.791 hours (Usage Time / Crashed Sessions)
 Longest Session  : 0.783 hours across all sessions
                  : 0.029 hours across crashed sessions
 Shortest Session : 0.000 hours across all sessions
                  : 0.000 hours across crashed sessions
 Average Session  : 0.049 hours across all sessions
                  : 0.012 hours across crashed sessions
```

Query on Database

If the report generated by the `wreport.py` does not provide enough information, you can use database queries for get more information.

Feedback

You can get more database info using the following methods:

- sqlite3 Command
- List the Tables
- Convert Database
- Number of Sessions

sqlite3 Command

Use the sqlite3 command:

```
sqlite3 [OPTIONS] database [SQL Commands]
```

List the Tables

`data.sqlite` can contain several tables which are corresponding to the versions of the Custom Compiler tool.

To list the names of the tables you can use the following command:

```
sqlite3 data.sqlite "SELECT name FROM sqlite_master WHERE type='table'"
```

Choose the version you want and do the query on it.

Convert Database

Sqlite3 allows you to format the output. The useful formats are CSV and HTML.

Use the following command to convert `data.sqlite` to CSV:

```
sqlite3 -header -csv data.sqlite "SELECT * FROM '<table name>'" >
 {output}
```

Number of Sessions

Use the following command to get the number of overall sessions for the version of the Custom Compiler tool:

```
sqlite3 data.sqlite "SELECT count(*) FROM '<table name>'"
```

Use the following command to get the number of crashed sessions:

```
sqlite3 data.sqlite "SELECT count(*) FROM '<table name>' WHERE ExitCode
 <> 'None' AND ExitCode <> 0"
```

11

Customizing the Software and Workspaces

The Custom Compiler environment provides ways for you to customize the user interface to meet your requirements and preferences. For example, you can pin tabs or define Custom workspaces to retain the arrangement of tabs, toolbars, assistants, and canvas. You can also configure the graphical user interface and add dialog boxes.

This chapter contains the following topics:

- Defining Custom Workspaces
- Configuring Custom Windows
- Creating Dialog Boxes
- Adding Assistants and Other User Interfaces Using Tcl/Tk
- Tcl Sample for Custom Graphic User Interface

Defining Custom Workspaces

You can define Custom workspaces to retain the arrangement of tabs, toolbars, assistants, and canvas. Different workspaces allow users with varying requirements and preferences to work from the same computer.

These sections cover:

- About Custom Workspaces
- Creating Workspaces
- Saving Workspaces
- Loading Workspaces
- Switching to Another Workspace
- Renaming Workspaces

Feedback

- Deleting Workspaces
- Preserving Workspace Geometries

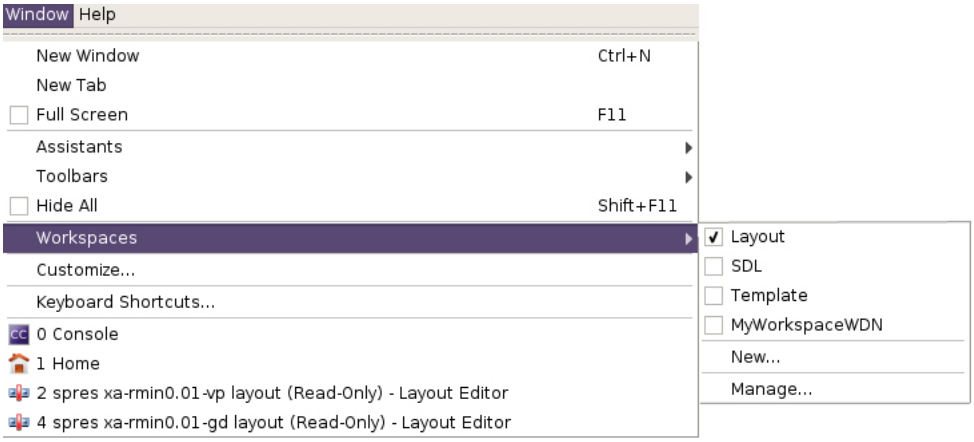
About Custom Workspaces

Custom workspaces preserve arrangements you make to the menus, toolbars, and assistants of a window type. Window types are editor and manager windows, such as the Layout Editor and Library Manager.

Multiple workspaces can be made for the design editors, allowing you to customize design sessions to your preference. During a design session, you can choose which workspace to display at any given time.

For example, you can set up a schematic window with the toolbar and assistant arrangement preferred for schematic editing. You can control the size and positions of all schematics as they open.

After arranging the workspace, you can save it using the **Window > Workspaces** menu. Changes to an active workspace are preserved when closing active sessions.



In the **Manage Workspaces** dialog box, you can save an XML file containing the workspace definition. You can load the XML file to restore a missing workspace or to modify an existing workspace.

The layout and schematic windows provide default workspaces that you can modify and save as custom workspaces. Default workspaces are also available for work in SDL and templates. Although you cannot customize the SDL and template workspaces, these workspaces will remember your setup between sessions.

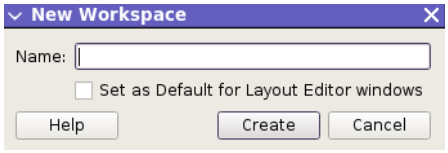
You can save a custom window you have defined as a custom workspace. To define a custom window, see Configuring Custom Windows.

Creating Workspaces

You can create workspaces to present a consistent user interface for design from one session to the next session.

To create a custom workspace:

1. In a Custom Compiler session, open a design, and position windows, toolbars, and assistants in the preferred arrangement.



2. Choose **Window > Workspaces > New**.
3. In the **New Workspace** dialog box, type a name for this new workspace.
4. (Optional) Select **Set as Default for Schematic Editor windows** or **Set as Default for Layout Editor windows** to make this workspace configuration the default for the window type.
5. Click **Create**.

   The workspace name is added to the **Window > Workspaces** menu.

   Changes to the workspace are automatically saved when closing designs. To save your workspace as an XML file, see Saving Workspaces.

Saving Workspaces

Save a custom workspace so that it is available across different design sessions, and to retain modifications made in this session.

To save a workspace:

1. Choose **Window > Workspaces > Manage**.
2. In the **Manage Workspaces** dialog box, select the workspaces to save.
3. Choose **File > Save**, or click **Save**.
4. In the **Select File to Save Workspace(s)** dialog box, type the name of the file in which to save the workspaces. The system saves workspace files in the XML format.
5. Click **Save**.

Feedback

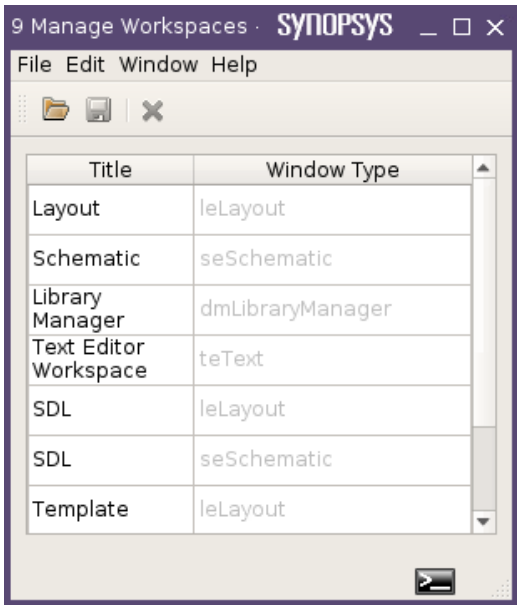
Loading Workspaces

Important: Make sure that the workspace you are loading is created and displayed in the **Manage Workspaces** dialog box.

To load a workspace:

1. Choose **Window > Workspaces > Manage**.

   The **Manage Workspaces** dialog box opens and lists the available workspaces and the corresponding window types.



2. Choose **File > Load**, or click **Load** .
3. In the **Select Workspace File to Open** dialog box, locate and select the workspace (.xml) file.
4. Click **Open**.

Switching to Another Workspace

Important: If you do not see the workspace you want to display, try loading it first. See Loading Workspaces.

To switch to another workspace:

► Choose **Window > Workspaces >** <workspace_name>.

A check mark next to the workspace name indicates that the workspace is displayed.

Renaming Workspaces

All workspaces except the default workspaces are available for renaming.

To rename a workspace:

1. Choose **Window > Workspaces > Manage**.
2. Select the workspace name, and then choose **Edit > Rename**.
3. Type a unique name for the workspace.
4. Press Enter.

Deleting Workspaces

Delete a custom workspace to remove it from your list of workspaces.

Caution:

You cannot undo this operation. Workspaces that have not been saved to an XML file cannot be retrieved.

To delete a workspace:

1. Choose **Window > Workspaces > Manage**.
2. In the **Manage Workspace(s)** dialog box, select the workspaces you want to delete.
3. Choose **Edit > Delete**, or click **Delete** .

   To retrieve a deleted workspace from an XML file, see Loading Workspaces.

Preserving Workspace Geometries

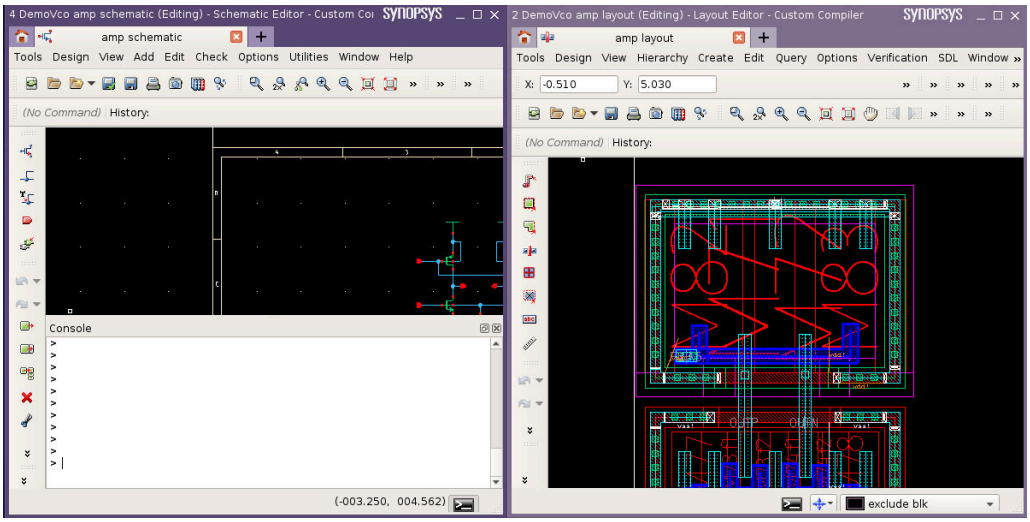
You can preserve the size and position of a window of your current session in a future session in the following cases:

- The **Window** tabs are disabled.
- The **Window** tabs are enabled and the current window is the only window in the frame.

For example, the you can configure the window geometry of the **Search and Replace** dialog box (without a frame) with your current window and save this geometry for future sessions.

Feedback

Further, if the **Layout Editor** and **Schematic Editor** windows are opened in separate frames in the SDL flow, you can configure the window geometries and save these for future sessions.

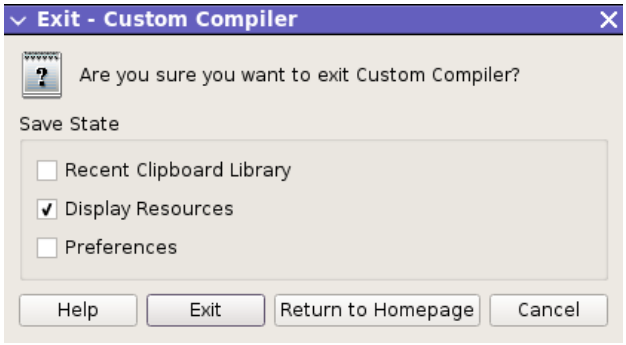


To open the **Layout Editor** and **Schematic Editor** windows in separate frames, set the `lxOpenInNewFrames` and `lxAutoArrangeWindows` Tcl preferences to `true`, as shown in the following example:

```
db::setPrefValue lxOpenInNewFrames -value true
db::setPrefValue lxAutoArrangeWindows -value true
```

To preserve the window geometries in a future session:

- Choose **File** > **Quit** from the Home tab. The **Exit** dialog box opens. Select **Display Resources** check box and click **Exit**.



Configuring Custom Windows

You can configure custom windows to create an environment with preferred menu and toolbar options for a specific type of designing. For example, you can create a new menu or toolbar that contains all the commands that you frequently use.

This topic describes the following:

- Planning the Custom Windows
- Creating Custom Windows
- Creating New Custom Menus and Toolbars
- Adding Commands in Custom Menus and Toolbars
- Deleting Menus, Menu Commands, and Toolbars
- Saving Custom Windows for Future Sessions

Planning the Custom Windows

Plan the custom window as follows:

1. For the type of custom window you want to create, open a design, and evaluate the current user interface. You can create a custom window for any window that contains menus and toolbars, including the Console.
2. Make a list of the items you want to add, move, and delete.
    - You can create new menus on menu bars and populate them with commands from a list of available operations called *actions*.
    - You can add new menu commands to existing menus from the list of available actions.
    - You can move menus and toolbars to different locations within the window.
    - You can delete menus, individual commands, and toolbars from the window.

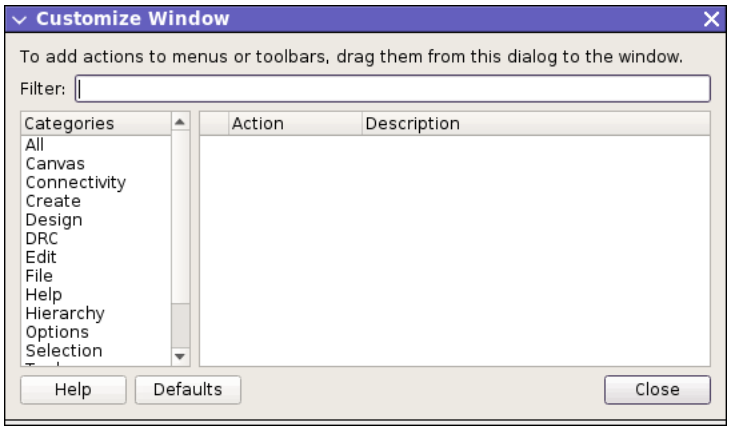
Creating Custom Windows

Before you start to add menus and toolbars, you must first create a new custom window. This is the window that will contain the custom menus, toolbars, and dialog boxes that you will be adding.

Feedback

To create a new custom window:

1. In the design editor, choose **Window > Customize**. The **Customize Window** dialog box opens.



   The system freezes all menus, toolbars, and assistants so that you can manipulate the menus and toolbars. For example, you can select and display a menu, but you cannot use it.

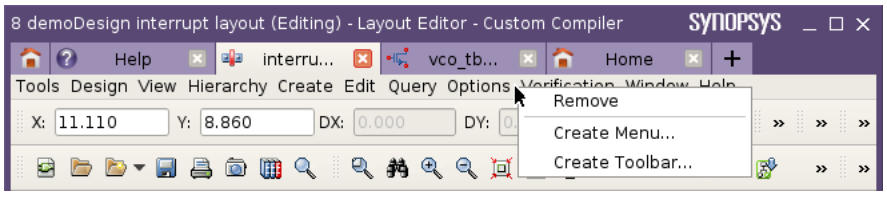
2. Do one of the following:
   - Creating New Custom Menus and Toolbars
   - Adding Commands in Custom Menus and Toolbars
   - Deleting Menus, Menu Commands, and Toolbars

Creating New Custom Menus and Toolbars

Based on the plan you created in Planning the Custom Windows, you can create new menus and toolbars in the custom window.

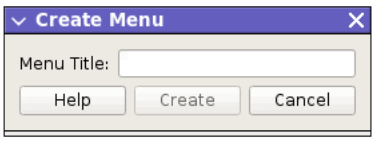
To create a new menu or toolbar:

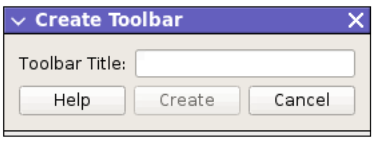
1. In the custom window you just created, right-click the menu bar, and choose **Create Menu** or **Create Toolbar**.



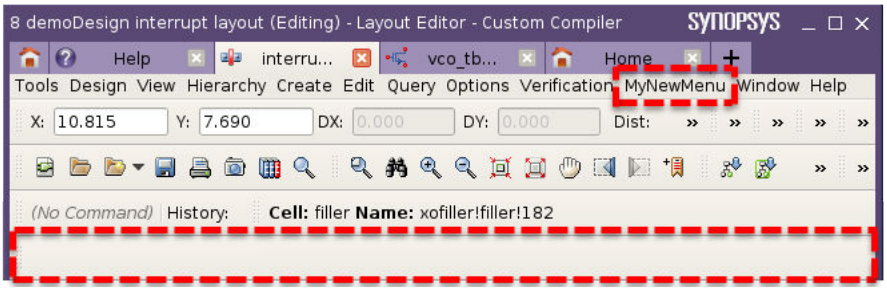
2. In the **Create Menu** or **Create Toolbar** dialog box, type the name of the new menu or toolbar.

Feedback





3. Click **Create**. The new menu name appears on the menu bar. If you created a new toolbar, the new toolbar appears below the set of existing toolbars, but it does not have a name.



4. Continue by developing the custom window as designed.
5. Choose one of the following:
   - Adding Commands in Custom Menus and Toolbars
   - Deleting Menus, Menu Commands, and Toolbars
   - Saving Custom Windows for Future Sessions

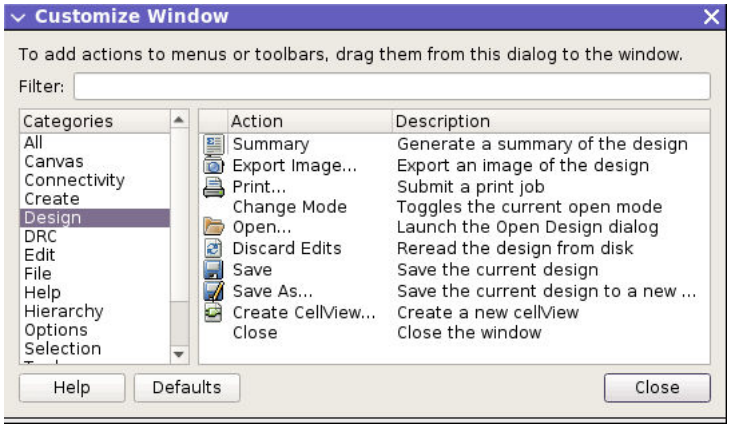
Adding Commands in Custom Menus and Toolbars

Based on the plan you created in Planning the Custom Windows, you can add commands in the custom menu or toolbar you just created.

To add new commands in a menu:

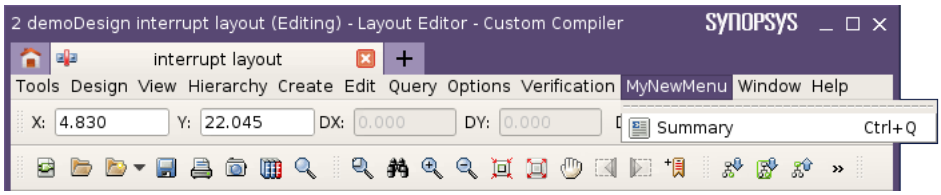
1. In the **Customize Window** dialog box, choose a category. The list of available actions (or commands) for that category appears.

   For example, if you choose the **Design** category, the list of design actions appears. The actions are commands that you can add to the menu you created.

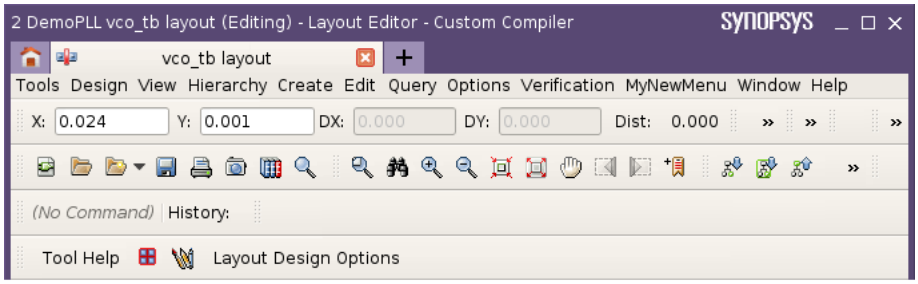


2. (Optional) To filter the list shown, type the action name in the **Filter** box.
3. To add a command, drag the action (command) to the newly created menu or toolbar. The command appears in the menu or on the toolbar.

   The following example shows that the **Design > Summary** command is added to the newly created menu called MyNewMenu.

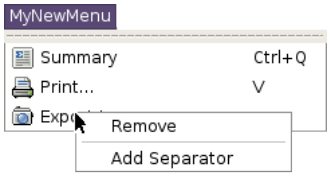


   The following example shows that the Tool Help, Create Via (icon), Display Resource Editor (icon), and Layout Display Options commands are added to the nameless custom toolbar.



4. (Optional) If you have more than one command in the custom menu or toolbar, reposition the command by dragging it to a new position.
5. (Optional) To add a line separator between commands, right-click where you want to add the separator, and choose **Add Separator**.
6. (Optional) To delete a command, right-click the command, and choose **Remove**.

Feedback



7. Continue by developing the custom window as designed.
8. Choose one of the following:
   - Deleting Menus, Menu Commands, and Toolbars
   - Saving Custom Windows for Future Sessions
   - Creating New Custom Menus and Toolbars

Deleting Menus, Menu Commands, and Toolbars

You can delete menus, menu commands, and toolbars in your custom window as follows:

- To delete an entire menu or toolbar, right-click the menu or toolbar, and choose **Remove**.
- To remove a menu command, right-click the command, and choose **Remove**.
- To remove all custom menus and toolbars, in the **Customize Window** dialog box, click **Defaults**.

Saving Custom Windows for Future Sessions

You can save custom windows for this session only or for future sessions. To keep the custom window only for this session, do nothing more. The custom settings will end when the session ends.

To save the custom window for future sessions:

1. In the **Customize Window** dialog box, click **Close** to exit the frozen user interface state.
2. Create a new workspace. See Creating Workspaces.

Creating Dialog Boxes

You can add new dialog boxes to accept user input for Tcl scripts you write. Dialog boxes are instantiated and visible by default. However, you have the option of hiding them.

- Creating a Sample Dialog Box
- Creating a Basic Dialog Box
- Creating Elements for Collecting Input
- Creating Grouping Elements
- Creating Push Buttons and Labels
- Creating Elements that Reconfigure
- Adjusting Custom Dialog Box Layouts

Creating a Sample Dialog Box

Create a sample dialog box to learn how to add some of the available elements you can include.

You can follow the sequence in Table 112 to create a dialog box sample.

Table 112 Dialog Box Sample Widgets

Widget	See instructions for:
Dialog	Creating a Basic Dialog Box
Tab Group	Creating Tab Groups
Group	Creating Groups
File Input	Collecting File Input
Library Input	Collecting Library Input
Text Input	Creating Text Input
Checkable Group	Creating Checkable Groups
Boolean Input	Creating Boolean Input Check Boxes
Mutex Input	Creating Mutually Exclusive Input
Layout Adjustment	Adjusting Custom Dialog Box Layouts

Feedback

Creating a Basic Dialog Box

Create a basic dialog box by adding the dialog box widget to your Tcl script.

You can visualize dialog box appearance by entering the Tcl for the dialog box, and all of its widget elements, directly in the Console. When you are done, you can copy the Tcl to your script for implementation.

To create a dialog box:

1. Add a Tcl procedure to create a graphic user interface, for example `proc showImportSPICE {} {}`.
2. Add the Tcl command `gi::createDialog` at the top of the Tcl procedure.

   By default, dialog boxes show **Help** and **Close** buttons, only. This dialog box format is useful for showing messages. The user reads the message, and can choose to either click the **Help** button or close the dialog box.

   To transform the default dialog box into one that executes a Tcl procedure, add the `-execProc` argument, and give it the name of a Tcl procedure to execute. Adding the `-execProc` argument adds **OK**, **Apply**, and **Cancel** buttons.

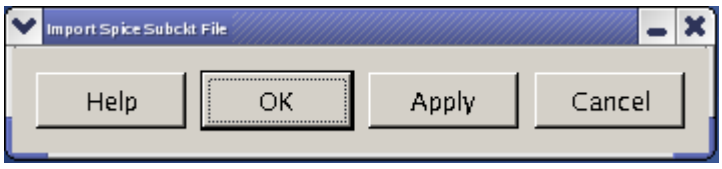
   If you do not plan to provide access to Help, add the `-showHelp` argument, and set it to `false`.

   If you want to render the dialog box as invisible, add the `-shown` argument, and set it to `false`.

   Example 6 creates a dialog box with the title *Import Spice Subckt File*.

Example 6 Dialog Box

```
set dialog [gi::createDialog importSpiceDialog -title \
    "Import Spice Subckt File" -execProc \
    cae::importSpiceFile]
```



3. Choose an option:
   - Continue to create the dialog box sample by adding a tab group to contain Main and Options tabs. See next Creating Tab Groups.
   - Add elements to the dialog box. See Creating Elements for Collecting Input.

See Also

- Sample GUI for Creating a Symbol from a SPICE Netlist

Creating Elements for Collecting Input

Create widgets for collecting input so that your Tcl procedure can operate on the input.

Table 113 lists widgets you can add to accept input in a dialog box, along with links to instructions in the sections that follow.

Table 113 Input Widgets

Input Type	Widget	See instructions for:
Single check box	Boolean Input	Creating Boolean Input Check Boxes
Text input field with folder icon offering the option to browse, and select	File Input	Collecting File Input
Text input field with pull down list of libraries	Library Input	Collecting Library Input
List box	List Input	Collecting Input through a List
Choice of only one option, either a radio button or selection from a list box	Mutually exclusive (mutex) input	Creating Mutually Exclusive Input
Number	Number Input	Creating Number Input
Text entry	Text Input	Creating Text Input

Creating Boolean Input Check Boxes

Create check boxes to collect Boolean input for user decisions that are either true or false.

To create a Boolean input check box:

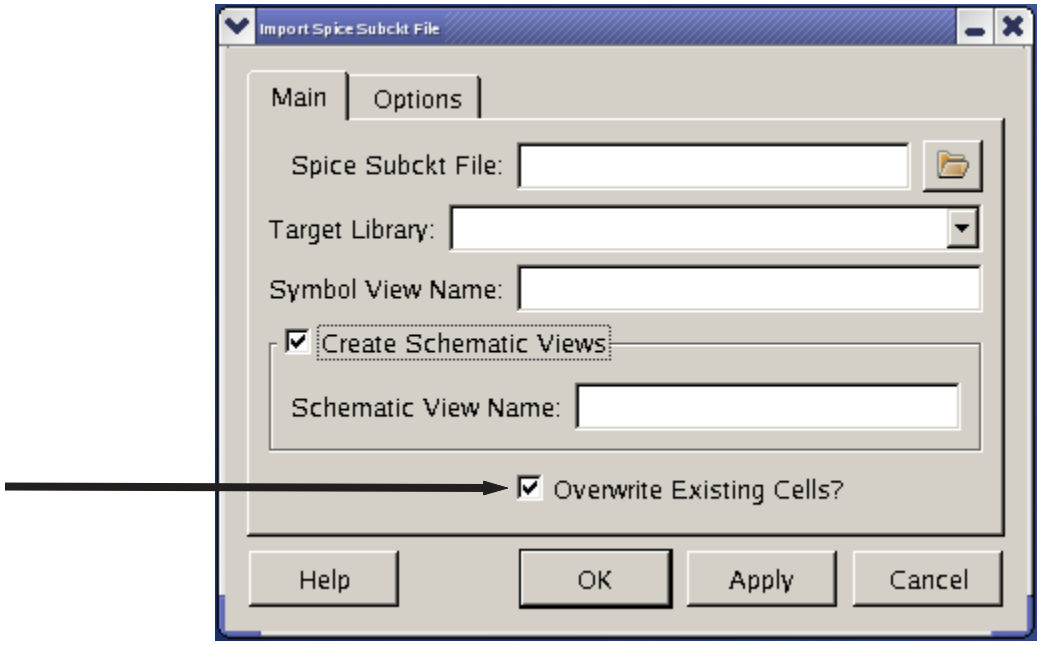
1. Add the Tcl command `gi::createBooleanInput`.

   Example 7 adds a Boolean input check box that offers the option to overwrite existing cells. Users can choose this option or deselect it.

Feedback

Example 7 Boolean Input Check Box

```
set overwriteDesign [gi::createBooleanInput overwriteDesign \
   -parent $mainOptions -label "Overwrite existing cells?"]
```



2. Choose an option:
   - Continue to create the dialog box sample by adding mutually exclusive input options. See next Creating Mutually Exclusive Input.

See Also

- Sample GUI for Creating a Symbol from a SPICE Netlist

Collecting File Input

Collect the name of a file for input by creating a file input widget.

File input widgets support inline completion of file and directory names. They also provide a browse button for selecting files through a file dialog box. They align with labeled inputs, and expand horizontally.

Feedback

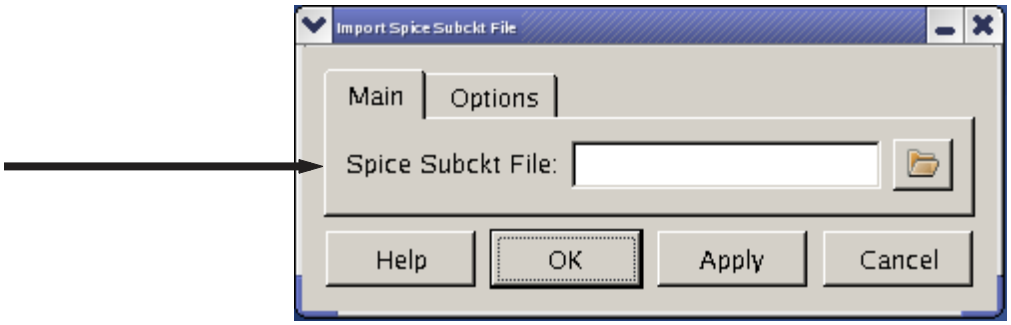
To create a file input widget:

1. Add the Tcl command `gi::createFileInput`.

   Example 8 adds a file input widget to the **Main** tab in the tab group mainOptions. This file input widget has a mask that filters for three specific file name extension patterns, and also accepts any file type.

Example 8 File Input Widget

```
set file [gi::createFileInput file -parent $mainOptions \
    -label "Spice Subcircuit File:" -mode open -fileMasks \
    {*.spi *.txt *.sp *} -fileType file]
```



2. Choose an option:
   - Continue to create the dialog box sample by capturing a target library. See next Collecting Library Input.

See Also

- Sample GUI for Creating a Symbol from a SPICE Netlist

Collecting Library Input

Collect a library as input by creating a library input widget.

Library input widgets offer a pull down menu for selecting a library.

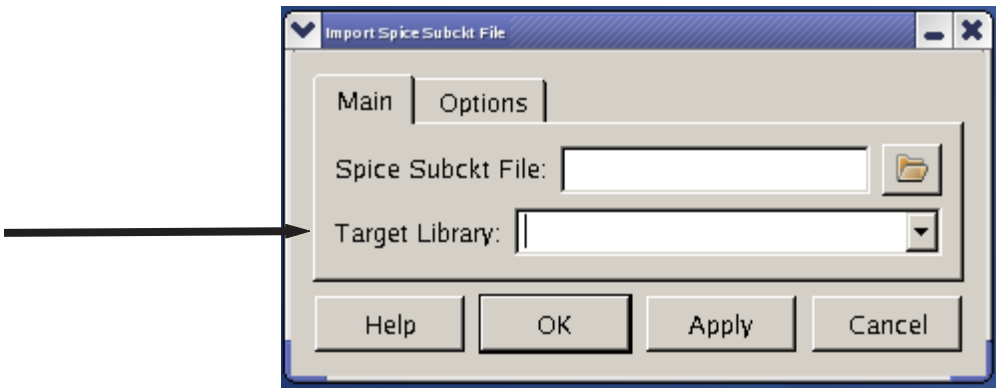
To create a library input widget:

1. Add the Tcl command `dm::createLibInput`.

   Example 9 adds a library input widget to the **Main** tab in the tab group mainOptions. This widget offers the list of currently available libraries for selection. See Sample GUI for Creating a Symbol from a SPICE Netlist for a script that uses this widget.

Example 9 Library Input Widget

```
set library [dm::createLibInput library -parent $mainOptions \
    -label "Target Library:" -prefName dbSpiceReaderLibName]
```



2. Choose an option:
   - Continue to create a dialog box sample by capturing the input of a symbol view name. See next Creating Text Input.

See Also

- Sample GUI for Creating a Symbol from a SPICE Netlist

Collecting Input through a List

Create a list input widget in a dialog box when you want to offer a list of one or more items for user selection.

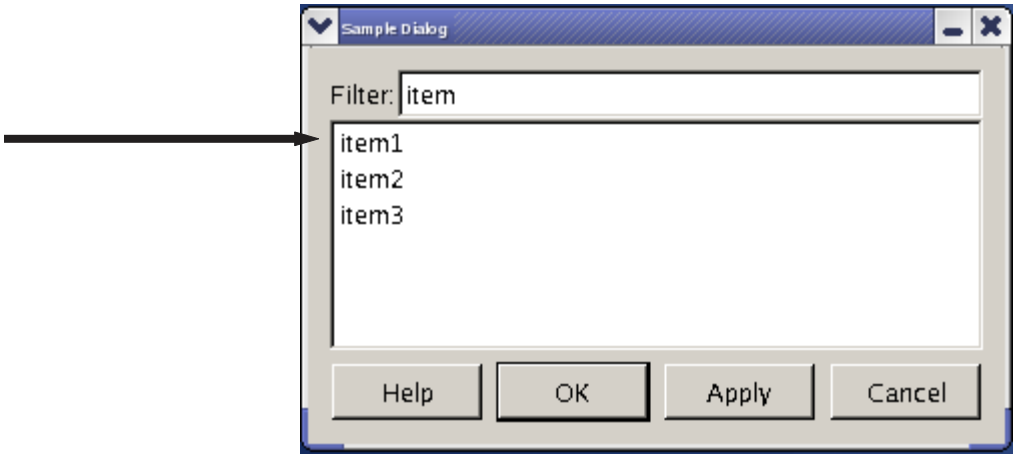
To create a list input widget:

1. Add the Tcl command `gi::createListInput`.

   Example 10 adds a list input widget to the sample dialog box, and, and shows user input of the word `item`, which leaves all listed items displayed. If the user now enters `1`, only `item1` will show in the filter box.

Example 10 List Input Widget

```
set list1 [gi::createListInput -parent $dialog -items \
   {item1 item2 item3} -showFilter true]
```



2. Choose an option:
   - More about Customizing the Software and Workspaces.

See Also

- Sample GUI for Creating a Symbol from a SPICE Netlist

Creating Mutually Exclusive Input

Create a mutually exclusive widget in a dialog box when you want to allow the selection of only one of many possible, mutually exclusive values. You can render this widget as either a set of radio buttons or a single *combo* box.

Example 11 adds two mutex input boxes where the user can select only one of the available options in each.

To create a mutually exclusive input widget:

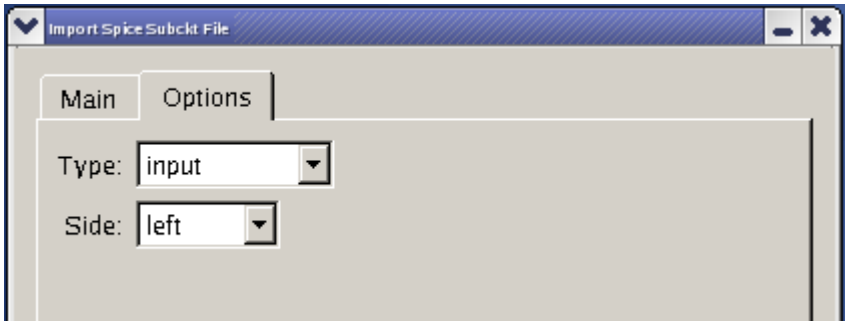
1. Add the Tcl command `gi::createMutexInput`.

   Example 11 adds two mutex input widgets. The first widget offers the option to set a pin type. The second widget offers the alternate, and mutually exclusive option to set a pin direction.

Example 11 Mutex Input Widget

```
set outputPinType [gi::createMutexInput outputPinType \
    -parent $subOptions -label "Type:" -enum {input \
    output inputOutput} -viewType combo]
set outputPinDirection [gi::createMutexInput \
    outputPinDirection -parent $subOptions -label "Side:" \
    -enum {left right top bottom} -viewType combo]
```

Feedback



2. Choose an option:
   - Continue to create a dialog box sample by adjusting the arrangement of the widgets. See next Adjusting Custom Dialog Box Layouts.

See Also

- Sample GUI for Creating a Symbol from a SPICE Netlist

Creating Number Input

Create a number input widget in a dialog box when you want the UI to accept input of an integer or float value.

1. Add the Tcl command `gi::createNumberInput`.
2. Choose an option:
   - More about Customizing the Software and Workspaces.

See Also

- Sample GUI for Creating a Symbol from a SPICE Netlist

Creating Text Input

Create a text input widget in a dialog box when you want to accept text input, and this information is not available to offer for selection through a list.

To create a text input widget:

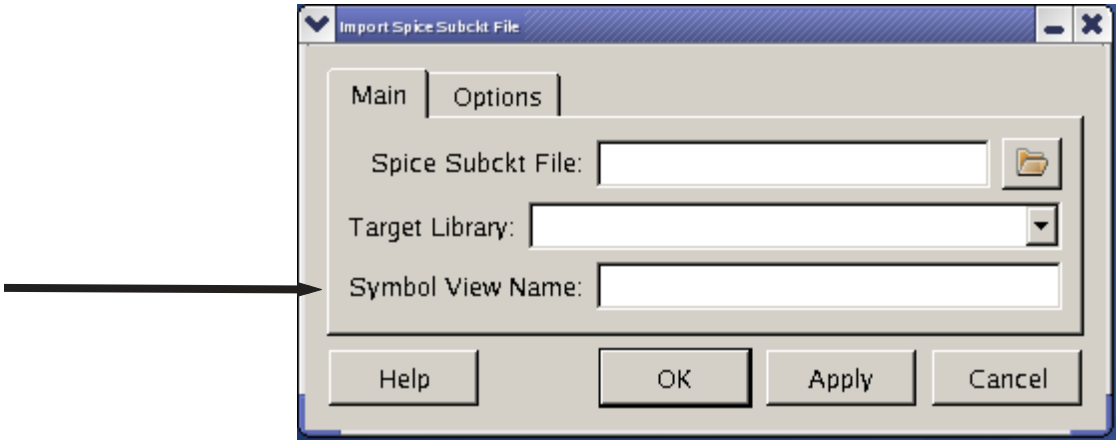
1. Add the Tcl command `gi::createTextInput`.

   Example 12 creates a text input widget on a **Main** tab in the mainOptions tab group. This widget captures the symbol view name the user enters.

Feedback

Example 12 Text Input Widget

```
set symbolView [gi::createTextInput symbolView -parent \
    $mainOptions -label "Symbol View Name:"]
```



2. Choose an option:
   - Continue to create a dialog box sample by adding a checkable group with a single check box, and a text input widget. See next Creating Checkable Groups.
   - More about Customizing the Software and Workspaces.

See Also

- Sample GUI for Creating a Symbol from a SPICE Netlist

Creating Grouping Elements

Create grouping elements when you want a dialog box to suggest a usage sequence by surrounding elements with borders and labels, and tabs.

Table 114 lists grouping element widgets you can add, along with links to instructions in the sections that follow.

Table 114 Input Widgets

Grouping Element	Widget	See instructions for:
Bordered area with or without a label	Group	Creating Groups
Labeled line of elements	Inline Group	Creating Inline Groups

Feedback

Table 114 Input Widgets (Continued)

Grouping Element	Widget	See instructions for:
Labeled tabs with separate usage areas	Tab Group	Creating Tab Groups
Multiple check boxes under one label	Checkable Group	Creating Checkable Groups

Creating Groups

Create a basic group in a dialog box when you want to arrange similar items together, and surround them graphically with a lined border. A group can be labeled in a group box or unlabeled in a frame.

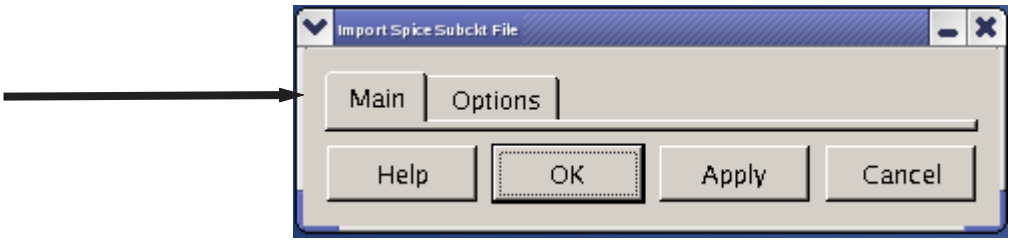
To create a group:

1. Add the Tcl command `gi::createGroup`.

   Example 13 adds two groups to the last defined tab group, and labels them *Main* and *Options*. Note that the tab group widget extends across the dialog box, and allows the addition of more tabs.

Example 13 Group Widgets

```
set mainOptions [gi::createGroup mainOptions -parent $tabG1 \
 -label "Main"]
set subOptions [gi::createGroup subOptions -parent $tabG1 \
    -label "Options"]
```



2. Choose an option:
   - Continue to create a dialog box sample by adding a file input widget. See next Collecting File Input.
   - Learn more about Customizing the Software and Workspaces.

See Also

- Sample GUI for Creating a Symbol from a SPICE Netlist

Feedback

Creating Inline Groups

Create an inline group in a dialog box when you want to present a group of widgets in a horizontal line.

To create an inline group:

1. Add the Tcl command `gi::createInlineGroup`.
2. Choose an option:
   - Learn more about Customizing the Software and Workspaces.

See Also

- Sample GUI for Creating a Symbol from a SPICE Netlist

Creating Tab Groups

Create tab groups in a dialog box when you want to offer separate usage areas. You can separate different types of input or provide a use model that progresses from one tab to another.

A tab group is an ordered, dynamic container of other group containers, which are rendered as separate tabs.

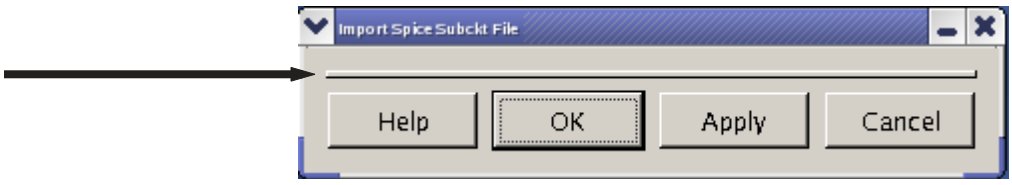
To create tab groups:

1. Add the Tcl command `gi::createTabGroup`.

   Example 14 creates a tab group called `tabG1` whose parent is the last dialog box defined. Before you add the tabs themselves, the tab group renders as a horizontal line above the standard buttons, as shown.

Example 14 Tab Group Widget

```
set tabG1 [gi::createTabGroup tabG1 -parent $dialog]
```



2. Choose an option:
   - Continue to create a dialog box sample by defining the tabs in the group. See next Creating Groups.
   - Learn more about Customizing the Software and Workspaces.

Feedback

See Also

- Sample GUI for Creating a Symbol from a SPICE Netlist

Creating Checkable Groups

Create a checkable group in a dialog box when you want to offer multiple check boxes under one label.

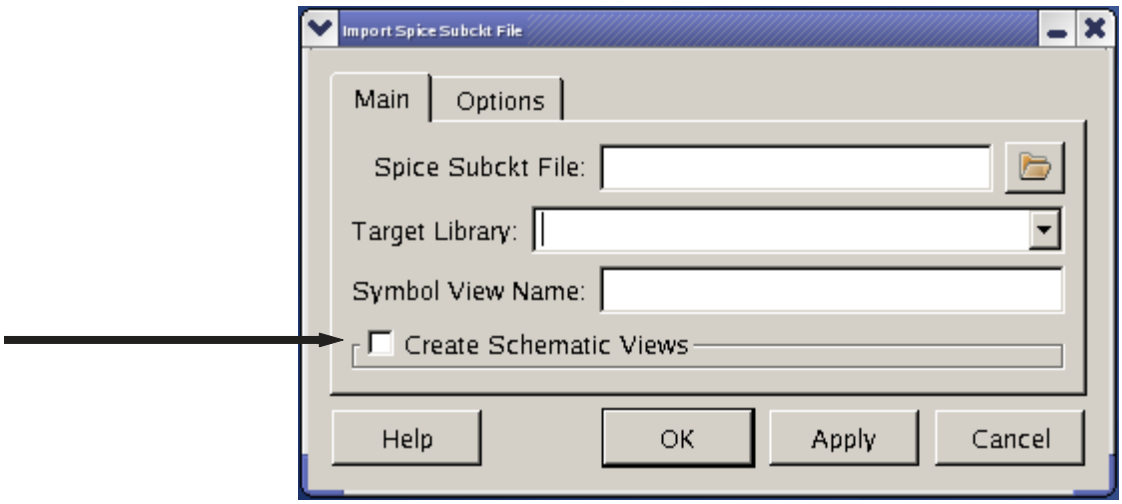
To create a checkable group:

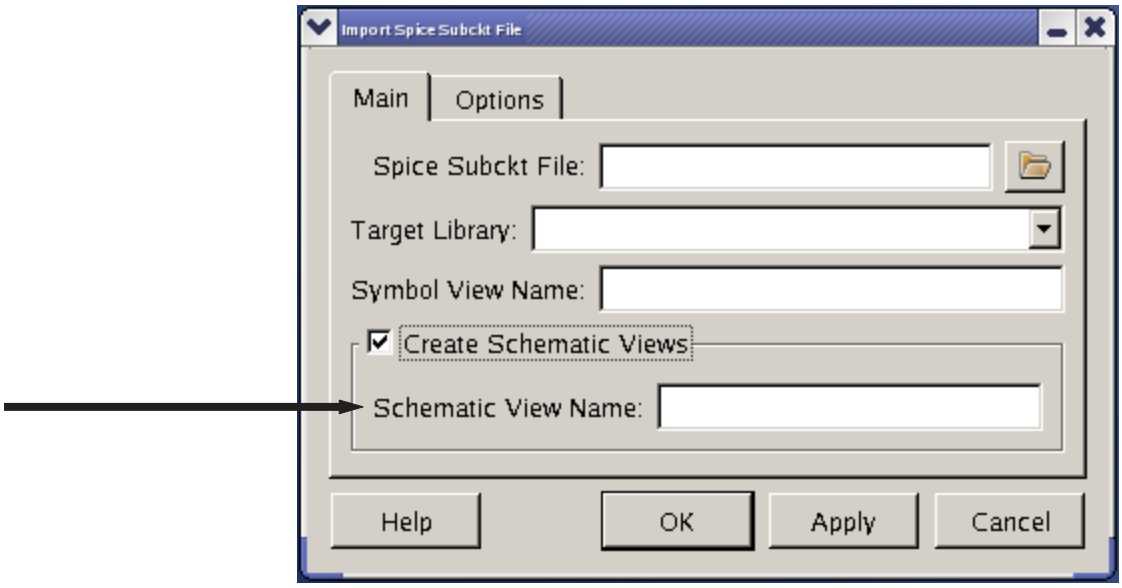
1. Add the Tcl command `gi::createCheckableGroup`.

   Example 15 creates a checkable group. When the user adds a check mark to the *Create Schematic Views* box, the second line of Tcl activates a text input widget to collect the text.

Example 15 Checkable Group Widget

```
set createSchem [gi::createCheckableGroup createSchem \
    -parent $mainOptions -label "Create schematic views"]
set schemView [gi::createTextInput schemView -parent  \
    $createSchem  -label "Schematic View Name:"]
```





2. Choose an option:
   - Continue to create a dialog box sample by adding a single check box for Boolean input. See next Creating Boolean Input Check Boxes.
   - Learn more about Customizing the Software and Workspaces.

See Also

- Sample GUI for Creating a Symbol from a SPICE Netlist

Creating Splitter Groups

Create a splitter group in a dialog box when you want to offer the ability to change the relative sizes of widgets.

To create a splitter group:

► Add the Tcl command `gi::createSplitterGroup`.

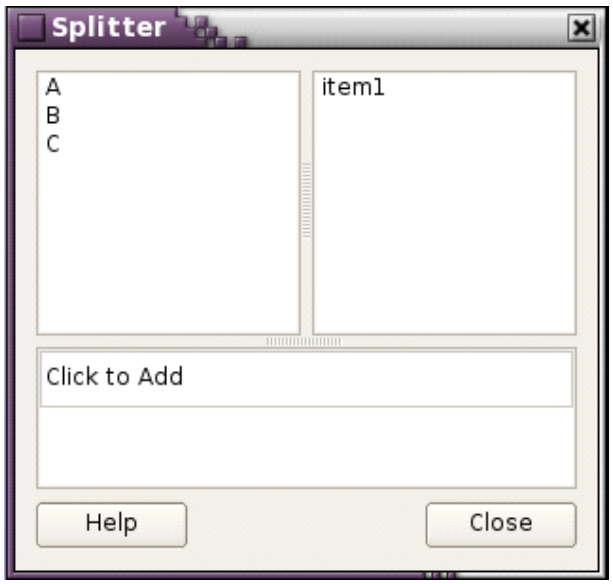
Example 16 adds horizontal and vertical splitters in a dialog box. The dialog box includes two List Input widgets (whose horizontal size can be changed relative to each other) and a Table widget (whose vertical size can be changed relative to the List Input widgets).

Example 16 Splitter Group Widget

```
set dialog [gi::createDialog myDialog -title "Splitter"]
set spv [gi::createSplitterGroup splitv -parent $dialog \
 -orientation vertical]
```

Feedback

```
set sph [gi::createSplitterGroup splith -parent $spv ]
gi::createListInput ll -parent $sph -viewType list \
 -items {A B C}
gi::createListInput rl -parent $sph -viewType list -items item1
set table [gi::createTable table -parent $spv \
 -allowAddRows true]
gi::createColumn  -parent $table -stretch true
```



The following commands specify the geometry of the dialog box and the vertical size of the List Input and Table widgets.

```
db::setAttr dialog.geometry -value 500x300
db::setAttr spv.childSizes -value {160 80}
```

To resize the dialog box vertically without changing the vertical sizes of the List Input widgets, use this command:

```
db::setAttr spv.childStretchFactors -value {0 1}
```

See Also

- Creating a Sample Dialog Box

Creating Push Buttons and Labels

Create push buttons to execute single operations immediately. Create labels to identify ungrouped elements.

These sections cover:

- Creating Push Buttons
- Creating Labels

Creating Push Buttons

Create a push button widget in a dialog box when you want to add a stateless button that performs a single operation. Push button widgets align with labeled inputs, and do not expand in either the horizontal or vertical direction.

To create a push button:

► Add the Tcl command `gi::createPushButton`.

See Also

- Sample GUI for Creating a Symbol from a SPICE Netlist

Creating Labels

Create a label in a dialog box when you want to add a stateless text label to show text.

To create a label:

► Add the Tcl command `gi::createLabel`.

See Also

- Sample GUI for Creating a Symbol from a SPICE Netlist

Creating Elements that Reconfigure

Create elements that reconfigure when you want to guide the user to a targeted choice from a larger set of information. You can create tables, and dynamic lists, which adjust in response to user interaction. These elements save space, and show only the information you designate.

These sections cover:

- Creating Tables
- Creating Dynamic Lists

Creating Tables

Create a table in a dialog box when you want to offer graphic arrangement in which each cell can contain data or an additional widget. You can choose to represent the table in standard format or as a tree where the parent of a row object is another row.

Tables are labeled widgets, and they expand both horizontally and vertically.

To create a table:

► Add the Tcl command `gi::createTable`.

See Also

- Sample GUI for Creating a Symbol from a SPICE Netlist

Creating Dynamic Lists

Create a dynamic list in a dialog box when you want offer a list in which the user can add or remove entries. The input value of a dynamic list is taken from the items which have been specified. Dynamic lists are labeled widgets, and expand both horizontally and vertically. Clicking on any item changes the item to an editable field. Erasing a name removes the row. Clicking on **Click to Add** adds an additional row.

To create a dynamic list:

► Add the Tcl command `gi::createDynamicList`.

See Also

- Sample GUI for Creating a Symbol from a SPICE Netlist

Adjusting Custom Dialog Box Layouts

Adjust dialog box layouts to arrange the elements in proper relationship to each other.

For example, the standard usage sequence for a dialog box progresses from left to right, and top to bottom. By default. However, dialog box creation adds each element in a vertical arrangement from top to bottom, and not from left to right. Adjust the layout to display the best suggested usage sequence from left to right in addition to top to bottom.

To arrange a dialog box layout:

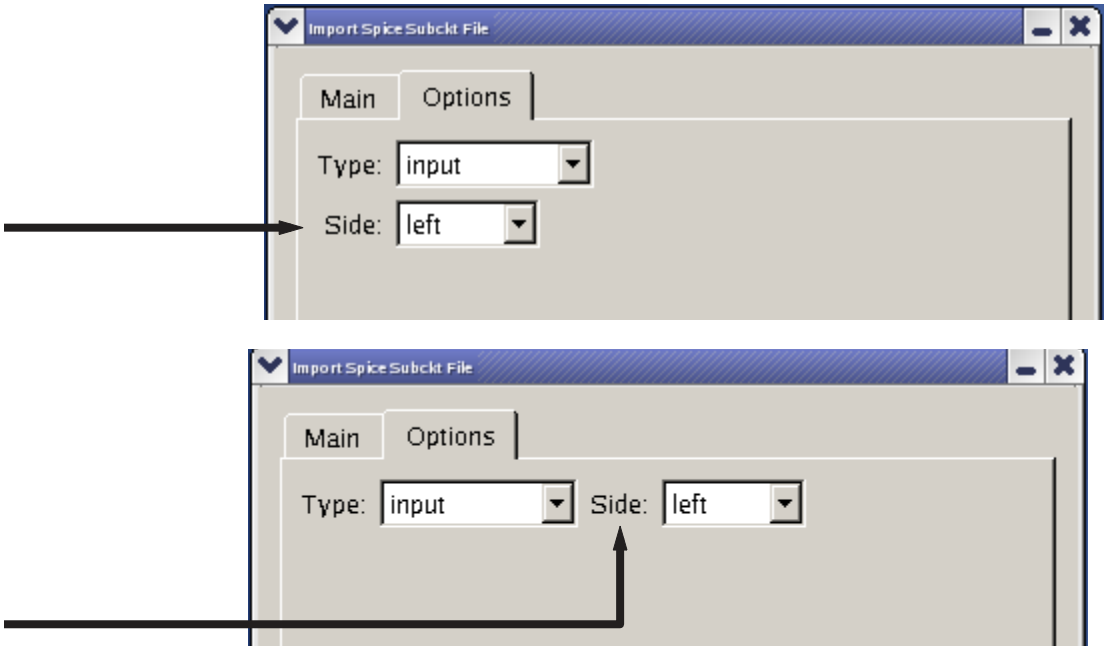
► Add the Tcl `gi::layout`.

Example 17 adjusts the arrangement of the two input fields from stacked to side by side.

Example 17 Adjusting Layouts

```
gi::layout $outputPinType -rightOf $outputPinRegExp
gi::layout $outputPinDirection -rightOf $outputPinType
```

Feedback



See Also

- Sample GUI for Creating a Symbol from a SPICE Netlist

Adding Assistants and Other User Interfaces Using Tcl/Tk

You have the option of using Tcl/Tk, the Tcl graphical user interface tool kit, to create custom interfaces, such as assistants. Having the flexibility to create new graphical user interfaces lets you meet your organization's unique requirements.

To create a new user interface:

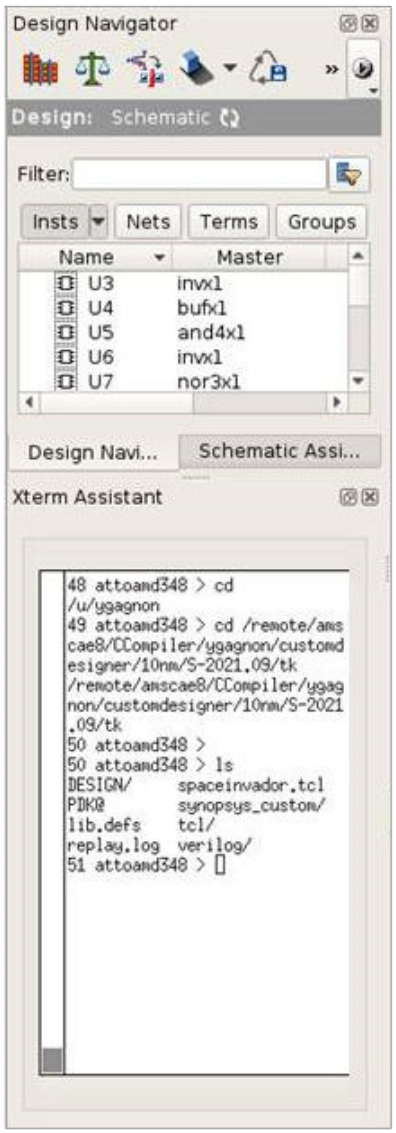
1. Use the available `gi::` Tcl commands, such as `gi::createAssistant` and `gi::getAssistants`.
2. Use the supported `gi` Tcl objects:
   - giContainer
   - giGroup
   - giAssistant
   - giWindow
   - giCentralWidget

Feedback

In the following example, the Tcl script excerpt creates a new assistant called Xterm Assistant. The `gi::createAssistant` command calls a Tcl procedure to create the graphical user interface for the assistant and displays the contents of the xterm (terminal emulator) for the X windows within the new assistant. The `gi::addAssistant` command adds the newly created assistant to the Custom Compiler window.

```
gi::createAssistant xtermAssistant -title "Xterm Assistant" \
        -createProc {apply {{window} {
            set grp [gi::createGroup grp -parent $window]
            set x11win [gi::createX11Window -parent $grp]
            exec xterm -into [db::getAttr id -of $x11win] &
}}}
gi::addAssistant [gi::getAssistants xtermAssistant] \
    -to [gi::getActiveWindow]
```

Feedback



Tcl Sample for Custom Graphic User Interface

This topic offers samples of custom user interface Tcl. The currently supplied sample consists of utility script that takes as input a SPICE netlist, and generates a symbol view.

- Sample GUI for Creating a Symbol from a SPICE Netlist
- Sample Preferences for Creating a Symbol from a SPICE Netlist

Feedback

Sample GUI for Creating a Symbol from a SPICE Netlist

The sample Tcl script that follows creates a symbol view using a SPICE netlist for input. Script execution dynamically creates a dialog box user interface with two tabs.

```
########################################
# Sample Custom User Interface Script
# Symbol creation from a spice netlist
########################################
namespace eval cae {

proc isPinInList { pin pinList rexp } {
 set returnCode 0
 if { $rexp == false } {
 foreach pinListElement $pinList {
 if [string match -nocase $pin $pinListElement] {
 set returnCode 1
 }
 }
 } else {
 if [regexp -nocase $pinList $pin] {
 set returnCode 1
 }
 }
 return $returnCode
}

# continued
proc importSPICEFile { d } {

### Preferences
set overwrite [db::getPrefValue dbSPICEReaderOverwriteDesigns]
set createSchematic [db::getPrefValue   \
          dbSPICEReaderCreateSchematic]
set libName [gi::findChild library.value -in $d]
set symbolViewName [db::getPrefValue    \
          dbSPICEReaderSymbolViewName]
set extraViewNames [db::getPrefValue    \
          dbSPICEReaderExtraViewNames]
set schematicViewName [db::getPrefValue \
          dbSPICEReaderSchematicViewName]

set outputPins [db::getPrefValue dbSPICEReaderOutputPins]
set outputPinType [db::getPrefValue dbSPICEReaderOutputPinType]
set outputPinSide [db::getPrefValue  \
          dbSPICEReaderOutputPinDirection]
set outputRexp [db::getPrefValue dbSPICEReaderOutputPinRexp]
set inputPins [db::getPrefValue dbSPICEReaderInputPins]
set inputPinType [db::getPrefValue dbSPICEReaderInputPinType]
set inputPinSide [db::getPrefValue   \
          dbSPICEReaderInputPinDirection]
set inputRexp [db::getPrefValue dbSPICEReaderInputPinRexp]
```

Feedback

```
set powerPins [db::getPrefValue dbSPICEReaderPowerPins]
set powerPinType [db::getPrefValue dbSPICEReaderPowerPinType]
set powerPinSide [db::getPrefValue  \
        dbSPICEReaderPowerPinDirection]
set powerRexp [db::getPrefValue dbSPICEReaderPowerPinRexp]
set groundPins [db::getPrefValue dbSPICEReaderGroundPins]
set groundPinType [db::getPrefValue dbSPICEReaderGroundPinType]
set groundPinSide [db::getPrefValue  \
        dbSPICEReaderGroundPinDirection]
set groundRexp [db::getPrefValue dbSPICEReaderGroundPinRexp]

set findSubckt 0
set file [db::getPrefValue dbSPICEReaderFileName]
if [file exists $file] {
 set fileID [open $file r]
 foreach line [split [read $fileID] \n] {
       if [regexp {(?i)^\.subckt.*} $line match] {
           set subline $line
           #create a hash table of the pins based on cell names
           set cellArray([lindex $subline 1]) [lrange
                    $subline 2 [llength $subline]]
           set findSubckt 1
           set cellName [lindex $subline 1]
       } elseif {$findSubckt} {
         if [regexp {(?i)^\+} $line match] {
            set portList $cellArray($cellName)
            regsub {^\+[ \t]*} $line {} newline
            append portList " "
            append portList $newline
            set cellArray($cellName) $portList
        } else {
           set findSubckt 0
        }
     }
 }
 close $fileID
} else {
 de::sendMessage "File not found!" -severity error
}


foreach cell [array names cellArray] {
  if {[oa::CellDMDataExists $libName $cell]} {
    if {$overwrite} {
 db::destroy [dm::findCell $cell -libName $libName]
 } else {
   de::sendMessage "Cell $cell exists! Skipping..."
   continue
  }

 }
 de::sendMessage "Importing subckt for $cell..."
 set dmCell [dm::createCell $cell -libName $libName]
```

Feedback

```
 if {$createSchematic} {
   set seCellView [dm::createCellView $schematicViewName -cell\
           $dmCell -viewType schematic]
### Retrieve the design from the dmCellView objct
  set context [de::open $seCellView -headless true]
  set design [db::getAttr editDesign -of $context]
  set y 0
 }
 set interfacePinList {}
 set termOrder [list "(" $cellArray($cell) ")"]
 set pos_in 0
 set pos_out 0
 set pos_gnd 0
 set pos_pwr 0
 foreach pin $cellArray($cell) {
   if [cae::isPinInList $pin $inputPins $inputRexp] {
     set type $inputPinType
     set side $inputPinSide
     incr pos_in
      lappend interfacePinList [db::createInterfacePin $pin \
               -type $type -side $side -position $pos_in]
   } elseif [cae::isPinInList $pin $outputPins $outputRexp] {
     set type $outputPinType
     set side $outputPinSide
     incr pos_out
      lappend interfacePinList [db::createInterfacePin $pin \
              -type $type -side $side -position $pos_out]
 } elseif [cae::isPinInList $pin $groundPins $groundRexp] {
 set type $groundPinType
 set side $groundPinSide
 incr pos_gnd
 lappend interfacePinList [db::createInterfacePin $pin -type $type -side
 $side -position $pos_gnd]
 } elseif [cae::isPinInList $pin $powerPins $powerRexp] {
 set type $powerPinType
 set side $powerPinSide
 incr pos_pwr
 lappend interfacePinList [db::createInterfacePin $pin -type $type -side
 $side -position $pos_pwr]
 } else {
 set type $inputPinType
 set side $inputPinSide
 incr pos_in
 lappend interfacePinList [db::createInterfacePin $pin -type $type -side
 $side -position $pos_in]
 }
 if {$createSchematic} {
 set points [list 0 $y]
 se::createPin $pin -design $design -type $type -points [list $points]
 set y [expr $y + 0.25]
 }
 }
```

Feedback

```
 set interface [db::createInterfaceSet -pins [db::createCollection
 $interfacePinList]]
 puts $interfacePinList
 set viewsToCreate $symbolViewName
 append viewsToCreate " "
 append viewsToCreate $extraViewNames
 foreach view $viewsToCreate {
 de::sendMessage "Creating symbol view for $cell..."
 set cellview [dm::createCellView $view -cell $dmCell -viewType
 schematicSymbol]
 db::generateCellView -dest $cellview -interface $interface -viewType
 symbolSchematic
 }

 oa::getAccess [oa::LibFind $libName] write 1
 set dmData [oa::DMDataOpen [oa::CellFind [oa::LibFind $libName] $cell ]
 write ]
 db::setNetlistInfo termOrder -data $dmData -netlister hspiceD -type list
 -value "($cellArray($cell))"
 oa::save $dmData
 oa::close $dmData


 oa::releaseAccess [oa::LibFind $libName]
}
 de::sendMessage "Done importing SPICE subcircuit file!"
}


# continued
proc showImportSPICE {} {
###### Create the user interface widgets
### This proc creates the GUI

### Create the dialog widget to contain the input widgets
### and capture input
set dialog [gi::createDialog importSPICEDialog -title \
          "Import SPICE Subcircuit File" -execProc \
          cae::importSPICEFile -showHelp false]

### Create a tab group to contain the two tabs
set tabG1 [gi::createTabGroup tabG1 -parent $dialog]

### Create an individual tab called Main
set mainOptions [gi::createGroup mainOptions -parent $tabG1 \
        -label "Main"]

### Create an individual tab called Options
set subOptions [gi::createGroup subOptions -parent $tabG1 \
        -label "Options"]

### Create a file input field on the Main tab to capture the SPICE
### file
```

Feedback

```
set file [gi::createFileInput file -parent $mainOptions \
          -label "SPICE Subcircuit File:" -mode open -fileMasks \
          {*.spi *.txt *.sp *} -fileType file -prefName \
          dbSPICEReaderFileName]

### Create a library input widget with a pulldown menu offering
### existing libraries to capture the target library
set library [dm::createLibInput library -parent $mainOptions \
               -label "Target Library:" -prefName dbSPICEReaderLibName]

### Create a text input widget to capture the symbol view name
set symbolView [gi::createTextInput symbolView -parent \
         $mainOptions -label "Symbol View Name:" -prefName \
         dbSPICEReaderSymbolViewName]

### Create a text input widget to capture optional extra view names
set extraViews [gi::createTextInput extraViews -parent \
    $mainOptions -label "Extra View Name(s):" -prefName \
    dbSPICEReaderExtraViewNames]

### Create a checkable group widget offering the option to create
### schematic views
set createSchem [gi::createCheckableGroup createSchem -parent \
    $mainOptions -label "Create schematic views" -prefName \
    dbSPICEReaderCreateSchematic]

### Create a single text input widget in the checkable group
### that is only enabled when the group is enabled
set schemView [gi::createTextInput schemView -parent  \
    $createSchem  -label "Schematic View Name:" -prefName \
    dbSPICEReaderSchematicViewName]

### Create a boolean input widget (a check box) for
### optionally overwriting existing designs
set overwriteDesign [gi::createBooleanInput overwriteDesign \
   -parent $mainOptions -label "Overwrite existing cells?" \
   -prefName dbSPICEReaderOverwriteDesigns]

### Create four required fields for the output pins on the
### Options tab: one text input, one boolean input,
### and two mutex input widgets
set outputPins [gi::createTextInput outputPins -parent \
    $subOptions -label "Output pins:" -prefName \
    dbSPICEReaderOutputPins]
set outputPinRegExp [gi::createBooleanInput outputPinRegExp \
    -parent $subOptions -label "Regexp" -prefName \
    dbSPICEReaderOutputPinRexp]
set outputPinType [gi::createMutexInput outputPinType \
    -parent $subOptions -label "Type:" -prefName \
    dbSPICEReaderOutputPinType -enum \
    {input output inputOutput} -viewType combo]
set outputPinDirection [gi::createMutexInput \
    outputPinDirection -parent $subOptions -label "Side:" \
```

```
    -prefName dbSPICEReaderOutputPinDirection -enum \
    {left right top bottom} -viewType combo]
set inputPins [gi::createTextInput inputPins -parent \
    $subOptions -label "Input pins:" -prefName \
    dbSPICEReaderInputPins]

### Create four fields for input pins on the Options tab:
### one text input, one boolean input, and two mutex
### input widgets
set inputPinRegExp [gi::createBooleanInput inputPinRegExp \
    -parent $subOptions -label "Regexp" -prefName \
    dbSPICEReaderInputPinRexp]
set inputPinType [gi::createMutexInput inputPinType \
    -parent $subOptions -label "Type:" -prefName \
    dbSPICEReaderInputPinType -enum \
    {input output inputOutput} -viewType combo]
set inputPinDirection [gi::createMutexInput \
    inputPinDirection -parent $subOptions -label "Side:" \
    -prefName dbSPICEReaderInputPinDirection -enum {left \
    right top bottom} -viewType combo]

### Create four fields for the power pins on the Options tab
set powerPins [gi::createTextInput powerPins -parent \
    $subOptions -label "Power pins:" -prefName \
    dbSPICEReaderPowerPins]
set powerPinRegExp [gi::createBooleanInput powerPinRegExp \
    -parent $subOptions -label "Regexp" -prefName \
    dbSPICEReaderPowerPinRexp]
set powerPinType [gi::createMutexInput powerPinType \
    -parent $subOptions -label "Type:" -prefName \
    dbSPICEReaderPowerPinType -enum {input output \
    inputOutput} -viewType combo]
set powerPinDirection [gi::createMutexInput powerPinDirection -parent
 $subOptions -label "Side:" -prefName dbSPICEReaderPowerPinDirection
 -enum {left right top bottom} -viewType combo]

### Create four fields for the ground pins on the Options tab
set groundPins [gi::createTextInput groundPins -parent \
    $subOptions -label "Ground pins:" -prefName \
    dbSPICEReaderGroundPins]
set groundPinRegExp [gi::createBooleanInput \
    groundPinRegExp -parent $subOptions -label "Regexp" \
    -prefName dbSPICEReaderGroundPinRexp]
set groundPinType [gi::createMutexInput groundPinType \
    -parent $subOptions -label "Type:" -prefName \
    dbSPICEReaderGroundPinType -enum {input output \
    inputOutput} -viewType combo]
set groundPinDirection [gi::createMutexInput \
    groundPinDirection -parent $subOptions -label \
    "Side:" -prefName dbSPICEReaderGroundPinDirection \
    -enum {left right top bottom} -viewType combo]

### Arrange the widgets in an orderly manner within the
```

Feedback

```
### dialog box
gi::layout $outputPinRegExp -rightOf $outputPins
gi::layout $outputPinType -rightOf $outputPinRegExp
gi::layout $outputPinDirection -rightOf $outputPinType
gi::layout $inputPinRegExp -rightOf $inputPins
gi::layout $inputPinType -rightOf $inputPinRegExp
gi::layout $inputPinDirection -rightOf $inputPinType
gi::layout $powerPinRegExp -rightOf $powerPins
gi::layout $powerPinType -rightOf $powerPinRegExp
gi::layout $powerPinDirection -rightOf $powerPinType
gi::layout $groundPinRegExp -rightOf $groundPins
gi::layout $groundPinType -rightOf $groundPinRegExp
gi::layout $groundPinDirection -rightOf $groundPinType

db::setAttr geometry -of $dialog -value \
    [db::getPrefValue dbSPICEReaderDialogGeometry]

}

}
```

Sample Preferences for Creating a Symbol from a SPICE Netlist

The sample preferences.tcl file that follows sets values for the preferences referenced in the supplied sample Tcl script in this chapter.

```
namespace eval ::startup {
   db::createPref "dbSPICEReaderOverwriteDesigns" -value true \
      -description "Overwrite Existing designs" \
      -labels {dataIO spice_reader} -type bool
   db::createPref "dbSPICEReaderCreateSchematic" -value false \
      -description "Create Schematics?" \
      -labels {dataIO spice_reader} -type bool
   db::createPref "dbSPICEReaderSymbolViewName" -value "symbol" \
      -description "Name of the Symbol View" \
      -labels {dataIO spice_reader}
   db::createPref "dbSPICEReaderExtraViewNames" -value {hspice} \
      -description "Name(s) of extra view(s) to create" \
      -labels {dataIO spice_reader}
   db::createPref "dbSPICEReaderSchematicViewName" -value \
         "schematic" \
      -description "Name of the schematic view" \
      -labels {dataIO spice_reader}
   db::createPref "dbSPICEReaderLibName" -value "" \
      -description "Target Library Name" \
      -labels {dataIO spice_reader}
   db::createPref "dbSPICEReaderFileName" -value "" \
      -description "Input File Name" \
      -labels {dataIO spice_reader}
   db::createPref "dbSPICEReaderOutputPins" -value "" \
      -description "Output Pins" \
      -labels {dataIO spice_reader}
```

```
  db::createPref "dbSPICEReaderOutputPinRexp" -value false \
     -description "Output Pin Regular Exp" \
     -labels {dataIO spice_reader} -type bool
  db::createPref "dbSPICEReaderInputPins" -value "" \
     -description "Input Pins" \
     -labels {dataIO spice_reader}
  db::createPref "dbSPICEReaderInputPinRexp" -value false \
     -description "Input Pin Regular Exp" \
     -labels {dataIO spice_reader} -type bool
  db::createPref "dbSPICEReaderPowerPins" -value "" \
     -description "Power Pins" \
     -labels {dataIO spice_reader}
  db::createPref "dbSPICEReaderPowerPinRexp" -value false \
     -description "Power Pin Regular Exp" \
     -labels {dataIO spice_reader} -type bool
  db::createPref "dbSPICEReaderGroundPins" -value "" \
     -description "Ground Pins" \
     -labels {dataIO spice_reader}
  db::createPref "dbSPICEReaderGroundPinRexp" -value false \
     -description "Ground Pin Regular Exp" \
     -labels {dataIO spice_reader} -type bool
  db::createPref "dbSPICEReaderDialogGeometry" -value
        "802x334+346+166" \
     -description "SPICE Reader Dialog Geometry" \
     -labels {dataOP spice_reader}
  db::createPref "dbSPICEReaderOutputPinType" -value "output" \
     -description "SPICE Reader Output Pin type" \
     -labels {dataOP spice_reader}
  db::createPref "dbSPICEReaderOutputPinDirection" -value \
        "right" \
     -description "SPICE Reader Output Pin Direction" \
     -labels {dataOP spice_reader}
  db::createPref "dbSPICEReaderInputPinType" -value "input" \
     -description "SPICE Reader Input Pin type" \
     -labels {dataOP spice_reader}
  db::createPref "dbSPICEReaderInputPinDirection" -value \
        "left" \
     -description "SPICE Reader Input Pin Direction" \
     -labels {dataOP spice_reader}
  db::createPref "dbSPICEReaderPowerPinType" -value
"inputOutput" \
     -description "SPICE Reader Power Pin type" \
     -labels {dataOP spice_reader}
  db::createPref "dbSPICEReaderPowerPinDirection" -value "top" \
     -description "SPICE Reader Power Pin Direction" \
     -labels {dataOP spice_reader}
  db::createPref "dbSPICEReaderGroundPinType" -value \
        "inputOutput" \
     -description "SPICE Reader Ground Pin type" \
     -labels {dataOP spice_reader}
  db::createPref "dbSPICEReaderGroundPinDirection" -value \
        "bottom" \
     -description "SPICE Reader Ground Pin Direction" \
```

```
        -labels {dataOP spice_reader}
    }
```

A

Integrating External Applications

This appendix describes how to extend Custom Compiler environment functionality by integrating external tools. The environment provides an easy-to-use interface to allow communication with other applications. This white paper covers a step-by-step procedure with examples for creating a custom tool integration. The complete source code is available in files gui.tcl and gui_launcher.tcl, which you can obtain from Synopsys Technical Support.

This appendix contains the following topics:

- Glossary of Terms
- Handshaking with the Custom Compiler Environment
- Walkup with the Custom Compiler Environment
- Blocking and Non-Blocking IPC
- A Sample Integration
- The Tcl Event Loop
- Launching From the Custom Compiler Environment

Glossary of Terms

IPC

(Inter-Process Communication) is the exchange of data between two programs. The exchange can be done between programs running on the same machine, or between programs running on different machines on a network.

RPC

(Remote Procedure Call) is a programming interface that allows one program to request a service from another program via IPC. The calling program sends a message and data to the other program, the request is executed, and the result is returned to the caller.

Feedback

Tcl

(Tool Command Language) is a freely available interpreted scripting language. See http://tcl.tk for documentation and information.

Tk

(Tool Kit) is a freely available extension to Tcl that provides commands to create graphical user interfaces. See http://tcl.tk for details.

wish

is a command-line Tcl interpreter with Tk command support. Graphical applications can be created by writing Tcl/Tk scripts, and executing them through the wish interpreter.

Tcl-Dp

is a freely available extension to Tcl/Tk that allows socket based remote procedure calls between Tcl interpreters. The Tcl-Dp website is http://tcldp.sourceforge.net.

API

(Application Programming Interface) - An API is a programmatic interface for accessing an application. In the context of this paper, the term API refers to the set of Tcl commands that the environment provides in its scripting interface.

Handshaking

is the process of forming an IPC channel between two programs.

Listening port

a file descriptor that accepts incoming network connections.

Walkup

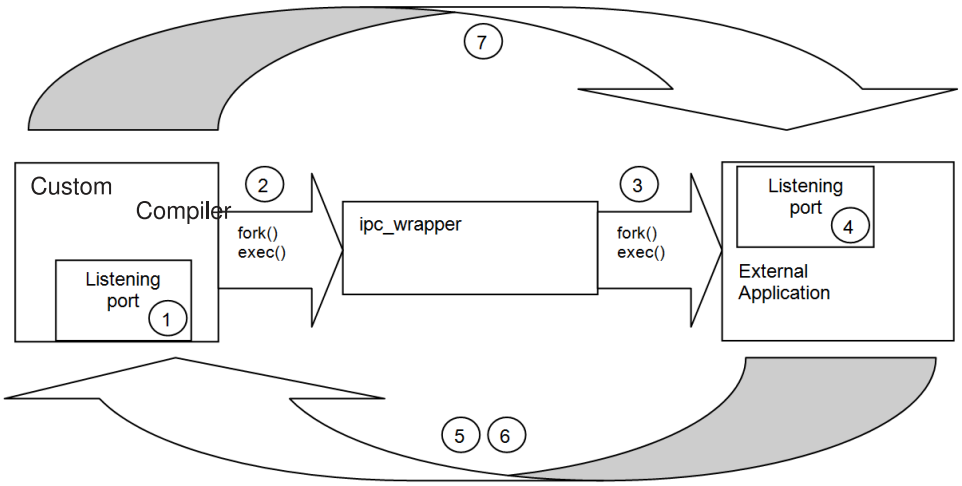
occurs when an external application that was not launched by the environment connects with it.

Handshaking with the Custom Compiler Environment

This section provides a high-level view of how the IPC channel is formed between the environment and an external application. A concrete coding example follows later in this chapter with sample code that an application integrator can copy.

There are seven steps in the handshaking process between the environment and an external application. The seven steps are illustrated in the following figure and described in this section.

Feedback



To launch the external application, the integrator must call the `xt::createJob` command within the environment.

The call to `xt::createJob` automatically performs the first three steps:

1. The environment opens a listening port.
2. The environment launches a light-weight monitoring process called *ipc_wrapper*.
3. The ipc_wrapper process launches the external application.

   The ipc_wrapper process sets up two special variables in the execution environment of the external application.

   They are:

   _SYNOPSYS_CUSTOM_SERVER - This is the network address of the listening port from the first step. It is a string variable of the form <hostname>:<port number>._SYNOPSYS_CUSTOM_JOBID - This is the unique job identifier for the external application. It is used during handshaking to associate the external application with the correct job in the Job Monitor.

   After the external application is launched, it must connect-back to the environment. The external application must perform the following three steps to continue handshaking.
4. The external application opens a listening port.
5. The external application forms a connection with the environment on the listening port that was defined in the first step of this procedure (using the _SYNOPSYS_CUSTOM_SERVER environment variable).

6. The external application uses the connection formed in the previous step to send the following details:

   The job identifier (the value of _SYNOPSYS_CUSTOM_JOBID)

   The network address of the listening port from Step 4 in the form <hostname>:<port number>

7. Finally, the environment uses the network address given in Step 6 to form a connection with the External Application.

   Steps 1, 2, 3, and 7 are performed automatically by the environment when the job is launched, and in response to incoming network events. It is the responsibility of the application integrator to write the code to perform steps 4, 5, and 6 during the external application startup.

Walkup with the Custom Compiler Environment

A running application can also connect to the environment through a process called walkup. Walkup is used to form a connection between the two applications without having to launch the application from the environment. External applications can walk up to the environment by performing the following steps:

1. The application opens a listening port.
2. The application scans the files in $HOME/.synopsys_custom/processes. Each file in the directory contains detailed information in XML format about all currently running environment applications for the user.

   The information directly required for walkup stored in the file includes:

   - The host name where the tool is running
   - The listening port number of the running session

   Each file contains other information that might be useful to the application to make special arrangements for the connection (for example, the product version number), or to select between multiple concurrently running sessions (for example, the display or process ID).
3. The application forms a connection with the environment given the host name and port number from Step 2.
4. The application sends its host name and listening port from Step 1 over the connection formed in Step 3.

   The following steps occur automatically on the environment.

5. The system automatically uses the information received in Step 4 to form a connection with the application.
6. The system internally generates a postman event called *onPostWalkup* to indicate that a walkup connection has been formed. Integrator code within the environment can be notified about the new connection by binding a Tcl callback procedure in Tcl or using a notifier in C++.

Blocking and Non-Blocking IPC

There are two methods to send commands to other applications: commands can be sent in blocking mode, or non-blocking mode. Each method has advantages and disadvantages, and the choice of which to use is application specific.

	Advantages	Disadvantages
Blocking Mode IPC	• The code to send commands to applications can be written linearly and is very simple to follow.	• Since the code blocks while waiting for the result of the RPC, it can lead to a sluggish user-interface if the remote application takes a long time to return a response. • If the RPC is done using a time-out to prevent infinite blocking, there can be cases where machine load or network latency can cause the time-out to expire erroneously.
Non-Blocking Mode IPC	• Because the command is sent and there is no waiting, the code will run smoothly without interruption, and generally will provide a more responsive user interface.	• The calling application does not receive a response, so if the command fails to execute in the remote application, the local application is not aware of a problem. However, this can be solved by having the remote application call back to the sender when it finishes processing a command. This capability is built into the Tcl-Dp framework.

Feedback

These sections cover:

- Blocking Mode IPC
- Non-Blocking Mode IPC
- Recommendation

Blocking Mode IPC

When a command is sent to a remote application using blocking mode IPC, the call blocks while waiting for the remote application to send a result. A time-out value can be provided to prevent infinite blocking in the local application when the remote application does not respond.

- How to do it with Tcl-Dp:

  The Tcl-Dp library provides the dp_RPC command to perform blocking RPC. See documentation for this library at the Cornell University Department of Computer Science website.

- How to do it within the environment:

  Calling `xt::sendCommand` in the environment with a non-zero time-out value blocks while waiting for a response. There is no way to block indefinitely using `xt::sendCommand`.

Non-Blocking Mode IPC

Non-blocking mode IPC sends the command to the remote application, but does not wait for a response.

- How to do it with Tcl-Dp:

  The Tcl-Dp library provides the dp_RDO command to perform non-blocking RPC. See documentation for this library at the Cornell University Department of Computer Science website.

- How to do it within the environment:

  Calling `xt::sendCommand` with a zero value time-out (or omitting the time-out argument) will send the command in non-blocking mode.

Recommendation

In general, the code author should strive to use non-blocking mode IPC wherever possible to have a smooth user-interface and prevent the need for time-outs.

Feedback

A Sample Integration

This section is a tutorial on how to integrate an application with the Custom Compiler environment. The example application is shown in the following figure and the code is contained in the file `gui.tcl`. This section does a step-by-step walk-through of the code to explain how it works.

- Using the Sample Integration
- The External Application

Using the Sample Integration

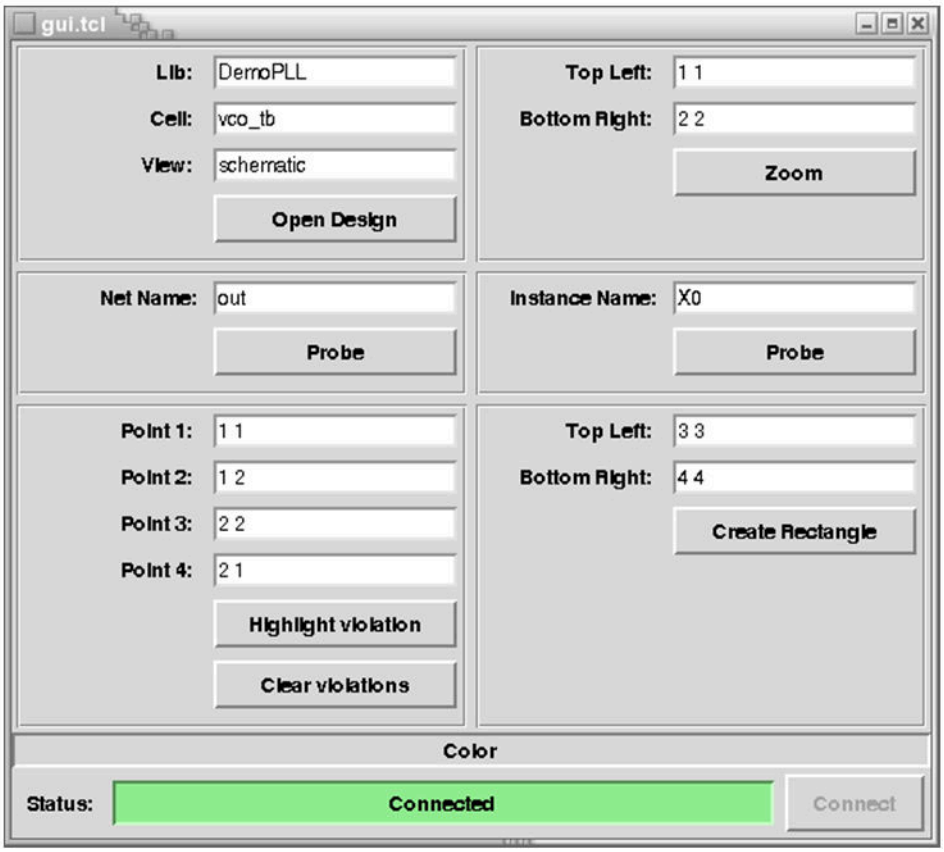
To use the sample integration:

1. Put the files gui.tcl and gui_launcher.tcl into the same directory.
2. Launch the Custom Compiler environment.
3. In the Console, source the `gui_launcher.tcl` file.

```
source ./gui_launcher.tcl
```

Feedback

4. Use the IPCTest menu in the main console window to launch and interact with the application.



The External Application

These sections cover:

- The Interpreter
- Opening a Design
- Zoom
- Probing a Net
- Probing an Instance
- Highlighting and Clearing Highlights
- Creating a Rectangle
- A Label to Modify

Feedback

- The Connection Status Frame
- The Window Layout
- Walkup Utilities
- Sending Commands to the Custom Compiler Environment
- The Handshaking Code

The Interpreter

The very first line of the external application spawns the interpreter.

```
#!/usr/bin/env wish
```

Following that, the script defines a single global variable that will store the Tcl-DP connection to the Custom Compiler environment.

```
# Global Vars
# This variable represents the connection to Custom Compiler
set cdesigner_fh ""
```

Then the script creates the basic Tk GUI. It is divided into eight frames. The first six frames provide examples of commands that can be sent to the environment. The seventh frame contains a horizontal label entitled *Color* and is used as an example of how to send a command from the environment to the external application. The eighth frame indicates the connection status.

Opening a Design

The first frame contains three regular single-line text entry fields. There, the lib name, cell name, and view name of a design can be entered to send a command to the environment to open that design using the `de::open` command.

```
# GUI Creation
# Open a Design
```

First, the script creates the frame to house the widgets:

```
frame .f1 -relief groove -bd
```

Then, the script creates three sets of labels and entry fields to put inside of the frame:

```
label .f1.l1 -text "Lib: " -width 15 -anchor e
entry .f1.e1 -bg white
label .f1.l2 -text "Cell: " -width 15 -anchor e
entry .f1.e2 -bg white
label .f1.l3 -text "View: " -width 15 -anchor e
entry .f1.e3 -bg white
```

Feedback

Then, the script adds a button to trigger sending the command:

```
button .f1.b -text "Open Design" -command openDesign
```

These lines of Tk code configure rows and columns within the frame so that the widgets will resize nicely when laid out in a grid:

```
grid columnconfigure .f1 1 -weight 1
grid rowconfigure .f1 10 -weight
```

This code lays out the widgets in four rows with some padding in between:

```
grid .f1.l1 .f1.e1 -sticky EW -padx 3 -pady 3
grid .f1.l2 .f1.e2 -sticky EW -padx 3 -pady 3
grid .f1.l3 .f1.e3 -sticky EW -padx 3 -pady 3
grid x .f1.b -sticky EW -padx 3 -pady 3
```

Finally, there is a Tcl callback procedure that is called when the button is clicked. The procedure first obtains the user-entered lib, cell and view names from the entry fields, and then it calls utility function to send the `de::open` command to the tool.

```
proc openDesign {} { global .f1.e1 .f1.e2 .f1.e3
 set lib [.f1.e1 get]
 set cell [.f1.e2 get]
 set view [.f1.e3 get]
```

Note that the square brackets below are escaped using a backslash. This is necessary for nested commands, and prevents the wish interpreter from trying to execute `dm::findCellView` locally.

The sendCommand utility procedure is defined in a section that follows: Sending Commands to the Custom Compiler Environment.

```
sendCommand "de::open \[dm::findCellView $view -libName $lib
 -cellName
 $cell\]"
}
```

Zoom

The next example calls `de::setViewPort` to change the zoom level on the currently opened design.

```
# Zoom
```

Then, the labels and entry fields are created to receive user input:

```
label .f2.l1 -text "Top Left: " -width 15 -anchor e
entry .f2.e1 -bg white
label .f2.l2 -text "Bottom Right: " -width 15 -anchor e
entry .f2.e2 -bg white
```

Feedback

And, a button with a callback to trigger sending the command:

```
button .f2.b -text "Zoom" -command zoom
```

Then, as in the previous example, the frame is configured so that the widgets will look nice when laid-out in a grid:

```
grid columnconfigure .f2 1 -weight 1
grid rowconfigure .f2 10 -weight 1
```

The labels, entries and button are laid-out in three rows:

```
grid .f2.l1 .f2.e1 -sticky EW -padx 3 -pady 3
grid .f2.l2 .f2.e2 -sticky EW -padx 3 -pady 3
grid x .f2.b -sticky EW -padx 3 -pady 3
```

The callback procedure for the button first obtains the values of the top-left and bottom-right coordinates of the zoom (allowing either comma-separated or space-separated coordinates), and then sends the `de::setViewPort` command with the coordinates:

```
proc zoom {} {
 global .f2.e1 .f2.e2
 set topleft [string map {"," " "} [.f2.e1 get]]
 set bottomright [string map {"," " "} [.f2.e2 get]]
 sendCommand "de::setViewport -box {{$topleft} {$bottomright}}"
}
```

Probing a Net

The next example uses the `de::createProbe` command to probe a net in the currently opened design.

```
# Probe a net
```

As usual, first the housing frame is created:

```
frame .f3 -relief groove -bd 2
```

And then a single label and entry field for the user to input the net name to probe.

```
label .f3.l -text "Net Name:" -width 15 -anchor e
entry .f3.e -bg white
```

And a button to trigger the operation:

```
button .f3.b -text "Probe" -command probeNet
```

Finally, the layout code:

```
grid columnconfigure .f3 1 -weight 1
grid rowconfigure .f3 10 -weight 1
grid .f3.l .f3.e -sticky EW -padx 3 -pady 3
grid x .f3.b -sticky EW -padx 3 -pady 3
```

Feedback

And the button callback to send the `de::createProbe` command with the given net name:

```
proc probeNet {} {
global .f3.e
set net [.f3.e get]
```

In this example, there are two nested calls. Each square bracket is escaped by a single backslash:

```
 sendCommand "de::createProbe $net -type net -context
 \[db::getAttr
  hierarchy -of \[de::getActiveContext\]\]"
}
```

Probing an Instance

This example uses `de::createProbe` to probe an instance in the currently opened design.

```
# Probe an instance
```

The housing frame and a label and entry widget for the user-input:

```
frame .f4 -relief groove -bd 2
label .f4.l -text "Instance Name:" -width 15 -anchor e
entry .f4.e -bg white
```

The button to trigger the operation:

```
button .f4.b -text "Probe" -command probeInst
```

The widget layout:

```
grid columnconfigure .f4 1 -weight 1
grid rowconfigure .f4 10 -weight 1
grid .f4.l .f4.e -sticky EW -padx 3 -pady 3
grid x .f4.b -sticky EW -padx 3 -pady 3
```

The callback procedure obtains the instance name from the entry and sends the `de::createProbe` command:

```
proc probeInst {} {
 global .f4.e
 set inst [.f4.e get]
 sendCommand "de::createProbe $inst -type instance -context
  \[db::getAttr hierarchy -of \[de::getActiveContext\]\]"
}
```

Highlighting and Clearing Highlights

The next example illustrates how to create and clear highlights in a design using `xt::highlightViolation` and `xt::clearViolations`. These commands create visible highlights on the design, but do not modify the underlying database.

Feedback

```
# Highlight and clear violations
```

The housing frame:

```
frame .f5 -relief groove -bd 2
```

For simplicity, this example contains four entry fields to create only rectangular shapes, but in general, the `xt::highlightViolation` command can be used to create an arbitrary polygon. Each entry field will be used for the x and y coordinates of each point in the rectangle.

```
label .f5.l1 -text "Point 1: " -width 15 -anchor e
entry .f5.e1 -bg white
label .f5.l2 -text "Point 2: " -width 15 -anchor e
entry .f5.e2 -bg white
label .f5.l3 -text "Point 3: " -width 15 -anchor e
entry .f5.e3 -bg white
label .f5.l4 -text "Point 4: " -width 15 -anchor e
entry .f5.e4 -bg white
```

This button triggers the highlight creation:

```
button .f5.b1 -text "Highlight violation"
 -command highlightViolation
```

This button clears existing highlights in the current design:

```
button .f5.b2 -text "Clear violations" -command clearViolations
```

The widgets are laid out in a grid within the housing frame:

```
grid columnconfigure .f5 1 -weight 1
grid rowconfigure .f5 10 -weight 1
grid .f5.l1 .f5.e1 -sticky EW -padx 3 -pady 3
grid .f5.l2 .f5.e2 -sticky EW -padx 3 -pady 3
grid .f5.l3 .f5.e3 -sticky EW -padx 3 -pady 3
grid .f5.l4 .f5.e4 -sticky EW -padx 3 -pady 3
grid x .f5.b1 -sticky EW -padx 3 -pady 3
grid x .f5.b2 -sticky EW -padx 3 -pady 3
```

The callback to create the highlights first obtains the four coordinates (allowing comma or space-separated strings), and then sends the `xt::highlightViolation` command:

```
proc highlightViolation {} {global .f5.e1 .f5.e2 .f5.e3 .f5.e4
set p1 [string map {"," " "} [.f5.e1 get]]
set p2 [string map {"," " "} [.f5.e2 get]]
set p3 [string map {"," " "} [.f5.e3 get]]
set p4 [string map {"," " "} [.f5.e4 get]]
```

Feedback

Note the use of the `ed` command. This special command provides shortcut access to the oaDesign from the currently active design:

```
sendCommand "xt::highlightViolation {{$p1} {$p2} {$p3} {$p4}}
 -design
\[ed\] -toolName testGui"
}
```

The callback to clear the highlights also uses the `ed` shortcut command to get the currently active oaDesign.

```
proc clearViolations {} {
sendCommand "xt::clearViolations -design \[ed\] -toolName  testGui"
}
```

Creating a Rectangle

This example creates a rectangle in the currently opened schematic using the `se::createRectangle` command.

```
# Create Rectangle
```

As usual, there is a frame to house the widgets:

```
frame .f6 -relief groove -bd 2
```

The rectangle is defined by its top-left and bottom-right coordinates, so there is an entry field for each:

```
label .f6.l1 -text "Top Left: " -width 15 -anchor e
entry .f6.e1 -bg white
label .f6.l2 -text "Bottom Right: " -width 15 -anchor e
entry .f6.e2 -bg white
```

There is a button to trigger the operation:

```
button .f6.b -text "Create Rectangle" -command createRectangle
```

This is the typical layout code:

```
grid columnconfigure .f6 1 -weight 1
grid rowconfigure .f6 10 -weight 1
grid .f6.l1 .f6.e1 -sticky EW -padx 3 -pady 3
grid .f6.l2 .f6.e2 -sticky EW -padx 3 -pady 3
grid x .f6.b -sticky EW -padx 3 -pady 3
```

The procedure to create the rectangle obtains the top-left and bottom-right coordinates from the entries, and sends the `se::createRectangle` command. Note that for a layout window, the `le::createRectangle` command could be used instead:

```
proc createRectangle {} {
global .f6.e1 .f6.e2
set topleft [string map {"," " "} [.f6.e1 get]]
```

Feedback

```
set bottomright [string map {"," " "} [.f6.e2 get]]
sendCommand "se::createRectangle {{$topleft} {$bottomright}}
 -design
\[ed\]"
}
```

A Label to Modify

This label's background color can be changed. It is here to demonstrate how to send commands from the environment to the external application.

```
# Color label
label .l -text "Color"
```

And here's a procedure that the tool can call that will change the color:

```
proc changeColor {color} {
 global .l
 .l configure -bg $color
}
```

The Connection Status Frame

The frame at the bottom of the window contains information about the connection to the Custom Compiler environment. Later, there will be examples of how to watch for changes to the connection so the widgets can be updated to reflect them.

```
frame .f7
label .f7.l1 -text "Status:"
pack .f7.l1 -side left -ipadx 3 -ipady 3 -padx 3 -pady 3
label .f7.l2 -text "Not connected" -relief sunken -bg pink
pack .f7.l2 -side left -expand yes -fill x -ipadx 3 -ipady 3 -
 padx 3 -pady 3
```

This button is used to initiate walkup to the Custom Compiler environment.

```
button .f7.b -text "Connect" -command walkupToCustomDesigner
pack .f7.b -side left -ipadx 3 -ipady 3 -padx 3 -pady 3
```

The Window Layout

This GUI setup code lays out the frames and the label in a grid, and configures the window resizing behavior.

```
# Arrange the frames in the window
grid columnconfigure . 0 -weight 1
grid columnconfigure . 1 -weight 1
grid rowconfigure . 0 -weight 1
grid rowconfigure . 1 -weight 1
grid rowconfigure . 2 -weight 1
grid rowconfigure . 3 -weight 1
grid .f1 .f2 -sticky NEWS -ipadx 3 -ipady 3 -padx 3 -pady 3
grid .f3 .f4 -sticky NEWS -ipadx 3 -ipady 3 -padx 3 -pady 3
```

Feedback

```
grid .f5 .f6 -sticky NEWS -ipadx 3 -ipady 3 -padx 3 -pady 3
grid .l -sticky NEWS -columnspan 2
grid .f7 -sticky NEWS -columnspan 2

# Set the minsize of the window
update
wm minsize . [winfo width .] [winfo height .]
```

Walkup Utilities

These procedures demonstrate a simple example of forming a walkup connection with the Custom Compiler environment.

```
# Walkup to Custom Compiler
proc walkupToCustomDesigner {} {
variable local_host
variable local_port
variable cdesigner_fh
```

This loop scans each process file to find a running session to connect to:

```
foreach filename [glob ~/.synopsys_custom/processes/*] {
 getHostPort $filename host port
 puts "Trying to connnect to $host:$port ..."
```

The code attempts to make a connection given the host and port information from the file:

```
if {![catch {dp_MakeRPCClient $host $port} fh]} {
```

When the connection is formed, the following line of code completes the handshaking with the environment from this end. It provides a string token to identify the type of application that is walking up, and also provides the address of the local listening port for connect-back. The string token (in this case demo_gui) will be used later in this example by the environment's Tcl integrator code.

```
dp_RDO $fh xt::createDPWalkupConnection "demo_gui"
 $local_host:$local_port
```

This registers a callback procedure to be invoked when the connection with the environment is closed.

```
dp_atclose $fh append connectionClosed
```

And the global variable for the connection is set.

```
set cdesigner_fh $fh
 puts " OK"
 break
} else {
```

Feedback

If the connection attempt fails, this code prints the error message to stdout:

```
 puts " $fh"
 }
}
```

If all of the process files have been tested and there is still no connection, this will bring up an error dialog box:

```
 if {$cdesigner_fh == ""} {
  error "Could not find a running Custom Compiler."
 }
}
```

The next procedure parses an individual system process file to obtain the host name and walkup port:

```
proc getHostPort {walkupFilename hostRef portRef} {
 upvar $hostRef host
 upvar $portRef port
 set fd [open $walkupFilename]
 set data [read $fd]
 close $fd
 set lines [split $data "\n"]
 set host ""
 set port ""
 foreach line $lines {
  regexp {<host>(.*?)</host>} $line match host
 regexp {<walkupPort>(.*?)</walkupPort>} $line match port
 }
}
```

This utility procedure controls the status widgets in the GUI depending on the state of the connection which is tracked by the global variable cdesigner_fh. It will be referenced later using the Tcl trace command:

```
proc connectionChanged { args } {
 variable cdesigner_fh
 if {$cdesigner_fh != ""} {
  .f7.b configure -state disabled
  .f7.l2 configure -text "Connected" -bg lightgreen
 } else {
  .f7.b configure -state normal
  .f7.l2 configure -text "Not connected" -bg pink
 }
}
```

And here's the callback that will be called when the connection to the environment is closed. It resets the cdesigner_fh variable:

```
proc connectionClosed { args } {
 variable cdesigner_fh
```

Feedback

```
 set cdesigner_fh ""
}
```

Sending Commands to the Custom Compiler Environment

This utility function sends commands to the environment if the connection exists; otherwise, it simply prints the command to *stdout*.

```
# Send a command to the environment
proc sendCommand {cmd} {
 global cdesigner_fh
 if {$cdesigner_fh == ""} {
# Not connected to Custom Compiler.
# Simply print out the command for debugging purposes.
 puts $cmd
} else {# Send the command to the environment and wait for the return
 value
# If there's an error, bring up an error dialog.
```

The `dp_RPC` command is a blocking remote procedure call, and so it waits for the command to complete execution on the environment before returning. The `catch` command can detect errors and then the code brings up an error dialog box to display the message:

```
  if {[catch {dp_RPC $cdesigner_fh $cmd} res]} {
  tkerror $res
  }
 }
}
```

The Handshaking Code

The environment provides the Tcl-DP extension library, and it can be loaded into the interpreter by the external application through the following code:

```
# load the Tcl-DP library
if {[catch {source "$::env(SYNOPSYS_CUSTOM_INSTALL)/ipc/
dp.tcl"}]} {
 source $::env(SYNOPSYS_CUSTOM_INSTALL)/etc/ipc/dp.tcl
}
```

The environment dependent Tcl/DP library is bundled with the environment and can be found in the installation (the script tries to load the 64-bit library if the 32-bit library fails to load):

```
set tcldp32 [file join [string map {amd64 linux} $env(TCLDP_PATH)]
 libdp40.so]
set tcldp64 [file join [string map {linux amd64} $env(TCLDP_PATH)]
 libdp40.so]
if {[catch {load $tcldp32}]} {
 load $tcldp64
```

Feedback

```
}
load "$::env(TCLDP_PATH)/libdp40.so"
```

After the Tcl-DP extension library is loaded, the external application creates a Tcl-DP listening socket, as described in the handshaking instructions, Handshaking with the Custom Compiler Environment:

```
# Create a socket for incoming connections
set local_fh [dp_MakeRPCServer]
```

The call to dp_MakeRPCServer above chooses a free port number at random. The following line of code queries the returned file handle to determine which port number was chosen:

```
set local_port [fconfigure $local_fh -myport]
set local_host [info hostname]
```

As described in the earlier section, the script tracks the *cdesigner_fh* variable to find out when connectivity changes so that the widgets can be updated:

```
# watch the cdesigner_fh variable
trace add variable cdesigner_fh write connectionChanged
```

Now that the listening port was created and callbacks are set, the application can operate in two different modes:

- Launched from the environment via `xt::createJob` - form a connection and send and receive commands via Tcl-DP
- Walkup: The user initiates a connection to a running session using a button in the GUI.

```
# Connect-back to Custom Compiler
# The presence of the environment variables
# _SYNOPSYS_CUSTOM_SERVER and _SYNOPSYS_CUSTOM_JOBID
# indicate that the script was launched by Custom Compilerif {[info
 exists ::env(_SYNOPSYS_CUSTOM_SERVER)]} {
```

Now, the external application can connect to the environment using the special environment variable _SYNOPSYS_CUSTOM_SERVER as described in Step 5:

```
# Create a connection to Custom Compiler's listening port
# contained in the _SYNOPSYS_CUSTOM_SERVER env var
set cdesigner_host [lindex [split $::env
 (_SYNOPSYS_CUSTOM_SERVER) ":"] 0]
set cdesigner_port [lindex [split $::env
 (_SYNOPSYS_CUSTOM_SERVER) ":"] 1]
```

Here, the connection to the environment is formed. The result is a file handle and it is stored in the global variable *cdesigner_fh*:

```
 set cdesigner_fh [dp_MakeRPCClient $cdesigner_host  $cdesigner_port]
 dp_atclose $cdesigner_fh append connectionClosed
```

Feedback

The last responsibility of the external application is to communicate the job identifier and local listening port address to the Custom Compiler environment, as described in Step 6.

```
# Send the environment the address of our listening port and
# our job id# The job id is contained in the _SYNOPSYS_CUSTOM_JOBID env
 var
set job_id $::env(_SYNOPSYS_CUSTOM_JOBID)
# dp_RDO is a non-blocking call
```

Note that the `xt::dpInteractiveConnectBack` procedure is only available for external applications to call via Tcl-DP. It cannot be called from the command Console:

```
dp_RDO $cdesigner_fh xt::dpInteractiveConnectBack $job_id \
 $local_host:$local_port
}
```

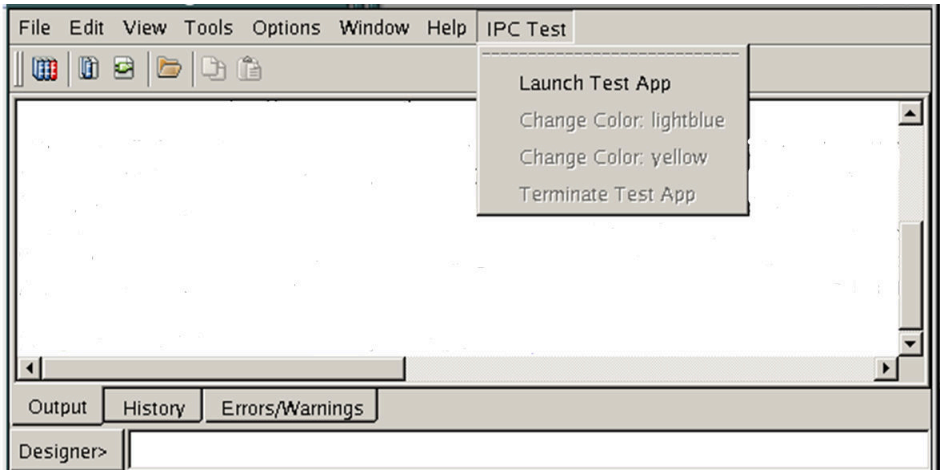
The Tcl Event Loop

The Tcl-DP package uses the Tcl interpreter's event loop for processing incoming connections and network messages. In this example, the use of the interpreter automatically starts the Tcl event loop and so the processing is done without any special code.

To integrate applications that do not have a Tcl event loop built-in, the integrator must make periodic calls to the Tcl update command, or the Tcl C API call `Tcl_DoOneEvent()` function. Unless the Tcl Event Loop in the external application periodically enters the Tcl event loop, the Tcl-DP library will not function correctly and will prevent a successful connection with Custom Compiler.

Launching From the Custom Compiler Environment

This section is a tutorial on how to create and manipulate menus and actions in Custom Compiler, and demonstrates how to launch the example external application. The following figure is a screen capture of the main Console window with the example menu. The complete code example can be found in the file `gui_launcher.tcl`.



These sections cover:

- Defining a Namespace
- Adding a Menu and Actions
- Action Callbacks
- Job Callbacks

Defining a Namespace

The first line of code defines a new namespace. Having a namespace that is specific to this particular application keeps the variables and procedures organized and separate from other utilities and top-level commands.

```
namespace eval guiTest {
# Namespace Vars
# Store the path to this file when it is first sourced
set script_path [info script]
# This contains a xtJob object when the test app is launched
set job ""
```

Adding a Menu and Actions

The example adds a Menu to the main console window with four menu actions.

```
# Create the IPC Test menu
proc createMenu {} {        # each of these actions is a menu item in the
 IPC Test menu
```

Actions represent items that can be inserted into menus, toolbars and context menus. The action has a different visual representation depending on what it is inserted into. For

Feedback

example, an action is rendered using only its icon if it is inserted into a toolbar widget, but it displays its textual title when placed into a menu, but, clicking on either the toolbar representation or the menu item runs the same command. And if the action is disabled, all of its inserted instances are also disabled.

The first action calls the launch procedure within the guiTest namespace when clicked:

```
gi::createAction "launchIPCTestApp" -title
 "Launch Test App" -command \
  [namespace current]::launch
```

The other actions also have command callback procedures that they call when clicked:

```
gi::createAction "changeIPCTestAppColor1" -title \
"Color: lightblue" -command [namespace current]::changeColor1
gi::createAction "changeIPCTestAppColor2" -title \
"Color: yellow"-command [namespace current]::changeColor2
gi::createAction "terminateIPCTestApp" -title \
"Terminate Test App" -command [namespace current]::terminate
```

Then, a menu is created and the actions are added to the menu:

```
# create the menu and add the actions to it
set m [gi::createMenu "ipcTest" -title "IPC Test"]
gi::addActions {"launchIPCTestApp" "changeIPCTestAppColor1" \
 "changeIPCTestAppColor2" "terminateIPCTestApp"} -to $m
```

And the menu is added to the main console window (window index 0):

```
# add the menu to the Console window
gi::addMenu $m -to [gi::getWindows 0]
```

Finally, each action is enabled or disabled so that the user can only activate the action that launches the example application. The other actions remain disabled until after the example application is launched:

```
 setActionEnabled "launchIPCTestApp" truesetActionEnabled
 "changeIPCTestAppColor1" falsesetActionEnabled "changeIPCTestAppColor2"
 false
setActionEnabled "terminateIPCTestApp" false
}
```

The following short utility procedure to enables or disables actions by name.

```
# Utility to enable / disable actions
proc setActionEnabled { actionName enabled } {
set m [gi::getMenus ipcTest -from [gi::getWindows 0]]
set a [gi::getActions $actionName -from $m]
db::setAttr enabled -of $a -value $enabled
}
```

Feedback

Action Callbacks

The next part of the file defines the action callbacks. Whenever an action is clicked, its callback procedure is called.

The launch callback is used to start the external application.

```
# Action callback procedures
proc launch {} {
 variable script_path
 variable job
```

This small section of code assumes that the example application is contained in the file gui.tcl and in the same directory as this script. It forms a Bourne Shell command for launching the job.

```
# Here the script assumes that gui.tcl is in the same directory as this
# script, so the tcl script takes the path to this script and replace the
 filename
# with gui.tcl and then the script
has our command.
set l [split $script_path "/"]
set l [lrange $l 0 end-1]
set l [lappend l "gui.tcl"]
set s [join $l "/"]
# This is the sh-style command-line syntax to launch the gui.tcl app
set cmd "$s > log.txt 2>&1"
set run_desc "Graphical Tcl/DP integration test application"
# start the job
```

The `xt::createJob` command on the environment launches external processes and associates them with a *xtJob* in the system. It is important to note the value of the `-type` argument used here. The string *dp* signifies a Tcl-Dp integration and initiates the communication handshaking sequence:

```
set job [xt::createJob \
 demo_gui \
 -type dp \
 -cmdLine "$cmd" \
 -runDesc "$run_desc" \
 -files "log.txt" \
```

The `-exitProc` argument specifies a Tcl procedure to call when the external application exits. In this case the procedure is called *exitProc*:

```
-exitProc [namespace current]::exitProc \
```

Feedback

The `-readyProc` argument specifies a Tcl procedure to call when handshaking with the external application is complete, indicating that commands may be sent and received:

```
-readyProc [namespace current]::readyProc]
# set this preference so that the script
can see interactive jobs
# in the job monitor for debugging
db::setPrefValue xtJobMonitorInteractiveJobs -value true
```

After the application has been launched, the action item for launching is disabled. This prevents the user from launching the app twice in one session:

```
# update the menu items
setActionEnabled "launchIPCTestApp" false
}
```

The next callback sends a command to the external application to change the label color to blue.

Note the use of the -timeout argument to `xt::sendCommand`.

If `xt::sendCommand` is not given a timeout, it operates in non-blocking mode, and does not return a value or produce an error if the command fails during remote execution. When `xt::sendCommand` is given a timeout, it operates in blocking mode, and returns the result of the sent command, or in the case of error during remote execution, it throws the error again.

```
proc changeColor1 {} {
variable job
# the "timeout" argument causes the command to be sent in blocking
# mode, so if there is an error, the script knows about it.
# with a 0 value for timeout, the command would be sent
# in non-blocking mode and if there's a return value / error,
# it would not be detected.
xt::sendCommand "changeColor lightblue" -job $job -timeout 1
}
```

This callback is very similar to the previous one, and it will send a command to the external application to change the label color to green.

```
proc changeColor2 {} {
 variable job
 xt::sendCommand "changeColor yellow" -job $job -timeout 1
}
```

The last callback sends the Tcl exit command to the external application. This causes the external application to quit.

```
proc terminate {} {
 variable job
 xt::sendCommand "exit" -job $job
}
```

Feedback

Job Callbacks

The last part of the file defines the *exitProc* and *readyProc* callbacks for the job.

```
# xtJob callbacks
# this proc is called when the job exits
proc exitProc {job} {
```

The following piece of code checks the exit status of the external application. If there was a problem during execution, it opens a text viewer window displaying the log file where *standard out* and *standard error* were captured. This helps with debugging problems that occur when starting up the external application. If the application exited normally, there is no need to view the log file.

```
# if the script fails to execute, bring up a text viewer window to
# display the job log file (if one is not already open)
if {[db::getAttr status -of $job] != "FINISHED"} {
set filename [db::getAttr files -of $job]
set windows [gi::getWindows -type \
 [gi::getWindowTypes xtTextViewer]]
set text_viewer ""
db::foreach win $windows {
 if {![catch {gi::getField $filename -in $win}]} {
  set text_viewer $win
 }
}
 if {$text_viewer == ""} {
  set text_viewer [xt::openTextViewer -files $filename]
 }
gi::setActiveWindow $text_viewer -raise true
}
```

When the application exits, the menu items are reset back to their original states so that you can launch them again, but not attempt to send commands:

```
# Reset the menu item states
 setActionEnabled "launchIPCTestApp" true
 setActionEnabled "changeIPCTestAppColor1" false
 setActionEnabled "changeIPCTestAppColor2" false
 setActionEnabled "terminateIPCTestApp" false
}
```

As described earlier, the following procedure is called when the handshaking is complete. This example enables the menu items that send commands to the external application:

```
proc readyProc {job} {
# the test app is ready to receive commands, enable the menu items
 setActionEnabled "changeIPCTestAppColor1" true
 setActionEnabled "changeIPCTestAppColor2" true
 setActionEnabled "terminateIPCTestApp" true
}
```

Feedback

The following code defines and registers a callback that will be called whenever a job walks up to the Custom Compiler environment.

```
#Walkup
proc registerWalkupProc {} { db::createCallback "walkup" \  -callbackType
 "onPostWalkup" \  -procedure [namespace current]::walkupProc \
  -priority 1}
```

Note the signature of the postman callback: the string token to identify the walked up app, and a special job object that was created to manage the connection.

```
proc walkupProc {token walkupJob} {
variable job
if {$token == "demo_gui"} {
 set job $walkupJob
 setActionEnabled "launchIPCTestApp" false
 setActionEnabled "changeIPCTestAppColor1" true
 setActionEnabled "changeIPCTestAppColor2" true
 setActionEnabled "terminateIPCTestApp" true
 db::setAttr exitProc -of $job -value \
  [namespace current]::exitProc
 }
}
```

Then the main flow control registers the walkup callback procedure and creates the menu in the Custom Compiler environment:

```
# main flow control
registerWalkupProc
createMenu
}
```